



Operating Systems: Three Easy Pieces

«Книга с кометой» (или OSTEP) — итог почти 20 лет преподавания курса «Введение в операционные системы» для студентов и магистрантов на факультете компьютерных наук Висконсинского университета.

В книге рассматриваются три фундаментальные концепции операционных систем:

- виртуализация (процессора и памяти)
- конкурентность (блокировки и условные переменные)
- долговременное хранение (диски, RAID-массивы, файловые системы).

Сочетая чтение книги с выполнением домашних заданий и работой над серьезными проектами, читатели приходят к более глубокому пониманию современных ОС.

Ремзи и Андреа Арпачи-Дюссо — профессора информатики Висконсинского университета в Мэдисоне. Занимаются исследованиями в области вычислительных систем более 20 лет. Опубликовали свыше 100 работ по производительности и надежности современных ОС, уделяя особое внимание файловым системам и системам хранения. Их труды были удостоены многочисленных наград, а некоторые новаторские предложения вошли в современные версии Linux и BSD.

Интернет-магазин:
www.dmkpress.com

Оптовая продажа:
КТК «Галактика»
books@aliants-kniga.ru



ISBN 978-5-97060-932-3



9 785970 609323 >

Операционные системы Три простых элемента

Ремзи Х. Арпачи-Дюссо
Андреа К. Арпачи-Дюссо

Операционные системы

Три простых элемента



Ремзи Х. Арпачи-Дюссо, Андреа К. Арпачи-Дюссо

Операционные системы

OPERATING SYSTEMS

Three Easy Pieces

**Remzi H. Arpaci-Dusseau
Andrea C. Arpaci-Dusseau**

Операционные системы

Три простых элемента

Ремзи Х. Арпачи-Дюссо
Андреа К. Арпачи-Дюссо



Москва, 2021

УДК 004.45
ББК 32.972.1
А79

Арпачи-Дюссо Р. Х., Арпачи-Дюссо А. К.

А79 Операционные системы: Три простых элемента / пер. с англ. А. А. Слинкина. – М.: ДМК Пресс, 2021. – 730 с.: ил.

ISBN 978-5-97060-932-3

В книге рассматриваются три фундаментальные концепции операционных систем: виртуализация (процессора и памяти), конкурентность (блокировки и условные переменные) и долговременное хранение (диски, RAID-массивы, файловые системы). В каждой главе представлена одна конкретная проблема и описано ее решение. Приводятся советы, которые могут пригодиться читателю при создании собственных систем.

Выполняя задания, предложенные авторами, и работая над серьезными проектами, читатели приходят к более глубокому пониманию современных ОС. Задания-эмуляторы способны генерировать практически бесконечное множество задач, благодаря чему можно многократно перепроверять свои знания. Все проекты, а также примеры кода написаны на языке программирования C.

Издание адресовано студентам технических вузов и всем, кто интересуется программированием. Преподаватели технических вузов могут использовать книгу в рамках курса информатики.

УДК 004.45
ББК 32.972.1

Russian language edition copyright © 2021 by DMK Press. All rights reserved.

Все права защищены. Любая часть этой книги не может быть воспроизведена в какой бы то ни было форме и какими бы то ни было средствами без письменного разрешения владельцев авторских прав.

ISBN 978-1-4842-6466-9 (англ.)
ISBN 978-5-97060-932-3 (рус.)

© Arpaci-Dusseau Books, LLC, 2008–2018
© Перевод, оформление, издание,
ДМК Пресс, 2021

*Посвящается Ведату С. Арпачи,
воодушевившему на всю жизнь*

Содержание

От издательства.....	27
Предисловие.....	28
Глава 1. Диалог о книге	36
Глава 2. Введение в операционные системы.....	38
2.1. Виртуализация процессора	40
2.2. Виртуализация памяти.....	42
2.3. Конкурентность	44
2.4. Хранение	46
2.5. Цели проектирования	48
2.6. Немного истории	50
Первые операционные системы: просто библиотеки	50
Не только библиотеки: защита	50
Эра мультипрограммирования.....	51
Современность	52
2.7. Резюме.....	54
Литература.....	55
Домашнее задание.....	56
Часть I. ВИРТУАЛИЗАЦИЯ	58
Глава 3. Диалог о виртуализации	59
Глава 4. Абстракция: процесс	61
4.1. Абстракция: процесс	62
4.2. API процессов	63
4.3. Создание процесса: подробности.....	64
4.4. Состояния процесса	65
4.5. Структуры данных	67
4.6. Резюме	69
Литература.....	70
Домашнее задание (эмуляция)	70
Вопросы.....	71
Глава 5. Интерлюдия: API процессов	72
5.1. Системный вызов <code>fork()</code>	72
5.2. Системный вызов <code>wait()</code>	74
5.3. И наконец, системный вызов <code>exec()</code>	75
5.4. Почему? Мотивация API.....	77

5.5. Управление процессами и пользователи	79
5.6. Полезные инструменты	80
5.7. Резюме.....	81
Литература.....	82
Домашнее задание (кодирование).....	83
Вопросы.....	83
Глава 6. Механизм: ограниченное прямое выполнение.....	85
6.1. Базовая техника: ограниченное прямое выполнение.....	86
6.2. Проблема 1: запрещенные операции	86
6.3. Проблема 2: переключение между процессами	91
Кооперативный подход: дождаться системного вызова	91
Некооперативный подход: ОС силой забирает управление	92
Сохранение и восстановление контекста	93
6.4. Сомневаетесь насчет конкурентности?.....	96
6.5. Резюме	97
Литература.....	98
Домашнее задание (измерение).....	99
Глава 7. Планирование: введение	101
7.1. Предположения о рабочей нагрузке.....	101
7.2. Метрики планирования.....	102
7.3. Первым пришел, первым ушел (FIFO).....	103
7.4. Сначала самое короткое	104
7.5. Сначала с наименьшим временем до завершения	106
7.6. Новая метрика: время отклика	106
7.7. Циклическое планирование.....	107
7.8. Учет ввода-вывода.....	110
7.9. Долой оракулов	111
7.10. Резюме.....	111
Литература.....	112
Домашнее задание (эмуляция).....	113
Вопросы.....	113
Глава 8. Планирование: многоуровневая аналитическая очередь.....	114
8.1. MLFQ: основные правила.....	115
8.2. Попытка 1: как изменять приоритеты	116
Пример 1: одно долго работающее задание	117
Пример 2: к нам приходит короткое задание	117
Пример 3: а как насчет ввода-вывода?.....	118
Проблемы текущей реализации MLFQ.....	119
8.3. Попытка 2: повышение приоритета	120
8.4. Попытка 3: улучшенный учет	121
8.5. Настройка MLFQ и другие вопросы.....	122
8.6. MLFQ: резюме.....	124

Литература.....	124
Домашнее задание (эмуляция).....	126
Вопросы.....	126
Глава 9. Планирование: пропорциональная доля.....	127
9.1. Основная идея: ваша доля представлена билетом.....	127
9.2. Механизмы обращения с билетами.....	129
9.3. Реализация.....	130
9.4. Пример.....	131
9.5. Как раздавать билеты?.....	132
9.6. Зачем отказываться от детерминированности?.....	132
9.7. Вполне равномерный планировщик в Linux.....	134
Принцип работы.....	134
Взвешивание (уровень nice).....	136
Использование красно-черных деревьев.....	137
Обращение со спящими процессами.....	138
Другие возможности CFS.....	138
9.8. Резюме.....	139
Литература.....	140
Домашнее задание (эмуляция).....	141
Вопросы.....	141
Глава 10. Планирование в многопроцессорных системах (материал повышенной сложности).....	142
10.1. Введение: многопроцессорная архитектура.....	143
10.2. Не забывайте о синхронизации.....	145
10.3. Последняя проблема: привязка к процессору.....	146
10.4. Планирование с одной очередью.....	147
10.5. Планирование с несколькими очередями.....	148
10.6. Планировщики мультипроцессоров в Linux.....	151
10.7. Резюме.....	152
Литература.....	152
Домашнее задание (эмуляция).....	153
Вопросы.....	154
Глава 11. Заключительный диалог о виртуализации процессора.....	156
Глава 12. Диалог о виртуализации памяти.....	158
Глава 13. Абстракция: адресное пространство.....	160
13.1. Ранние системы.....	160
13.2. Мультипрограммирование и разделение времени.....	161
13.3. Адресное пространство.....	162
13.4. Цели.....	164

13.5. Резюме	166
Литература	167
Домашнее задание (код)	168
Вопросы	168
Глава 14. Интерлюдия: API памяти	170
14.1. Типы памяти	170
14.2. Вызов <code>malloc()</code>	171
14.3. Вызов <code>free()</code>	173
14.4. Типичные ошибки	173
Забыли выделить память	173
Выделили недостаточно памяти	174
Забыли инициализировать выделенную память	175
Забыли освободить память	175
Освободили память раньше, чем закончили с ней работать	175
Освободили память несколько раз	176
Неправильно вызвали <code>free()</code>	176
Итоги	177
14.5. Поддержка со стороны ОС	177
14.6. Другие вызовы	177
14.7. Резюме	178
Литература	178
Домашнее задание (код)	179
Вопросы	179
Глава 15. Механизм: трансляция адресов	181
15.1. Предположения	182
15.2. Пример	182
15.3. Динамическое (аппаратное) перемещение	185
Пример трансляции	187
15.4. Аппаратная поддержка: итоги	188
15.5. Требования к операционной системе	189
15.6. Резюме	192
Литература	193
Домашнее задание (эмуляция)	194
Вопросы	194
Глава 16. Сегментация	195
16.1. Сегментация: обобщение идеи базы и границы	195
16.2. К какому сегменту мы обращаемся?	198
16.3. А что насчет стека?	200
16.4. Поддержка разделения	200
16.5. Мелкоструктурная и крупноструктурная сегментация	201
16.6. Поддержка со стороны ОС	202
16.7. Резюме	203
Литература	204

Домашнее задание (эмуляция)	205
Вопросы	205
Глава 17. Управление свободным пространством	207
17.1. Предположения	208
17.2. Низкоуровневые механизмы	209
Разделение и объединение	209
Запоминание размеров выделенных блоков	211
Встраивание списка свободных	212
Увеличение размера кучи	217
17.3. Основные стратегии	218
Лучший подходящий	218
Худший подходящий	218
Первый подходящий	219
Следующий подходящий	219
Примеры	219
17.4. Другие подходы	220
Сегрегированные списки	220
Метод близнецов	221
Другие идеи	222
17.5. Резюме	223
Литература	223
Домашнее задание (эмуляция)	224
Вопросы	224
Глава 18. Страничная организация: введение	226
18.1. Простой пример и общий обзор	226
18.2. Где хранятся таблицы страниц?	230
18.3. Что хранится в таблице страниц?	231
18.4. Страничная организация: тоже слишком медленно	232
18.5. Трассировка доступа к памяти	234
18.6. Резюме	237
Литература	237
Домашнее задание (эмуляция)	238
Вопросы	238
Глава 19. Страничная организация: более быстрая трансляция (TLB)	240
19.1. Основной алгоритм TLB	241
19.2. Пример: доступ к массиву	242
19.3. Кто обрабатывает непопадание в TLB?	245
19.4. Содержимое TLB: что там хранится?	247
19.5. Проблема TLB: контекстные переключения	248
19.6. Проблема: политика вытеснения	250
19.7. Реальная запись TLB	251
19.8. Резюме	252

Литература.....	253
Домашнее задание (измерение).....	254
Вопросы.....	256
Глава 20. Страничная организация: уменьшенные таблицы	257
20.1. Простое решение: увеличенные страницы.....	257
20.2. Гибридный подход: страничная организация и сегменты.....	258
20.3. Многоуровневые таблицы страниц.....	261
Подробный пример работы с многоуровневой таблицей страниц.....	264
Больше двух уровней.....	267
Процесс трансляции: вспомним про TLB	268
20.4. Инвертированные таблицы страниц	269
20.5. Выгрузка таблиц страниц на диск	270
20.6. Резюме	270
Литература.....	270
Домашнее задание (эмуляция).....	271
Вопросы.....	271
Глава 21. За пределами физической памяти: механизмы	273
21.1. Область подкачки.....	274
21.2. Бит присутствия	275
21.3. Отказ страницы	276
21.4. А что, если память заполнена?.....	277
21.5. Поток управления при обработке отказа страницы	278
21.6. Когда на самом деле происходит замещение	279
21.7. Резюме.....	280
Литература.....	281
Домашнее задание (измерение).....	281
Вопросы.....	282
Глава 22. За пределами физической памяти: политики.....	284
22.1. Управление кешем	284
22.2. Оптимальная политика замещения.....	286
22.3. Простая политика: FIFO	288
22.4. Еще одна простая политика: случайная	289
22.5. Учет истории: LRU	290
22.6. Примеры рабочей нагрузки.....	292
22.7. Реализация алгоритмов, учитывающих историю	295
22.8. Аппроксимация LRU	296
22.9. Учет модифицированных страниц.....	297
22.10. Другие политики ВП	298
22.11. Пробуксовка	298
22.12. Резюме	299
Литература.....	299
Домашнее задание (эмуляция).....	301
Вопросы.....	301

Глава 23. Полные примеры систем виртуальной памяти	302
23.1. Виртуальная память в VAX/VMS	303
Оборудование управления памятью	303
Реальное адресное пространство	304
Замещение страниц.....	306
Другие хитрости.....	308
23.2. Система виртуальной памяти в Linux.....	309
Адресное пространство Linux.....	310
Структура таблицы страниц.....	312
Поддержка больших страниц.....	313
Страничный кеш.....	314
Безопасность и переполнение буфера.....	316
Другие проблемы безопасности: Meltdown и Spectre	318
23.3. Резюме	319
Литература.....	320
Глава 24. Заключительный диалог о виртуализации памяти	322
Часть II. КОНКУРЕНТНОСТЬ	325
Глава 25. Диалог о конкурентности	326
Глава 26. Конкурентность: введение	328
26.1. Зачем нужны потоки?.....	329
26.2. Пример: создание потока	330
26.3. Почему становится хуже: разделяемые данные.....	333
26.4. Суть проблемы: неконтролируемое планирование.....	335
26.5. Жажда атомарности.....	337
26.6. Еще одна проблема: ожидание другого потока.....	339
26.7. Резюме: почему на курсе по ОС?	339
Литература.....	340
Домашнее задание (эмуляция).....	341
Вопросы.....	341
Глава 27. Интерлюдия: API потоков	343
27.1. Создание потока	343
27.2. Завершение потока	345
27.3. Блокировки.....	348
27.4. Условные переменные	350
27.5. Компиляция и выполнение.....	352
27.6. Резюме.....	352
Литература.....	353
Домашнее задание (код).....	354
Вопросы.....	354

Глава 28. Блокировки	355
28.1. Блокировки: основная идея	355
28.2. Блокировки в pthread	356
28.3. Конструирование блокировок	357
28.4. Оценивание блокировок	357
28.5. Управление прерываниями	358
28.6. Неудачная попытка: пробуем обойтись командами загрузки и сохранения	359
28.7. Построение работоспособных спин-блокировок с помощью команды проверки и установки	361
28.8. Оценка спин-блокировок	363
28.9. Сравнить и обменять	364
28.10. Загрузить по связи и сохранить условно	366
28.11. Выбрать и прибавить	368
28.12. Слишком много активного ожидания: и как с этим быть?	369
28.13. Простой подход: уступи	369
28.14. Очереди: засыпание вместо активного ожидания	371
28.15. Разные ОС, разная поддержка	374
28.16. Двухфазная блокировка	375
28.17. Резюме	376
Литература	376
Домашнее задание (эмуляция)	378
Вопросы	378
Глава 29. Конкурентные структуры данных с блокировками	380
29.1. Конкурентные счетчики	380
Простой, но немасштабируемый	380
Масштабируемый подсчет	382
29.2. Конкурентные связные списки	385
Масштабирование связных списков	388
29.3. Конкурентные очереди	389
29.4. Конкурентная хеш-таблица	390
29.5. Резюме	392
Литература	392
Домашнее задание (код)	393
Вопросы	393
Глава 30. Условные переменные	395
30.1. Определение и функции	396
30.2. Задача о производителе и потребителе (об ограниченном буфере)	399
Неправильное решение	401
Лучше, но все равно неправильно: While, а не If	404
Решение задачи о производителе и потребителе с буфером на один элемент	406
Правильное решение задачи о производителе и потребителе	407

30.3. Покрывающие условия.....	408
30.4. Резюме	410
Литература.....	410
Домашнее задание (код).....	411
Вопросы.....	411

Глава 31. Семафоры

31.1. Семафоры: определение	413
31.2. Двоичные семафоры (блокировки)	415
31.3. Использование семафоров для упорядочения	416
31.4. Задача о производителе и потребителе (об ограниченном буфере).....	418
Первая попытка	419
Решение: добавление взаимного исключения.....	421
Предотвращение взаимоблокировки	422
Наконец-то правильное решение	422
31.5. Блокировки чтения-записи.....	422
31.6. Обедающие философы	425
Неправильное решение	426
Решение: разрыв зависимости	427
31.7. Как реализуются семафоры	428
31.8. Резюме	429
Литература.....	429
Домашнее задание (код).....	431
Вопросы.....	431

Глава 32. Типичные ошибки в конкурентных программах.....

32.1. Какие бывают ошибки?	433
32.2. Ошибки, не связанные с взаимоблокировкой.....	434
Ошибки нарушения атомарности	434
Ошибка нарушения порядка.....	435
Ошибки, не связанные с взаимоблокировкой: резюме	437
32.3. Ошибки, связанные с взаимоблокировкой	437
Почему возникают взаимоблокировки?	438
Условия возникновения взаимоблокировки	439
Предотвращение.....	440
Циклическое ожидание	440
Ожидание с удержанием	441
Отсутствие вытеснения.....	441
Взаимное исключение.....	442
Избегание взаимоблокировок с помощью планирования.....	444
Найди и исправь	446
32.4. Резюме	446
Литература.....	447
Домашнее задание (код).....	448
Вопросы.....	448

Глава 33. Событийно-управляемая конкурентность (материал повышенной сложности)	450
33.1. Основная идея: цикл событий	451
33.2. Важный API: <code>select()</code> (или <code>poll()</code>)	451
33.3. Использование <code>select()</code>	452
33.4. Почему проще? Потому что не нужны блокировки.....	454
33.5. Проблема: блокирующие системные вызовы.....	454
33.6. Решение: асинхронный ввод-вывод	455
33.7. Еще одна проблема: управление состоянием	457
33.8. Какие еще трудности сопряжены с событиями.....	458
33.9. Резюме	459
Литература.....	459
Домашнее задание (код).....	460
Вопросы.....	460
Глава 34. Итоговый диалог о конкурентности	462
Часть III. ХРАНЕНИЕ	464
Глава 35. Диалог о хранении	465
Глава 36. Устройства ввода-вывода	466
36.1. Архитектура системы	466
36.2. Каноническое устройство	468
36.3. Канонический протокол	469
36.4. Прерывания помогают снизить затраты CPU.....	470
36.5. Более эффективное перемещение данных с помощью ПДП	472
36.6. Методы взаимодействия с устройствами.....	473
36.7. Сопряжение с ОС: драйвер устройства	474
36.8. Практический пример: простой драйвер IDE-диска	475
36.9. Исторические замечания.....	478
36.10. Резюме	478
Литература.....	479
Глава 37. Жесткие диски	481
37.1. Интерфейс	481
37.2. Базовая геометрия.....	482
37.3. Простой диск	483
Одна дорожка: задержка вращения	483
Несколько дорожек: время поиска.....	484
Дополнительные детали.....	485
37.4. Время ввода-вывода: немного арифметики	487
37.5. Планирование диска	490
SSTF: с наименьшим временем поиска первым.....	490
Лифт (он же SCAN или C-SCAN).....	491

SPTF: с наименьшим временем позиционирования первым	492
Другие проблемы планирования	493
37.6. Резюме.....	494
Литература.....	494
Домашнее задание (эмуляция).....	495

Глава 38. Избыточный массив недорогих дисков (RAID)

38.1. Интерфейс и внутреннее устройство RAID.....	499
38.2. Модель отказов.....	500
38.3. Как оценивать RAID.....	500
38.4. RAID уровня 0: чередование.....	501
Размеры порций	502
Возвращаясь к анализу RAID-0.....	503
Оценка производительности RAID	503
Снова возвращаемся к анализу RAID-0.....	505
38.5. RAID уровня 1: зеркалирование	505
Анализ RAID-1.....	506
38.6. RAID уровня 4: экономия места за счет четности.....	508
Анализ RAID-4.....	509
38.7. RAID уровня 5: ротация четности	512
Анализ RAID-5.....	512
38.8. Сравнение RAID: итоги	513
38.9. Другие интересные вопросы RAID	514
38.10. Резюме	514
Литература.....	515
Домашнее задание (эмуляция).....	516
Вопросы.....	516

Глава 39. Интерлюдия: файлы и каталоги.....

39.1. Файлы и каталоги.....	518
39.2. Интерфейс файловой системы.....	519
39.3. Создание файлов.....	519
39.4. Чтение и запись файлов	521
39.5. Непоследовательные чтение и запись.....	522
39.6. Разделяемые записи таблицы файлов: <code>fork()</code> и <code>dup()</code>	525
39.7. Безотлагательная запись с помощью <code>fsync()</code>	527
39.8. Переименование файлов	528
39.9. Получение информации о файлах.....	529
39.10. Удаление файлов	530
39.11. Создание каталога.....	530
39.12. Чтение каталогов	531
39.13. Удаление каталогов.....	532
39.14. Жесткие ссылки.....	533
39.15. Символические ссылки	534
39.16. Биты полномочий и списки контроля доступа	536
39.17. Создание и монтирование файловой системы.....	539

39.18. Резюме	541
Литература	541
Домашнее задание (код)	542
Вопросы	543

Глава 40. Реализация файловой системы

40.1. Ход мыслей	544
40.2. Общая организация	545
40.3. Организация файла: индексный дескриптор	548
Многоуровневый индекс	550
40.4. Организация каталогов	552
40.5. Управление свободным местом	553
40.6. Пути доступа: чтение и запись	554
Чтение файла с диска	554
Запись на диск	556
40.7. Кеширование и буферизация	558
40.8. Резюме	560
Литература	561
Домашнее задание (эмуляция)	562
Вопросы	562

Глава 41. Локальность и быстрая файловая система

41.1. Проблема: низкая производительность	563
41.2. FFS: решение – осведомленность о диске	565
41.3. Организационная структура: группа цилиндров	565
41.4. Политики: как выделять место для файлов и каталогов	567
41.5. Измерение локальности файлов	569
41.6. Исключение для больших файлов	571
41.7. Другие аспекты FFS	573
41.8. Резюме	575
Литература	575
Домашнее задание (эмуляция)	576
Вопросы	576

Глава 42. Согласованность после отказа: FSCK

и журналирование

42.1. Подробный пример	579
Сценарии отказа	581
Проблема согласованности после отказа	582
42.2. Решение 1: средство проверки файловой системы	582
42.3. Решение 2: журналирование (или упреждающая запись в журнал)	584
Журналирование данных	585
Восстановление	588
Группировка обновлений журнала	589
Ограничение размера журнала	590
Журналирование метаданных	591

Интересный случай: повторное использование блока	593
Подводя итоги: хронология журналирования	594
42.4. Решение 3: другие подходы	595
42.5. Резюме	596
Литература	597
Домашнее задание (эмуляция)	599
Вопросы	599

Глава 43. Файловые системы со структурой журнала

43.1. Записывать на диск последовательно	601
43.2. Записывать последовательно и эффективно	602
43.3. Сколько буферизовать?	603
43.4. Проблема: нахождение индексных дескрипторов	604
43.5. Решение дает косвенность: карта индексных дескрипторов	605
43.6. Полное решение: область контрольной точки	606
43.7. Чтение файла с диска: повторение пройденного	607
43.8. А как насчет каталогов?	607
43.9. Новая проблема: сборка мусора	608
43.10. Нахождение живых блоков	610
43.11. Политика: какие блоки очищать и когда?	611
43.12. Структура журнала и восстановление после аварии	612
43.13. Резюме	613
Литература	614
Домашнее задание (эмуляция)	615
Вопросы	615

Глава 44. SSD-диски на основе флеш-памяти

44.1. Сохранение одного бита	619
44.2. От битов к банкам и плоскостям	619
44.3. Основные операции с флеш-памятью	620
Подробный пример	621
Резюме	622
44.4. Производительность и надежность флеш-памяти	622
44.5. От голой флеш-памяти к SSD на ее основе	624
44.6. Организация FTL: неправильный подход	625
44.7. FTL со структурой журнала	625
44.8. Сборка мусора	628
44.9. Размер таблицы отображения	631
Блочное отображение	631
Гибридное отображение	632
Страничное отображение плюс кеширование	635
44.10. Выравнивание износа	635
44.11. Производительность и стоимость SSD	636
Производительность	636
Стоимость	637
44.12. Резюме	638

Литература.....	639
Домашнее задание (эмуляция).....	641
Вопросы.....	642
Глава 45. Целостность и защита данных	644
45.1. Виды отказа дисков	644
45.2. Обработка скрытых ошибок секторов	646
45.3. Обнаружение искажения: контрольная сумма.....	647
Распространенные функции вычисления контрольной суммы.....	648
Хранение контрольных сумм	649
45.4. Использование контрольных сумм	650
45.5. Новая проблема: запись не по адресу.....	651
45.6. Последняя проблема: потерянные записи.....	652
45.7. Очистка	653
45.8. Накладные расходы контрольных сумм.....	653
45.9. Резюме	654
Литература.....	654
Домашнее задание (эмуляция).....	656
Вопросы.....	656
Домашнее задание (код).....	657
Вопросы.....	657
Глава 46. Итоговый диалог о долговременном хранении	658
Глава 47. Диалог о распределенности	660
Глава 48. Распределенные системы	662
48.1. Основы коммуникации	663
48.2. Ненадежные уровни коммуникации	664
48.3. Надежные коммуникационные уровни.....	666
48.4. Абстракции коммуникации.....	669
48.5. Удаленный вызов процедур (RPC)	670
Генератор заглушек	670
Библиотека времени выполнения.....	672
Другие проблемы	673
48.6. Резюме	675
Литература.....	675
Домашнее задание (код).....	676
Вопросы.....	676
Глава 49. Сетевая файловая система Sun (NFS).....	678
49.1. Простая распределенная файловая система.....	679
49.2. Вперед к NFS	680
49.3. Акцент на простом и быстром восстановлении после аварии файлового сервера	680

49.4. Ключ к быстрому восстановлению: отсутствие информации о состоянии	681
49.5. Протокол NFSv2	682
49.6. От протокола к распределенной файловой системе	684
49.7. Обработка отказов сервера благодаря идемпотентным операциям.....	686
49.8. Повышение производительности: кеширование на стороне клиента ...	688
49.9. Проблема согласованности кешей.....	689
49.10. Оценка согласованности кешей в NFS	690
49.11. Последствия для буферизации записи на стороне сервера.....	691
49.12. Резюме	693
Литература.....	694
Домашнее задание (измерение).....	695
Вопросы.....	695
Глава 50. Файловая система Andrew (AFS)	697
50.1. AFS версии 1	697
50.2. Проблемы версии 1	699
50.3. Улучшение протокола.....	700
50.4. AFS версии 2	700
50.5. Согласованность кешей.....	702
50.6. Восстановление после аварии.....	703
50.7. Масштабируемость и производительность AFSv2.....	705
50.8. AFS: другие усовершенствования	707
50.9. Резюме	708
Литература.....	708
Домашнее задание (эмуляция)	709
Вопросы.....	709
Глава 51. Заключительный диалог о распределенных файловых системах	711
Предметный указатель.....	713

Список рисунков

Рис. 2.1	Простой пример: программа крутится в цикле и печатает (срп.с)	40
Рис. 2.2	Выполнение сразу нескольких программ.....	41
Рис. 2.3	Программа, обращающаяся к памяти	43
Рис. 2.4	Выполнение нескольких экземпляров программы обращения к памяти	43
Рис. 2.5	Многопоточная программа (threads.c).....	45
Рис. 2.6	Программа ввода-вывода (io.c)	47
Рис. 4.1	Загрузка: от программы к процессу	64
Рис. 4.2	Процесс: диаграмма перехода состояний.....	66
Рис. 4.3	Последовательность состояний процесса: только CPU.....	66
Рис. 4.4	Последовательность состояний процесса: CPU и ввод-вывод.....	67
Рис. 4.5	Структура proc в xv6	68
Рис. 5.1	Вызов fork() (p1.c).....	73
Рис. 5.2	Вызов fork() и wait() (p2.c)	75
Рис. 5.3	Вызов fork(), wait() и exec() (p3.c)	76
Рис. 5.4	Все вышеперечисленное плюс перенаправление (p4.c)	78
Рис. 6.1	Протокол прямого выполнения (без ограничений)	86
Рис. 6.2	Протокол ограниченного прямого выполнения.....	90
Рис. 6.3	Протокол ограниченного прямого выполнения (прерывание от таймера)	94
Рис. 6.4	Код контекстного переключения в xv6	95
Рис. 7.1	Простой пример FIFO.....	103
Рис. 7.2	Почему алгоритм FIFO не так уж хорош	103
Рис. 7.3	Простой пример SJF.....	105
Рис. 7.4	SJF, когда В и С поступают с задержкой	105
Рис. 7.5	Простой пример SCTF	106
Рис. 7.6	Снова SJF (не годится, когда метрикой является время отклика).....	108
Рис. 7.7	Циклическое планирование (годится для времени отклика).....	108
Рис. 7.8	Неэффективное использование ресурсов	110
Рис. 7.9	Перекрытие позволяет лучше использовать ресурсы.....	111
Рис. 8.1	Пример MLFQ	116
Рис. 8.2	Эволюция долго работающего задания.....	117
Рис. 8.3	Поступает интерактивное задание	118
Рис. 8.4	Смешанная рабочая нагрузка: задания с вводом-выводом и счетные задания	119
Рис. 8.5	Без повышения приоритета (слева) и с повышением приоритета (справа).....	121
Рис. 8.6	Без препятствования переигрыванию (слева) и с препятствованием (справа)	122
Рис. 8.7	Пониженный приоритет, увеличенный квант	123

Рис. 9.1	Код принятия решения в лотерейном планировании	131
Рис. 9.2	Изучение справедливости лотерейного планирования	132
Рис. 9.3	Шаговое планирование: трасса	133
Рис. 9.4	Простой пример CFS	135
Рис. 9.5	Красное-черное дерево в CFS	138
Рис. 10.1	Один CPU с кешем	143
Рис. 10.2	Два CPU с раздельными кешами и общей памятью	144
Рис. 10.3	Простой код удаления элемента из списка	146
Рис. 13.1	Операционная система: прежние деньки	160
Рис. 13.2	Три процесса: разделение памяти	162
Рис. 13.3	Пример адресного пространства	163
Рис. 15.1	Процесс и его адресное пространство	184
Рис. 15.2	Физическая память и один перемещенный процесс	185
Рис. 15.3	Динамическое перемещение: требования к оборудованию	188
Рис. 15.4	Динамическое перемещение: обязанности операционной системы	190
Рис. 15.5	Протокол ограниченного прямого выполнения (динамическое перемещение)	191
Рис. 16.1	Адресное пространство (повтор)	196
Рис. 16.2	Размещение сегментов в физической памяти	197
Рис. 16.3	Значения сегментных регистров	197
Рис. 16.4	Сегментные регистры (с поддержкой роста вверх)	200
Рис. 16.5	Значения сегментных регистров (с защитой)	201
Рис. 16.6	Неуплотненная и уплотненная память	203
Рис. 17.1	Выделенная область плюс заголовок	211
Рис. 17.2	Содержимое заголовка	212
Рис. 17.3	Куча с одним свободным блоком	213
Рис. 17.4	Куча: после одного выделения памяти	214
Рис. 17.5	Свободное пространство с тремя выделенными блоками	215
Рис. 17.6	Свободное пространство с двумя выделенными блоками	216
Рис. 17.7	Список свободных с необъединенными блоками	217
Рис. 18.1	Простое 64-байтовое адресное пространство	227
Рис. 18.2	64-байтовое адресное пространство в 128-байтовой физической памяти	227
Рис. 18.3	Процесс трансляции адреса	230
Рис. 18.4	Пример: таблица страниц в физической памяти ядра	231
Рис. 18.5	Запись таблицы страниц (PTE) на x86	232
Рис. 18.6	Доступ к странично организованной памяти	234
Рис. 18.7	Трассировка доступа к виртуальной и физической памяти	236
Рис. 19.1	Алгоритм управления TLB	241
Рис. 19.2	Пример: массив в крохотном адресном пространстве	243
Рис. 19.3	Алгоритм управления TLB (средствами ОС)	246
Рис. 19.4	Запись TLB в архитектуре MIPS	251
Рис. 19.5	Определение размера TLB и стоимости непопадания	255
Рис. 20.1	Адресное пространство размером 16 КБ со страницами размером 1 КБ	259
Рис. 20.2	Таблица страниц для адресного пространства размером 16 КБ	259

Рис. 20.3	Линейная (слева) и многоуровневая (справа) таблицы страниц.....	262
Рис. 20.4	16-килобайтовое адресное пространство с 64-байтовыми страницами.....	264
Рис. 20.5	Каталог страниц и части таблицы страниц	266
Рис. 20.6	Поток управления при поиске в многоуровневой таблице страниц.....	269
Рис. 21.1	Физическая память и область подкачки	275
Рис. 21.2	Поток управления при обработке отказа страницы (оборудование).....	279
Рис. 21.3	Поток управления при обработке отказа страницы (ОС).....	279
Рис. 22.1	Трассировка оптимальной политики	287
Рис. 22.2	Трассировка политики FIFO.....	288
Рис. 22.3	Трассировка случайной политики	289
Рис. 22.4	Результаты случайной политики при более 10 000 испытаний	290
Рис. 22.5	Трассировка политики LRU.....	291
Рис. 22.6	Рабочая нагрузка без локальности.....	293
Рис. 22.7	Рабочая нагрузка 80-20.....	293
Рис. 22.8	Циклическая рабочая нагрузка	294
Рис. 22.9	Рабочая нагрузка 80-20 с часами.....	297
Рис. 23.1	Адресное пространство VAX/VMS	305
Рис. 23.2	Адресное пространство в Linux.....	311
Рис. 26.1	Адресное пространство однопоточного и многопоточного процесса	329
Рис. 26.2	Простая программа создания потоков (t0.c)	331
Рис. 26.3	Трассировка потоков (1)	331
Рис. 26.4	Трассировка потоков (2)	332
Рис. 26.5	Трассировка потоков (3)	332
Рис. 26.6	Разделение данных: ай и ой (t1.c)	334
Рис. 26.7	Проблема с близкого расстояния	337
Рис. 27.1	Создание потока	345
Рис. 27.2	Ожидание завершения потока	346
Рис. 27.3	Упрощенная передача аргументов потоку	347
Рис. 27.4	Пример обертки	349
Рис. 28.1	Первая попытка: простой флаг	360
Рис. 28.2	Трассировка: взаимного исключения нет	360
Рис. 28.3	Простая спин-блокировка с применением команды проверки и установки	362
Рис. 28.4	Сравнить и обменять	365
Рис. 28.5	Команды «загрузить по связи» и «сохранить условно»	366
Рис. 28.6	Применение пары команд LL/SC для построения блокировки	367
Рис. 28.7	Билетная блокировка	368
Рис. 28.8	Блокировка на основе команды «проверить и установить» и уступки.....	370
Рис. 28.9	Блокировка на основе очередей, команды «проверить и установить», уступки и пробуждения.....	372
Рис. 28.10	Фьютексная блокировка в Linux.....	375

Рис. 29.1	Счетчик без блокировок	381
Рис. 29.2	Счетчик с блокировками	381
Рис. 29.3	Производительность традиционного и приближенного счетчика	382
Рис. 29.4	Трассировка приближенных счетчиков	383
Рис. 29.5	Реализация приближенного счетчика	384
Рис. 29.6	Масштабируемость приближенных счетчиков	385
Рис. 29.7	Конкурентный связный список	386
Рис. 29.8	Конкурентный связный список: переписанный вариант	387
Рис. 29.9	Конкурентная очередь Майкла и Скотта	389
Рис. 29.10	Конкурентная хеш-таблица	390
Рис. 29.11	Масштабирование хеш-таблицы	391
Рис. 30.1	Родитель, ожидающий потомка	395
Рис. 30.2	Родитель, ожидающий потомка	396
Рис. 30.3	Родитель, ожидающий потомка: использование условной переменной	397
Рис. 30.4	Функции put и get (версия 1)	400
Рис. 30.5	Потоки производителя и потребителя (версия 1)	401
Рис. 30.6	Производитель-потребитель: одна условная переменная и предложение if	402
Рис. 30.7	Трассировка потоков: неправильное решение (версия 1)	403
Рис. 30.8	Производитель-потребитель: одна условная переменная и цикл while	404
Рис. 30.9	Трассировка потоков: неправильное решение (версия 2)	405
Рис. 30.10	Производитель-потребитель: две условные переменные и цикл while	406
Рис. 30.11	Правильные функции put и get	407
Рис. 30.12	Правильная синхронизация производителя и потребителя	408
Рис. 30.13	Покрывающие условия: пример	409
Рис. 31.1	Инициализация семафора	414
Рис. 31.2	Семафор: определение операций wait и post	414
Рис. 31.3	Двоичный семафор (т. е. блокировка)	415
Рис. 31.4	Трассировка потоков: один поток использует семафор	415
Рис. 31.5	Трассировка потоков: два потока используют семафор	416
Рис. 31.6	Родитель ждет потомка	417
Рис. 31.7	Трассировка потоков: родитель ждет потомка (случай 1)	418
Рис. 31.8	Трассировка потоков: родитель ждет потомка (случай 2)	418
Рис. 31.9	Функции put и get	419
Рис. 31.10	Добавление семафоров full и empty	420
Рис. 31.11	Добавление взаимного исключения (некорректное)	421
Рис. 31.12	Добавление взаимного исключения (корректное)	423
Рис. 31.13	Простая блокировка чтения записи	424
Рис. 31.14	Обедающие философы	426
Рис. 31.15	Функции getforks() и putforks()	427
Рис. 31.16	Реализация земафоров с помощью блокировок и условных переменных	428
Рис. 32.1	Ошибки в современных приложениях	434

Рис. 32.2	Граф зависимостей при взаимоблокировке	438
Рис. 33.1	Простой код с применением select()	453
Рис. 36.1	Архитектура типовой системы	467
Рис. 36.2	Архитектура современной системы	468
Рис. 36.3	Каноническое устройство	469
Рис. 36.4	Стек файловой системы	474
Рис. 36.5	Интерфейс IDE	476
Рис. 36.6	Драйвер IDE-диска в xv6 (упрощенный)	477
Рис. 37.1	Диск с одной дорожкой	483
Рис. 37.2	Дорожка и головка	483
Рис. 37.3	Три дорожки и головка (справа: с поиском)	484
Рис. 37.4	Три дорожки: сдвиг дорожек на 2	485
Рис. 37.5	Спецификации дисков: сравнение SCSI и SATA	487
Рис. 37.6	Производительность дисков: сравнение SCSI и SATA	489
Рис. 37.7	SSTF: планирование запросов к секторам 21 и 2	491
Рис. 37.8	SSTF: иногда недостаточно хорош	493
Рис. 38.1	RAID-0: простое чередование	501
Рис. 38.2	Чередование с большим размером порции	502
Рис. 38.3	Простой RAID-1: зеркалирование	505
Рис. 38.4	RAID-4 с четностью	508
Рис. 38.5	Цельнополосная запись в RAID-4	510
Рис. 38.6	Пример: запись в блоки 4 и 13 и соответствующие блоки четности	511
Рис. 38.7	RAID-5 с чередованием четности	512
Рис. 38.8	Емкость, надежность и производительность RAID	513
Рис. 39.1	Пример дерева каталогов	519
Рис. 39.2	Разделяемая родителем и потомком запись в таблице файлов (fork- <i>seek.c</i>)	526
Рис. 39.3	Процессы, разделяющие запись в таблице открытых файлов	526
Рис. 39.4	Разделение записи таблицы файлов с помощью dup() (<i>dup.c</i>)	527
Рис. 40.1	Упрощенный индексный дескриптор в ext2	549
Рис. 40.2	Результаты измерения файловых систем	551
Рис. 40.3	Хронология чтения файла	556
Рис. 40.4	Хронология создания файла	557
Рис. 41.1	Локальность FFS для SEER-трассировки	570
Рис. 41.2	Амортизация: каким должен быть размер порции?	572
Рис. 41.3	FFS: стандартное и параметризованное размещение	574
Рис. 42.1	Хронология журналирования данных	594
Рис. 42.2	Хронология журналирования метаданных	594
Рис. 44.1	Организация простой флеш-памяти: страницы с несколькими блоками	620
Рис. 44.2	Характеристики аппаратной производительности флеш-памяти	623
Рис. 44.3	SSD на основе флеш-памяти: логическая диаграмма	624
Рис. 44.4	Сравнение производительности SSD и жестких дисков	636
Рис. 45.1	Частота скрытых ошибок сектора и повреждений блоков	645

Рис. 48.1	Пример клиент-серверного кода с применением протокола UDP/IP.....	665
Рис. 48.2	Простая библиотека UDP	666
Рис. 48.3	Сообщение и подтверждение	667
Рис. 48.4	Сообщение и подтверждение: отброшенный запрос	667
Рис. 48.5	Сообщение и подтверждение: отброшенный ответ.....	668
Рис. 49.1	Обобщенная клиент-серверная система.....	678
Рис. 49.2	Архитектура распределенной файловой системы	679
Рис. 49.3	Клиентский код: чтение из файла	681
Рис. 49.4	Протокол NFS: примеры	683
Рис. 49.5	Чтение файла: действия на стороне клиента и файлового сервера	685
Рис. 49.6	Три вида потерь	687
Рис. 49.7	Проблема согласованности кешей	689
Рис. 50.1	Основы протокола AFSv1.....	698
Рис. 50.2	Чтение файла: действия на стороне клиента и сервера	701
Рис. 50.3	Хронология согласованности кешей	704
Рис. 50.4	Сравнение AFS с NFS	705

От издательства

Отзывы и пожелания

Мы всегда рады отзывам наших читателей. Расскажите нам, что вы думаете об этой книге – что понравилось или, может быть, не понравилось. Отзывы важны для нас, чтобы выпускать книги, которые будут для вас максимально полезны.

Вы можете написать отзыв на нашем сайте www.dmkpress.com, зайдя на страницу книги и оставив комментарий в разделе «Отзывы и рецензии». Также можно послать письмо главному редактору по адресу dmkpress@gmail.com; при этом укажите название книги в теме письма.

Если вы являетесь экспертом в какой-либо области и заинтересованы в написании новой книги, заполните форму на нашем сайте по адресу http://dmkpress.com/authors/publish_book/ или напишите в издательство по адресу dmkpress@gmail.com.

Скачивание исходного кода примеров

Скачать файлы с дополнительной информацией для книг издательства «ДМК Пресс» можно на сайте www.dmkpress.com на странице с описанием соответствующей книги.

Список опечаток

Хотя мы приняли все возможные меры для того, чтобы обеспечить высокое качество наших текстов, ошибки все равно случаются. Если вы найдете ошибку в одной из наших книг, мы будем очень благодарны, если вы сообщите о ней главному редактору по адресу dmkpress@gmail.com. Сделав это, вы избавите других читателей от недопонимания и поможете нам улучшить последующие издания этой книги.

Нарушение авторских прав

Пиратство в интернете по-прежнему остается насущной проблемой. Издательство «ДМК Пресс» очень серьезно относится к вопросам защиты авторских прав и лицензирования. Если вы столкнетесь в интернете с незаконной публикацией какой-либо из наших книг, пожалуйста, пришлите нам ссылку на интернет-ресурс, чтобы мы могли применить санкции.

Ссылку на подозрительные материалы можно прислать по адресу электронной почты dmkpress@gmail.com.

Мы высоко ценим любую помощь по защите наших авторов, благодаря которой мы можем предоставлять вам качественные материалы.

Предисловие

Всем

Добро пожаловать! Надеемся, что вы получите такое же удовольствие от чтения этой книги, какое получили мы, когда ее писали. Книга называется «Операционные системы: три простых элемента» (Operating Systems: Three Easy Pieces) (доступна на сайте <http://www.ostep.org>), это название, очевидно, является данью уважения одному из самых блестящих в истории курсов лекций (точнее, заметок к ним) – лекциям Ричарда Фейнмана по физике [F96]¹. Без сомнения, этой книге не сравниться с высокой планкой, заданной знаменитым физиком, но, быть может, ее хватит, чтобы удовлетворить взыскующего ответа на вопрос, что такое операционные системы (и системы вообще).

Вот те три простых элемента, которые составляют тематический костяк книги: **виртуализация**, **конкурентность** и **хранение**. Говоря о них, мы в конечном итоге обсудим большую часть того, что делает операционная система; льстим себя надеждой, что попутно вы узнаете много интересного. Ведь узнавать новое всегда интересно, не правда ли? Ну, по крайней мере, должно быть так.

Каждой из основополагающих концепций посвящено несколько глав. В каждой главе представлена одна конкретная проблема и описано ее решение. Главы короткие, и мы старались (в меру возможностей) ссылаться на источники, откуда берут начало идеи. Одна из наших целей при написании этой книги – пролить свет на пути истории, поскольку, на наш взгляд, это поможет студенту лучше понять, что было, что есть и что будет. В данном случае знать, из чего делается колбаса, почти так же важно, как понимать, для чего она предназначена².

На протяжении всей книги мы прибегаем к нескольким приемам, о которых, пожалуй, стоит рассказать прямо сейчас. Первый – **существо** проблемы. Прежде чем приступить к решению проблемы, мы всегда пытаемся сформулировать самый важный вопрос; вот это **существо проблемы** явно выделено в тексте, и, хочется надеяться, проблема разрешается с помощью методов, алгоритмов и идей, изложенных далее.

Во многих местах мы объясняем, как система работает, демонстрируя ее поведение во времени. Эта **хронология** очень важна для понимания; уяснив, что происходит, к примеру, при страничном отказе, вы встали на путь понимания принципов работы виртуальной памяти. А осознав, что случается, когда журналируемая файловая система записывает блок на диск, вы сделали первый шаг к овладению тайнствами систем хранения.

¹ Русский перевод: Ричард Фейнман. Дюжина лекций. Шесть попроще и шесть сложнее. М.: Лаборатория знаний, 2018.

² Подсказка: чтобы ее съесть! Или, если вы вегетарианец, чтобы бежать от нее куда подальше.

В тексте вам также встретится много **отступлений** и **советов** – они расцвечивают магистральную линию изложения. В отступлениях обсуждаются вопросы, имеющие какое-то отношение (пусть и отдаленное) к основному тексту, советы содержат общие уроки, которые могут пригодиться вам при создании собственных систем. Для удобства в конце книги присутствует указатель, в котором сведены все советы и отступления (а заодно и темы, которые мы называли «существом проблемы»).

В разных местах книги мы используем один из старейших дидактических приемов, **диалог**, как способ представить материал в ином свете. Диалоги позволяют подойти к рассмотрению важных тем (как нам кажется, в увлекательной манере), а иногда резюмировать тот или иной материал. Ну, и дают шанс добавить немного юмора. А уж покажутся они полезными или смешными вам – это совсем другой вопрос.

Каждая часть начинается с описания **абстракции**, предоставляемой операционной системой, а в последующих главах детализируются механизмы, политики и прочие вспомогательные средства, необходимые для реализации этой абстракции. Абстракции – вещь, фундаментальная для всех аспектов информатики, неудивительно, что они играют важнейшую роль и в операционных системах.

Во всех главах мы стараемся приводить **настоящий код** (а не **псевдокод**) всюду, где возможно, так что практически во всех примерах вы сможете набрать код и выполнить его. Исполнение реального кода в реальной системе – лучший способ изучить операционную систему, поэтому мы рекомендуем делать это, если есть возможность. Для лучшего восприятия мы также разместили код по адресу <https://github.com/remzi-arpacidusseau/ostep-code>.

Мы включили в текст несколько **домашних заданий**, чтобы вы могли оценить, насколько хорошо поняли прочитанное. Часто эти задания представляют собой небольшие **эмуляции** частей операционной системы; скачайте их и выполните для самопроверки. У домашних заданий-эмуляторов есть одна общая черта: задав начальное значение генератора случайных чисел, вы сможете сгенерировать практически бесконечное множество задач; кроме того, эмулятор можно попросить решить задачу за вас. Таким образом, вы можете проверять и перепроверять себя, пока не сочтете, что достигли хорошего уровня понимания.

Самое важное дополнение к этой книге – набор **проектов**, которые на примере проектирования, реализации и тестирования собственного кода позволяют узнать, как работают реальные системы. Все проекты (а равно и вышеупомянутые примеры кода) написаны на **языке программирования C** [KR88]; C – простой и мощный язык, лежащий в основе большинства операционных систем, поэтому его стоит добавить в свой арсенал. Есть два типа проектов (идеи см. в онлайн-овом приложении). Первый тип – **системное программирование**, эти проекты ориентированы на тех, кто только начинает изучать C и UNIX и хочет узнать о низкоуровневом программировании на C. Второй тип – ядро реальной операционной системы, разработанной в MIT и называемой xv6 [CK+08]; эти проекты ориентированы на студентов, уже знакомых с языком C и желающих покопаться в потрохах ОС. Мы в Висконсинском университете читаем этот курс в трех вариантах:

только системное программирование, только программирование xv6 или то и другое вместе.

Мы потихоньку выкладываем описания проектов и систему тестирования. Подробнее см. на странице <https://github.com/remzi-arpacidusseau/ostep-projects>. Если вы не студент нашего курса, то эти материалы дадут вам возможность выполнить проекты самостоятельно, чтобы лучше усвоить материал. К сожалению, у вас не будет ассистента кафедры, к которому можно обратиться, если ничего не выходит, но не все в этой жизни можно получить бесплатно (некоторые книги – можно!).

Преподавателям

Если вы преподаватель или профессор, желающий использовать эту книгу, нет никаких препятствий. На всякий случай отметим, что английский текст книги бесплатно размещен на сайте <http://www.ostep.org>.

Курс довольно хорошо укладывается в 15-недельный семестр, в течение этого времени можно рассмотреть большинство тем на достаточно глубоком уровне. Если вы попытаетесь втиснуть курс в 10-недельную четверть, то, вероятно, какие-то детали из каждой части придется опустить. Есть также несколько глав, посвященных мониторам виртуальных машин, которые мы обычно проходим либо в самом конце большого раздела о виртуализации, либо ближе к концу в качестве отступления.

Трактовка конкурентности в этой книге немного необычна. Во многих книгах по ОС эта тема рассматривается в самом начале, а мы отложили ее до того момента, как студент будет понимать виртуализацию процессора и памяти. Наш опыт чтения этого курса на протяжении почти 20 лет показал, что студентам трудно понять, как возникает проблема конкурентности и почему ее надо решать, пока они не разобрались в том, что такое адресное пространство, что такое процесс и почему контекстное переключение может происходить в любой момент времени. Зато потом понятие потоков и вся возникающая в связи с ними проблематика дается им легко или, по крайней мере, легче.

Во время лекций мы пишем на доске – мелом или маркером. Если лекция посвящена преимущественно теории, то мы приходим на занятия, держа в голове несколько основных идей и примеров, и излагаем их на доске. Конкретные задачи для самостоятельной проработки раздаются студентам в виде заданий. На лекциях более практической направленности мы просто подключаем ноутбук к проектору и показываем реальный код; этот метод особенно хорош при изложении вопросов конкурентности, а также на дискуссионных занятиях, когда студентам демонстрируется код, имеющий отношение к их проектам. Обычно мы не используем слайды, но подготовили набор таковых для тех, кто предпочитает такой стиль изложения.

Если вы захотите получить копию этих материалов, напишите нам по электронной почте. Мы уже разослали их многим преподавателям из разных стран, и кто-то поделился в ответ своими материалами.

И напоследок одна просьба: если вы используете бесплатные онлайн-главы, пожалуйста, давайте только **ссылку** на них, не создавая локальную

копию. Это поможет нам отслеживать, как ими пользуются (за прошедшие несколько лет эти главы скачивали более миллиона раз!), и гарантирует, что студентам будут доступны самые актуальные (и самые лучшие?) версии.

Студентам

Если вы студент, спасибо, что выбрали эту книгу! Для нас большая честь предложить материал, который поможет вам в приобретении знаний об операционных системах. Мы с теплотой вспоминаем об учебниках нашей юности (например, о Hennessy and Patterson [HP90] – классической книге по архитектуре компьютеров) и надеемся, что у вас останутся положительные воспоминания об этой книге.

Вероятно, вы обратили внимание, что англоязычный оригинал данной книги свободно доступен в сети¹. Тому есть веская причина – учебники очень дороги. Мы надеемся, что эта книга станет первой в череде бесплатных материалов, доступных всем, кто ищет образования, вне зависимости от того, в какой части мира они проживают и сколько денег готовы потратить на книгу. Ну а если не получится, что ж, и одна бесплатная книга лучше, чем ничего.

Мы также надеемся всюду, где возможно, указывать оригинальные источники представленных в книге материалов: великие статьи и людей, которые на протяжении многих лет формировали научный подход к операционным системам. Идеи не возникают из ничего, это результат труда умных и напряженно работавших людей (включая многих лауреатов премии Тьюринга²), поэтому мы должны отдавать должное этим людям и их идеям. Надеемся, что так будет понятнее, какие грандиозные революции имели место, чем если бы мы просто делали вид, что эти идеи были известны всегда [K62]. И быть может, такие отсылки побудят вас к более глубоким самостоятельным изысканиям, ведь чтение основополагающих статей – безусловно, один из лучших способов изучить соответствующую дисциплину.

Благодарности

В этом разделе мы благодарим тех, кто помогал нам в создании книги. Кстати, заметьте: **и ваше имя могло бы здесь появиться!** Но для этого вы должны чем-то помочь. Пришлите нам свой отзыв, помогите «отладить» эту книгу – и вы прославитесь! Ну, или по крайней мере увидите свое имя в книге.

Вот имена тех, кто помогал нам: Aaron Gember (Colgate), Aashrith H Govindraj (USF), Abhinav Mehra, Abhirami Senthilkumaran*, Adam Drescher* (WUSTL),

¹ Небольшое отступление: здесь «свободно» не означает «с открытым кодом» и вовсе не значит, что книга не защищена, как положено, авторским правом, – защищена! А означает это, что вы можете скачивать главы и пользоваться ими при изучении операционных систем. Почему текст книги не является открытым по аналогии с открытым кодом ядра Linux? Потому что мы считаем, что у книги должен быть один авторитетный источник, и приложили немало усилий, чтобы стать таковым. Предполагается, что процесс чтения книги должен восприниматься как диалог с человеком, который что-то вам объясняет. Таков наш взгляд на вещи.

² Премия Тьюринга – высшая награда по информатике, ее можно сравнить с Нобелевской премией, правда, известна она не так широко.

Adam Eggum, Aditya Venkataraman, Adriana Iamnitchi and class (USF), Ahmad Jarara, Ahmed Fikri*, Ajaykrishna Raghavan, Akiel Khan, Alex Curtis, Alex Wyler, Alex Zhao (U. Colorado at Colorado Springs), Ali Razeen (Duke), Alistair Martin, Amir Behzad Eslami, Anand Mundada, Andrew Mahler, Andrew Valencik (Saint Mary's), Angela Demke Brown (Toronto), Antonella Bernobich (UoPeople)*, Arek Bulski, B. Brahmananda Reddy (Minnesota), Bala Subrahmanyam Kambala, Bart Miller, Ben Kushigian (U. Mass), Benita Bose, Biswajit Mazumder (Clemson), Bobby Jack, Björn Lindberg, Brandon Harshe (U. Minn), Brennan Payne, Brian Gorman, Brian Kroth, Caleb Sumner (Southern Adventist), Cara Lauritzen, Charlotte Kissinger, Cheng Su, Chien-Chung Shen (Delaware)*, Christian Stober, Christoph Jaeger, C. J. Stanbridge (Memorial U. of Newfoundland), Cody Hanson, Constantinos Georgiades, Dakota Crane (U. Washington-Tacoma), Dan Soendergaard (U. Aarhus), Dan Tsafrir (Technion), Danilo Bruschi (Universita Degli Studi Di Milano), Darby Asher Noam Haller, David Hanle (Grinnell), David Hartman, Deepika Muthukumar, Demir Delic, Dennis Zhou, Dheeraj Shetty (North Carolina State), Dorian Arnold (New Mexico), Dustin Metzler, Dustin Passofaro, Eduardo Stelmaszczyk, Emad Sadeghi, Emil Hessman, Emily Jacobson, Emmett Witchel (Texas), Eric Freudenthal (UTEP), Eric Johansson, Erik Turk, Ernst Biersack (France), Fangjun Kuang (U. Stuttgart), Feng Zhang (IBM), Finn Kuusisto*, Giovanni Lagorio (DIBRIS), Glenn Bruns (CSU Monterey Bay), Glen Granzow (College of Idaho), Guilherme Baptista, Hamid Reza Ghasemi, Hao Chen, Henry Abbey, Hilmar Gustafsson (Aalborg University), Hrishikesh Amur, Huanchen Zhang*, Huseyin Sular, Hugo Diaz, Ilya Oblomkov, Itai Hass (Toronto), Jackson «Jake» Haenchen (Texas), Jagannathan Eachambadi, Jake Gillberg, Jakob Olandt, James Earley, James Perry (U. Michigan-Dearborn)*, Jan Reineke (Universität des Saarlandes), Jason MacLafferty (Southern Adventist), Jason Waterman (Vassar), Jay Lim, Jerod Weinman (Grinnell), Jhih-Cheng Luo, Jiao Dong (Rutgers), Jia-Shen Boon, Jiawen Bao, Jingxin Li, Joe Jean (NYU), Joel Kuntz (Saint Mary's), Joel Sommers (Colgate), John Brady (Grinnell), John Komenda, Jonathan Perry (MIT), Joshua Carpenter (NCSU), Jun He, Karl Wallinger, Kartik Singhal, Katherine Dudenas, Katie Coyle (Georgia Tech), Kaushik Kannan, Kemal Bıçakcı, Kevin Liu*, Lanyue Lu, Laura Xu, Lei Tian (U. Nebraska-Lincoln), Leonardo Medici (U. Milan), Leslie Schultz, Liang Yin, Lihao Wang, Looserof, Manav Batra (IIIT-Delhi), Manu Awasthi (Samsung), Marcel van der Holst, Marco Guazzzone (U. Piemonte Orientale), Mart Oskamp, Martha Ferris, Masashi Kishikawa (Sony), Matt Reichoff, Mattia Monga (U. Milan), Matty Williams, Meng Huang, Michael Machtel (Hochschule Konstanz), Michael Walfish (NYU), Michael Wu (UCLA), Mike Griepentrog, Ming Chen (Stonybrook), Mohammed Alali (Delaware), Mohamed Omran (GUST), Murugan Kandaswamy, Nadeem Shaikh, Natasha Eilbert, Natasha Stopa, Nathan Dipiazza, Nathan Sullivan, Neeraj Badlani (N. C. State), Neil Perry, Nelson Gomez, Nghia Huynh (Texas), Nicholas Mandal, Nick Weinandt, Patel Pratyush Ashesh (BITS-Pilani), Patricio Jara, Pavle Kostovic, Perry Kivolowitz, Peter Peterson (Minnesota), Pieter Kockx, Radford Smith, Riccardo Mutschlechner, Ripudaman Singh, Robert Ordóñez со своим классом (Southern Adventist), Roger Wattenhofer (ETH), Rohan Das (Toronto)*, Rohan Pasalkar (Minnesota), Rohan Puri, Ross Aiken, Ruslan Kiselev, Ryland Herrick, Sam Kelly, Sam Noh (UNIST), Samer Al-Kiswany, Sandeep Um-madi (Minnesota), Sankaralingam Panneerselvam, Satish Chebrolu (Net App), Satyanarayana Shanmugam*, Scott Catlin, Scott Lee (UCLA), Seth Pollen, Sharad

Punuganti, Shreevatsa R., Simon Pratt (Waterloo), Sivaraman Sivaraman*, Song Jiang (Wayne State), Spencer Harston (Weber State), Srinivasan Thirunarayanan*, Stefan Dekanski, Stephen Bye, Suriyaparakhas Balaram Sankari, Sy Jin Cheah, Teri Zhao (EMC), Thanumalayan S. Pillai, Thomas Griebel, Thomas Scrace, Tianxia Bai, Tong He, Tongxin Zheng, Tony Adkins, Torin Rudeen (Princeton), Tuo Wang, Tyler Couto, Varun Vats, Vikas Goel, Waciuma Wanjohi, William Royle (Grinnell), Xiang Peng, Xu Di, Yifan Hao, Yuanyuan Chen, Yubin Ruan, Yudong Sun, Yue Zhuo (Texas A&M), Yufui Ren, Zef Rosn Brick, Zeyuan Hu (Texas), Zihan Zheng (USTC), Zuyu Zhang. Отдельное спасибо людям, чьи имена помечены звездочкой, – они не ограничились просто предложениями, как улучшить книгу.

Также сердечное спасибо профессору Джо Михину (Joe Meehean, Lynchburg) за подробные комментарии к каждой главе, профессору Джероу Вейнману (Jerod Weinman, Grinnell) и всему его курсу за потрясающие буклеты, профессору Чие Чунь Шену (Chien-Chung Shen, Delaware) за бесценные и очень подробные комментарии, Адаму Дрешеру (Adam Drescher, WUSTL) за внимательное прочтение и предложения, Глену Гранцоу (Glen Granzow, College of Idaho) за невероятно подробные замечания и советы, Майклу Волфишу (Michael Walfish, NYU) за энтузиазм и предложения по улучшению, Питеру Петерсону (Peter Peterson, UMD) за полезные отзывы и комментарии, Марку Кампе (Mark Kampe, Pomona) за детальные критические замечания (еще бы нам учесть все его предложения!) и Юджиу Вону (Youjip Won, Hanyang) за перевод на корейский (!) язык и многочисленные пронизательные предложения. Все они оказали авторам неоценимую помощь в улучшении представленного в книге материала.

Также большое спасибо сотням, выбравшим курс под номером 537. В частности, слушателям, пришедшим осенью 2008 года, которые побудили нас подготовить первый письменный вариант этих заметок (они так страдали от отсутствия учебника, настырные наши студенты), а затем так высоко оценили их, что мы вынуждены были продолжить (вспоминается это зазорное «Блин! Вы определенно должны написать целый учебник!» в студенческих оценках курса за тот год).

Мы также в большом долгу перед теми немногими храбрецами, которые брали в качестве темы для лабораторных работ проект xv6, значительная часть которого теперь включена в основной курс 537. Весна 2009: Justin Cherniak, Patrick Deline, Matt Czech, Tony Gregerson, Michael Griepentrog, Tyler Harter, Ryan Kroiss, Eric Radzikowski, Wesley Reardan, Rajiv Vaidyanathan, Christopher Waclawik. Осень 2009: Nick Bearson, Aaron Brown, Alex Bird, David Capel, Keith Gould, Tom Grim, Jeffrey Hugo, Brandon Johnson, John Kjell, Boyan Li, James Loethen, Will McCardell, Ryan Szaroletta, Simon Tso, Ben Yule. Весна 2010: Patrick Blesi, Aidan Dennis-Oehling, Paras Doshi, Jake Friedman, Benjamin Frisch, Evan Hanson, Pikkili Hemanth, Michael Jeung, Alex Langenfeld, Scott Rick, Mike Treffert, Garret Staus, Brennan Wall, Hans Werner, Soo-Young Yang, Carlos Griffin (почти).

Хотя прямой помощи в работе над книгой наши выпускники не оказали, они научили нас многому из того, что мы знаем о системах. Мы регулярно общаемся с ними в стенах Висконсинского университета, но все они заняты реальным делом – и каждую неделю, когда они рассказывают нам о том, чем занимаются, мы узнаем что-то новое. Ниже приведен перечень настоящих и бывших студентов и постдоков, с которыми мы публиковали статьи,

звездочкой отмечены те, кто защитил докторскую диссертацию под нашим руководством: Abhishek Rajimwale, Aishwarya Ganesan, Andrew Krioukov, Ao Ma, Brian Forney, Chris Dragga, Deepak Ramamurthi, Dennis Zhou, Edward Oakes, Florentina Popovici*, Hariharan Gopalakrishnan, Haryadi S. Gunawi*, James Nugent, Joe Meehan*, John Bent*, Jun He, Kevin Houck, Lanyue Lu*, Lakshmi Bairavasundaram*, Laxman Visampalli, Leo Arulraj*, Leon Yang, Meenali Rungta, Muthian Sivathanu*, Nathan Burnett*, Nitin Agrawal*, Ram Alagappan, Samer Al-Kiswany, Scott Hendrickson, Sriram Subramanian*, Stephen Todd Jones*, Stephen Sturdevant, Sudarsun Kannan, Suli Yang*, Swaminathan Sundararaman*, Swetha Krishnan, Thanh Do*, Thanumalayan S. Pillai*, Timothy Denehy*, Tyler Harter*, Venkat Venkataramani, Vijay Chidambaram*, Vijayan Prabhakaran*, Yiy-ing Zhang*, Yupu Zhang*, Yuvraj Patel, Zev Weiss*.

Наши выпускники в основном получали финансирование от Национального научного фонда (NSF), от научного управления Министерства энергетики и по отраслевым грантам. Мы особенно благодарны NSF за многолетнюю поддержку, поскольку наши исследования легли в основу многих глав этой книги.

Мы благодарим Томаса Грибеля (Thomas Griebel), который настоял на более качественной обложке книги. Хотя конкретно его предложение мы отклонили (динозавр, можете себе представить?), без него на обложке не появилось бы красивое изображение кометы Галлея.

И напоследок мы хотели бы выразить благодарность Аарону Брауну (Aaron Brown), который сначала выбрал этот курс (весной 2009), затем в качестве лабораторного проекта выбрал xv6 (осенью 2009) и наконец, уже после выпуска, два с лишним года работал ассистентом профессора на курсе (с весны 2010-го до весны 2012-го). Его неустанными трудами состояние проектов (особенно относящихся к xv6) значительно улучшилось, а стало быть, бесчисленным студентам и аспирантам Висконсинского университета стало приятнее учиться. Аарон сказал бы (в своей обычной лаконичной манере): «Спасибки».

В заключение

Хорошо известно знаменитое высказывание Йейтса: «Ученик – это не сосуд, который надо наполнить, а факел, который надо зажечь»¹. Он был одновременно прав и неправ². Сосуд все-таки надо немного заполнить, а эти заметки как раз и призваны способствовать данной части обучения; в конце концов, когда на собеседовании в Google вас спрашивают, как используются семафоры, неплохо бы знать, что такое семафор, верно?

Но в более общем смысле Йейтс, конечно, прав: истинная цель образования – заинтересовать чем-то, узнать больше о предмете самостоятельно, а не просто сделать краткий доклад в аудитории и получить хорошую оценку. Как говаривал отец одного из нас (папа Ремзи, Ведат Арпачи): «Учитесь за пределами классной комнаты».

¹ Эти слова часто приписывают поэту Йейтсу в англосаксонском мире, тогда как в русской традиции их автором принято считать Плутарха. – *Прим. перев.*

² Если это сказал действительно он; как и со многими знаменитыми цитатами, история этого перла темна и запутанна.

Мы написали эти заметки, чтобы заинтересовать вас изучением операционных систем, чтобы побудить вас читать дальше уже самостоятельно, беседовать с профессором обо всем, что происходит в этой области, и даже принять участие в исследованиях. Это потрясающая область знаний (!), полная чудесных и волнующих идей, которые стали важными вехами в истории вычислительной техники и информатики. И хотя мы понимаем, что этот факел зажжется не для каждого из вас, надеемся, что его увидят многие, ну хотя бы некоторые студенты. Потому что если факел зажегся, то вы сможете создать что-то по-настоящему великое. А это и есть истинная цель процесса обучения: идти вперед, изучать новые влекущие дисциплины, учиться, взрослеть и, самое главное, найти то, что станет для вас факелом.

Андреа и Ремзи, супруги.
Профессора информатики в Висконсинском университете,
главные возжигатели факелов (хочется надеяться)¹

Литература

[CK+08] «The xv6 Operating System» by Russ Cox, Frans Kaashoek, Robert Morris, Nickolai Zeldovich. Из: <http://pdos.csail.mit.edu/6.828/2008/index.html>. *Проект xv6 разрабатывался как перенос оригинальной системы UNIX версии 6 и представляет собой элегантный, чистый и простой способ разобраться в современной операционной системе.*

[F96] «Six Easy Pieces: Essentials Of Physics Explained By Its Most Brilliant Teacher» by Richard P. Feynman. Basic Books, 1996. *Эта книга представляет собой репринт шести более легких глав фейнмановских лекций по физике, прочитанных в 1963 году. Фантастическое чтение для любителей физики.*

[HP90] «Computer Architecture a Quantitative Approach» (1st ed.) by David A. Patterson and John L. Hennessy. Morgan-Kaufman, 1990. *Когда мы были студентами-младшекурсниками, эта книга вдохновила нас продолжить изучение на старших курсах, впоследствии нам обоим посчастливилось работать с Паттерсоном, который во многом определил направление наших научных карьер.*

[KR88] «The C Programming Language» by Brian Kernighan and Dennis Ritchie. Prentice-Hall, April 1988. *Главный учебник языка программирования C, написанный его авторами; должен стоять у каждого на книжной полке.*

[K62] «The Structure of Scientific Revolutions» by Thomas S. Kuhn. University of Chicago Press, 1962. *Знаменитая и замечательная книга о фундаментальных основах научного процесса. Черновая работа, аномалия, кризис и революция. Мы в основном занимаемся черновой работой, увы.*

¹ Если вам показалось, будто этим мы признаемся, что в прошлом были поджигателями, то, вероятно, вы чего-то не поняли. Вероятно. Если это прозвучало глуповато, что ж, наверное, так оно и есть, но вам придется простить нас за это.

Глава 1

Диалог о книге

Профессор. Добро пожаловать! Эта книга называется «Операционные системы: Три простых элемента», а я здесь для того, чтобы научить вас тому, что нужно знать об операционных системах. Меня зовут «Профессор», а тебя?

Студент. Добрый день, Профессор! Меня зовут «Студент», как вы, наверное, уже догадались. И я здесь для того, чтобы учиться!

Профессор. Звучит неплохо. Есть какие-нибудь вопросы?

Студент. А как же! Почему книга называется «Три простых элемента»?

Профессор. Это простой вопрос. Видишь ли, существуют знаменитые лекции Ричарда Фейнмана по физике...

Студент. А! Тот чувак, что написал «Вы, конечно, шутите, мистер Фейнман», да? Суперская книга! А эта такая же веселая, как та?

Профессор. Гм... ну, нет. Та книга великолепна, и я рад, что ты ее прочитал. Лыцу себя надеждой, что эта книга больше похожа на его заметки по физике. Некоторые основные положения кратко изложены в книге «Шесть простых элементов». Он писал о физике, а тема этой книги – три простых элемента операционных систем. Это разумно, потому что операционные системы примерно в два раза проще физики.

Студент. Что ж, физика мне нравилась, так что это, наверное, неплохо. А что это за элементы?

Профессор. Это три ключевые идеи, о которых мы будем говорить: виртуализация, конкурентность и хранение. По ходу дела мы узнаем, как работает операционная система и, в частности, как она решает, какая программа займет процессор в следующий раз, как система виртуальной памяти помогает справиться с нехваткой памяти физической, как работают мониторы виртуальных машин, как управлять информацией, хранящейся на дисках, и даже немного поговорим о том, как построить распределенную систему, которая продолжает работать, когда некоторые ее части выходят из строя. Такой вот план.

Студент. Честно говоря, вообще не понял, о чем вы толкуете.

Профессор. И прекрасно! Значит, ты попал точно по адресу.

Студент. У меня еще вопрос: как лучше всего научиться всему этому?

Профессор. *Отличный вопрос! Конечно, у каждого человека свой способ, но вот как поступил бы я сам: записался бы на курс, слушал, как профессор излагает новый материал. Затем в конце каждой недели читал бы эти заметки, чтобы идеи лучше осели в голове. Разумеется, спустя некоторое время (подсказка: до экзамена!) прочитал бы эти заметки снова, чтобы закрепить знания. Без сомнения, профессор будет давать домашние задания и учебные проекты, их надо выполнять; кстати, работа над проектами, в которых нужно писать реальный код для решения реальных задач, – лучший способ воплотить изложенные в этих заметках идеи на практике. Как сказал Конфуций...*

Студент. *А, знаю, знаю! «Я слышу и забываю. Я вижу и забываю. Я делаю и понимаю». Или что-то в этом роде.*

Профессор (удивленно). *Откуда ты знаешь, что я собирался сказать?!*

Студент. *Ну, это же напрашивалось. К тому же я большой поклонник Конфуция и еще больший поклонник Сунь Цзы, который, пожалуй, лучше подходит на роль автора этого афоризма¹.*

Профессор (ошеломленно). *Что же, мне кажется, что мы отлично поладим. Просто отлично.*

Студент. *Профессор, еще один вопрос, если позволите. А для чего эти диалоги? В смысле, разве это не просто книга? Почему не излагать материал прямо?*

Профессор. *Ага, превосходный вопрос! Думается мне, что иногда полезно разорвать повествование и немного подумать; эти диалоги как раз на такой случай. Итак, ты и я, вместе, попытаемся разобраться во всех этих довольно сложных идеях. Готов?*

Студент. *Стало быть, нужно будет думать? Что ж, я готов. Да и что мне остается? Не похоже, что эта книга оставит мне много времени на личную жизнь.*

Профессор. *Как и мне, увы. Итак, за работу!*

¹ Если верить страничке http://www.barrypopik.com/index.php/new_york_city/entry/tell_me_and_i_forget_teach_me_and_i_may_remember_involve_me_and_i_will_learn/, философ-конфуцианец Сунь Цзы говорил: «Не слышать о чем-то хуже, чем слышать об этом; слышать о чем-то хуже, чем видеть это: видеть что-то хуже, чем знать это; знать что-то хуже, чем воплотить это в действии». Позже это мудрое высказывание почему-то было приписано Конфуцию. Спасибо Сяо Донгу (из Ратгерского университета), рассказавшему нам об этом.

Глава 2

Введение в операционные системы

Коль скоро вы записались на курс по операционным системам, то, надо полагать, уже имеете представление о том, что делает работающая компьютерная программа. Если же нет, то эта книга и соответствующий курс покажутся вам трудными – так что лучше, наверное, будет на время отложить ее, сбегать в ближайший книжный магазин и быстренько проглотить необходимый подготовительный материал (подойдут, например, прекрасные книги Patt & Patel [PP03] и Bryant & O'Hallaron [BOH10]).

Так что же происходит, когда программа запущена?

Работающая программа делает одну очень простую вещь – выполняет команды. Много миллионов (а в наши дни уже миллиардов) раз в секунду процессор **выбирает** команду из памяти, **декодирует** ее (т. е. выясняет, что это за команда) и **выполняет** (т. е. делает то, для чего предназначена команда, например складывает два числа, обращается к памяти, проверяет условие, совершает переход к функции и т. д.). Прodelав все это для данной команды, процессор переходит к следующей – и так, пока программа не завершится¹.

Только что мы описали основы модели вычислений **фон Неймана**². Кажется просто, не правда ли? Но на этом курсе мы узнаем, что пока программа работает, происходит много чего еще – и все для того, чтобы системой было **легко пользоваться**.

Существует большой пласт программного обеспечения, которое отвечает за простоту выполнения программ (и даже создание иллюзии, будто одно-

¹ Разумеется, современные процессоры совершают много странных и пугающих вещей, чтобы программы работали быстрее, например выполняют несколько команд одновременно и даже, возможно, не по порядку! Но сейчас нас это не интересует; нам важна только простая модель, предполагаемая большинством программ: стороннему наблюдателю представляется, что команды выполняются по одной, последовательно и строго по порядку.

² Фон Нейман – один из пионеров создания вычислительных систем. Он также автор пионерских работ по теории игр и атомной бомбе, а еще шесть лет играл в НБА. Ладно, признаемся, что одно из этих утверждений – ложь.

временно работает много программ), за то, чтобы программы могли общаться использовать память, чтобы они могли взаимодействовать с периферийными устройствами и много других вещей. Это программное обеспечение называется **операционной системой (ОС)**¹, поскольку несет ответственность за правильное и эффективное функционирование системы, притом так, чтобы использовать ее было легко.

СУЩЕСТВО ПРОБЛЕМЫ: КАК ВИРТУАЛИЗИРОВАТЬ РЕСУРСЫ

Главный вопрос, на который мы дадим ответ в этой книге, прост: как система виртуализует ресурсы? Это и есть существо нашей проблемы. *Почему ОС это делает – не главное, поскольку ответ очевиден: чтобы системой было проще пользоваться.* Таким образом, нас будет интересовать, прежде всего, *как*: какие механизмы и политики задействует ОС, чтобы добиться виртуализации? Как ОС достигает эффективности? Какая нужна поддержка со стороны оборудования?

На подобных врезках «существо проблемы» на сером фоне мы будем формулировать конкретные проблемы, решаемые при построении операционной системы. В одной главе можете встретиться несколько таких врезок, освещающих проблему с разных сторон. А в тексте главы, конечно, описано решение или, по крайней мере, основные его параметры.

Основной способ, которым ОС решает эту задачу, – общая техника, именуемая **виртуализацией**. Это значит, что ОС берет **физический** ресурс (например, процессор, память или диск) и преобразует его в более общую, более мощную и более простую в использовании **виртуальную форму**. Поэтому иногда операционная система называется **виртуальной машиной**.

Разумеется, чтобы пользователи могли сказать ОС, что делать, и тем самым воспользоваться средствами виртуальной машины (например, выполнить программу, выделить память или обратиться к файлу), ОС должна предоставлять какие-то интерфейсы (API), которые можно вызвать. Типичная ОС экспортирует сотни **системных вызовов**, доступных приложениям. Поскольку ОС предоставляет эти вызовы, чтобы выполнять программы, обращаться к памяти и устройствам и совершать другие подобные действия, иногда говорят, что ОС предоставляет приложениям **стандартную библиотеку**.

Наконец, поскольку виртуализация позволяет запускать много программ (и, стало быть, общаться использовать один центральный процессор), а эти программы могут одновременно обращаться к своим командам и данным (т. е. общаться использовать память), а также к устройствам (т. е. общаться использовать диски и т. д.), то ОС иногда называют **диспетчером ресурсов**. Процессор, память и диск – примеры **ресурсов** системы, а роль операционной системы сводится к тому, чтобы **управлять** этими ресурсами, делая это эффективно или справедливо или стремясь к достижению еще какой-то ведомой ей цели. Чтобы лучше понять роль ОС, рассмотрим несколько примеров.

¹ Раньше для ОС были в ходу и другие названия: **супервизор** и даже **главная управляющая программа**. Последнее название звучит уж слишком грозно (посмотрите фильм «Трон», там найдете все детали), так что, к счастью, устоялось название «операционная система».

2.1. ВИРТУАЛИЗАЦИЯ ПРОЦЕССОРА

На рис. 2.1 показана наша первая программа. Делает она немного – всего-то вызывает функцию `Spin()`, которая повторно проверяет время и возвращает управление по прошествии одной секунды. Затем программа печатает строку, переданную пользователем в командной строке, и возвращается в начало цикла. И так бесконечно.

```

1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <sys/time.h>
4 #include <assert.h>
5 #include "common.h"
6
7 int
8 main(int argc, char *argv[])
9 {
10     if (argc != 2) {
11         fprintf(stderr, "usage: cpu <string>\n");
12         exit(1);
13     }
14     char *str = argv[1];
15     while (1) {
16         Spin(1);
17         printf("%s\n", str);
18     }
19     return 0;
20 }
```

Рис. 2.1 ❖ Простой пример:
программа крутится в цикле и печатает (cpu.c)

Допустим, что мы сохранили этот код в файле `cpu.c` и решили откомпилировать и запустить его в системе с одним процессором (иногда мы будем называть его **CPU**). Вот что мы увидим:

```

prompt> gcc -o cpu cpu.c -Wall
prompt> ./cpu "A"
A
A
A
A
^C
prompt>
```

Не особенно интересно – система начинает выполнять программу, которая повторно проверяет время, пока не пройдет одна секунда. Как только секунда прошла, программа печатает строку, переданную пользователем (в данном случае букву «А»), и все повторяется. Эта программа будет работать вечно; лишь нажатием комбинации клавиш **Control-c** (в UNIX-системах

она останавливает программу, работающую в приоритетном режиме) мы можем ее снять.

Теперь повторим эксперимент, но на этот раз запустим несколько экземпляров одной и той же программы. На рис. 2.2 показаны результаты этого чуть более сложного примера.

```
prompt> ./cpu A & ; ./cpu B & ; ./cpu C & ; ./cpu D &
[1] 7353
[2] 7354
[3] 7355
[4] 7356
A
B
D
C
A
B
D
C
A
C
B
D
...
```

Рис. 2.2 ❖ Выполнение сразу нескольких программ

Вот это уже интереснее. Хотя процессор всего один, создается впечатление, что все четыре программы каким-то образом работают одновременно! И как такое чудо могло случиться¹?

На самом деле операционная система, которой помогает оборудование, создает **иллюзию**, будто в системе имеется очень много виртуальных процессоров. Превращение одного (или нескольких) CPU в кажущееся бесконечным число CPU и как следствие иллюзия одновременного выполнения большого числа программ и называется **виртуализацией процессора**. Это главная тема первого большого раздела книги.

Разумеется, чтобы запускать программы, останавливать их и вообще сообщать ОС, какие программы должны работать, необходимы интерфейсы (API), позволяющие поведать ОС о наших желаниях. Мы будем говорить об этих API на всем протяжении книги, они-то и являются основным способом взаимодействия пользователей с операционной системой.

Возможно, вы заметили, что возможность запускать одновременно несколько программ ставит новые вопросы. Например, если в какой-то момент

¹ Обратите внимание, что для запуска четырех процессов одновременно мы воспользовались символом &. В оболочке tcsh он запускает задание в фоновом режиме, т. е. пользователь может сразу же вводить следующую команду – в нашем случае запустить другую программу. Точка с запятой между командами позволяет запустить несколько программ одновременно. В другой оболочке (например, bash) все может быть устроено немного иначе, подробности ищите в документации.

времени хотят работать две программы, какой отдать предпочтение? На этот вопрос отвечает **политика** ОС; политики используются во многих местах для ответа на подобные вопросы, поэтому мы будем изучать их, когда узнаем о базовых **механизмах**, реализуемых операционными системами (в т. ч. механизме, позволяющем выполнять сразу несколько программ). Отсюда и роль ОС как **диспетчера ресурсов**.

2.2. Виртуализация памяти

Теперь переключим внимание на память. Модель **физической памяти** в современных компьютерах очень проста. Память – это просто массив байтов; чтобы **прочитать** из памяти, нужно задать адрес хранящихся в ней данных, а чтобы **записать** в память (или **обновить** ее), нужно дополнительно задать данные, помещаемые по указанному адресу.

Доступ к памяти производится на протяжении всего времени работы программы. Программа хранит все свои данные в памяти и обращается к ним с помощью различных команд: загрузки, сохранения и др. Не забывайте, что все команды программы тоже хранятся в памяти, поэтому доступ к памяти производится при выборе каждой команды.

Рассмотрим программу (рис. 2.3), которая выделяет память путем обращения к функции `malloc()`. Вот что она выводит на экран:

```
prompt> ./mem
(2134) адрес, на который указывает p: 0x200000
(2134) p: 1
(2134) p: 2
(2134) p: 3
(2134) p: 4
(2134) p: 5
^C
```

Эта программа делает две вещи. Сначала она выделяет блок памяти (строка a1). Затем печатается адрес этого блока (a2), после чего в первый элемент только что выделенного блока записывается нуль (a3). Наконец, программа возвращается в начало цикла, ожидает в течение одной секунды и увеличивает на единицу значение, хранящееся по адресу, который находится в `p`. В каждом предложении печати выводится также идентификатор процесса (PID) выполняемой программы. У каждого работающего процесса имеется свой уникальный PID.

И снова результат не особенно впечатляет. Блок памяти выделен по адресу `0x200000`. По ходу работы программа медленно обновляет значение и печатает результат.

Теперь, как и раньше, запустим несколько экземпляров этой программы и посмотрим, что будет (рис. 2.4). Из распечатки видно, что обе работающие программы выделили блок памяти по одному и тому же адресу (`0x200000`), но каждая обновляет значение по адресу `0x200000` независимо! Складывается

впечатление, что работающие программы не разделяют одну и ту же физическую память, а владеют собственной частной памятью¹.

```

1 #include <unistd.h>
2 #include <stdio.h>
3 #include <stdlib.h>
4 #include "common.h"
5
6 int
7 main(int argc, char *argv[])
8 {
9     int *p = malloc(sizeof(int)); // a1
10    assert(p != NULL);
11    printf("(24113) адрес, на который указывает p: %p\n",
12           getpid(), p); // a2
13    *p = 0; // a3
14    while (1) {
15        Spin(1);
16        *p = *p + 1;
17        printf("(24114) p: %d\n", getpid(), *p); // a4
18    }
19    return 0;
20 }
```

Рис. 2.3 ❖ Программа, обращающаяся к памяти

```

prompt> ./mem & ./mem &
[1] 24113
[2] 24114
(24113) адрес, на который указывает p: 0x200000
(24114) адрес, на который указывает p: 0x200000
(24113) p: 1
(24114) p: 1
(24114) p: 2
(24113) p: 2
(24113) p: 3
(24114) p: 3
(24113) p: 4
(24114) p: 4
...
```

Рис. 2.4 ❖ Выполнение нескольких экземпляров программы обращения к памяти

И действительно, именно это здесь и происходит – вследствие **виртуализации памяти**, осуществляемой ОС. Каждый процесс обращается к собст-

¹ Чтобы этот пример работал, как показано, необходимо отключить рандомизацию адресного пространства. Рандомизация – это механизм защиты от некоторых видов атак. Почитайте об этом самостоятельно, особенно если хотите узнать, как можно взломать компьютерную систему путем атаки с переполнением стека. Не подумайте, что это рекомендация...

венному частному **виртуальному адресному пространству** (иногда оно называется просто **адресным пространством**), которое ОС каким-то образом отображает на физическую память машины. Обращение к памяти внутри одной программы никак не влияет на адресное пространство других процессов (и самой ОС); с точки зрения работающей программы, вся физическая память принадлежит ей. Но на самом деле физическая память – это разделяемый ресурс, управляемый операционной системой. Как именно она это делает, также обсуждается в первой части книги, посвященной **виртуализации**.

2.3. Конкурентность

Вторая из основных тем этой книги – **конкурентность**. Мы пользуемся этим общим термином для обозначения совокупности проблем, которые возникают и должны решаться, когда одновременно (т. е. конкурентно) в одной программе производятся действия с несколькими объектами. Первоначально проблемы конкурентности возникли в самой операционной системе; в приведенных выше примерах мы видели, что ОС манипулирует сразу несколькими сущностями: запускает один процесс, потом другой и т. д. Как выясняется, это поднимает ряд глубоких и интересных вопросов.

К сожалению, проблемы конкурентности вышли за пределы самой операционной системы. И в современных **многопоточных** программах возникают те же самые проблемы. Продемонстрируем их на примере программы на рис. 2.5.

Хотя в данный момент вы, возможно, не понимаете все детали этого кода (мы разберемся с ними в разделе, посвященном конкурентности), основная идея проста. Главная программа создает два **потока**, вызывая функцию `Pthread_create()`¹. Потоки можно представлять себе как функции, одновременно работающие в одном и том же пространстве памяти. В примере выше каждый поток запускает функцию `worker()`, которая просто `loops` раз увеличивает счетчик в цикле.

СУЩЕСТВО ПРОБЛЕМЫ: КАК ПРАВИЛЬНО ПИСАТЬ КОНКУРЕНТНЫЕ ПРОГРАММЫ

Как правильно написать программу, если в одном и том же пространстве памяти может конкурентно выполняться несколько потоков? Какие примитивы ОС нам для этого понадобятся? Какие механизмы должно обеспечить оборудование? Как ими воспользоваться для решения проблем конкурентности?

Ниже показано, что печатает эта программа, когда переменной `loops` присвоено начальное значение 1000. Значение `loops` определяет, сколько раз

¹ Вообще-то, функция правильно называется `pthread_create()`; имя, начинающееся заглавной буквой, принадлежит нашей обертке, которая вызывает `pthread_create()` и проверяет, что та завершилась успешно. Детали см. в коде.

каждый из исполнителей (экземпляров функции `worker`) увеличивает разделяемый счетчик. Если вначале оно равно 1000, то как вы думаете, каким будет конечное значение `counter`?

```
prompt> gcc -o thread thread.c -Wall -pthread
```

```
prompt> ./thread 1000
```

```
Начальное значение : 0
```

```
Конечное значение  : 2000
```

```

1 #include <stdio.h>
2 #include <stdlib.h>
3 #include "common.h"
4
5 volatile int counter = 0;
6 int loops;
7
8 void *worker(void *arg) {
9     int i;
10    for (i = 0; i < loops; i++) {
11        counter++;
12    }
13    return NULL;
14 }
15
16 int
17 main(int argc, char *argv[])
18 {
19     if (argc != 2) {
20         fprintf(stderr, "usage: threads <value>\n");
21         exit(1);
22     }
23     loops = atoi(argv[1]);
24     pthread_t p1, p2;
25     printf("Начальное значение : %d\n", counter);
26
27     Pthread_create(&p1, NULL, worker, NULL);
28     Pthread_create(&p2, NULL, worker, NULL);
29     Pthread_join(p1, NULL);
30     Pthread_join(p2, NULL);
31     printf("Конечное значение : %d\n", counter);
32     return 0;
33 }
```

Рис. 2.5 ❖ Многопоточная программа (`threads.c`)

Вы, наверное, уже догадались, что по завершении обоих потоков конечное значение `counter` будет равно 2000, поскольку каждый поток увеличил счетчик 1000 раз. Действительно, если начальное значение `loops` равно N , то следует ожидать, что в конце программы счетчик окажется равным $2N$. Но, как выясняется, жизнь устроена сложнее. Давайте-ка запустим ту же программу с большим значением `loops` и посмотрим, что произойдет:


```
prompt> ./thread 100000
Начальное значение : 0
Конечное значение : 143012 // странно
prompt> ./thread 100000
Начальное значение : 0
Конечное значение : 137298 // что за черт?!
```

На этот раз мы задали начальное значение 100 000, но вместо конечного значения 200 000 получили 143 012. А запустив программу еще раз, мы мало того что получили *неверное* значение, так еще и *отличающееся* от полученного в первый раз. На самом деле если запускать программу снова и снова с большим значением `loops`, то иногда будет даже получаться правильный ответ! Так что же происходит?

Причина таких странных результатов связана с тем, что команды выполняются по одной. Но, к сожалению, самая главная часть этой программы – та, где счетчик увеличивается на единицу, – требует трех команд: первая загружает значение счетчика из памяти в регистр, вторая увеличивает его, а третья записывает обратно в память. Поскольку эти три команды выполняются не **атомарно** (как неделимое целое), происходят странные вещи. Именно эту проблему конкурентности мы будем подробнейшим образом рассматривать во второй части книги.

2.4. ХРАНЕНИЕ

Третья из главных тем этого курса – **хранение**. В памяти системы данные легко могут быть утрачены, потому что такие запоминающие устройства, как ДЗУПВ (англ. *DRAM*), являются **энергозависимыми** – если питание пропадает или система внезапно выходит из строя, то все данные в памяти теряются. Таким образом, нам нужно, чтобы оборудование и программное обеспечение могли сохранять данные на постоянной основе; подобные системы хранения – важнейшая часть любой системы, т. к. пользователи дорожат своими данными.

Оборудование выступает в форме того или иного устройства **ввода-вывода**; в современных системах типичным хранилищем долговечной информации является **жесткий диск**, хотя постепенно на эту роль выдвигаются **твердотельные запоминающие устройства** (англ. *SSD*).

Та часть операционной системы, которая занимается управлением диском, называется **файловой системой**, она отвечает за надежное и эффективное хранение созданных пользователями **файлов** на системных дисках.

В отличие от абстракций процессора и памяти, ОС не создает частного виртуализированного диска для каждого приложения. Напротив, предполагается, что пользователи зачастую желают **разделять** информацию, находящуюся в файлах, т. е. пользоваться ей совместно. Например, при написании программы на С вы сначала используете редактор (например, Emacs¹), чтобы

¹ Верим, что вы используете именно Emacs. Если вы пользуетесь vi, то, видимо, с вами что-то не так. А уж если вы работаете с чем-то, вовсе не являющимся настоящим редактором кода, то дела обстоят совсем плохо.

создать и редактировать С-файл (`emacs -nw main.c`). Затем вы запускаете компилятор, чтобы преобразовать исходный код в исполняемый (например, `gcc -o main main.c`). Покончив с этим, вы запускаете новый исполняемый файл (например, `./main`). Таким образом, одни и те же файлы используются разными процессами. Сначала Emacs создает файл, который подается на вход компилятору, затем компилятор создает из него новый исполняемый файл (для чего требуется много шагов, о которых вы можете узнать на курсе по компиляторам), и, наконец, этот исполняемый файл запускается. Так рождается новая программа!

Чтобы лучше разобраться в этом, возьмем какой-нибудь код. На рис. 2.6 показана программа создания файла (`/tmp/file`), содержащего строку «hello world».

```

1 #include <stdio.h>
2 #include <unistd.h>
3 #include <assert.h>
4 #include <fcntl.h>
5 #include <sys/types.h>
6
7 int
8 main(int argc, char *argv[])
9 {
10     int fd = open("/tmp/file", O_WRONLY | O_CREAT | O_TRUNC, S_IRWXU);
11     assert(fd > -1);
12     int rc = write(fd, "hello world\n", 13);
13     assert(rc == 13);
14     close(fd);
15     return 0;
16 }
```

Рис. 2.6 ❖ Программа ввода-вывода (`io.c`)

СУЩЕСТВО ПРОБЛЕМЫ: КАК СОХРАНЯТЬ ДАННЫЕ НА ПОСТОЯННОЙ ОСНОВЕ

Файловая система – это часть ОС, отвечающая за управление постоянно хранимыми данными. Какие методы необходимы для корректного решения этой задачи? Какие механизмы и политики позволят добиться при этом высокой производительности? Как обеспечить надежность в условиях возможных отказов оборудования и программ?

Для решения этой задачи программа трижды обращается к операционной системе. Первый вызов, `open()`, создает файл и открывает его, второй, `write()`, записывает в файл данные, а третий, `close()`, закрывает файл и тем самым дает программе знать, что больше операций записи в него не будет. Эти **системные вызовы** передаются части ОС, которая называется **файловой системой**, она обрабатывает запросы и возвращает программе код успеха или ошибки.

Вам, наверное, любопытно, что именно делает ОС, для того чтобы записать данные на диск. Мы покажем, но обещайте сначала закрыть глаза – непри-

ятное это зрелище. Файловая система должна проделать немало работы: сначала она вычисляет, в каком месте диска должны оказаться новые данные, а затем работает с различными структурами данных. Для этого необходимо отправлять команды ввода-вывода физическому запоминающему устройству, чтобы либо прочитать существующие структуры, либо обновить их. Всякий, кому доводилось писать **драйвер устройства**¹, знает, что заставить устройство что-то сделать от имени пользователя – сложная процедура, изобилующая тонкими деталями. Требуется глубокое знание низкоуровневого интерфейса устройства и его точной семантики. По счастью, ОС предлагает стандартный и простой способ доступа к устройствам с помощью системных вызовов. Поэтому ОС иногда рассматривают как **стандартную библиотеку**.

Конечно, поверх собственно устройств файловая система надстраивает много механизмов доступа к устройствам и управления данными. Из соображений производительности большинство файловых систем на некоторое время откладывают операции записи в надежде сформировать из них более крупные группы. Чтобы справиться с крахом системы во время записи, многие файловые системы включают хитроумные протоколы записи, например **журналирование** или **копирование при записи**, тщательно упорядочивая операции записи на диск таким образом, чтобы в случае сбоя система могла гарантированно восстановиться в каком-то разумном состоянии. Для повышения эффективности типичных операций файловые системы прибегают к разнообразным структурам данных и методам доступа, от простых списков до сложных B-деревьев. Если пока это вам ни о чем не говорит, не страшно! Мы разберемся во всем этом гораздо подробнее в третьей части книги, где будем обсуждать устройства и ввод-вывод сначала в общем, а затем на примере конкретных дисков, RAID-массивов и файловых систем.

2.5. Цели проектирования

Итак, теперь у вас есть представление о том, что же делает ОС: она берет физические **ресурсы** (процессор, память или диск) и **виртуализирует** их. Она берет на себя сложные вопросы, связанные с **конкурентностью**. И **хранит** файлы, обеспечивая безопасность данных на долгое время. Поскольку мы хотим построить такую систему, необходимо сформулировать цели, на достижении которых сосредоточить усилия по проектированию и реализации, идя по мере необходимости на компромиссы. Отыскание подходящего набора компромиссов – ключ к проектированию систем.

Одна из самых важных целей – выбор абстракций, при которых системой будет легко и удобно пользоваться. Абстракции – вообще основа всего в информатике. Благодаря абстрагированию мы можем написать большую программу, разделив ее на меньшие части, которые проще понять. Благо-

¹ Драйвером устройства называется часть операционной системы, которая знает, как обращаться с конкретным устройством. Ниже мы еще поговорим об устройствах и их драйверах.

даря ему же мы можем писать программы на языках высокого уровня типа C¹, не думая о языке ассемблера, писать код на языке ассемблера, не думая о логических вентилях, и собирать процессор из вентилях, не «замораживаясь» мыслями о транзисторах. Абстрагирование – настолько фундаментальная вещь, что иногда забывают о его важности, но мы такого не допустим и в каждом разделе будем обсуждать, какие абстракции уже разработали. Это поможет вам рассуждать о частях ОС.

Одна из целей проектирования и реализации операционной системы – обеспечить высокую **производительность**, или, как еще говорят, **минимизировать накладные расходы** ОС. Виртуализация и удобство пользования – достойные цели, но не любой ценой; необходимо, чтобы эти и другие механизмы ОС не сопровождалась непомерными накладными расходами. Накладные расходы могут принимать разную форму: перерасход времени (больше команд) или перерасход места (в памяти или на диске). Мы будем искать решения, которые минимизируют одно, другое или то и другое сразу, если возможно. Однако совершенство не всегда достижимо, мы научимся подмечать это и (там, где это приемлемо) мириться с этим.

Еще одна цель – обеспечить **защиту** приложений друг от друга и ОС от приложений. Поскольку мы хотим, чтобы много программ могли работать одновременно, требуется гарантировать, что намеренное или случайное некорректное поведение одной программы не нанесет урона другим; и уж точно мы не хотим, чтобы какое-то приложение могло повредить саму ОС (поскольку это отразилось бы на *всех* вообще программах, работающих в системе). Защита – сердцевина одного из главных принципов операционной системы – **изоляции**; изолирование процессов друг от друга – ключевая характеристика защиты и потому определяет многое из того, что должна делать ОС.

Операционная система должна работать бесперебойно; если она отказывает, то отказывают сразу *все* работающие в системе приложения. Поэтому ОС стремятся обеспечить высочайший уровень **надежности**. Поскольку со временем операционные системы только усложняются (иногда они содержат миллионы строк кода), построение надежной ОС – очень трудная задача, огромное число исследований в этой области (в т. ч. и наши собственные работы [BS+09, SS+10]) посвящены именно этой проблеме.

Есть и другие цели: **энергоэффективность**, которая приобретает особую важность в мире, который все сильнее озабочен экологией; **безопасность** (в действительности обобщение защиты) в условиях, когда существуют вредоносные приложения, особенно в наши времена повсеместного распространения сетей; **мобильность** становится все важнее, поскольку ОС работают на устройствах все меньшего и меньшего размера. В зависимости от характера использования системы стоящие перед ОС цели разнятся и потому реализуются несколькими различающимися способами. Однако, как мы увидим, многие описанные нами принципы построения ОС применимы к широкому кругу различных устройств.

¹ Возможно, кто-то не согласится с тем, что C – язык высокого уровня. Но напомним, что это курс по ОС, так что мы должны быть счастливы уже тем, что не приходится всю дорогу писать на ассемблере!

2.6. Немного истории

Прежде чем расстаться с этим введением, мы хотели бы изложить краткую историю развития операционных систем. Как и для любой созданной людьми системы, хорошие идеи аккумулировались на протяжении времени, по мере того как инженеры осознавали, что именно важно при проектировании ОС. Здесь мы обсудим лишь несколько наиболее заметных вех. Более полный рассказ можно найти в великолепной истории операционных систем, написанной Бринчем Хансеном [BH00].

Первые операционные системы: просто библиотеки

В самом начале операционные системы делали не слишком много. По существу, это был просто набор библиотек, содержащий часто используемые функции. Например, чтобы не заставлять каждого программиста писать низкоуровневый код ввода-вывода, «ОС» предоставляла соответствующие API, облегчая тем самым жизнь разработчикам.

Обычно на этих старых больших машинах – мейнфреймах – в каждый момент времени работала одна программа, и управлял всем этим оператор-человек. Много из того, что делала бы современная ОС (например, решение о том, в каком порядке запускать задания), возлагалось на оператора. Разумный разработчик завязал бы хорошие отношения с оператором, чтобы тот передвинул его задание в начало очереди.

Такой режим обработки назывался **пакетным**, поскольку задания собирались оператором в «пакет», который потом запускался на выполнение. В то время еще не было интерактивного режима работы, потому что заставлять пользователя сидеть перед компьютером и время от времени взаимодействовать с ним было бы попросту слишком дорого – ведь большую часть времени он бы ничего не делал, а вычислительному центру это обходилось бы в сотни долларов в час [BH00].

Не только библиотеки: защита

Перерастая простую библиотеку общеупотребительных служб, операционные системы начать играть более заметную роль в управлении машинами. И тут важно отметить осознание того, что код, работающий от имени ОС, имел особенности; поскольку он управлял устройствами, с ним следовало обращаться не как с кодом обычного приложения. Почему? Представьте, что любому приложению разрешено читать любой участок диска; тогда ни о какой конфиденциальности не может быть и речи, ведь любая программа может прочитать любой файл. Поэтому реализация **файловой системы** (с целью управления вашими файлами) в виде библиотеки не имеет смысла. Нужно было что-то другое.

Так возникла идея **системного вызова**, которая впервые была опробована в вычислительной системе Atlas [K+61, L78]. Вместо того чтобы реализовывать подпрограммы ОС в виде библиотеки (и для доступа к ним выполнять просто **вызов процедуры**), было предложено включить две специальные аппаратные команды и аппаратное же состояние, позволяющие переходить в ОС более формальным и контролируемым способом.

Ключевое отличие системного вызова от вызова процедур заключается в том, что системный вызов передает управление (т. е. совершает переход) ОС и одновременно увеличивает **аппаратный уровень привилегий**. Пользовательские приложения работают в так называемом **режиме пользователя**, когда оборудование ограничивает доступные приложению возможности; например, приложение, работающее в режиме пользователя, обычно не может инициировать запрос ввода-вывода к диску, обратиться к произвольной странице физической памяти или отправить пакет в сеть. Когда произведен системный вызов (обычно с помощью специальной команды **системного прерывания** (англ. *trap*), оборудование передает управление предопределенному **обработчику системных прерываний** (который ОС предварительно подготовила) и в то же время поднимает уровень привилегий до **режима ядра**. В режиме ядра ОС имеет полный доступ ко всему оборудованию и, следовательно, может инициировать запрос ввода-вывода, выделить программе дополнительную память и т. д. Закончив обслуживать запрос, ОС возвращает управление пользователю с помощью специальной команды **возврата из системного прерывания**, которая не только возобновляет работу с того места, где приложение прервалось, но и восстанавливает режим пользователя.

Эра мультипрограммирования

Настоящий взлет операционные системы пережили, когда на смену мейнфреймам пришли **мини-компьютеры**. Классические машины, в частности семейство PDP компании Digital Equipment, сделали компьютеры гораздо более доступными по цене; теперь вместо одного мейнфрейма на всю крупную организацию стало возможно выделить отдельный компьютер сравнительно небольшому коллективу. Неудивительно, что одним из главных последствий снижения стоимости стало возрастание активности разработчиков — у большего числа талантливых людей появилась возможность дорваться до компьютера, и в результате вычислительные системы стали делать более интересные и красивые вещи.

В частности, широкое распространение получило **мультипрограммирование** — вследствие желания более эффективно использовать машинные ресурсы. Вместо того чтобы выполнять задания по одному, ОС научились загружать несколько заданий в память и быстро переключаться между ними, повышая тем самым коэффициент использования процессора. Это переключение было особенно важно, потому что драйверы ввода-вывода работали медленно, а заставлять программу ждать, пока будет обслужено устройство ввода-вывода, значило бы непроизводительно расходовать процессорное

время. Почему бы вместо этого не переключиться на другое задание и не дать ему поработать?

Стремление поддержать мультипрограммирование и перекрытие во времени при наличии ввода-вывода и прерываний повлекло за собой концептуальное развитие операционных систем сразу в нескольких направлениях. Приобрели важность такие вопросы, как **защита памяти**; мы же не хотим, чтобы одна программа могла обращаться к памяти другой. Не менее важно было понять, как решать проблемы **конкурентности**, сопутствующие мультипрограммированию; во весь рост встал вопрос о том, как заставить ОС корректно вести себя, несмотря на прерывания. Мы изучим эти и смежные вопросы далее в этой книге.

Одним из главных практических достижений в то время стало появление операционной системы UNIX в основном благодаря усилиям Кена Томпсона (и Денниса Ритчи) из компании Bell Labs (да-да, телефонной компании). UNIX заимствовала много хороших идей у других операционных систем (в особенности Multics [O72] и кое-что у системы TENEX [B+72] и Berkeley Time-Sharing System [S+68]), но упростила их и сделала использование более удобным. Вскоре эта команда начала рассылать магнитные ленты с исходным кодом UNIX людям по всему миру, многие из них затем подключились к работе и начали вносить в систему свои добавления; подробнее см. врезку «Отступление» ниже¹.

Современность

На мини-компьютерах история не закончилась, вскоре появились машины нового типа – дешевле, быстрее и рассчитанные на массовое производство: **персональные компьютеры**, или **ПК**, как мы их сегодня называем. Вслед за ранними машинами Apple (например, Apple II) и IBM PC эта новая порода машин очень быстро завоевала доминирующие позиции на рынке вычислительной техники, поскольку низкая стоимость позволяла иметь отдельную машину на каждом рабочем столе вместо общего мини-компьютера на всю группу.

К сожалению, с точки зрения операционных систем, ПК поначалу знаменовали гигантский отскок назад, поскольку первые системы забыли (или никогда не знали) уроки, выученные в эру господства мини-компьютеров. Например, ранние операционные системы, в частности **DOS (Disk Operating System)** от компании **Microsoft**, не задумывались о важности защиты памяти, а потому злонамеренное (или просто плохо написанное) приложение могло затереть всю память. В первых поколениях **Mac OS** (v9 и более ранних) был принят кооперативный подход к планированию заданий, поэтому поток, который случайно заикливался, мог остановить всю систему, так что ее приходилось перезагружать. Скорбный список функций ОС, отсутствующих в этом поколении систем, длинен, слишком длинен, чтобы обсуждать его здесь подробно.

¹ Мы используем врезки «Отступление» и другие, чтобы привлечь внимание к различным фактам, которые не укладываются в основной поток изложения. Иногда они просто дают возможность пошутить, ведь нет же ничего плохого в том, чтобы немного рассеяться? Да, наверное, многие шутки неудачны.

К счастью, после нескольких лет страданий давно известные механизмы операционных систем для мини-компьютеров начали постепенно проникать и на рабочий стол. Например, в основу системы Mac OS X/macOS была положена UNIX со всей функциональностью, которую можно ожидать от такой зрелой системы. Windows также восприняла многие из величайших идей в истории вычислительной техники, и началось это с Windows NT, знаменовавшей гигантский скачок к технологии ОС от Microsoft. Даже современные сотовые телефоны управляются операционными системами (например, Linux), которые гораздо больше напоминают мини-компьютеры 1970-х годов, чем ПК 1980-х (и слава богу); приятно видеть, что хорошие идеи, заложенные в лучшую пору разработки ОС, проложили путь в современный мир. А еще приятнее, что эти идеи продолжают развиваться, обретают новые возможности и делают современные системы еще удобнее для пользователей и приложений.

ОТСТУПЛЕНИЕ: ВАЖНОСТЬ UNIX

Невозможно переоценить важность Unix в истории операционных систем. Испытав влияние более ранних систем (в частности, знаменитой системы **Multics**, созданной в MIT), Unix объединила в себе многие великие идеи, что позволило создать систему одновременно простую и мощную.

В основу оригинальной Unix от «Bell Labs» лег универсальный принцип, заключающийся в построении небольших узкоспециализированных программ, которые можно было соединять для решения более крупных задач. **Оболочка**, в которой вводятся команды, предоставляла такие примитивы, как **конвейеры**, открывавшие возможность для подобного программирования на метауровне. Например, чтобы найти все строки текстового файла, в которых встречается слово «foo», а затем подсчитать количество таких строк, нужно было ввести команду `grep foo file.txt | wc -l`, в составе которой используются программы `grep` и `wc` (word count – счетчик слов).

Среда Unix была удобна для программистов и разработчиков и предлагала также компилятор нового **языка программирования C**. Благодаря простоте написания программ и обмена ими Unix приобрела невероятную популярность. И, наверное, тому сильно способствовала готовность авторов бесплатно раздавать копии всем, кто попросит, – ранняя форма **ПО с открытым исходным кодом**.

Также очень важную роль сыграли доступность и удобочитаемость кода. Элегантное небольшое ядро, написанное на C, как бы приглашало всех желающих к экспериментам, к добавлению новой «крутой» функциональности. Например, инициативная группа в Беркли под руководством **Билла Джоя** создала потрясающий дистрибутив (**Berkeley Systems Distribution**, или **BSD**), в который вошла передовая система виртуальной памяти, файловая система и сетевая подсистема. Впоследствии Джой стал одним из основателей компании **Sun Microsystems**.

К сожалению, распространение Unix немного замедлилось, поскольку компании пытались закрепить за собой права собственности и извлекать прибыль – печальное (но типичное) следствие привлечения юристов. Многие компании начали выпускать собственные варианты: **SunOS** от Sun Microsystems, **AIX** от IBM, **HP-UX** (или «H-Pucks») от HP, **IRIX** от SGI. Из-за судебных тяжб между AT&T/Bell Labs и другими игроками над Unix нависли тучи, и многие сомневались, выживет ли она, особенно в связи с появлением Windows, воцарившейся на рынке ПК.

ОТСТУПЛЕНИЕ: И ТОГДА ПРИШЛА LINUX

К счастью для Unix, молодой финский хакер **Линус Торвальдс** решил написать собственную версию Unix. Он позаимствовал у оригинальной системы многие принципы и идеи, но не кодовую базу и тем самым избежал юридических сложностей. Он рекрутировал в добровольные помощники многих программистов со всего мира, воспользовался развитыми инструментами GNU, уже существовавшими к тому времени [G85], и вскоре родилась ОС **Linux** (а вместе с ней современное движение за ПО с открытым исходным кодом).

Когда настала эра интернета, многие компании (например, Google, Amazon, Facebook и другие) выбрали Linux, поскольку она распространялась свободно и ее можно было модифицировать под свои нужды; трудно представить, что все эти компании добились бы такого успеха, если бы подобной системы не существовало. Когда смартфоны стали доминирующей платформой, ориентированной на пользователя, Linux закрепилась и там тоже (посредством Android) – и по тем же причинам. А Стив Джобс забрал свою основанную на Unix операционную среду **NeXTStep** в Apple и тем самым сделал Unix популярной на настольных компьютерах (хотя многие пользователи технологий Apple, возможно, и не подозревают об этом). Таким образом, Unix продолжает здравствовать и даже более важна, чем когда-либо прежде. Компьютерные боги, если вы в них верите, должны быть благодарны за это великолепное подношение.

2.7. РЕЗЮМЕ

Итак, краткое введение в ОС позади. Благодаря современным операционным системам с компьютерами стало относительно легко работать, и практически все используемые сегодня операционные системы так или иначе испытывали влияние идей, которые мы будем обсуждать далее в этой книге.

К сожалению, из-за ограничений по времени мы не сможем рассмотреть все части ОС. Например, в операционной системе много **сетевых** кода, но чтобы подробнее узнать о нем, вам придется записаться на курс по сетям. Не менее важны **графические** устройства, походите на курс по графике, чтобы расширить свои знания в этом направлении. Наконец, в некоторых книгах по операционным системам большое внимание уделено **безопасности**; мы тоже затронем эту тему, поскольку ОС должна защищать работающие программы друг от друга и давать пользователям возможность защитить свои файлы, но не станем глубоко вдаваться в детали, которые излагаются в курсе по безопасности.

Однако же все равно остается много важных тем для изучения, включая основы виртуализации процессора и памяти, конкурентность и постоянное хранение на запоминающих устройствах с помощью файловых систем. Не пугайтесь! Да, нам предстоит длинная дорога, но это будет интересное путешествие, и в конце пути вы сможете по-новому взглянуть на то, как в действительности работают вычислительные системы. А теперь – за работу!

Литература

[BS+09] «Tolerating File-System Mistakes with EnvyFS» by L. Bairavasundaram, S. Sundararaman, A. Arpaci-Dusseau, R. Arpaci-Dusseau. USENIX '09, San Diego, CA, June 2009. *Интересная статья о том, как использовать сразу несколько файловых систем на случай, если в одной произойдет ошибка.*

[BH00] «The Evolution of Operating Systems» by P. Brinch Hansen. В книге «Classic Operating Systems: From Batch Processing to Distributed Systems». Springer-Verlag, New York, 2000. *В этом эссе вы найдете введение в замечательное собрание статей об исторически значимых системах.*

[B+72] «TENEX, A Paged Time Sharing System for the PDP-10» by D. Bobrow, J. Burchfiel, D. Murphy, R. Tomlinson. CACM, Volume 15, Number 3, March 1972. *TENEX вобрала в себя многие механизмы, встречающиеся в современных операционных системах; почитайте о ней – и узнаете, сколько инноваций было придумано еще в начале 1970-х годов.*

[B75] «The Mythical Man-Month» by F. Brooks. Addison-Wesley, 1975. *Классическая книга по программной инженерии, очень рекомендуем прочитать¹.*

[BOH10] «Computer Systems: A Programmer's Perspective» by R. Bryant and D. O'Hallaron. Addison-Wesley, 2010. *Еще одно великолепное введение в компьютерные системы. Немного пересекается с этой книгой, поэтому, если хотите, можете пропустить несколько последних глав или прочитайте их, чтобы познакомиться с другим взглядом на тот же самый материал. В конце концов, один из способов познакомиться с предметом – изучить как можно больше разных точек зрения, а затем сформулировать собственное мнение и мысли. Думать надо, думать!*

[G85] «The GNU Manifesto» by R. Stallman. 1985. www.gnu.org/gnu/manifesto.html. *Своим успехом Linux, без сомнения, в огромной степени обязана великолепному компилятору gcc и другому программному обеспечению, появившемуся в результате усилий по созданию фонда GNU, который возглавил Ричард Столлмен – провидец в области ПО с открытым исходным кодом. А в этом манифесте изложены его мысли на эту тему.*

[K+61] «One-Level Storage System» by T. Kilburn, D. B. G. Edwards, M. J. Lanigan, F. H. Sumner. IRE Transactions on Electronic Computers, April 1962. *В системе Atlas впервые было реализовано многое из того, что мы встречаем в современных системах. Однако эта статья – не самое лучшее чтение. Если у вас есть время для чтения только одной работы, то лучше обратиться к историческому обзору [L78].*

[L78] «The Manchester Mark I and Atlas: A Historical Perspective» by S. H. Lavington. Communications of the ACM, Volume 21:1, January 1978. *Изящно изложенная история ранних разработок компьютерных систем и первопрототипической*

¹ Фредерик Брукс младший. Мифический человеко-месяц. Питер, 2020.

роли Atlas. Конечно, можно было бы прочитать оригинальные статьи по Atlas, но эта работа содержит отличный обзор и открывает определенную историческую перспективу.

[O72] «The Multics System: An Examination of its Structure» by Elliott Organick. MIT Press, 1972. Прекрасный обзор системы Multics. Изобилие блестящих идей, но при этом система замахнулась на слишком многое и потому в реальности так никогда и не работала. Классический пример того, что Фред Брукс назвал «эффектом второй системы» [B75].

[PP03] «Introduction to Computing Systems: From Bits and Gates to C and Beyond» by Yale N. Patt, Sanjay J. Patel. McGraw-Hill, 2003. Одно из наших любимых введений в компьютерные системы. Начав с транзисторов, проводит по всему пути к языку C; особенно хороши начальные главы.

[RT74] «The Unix Time-Sharing System» by Dennis M. Ritchie, Ken Thompson. CACM, Volume 17: 7, July 1974. Блестящий обзор Unix, написанный ее создателями в то время, когда эта система захватывала мир.

[S68] «SDS 940 Time-Sharing System» by Scientific Data Systems. TECHNICAL MANUAL, SDS 90 11168, August 1968. Да, техническое руководство – лучшее, что нам удалось найти. Но как же захватывает чтение этой документации по старой системе, когда понимаешь, как много было сделано еще в конце 1960-х годов. Одним из гениев, стоявших у истоков Berkeley Time-Sharing System (которая в итоге превратилась в систему SDS), был Батлер Лэм্পсон, позже получивший премию Тьюринга за вклад в разработку вычислительных систем.

[SS+10] «Membrane: Operating System Support for Restartable File Systems» by S. Sundararaman, S. Subramanian, A. Rajimwale, A. Arpaci-Dusseau, R. Arpaci-Dusseau, M. Swift. FAST '10, San Jose, CA, February 2010. Что хорошо, когда пишешь заметки к собственным лекциям, – можно рекламировать свои исследования. Но эта статья действительно интересна – когда файловая система из-за ошибки «падает», Membrane автоматически перезапускает ее, так что ни приложения, ни прочие части системы не страдают.

Домашнее задание

Большинство глав этой книги (а в конечном итоге так будет со всеми) заканчиваются домашним заданием. Выполнять их важно, поскольку так читатель приобретает практический опыт работы с изложенным в главе материалом.

Есть два типа домашних заданий. Первый основан на **эмуляции**. Эмулятор компьютерной системы – это простая программа, которая моделирует какие-то интересные части настоящей системы и формирует отчет о метриках, показывающих, как ведет себя система. Например, эмулятор жесткого диска отправляет серию запросов, моделирует время, необходимое диску для их обслуживания в предположении определенных характеристик производительности, а затем формирует отчет о средней задержке обработки запроса.

Эмуляция хороша тем, что позволяет исследовать поведение систем, обходя трудности, связанные с эксплуатацией реальной системы. На самом деле

так можно даже создать систему, не существующую в реальности (скажем, невообразимо быстрый жесткий диск), и изучать потенциальное влияние будущих технологий.

Конечно, у эмуляции есть и недостатки. По самой своей природе, эмуляция – это лишь приближение к реальной системе. Если упустить из виду какой-то важный аспект реального поведения, то результаты эмулятора окажутся бесполезны. Поэтому к результатам эмуляции всегда следует относиться с подозрением. В конце концов, по-настоящему важно, как система ведет себя в реальном мире.

Домашние задания второго типа предполагают взаимодействие с **реальным кодом**. Некоторые из них нацелены на измерения, другие требуют не слишком масштабной разработки и экспериментирования. В любом случае это лишь небольшие вылазки в обширный мир разработки системного кода на C в UNIX-системах, куда вам рано или поздно предстоит попасть. Чтобы дальше продвинуться в этом направлении, необходимы более крупные проекты, выходящие за рамки домашних заданий, поэтому мы настоятельно рекомендуем не ограничиваться домашними заданиями и выполнять проекты, чтобы закрепить приобретенные навыки. Некоторые проекты такого рода описаны на странице <https://github.com/remzi-arpacidusseau/ostep-projects>.

Для выполнения домашних заданий вам, вероятно, понадобится машина под управлением Linux, macOS или аналогичной системы. Должен быть также установлен компилятор C (например, **gcc**) и Python. Кроме того, вы должны знать, как писать код в каком-нибудь редакторе кода.

Часть I

.....

ВИРТУАЛИЗАЦИЯ

Глава 3

Диалог о виртуализации

Профессор. Вот мы и подошли к первому из трех элементов операционной системы: виртуализации.

Студент. Но что такое виртуализация, о великодушный профессор?

Профессор. Представь, что у тебя есть персик.

Студент (недоверчиво). Персик?

Профессор. Именно персик. Назовем его физическим персиком. Но у нас тут много желающих этот персик съесть. А мы хотели бы дать каждому по персику, чтобы все были довольны. Назовем персики, которые мы раздаем всем желающим, виртуальными; каким-то образом мы создали много виртуальных персиков из одного физического. И вот что важно: это иллюзия – каждому кажется, что он получил физический персик, хотя в действительности это не так.

Студент. То есть я разделяю один персик со всеми, даже не зная об этом?

Профессор. Верно! В самую точку.

Студент. Но персик-то всего один.

Профессор. Да. И...?

Студент. Ну, это... если бы я разделил персик с кем-то еще, то, надо полагать, заметил бы это.

Профессор. О да! Тонко подмечено. Но когда едоков много, они большую часть времени дремлют или еще чем-то заняты, и в этот момент можно умыкнуть персик и на время дать его кому-то другому. Так мы создаем иллюзию, что виртуальных персиков много, по одному на каждого!

Студент. Звучит как плохой рекламный слоган. Но ведь мы же говорим о компьютерах. Или нет, профессор?

Профессор. Ах, мой юный друг, ты жаждешь более конкретного примера. Отлично! Возьмем самый главный ресурс, процессор. Предположим, что в системе имеется один физический процессор (хотя теперь их чаще два, четыре или еще больше). В чем суть виртуализации? В том, чтобы взять этот единственный

процессор и сделать так, чтобы приложениям, работающим в системе, казалось, что имеется много виртуальных процессоров. Таким образом, каждое приложение думает, что располагает своим собственным процессором, хотя в действительности он один. ОС создала прекрасную иллюзию: она виртуализовала CPU.

Студент. *Ух ты! Прямо волшебство какое-то. Расскажите еще! Как это работает.*

Профессор. *Всему свое время, юный ученик, всему свое время. Но ты, похоже, готов начать.*

Студент. *Готов! Правда, немного опасаясь, что вы снова затеете разговор о персиках.*

Профессор. *Не стоит, я даже и не люблю персики. Итак, начнем...*

Глава 4

Абстракция: процесс

В этой главе мы обсудим одну из самых фундаментальных абстракций, предоставляемых ОС: **процесс**. Неформальное определение процесса очень простое – **исполняемая программа** [V+65, BH70]. Сама программа – вещь безжизненная: просто набор команд (и, возможно, данных); она хранится на диске и ждет, когда ее призовут к действию. Именно операционная система заставляет эти байты выполняться, превращая их в нечто полезное.

Но часто бывает, что нужно одновременно запустить более одной программы; например, на вашем настольном компьютере или ноутбуке может работать веб-браузер, почтовая программа, игра, музыкальный проигрыватель и т. д. Вообще, в типичной системе могут одновременно присутствовать десятки и даже сотни процессов. Благодаря этой иллюзии с системой легко работать, потому что не нужно думать о том, доступен процессор или нет; мы просто запускаем программы. И в этом и состоит наша задача.

Существо проблемы:

КАК СОЗДАТЬ ИЛЛЮЗИЮ СУЩЕСТВОВАНИЯ МНОГИХ ПРОЦЕССОРОВ?

Учитывая, что физических CPU всего несколько штук, как ОС может создать иллюзию почти бесконечного запаса процессоров?

ОС создает эту иллюзию путем виртуализации CPU. Она запускает один процесс, затем останавливает его, запускает другой и т. д., в результате чего складывается впечатление, будто существует много виртуальных CPU, хотя на самом деле имеется только один (или небольшое число) физический. Эта базовая техника, называемая **разделением времени**, позволяет запускать столько конкурентных процессов, сколько нужно пользователям; расплачиваться за это приходится производительностью, потому что каждый процесс будет работать медленнее, если процессор разделяется.

Чтобы реализовать виртуализацию и сделать это эффективно, ОС должна располагать как низкоуровневыми средствами, так и высокоуровневым координатором. Низкоуровневые средства называются **механизмами**, это методы или протоколы, необходимые для реализации определенной функциональности. Например, ниже мы узнаем, как реализуется **контекстное переключение**, наделяющее ОС способностью приостанавливать одну про-

грамму и запускать другую на том же процессоре; этот механизм **разделения времени** используется во всех современных ОС.

Поверх этих механизмов располагаются **политики** ОС. Это алгоритмы принятия решений внутри ОС. Например, при заданном множестве возможных программ, готовых к использованию CPU, какой программе ОС должна отдать процессор? За это решение отвечает **политика планирования**, которая, вероятно, учитывает исторические данные (например, какая программа дольше работала в течение последней минуты?), сведения о рабочей нагрузке (программы какого типа сейчас работают?) и показатели производительности (например, что пытается оптимизировать система: производительность интерактивной работы или пропускную способность?).

СОВЕТ: ПРИМЕНЯЙТЕ РАЗДЕЛЕНИЕ ВРЕМЕНИ (И ПРОСТРАНСТВА)

Разделение времени — фундаментальная техника, применяемая ОС для совместного использования ресурса. Смысл ее в том, что в течение короткого времени ресурс (например, процессор или сетевой канал) используется одним потребителем, затем другим и т. д., поэтому один и тот же ресурс разделяется между многими потребителями. Альтернативой разделению времени является разделение пространства, когда ресурс распределяется между всеми, кто хочет его использовать. Например, дисковое пространство естественным образом является пространственно разделяемым ресурсом; блок, выделенный одному файлу, при обычных обстоятельствах не может быть выделен другому, пока пользователь не удалит первый файл.

4.1. АБСТРАКЦИЯ: ПРОЦЕСС

Предоставляемая ОС абстракция работающей программы называется **процессом**. В любой момент времени мы можем получить представление о процессе, посмотрев, к каким частям системы он обращается и на какие части оказывает влияние в ходе своего выполнения.

Чтобы понять, из чего состоит процесс, мы должны понять, что такое **машинное состояние**: что именно может читать и обновлять работающая программа. Какие части машины важны для выполнения данной программы?

Очевидный компонент машинного состояния, составляющий процесс, — его *память*. Команды находятся в памяти, там же размещаются данные, которые читает и изменяет работающая программа. Таким образом, память, которую может адресовать процесс (его **адресное пространство**), является частью этого процесса.

Кроме того, частью машинного состояния процесса являются *регистры*; многие команды явно читают или изменяют регистры, так что они, безусловно, важны для выполнения процесса.

Отметим также, что в состав машинного состояния входят некоторые специальные регистры. Например, **счетчик команд** (англ. *program counter* — PC или *instruction pointer* — IP) говорит, какая команда исполняется в данный

момент, а **указатель стека** и связанный с ним **указатель кадра** служат для управления стеком, в котором хранятся параметры функции, локальные переменные и адрес возврата.

СОВЕТ: РАЗЛИЧАЙТЕ ПОЛИТИКУ И МЕХАНИЗМ

При проектировании многих операционных систем принято отделять высокоуровневые политики от соответствующих низкоуровневых механизмов [L+75]. Можно считать, что механизм дает ответ на вопрос *как*, например *как* операционная система выполняет контекстное переключение? Политика же отвечает на вопрос *какой*, например на *какой* процесс должна переключиться операционная система в данный момент? Разделение того и другого позволяет легко менять политики, не меняя механизма, это форма **модульности**, общего принципа проектирования программного обеспечения.

Наконец, программы часто обращаются к запоминающим устройствам. Такая *информация о вводе-выводе* может также включать список открытых процессом файлов.

4.2. API ПРОЦЕССОВ

Обсуждение настоящего API процессов мы отложим до следующей главы, но уже сейчас попробуем составить представление о том, что должно быть включено в интерфейс операционной системы. Эти API в том или ином виде имеются в любой современной операционной системе.

- **Создание.** ОС должна включать какой-то метод создания новых процессов. Когда мы вводим команду в оболочке или дважды щелкаем мышью по значку приложения, ОС создает новый процесс для выполнения указанной программы.
- **Снятие.** Раз имеется интерфейс для создания процесса, то должен быть и интерфейс для его принудительного завершения. Конечно, многие процессы завершаются сами, закончив работу, но если процесс этого не делает, то у пользователя должна быть возможность снять его, поэтому интерфейс для остановки вышедшего из-под контроля процесса весьма полезен.
- **Ожидание.** Иногда полезно подождать, пока процесс завершит работу, поэтому часто предоставляется какой-то интерфейс для ожидания.
- **Разные средства управления.** Помимо снятия и ожидания процесса, бывают необходимы и другие средства управления. Например, в большинстве операционных систем имеется метод приостановки процесса и последующего возобновления.
- **Состояние.** Обычно предлагаются интерфейсы для получения информации о состоянии процесса, например сколько времени он уже работает и в каком состоянии сейчас находится.

4.3. СОЗДАНИЕ ПРОЦЕССА: ПОДРОБНОСТИ

Пора сорвать покров тайны с одного вопроса: как программы преобразуются в процессы? Точнее, как ОС запускает программу на выполнение? Как в действительности устроено создание процесса?

Чтобы запустить программу, ОС первым делом должна **загрузить** ее код и статические данные (например, инициализированные переменные) в память – в адресное пространство процесса. Поначалу программы располагаются на **диске** (в некоторых современных системах – на **SSD-диске**, основанном на технологии **флеш-памяти**) в том или ином **исполняемом формате**; таким образом, для загрузки программы и статических данных ОС должна прочитать байты с диска и поместить их куда-то в память (как показано на рис. 4.1).

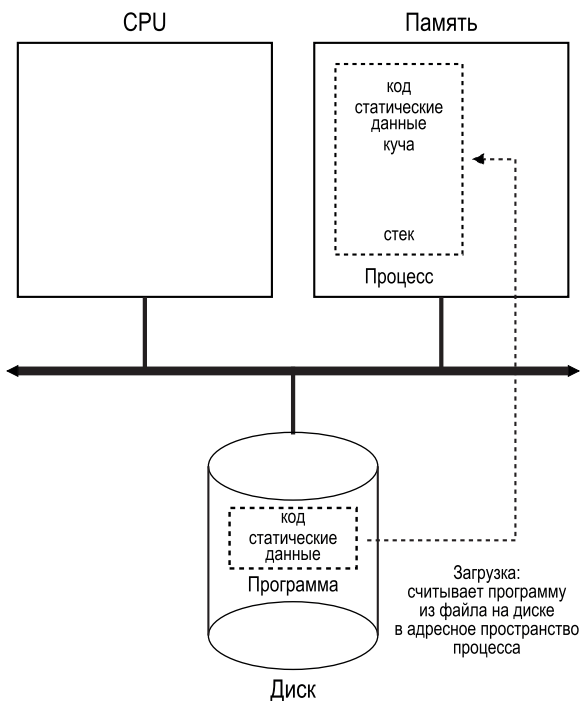


Рис. 4.1 ❖ Загрузка: от программы к процессу

В ранних (и простых) операционных системах процедура загрузки выполнялась **безотлагательно**, т. е. загружалась вся программа целиком; в современных ОС это делается **лениво**, т. е. код и данные загружаются частями, по мере необходимости. Чтобы разобраться в ленивой загрузке кода и данных, нужно больше знать о механизмах **страничной организации** (paging) и **подкачки** (swapping); эти вопросы мы рассмотрим ниже, когда будем обсуждать виртуализацию памяти. А пока просто запомните, что прежде чем

что-то запустить, ОС обязательно должна проделать определенную работу по переносу байтов с диска в память.

После того как код и статические данные загружены в память, ОС должна сделать еще кое-что. Часть памяти следует выделить под **стек** программы. Как вы, вероятно, уже знаете, в программах на С стек используется для размещения локальных переменных, параметров функций и адресов возврата. ОС выделяет под это память и передает ее процессу. Кроме того, ОС, скорее всего, поместит в стек аргументы, а именно параметры функции `main()`: целое число `argc` и массив `argv`.

Также ОС может выделить память для программной **кучи**. В программах на С из кучи динамически выделяется память; программа запрашивает память с помощью библиотечной функции `malloc()` и явно освобождает ее, вызывая функцию `free()`. Куча нужна для размещения таких структур данных, как связные списки, хеш-таблицы, деревья и прочее. Сначала куча невелика, но если по ходу работы программа запрашивает дополнительную память, обращаясь к `malloc()`, то ОС может вступить в дело и выделить процессу дополнительную память, чтобы он мог удовлетворить запросы.

Кроме того, ОС выполняет другие операции по инициализации, прежде всего относящиеся к вводу-выводу. Например, в Unix-системах каждый процесс по умолчанию получает три открытых **файловых дескриптора**: для стандартного ввода, стандартного вывода и стандартного вывода ошибок; благодаря этим дескрипторам программа может читать данные, вводимые с клавиатуры, и осуществлять вывод на экран. Мы еще вернемся к вводу-выводу, файловым дескрипторам и прочему в третьей части книги, посвященной **хранению**.

Загрузив код и статические данные в память, создав и инициализировав стек, организовав все необходимое для ввода-вывода, ОС (наконец-то) подготовила сцену для выполнения программы. Остался последний шаг: начать выполнение программы с точки входа, а именно с функции `main()`. Совершив переход в начало функции `main()` (с помощью специального механизма, который мы обсудим в следующей главе), ОС передает процессор в распоряжение вновь созданного процесса, и программа начинает исполняться.

4.4. Состояния ПРОЦЕССА

Итак, мы теперь представляем, что такое процесс (хотя будем постепенно уточнять это представление) и как он создается (хотя бы вчерне). Теперь можно поговорить о различных **состояниях**, в которых может находиться процесс. Понятие состояния процесса возникло еще в ранних компьютерных системах [DV66, V+65]. Упрощая, можно сказать, что процесс может находиться в одном из трех состояний.

- **Исполняется:** в этом состоянии процесс владеет процессором, т. е. выполняются его команды.
- **Готов:** в этом состоянии процесс готов к исполнению, но по какой-то причине ОС его пока не выбрала.

- **Заблокирован:** в это состояние процесс переходит, выполнив какую-то операцию, в результате которой перестал быть готовым к выполнению, и остается в нем, пока не произойдет некое событие. Типичный пример: когда процесс инициирует запрос ввода-вывода к диску, он блокируется, поэтому доступ к процессору получает какой-то другой процесс.

Если представить эти состояния в графическом виде, то получится диаграмма на рис. 4.2. Как видим, процесс может переходить из состояния «исполняется» в состояние «готов» и обратно по усмотрению ОС. Переход из состояния «готов» в состояние «исполняется» означает, что процесс был **запланирован**, а обратный переход – что процесс был **снят с процессора**. После того как процесс был заблокирован (например, в результате инициирования операции ввода-вывода), ОС держит его в этом состоянии, пока не произойдет какое-то событие (например, завершение ввода-вывода), а в этот момент процесс снова переходит в состояние «готов» (и, возможно, сразу же начинает исполняться, если так решит ОС).

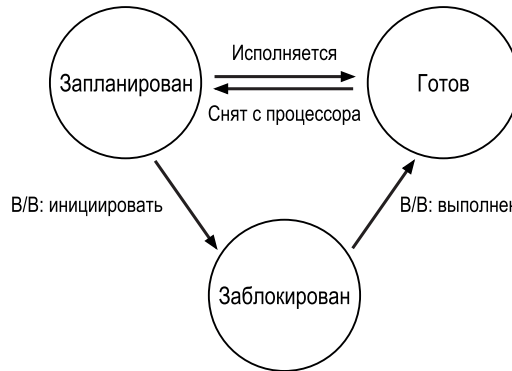


Рис. 4.2 ❖ Процесс: диаграмма перехода состояний

Рассмотрим на примере, как могут меняться состояния двух процессов. Сначала представим себе, что во время работы оба процесса используют только CPU (т. е. не совершают никакого ввода-вывода). В этом случае последовательность состояний может выглядеть, как показано на рис. 4.3.

Время	Процесс ₀	Процесс ₁	Примечания
1	Исполняется	Готов	
2	Исполняется	Готов	
3	Исполняется	Готов	
4	Исполняется	Готов	Процесс ₀ завершился
5	–	Исполняется	
6	–	Исполняется	
7	–	Исполняется	
8	–	Исполняется	Процесс ₁ завершился

Рис. 4.3 ❖ Последовательность состояний процесса: только CPU

В следующем примере первый процесс некоторое время работает, а затем инициирует ввод-вывод. В этот момент процесс блокируется, давая другому процессу шанс поработать. Последовательность состояний в данном случае показана на рис. 4.4.

Время	Процесс ₀	Процесс ₁	Примечания
1	Исполняется	Готов	
2	Исполняется	Готов	
3	Исполняется	Готов	Процесс ₀ инициирует В/В
4	Блокирован	Исполняется	Процесс ₀ заблокирован
5	Блокирован	Исполняется	Поэтому выполняется Процесс ₁
6	Блокирован	Исполняется	
7	Готов	Исполняется	В/В завершился
8	Готов	Исполняется	Процесс ₁ завершился
9	Исполняется	–	
10	Исполняется	–	Процесс ₀ завершился

Рис. 4.4 ❖ Последовательность состояний процесса: CPU и ввод-вывод

Именно Процесс₀ инициирует ввод-вывод и оказывается заблокированным в ожидании завершения операции, например чтения с диска или получения пакета из сети. ОС обнаруживает, что Процесс₀ не использует CPU, и начинает выполнять Процесс₁. Пока Процесс₁ исполняется, операция ввода-вывода завершается, и Процесс₀ переходит в состояние «готов». Наконец, Процесс₁ завершается, после чего Процесс₀ возобновляет выполнение и тоже завершается.

Заметим, что даже в этом простом примере ОС должна принимать много решений. Во-первых, система должна решить, что нужно запустить Процесс₁, когда Процесс₀ инициировал операцию ввода-вывода; это позволяет более эффективно использовать ресурсы, не давая простаивать процессору. Во-вторых, система решила не переключаться обратно на Процесс₀, когда ввод-вывод завершился; не вполне ясно, хорошее это решение или нет. Как вы сами думаете? Решения такого рода принимает **планировщик** ОС, мы вернемся к этой теме через несколько глав.

4.5. СТРУКТУРЫ ДАННЫХ

ОС – это программа, и, как в любой программе, в ней имеются важные структуры данных, в которых хранится необходимая информация. Например, для отслеживания состояния каждого процесса ОС, вероятно, должна хранить **список процессов**, в котором перечислены все готовые процессы, а также дополнительную информацию о том, какой процесс исполняется в данный момент. ОС также должна каким-то образом отслеживать заблокированные процессы, а когда возникает событие завершения ввода-вывода, ОС должна

позаботиться о том, чтобы разбудить нужный процесс и перевести его в состояние готовности.

На рис. 4.5 на примере ядра xv6 показано, какую информацию должна хранить ОС о каждом процессе [CK+08]. Аналогичные структуры существуют в «настоящих» операционных системах: Linux, Mac OS X или Windows; поинтересуйтесь этим вопросом и посмотрите, насколько они сложнее.

```
// регистры, которые сохраняет и восстанавливает xv6,
// чтобы остановить и возобновить процесс
struct context {
    int eip;
    int esp;
    int ebx;
    int ecx;
    int edx;
    int esi;
    int edi;
    int ebp;
};

// различные состояния, в которых может находиться процесс
enum proc_state { UNUSED, EMBRYO, SLEEPING,
                  RUNNABLE, RUNNING, ZOMBIE };

// информация, которую xv6 хранит о каждом процессе,
// в т. ч. регистровый контекст и состояние
struct proc {
    char *mem;                // начало памяти процесса
    uint sz;                  // размер памяти процесса
    char *kstack;             // конец стека ядра для этого процесса
    enum proc_state state;     // состояние процесса
    int pid;                  // идентификатор процесса
    struct proc *parent;       // родительский процесс
    void *chan;               // если не 0, спит в ожидании канала
    int killed;               // если не 0, был снят
    struct file *ofile[NOFILE]; // открытые файлы
    struct inode *cwd;         // текущий каталог
    struct context context;    // переключиться сюда для исполнения процесса
    struct trapframe *tf;      // кадр текущего системного прерывания
};
```

Рис. 4.5 ❖ Структура proc в xv6

На рисунке мы видим два важных элемента информации, которую ОС хранит о процессе. В **регистровом контексте** для каждого остановленного процесса запоминает содержимое его регистров. Когда процесс снимается с процессора, его регистры сохраняются в памяти, а путем восстановления этих регистров (т. е. копирования их значений из памяти обратно в соответствующие регистры) ОС возобновляет выполнение процесса. В последующих главах мы рассмотрим это **контекстное переключение** более подробно.

Из этого рисунка также видно, что существуют и другие состояния, помимо «исполняется», «готов» и «заблокирован». Иногда система поддерживает **начальное** состояние, в котором процесс пребывает, пока его создание не закончено. Кроме того, процесс может находиться в **финальном** состоянии, когда он уже завершился, но еще не прибран (в Unix-системах оно называется состоянием **зомби**¹). Это финальное состояние может быть полезно, поскольку позволяет другим процессам узнать код возврата и оценить, было ли завершение процесса успешным (в Unix-системах принято возвращать 0, если завершение было успешным, и не 0 в противном случае). После завершения родительский процесс выполняет еще один последний вызов (например, `wait()`), чтобы дождаться кончины потомка и сообщить ОС, что она может очистить все структуры данных, ссылающиеся на почивший процесс.

ОТСТУПЛЕНИЕ: СТРУКТУРА ДАННЫХ – СПИСОК ПРОЦЕССОВ

Операционные системы изобилуют различными важными **структурами данных**, которые мы будем обсуждать в таких вот врезках. **Список процессов** (или **список задач**) – первая из таких структур. Она совсем простая, но, разумеется, любая ОС, способная выполнять несколько программ одновременно, должна включать что-то в этом роде, чтобы отслеживать все работающие программы. Иногда отдельную структуру, в которой хранится информация о процессе, называют **блоком управления процессом** (англ. *Process Control Block – PCB*), или **дескриптором процесса**.

ОТСТУПЛЕНИЕ: ТЕРМИНЫ, СВЯЗАННЫЕ С ПРОЦЕССОМ

- **Процесс** – абстракция работающей программы в ОС. В любой момент времени процесс можно описать его состоянием: содержимым памяти в его **адресном пространстве**, содержимым регистров CPU (включая **счетчик команд** и **указатель стека**) и информацией о вводе-выводе (дескрипторы открытых для чтения или записи файлов).
- **API процессов** включает системные вызовы, относящиеся к процессам. В него входит создание процесса, его снятие и другие полезные операции.
- Существует несколько **состояний процесса**, в т. ч. «исполняется», «готов» и «заблокирован». Различные события (например, планирование, снятие с процессора, ожидание завершения ввода-вывода) вызывают переход процесса из одного состояния в другое.
- **Список процессов** содержит информацию обо всех процессах в системе. Каждый элемент этого списка называется **блоком управления процессом (PCB)**; это просто структура данных, содержащая информацию о конкретном процессе.

4.6. РЕЗЮМЕ

Мы познакомились с самой базовой абстракцией ОС: процессом. Попросту говоря, это исполняемая программа. Имея в виду это концептуальное пред-

¹ Да, именно так. Как и настоящих зомби, такие зомби-процессы довольно легко убить. Но обычно рекомендуются другие методы.

ставление, мы перейдем к техническим деталям: механизмам, необходимым для реализации процессов, и высокоуровневым политикам, применяемым для их планирования. Сочетание механизмов и политик позволит нам понять, как операционная система виртуализирует процессор.

Литература

[BH70] «The Nucleus of a Multiprogramming System» by Per Brinch Hansen. Communications of the ACM, Volume 13:4, April 1970. *В этой статье описано одно из первых **микроядер** в истории операционных систем – Nucleus. Идея небольшой, обладающей минимальной функциональностью системы много раз возникала в истории ОС, но все началось с этой работы Бринча Хансена.*

[CK+08] «The xv6 Operating System» by Russ Cox, Frans Kaashoek, Robert Morris, Nickolai Zeldovich. Из <https://github.com/mit-pdos/xv6-public>. *Самая крутая в мире настоящая небольшая ОС. Скачайте ее код и поэкспериментируйте с ним, чтобы лучше понять, как в действительности работают операционные системы. Мы пользуемся довольно старой версией (2012-01-30-1-g1c41342), поэтому некоторые примеры могут расходиться с кодом в репозитории.*

[DV66] «Programming Semantics for Multiprogrammed Computations» by Jack B. Dennis, Earl C. Van Horn. Communications of the ACM, Volume 9, Number 3, March 1966. *В этой работе даны определения многих ранних терминов и концепций мультипрограммных систем.*

[L+75] «Policy/mechanism separation in Hydra» by R. Levin, E. Cohen, W. Corwin, F. Pollack, W. Wulf. SOSP '75, Austin, Texas, November 1975. *Одна из первых статей о структурировании операционных систем на примере исследовательской ОС Hydra. Хотя Hydra так и не получила широкого распространения, некоторые заложенные в ней идеи оказали влияние на проектировщиков ОС.*

[V+65] «Structure of the Multics Supervisor» by V. A. Vyssotsky, F. J. Corbato, R. M. Graham. Fall Joint Computer Conference, 1965. *Ранняя работа по системе Multics, в которой описаны многие базовые идеи и термины, встречающиеся в современных системах. Представления о вычислениях как услуге частично реализованы в нынешних облачных системах.*

Домашнее задание (эмуляция)

Программа `process-run.py` позволяет увидеть, как меняется во время работы состояние процесса, если программа то использует CPU (например, выполняет команду сложения), то производит ввод-вывод (отправляет запрос диску и ожидает его завершения). Детали см. в файле README.

Вопросы

1. Запустите программу `process-run.py` со следующими флагами: `-l 5:100,5:100`. Каким должен быть уровень использования процессора (например, в процентах от общего времени занятости CPU)? Откуда вы это знаете? Воспользуйтесь флагами `-с` и `-р` для проверки своей гипотезы.
2. Теперь запустите ее следующим образом: `./process-run.py -l 4:100,1:0`. Эти флаги означают, что один процесс содержит 4 команды (все работают с CPU), а другой просто иницирует операцию ввода-вывода и ждет ее завершения. Сколько времени понадобится для завершения обоих процессов? Воспользуйтесь флагами `-с` и `-р` для проверки своей гипотезы.
3. Измените порядок запуска процессов на противоположный: `./process-run.py -l 1:0,4:100`. Что теперь происходит? Влияет ли на что-нибудь порядок запуска? Почему? (Как всегда, для проверки гипотезы воспользуйтесь флагами `-с` и `-р`.)
4. Теперь изучим другие флаги. Интересен флаг `-S`, который определяет реакцию системы на иницирование процессом ввода-вывода. Если этот флаг равен `SWITCH_ON_END`, то система НЕ будет переключаться на другой процесс, пока первый занимается вводом-выводом, а вместо этого будет ждать завершения данного процесса. Что произойдет, если запустить два процесса (`-l 1:0,4:100 -с -S SWITCH_ON_END`), один из которых занят вводом-выводом, а другой выполняет команды процессора?
5. Теперь запустите те же процессы, но задайте поведение, так чтобы система переключалась на другой процесс, когда один ожидает завершения ввода-вывода (`-l 1:0,4:100 -с -S SWITCH_ON_IO`). Что происходит? Воспользуйтесь флагами `-с` и `-р` для подтверждения своей правоты.
6. Еще одно важное поведение – что делать, когда ввод-вывод завершен. Если задан флаг `-I IO_RUN_LATER`, то по завершении ввода-вывода иницировавший его процесс необязательно возобновляется немедленно; возможно, продолжит работать процесс, исполняемый в данный момент. Что произойдет, если запустить два процесса с такой комбинацией флагов: `./process-run.py -l 3:0,5:100,5:100,5:100 -S SWITCH_ON_IO -I IO_RUN_LATER -с -р`? Эффективно ли при этом используются системные ресурсы?
7. Теперь запустите те же процессы, но с флагом `-I IO_RUN_IMMEDIATE`, который означает, что процесс, иницировавший ввод-вывод, должен возобновиться немедленно. Как изменилось поведение? Почему возобновление процесса, только что завершившего ввод-вывод, может оказаться здоровой идеей?
8. Теперь запустите несколько случайно порождаемых процессов: `-s 1 -l 3:50,3:50`, или `-s 2 -l 3:50,3:50`, или `-s 3 -l 3:50,3:50`. Сможете ли вы предсказать, как будет выглядеть последовательность состояний? Сравните поведение, наблюдаемое при задании флагов `-I IO_RUN_IMMEDIATE` и `-I IO_RUN_LATER`. А также поведение при задании флагов `-S SWITCH_ON_IO` и `-S SWITCH_ON_END`.

Глава 5

Интерлюдия: API процессов

ОТСТУПЛЕНИЕ: ИНТЕРЛЮДИИ

В интерлюдиях описываются более практические аспекты систем с упором на API операционных систем и порядок их использования. Если практические вещи вас не интересуют, можете пропускать интерлюдии. Но вообще-то должны интересовать, поскольку они полезны в реальной жизни; так, компании обычно нанимают сотрудников не за теоретические познания.

В этой интерлюдии мы обсудим создание процессов в Unix-системах, где применяется один из самых загадочных способов создания нового процесса – с помощью двух системных вызовов: `fork()` и `exec()`. Третий вызов, `wait()`, позволяет процессу дожидаться завершения созданного им процесса. Мы опишем эти интерфейсы в деталях, проиллюстрировав процедуру на нескольких простых примерах. Итак, вот наша проблема:

СУЩЕСТВО ПРОБЛЕМЫ: КАК СОЗДАВАТЬ ПРОЦЕССОРЫ И УПРАВЛЯТЬ ИМИ?

Какие интерфейсы должна предоставлять ОС для создания и управления процессами? Как следует проектировать эти интерфейсы, чтобы обеспечить богатую функциональность, простоту использования и высокую производительность?

5.1. Системный вызов `fork()`

Системный вызов `fork()` служит для создания нового процесса [С63]. Однако предупреждаем: это, безусловно, самая странная из всех функций¹. Конкрет-

¹ Ладно, признаем, что не можем утверждать это с полной уверенностью. Кто знает, какие функции вызываете вы, когда никто не смотрит? Но `fork()` и вправду далеко от стереотипов, сколько бы странными ни были ваши привычки.

но, код программы выглядит, как показано на рис. 5.1; изучите его, а еще лучше наберите и выполните!

```

1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <unistd.h>
4
5 int main(int argc, char *argv[]) {
6     printf("hello world (pid:%d)\n", (int) getpid());
7     int rc = fork();
8     if (rc < 0) { // ошибка fork, выйти
9         fprintf(stderr, "ошибка fork\n");
10        exit(1);
11    } else if (rc == 0) { // потомок (новый процесс)
12        printf("hello, I am child (pid:%d)\n", (int) getpid());
13    } else { // родитель следует по этому пути (в main)
14        printf("hello, I am parent of %d (pid:%d)\n",
15              rc, (int) getpid());
16    }
17    return 0;
18 }
19

```

Рис. 5.1 ❖ Вызов fork() (p1.c)

Эта программа (файл называется p1.c) печатает следующее:

```

prompt> ./p1
hello world (pid:29146)
hello, I am parent of 29147 (pid:29146)
hello, I am child (pid:29147)
prompt>

```

Разберемся, что же здесь произошло. Сразу после запуска процесс печатает сообщение «hello world», включив в него свой **идентификатор процесса**, или **PID**, который в данном случае равен 29146; в Unix PID служит для именования процесса на случай, если с ним понадобится что-то сделать, например снять. Пока все понятно.

Но дальше начинается самое интересное. Процесс делает системный вызов `fork()`, т. е. просит ОС создать новый процесс. И вот что странно: созданный процесс является (почти) *точной копией вызвавшего*. Это означает, что с точки зрения ОС выполняются две копии программы p1 и обе вот-вот вернутся из системного вызова `fork()`. Вновь созданный процесс (который называется **потомком**, или **дочерним** процессом в отличие от создавшего его **родителя**) начинает исполнение не в `main()`, как вы могли бы подумать (обратите внимание, что сообщение «hello, world» печатается только один раз), а вступает в жизнь так, будто сам только что вызвал `fork()`.

Вы, наверное, заметили, что потомок не является *точной копией*. У него имеется своя копия адресного пространства (собственная частная память), свои регистры, свой счетчик команд и т. д., и главное – он получает от `fork()`

другое значение. Точнее, родитель получает PID вновь созданного потомка, а потомок – нулевой код возврата. Это различие полезно, потому что позволяет писать код, по-разному обрабатывающий эти два случая (как в примере выше).

Вы, вероятно, также заметили, что вывод программы p1.c не **детерминирован**. После того как создан дочерний процесс, в системе имеется два интересующих нас процесса: родитель и потомок. В предположении, что процессор один (для простоты), в этой точке должен выполняться либо один процесс, либо другой. В нашем примере таковым оказался родитель, поэтому он напечатал свое сообщение первым. Но может случиться и обратное, и тогда мы увидим такую картину:

```
prompt> ./p1
hello world (pid:29146)
hello, I am child (pid:29147)
hello, I am parent of 29147 (pid:29146)
prompt>
```

Какой именно процесс будет работать в каждый момент времени, определяет **планировщик**; эту тему мы подробно рассмотрим чуть ниже. Поскольку планировщик устроен сложно, мы обычно не можем с уверенностью сказать, какое решение он выберет и, стало быть, какой процесс будет работать первым. Эта **недетерминированность** ведет к интересным проблемам, в частности в **многопоточных программах**, поэтому подробнее мы будем рассматривать эту тему во второй части книги, посвященной конкурентности.

5.2. Системный вызов wait()

Пока что мы почти ничего не сделали, только создали дочерний процесс, который печатает сообщение и выходит. Но иногда нужно, чтобы родитель подождал, пока потомок закончит свои дела. Для этого предназначен системный вызов wait() (или его более полный вариант waitpid()); детали см. на рис. 5.2.

В этом примере (p2.c) родитель вызывает wait(), чтобы задержать выполнение до момента, когда потомок закончит работу. По завершении потомка wait() возвращает управление родителю.

Если включить в программу вызов wait(), то результат станет детерминированным. Понимаете, почему? Остановитесь и подумайте.

(ждем, пока вы думаете.... дождались)

Итак, вы поразмышляли, теперь приведем распечатку:

```
pt> ./p2
hello world (pid:29266)
hello, I am child (pid:29267)
hello, I am parent of 29267 (rc_wait:29267) (pid:29266)
prompt>
```

```

1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <unistd.h>
4 #include <sys/wait.h>
5
6 int main(int argc, char *argv[]) {
7     printf("hello world (pid:%d)\n", (int) getpid());
8     int rc = fork();
9     if (rc < 0) {           // ошибка fork; выход
10         fprintf(stderr, "ошибка fork\n");
11         exit(1);
12     } else if (rc == 0) { // потомок (новый процесс)
13         printf("hello, I am child (pid:%d)\n", (int) getpid());
14     } else {               // родитель следует по этому пути (в main)
15         int rc_wait = wait(NULL);
16         printf("hello, I am parent of %d (rc_wait:%d) (pid:%d)\n",
17             rc, rc_wait, (int) getpid());
18     }
19     return 0;
20 }
21

```

Рис. 5.2 ❖ Вызов `fork()` и `wait()` (p2.c)

При таком коде потомок всегда будет печатать свое сообщение первым. Откуда мы это знаем? Если он просто был запланирован первым, как прежде, то и напечатает раньше родителя. А если первым был запланирован родитель, то он сразу же вызовет `wait()`, а эта функция не вернет управление, пока не запустится и не завершится потомок¹. Таким образом, даже если родитель начнет выполняться первым, он вежливо подождет завершения работы потомка, а когда `wait()` вернется, напечатает свое сообщение.

5.3. И НАКОНЕЦ, СИСТЕМНЫЙ ВЫЗОВ `exec()`

Последняя важная часть API процессов – системный вызов `exec()`². Он полезен, когда требуется запустить программу, отличную от вызвавшей. Например, вызов `fork()` в программе p2.c полезен, только если мы хотим, чтобы работало две копии одной и той же программы. Но зачастую нужно выполнять две *разные* программы, такую возможность и дает `exec()` (рис. 5.3).

¹ Существует несколько случаев, когда `wait()` возвращает управление до выхода потомка; как обычно, читайте страницы руководства. И не слишком доверяйте непререкаемым и ничем не обоснованным утверждениям, встречающимся в этой книге, например: «потомок всегда печатает сообщение первым» или «UNIX – лучшая вещь на свете, даже лучше мороженого».

² В Linux имеется шесть вариантов `exec()`: `execl()`, `execle()`, `execv()`, `execvp()` и `execvpe()`. Подробности смотрите на страницах руководства.

```

1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <unistd.h>
4 #include <string.h>
5 #include <sys/wait.h>
6
7 int main(int argc, char *argv[]) {
8     printf("hello world (pid:%d)\n", (int) getpid());
9     int rc = fork();
10    if (rc < 0) {           // ошибка fork; выход
11        fprintf(stderr, "ошибка fork\n");
12        exit(1);
13    } else if (rc == 0) { // потомок (новый процесс)
14        printf("hello, I am child (pid:%d)\n", (int) getpid());
15        char *myargs[3];
16        myargs[0] = strdup("wc"); // программа: "wc" (счетчик слов)
17        myargs[1] = strdup("p3.c"); // аргумент: обсчитываемый файл
18        myargs[2] = NULL; // признак конца массива
19        execvp(myargs[0], myargs); // подсчитывает слова
20        printf("эта строка не должна печататься");
21    } else { // родитель следует по этому пути (в main)
22        int rc_wait = wait(NULL);
23        printf("hello, I am parent of %d (rc_wait:%d) (pid:%d)\n",
24               rc, rc_wait, (int) getpid());
25    }
26    return 0;
27 }
28

```

Рис. 5.3 ❖ Вызов `fork()`, `wait()` и `exec()` (p3.c)

В этом примере дочерний процесс вызывает `execvp()`, чтобы запустить программу подсчета слов `wc`. При этом `wc` запускается для исходного файла `p3.c` и, следовательно, сообщает, сколько строк, слов и байтов в этом файле.

```

prompt> ./p3
hello world (pid:29383)
hello, I am child (pid:29384)
    29    107    1030 p3.c
hello, I am parent of 29384 (rc_wait:29384) (pid:29383)
prompt>

```

Системный вызов `fork()` странен, да и его подельник `exec()` не вполне нормален. Вот что он делает: получив имя исполняемого файла (например, `wc`) и аргументы (например, `p3.c`), он **загружает** код (и статические данные) из этого файла и перезаписывает им текущий сегмент кода (и текущие статические данные); куча, стек и другие участки памяти программы инициализируются заново. Затем ОС запускает эту программу, передавая ей аргументы в массиве `argv`. Таким образом, новый процесс *не* создается, вместо этого уже работающая программа (бывшая `p3`) преобразуется в другую работающую программу (`wc`). После выполнения `exec()` в дочернем процессе ситуация вы-

глядит так, будто `р3.c` никогда не запускалась, а успешный вызов `exec()` не возвращает управление.

5.4. Почему? Мотивация API

Разумеется, у вас возник большой вопрос: почему для такого простого действия, как создание нового процесса, нужно было изобретать столь странный интерфейс? Дело в том, что разделение `fork()` и `exec()` принципиально важно для построения оболочки Unix, поскольку позволяет оболочке выполнять код *после* вызова `fork()`, но *до* вызова `exec()`; в этом коде можно изменить окружение еще не запущенной программы и тем самым реализовать целый ряд интересных возможностей.

Совет: сделать правильно (закон Лэмпсона)

Лэмпсон в авторитетной работе «Hints for Computer Systems Design» [L83] писал: **«Делайте правильно**. Ни абстракция, ни простота не являются заменой правильности». Иногда просто необходимо сделать что-то правильно, а когда это сделано, то оказывается гораздо лучше альтернатив. Есть много способов спроектировать API создания процесса, но комбинация `fork()` и `exec()` проста и открывает огромные возможности. В данном случае проектировщики Unix сделали правильно. А поскольку сам Лэмпсон очень часто «делал правильно», закон назван в его честь.

Оболочка – это обычная пользовательская программа¹. Она выводит **приглашение** и ждет, пока вы что-нибудь введете. Затем вы вводите команду (например, имя исполняемой программы и ее аргументы); в большинстве случаев оболочка определяет, где находится программа в файловой системе, вызывает `fork()`, чтобы создать новый дочерний процесс, потом какой-то вариант `exec()`, чтобы выполнить команду, и `wait()`, чтобы дождаться ее завершения. По завершении дочернего процесса `wait()` возвращает управление оболочке, которая снова выводит приглашение в ожидании следующей команды.

Разделение `fork()` и `exec()` позволяет оболочке без особого труда проделать целый ряд полезных вещей. Например:

```
prompt> wc р3.c > newfile.txt
```

В этом примере вывод программы `wc` **перенаправляется** в файл `newfile.txt` (знак «больше» как раз и обозначает перенаправление). И делается это совсем просто: после создания потомка, но перед вызовом `exec()` оболочка закрывает **стандартный вывод** и открывает файл `newfile.txt`. В результате весь вывод еще не запущенной программы `wc` будет отправлен в файл, а не на экран.

¹ И оболочек много: `tcsh`, `bash`, `zsh` – и это далеко не все. Выберите какую-нибудь, прочитайте страницу руководства и изучите ее подробнее; все специалисты по Unix так делают.

На рис. 5.4 показана программа, которая делает то же самое. Такое перенаправление работает благодаря тому, как операционная система обращается с файловыми дескрипторами. Именно Unix начинает поиск свободных файловых дескрипторов с нуля. В данном случае первым будет доступен дескриптор `STDOUT_FILENO`, поэтому он и будет занят в результате вызова `open()`. Поэтому последующие операции записи в дескриптор стандартного вывода в дочернем процессе, например с помощью функции `printf()`, прозрачно перенаправляются во вновь открытый файл, а не на экран.

```

1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <unistd.h>
4 #include <string.h>
5 #include <fcntl.h>
6 #include <sys/wait.h>
7
8 int main(int argc, char *argv[]) {
9     int rc = fork();
10    if (rc < 0) { // ошибка fork; выйти
11        fprintf(stderr, "ошибка fork\n");
12        exit(1);
13    } else if (rc == 0) { // потомок: перенаправить стандартный вывод в файл
14        close(STDOUT_FILENO);
15        open("./p4.output", O_CREAT|O_WRONLY|O_TRUNC, S_IRWXU);
16
17        // теперь выполнить "wc"...
18        char *myargs[3];
19        myargs[0] = strdup("wc"); // программа: "wc" (word count)
20        myargs[1] = strdup("p4.c"); // аргумент: обсчитываемый файл
21        myargs[2] = NULL; // признак конца массива
22        execvp(myargs[0], myargs); // подсчитывает слова
23    } else { // родитель следует по этому пути (в main)
24        int rc_wait = wait(NULL);
25    }
26    return 0;
27 }
```

Рис. 5.4 ❖ Все вышеперечисленное плюс перенаправление (p4.c)

Вот как выглядит результат работы программы `p4.c`:

```

prompt> ./p4
prompt> cat p4.output
32 109 846 p4.c
prompt>
```

Обратите внимание на два (по крайней мере) интересных нюанса этого результата. Во-первых, складывается впечатление, что после запуска `p4` вообще ничего не произошло; оболочка сразу печатает приглашение и готова к вводу следующей команды. Однако впечатление обманчиво – программа `p4` все-таки вызвала `fork()` для создания нового потомка, а затем запустила

программу `wc` посредством вызова `execvp()`. Мы не видим никаких сообщений на экране, потому что вывод перенаправлен в файл `p4.output`. Во-вторых, после распечатки содержимого этого файла программой `cat` видно, что весь ожидаемый вывод оказался в нем. Круто, правда?

Каналы в Unix реализованы похоже, только используется системный вызов `pipe()`. В этом случае выход одного процесса соединяется с находящимся в ядре **каналом** (очередью), и к тому же каналу присоединяется вход другого процесса. Таким образом, выход одного процесса поступает на вход следующего, так что можно сцеплять длинные цепочки команд и получать полезные результаты. В качестве простого примера рассмотрим поиск слова в файле и подсчет числа его вхождений. С помощью каналов и программ `grep` и `wc` сделать это очень легко, нужно лишь ввести `grep -o foo file | wc -l` в ответ на приглашение и полюбоваться результатом.

Наконец, отметим, что пока мы дали лишь общий обзор API, а есть еще и много деталей, которые предстоит изучить. Например, в третьей части книги, при рассмотрении файловых систем мы узнаем о файловых дескрипторах. Пока же достаточно сказать, что комбинация `fork()` и `exec()` – действенный способ создания и манипулирования процессами.

Отступление: RTFM¹ – ЧИТАЙТЕ СТРАНИЦЫ РУКОВОДСТВА

В этой книге при рассмотрении различных системных или библиотечных вызовов мы не раз будем призывать вас читать **страницы руководства** (англ. *man pages*). Это форма документации в Unix-системах, созданная еще до того, как появилась сеть **web**.

Чтение страниц руководства – важный шаг становления системного программиста, это неиссякаемый источник драгоценной информации. Особенно полезны страницы, посвященные используемой вами оболочке (например, **tcsh** или **bash**) и, конечно, системным вызовам, которые делает ваша программа (чтобы узнать, что возвращает система и какие могут быть ошибки).

Наконец, чтение страниц руководства может избавить вас от неловких ситуаций. Если вы спросите коллегу о каком-нибудь тонком моменте `fork()`, то можете услышать в ответ краткое «RTFM». Так коллега мягко и ненавязчиво предлагает вам почитать страницы руководства. Буква F в акрониме RTFM добавляет фразе немного экспрессии...

5.5. УПРАВЛЕНИЕ ПРОЦЕССАМИ И ПОЛЬЗОВАТЕЛИ

Помимо `fork()`, `exec()` и `wait()`, есть еще много интерфейсов для взаимодействия с процессами в Unix-системах. Например, системный вызов `kill()` позволяет отправлять процессу **сигналы**, в т. ч. указания приостановиться, завершиться и т. п. Для удобства в большинстве оболочек Unix определены специальные комбинации клавиш для доставки некоторых сигналов текущему исполняемому процессу; например, `control-c` отправляет сигнал `SIGINT`.

¹ Read the following manual – читайте прилагаемое руководство (менее цензурную расшифровку можно найти в интернете). – *Прим. перев.*

GINT (прерывание), который обычно приводит к снятию процесса, а control-z отправляет сигнал SIGTSTP (стоп), что заставляет процесс приостановиться на время (позже его можно возобновить, например с помощью команды fg, встроенной в большинство оболочек).

Подсистема сигналов в целом предлагает развитую инфраструктуру для доставки процессам внешних событий, включающую возможность принимать и обрабатывать сигналы внутри процессов, а также отправлять сигналы как отдельным процессам, так и **группам процессов**. Чтобы воспользоваться этой формой коммуникации, процесс должен с помощью системного вызова `signal()` «перехватывать» различные сигналы, тогда при получении сигнала нормальное выполнение будет приостановлено и управление передано функции, назначенной в качестве обработчика сигнала. О сигналах и многочисленных связанных с ними тонкостях можно прочесть, например, в работе [SR05].

Естественно возникает вопрос: кто может, а кто не может отправлять сигнал процессу? В общем случае с рассматриваемыми нами системами может одновременно работать несколько человек; если бы любой из них мог посылать сигналы, тот же SIGINT (который прерывает и, скорее всего, завершает процесс) любому процессу, то безопасность системы оказалась бы под вопросом. Поэтому в современных системах имеется четко определенная концепция **пользователя**. Пользователь вводит свой пароль и входит в систему, получая доступ к ее ресурсам. Затем пользователь может запускать процессы и управлять ими, как пожелает (приостанавливать, снимать и т. д.). Обычно пользователь вправе управлять только своими процессами, а распределение ресурсов (таких как процессор, память и диск) между пользователями с учетом общесистемных целей – задача операционной системы.

ОТСТУПЛЕНИЕ: СУПЕРПОЛЬЗОВАТЕЛЬ (root)

Как правило, в системе должен быть пользователь, который ее **администрирует** и не ограничен в правах, как обычные пользователи. Он должен иметь право снимать любой процесс (например, если сочтет, что тот ведет себя некорректно), пусть даже запущенный другим пользователем. У такого пользователя должно быть также право выполнять такие опасные команды, как shutdown (которая останавливает систему). В Unix-системах подобные особые права выдаются **суперпользователю** (иногда его называют **root**). Большинство пользователей не могут снимать чужие процессы, суперпользователь может. Суперпользователь сродни человеку-пауку: большая власть подразумевает большую ответственность [Q15]. Таким образом, чтобы повысить уровень **безопасности** (и избежать дорогостоящих ошибок), лучше работать от имени обычного пользователя; если же вам все-таки необходимо быть суперпользователем, ступайте осторожно, потому что теперь в ваших руках вся мощь компьютера.

5.6. ПОЛЕЗНЫЕ ИНСТРУМЕНТЫ

Существует много других полезных командных инструментов. Например, команда `ps` позволяет узнать, какие процессы сейчас работают, о флагах `ps` читайте на **страницах руководства**. Очень полезна также команда `top`, ко-

торая отображает работающие процессы и сколько они потребляют ресурсов, в т. ч. процессорного времени. Забавно, что довольно часто `top` показывает, что главным пожирателем ресурсов является она сама, быть может, это проявление самолюбования. Команда `kill` позволяет посылать произвольные сигналы процессам, как и чуть более дружественная к пользователю команда `killall`. Но пользуйтесь ими осторожно; если вы случайно снимете свой оконный менеджер, то использовать компьютер, перед которым вы сидите, станет затруднительно.

Наконец, есть много других средств измерения, позволяющих с одного взгляда оценить, как загружена система. Например, на панели инструментов в нашем Macintosh всегда присутствует программа **MenuMeters** (от компании Raging Menace), поэтому мы видим, какая часть ресурсов CPU потребляется в каждый момент времени. Вообще, чем больше у вас информации о происходящем, тем лучше.

5.7. РЕЗЮМЕ

Мы познакомились с некоторыми API, относящимися к созданию процессов в Unix: `fork()`, `exec()` и `wait()`. Но это лишь верхушка айсберга. Дополнительные сведения см. в книге Stevens and Rago [SR05], разумеется, особый интерес представляют главы, посвященные управлению процессами, связям между процессами и сигналам. В этой книге масса полезной информации.

ОТСТУПЛЕНИЕ: ОСНОВНЫЕ ТЕРМИНЫ API ПРОЦЕССОВ

- У любого процесса есть имя; в большинстве систем в качестве имени используется число, называемое **идентификатором процесса (PID)**.
- Системный вызов **`fork()`** используется в Unix-системах для создания нового процесса. Процесс-создатель называется **родителем**, а вновь созданный процесс – **потомком**, или **дочерним процессом**. Как и в реальной жизни [16], процесс-потомок является почти точной копией своего родителя.
- Системный вызов **`wait()`** позволяет родителю ждать завершения потомка.
- Семейство системных вызовов **`exec()`** позволяет потомку освободиться от бремени сходства с родителем и выполнить совершенно другую программу.
- В Unix **оболочка** обычно использует системные вызовы `fork()`, `wait()` и `exec()` для запуска пользовательских команд. Разделение `fork` и `exec` открывает возможность для **перенаправления ввода-вывода**, создания **конвейеров** и других полезных функций, при этом в запускаемых программах ничего изменять не нужно.
- Управление процессами выступает в форме **сигналов**, которые позволяют приостанавливать, возобновлять и даже завершать задания.
- Концепция **пользователя** определяет, какими именно процессами может управлять конкретное лицо. Операционная система допускает наличие нескольких пользователей в системе и гарантирует, что каждый сможет управлять только своими процессами.
- **Суперпользователь** может управлять всеми процессами (и еще много чего делать), примеривать на себя эту роль следует нечасто и с осторожностью – из соображений безопасности.

Литература

[C63] «A Multiprocessor System Design» by Melvin E. Conway. AFIPS '63 Fall Joint Computer Conference. New York, USA 1963. *Ранняя статья о проектировании многопроцессорных систем; быть может, именно здесь впервые был употреблен термин `fork()` в контексте обсуждения запуска новых процессов.*

[DV66] «Programming Semantics for Multiprogrammed Computations» by Jack B. Dennis and Earl C. Van Horn. Communications of the ACM, Volume 9, Number 3, March 1966. *Классическая работа, в которой описаны основы мультипрограммных вычислительных систем. Безусловно, оказала большое влияние на проекты Project MAC, Multics и в конечном итоге Unix.*

[J16] «They could be twins!» by Phoebe Jackson-Edwards. The Daily Mail. March 1, 2016. Доступно по адресу www.dailymail.co.uk/femail/article-3469189/Photos-children-look-IDENTICAL-parents-age-sweep-web.html. *В этой бьющей наповал журналистской работе приведены фотографии поразительно похожих детей и родителей, они буквально завораживают. Потратьте пару минут своей жизни, чтобы посмотреть на это. Но не забудьте вернуться сюда! Блуждание по вебу может быть опасно.*

[L83] «Hints for Computer Systems Design» by Butler Lampson. ACM Operating Systems Review, Volume 15:5, October 1983. *Знаменитые заметки Лэмпсона о проектировании компьютерных систем. Их надо прочитать хотя бы раз в жизни, а может быть, и несколько раз.*

[QI15] «With Great Power Comes Great Responsibility» by The Quote Investigator. Доступно по адресу <https://quoteinvestigator.com/2015/07/23/great-power>. *Исследователь цитат приходит к выводу, что самое раннее упоминание вынесенного в заголовок высказывания «большая власть подразумевает большую ответственность» относится к 1793 году, оно встретилось в собрании декретов Национального Конвента Франции. На языке оригинала оно звучит так: «Ils doivent envisager qu'une grande responsabilite est la suite insparable d'un grand pouvoir», что приблизительно переводится как «Надлежит им помнить, что с великой властью неразделима великая ответственность». Лишь в 1962 году в фильме «Человек-паук» прозвучали слова «...with great power there must also come great responsibility!». Похоже, что воздать за них должно следует Французской революции, а не Стэну Ли. Уж извини, Стэн.*

[SR05] «Advanced Programming in the UNIX Environment»¹ by W. Richard Stevens, Stephen A. Rago. Addison-Wesley, 2005. *Здесь вы найдете все нюансы и тонкости использования API в UNIX. Купите эту книгу! Читайте ее! И самое главное – **живите** по ее заповедям.*

¹ У. Ричард Стивенс, Стивен А. Раго. UNIX. Профессиональное программирование. 3-е изд. Питер, 2018.

ОТСТУПЛЕНИЕ: ЗАДАЧИ НА КОДИРОВАНИЕ

Домашние задачи на кодирование – это небольшие упражнения, в которых вам предлагается написать код и выполнить его на реальной машине, чтобы получить опыт работы с базовыми API операционной системы. В конце концов, вы (предположительно) специалист по компьютерным наукам, и, значит, вам должно нравиться кодирование, верно? (А если нет, что ж, есть еще и теоретическая информатика, но это довольно трудная наука.) Конечно, чтобы стать настоящим спецом, нужно провести с машиной несколько больше, чем «немного времени». В общем, ищите любой предлог для того, чтобы попрактиковаться в написании кода и посмотреть, как он работает. Тратьте свое время и станьте мудрым гуру – вы, безусловно, можете им стать.

Домашнее задание (кодирование)

В этом домашнем задании вы получите поближе познакомиться с API управления процессами, о которых только что прочитали. Не пугайтесь – это даже интереснее, чем кажется! Вообще, чем больше времени вы найдете для написания кода, тем лучше. Так почему бы не начать прямо сейчас?

Вопросы

1. Напишите программу, вызывающую `fork()`. Но прежде чем вызвать `fork()`, главный процесс должен присвоить какой-нибудь переменной (скажем, `x`) какое-то значение (скажем, `100`). Какое значение эта переменная будет иметь в дочернем процессе? Что произойдет с переменной `x`, если и родитель, и потомок изменят ее значение?
2. Напишите программу, которая открывает файл (с помощью системного вызова `open()`), а затем вызывает `fork()` для создания нового процесса. Могут ли родитель и потомок обратиться к файловому дескриптору, возвращенному `open()`? Что произойдет, если они будут писать в файл одновременно?
3. Напишите еще программу, в которой используется `fork()`. Дочерний процесс должен напечатать «hello», а родительский процесс – «goodbye». Сделайте так, чтобы дочерний процесс всегда печатал свое сообщение первым. Сможете ли вы сделать это, *не* вызывая `wait()` в родителе?
4. Напишите программу, которая вызывает `fork()`, а затем какую-то форму `exec()` для запуска программы `/bin/ls`. Попробуйте использовать все варианты `exec()`, в т. ч. (в Linux) `execl()`, `execle()`, `execlp()`, `execv()`, `execvp()` и `execve()`. Как вы думаете, почему существует так много вариантов одного и того же по сути вызова?
5. Теперь напишите программу, которая вызывает в родителе `wait()`, чтобы дожидаться завершения потомка. Что возвращает `wait()`? Что будет, если вызвать `wait()` в потомке?

6. Модифицируйте предыдущую программу, воспользовавшись `waitpid()` вместо `wait()`. Когда стоило бы использовать `waitpid()`?
7. Напишите программу, которая создает дочерний процесс и в нем закрывает стандартный вывод (`STDOUT_FILENO`). Что будет, если потомок вызовет функцию `printf()`, чтобы напечатать что-то после закрытия дескриптора?
8. Напишите программу, которая создает двух потомков и соединяет стандартный вывод первого со стандартным вводом второго, используя системный вызов `pipe()`.

Глава 6

Механизм: ограниченное прямое выполнение

Чтобы виртуализировать CPU, операционной системе нужно каким-то способом разделить физический процессор между несколькими заданиями, которые будто бы выполняются одновременно. Идея проста: дать немного поработать одному процессу, затем другому и т. д. Благодаря такому **разделению времени** процессора и достигается виртуализация.

Однако реализация подобного механизма наталкивается на некоторые трудности. Первая из них – *производительность*: как осуществить виртуализацию, не допустив слишком больших накладных расходов? Вторая – *контроль*: как эффективно выполнять процессы, сохраняя в то же время контроль над CPU? Вопрос контроля особенно важен для ОС, потому что она управляет всеми ресурсами; без контроля процесс мог бы работать бесконечно, завладев всей машиной, или получать доступ к информации, которая должна быть ему недоступна. Таким образом, достижение высокой производительности при сохранении контроля – одна из главных проблем при построении операционной системы.

Существо проблемы:

КАК ЭФФЕКТИВНО ВИРТУАЛИЗИРОВАТЬ CPU, СОХРАНИВ НАД НИМ КОНТРОЛЬ

ОС должна эффективно виртуализировать CPU, сохранив при этом контроль над системой. Для этого необходима поддержка со стороны оборудования. ОС часто пользуется аппаратной поддержкой для эффективного достижения своих целей.

6.1. БАЗОВАЯ ТЕХНИКА: ОГРАНИЧЕННОЕ ПРЯМОЕ ВЫПОЛНЕНИЕ

Программа должна работать с той скоростью, какой от нее ожидают, поэтому неудивительно, что разработчики ОС придумали метод, который мы будем называть **ограниченным прямым выполнением**. Часть «прямое выполнение» проста: программа выполняется непосредственно на процессоре. Таким образом, когда ОС хочет запустить программу, она создает запись в списке процессов, выделяет для процесса память, загружает в нее код программы (с диска), находит точку входа в программу (функцию `main()` или нечто подобное), переходит к ней и начинает исполнять пользовательский код. На рис. 6.1 показан этот протокол прямого выполнения (пока без ограничений), в котором используются обычные команды вызова и возврата для перехода в начало `main()` и последующего возврата в ядро.

ОС	Программа
Создать запись в списке процессов	
Выделить память для программы	
Загрузить программу в память	
Поместить в стек <code>argc</code> и <code>argv</code>	
Очистить регистры	
Выполнить вызов <code>main()</code>	Выполнить <code>main()</code>
	Выполнить возврат из <code>main()</code>
Освободить память процесса	
Удалить запись из списка процессов	

Рис. 6.1 ❖ Протокол прямого выполнения (без ограничений)

Кажется просто, да? Но при таком подходе возникает несколько проблем на пути к задуманной виртуализации CPU. Первая проста: если мы запустим программу, то как ОС сможет гарантировать, что программа не сможет делать то, что ей запрещено, но при этом будет работать эффективно? Вторая: как ОС сможет остановить работающий процесс и переключиться на другой, реализовав тем самым **разделение времени**?

Отвечая на эти вопросы, мы гораздо лучше поймем, что необходимо для виртуализации CPU. Заодно мы увидим, когда возникают «ограничения»; ни в чем не ограничивая программу, ОС не смогла бы ничего контролировать, а значит, была бы «простой библиотекой» – весьма униженное положение для амбициозной операционной системы!

6.2. ПРОБЛЕМА 1: ЗАПРЕЩЕННЫЕ ОПЕРАЦИИ

Прямое выполнение имеет очевидное преимущество – быстроту; программа выполняется непосредственно аппаратурой, поэтому работает настолько

быстро, насколько это вообще возможно. Но исполнение на CPU порождает проблему: что, если процесс захочет выполнить какую-то запрещенную операцию, например отправить запрос ввода-вывода диску или получить доступ к дополнительным системным ресурсам, скажем памяти?

Существо проблемы: как выполнять операции с ограниченным доступом

ОС должна давать процессу возможность выполнять ввод-вывод и другие операции с ограниченным доступом, не предоставляя при этом полного контроля над системой. Как ОС совместно с оборудованием могут решить эту проблему?

Отступление: почему системный вызов выглядит как вызов функции

Вам, наверное, интересно, почему системный вызов, например `open()` или `read()`, выглядит в точности как вызов типичной функции на C. Точнее, если он выглядит как вызов функции, то откуда система знает, что это системный вызов, и выполняет все положенные действия? Ответ прост: это *и есть* вызов функции, но внутри нее находится команда системного прерывания. Когда мы вызываем, например, `open()`, производится обращение к библиотечной функции. А уже сама библиотека следует принятому соглашению о вызове ядра: помещает аргументы в точно определенные места (например, в стек или в конкретные регистры), записывает номер системного вызова в еще одно точно определенное место (снова в стек или в регистр) и выполняет команду системного прерывания. Библиотечный код, находящийся после команды прерывания, распаковывает возвращенные значения и возвращает управление программе, выполнившей системный вызов. Части библиотеки, отвечающие за системные вызовы, написаны на ассемблере, поскольку должны точно следовать соглашению о передаче аргументов и обработке возвращенных значений и вызывать аппаратную команду прерывания – на C этот код написать невозможно. Теперь вы знаете, что лично вам не придется писать ассемблерный код для вызова ядра, это уже сделал кто-то другой.

Один из возможных подходов – просто позволить процессу делать все, что ему заблагорассудится, в плане ввода-вывода и родственных операций. Но тогда мы не смогли бы сконструировать крайне желательные нам системы. Например, если мы хотим построить файловую систему, которая проверяет права, перед тем как дать доступ к файлу, то никак не можем позволить любому пользовательскому процессу выполнять ввод-вывод на диск, иначе процесс попросту прочитал бы или записал бы весь диск, и все наши меры защиты пошли бы насмарку.

Поэтому мы введем новый режим процессора, называемый **режимом пользователя**; в этом режиме возможности кода ограничены. Например, код, работающий в режиме пользователя, не может отправлять запросы ввода-вывода – любая попытка сделать это привела бы к исключению процессора, и ОС, скорее всего, сняла бы процесс.

С другой стороны, в **режиме ядра** работает сама операционная система (или ядро). В этом режиме код может делать все, что угодно, в т. ч. выполнять привилегированные операции – ввод-вывод и все специальные команды.

Однако проблема осталась: что должен сделать пользовательский процесс, если ему требуется выполнить привилегированную операцию, например прочитать данные с диска? Для этого практически любое современное оборудование позволяет пользовательской программе выполнять **системный вызов**. Впервые системные вызовы появились еще в древних машинах, например Atlas [K+61, L78], они дают ядру возможность аккуратно раскрывать часть своей функциональности пользовательской программе, например обращаться к файловой системе, создавать и уничтожать процессы, взаимодействовать с другими процессами и выделять память. В большинстве операционных систем насчитывается несколько сотен системных вызовов (подробнее см. стандарт POSIX [P10]); в ранних системах Unix их число было скромнее – примерно два десятка.

СОВЕТ: ПРИМЕНЯЙТЕ ЗАЩИЩЕННУЮ ПЕРЕДАЧУ УПРАВЛЕНИЯ

Оборудование помогает ОС, предоставляя различные режимы выполнения. В режиме пользователя приложения не имеют полного доступа к аппаратным ресурсам. В режиме ядра ОС имеет доступ ко всем ресурсам машины. Предоставляются также специальные команды: системного прерывания (для входа в ядро) и возврата из прерывания (для возврата в режим пользователя), а также команды, позволяющие ОС сообщить оборудованию, где в памяти находится таблица прерываний.

Чтобы обратиться к системе, программа должна выполнить специальную команду **системного прерывания** (trap), которая делает сразу две вещи: переход в ядро и увеличение уровня привилегий до режима ядра. Находясь в режиме ядра, система может выполнять любые привилегированные операции и, стало быть, проделать работу, запрошенную вызывающим процессом (если ему это разрешено). По завершении ОС вызывает специальную команду **возврата из прерывания**, которая возвращает управление пользовательской программе и одновременно понижает уровень привилегий.

Оборудование должно принять меры предосторожности при выполнении команды прерывания: сохранить необходимые регистры, чтобы корректно вернуть управление пользовательской программе. Например, процессор x86 помещает в **стек ядра** счетчик команд, флаги и еще несколько регистров, а команда возврата из прерывания выбирает эти значения из стека и возобновляет выполнение программы в пользовательском режиме (детали см. в системных руководствах Intel [I11]). В других компьютерах действуют иные соглашения, но основная идея всюду одинакова.

Мы упустили еще одну важную деталь: откуда команда системного прерывания знает, какой код внутри ОС выполнять? Очевидно, что вызывающий процесс не может указать адрес перехода (как при вызове обычной функции), иначе он смог бы перейти в любое место ядра, а это **Очень Плохая Идея**¹.

¹ Представьте, что вы перешли в код доступа к файлу, но уже после проверки прав. Вообще, наличие такой возможности позволило бы злокозненному программисту выполнить произвольный код [S07]. Старайтесь всячески избегать Очень Плохих Идей такого рода.

Таким образом, ядро должно тщательно контролировать, какой код выполняется после прерывания.

Для этого ядро на этапе начальной загрузки инициализирует **таблицу прерываний**. В процессе начальной загрузки система работает в привилегированном режиме ядра, поэтому может конфигурировать оборудование как угодно. Едва ли не первым делом ОС устанавливает, что должно делать оборудование при возникновении определенных исключительных событий. Например, какой код выполнять, когда происходит прерывание по обращению к жесткому диску, прерывание от клавиатуры или системный вызов? ОС информирует оборудование об адресах **обработчиков прерываний**, для чего обычно предусмотрена специальная команда. Переданные оборудованию адреса не меняются до следующей перезагрузки машины, поэтому оборудование знает, что делать (к какому коду переходить) в случае системных вызовов и других исключительных событий.

Обычно каждому системному вызову присваивается уникальный **номер системного вызова**. Таким образом, пользовательский код должен поместить номер нужного системного вызова в регистр или в определенную позицию стека, а ОС в ходе обработки извлекает этот номер, проверяет его и выполняет соответствующий код. Эта косвенность является формой **защиты**; пользовательский код не может задать точный адрес перехода, а должен запрашивать обслуживание по номеру.

И последнее замечание: уведомление оборудования о месте нахождения таблицы прерываний – чрезвычайно опасная операция. Поэтому, как вы наверняка догадались, она относится к числу **привилегированных**. Оборудование пресечет попытку выполнить ее в режиме пользователя, и, надо полагать, вы понимаете, что за этим последует (подсказка: прощай, нехорошая программа). Тема для размышления: что страшного могло бы произойти с системой, если бы вам было разрешено устанавливать собственную таблицу прерываний? Могли бы перехватить управление машиной?

Совет: берегитесь данных от пользователя в безопасных системах

Хотя мы предприняли все меры, чтобы защитить ОС во время системных вызовов (добавив аппаратный механизм прерываний и гарантировав, что обращения к ОС проходят через него), для реализации **безопасной** операционной системы нужно учесть еще множество аспектов. Один из них – проверка аргументов на границе системного вызова; ОС должна проверять все, что передает пользователь, и отвергать вызов, если аргументы недопустимы. Например, в случае системного вызова `write()` пользователь задает адрес буфера, в котором находятся записываемые данные. Если указан (случайно или намеренно) «плохой» адрес (например, принадлежащий части адресного пространства, занятого ядром), то ОС должна это обнаружить и отвергнуть вызов. В противном случае пользователь мог бы прочитать всю память ядра, а учитывая, что (виртуальная) память ядра обычно включает всю физическую память системы, это небольшое упущение позволило бы программе прочитать память любого другого процесса.

Вообще, безопасная система должна относиться к данным, полученным от пользователя, с большим подозрением. Пренебрежение этим правилом ведет к легко взламываемым программам, ощущению, что мир – опасное и устрашающее место, и потере работы излишне доверчивым разработчиком ОС.

Хронология событий на рис. 6.2 (время увеличивается сверху вниз) отражает протокол. Мы предполагаем, что у каждого процесса имеется стек ядра, в котором сохраняются регистры (в т. ч. регистры общего назначения и счетчик команд) и из которого они затем восстанавливаются (аппаратно) при входе и выходе из ядра.

ОС на этапе начальной загрузки (режим ядра)	Оборудование	
Инициализировать таблицу прерываний	Запомнить адрес обработчика системного вызова	
ОС на этапе выполнения (режим ядра)	Оборудование	Программа (режим пользователя)
Создать запись в списке процессов Выделить память для программы Загрузить программу в память Поместить argv в стек пользователя Поместить регистры и PC в стек ядра Возврат из прерывания	Восстановить регистры из стека ядра Перейти в режим пользователя Вернуться в main	Выполнить main() ... Выполнить системный вызов Системное прерывание с переходом в ОС
Обработать прерывание Проделать работу системного вызова Возврат из прерывания	Сохранить регистры в стеке ядра Перейти в режим ядра Перейти к обработчику прерывания	
	Сохранить регистры в стеке ядра Перейти в режим ядра Перейти к обработчику прерывания	... Возврат из main Системное прерывание (из-за exit())
Освободить память процесса Удалить запись из списка процессов		

Рис. 6.2 ❖ Протокол ограниченного прямого выполнения

Протокол ограниченного прямого выполнения (limited direct execution – **LDE**) состоит из двух этапов. На первом этапе (во время начальной загрузки) ядро инициализирует таблицу прерываний, а CPU запоминает ее местоположение для последующего использования. Для этого ядро применяет привилегированную команду (все привилегированные команды показаны полужирным шрифтом).

На втором этапе (во время выполнения процесса) ядро производит инициализацию (в частности, создает запись в списке процессов, выделяет память), прежде чем выполнить команду возврата из прерывания; при этом процессор переключается в режим пользователя и начинает выполнять процесс. Когда процесс хочет сделать системный вызов, он выполняет команду системного прерывания, ОС обрабатывает ее и опять-таки возвращает управ-

ление с помощью команды возврата из прерывания. Процесс завершает работу и выходит из `main()`, обычно он при этом попадает в некий специальный код, который осуществляет правильный выход из программы (например, с помощью системного вызова `exit()`, который передает управление ОС). В этот момент ОС производит очистку, и на этом все заканчивается.

6.3. ПРОБЛЕМА 2: ПЕРЕКЛЮЧЕНИЕ МЕЖДУ ПРОЦЕССАМИ

Следующая проблема прямого выполнения – как переключиться между процессами. Это ведь должно быть просто, правда? ОС только и нужно, что решить, какой процесс остановить и какой запустить. Не такая уж сложная задача? Ан нет, ведь если процесс занимает CPU, значит, по определению, ОС не выполняется. А если она не выполняется, то как может вообще что-то сделать? (Подсказка: никак.) Звучит это как философский вопрос, но проблема реальная: у ОС нет никакой возможности предпринять действие, если она не выполняется процессором. Вот мы и подошли к сути проблемы.

СУЩЕСТВО ПРОБЛЕМЫ: КАК ВЕРНУТЬ СЕБЕ КОНТРОЛЬ НАД CPU

Как операционная система может восстановить контроль над CPU, чтобы переключить процессы?

Кооперативный подход: дождаться системного вызова

В прошлом в некоторых системах (например, в первых версиях ОС Macintosh [M11] и в старой системе Xerox Alto [A79]) использовался **кооперативный** подход. Это означает, что ОС *доверяет* процессам, полагая, что они ведут себя разумно. Предполагается, что если процесс работает долго, то он будет время от времени уступать процессор, чтобы ОС могла дать поработать другой задаче.

Но каким образом благородный процесс уступает CPU в этом утопическом мире? Оказывается, что большинство процессов довольно часто передают управление ОС, выполняя системные вызовы, например чтобы открыть файл и прочитать из него данные, или чтобы отправить сообщение другой машине, или чтобы создать новый процесс. В таких системах часто имеется явный системный вызов **уступить** (`yield`), который не делает ничего, кроме передачи управления ОС, чтобы та могла запустить другой процесс.

Кроме того, приложения передают управление ОС, когда делают что-то недопустимое. Например, если приложение выполняет деление на ноль или пытается обратиться к памяти, к которой не должна обращаться, то возни-

кает **системное прерывание**. Тогда ОС получает CPU в свое распоряжение (и, вероятно, снимает сбойный процесс).

СОВЕТ: ОБРАБОТКА НЕПРАВИЛЬНОГО ПОВЕДЕНИЯ ПРИЛОЖЕНИЯ

Операционным системам часто приходится иметь дело с некорректными процессами, которые специально (злонамеренно) или случайно (по ошибке) пытаются сделать нечто такое, чего делать не должны. Современные ОС реагируют на такое пренебрежение приличиями, просто «выгоняя» провинившийся процесс. Один удар – и ты в ауте! Быть может, жестоко, но что же делать ОС, если вы пытаетесь незаконно обратиться к памяти или выполнить не разрешенную вам команду?

Таким образом, в системе с кооперативным планированием ОС для возврата себе контроля над CPU должна терпеливо ждать, когда случится системный вызов или какая-то недопустимая операция. Вы, наверное, тоже думаете, что такой пассивный подход далеко не идеален. Что, например, произойдет, если процесс (вредоносный или просто полный ошибок) зациклится и никогда не будет вызывать систему? Что делать ОС в таком случае?

Некооперативный подход: ОС силой забирает управление

Без поддержки со стороны оборудования ОС ничего не могла бы сделать, если процесс отказывается совершать системные вызовы (и не делает ошибок) и не хочет передавать ей управление. На самом деле кооперативный подход оставляет нам только один выход, когда процесс зацикливается, – прибегнуть к древнему решению всех проблем в компьютерных системах: **перезагрузить машину**. Так что мы снова пришли к тому же вопросу.

СУЩЕСТВО ПРОБЛЕМЫ: КАК ВЕРНУТЬ СЕБЕ КОНТРОЛЬ БЕЗ КООПЕРАЦИИ

Как ОС может восстановить контроль над CPU, даже если процессы отказываются сотрудничать? Что может сделать ОС, чтобы жуликоватый процесс не захватил контроль над машиной?

Ответ простой, он был найден разными людьми, занимавшимися созданием компьютерных систем много лет назад: **прерывание от таймера** [М+63]. Таймер можно запрограммировать, так чтобы он генерировал прерывание каждые несколько миллисекунд. Когда возникает прерывание, текущий процесс приостанавливается, и начинает работать заранее сконфигурированный **обработчик прерывания** внутри ОС. В этот момент ОС вновь получила контроль над CPU и может делать все, что захочет, в частности остановить текущий процесс и запустить новый.

При обсуждении системных вызовов мы уже говорили, что ОС должна проинформировать оборудование, какой код выполнять при возникновении

прерывания от таймера; это делается на этапе начальной загрузки. Кроме того, в процессе начальной загрузки ОС должна запустить таймер, и это, конечно же, привилегированная операция. После того как таймер запущен, ОС может быть уверена, что рано или поздно контроль вернется к ней, а потому может со спокойной душой запускать пользовательские программы. Таймер можно и остановить (и это тоже привилегированная операция), мы вернемся к этому вопросу, когда дойдем до обсуждения конкурентности.

Совет: использование таймера для восстановления контроля

Добавление **прерывания от таймера** позволяет ОС вернуть себе контроль над CPU, даже если процессы отказываются сотрудничать. Таким образом, это аппаратное средство принципиально необходимо для контроля над машиной со стороны ОС.

Совет: перезагрузка полезна

Выше мы отметили, что единственное решение проблемы бесконечных циклов (и любого подобного поведения) при кооперативном вытеснении – **перезагрузить** машину. И не надо презрительно фыркать – исследователи показали, что перезагрузка (или, более общо, перезапуск какой-то программы) может быть очень полезным инструментом для построения надежных систем [C+04].

Конкретно перезагрузка полезна тем, что переводит программу в известное и, вероятно, более тщательно протестированное состояние. К тому же перезагрузка позволяет вернуть забытые или утекающие ресурсы (например, память), что не всегда просто сделать иным способом. Наконец, перезагрузку легко автоматизировать. Поэтому в крупномасштабных кластерных интернет-службах нередко можно встретить ПО управления системой, которое периодически перезагружает машины, чтобы сбросить их в исходное состояние и тем самым получить все вышеперечисленные преимущества.

Так что в следующий раз, когда будете перезагружать машину, не думайте, что совершили гадкое дело. Отнюдь! На самом деле вы применили проверенный временем подход, улучшающий поведение компьютерной системы. Отлично сработано!

Отметим, что у оборудования есть определенные обязанности в момент возникновения прерывания, а именно сохранить достаточную часть состояния работавшей программы, чтобы при последующем возврате из прерывания правильно восстановить его. Эти действия очень похожи на те, что оборудование производит при явном переходе в ядро с помощью системного прерывания: различные регистры сохраняются (например, в стеке ядра), а затем легко восстанавливаются при выполнении команды возврата из прерывания.

Сохранение и восстановление контекста

Итак, ОС вернула себе контроль – не важно, кооперативно или силой с помощью прерывания от таймера, – и теперь нужно принимать решение: дать возможность и дальше работать текущему процессу или переключиться на дру-

гой. Это решение принимает **планировщик** – часть операционной системы; политики планирования мы будем подробнее обсуждать в следующих главах.

Если ОС решила переключиться, то выполняет низкоуровневый код **контекстного переключения**. Концептуально тут все просто: ОС должна сохранить несколько регистров текущего процесса (например, в его стеке ядра) и восстановить регистры будущего процесса (из его стека ядра). Таким образом, ОС гарантирует, что после команды возврата из прерывания управление не вернется процессу, который выполнялся перед этим, а возобновится выполнение другого процесса.

Чтобы переключить контекст, ОС выполняет написанный на ассемблере код сохранения регистров общего назначения, счетчика команд и указателя стека ядра текущего процесса, а затем восстанавливает все то же самое для процесса, который будет выполняться следующим. Благодаря переключению стеков ядро входит в код переключения в контексте одного процесса (того, что был прерван), а возвращается уже в контексте другого (того, который вот-вот начнет выполняться). Когда ОС наконец выполнит команду возврата из прерывания, новый процесс станет текущим. И на этом контекстное переключение завершается.

На рис. 6.3 показана хронология всей этой процедуры. В данном случае во время выполнения процесса А возникает прерывание от таймера. оборудо-



Рис. 6.3 ❖ Протокол ограниченного прямого выполнения (прерывание от таймера)

вание сохраняет его регистры (в его стеке ядра) и входит в ядро (переключаясь в режим ядра). В обработчике прерывания от таймера ОС решает переключиться с процесса А на процесс В. В этот момент она вызывает функцию `switch()`, которая сохраняет текущие значения регистров (в записи о процессе А), восстанавливает регистры процесса В (из записи о нем) и **переключает контекст**, т. е. заменяет указатель стека, так чтобы он указывал на стек ядра В (не А, как раньше). Наконец, ОС возвращается из прерывания, при этом восстанавливаются регистры В, и процесс В начинает выполняться.

Отметим, что в этом протоколе присутствуют два вида сохранения и восстановления регистров. Первый – когда происходит прерывание от таймера; в этом случае *пользовательские регистры* работающего процесса неявно сохраняются *оборудованием* в стеке ядра данного процесса. Второй – когда ОС решает переключиться с А на В; тогда *ядерные регистры* явно сохраняются программой (т. е. ОС), но на этот раз в записи о процессе. Последнее действие переводит систему из состояния, когда она вошла в ядро из А, в состояние, когда кажется, что она только что вошла в ядро из В.

Чтобы вы лучше поняли, как происходит контекстное переключение, на рис. 6.4 показан его код в системе xv6. Попробуйте разобраться в нем (для этого нужно кое-что знать о процессоре x86 и о xv6). Структуры `old` и `new` типа `context` – это записи о старом и новом процессах соответственно.

```

1 # void swtch(struct context **old, struct context *new);
2 #
3 # Сохранить текущий регистровый контекст в old,
4 # а затем загрузить регистровый контекст из new.
5 .globl swtch
6 swtch:
7 # Сохранить старые значения регистров
8 movl 4(%esp), %eax # поместить старый указатель стека в eax
9 popl 0(%eax)      # сохранить старый счетчик команд
10 movl %esp, 4(%eax) # и указатель стека
11 movl %ebx, 8(%eax) # и остальные регистры
12 movl %ecx, 12(%eax)
13 movl %edx, 16(%eax)
14 movl %esi, 20(%eax)
15 movl %edi, 24(%eax)
16 movl %ebp, 28(%eax)
17
18 # Загрузить новые значения регистров
19 movl 4(%esp), %eax # поместить новый указатель стека в eax
20 movl 28(%eax), %ebp # восстановить остальные регистры
21 movl 24(%eax), %edi
22 movl 20(%eax), %esi
23 movl 16(%eax), %edx
24 movl 12(%eax), %ecx
25 movl 8(%eax), %ebx
26 movl 4(%eax), %esp # здесь происходит переключение стеков
27 pushl 0(%eax)      # записать в нужное место адрес возврата
28 get               # и наконец вернуться уже в новом контексте

```

Рис. 6.4 ❖ Код контекстного переключения в xv6

6.4. Сомневаетесь насчет конкурентности?

Вдумчивые и внимательные читатели, возможно, задумались над следующими вопросами: «А что будет, если во время системного вызова произойдет прерывание от таймера?» или «А что, если во время обработки одного прерывания произойдет другое? Можно ли это вообще обработать в ядре?». Отличные вопросы – но не отчаивайтесь, надежда есть!

Да, ОС действительно должна позаботиться о том, что произойдет, если во время обработки прерывания или системного вызова возникнет другое прерывание. Это, собственно, и есть тема всей второй части книги, посвященной **конкурентности**, поэтому мы до поры отложим подробное обсуждение.

ОТСТУПЛЕНИЕ:

СКОЛЬКО ВРЕМЕНИ ЗАНИМАЕТ КОНТЕКСТНОЕ ПЕРЕКЛЮЧЕНИЕ

Возникает естественный вопрос: сколько времени может занять контекстное переключение? Или системный вызов? Для любопытствующих существует инструментальная программа **lmbench** [MS96], которая измеряет как раз такие вещи, а также еще несколько важных показателей производительности.

Результаты со временем довольно сильно улучшились, что отражает возросшую производительность процессоров. Например, в 1996 году при выполнении Linux 1.3.37 на процессоре P6 частотой 200 МГц системный вызов занимал приблизительно 4 микросекунды, а контекстное переключение – 6 микросекунд [MS96]. В современных системах время почти на порядок меньше, т. е. на процессорах частотой 2–3 ГГц меньше микросекунды.

Отметим, что не все действия операционной системы занимают время, обратно пропорциональное производительности процессора. Как обратил внимание Оустерхаут, многие операции ОС требуют интенсивного доступа к памяти, а быстродействие памяти со временем возросло не так сильно, как быстродействие процессоров [O90]. Таким образом, покупка самого последнего и мощного процессора необязательно ускорит работу ОС так, как вы ожидаете, – все зависит от характера рабочей нагрузки.

Чтобы разжечь у вас аппетит, мы все же напомним, как ОС обрабатывает эти хитрые ситуации. Прежде всего ОС может сделать одну простую вещь – **запретить прерывания** на время обработки прерывания, тогда пока одно прерывание обрабатывается, никакое другое не будет доставлено процессору. Разумеется, ОС должна быть осторожна – если запретить прерывания надолго, то какие-то прерывания могут быть потеряны, а это плохо.

Кроме того, для операционных систем разработан целый ряд хитроумных схем **блокировки**, чтобы защититься от конкурентного доступа к внутренним структурам данных. Это позволяет производить внутри ядра сразу несколько действий, особенно когда машина оборудована несколькими процессорами. Однако в следующей части книги мы увидим, что эти схемы довольно сложны и могут стать причиной интересных и трудно обнаруживаемых ошибок.

6.5. РЕЗЮМЕ

Мы описали некоторые низкоуровневые механизмы виртуализации CPU – набор методов под общим названием **ограниченное прямое выполнение**. Основная идея проста: программа выполняется непосредственно процессором, но предварительно нужно сконфигурировать оборудование, ограничив действия, которые процесс может производить без помощи со стороны ОС.

Этот общий подход применяется и в обычной жизни. Например, те из вас, кто имеет детей или хотя бы слышал об обращении с детьми, наверное, знают о **защите помещений от детей**: запирать шкафчики с опасными веществами, закрывать электрические розетки и т. д. Когда комната подготовлена таким образом, можно спокойно запускать в нее младенца, сознавая, что от самых опасных вещей он изолирован.

ОТСТУПЛЕНИЕ:

ОСНОВНЫЕ ТЕРМИНЫ, ОТНОСЯЩИЕСЯ К ВИРТУАЛИЗАЦИИ CPU (МЕХАНИЗМЫ)

- CPU должен поддерживать по крайней мере два режима работы: ограниченный **режим пользователя** и привилегированный (неограниченный) **режим ядра**.
- Типичные пользовательские приложения работают в режиме пользователя и используют **системные вызовы**, реализованные с помощью **системных прерываний**, чтобы запросить у операционной системы какие-то услуги.
- Команда системного прерывания сохраняет состояние регистров, входит в режим ядра и переходит по адресу, хранящемуся в заранее инициализированной **таблице прерываний**.
- Завершив обслуживание системного вызова, ОС возвращает управление пользовательской программе с помощью команды **возврата из прерывания**, которая понижает уровень привилегий и возобновляет выполнение с команды, следующей за командой системного прерывания.
- Таблица прерываний должна быть инициализирована ОС на этапе начальной загрузки, при этом следует позаботиться о том, чтобы пользовательские программы не могли ее модифицировать. Все это части протокола **ограниченного прямого выполнения**, благодаря которому программы могут работать эффективно, но не лишая ОС контроля.
- Когда программа запущена, ОС должна задействовать аппаратные механизмы, чтобы не дать программе работать вечно, а именно **прерывание от таймера**. Это пример **некооперативного** подхода к планированию CPU.
- Иногда во время прерывания от таймера или системного вызова ОС хочет переключиться с текущего работающего процесса на другой. Для этого применяется низкоуровневая техника **контекстного переключения**.

Точно так же ОС «защищает от детей» процессор: сначала (во время начальной загрузки) инициализирует обработчики системных прерываний и запускает таймер, а только потом разрешает процессам выполняться в ограниченном режиме. При этом ОС может быть уверена, что процессы будут работать эффективно, требуя вмешательства только для выполнения

привилегированных операций или когда монополизировали процессор надолго и потому должны быть вытеснены.

Итак, теперь мы располагаем основными механизмами для виртуализации CPU. Но еще нет ответа на главный вопрос: какой процесс должен работать в каждый момент времени? На него должен ответить планировщик, это и станет нашей следующей темой.

Литература

[A79] «Alto User's Handbook» by Xerox. Xerox Palo Alto Research Center, September 1979. Доступно на <http://history-computer.com/Library/AltoUsersHandbook.pdf>. *Удивительная система, далеко опередившая свое время. Прославилась, потому что ее увидел Стив Джобс, взял на заметку, а затем создал Lisa и в конечном итоге Mac.*

[C+04] «Microreboot – A Technique for Cheap Recovery» by G. Candea, S. Kawamoto, Y. Fujiki, G. Friedman, A. Fox. OSDI '04, San Francisco, CA, December 2004. *Великолепная статья, показывающая, сколь многого можно достичь с помощью перезагрузки при конструировании надежных систем.*

[I11] «Intel 64 and IA-32 Architectures Software Developer's Manual» by Volume 3A and 3B: System Programming Guide. Intel Corporation, January 2011. *Просто скучное руководство, но иногда и они полезны.*

[K+61] «One-Level Storage System» by T. Kilburn, D. B. G. Edwards, M. J. Lanigan, F. H. Sumner. IRE Transactions on Electronic Computers, April 1962. *В системе Atlas впервые было реализовано многое из того, что мы встречаем в современных системах. Однако эта статья – не самое лучшее чтение. Если у вас есть время для чтения только одной работы, то лучше обратиться к историческому обзору [L78].*

[L78] «The Manchester Mark I and Atlas: A Historical Perspective» by S. H. Lavington. Communications of the ACM, Volume 21:1, January 1978. *Изящно изложенная история ранних разработок компьютерных систем и первопроходческой роли Atlas. Конечно, можно было бы прочитать оригинальные статьи по Atlas, но эта работа содержит отличный обзор и открывает определенную историческую перспективу.*

[M+63] «A Time-Sharing Debugging System for a Small Computer» by J. McCarthy, S. Boilen, E. Fredkin, J. C. R. Licklider. AFIPS '63 (Spring), May, 1963, New York, USA. *Ранняя работа, посвященная разделению времени, в которой упоминается прерывание от таймера. Вот соответствующая цитата: «Основная задача подпрограммы таймера на канале 17 – решить, следует ли удалять текущего пользователя из ядра и какую пользовательскую программу загрузить на его место».*

[MS96] «lmbench: Portable tools for performance analysis» by Larry McVoy and Carl Staelin. USENIX Annual Technical Conference, January 1996. *Любопытная статья о том, как измерить различные характеристики ОС. Скачайте lmbench и попробуйте ее запустить.*

[M11] «Mac OS 9» by Apple Computer, Inc. January 2011. http://en.wikipedia.org/wiki/Mac_OS_9. Возможно, вам даже удастся найти эмулятор OS 9, если по-стараетесь. Попробуйте, будет забавный маленький Mac!

[O90] «Why Aren't Operating Systems Getting Faster as Fast as Hardware?» by J. Ousterhout. USENIX Summer Conference, June 1990. Классическая работа о природе производительности операционных систем.

[P10] «The Single UNIX Specification, Version 3» by The Open Group, May 2010. Доступно на <http://www.unix.org/version3/>. Трудное чтение, поэтому лучше держаться от него подальше. Если, конечно, кто-то не платит за то, чтобы вы прочитали этот труд. Или если вы настолько любопытны, что не можете удержаться!

[S07] «The Geometry of Innocent Flesh on the Bone: Return-into-libc without Function Calls (on the x86)» by Hovav Shacham. CCS '07, October 2007. Одна из фантастических мозгодробительных идей, которые время от времени встречаешь в научных работах. Автор показывает, что если можно переходить в произвольную точку кода, то рано или поздно удастся сшить любую желаемую последовательность команд (если кодовая база достаточно велика); если хотите узнать детали, прочитайте. Увы, эта техника еще сильнее усложняет защиту от вредоносных атак.

Домашнее задание (измерение)

ОТСТУПЛЕНИЕ: ДОМАШНИЕ ЗАДАНИЯ НА ИЗМЕРЕНИЕ

В таких домашних заданиях от вас требуется написать код, который будет работать на реальной машине и измерять какой-то аспект производительности ОС или оборудования. Идея в том, чтобы получить опыт практической работы с настоящей операционной системой.

В этом домашнем задании вам предстоит измерить стоимость системного вызова и контекстного переключения. Измерить стоимость системного вызова сравнительно просто. Например, можно многократно выполнить какой-нибудь простой системный вызов (например, чтение 0 байт) и замерить общее время, а затем разделить время на количество итераций.

Необходимо принимать во внимание точность и разрешающую способность таймера. Типовой таймер дает функция `gettimeofday()`, подробности см. на странице руководства. Эта функция возвращает время в микросекундах, прошедшее с полуночи 1 января 1970 года, однако это не значит, что разрешающая способность действительно равна одной микросекунде. Несколько измерений с помощью `gettimeofday()`, выполненных подряд, позволят получить представление об истинной разрешающей способности таймера; это даст информацию о том, сколько итераций предложенного теста системного вызова с чтением 0 байт необходимо для получения приемлемого результата измерения. Если `gettimeofday()` для вас недостаточно точна, то можете поинтересоваться командой `rdtsc`, имеющейся в машинах с процессором x86.

Измерить стоимость контекстного переключения труднее. Lmbench для этого запускает на одном CPU два процесса и организует между ними два Unix-канала; канал – это один из многих способов межпроцессного взаимодействия в Unix. Затем первый процесс производит запись в первый канал и ждет возможности чтения из второго. Увидев, что первый процесс ждет, ОС переводит его в состояние «заблокирован» и переключается на другой процесс, который читает из первого канала, а затем пишет во второй. Когда второй процесс снова попытается прочитать из первого канала, он будет заблокирован. Такое попеременное взаимодействие повторяется в цикле. Многократно измеряя стоимость подобного взаимодействия, Lmbench получает довольно точную оценку стоимости контекстного переключения. Вы можете попытаться воспроизвести нечто подобное, воспользовавшись каналами или каким-то другим механизмом коммуникации, например Unix-сокетами.

Измерение контекстного переключения сталкивается с трудностями, когда в системе несколько процессоров; в этом случае нужно сделать так, чтобы переключаемые процессы исполнялись одним и тем же процессором. К счастью, в большинстве ОС имеются системные вызовы для привязки процесса к конкретному процессору; например, в Linux это вызов `sched_setaffinity()`. Если скоро оба процесса выполняются одним процессором, вы гарантированно будете измерять, сколько времени требуется ОС, чтобы остановить один процесс и возобновить другой на том же CPU.

Глава 7

Планирование: введение

К настоящему моменту низкоуровневые **механизмы** выполнения процессов (например, контекстное переключение) должны быть уже ясны; если нет, вернитесь на одну-две главы и перечитайте их описание. Но пока мы еще не разобрались в высокоуровневых политиках, применяемых планировщиком ОС. Этим мы сейчас и займемся и представим серию **политик планирования** (иногда их называют **дисциплинами**), придуманных умными и трудолюбивыми разработчиками на протяжении ряда лет.

Планирование уходит корнями во времена, предшествующие вычислительным системам; поначалу оно развивалось в рамках управления операциями и только потом было применено к компьютерам. Это и не удивительно, ведь для работы сборочных линий и во многих других видах человеческой деятельности планирование тоже необходимо, и там возникают те же проблемы, в т. ч. неумное стремление к эффективности. И вот в чем состоит наша проблема.

СУЩЕСТВО ПРОБЛЕМЫ: КАК РАЗРАБОТАТЬ ПОЛИТИКУ ПЛАНИРОВАНИЯ

Как должна выглядеть система, в рамках которой можно рассуждать о политиках планирования? Каковы основные предположения? Какие метрики важны? Какие подходы использовались в самых ранних компьютерных системах?

7.1. ПРЕДПОЛОЖЕНИЯ О РАБОЧЕЙ НАГРУЗКЕ

Прежде чем перечислять возможные политики, сделаем ряд упрощающих предположений о работающих в системе процессах, совокупность которых иногда называют **рабочей нагрузкой**. Определение рабочей нагрузки – важнейшая часть построения политик; чем больше мы знаем о рабочей нагрузке, тем точнее можно настроить политику.

Наши предположения по большей части нереалистичны, но это не страшно (пока), поскольку позже мы их ослабим и в конечном итоге придем к тому,

что называется ... (*драматическая пауза*) ... **полнофункциональной дисциплиной планирования**¹.

Мы примем следующие предположения о процессах (или **заданиях**), работающих в системе.

1. Время выполнения всех заданий одинаково.
2. Все задания поступают в одно и то же время.
3. Запущенное задание работает до естественного завершения.
4. Все задания используют только процессор (не выполняют ввода-вывода).
5. Время работы каждого задания известно.

Мы уже сказали, что многие из этих предположений нереалистичны, но подобно тому, как некоторые животные равнее других в «Скотном дворе» Оруэлла [O45], так и некоторые предположения нереалистичнее других. В частности, очень подозрительно выглядит предположение о том, что время работы каждого задания известно; это означало бы, что планировщик всеведущ, что, наверное, было бы замечательно, но вряд ли случится в обозримом будущем.

7.2. МЕТРИКИ ПЛАНИРОВАНИЯ

Помимо предположений о рабочей нагрузке, нам для сравнения политик планирования понадобится еще одна вещь: **метрика планирования**. Метрика – это нечто такое, что используется для *измерения* чего-либо; для оценки качества планирования применяются разные метрики.

Но пока упростим себе жизнь, ограничившись единственной метрикой: **оборотным временем**, которое определено как время завершения задания минус время поступления его в систему. Формально, обратное время $T_{\text{turnaround}}$ равно:

$$T_{\text{turnaround}} = T_{\text{completion}} - T_{\text{arrival}}. \quad (7.1)$$

Поскольку мы предположили, что все задания поступают в одно и то же время, имеем $T_{\text{arrival}} = 0$ и, значит, $T_{\text{turnaround}} = T_{\text{completion}}$. Это предположение впоследствии будет изменено.

Отметим, что обратное время – метрика **производительности** и потому больше всего интересна нам в этой главе. Также представляет интерес еще одна метрика – **справедливость**, измеряемая, например, с помощью **индекса справедливости Джейна** [J91]. Производительность и справедливость часто конфликтуют; например, планировщик может оптимизировать производительность, лишая некоторые задания возможности выполниться и тем самым снижая справедливость. Этот парадокс лишний раз показывает, что жизнь не всегда совершенна.

¹ Произносится с такой же интонацией, как «полнофункциональная Звезда Смерти» (см. https://ru.wikipedia.org/wiki/Звезда_Смерти).

7.3. ПЕРВЫМ ПРИШЕЛ, ПЕРВЫМ УШЕЛ (FIFO)

Самый простой алгоритм планирования известен под названием «**первым пришел, первым ушел**» (First In, First Out – **FIFO**), или «**первым пришел, первым обслужен**» (First Come, First Served – **FCFS**). У FIFO имеется ряд полезных свойств: он прост и легко реализуется. И в наших предположениях он хорошо работает.

Разберем простой пример. Допустим, что три задания А, В и С поступают в систему примерно в одно время ($T_{arrival} = 0$). Поскольку алгоритм FIFO должен поставить первым какое-то задание, будем предполагать, что А поступило чуть-чуть раньше В, а В чуть-чуть раньше С. Предположим также, что каждое задание работает 10 секунд. Каково **среднее оборотное время** этих заданий?

На рис. 7.1 видно, что А завершилось в момент 10, В в момент 20, а С в момент 30. Следовательно, среднее оборотное время равно $(10 + 20 + 30)/3 = 20$. Ничего сложного.

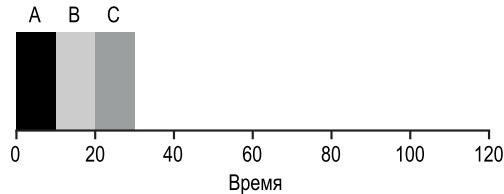


Рис. 7.1 ❖ Простой пример FIFO

Теперь ослабим одно из наших предположений, а именно предположение 1 – больше не будем предполагать, что все задания работают одинаковое время. Как тогда работает FIFO? При какой рабочей нагрузке FIFO будет показывать плохую производительность?

(Думайте, прежде чем читать дальше... думайте... придумали?!)

Наверное, вы уже сами догадались, но на всякий случай приведем пример, показывающий, что задания разной продолжительности могут сделать FIFO-планирование малопригодным. Снова рассмотрим три задания (А, В, С), но теперь пусть А работает 100 секунд, а В и С – по 10.

Как видно на рис. 7.2, сначала задание А отрабатывает все свои 100 секунд, и только потом задания В и С получают шанс поработать. Поэтому среднее оборотное время велико: чудовищные 110 секунд $((100 + 110 + 120)/3 = 110)$.

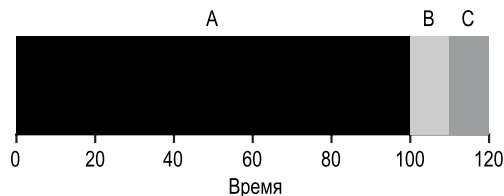


Рис. 7.2 ❖ Почему алгоритм FIFO не так уж хорош

Эту проблему часто называют **эффектом сопровождения** (convoy effect) [B+79]: когда несколько быстрых потенциальных потребителей ресурса выстраиваются в очередь за медленным потребителем. Данный вариант планирования похож на очередь к кассе в бакалейной лавке – вспомните, какие чувства вы испытываете, когда перед вами стоит человек с тремя полными тележками, а в кассовом аппарате кончилась лента – придется задержаться¹.

СОВЕТ: ПРИНЦИП «СНАЧАЛА САМОЕ КОРОТКОЕ»

«Сначала самое короткое» – общий принцип планирования, который можно применять в любой системе, где имеет значение воспринимаемое обратное время в расчете на одного клиента (в нашем случае – задания). Вспомните любую очередь, в которой вам доводилось стоять: если заведение думает о том, чтобы клиент остался доволен, то, вероятно, как-то взяло на вооружение этот принцип. Например, в супермаркетах часто имеется касса для клиентов, купивших «не больше десяти предметов», чтобы таким покупателям не приходилось стоять в очереди за семейством, готовящимся к грядущей ядерной зиме.

Так что же нам делать? Как разработать улучшенный алгоритм, который принимал бы во внимание тот факт, что задания работают разное время? Сначала подумайте, потом читайте дальше.

7.4. СНАЧАЛА САМОЕ КОРОТКОЕ

Для решения этой проблемы можно применить совсем простую идею, заимствованную у исследования операций [C54, PV56]. Новая дисциплина планирования называется **«сначала самое короткое»** (Shortest Job First – SJF), это название легко запомнить, потому что оно исчерпывающим образом описывает суть политики: сначала выполнить самое короткое задание, затем следующее по продолжительности и т. д.

Применим к рассмотренному выше примеру политику планирования SJF. На рис. 7.3 показаны результаты выполнения заданий А, В и С. Надеемся, что из рисунка понятно, почему SJF работает гораздо лучше с точки зрения среднего обратного времени. Если В и С пустить раньше А, то среднее обратное время уменьшается со 110 до 50 секунд ($(10 + 20 + 120)/3 = 50$) – в два с лишним раза.

На самом деле, в предположении, что все задания поступают в одно время, можно доказать, что SJF – **оптимальный** алгоритм планирования. Однако это курс по операционным системам, а не по исследованию операций, поэтому доказательства не приводятся.

¹ Рекомендуемые действия: быстро перейти в другую очередь или глубоко вдохнуть и расслабиться. Вдох-выдох, вдох-выдох... Все будет хорошо, нет причин волноваться.

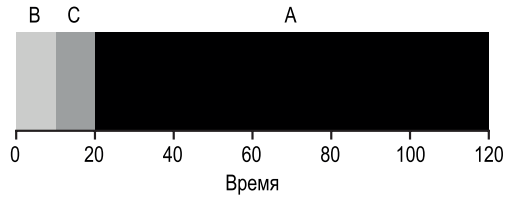


Рис. 7.3 ❖ Простой пример SJF

ОТСТУПЛЕНИЕ: ВЫТЕСНЯЮЩИЕ ПЛАНИРОВЩИКИ

Во времена пакетных вычислений было разработано несколько **невывесняющих** планировщиков; в таких системах каждое задание дорабатывало до конца, и только потом ставился вопрос о том, какое задание запустить следующим. Практически все современные планировщики **вытесняющие**, т. е. готовы временно приостановить один процесс и запустить другой. Отсюда следует, что планировщик использует уже рассмотренные выше механизмы, в частности умеет **переключать контекст**.

Таким образом, мы придумали хороший подход к планированию методом SJF, но наши предположения по-прежнему малореалистичны. Пойдем дальше по пути их ослабления. Теперь нашей мишенью станет предположение 2 – будем считать, что задания могут поступать в любое время, необязательно одновременно. Какие при этом могут возникнуть проблемы?

(Еще одна пауза на размышления... вы думаете? Давайте, вы сможете.)

Снова проиллюстрируем на примере. На этот раз предположим, что задание А поступает в момент $t = 0$ и работает 100 секунд, а задания В и С поступают в момент $t = 10$ и работают по 10 секунд. Если использовать SJF в чистом виде, то получится картинка, показанная на рис. 7.4.

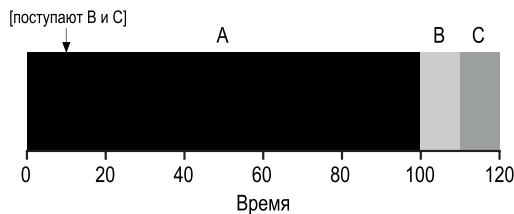


Рис. 7.4 ❖ SJF, когда В и С поступают с задержкой

Как видим, хотя В и С поступили вскоре после А, они все равно вынуждены ждать завершения А и, стало быть, являются жертвами все той же проблемы сопровождения. Среднее обратное время равно 103.33 с $((100 + (110 - 10) + (120 - 10))/3)$. Что может сделать планировщик?

7.5. СНАЧАЛА С НАИМЕНЬШИМ ВРЕМЕНЕМ ДО ЗАВЕРШЕНИЯ

Чтобы решить эту проблему, необходимо ослабить предположение 3 (что все задания работают до естественного завершения). Кроме того, нам понадобятся некоторые механизмы внутри самого планировщика. Памятуя обсуждение прерываний от таймера и контекстного переключения, вы, наверное, догадались, что может сделать планировщик в момент поступления заданий В и С: вытеснить задание А и запустить другое задание, а А возобновить позже. Согласно нашему определению, SJF – **невытесняющий** планировщик, поэтому и испытывает описанные выше проблемы.

По счастью, существует планировщик, который делает именно то, что необходимо: добавляет вытеснение в SJF. Он называется «**сначала с наименьшим временем до завершения**» (Shortest Time-to-Completion First – STCF) или «**сначала самое короткое с вытеснением**» (Preemptive Shortest Job First – PSJF) [CK68]. Всякий раз, как в систему поступает новое задание, планировщик STCF решает, какому из имеющихся заданий (включая и только что поступившее) осталось меньше времени до завершения, и планирует его. Таким образом, в нашем примере STCF вытеснил бы А и дал возможность В и С проработать до завершения – и лишь после этого запланировал бы продолжение А. Эта ситуация показана на рис. 7.5.

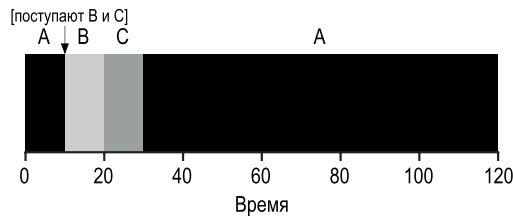


Рис. 7.5 ❖ Простой пример STCF

Среднее оборотное время значительно уменьшилось и стало равно 50 секунд ($((120 - 0) + (20 - 10) + (30 - 10))/3$). Как и раньше, можно доказать, что в наших новых предположениях STCF оптимален; учитывая, что SJF оптимален, когда все задания поступают одновременно, вы, наверное, интуитивно понимаете, почему оптимален STCF.

7.6. НОВАЯ МЕТРИКА: ВРЕМЯ ОТКЛИКА

Итак, если бы продолжительность заданий была известна, и если бы задания использовали только CPU, и если бы единственной метрикой было оборотное время, то политика STCF была бы замечательна. И действительно, во многих ранних пакетных вычислительных системах такие алгоритмы планиро-

вания имели смысл. Однако появление машин с разделением времени все изменило. Теперь пользователи работали с машиной интерактивно, сидя за терминалом, и требовали от системы соответствующей производительности. Поэтому появилась новая метрика: **время отклика**.

Мы определим время отклика как время от момента поступления задачи в систему до момента ее первого планирования¹. Формально:

$$T_{response} = T_{firstrun} - T_{arrival} \quad (7.2)$$

Например, в примере выше (А поступает в момент 0, а В и С в момент 10) времена отклика были бы такими: 0 для задания А, 0 для В и 10 для С (среднее равно 3.33).

Как вы понимаете, STCF и родственные политики не особенно хороши для минимизации времени отклика. Если три задания поступают в одно время, то, например, третье задание должно будет ждать, пока два других завершатся *полностью*, и лишь потом может рассчитывать на получение процессора. С точки зрения оборотного времени, это отличный подход, но в интерактивном окружении, когда требуется минимизировать время отклика, он никуда не годится. Действительно, представьте, что вы сидите за терминалом, ввели какую-то команду и должны 10 секунд ждать ответа от системы только потому, что какое-то другое задание было запланировано раньше, – приятного мало.

Таким образом, мы получили другую проблему: как построить планировщик, оптимизирующий время отклика?

7.7. ЦИКЛИЧЕСКОЕ ПЛАНИРОВАНИЕ

Для решения этой проблемы мы введем новый алгоритм планирования, который принято называть **циклическим** (Round-Robin – **RR**) [K64]. Идея проста: вместо того чтобы давать заданиям работать до конца, RR позволяет заданию владеть процессором в течение **временного кванта** (иногда его называют **квантом планирования**), а затем переключается на следующее задание в очереди на выполнение. Так происходит, пока задания не завершатся. По этой причине RR иногда называют планированием **с квантованием времени**. Заметим, что длительность временного кванта должна быть кратна разрешающей способности таймера: если таймер генерирует прерывания раз в 10 миллисекунд, то квант может быть равен 10, 20 и вообще любому кратному 10 мс.

Чтобы лучше разобраться в RR-планировании, рассмотрим пример. Пусть три задания А, В и С поступают в систему одновременно и каждое хочет поработать 5 секунд. SJF-планировщик выполняет каждое задание до конца,

¹ Иногда эта метрика определяется немного по-другому, например включает время до момента выдачи заданием какого-то «отклика». Наше определение является вариантом этого в лучшем случае, поскольку предполагается, что задание откликается мгновенно.

а затем планирует следующее (рис. 7.6). Напротив, RR-планировщик с временным квантом 1 секунда быстро перебирает задания в цикле (рис. 7.7).

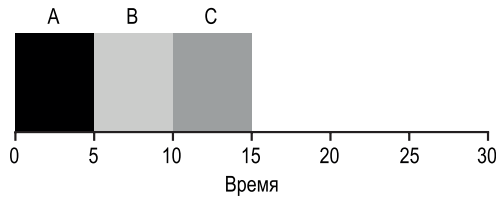


Рис. 7.6 ❖ Снова SJF
(не годится, когда метрикой является время отклика)

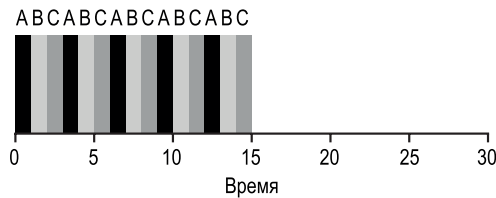


Рис. 7.7 ❖ Циклическое планирование
(годится для времени отклика)

Среднее время отклика для RR-планировщика составляет $(0 + 1 + 2)/3 = 1$, а для SJF-планировщика оно было бы равно $(0 + 5 + 10)/3 = 5$.

Как видим, величина временного кванта – важнейший параметр RR. Чем она меньше, тем лучше производительность RR-планировщика с точки зрения времени отклика. Но и слишком короткий временной квант – плохо: очень большая доля времени будет уходить на контекстное переключение. Решение о том, какая длительность временного кванта оптимальна, принимает проектировщик системы. Она должна быть достаточно большой, чтобы **амортизировать** стоимость переключения, но достаточно малой, чтобы система откликалась быстро.

СОВЕТ: АМОРТИЗАЦИЯ МОЖЕТ УМЕНЬШИТЬ СТОИМОСТЬ

Общая техника **амортизации** часто применяется в системах, где стоимость некоторой операции фиксирована. Неся этот расход реже (т. е. реже выполняя операцию), мы можем сократить общие системные затраты. Например, если квант времени равен 10 мс, а контекстное переключение занимает 1 мс, то примерно 10 % времени уходят на контекстное переключение и, следовательно, расходуются впустую. Если мы хотим *амортизировать* эту стоимость, то можем увеличить временной квант, например до 100 мс. В этом случае на контекстное переключение будет уходить менее 1 % времени, т. е. затраты на квантование амортизируются.

Отметим, что стоимость контекстного переключения не сводится к одному лишь времени сохранения и восстановления нескольких регистров.

Состояние работающей программы присутствует в процессорных кешах, буферах ассоциативной трансляции (TLB), предсказателях ветвлений и другом оборудовании, размещенном на кристалле. При переключении на другое задание все это состояние сбрасывается, и загружается состояние, относящееся к новому заданию, а стоимость этих операций может быть довольно высока [MB91].

Таким образом, RR-планировщик с разумно выбранным временным квантом прекрасно справляется с задачей, если время отклика – единственная метрика. Но как быть с нашим старым знакомым – оборотным временем? Вернемся к нашему примеру. Задания А, В, С, каждое по 5 секунд, поступают в одно время, а планирование производится согласно алгоритму RR с длинным 1-секундным интервалом. Из рисунка выше видно, что А завершается в момент 13, В в момент 14, С в момент 15, так что среднее оборотное время равно 14. Просто ужас!

Неудивительно, что RR является одной из *худших* политик, если в качестве метрики выбрано оборотное время. Интуитивно это понятно: ведь RR растягивает каждое задание на максимально возможное время, поскольку дает ему чуть-чуть поработать, а потом передает процессор следующему заданию. Поскольку с точки зрения оборотного времени важно только то, когда задание завершается, RR – едва ли не самый неудачный алгоритм, во многих случаях даже хуже простого FIFO.

Вообще, любая политика (типа RR), которая **справедлива**, т. е. поровну распределяет процессорное время между активными процессами в локальном временном масштабе, непригодна для таких метрик, как оборотное время. Это принципиальное противоречие: если мы готовы к несправедливости, то можем давать коротким заданиям доработать до конца, но будем расплачиваться за это временем отклика; если же мы ценим справедливость, то нельзя рассчитывать на хорошее оборотное время. Такого рода **компромиссы** часто встречаются в системах: нельзя и рыбку съесть, и в воду не лезть.

Мы разработали два типа планировщиков. Одни (SJF, STCF) оптимизируют оборотное время, но плохо ведут себя с точки зрения времени отклика. Другие (RR) оптимизируют время отклика, но теряют в оборотном времени. И у нас по-прежнему осталось два неослабленных предположения: номер 4 (что задания не выполняют ввода-вывода) и номер 5 (что время работы каждого задания известно). Далее мы займемся ими.

СОВЕТ: ПЕРЕКРЫТИЕ ОТКРЫВАЕТ ВОЗМОЖНОСТЬ ДЛЯ БОЛЕЕ ВЫСОКОЙ СТЕПЕНИ ИСПОЛЬЗОВАНИЯ

При любой возможности **перекрывайте** операции во времени, чтобы добиться максимального использования ресурсов системы. Перекрывание полезно в разных ситуациях, в т. ч. при выполнении дискового ввода-вывода и отправке сообщений удаленным машинам; в обоих случаях имеет смысл начать операцию, а затем переключиться на другую работу, тогда коэффициент использования будет выше и общая эффективность системы увеличится.

7.8. УЧЕТ ВВОДА-ВЫВОДА

Сначала ослабим предположение 4 – конечно же, все программы выполняют ввод-вывод. Если бы программа вообще не выполняла ввода, то при каждом запуске давала бы один и тот же результат. А программа, которая ничего не выводит, заставляет вспомнить о философской загадке: слышен ли звук падающего дерева в лесу, если рядом никого нет, – никто не узнает, работала она или нет.

Очевидно, что планировщик должен принять какое-то решение, когда текущее задание инициирует запрос ввода-вывода, поскольку во время ожидания завершения оно не будет использовать CPU – оно **заблокировано**. Если речь идет о дисковом вводе-выводе, то блокировка может продолжаться несколько миллисекунд или даже дольше в зависимости от текущей загруженности диска. Поэтому планировщик, вероятно, должен на это время отдать процессор другому заданию.

Также планировщик должен решить, что делать, когда ввод-вывод завершится. В этот момент генерируется прерывание, ОС получает управление и переводит процесс, инициировавший ввод-вывод из состояния «заблокирован» в состояние «готов». Конечно, она могла бы в этот момент заодно запустить это задание. Как ОС должна обращаться с заданиями?

Чтобы лучше разобраться в этом вопросе, предположим, что имеется два задания, А и В, каждому из которых нужно 50 мс процессорного времени. Однако между ними есть существенное различие: А работает 10 мс, затем инициирует запрос ввода-вывода (допустим, что каждый такой запрос занимает 10 мс), тогда как В просто использует CPU в течение 50 мс, не выполняя никакого ввода-вывода. Планировщик сначала запускает А, потом В (рис. 7.8).

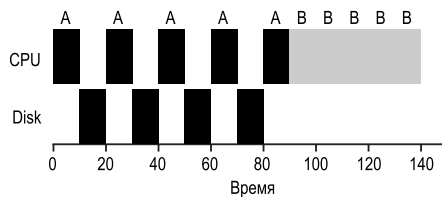


Рис. 7.8 ❖ Неэффективное использование ресурсов

Предположим, что мы пытаемся разработать STCF-планировщик. Как он мог бы поступить с учетом того факта, что А разбита на 5 подзаданий по 10 мс каждое, тогда как В процессор нужен на 50 мс непрерывно? Очевидно, что просто запустить одно задание, а затем другое, вообще не принимая во внимание ввод-вывод, не слишком осмысленно.

Типичный подход – рассматривать каждое 10-мс подзадание А как независимое задание. Тогда системе нужно решить, запланировать ли 10-мс задание А или 50-мс задание В. В случае алгоритма STCF выбор ясен: выбрать более короткое задание, т. е. А. Затем, когда первое подзадание А завершится,

останется только В, оно и начнет выполняться. После этого поступает новое подзадание А, вытесняет В и работает в течение 10 мс. Такой подход открывает возможность **перекрытия**, когда CPU используется одним процессом, пока другой ждет завершения ввода-вывода; при этом вся система используется более эффективно (см. рис. 7.9).

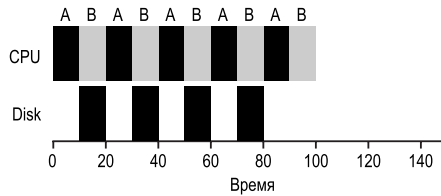


Рис. 7.9 ❖ Перекрытие позволяет лучше использовать ресурсы

Таким образом, мы видим, как планировщик мог бы учесть ввод-вывод. Рассматривая каждый отрезок непрерывного использования CPU как задание, планировщик гарантирует, что «интерактивные» процессы получают процессор чаще. А когда эти интерактивные задания выполняют ввод-вывод, работают другие занимающие CPU задания, и общий коэффициент использования процессора повышается.

7.9. Долой ОРАКУЛОВ

Приняв базовый подход к обработке ввода-вывода, мы можем покончить с нашим последним предположением: что планировщик заранее знает продолжительность каждого задания. Как уже было сказано, это, пожалуй, худшее из всех сделанных предположений. На самом деле ОС общего назначения (а мы только о таких и говорим) обычно очень мало знает о продолжительности отдельных заданий. Но как же без такого *априорного* знания реализовать алгоритмы типа SJF и STCF? Да и воплотить в жизнь некоторые идеи RR-планировщика, гарантирующего малое время отклика, тоже будет затруднительно.

7.10. РЕЗЮМЕ

Мы познакомились с основными идеями планирования и разработали два семейства планировщиков. Одни выполняют самое короткое из оставшихся заданий и таким образом оптимизируют оборотное время, другие поочередно перебирают все задания, оптимизируя время отклика. Увы, каждое семейство хорошо проявляет себя именно там, где другое терпит неудачу, – таков неизбежный компромисс при проектировании систем. Мы также видели, как включить в общую картину ввод-вывод, но пока не решили проблему

принципиальной невозможности для ОС заглянуть в будущее. Вскоре мы покажем, как преодолеть эти трудности, построив планировщик, который использует знания о недавнем прошлом для предсказания будущего. Соответствующий алгоритм, называемый **многоуровневой аналитической очередью** (multi-level feedback queue), является темой следующей главы.

Литература

[B+79] «The Convoy Phenomenon» by M. Blasgen, J. Gray, M. Mitoma, T. Price. ACM Operating Systems Review, 13:2, April 1979. *Наверное, первое упоминание о феномене сопровождения, который встречается не только в ОС, но и в базах данных.*

[C54] «Priority Assignment in Waiting Line Problems» by A. Cobham. Journal of Operations Research, 2:70, pages 70–76, 1954. *Основополагающая работа по использованию алгоритма SJF при планировании ремонта машин.*

[K64] «Analysis of a Time-Shared Processor» by Leonard Kleinrock. Naval Research Logistics Quarterly, 11:1, pages 59–73, March 1964. *Наверное, первое упоминание об алгоритме циклического планирования; безусловно, один из первых анализов применения этого подхода к планированию в системах с разделением времени.*

[CK68] «Computer Scheduling Methods and their Countermeasures» by Edward G. Coffman and Leonard Kleinrock. AFIPS '68 (Spring), April 1968. *Великолепное раннее введение в различные базовые дисциплины планирования и их анализ.*

[J91] «The Art of Computer Systems Performance Analysis: Techniques for Experimental Design, Measurement, Simulation, and Modeling» by R. Jain. Interscience, New York, April 1991. *Стандартный учебник по измерению компьютерных систем. Без сомнения, отличное справочное руководство, которое полезно иметь в своей библиотеке.*

[O45] «Animal Farm» by George Orwell. Secker and Warburg (London)¹, 1945. *Великая, но удручающая аллегория на тему власти и злоупотребления ей. Некоторые говорят, что это сатира на Сталина и сталинский СССР в преддверии Второй мировой войны, мы же скажем, что это сатира на свиней.*

[PV56] «Machine Repair as a Priority Waiting-Line Problem» by Thomas E. Phipps Jr., W. R. Van Voorhis. Operations Research, 4:1, pages 76–86, February 1956. *В этой работе обобщается подход к ремонту машин на основе алгоритма SJF, описанный в оригинальной работе Кобхэма; также постулируется полезность подхода STCF в таком окружении. Точнее, «существуют такие виды ремонтных работ, ... сопровождающиеся масштабной разборкой, когда весь пол усеян винтами и гайками, которые, безусловно, не следует прерывать до самого завершения; но есть и другие случаи, когда неразумно продолжать работу над длинным заданием, если появилось одно или несколько более коротких» (стр. 81).*

¹ Дж. Оруэлл. Скотный двор. Эксмо, 2020.

[MB91] «The effect of context switches on cache performance» by Jeffrey C. Mogul, Anita Borg. ASPLOS, 1991. *Эlegantное исследование влияния контекстного переключения на производительность кеша; не столь актуально в современных системах, выполняющих миллиарды операций в секунду, но все равно контекстное переключение занимает время порядка нескольких миллисекунд.*

[W15] «You can't have your cake and eat it» by Authors: Unknown. Wikipedia (as of December 2015). http://en.wikipedia.org/wiki/You_can't_have_your_cake_and_eat_it. *Самое интересное на этой странице – аналогичные идиомы на других языках. Например, в тамильском это звучит так: «нельзя и суп съесть, и усы не замочить».*

Домашнее задание (эмуляция)

Программа `scheduler.py` поможет понять, как работают разные планировщики с точки зрения таких метрик, как время отклика, обратное время и полное время ожидания. Детали см. в файле README.

Вопросы

1. Вычислить время отклика и обратное время при выполнении трех заданий продолжительностью 200 с применением планировщиков SJF и FIFO.
2. То же самое, но для заданий разной продолжительности: 100, 200 и 300.
3. То же самое, но с применением RR-планировщика с временным квантом 1.
4. Для рабочих нагрузок какого типа алгоритм SJF дает такое же обратное время, как FIFO?
5. Для рабочих нагрузок какого типа и какой длины кванта алгоритм SJF дает такое же время отклика, как RR?
6. Что будет со временем отклика при планировании методом SJF, когда продолжительность заданий увеличивается? Можете ли вы с помощью эмулятора продемонстрировать эту тенденцию?
7. Что будет со временем отклика при планировании методом RR, когда величина временного кванта увеличивается? Попробуйте вывести формулу, дающую время отклика в худшем случае при наличии N заданий.

Глава 8

Планирование: многоуровневая аналитическая очередь

В этой главе мы займемся разработкой одного из самых хорошо известных подходов к планированию – **многоуровневой аналитической очереди** (Multi-level Feedback Queue – MLFQ). MLFQ-планировщик впервые был описан в статье Corbato et al. 1962 года [C+62] для системы под названием Compatible Time-Sharing System (CTSS). Эта работа наряду с более поздней работой над системой Multics побудила Ассоциацию по вычислительной технике (АСМ) присудить Корбато свою высшую награду – **премию Тьюринга**. С годами планировщик совершенствовался, и его нынешние реализации используются в некоторых современных системах.

Фундаментальная проблема, которую пытается решить MLFQ, имеет две стороны. Во-первых, желательно оптимизировать *оборотное время*, что, как мы видели в предыдущей главе, делается путем исполнения в первую очередь более коротких заданий; но, к сожалению, ОС в общем случае не знает, сколько времени будет работать задание, а без этого алгоритмы типа SJF (или STCF) не работают. Во-вторых, хотелось бы, чтобы система быстро отвечала интерактивным пользователям (тем, кто сидит за экраном терминала и ждет, когда процесс закончится), а значит, требуется минимизировать *время отклика*; однако алгоритм циклического планирования и ему подобные хотя и снижают время отклика, но никуда не годны с точки зрения оборотного времени. Итак, проблема ставится следующим образом: при условии что в общем случае о процессе ничего не известно, как построить планировщик, отвечающий этим целям? Как планировщик может во время работы системы узнать характеристики выполняемых заданий и на основе этого знания принимать более качественные решения?

СУЩЕСТВО ПРОБЛЕМЫ: КАК ПЛАНИРОВАТЬ, НЕ ИМЕЯ ТОЧНЫХ ЗНАНИЙ?

Как спроектировать планировщик, который минимизирует время отклика для интерактивных заданий и одновременно минимизирует обратное время, не имея априорных знаний о продолжительности заданий?

СОВЕТ: ОБУЧАТЬСЯ НА ИСТОРИЧЕСКИХ ДАННЫХ

Многоуровневая аналитическая очередь – прекрасный пример системы, которая учится предсказывать будущее, опираясь на прошлые данные. Такие подходы часто встречаются в операционных системах (и в других областях информатики, в т. ч. аппаратных предсказателях ветвлений и алгоритмах кеширования). Они работают, когда в поведении заданий можно выделить определенные этапы, благодаря чему они предсказуемы. Но, конечно, следует проявлять осторожность, потому что предположение может оказаться ошибочным, в результате чего решение, принятое системой, может оказаться хуже, чем если бы она вообще не располагала никакой информацией.

8.1. MLFQ: ОСНОВНЫЕ ПРАВИЛА

В этой главе мы опишем основные алгоритмы, лежащие в основе планировщика с многоуровневой аналитической очередью; хотя у различных реализаций MLFQ есть специфические особенности [E95], по сути своей они в большинстве своем схожи.

В нашем изложении MLFQ состоит из нескольких **очереди**, каждой из которых назначен **уровень приоритета**. В каждый момент времени задание, готовое к выполнению, находится в какой-то одной очереди. Приоритеты используются, для того чтобы решить, какое задание выполнять следующим: выбирается задание с наибольшим приоритетом (находящееся в очереди, имеющий наибольший уровень приоритета).

Разумеется, в каждой очереди может находиться более одного задания, и все они будут иметь *одинаковый* приоритет. В таком случае к задачам, стоящим в очереди, применяется циклический алгоритм.

Итак, вот первые два правила MLFQ:

- **правило 1:** если Приоритет(A) > Приоритет(B), то выполнять A (а не B);
- **правило 2:** если Приоритет(A) = Приоритет(B), то применять к A и B алгоритм циклического планирования.

Поэтому ключ к MLFQ-планированию – задание приоритетов планировщиком. Вместо того чтобы присваивать приоритет заданию раз и навсегда, MLFQ *изменяет* приоритет, опираясь на *наблюдаемое поведение*. Например, если задание раз за разом уступает процессор в ожидании ввода с клавиатуры, то MLFQ не понижает его приоритет, считая, что так может себя вести интерактивный процесс. Если же, напротив, задание долго удерживает CPU, то его приоритет уменьшается. Таким образом, MLFQ пытается *обучаться*

во время выполнения процессов и использовать *историю* заданий для предсказания их будущего поведения.

На рис. 8.1 изображено, как могли бы выглядеть очереди в некоторый момент времени. Два задания (А и В) имеют наивысший приоритет, задание С – средний, а задание D – самый низкий. Из наших текущих знаний о работе MLFQ можно сделать вывод, что планировщик будет попеременно выполнять задания А и В, поскольку они самые приоритетные, а бедным заданиям С и D ничего не светит – какое бесчинство!

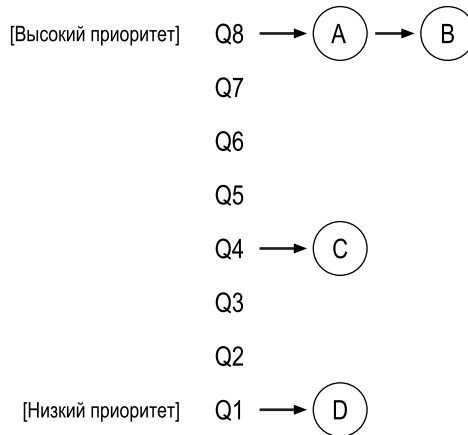


Рис. 8.1 ❖ Пример MLFQ

Разумеется, статическая картина некоторых очередей не дает полного представления о работе MLFQ. Нужно понимать, как приоритеты *изменяются* со временем. И именно этим мы сейчас и займемся – к вящему удивлению тех, кто начал читать книгу с этой главы.

8.2. Попытка 1: КАК ИЗМЕНЯТЬ ПРИОРИТЕТЫ

Теперь нужно решить, как MLFQ должен изменять приоритет задания (т. е. в какую очередь он будет ее помещать) на протяжении его жизни. Для этого следует иметь в виду нашу рабочую нагрузку: смесь интерактивных заданий, которые работают недолго (и могут часто уступать CPU), и долго работающих «счетных» заданий, которые потребляют много процессорного времени, но для которых время отклика не важно. Вот как выглядит первая попытка придумать алгоритм корректировки приоритетов.

- **Правило 3:** когда задание поступает в систему, оно помещается в очередь с высшим приоритетом (верхнюю).
- **Правило 4а:** если задание использовало весь свой временной квант целиком, то его приоритет *понижается* (оно перемещается на одну очередь вниз).

- **Правило 4b:** если задание уступает CPU до истечения своего временного кванта, его приоритет *не изменяется*.

Пример 1: одно долго работающее задание

Приведем несколько примеров. Сначала посмотрим, что будет, когда в системе имеется одно долго работающее задание. На рис. 8.2 показано, что происходит с этим заданием, когда планировщик поддерживает три очереди.

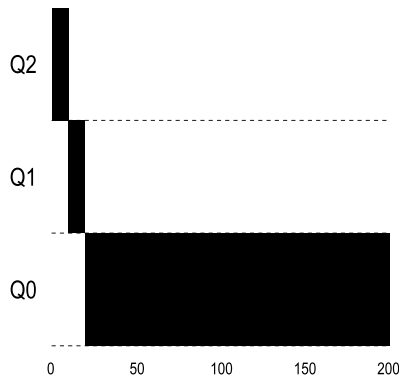


Рис. 8.2 ❖ Эволюция долго работающего задания

Как видно из этого примера, задание поступает в очередь с высшим приоритетом (Q2). После первого же временного кванта длительностью 10 мс планировщик понижает приоритет задания на единицу и, следовательно, перемещает его в очередь Q1. Задержавшись в Q1 на один временной квант, задание перемещается в очередь с низшим приоритетом (Q0), где и остается. Просто, правда?

Пример 2: к нам приходит короткое задание

Теперь рассмотрим пример посложнее в надежде увидеть, как MLFQ попытается аппроксимировать SJF. Теперь у нас два задания: А, долго работающее счетное задание, и В, короткое интерактивное задание. Предположим, что А проработало некоторое время, а потом в систему поступило В. Что произойдет? Сможет ли MLFQ аппроксимировать алгоритм SJF для В?

Этот сценарий изображен на рис. 8.3. Задание А (показано черным цветом) работает, находясь в очереди с низшим приоритетом (как и любое долго работающее счетное задание). Задание В (показано серым цветом) поступает в момент $T = 100$ и помещается в верхнюю очередь. Поскольку В короткое (всего 20 мс), оно завершается, не достигнув нижней очереди, за два временных кванта, а затем выполнение А возобновляется (с низким приоритетом).

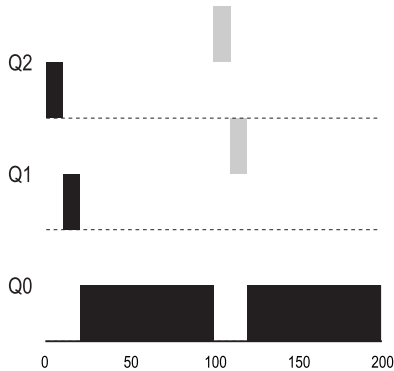


Рис. 8.3 ❖ Поступает интерактивное задание

Надеемся, что из этого примера вы уяснили одну из главных целей алгоритма: поскольку он не *знает*, будет ли задание коротким или долгим, то сначала *предполагает*, что задание короткое, и назначает ему высокий приоритет. Если задание действительно короткое, то оно быстро выполнится и завершится, а если нет, то оно будет медленно опускаться вниз и вскоре проявит свою истинную сущность – долго работающее задание, больше напоминающее пакетный процесс. Именно так MLFQ аппроксимирует SJF.

Пример 3: а как насчет ввода-вывода?

Теперь разберем пример с участием ввода-вывода. Правило 4b гласит, что если процесс уступает процессор, не исчерпав свой временной квант, то его приоритет не изменяется. Идея тут проста: если интерактивное задание часто выполняет ввод-вывод (например, ожидая от пользователя действий с клавиатурой или мышью), то оно будет уступать CPU, прежде чем его квант закончится; в таком случае мы не станем штрафовать задание, а просто оставим его на том же уровне.

На рис. 8.4 показано, как это работает на примере интерактивного задания В (серого цвета), которое нуждается в CPU всего 1 мс, а затем приступает к вводу-выводу, и долго работающего пакетного задания А (черного цвета), которое конкурирует за процессор. MLFQ сохраняет за В высший приоритет, поскольку В постоянно и добровольно освобождает CPU; если В – интерактивное задание, то MLFQ достигает своей цели – выполнять такие задания быстро.

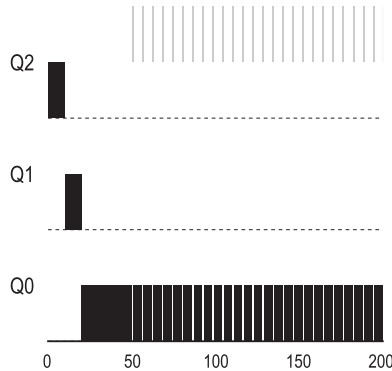


Рис. 8.4 ❖ Смешанная рабочая нагрузка: задания с вводом-выводом и счетные задания

Проблемы текущей реализации MLFQ

Итак, у нас есть базовый вариант MLFQ. Похоже, он неплохо справляется со своим делом – справедливо распределяет CPU между долго работающими заданиями и дает возможность коротким и выполняющим ввод-вывод интерактивным заданиям работать быстро. К сожалению, текущий алгоритм обладает серьезными изъянами. Можете ли вы назвать их?

(В этом месте сделайте паузу и думайте, думайте, думайте...)

Во-первых, имеется проблема **голодной смерти** (starvation): если в системе «слишком много» интерактивных заданий, то в совокупности они потребят *все* процессорное время, поэтому долго работающим заданиям не достанется *ничего* (они умрут от голода). Мы хотели бы, чтобы и в такой ситуации эти задания хоть как-то двигались.

Во-вторых, хитрый пользователь может переписать свою программу, так чтобы **переиграть планировщик**. В общем случае это означает действие, направленное на то, чтобы обманом заставить планировщик дать больше ресурса, чем положено. Описанный выше алгоритм уязвим для следующей атаки: перед тем как временной квант будет исчерпан, инициировать операцию ввода-вывода (в какой-нибудь файл, все равно, в какой) и тем самым освободить CPU; тогда задание останется в той же очереди и, следовательно, получит большую долю процессорного времени. Если все сделано правильно (например, перед уступкой CPU квант времени уже исчерпан на 99 %), то задание может практически монополизировать процессор.

Наконец, программа может со временем *изменять свое поведение*; программа, до этого занимавшаяся вычислениями, может перейти в фазу интерактивности. При теперешнем подходе такому заданию не повезет, и оно не будет рассматриваться наравне с другими интерактивными заданиями.

СОВЕТ: ПЛАНИРОВАНИЕ ДОЛЖНО БЫТЬ ЗАЩИЩЕНО ОТ АТАК

Возможно, вы думаете, что политика планирования, будь то внутри самой ОС (как обсуждалось выше) или в более широком контексте (например, при обработке запросов ввода-вывода в распределенной системе [Y+18]), не имеет отношения к **безопасности**. Но во многих случаях – и их число постоянно растет – это не так. Рассмотрим современный центр обработки данных (ЦОД), в котором пользователи со всего света совместно пользуются процессорами, памятью, сетями и системами хранения; без тщательного проектирования и соблюдения политики один пользователь мог бы своими злонамеренными действиями нанести ущерб остальным и получить преимущества для себя. Поэтому политика планирования – важная часть безопасности системы, и разрабатывать ее надо со всем тщанием.

8.3. Попытка 2: повышение приоритета

Попробуем изменить правила и посмотрим, удастся ли избежать проблемы голодной смерти. Что можно сделать, для того чтобы счетные задания гарантированно получили возможность продвигаться к завершению (хотя бы медленно)?

Идея проста: периодически повышать приоритет всех заданий в системе. Сделать это можно по-разному, но выберем простой способ – будем перебрасывать их в верхнюю очередь. Итак, вот новое правило:

- **правило 5:** по истечении некоторого времени S перемещать все присутствующие в системе задания в очередь с высшим приоритетом.

Наше новое правило решает сразу две проблемы. Во-первых, процессы гарантированно не будут голодать: оказавшись в верхней очереди, задание будет разделять CPU с прочими высокоприоритетными заданиями, которые перебираются циклически, поэтому рано или поздно получит обслуживание. Во-вторых, если счетное задание стало интерактивным, то после повышения приоритета планировщик будет обращаться с ним соответствующим образом.

Рассмотрим пример. Мы продемонстрируем поведение долго работающего задания, которое конкурирует за CPU с двумя короткими интерактивными заданиями. На рис. 8.5 мы видим две диаграммы. На левой приоритет не повышался, и потому долго работающие задания перестают обслуживаться после поступления двух коротких заданий. На правой приоритет повышается каждые 50 мс (это, пожалуй, слишком мало, но для примера пусть будет так), поэтому гарантируется, что долго работающее задание будет потихоньку продвигаться вперед.

Разумеется, добавление интервала S ставит перед нами очевидный вопрос: чему должно быть равно S ? Джон Оустерхаут, хорошо известный исследователь систем [O11], называл подобные величины в системе **КОЛДОВСКИМИ константами**, потому что для их правильного задания, похоже, нужна черная магия. К сожалению, S именно из их числа. Если сделать S слишком

большим, то долго работающие задания будут голодать, а если слишком малым, то интерактивные задания не будут получать подобающую им долю процессорного времени.

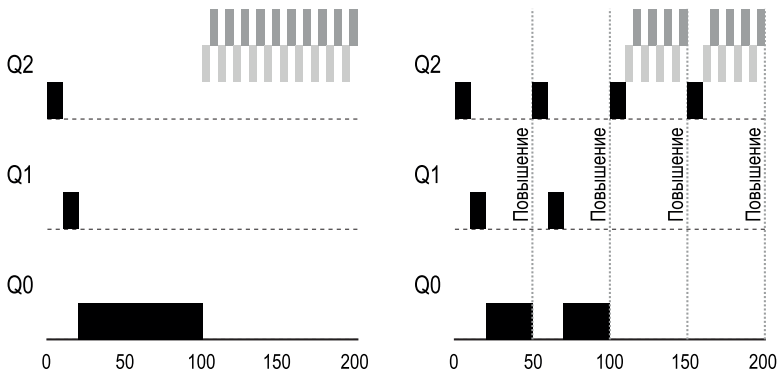


Рис. 8.5 ❖ Без повышения приоритета (слева) и с повышением приоритета (справа)

8.4. Попытка 3: улучшенный учет

Теперь нужно решить еще одну проблему: как предотвратить переигрывание планировщика? Виновниками, как вы, наверное, догадались, являются правила 4а и 4б, которые позволяют заданию сохранить приоритет путем добровольной уступки CPU до исчерпания его временного кванта. Так что же делать?

Решение заключается в том, чтобы вести более полный **учет** процессорного времени на каждом уровне MLFQ. Планировщик должен запоминать, какую часть кванта процесс использовал на данном уровне; после того как процесс использовал свою квоту, он перемещается в очередь с более низким приоритетом. А использует он ее за один раз или за несколько, не имеет значения. Поэтому правила 4а и 4б можно заменить одним правилом:

- **правило 4:** после того как задание израсходовало свою квоту на данном уровне (вне зависимости от того, сколько раз оно получало CPU), его приоритет понижается, т. е. оно опускается вниз на одну очередь.

Рассмотрим пример. На рис. 8.6 показано, что происходит, когда пользователь пытается переиграть планировщик по старым правилам 4а и 4б (слева) и по новому, препятствующему манипуляциям, правилу 4. Не будь защиты от переигрывания, процесс мог бы инициировать ввод-вывод непосредственно перед исчерпанием своего временного кванта и тем самым получить в свое распоряжение почти все процессорное время. Но при наличии защиты процесс независимо от того, как он использует ввод-вывод, медленно перемещается вниз и не может получить несправедливо большую долю времени.

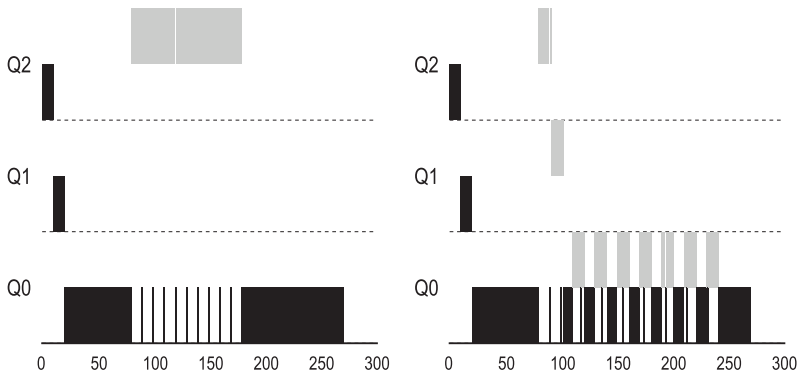


Рис. 8.6 ❖ Без препятствования переигрыванию (слева)
и с препятствованием (справа)

8.5. Настройка MLFQ и другие вопросы

В связи с MLFQ-планированием возникают и другие вопросы. Например, сколько должно быть очередей? Какова должна быть временная квота в каждой очереди? Как часто следует повышать приоритет, чтобы избежать голодной смерти и надлежащим образом учесть изменения в поведении? На эти вопросы нет простых ответов, только эксперименты с рабочими нагрузками и последующая настройка планировщика помогут найти приемлемый баланс.

Совет: избегайте колдовских констант (закон Оустерхаута)

Если есть такая возможность, избегайте колдовских констант. К сожалению, как показывает приведенный пример, часто это бывает затруднительно. Можно попытаться заставить систему обучиться хорошему значению, но и это непросто. Поэтому лучше создать конфигурационный файл параметров, которые опытный администратор сможет подправить, если что-то работает не оптимально. Как вы понимаете, чаще всего параметры никто не модифицирует, поэтому остается надеяться, что значения по умолчанию работают хорошо. Этот совет дал профессор, который читал нам курс по ОС, Джон Оустерхаут, поэтому мы назвали его **законом Оустерхаута**.

Например, в большинстве реализаций MLFQ допускаются разные временные кванты для разных очередей. Высокоприоритетным очередям обычно назначаются короткие кванты; в них находятся интерактивные задания, поэтому имеет смысл переключаться как можно быстрее (например, через каждые 10 мс или даже чаще). С другой стороны, в низкоприоритетных очередях находятся долго работающие счетные задания, поэтому разумнее выбирать кванты побольше (например, 100 мс). На рис. 8.7 приведен пример, когда два задания работают в течение 20 мс в верхней очереди (с квантом 10 мс), 40 мс в средней очереди (с квантом 20 мс) и 80 мс в нижней очереди с квантом 40 мс.

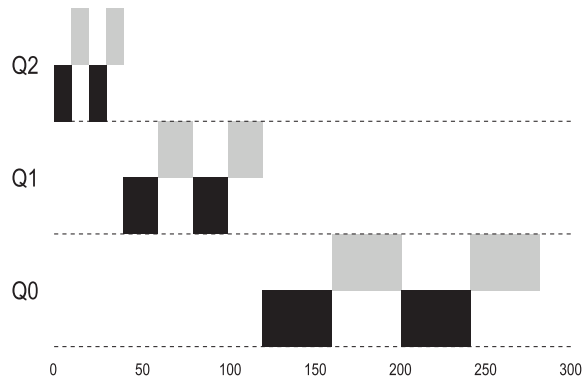


Рис. 8.7 ❖ Пониженный приоритет, увеличенный квант

В реализации MLFQ в Solaris класс планирования с разделением времени (TS) особенно легко конфигурируется; он предоставляет набор таблиц, точно определяющих, как меняется приоритет процесса на протяжении его жизни, какова длительность каждого временного кванта и как часто следует повышать приоритет задания [AD00]. Администратор может настроить эту таблицу и тем самым изменить поведение планировщика. По умолчанию имеется 60 очередей, а временной квант медленно увеличивается от 20 мс (высший приоритет) до нескольких сотен миллисекунд (низший приоритет); повышение приоритета производится примерно 1 раз в секунду.

В других MLFQ-планировщиках не используются ни таблицы, ни точные правила, описанные в этой главе; вместо этого приоритеты изменяются в соответствии с математической формулой. Например, в ОС FreeBSD версии 4.3 текущий приоритет задания вычисляется по формуле, учитывающей, сколько процессорного времени процесс уже использовал [LM+89]; кроме того, когда задание не использует процессор, его приоритет повышается, что приводит к желаемому результату, но способом, отличным от описанного выше. Прекрасный обзор алгоритмов с **понижением приоритета** и их свойствами см. в работе Ерема [E95].

Наконец, во многих планировщиках есть дополнительные возможности. Например, некоторые планировщики резервируют наибольшие уровни приоритета для самой операционной системы, поэтому пользовательские задания не могут получить наивысший приоритет. Есть системы, допускающие вмешательство пользователя в задание приоритетов; например, командная утилита `nice` позволяет повышать или понижать приоритет задания (в ограниченных пределах), а стало быть, его шансы на получение процессора. Дополнительные сведения см. на странице руководства.

Совет: по возможности пользуйтесь механизмом указаний

Поскольку операционная система редко знает, что лучше для каждого работающего процесса, полезно предоставлять интерфейсы, позволяющие пользователям или администраторам давать **указания**, или **рекомендации** ОС. Это именно рекомендации, а не приказы,

потому что ОС не обязана принимать их во внимание, но может учесть при принятии решений. Такие указания полезны во многих частях ОС, в т. ч. в планировщике (с помощью `nice`), в диспетчере памяти (с помощью `madvise`) и в файловой системе (например, для разумной предвыборки и кеширования [P+95]).

8.6. MLFQ: РЕЗЮМЕ

Мы описали подход к планированию, называемый многоуровневой аналитической очередью (MLFQ). Надеемся, что вы теперь понимаете, почему он называется именно так: имеется *несколько уровней* очередей, а для определения приоритета задания применяется *анализ*. Анализ основан на исторических сведениях – запоминается, как задание вело себя в прошлом, и принимаются соответствующие решения о его будущем.

Для удобства повторим правила MLFQ, разбросанные по тексту.

- **Правило 1:** если Приоритет(A) > Приоритет(B), то выполнять A (а не B).
- **Правило 2:** если Приоритет(A) = Приоритет(B), то применять к A и B алгоритм циклического планирования.
- **Правило 3:** когда задание поступает в систему, оно помещается в очередь с высшим приоритетом (верхнюю).
- **Правило 4:** после того как задание израсходовало свою квоту на данном уровне (вне зависимости от того, сколько раз оно получало CPU), его приоритет понижается, т. е. оно опускается вниз на одну очередь.
- **Правило 5:** по истечении некоторого времени *S* перемещать все присутствующие в системе задания в очередь с высшим приоритетом.

MLFQ интересен по следующей причине: он не требует априорных знаний о природе задания, а наблюдает за его поведением и соответственно назначает ему приоритет. Таким образом, ему удастся достичь сразу двух целей: обеспечить отличную общую производительность (как SJF и STCF) для коротких интерактивных заданий, а также справедливость и возможность продвижения для долго работающих счетных заданий. Поэтому во многих системах, в т. ч. производных от BSD UNIX [LM+89, B86], Solaris [M06], Windows NT и последующих версий Windows [CS97], MLFQ является основным планировщиком.

Литература

[AD00] «Multilevel Feedback Queue Scheduling in Solaris» by Andrea Arpaci-Dusseau. Доступно по адресу <http://www.ostep.org/Citations/notes-solaris.pdf>. *Прекрасные заметки одного из авторов этой книги по особенностям планировщика в Solaris. Ну хорошо, мы, наверное, пристрастны в оценке, но честно – заметки чертовски хороши.*

[B86] «The Design of the UNIX Operating System» by M.J. Bach. Prentice-Hall, 1986. *Одна из классических старых книг о том, как устроена реальная операционная система Unix; обязательное чтение для желающих разобраться в ядре.*

[C+62] «An Experimental Time-Sharing System» by F. J. Corbato, M. M. Daggett, R. C. Daley. IFIPS 1962. *Трудный для чтения текст, но является источником многих оригинальных идей планирования с многоуровневой аналитической очередью. Многие из них впоследствии вошли в систему Multics, которую, пожалуй, можно назвать самой влиятельной операционной системой всех времен.*

[CS97] «Inside Windows NT» by Helen Custer and David A. Solomon. Microsoft Press, 1997. *Та самая книга по NT, которую стоит прочитать, если вы хотите узнать о чем-то, кроме Unix. Впрочем, зачем бы вам это? Шутим, конечно, не исключено, что когда-нибудь вы будете работать в Microsoft.*

[E95] «An Analysis of Decay-Usage Scheduling in Multiprocessors» by D. H. J. Epema. SIGMETRICS '95. *Прекрасная статья о состоянии дел, сложившемся в планировании в середине 1990-х годов. Включает хороший обзор подхода, лежащего в основе планировщиков с понижением приоритета при использовании процессора.*

[LM+89] «The Design and Implementation of the 4.3BSD UNIX Operating System» by S. J. Leffler, M. K. McKusick, M. J. Karels, J. S. Quarterman. Addison-Wesley, 1989. *Еще одна классическая книга, написанная четырьмя основными идеологами BSD. Последующие ее издания, хотя и более актуальные, лишены очарования этой редакции.*

[M06] «Solaris Internals: Solaris 10 and OpenSolaris Kernel Architecture» by Richard Mc-Dougall. Prentice-Hall, 2006. *Хорошая книга о внутреннем устройстве Solaris.*

[O11] «John Ousterhout's Home Page» by John Ousterhout. www.stanford.edu/~ouster/. *Домашняя страница знаменитого профессора Оустерхаута. Оба автора этой книги имели удовольствие прослушать его курс по операционным системам для магистрантов. Собственно, именно там соавторы и познакомились, что привело к браку, детишкам и даже этой книге. Так что вините Оустерхаута за то, что вляпались в это дело.*

[P+95] «Informed Prefetching and Caching» by R. H. Patterson, G. A. Gibson, E. Ginting, D. Stodolsky, J. Zelenka. SOSP '95, Copper Mountain, Colorado, October 1995. *Любопытная статья о некоторых очень важных для файловых систем идеях, в т. ч. о том, как приложение может проинформировать ОС о том, к каким файлам оно обращается, и как планирует с ними работать.*

[Y+18] «Principled Schedulability Analysis for Distributed Storage Systems using Thread Architecture Models» by Suli Yang, Jing Liu, Andrea C. Arpaci-Dusseau, Remzi H. Arpaci-Dusseau. OSDI '18, San Diego, California. *Недавняя работа нашей группы, которая демонстрирует трудности планирования запросов ввода-вывода в таких современных распределенных системах хранения, как Hive/HDFS, Cassandra, MongoDB и Riak. Без должной аккуратности один пользователь сможет монополизировать системные ресурсы.*

Домашнее задание (эмуляция)

Программа `mlfq.py` позволяет наглядно увидеть, как ведет себя MLFQ-планировщик, описанный в этой главе. Детали см. в файле README.

Вопросы

1. Запустите несколько случайно сгенерированных задач с двумя заданиями и двумя очередями; вычислите для каждой трассу выполнения MLFQ. Упростите себе жизнь, ограничив продолжительность каждого задания и выключив ввод-вывод.
2. Как следует запустить планировщик для воспроизведения всех примеров, приведенных в этой главе?
3. При какой конфигурации параметров планировщик будет вести себя как циклический?
4. Подберите рабочую нагрузку, состоящую из двух заданий, и параметры планировщика, так чтобы одно задание могло воспользоваться правилами 4a и 4b (включаются флагом `-S`), чтобы обмануть планировщик и захватить 99 % процессорного времени на определенном временном интервале.
5. Пусть величина временного кванта для самой высокоприоритетной очереди равна 10 мс. Как часто нужно повышать приоритет заданий до максимального (флаг `-B`), чтобы одно долго работающее (и потенциально голодающее) задание гарантированно получило по меньшей мере 5 % процессорного времени?
6. При планировании возникает такой вопрос: в какой конец очереди помещать задание, только что завершившее ввод-вывод? Это поведение задает флаг эмулятора `-I`. Поэкспериментируйте с разными рабочими нагрузками и изучите, дает ли этот флаг какой-нибудь эффект.

Глава 9

Планирование: пропорциональная доля

В этой главе мы рассмотрим другой тип планирования – **пропорциональный**, или **равномерный**, планировщик. Идея проста: вместо оптимизации оборотного времени или времени отклика планировщик может попытаться гарантировать, что каждое задание получает определенную долю процессорного времени.

Великолепный ранний пример равномерного планирования можно найти в статье Waldspurger and Weihl [WW94], он известен также под названием **лотерейное планирование**. Однако сама идея, конечно, старше [KL88] и заключается в том, чтобы периодически проводить лотерею и решать, какой процесс должен выполняться следующим. Процессам, которые должны выполняться чаще, дается больше шансов выиграть в лотерею. Просто, да? Но теперь надо заняться деталями. Однако прежде:

Существо проблемы: как разделить CPU пропорционально

Как спроектировать планировщик, который распределяет процессорное время пропорционально? Какие ключевые механизмы при этом применяются? Насколько они эффективны?

9.1. Основная идея: ВАША ДОЛЯ ПРЕДСТАВЛЕНА БИЛЕТОМ

В основе лотерейного планирования лежит очень простая идея: **билеты**, представляющие долю ресурса, которую должен получить процесс (или пользователь, или еще что-то). Процентная доля билетов, которыми владеет процесс, соответствует его доле в рассматриваемом системном ресурсе.

Рассмотрим пример. Пусть есть два процесса, А и В, причем процесс А владеет 75 билетами, а В только 25. То есть мы хотели бы, чтобы А получил 75 % процессорного времени, а В – оставшиеся 25 %.

При лотерейном планировании задача решается вероятностно (недетерминированно) – периодически (скажем, в каждом временном кванте) проводится лотерея. Провести лотерею просто: планировщик должен знать, сколько всего выпущено билетов (в нашем примере – 100). Затем планировщик тянет выигрышный билет, т. е. число от 0 до 99¹. В предположении, что А владеет билетами от 0 до 74, а В – билетами от 75 до 99, выигрышный билет определяет, кто будет работать: А или В. Затем планировщик загружает состояние выигравшего процесса и запускает его.

Совет: о пользе случайности

Один из самых полезных аспектов лотерейного планирования – **случайность**. Такой **рандомизированный** подход часто оказывается самым простым и надежным способом принятия решения.

У рандомизированных подходов есть по крайней мере три преимущества по сравнению с традиционными. Во-первых, рандомизация позволяет избежать странных и редких видов поведения, которые должен учитывать традиционный алгоритм. Например, рассмотрим политику замещения LRU (подробнее мы будем изучать ее в следующей главе, посвященной виртуальной памяти); для этого, в общем-то, хорошего алгоритма худшим случаем являются некоторые циклические последовательные рабочие нагрузки. А у рандомизированного алгоритма такого худшего случая нет.

Во-вторых, рандомизация почти не требует хранения состояния для выбора альтернатив. В традиционном равномерном алгоритме планирования нужно запоминать, сколько процессорного времени потребил каждый процесс, и обновлять эти учетные данные всякий раз после того, как процесс исчерпал свой квант. Рандомизированный же алгоритм должен хранить только минимальные сведения о состоянии процесса (например, количество выданных ему билетов).

Наконец, рандомизированный алгоритм может работать очень быстро. При условии, что случайные числа генерируются быстро, принятие решения тоже не требует много времени, поэтому такой алгоритм можно использовать в разных местах, где необходима высокая скорость работы. Конечно, чем больше потребность в скорости, тем ближе случайные числа к псевдослучайным.

В примере ниже показаны выигравшие билеты лотерейного планировщика:

63 85 70 39 76 17 29 41 36 39 10 99 68 83 63 62 43 0 49 49

А вот как выглядит результат планирования:

А	А	А	А	А	А	А	А	А	А	А	А	А	А	А	А	А	А	А	А
В		В								В		В							

Как видим, рандомизированный алгоритм лотерейного планирования в вероятностном смысле соблюдает желаемую пропорцию, но никаких га-

¹ В информатике отсчет всегда начинается с 0. Для людей, далеких от компьютеров, это настолько странно, что знаменитые люди посчитали необходимым написать, почему так делается [D82].

Один из самых мощных (и основных) механизмов при проектировании лотерейного планирования – **билет**. В наших примерах он представляет долю процессорного времени, выделенного процессу, но может применяться и более широко. Например, в недавней работе по управлению виртуальной памятью для гипервизоров Вальдспургер показывает, как билеты можно использовать для представления доли памяти, выделяемой гостевой операционной системе [W02]. Таким образом, если вам когда-нибудь понадобится механизм для представления доли владения, то это может быть... погодите-ка... ага, билет.

Лотерейное планирование предлагает также ряд механизмов для манипулирования билетами различными полезными способами. Один из них – концепция **валюты билетов**. Валюта позволяет пользователю, владеющему набором билетов, распределять их между своими заданиями в той валюте, в какой они пожелают, а система затем автоматически конвертирует эту валюту в глобальную.

Например, предположим, что пользователям А и В выдано по 100 билетов. Пользователь А запустил два задания, А1 и А2, и выделил каждому по 500 (из имеющихся 1000) в своей валюте. Пользователь В запустил только одно задание и выдал ему 10 билетов (из имеющихся 10). Тогда система конвертирует полученные А1 и А2 500 билетов в валюте А в 50 в глобальной валюте; аналогично 10 билетов В1 будут конвертированы в 100. Затем будет проведена лотерея в глобальной валюте билетов (всего 200), чтобы определить, какое задание выполнять.

Пользователь В -> 10 (в валюте В) в В1 -> 100 (в глобальной валюте)

Еще один полезный механизм – **передача билетов**, когда процесс временно передает свои билеты другому процессу. Эта возможность особенно полезна в клиент-серверных приложениях, когда клиентский процесс отправляет серверу сообщение с просьбой выполнить какую-то работу от имени клиента. Чтобы ускорить работу, клиент может передать свои билеты серверу и таким образом попытаться максимизировать производительность сервера на время обработки своего запроса. По завершении работы сервер передает билеты обратно клиенту, и все возвращается на круги своя.

Наконец, иногда бывает полезна **инфляция билетов**, когда процесс временно увеличивает или уменьшает количество принадлежащих ему билетов.

Разумеется, в случае конкуренции, когда процессы не доверяют друг другу, это не имеет особого смысла; один жадный процесс мог бы выдать себе огромное число билетов и монополизировать машину. Инфляция применяется в ситуации, когда имеется группа доверяющих друг другу процессов; тогда, если какой-то процесс понимает, что ему нужно больше процессорного времени, он может увеличить ценность своего билета, чтобы сообщить об этой необходимости системе, не вступая в переговоры с другими процессами.

9.3. РЕАЛИЗАЦИЯ

Пожалуй, самое удивительное в лотерейном планировании – простота реализации. Нужен всего лишь хороший генератор случайных чисел для выбора выигрышного билета, структура данных для хранения процессов в системе (например, список) и общее число билетов.

Предположим, что процессы хранятся в списке. Ниже приведен пример трех процессов, А, В и С, у каждого из которых есть сколько-то билетов.



Чтобы принять решение, планировщик должен сначала выбрать случайное число (победителя) из общего числа билетов (400)¹. Предположим, что выбрано число 300. Затем мы просто обходим список, а счетчик позволяет найти победителя (рис. 9.1).

Программа обходит список процессов, добавляя стоимость каждого билета к `counter`, пока не будет превышено значение `winner`. В этот момент текущий элемент списка объявляется победителем. В нашем примере, где выигрышный билет равен 300, происходит следующее. Сначала `counter` увеличивается на 100 – учитываются билеты А. Поскольку 100 меньше, чем 300, цикл продолжается. Затем `counter` увеличивается еще на 50 (билеты В), но результат по-прежнему меньше 300, так что цикл снова продолжается. Наконец, `counter` возрастает до 400, что заведомо больше 300, поэтому мы выходим из цикла, а `suggest` указывает на С (победителя).

Чтобы повысить эффективность процедуры, имеет смысл хранить список отсортированным по числу билетов – от наибольшего к наименьшему. Правильность алгоритма от порядка следования не зависит, но количество итераций минимизируется, что особенно важно, когда имеется небольшое число процессов, владеющих большинством билетов.

¹ Бьёрн Линдберг указал, что, как ни странно, сделать это правильно довольно трудно; см. <http://stackoverflow.com/questions/2509679/how-to-generate-a-random-number-from-within-a-range>.


```

1 // counter (счетчик): отслеживает, определен победитель или еще нет
2 int counter = 0;
3
4 // winner (победитель): использовать какой-нибудь генератор случайных чисел,
5 // возвращающий значение от 0 до общего числа билетов
6 int winner = getrandom(0, totaltickets);
7
8 // current: используется для обхода списка заданий
9 node_t *current = head;
10
11 // цикл, пока сумма стоимостей билетов не станет > winner
12 while (current) {
13     counter = counter + current->tickets;
14     if (counter > winner)
15         break; // победитель найден
16     current = current->next;
17 }
18 // 'current' – победитель: запланировать его...

```

Рис. 9.1 ❖ Код принятия решения в лотерейном планировании

9.4. ПРИМЕР

Чтобы лучше разобраться в динамике лотерейного планирования, рассмотрим время завершения двух конкурирующих заданий с одинаковым числом билетов (100) и одинаковым временем работы (параметр R , который мы будем изменять).

В этом сценарии мы хотим, чтобы все задания завершались примерно в одно время, но в силу случайности лотерейного планирования одно задание иногда будет завершаться раньше другого. Чтобы количественно оценить эту разность, мы определим простую **метрику неравномерности** U , равную частному от деления времени завершения первого задания на время завершения второго. Например, если $R = 10$ и первое задание завершается в момент 10 (а второе – в момент 20), то $U = 10/20 = 0.5$. Если оба задания завершаются примерно в одно время, то U будет близко к 1. Это и есть наша цель: для идеально равномерного планировщика $U = 1$.

На рис. 9.2 показан график зависимости средней неравномерности от продолжительности обоих заданий (R), изменяющейся в диапазоне от 1 до 1000, в 30 испытаниях (результаты сгенерированы эмулятором, приведенным в конце этой главы). Как видно из графика, когда задание не слишком продолжительно, средняя неравномерность довольно велика. И лишь когда задание работает в течение многих временных квантов, лотерейный планировщик приближается к желаемому результату.

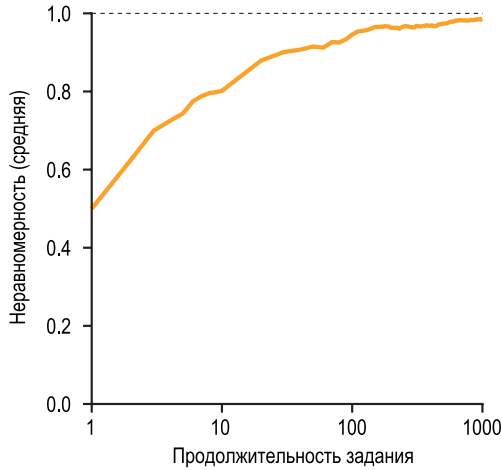


Рис. 9.2 ❖ Изучение справедливости лотерейного планирования

9.5. КАК РАЗДАВАТЬ БИЛЕТЫ?

Мы пока не рассмотрели одну проблему, возникающую при лотерейном планировании: как раздавать билеты заданиям? Проблема серьезная, потому что поведение системы, естественно, сильно зависит от того, как распределяются билеты. Один из возможных подходов – считать, что пользователям виднее, тогда каждому пользователю выдается определенное число билетов, а он уже раздает их своим заданиям, как пожелает. Но это решение – обманка: что делать, мы так и не знаем. Поэтому «проблема раздачи билетов» данному множеству заданий остается открытой.

9.6. ЗАЧЕМ ОТКАЗЫВАТЬСЯ ОТ ДЕТЕРМИНИРОВАННОСТИ?

Может возникнуть вопрос: а зачем нам вообще случайность? Выше мы уже видели, что хотя благодаря случайности мы получили простой (и приближенно правильный) планировщик, иногда пропорции оказываются неточными, особенно на коротких временных интервалах. Поэтому Вальдспургер придумал **шаговое планирование** – детерминированный равномерный планировщик [W95]. Идея шагового планирования проста. Для каждого задания в системе определен **шаг** (stride), обратно пропорциональный количеству принадлежащих ему билетов. В нашем примере имеются три задания, А, В и С, владеющих соответственно 100, 50 и 250 билетами, поэтому для вычисления шага нужно разделить какое-нибудь большое число на количество билетов, выданных каждому процессу. Например, если в качестве большого

числа взять 10 000, то шаги заданий А, В и С будут равны 100, 200 и 40. Всякий раз, как некоторый процесс работает, мы будем увеличивать для него счетчик (называемый **прогрессом**) на величину шага; это позволит отслеживать, как далеко он продвинулся.

Планировщик использует шаг и прогресс, чтобы решить, какой процесс выполнять следующим. Идея простая: в каждый момент времени выбирать процесс с наименьшим прогрессом и для выбранного процесса увеличивать прогресс на величину шага. Псевдокод предложен Вальдспургером в работе [W95]:

```
current = remove_min(queue);    // выбрать клиента с минимальным прогрессом
schedule(current);             // использовать ресурс на протяжении одного кванта
current->pass += current->stride; // вычислить следующее значение прогресса, прибавив шаг
insert(queue, current);        // поместить обратно в очередь
```

В нашем примере вначале имеется три процесса (А, В и С) с шагами 100, 200 и 40, для каждого из них начальное значение прогресса равно 0. Поэтому сначала может быть выбран любой процесс. Предположим, что выбран А (выбор произволен, можно было бы взять любой другой). По завершении А его прогресс принимает значение 100. Затем мы выбираем В и по завершении присваиваем его прогрессу значение 200. Наконец, выполняется С, прогресс которого по завершении станет равен 40. В этот момент алгоритм выберет процесс с наименьшим прогрессом, т. е. С, выполнит его и присвоит прогрессу значение 80 (напомним, что шаг С равен 40). Затем С будет выбран снова (его прогресс все еще минимален), и прогресс достигнет 120. Теперь будет выполнен процесс А, а его прогресс увеличится до 200 (и сравняется с прогрессом В). Затем С будет выбран еще два раза, так что его прогресс станет равен сначала 160, а потом 200. В этот момент все значения прогресса снова стали равны, и весь цикл повторяется – и так до бесконечности. На рис. 9.3 показана динамика планировщика.

Прогресс(А) (шаг = 100)	Прогресс(В) (шаг = 200)	Прогресс(С) (шаг = 40)	Кто работает?
0	0	0	А
100	0	0	В
100	200	0	С
100	200	40	С
100	200	80	С
100	200	120	А
200	200	120	С
200	200	160	С
200	200	200	...

Рис. 9.3 ❖ Шаговое планирование: трасса

Как видно из таблицы, С выполнялся пять раз, А два раза, а В всего один раз – в точном соответствии со стоимостью их билетов: 250, 100 и 50. При лотерейном планировании пропорции выдерживаются с некоторой вероят-

ностью, а при шаговом – абсолютно точно в конце каждого цикла планирования. Поэтому возникает естественный вопрос: если шаговое планирование точное, то зачем вообще нужно лотерейное? А дело в том, что у лотерейного планирования есть одно хорошее свойство – отсутствие глобального состояния. Представьте себе, что в середине рассмотренного выше примера поступает новое задание; какой прогресс ему назначить? Может быть, 0? Но тогда оно монополизировало процессор. При лотерейном планировании у процесса нет никакого глобального состояния; мы просто добавляем новый процесс вместе с его билетами, изменяем единственную глобальную переменную, равную общему количеству билетов, и продолжаем работу, как ни в чем не бывало. Таким образом, лотерейное планирование существенно упрощает включение новых процессов.

9.7. ВПОЛНЕ РАВНОМЕРНЫЙ ПЛАНИРОВЩИК в LINUX

Несмотря на ранние работы по равномерному планированию, в современных версиях Linux схожие цели достигаются другим методом. **Вполне равномерный планировщик** (Completely Fair Scheduler – CFS) [J09] реализует политику равномерного планирования эффективным и масштабируемым способом.

Для обеспечения эффективности CFS старается тратить как можно меньше времени на принятие решений, и достигается это как на проектном уровне, так и благодаря умному использованию структур данных, специально предназначенных для этой задачи. Недавние исследования показали, что эффективность планировщика чрезвычайно важна; именно, в работе Канева и др., посвященной ЦОДам Google, показано, что даже после агрессивной оптимизации планирование занимает приблизительно 5 % всего времени работы ЦОДа. Поэтому всемерное сокращение накладных расходов является главной целью современной архитектуры планировщика.

Принцип работы

Большинство планировщиков построены в предположении, что временной квант фиксирован, но CFS работает по-другому. Его цель проста: равномерно распределить процессорное время между всеми конкурирующими процессами. Для этого применяется основанная на подсчете техника **виртуального времени работы** (virtual runtime – *vruntime*).

По ходу работы каждый процесс аккумулирует *vruntime*. В самом простом случае *vruntime* всех процессов увеличивается с одинаковой скоростью, так что виртуальное время пропорционально физическому (реальному). В момент принятия решения планировщик CFS выбирает в качестве следующего процесс с наименьшим *vruntime*.

Тогда возникает вопрос: откуда планировщик знает, когда остановить текущий процесс и запустить следующий? Конфликт целей очевиден: если CFS переключает процессы слишком часто, то равномерность повышается, поскольку CFS гарантирует, что каждый процесс получит свою долю CPU даже в крохотном временном окне, – но ценой падения производительности. Если же CFS переключает процессы реже, то производительность возрастает (меньше контекстных переключений), но за счет утраты равномерности в краткосрочной перспективе.

CFS разрешает этот конфликт с помощью различных управляющих параметров. Первый называется `sched_latency`. Его значение позволяет CFS решить, сколько времени должен проработать процесс, прежде чем будет решаться вопрос о переключении (по сути, это временной квант, но определяемый динамически). Типичное значение `sched_latency` равно 48 (миллисекунд); CFS делит это значение на число (n) процессов, исполняемых процессором, чтобы определить временной квант каждого процесса, и таким образом гарантирует, что на этом отрезке времени планировщик вполне равномерный.

Например, если работает $n = 4$ процесса, то CFS делит значение `sched_latency` на n и получает квант, равный 12 мс. Затем CFS планирует первое задание и позволяет ему работать до истечения 12 мс (виртуального) времени, после чего проверяет, существует ли задание с меньшим значением `vruntime`, которым можно было бы заместить текущее. В данном случае такое задание существует, и CFS переключится на одно из трех других заданий. И так далее. На рис. 9.4 приведен пример, когда имеется четыре задания (A, B, C, D), каждое из которых работает в течение двух временных квантов; затем два из них (C, D) завершаются, а оставшиеся два будут работать по 24 мс, сменяя друг друга.

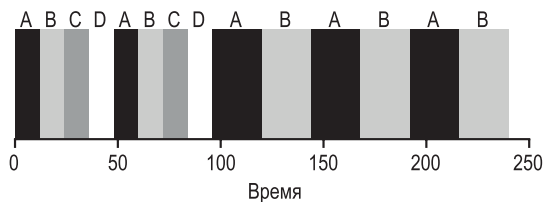


Рис. 9.4 ❖ Простой пример CFS

Но что, если работает «слишком много» процессов? Не приведет ли это к слишком малому значению временного кванта и, как следствие, к чрезмерно большому числу контекстных переключений? Хороший вопрос! И да – обязательно приведет.

Чтобы решить эту проблему, вводится еще один параметр – `min_granularity`, который обычно равен 6 мс или близко к этому. CFS никогда не присваивает временному кванту меньшее значение, гарантируя тем самым, что накладные расходы на планирование не станут слишком велики.

Например, если работает десять процессов, то приведенное выше вычисление (деление `sched_latency` на десять) дало бы квант 4.8 мс. Но вместо это-

го каждому процессу выделяется квант 6 мс. Хотя CFS не достигает полной равномерности на интервале длительностью 48 мс (`sched_latency`), он близок к желанной цели, но при этом сохраняет высокую эффективность.

Заметим, что CFS пользуется периодическим прерыванием от таймера, поэтому может принимать решения только через фиксированные промежутки времени. Таймер срабатывает часто (например, раз в миллисекунду), давая CFS шанс осмотреться и решить, не пора ли менять текущее задание. Если временной квант задания не является точным кратным интервала таймера, ничего страшного; CFS точно отслеживает `vruntime`, т. е. в долгосрочной перспективе распределяет CPU почти идеально.

Взвешивание (уровень `nice`)

CFS также позволяет управлять приоритетом процесса, т. е. пользователи или администраторы могут увеличивать долю процессорного времени, выделяемого конкретным процессам. Для этого используются не билеты, а классический механизм Unix, называемый уровнем **nice** процесса. Параметр `nice` может принимать значения от -20 до $+19$, по умолчанию он равен 0. Как ни странно, положительные значения `nice` означают *меньший* приоритет, а отрицательные – *большой*, это еще одна вещь, которую нужно просто запомнить.

CFS следующим образом отображает значения `nice` на вес:

```
static const int prio_to_weight[40] = {
/* -20 */ 88761, 71755, 56483, 46273, 36291,
/* -15 */ 29154, 23254, 18705, 14949, 11916,
/* -10 */ 9548, 7620, 6100, 4904, 3906,
/* -5 */ 3121, 2501, 1991, 1586, 1277,
/* 0 */ 1024, 820, 655, 526, 423,
/* 5 */ 335, 272, 215, 172, 137,
/* 10 */ 110, 87, 70, 56, 45,
/* 15 */ 36, 29, 23, 18, 15,
};
```

Эти веса позволяют вычислить эффективный временной квант каждого процесса (как мы делали раньше), но с учетом различия в приоритетах. Делается это по следующей формуле:

$$\text{time_slice}_k = \frac{\text{weight}_k}{\sum_{i=0}^{n-1} \text{weight}_i} \cdot \text{sched_latency}. \quad (9.1)$$

Проиллюстрируем на примере, как это работает. Пусть имеется два задания, А и В. Задание А для нас особенно ценно, поэтому назначим ему более высокий приоритет, присвоив значение `nice` = -5 . Задание В мы не любим, поэтому оставим для него приоритет по умолчанию (`nice` = 0). Тогда вес weight_A (по таблице) будет равен 3121, а вес weight_B – 1024. Если теперь вычислить временной квант каждого задания, то окажется, что квант А приблизительно равен $3/4$ от `sched_latency` (36 мс), а квант В – приблизительно $1/4$ (12 мс).

Помимо обобщенного вычисления временного кванта, необходимо также изменить способ вычисления `vruntime`. Ниже приведена новая формула, которая учитывает фактическое время работы i -го процесса (`runtimei`) и уменьшает его пропорционально весу процесса. В нашем примере `vruntime` процесса А растет в три раза медленнее, чем для процесса В.

$$\text{vruntime}_i = \text{vruntime}_i + \frac{\text{weight}_0}{\text{weight}_i} \cdot \text{runtime}_i. \quad (9.2)$$

Интересная особенность показанной выше таблицы весов заключается в том, что при одной и той же разности значений `nice` отношение долей CPU приблизительно одинаково. Так, если бы для процесса А `nice` было равно 5 (а не -5), а для процесса В - 10 (а не 0), то CFS планировал бы их точно так же, как и раньше. Прodelайте сами математические вычисления, чтобы убедиться в этом.

Использование красно-черных деревьев

Мы уже говорили, что одна из главных целей CFS – эффективность. У эффективности планировщика много граней, но одна из них особенно очевидна: поиск следующего подлежащего выполнению задания должен производиться максимально быстро. Простые структуры данных типа списка масштабируются плохо: в современных системах одновременно работают тысячи процессов, поэтому тратить драгоценные миллисекунды на просмотр длинного списка расточительно.

В CFS эта проблема решается с помощью хранения процессов в **красно-черном дереве** [B72]. Это один из многих вариантов сбалансированного дерева; в отличие от простого двоичного дерева (которое в худшем случае может вырождаться в линейный список), сбалансированные деревья выполняют дополнительные действия, чтобы глубина всегда оставалась минимальной, поэтому время выполнения операций зависит от количества узлов логарифмически (а не линейно).

CFS хранит в этой структуре не *все* процессы, а только исполняемые (или готовые к исполнению). Если процесс засыпает (например, в ожидании завершения ввода-вывода или получения пакета), то он удаляется из дерева и хранится в другом месте.

Чтобы стало яснее, рассмотрим пример. Пусть имеется десять заданий со следующими значениями `vruntime`: 1, 5, 9, 10, 14, 18, 17, 21, 22, 24. Если бы мы хранили их в упорядоченном списке, то найти следующее задание было бы просто: ему соответствует первый элемент. Но чтобы поместить задание в нужное место списка, пришлось бы просмотреть его целиком (по порядку), на что потребовалось бы время порядка $O(n)$. Поиск произвольного элемента также был бы неэффективен и в среднем занимал бы линейное время.

Хранение тех же значений в красно-черном дереве повышает эффективность большинства операций, как показано на рис. 9.5. Процессы в дереве упорядочены по `vruntime`, а типичные операции (например, вставка и удаление)

ние) занимают логарифмическое время, $O(\log n)$. Если n исчисляется тысячами, то логарифмическое время заметно меньше линейного.

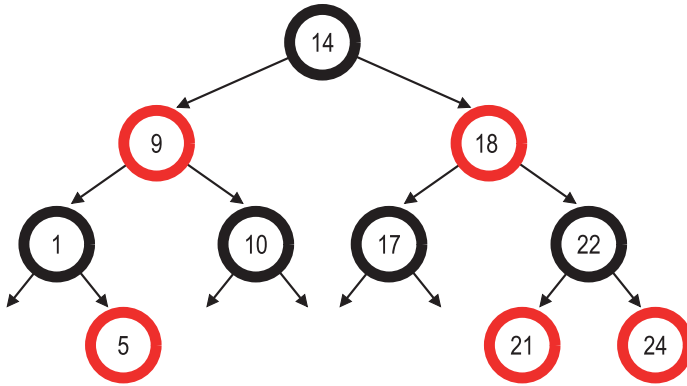


Рис. 9.5 ❖ Красное-черное дерево в CFS

Обращение со спящими процессами

С выбором процесса с наименьшим `vruntime` для запуска следующим связана еще одна проблема: задания, которые надолго засыпают. Пусть имеется два процесса, А и В, один из которых (А) работает непрерывно, а другой (В) заснул на длительное время (скажем, 10 секунд). Когда В проснется, его `vruntime` будет на 10 секунд меньше, чем у А, и, значит, если не принять мер, В монополизировать процессор на следующие 10 секунд, чтобы догнать А, лишив тем самым А возможности выполняться.

CFS решает эту проблему путем изменения `vruntime` задания в момент его просыпания. Точнее, CFS устанавливает `vruntime` этого задания равным минимальному значению в дереве (напомним, что в дереве хранятся только исполняемые и готовые к исполнению процессы) [B+18]. Таким образом, CFS не допускает голодания, но не даром: задания, которые часто засыпают на короткое время, не будут получать справедливой доли процессорного времени [AC97].

Другие возможности CFS

У CFS есть много других возможностей – слишком много, чтобы обсуждать их в этой книге. Это и многочисленные эвристики для повышения производительности кеша, и стратегии эффективной работы с несколькими CPU (мы рассмотрим их ниже), и возможность планирования больших групп процессов (вместо того чтобы рассматривать каждый процесс независимо от других), и еще много чего интересного. Чтобы узнать обо всем этом, обратитесь к недавним исследованиям и начните с работы Bouron [B+18].

Совет: используйте эффективные структуры данных там, где это оправдано

Во многих случаях списка вполне достаточно. Но далеко не всегда. Понимание того, какую структуру данных использовать, – отличительный признак хорошего инженера. В рассмотренном выше случае простые списки, характерные для ранних планировщиков, не годятся для современных систем, где имеются тысячи активных процессов; раз в несколько миллисекунд для каждого процессорного ядра искать в длинном списке следующее задание, подлежащее выполнению, – недопустимое расточительство. Необходима более эффективная структура, и CFS блестяще использует для этого красно-черное дерево. Вообще, при выборе структуры данных для своей системы внимательно изучите закономерности доступа и частоту использования, тогда вы сможете подобрать структуру, подходящую для конкретной задачи.

9.8. РЕЗЮМЕ

Мы познакомились с концепцией пропорционального планирования и кратко обсудили три подхода: лотерейное планирование, шаговое планирование и вполне равномерный планировщик (CFS), применяемый в Linux. При лотерейном планировании используется вероятностный способ достижения пропорционального разделения процессора, а при шаговом эта задача решается детерминированно. CFS, единственный «реальный» планировщик, обсуждаемый в этой главе, напоминает взвешенное циклическое планирование с динамическими временными квантами, но допускает масштабирование и хорошо работает при высокой нагрузке; насколько нам известно, это самый распространенный из существующих на сегодня равномерных планировщиков.

Никакой планировщик не является панацеей, и у равномерных планировщиков тоже есть свои проблемы. Одна из них – неидеальное сочетание с вводом-выводом [AC97]; как уже было сказано, задания, которые часто выполняют ввод-вывод, могут не получать справедливой доли CPU. Другая проблема заключается в том, что они оставляют открытым трудный вопрос о билетах или назначении приоритетов, т. е. как узнать, сколько билетов следует выделить вашему браузеру или какое значение `nice` назначить вашему текстовому редактору? Другие планировщики общего назначения (например, обсуждавшийся выше MLFQ и аналогичные планировщики в Linux) решают эти проблемы автоматически, поэтому их проще развертывать.

Но есть и хорошие новости: во многих предметных областях эти проблемы не особенно важны, поэтому пропорциональные планировщики вполне эффективны. Например, в **виртуализированном ЦОДе** (или в **облаке**), когда требуется выделить четверть процессорного времени VM Windows, а остальное – VM Linux, пропорциональное планирование оказывается простым и эффективным. Эту идею можно обобщить и на другие ресурсы; дополнительные сведения о том, как производится пропорциональное разделение памяти в программе ESX Server от компании VMware, см. в работе Вальдспургера [W02].

Литература

[AC97] «Extending Proportional-Share Scheduling to a Network of Workstations» by Andrea C. Arpaci-Dusseau and David E. Culler. PDPTA'97, June 1997. *Работа одного из авторов этой книги о том, как обобщить пропорциональное планирование на кластерную среду.*

[B+18] «The Battle of the Schedulers: FreeBSD ULE vs. Linux CFS» by J. Bouron, S. Chevalley, B. Lepers, W. Zwaenepoel, R. Gouicem, J. Lawall, G. Muller, J. Sopena. USENIX ATC '18, July 2018, Boston, Massachusetts. *Недавняя обстоятельная работа, посвященная сравнению планировщиков Linux CFS и FreeBSD. Приведен также прекрасный обзор обоих планировщиков. Результат сравнения: неоднозначно – в одних случаях лучше был CFS, в других ULE (планировщик, применяемый в BSD). Что поделаешь, иногда жизнь не дает простых ответов.*

[B72] «Symmetric binary B-Trees: Data Structure And Maintenance Algorithms» by Rudolf Bayer. Acta Informatica, Volume 1, Number 4, December 1972. *Элегантное сбалансированное дерево, придуманное еще до вашего рождения (скорее всего). Одно из многих существующих сбалансированных деревьев, посмотрите в своей любимой книге по алгоритмам другие варианты!*

[D82] «Why Numbering Should Start At Zero» by Edsger Dijkstra, August 1982. Доступно по адресу <http://www.cs.utexas.edu/users/EWD/ewd08xx/EWD831.PDF>. *Небольшая заметка Э. Дейкстры, одного из основоположников информатики. Мы еще встретимся с ним в разделе о конкурентности. А пока насладитесь чтением этой статьи, из которой взята следующая цитата: «Один из моих коллег-математиков, но не информатик, упрекнул более молодых специалистов по информатике в педантизме из-за их привычки нумеровать последовательности с нуля». В заметке объясняется, почему это логично.*

[K+15] «Profiling A Warehouse-scale Computer» by S. Kanev, P. Ranganathan, J. P. Darago, K. Hazelwood, T. Moseley, G. Wei, D. Brooks. ISCA '15, June, 2015, Portland, Oregon. *Удивительное исследование на тему о том, на что тратится время в современных центрах обработки данных, где сосредотачивается все больше вычислений, производимых на планете. Почти 20 % процессорного времени забирают операционные системы, причем 5 % приходится только на планировщик!*

[J09] «Inside The Linux 2.6 Completely Fair Scheduler» by M. Tim Jones. December 15, 2009. www.ibm.com/developerworks/linux/library/l-completely-fair-scheduler. *Простое описание CFS, начиная с момента его создания. CFS был создан Инго Молнаром в течение краткого выплеска творческой энергии, результатом которого стала заплата для ядра размером 100K, написанная за 62 часа.*

[KL88] «A Fair Share Scheduler» by J. Kay and P. Lauder. CACM, Volume 31 Issue 1, January 1988. *Одно из первых справочных руководств по равномерному планировщику.*

[WW94] «Lottery Scheduling: Flexible Proportional-Share Resource Management» by Carl A. Waldspurger and William E. Weihl. OSDI '94, November 1994.

Основополагающая работа по лотерейному планированию, которая возродила интерес сообщества системщиков к планированию, справедливому распределению ресурсов и удивительным возможностям простых рандомизированных алгоритмов.

[W95] «Lottery and Stride Scheduling: Flexible Proportional-Share Resource Management» by Carl A. Waldspurger. Ph.D. Thesis, MIT, 1995. *Удостоенная премии диссертация Вальдспургера, в которой описывается лотерейное и шаговое планирование. Всякому, кто решит писать диссертацию на звание доктора философии, нужно иметь пример, показывающий, к чему стремиться; эта работа – один из достойных примеров.*

[W02] «Memory Resource Management in VMware ESX Server» by Carl A. Waldspurger. OSDI '02, Boston, Massachusetts. *Эта работа посвящена управлению виртуальной памятью в гипервизорах. Она легко читается и содержит немало интереснейших идей об этом новом типе управления памятью.*

Домашнее задание (эмуляция)

Программа `lottery.py` поможет вам понять, как работает лотерейный планировщик. Детали см. в файле `README`.

Вопросы

1. Вычислите решения для эмуляции с тремя заданиями и начальными значениями 1, 2 и 3.
2. Теперь выполните программу с двумя конкретными заданиями: каждое продолжается 10 единиц времени, но у одного (задание 0) имеется только 1 билет, а у другого (задание 1) – 100 (запуск с флагами `-l 10:1,10:100`). Что происходит, когда число билетов настолько сильно разнится? Запускается ли вообще задание 0 до завершения задания 1? Как часто? Вообще, как такая несбалансированность влияет на поведение лотерейного планирования?
3. При выполнении двух заданий продолжительностью 100 с равным числом билетов, по 100 (`-l 100:100,100:100`), насколько неравномерным будет планировщик? Прогоните эмулятор с разными начальными значениями для нахождения (вероятного) ответа; будем считать, что степень неравномерности определяется тем, насколько одно задание завершается быстрее другого.
4. Как изменится ответ на предыдущий вопрос при увеличении временного кванта (флаг `-q`)?
5. Можете ли вы придумать варианты графика, приведенного в этой главе? Что еще имеет смысл исследовать? Как будет выглядеть график для шагового планировщика?

Глава 10

Планирование в многопроцессорных системах (материал повышенной сложности)

В этой главе рассматриваются основы **планирования в многопроцессорных системах**. Поскольку это материал повышенной сложности, быть может, будет разумнее прочитать его после знакомства с конкурентностью (вторым из «простых элементов», изучаемых в этой книге).

Многие годы **многопроцессорные системы** (или **мультипроцессоры**) занимали место в верхней части спектра вычислительной техники, но теперь вошли в обиход и проложили путь в настольные компьютеры, ноутбуки и даже мобильные устройства. Причиной такого повсеместного распространения стало появление **многоядерных** процессоров, в которых несколько процессорных ядер размещено на одном кристалле; такие кристаллы стали популярны, когда архитекторы компьютеров столкнулись с трудностями при попытке ускорить работу одного процессора, не слишком увеличивая энергопотребление. В результате все мы получили в свое распоряжение несколько CPU, что, конечно, неплохо.

Разумеется, с появлением нескольких процессоров возникли новые трудности. Основная из них заключается в том, что типичное приложение (т. е. написанная вами программа на C) работает только с одним процессором, а добавление CPU не сделает ее быстрее. Чтобы решить эту проблему, приложение придется **распараллелить**, например с помощью **поток**ов (они подробно обсуждаются во второй части книги). Многопоточные приложения могут распределить работу между несколькими CPU и, стало быть, будут работать тем быстрее, чем больше в машине процессоров.

ОТСТУПЛЕНИЕ: ГЛАВЫ ПОВЫШЕННОЙ СЛОЖНОСТИ

Для полного понимания глав повышенной сложности необходимо знакомство с материалом из разных частей книги, но расположены они до того, как этот материал изложен. Например, эту главу о планировании в многопроцессорных системах было бы гораздо логичнее читать, уже зная о конкурентности, однако ее место все же в той части книги, которая посвящена виртуализации вообще и планированию процессора в частности. Поэтому рекомендуется читать такие главы не по порядку, в данном случае – после ознакомления со второй частью книги.

Помимо приложений, новая проблема возникает и в самой операционной системе, а именно (неудивительно!) в сфере **планирования мультипроцессора**. До сих пор мы обсуждали принципы планирования в однопроцессорной системе, а как распространить эти идеи на случай нескольких CPU? Какие новые проблемы придется преодолеть?

СУЩЕСТВО ПРОБЛЕМЫ: КАК ПЛАНИРОВАТЬ ЗАДАНИЯ НА НЕСКОЛЬКИХ CPU

Как ОС должна планировать задания при наличии нескольких CPU? Какие возникают новые проблемы? Будут ли работать старые подходы или необходимы новые идеи?

10.1. ВВЕДЕНИЕ: МНОГОПРОЦЕССОРНАЯ АРХИТЕКТУРА

Чтобы разобраться в новых вопросах, которые поднимает планирование мультипроцессора, необходимо понять фундаментальное различие между оборудованием с одним и несколькими CPU. Суть его – в использовании аппаратных кешей (см. рис. 10.1) и в том, как именно данные разделяются между несколькими процессорами. Мы обсудим этот вопрос в общем плане, а детали можно найти в другом месте [CSG99], например на курсе по компьютерной архитектуре для магистрантов.

В системе с одним CPU имеется иерархия аппаратных **кешей**, которые помогают процессору быстрее выполнять программы. Кеш – это быстрая память небольшого размера, в которой (в общем случае) хранятся копии *востребованных* данных, находящихся в основной памяти системы. Основная память содержит *все* данные, но доступ к ней медленнее. Благодаря хранению часто используемых данных в кеше система может представить большую, но медленную память как быструю.

В качестве примера рассмотрим программу, которая выполняет команду загрузки, чтобы выбрать некоторое значение из памяти, и простую систему

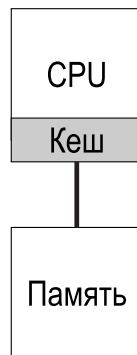


Рис. 10.1 ❖ Один CPU с кешем

с одними CPU; в системе имеется небольшой кеш (скажем, 64 КБ) и большая основная память. При первом выполнении команды загрузки данные находятся в основной памяти, поэтому выборка занимает много времени (быть может, десятки или даже сотни наносекунд). Процессор, предвидя, что данные вскоре могут понадобиться снова, помещает копию загруженных данных в свой кеш. Если впоследствии программе понадобится выбрать те же данные, то CPU сначала проверит, нет ли их в кеше. Если есть, то на выборку данных уходит гораздо меньше времени (скажем, несколько наносекунд), поэтому и программа работает быстрее.

Таким образом, в основе кешей лежит концепция **локальности**, каковая бывает двух видов: **временная** и **пространственная**. Идея временной локальности состоит в том, что если было обращение к какому-то элементу данных, то весьма вероятно, что он понадобится в ближайшем будущем; представьте себе переменные и даже команды, к которым снова и снова производятся обращения в цикле. Идея пространственной локальности состоит в том, что если программа обратилась к элементу данных по адресу x , то, вероятно, ей понадобятся также данные, расположенные рядом с x ; представьте, что программа перебирает элементы массива или выполняет команды одну за другой. Поскольку такого рода локальность встречается во многих программах, аппаратные системы могут строить разумные гипотезы относительно того, какие данные помещать в кеш, и потому работают хорошо.

А теперь переходим к сложным вещам: что будет, если в системе несколько процессоров с одной общей памятью, как показано на рис. 10.2?

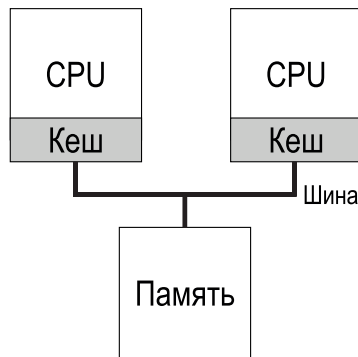


Рис. 10.2 ❖ Два CPU с отдельными кешами и общей памятью

Как выясняется, кеширование при наличии нескольких CPU гораздо сложнее. Допустим, к примеру, что программа, работающая на CPU 1, читает элемент данных (имеющий значение D) по адресу A ; поскольку данные отсутствуют в кеше CPU 1, система выбирает их из основной памяти и получает значение D . Затем программа модифицирует значение по адресу A , но при этом изменяется (становится равным D') только значение в кеше; записывать данные в основную память долго, поэтому система обычно откладывает это на потом. Далее предположим, что ОС решила приостановить работу

программы и возобновить ее на CPU 2. Программа снова читает значение по адресу *A*; оно отсутствует в кеше CPU 2, поэтому система выбирает его из основной памяти – и получает старое значение *D* вместо правильного *D'*. Незадача!

Эта общая проблема получила название проблемы **когерентности кешей**, ей посвящена обширная литература, в которой описываются различные нюансы решения [SHW11]. Мы опустим обсуждение всех этих тонкостей, а отметим лишь некоторые существенные моменты; если хотите узнать больше, запишитесь на курс по архитектуре компьютеров (или на три таких курса).

В основном решение аппаратное: наблюдая за доступом к памяти, оборудование может гарантировать, что все делается «правильно», т. е. все процессоры видят общую память одинаково. Один из способов достичь этого в системе с шинной архитектурой (описанной выше) – воспользоваться давно известной техникой **слежения за шиной** (bus snooping) [G83]; каждый кеш следит за операциями обновления памяти, наблюдая за шиной, связывающей кеш с основной памятью. Когда CPU видит обновление элемента данных, которые хранятся в его кеше, он либо **объявляет недействительной** свою копию (т. е. удаляет ее из кеша), либо **обновляет** ее (т. е. помещает новое значение и в кеш тоже). Для кешей с отложенной записью все немного сложнее (потому что запись в основную память становится видна позже), но принцип работы вы поняли.

10.2. НЕ ЗАБЫВАЙТЕ О СИНХРОНИЗАЦИИ

Учитывая, что всю работу по обеспечению когерентности берут на себя кеши, должны ли программы (или сама ОС) беспокоиться о чем-то при обращении к общим данным? К сожалению, да, и это подробно описано во второй части книги. Сейчас мы не будем вдаваться в детали, но кратко упомянем некоторые основные идеи (предполагая, что вы знакомы с темой конкурентности).

При доступе к элементам или структурам данных (и в особенности при их обновлении), разделяемым между процессорами, следует использовать примитивы взаимного исключения (например, блокировки), чтобы гарантировать правильность (другие подходы, например разработка **безблокировочных** структур данных, сложны и используются в особых случаях; детали см. в главе, посвященной взаимоблокировкам). Например, предположим, что имеется разделяемая очередь, к которой одновременно обращается несколько процессоров. Если бы блокировок не было, то конкурентное добавление или удаление элементов работало бы некорректно, несмотря на наличие протоколов когерентности кешей; блокировка необходима, чтобы атомарно перевести структуру данных в новое состояние.

Чтобы сделать рассмотрение более предметным, возьмем код, который удаляет элемент из разделяемого связанного списка (рис. 10.3). Допустим, что потоки, работающие на двух процессорах, входят в эту функцию одновременно. Когда поток 1 выполняет первую строку, он сохраняет текущее значение *head* в своей переменной *tmp*; когда поток 2 тоже выполнит первую строку, то же

самое значение `head` будет сохранено в его переменной `tmp` (память для этой переменной выделена в стеке, а у каждого потока свой собственный стек). Таким образом, вместо того чтобы удалить два первых элемента, каждый поток попытается удалить один и тот же элемент из начала списка, что может приводить к разнообразным проблемам (например, двойное освобождение памяти, на которую указывает `head`, или возврат одних и тех же данных дважды).

```

1 typedef struct __Node_t {
2     int          value;
3     struct __Node_t *next;
4 } Node_t;
5
6 int List_Pop() {
7     Node_t *tmp = head;    // запомнить старое начало ...
8     int value = head->value; // ... и хранящееся там значение
9     head = head->next;      // сдвинуть указатель вперед
10    free(tmp);              // освободить старое начало
11    return value;           // вернуть хранившееся там значение
12 }
```

Рис. 10.3 ❖ Простой код удаления элемента из списка

Чтобы исправить ситуацию, нужно, конечно, воспользоваться **блокировкой**. В данном случае достаточно создать простой мьютекс (например, `pthread_mutex_t m;`) и добавить вызов `lock(&m)` в начало функции и `unlock(&m)` в ее конец, тогда код будет работать, как задумано. К сожалению, как мы позже увидим, у этого решения есть свои проблемы, в т. ч. связанные с производительностью. Именно, при увеличении количества процессоров доступ к синхронизированным разделяемым структурам данных может оказаться весьма медленным.

10.3. Последняя проблема:

ПРИВЯЗКА К ПРОЦЕССОРУ

Последняя проблема при построении планировщика мультипроцессора – **привязка к процессору** [TTG95]. Суть ее проста: когда процесс работает на конкретном CPU, он накапливает в кешах этого CPU (и в буферах TLB) большой объем состояния. При следующем планировании было бы выгодно выполнять его на том же CPU, поскольку процесс будет работать быстрее, если часть его состояния уже присутствует в кешах. Если же процесс каждый раз запускается на разных CPU, то производительность снижается, потому что приходится заново загружать состояние (заметим, что благодаря аппаратным протоколам когерентности кешей процесс и на другом CPU будет работать правильно, только медленнее). Таким образом, планировщик мультипроцессора должен учитывать привязку к процессору – быть может, предпочитая запускать процесс на том же CPU, если это возможно.

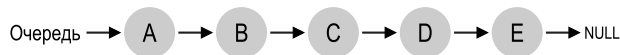
10.4. ПЛАНИРОВАНИЕ С ОДНОЙ ОЧЕРЕДЬЮ

После этого краткого введения обсудим, как построить планировщик в многопроцессорной системе. Самый простой подход – воспользоваться инфраструктурой, созданной для одного процессора, поместив все подлежащие планированию задачи в одну очередь; это называется **планированием мультипроцессора с одной очередью** (single-queue multiprocessor scheduling – **SQMS**). Преимущество такого подхода в простоте; не надо больших усилий, чтобы адаптировать существующую политику выбора следующего задания к работе на нескольких CPU (если процессоров два, то можно, например, выбирать два лучших задания).

Однако у SQMS есть очевидные недостатки. Первая проблема – отсутствие **масштабируемости**. Чтобы планировщик правильно работал для нескольких CPU, пришлось бы включить в код какую-то форму **блокировки**, как описано выше. Блокировки гарантируют, что когда код SQMS обращается к единственной очереди (скажем, чтобы найти следующую подлежащую выполнению задачу), результат будет правильным.

К несчастью, блокировки могут сильно снижать производительность, особенно если количество процессоров в системе растет [A91]. По мере того как увеличивается конкуренция за единственную блокировку, система тратит все больше времени на накладные расходы и все меньше на полезную работу (примечание: неплохо было бы включить в будущем реальное измерение этих расходов).

Вторая проблема SQMS – привязка к процессору. Предположим, к примеру, что имеется пять заданий (A, B, C, D, E) и четыре процессора. Очередь планировщика выглядит следующим образом:



Со временем, в предположении, что каждое задание работает в течение временного кванта, а затем выбирается другое задание, может сложиться такая картина назначения процессоров задачам:

CPU 0	A	E	D	C	B	... (повторяется) ...
CPU 1	B	A	E	D	C	... (повторяется) ...
CPU 2	C	B	A	E	D	... (повторяется) ...
CPU 3	D	C	B	A	E	... (повторяется) ...

Поскольку каждый CPU просто выбирает следующее задание из глобальной разделяемой очереди, то задания перемещаются с одного CPU на другой,

т. е. происходит ровно то, чего следует избегать с точки зрения привязки к процессору.

Чтобы справиться с этой проблемой, большинство SQMS-планировщиков включают какой-то механизм привязки, повышающий вероятность того, что процесс продолжит работу на прежнем CPU. Именно, можно обеспечить привязку для некоторых задач, а другие перемещать с целью балансировки нагрузки. Например, те же пять заданий можно было бы планировать следующим образом:

CPU 0	A	E	A	A	A	... (повторяется) ...
CPU 1	B	B	E	B	B	... (повторяется) ...
CPU 2	C	C	C	E	C	... (повторяется) ...
CPU 3	D	D	D	D	E	... (повторяется) ...

В этом случае задания А–D не перемещаются между процессорами, а **мигрирует** только задание Е, так что для большинства заданий привязка сохраняется. Через некоторое время можно было бы сменить мигрирующее задание, чтобы обеспечить хоть какую-то справедливость привязки. Но реализация такой схемы может оказаться сложной.

Итак, мы видим, что у подхода SQMS есть сильные и слабые стороны. Его нетрудно реализовать, учитывая, что уже имеется планировщик с одной очередью. Однако он плохо масштабируется (из-за накладных расходов на синхронизацию) и слабо приспособлен к сохранению привязки к процессору.

10.5. ПЛАНИРОВАНИЕ С НЕСКОЛЬКИМИ ОЧЕРЕДЯМИ

Из-за проблем, свойственных планировщикам с одной очередью, в некоторых системах предпочтение отдано нескольким очередям, например по одной на каждый CPU. Мы называем такой подход **планированием мультипроцессора с несколькими очередями** (multi-queue multiprocessor scheduling – MQMS).

В случае MQMS имеется несколько очередей планирования. Как правило, каждая очередь обслуживается согласно одной и той же дисциплине, например циклически, хотя, конечно, можно использовать любой алгоритм. Задание, поступающее в систему, помещается ровно в одну очередь, исходя из некоторой эвристики (например, случайно или в очередь, где меньше заданий). Затем оно планируется по существу независимо, что позволяет избежать проблем разделения данных и синхронизации, которые преследуют планировщик с одной очередью.

Предположим, например, что в систему с двумя CPU (обозначим их CPU 0 и CPU 1) поступает несколько заданий: A, B, C, D. Поскольку теперь с каждым CPU связана своя очередь планирования, ОС должна решить, в какую очередь поместить каждое задание. Она может поступить следующим образом:



Теперь для выполнения на каждом CPU можно выбрать одно из двух заданий. Если используется **циклический** алгоритм, то система может запланировать задания, как показано ниже:

CPU 0	A	A	C	C	A	A	C	C	A	A	C	C	...
CPU 1	B	B	D	D	B	B	D	D	B	B	D	D	...

У MQMS есть очевидное преимущество перед SQMS: он, по определению, лучше масштабируется. С ростом количества процессоров увеличивается и количество очередей, поэтому блокировка и конкуренция за кеш не должны стать серьезной проблемой. Кроме того, MQMS автоматически обеспечивает привязку к кешу – задания остаются на одном и том же CPU, поэтому пользуются всеми благами кеширования.

Но если вы были внимательны, то, наверное, заметили новую проблему, внутренне присущую подходу с несколькими очередями: **несбалансированность нагрузки**. Пусть все так же, как и раньше (четыре задания, два CPU), но одно задание (скажем, C) завершается. Теперь имеются такие две очереди планирования:



Если применить к каждой очереди политику циклического планирования, то получится такой результат:

CPU 0	A	A	A	A	A	A	A	A	A	A	A	...	
CPU 1	B	B	D	D	B	B	D	D	B	B	D	D	...

Как видим, A получает в два раза больше процессорного времени, чем B и D, – не этого мы хотели. Хуже того, представим, что завершились оба задания A и C, а остались только B и D. Тогда очереди планирования будут выглядеть так:



В итоге CPU 0 вообще простаивает (здесь должна звучать драматическая зловеющая музыка)! А хронология использования CPU выглядит совсем грустно:

CPU 0

CPU 1



Так что же делать бедному планировщику мультипроцессора с несколькими очередями? Как решить предательскую проблему несбалансированной нагрузки и одолеть злые силы... Десептиконов¹? Как нам перестать задавать вопросы, которые никаким боком не связаны с темой этой во всех прочих отношениях замечательной книги?

СУЩЕСТВО ПРОБЛЕМЫ: КАК СПРАВИТЬСЯ С НЕСБАЛАНСИРОВАННОЙ НАГРУЗКОЙ

Как планировщик мультипроцессора с несколькими очередями должен обрабатывать несбалансированную нагрузку, чтобы достичь желаемых целей планирования?

Очевидный ответ – перебрасывать задания с одного процессора на другой; эта техника называется **миграцией**. С помощью миграции заданий можно добиться по-настоящему сбалансированной нагрузки.

Чтобы стало яснее, рассмотрим два примера. Мы остановились на ситуации, когда один CPU простаивает, а другой выполняет задачи.



В этом случае желаемая миграция проста: ОС должна переместить *B* или *D* на CPU 0. В результате миграции всего одного задания баланс восстановлен и все довольны.

Более сложный случай возникает в предыдущем примере, когда на CPU 0 осталось только задание *A*, а на CPU 1 попеременно выполняются *B* и *D*:



¹ Мало кому известно, что их родная планета Кибертрон была уничтожена в результате неудачных решений, принятых планировщиком. И да будет это первым и последним упоминанием трансформеров в этой книге, за которое мы приносим искренние извинения.

В этом случае одной миграцией проблему не решить. И что же делать? Увы, придется постоянно перемещать одно или несколько заданий с одного процессора на другой. Одно из возможных решений – переключать задания, как показано на рисунке ниже. Сначала на CPU 0 работает только *A*, а *B* и *D* сменяют друг друга на CPU 1. После нескольких временных квантов *B* перемещается на CPU 0, где конкурирует с *A*, а *D* на протяжении нескольких квантов наслаждается одиночеством на CPU 1. Нагрузка, таким образом, сбалансирована:

CPU 0	A	A	A	A	B	A	B	A	B	B	B	B	...
CPU 1	B	D	B	D	D	D	D	D	A	D	A	D	...

Конечно, возможно много других вариантов миграции. Но вот в чем вопрос: почему система должна решить, что стоит вообще прибегнуть к миграции?

Один из известных подходов – техника **заимствования работ** [FLR98]. В этом случае (исходная) очередь, в которой мало заданий, время от времени заглядывает в другую очередь (целевую), чтобы узнать, сильно ли она заполнена. Если в целевой очереди значительно больше заданий, чем в исходной, то исходная очередь «заимствует» у целевой одно или несколько заданий, чтобы сделать нагрузку более сбалансированной.

Разумеется, сразу возникает конфликт целей. Если заглядывать в другие очереди слишком часто, то возрастут накладные расходы, а значит, пострадает масштабируемость, что противоречит самой идее планирования с несколькими очередями! Если же заглядывать в другие очереди редко, то возникает риск значительной несбалансированности. Нахождение правильного компромисса, как всегда при проектировании системной политики, остается черной магией.

10.6. Планировщики мультипроцессоров в Linux

Интересно, что в сообществе Linux нет единого подхода к построению планировщика мультипроцессора. С течением времени было создано три разных планировщика: $O(1)$, вполне равномерный планировщик (CFS) и планировщик BF (BFS)¹. В диссертации Михина приведен отличный обзор плюсов и минусов указанных планировщиков [M11]; мы же ограничимся только самыми основными сведениями.

¹ Посмотрите, что означает аббревиатура BF, самостоятельно, но предупреждаем, эта информация не для слабонервных.

O(1) и CFS – планировщики с несколькими очередями, а BFS – с одной очередью. Как видим, оба подхода работоспособны. Конечно, эти планировщики во многих деталях различаются. Например, O(1) – приоритетный планировщик (в этом отношении он похож на обсуждавшийся выше MLFQ): приоритеты процессов изменяются со временем, а затем выбираются процессы с наивысшим приоритетом, удовлетворяющие различным целям планирования; особое внимание уделяется интерактивности. Напротив, CFS – детерминированный пропорциональный планировщик (больше напоминающий шаговое планирование). BFS, единственный из трех планировщик с одной очередью, тоже пропорциональный, но основан на более сложной схеме «сначала с самым ранним виртуальным крайним сроком» (Earliest Eligible Virtual Deadline First – EEVDF) [SA96]. Можете прочитать об этих современных алгоритмах самостоятельно, теперь вы уже способны понять, как они работают!

10.7. РЕЗЮМЕ

Мы рассмотрели различные подходы к планированию в многопроцессорных системах. Решение с одной очередью (SQMS) довольно просто реализуется и хорошо балансирует нагрузку, но с трудом масштабируется на большое число процессоров и плохо согласуется с привязкой к процессору. Решение с несколькими очередями (MQMS) масштабируется лучше и обеспечивает привязку к процессору, но при этом нагрузка может оказаться несбалансированной, а реализация сложнее. На вопрос, какой подход выбрать, нет простого ответа: создание планировщика общего назначения остается чрезвычайно трудной задачей, потому что даже небольшое изменение кода может стать причиной значительного изменения поведения. Беритесь за это дело, только если точно знаете, что делаете, или, по крайней мере, получаете за это хорошие деньги.

Литература

[A90] «The Performance of Spin Lock Alternatives for Shared-Memory Multiprocessors» by Thomas E. Anderson. IEEE TPDS Volume 1:1, January 1990. *Классическая работа о том, как масштабируются – или не масштабируются – различные схемы блокировки. Написана Томом Андерсоном, хорошо известным специалистом по системам и сетям. И добавим, автором прекрасного учебника по ОС.*

[B+10] «An Analysis of Linux Scalability to Many Cores Abstract» by Silas Boyd-Wickizer, Austin T. Clements, Yandong Mao, Aleksey Pesterev, M. Frans Kaashoek, Robert Morris, Nickolai Zeldovich. OSDI '10, Vancouver, Canada, October 2010. *Потрясающая современная работа по трудностям масштабирования Linux на большое число ядер.*

[CSG99] «Parallel Computer Architecture: A Hardware/Software Approach» by David E. Culler, Jaswinder Pal Singh, and Anoop Gupta. Morgan Kaufmann, 1999.

Сокровищница, полная подробной информации о параллельных машинах и алгоритмах. На обложке приведено юмористическое замечание Марка Хилла о том, что книга содержит больше информации, чем большинство научных статей.

[FLR98] «The Implementation of the Cilk-5 Multithreaded Language» by Matteo Frigo, Charles E. Leiserson, Keith Randall. PLDI '98, Montreal, Canada, June 1998. *Cilk – облегченный язык и среда выполнения для написания параллельных программ, а также отличный пример реализации заимствования работ.*

[G83] «Using Cache Memory To Reduce Processor-Memory Traffic» by James R. Goodman. ISCA '83, Stockholm, Sweden, June 1983. *Основополагающая работа по технике наблюдения за шиной, т. е. построения протокола когерентности кешей на основе анализа запросов, присутствующих на шине. Многолетние исследования Гудмана в Висконсинском университете – кладезь мудрости, это лишь один пример.*

[M11] «Towards Transparent CPU Scheduling» by Joseph T. Meehan. Doctoral Dissertation at University of Wisconsin–Madison, 2011. *В этой диссертации содержится масса деталей, касающихся работы планировщиков мультипроцессоров в современных версиях Linux. Потрясающе! Но, будучи научными руководителями Джо, мы, возможно, несколько пристрастны.*

[SHW11] «A Primer on Memory Consistency and Cache Coherence» by Daniel J. Sorin, Mark D. Hill, and David A. Wood. Synthesis Lectures in Computer Architecture. Morgan and Claypool Publishers, May 2011. *Авторитетный обзор согласованности памяти и кеширования в многопроцессорных системах. Обязательное чтение для любого, кому нужна полная информация по этой теме.*

[SA96] «Earliest Eligible Virtual Deadline First: A Flexible and Accurate Mechanism for Proportional Share Resource Allocation» by Ion Stoica and Hussein Abdel-Wahab. Technical Report TR-95-22, Old Dominion University, 1996. *Технический отчет по этой интереснейшей идее планирования, написанный Ионом Стойка, ныне профессором Калифорнийского университета в Беркли и всемирно признанным специалистом по сетям, распределенным системам и многим другим вещам.*

[TTG95] «Evaluating the Performance of Cache-Affinity Scheduling in Shared-Memory Multiprocessors» by Josep Torrellas, Andrew Tucker, Anoop Gupta. Journal of Parallel and Distributed Computing, Volume 24:2, February 1995. *Это не первая статья по данной теме, но в ней есть отсылки к более ранним работам. При этом она легче читается и имеет более практическую направленность, чем некоторые из предшествующих работ по анализу планирования с очередями.*

Домашнее задание (эмуляция)

В этом домашнем задании вы будете использовать программу `multi.py` для эмуляции планировщика мультипроцессора, что позволит узнать больше о некоторых его особенностях. Дополнительные сведения об эмуляторе и его параметрах см. в файле `README`.

Вопросы

1. Для начала научимся использовать эмулятор для изучения методов построения эффективного планировщика мультипроцессора. Первой попыткой будет запуск одного задания продолжительностью 30 с размером рабочего набора 200. Выполните это задание (которое будем обозначать «а») на одном эмулированном CPU следующим образом: `./multi.py -n 1 -L a:30:200`. Сколько времени понадобится для его завершения? Добавьте флаг `-c`, чтобы увидеть ответ, и флаг `-t`, чтобы увидеть трассу задания тик за тиком и посмотреть, как оно планируется.
2. Теперь увеличим размер кеша, чтобы весь рабочий набор задания (`size=200`) помещался в кеш (по умолчанию его размер равен 100); запустите задание командой `./multi.py -n 1 -L a:30:200 -M 300`. Попробуйте предсказать, как быстро будет работать задание, когда все данные находятся в кеше (указание: вспомните о ключевом параметре скорости прогрева, который задается флагом `-r`). Проверьте себя, запустив команду с флагом `-c`.
3. У программы `multi.py` есть очень полезная возможность: детально изучить происходящее с помощью различных флагов трассировки. Запустите ту же эмуляцию, что и выше, с включенной трассировкой оставшегося времени (`-T`). Этот флаг показывает как задание, планируемое на каждом временном шаге, так и время выполнения, оставшееся ему после каждого тика. Какие у вас мысли по поводу убывания значений во втором столбце?
4. Добавим еще один флаг трассировки, `-C`, который показывает состояние каждого кеша CPU для каждого задания. Для каждого задания показывается либо пробел (если кеш для него еще не прогрет), либо `w` (если кеш прогрет). В какой момент прогревается кеш для задания «а» в этом простом примере? Что будет, если увеличить или уменьшить параметр времени прогрева (`-w`) по сравнению со значением по умолчанию?
5. Теперь вы уже неплохо понимаете, как эмулятор работает для одного задания на одном CPU. Но ведь это глава о планировании мультипроцессора! Так давайте запустим несколько заданий. Конкретно, запустите три задания в двухпроцессорной системе, т. е. введите команду `./multi.py -n 2 -L a:100:100,b:100:50,c:100:50`. Сможете ли вы предсказать, сколько это займет времени в предположении, что используется централизованный циклический планировщик? Проверьте себя с помощью флага `-c`, а затем углубитесь в детали, задав флаг `-t` для просмотра всех шагов планирования, и флаг `-C`, чтобы узнать, эффективно ли прогревается кеш для этих заданий. Что вы заметили?
6. Теперь воспользуемся средствами управления для изучения **привязки к процессору**, описанной в этой главе. Для этого нам понадобится флаг `-A`, который позволяет ограничить набор процессоров, которые планировщик может предоставить данной задаче. В данном случае разрешим помещать задачи «b» и «c» на CPU 1, а задачу «а» только на CPU 0. Для этого нужно ввести команду `./multi.py -n 2 -L a:100:100,b:100:50,c:100:50 -A a:0,b:1,c:1`. Не забудьте включить различные флаги, помогающие понять, что происходит! Сможете ли вы предсказать, сколько времени уйдет на

выполнение этой версии? Почему она работает лучше? А при других сочетаниях заданий «а», «b» и «с» с двумя процессорами получится быстрее или медленнее?

7. Интересный аспект кеширования мультипроцессоров – возможность получить ускорение больше ожидаемого при использовании нескольких CPU (и их кешей) по сравнению с работой на одном процессоре. Точнее, если задания запускаются на N процессорах, то иногда удастся ускорить работу больше, чем в N раз; это называется **сверхлинейным ускорением**. Для экспериментов создадим три задания с описанием -L a:100:100,b:100:100,c:100:100 и небольшой кеш (-M 50). Запустите эти задания в системе с 1, 2 и 3 процессорами (-n 1, -n 2, -n 3). Теперь увеличьте размер кеша каждого процессора до 100. Как изменяется производительность при увеличении числа CPU? Воспользуйтесь флагом -с для проверки своих гипотез и другими флагами, чтобы получить дополнительную информацию.
8. Еще один достойный внимания аспект эмулятора – флаг -р, задающий режим планирования с отдельными очередями для каждого процессора. Прогоните пример с двумя CPU и такой смесью заданий: -L a:100:100,b:100:50,c:100:50. Какой результат дает этот режим по сравнению с управляемыми вручную ограничениями на привязку, которые мы изучали выше? Как изменится производительность, если увеличить или уменьшить «интервал заглядывания» (-P)? Насколько хорошо работает этот подход при увеличении числа процессоров?
9. Наконец, сгенерируйте случайные рабочие нагрузки и попробуйте предсказать их производительность при разном числе процессоров, разных размерах кешей и параметрах планирования. Если получится, то вы скоро станете гуру планирования мультипроцессора, что весьма почетно. Удачи!

Глава 11

.....

Заключительный диалог о виртуализации процессора

Профессор. Ну, ученик, научился ли ты чему-нибудь?

Студент. Э, профессор, это провокационный вопрос. Полагаю, Вы хотите получить только ответ «да» и никакого другого.

Профессор. Это правда. Но все-таки я хочу услышать честный ответ. Хватит уже подкалывать профессора, ладно?

Студент. Хорошо, хорошо. Думаю, что чему-то я все же научился. Во-первых, я кое-что узнал о том, как ОС виртуализирует процессор. Для этого мне пришлось разобраться в ряде важных **механизмов**: системные прерывания и их обработчики, прерывания от таймера и как ОС на пару с оборудованием должны аккуратно сохранять и восстанавливать состояние при переключении процессов.

Профессор. Отлично, отлично!

Студент. Правда, все эти взаимодействия сложноваты. Как мне разобраться в этом лучше?

Профессор. Хороший вопрос. Я так думаю, что нужно пробовать, другого способа нет. Если будешь только читать, то по-настоящему никогда не поймешь. Выполняй учебные проекты, и держу пари, что в конце концов все станет на свои места.

Студент. Звучит неплохо. Что еще Вы хотите услышать от меня?

Профессор. Получил ли ты представление о философии ОС, понял ли ты, как она устроена?

Студент. Гм... думаю, что да. Мне ОС представляется немного параноидальной. Она желает все время контролировать машину. С одной стороны, она хочет, чтобы программа работала максимально эффективно (отсюда и вся эта байда с **ограниченным прямым выполнением**), а с другой – чтобы всегда можно было прикрикнуть «Постой, охолони, дружок!», если процесс ошибочный или вредоносный. Паранойя правит бал и, само собой, оставляет за ОС

контроль над машиной. Быть может, именно по этой причине ОС иногда называют диспетчером ресурсов.

Профессор. Отлично – кажется, ты начинаешь понимать, откуда ноги растут! Я доволен.

Студент. Спасибо.

Профессор. Ну, а как насчет политик поверх механизмов – вынес что-нибудь интересное?

Студент. Кое-что вынес, конечно. Пожалуй, малость очевидное, но и очевидное тоже хорошо. Например, идею о том, чтобы помещать короткие задания в начало очереди, – я знал, что это хорошая мысль, еще с тех пор, когда покупал жвачку в магазине, а у мужика передо мной не работала кредитка. Вот уж он-то точно не был коротким заданием, скажу я вам.

Профессор. Как-то грубовато звучит по отношению к бедному парню. Еще что?

Студент. Ну, что можно построить умный планировщик, который пытается совместить SJF и RR в одном лице, этот MLFQ – симпатичная штучка. Но сделать реальный планировщик, похоже, трудное дело.

Профессор. Не похоже, а так и есть. Потому-то до сих пор и спорят, какой планировщик использовать, об этом, например, бодания в Linux по поводу CFS, BFS и O(1). Нет-нет, я не стану произносить полное название BFS, и не проси.

Студент. Да я и не прошу. Эти битвы по поводу политик, наверное, могут продолжаться вечно. А правильный отчет есть?

Профессор. Пожалуй, нет. Ведь даже наши метрики противоречивы: если планировщик хорош с точки зрения обратного времени, то он дает плохое время отклика, и наоборот. Как говорил Лэмпсон, цель, наверное, не в том, чтобы найти лучшее решение, а в том, чтобы избежать катастрофы.

Студент. Как-то невесело.

Профессор. Хорошее инженерное решение часто бывает таким. Но оно же внушает надежду! Все дело в том, как на это посмотреть. Лично я думаю, что прагматизм – вещь хорошая, прагматик понимает, что не у каждой проблемы есть простое и идеальное решение. Что-то еще тебя зацепило?

Студент. Мне понравилась идея обыгрывания планировщика, может, я займусь этим в следующий раз, когда буду запускать задание на Amazon EC2. Вдруг получится украсть пару-другую тактов у какого-нибудь ничего не подозревающего (и, главное, не знающего об ОС) пользователя!

Профессор. Э, да я, кажется, породил монстра! Я не хочу, чтобы меня называли профессором Франкенштейном, знаешь ли.

Студент. Но разве не в этом идея? Заинтересовать нас чем-то таким, чтобы мы потом погрузились в это с головой? Зажечь факел и все такое?

Профессор. Ну, да. Но я не думал, что это сработает таким образом!

Глава 12

Диалог о виртуализации памяти

Студент. Ну, что, мы закончили с виртуализацией?

Профессор. Нет!

Студент. Да не надо так нервничать, я просто спросил. От студентов ведь именно этого и ждешь, разве не так?

Профессор. Да, профессора всегда так говорят, но на самом деле они имеют в виду другое: задавай вопросы, **но** только хорошие **и** если ты уже немного над ними поразмыслил.

Студент. Эх, опустили на грешную землю.

Профессор. Миссия выполнена. Так или иначе, с виртуализацией мы не закончили! Мы только узнали, как виртуализировать процессор, но в шкафу нас поджидает еще один гигантский монстр – память. Виртуализация памяти – сложное дело, оно потребует от нас понимания многих тонких деталей взаимодействия оборудования и ОС.

Студент. Круто! А почему так трудно-то?

Профессор. Так деталей много, и тебе придется все уложить в голову, чтобы построить умозрительную модель. Мы начнем с очень простых вещей и постепенно будем добавлять все более сложные, включая буфер ассоциативной трансляции и многоуровневые таблицы страниц. И в конечном итоге опишем, как устроен современный полнофункциональный диспетчер виртуальной памяти.

Студент. Супер! А какие-нибудь советы бедному студенту, по самую макушку набитому всей этой информацией и вечно недосыпающему?

Профессор. Ну, насчет недосыпа все просто: больше спать (и меньше шастать по кабакам). А насчет понимания виртуальной памяти начни вот с чего: **любой адрес, генерируемый пользовательской программой, виртуальный.** ОС лишь создает иллюзию для каждого процесса, будто он располагает огромной собственной памятью; благодаря поддержке со стороны оборудования ОС преобразует эти воображаемые виртуальные адреса в реальные физические и таким образом находит требуемую информацию.

Студент. ОК, думаю, я в состоянии это запомнить... (себе под нос) любой адрес в пользовательской программе виртуальный, любой адрес в пользовательской программе виртуальный, любой...

Профессор. Что ты там бормочешь?

Студент. Да так, ничего... (неловкая пауза)... Да, а почему ОС снова хочет создавать иллюзии?

Профессор. В основном ради **простоты использования**: ОС создает у каждой программы впечатление, будто ей доступно большое непрерывное **адресное пространство**, в котором можно разместить код и данные; таким образом, программисту не приходится задумываться о том, «где бы сохранить эту переменную», – ведь виртуальное адресное пространство программы велико и в нем полно места. Жизнь программиста стала бы куда труднее, если бы пришлось ломать голову над тем, как впихнуть код и данные в небольшую память, где все уже занято.

Студент. А еще почему?

Профессор. А еще **изоляция и защита** – тоже вещи немаловажные. Мы же не хотим, чтобы какая-нибудь заблудшая программа могла читать или, того хуже, перезаписывать память другой программы, верно?

Студент. Наверное, нет. Если только эта другая программа не написана тем, кто нам не нравится.

Профессор. Гм... Думается, нам стоит запланировать для тебя курс по морали и этике в следующем семестре. Вероятно, курс по ОС заводит тебя на ложный путь.

Студент. Может, и стоит. Но хочу заметить: не я учил нас, что правильная реакция ОС на недостойное поведение процесса – убить его!

Глава 13

Абстракция: адресное пространство

Когда-то давным-давно сконструировать вычислительную систему было просто. Почему, спросите вы? Да потому, что пользователи многого и не ожидали. Это ведь те самые чертовы пользователи, с их ожиданиями «простоты использования», «высокой производительности», «надежности» и т. д. и т. п., все портят. В следующий раз как встретите такого, поблагодарите его за все проблемы, которые он создал.

13.1. Ранние системы

С точки зрения памяти, ранние машины не предоставляли пользователям каких-то мудреных абстракций. Физическая память машины выглядела примерно так, как показано на рис. 13.1.

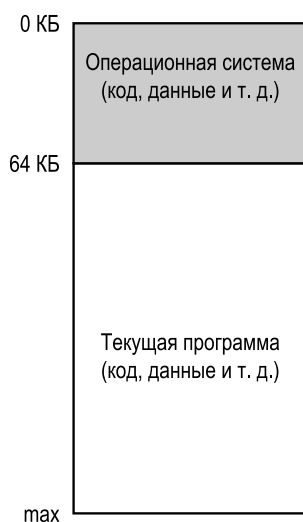


Рис. 13.1 ❖ Операционная система: прежние деньки

ОС представляла собой набор подпрограмм (по сути дела, библиотеку) и располагалась в памяти (в данном случае начиная с физического адреса 0), а кроме нее, в памяти (в данном случае начиная с адреса 64К) находилась одна работающая программа (процесс), которая использовала всю оставшуюся память. Особых иллюзий здесь нет, и пользователь не ожидал от ОС слишком многого. Отличная жизнь была в те времена для разработчиков ОС, не находите?

13.2. МУЛЬТИПРОГРАММИРОВАНИЕ И РАЗДЕЛЕНИЕ ВРЕМЕНИ

Со временем, поскольку вычислительные машины стоили дорого, люди начали использовать их более эффективно. Так началась эра **мультипрограммирования** [DV66], когда в каждый момент времени могло существовать несколько процессов, готовых к выполнению, а ОС должна была переключаться между ними, например, потому что какой-то один решил выполнить ввод-вывод. Это повысило **коэффициент использования** процессора. В те дни такое увеличение **эффективности** было особенно важно, потому что одна машина стоила сотни тысяч, а то и миллионы долларов (а вы-то думали, что ваш Мас чертовски дорогой!).

Но вскоре люди стали предъявлять больше требований к машинам, и началась эра **разделения времени** [S59, L60, M62, M83]. А именно, многие осознавали ограничения пакетных вычислений, и прежде всего сами программисты [CV65], которые устали от долгих (и потому неэффективных) циклов кодирования-отладки. Появилось и стало очень важным понятие **интерактивности**, когда много пользователей могли использовать машину одновременно и каждый ожидал (или надеялся), что машина будет быстро откликаться на их действия.

Для реализации разделения времени применялся, в частности, такой способ: в течение короткого промежутка времени выполнять один процесс, предоставив в его распоряжение всю память (рис. 13.1), затем остановить его, сохранить все его состояние (включая физическую память) на диске, загрузить состояние другого процесса, дать ему немного поработать и таким образом добиться разделения машины, пусть и грубого [M+63].

К сожалению, у этого подхода есть серьезная проблема: он слишком медленный, особенно в ситуации, когда объем памяти увеличивается. Если сохранение и восстановление регистров (РС, регистры общего назначения и т. д.) производится сравнительно быстро, то сброс всего содержимого памяти на диск – катастрофически медленная операция. На самом деле мы хотели бы оставлять процессы в памяти и переключаться между ними, что позволило бы ОС реализовать разделение времени эффективно (рис. 13.2).

На рисунке показаны три процесса (А, В, С), каждому из которых выделена небольшая часть физической памяти размером 512 КБ. В предположении, что имеется всего один процессор, ОС выбирает для выполнения какой-то

один процесс (скажем, А), а остальные (В и С) в это время ожидают в очереди готовых к выполнению. Поскольку режим разделения времени приобрел популярность, к операционной системе, как вы догадываетесь, стали предъявлять новые требования. В частности, поскольку в памяти может располагаться несколько программ, возникла важная проблема **защиты**; ведь не хотим же мы, чтобы процесс мог читать или, того хуже, перезаписывать память другого процесса.



Рис. 13.2 ❖ Три процесса: разделение памяти

13.3. АДРЕСНОЕ ПРОСТРАНСТВО

Однако надо помнить об этих докучливых пользователях, а это значит, что ОС должна создать **простую для использования** абстракцию физической памяти. Мы называем ее **адресным пространством**, это взгляд на память с точки зрения работающей программы. Понимание данной фундаментальной абстракции ОС – ключ к пониманию того, как работает виртуализация памяти.

Адресное пространство процесса содержит все состояние памяти работающей программы. Например, **код** программы (команды) должен находиться где-то в памяти, т. е. в адресном пространстве. Во время работы программа использует **стек**, чтобы следить за цепочкой вызовов функций, там же хранятся локальные переменные, в стеке передаются параметры и возвращаемые значения функций. Наконец, **куча** служит для динамического выделения памяти пользователем, например с помощью функции `malloc()` в С или оператора `new` в объектно-ориентированном языке типа С++ или Java.

Конечно, есть и другие вещи (например, статически инициализированные переменные), но пока остановимся на трех компонентах: код, стек и куча.

В примере на рис. 13.3 адресное пространство крохотное (всего 16 КБ)¹. Код программы размещается в начале адресного пространства (в данном примере начинается с адреса 0 и помещается в первый 1 КБ адресного пространства). Код статический (и потому его легко разместить в памяти), поэтому мы можем расположить его в начале адресного пространства, будучи уверенными, что выделять для него дополнительную память в процессе работы не понадобится.

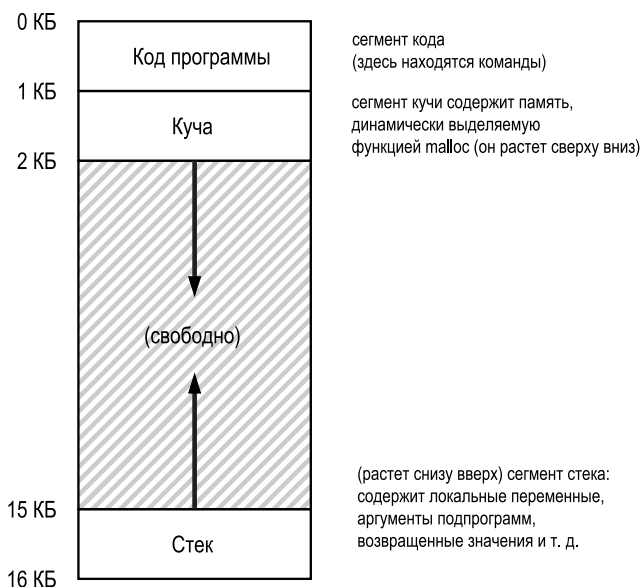


Рис. 13.3 ❖ Пример адресного пространства

Далее идут две области, которые могут увеличиваться (и уменьшаться) по ходу работы программы. Это куча (сверху) и стек (снизу). Мы расположили их именно так, потому что обе области могут расти, а разместив их в противоположных концах адресного пространства, мы допускаем такую возможность – они просто растут навстречу друг другу. Куча начинается сразу после кода (с адреса 1 КБ) и растет вниз (например, когда пользователь запрашивает память с помощью `malloc()`); стек начинается с адреса 16 КБ и растет вверх (например, когда пользователь вызывает функцию). Однако такое размещение стека и кучи – не более чем соглашение; при желании адресное пространство можно организовать и по-другому (как мы увидим ниже, когда в адресном пространстве сосуществует несколько **потоков**, так удобно разделить его, к сожалению, не получится).

¹ Мы часто будем использовать такие небольшие примеры, потому что (а) изображать 32-разрядное адресное пространство мутрно и (б) математика оказывается сложнее. А нам нравится простая математика.

Конечно, описывая адресное пространство, мы описываем **абстракцию**, которую ОС предоставляет работающей программе. В действительности программа не находится в памяти по физическим адресам от 0 до 16 КБ, а загружена начиная с какого-то произвольного физического адреса. Взгляните на процессы А, В, С на рис. 13.2, вы увидите, что все они начинаются с разных адресов. И возникает главный вопрос.

СУЩЕСТВО ПРОБЛЕМЫ: КАК ВИРТУАЛИЗИРОВАТЬ ПАМЯТЬ

Как ОС может построить абстракцию частного потенциально очень большого адресного пространства для нескольких работающих процессов (разделяющих память) поверх одной физической памяти?

Этот подвиг называется **виртуализацией памяти**, потому что работающая программа думает, что загружена в память с определенного физического адреса (скажем, 0) и имеет в своем распоряжении очень большое адресное пространство (32- или 64-разрядное); реальность же совершенно иная.

Например, хотя процесс А на рис. 13.2 думает, что он загружен по адресу 0 (который мы будем называть **виртуальным адресом**), ОС в паре с оборудованием каким-то образом загружает его не с адреса 0, а с физического адреса 320 КБ. Это ключ к виртуализации памяти, существующей во всех современных вычислительных системах.

Совет: принцип изоляции

Изоляция – главный принцип построения надежных систем. Если две сущности хорошо изолированы, то отказ одной не повлияет на работоспособность другой. Операционные системы прилагают массу усилий, чтобы изолировать процессы друг от друга и тем самым не дать одному нанести вред другому. С помощью изоляции памяти ОС дополнительно гарантирует, что исполняемые программы не смогут повлиять на работу самой ОС. В некоторых современных ОС изоляция заходит еще дальше, отделяя стеной одни части ОС от других. Такие **микроядра** [BH70, R+89, S+03] обеспечивают большую надежность, чем типичные монолитные ядра.

13.4. Цели

Итак, мы подошли к очередной задаче ОС: виртуализировать память. Впрочем, ОС виртуализирует память не просто так, а стильно. Но для этого мы должны поставить какие-то цели. Мы уже формулировали их раньше (смотрите введение) и сделаем это снова, поскольку они, безусловно, заслуживают повторения.

Основная цель виртуальной памяти (ВП) – **прозрачность**¹. ОС должна реализовывать виртуальную память, так чтобы она была не видна исполняемой программе. То есть программа не должна знать, что память виртуализирована, а должна вести себя так, будто это ее собственная частная физическая память. За кулисами ОС (и оборудование) делает все необходимое, чтобы распределить память между разными заданиями, и, значит, поддерживает эту иллюзию.

Еще одна цель ВП – **эффективность**. ОС должна стремиться, чтобы виртуализация была максимально эффективной с точки зрения как времени (т. е. программы не должны существенно замедляться), так и пространства (т. е. на структуры данных, необходимые для поддержки виртуализации, не должно расходоваться слишком много памяти). При реализации эффективной по времени виртуализации ОС опирается на аппаратную поддержку, в т. ч. такие специализированные аппаратные средства, как буфер ассоциативной трансляции TLB (о котором мы узнаем в свое время).

Наконец, третья цель ВП – **защита**. ОС должна гарантированно защищать процессы друг от друга, а также себя от процессов. Когда один процесс выполняет загрузку данных, сохранение данных или выборку команды, он не должен иметь возможность обратиться или каким-то образом повлиять на содержимое памяти любого другого процесса или самой ОС (т. е. всего, что находится *вне* его адресного пространства). Таким образом, защита обеспечивает свойство **изоляции** процессов; каждый процесс должен работать в своем изолированном коконе, надежно защищенном от поползновений других ошибочных или даже вредоносных процессов.

ОТСТУПЛЕНИЕ:

ВСЕ АДРЕСА, КОТОРЫЕ ВЫ ВИДИТЕ, ВИРТУАЛЬНЫЕ

Вы когда-нибудь писали программу на С, которая распечатывает указатель? Так вот, значение, которое она выводит (большое число, часто в шестнадцатеричном виде), – это **виртуальный адрес**. Вас когда-нибудь интересовало, где в памяти размещается код вашей программы? Этот адрес тоже можно напечатать, и он тоже будет виртуальным. На самом деле любой адрес, который видит автор пользовательской программы, виртуальный. Лишь сама ОС, благодаря хитрым приемам виртуализации памяти, знает, где в физической памяти находятся команды и данные. Так что помните: любой распечатанный адрес, относящийся к программе, виртуальный, это иллюзорное представление о размещении программы в памяти, и только ОС (и оборудование) знает истину.

¹ Такое употребление слова «прозрачность» может внести путаницу; некоторые студенты думают, что «быть прозрачным» значит выставлять на всеобщее обозрение – так должно вести себя правительство. Но здесь смысл прямо противоположный: иллюзия, создаваемая ОС, не должна быть видна приложениям. То есть прозрачная система – это такая система, которую трудно заметить, а не такая, которая отвечает на запросы, как того требует Закон о свободном доступе к информации.

Ниже приведена небольшая программа (va.c), которая печатает адрес функции `main()` (где находится код), значение адреса блока памяти в куче, возвращенное `malloc()`, и адрес целого числа в стеке:

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 int main(int argc, char *argv[]) {
4     printf("адрес кода : %p\n", (void *) main);
5     printf("адрес в куче : %p\n", (void *) malloc(1));
6     int x = 3;
7     printf("адрес в стеке : %p\n", (void *) &x);
8     return x;
9 }
```

Запустив эту программу на 64-разрядном Mac, получим такой результат:

```
адрес кода : 0x1095afe50
адрес в куче : 0x1096008c0
адрес в стеке : 0x7fff691aea64
```

Отсюда видно, что первым в адресном пространстве расположен код, за ним куча, а стек – в самом конце большого виртуального адресного пространства. Все эти адреса виртуальные, и ОС совместно с оборудованием подвергает их трансляции, чтобы выбрать значения из настоящей физической памяти.

В следующих главах мы сосредоточимся на основных **механизмах** виртуализации памяти – как аппаратных, так и предоставляемых операционной системой. Мы также рассмотрим некоторые относящиеся к этому предмету **политики** операционной системы, в частности как она управляет свободным пространством и какие страницы вытесняет из памяти, если ее оказывается недостаточно. По ходу дела вы сможете составить представление о том, как в действительности работает современная система виртуальной памяти¹.

13.5. РЕЗЮМЕ

Мы познакомились с одной из основных подсистем ОС: виртуальной памятью. Система ВП создает иллюзию большого разреженного частного адресного пространства, в котором находятся все команды и данные программы. ОС с помощью серьезной поддержки со стороны оборудования преобразует каждый виртуальный адрес в физический, который можно предъявить физической памяти и выбрать хранящуюся по нему информацию. ОС делает это сразу для многих процессов, защищая программы друга от друга и саму себя от пользовательских программ. В этом подходе используется много разных низкоуровневых механизмов, а также ряд высокоуровневых

¹ Или же решите бросить этот курс. Но не торопитесь – если вы сможете продраяться сквозь дебри ВП, то сумеете дойти до самого конца!

политик; мы будем двигаться снизу вверх, т. е. сначала опишем критические механизмы. Приступим!

Литература

[BH70] «The Nucleus of a Multiprogramming System» by Per Brinch Hansen. Communications of the ACM, 13:4, April 1970. *Первая статья, в которой была предложена идея ядра, – минимального и гибкого субстрата для создания специализированных операционных систем; к этой теме неоднократно возвращались на протяжении всей истории ОС.*

[CV65] «Introduction and Overview of the Multics System» by F. J. Corbato, V. A. Vyssotsky. Fall Joint Computer Conference, 1965. *Великолепная ранняя статья по системе Multics. Вот важная цитата из нее, касающаяся разделения времени: «Движущей силой разделения времени являются в первую очередь профессиональные программисты, которым надоело отлаживать программы в вычислительных центрах с пакетной дисциплиной обслуживания. Таким образом, первоначальной целью было организовать разделение компьютерного времени, чтобы к машине одновременно могло обращаться несколько человек и каждый думал, что вся машина находится в его распоряжении».*

[DV66] «Programming Semantics for Multiprogrammed Computations» by Jack B. Dennis, Earl C. Van Horn. Communications of the ACM, Volume 9, Number 3, March 1966. *Ранняя (но не первая) статья о мультипрограммировании.*

[L60] «Man-Computer Symbiosis» by J. C. R. Licklider. IRE Transactions on Human Factors in Electronics, HFE-1:1, March 1960. *Фантастическая статья о том, как компьютеры и люди сольются в симбиозе; далеко опередила свое время, но читать ее тем не менее интересно.*

[M62] «Time-Sharing Computer Systems» by J. McCarthy. Management and the Computer of the Future, MIT Press, Cambridge, MA, 1962. *Вероятно, первая опубликованная статья Маккарти на тему разделения времени. В другой статье [M83] он пишет, что обдумывал эту идею с 1957 года. Маккарти оставит системное программирование, чтобы стать гигантом искусственного интеллекта в Стэнфорде, в частности он создал язык программирования LISP. Дополнительные сведения см. на домашней странице Маккарти: <http://www-formal.stanford.edu/jmc/>.*

[M+63] «A Time-Sharing Debugging System for a Small Computer» by J. McCarthy, S. Boilen, E. Fredkin, J. C. R. Licklider. AFIPS '63 (Spring), New York, NY, May 1963. *Прекрасный ранний пример системы, которая выгружала память программы на «барабан», когда та не выполнялась, а затем загружала ее обратно в «оперативную» память, перед тем как запустить снова.*

[M83] «Reminiscences on the History of Time Sharing» by John McCarthy. 1983. Доступно по адресу <http://www-formal.stanford.edu/jmc/history/timesharing/timesharing.html>. *Блестательные исторические заметки о том, откуда взялась идея разделения времени, содержащая в том числе критику в адрес тех, кто цитирует работу Стрейчи [S59], называя ее пионерской в этой области.*

[NS07] «Valgrind: A Framework for Heavyweight Dynamic Binary Instrumentation» by N. Nethercote, J. Seward. PLDI 2007, San Diego, California, June 2007. *Valgrind – спасательный пояс для тех, кто пользуется небезопасными языками типа C. Прочитайте эту статью, чтобы узнать о весьма изоцированной технике оснащения инструментальными средствами двоичной программы, – очень впечатляет.*

[R+89] «Mach: A System Software kernel» by R. Rashid, D. Julin, D. Orr, R. Sanzi, R. Baron, A. Forin, D. Golub, M. Jones. COMPCON '89, February 1989. *Проект Mach, выполненный в университете Карнеги-Меллона, – не первый проект микроядра как такового, но самый известный и влиятельный; он по сей день скрыт глубоко в недрах Mac OS X.*

[S59] «Time Sharing in Large Fast Computers» by C. Strachey. Proceedings of the International Conference on Information Processing, UNESCO, June 1959. *Одно из самых ранних упоминаний о разделении времени.*

[S+03] «Improving the Reliability of Commodity Operating Systems» by M. M. Swift, B. N. Bershad, H. M. Levy. SOSP '03. *Первая статья, в которой показано, как подход на основе микроядра может повысить надежность операционной системы.*

Домашнее задание (код)

В этом домашнем задании вы узнаете о нескольких полезных инструментах для исследования виртуальной памяти в Linux-системах. Это лишь намек на имеющиеся возможности; чтобы стать экспертом, придется копнуть глубже (как всегда).

Вопросы

1. Первый инструмент, с которым вам следует познакомиться, – очень простая программа `free`. Сначала наберите `man free` и прочитайте всю страницу руководства; она коротенькая, не переживайте!
2. Затем выполните `free`, быть может, с какими-то аргументами (например, `-m`, чтобы показывать объемы в мегабайтах). Сколько памяти в вашей системе? Какая ее часть свободна? Совпадают ли эти данные с тем, что вам подсказывает интуиция?
3. Затем напишите простенькую программу, которая запрашивает сколько-то памяти, назовите ее `memog-user.c`. Эта программа должна принимать один аргумент командной строки: количество запрашиваемых мегабайтов памяти. После запуска она должна выделить массив такого размера и пройтись по нему, обратившись к каждому элементу. Программа должна обходить массив в бесконечном цикле или, быть может, в течение времени, которое также задается в командной строке.
4. Теперь запустите свою программу `memog-user`, и в другом окне терминала на той же машине запустите программу `free`. Как при этом изменятся данные о свободной памяти? А что будет, когда вы снимете программу

memoq-user? Это совпадает с вашими ожиданиями? Попробуйте задать другой объем запрашиваемой памяти. Что произойдет, если вы запросите очень много памяти?

5. Теперь поработаем еще с одним инструментом, rmar. Потратьте немного времени на ознакомление с его страницей руководства.
6. Чтобы использовать rmar, нужно знать **идентификатор** интересующего вас процесса (PID). Поэтому сначала выполните команду `ps auxw`, которая выводит список всех процессов, а затем выберите какой-нибудь, например браузер. Или возьмите свою программу memoq-user (собственно, можете просто вызвать в этой программе функцию `getpid()` и напечатать ее PID).
7. Теперь запустите rmar для некоторых процессов с разными флагами (например, -X), чтобы получить разнообразные сведения. Что вы видите? Сколько различных частей в современном адресном пространстве, помимо нашего упрощенного представления код-стек-куча?
8. Наконец, запустите rmar для своей программы memoq-user с различными значениями параметра. Что вы видите? Совпадает ли результат rmar с вашими ожиданиями?

Глава 14

Интерлюдия: API памяти

В этой интерлюдии мы обсудим интерфейсы выделения памяти в Unix-системах. Эти интерфейсы очень просты, поэтому глава получилась коротенькой и по делу¹. И вот какую проблему мы в ней обсуждаем.

Существо проблемы: как выделить память и управлять ей

При программировании на C в Unix понимать, как осуществляется выделение и управление памятью, критически важно для создания надежного программного обеспечения. Какие интерфейсы обычно используются? Каких ошибок следует избегать?

14.1. Типы памяти

Работающая программа C выделяет память двух типов. Во-первых, память в **стеке** – она выделяется и освобождается *неявно* компилятором, поэтому такую память часто называют **автоматической**.

Объявить память в стеке легко. Пусть, например, в функции `func()` требуется память для целого числа `x`. Чтобы объявить такой участок памяти, нужно всего лишь написать что-то вроде:

```
void func() {  
    int x; // объявляется целое в стеке  
    ...  
}
```

Все остальное сделает компилятор – он гарантирует, что при вызове `func()` в стеке будет достаточно места. А при возврате из функции компилятор освободит эту память, поэтому если вы хотите, чтобы какая-то информация продолжала существовать после возврата из функции, лучше не оставлять ее в стеке.

¹ Вообще-то мы надеемся, что все главы такие! Но эта равнее других, так нам кажется.

Именно потребность в долговечной памяти привела ко второму типу памяти – **куче**; в этом случае выделение и освобождение производятся *явно* программистом. Без сомнения, это тяжкая ответственность! И разумеется, причина многих ошибок. Но если аккуратно и правильно использовать предоставляемые интерфейсы, то проблем удастся избежать. Вот пример выделения памяти для целого числа в куче:

```
void func() {
    int *x = (int *) malloc(sizeof(int));
    ...
}
```

По поводу этого фрагмента кода есть два замечания. Во-первых, вы, наверное, заметили, что в этой строке имеет место выделение как из стека, так и из кучи: сначала компилятор выделяет место для указателя на целое, увидев объявление указателя (`int *x`), а затем программа вызывает функцию `malloc()`, запрашивая тем самым место для целого числа в куче; функция возвращает адрес этого числа (в случае успеха или `NULL` в случае ошибки), который запоминается в стеке.

Поскольку память в куче выделяется явно и используется в разнообразных ситуациях, работа с ней представляет больше трудностей и для пользователей, и для системы. Именно ее мы и будем обсуждать далее.

14.2. Вызов malloc()

Функции `malloc()` просто передается размер запрашиваемой памяти в куче, а она либо успешно выполняет операцию и возвращает указатель на выделенный блок, либо не выполняет и возвращает `NULL`¹.

На странице руководства написано, что нужно для использования `malloc`; набрав команду `man malloc`, вы увидите:

```
#include <stdlib.h>
...
void *malloc(size_t size);
```

Отсюда понятно, что для использования `malloc` необходимо включить заголовочный файл `stdlib.h`. На самом деле можно и этого не делать, потому что стандартная библиотека C, с которой по умолчанию компонуется все написанные на C программы, включает код `malloc()`; добавление заголовка просто позволяет компилятору проверить, что `malloc()` вызывается правильно (что ей передается нужное число аргументов нужных типов).

Единственный параметр `malloc()` должен иметь тип `size_t`, это количество запрашиваемых байтов. Однако программисты, как правило, не пишут это число явно (например, 10), это считается дурным тоном. Вместо этого ис-

¹ Заметим, что `NULL` в языке C – не зарезервированное слово, а просто макрос, обозначающий нуль.

пользуются различные функции и макросы. Так, чтобы выделить место для числа с плавающей точкой двойной точности, можно написать:

```
double *d = (double *) malloc(sizeof(double));
```

СОВЕТ: СОМНЕВАЕШЬСЯ – ПОПРОБУЙ

Если вы не уверены, как ведет себя используемая функция или оператор, просто попробуйте и убедитесь, что ее поведение совпадает с вашими ожиданиями. Читать страницы руководства и прочую документацию полезно, но практика – лучший учитель. Напишите код и протестируйте его! Это, без сомнения, лучший способ убедиться, что код работает так, как вам нужно. На самом деле именно так мы проверяли, что все сказанное ниже о `sizeof()` – правда!

Однако же сколько тут `double`! В этом обращении к `malloc()` используется оператор `sizeof()`, чтобы запросить нужное количество памяти; в С он является оператором *времени компиляции*, т. е. фактический размер известен *на этапе компиляции* и подставляется в качестве аргумента `malloc()` (в данном случае он равен 8 – размеру типа `double`). Поэтому `sizeof()` правильно рассматривать именно как оператор, а не как вызов функции (который производится на этапе выполнения).

Можно также передать `sizeof()` имя переменной (а не просто тип), но в некоторых случаях вы можете получить не то, что хотели, так что будьте осторожны. Рассмотрим, например, следующий фрагмент:

```
int *x = malloc(10 * sizeof(int));
printf("%d\n", sizeof(x));
```

В первой строке мы выделили место для массива 10 целых чисел – никаких вопросов. Но при использовании `sizeof()` в следующей строке мы получим небольшое значение: 4 (на 32-разрядной машине) или 8 (на 64-разрядной). Дело в том, что `sizeof()` думает, что мы запрашиваем размер *указателя* на целое, а не размер динамически выделенной памяти. Но иногда `sizeof()` делает именно то, что вы хотели:

```
int x[10];
printf("%d\n", sizeof(x));
```

В этом случае у компилятора достаточно статической информации, чтобы понять, что было выделено 40 байт.

Еще одно место, где требуется аккуратность, – строки. При выделении места для строки вспоминайте следующую идиому: `malloc(strlen(s) + 1)`; так вы получаете длину строки с помощью функции `strlen()` и прибавляете к ней единицу, чтобы зарезервировать место для завершающего символа конца строки. Использование в этой ситуации `sizeof()` чревато неприятностями.

Вы, вероятно, также заметили, что `malloc()` возвращает указатель на тип `void`. Это позволяет передать адрес и дать возможность программисту решить, что с ним делать. Дальше программист выполняет **приведение типа**;

в примере выше значение, возвращенное `malloc()`, приводится к типу указателя на `double`. Приведение типа – это всего лишь способ сообщить компилятору и другим программистам, читающим код: «не волнуйтесь, я знаю, что делаю»; никаких реальных действий при этом не производится. Приводя результат `malloc()`, программист просто уверяет себя и компилятор в правильности действий, на корректность программы это не влияет.

14.3. Вызов `free()`

Выделение памяти – это еще цветочки, а ягодки – знать, когда и как освободить память, да и вообще нужно ли это делать. Для освобождения памяти, выделенной из кучи, программист просто вызывает функцию `free()`:

```
int *x = malloc(10 * sizeof(int));  
...  
free(x);
```

Эта функция принимает один аргумент – указатель, возвращенный `malloc()`. Как видите, размер выделенной области не передается, его должна хранить сама библиотека работы с памятью.

14.4. Типичные ошибки

При работе с `malloc()` и `free()` часто допускают типичные ошибки. Мы перечислим те, с которыми снова и снова встречаемся на занятиях по операционным системам для студентов бакалавриата. Все эти примеры компилируются без единого возражения со стороны компилятора, но хотя успешная компиляция программы на С – необходимое условие ее правильности, оно отнюдь не достаточно – и зачастую за этот опыт приходится дорого платить.

Правильно управлять памятью всегда было нелегко, именно поэтому многие современные языки поддерживают **автоматическое управление памятью**. В таких языках существуют средства, подобные `malloc()`, для выделения памяти (обычно оператор **new** или нечто похожее для выделения памяти под новый объект), но освобождать память вручную не нужно; вместо этого работающий в фоновом режиме **сборщик мусора** определяет, на какую память больше нет ссылок, и освобождает ее самостоятельно.

Забыли выделить память

Многие функции ожидают, что перед обращением к ним была выделена память. Например, функция `strcpy(dst, src)` копирует строку, на которую указывает `src`, в область памяти, на которую указывает `dst`. Но при небрежном программировании может случиться такое:

```
char *src = "hello";
char *dst;           // Ой! Память не выделена
strcpy(dst, src);    // segfault и аварийное завершение
```

При выполнении этого кода, скорее всего, возникнет **ошибка сегментации** (segmentation fault)¹, а это не что иное, как вежливый эвфемизм для следующей простой мысли: **ТЫ СДЕЛАЛ ЧТО-ТО НЕПРАВИЛЬНОЕ С ПАМЯТЮ, ТЫ ТУПОЙ ПРОГРАММИСТ, И Я ТОБОЙ НЕДОВОЛЬНА.**

Совет: компилируется и запускается ≠ правильно

Из того, что программа компилируется и даже один или несколько раз показывает правильные результаты при выполнении, вовсе не следует, что программа правильна. Сочетание многих событий может заставить вас поверить в то, что программа работает, но потом что-то меняется – и везение кончилось. Обычно студент реагирует на такой поворот событий восклицанием «Но раньше же работала!», а потом начинает винить компилятор, операционную систему, оборудование и даже (осмелимся сказать) профессора. Но проблема-то, как правило, кроется именно там, где вы и должны были ее заподозрить, – в вашем коде. Принимайтесь за отладку, прежде чем валить все на другие компоненты.

В данном случае правильный код мог бы выглядеть следующим образом:

```
char *src = "hello";
char *dst = (char *) malloc(strlen(src) + 1);
strcpy(dst, src); // работает правильно
```

Или можно было бы упростить себе жизнь и воспользоваться функцией `strdup()`, о которой можно прочесть на странице руководства.

Выделили недостаточно памяти

Родственная ошибка – выделение памяти недостаточного размера, иногда это называется **переполнением буфера**. В примере выше типичная ошибка – выделить память, которой *почти* хватает для буфера-приемника.

```
char *src = "hello";
char *dst = (char *) malloc(strlen(src)); // маловато будет!
strcpy(dst, src);
```

Как ни странно, но в зависимости от особенностей реализации `malloc` и многих других факторов часто кажется, что эта программа работает правильно. При копировании строки записывается один байт за концом выделенного буфера, но иногда это безвредно, например потому, что затертая переменная больше не используется. А иногда переполнение очень даже

¹ Звучит странно, но скоро вы поймете, почему недопустимый доступ к памяти называется ошибкой сегментации. Если уж это недостаточный стимул читать дальше, то что тогда?

вредно и является причиной многих уязвимостей в системах [W06]. А бывает, что библиотечная функция `malloc` в любом случае выделяет немного места сверх запрошенного, тогда программа не испортит другую переменную и все будет работать хорошо. Бывает и так, что дело заканчивается ошибкой сегментации и крахом. И это нам еще один урок – даже если программа один раз отработала правильно, это еще не значит, что она правильна.

Забыли инициализировать выделенную память

В этом случае обращение к `malloc()` правильно, но вы забыли заполнить выделенную область какими-то значениями. Не поступайте так! Если вы так сделаете, то программа рано или поздно произведет **чтение неинициализированной памяти** и прочитает какие-то неизвестные данные, незнамо как оказавшиеся в куче. Кто знает, что там может быть? Если повезет, это может быть значение, при котором программа все же работает (например, ноль). А если не повезет, то что-то случайное и потенциально опасное.

Забыли освободить память

Еще одна типичная ошибка – **утечка памяти**, это происходит, когда вы забыли освободить память. В долго работающих приложениях и системах (в т. ч. в самой ОС) это огромная проблема, поскольку медленная утечка рано или поздно приводит к нехватке памяти, после чего необходим перезапуск. Поэтому в общем случае, закончив работу с блоком памяти, обязательно освободите его. Заметим, что сборщик мусора тут не поможет: если сохраняется ссылка на блок памяти, то никакой сборщик мусора этот блок не освободит, поэтому утечки остаются проблемой даже в современных языках.

Иногда может показаться, что не вызывать `free()` разумно. Например, программа работает недолго и все равно скоро завершится; в таком случае процесс умрет, и ОС освободит все занятые им страницы памяти, так что никакой утечки не будет. Это, конечно, «работает» (см. отступление ниже), но способствует развитию дурных привычек, поэтому относитесь к такой стратегии с подозрением. В перспективе одна из наших целей – выработать у программиста хорошие привычки, а одна из них – понимать, что освобождение выделенной памяти в языках типа C – неотъемлемая часть управления памятью. Даже если иногда это может сойти с рук, лучше все же взять за правило освобождать каждый байт, который вы явно выделили.

Освободили память раньше, чем закончили с ней работать

Иногда программа освобождает память, не закончив использовать ее; такая ошибка называется **висячим указателем** и, как вы догадываетесь, ничего

хорошего не сулит. Если впоследствии воспользоваться этим указателем, то программа может «упасть». Или, того хуже, можно затереть нужную область памяти (например, если вызвать `free()`, а затем снова `malloc()` с целью получить память под что-то другое, то может оказаться, что ошибочно освобожденная память будет занята другими данными).

Освободили память несколько раз

Иногда память освобождается несколько раз, эта ошибка называется **двойным освобождением**. Результат такого действия не определен. Как легко представить, библиотека работы с памятью может запутаться и сделать странное, обычно все кончается крахом программы.

Неправильно вызвали `free()`

И последняя проблема, которую мы обсудим, – неправильный вызов `free()`. Функция `free()` ожидает получить указатель, который раньше вернула `malloc()`. Если передать ей что-то другое, то может произойти (и обязательно произойдет) нехорошее. Такие **неправильные освобождения** опасны, и, конечно, их следует избегать.

ОТСТУПЛЕНИЕ:

ПОЧЕМУ ПАМЯТЬ НЕ УТЕКАЕТ ПОСЛЕ ЗАВЕРШЕНИЯ ПРОЦЕССА

В программе, которая работает недолго, тоже могут быть обращения к `malloc()`. И вот программа готова завершиться; нужно ли вызывать `free()` перед выходом? На первый взгляд, нужно, но даже если этого не сделать, никакой реальной утечки памяти не произойдет. Причина проста: в системе есть два уровня управления памятью. За первый уровень отвечает сама ОС, которая выделяет память работающим процессам и забирает ее, когда процесс завершается добровольно или принудительно. Второй уровень находится *внутри* каждого процесса, это вызовы `malloc()` и `free()`. Даже если вы не вызовете `free()` (и, значит, произойдет утечка памяти из кучи), операционная система получит обратно *всю* память процесса (включая страницы кода, стека и, что нам особенно важно, кучи), когда программа завершится. Состояние кучи в адресном пространстве процесса не имеет никакого значения, ОС возвращает себе всё, так что память никуда не девается, даже если вы ее не освободили сами.

Поэтому для коротких программ утечка памяти обычно не приводит к эксплуатационным проблемам (хотя все равно считается дурным тоном). Но если вы пишете долго работающий сервер (например, веб-сервер или систему управления базами данных, которые никогда не завершаются), утечка памяти является куда более серьезной проблемой и в конечном итоге приведет к краху приложения из-за нехватки памяти. И конечно, утечка принимает совсем уж катастрофические масштабы, если имеет место в одной конкретной программе – самой операционной системе. Что еще раз подчеркивает: у тех, кто пишет код ядра, самая тяжелая работа...

Итоги

Как видите, для неправильной работы с памятью есть масса возможностей. Из-за того, что такие ошибки случаются очень часто, появилась целая экосистема инструментов, предназначенных для их поиска. Поинтересуйтесь программами **purify** [H]92] и **valgrind** [SN05]; обе отлично помогают находить источник связанных с памятью проблем. Освоившись с этими мощными инструментами, вы уже не сможете представить, как обходились без них раньше.

14.5. Поддержка со стороны ОС

Вы, наверное, заметили, что при обсуждении `malloc()` и `free()` мы ни разу не упомянули о системных вызовах. Причина проста: это не системные вызовы, а библиотечные функции. Библиотека `malloc` отвечает за управление памятью внутри виртуального адресного пространства процесса, но сама построена поверх системных вызовов, которые запрашивают у ОС дополнительную память и возвращают память системе.

Один из таких вызовов называется `brk`, он позволяет изменить границу сегмента данных программы, т. е. адрес конца кучи. Функция принимает один аргумент (адрес новой границы) и таким образом увеличивает или уменьшает размер кучи в зависимости от того, что больше: новая граница или текущая. Есть еще системный вызов `sbrk`, которому передается величина инкремента, но в остальном он ничем не отличается.

Отметим, что вы сами никогда не должны вызывать `brk` или `sbrk`. Эти вызовы используются библиотекой работы с памятью; если вы попытаетесь вызвать их самостоятельно, то, скорее всего, произойдет нечто ужасное. Используйте `malloc()` и `free()` – они для нас и предназначены.

Наконец, можно получить память от операционной системы с помощью вызова `mmap()`. Получив правильные аргументы, `mmap()` создает **анонимную** область памяти внутри вашей программы – связанную не с конкретным файлом, а с **областью подкачки** (`swap space`), которую мы обсудим ниже. Далее эту область можно рассматривать как кучу и управлять ей точно так же. Дополнительные сведения см. на странице руководства по `mmap()`.

14.6. Другие вызовы

Библиотека работы с памятью поддерживает еще несколько функций. Например, `calloc()` выделяет и обнуляет область памяти; это предотвращает ошибки из-за того, что вы забыли инициализировать память, которая должна содержать нули (см. раздел о чтении неинициализированной памяти выше). Функция `realloc()` полезна, когда уже имеется блок памяти, выделенной

для чего-то (например, массива), и возникла необходимость увеличить его; `realloc()` выделяет новый, больший блок памяти, копирует в него содержимое прежнего блока и возвращает указатель на новый блок.

14.7. РЕЗЮМЕ

Мы познакомились с некоторыми API выделения и освобождения памяти. Как обычно, мы рассказали лишь об основах, дополнительные сведения следует искать в другом месте. Можете прочитать учебник C [KR88] и книгу Стивенса [SR05] (глава 7). Более современный материал о том, как автоматически обнаруживать и исправлять многие из описанных проблем, см. в статье Novark et al. [N+07].

Литература

[HJ92] «Purify: Fast Detection of Memory Leaks and Access Errors» by R. Hastings, B. Joyce. USENIX Winter '92. *В этой статье описывается программа Purify, ставшая позднее коммерческой.*

[KR88] «The C Programming Language» by Brian Kernighan, Dennis Ritchie. Prentice-Hall 1988. *Учебник языка C, написанный его авторами. Прочитайте один раз, попрограммируйте, затем прочитайте снова и держите рядом с рабочим столом или где вы там пишете код.*

[N+07] «Exterminator: Automatically Correcting Memory Errors with High Probability» by G. Novark, E. D. Berger, B. G. Zorn. PLDI 2007, San Diego, California. *Статья посвящена автоматическому поиску и исправлению ошибок работы с памятью, содержит прекрасный обзор многих типичных ошибок программирования на C и C++. Расширенный вариант статьи доступен в журнале CACM (Volume 51, Issue 12, December 2008).*

[SN05] «Using Valgrind to Detect Undefined Value Errors with Bit-precision» by J. Seward, N. Nethercote. USENIX '05. *Как использовать valgrind для поиска некоторых типов ошибок.*

[SR05] «Advanced Programming in the Unix Environment» by W. Richard Stevens, Stephen A. Rago. Addison-Wesley, 2005. *Мы уже говорили раньше и говорим еще раз: читайте эту книгу много раз и обращайтесь к ней как к справочнику, когда вас терзают сомнения. Авторы не устают удивляться тому, что каждый раз, перечитывая ее, находят что-то новое, хотя уже много лет программируют на C.*

[W06] «Survey on Buffer Overflow Attacks and Countermeasures» by T. Werthman. Доступно по адресу www.nds.rub.de/lehre/seminar/SS06/Werthmann/BufferOverflow.pdf. *Удачный обзор ошибок переполнения буфера и вызываемых ими проблем безопасности. Рассматриваются многие знаменитые эксплойты.*

Домашнее задание (код)

В этом домашнем задании вы немного попрактикуетесь в выделении памяти. Сначала вы напишете несколько дефектных программ (забавно же!). А потом воспользуетесь инструментами для нахождения ошибок, которые сами же и насажали. И таким образом узнаете о крутизне этих инструментов и будете использовать их в будущем, что принесет вам радость и продуктивность. Какие инструменты? – так отладчик (например, `gdb`) и детектор ошибок работы с памятью `valgrind` [SN05].

Вопросы

1. Для начала напишите простую программу `null.c`, которая создает указатель на целое число, присваивает ему значение `NULL`, а потом пытается разыменовать его. Откомпилируйте ее, назвав исполняемый файл `null`. Что произойдет при попытке выполнить эту программу?
2. Далее откомпилируйте эту программу, включив информацию о символах (с флагом `-g`). При этом в исполняемый файл помещается дополнительная информация, позволяющая отладчику получить сведения об именах переменных, и не только. Запустите программу под отладчиком, набрав `gdb null`, а после того как отладчик загрузится, наберите `run`. Что показывает `gdb`?
3. Наконец, примените к этой программе инструмент `valgrind`. Для анализа происходящего нам понадобится его часть `memcheck`. Запустите программу командой `valgrind --leak-check=yes null`. Что происходит? Можете ли вы интерпретировать полученный результат?
4. Напишите простую программу, которая выделяет память с помощью `malloc()`, но забывает освободить ее перед выходом. Что происходит при запуске этой программы? Можете ли вы с помощью `gdb` найти в ней ошибки? А с помощью `valgrind` (снова с флагом `--leak-check=yes`)?
5. Напишите программу, которая создает массив целых чисел `data` размером 100 с помощью `malloc`; затем присвойте `data[100]` значение 0. Что произойдет при запуске этой программы? А при запуске под управлением `valgrind`? Правильна эта программа?
6. Напишите программу, которая выделяет память для массива целых чисел (как и выше), освобождает ее, а затем пытается напечатать значение одного из элементов массива. Завершается ли программа успешно? А что говорит о ней `valgrind`?
7. Теперь передайте `free` какое-нибудь недопустимое значение (например, указатель на средний элемент созданного выше массива). Что происходит? Нужны ли вам инструменты для обнаружения такого рода проблем?
8. Поэкспериментируйте с другими интерфейсами выделения памяти. Например, создайте простую структуру данных типа вектора и напишите

функции, которые используют `realloc()` для управления вектором. Элементы вектора храните в массиве, а когда понадобится расширить вектор, используйте `realloc()`. Какова производительность такого вектора? Сравните ее с производительностью связного списка. Воспользуйтесь `valgrind` для поиска ошибок.

9. Потратьте время и почитайте о `gdb` и `valgrind`. Знать свои инструменты очень важно, не поленитесь и станьте экспертом по отладке в среде Unix и C.

Глава 15

Механизм: трансляция адресов

При разработке виртуализации CPU мы акцентировали внимание на общем механизме **ограниченного прямого выполнения** (LDE). Идея LDE проста: ОС позволяет программе большую часть времени выполняться непосредственно на оборудовании, но в некоторых точках (например, когда процесс выполняет системный вызов или возникает прерывание от таймера) вмешивается и наводит порядок. Таким образом, ОС, пользуясь небольшой поддержкой со стороны оборудования, делает все возможное, чтобы не стоять на пути у программы и тем самым обеспечить *эффективную* виртуализацию, но, вклиниваясь в критические моменты, сохраняет за собой *контроль* над оборудованием. Эффективность и контроль – две главные цели современной операционной системы.

При виртуализации памяти мы следуем аналогичной стратегии, достигая одновременно эффективности и контроля. Стремление к эффективности вынуждает нас прибегать к аппаратной поддержке, которая поначалу будет рудиментарной (всего несколько регистров), но постепенно будет усложняться (TLB, поддержка таблиц страниц и т. д. – мы обо всем этом поговорим). Контроль подразумевает, что ОС не должна позволять приложению обращаться к какой-либо памяти, кроме его собственной; поэтому, чтобы защитить приложения друг от друга и себя от приложений, ОС тоже нужна помощь со стороны оборудования. Наконец, нам потребуется еще кое-что от системы ВП, а именно *гибкость*: мы хотим, чтобы программы могли пользоваться своим адресным пространством, как им заблагорассудится, что должно упростить программирование системы.

СУЩЕСТВО ПРОБЛЕМЫ: КАК ЭФФЕКТИВНО И ГИБКО ВИРТУАЛИЗИРОВАТЬ ПАМЯТЬ

Как организовать эффективную виртуализацию памяти? Как обеспечить необходимую приложениям гибкость? Как сохранить контроль над тем, какие области памяти доступны приложению, и тем самым гарантировать, что возможность обращения к памяти надежно ограничена? И как все это сделать эффективно?

Общая техника, которую мы будем рассматривать как добавление к стратегии ограниченного прямого выполнения, называется **аппаратной транс-**

ляцией адресов, или просто **трансляцией адресов**. Идея в том, что оборудование преобразует каждую операцию доступа к памяти (выборку команд, загрузку и сохранение данных), заменяя **виртуальный** адрес в команде **физическим** адресом, по которому располагается нужная информация. Таким образом, трансляция адресов производится оборудованием для всех без исключения обращений к памяти.

Разумеется, оборудование не может виртуализировать память в одиночку, оно лишь предоставляет низкоуровневый механизм, позволяющий сделать это эффективно. ОС должна вмешиваться в стратегически важных точках, чтобы настроить оборудование, т. е. она должна **управлять памятью**, запоминать, какие участки свободны, а какие используются, и рассудительно руководить, чтобы сохранить контроль над использованием памяти.

Как и раньше, цель всей этой работы – создать красивую **иллюзию**, будто программа располагает собственной частной памятью, в которой располагаются ее код и данные. Но за этой виртуальной видимостью лежит уродливая правда жизни: много программ разделяют между собой память одновременно, точно так же, как процессор (или несколько процессоров) переключается с одной программы на другую. Посредством виртуализации ОС (при поддержке оборудования) превращает эту неприглядную реальность в полезную, мощную и простую для использования абстракцию.

15.1. ПРЕДПОЛОЖЕНИЯ

Наши первые попытки виртуализировать память будут до смешного просты. Ну и ладно, смейтесь, сколько хотите; очень скоро настанет черед смеяться ОС, когда вы будете ломать голову, пытаясь разобраться в хитросплетениях TLB, многоуровневых таблиц структур и других технических чудесах. Не нравится такой поворот? Ну, что поделать, так уж ОС устроена.

Итак, будем предполагать, что пользовательское адресное пространство должно занимать *непрерывный* участок физической памяти. Для простоты предположим также, что размер адресного пространства не слишком велик, а именно что он *меньше размера физической памяти*. Наконец, предположим еще, что все адресные пространства в точности *одинакового размера*. Не переживайте, если эти предположения покажутся нереалистичными, по ходу дела мы их ослабим и в итоге придем к реалистичной виртуализации памяти.

15.2. ПРИМЕР

Чтобы лучше понять, что необходимо для трансляции адресов и почему нам нужен такой механизм, рассмотрим простой пример. Представим, что адресное пространство некоторого процесса выглядит, как показано на рис. 15.1. Мы хотим изучить короткий код, который загружает значение из памяти, прибавляет к нему 3 и сохраняет результат обратно в памяти. На С такой код может выглядеть следующим образом:

```
void func() {
    int x = 3000;
    x = x + 3;    // нас интересует эта строчка
    ...
}
```

СОВЕТ: ВМЕШАТЕЛЬСТВО – ЭТО КРУТО

Вмешательство – общий и весьма эффективный метод, часто применяемый в компьютерных системах. При виртуализации памяти оборудование вмешивается при каждом обращении к памяти и транслирует виртуальные адреса процесса в физические. Однако спектр применения этой техники гораздо шире; почти в любой четко определенный интерфейс можно вклиниться, чтобы добавить новую функциональность или что-то улучшить. Одно из самых известных достоинств такого подхода – **прозрачность**; вмешательство часто производится без изменения клиентской части интерфейса, поэтому и клиента модифицировать не приходится.

Компилятор преобразует эту строчку в ассемблерный код, который может выглядеть, как показано ниже (в случае процессора x86). Для дизассемблирования можете воспользоваться программой `objdump` в Linux или `otool` в Mac:

```
128: movl 0x0(%ebx), %eax ; загрузить 0+ebx в eax
132: addl $0x03, %eax      ; прибавить 3 к регистру eax
135: movl %eax, 0x0(%ebx) ; сохранить eax в памяти
```

В этом фрагменте нет ничего сложного; предполагается, что адрес `x` помещен в регистр `ebx`, а затем значение, хранящееся по этому адресу, загружено в регистр общего назначения `eax` командой `movl` («переместить длинное слово»). Следующая команда прибавляет 3 к `eax`, а последняя сохраняет находящееся в `eax` значение в памяти по тому же адресу, откуда оно было взято.

На рис. 15.1 показано, как код и данные размещены в адресном пространстве процесса; указанная последовательность трех команд начинается с адреса 128 (близко к началу кода), а значение переменной `x` хранится по адресу 15 КБ (ближе к концу стека). Начальное значение `x` равно 3000, что и показано на рисунке.

Когда эти команды выполняются, с точки зрения процесса имеют место следующие обращения к памяти:

- выбрать команду по адресу 128;
- выполнить эту команду (загрузить данные по адресу 15 КБ);
- выбрать команду по адресу 132;
- выполнить эту команду (без обращения к памяти);
- выбрать команду по адресу 135;
- выполнить эту команду (сохранить данные по адресу 15 КБ).

С точки зрения программы, ее **адресное пространство** начинается с адреса 0 и простирается до максимального адреса 16 КБ; все генерируемые адреса должны находиться в этом диапазоне. Но для виртуализации памяти ОС хочет поместить процесс в какое-то другое место в физической памяти,

необязательно начиная с адреса 0. Поэтому налицо проблема: как **переместить** процесс в памяти способом, **прозрачным** для самого процесса? Как создать иллюзию, что виртуальное адресное пространство начинается с 0, хотя в действительности она находится совсем в другом месте?

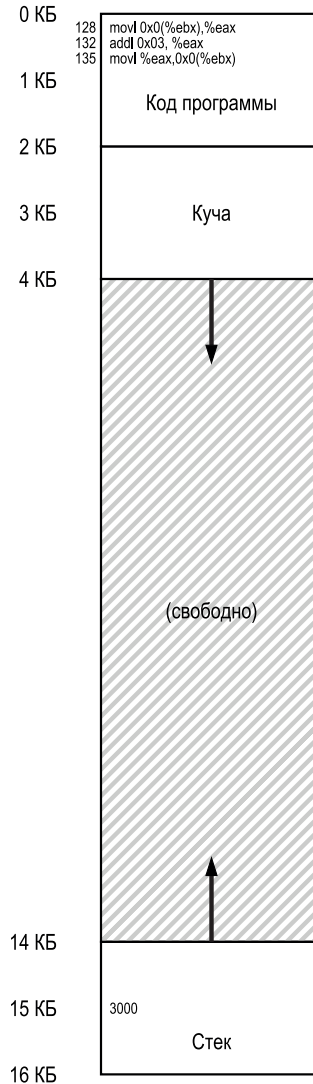


Рис. 15.1 ❖ Процесс и его адресное пространство

На рис. 15.2 показано, как может выглядеть физическая память, когда в ней размещено адресное пространство этого процесса. Мы видим, что ОС занимает начальную область физической памяти для себя и что она переместила наш процесс в область, начинающуюся с физического адреса 32 КБ,

которая расположена после нее самой. Оставшиеся два участка (16–32 КБ и 48–64 КБ) свободны.

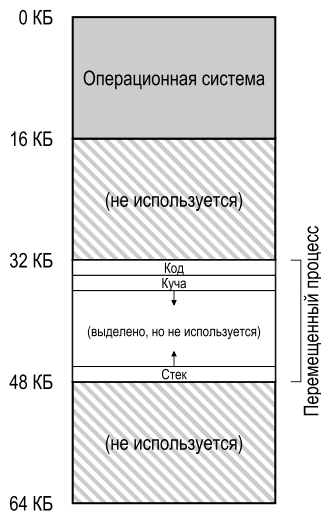


Рис. 15.2 ❖ Физическая память и один перемещенный процесс

15.3. ДИНАМИЧЕСКОЕ (АППАРАТНОЕ) ПЕРЕМЕЩЕНИЕ

Чтобы лучше разобраться в аппаратной трансляции адресов, обсудим для начала ее первое воплощение. В ранних машинах с разделением времени, появившихся в конце 1950-х годов, эта простая идея называлась **база и граница** (base and bounds), или **динамическое перемещение**; мы будем употреблять оба термина как синонимы [SS74].

Конкретно, в каждом CPU должно быть два регистра: один называется **базой**, другой – **границей**, или **пределом**. Смысл этой пары регистров состоит в том, чтобы поместить адресное пространство в любое место физической памяти таким образом, что процесс может получить доступ только к своему собственному адресному пространству.

При такой организации любая программа пишется и компилируется так, будто она загружена с нулевого адреса. Но когда программа запускается, ОС решает, в какое место физической памяти ее загрузить, и соответственно устанавливает регистр базы. В примере выше ОС решила загрузить процесс с физического адреса 32 КБ и записала это значение в регистр базы.

Интересное начинается, когда процесс приступает к выполнению. Любой адрес, сгенерированный процессом, **транслируется** оборудованием следующим образом:

физический адрес = виртуальный адрес + база

ОТСТУПЛЕНИЕ: ПРОГРАММНОЕ ПЕРЕМЕЩЕНИЕ

В начале времен, когда аппаратной поддержки еще не было, в некоторых системах перемещение производилось чисто программными методами. Основная техника называлась **статическим перемещением** – программа-загрузчик принимала подлежащий выполнению исполняемый файл и прибавляла к встречающимся в нем адресам нужное смещение относительно начала физической памяти.

Например, для команды загрузки значения по адресу 1000 в регистр (`movl 1000, %eax`), при условии что адресное пространство программы загружено начиная с адреса 3000 (а не 0, как думает программа), загрузчик переписал бы команду, прибавив смещение 3000 к адресу (`movl 4000, %eax`). Таким образом достигалось простое статическое перемещение адресного пространства процесса.

Однако со статическим перемещением сопряжено много проблем. Первая и самая главная – оно не обеспечивает защиты, потому что процесс может сгенерировать недопустимые адреса и тем самым незаконно получить доступ к другим процессам и даже к памяти ОС; в общем случае для настоящей защиты, вероятно, необходима аппаратная поддержка [WL+93]. Другая проблема заключается в том, что после размещения трудно переместить адрес в другое место [M65].

Любой адрес, сгенерированный процессом, **виртуальный**; оборудование затем прибавляет к нему содержимое регистра базы, и получается **физический адрес**, который можно передать подсистеме памяти.

Чтобы лучше понять это, посмотрим внимательно, что происходит при выполнении одной команды, а именно команды

```
128: movl 0x0(%ebx), %eax
```

Счетчик команд (PC) равен 128. Когда оборудованию нужно будет выбрать эту команду, оно сначала прибавит к PC значение в регистре базы (32 КБ = 32 768) и получит физический адрес 32 896, а затем выберет команду по этому адресу. Далее процессор начинает выполнять эту команду. В какой-то момент процесс требует загрузить данные по виртуальному адресу 15 КБ, к которому процессор снова прибавляет регистр базы (32 КБ), получает физический адрес 47 КБ и загружает хранящееся там значение.

Преобразование виртуального адреса в физический и называется **трансляцией адреса** – оборудование принимает виртуальный адрес, к которому обращается процесс (по его мнению), и преобразует его в физический адрес, где в действительности находятся данные. Поскольку перемещение производится на этапе выполнения и адресное пространство процесса можно перемещать, даже после того как он начал выполняться, этот метод называется **динамическим перемещением** [M65].

Возникает вопрос: а куда делся регистр границы? Ведь метод-то называется база и граница, разве не так? Так, так. И вы, наверное, уже догадались, что регистр границы имеет отношение к защите. Именно, процессор сначала проверяет, что адрес *не выходит за границы* и, стало быть, является допустимым; в простом примере выше в регистр границы записано значение 16 КБ. Если процесс генерирует виртуальный адрес, который больше границы или

отрицателен, то CPU возбudit исключение, и процесс, скорее всего, будет снят. Таким образом, цель регистра границы – гарантировать, что все генерируемые процессом адреса допустимы и не выходят за границы процесса.

СОВЕТ: АППАРАТНОЕ ДИНАМИЧЕСКОЕ ПЕРЕМЕЩЕНИЕ

С введением динамического перемещения крохотное оборудование совершило большой скачок. Именно, регистр **базы** используется для преобразования виртуальных адресов (генерируемых программой) в физические. А регистр **границы** (или **предела**) гарантирует, что адреса не выйдут за пределы адресного пространства. В совокупности они определяют простую и эффективную схему виртуализации памяти.

Отметим, что регистры базы и границы – это аппаратные конструкции, находящиеся на кристалле (по одной паре на каждый процессор). Иногда часть процессора, занимающуюся трансляцией адресов, называют **устройством управления памятью** (memory management unit – MMU); по мере усложнения методов управления памятью мы будем добавлять в MMU новые цепи.

Сделаем небольшое отступление по поводу регистра границы. Его можно определить двумя способами. В первом случае (как выше) в нем хранится *размер* адресного пространства, поэтому оборудование сначала сравнивает с ним виртуальный адрес, а затем прибавляет базу. Во втором случае в нем хранится *физический адрес* конца адресного пространства, и тогда оборудование сначала прибавляет базу, а потом проверяет, что результат не вышел за границу. Оба способа логически эквивалентны, для простоты будем предполагать, что используется первый.

Пример трансляции

Чтобы глубже разобраться в трансляции методом базы и границы, рассмотрим пример. Пусть размер адресного пространства процесса равен 4 КБ (да, нереально мало), и этот процесс загружен начиная с физического адреса 16 КБ. Приведем результаты трансляции нескольких адресов.

Виртуальный адрес		Физический адрес
0	→	16 КБ
1 КБ	→	17 КБ
3000	→	19384
4400	→	Ошибка (выход за границу)

Как видим, совсем нетрудно прибавить базовый адрес к виртуальному (который можно рассматривать как *смещение* относительно начала адресного пространства) и получить физический адрес. Только если виртуальный адрес «слишком велик» или отрицателен, возникает ошибка, приводящая к исключению.

ОТСТУПЛЕНИЕ: СТРУКТУРА ДАННЫХ – СПИСОК СВОБОДНЫХ

ОС должна знать, какие участки памяти не используются, иначе она не сможет выделять память процессам. Для решения этой задачи можно придумать много разных структур данных, но самая простая (и дальше мы будем считать, что так и есть) – **список свободных**, т. е. обычный список диапазонов физической памяти, которые в данный момент не используются.

15.4. АППАРАТНАЯ ПОДДЕРЖКА: ИТОГИ

Давайте теперь подытожим, что нам нужно от оборудования (см. также рис. 15.3). Прежде всего, как мы говорили в главе о виртуализации процессора, требуется два режима работы CPU. ОС работает в **привилегированном режиме** (или **режиме ядра**), в котором имеет доступ ко всей машине, а приложения – в **режиме пользователя**, в котором их возможности ограничены. В каком режиме работает процессор, определяет один бит, который хранится в каком-то **слове состояния процессора**; при наступлении определенных событий (например, системный вызов или другой вид исключения или прерывания) CPU переключает режим.

Требования к оборудованию	Примечания
Привилегированный режим	<i>Необходимо, чтобы пользовательские процессы не могли выполнять привилегированные операции</i>
Регистры базы и границы	<i>Необходимо два регистра на каждый CPU для поддержки трансляции адресов и проверки выхода за границы адресного пространства</i>
Способность транслировать виртуальные адреса и проверять, не выходят ли они за границы	<i>Цепи для трансляции и проверки выхода за границы; в данном случае совсем простые</i>
Привилегированные команды для изменения регистров базы и границы	<i>ОС должна установить эти регистры перед запуском пользовательской программы</i>
Привилегированные команды для регистрации обработчиков исключений	<i>ОС должна сообщить оборудованию, что делать, когда в коде возникает исключение</i>
Способность возбуждать исключения	<i>Когда процесс пытается выполнить привилегированную команду или обратиться к памяти за пределами своего адресного пространства</i>

Рис. 15.3 ❖ Динамическое перемещение: требования к оборудованию

Оборудование должно также предоставить **регистры базы и границы**; пара таких регистров есть у каждого CPU и является частью **устройства управления памятью (MMU)**. Когда работает пользовательская программа, оборудование транслирует все виртуальные адреса, прибавляя к ним значение базы. Кроме того, оборудование должно проверять, является ли адрес допустимым, для чего используется регистр границы и определенные цепи внутри CPU.

Оборудование должно также предоставлять специальные команды для модификации регистров базы и границы, что позволит ОС изменять их при смене процессов. Эти команды **привилегированные**; изменять эти регистры можно только в режиме ядра. Представьте только, какой хаос мог бы учинить пользователь, если бы имел возможность загрузить произвольное значение в регистр базы! Представили? А теперь быстренько прогоните эти черные мысли, поскольку именно из такого материала ткутся ночные кошмары.

Наконец, CPU должен генерировать **исключения** в ситуациях, когда пользовательская программа пытается выполнить недопустимый доступ к памяти (по адресу, выходящему за границы); в этом случае CPU должен остановить выполнение программы и передать управление **обработчику исключения** «выход за границы» внутри ОС. Обработчик решает, как отреагировать, в данном случае он, скорее всего, снимет процесс. Аналогично, если пользовательская программа попытается изменить значения привилегированных регистров базы и границы, то CPU должен возбудить исключение и запустить обработчик «попытки выполнить привилегированную операцию в режиме пользователя». CPU должен также предоставить метод, позволяющий проинформировать о местоположении этих обработчиков, поэтому нужно еще несколько привилегированных команд.

15.5. ТРЕБОВАНИЯ К ОПЕРАЦИОННОЙ СИСТЕМЕ

Для поддержки динамического перемещения свою часть пути должно пройти не только оборудование, но и ОС, только сочетание того и другого позволит реализовать простую виртуальную память. Существует несколько критических точек, в которых ОС должна вмешаться, чтобы довести до логического завершения нашу версию виртуальной памяти на основе базы и границы.

Во-первых, ОС должна предпринять некоторые действия при создании процесса, а именно найти в памяти место для его адресного пространства. По счастью, в силу наших предположений, (а) каждое адресное пространство меньше физической памяти и (б) все они одного размера, поэтому задача ОС упрощается – она может рассматривать физическую память как массив слотов и запоминать, какие из них заняты, а какие свободны. При создании процесса ОС просматривает структуру данных (**список свободных**), пытаясь найти в ней место для адресного пространства, и затем помечает его как занятое. Если размеры адресных пространств могут различаться, то ситуация усложняется, но эту проблему мы оставим для последующих глав.

Рассмотрим пример. На рис. 15.2 видно, что ОС заняла первый слот физической памяти для себя, а процесс переместила в слот, начинающийся с физического адреса 32 КБ. Остальные два слота (16–32 КБ и 48–64 КБ) свободны, поэтому они должны войти в список свободных.

Во-вторых, ОС должна выполнить некоторые действия при завершении процесса (все равно, добровольном или принудительном, когда он был снят из-за неподобающего поведения), а именно вернуть всю занятую им память, чтобы ее можно было отдать другим процессам. Стало быть, по завершении

процесса ОС помещает его память назад в список свободных и соответственно модифицирует связанные с ним структуры данных.

В-третьих, ОС должна выполнять дополнительные действия при контекстном переключении. Ведь у каждого процессора есть только одна пара регистров базы и границы, и их значения меняются при смене работающей программы, потому что все программы загружаются в память по разным адресам. Таким образом, ОС должна *сохранять и восстанавливать* регистры базы и границы при контекстном переключении. Точнее, когда ОС решает приостановить работающий процесс, она должна сохранить его регистры базы и границы в структуре данных в памяти, которая называется **блоком управления процессом** (process control block – PCB). Аналогично, когда ОС возобновляет работу процесса (или запускает его впервые), она должна правильно установить регистры базы и границы.

Требования к ОС	Примечания
Управление памятью	<i>Необходимо для выделения памяти новым процессам и возврата памяти завершившихся процессов. Обычно для управления используется список свободных</i>
Управление регистрами базы и границы	<i>Должна перезагружать эти регистры при контекстном переключении</i>
Обработка исключений	<i>Код, который выполняется при возникновении исключения; чаще всего снимает сбойный процесс</i>

Рис. 15.4 ❖ Динамическое перемещение: обязанности операционной системы

Отметим, что когда процесс приостановлен (т. е. не выполняется), ОС легко может переместить его адресное пространство из одного места в другое. Для этого ОС сначала снимает процесс с процессора, затем копирует адресное пространство в новое место и напоследок изменяет сохраненный регистр базы (в блоке управления процессом), так чтобы он указывал на новое место. При возобновлении процесса его (новый) регистр базы восстанавливается, и процесс начинает выполняться, не ведая о том, что его команды и данные переехали в другое место.

В-четвертых, ОС должна предоставить **обработчики исключений**, т. е. функции, которые мы обсуждали выше. Эти обработчики ОС устанавливает на этапе начальной загрузки (с помощью привилегированных команд). Например, если процесс пытается обратиться к памяти за пределами своего адресного пространства, то процессор возбуждает исключение, а ОС должна быть готова к действиям, когда такое исключение возникнет. Типичная реакция ОС враждебная: она, скорее всего, завершит сбойный процесс. ОС должна в первую очередь думать о защите машины, на которой работает, поэтому не прощает процессу попытки обратиться к запрещенной для него памяти или выполнить команды, которые не для него предназначены. Бывай, дурной процесс, было приятно познакомиться.

На рис. 15.5 показана хронология взаимодействия ОС и оборудования в различных ситуациях. Мы видим, что делает ОС на этапе начальной загрузки, чтобы подготовить машину к работе, и что происходит, ког-

ОС на этапе начальной загрузки (режим ядра)	Оборудование	
Инициализировать таблицу прерываний	Запомнить адреса... обработчика системных вызовов обработчика прерывания от таймера обработчика ошибки доступа к памяти обработчика выполнения недопустимых команд	
Запустить таймер	Запустить таймер, прервать через X мс	
Инициализировать таблицу процессов		
Инициализировать список свободных		
ОС на этапе выполнения (режим ядра)	Оборудование	Программа (режим пользователя)
Чтобы запустить процесс А: выделить запись в таблице процессов; выделить память для процесса; установить регистры базы и границы; вернуться из прерывания (в А)	Восстановить регистры А Перейти в режим пользователя Перейти по адресу, хранящемуся в РС (начальному)	Процесс А исполняется Выбрать команду
	Транслировать виртуальный адрес и выполнить выборку	Выполнить команду
	Если команда явной загрузки или сохранения: проверить, что адрес не выходит за границы; транслировать виртуальный адрес и выполнить загрузку или сохранение	...
	Прерывание от таймера Перейти в режим ядра Перейти на начало обработчика прерывания	
Обработать системное прерывание Вызвать функцию switch() сохранить регистры(А) в РСВ(А) (включая регистры базы и границы); восстановить регистры(В) из РСВ(В) (включая регистры базы и границы) Вернуться из прерывания (в В)	Восстановить регистры В Перейти в режим пользователя Перейти по адресу, хранящемуся в РС В	Процесс В исполняется Выполнить некорректную команду загрузки
	Адрес за границами; Перейти в режим ядра Перейти на начало обработчика системного прерывания	
Обработать системное прерывание Решение завершить процесс В Освободить память В Освободить запись В в таблице процессов		

Рис. 15.5 ❖ Протокол ограниченного прямого выполнения (динамическое перемещение)

да процесс (Процесс А) начинает выполняться. Обратите внимание, что трансляция адресов производится оборудованием без участия ОС. В какой-то момент происходит прерывание от таймера, и ОС переключается на процесс В, который выполняет некорректную команду загрузки (из недоступного ему адреса); в этот момент ОС вмешивается, завершает процесс, освобождает его память и удаляет запись о нем из таблицы процессов. Как следует из диаграммы, мы по-прежнему придерживаемся стратегии **ограниченного прямого выполнения**. В большинстве случаев ОС просто настраивает оборудование и позволяет процессу исполняться непосредственно на CPU, и лишь когда процесс ведет себя неподобающим образом, ОС вмешивается.

15.6. РЕЗЮМЕ

В этой главе мы расширили концепцию ограниченного прямого выполнения, добавив механизм виртуализации памяти – **трансляцию адресов**. Располагая этим механизмом, ОС может контролировать любой доступ процесса к памяти и гарантировать, что процесс никогда не обратится к памяти вне своего адресного пространства. Ключом к эффективности этого метода является аппаратная поддержка, благодаря которой трансляция, т. е. преобразование виртуального адреса (взгляд на память со стороны процесса) в физический (истинная картина), производится очень быстро. Причем все это делается *прозрачно* для перемещенного процесса; процесс вообще не в курсе, что все его обращения к памяти подвергаются трансляции, так что иллюзия полная.

Мы также рассмотрели одну конкретную форму визуализации – методы базы и границы, или динамическое перемещение. Этот метод весьма эффективен, поскольку для прибавления регистра базы к виртуальному адресу и проверки того, что сгенерированный процессом адрес не выходит за границы его адресного пространства, нужно очень немного аппаратной логики. Динамическое перемещение обеспечивает также *защиту*: ОС и оборудование совместными усилиями гарантируют, что процесс не может обратиться к памяти вне своего адресного пространства. Защита – безусловно, одна из главных целей ОС, без нее ОС не могла бы управлять машиной (если бы процессы могли перезаписывать чужую память, то ничто не помешало бы им переустановить таблицу прерываний и перехватить контроль над системой).

К сожалению, у этого простого метода динамического перемещения есть недостатки. Например, на рис. 15.2 показано, что перемещенный процесс использует область физической памяти от 32 до 48 КБ; однако стек и куча процесса невелики, так что все пространство между ними *растрачено впустую*. Такие бесполезные потери обычно называются **внутренней фрагментацией**, поскольку место *внутри* выделенного блока используется не полностью, а значит, потеряно. В описанном подходе памяти, быть может, хватило бы места еще для нескольких процессов, но мы распределяем ее блоками фикс-

рованного размера, из-за чего возникает внутренняя фрагментация¹. Таким образом, нам необходимы более изощренные механизмы, которые позволили бы эффективнее использовать физическую память, избежав внутренней фрагментации. Первой попыткой в этом направлении будет небольшое обобщение подхода на основе базы и границы, называемое **сегментацией**.

Литература

[M65] «On Dynamic Program Relocation» by W.C. McGee. IBM Systems Journal, Volume 4:3, 1965, pages 184–199. *Эта статья – прекрасный обзор ранних работ по динамическому перемещению, излагаются также основы статического перемещения.*

[P90] «Relocating loader for MS-DOS .EXE executable files» by Kenneth D. A. Pillay. Microprocessors & Microsystems archive, Volume 14:7 (September 1990). *Пример перемещающего загрузчика для MS-DOS. Не первый, но довольно современный пример такой системы.*

[SS74] «The Protection of Information in Computer Systems» by J. Saltzer and M. Schroeder. CACM, July 1974. *Цитата из этой статьи: «Идеи регистров базы и границы и аппаратно интерпретируемых дескрипторов появились, по всей видимости, независимо, между 1957 и 1959 годом в трех проектах с разными целями. В M.I.T. Маккарти предложил идею базы и границы как часть системы защиты памяти, необходимой для реализации разделения времени. IBM независимо разработала регистры базы и границы как механизм надежного мультипрограммирования в вычислительной системе Stretch (7030). В компании Burroughs Р. Бартон предположил, что аппаратно интерпретируемые дескрипторы смогут предоставить прямую поддержку правил видимости имен в языках высокого уровня, применявшихся в системе B5000». Мы нашли эту цитату в интереснейших исторических заметках Марка Смотермана [S04]; почитайте их, если интересна дополнительная информация.*

[S04] «System Call Support» by Mark Smotherman. May 2004. people.cs.clemson.edu/~mark/syscall.html. *История поддержки системных вызовов. Смотерман собрал также исторические факты о прерываниях и других интересных аспектах истории компьютеров. Дополнительные сведения см. на его веб-страницах.*

[WL+93] «Efficient Software-based Fault Isolation» by Robert Wahbe, Steven Lucco, Thomas E. Anderson, Susan L. Graham. SOSP '93. *Потрясающая статья о том, как можно использовать поддержку со стороны компилятора, чтобы ограничить адресацию памяти, не прибегая к аппаратной поддержке. Статья всколыхнула интерес к программным методам изоляции ссылок.*

¹ Альтернативное решение – разместить в адресном пространстве, сразу под областью кода, стек фиксированного размера, а под ним растущую кучу. Но тогда мы потеряем в гибкости, потому что возникнут проблемы с рекурсией и глубоко вложенными функциями, а нам бы хотелось этого избежать.

Домашнее задание (эмуляция)

Программа `relocation.py` позволяет увидеть, как выполняется трансляция адресов в системе с регистрами базы и границы. Детали см. в файле `README`.

Вопросы

1. Запустите с начальными значениями 1, 2 и 3 и вычислите, выходит виртуальный адрес, сгенерированный процессом, за границы адресного пространства или нет. Если не выходит, вычислите результат трансляции.
2. Запустите с флагами `-s 0 -n 10`. Какое значение следует присвоить флагу `-l` (регистр границы), чтобы все сгенерированные виртуальные адреса гарантированно попадали внутри адресного пространства?
3. Запустите с флагами `-s 1 -n 10 -l 100`. Какое максимальное значение можно присвоить регистру базы, чтобы адресное пространство полностью помещалось в физической памяти?
4. Выполните некоторые из предыдущих задач с большим адресным пространством (`-a`) и физической памятью (`-p`).
5. Выразите долю допустимых случайно сгенерированных виртуальных адресов в виде функции от значения регистра границы. Постройте график, выполнив программу с различными случайными начальными значениями, когда величина границы меняется от 0 до максимального размера адресного пространства.

Глава 16

Сегментация

До сих пор мы помещали все адресное пространство каждого процесса в память. При наличии регистров базы и границы ОС легко может перемещать процессы из одного места физической памяти в другое. Но вы, наверное, обратили внимание на интересную особенность этих адресных пространств: большой участок «свободного» пространства в середине, между стеком и кучей.

На рис. 16.1 видно, что хотя место между стеком и кучей не используется процессом, оно все равно занимает физическую память, куда бы мы ни переместили адресное пространство. Поэтому простой подход на основе регистров базы и границы приводит к расточительному расходованию памяти. Кроме того, очень трудно запустить программу, если ее адресное пространство не умещается целиком в физическую память. Таким образом, метод базы и границы не настолько гибкий, как нам хотелось бы.

Существо проблемы: как поддержать большое адресное пространство

Как поддержать большое адресное пространство, в котором может быть много свободного места между стеком и кучей? Заметим, что в наших примерах адресные пространства были крохотными, так что расточительство не кажется таким уж страшным. Но представьте себе 32-разрядное адресное пространство (размером 4 ГБ); типичная программа использует лишь несколько мегабайтов памяти, однако мы все равно требуем, что все адресное пространство находилось в памяти.

16.1. СЕГМЕНТАЦИЯ: ОБОБЩЕНИЕ ИДЕИ БАЗЫ И ГРАНИЦЫ

Для решения этой проблемы была выдвинута идея **сегментации**. Сама идея довольно старая, она восходит еще к началу 1960-х годов [H61, G62]. Смысл ее в том, чтобы вместо одной пары регистров базы и границы в MMU завести по паре для каждого логического **сегмента** адресного пространства. Сегментом называется непрерывный участок памяти определенной длины, и в нашем каноническом адресном пространстве имеется три логически различных

сегмента: кода, стека и кучи. Сегментация позволяет ОС поместить каждый сегмент в свою часть физической памяти и тем самым избежать замусоривания физической памяти неиспользуемыми виртуальными адресными пространствами.

Рассмотрим пример. Допустим, мы хотим разместить адресное пространство на рис. 16.1 в физической памяти. Если для каждого сегмента имеется отдельная пара регистров базы и границы, то их можно размещать *независимо*. Например, на рис. 16.2 показана физическая память размером 64 КБ с этими тремя сегментами (и еще 16 КБ отведено под ОС).

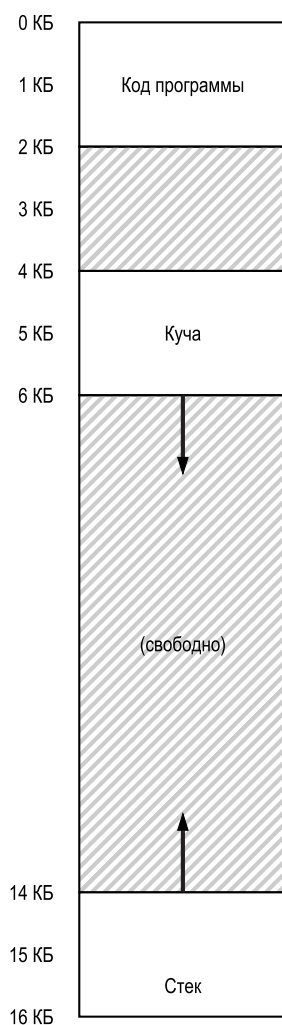


Рис. 16.1 ❖ Адресное пространство (повтор)

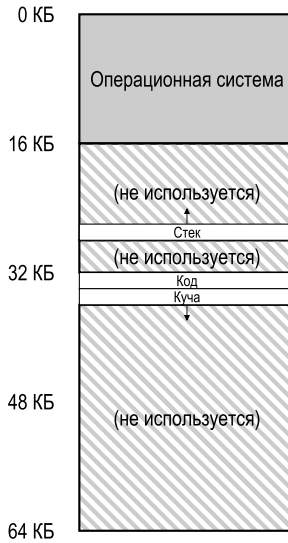


Рис. 16.2 ❖ Размещение сегментов в физической памяти

Как видим, в физической памяти располагаются только используемые области памяти, поэтому можно разместить большие адресные пространства, в которых имеются длинные неиспользуемые участки (иногда такие адресные пространства называются **разреженными**).

Для поддержки сегментации, как и следовало ожидать, нужны аппаратные схемы в MMU: в данном случае набор из трех пар регистров базы и границы. На рис. 16.3 ниже показаны значения регистров для рассматриваемого примера; в регистрах границ хранятся размеры сегментов.

Сегмент	База	Размер
Код	32 КБ	2 КБ
Куча	34 КБ	2 КБ
Стек	28 КБ	2 КБ

Рис. 16.3 ❖ Значения сегментных регистров

Из рисунка видно, что сегмент кода размещен по физическому адресу 32 КБ и имеет размер 2 КБ, а сегмент кучи – по адресу 34 КБ и тоже имеет размер 2 КБ.

Давайте выполним трансляцию адресов на примере адресного пространства на рис. 16.1. Пусть производится обращение к виртуальному адресу 100 (он находится в сегменте кода). Видя обращение (например, во время выборки команды), оборудование прибавляет величину базы к *смещению* относительно начала сегмента (в данном случае 100) и получает искомый физический адрес: $100 + 32 \text{ КБ}$, или 32868. Затем оно проверяет, не вышел ли адрес за границы (100 меньше 2 КБ), находит, что нет, и выполняет обращение к физической памяти по адресу 32868.

ОТСТУПЛЕНИЕ: ОШИБКА СЕГМЕНТАЦИИ

Термин «ошибка сегментации» (segmentation fault или segmentation violation) означал попытку доступа к недопустимому адресу на машине с сегментированной памятью. Смешно, но термин прижился даже для машин, не поддерживающих сегментацию. Впрочем, охота смеяться пропадает, если вы не можете понять, почему никак не удастся устранить ошибку.

Теперь рассмотрим виртуальный адрес 4200 в куче (снова см. рис. 16.1). Если просто прибавить 4200 к базе кучи (34 КБ), то получится физический адрес 39016 – недопустимый. Сначала нужно выделить *смещение* относительно начала кучи, т. е. понять, к какому байту *в этом сегменте* относится адрес. Поскольку куча начинается с виртуального адреса 4 КБ (4096), то смещение равно $4200 - 4096 = 104$. Вот теперь нужно прибавить это смещение к физическому адресу в регистре базы (34 КБ), тогда получится искомый результат: 34920.

А что, если бы мы попробовали обратиться к недопустимому адресу, например 7 КБ, находящемуся за границей кучи? Понятно, что произошло бы: оборудование обнаружило бы, что адрес вышел за границы сегмента, и вызвало бы исключение, которое ОС обработала бы и, скорее всего, завершила сбойный процесс. Теперь вы знаете о происхождении знаменитого термина, которого как огня боятся все программисты на С: **ошибка сегментации**.

16.2. К какому сегменту мы обращаемся?

Оборудование использует сегментные регистры во время трансляции. Но откуда оно знает смещение относительно начала сегмента, да и вообще к какому сегменту относится адрес? Один из распространенных подходов, называемый **явным**, – разбивать адресное пространство на сегменты, анализируя несколько старших битов виртуального адреса; этот метод использовался в системе VAX/VMS [LL82]. В примере выше мы имеем три сегмента, поэтому для решения задачи хватит двух битов. Если два старших бита нашего 14-рядного виртуального адреса используются для выбора сегмента, то адрес имеет такую структуру:



Таким образом, если в нашем примере два старших бита равны 00, то оборудование знает, что виртуальный адрес находится в сегменте кода, и использует соответствующие регистры базы и границы для трансляции этого адреса в физический. Если старшие биты равны 01, то оборудование знает,

что адрес находится в куче, и использует связанные с кучей регистры базы и границы. Возьмем наш пример виртуального адреса из кучи (4200) и транслируем его – просто чтобы убедиться, что все понятно. Виртуальный адрес 4200 в двоичном виде выглядит так:



Как видно из рисунка, два старших бита (01) говорят оборудованию, к какому *сегменту* мы обращаемся. Младшие 12 бит определяют смещение в этом сегменте: 0000 0110 1000, или 0x068h, или 104 в десятичной записи. Таким образом, оборудование просто выделяет первые два бита, чтобы понять, какой сегментный регистр использовать, а остальные 12 бит интерпретирует как смещение относительно начала этого сегмента. Прибавив регистр базы к смещению, оборудование находит окончательный физический адрес. Отметим, что знание смещения упрощает и проверку границы: нужно просто проверить, что смещение меньше регистра границы; если это не так, то адрес недопустим. Если бы базы и границы были массивами (по одному элементу на сегмент), то действия оборудования для получения искомого физического адреса можно было бы описать таким кодом:

```

1 // выделить 2 старших бита 14-разрядного виртуального адреса
2 Segment = (VirtualAddress & SEG_MASK) >> SEG_SHIFT
3 // получить смещение
4 Offset = VirtualAddress & OFFSET_MASK
5 if (Offset >= Bounds[Segment])
6     RaiseException(PROTECTION_FAULT)
7 else
8     PhysAddr = Base[Segment] + Offset
9     Register = AccessMemory(PhysAddr)
```

Для нашего сквозного примера мы можем задать конкретные значения констант в этом коде: SEG MASK должно быть равно 0x3000, SEG SHIFT – 12, а OFFSET MASK – 0xFFF.

Возможно, вы также обратили внимание, что если используется два старших бита, а сегментов всего три (код, куча, стек), то одна комбинация битов остается неиспользованной. Поэтому в некоторых системах код помещается в тот же сегмент, что и куча, тогда для выбора сегмента нужен всего один бит [LL82].

Существуют и другие способы аппаратного определения сегмента, которому принадлежит адрес. В случае **неявного** подхода оборудование определяет сегмент, анализируя, как сформирован адрес. Если, например, адрес сгенерирован по счетчику команд (т. е. это была выборка команды), значит, он находится в сегменте кода; если по указателю стека, то это должен быть сегмент стека; в противном случае адрес принадлежит куче.

16.3. А ЧТО НАСЧЕТ СТЕКА?

До сих пор мы не говорили о еще одном важном компоненте адресного пространства: стеке. На рисунке выше стек перемещен по физическому адресу 28 КБ, но есть одно важное отличие: *он растет в другую сторону*. В физической памяти стек начинается с адреса 28 КБ и растет вверх в сторону 26 КБ, что соответствует виртуальным адресам от 16 до 14 КБ; трансляция должна выполняться по-другому.

Прежде всего нам необходима небольшая помощь со стороны оборудования. Помимо значений базы и границы, оборудование должно еще знать, в каком направлении растет сегмент (достаточно одного бита, равного, например, 1, если сегмент растет вниз, и 0 в противном случае). Обновленная картина того, что запоминает оборудование, показана на рис. 16.4.

Сегмент	База	Размер	Растет вниз?
Код	32 КБ	2 КБ	1
Куча	34 КБ	2 КБ	1
Стек	28 КБ	2 КБ	0

Рис. 16.4 ❖ Сегментные регистры
(с поддержкой роста вверх)

Понимая, что сегменты могут расти вверх, оборудование должно изменить схему трансляции виртуальных адресов. Рассмотрим пример виртуального адреса в стеке и порядок его трансляции.

Пусть имеется обращение к виртуальному адресу 15 КБ, который должен быть отображен на физический адрес 27 КБ. В двоичном виде этот виртуальный адрес записывается как 11 1100 0000 0000 (0x3C00). Оборудование выделяет два старших бита (11), определяющих сегмент, после чего остается смещение 3 КБ. Чтобы получить правильное отрицательное смещение, необходимо вычесть из 3 КБ максимальный размер сегмента; в данном случае сегмент может достигать 4 КБ, поэтому смещение равно 3 КБ – 4 КБ = –1 КБ. Теперь прибавим это отрицательное смещение к базе (28 КБ) и получим правильный физический адрес: 27 КБ. Проверка границы должна удостоверить, что отрицательное смещение по абсолютной величине меньше размера сегмента.

16.4. ПОДДЕРЖКА РАЗДЕЛЕНИЯ

По мере распространения сегментации разработчики систем осознали, что можно еще повысить эффективность, совсем немного расширив аппаратную поддержку. Именно, для экономии памяти иногда полезно **разделять** некоторые сегменты между разными адресными пространствами. В частности, **разделение кода** применяется в системах и по сей день.

Для реализации разделения нужна дополнительная аппаратная поддержка в виде **битов защиты**. Для этого нужно как минимум добавить несколько

битов на каждый сегмент, показывающих, может ли программа читать сегмент, записывать в него или исполнять содержащийся в нем код. Если сегмент кода помечен как допускающий только чтение, то его можно разделять между несколькими процессами, не опасаясь нарушить изоляцию; каждый процесс по-прежнему думает, что обращается к своей частной памяти, но ОС тайком предоставляет в общее пользование память, которую процессы не могут модифицировать, поэтому иллюзия сохраняется.

Пример дополнительной информации, запоминаемой оборудованием (и ОС), приведен на рис. 16.5. Как видим, сегмент кода помечен как доступный для чтения и исполнения, поэтому один и тот же сегмент физической памяти можно отобразить на несколько виртуальных адресных пространств.

Сегмент	База	Размер	Растет вниз?	Защита
Код	32 КБ	2 КБ	1	Чтение-выполнение
Куча	34 КБ	2 КБ	1	Чтение-запись
Стек	28 КБ	2 КБ	0	Чтение-запись

Рис. 16.5 ❖ Значения сегментных регистров (с защитой)

После добавления битов защиты описанный выше алгоритм нуждается в изменении. При проверке попадания виртуального адреса в допустимый диапазон оборудование должно также проверять, разрешен ли запрошенный доступ. Если пользовательский процесс пытается записать в сегмент, допускающий только чтение, или выполнить код в неисполняемом сегменте, то оборудование должно возбудить исключение и тем самым препоручить нарушивший обязательства процесс заботам ОС.

16.5. МЕЛКОСТРУКТУРНАЯ И КРУПНОСТРУКТУРНАЯ СЕГМЕНТАЦИЯ

До сих пор мы обсуждали системы, в которых сегментов совсем мало (код, стек, куча); такую сегментацию можно назвать **крупноструктурной**, поскольку адресное пространство разбивается на сравнительно крупные части. Но некоторые ранние системы (например, Multics [CV65, DD68]) были более гибкими и допускали адресные пространства, состоящие из большого числа мелких сегментов; такая сегментация называется **мелкоструктурной**.

Для реализации большого количества сегментов требуется дополнительная аппаратная поддержка – в памяти нужно хранить **таблицу сегментов**. Такие таблицы обычно допускают создание очень большого числа сегментов, а значит, позволяют системе использовать сегменты более гибко, чем описано выше. Например, одна из ранних машин, Burroughs B5000, поддерживала тысячи сегментов, и ожидалось, что компилятор будет «нарезать» код и данные на отдельные сегменты, которые ОС вкупе с оборудованием потом смогут обслужить [RK68]. Тогда считалось, что при наличии мелких сегмен-

тов ОС сможет получить более полную информацию о том, какие сегменты используются, а какие нет, и таким образом будет использовать память эффективнее.

16.6. ПОДДЕРЖКА СО СТОРОНЫ ОС

Теперь вы примерно представляете, как работает сегментация. В ходе работы системы части адресного пространства перемещаются в физической памяти, за счет чего достигается огромная экономия памяти по сравнению с более простым подходом на основе единственной пары регистров базы и границы для всего адресного пространства. Точнее, неиспользуемое пространство между стеком и кучей необязательно выделять в физической памяти, а значит, мы можем уместить в физической памяти больше адресных пространств.

Однако сегментация поднимает ряд новых вопросов. Сначала опишем, какие новые проблемы должна разрешить ОС. Первый нам уже знаком: что должна делать ОС при контекстном переключении? Наверное, вы уже догадываетесь: необходимо сохранять и восстанавливать сегментные регистры. Понятно, что у каждого процесса имеется свое адресное пространство, и ОС должна правильно настроить эти регистры перед повторным запуском процесса.

Вторая, более важная проблема – управление свободным местом в физической памяти. При создании нового адресного пространства ОС должна найти в физической памяти место для его сегментов. Раньше мы предполагали, что размеры всех адресных пространств одинаковы, поэтому физическую память можно рассматривать как набор слотов, в которые должны уместиться процессы. Теперь же процесс состоит из нескольких сегментов, и все они могут быть разного размера.

Общая проблема заключается в том, что в физической памяти очень скоро образуется множество небольших участков свободного пространства, «дыр», что не дает возможности выделить место для новых сегментов или увеличить размер существующих. Эта проблема называется внешней сегментацией [R69]; см. рис. 16.6 (слева).

В этом примере новый процесс хочет выделить сегмент размером 20 КБ. Свободной памяти у нас 24 КБ, но она расположена не в одном непрерывном участке, а в трех несмежных. Поэтому ОС не может удовлетворить запрос на выделение 20 КБ.

Одно из возможных решений – **уплотнить** физическую память путем реорганизации имеющихся сегментов. Например, ОС могла бы приостановить все работающие процессы, скопировать их данные в одну непрерывную область памяти, изменить значение сегментных регистров, записав в них новые физические адреса, и тем самым освободить большой участок памяти для работы. В результате ОС сможет удовлетворить новый запрос на выделение памяти. Но такое уплотнение обходится дорого, поскольку для копирования сегментов нужно много процессорного времени. Уплотненная физическая память показана на рис. 16.6 (справа).

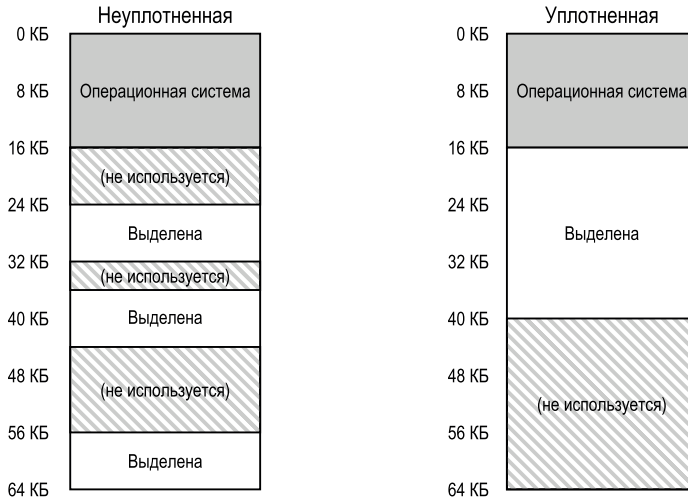


Рис. 16.6 ❖ Неуплотненная и уплотненная память

Проще написать алгоритм управления списком свободных, так чтобы он оставлял большие участки памяти свободными и доступными для выделения. Существуют буквально сотни подобных подходов, включая такие классические алгоритмы, как **лучший подходящий** (хранить список свободных участков и возвращать тот, размер которого ближе всего к запрошенному), **худший подходящий**, **первый подходящий** и более сложные схемы, например **алгоритм близнецов** [K68]. Отличный обзор Wilson et al. [W+95] – неплохая отправная точка для тех, кто хочет больше узнать о таких алгоритмах. Или можно дожидаться описания основных идей в следующей главе. Но, к сожалению, каким бы изощренным ни был алгоритм, внешняя фрагментация все равно существует, так что хороший алгоритм лишь пытается свести ее к минимуму.

Совет: если существует 1000 решений, то лучшего среди них нет

Тот факт, что алгоритмов, пытающихся минимизировать внешнюю фрагментацию, так много, – свидетельство неприглядной истины: не существует одного «лучшего» решения. Поэтому мы должны согласиться на что-то разумное и надеяться, что этого будет достаточно. Единственное настоящее решение (как мы увидим в последующих главах) – вообще уйти от проблемы и никогда не выделять память блоками разного размера.

16.7. РЕЗЮМЕ

Сегментация решает целый ряд проблем и помогает сконструировать более эффективную систему виртуализации памяти. Сегментация не только реализует динамическое перемещение, но и позволяет лучше поддерживать разре-

женные адресные пространства, поскольку избегает потенциально безумного расточительства, связанного с резервированием памяти между логическими сегментами. Кроме того, она работает быстро, потому что арифметические операции с сегментными регистрами просты и эффективно реализуются аппаратно; накладные расходы на трансляцию адресов минимальны. Есть и побочная выгода: разделение кода. Если поместить код в отдельный сегмент, то потенциально такой сегмент можно будет разделить между несколькими работающими программами.

Однако, как мы выяснили, выделение памяти блоками переменного размера ведет к некоторым проблемам, которые хотелось бы преодолеть. Первая из них – внешняя фрагментация. Поскольку размеры сегментов различны, свободная память выделяется неоднородными кусками, из-за чего удовлетворение запроса на выделение памяти может столкнуться с трудностями. Можно попытаться применить изошренные алгоритмы [W+95] или периодически уплотнять память, но проблема имеет фундаментальную природу и полностью избежать ее трудно.

Вторая и, пожалуй, более важная проблема заключается в том, что сегментация все же не обладает достаточной гибкостью для поддержки разреженных адресных пространств общего вида. Например, если имеется большая, но используемая разреженно куча в одном логическом сегменте, то вся она по-прежнему должна располагаться в памяти. Иными словами, если наша модель использования адресного пространства не соответствует механизму сегментации, то сегментация будет работать плохо. Поэтому нужно искать другие решения. Готовы?

Литература

[CV65] «Introduction and Overview of the Multics System» by F. J. Corbato, V. A. Vysotsky. Fall Joint Computer Conference, 1965. *Одна из пяти работ по системе Multics, представленных на Объединенной осенней конференции по компьютерам. Эх, хотя бы мухой на стене побывать на той конференции.*

[DD68] «Virtual Memory, Processes, and Sharing in Multics» by Robert C. Daley and Jack B. Dennis. Communications of the ACM, Volume 11:5, May 1968. *Ранняя работа по динамической компоновке в Multics, далеко опередившая свое время. Динамическая компоновка в конце концов добралась до систем примерно 20 лет спустя, когда потребовалась в больших библиотеках X-Windows. Говорят, что эти библиотеки X11 были мстью MIT, за исключением поддержки динамической компоновки из ранних версий Unix!*

[G62] «Fact Segmentation» by M. N. Greenfield. Proceedings of the SJCC, Volume 21, May 1962. *Еще одна ранняя работа по сегментации; настолько ранняя, что даже не содержит ссылок на другие работы.*

[H61] «Program Organization and Record Keeping for Dynamic Storage» by A. W. Holt. Communications of the ACM, Volume 4:10, October 1961. *Очень ранняя и трудная для чтения работа о сегментации и некоторых ее применениях.*

[I09] «Intel 64 and IA-32 Architectures Software Developer's Manuals» by Intel. 2009. Доступно по адресу <http://www.intel.com/products/processor/manuals>. *Попробуйте почитать о сегментации здесь (глава 3 в томе 3а); это заставит вас поднапрячься.*

[K68] «The Art of Computer Programming: Volume I» by Donald Knuth. Addison-Wesley, 1968. *Кнут знаменит не только своими ранними книгами по «искусству программирования», но и разработанной им системой типографской верстки TeX, которая все еще используется профессионалами, в т. ч. для верстки этой книги. Его многотомный труд – прекрасный ранний справочник по многочисленным алгоритмам, лежащим в основе современных компьютерных систем.*

[L83] «Hints for Computer Systems Design» by Butler Lampson. ACM Operating Systems Review, 15:5, October 1983. *Сокровищница знаний о построении систем. Прочитать за один присест трудно, нужно распробовать, как хорошее вино. Или как справочное руководство.*

[LL82] «Virtual Memory Management in the VAX/VMS Operating System» by Henry M. Levy, Peter H. Lipman. IEEE Computer, Volume 15:3, March 1982. *Классическая система управления памятью, в проект которой заложено немало здравых идей. В следующей главе мы изучим ее подробнее.*

[RK68] «Dynamic Storage Allocation Systems» by B. Randell and C. J. Kuehner. Communications of the ACM, Volume 11:5, May 1968. *Прекрасный обзор различий между страничной организацией и сегментацией, включающий исторический экскурс по различным машинам.*

[R69] «A note on storage fragmentation and program segmentation» by Brian Randell. Communications of the ACM, Volume 12:7, July 1969. *Одна из первых работ, где обсуждается фрагментация.*

[W+95] «Dynamic Storage Allocation: A Survey and Critical Review» by Paul R. Wilson, Mark S. Johnstone, Michael Neely, David Boles. International Workshop on Memory Management, Scotland, UK, September 1995. *Великолепная обзорная статья по распределителям памяти.*

Домашнее задание (эмуляция)

Эта программа позволит вам понять, как выполняется трансляция адресов в системе с сегментацией. Детали см. в файле README.

Вопросы

1. Сначала посмотрим, как транслируются адреса в крохотном адресном пространстве. Ниже приведены наборы параметров с разными начальными значениями; попробуйте транслировать адреса.

```
segmentation.py -a 128 -p 512 -b 0 -l 20 -B 512 -L 20 -s 0
segmentation.py -a 128 -p 512 -b 0 -l 20 -B 512 -L 20 -s 1
segmentation.py -a 128 -p 512 -b 0 -l 20 -B 512 -L 20 -s 2
```

2. Теперь посмотрим, понимаете ли вы это сконструированное нами крохотное адресное пространство (с параметрами из предыдущей задачи). Каков наибольший допустимый виртуальный адрес в сегменте 0? А наименьший виртуальный адрес в сегменте 1? Каковы наименьший и наибольший *недопустимые* адреса во всем адресном пространстве? Наконец, как запустить программу `segmentation.py` с флагом `-A` для проверки ваших ответов?
3. Пусть имеется 16-разрядное адресное пространство в 128-разрядной физической памяти. Как нужно задать базу и границу, чтобы эмулятор генерировал следующие результаты трансляции для заданного потока адресов: допустимый, допустимый, ошибка, ..., ошибка, допустимый, допустимый? Решите задачу для следующих значений параметров:

```
segmentation.py -a 16 -p 128  
-A 0,1,2,3,4,5,6,7,8,9,10,11,12,13,14,15  
--b0 ? --l0 ? --b1 ? --l1 ?
```

4. Предположим, что требуется сгенерировать задачу, в которой приблизительно 90 % случайно сгенерированных виртуальных адресов допустимы (не приводят к ошибке сегментации). Как бы вы сконфигурировали эмулятор? Какие параметры важны для получения такого результата?
5. Сможете ли вы запустить эмулятор, так чтобы все виртуальные адреса были недопустимы? Как именно?

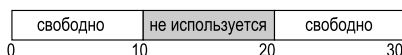
Глава 17

Управление свободным пространством

В этой главе мы сделаем небольшое отступление в сторону от обсуждения виртуализации памяти и поговорим о фундаментальном аспекте любой системы управления памятью, будь то библиотека `malloc` (для управления кучей процесса) или сама ОС (для управления частями адресного пространства). Конкретно мы обсудим проблемы, связанные с **управлением свободным пространством**.

Сначала уточним постановку задачи. Управление свободным пространством, конечно, может быть простым, как мы убедимся, когда будем рассматривать **страничную организацию**. Это просто, когда пространство, которым мы управляем, разделено на блоки фиксированного размера; в таком случае нужно лишь поддерживать список этих блоков, а в ответ на запрос клиента возвращать первый элемент списка.

Но проблема становится более трудной (и интересной), если свободное пространство состоит из блоков переменного размера; такая ситуация имеет место в библиотеках выделения памяти пользовательского уровня (например, `malloc()` и `free()`) и при управлении физической памятью со стороны ОС, когда для реализации виртуальной памяти используется сегментация. В обоих случаях проблема заключается во **внешней фрагментации**: свободное пространство «нарезается» на небольшие участки разных размеров, из-за чего возникает фрагментация; в последующих запросах может быть отказано, потому что не нашлось ни одного достаточно большого непрерывного участка, хотя общий размер свободного пространства превышает размер запроса.



На рисунке показан пример этой проблемы. Общий размер свободной памяти составляет 20 байт, но она разбита на два блока по 10 байт каждый.

Поэтому запрос на выделение 15 байт невозможно удовлетворить, хотя всего свободных байтов 20.

СУЩЕСТВО ПРОБЛЕМЫ: КАК УПРАВЛЯТЬ СВОБОДНЫМ ПРОСТРАНСТВОМ

Как следует управлять свободным пространством, чтобы удовлетворять запросы переменного размера? Какие стратегии можно использовать для минимизации фрагментации? Каковы накладные расходы различных подходов с точки зрения времени и пространства?

17.1. Предположения

Это обсуждение в основном посвящено великой истории распределителей памяти, которые встречаются в библиотеках пользовательского уровня. Мы опираемся на великолепный обзор Уилсона [W+95], но призываем интересующихся читателей ознакомиться с исходным документом, где имеются дополнительные сведения¹.

Мы будем рассматривать простейший интерфейс, предлагаемый функциями `malloc()` и `free()`. Именно, функция `void *malloc(size_t size)` принимает один параметр, `size` – количество запрашиваемых байтов, и возвращает указатель (без конкретизации типа, в языке C он называется **указателем на void**) на область этого (или большего) размера. Парная функция `void free(void *ptr)` принимает указатель и освобождает соответствующий блок. Отметим следствие такого интерфейса: при освобождении памяти пользователь не сообщает библиотеке о размере блока, а это значит, что библиотека сама должна определить этот размер, зная указатель на него. Как это делается, мы обсудим ниже в данной главе.

Пространство, управляемое библиотекой, исторически называется кучей, а общая структура данных для управления свободным пространством в куче – **списком свободных**. Эта структура содержит ссылки на все свободные участки в управляемой области памяти. Разумеется, она не является списком в строгом смысле слова, а просто позволяет отслеживать свободное пространство.

Далее будем предполагать, что нас интересует в первую очередь **внешняя фрагментация**, описанная выше. Распределители памяти могут, конечно, сталкиваться и с проблемой внутренней фрагментации; если распределитель возвращает блоки, большие, чем запрашивалось, то избыточное (а значит, неиспользуемое) пространство в таком блоке считается **внутренней фрагментацией** (поскольку выброшенное на ветер место находится внутри выделенного блока). Но для простоты и потому, что из двух типов фрагментации именно внешняя наиболее интересна, мы будем рассматривать только ее.

¹ Он занимает почти 80 страниц, так что интерес должен быть поистине велик!

Будем также предполагать, что после того как блок памяти передан клиенту, его нельзя перемещать в другое место. Например, если программа вызывает `malloc()` и получает указатель на какой-то блок в куче, то этот блок считается «принадлежащим» программе (и не может быть перемещен библиотекой) до тех пор, пока программа не вернет его, обратившись к `free()`. Таким образом, **уплотнение** свободного пространства невозможно, как бы полезно оно ни было для борьбы с фрагментацией¹. Однако сама ОС может использовать уплотнение при реализации сегментации (как было описано в предыдущей главе).

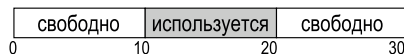
Наконец, будем предполагать, что распределитель работает с непрерывной областью байтов. В некоторых случаях распределитель мог бы попросить увеличить размер этой области, например пользовательская библиотека выделения памяти могла бы обратиться к ядру с просьбой увеличить кучу (с помощью системного вызова `sbrk()`), если ей не хватает места. Но для простоты будем считать, что область сохраняет размер на протяжении всего времени существования.

17.2. НИЗКОУРОВНЕВЫЕ МЕХАНИЗМЫ

Прежде чем переходить к деталям политики, рассмотрим общие механизмы, применяемые в большинстве распределителей памяти. Сначала обсудим основы разделения и объединения, затем покажем, как можно быстро и сравнительно легко определить размер выделенного блока. И наконец, поговорим о том, как встроить простой список прямо в свободные блоки, чтобы можно было находить свободные и занятые участки памяти.

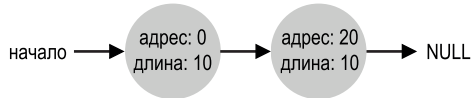
Разделение и объединение

Список свободных содержит элементы, описывающие свободное пространство, еще оставшееся в куче. Предположим, что имеется такая куча размером 30 байт:



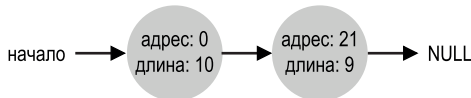
В списке свободных для этой кучи два элемента: один описывает первый свободный участок длиной 10 байт (с 0 по 9), а другой – второй свободный участок (байты 20–29):

¹ После того как указатель на блок памяти передан программе на С, очень трудно отследить все ссылки (указатели) на этот блок, потому что они могут храниться в переменных и даже в регистрах. В более сильно типизированных языках со сборкой мусора это необязательно так, поэтому уплотнение в качестве метода борьбы с фрагментацией в них возможно.



Как было сказано выше, запрос более 10 байт не будет удовлетворен (возвращается NULL), т. к. не существует одного непрерывного блока памяти такого размера. Запрос ровно на 10 байт легко удовлетворить, выделив любой из двух свободных блоков. Но что, если запрошено *меньше* 10 байт?

Пусть запрашивается всего один байт памяти. В этом случае распределитель выполнит операцию **разделения**: найдет свободный блок, способный удовлетворить запрос, и разобьет его на два. Первый блок будет возвращен вызывающей стороне, а второй останется в списке. В примере выше, если бы распределитель решил использовать второй из двух блоков в списке свободных, то вызов `malloc()` вернул бы 20 (адрес 1-байтовой выделенной области), а список принял бы такой вид:



Из рисунка видно, что список остался почти таким же, как был; единственное изменение состоит в том, что свободный блок теперь начинается с адреса 21, а не 20, а длина этого блока равна 9¹. Таким образом, разделение используется, когда размер запрашиваемого блока меньше размера доступного свободного блока.

Парный механизм, присутствующий во многих распределителях, называется **объединением** свободного пространства. Снова рассмотрим тот же пример (10 свободных байт, 10 занятых и снова 10 свободных).

Что произойдет в этой крохотной куче, если приложение вызовет функцию `free(10)`, чтобы освободить место в середине куче? Если, не мудрствуя лукаво, включить этот блок в список свободных, то получится такая картина:



И вот ведь какая проблема: свободна-то теперь вся куча, но выглядит это так, будто она разделена на три блока по 10 байт. Следовательно, если пользователь запросит 20 байт, то при простом обходе списка подходящий свободный блок не будет найден.

¹ Мы предполагаем, что у блоков нет заголовков – нереалистичное, но упрощающее допущение, которое мы пока примем.

Во избежание подобных проблем распределитель старается объединить свободное пространство после освобождения блока памяти. Идея проста: после освобождения блока посмотреть, какие блоки находятся слева и справа от него. Если с освобожденным блоком непосредственно соседствует один (или два, как в нашем примере) блок, то они объединяются в один больший свободный блок. Таким образом, после объединения список свободных будет выглядеть так:



Именно так список свободных выглядел в самом начале, когда из кучи еще не было выделено ни одного блока. Благодаря объединению для последующего выделения памяти оказываются доступны свободные участки большего размера.

Запоминание размеров выделенных блоков

Вы, наверное, заметили, что функция `free(void *ptr)` не принимает параметра размера, т. е. предполагается, что, зная указатель, библиотека `malloc` сама может быстро определить размер освобождаемой области и включить ее в список свободных.

Для этого в большинстве распределителей хранится небольшой **заголовок**, который обычно располагается непосредственно перед возвращенным пользователю блоком. Снова взгляните на рис. 17.1; нас интересует выделенный блок размера 20 байт, на который указывает `ptr`. Представьте, что пользователь вызвал `malloc()` и сохранил результат в `ptr`: `ptr = malloc(20);`.

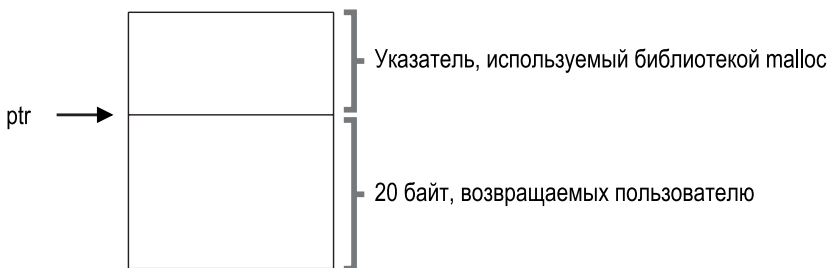


Рис. 17.1 ❖ Выделенная область плюс заголовок

Как минимум заголовок содержит размер выделенной области (в данном случае 20), но может также содержать дополнительные указатели, чтобы ускорить освобождение, специальную константу-признак для контроля целостности и другую информацию. Предположим, что заголовок простой и содержит только размер области и признак:

```
typedef struct __header_t {
    int size;
    int magic;
} header_t;
```

Наш пример будет выглядеть, как показано на рис. 17.2. Когда пользователь вызовет `free(ptr)`, библиотека выполнит простое арифметическое действие над указателем и найдет, где начинается заголовок:

```
void free(void *ptr) {
    header_t *hptr = (void *)ptr - sizeof(header_t);
    ...
}
```

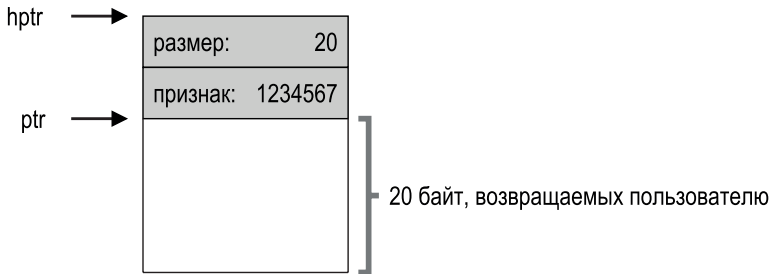


Рис. 17.2 ❖ Содержимое заголовка

Получив этот указатель на заголовок, библиотека легко проверит целостность, сравнив признак с ожидаемым значением (`assert(hptr->magic == 1234567)`), и вычислит полный размер освобождаемой области (прибавив размер заголовка к размеру области). Обратите внимание на мелкую, но очень важную деталь в последнем предложении: размер освобождаемой области равен сумме размера заголовка и размера запрошенного пользователем блока. Следовательно, когда пользователь запрашивает N байт памяти, библиотека ищет свободный блок размера не N , а N плюс размер заголовка.

Встраивание списка свободных

До сих пор мы рассматривали список свободных концептуально – просто как список, описывающий свободные блоки памяти в куче. Но как построить такой список непосредственно в самом свободном пространстве?

Если речь идет об обычном списке, то для создания нового узла мы просто вызвали бы `malloc()`. Увы, внутри библиотеки выделения памяти так сделать нельзя! Мы должны каким-то образом построить список в самом свободном пространстве. Не переживайте, если звучит сюрреалистично; оно, конечно, так, но не настолько сюрреалистично, чтобы это было невозможно!

Пусть мы управляем областью памяти размером 4096 байт (т. е. 4К-байтовой кучей). Чтобы рассматривать ее как список свободных, мы предварительно должны этот список инициализировать: в начальный момент список должен

содержать один элемент размера 4096 (минус размер заголовка). Вот как выглядит описание узла списка:

```
typedef struct __node_t {
    int size;
    struct __node_t *next;
} node_t;
```

Теперь взглянем на код, который инициализирует кучу и помещает первый элемент списка свободных в это пространство. Предполагается, что куча строится в памяти, полученной путем обращения к системному вызову `mmap()`; это не единственный способ ее построения, но в рассматриваемом примере он нас устраивает. Вот как выглядит код:

```
// mmap() возвращает указатель на блок свободной памяти
node_t *head = mmap(NULL, 4096, PROT_READ|PROT_WRITE,
                    MAP_ANON|MAP_PRIVATE, -1, 0);
head->size = 4096 - sizeof(node_t);
head->next = NULL;
```

После выполнения этого кода список содержит один элемент размером 4088. Согласно, куча крохотная, но в качестве примера сойдет. Указатель `head` содержит адрес начала этого диапазона, пусть он равен 16 КБ (подошел бы любой виртуальный адрес). Визуально куча выглядит, как показано на рис. 17.3.

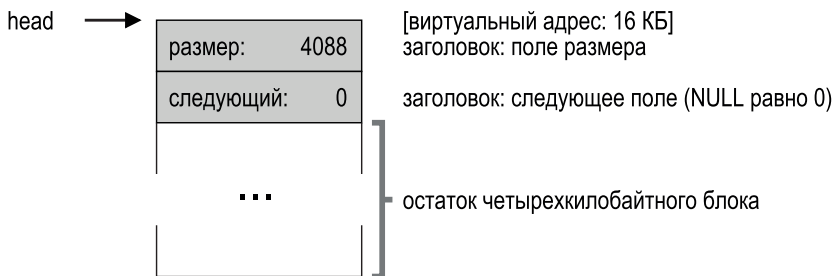


Рис. 17.3 ❖ Куча с одним свободным блоком

Теперь предположим, что запрошен блок памяти размером, скажем, 100 байт. Чтобы удовлетворить запрос, библиотека сначала ищет достаточно большой блок; поскольку существует всего один свободный блок (размером 4088), он и будет выбран. Затем этот блок **разделяется** на два: один достаточно для удовлетворения запроса (плюс заголовок), второй остается свободным. В предположении, что заголовок занимает 8 байт (два целых – размер и признак), куча теперь выглядит, как показано на рис. 17.4.

Таким образом, после запроса 100 байт библиотека выделила 108 байт из единственного существующего свободного блока, сохранила информацию заголовка в начале выделенного блока, для того чтобы впоследствии ей могла воспользоваться `free()`, уменьшила размер оставшегося свободного узла

списка до 3980 байт (4088 – 108) и вернула указатель на первый байт после заголовка (обозначен на рисунке ptr).



Рис. 17.4 ❖ Куча: после одного выделения памяти

Теперь посмотрим на кучу, из которой выделено три блока по 100 байт каждый (или 108, если учитывать заголовок). Виртуализация этой кучи показана на рис. 17.5. Как видим, теперь выделены первые 324 байта кучи, в этой области находится три заголовка и три 100-байтовых блока, используемых вызывающей программой. Список свободных по-прежнему неинтересен: единственный узел (на который указывает head), но размером уже 3764 байта (после трех разделений). А что будет, если вызывающая программа вернет в кучу часть памяти с помощью `free()`?

В этом примере приложение возвращает средний блок выделенной памяти, вызывая `free(16500)` (значение 16500 получено путем сложения адреса начала кучи, 16384, с размером предыдущего блока, 108, и размером заголовка этого блока, 8). Этому значению соответствует указатель `sptr` на рис. 17.5.

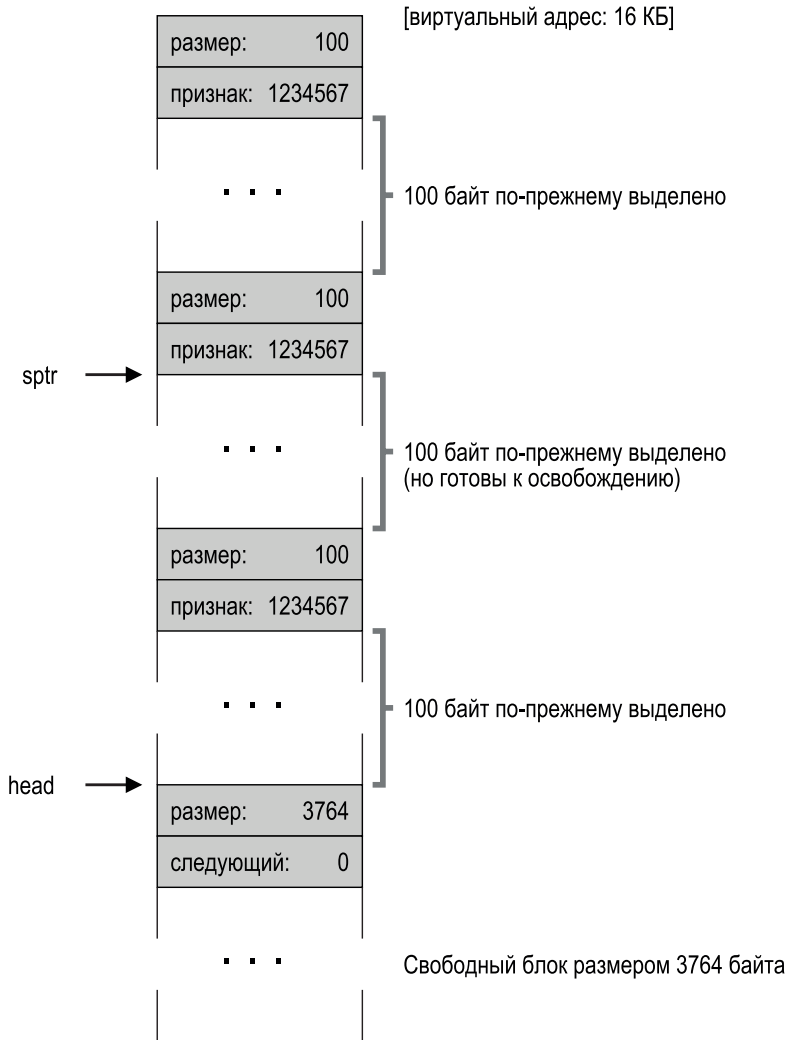


Рис. 17.5 ❖ Свободное пространство с тремя выделенными блоками

Библиотека вычисляет размер освобождаемого блока и помещает его обратно в список свободных. В предположении, что он вставлен в начало списка, куча теперь выглядит, как показано на рис. 17.6.

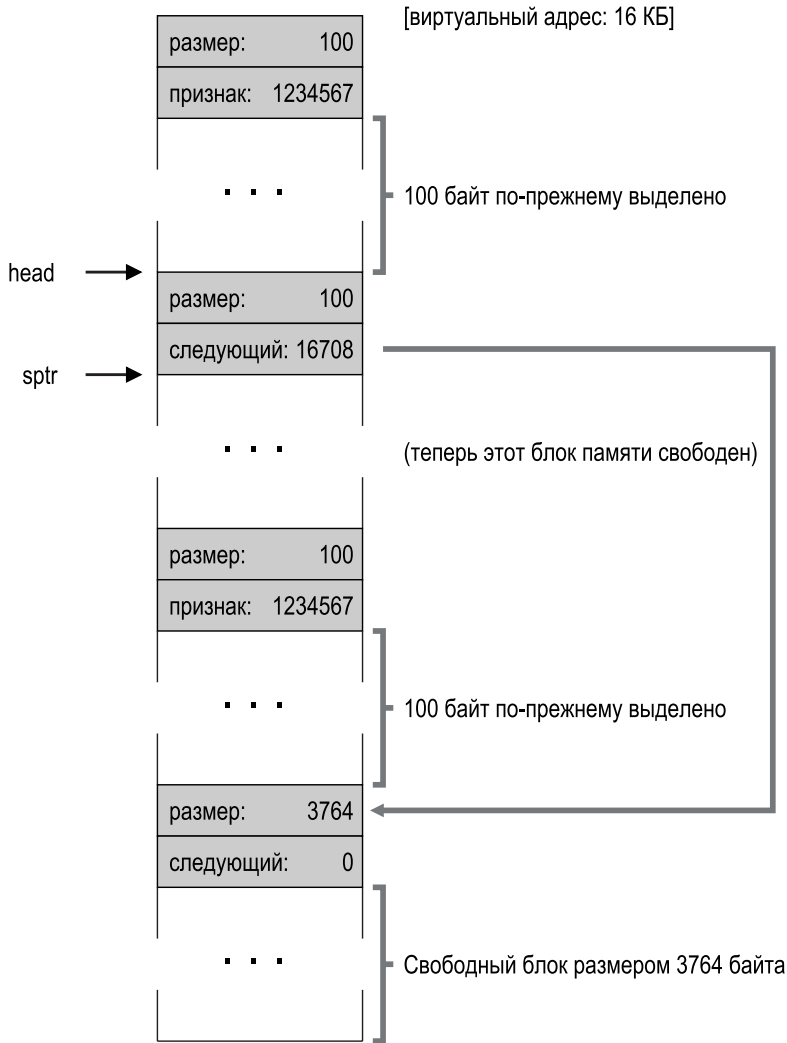


Рис. 17.6 ❖ Свободное пространство
с двумя выделенными блоками

Теперь список начинается небольшим свободным блоком (100 байт, на которые указывает *head*) и содержит еще один большой свободный блок (3764 байта). Наконец-то в списке оказалось больше одного элемента! И да, свободное пространство фрагментировано – печальное, но нередкое явление.

И последний пример: предположим, что и последние два используемых блока освобождены. Не будь объединения, мы могли бы получить сильно фрагментированный список (см. рис. 17.7).

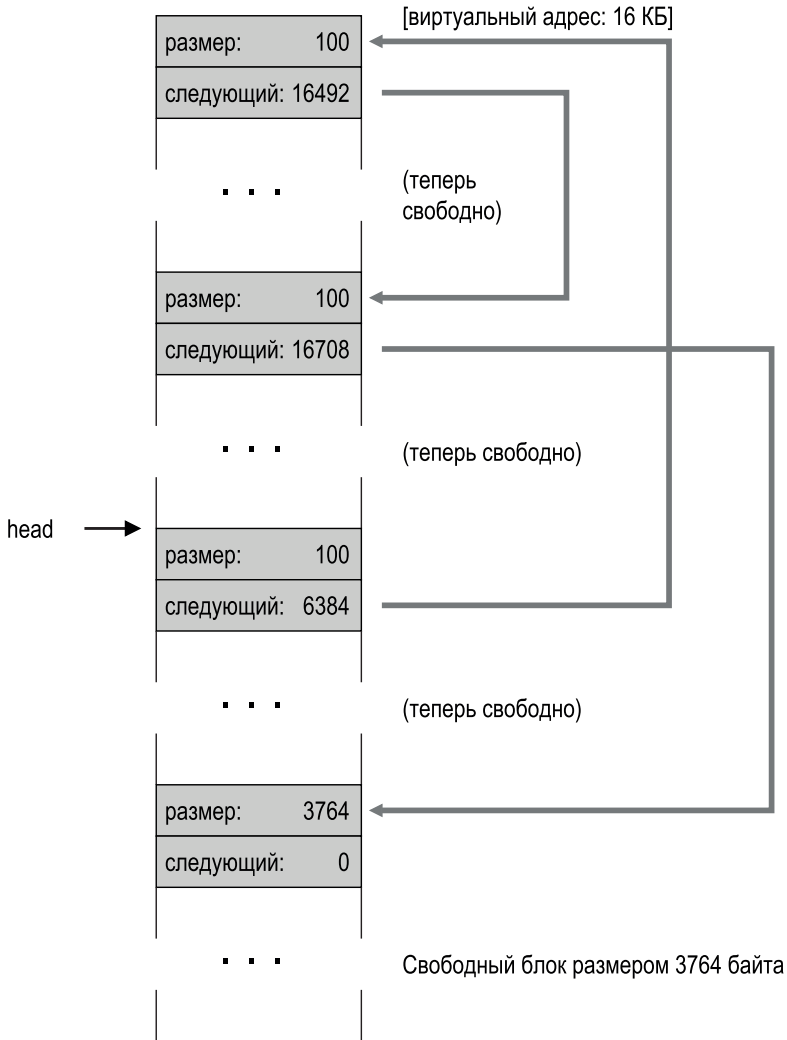


Рис. 17.7 ❖ Список свободных с необъединенными блоками

Как видно из рисунка, все перепуталось! Почему? Да все просто – мы забыли **объединить** список. Вся память свободна, но нарезана на кусочки, поэтому выглядит фрагментированной. Решение тоже простое – пройти по списку и объединить соседние блоки; по завершении куча снова станет единой.

Увеличение размера кучи

Осталось обсудить последний механизм, присутствующий во многих библиотеках работы с памятью: что делать, если в куче не осталось места? Проще всего вернуть ошибку. Иногда ничего другого и не остается, поэтому возврат

NULL – достойный подход. Не надо огорчаться! Вы старались и, хотя не победили, сражались хорошо.

В большинстве традиционных распределителей начальный размер кучи невелик, а затем по мере необходимости запрашивается дополнительная память у ОС. Обычно для этого выполняется тот или иной системный вызов (в большинстве Unix-систем `sbk`), после чего продолжается выделение новых блоков оттуда. Чтобы обслужить запрос `sbk`, ОС находит свободные физические страницы, отображает их на адресное пространство запросившего процесса и возвращает адрес конца новой кучи; в этот момент доступна куча большего размера, и запрос может быть успешно удовлетворен.

17.3. ОСНОВНЫЕ СТРАТЕГИИ

Теперь, располагая кое-какими механизмами, перейдем к рассмотрению основных стратегий управления свободным пространством. В основном эти подходы основаны на довольно простых политиках, которые вы и сами можете придумать; кстати, попробуйте, прежде чем читать дальше, и проверьте, удалось ли вам найти все варианты (а быть может, и какие-то новые!).

Идеальный распределитель должен быть быстрым и минимизировать фрагментацию. К сожалению, поскольку поток запросов на выделение и освобождение памяти может быть произвольным (по усмотрению программиста), любая наперед выбранная стратегия может оказаться очень плохой на не подходящих для нее данных. Поэтому мы не станем описывать «лучший» подход, а поговорим об основных и обсудим их плюсы и минусы.

Лучший подходящий

Стратегия **лучший подходящий** очень проста: сначала пробежаться по всему списку свободных и найти свободные блоки, размер которых больше или равен размеру запрошенного, а затем выбрать из них наименьший. Этот блок называется лучшим подходящим (можно было бы назвать его также наименьшим подходящим). Одного прохода по списку свободных достаточно для нахождения нужного блока.

Обоснование стратегии лучшего подходящего интуитивно понятно: возвращая блок, близкий по размеру к запрошенному, мы пытаемся уменьшить бесполезный расход памяти. Но за это приходится платить: при наивной реализации накладные расходы велики, поскольку нужно обойти весь список целиком.

Худший подходящий

Стратегия **худший подходящий** прямо противоположна: найти наибольший блок и вернуть из него запрошенный объем памяти, а оставшийся (большой) блок оставить в списке свободных. Смысл в том, чтобы оставлять свободными

ми большие блоки, а не множество мелких, как в подходе лучший подходящий. Но все равно необходим полный просмотр списка свободных, поэтому накладные расходы высоки. Хуже того, многие исследования показывают, что этот подход работает плохо – приводит к избыточной фрагментации, не отменяя больших издержек.

Первый подходящий

Стратегия **первый подходящий** находит первый достаточно большой блок и возвращает из него запрошенную память. Как и в остальных случаях, остаток помещается в список свободных для удовлетворения последующих запросов.

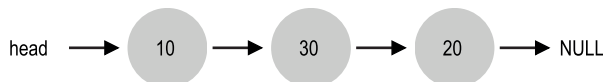
Преимуществом этого метода является скорость – исчерпывающий просмотр всех свободных блоков не нужен, – но иногда начало списка свободных засоряется мелкими объектами. Поэтому вопрос о том, как распределитель управляет порядком блоков в списке свободных, приобретает важность. Наш подход – **упорядочение по адресу**; если список хранится в порядке возрастания адресов свободных блоков, то объединение упрощается, а фрагментация имеет тенденцию к уменьшению.

Следующий подходящий

Алгоритм **следующий подходящий** начинает поиск первого подходящего не с начала списка, а хранит дополнительный указатель на место в списке, где закончился предыдущий поиск. Идея в том, чтобы более равномерно производить поиск свободных блоков во всем списке, избежав тем самым расщепления в его начале. Производительность такого подхода близка к методу первого подходящего, поскольку исчерпывающего просмотра тоже нет.

Примеры

Ниже приведено несколько примеров описанных стратегий. Рассматривается список свободных с тремя элементами размера 10, 30 и 20 (мы игнорируем заголовки и другие детали, а интересуемся только тем, как работают стратегии).



Пусть запрашивается блок размером 15. Метод лучшего подходящего просмотрит весь список и найдет, что лучшим подходящим является блок размером 20, поскольку это наименьший блок, способный удовлетворить запрос. После этого список свободных примет вид:



В этом примере и вообще при применении метода лучшего подходящего в списке остался небольшой блок. Метод худшего подходящего работал бы аналогично, но нашел наибольший блок размером 30. Вот как выглядит результирующий список:



В этом примере метод первого подходящего дает такой же результат, как метод худшего подходящего. Разница – в стоимости поиска: методы лучшего и худшего подходящего должны просмотреть весь список, а метод первого подходящего останавливается, как только находит первый подходящий блок, так что стоимость поиска уменьшается.

В этих примерах мы лишь слегка затронули тему политик выделения памяти. Для углубленного понимания необходим более детальный анализ с привлечением реальных рабочих нагрузок и более сложных поведений распределителя (например, объединения). Быть может, стоит заняться этим в домашнем задании?

17.4. Другие подходы

Помимо описанных выше основных подходов, было предложено много других методов и алгоритмов для улучшения выделения памяти. Мы перечислим несколько из них для самостоятельного изучения (чтобы заставить вас подумать немного больше, чем требует метод лучшего подходящего).

Сегрегированные списки

Интересный подход, применяемый уже довольно давно, – **сегрегированные списки**. Основная идея проста: если некоторое приложение особенно часто запрашивает блоки какого-то одного (или нескольких) размера, то следует завести отдельный список для объектов этого размера, а все остальные запросы переадресовывать общему распределителю памяти.

Преимущества этого подхода очевидны. Резервируя область памяти специально для запросов определенного размера, мы можем заметно снизить фрагментацию. Кроме того, запросы выделения и освобождения памяти такого специального размера можно обслужить очень быстро, поскольку не требуется сложный поиск в списке.

Как и любая хорошая идея, этот подход привносит и новые сложности в систему. Например, сколько памяти отвести под пул, из которого обслуживаются запросы данного размера? В одном конкретном, **слябовом** распределителе, созданном блестящим инженером Джеффом Бонвиком (для использования в ядре Solaris), эта проблема решается довольно изящно [B94].

На этапе начальной загрузки ядро выделяет некоторое количество **объектных кешей** для тех объектов ядра, которые с большой вероятностью будут запрашиваться часто (например, блокировки, индексные узлы файловой системы и т. д.). Таким образом, объектные кеши являются сегрегированными списками заданного размера и позволяют быстро обслужить запросы выделения и освобождения памяти. Если в данном кеше заканчивается место, он запрашивает несколько **слябов** памяти у общего распределителя (полный объем запрашиваемой памяти равен произведению размера страницы на размер объектов в списке). Наоборот, когда счетчик ссылок на объекты из данного сляба обращается в нуль, общий распределитель может отобрать сляб у специализированного, и это часто делается, когда системе ВП нужна дополнительная память.

ОТСТУПЛЕНИЕ: ВЕЛИКИЕ ИНЖЕНЕРЫ ДЕЙСТВИТЕЛЬНО ВЕЛИКИЕ

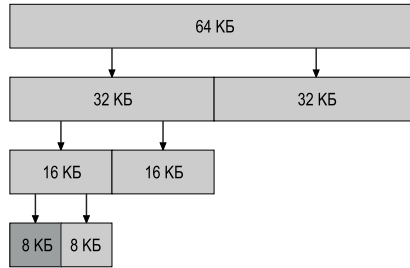
Инженеры типа Джеффа Бонвика (который написал не только слябовый распределитель, но и руководил группой, создавшей потрясающую файловую систему ZFS) – соль Кремниевой долины. За любым гениальным продуктом или технологией стоит человек (или небольшая группа людей) выше среднего уровня по талантам, способностям и одержимости. Марк Цукерберг (из Facebook) когда-то сказал: «Человек, исключительный в своей профессии, не просто чуть лучше довольно хорошего. Он в сто раз лучше». Именно поэтому даже в наши дни один или два человека могут основать компанию, которая навсегда изменит облик мира (вспомните Google, Apple или Facebook). Работайте усердно – и, возможно, тоже станете человеком «в сто раз лучше». А если не получится, то поработайте *вместе* с таким человеком, за один день вы узнаете больше, чем многие узнают за месяц. Если и это не получится, можете погрустить.

Слябовый распределитель идет дальше большинства подходов на основе сегрегированных списков, поскольку свободные объекты в списках заранее инициализированы. Бонвик показал, что инициализация и очистка структур данных обходятся дорого [B94]; благодаря хранению освобожденных объектов в инициализированном состоянии слябовый распределитель избегает частых циклов инициализации и очистки, за счет чего заметно снижаются накладные расходы.

Метод близнецов

Поскольку объединение – критически важная часть распределителя, было предложено несколько подходов, упрощающих именно этот аспект. Один из них – **метод близнецов** [K65].

В такой системе свободная память вначале рассматривается концептуально как одно большое пространство размером 2^N . При поступлении запроса на выделение памяти алгоритм поиска блока рекурсивно делит свободное пространство пополам, пока не будет найден блок достаточного размера (и такой, что его дальнейшее деление пополам привело бы к недостаточно большому блоку). В этот момент пользователю возвращается запрошенный блок. В примере ниже в свободном пространстве размером 64 КБ ищется блок размером 7 КБ:



Здесь будет выделен и возвращен пользователю самый левый блок размером 8 КБ (показан более темным цветом). Отметим, что эта схема подвержена **внутренней фрагментации**, поскольку разрешается выделять только блоки, размер которых является степенью двойки.

Вся красота метода близнецов раскрывается, когда происходит освобождение блока. Возвращая блок размером 8 КБ, распределитель проверяет, свободен ли его «близнец» тоже размером 8 КБ; если да, то оба блока объединяются в один блок размером 16 КБ. Затем распределитель проверяет, свободен ли близнец размером 16 КБ, и если да, близнецы объединяются. Этот процесс рекурсивного объединения продвигается вверх по дереву и останавливается, когда либо восстановлено все свободное пространство, либо очередной близнец в данный момент используется.

Причина такой хорошей работы метода близнецов – простота нахождения близнеца данного блока. Вы спросите, как? Подумайте об адресах блоков в свободном пространстве, и вы поймете, что адреса каждой пары близнецов различаются лишь в одном бите, зависящем от уровня в дереве. Теперь вы понимаете основную идею выделения памяти методом близнецов. Дополнительные сведения см. в обзоре Уилсона [W+95].

Другие идеи

Одна из главных проблем многих описанных выше подходов – недостаточная **масштабируемость**. Просмотр списков может оказаться очень медленным. Поэтому в передовых распределителях используются более сложные структуры данных, т. е. простотой жертвуют в обмен на производительность. Примерами могут служить сбалансированные двоичные деревья, косые деревья (*англ.* splay tree) и частично упорядоченные деревья [W+95].

Учитывая, что современные системы часто оснащены несколькими процессорами и исполняют многопоточные рабочие нагрузки (о чем мы узнаем во второй части книги), неудивительно, что масса усилий была приложена к тому, чтобы распределители хорошо работали в многопроцессорных системах. Два замечательных примера имеются в работах Berger et al. [B+00] и Evans [E06].

Это всего лишь две из тысяч идей, исторически предложенных при разработке распределителей памяти; почитайте о них, если интересно. А нет, так почитайте о том, как устроен распределитель в библиотеке glibc[S15], чтобы получить представление о жизни в реальном мире.

17.5. РЕЗЮМЕ

В этой главе мы обсудили самые рудиментарные формы распределителей памяти. Такие распределители встречаются повсюду, они компонуется с любой написанной вами программой на C, а также используются в самой ОС, которая управляет памятью для собственных структур данных. Как всегда, при построении подобных систем приходится идти на компромиссы, и чем больше мы знаем о характере рабочей нагрузки на распределитель, тем лучше сможем настроить его работу. Создать быстрый, эффективно использующий память и масштабируемый распределитель, который хорошо работал бы для широкого круга рабочих нагрузок, – все еще нерешенная задача в современных компьютерных системах.

Литература

[B+00] «Hoard: A Scalable Memory Allocator for Multithreaded Applications» by Emery D. Berger, Kathryn S. McKinley, Robert D. Blumofe, Paul R. Wilson. ASPLOS-IX, November 2000. *Великолепный распределитель Бергера и компании для многопроцессорных систем. И это не просто любопытная статья, он используется на практике!*

[B94] «The Slab Allocator: An Object-Caching Kernel Memory Allocator» by Jeff Bonwick. USENIX '94. *Интересная статья о построении распределителя для ядра операционной системы. Прекрасный пример того, как специализировать алгоритм для конкретных размеров объектов.*

[E06] «A Scalable Concurrent malloc(3) Implementation for FreeBSD» by Jason Evans. April, 2006. <http://people.freebsd.org/~jasone/jemalloc/bsdcan2006/jemalloc.pdf>. *Подробное рассмотрение того, как построить реальный современный распределитель для многопроцессорных систем. Распределитель jemalloc широко используется в настоящее время, в частности в FreeBSD, NetBSD, Mozilla Firefox и Facebook.*

[K65] «A Fast Storage Allocator» by Kenneth C. Knowlton. Communications of the ACM, Volume 8:10, October 1965. *Хорошо известное руководство по методу*

близнецов. Интересный факт: Кнут отдает приоритет не Ноултону, а Гарри Марковицу, лауреату Нобелевской премии по экономике. Еще один интересный факт: Кнут посылает все электронные письма через секретаря, а не сам; он говорит секретарю, что куда послать, а секретарь делает всю работу. И последний факт о Кнуте: он создал систему TeX, которая использовалась для верстки этой книги. Потрясающая программа¹.

[S15] «Understanding glibc malloc» by Sploitfun. February, 2015. sploitfun.wordpress.com/2015/02/10/understanding-glibc-malloc/. Углубленное обсуждение библиотеки *glibc malloc*. Очень подробное и читается с огромным интересом.

[W+95] «Dynamic Storage Allocation: A Survey and Critical Review» by Paul R. Wilson, Mark S. Johnstone, Michael Neely, David Boles. International Workshop on Memory Management, Scotland, UK, September 1995. Отличный и масштабный обзор многих граней выделения памяти. Деталей настолько много, что включить все в эту крохотную главу не получилось!

Домашнее задание (эмуляция)

Программа `malloc.py` позволяет изучить поведение простого распределителя свободного пространства, описанного в этой главе. Детали см. в файле `README`.

Вопросы

1. Сначала запустите программу с флагами `-n 10 -H 0 -p BEST -s 0` для генерирования нескольких случайных операций выделения и освобождения. Можете ли вы предсказать, что вернет `alloc()` или `free()`? Каково, на ваш взгляд, будет состояние списка свободных после каждого запроса? Замечали вы какие-нибудь особенности эволюции списка свободных со временем?
2. Как будут отличаться результаты при использовании политики худшего подходящего для поиска в списке свободных (`-p WORST`)?
3. А как насчет политики первого подходящего (`-p FIRST`)? Что ускорилось при использовании этой политики?
4. Для всех предыдущих задач способ упорядочения списка свободных может влиять на время поиска свободного блока в некоторых политиках. Задайте разные способы упорядочения (`-l ADDRSORT`, `-l SIZESORT+`, `-l SIZESORT-`) и посмотрите, как они взаимодействуют с политиками.
5. Объединение блоков в списке свободных может быть очень важно. Увеличьте число случайных операций выделений (скажем, до `-n 1000`). Что происходит для запросов на выделение больших блоков со временем? Запустите программу с объединением и без него (т. е. с флагом и без флага

¹ На самом деле мы использовали `LaTeX`, включающую добавления Лампорта к `TeX`, но обе программы близки.

- С). Что изменилось? Насколько большим оказывается со временем список свободных в каждом случае? Влияет ли на это упорядочение списка?
6. Что будет, если изменить процентную долю выделенных блоков -Р до уровня выше 50? А если она близка к 100? А если процентная доля близка к 0?
 7. Какие конкретно запросы можно отправить, чтобы получилось сильно фрагментированное свободное пространство? Воспользуйтесь флагом -А для создания фрагментированных списков свободных и посмотрите, как задание политик и параметров изменяет организацию списка свободных.

Глава 18

.....

Страничная организация: введение

Иногда говорят, что при решении большинства проблем управления пространством операционная система принимает один из двух подходов. Первый – нарезать на куски *переменного размера*, как мы видели при обсуждении **сегментации** виртуальной памяти. К сожалению, этому решению присущи трудности. В частности, при разбиении пространства на блоки разного размера это пространство **фрагментируется**, поэтому со временем выделить блок становится проблематично.

Таким образом, мы приходим ко второму подходу: нарезать пространство на куски *фиксированного размера*. В подсистеме виртуальной памяти эта идея называется **страничной организацией** и восходит к ранней и оказавшей большое влияние системе Atlas [KE+62, L78]. Вместо того чтобы разбивать адресное пространство процесса на логические сегменты переменного размера (например, код, кучу, стек), мы разбиваем его на участки фиксированного размера, каждый из которых называется **страницей**. Соответственно, физическая память представляется в виде массива слотов фиксированного размера, называемых **страничными рамками**; каждая рамка может содержать одну страницу виртуальной памяти.

Существо проблемы: как виртуализировать память с помощью страниц

Как виртуализировать память с помощью страниц, чтобы избежать проблем, свойственных сегментации? Каковы основные методы? Как сделать так, чтобы эти методы работали хорошо, с минимальными накладными расходами по времени и по памяти?

18.1. Простой пример и общий обзор

Чтобы прояснить этот подход, проиллюстрируем его на простом примере. На рис. 18.1 приведен пример крохотного адресного пространства размером

всего 64 байта, разбитого на четыре 16-байтовые виртуальные страницы (с номерами 0, 1, 2, 3). Реальные адресные пространства, конечно, гораздо больше, обычно 32-разрядные (т. е. способны адресовать 4 ГБ) или даже 64-разрядные¹. В этой книге мы часто используем крохотные примеры, чтобы их легче было переварить.



Рис. 18.1 ❖ Простое 64-байтовое адресное пространство

Физическая память, показанная на рис. 18.2, также состоит из слотов фиксированного размера, в данном случае восьми страничных рамок (так что ее полный объем 128 байт, до смешного мало). Как видно из рисунка, страницы виртуального адресного пространства разбросаны по разным адресам физической памяти, видно также, что ОС использует часть физической памяти для себя.

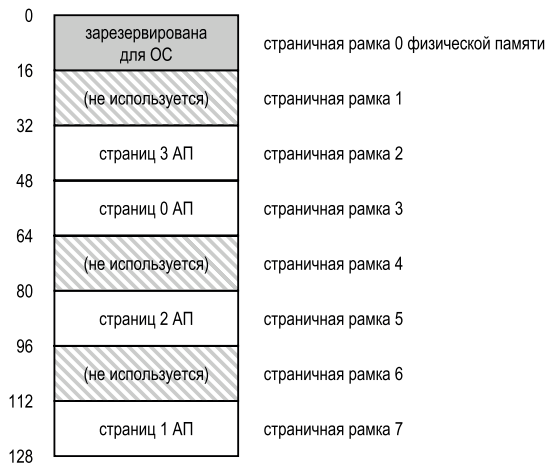


Рис. 18.2 ❖ 64-байтовое адресное пространство в 128-байтовой физической памяти

У страничной организации, как мы увидим ниже, есть ряд преимуществ по сравнению с предыдущими подходами. Пожалуй, самым важным из них

¹ 64-разрядное адресное пространство трудно себе вообразить, настолько оно огромно. Поможет аналогия: если считать 32-разрядное адресное пространство теннисным кортом, то 64-разрядное будет размером с Европу.

является *гибкость*; при тщательно проработанном страничном подходе система сможет эффективно поддерживать абстракцию адресного пространства независимо от того, как процесс ее использует. Например, мы не будем делать никаких предположений о направлении роста и использовании стека и кучи.

Еще одно преимущество – *простота* управления свободным пространством при страничной организации. Например, если ОС захочет поместить наше крохотное 64-байтовое адресное пространство в физическую память из восьми страниц, то ей нужно будет просто найти четыре свободные страницы. Быть может, ОС ведет для этого **список свободных** страниц и просто выбирает из него первые четыре страницы. В нашем примере ОС поместила виртуальную страницу 0 адресного пространства (АП) в физическую рамку 3, виртуальную страницу 1 – в физическую рамку 7, страницу 2 – в рамку 5, а страницу 3 – в рамку 2. Рамки 1, 4 и 6 остались свободными.

Чтобы запомнить, в каком месте физической памяти находится каждая виртуальная страница адресного пространства, операционная система обычно хранит для каждого процесса структуру данных, называемую **таблицей страниц**. Ее основная роль – хранить **трансляции адресов** для каждой виртуальной страницы адресного пространства, сообщая тем самым, в каком месте физической памяти она находится. В нашем простом примере (рис. 18.2) таблица страниц должна была бы содержать четыре записи: (Виртуальная Страница 0 → Физическая Рамка 3), (ВС 1 → ФР 7), (ВС 2 → ФР 5), (ВС 3 → ФР 2).

Важно помнить, что такая таблица страниц существует для *каждого* процесса (как и большая часть других обсуждаемых таблиц страниц, исключение составляет **инвертированная таблица страниц**). Если бы в примере выше был запущен еще один процесс, то ОС создала бы для него другую таблицу страниц, поскольку его виртуальные страницы отображаются на другие физические (хотя некоторые страницы могут быть разделяемыми).

Теперь мы знаем достаточно, чтобы выполнить трансляцию адресов. Предположим, что процесс с этим крохотным адресным пространством (64 байта) выполняет команду доступа к памяти:

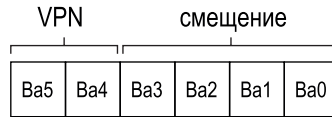
```
movl <виртуальный адрес>, %eax
```

Обратим внимание на явную загрузку данных по адресу <виртуальный адрес> в регистр `eax` (и забудем о выборке команды, которая должна была произойти раньше).

Для **трансляции** этого виртуального адреса, сгенерированного процессом, мы должны сначала выделить из него две компоненты: **номер виртуальной страницы** (virtual page number – **VPN**) и **смещение** от начала этой страницы. В этом примере, поскольку виртуальное адресное пространство составляет 64 байта, нам нужно всего 6 бит для виртуального адреса ($2^6 = 64$). Таким образом, концептуально виртуальный адрес можно представить следующим образом:

Ba5	Ba4	Ba3	Ba2	Ba1	Ba0
-----	-----	-----	-----	-----	-----

На этом рисунке Ba5 – старший бит виртуального адреса, а Ba0 – его младший бит. Поскольку размер страницы известен (16 байт), можно разбить виртуальный адрес следующим образом:

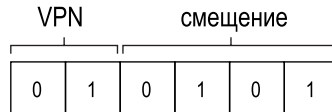


Размер страницы равен 16 байт в 64-байтовом адресном пространстве, поэтому у нас должна быть возможность выбрать любую из четырех страниц, для этого вполне достаточно 2 битов адреса. Таким образом, мы имеем 2-битовый номер виртуальной страницы (VPN). Остальные биты (в данном случае их четыре) содержат номер байта на этой странице – мы называем его **смещением**.

Когда процесс генерирует виртуальный адрес, ОС и оборудование должны совместно транслировать его в осмысленный физический адрес. Например, предположим, что приведенная выше команда загрузки ссылается на виртуальный адрес 21:

```
movl 21, %eax
```

В двоичной записи 21 равно 010101, и нам надо понять, как этот виртуальный адрес раскладывается на номер виртуальной страницы и смещение:



Таким образом, виртуальный адрес 21 – это пятый (5=0101) байт виртуальной страницы 01 (или 1). Зная номер виртуальной страницы, мы можем найти соответствующую запись в таблице страниц и определить, в какой физической рамке находится эта виртуальная страница. В приведенной выше таблице **номер физической рамки** (physical frame number – **PFN**), иногда называемый также **номером физической страницы** (physical page number – **PPN**), равен 7 (двоичное 111). Поэтому мы можем транслировать этот виртуальный адрес, заметив VPN на PFN, а затем выполнить загрузку из физической памяти (рис. 18.3).

Заметим, что смещение не изменяется (т. е. не транслируется), потому что оно говорит только о том, какой нам нужен байт *внутри* страницы. Окончательный физический адрес равен 1110101 (десятичное 117), отсюда мы и хотим загрузить данные (рис. 18.2).

Познакомившись с общей картиной, мы теперь можем задать несколько базовых вопросов о страничной организации (и, надеемся, ответить на них). Например, где хранятся таблицы страниц? Каково физическое содержимое

таблицы страниц, и насколько она велика? Не слишком ли табличная организация замедляет работу системы? На эти и другие интересные вопросы будут даны ответы, по крайней мере частичные, далее в тексте. Читайте!

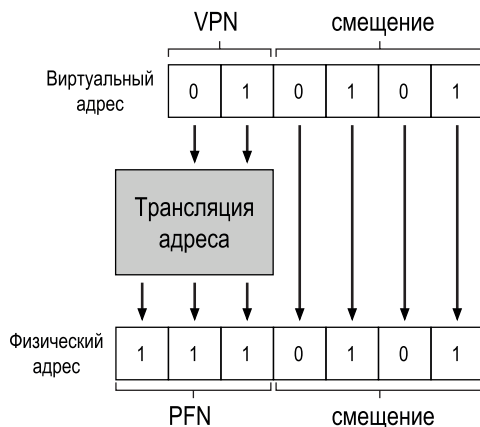


Рис. 18.3 ❖ Процесс трансляции адреса

18.2. ГДЕ ХРАНЯТСЯ ТАБЛИЦЫ СТРАНИЦ?

Табличные страницы могут быть очень велики, гораздо больше, чем небольшая таблица сегментов или пара регистров базы и границы. Например, представьте себе типичное 32-разрядное адресное пространство со страницами размером 4 КБ. Такой виртуальный адрес расщепляется на 20-разрядный VPN и 12-разрядное смещение (напомним, что для страницы размером 1 КБ нужно было бы 10 бит смещения и еще два, чтобы получить 4 КБ).

20-разрядный VPN означает, что существует 2^{20} трансляций, которыми ОС должна была бы управлять в интересах каждого процесса (это приблизительно миллион). В предположении, что на каждую **запись таблицы страниц** (page table entry – **PTE**) нужно 4 байта для размещения физической трансляции и дополнительных полезных данных, мы получаем, что для хранения каждой таблицы страниц требуется аж 4 МБ! Это довольно много. Если работает 100 процессов, то ОС понадобится 400 МБ памяти только для трансляции их адресов! Даже в наши дни, когда машины оснащаются гигабайтами памяти, кажется безумием использовать такую гигантскую область для одних лишь трансляций, как вы полагаете? А мы еще даже не думали, сколько будет весить таблица страниц для 64-разрядного адресного пространства; это совсем печально и может вас окончательно отпугнуть.

Поскольку таблицы страниц столь велики, мы не заводим в аппаратном устройстве управления памятью на кристалле места для хранения таблицы страниц текущего процесса. Вместо этого таблицы страниц для каждого процесса нужно хранить где-то в памяти. Предположим пока, что таблицы страниц находятся в памяти, управляемой ОС; ниже мы увидим, что значи-

тельную часть памяти самой ОС можно виртуализировать, поэтому таблицы страниц можно хранить в виртуальной памяти ОС (и даже выгружать на диск), но сейчас для этого еще не пришло время. На рис. 18.4 представлена картина таблицы страниц в памяти ОС – видите этот малюсенький набор трансляций?

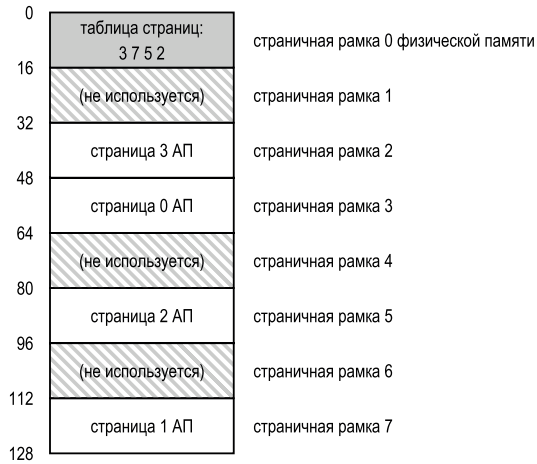


Рис. 18.4 ❖ Пример: таблица страниц в физической памяти ядра

18.3. Что хранится в таблице страниц?

Теперь поговорим об устройстве таблицы страниц. Это просто структура данных, используемая для отображения виртуальных адресов (точнее, номеров виртуальных страниц) в физические (номера страничных рамок). Так что нам подошла бы любая структура данных. Простейшая ее форма называется **линейной таблицей страниц**, это обычный массив. ОС ищет в этом массиве запись таблицы страниц (PTE) по *индексу*, равному номеру виртуальной страницы (VPN), и выбирает из этой записи номер физической рамки (PFN). Пока что будем предполагать, что это линейная структура, а в последующих главах воспользуемся более сложными структурами, чтобы решить некоторые возникающие проблемы.

В каждой PTE хранится несколько битов, заслуживающих внимания. **Бит достоверности** (valid bit) показывает, действует ли данная трансляция; в самом начале работы программы код и куча находятся в одном конце адресного пространства, а стек – в другом. Все неиспользуемое место между ними помечено как **недействительное**, и если процесс попытается обратиться к этой памяти, то будет сгенерировано системное прерывание, после чего ОС, скорее всего, снимет процесс. Таким образом, этот бит не-обходим для поддержки разреженного адресного пространства: просто помечая неиспользуемые страницы как недействительные, мы устраним

необходимость выделять для них физические рамки и тем самым экономим уйму памяти.

Биты защиты показывают, что можно делать со страницей: читать, записывать или выполнять. Попытка обратиться к странице неразрешенным способом также генерирует системное прерывание. Есть еще два важных бита, но о них мы пока подробно говорить не будем. **Бит присутствия** показывает, где находится данная страница: в физической памяти или на диске (т. е. **выгружена**). В этом механизме мы разберемся позже, когда будем изучать, как **выгрузить** части адресного пространства на диск в случае, когда адресное пространство больше физической памяти; выгрузка позволяет ОС освободить физическую память, перемещая редко используемые страницы на диск. **Бит изменения** (dirty bit) говорит, что страница была изменена с момента ее загрузки в память. **Бит обращения** (или **бит доступа**) иногда используется, чтобы показать, что к странице было обращение. Это полезно, когда нужно определить, какие страницы популярны, так что лучше их оставить в памяти; знать об этом необходимо для **вытеснения страниц**, о котором мы будем подробно говорить в последующих главах.

На рис. 18.5 показан пример записи таблицы страниц в архитектуре x86 [I09]. Она содержит бит присутствия (P), бит чтения-записи (R/W), который определяет, разрешена ли запись в эту страницу, бит пользователя-супервизора (U/S), который определяет, разрешено ли обращаться к этой странице процессам, работающим в режиме пользователя, несколько битов (PWT, PCD, PAT и G), определяющих порядок работы аппаратуры кеширования с этой страницей, бит доступа (A), бит изменения (D) и, наконец, сам номер физической рамки (PFN).

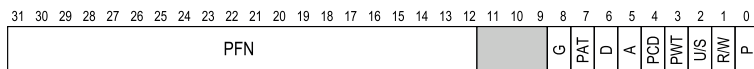


Рис. 18.5 ❖ Запись таблицы страниц (PTE) на x86

Подробнее о поддержке страничной организации в x86 читайте в руководствах по архитектуре Intel [I09]. Но предупреждаем: чтение подобных руководств, весьма информативных (и, конечно же, необходимых тем, кто пишет код работы с таблицами страниц в ОС), может поначалу показаться трудным. Потребуется немного терпения и много желания.

18.4. СТРАНИЧНАЯ ОРГАНИЗАЦИЯ:

ТОЖЕ СЛИШКОМ МЕДЛЕННО

Мы знаем, что таблицы страниц, хранящиеся в памяти, могут занимать слишком много места. И как выясняется, они могут еще и замедлять работу. Рассмотрим, например, простую команду

```
movl 21, %eax
```

Снова нас интересует только явное обращение к адресу 21, а не выборка команды. Предполагается, что трансляцию выполняет оборудование. Чтобы выбрать нужные данные, система сначала должна транслировать виртуальный адрес (21) в физический (117). Следовательно, прежде чем выбирать данные по адресу 117, система должна найти нужную запись в таблице страниц процесса, выполнить трансляцию и только потом загрузить данные из физической памяти.

Для этого оборудование должно знать, где находится таблица страниц текущего процесса. Пока предположим, что существует **регистр базы таблицы страниц**, содержащий физический адрес начала таблицы. Чтобы найти нужную PTE, оборудование, следовательно, должно выполнить эквивалент такого кода:

```
VPN = (VirtualAddress & VPN_MASK) >> SHIFT
PTEAddr = PageTableBaseRegister + (VPN * sizeof(PTE))
```

В нашем примере маска VPN_MASK будет установлена в 0x30 (двоичное 110000), чтобы выбирать биты VPN из полного виртуального адреса, а SHIFT будет равно 4 (количество битов смещения), чтобы после сдвига битов VPN вправо получился правильный номер виртуальной страницы. Например, если виртуальный адрес равен (010101), то после маскирования получаем 010000, а результатом сдвига будет 01, т. е. виртуальная страница 1, что и требовалось. Далее это значение используется как индекс в массив PTE, на который указывает регистр базы таблицы страниц.

Зная физический адрес, оборудование может выбрать PTE из памяти, выделить PFN и объединить его со смещением из виртуального адреса, чтобы сформировать физический адрес. Именно, PFN сдвигается влево на SHIFT бит, а затем объединяется со смещением с помощью операции ПОРАЗРЯДНОЕ ИЛИ:

```
offset = VirtualAddress & OFFSET_MASK
PhysAddr = (PFN << SHIFT) | offset
```

Наконец, оборудование может выбрать искомые данные из памяти и поместить их в регистры `eax`. Программа удачно загрузила значение из памяти!

Подводя итог, опишем первоначальную версию протокола действий при обращении к памяти. На рис. 18.6 показаны основные операции. При каждом обращении к памяти (будь то выборка команды или явная загрузка или сохранение) от нас требуется выполнить одну дополнительную операцию доступа к памяти, чтобы выбрать данные о трансляции из таблицы страниц. Это очень много работы! Дополнительные обращения к памяти обходятся дорого и в данном случае, скорее всего, замедлят процесс раза в два, а то и больше.

Надеемся, что теперь вы видите две реальные проблемы, нуждающиеся в решении. Без тщательного проектирования оборудования и программного обеспечения таблицы страниц будут слишком сильно замедлять работу системы и пожирать слишком много памяти. Описанное решение задачи виртуализации памяти кажется очень неплохим, но сначала необходимо разрешить эти две насущные проблемы.

```

1 // Выделить VPN из виртуального адреса
2 VPN = (VirtualAddress & VPN_MASK) >> SHIFT
3
4 // Сформировать адрес записи таблицы страниц (PTE)
5 PTEAddr = PTBR + (VPN * sizeof(PTE))
6
7 // Выбрать PTE
8 PTE = AccessMemory(PTEAddr)
9
10 // Проверить, имеет ли процесс право обращаться к странице
11 if (PTE.Valid == False)
12     RaiseException(SEGMENTATION_FAULT)
13 else if (CanAccess(PTE.ProtectBits) == False)
14     RaiseException(PROTECTION_FAULT)
15 else
16     // Доступ разрешен: сформировать физический адрес и выбрать
17     offset = VirtualAddress & OFFSET_MASK
18     PhysAddr = (PTE.PFN << PFN_SHIFT) | offset
19     Register = AccessMemory(PhysAddr)

```

Рис. 18.6 ❖ Доступ к странично организованной памяти

18.5. ТРАССИРОВКА ДОСТУПА К ПАМЯТИ

Прежде чем закруглиться, мы протрассируем простой пример доступа к памяти, чтобы продемонстрировать все обращения к странично организованной памяти. Нас интересует такой фрагмент кода на С (находится в файле `array.c`):

```

int array[1000];
...
for (i = 0; i < 1000; i++)
    array[i] = 0;

```

ОТСТУПЛЕНИЕ: СТРУКТУРА ДАННЫХ – ТАБЛИЦА СТРАНИЦ

Одна из самых важных структур данных в подсистеме управления памятью современной ОС – **таблица страниц**. В таблице страниц хранятся **трансляции виртуальных адресов в физические**, что позволяет системе узнать, где находится каждая страница адресного пространства в физической памяти. Поскольку трансляции подвергается адресное пространство каждого процесса, в общем случае в системе должно присутствовать по одной таблице страниц на процесс. Точная структура таблицы процессов определяется либо оборудованием (в старых системах), либо ОС (в современных системах), что дает дополнительную гибкость.

Мы откомпилировали `array.c` и запустили его:

```

prompt> gcc -o array array.c -Wall -O
prompt> ./array

```


Разумеется, чтобы по-настоящему понять, какие операции доступа к памяти выполняет этот фрагмент кода (он просто инициализирует массив), нужно знать еще кое-что. Во-первых, нужно дизассемблировать получившийся двоичный код (с помощью программы `objdump` в Linux или `otool` в Mac), чтобы увидеть, какие команды применяются для инициализации массива в цикле. Вот как выглядит ассемблерный код:

```
1024 movl $0x0, (%edi,%eax,4)
1028 incl %eax
1032 cmpl $0x03e8,%eax
1036 jne 0x1024
```

Если вы хоть немного знакомы с **x86**, то понять этот код будет несложно¹. Первая команда помещает значение 0 (показанное как `$0x0`) по виртуальному адресу массива; этот адрес вычисляется путем сложения регистра `%edi` со значением регистра `%eax`, умноженным на 4. Таким образом, `%edi` содержит базовый адрес массива, а `%eax` – индекс массива (*i*), а на 4 мы умножаем, потому что массив содержит целые числа, каждое из которых занимает 4 байта.

Вторая команда увеличивает на 1 индекс массива, хранящийся в `%eax`, а третья сравнивает содержимое этого регистра с шестнадцатеричным значением `0x03e8` (десятичное 1000). Если эти два значения еще не равны (что проверяет команда `jne`), то четвертая команда переходит в начало цикла.

Чтобы понять, какие обращения к памяти происходят при выполнении этой последовательности команд (на виртуальном и физическом уровнях), нам нужно сделать какие-то предположения о том, где в виртуальной памяти находятся этот код и массив, а также о местоположении и содержании таблицы страниц.

В этом примере предполагается, что виртуальное адресное пространство имеет размер 64 КБ (нереалистично мало), а размер страницы равен 1 КБ.

Теперь нам осталось только узнать содержимое таблицы страниц и ее местоположение в памяти. Будем предполагать, что таблица страниц линейна (является массивом) и расположена начиная с физического адреса 1 КБ (1024).

Что касается ее содержания, то нас интересуют лишь несколько виртуальных страниц, отображаемых в нашем примере. Во-первых, это виртуальная страница с кодом. Поскольку размер страницы равен 1 КБ, виртуальный адрес 1024 принадлежит второй странице виртуального адресного пространства ($VPN=1$, поскольку первой странице соответствует $VPN=0$). Предположим, что эта виртуальная страница отображается на физическую рамку 4 ($VPN\ 1 \rightarrow PFN\ 4$).

Далее – сам массив. Его размер равен 4000 байт (1000 целых чисел), и мы предполагаем, что он находится по виртуальным адресам от 40000 до 44000 (последний байт не включается). Этому диапазону десятичных адресов соответствуют виртуальные страницы $VPN=39, \dots, VPN=42$. Следовательно, нам нужны отображения этих страниц. Будем предполагать следующие отобра-

¹ Тут мы немножко лукавили, предположив для простоты, что все команды занимают четыре байта; на самом деле в **x86** есть команды разного размера.

жения виртуальных страниц на физические: (VPN 39 → PFN 7), (VPN 40 → PFN 8), (VPN 41 → PFN 9), (VPN 42 → PFN 10).

Теперь все готово для трассировки доступа к памяти. Во время работы программы каждая выборка команды генерирует два обращения к памяти: одно к таблице страниц для поиска физической страницы, на которой находится команда, а второе к самой команде, чтобы передать ее процессору для выполнения. Кроме того, имеется одно явное обращение к памяти в команде `mov`; оно влечет за собой второй доступ к таблице страниц (чтобы транслировать виртуальный адрес в физический) и затем доступ к самому массиву.

Первые пять итераций процесса представлены на рис. 18.7. Внизу черными квадратиками показаны обращения к памяти команд (виртуальные адреса слева, соответствующие им физические справа), на среднем рисунке серыми квадратиками показаны обращения к массиву (снова виртуальные слева, физические справа), а на верхнем светло-серыми квадратиками – обращения к таблице страниц (только физические адреса, потому что в этом примере таблица страниц располагается в физической памяти). На оси *x*, общей для всех рисунков, показаны операции доступа к памяти на первых пяти итерациях цикла. Всего на каждой итерации выполняется 10 обращений: четыре выборки команд, одно явное обновление памяти и пять обращений к таблице страниц для трансляции адресов в этих командах.

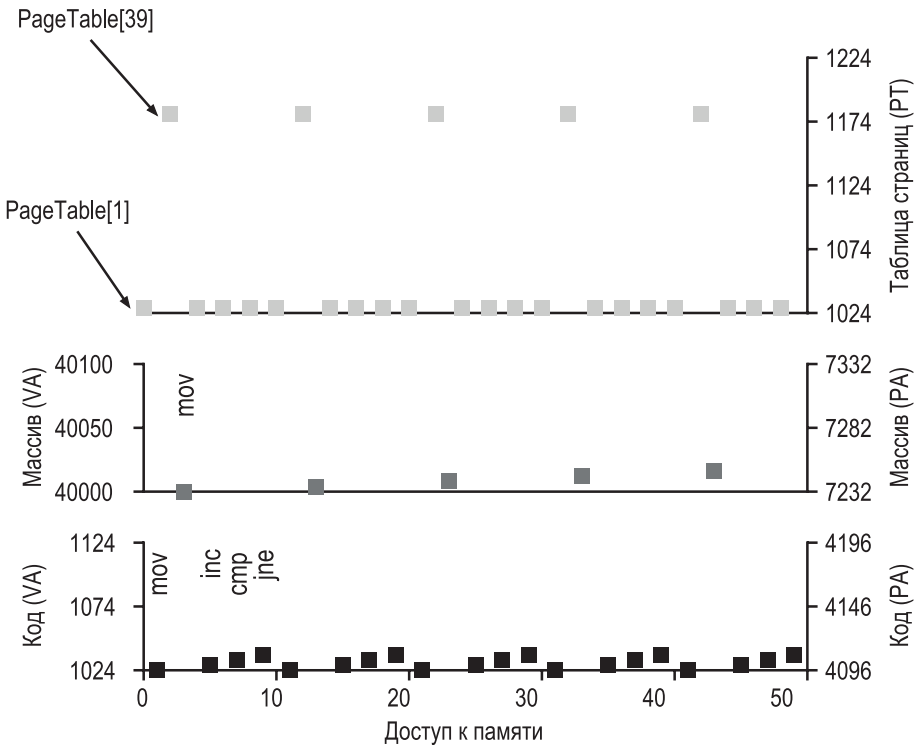


Рис. 18.7 ❖ Трассировка доступа к виртуальной и физической памяти

Разберитесь в закономерностях, проявляющихся в коде этой визуализации. В частности, что изменится, если цикл продолжит работать после этих пяти итераций? К каким новым адресам в памяти будут произведены обращения?

Это был очень простой пример (всего несколько строк на C), но даже он показывает, как трудно разобраться в характеристиках доступа к памяти в реальных приложениях. Но не переживайте: будет еще хуже, потому что механизмы, которые мы собираемся объяснить далее, только усложняют и без того сложную картину. Ну, извиняйте¹!

18.6. РЕЗЮМЕ

Мы познакомились с концепцией **страничной организации** как решением проблемы виртуализации памяти. У страничной организации много преимуществ по сравнению с предыдущими подходами (например, сегментацией). Во-первых, она не приводит к внешней фрагментации, поскольку память разбивается на блоки фиксированного и одинакового размера. Во-вторых, она весьма гибкая, т. к. допускает разреженные адресные пространства.

Однако при реализации страничной организации нужно проявлять осторожность, т. к. при этом замедляется работа машины (появляется много дополнительных обращений к памяти, необходимых для доступа к таблицам страниц) и потребляется очень много памяти (заполненной таблицами страниц, а не полезными данными). Поэтому необходимо еще подумать и изобрести страничную систему, которая не просто работает, а работает хорошо. В следующих двух главах мы покажем, как это делается.

Литература

[KE+62] «One-level Storage System» by T. Kilburn, D. B. G. Edwards, M. J. Lanigan, F. H. Sumner. IRE Trans. EC-11, 2, 1962. Reprinted in Bell and Newell, «Computer Structures: Readings and Examples». McGraw-Hill, New York, 1971. *В системе Atlas впервые высказана идея о разбиении памяти на страницы фиксированного размера, во многих отношениях это была ранняя форма управления памятью, впоследствии воспринятая современными компьютерными системами.*

[I09] «Intel 64 and IA-32 Architectures Software Developer's Manuals» Intel, 2009. Доступно по адресу <http://www.intel.com/products/processor/manuals>. *В особенности обратите внимание на том 3А «System Programming Guide Part 1» и том 3В «System Programming Guide Part 2».*

[L78] «The Manchester Mark I and Atlas: A Historical Perspective» by S. H. Lavington. Communications of the ACM, Volume 21:1, January 1978. *Эта статья – замечательный ретроспективный анализ истории разработки некоторых важ-*

¹ На самом-то деле мы себя виноватыми не чувствуем. Так что извините за то, в чем мы не виноваты, если в таких извинениях есть смысл.

ных вычислительных систем. Хотя мы в США часто забываем об этом, многие из этих новых идей пришли к нам из-за океана.

Домашнее задание (эмуляция)

В этом домашнем задании вы будете работать с простой программой `paging-linear-translate.py`, чтобы понять, как работает простая трансляция виртуальных адресов в физические на примере линейных таблиц страниц. Детали см. в файле `README`.

Вопросы

1. Прежде чем приступить к трансляции адресов, воспользуемся эмулятором, чтобы изучить, как меняется размер линейных таблиц страниц при изменении различных параметров. Ниже приведено несколько рекомендуемых наборов параметров, вычислите соответствующий каждому размер таблицы страниц. Флаг `-v` показывает, сколько записей в таблице страниц заполнено. Сначала разберитесь, что происходит с размером линейной таблицы страниц при росте адресного пространства:

```
paging-linear-translate.py -P 1k -a 1m -p 512m -v -n 0
paging-linear-translate.py -P 1k -a 2m -p 512m -v -n 0
paging-linear-translate.py -P 1k -a 4m -p 512m -v -n 0
```

Затем посмотрите, как изменяется размер таблицы при росте размера страницы:

```
paging-linear-translate.py -P 1k -a 1m -p 512m -v -n 0
paging-linear-translate.py -P 2k -a 1m -p 512m -v -n 0
paging-linear-translate.py -P 4k -a 1m -p 512m -v -n 0
```

Прежде чем запускать программу, подумайте, чего ждать. Как должен изменяться размер таблицы страниц при росте адресного пространства? А при росте размера страницы? Почему в общем случае не стоит использовать очень большие страницы?

2. Теперь перейдем к трансляции. Начнем с небольших примеров и изменим количество страниц, выделенных адресному пространству, с помощью флага `-u`. Например:

```
paging-linear-translate.py -P 1k -a 16k -p 32k -v -u 0
paging-linear-translate.py -P 1k -a 16k -p 32k -v -u 25
paging-linear-translate.py -P 1k -a 16k -p 32k -v -u 50
paging-linear-translate.py -P 1k -a 16k -p 32k -v -u 75
paging-linear-translate.py -P 1k -a 16k -p 32k -v -u 100
```

Что происходит при увеличении доли страниц, выделяемых каждому адресному пространству?

3. Теперь для разнообразия попробуем взять другие случайные начальные значения и другие (иногда совершенно безумные) параметры адресного пространства:

```
paging-linear-translate.py -P 8 -a 32 -p 1024 -v -s 1
paging-linear-translate.py -P 8k -a 32k -p 1m -v -s 2
paging-linear-translate.py -P 1m -a 256m -p 512m -v -s 3
```

Какие из этих комбинаций параметров нереалистичны? Почему?

4. Примените эту программу для изучения других проблем. Сможете ли вы найти пределы, за которыми программа перестает работать? Например, что будет, если размер адресного пространства *больше* физической памяти?

Глава 19

Страничная организация: более быстрая трансляция (TLB)

Применение страничной организации как основного механизма поддержки виртуальной памяти может стать причиной высоких накладных расходов. Разбиение адресного пространства на небольшие блоки фиксированного размера (страницы) требует хранения большого объема информации об отображении. Поскольку эта информация обычно хранится в физической памяти, для страничной организации, естественно, требуется дополнительное обращение к памяти на каждый генерируемый программой виртуальный адрес. Обращаться к памяти за информацией о трансляции перед выборкой каждой команды или выполнением явной команды загрузки или сохранения недопустимо дорого.

СУЩЕСТВО ПРОБЛЕМЫ: КАК УСКОРИТЬ ТРАНСЛЯЦИЮ АДРЕСОВ

Как можно ускорить трансляцию адресов и вообще избежать дополнительных обращений к памяти, которые, похоже, неотъемлемы от страничной организации? Какая нужна поддержка со стороны оборудования? А какая со стороны ОС?

Чтобы что-то ускорить, ОС обычно нуждается в помощи. И помощь часто приходит со стороны доброго друга ОС – оборудования. Чтобы ускорить трансляцию адресов, мы добавим так называемый (в силу исторических причин [CP78]) **буфер ассоциативной трансляции** (translation-lookaside buffer – **TLB** [CG68, C95]. TLB – часть устройства управления памятью (**MMU**), расположенного на кристалле, и по сути дела является аппаратным кешем часто повторяющихся трансляций виртуального адреса в физический, поэтому лучше было бы назвать его **кешем трансляции адресов**. При каждом обращении к виртуальной памяти оборудование сначала смотрит,

нет ли нужной трансляции в TLB; если есть, то трансляция производится быстро без необходимости заглядывать в таблицу страниц (где хранятся все трансляции). Из-за своего огромного влияния на производительность TLB в самом буквальном смысле превращают идею виртуальной памяти в реальность [С95].

19.1. Основной алгоритм TLB

На рис. 19.1 приведено приблизительное описание того, как оборудование могло бы обрабатывать трансляцию виртуальных адресов, если предполагается простая **линейная таблица страниц** (т. е. таблица страниц является массивом) и **аппаратно управляемый TLB** (т. е. оборудование берет на себя большую часть ответственности за доступ к таблице страниц, мы подробнее объясним это ниже).

```

1 VPN = (VirtualAddress & VPN_MASK) >> SHIFT
2 (Success, TlbEntry) = TLB_Lookup(VPN)
3 if (Success == True) // присутствует в TLB
4     if (CanAccess(TlbEntry.ProtectBits) == True)
5         Offset = VirtualAddress & OFFSET_MASK
6         PhysAddr = (TlbEntry.PFN << SHIFT) | Offset
7         Register = AccessMemory(PhysAddr)
8     else
9         RaiseException(PROTECTION_FAULT)
10 else // отсутствует в TLB
11     PTEAddr = PTBR + (VPN * sizeof(PTE))
12     PTE = AccessMemory(PTEAddr)
13     if (PTE.Valid == False)
14         RaiseException(SEGMENTATION_FAULT)
15     else if (CanAccess(PTE.ProtectBits) == False)
16         RaiseException(PROTECTION_FAULT)
17     else
18         TLB_Insert(VPN, PTE.PFN, PTE.ProtectBits)
19         RetryInstruction()
```

Рис. 19.1 ❖ Алгоритм управления TLB

Алгоритм работы оборудования устроен следующим образом. Сначала выделить номер виртуальной страницы (VPN) из виртуального адреса (строка 1) и проверить, присутствует ли в TLB трансляция этого VPN (строка 2). Если да, имеет место **попадание в TLB**. Замечательно! Теперь можно взять номер страничной рамки (PFN) из соответствующей записи в TLB, объединить его со смещением из исходного виртуального адреса, образовав нужный физический адрес, и обратиться к памяти (строки 5–7) в предположении, что с защитой все в порядке (строка 4).

Если CPU не найдет трансляции в TLB (**непопадание в TLB**), то придется сделать еще кое-что. Оборудование обращается за трансляцией к таблице

страниц (строки 11–12) и, в предположении, что сгенерированный программой виртуальный адрес корректен и доступен (строки 13, 15), записывает эту трансляцию в TLB (строка 18). Эти действия обходятся дорого, прежде всего из-за дополнительного обращения к памяти для доступа к таблице страниц (строка 12). Наконец, после обновления TLB программа заново выполняет команду; на этот раз трансляция обнаруживается в TLB, и обращение к памяти обрабатывается быстро.

TLB, как и все кеши, основан на допущении, что в типичном случае трансляция будет найдена в кеше. Если так, то накладные расходы невелики, поскольку TLB расположен рядом с процессорным ядром и спроектирован так, что работает очень быстро. Если же имеет место непопадание в кеш, то приходится платить высокую цену: нужно найти трансляцию в таблице страниц, так что происходит дополнительное обращение к памяти (и не одно, если используются более сложные таблицы страниц). Если это происходит часто, то программа будет работать заметно медленнее, т. к. обращения к памяти требуют куда больше времени, чем большинство других команд. Поэтому вся надежда на то, что мы сумеем по возможности избежать непопаданий в кеш.

19.2. ПРИМЕР: ДОСТУП К МАССИВУ

Чтобы лучше понять работу TLB, возьмем простое виртуальное адресное пространство и посмотрим, как TLB может улучшить его производительность. Будем предполагать, что в памяти расположен массив из 10 4-байтовых целых чисел, начиная с виртуального адреса 100. Пусть также имеется небольшое 8-разрядное виртуальное адресное пространство с 16-байтовыми страницами, которые, следовательно, разлагаются на 4-битовый VPN (существует 16 виртуальных страниц) и 4-битовое смещение (на каждой странице по 16 байт).

На рис. 19.2 показано, как массив расположен в 16 16-байтовых страницах системы. Как видим, первый элемент массива ($a[0]$) имеет адрес (VPN=06, смещение=04); на этой странице помещается только три 4-байтовых целых. Массив продолжается на следующей странице (VPN=07), где уместилось четыре элемента ($a[3] \dots a[6]$). Наконец, последние три элемента ($a[7] \dots a[9]$) находятся на следующей странице адресного пространства (VPN=08).

Теперь рассмотрим простой цикл, в котором производится доступ к каждому элементу массива. На C он выглядел бы так:

```
int sum = 0;
for (i = 0; i < 10; i++) {
    sum += a[i];
}
```

Для простоты будем считать, что единственные обращения к памяти в этом цикле – это обращения к массиву (игнорируя переменные i и sum , а также выборку самих команд). При обращении к первому элементу ($a[0]$) CPU увидит загрузку из виртуального адреса 100. Оборудование выделит из

него VPN (VPN=06) и проверит, есть ли в TLB соответствующая ему трансляция. В предположении, что программа обращается к массиву в первый раз, трансляции не будет, т. е. мы имеем непопадание в кеш.

	Смещение				
	00	04	08	12	16
VPN = 00					
VPN = 01					
VPN = 02					
VPN = 03					
VPN = 04					
VPN = 05					
VPN = 06		a[0]	a[1]	a[2]	
VPN = 07	a[3]	a[4]	a[5]	a[6]	
VPN = 08	a[7]	a[8]	a[9]		
VPN = 09					
VPN = 10					
VPN = 11					
VPN = 12					
VPN = 13					
VPN = 14					
VPN = 15					

Рис. 19.2 ❖ Пример:
массив в крохотном адресном пространстве

Далее производится доступ к элементу $a[1]$, и тут нас ожидают хорошие новости: попадание в TLB! Поскольку второй элемент массива находится рядом с первым на той же странице, а к этой странице мы уже обращались при доступе к первому элементу, то трансляция уже присутствует в TLB. Что и является причиной успеха. При доступе к элементу $a[2]$ нам тоже сопутствует успех – и по той же причине.

К сожалению, когда программа обращается к элементу $a[3]$, снова происходит непопадание в TLB. Однако для следующих трех элементов ($a[4]$... $a[6]$) будет иметь место попадание, т. к. все они находятся на той же странице в памяти. Наконец, доступ к $a[7]$ приводит к последнему непопаданию. Оборудование еще раз заглядывает в таблицу страниц, чтобы найти местоположение этой виртуальной страницы в физической памяти, и соответственно обновляет TLB. Последние два обращения ($a[8]$ и $a[9]$) пользуются благами этого обновления.

Подведем итоги деятельности TLB на протяжении этих десяти доступов к массиву: **непопадание**, попадание, попадание, **непопадание**, попадание, попадание, попадание, **непопадание**, попадание, попадание. Таким образом, **коэффициент попаданий** в TLB – частное от деления числа попаданий на общее число обращений – равен 70 %. Это не очень много (нам вообще-то

хотелось бы, чтобы коэффициент попаданий был близок к 100 %), но все же не нуль, что, возможно, вызвало у вас удивление. Даже при первом проходе этой программы по массиву TLB повышает эффективность благодаря **пространственной локальности**. Элементы массива плотно упакованы на страницах (т. е. близки друг к другу в **пространстве**), поэтому только первое обращение к элементу на странице заканчивается непопаданием в TLB.

Отметим также роль размера страницы в этом примере. Если бы размер страницы был всего в два раза больше (32 байта вместо 16), то количество непопаданий при доступе к массиву еще уменьшилось бы. Поскольку типичные размеры страниц гораздо больше 4 КБ, эффективность доступа к таким плотным массивам очень высока – всего одно непопадание на целую страницу элементов.

И последнее замечание относительно производительности TLB: если вскоре после завершения цикла программа снова обратится к этому массиву, то результат, скорее всего, будет еще лучше в предположении, что TLB достаточно велик, чтобы кэшировать все трансляции. В этом случае коэффициент попаданий в TLB был бы велик вследствие **временной локальности**, т. е. близкого соседства обращений к одним и тем же элементам во **времени**. Как и для любого кеша, успех применения TLB зависит от пространственной и временной локальности, а это свойства программы. Если программа обладает такими свойствами (а многие программы именно таковы), то коэффициент попаданий в TLB, скорее всего, будет высоким.

Совет: при любой возможности задействуйте кэширование

Кэширование – один из самых главных методов повышения производительности в вычислительных системах, он используется сплошь и рядом, чтобы «ускорить типичный случай» [HP06]. Идея аппаратных кешей заключается в том, чтобы воспользоваться локальностью обращений к командам и данным. Существует два типа локальности: **временная** и **пространственная**. Временная локальность базируется на предположении о том, что если к команде или элементу данных недавно производился доступ, то с большой вероятностью к ним будет повторное обращение в ближайшем будущем. Вспомните о переменных или командах в цикле: обращения к ним повторяются во времени. Пространственная локальность возникает благодаря тому, что программа, обращающаяся к памяти по адресу x , скорее всего, вскоре обратится к памяти по соседству с x . Тут на память приходит какой-нибудь вид потоковой обработки, когда один элемент обрабатывается вслед за другим. Конечно, эти свойства зависят от природы программы, поэтому являются не непреложными законами, а, скорее, эвристическими правилами.

Аппаратные кешы – все равно, для команд, данных или трансляций адресов (как в случае TLB) – обращают себе на пользу локальность, храня копии участков памяти в небольшой, но очень быстрой памяти, расположенной на кристалле. Вместо того чтобы для удовлетворения запроса сразу обращаться к (медленной) основной памяти, процессор сначала может проверить, есть ли копия в находящемся поблизости кеше, и если да, то очень быстро получить к ней доступ (всего за несколько тактов процессора), избежав временных затрат на доступ к основной памяти (много наносекунд).

Возникает вопрос: если кешы (в частности, TLB) так хороши, то почему бы не изготовить большие кешы и хранить в них все данные? К сожалению, здесь мы сталкиваемся с фундаментальными законами физики. Если вы хотите быстрый кеш, то он должен быть малень-

ким, поскольку в игру вступают скорость света и другие физические ограничения. Любой большой кеш по определению был бы медленным, так что вся затея потеряла бы смысл. Итак, мы обречены на небольшие быстрые кеши, и вопрос в том, как их лучше использовать для повышения производительности.

19.3. КТО ОБРАБАТЫВАЕТ НЕПОПАДАНИЕ В TLB?

Мы еще не ответили на вопрос, кто обрабатывает непопадание в TLB. Возможно два ответа: оборудование или программа (ОС). В стародавние времена оборудование располагало сложными наборами команд (такие компьютеры так и назывались: **CISC** – complex-instruction set computer), а их конструкторы не слишком доверяли этим ловкачам от ОС. Поэтому оборудование обрабатывало непопадание в кеш полностью самостоятельно. Для этого оборудованию нужно было точно знать, где в памяти находятся таблицы страниц (для чего использовался **регистр базы таблицы страниц**, см. строку 11 на рис. 19.1), а также их *точный формат*; в случае непопадания оборудование заглядывало в таблицу страниц, находило в ней нужную запись и выделяло из нее искомую трансляцию, после чего записывало эту трансляцию в TLB и повторяло выполнение команды. Примером такой «устаревшей» архитектуры с **аппаратно управляемым TLB** является архитектура Intel x86, в которой применяется фиксированная **многоуровневая таблица страниц** (детали см. в следующей главе), а на текущую таблицу страниц указывает регистр CR3 [I09].

В более современных архитектурах (например, MIPS R10k [H93] или Sun SPARC v9 [WG00], то и другое **RISC**-компьютеры, т. е. компьютеры с сокращенным набором команд) имеется **программно управляемый TLB**. В случае непопадания в TLB оборудование просто возбуждает исключение (строка 11); в результате текущий поток команд приостанавливается, уровень привилегий повышается до уровня ядра, и производится переход к **обработчику системного прерывания**. Легко догадаться, что этот обработчик – часть ОС, написанная специально с целью обработки непопаданий в кеш. Этот код ищет трансляцию в таблице страниц, с помощью специальных привилегированных команд обновляет TLB и возвращает управление, после чего оборудование повторяет выполнение команды (и обнаруживает искомое в TLB).

Обсудим две важные детали. Во-первых, команда возврата из прерывания должна немного отличаться от той, что мы видели раньше при обсуждении обслуживания системных вызовов. Тогда эта команда возобновляла выполнение, начиная с команды, расположенной *после* системного прерывания, точно так же, как возврат из функции передает управление команде, следующей за командой вызова этой функции. Теперь же после возврата из обработчика непопадания в TLB оборудование должно возобновить выполнение с той команды, которая *вызвала* прерывание, это позволит выполнить ее повторно, на сей раз с попаданием в TLB. Таким образом, в зависимости от того, чем было вызвано системное прерывание или исключение, оборудо-

вание должно перед переходом в ОС сохранять разные значения РС, чтобы правильно возобновить выполнение впоследствии.

Во-вторых, при выполнении кода обработки непадания в TLB ОС должна очень внимательно следить за тем, чтобы не вызвать бесконечной цепочки таких непадания. Решений существует много, например можно хранить обработчик непадания в TLB в физической памяти (где адреса не **отображаются** и, значит, не подвергаются трансляции) или зарезервировать в TLB несколько элементов для постоянного хранения трансляций и использовать часть из них для самого кода обработчика; при поиске таких **защитых** трансляций всегда имеет место попадание.

Главное достоинство программно управляемого решения – в его *гибкости*; ОС может использовать любую структуру данных, которую сочтет выгодной для реализации таблицы страниц, не требуя переделывать оборудование. Другое достоинство – *простота*; как видно в алгоритме управления TLB (строка 11 на рис. 19.3 в отличие от строк 11–19 на рис. 19.1), оборудованию не нужно делать много работы в случае непадания, оно возбуждает исключение, а обработчик в ОС берет на себя все остальное.

```

1 VPN = (VirtualAddress & VPN_MASK) >> SHIFT
2 (Success, TlbEntry) = TLB_Lookup(VPN)
3 if (Success == True) // присутствует в TLB
4     if (CanAccess(TlbEntry.ProtectBits) == True)
5         Offset = VirtualAddress & OFFSET_MASK
6         PhysAddr = (TlbEntry.PFN << SHIFT) | Offset
7         Register = AccessMemory(PhysAddr)
8     else
9         RaiseException(PROTECTION_FAULT)
10    else // отсутствует в TLB
11        RaiseException(TLB_MISS)

```

Рис. 19.3 ❖ Алгоритм управления TLB (средствами ОС)

ОТСТУПЛЕНИЕ: RISC и CISC

В 1980-х годах в сообществе, связанном с компьютерной архитектурой, разразилась великая битва. На одной стороне стали лагерем сторонники **CISC (Complex Instruction Set Computing)** – вычисления со сложным набором команд, а на другой – сторонники **RISC (Reduced Instruction Set Computing)** – вычисления с сокращенным набором команд [PS81]. Во главе сторонников RISC стояли Дэвид Паттерсон из Беркли и Джон Хеннесси из Стэнфорда (оба соавторы нескольких знаменитых книг [HP06]), хотя впоследствии Джон Кок получил премию Тьюринга за ранние работы по RISC [CM00].

В CISC-компьютерах много команд, и каждая из них довольно мощная. Например, есть команда копирования строки, которая принимает два указателя и длину и копирует байты из источника в приемник. Идея CISC заключается в том, что команды должны быть высокоуровневыми примитивами, чтобы язык ассемблера было проще использовать, а код был более компактным.

В RISC-архитектурах все с точностью до наоборот. Основное наблюдение, приведшее к RISC, заключается в том, что набор команд важен прежде всего для компилятора, а все

компиляторы хотят иметь немного простых примитивов, из которых можно составить высокопроизводительный код. Поэтому, аргументировали свою позицию сторонники RISC, давайте уберем из оборудования все лишнее (особенно микрокод), а то, что останется, сделаем простым, единообразным и быстрым.

Поначалу RISC-кристаллы оказали огромное влияние на индустрию, потому что оказались заметно быстрее [BC91]. Было написано много статей, образовано несколько компаний (например, MIPS и Sun). Однако прогресс не стоял на месте, и производители CISC-кристаллов, в частности Intel, включили многие идеи RISC в ядра своих процессоров, например добавили ранние стадии конвейера, где сложные команды преобразовывались в микрокоманды, которые затем можно было обработать в духе RISC. Эти новшества, а также рост количества транзисторов на кристалле позволили архитектуре CISC остаться на плаву. В конце концов, споры утихи, и теперь процессоры обоих типов могут работать быстро.

19.4. Содержимое TLB: что там хранится?

Рассмотрим содержимое аппаратного TLB внимательнее. Типичный TLB может иметь 32, 64 или 128 элементов и быть **полностью ассоциативным**. По сути дела, это просто означает, что любая конкретная трансляция может находиться в любом месте TLB, а оборудование просматривает весь TLB параллельно в поисках искомого. Запись TLB может выглядеть так:

VPN	PFN	Прочие биты
-----	-----	-------------

Заметим, что в каждой записи присутствуют и VPN, и PFN, поскольку трансляция может оказаться в любом месте TLB (это и называется **полной ассоциативностью**). Оборудование ищет совпадение во всех записях параллельно.

Отступление:

БИТ ДОСТОВЕРНОСТИ В TLB ≠ БИТ ДОСТОВЕРНОСТИ В ТАБЛИЦЕ СТРАНИЦ

Часто путают биты достоверности в TLB и в таблице страниц. Если запись таблицы страниц помечена как недействительная, значит, она не была выделена процессу, и правильно написанная программа не должна к ней обращаться. Обычная реакция на обращение к недействительной странице – системное прерывание и снятие процесса операционной системой.

С другой стороны, бит достоверности в TLB означает лишь, что в этой записи хранится недействительная трансляция. На этапе начальной загрузки все записи TLB обычно помечаются как недействительные, поскольку никакие трансляции еще не были кешированы. После активации виртуальной памяти, когда программы начинают выполняться и обращаться к своим виртуальным адресным пространствам, TLB потихоньку заполняется, и вскоре в нем остаются только действительные записи.

Бит достоверности TLB также полезен при контекстном переключении, о чем мы поговорим чуть ниже. Помечая все записи TLB недействительными, система может гарантировать, что процесс, который вот-вот начнет работать, случайно не унаследует трансляции виртуальных адресов в физические, оставшиеся от предыдущего процесса.

Более интересны «прочие биты». Например, в TLB обычно имеется бит **достоверности**, который говорит, находится в записи действительная трансляция или нет. Также присутствуют биты **защиты**, которые определяют, как можно обращаться к странице (такие же, как в таблице страниц). Например, страницу кода можно было бы пометить битами *чтения и выполнения*, а страницу кучи – битами *чтения и записи*. Могут быть и другие поля, в т. ч. **идентификатор адресного пространства**, **бит изменения** и т. д. (дополнительные сведения см. ниже).

19.5. Проблема TLB:

КОНТЕКСТНЫЕ ПЕРЕКЛЮЧЕНИЯ

При использовании TLB возникают новые проблемы, связанные с переключением между процессами (а значит, и адресными пространствами). Именно, TLB содержит трансляции виртуальных адресов в физические, действительные только для текущего процесса; для остальных же процессов они не имеют смысла. В результате при переключении на другой процесс оборудование или ОС (или совместно) должны позаботиться о том, чтобы новый процесс по ошибке не воспользовался трансляциями, принадлежащими предыдущему.

Чтобы лучше разобраться в этой ситуации, рассмотрим пример. Когда работает один процесс (P1), он предполагает, что в TLB кешированы действительные для него трансляции, т. е. взятые из таблицы страниц P1. Предположим, что десятая виртуальная страница P1 отображена на физическую рамку 100.

Предположим также, что существует другой процесс (P2), и ОС вскоре может решить, что нужно переключиться на него. Пусть десятая виртуальная страница P2 отображена на физическую рамку 170. Если бы записи, относящиеся к обоим процессам, одновременно присутствовали в TLB, то он выглядел бы так:

VPN	PFN	Бит достоверности	Биты защиты
10	100	1	rwX
–	–	0	–
10	170	1	rwX
–	–	0	–

В этом TLB наблюдается очевидная проблема: VPN 10 транслируется либо в PFN 100 (P1), либо в PFN 170 (P2), но оборудование не может установить, какая запись к какому процессу относится. Поэтому нужно проделать дополнительную работу, чтобы TLB правильно и эффективно поддерживал виртуализацию при наличии нескольких процессов.

СУЩЕСТВО ПРОБЛЕМЫ:**КАК УПРАВЛЯТЬ СОДЕРЖИМЫМ TLB ПРИ КОНТЕКСТНОМ ПЕРЕКЛЮЧЕНИИ**

После контекстного переключения процессов находящиеся в TLB трансляции предыдущего процесса не имеют смысла для последующего. Что оборудование или ОС должны сделать для решения этой проблемы?

Существует несколько потенциальных решений. Одно из них – просто **сбросить** TLB в начальное состояние, т. е. опустошить его перед запуском следующего процесса. В программно управляемой системе этого можно достичь с помощью явной (и привилегированной) команды, а в аппаратно управляемой сброс можно инициировать в момент изменения регистра базы таблицы страниц (PBTR) (отметим, что при контекстном переключении ОС должна в любом случае изменить этот регистр). Так или иначе, операция сброса просто устанавливает все биты достоверности в 0, очищая тем самым TLB от содержимого.

Сбрасывая TLB при каждом контекстном переключении, мы получаем работоспособное решение, поскольку процесс никогда не увидит чужие трансляции. Но у этого решения есть цена: при каждом возобновлении процесс поначалу будет сталкиваться с непопаданиями в TLB при попытке обратиться к своим страницам данных и кода. Если процессы переключаются часто, то цена эта будет высокой.

Чтобы уменьшить накладные расходы, в некоторых системах добавлена аппаратная поддержка разделения TLB после контекстных переключений. В частности, в TLB может присутствовать поле **идентификатора адресного пространства** (address space identifier – **ASID**). Можно считать это аналогом **идентификатора процесса (PID)**, но обычно в нем меньше битов (например, ASID может занимать 8 бит, тогда как PID занимает 32 бита).

Если продолжить пример и добавить в TLB значения ASID, то становится ясно, что процессы в принципе могут разделять TLB: для различения кажущихся одинаковыми трансляций нужно только поле ASID. Вот как выглядит TLB после добавления ASID:

VPN	PFN	Бит достоверности	Биты защиты	ASID
10	100	1	rwX	1
–	–	0	–	–
10	170	1	rwX	2
–	–	0	–	–

Таким образом, при наличии идентификаторов адресного пространства в TLB можно одновременно хранить трансляции разных процессов, не внося путаницы. Конечно, оборудованию нужно также знать, какой процесс выполняется в данный момент, чтобы выполнять трансляции для него, поэтому ОС при контекстном переключении должна заносить в некий привилегированный регистр ASID текущего процесса.

Заодно рассмотрим еще один случай, когда две записи TLB очень похожи. В примере ниже имеются две записи, принадлежащие разным процессам, в которых разные VPN указывают на *одну и ту же* физическую страницу.

VPN	PFN	Бит достоверности	Биты защиты	ASID
10	101	1	г-х	1
–	–	0	–	–
50	101	1	г-х	2
–	–	0	–	–

Такая ситуация может возникнуть, например, когда два процесса *разделяют* страницу (скажем, страницу кода). В этом примере процесс 1 разделяет физическую страницу 101 с процессом 2; P1 отображает эту страницу на 10-ю страницу своего адресного пространства, а P2 – на 50-ю. Разделение страниц кода (исполняемыми файлами или разделяемыми библиотеками) полезно, потому что уменьшает общее число используемых физических страниц, а значит, и нагрузку на память.

19.6. ПРОБЛЕМА: ПОЛИТИКА ВЫТЕСНЕНИЯ

При работе с TLB, как и с любым другим кешем, необходимо рассмотреть проблему **вытеснения из кеша**. Именно, помещая в TLB новую запись, мы должны **вытеснить** какую-то старую – а какую?

СУЩЕСТВО ПРОБЛЕМЫ:

КАК СПРОЕКТИРОВАТЬ ПОЛИТИКУ ВЫТЕСНЕНИЯ

Какую запись TLB следует вытеснить, чтобы добавить новую? Разумеется, наша цель – минимизировать **коэффициент непопаданий** (или, что то же самое, максимизировать **коэффициент попаданий**) и тем самым повысить производительность.

Мы более подробно изучим такие политики при обсуждении проблемы выгрузки страниц на диск, а пока просто перечислим несколько распространенных политик. Один из самых популярных подходов – вытеснить запись **по давности использования** (least recently used – **LRU**). Идея в том, чтобы воспользоваться локальностью ссылок в памяти и предполагать, что запись, которая давно не использовалась, – лучший кандидат на вытеснение. Другой типичный подход – **рандомизированная** политика, когда из TLB вытесняется случайная запись. Такая политика хороша своей простотой и способностью обходить редко встречающиеся случаи; например, «разумная» политика типа LRU будет вести себя крайне неразумно, когда программа в цикле обращается к $(n + 1)$ -й странице, притом что размер TLB равен n , – в этом случае LRU приводит к непопаданию в кеш при каждом обращении, так что рандомизированная политика намного лучше.

19.7. РЕАЛЬНАЯ ЗАПИСЬ TLB

Напоследок кратко рассмотрим один реальный TLB. Пример взят из процессора MIPS R4000 [H93], современной системы, в которой используется программно управляемый TLB; немного упрощенный вариант записи MIPS TLB показан на рис. 19.4.

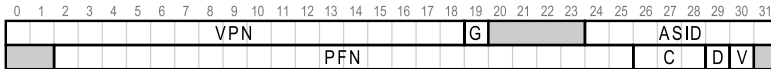


Рис. 19.4 ❖ Запись TLB в архитектуре MIPS

Процессор MIPS R4000 поддерживает 32-разрядное адресное пространство со страницами размером 4 КБ. Поэтому следовало бы ожидать, что в типичном виртуальном адресе длина VPN будет равна 20 бит, а смещения – 12 бит. Однако, как видно на рисунке, в записи TLB под VPN отведено всего 19 бит. Оказывается, что пользовательские адреса могут находиться только в половине адресного пространства (остальное зарезервировано для ядра), и потому требуется лишь 19 бит. VPN транслируется в 24-битовый номер физической рамки (PFN), так что поддерживаются системы, имеющие до 64 ГБ физической памяти (2^{24} 4-килобайтовых страниц).

В MIPS TLB есть еще несколько интересных битов. Мы видим *глобальный* бит (G), который выставляется для страниц, глобально разделяемых между всеми процессами. Таким образом, если глобальный бит установлен, то ASID игнорируется. Также мы видим 8-битовое поле *ASID*, которое ОС может использовать, чтобы различать адресные пространства (как описано выше). А вот вам вопрос: что должна сделать ОС, если одновременно работает более 256 (2^8) процессов? Наконец, мы видим 3 бита *когерентности* (C), которые определяют, как страница кешируется оборудованием (это выходит за рамки нашего обсуждения), бит *изменения* (D), который выставляется, если в страницу была произведена запись (ниже мы увидим, как он используется), бит *достоверности* (V), который сообщает оборудованию, находится ли в данной записи действительная трансляция. Еще имеется поле *страничной маски* (не показано), позволяющее поддерживать страницы нескольких размеров; ниже мы покажем, почему могут быть полезны страницы большего размера. Наконец, часть из 64 битов не используется (закрашены серым цветом).

В MIPS TLB обычно присутствует 32 или 64 такие записи, и большая их часть используется пользовательскими процессами. Но несколько записей зарезервировано для ОС. ОС может записать в регистр *wired*, сколько записей TLB зарезервировать для себя, а затем использует зарезервированные записи для страниц кода и данных, которые должны быть доступны в критических ситуациях, когда непопадание в TLB может вызвать серьезные проблемы (как, например, внутри обработчика непопаданий).

Поскольку MIPS TLB управляется программно, должны быть предоставлены команды для обновления TLB. В MIPS есть четыре такие команды: TLBR смотрит, существует ли конкретная трансляция в TLB; TLBR читает содержи-

мое записи TLB в регистры; TLBWI вытесняет определенную запись из TLB; TLBWR вытесняет случайную запись. ОС пользуется этими командами для управления содержимым TLB. Конечно же, все эти команды **привилегированные**; только представьте себе, что мог бы натворить пользовательский процесс, если бы ему было разрешено модифицировать TLB (подсказка: почти всё, в т. ч. перехватить управление машиной, выполнить собственную вредоносную «ОС» и даже убрать солнце с небосвода).

Совет: ЗУПВ НЕ ВСЕГДА ЗУПВ (закон Каллера)

Термин **запоминающее устройство с произвольной выборкой (ЗУПВ)** (англ. *random-access memory – RAM*) означает, что время обращения к любой части ЗУПВ одинаково. Вообще говоря, такое представление о ЗУПВ разумно, но благодаря программно-аппаратным средствам, подобным TLB, доступ к конкретной странице памяти может обходиться дорого, особенно если для нее нет трансляции в TLB. Поэтому всегда стоит помнить о детали реализации: **ЗУПВ не всегда ЗУПВ**. Иногда произвольный доступ к адресному пространству, в особенности когда число страниц, к которым производится обращение, больше емкости TLB, может стать причиной весьма заметного падения производительности. Поскольку один из наших научных руководителей, Дэвид Каллер, всегда подчеркивал, что TLB может быть источником многих проблем с производительностью, мы назвали это наблюдение в его честь: **закон Каллера**.

19.8. РЕЗЮМЕ

Мы видели, как оборудование может помочь с ускорением трансляции адресов. Благодаря небольшому специализированному TLB на кристалле, который используется в роли кеша для трансляции адресов, можно надеяться, что большинство обращений к памяти удастся обработать, обойдясь без доступа к таблице страниц в основной памяти. Таким образом, в типичном случае производительность программы будет почти такой же, как если бы память не виртуализировалась вовсе, – отличное достижение для операционной системы и, безусловно, крайне важное для использования страничной организации в современных системах.

Однако TLB не обещает рая всем существующим программам. В частности, если за короткий период времени программа обращается к большему числу страниц, чем помещается в TLB, то количество непопаданий в кеш резко возрастает, поэтому работа замедляется. Это явление, называемое превышением **емкости TLB**, может стать серьезной проблемой для некоторых программ. Одно из решений, которое мы обсудим в следующей главе, – включить поддержку страниц большего размера; если важные структуры данных располагаются в областях адресного пространства программы, которые отображаются на большие страницы, то эффективная емкость TLB увеличивается. Поддержка больших страниц часто используется в таких программах, как **системы управления базами данных (СУБД)**, где имеются структуры дан-

ных, с одной стороны большие, а с другой – характеризующие произвольной выборкой.

Стоит упомянуть еще одну проблему: доступ к TLB легко может оказаться узким местом в конвейере команд, особенно в сочетании с **физически индексируемым кешем**. В этом случае трансляция адресов должна иметь место до обращения к кешу, что может довольно сильно замедлить работу. Из-за этой потенциальной проблемы были придуманы разные хитроумные способы обращаться к кешу по *виртуальному* адресу, избежав тем самым дорогостоящего шага трансляции в случае попадания в кеш. Такой **виртуально индексируемый кеш** решает некоторые проблемы производительности, но привносит новые проблемы в проектирование оборудования. Дополнительные сведения см. в обзоре Уиггинса [W03].

Литература

[BC91] «Performance from Architecture: Comparing a RISC and a CISC with Similar Hardware Organization» by D. Bhandarkar and Douglas W. Clark. Communications of the ACM, September 1991. *Прекрасное и честное сравнение RISC и CISC. Вывод: на схожем оборудовании производительность RISC была примерно в три раза выше.*

[CM00] «The evolution of RISC technology at IBM» by John Cocke, V. Markstein. IBM Journal of Research and Development, 44:1/2. *Обзор идей и работы, проделанной над изделием IBM 801, которое многие считают первым настоящим RISC-микропроцессором.*

[C95] «The Core of the Black Canyon Computer Corporation» by John Couleur. IEEE Annals of History of Computing, 17:4, 1995. *В этом увлекательном историческом экскурсе Кулер рассказывает о том, как изобрел TLB в 1964 году, работая в компании «Дженерал Электрик», и о последовавшем за этим плодотворном сотрудничестве с участниками проекта Project MAC в MIT.*

[CG68] «Shared-access Data Processing System» by John F. Couleur, Edward L. Glaser. Patent 3412382, November 1968. *Патент, в котором излагается идея ассоциативной памяти для хранения трансляций адресов. Эта идея, по словам самого Кулера, посетила его в 1964 году.*

[CP78] «The architecture of the IBM System/370» by R. P. Case, A. Padegs. Communications of the ACM. 21:1, 73-96, January 1978. *Быть может, первая работа, в которой использовался термин translation lookaside buffer (буфер ассоциативной трансляции). Он происходит от исторического названия кеша – lookaside buffer, – предложенного разработчиками системы Atlas в Манчестерском университете. Хотя термин lookaside buffer вышел из употребления, TLB неизвестно почему прижился.*

[H93] «MIPS R4000 Microprocessor User's Manual». by Joe Heinrich. Prentice-Hall, June 1993. Доступно по адресу http://cag.csail.mit.edu/raw/documents/R4400_Uman_book_Ed2.pdf. *Техническое руководство, которое на удивление легко читается. Или нет?*

[HP06] «Computer Architecture: A Quantitative Approach» by John Hennessy and David Patterson. Morgan-Kaufmann, 2006. *Потрясающая книга по компьютерной архитектуре. Нам особенно нравится классическое первое издание.*

[I09] «Intel 64 and IA-32 Architectures Software Developer's Manuals» Intel, 2009. Доступно по адресу <http://www.intel.com/products/processor/manuals>. *В особенности обратите внимание на том 3А «System Programming Guide Part 1» и том 3В «System Programming Guide Part 2».*

[PS81] «RISC-I: A Reduced Instruction Set VLSI Computer» by D.A. Patterson and C.H. Sequin. ISCA '81, Minneapolis, May 1981. *В этой статье впервые появился термин RISC, с нее началась лавина исследований по упрощению компьютерных микросхем ради повышения производительности.*

[SB92] «CPU Performance Evaluation and Execution Time Prediction Using Narrow Spectrum Benchmarking» by Rafael H. Saavedra-Barrera. EECS Department, University of California, Berkeley. Technical ReportNo. UCB/CSD-92-684, February 1992. *Диссертация на тему о том, как предсказывать время выполнения приложений путем разбиения их на составные части известной стоимости. Пожалуй, самое интересное следствие этой работы – инструмент, позволяющий измерять детали работы иерархии кешей (см. главу 5). Обязательно посмотрите на прекрасные рисунки.*

[W03] «A Survey on the Interaction Between Caching, Translation and Protection» by Adam Wiggins. University of New South Wales TR UNSW-CSE-TR-0321, August, 2003. *Отличный обзор взаимодействия TLB с другими частями процессорного конвейера, а именно с аппаратными кешами.*

[WG00] «The SPARC Architecture Manual: Version 9» by David L. Weaver and Tom Germond. SPARC International, San Jose, California, September 2000. Доступно по адресу www.sparc.org/standards/SPARCV9.pdf. *Еще одно техническое руководство. Держу пари, что вы надеялись на какую-нибудь более интересную ссылку в конце этой главы.*

Домашнее задание (измерение)

В этом домашнем задании вы будете измерять размер TLB и стоимость доступа к нему. Идея подсказана работой Saavedra-Barrera [SB92], придумавшего изящный метод измерения различных аспектов работы иерархии кешей, для которого нужна очень простая программа пользовательского уровня. Детали см. в его работе.

Идея заключается в том, чтобы обращаться к страницам, на которых размещена большая структура данных (например, массив), и замерять время обращений. Предположим, например, что размер TLB составляет 4 (это очень мало, но полезно для обсуждения). Если написать программу, которая обращается к четырем или меньшему числу страниц, то каждое обращение будет заканчиваться попаданием в TLB и, следовательно, производится сравнительно быстро. Но если мы будем в цикле обращаться к пяти или большему

числу страниц, то стоимость каждого обращения внезапно увеличится, поскольку будет иметь место непопадание в TLB.

Код однократного обхода массива выглядит так:

```
int jump = PAGESIZE / sizeof(int);
for (i = 0; i < Numpages * jump; i += jump) {
    a[i] += 1;
}
```

На каждой итерации цикла обновляется один целый элемент массива, а количество итераций равно числу страниц Numpages. Замеряя время многократного повторения этого цикла (например, можно организовать внешний цикл, повторяющийся миллион раз или на протяжении одной секунды), мы можем узнать среднее время одного доступа. Наблюдая за тем, при каком значении Numpages стоимость доступа скачкообразно возрастает, можно грубо оценить размер TLB первого уровня и определить, существует ли TLB второго уровня (и если да, то его размер), а также получить общее представление о влиянии попаданий и непопаданий в TLB на производительность.

На рис. 19.5 изображена зависимость среднего времени доступа от количества страниц, перебираемых в цикле. Как видно из графика, пока обращения производятся к небольшому числу страниц (8 или меньше), среднее время доступа составляет примерно 5 наносекунд. Когда число страниц возрастает до 16, происходит внезапный скачок до 20 наносекунд на один доступ. И последний скачок стоимости имеет место в районе 1024 страниц, когда время доступа увеличивается до 70 нс. Отсюда можно сделать вывод о присутствии двухуровневой иерархии TLB; первый уровень совсем мал (от 8 до 16 записей), а второй больше, но медленнее (где-то около 512 записей). Разница между попаданием в TLB первого уровня и непопаданием очень велика – 14 раз. Так что производительность TLB имеет значение!

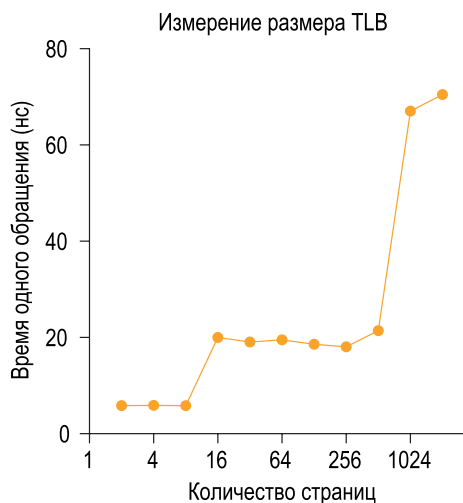


Рис. 19.5 ❖ Определение размера TLB и стоимости непопадания

Вопросы

1. Для хронометража нам понадобится какой-нибудь таймер (например, `gettimeofday()`). Насколько он точен? Сколько времени должна занимать операция, чтобы его можно было измерить точно (это поможет определить, сколько должно быть итераций в цикле повторного доступа к страницам, чтобы можно было рассчитывать на успех измерения)?
2. Напишите программу `tlb.c`, которая приблизительно измеряет стоимость доступа к странице. Параметрами программы должны быть количество страниц, к которым производится обращение, и количество испытаний.
3. Напишите на своем любимом скриптовом языке (`csh`, `python` и т. д.) скрипт, который будет запускать эту программу, изменяя количество страниц от 1 до нескольких тысяч. Имеет смысл на каждой итерации увеличивать это число вдвое. Выполните скрипт на разных машинах и соберите статистику. Сколько испытаний необходимо для получения надежных результатов?
4. Нанесите результаты на график, похожий на тот, что представлен на рисунке выше. Воспользуйтесь каким-нибудь хорошим инструментальным средством, например `ploticus` или даже `zplot`. Обычно визуализация сильно упрощает восприятие данных; почему, на ваш взгляд?
5. Замечание по поводу оптимизации компилятором. Компиляторы часто «умничают», например удаляют циклы, в которых инкрементируется переменная, впоследствии нигде не используемая. Как гарантировать, что компилятор не удалит главный цикл из вашей программы оценки размера TLB?
6. Следует также помнить о том, что в наши дни большинство систем оснащены несколькими CPU, и, разумеется, у каждого CPU своя иерархия кешей. Чтобы добиться высокого качества измерений, нужно запускать код ровно на одном CPU, не позволяя планировщику перебрасывать его с одного CPU на другой. Как это сделать (указание: поищите в Google по словам «привязка к процессору» (англ. *pinning a thread*)? Что будет, если вы не предпримете таких мер и код будет перемещаться с одного CPU на другой?
7. Возможна еще одна проблема, связанная с инициализацией. Если не инициализировать массив перед доступом к нему в цикле, то первая итерация может оказаться очень дорогой из-за затрат на начальный доступ, например обнуления по требованию. Повлияет ли это на хронометраж вашего кода? Как можно компенсировать такие потенциальные затраты?

Глава 20

Страничная организация: уменьшенные таблицы

Теперь мы займемся второй проблемой страничной организации: таблицы страниц слишком велики и потому потребляют чересчур много памяти. Начнем с линейной таблицы страниц. Как вы, наверное, знаете¹, линейные таблицы страниц довольно велики. Снова предположим, что адресное пространство 32-разрядное (2^{32} байт), страницы имеют размер 4 КБ (2^{12} байт), а одна запись таблицы страниц занимает 4 байта. Следовательно, адресное пространство содержит приблизительно миллион виртуальных страниц ($2^{32}/2^{12}$). Умножим это на размер одной записи и получим, что размер таблицы страниц составляет 4 МБ. Напомним: обычно в системе существует по одной таблице страниц на *каждый процесс*! При ста активных процессах (не так уж много для современных систем) мы будем выделять сотни мегабайтов памяти только для таблиц страниц! Нужно придумать какой-нибудь способ уменьшить этот тяжкий груз. Таких способов много, и сейчас мы их рассмотрим. Но сначала

СУЩЕСТВО ПРОБЛЕМЫ: КАК УМЕНЬШИТЬ РАЗМЕР ТАБЛИЦ СТРАНИЦ?

Простые таблицы страниц в виде массива (обычно называемые линейными) слишком велики, т. е. занимают чрезмерно много памяти в типичной системе. Как уменьшить их размер? Могут ли новые структуры данных стать причиной неэффективности?

20.1. ПРОСТОЕ РЕШЕНИЕ: УВЕЛИЧЕННЫЕ СТРАНИЦЫ

Размер таблицы страниц можно было бы уменьшить очень просто: использовать увеличенные страницы. Снова рассмотрим 32-разрядное адресное

¹ А может, и не знаете; ведь страничная организация – вещь, которой мы не управляем. Тем не менее всегда следует сначала понять проблему, которую вы пытаетесь решить, а только потом приступать к решению. Поняв, в чем проблема, вы зачастую сможете самостоятельно найти решение. В данном случае проблема очевидна: таблицы страниц в виде простого линейного массива слишком велики.

пространство, но будем считать, что размер страницы равен 12 КБ, т. е. 18 бит отводится на VPN, 14 на смещение. В предположении, что размер записи такой же (4 байта), общее число записей в нашей таблице равно 2^{18} , значит, ее размер равен 1 МБ. Мы получили сокращение в четыре раза (что и неудивительно, ведь в те же четыре раза увеличился размер страницы).

ОТСТУПЛЕНИЕ: СТРАНИЦЫ НЕСКОЛЬКИХ РАЗМЕРОВ

Отметим, что во многих современных архитектурах (например, MIPS, SPARC, x86-64) поддерживаются страницы нескольких размеров. Обычно используется небольшая страница (4 или 8 КБ). Но если «умное» приложение попросит, то в определенной области адресного пространства можно использовать одну большую страницу (скажем, размером 4 МБ), что позволит приложению размещать в ней особо востребованные (и большие) структуры данных, потребляя всего одну запись в TLB. Большие страницы часто находят применение в системах управления базами данных и других коммерческих приложениях высшей ценовой категории. Но основная причина использования страниц разных размеров – не столько экономия места для таблицы страницы, сколько уменьшение нагрузки на TLB, чтобы программа могла обращаться к большей части своего адресного пространства, не слишком часто сталкиваясь с непопаданием в TLB. Но, как показывают исследования [N+02], наличие нескольких размеров страниц заметно усложняет диспетчер виртуальной памяти в ОС, поэтому иногда большие страницы проще всего использовать напрямую, экспортируя приложениям новый интерфейс.

Однако у этого подхода есть серьезная проблема: большие страницы ведут к растраживанию места *внутри* каждой страницы, т. е. к **внутренней фрагментации**. Так как приложение выделяет страницы, но использует лишь их малую часть, память быстро забивается такими чрезмерно большими страницами. Поэтому в большинстве систем используются лишь сравнительно малые страницы: 4 КБ (как в x86) или 8 КБ (как в SPARCv9). Увы, так просто нашу проблему не решить.

20.2. Гибридный подход:

СТРАНИЧНАЯ ОРГАНИЗАЦИЯ И СЕГМЕНТЫ

Всякий раз, как к какой-то жизненной ситуации имеется два разумных подхода, следует рассмотреть их комбинацию – быть может, удастся получить лучшее из обоих миров. Такая комбинация называется **гибридом**. Например, чем есть отдельно шоколад и арахис, не лучше ли объединить в чудесном гибриде под названием арахисовая паста Риза [M28]?

Много лет назад создатели Multics (в особенности Джек Деннис) наткнулись на такую идею при конструировании системы виртуальной памяти [M07]. Именно Деннису пришла мысль объединить страничную организацию и сегментацию, чтобы уменьшить накладные расходы, связанные с таблицами страниц. Почему это может сработать, мы поймем, рассмотрев типичную линейную таблицу страниц более пристально. Предположим, что имеется

адресное пространство, в котором куча и стек занимают мало места. Например, пусть размер адресного пространства равен 16 КБ, а размер страницы 1 КБ (рис. 20.1); таблица страниц для него показана на рис. 20.2.

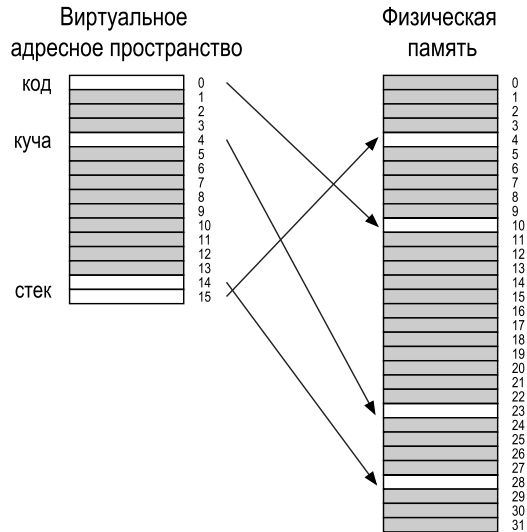


Рис. 20.1 ❖ Адресное пространство размером 16 КБ со страницами размером 1 КБ

PFN	Бит достоверности	Биты защиты	Бит присутствия	Бит изменения
10	1	rw-	1	0
-	0	---	-	-
-	0	---	-	-
-	0	---	-	-
23	1	rw-	1	1
-	0	---	-	-
-	0	---	-	-
-	0	---	-	-
-	0	---	-	-
-	0	---	-	-
-	0	---	-	-
-	0	---	-	-
-	0	---	-	-
-	0	---	-	-
28	1	rw-	1	1
4	1	rw-	1	1

Рис. 20.2 ❖ Таблица страниц для адресного пространства размером 16 КБ

В этом примере предполагается, что единственная страница кода (VPN 0) отображена на физическую страницу 10, единственная страница кучи (VPN 4) на физическую страницу 23, а две страницы стека на другом конце

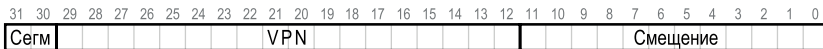
адресного пространства (VPN 14 и 15) на физические страницы 28 и 4 соответственно. Из рисунка видно, что *большая часть* таблицы страниц не используется и занята **недействительными** записями. Какое расточительство! И это для крохотного адресного пространства. А вообразите таблицу страниц для 32-разрядного адресного пространства, это сколько же места мы там потеряем! Нет, не стоит даже воображать – уж больно безотрадная картина получается.

Отсюда вытекает идея гибридного подхода: вместо того чтобы заводить единственную линейную таблицу страниц для всего адресного пространства процесса, почему бы не завести по одной таблице на каждый логический сегмент? В нашем примере нам понадобилось бы три таблицы страниц: для кода, кучи и стека.

Теперь вспомним, что в системе с сегментацией у нас был регистр **базы**, который сообщал, в каком месте физической памяти находится сегмент, и регистр **границы**, содержащий размер сегмента. В гибридной модели эти структуры по-прежнему присутствуют внутри MMU, но только база указывает не на сам сегмент, а содержит *физический адрес таблицы страниц* этого сегмента. А регистр границы обозначает конец таблицы страниц (т. е. сколько в ней действительных страниц).

Поясним на простом примере. Пусть имеется 32-разрядное адресное пространство со страницами размером 4 КБ, которое разбито на четыре сегмента. Мы будем использовать только три из них: для кода, для кучи и для стека.

Чтобы понять, к какому сегменту относится адрес, мы будем использовать два старших бита адреса. Пусть 00 обозначает неиспользуемый сегмент, 01 – сегмент кода, 10 – сегмент кучи и 11 – сегмент стека. Таким образом, виртуальный адрес имеет такую структуру:



Что касается оборудования, будем предполагать, что имеется три пары регистров базы и границы, по одному для сегментов кода, кучи и стека. Когда процесс выполняется, в регистре базы каждого сегмента находится физический адрес соответствующей ему линейной таблицы страниц, так что с каждым процессом в системе теперь ассоциировано *три* таблицы страниц. При контекстном переключении эти регистры необходимо изменить, отразив положение таблиц страниц замещающего процесса.

В случае непопадания в TLB (предполагается аппаратно управляемый TLB, когда за обработку непопаданий отвечает оборудование) биты сегмента (SN) служат для определения того, какую пару регистров базы и границы использовать. Оборудование извлекает хранящийся в регистре базы адрес и объединяет его с VPN для формирования адреса записи таблицы страниц (PTE):

```
SN      = (VirtualAddress & SEG_MASK) >> SN_SHIFT
VPN     = (VirtualAddress & VPN_MASK) >> VPN_SHIFT
AddressOfPTE = Base[SN] + (VPN * sizeof(PTE))
```

Эта последовательность нам уже знакома, она практически идентична той, что мы видели при рассмотрении линейных таблиц страниц. Единственная разница в том, что теперь используется один из трех регистров базы сегментов, а не единственный регистр базы таблицы страниц.

Критически важное отличие нашей гибридной схемы – наличие регистра границы для каждого сегмента; в этом регистре хранится номер максимальной действительной страницы в сегменте. Например, если в сегменте кода использованы первые три страницы (0, 1 и 2), то в таблице страниц сегмента кода будет выделено только три записи и регистр границы будет равен 3; обращение к памяти за границей сегмента приведет к исключению и, вероятно, завершению процесса. Таким образом, гибридный подход значительно экономит память по сравнению с линейной таблицей; невыделенные страницы между стеком и кучей больше не занимают места в таблице страниц (просто для того, чтобы быть помеченными как недействительные).

СОВЕТ: ПРИМЕНЯЙТЕ ГИБРИДНЫЕ ПОДХОДЫ

Если имеется две хорошие, но кажущиеся несовместимыми идеи, всегда старайтесь найти способ объединить их в **гибридное** решение, чтобы получить лучшее из обоих миров. Например, известно, что гибриды кукурузы более устойчивы к вредителям и болезням, чем природные сорта. Конечно, не всегда гибрид – хорошая идея; взять, к примеру, зеброидов – гибрид зебры с ослом. Если вы не верите в существование подобного животного, поищите в сети – будете удивлены.

Но, как вы наверняка заметили, у этого подхода есть свои проблемы. Во-первых, приходится использовать сегментацию, а, как было сказано выше, сегментация – не такое гибкое решение, как нам хотелось бы, потому что предполагается определенный способ использования адресного пространства, например при наличии большой, но разреженной кучи значительная часть таблицы страниц может расходоваться впустую. Во-вторых, снова возникает проблема внешней фрагментации. Хотя большая часть памяти выделяется блоками размером со страницу, таблицы страниц могут теперь иметь произвольный размер (кратный PTE). Таким образом, найти для них свободное место в памяти оказывается сложнее. Поэтому усилия, направленные на поиск лучших путей реализации уменьшенных таблиц страниц, были продолжены.

20.3. МНОГОУРОВНЕВЫЕ ТАБЛИЦЫ СТРАНИЦ

Другой подход не опирается на сегментацию, но призван решить ту же проблему: как не хранить в памяти области, занятые недействительными записями таблицы страниц? Этот подход называется **многоуровневой таблицей страниц**, поскольку линейная таблица преобразуется в древовидную. Он настолько эффективен, что применяется в большинстве современных систем (например, в x86 [ВОН10]). Опишем его более подробно.

Основная идея проста. Сначала разобьем таблицу страниц на блоки размером со страницу. Затем, если целая страница состоит только из недействительных записей (PTE), не станем выделять для нее память. Для отслеживания того, какие страницы действительны (и где они в таком случае находятся в памяти), используется новая структура – **каталог страниц**. Таким образом, каталог можно использовать для двух целей: узнать, где расположена страница, присутствующая в таблице страниц, и имеется ли в данной странице хотя бы одна действительная запись.

На рис. 20.3 приведен пример. В левой части мы видим классическую линейную таблицу страниц; хотя средняя часть (вторая и третья страницы) адресного пространства в основном содержит только недействительные страницы, для этих областей все равно приходится выделять память. Справа показана многоуровневая таблица страниц. В каталоге страниц лишь две ее страницы помечены как действительные (первая и последняя), т. е. только эти две страницы и находятся в памяти. Таким образом, можно наглядно представить себе, в чем смысл многоуровневой таблицы страниц: она исключает части линейной таблицы (освобождая соответствующие физические рамки для других целей) и с помощью каталога следит за тем, какие страницы таблицы страниц находятся в памяти.

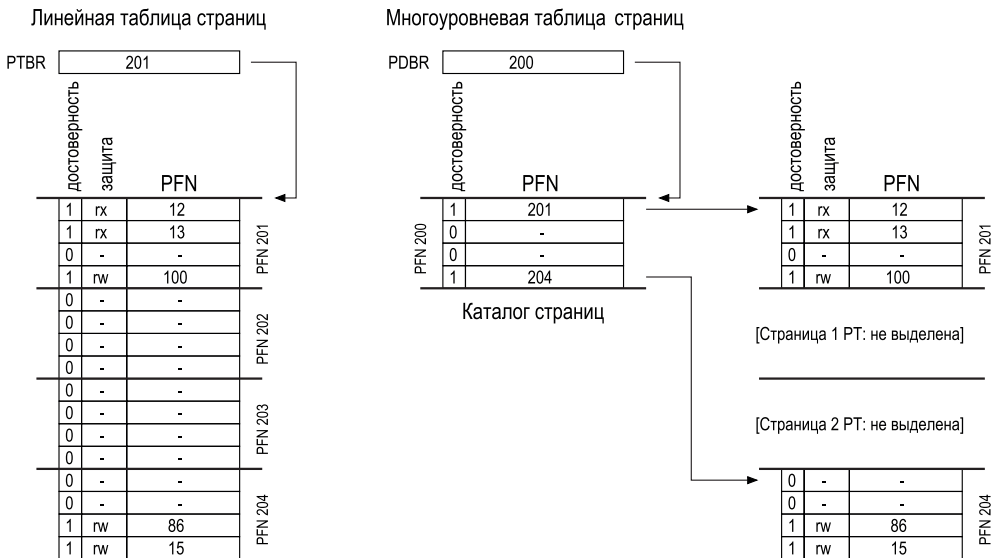


Рис. 20.3 ❖ Линейная (слева) и многоуровневая (справа) таблицы страниц

Каталог страниц – это просто таблица второго уровня, которая содержит по одной записи на каждую страницу таблицы страниц. Он состоит из **записей каталога страниц** (page directory entry – **PDE**). PDE содержит (как минимум) бит достоверности и номер физической рамки (PFN) – так же, как PTE. Однако интерпретация бита достоверности отличается: если запись PDE действительна, значит, по крайней мере одна из записей на той странице таблицы

страниц, на которую указывает эта запись (посредством PFN), действительна, т. е. хотя бы в одной записи PTE бит достоверности установлен в 1. Если же PDE недействительна (т. е. бит достоверности равен 0), то остальные поля этой PDE не определены.

У многоуровневых таблиц страниц есть очевидные преимущества по сравнению с предыдущими решениями. Первое и самое главное – тот факт, что объем выделенного в таблице страниц места пропорционален доле используемого адресного пространства, т. е. в общем случае это представление компактно и поддерживает разреженные адресные пространства.

Во-вторых, при аккуратном конструировании каждая часть таблицы страниц точно уместается на странице, что упрощает управление памятью; ОС может просто взять следующую свободную страницу, когда ей понадобится выделить новую таблицу страниц или увеличить существующую. Сравните это с простой линейной таблицей страниц¹, которая представляет собой массив записей PTE, индексированный VPN; при такой структуре вся линейная таблица страниц должна располагаться в непрерывном участке физической памяти. Для большой таблицы страниц (скажем, размером 4 МБ) найти такой большой непрерывный участок свободной физической памяти может оказаться непростой проблемой. В случае многоуровневой структуры мы добавляем еще один **уровень косвенности** – каталог страниц, который указывает на куски таблицы страниц; косвенность позволяет размещать страницы таблицы страниц в любом месте физической памяти.

СОВЕТ: РАЗБЕРИТЕСЬ В КОМПРОМИССАХ МЕЖДУ ПРОСТРАНСТВОМ И ВРЕМЕНЕМ

При проектировании структуры данных всегда следует рассматривать **компромиссы между пространством и временем**. Обычно за ускорение доступа к некоторой структуре данных приходится расплачиваться повышенным потреблением памяти.

Следует отметить, что у многоуровневых таблиц тоже есть цена: при попадании в TLB, чтобы получить информацию о трансляции из таблицы страниц, придется обратиться к памяти дважды (к каталогу страниц и к самой записи PTE), а не один раз, как в случае линейной таблицы страниц. Так что многоуровневая таблица – пример **компромисса между пространством и временем**. Мы хотели уменьшенные таблицы и получили их, но не бесплатно; хотя в типичном случае (попадание в TLB) производительность такая же, как и раньше, но при попадании стоимость работы с меньшей таблицей увеличивается.

Еще один очевидный недостаток – *сложность*. Не важно, кто занимается поиском в таблице страниц – оборудование или ОС, это, вне всяких сомнений, сложнее, чем поиск в линейной таблице. Часто мы готовы расплачиваться сложностью за повышение производительности или снижение накладных

¹ Здесь мы вводим одно предположение, а именно что все таблицы страниц полностью помещаются в физической памяти (т. е. не выгружаются на диск); вскоре мы его ослабим.

расходов, конкретно в случае многоуровневых таблиц мы согласились на усложнение поиска, чтобы сэкономить ценную память.

Подробный пример работы с многоуровневой таблицей страниц

Чтобы лучше разобраться в многоуровневых таблицах страниц, рассмотрим пример. Возьмем небольшое адресное пространство размером 16 КБ со страницами размером 64 байта. Имеем, стало быть, 14-разрядное виртуальное адресное пространство, так что 8 бит отведено под VPN, а 6 бит под смещение. Линейная таблица страниц содержала бы 2^8 (256) записей, даже если используется лишь ее малая часть. На рис. 20.4 показано такое адресное пространство.

0000 0000	код
0000 0001	код
0000 0010	(свободно)
0000 0011	(свободно)
0000 0100	куча
0000 0101	куча
0000 0110	(свободно)
0000 0111	(свободно)
.....	... все свободно ...
1111 1100	(свободно)
1111 1101	(свободно)
1111 1110	стек
1111 1111	стек

Рис. 20.4 ❖ 16-килобайтовое адресное пространство с 64-байтовыми страницами

СОВЕТ: БЕРЕГИТЕСЬ СЛОЖНОСТИ

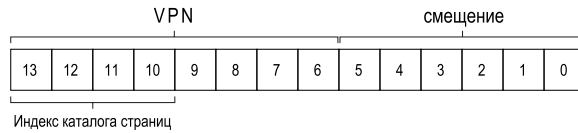
Проектировщики должны с осторожностью относиться к увеличению сложности системы. Хороший проектировщик всегда стремится реализовать самую простую систему, достигающую поставленных целей. Например, если места на диске в избытке, то не следует проектировать файловую систему так, чтобы минимизировать число используемых байтов до предела. Аналогично, если процессор быстрый, то лучше написать простой и понятный модуль ОС, чем вручную оптимизировать ассемблерный код. Берегитесь избыточной сложности, будь то в форме преждевременно оптимизируемого кода или в какой-то другой, потому что такое решение будет труднее понять, сопровождать и отлаживать. Как писал Антуан де Сент-Экзюпери: «Совершенство достигается не тогда, когда уже нечего прибавить, но когда уже ничего нельзя отнять». А вот чего он не писал: «Гораздо проще что-то сказать о совершенстве, чем на деле достичь его».

В этом примере виртуальные страницы 0 и 1 заняты кодом, страницы 4 и 5 – кучей, а страницы 254 и 255 – стеком, остальные страницы адресного пространства не используются.

Чтобы построить двухуровневую таблицу страниц для этого адресного пространства, мы начнем с полной линейной таблицы и разобьем ее на блоки длиной со страницу. Напомним, что в полной таблице из этого примера имеется 256 записей, будем предполагать, что длина каждой записи 4 байта. Таким образом, вся таблица страниц занимает 1 КБ (256 × 4 байта). Поскольку размер страницы в нашем случае 64 байта, эту таблицу страниц можно разбить на 16 страниц, и в каждой из них будет 16 PTE.

Теперь нужно понять, как использовать VPN для индексирования сначала каталога страниц, а затем страницы в таблице страниц. Напомним, что то и другое – массив записей, поэтому нужно лишь придумать, как образовать индексы каждой части из VPN.

Сначала проиндексируем каталог страниц. В этом примере таблица страниц мала: 256 записей, распределенных по 16 страницам. В каталоге страниц должно быть по одной записи на каждую страницу таблицы страниц, т. е. всего 16 записей. В результате нам нужно отвести четыре бита VPN под индекс записи каталога, зарезервируем для этой цели четыре старших бита, как показано на рисунке ниже:



Выделив **индекс каталога страниц** (для краткости PDIIndex) из VPN, мы можем с его помощью без труда найти адрес записи каталога страниц (PDE): $PDEAddr = PageDirBase + (PDIIndex * sizeof(PDE))$, после чего можно двигаться дальше.

Если эта запись каталога страниц недействительна, то доступ к ней запрещен и нужно возбудить исключение. Если же она действительна, то придется еще немного поработать. Именно, мы должны найти конкретную запись на той странице таблицы страниц, на которую указывает данная запись каталога страниц. Чтобы найти эту PTE, нужно использовать оставшиеся биты VPN как индекс в эту часть таблицы страниц:



Затем этот **индекс таблицы страниц** (для краткости PTIndex) можно использовать для нахождения PTE в самой таблице страниц:

$$PTEAddr = (PDE.PFN \ll \text{SHIFT}) + (PTIndex * \text{sizeof}(PTE))$$

Заметим, что номер физической рамки (PFN), взятый из записи каталога страниц, необходимо сдвинуть влево, прежде чем объединять с индексом таблицы страниц для образования адреса РТЕ.

Чтобы посмотреть, как все это выглядит на практике, заполним многоуровневую таблицу страниц конкретными значениями и транслируем один виртуальный адрес. Начнем с каталога страниц (левая часть рис. 20.5).

Каталог страниц		Страница таблицы страниц (@PFN:100)			Страница таблицы страниц (@PFN:101)		
PFN	достоверна	PFN	достоверна	защита	PFN	достоверна	защита
100	1	10	1	г-х	—	0	—
—	0	23	1	г-х	—	0	—
—	0	—	0	—	—	0	—
—	0	—	0	—	—	0	—
—	0	80	1	гw-	—	0	—
—	0	59	1	гw-	—	0	—
—	0	—	0	—	—	0	—
—	0	—	0	—	—	0	—
—	0	—	0	—	—	0	—
—	0	—	0	—	—	0	—
—	0	—	0	—	—	0	—
—	0	—	0	—	—	0	—
—	0	—	0	—	—	0	—
—	0	—	0	—	—	0	—
—	0	—	0	—	55	0	гw-
101	1	—	0	—	45	1	гw-

Рис. 20.5 ❖ Каталог страниц и части таблицы страниц

На рисунке видно, что каждая запись **каталога страниц** (PDE) сообщает что-то о странице таблицы страниц для нашего адресного пространства. В данном примере имеется две используемые области адресного пространства (в начале и в конце), а между ними – ряд недействительных записей.

На физической странице 100 (номер физической рамки нулевой страницы таблицы страниц) мы имеем первую страницу, содержащую 16 записей, соответствующих первым 16 VPN в адресном пространстве. Содержимое этой части таблицы страниц см. на рис. 20.5 в средней части.

Эта страница таблицы страниц содержит отображения для первых 16 VPN; в нашем примере действительны VPN 0 и 1 (сегмент кода), а также 4 и 5 (куча). И в таблице присутствуют отображения для этих страниц. Все остальные записи помечены как недействительные.

Другая действительная страница имеет PFN 101. Она содержит отображения для последних 16 VPN адресного пространства, детали см. на рис. 20.5 справа.

В этом примере VPN 254 и 255 (стек) содержат действительные отображения. Пример показывает, сколько места можно сэкономить с помощью

многоуровневой индексной структуры. Вместо того чтобы выделять полные *шестнадцать* страниц для линейной таблицы страниц, мы выделили только *три*: одну для каталога страниц и две для тех участков таблицы страниц, в которых имеются действительные трансляции. Очевидно, что экономия для больших (32- и 64-разрядных) адресных пространств будет гораздо больше.

Наконец, воспользуемся этой информацией, чтобы выполнить трансляцию. Адрес $0x3F80$ (двоичное 11 1111 1000 0000) относится к нулевому байту страницы с VPN 254.

Напомним, что старшие четыре бита VPN интерпретируются как индекс каталога страниц. Следовательно, биты 1111 выбирают последнюю (15-ю, если отсчитывать от нуля) запись приведенного выше каталога страниц. Она указывает на действительную страницу таблицы страниц по адресу 101. Затем мы используем следующие четыре бита VPN (1110) для индексирования страницы таблицы страниц и находим нужную PTE. 1110 – это номер предпоследней (14-й) записи на странице, которая сообщает, что страница 254 нашего виртуального адресного пространства отображена на физическую страницу 55. Объединяя PFN=55 ($0x37$) со смещением $offset=000000$, мы формируем физический адрес и передаем запрос подсистеме памяти: $PhysAddr = (PTE.PFN \ll SHIFT) + offset = 00\ 1101\ 1100\ 0000 = 0x0DC0$.

Теперь вы понимаете, как устроена двухуровневая таблица страниц, в которой имеется каталог страниц, указывающий на страницы таблицы страниц. Но, к сожалению, работа еще не окончена. Ниже мы увидим, что иногда и двух уровней недостаточно!

Больше двух уровней

До сих пор мы предполагали, что в многоуровневой таблице страниц имеется всего два уровня: каталог страниц и части таблицы страниц. Но в некоторых случаях возможно (и даже необходимо) более глубокое дерево.

На простом примере покажем, почему более глубокая многоуровневая таблица может быть полезна. Будем предполагать, что виртуальное адресное пространство 3-разрядное, а страница мала (512 байт). Это означает, что номер виртуальной страницы занимает 21 бит, а смещение 9 бит.

Напомним цель построения многоуровневой таблицы страниц: сделать так, чтобы каждый кусок таблицы страниц умещался в одной странице. Пока что мы рассматривали только саму таблицу страниц, но что, если слишком большим становится каталог страниц?

Чтобы определить, сколько уровней должно быть в многоуровневой таблице, чтобы не было кусков таблицы страниц, занимающих более одной страницы, начнем с вычисления того, сколько записей находится на одной странице. Поскольку размер страницы равен 512 байт, а размер PTE составляет 4 байта, понятно, что на странице помещается 128 PTE. Поэтому для индексирования страницы таблицы страниц нам понадобятся младшие 7 бит ($\log_2 128$) VPN:



Из рисунка также видно, сколько битов остается в (большом) каталоге страниц: 14. Если в нашем каталоге страниц имеется 2^{14} записей, то он занимает не одну страницу, а 128, поэтому цель добиться того, чтобы каждый кусок многоуровневой таблицы страниц умещался в одной странице, остается недостигнутой.

Чтобы исправить ситуацию, мы построим еще один уровень дерева, разбив сам каталог страниц на несколько страниц и добавив новый каталог, который указывает на страницы каталога страниц. Тогда виртуальный адрес нужно будет интерпретировать следующим образом:



Теперь для индексирования каталога страниц верхнего уровня используются старшие биты виртуального адреса (индекс PD 0 на рисунке); зная этот индекс, мы выбираем запись из каталога страниц верхнего уровня. Если она действительна, то мы заглядываем в каталог страниц второго уровня, объединяя номер физической рамки из PDE верхнего уровня со следующей частью VPN (индекс PD 1). Наконец, если эта запись действительна, то можно сформировать адрес PTE, объединив индекс таблицы страниц с адресом, хранящимся в PDE второго уровня. Уфф! Сколько работы! И все для того, чтобы найти что-то в многоуровневой таблице.

Процесс трансляции: вспомним про TLB

Чтобы описать весь процесс трансляции с применением двухуровневой таблицы страниц, снова представим псевдокод алгоритма (рис. 20.6). Здесь показано, что делает оборудование (в предположении, что TLB управляется аппаратно) при *каждом* обращении к памяти.

Перед тем как заглянуть в таблицу на любом уровне, оборудование справляется с TLB; если имеет место попадание, физический адрес формируется вообще без обращения к таблице страниц, как и раньше. И только в случае непадания в TLB оборудование должно выполнить полный поиск в многоуровневой таблице. И тут видно, во что обходится наша традиционная двух-

уровневая таблица страниц: два дополнительных обращения к памяти для поиска нужной трансляции.

```

1 VPN = (VirtualAddress & VPN_MASK) >> SHIFT
2 (Success, TlbEntry) = TLB_Lookup(VPN)
3 if (Success == True)    // попадание в TLB
4     if (CanAccess(TlbEntry.ProtectBits) == True)
5         Offset = VirtualAddress & OFFSET_MASK
6         PhysAddr = (TlbEntry.PFN << SHIFT) | Offset
7         Register = AccessMemory(PhysAddr)
8     else
9         RaiseException(PROTECTION_FAULT)
10 else    // не попадание в TLB
11     // сначала получить запись каталога страниц
12     PDIndex = (VPN & PD_MASK) >> PD_SHIFT
13     PDEAddr = PDBR + (PDIndex * sizeof(PDE))
14     PDE = AccessMemory(PDEAddr)
15     if (PDE.Valid == False)
16         RaiseException(SEGMENTATION_FAULT)
17     else
18         // PDE действительна: теперь выбрать PTE из таблицы страниц
19         PTIndex = (VPN & PT_MASK) >> PT_SHIFT
20         PTEAddr = (PDE.PFN << SHIFT) + (PTIndex * sizeof(PTE))
21         PTE = AccessMemory(PTEAddr)
22         if (PTE.Valid == False)
23             RaiseException(SEGMENTATION_FAULT)
24         else if (CanAccess(PTE.ProtectBits) == False)
25             RaiseException(PROTECTION_FAULT)
26         else
27             TLB_Insert(VPN, PTE.PFN, PTE.ProtectBits)
28             RetryInstruction()

```

Рис. 20.6 ❖ Поток управления
при поиске в многоуровневой таблице страниц

20.4. ИНВЕРТИРОВАННЫЕ ТАБЛИЦЫ СТРАНИЦ

Еще большей экономии памяти позволяют добиться **инвертированные таблицы страниц**. В этом случае вместо нескольких таблиц страниц (по одной на каждый работающий в системе процесс) мы храним одну таблицу страниц, в которой имеется запись для каждой *физической страницы* в системе. Эта запись сообщает, какой процесс использует данную страницу и какая виртуальная страница этого процесса на нее отображена.

Для нахождения нужной записи необходимо произвести поиск в этой структуре данных. Линейный поиск обошелся бы слишком дорого, поэтому часто над базовой структурой надстраивается хеш-таблица. Примером такой архитектуры может служить PowerPC [JM98].

Вообще, инвертированные таблицы страниц иллюстрируют высказанную в самом начале мысль о том, что таблицы страниц – обычные структуры данных. Со структурами данных можно производить самые разные вещи: умень-

шать или увеличивать, ускорять или замедлять. Многоуровневые и инвертированные таблицы страниц – всего лишь два примера возможных действий.

20.5. ВЫГРУЗКА ТАБЛИЦ СТРАНИЦ НА ДИСК

Наконец, обсудим ослабление последнего оставшегося предположения. До сих пор мы считали, что таблицы страниц находятся в физической памяти ядра. И хотя наши ухищрения позволили уменьшить размер таблиц страниц, все еще может оказаться, что они настолько велики, что не помещаются в память все сразу. Поэтому в некоторых системах такие таблицы страниц размещаются в *виртуальной памяти ядра*, что позволяет системе выгружать их на диск, когда памяти не хватает. Мы поговорим об этом подробнее в следующей главе (когда будем изучать VAX/VMS), после того как поймем, каким образом страницы выгружаются из памяти и подкачиваются в нее.

20.6. РЕЗЮМЕ

Мы видели, как конструируются реальные таблицы страниц, это могут быть не только простые линейные массивы, но и более сложные структуры данных. При конструировании приходится искать компромисс между временем и пространством: чем больше таблица, тем быстрее обрабатывается непопадание в TLB, и наоборот. Поэтому при выборе структуры необходимо учитывать ограничения в имеющемся окружении.

В системах с ограниченной памятью (многие системы постарше) имеет смысл использовать небольшие структуры, а если памяти достаточно, а рабочая нагрузка такова, что активно используется много страниц, то будет правильнее взять таблицу побольше, чтобы непопадания в TLB обрабатывались быстрее. В случае программно управляемых TLB изобилие структур данных открывает простор для творчества новаторски настроенного проектировщика операционных систем (намек: для вас). Какие новые структуры можно придумать? Какие проблемы они решают? Подумайте об этих вопросах на сон грядущий, и пусть вам приснятся прекрасные сны, какие могут сниться только разработчикам операционных систем.

Литература

[BOH10] «Computer Systems: A Programmer's Perspective» by Randal E. Bryant and David R. O'Hallaron. Addison-Wesley, 2010. *Мы еще не нашли первое хорошее техническое руководство по многоуровневым таблицам страниц. Но в этом прекрасном учебнике рассматриваются детали x86, а это одна из ранних систем, в которых такие структуры использовались. Да и вообще отличная книга.*

[JM98] «Virtual Memory: Issues of Implementation» by Bruce Jacob, Trevor Mudge. IEEE Computer, June 1998. *Прекрасный обзор нескольких систем и принятых в них подходов к виртуализации памяти. Приведено много деталей устройства x86, PowerPC, MIPS и других архитектур.*

[LL82] «Virtual Memory Management in the VAX/VMS Operating System» by Hank Levy, P. Lipman. IEEE Computer, Vol. 15, No. 3, March 1982. *Потрясающая статья о реальном диспетчере виртуальной памяти в классической операционной системе VMS. Насколько потрясающая, что в одной из последующих глав мы воспользуемся ей, чтобы проиллюстрировать все, что узнали о виртуальной памяти.*

[M28] «Reese's Peanut Butter Cups» by Mars Candy Corporation¹. *По-видимому, эти конфетки придумал в 1928 году Гарри Бэрнетт Риз, бывший владелец молочной фермы и начальник цеха отгрузки в компании Милтона С. Херши. По крайней мере, так написано в Википедии. Если это правда, то Херши и Риз, вероятно, люто ненавидели друг друга, как и положено двум шоколадным баронам.*

[N+02] «Practical, Transparent Operating System Support for Superpages» by Juan Navarro, Sitaram Iyer, Peter Druschel, Alan Cox. OSDI '02, Boston, Massachusetts, October 2002. *В этой статье все детали, которые необходимо учесть при включении больших страниц, или **суперстраниц**, в современную ОС. Увы, это не так просто, как кажется.*

[M07] «Multics: History» Доступно по адресу <http://www.multicians.org/history.html>. *На этом потрясающем сайте приведена масса исторических сведений о системе Multics, безусловно, одной из самых влиятельных ОС в истории. Вот цитата оттуда: «Джек Деннис из MIT привнес важные архитектурные идеи в начале работы над Multics, в особенности идею сочетания страничной организации и сегментации» (из раздела 1.2.1).*

Домашнее задание (эмуляция)

В этом небольшом домашнем задании проверяется, насколько хорошо вы поняли принцип работы многоуровневой таблицы страниц. Программа называется без претензий: `paging-multilevel-translate.py`; детали см. в файле README.

Вопросы

1. В случае линейной таблицы страниц необходим всего один регистр для указания местоположения таблицы страниц в предположении, что обо-

¹ Шоколадные тарталетки с арахисовой пастой. В России тоже продаются – на любителя. – Прим. перев.

рудование выполняет поиск трансляции при непадании в кеш. Сколько регистров необходимо для двухуровневой таблицы страниц? А для трехуровневой?

2. Воспользуйтесь эмулятором для выполнения трансляций. Задайте начальные значения генератора случайных чисел 0, 1 и 2 и проверьте свои ответы с помощью флага -с. Сколько обращений к памяти нужно для каждого поиска?
3. В предположении, что вы понимаете принцип работы кеша памяти, объясните, как будут вести себя обращения к таблице страниц в кеше. Приведут ли они к большому количеству паданий в кеш (и, значит, к быстрому доступу)? Или к большому количеству непаданий (и, значит, к медленному доступу)?

Глава 21

.....

За пределами физической памяти: механизмы

До сих пор мы предполагали, что адресное пространство нереалистично маленькое и целиком помещается в физическую память. Более того, мы предполагали, что адресные пространства *всех* работающих процессов помещаются в память. Теперь мы ослабим эти предположения и обсудим, как поддерживать много больших адресных пространств одновременно работающих процессов.

Для этого нам понадобится дополнительный уровень **иерархии запоминающих устройств**. До сих пор мы считали, что все страницы располагаются в физической памяти. Но чтобы поддерживать большие адресные пространства, ОС должна куда-то убирать части адресных пространств, которые в данный момент не пользуются спросом. В общем случае такое место должно иметь большую емкость, чем оперативная память, а значит, оно, скорее всего, будет медленнее (иначе зачем бы нам использовать память?). В современных системах эта роль обычно возлагается на **жесткий диск**. Таким образом, в нашей иерархии запоминающих устройств большой и медленный жесткий диск находится в самом низу, а оперативная память прямо над ним.

Существо проблемы: как выйти за пределы физической памяти

Как ОС может воспользоваться большим, но более медленным устройством, чтобы прозрачно создать иллюзию большого виртуального адресного пространства?

Возникает вопрос: зачем нужно поддерживать большое адресное пространство процесса? Ответ все тот же: для удобства и простоты использования. Располагая большим адресным пространством, мы избавлены от необходимости думать, хватит ли в памяти места для структур данных, мы можем писать код свободно, выделяя память по мере необходимости. Убедительная иллюзия, предоставляемая ОС, здорово упрощает нам жизнь. Противоположный пример дают старые системы, в которых использовалась **оверлей-**

ная память, когда программист вынужден был вручную перемещать код и данные в память и из памяти по мере необходимости [D97]. Попробуйте представить, как это выглядело: перед тем как обратиться к функции или данным, мы должны были сами загрузить их в память¹. Брррр!

ОТСТУПЛЕНИЕ: ТЕХНОЛОГИИ ХРАНЕНИЯ ДАННЫХ

Мы будем гораздо подробнее обсуждать устройства ввода-вывода ниже, так что потерпите. И конечно, более медленное устройство может быть не только жестким диском, но и более современным изделием, например твердотельным диском (SSD) на основе флеш-памяти. Об этом мы тоже поговорим. А пока просто предположим, что имеется емкое и сравнительно медленное устройство, которым можно воспользоваться, чтобы создать иллюзию очень большой виртуальной памяти, даже большей, чем сама физическая память.

Добавление пространства подкачки позволяет ОС не ограничиваться одним процессом, а создать иллюзию огромной виртуальной памяти для нескольких одновременно работающих процессов. С изобретением мультипрограммирования (исполнения «сразу» нескольких программ для более эффективного использования машинных ресурсов) возникла настоятельная необходимость выгружать часть страниц, потому что ранние компьютеры просто не могли хранить все страницы всех процессов в памяти. Таким образом, желание обеспечить мультипрограммирование вкупе с простотой использования заставляет нас поддерживать иллюзию большей памяти, чем физически доступно. Так поступают все современные системы ВП, и именно этим мы сейчас займемся.

21.1. Область подкачки

Прежде всего мы должны зарезервировать на диске место для выгрузки страниц. Обычно в операционных системах это место называется **областью подкачки**, или **областью выгрузки**, потому что мы *выгружаем* страницы из памяти и *подкачиваем* их обратно в память. Будем предполагать, что ОС умеет читать область подкачки и записывать в нее блоки размером со страницу. Для этого ОС должна запоминать **дисковый адрес** данной страницы.

Размер области подкачки важен, потому что именно он в конечном итоге определяет, какое максимальное количество страниц памяти может использоваться системой одновременно. Пока для простоты предположим, что этот размер *очень* велик.

На рис. 21.1 показан пример крохотной физической памяти на 4 страницы с 8-страничной областью подкачки. Три процесса (Proc 0, Proc 1 и Proc 2) со-

¹ По крайней мере, в плане обращения к функциям все выглядело не так ужасно, об этом заботился специальный оверлейный компоновщик. Но все равно приходилось вручную проектировать оверлейные сегменты, которые никогда не могли находиться в памяти одновременно. – Прим. перев.

вместно используют физическую память, но для каждого часть действительных страниц находится в памяти, а остальные в области подкачки на диске. Что касается четвертого процесса (Proc 3), то все его страницы выгружены на диск, так что он, очевидно, в данный момент не работает. В области подкачки остался один свободный блок. Даже на этом модельном примере можно видеть, как область подкачки позволяет системе создать иллюзию, будто памяти больше, чем на самом деле.

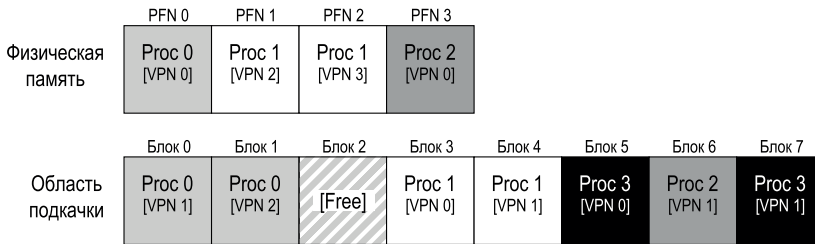


Рис. 21.1 ❖ Физическая память и область подкачки

Отметим, что область подкачки – не единственное место на диске, связанное с процессом выгрузки и подкачки. Пусть, например, мы запустили двоичный исполняемый файл (скажем, программу `ls` или свою собственную). Страницы ее кода первоначально находятся на диске, а при выполнении загружаются в память (либо все сразу в момент запуска, либо, как это делается в современных системах, по одной по мере необходимости). Но когда системе понадобится место в физической памяти для других надобностей, она может, ничего не опасаясь, воспользоваться страницами, в которые загружен код, точно зная, что впоследствии сможет их снова загрузить из двоичного файла.

21.2. Бит присутствия

Теперь, обзаведясь местом на диске, мы должны добавить в систему механизмы более высокого уровня, которые позволили бы поддерживать выгрузку страниц на диск и их подкачку обратно в память. Для простоты предположим, что TLB в системе управляется аппаратно.

Сначала вспомним, что происходит при обращении к памяти. Выполняемый процесс генерирует адреса виртуальной памяти (для выборки команд и доступа к данным), а оборудование транслирует их в физические адреса перед фактической выборкой из памяти.

Напомним, что оборудование сначала выделяет VPN из виртуального адреса, проверяет, нет ли нужной трансляции в TLB (**попадание в TLB**), и, если есть, берет соответствующий физический адрес и производит по нему выборку из памяти. Хочется надеяться, что это типичный случай, потому что он быстрый (не требуется дополнительных обращений к памяти).

Если же VPN не найден в TLB (**непопадание в TLB**), то оборудование находит таблицу страниц в памяти (пользуясь **регистром базы таблицы страниц**) и из нее получает **запись таблицы страниц (PTE)** для интересующей страницы, используя VPN как индекс. Если страница действительна и присутствует в физической памяти, то оборудование выделяет из PTE номер физической рамки PFN, сохраняет результат трансляции в TLB и повторяет выполнение команды, причем на этот раз гарантированно произойдет попадание в TLB. Пока все хорошо.

Но если мы хотим, чтобы страницы можно было выгружать на диск, то понадобится дополнительный механизм. Именно, может оказаться, что страница, на которую указывает PTE, *отсутствует* в физической памяти. Чтобы выяснить это, оборудование (или ОС в случае программно управляемого TLB) использует новый флаг в записи таблицы страниц – **бит присутствия**. Если этот бит равен 1, то страница присутствует в физической памяти, и все происходит, как раньше. Если же он равен 0, то страница находится не в памяти, а где-то на диске. Доступ к странице, отсутствующей в физической памяти, называется **отказом страницы**.

ОТСТУПЛЕНИЕ: ТЕРМИНОЛОГИЯ И ДРУГИЕ ВОПРОСЫ

Применяемая в системах виртуальной памяти терминология запутанна и зависит от конкретного компьютера и операционной системы. Например, под **отказом страницы** иногда понимают любую ошибку, связанную с таблицей страниц; это может быть как ошибка из-за отсутствия страницы в памяти, так и недопустимый доступ к памяти. Вообще-то, странно называть «отказом» совершенно законный доступ (к странице, отображенной в виртуальное адресное пространство процесса, но просто выгруженной в данный момент из памяти); правильнее было бы говорить об **отсутствии страницы**. Но зачастую, говоря, что программа «генерирует страничные отказы», люди имеют в виду доступ к частям адресного пространства, который ОС выгрузила на диск.

Мы подозреваем, что употребление слова «отказ» связано со способом обработки этой ситуации внутри операционной системы. Когда происходит что-то необычное, оборудование, не зная, как это обработать, просто передает ответственность ОС в надежде, что та разберется. В данном случае страница, нужная процессу, отсутствует в памяти, и оборудование делает то единственное, что может сделать, – возбуждает исключение, после чего управление перехватывает ОС. Поскольку то же самое происходит, когда процесс делает что-то недопустимое, использование термина «отказ» не должно вызывать удивления.

После отказа страницы для его обработки вызывается ОС. Начинает работать специальный код – **обработчик отказа страницы**, который мы опишем ниже.

21.3. ОТКАЗ СТРАНИЦЫ

Напомним, что в плане работы с TLB системы делятся на две категории: с аппаратно управляемым TLB (когда оборудование ищет нужную трансляцию в таблице страниц) и с программно управляемым TLB (когда это делает

OS). В любом случае, если страница отсутствует в памяти, ОС поручается обработать отказ страницы. **Обработчик отказа страницы** решает, что делать дальше. Практически во всех системах отказы страниц обрабатываются программно, даже в системе с аппаратно управляемым TLB оборудование оставляет эту важную работу ОС.

Если страница отсутствует и была выгружена на диск, то в процессе обработки отказа ОС должна подкачать страницу в память. Возникает вопрос: как ОС узнает, где искать нужную страницу? Во многих системах самое подходящее место для хранения такой информации – таблица страниц. Таким образом, ОС могла бы использовать для хранения дискового адреса те биты PTE, в которых обычно находится PFN страницы. Видя отказ страницы, ОС берет адрес из PTE и обращается к диску с запросом загрузить страницу в память.

ОТСТУПЛЕНИЕ: ПОЧЕМУ ОБОРУДОВАНИЕ НЕ ОБРАБАТЫВАЕТ ОТКАЗЫ СТРАНИЦ

Мы на примере TLB знаем, что конструкторы оборудования с неохотой доверяют ОС делать многие вещи. Так почему же обработку страничного отказа все-таки доверили? Тому есть несколько причин. Во-первых, подкачка страницы с диска происходит *медленно*; даже если ОС для обработки отказа потребуется выполнить тысячи команд, все равно дисковые операции настолько медленны, что эти дополнительные накладные расходы окажутся незаметны. Во-вторых, чтобы обработать отказ страницы, оборудование должно понимать, что такое область подкачки, как производить дисковый ввод-вывод, и уйму других деталей, о которых оно мало что знает. Поэтому ради простоты и производительности обработкой отказов страниц занимается ОС, и даже упертые «железячники» не возражают.

По завершении операции ввода-вывода ОС обновляет таблицу страниц: помечает, что страница присутствует, помещает в поле PFN записи таблицы страниц (PTE) адрес только что подкачанной страницы в памяти, а затем заново выполняет команду. При следующей попытке может случиться не-попадание в TLB, которое будет должным образом обработано с записью трансляции в TLB (альтернатива – обновить TLB в процессе обработки отказа страницы, чтобы избежать этого шага). И наконец, при втором повторном выполнении трансляция будет найдена в TLB, так что выборка команды или данных из памяти по найденному физическому адресу окажется успешной.

Отметим, что пока операция ввода-вывода выполняется, процесс пребывает в **блокированном** состоянии. Поэтому во время обслуживания отказа страницы ОС вправе выполнять другие процессы. Поскольку ввод-вывод обходится дорого, такое **перекрывание** ввода-вывода (отказа страниц) в одном процессе и выполнения другого – еще один способ эффективного использования оборудования в системе мультипрограммирования.

21.4. А что, если память заполнена?

В приведенном выше описании предполагалось, что имеется свободная память, в которую можно **подкачать** страницу. Но это вовсе не обязательно, память может быть полностью занята (или близка к этому). Поэтому ОС,

возможно, придется **выгрузить** одну или несколько страниц, чтобы освободить место для новых. Процедура выбора подлежащей выгрузке страницы называется **политикой замещения страниц**.

На изобретение хорошей политики замещения было потрачено много усилий, потому что выбор неподходящей страницы может крайне негативно отразиться на производительности программы. Из-за неправильного решения программа, возможно, будет работать со скоростью диска, а не памяти, т. е. при текущем уровне развития технологий от 10 000 до 100 000 раз медленнее. Поэтому нам надлежит поговорить об этой политике подробнее, и мы обязательно займемся этим – но в следующей главе. А пока достаточно знать, что такая политика существует и основана на описываемых здесь механизмах.

21.5. Поток управления при обработке ОТКАЗА СТРАНИЦЫ

Вооружившись знаниями, мы теперь можем грубо очертить полный поток управления при доступе к памяти. Иными словами, если кто-нибудь спросит вас «Что происходит, когда программа выбирает данные из памяти?», вы будете неплохо представлять себе все варианты развития событий. Детали см. на рис. 21.2 и 21.3; на первом показано, что делает оборудование во время трансляции адресов, а на втором – что делает ОС во время отказа страницы.

На рис. 21.2 обратите внимание, что теперь при непопадании в TLB нужно рассмотреть три важных случая. Во-первых, когда страница **присутствует и действительна** (строки 18–21); в этом случае обработчик непопадания в TLB может просто взять PFN из PTE, повторить команду (на этот раз будет иметь место попадание в кеш) и продолжить, как было многократно описано выше. Во втором случае (строки 22–23) необходимо выполнить обработчик отказа страницы; хотя доступ к странице не вызывает никаких нареканий (страница-то действительная), в физической памяти ее нет. И наконец, производится недопустимый доступ к странице из-за ошибки в программе (строки 13–14). В этом случае все остальные биты в PTE не играют роли; оборудование генерирует системное прерывание, управление передается обработчику прерывания, и операционная система, скорее всего, снимает процесс.

На рис. 21.3 мы видим основные действия, которые должна выполнить ОС для обработки отказа страницы. Сначала ОС ищет физическую страницу, в которую следует поместить страницу, которая вот-вот будет подкачана с диска; если такой страницы нет, то следует запустить алгоритм замещения и подождать, пока он выгрузит какие-то страницы из памяти, освободив тем самым место.

Располагая свободной физической страницей, обработчик отправляет запрос ввода-вывода, чтобы прочитать страницу из области подкачки. Наконец, когда эта медленная операция завершится, ОС обновляет таблицу страниц и повторяет команду. При повторном выполнении будет иметь место

непопадание в TLB, а затем, при следующем повторе, – попадание в TLB, так что оборудование сможет наконец обратиться по требуемому адресу.

```

1 VPN = (VirtualAddress & VPN_MASK) >> SHIFT
2 (Success, TlbEntry) = TLB_Lookup(VPN)
3 if (Success == True)    // попадание в TLB
4     if (CanAccess(TlbEntry.ProtectBits) == True)
5         Offset = VirtualAddress & OFFSET_MASK
6         PhysAddr = (TlbEntry.PFN << SHIFT) | Offset
7         Register = AccessMemory(PhysAddr)
8     else
9         RaiseException(PROTECTION_FAULT)
10    else    // непопадание в TLB
11        PTEAddr = PTBR + (VPN * sizeof(PTE))
12        PTE = AccessMemory(PTEAddr)
13        if (PTE.Valid == False)
14            RaiseException(SEGMENTATION_FAULT)
15        else
16            if (CanAccess(PTE.ProtectBits) == False)
17                RaiseException(PROTECTION_FAULT)
18            else if (PTE.Present == True)
19                // в предположении, что TLB управляется аппаратно
20                TLB_Insert(VPN, PTE.PFN, PTE.ProtectBits)
21                RetryInstruction()
22            else if (PTE.Present == False)
23                RaiseException(PAGE_FAULT)

```

Рис. 21.2 ❖ Поток управления
при обработке отказа страницы (оборудование)

```

1 PFN = FindFreePhysicalPage()
2 if (PFN == -1)    // нет свободных страниц
3     PFN = EvictPage()    // запустить алгоритм замещения
4 DiskRead(PTE.DiskAddr, PFN) // спать (в ожидании завершения ввода-вывода)
5 PTE.present = True    // поднять бит присутствия в таблице страниц
6 PTE.PFN = PFN    // и записать результат трансляции (PFN)
7 RetryInstruction()    // повторить команду

```

Рис. 21.3 ❖ Поток управления при обработке отказа страницы (ОС)

21.6. Когда на самом деле ПРОИСХОДИТ ЗАМЕЩЕНИЕ

До сих пор при описании процедуры замещения мы предполагали, что ОС ждет, когда память заполнится, и только тогда замещает (вытесняет) страницу, чтобы освободить место для другой. Но в действительности все не совсем так, и по разным причинам ОС всегда придерживает немного свободной памяти.

Для этой цели в большинстве операционных систем определены **верхний** (high watermark – HW) и **нижний** (low watermark – LW) **пределы**, помогающие решить, когда начинать вытеснение страниц из памяти. Работает это следующим образом. Когда ОС замечает, что осталось меньше LW свободных страниц, запускается фоновый поток, отвечающий за освобождение памяти. Этот поток вытесняет страницы, пока количество свободных не станет равно HW. Фоновый поток, иногда называемый **демоном выгрузки**, или **демоном страничного обмена**¹, затем засыпает, довольный тем, что освободил память, чтобы ей могли воспользоваться работающие процессы и ОС.

Поскольку за раз производится несколько замещений, открываются новые возможности для оптимизации. Например, во многих системах страницы **кластеризуются**, или **группируются**, и записываются в область подкачки одной порцией, что повышает эффективность работы с диском [LL82]. Ниже, при обсуждении дисков, мы объясним, что такая кластеризация снижает накладные расходы на поиск и вращение и тем самым заметно увеличивает производительность.

Чтобы учесть наличие фонового потока выгрузки, поток управления на рис. 21.3 следует немного подправить; вместо того чтобы выполнять замещение непосредственно, алгоритм проверяет, есть ли свободные страницы. Если нет, он информирует фоновый поток выгрузки о том, что нужно освободить память. Выгрузив несколько страниц, фоновый поток пробуждает исходный, который теперь может подкачать нужную страницу и продолжить работу.

СОВЕТ: РАБОТАЙТЕ В ФОНОВОМ РЕЖИМЕ

Когда нужно выполнить какую-то работу, бывает разумно сделать это в **фоновом режиме**, чтобы повысить эффективность и сгруппировать несколько операций. Операционные системы часто делают разные вещи в фоновом режиме, например буферизуют файлы в памяти, прежде чем окончательно записать данные на диск. У такого подхода много преимуществ: диск используется эффективнее, потому что получает запрос, содержащий сразу несколько операций записи, и может лучше запланировать их выполнение; уменьшается задержка записи, потому что приложение думает, что операция завершилась очень быстро; открывается возможность снизить объем работы, потому что не исключено, что и записывать-то не нужно будет (т. к. файл удален); лучше используется **время простоя**, поскольку фоновую работу можно выполнить, когда система не делает ничего полезного, и тем самым эффективнее задействовать аппаратные ресурсы [G+95].

21.7. РЕЗЮМЕ

В этой короткой главе мы познакомились с идеей доступа к памяти, размер которой больше, чем физически существует в системе. Для этого необходимо еще усложнить структуру таблицы страниц, включив **бит присутствия**,

¹ Слово «демон» – устаревшее название фонового процесса или потока, делающего нечто полезное. Своим происхождением он обязан (в который раз!) системе Multics [CS94].

который сообщает системе, находится страница в памяти или нет. Если нет, то управление получает **обработчик отказа страницы**, который организует перемещение нужной страницы с диска в память, быть может, предварительно вытеснив некоторые страницы, чтобы освободить место в памяти.

Напомним важную (и удивительную) вещь – все эти действия происходят **прозрачно** для процесса. Процессу кажется, что он просто обращается к своей частной непрерывной виртуальной памяти. А за кулисами страницы размещаются в произвольных (необязательно соседних) местах физической памяти, а иногда даже отсутствуют в памяти и должны быть подкачаны с диска. Мы надеемся, что в типичном случае доступ к памяти производится быстро, но иногда для этого приходится прибегать к операциям с диском; то, что кажется всего лишь одной командой, может в худшем случае занять много миллисекунд.

Литература

[CS94] «Take Our Word For It» by F. Corbato, R. Steinberg. www.takeourword.com/TOW146 (стр. 4). Ричард Стейнберг пишет: «Кто-то спросил меня о происхождении слова демон применительно к компьютерам. Основываясь на собственных изысканиях, могу лишь сказать, что впервые его употребили сотрудники проекта Project MAC, работавшие на IBM 7094 в 1963 году». Профессор Корбатто отвечает: «На употребление слова демон нас натолкнула идея демона Максвелла в физике и термодинамике (я по образованию физик). Демон Максвелла – это воображаемый агент, который помогает сортировать молекулы по скорости и неустанно трудится в фоновом режиме. Нам показалось забавным использовать это слово для описания фоновых процессов, которые без устали занимаются грязной работой в системе».

[D97] «Before Memory Was Virtual» by Peter Denning. In the Beginning: Recollections of Software Pioneers, Wiley, November 1997. Блестящее историческое эссе, написанное одним из тех, кто заложил основы виртуальной памяти и рабочих наборов.

[G+95] «Idleness is not sloth» by Richard Golding, Peter Bosch, Carl Staelin, Tim-Sullivan, John Wilkes. USENIX ATC '95, New Orleans, Louisiana. Увлекательное и легкое для чтения обсуждение того, как лучше использовать время простоя в системах. Изобилует хорошими примерами.

[LL82] «Virtual Memory Management in the VAX/VMS Operating System» by Hank Levy, P. Lipman. IEEE Computer, Vol. 15, No. 3, March 1982. Не первое упоминание кластеризации страниц, но ясное и простое объяснение того, как работает механизм. Мы очень часто цитируем эту работу!

Домашнее задание (измерение)

В этом домашнем задании вы познакомитесь с новым инструментом, **vmstat**, и научитесь с его помощью понимать, как работает процессор, память

и устройства ввода-вывода. Прочитайте файл README и изучите код в файле `mem.c`, прежде чем переходить к упражнениям и вопросам.

Вопросы

1. Сначала откройте два разных окна терминала на одной машине, чтобы можно было запускать что-то в обоих окнах. Теперь в одном окне выполните команду `vmstat 1`, которая каждую секунду выдает статистику работы машины. Прочитайте страницу руководства, ассоциированный файл README и другие материалы, необходимые для понимания того, что она выводит. Оставьте это окно открытым, пока будете выполнять другие упражнения.
Теперь запустим программу `mem.c`, но с очень небольшим потреблением памяти. Для этого нужно ввести команду `./mem 1` (она потребляет всего 1 МБ памяти). Как изменилась статистика CPU после запуска `mem`? Имеют ли смысл числа в столбце `user time`? Что изменится, если запустить одновременно несколько экземпляров `mem`?
2. Теперь посмотрим на статистику памяти во время работы `mem`. Нас будут интересовать два столбца: `swpd` (объем использованной виртуальной памяти) и `free` (объем свободной памяти). Выполните команду `./mem 1024` (она выделяет 1024 МБ) и посмотрите, как изменятся эти значения. Затем снимите программу (нажав **Control-c**) и снова посмотрите, как изменятся значения. Что вы заметили? В частности, как изменился столбец `free` после завершения программы? Увеличился ли объем свободной памяти на ожидаемую величину?
3. Теперь рассмотрим столбцы `swap` (`si` и `so`), которые показывают интенсивность выгрузки на диск и подкачки с диска. Конечно, чтобы значения в них стали ненулевыми, нужно запустить `mem`, запросив выделение большего объема памяти. Для начала узнайте, сколько свободной памяти в вашей системе Linux (например, выполнив команду `cat /proc/meminfo`; чтобы узнать о файловой системе `/proc` и об информации, которую можно в ней найти, наберите `man proc`). Одно из первых полей в `/proc/meminfo` – общий объем памяти в системе. Предположим, что имеется около 8 ГБ; если так, начните с выполнения `mem 4000` (около 4 ГБ) и наблюдайте за столбцами `in/out`. Появляются ли в них ненулевые значения? Потом попробуйте 5000, 6000 и т. д. Что происходит со значениями в этих столбцах, когда программа приступает ко второй (и последующим) итерации цикла, – есть ли отличия от первой итерации? Сколько всего данных подкачено и выгружено на второй, третьей и последующих итерациях? Эти значения кажутся вам осмысленными?
4. Выполните те же эксперименты, что и выше, но теперь наблюдайте за другой статистикой (например, потребление процессора и статистика блочного ввода-вывода). Как изменяются цифры во время работы `mem`?
5. Теперь обратимся к производительности. Задайте такой параметр `mem`, чтобы памяти хватало с большим запасом (скажем, 4000 в системе с 8 ГБ памяти). Сколько времени занимает итерация цикла 0 (и последующие

итерации 1, 2 и т. д.)? Теперь выберите параметр, так чтобы памяти сильно не хватало (скажем, 12000 в той же системе с 8 ГБ памяти). Сколько теперь времени занимают итерации цикла? Объясните результаты сравнения пропускной способности (bandwidth в выдаче программы). Насколько сильно отличается производительность в случае постоянной выгрузки-подкачки по сравнению со случаем, когда памяти хватает? Постройте график, отложив размер памяти по оси x , а пропускную способность по оси y . И последнее – сравните производительность на первой и последующих итерациях цикла для случая, когда все помещается в памяти, и для случая, когда памяти не хватает.

6. Область подкачки не безгранична. Программа `swapon` с флагом `-s` позволяет узнать, сколько места доступно в области подкачки. Что будет, если запускать `mem` со все большими значениями параметра и наконец превысить размер области подкачки? В какой момент выделение памяти завершается неудачно?
7. Наконец, продвинутый пользователь с помощью программ `swapon` и `swaponoff` может сконфигурировать систему, так чтобы для подкачки использовались разные устройства. Детали можете посмотреть на страницах руководства. Если у вас есть доступ к разному оборудованию, посмотрите, как изменяется производительность подкачки при использовании классического жесткого диска, SSD-диска и даже RAID-массива. Насколько можно улучшить производительность подкачки, воспользовавшись новыми технологиями? Насколько близко вы сможете подобраться к производительности оперативной памяти?

Глава 22

За пределами физической памяти: политики

Для диспетчера виртуальной памяти все просто, когда свободной памяти полно. Происходит отказ страницы – диспетчер находит свободную страницу в списке свободных и назначает ее отказавшей странице. Да здравствует Операционная Система, ты снова справились с этим!

К сожалению, все становится не так радужно, когда свободной памяти мало. В таком случае **дефицит памяти** заставляет ОС **выгружать** страницы, чтобы освободить место для активно используемых страниц. Решение о том, какую страницу (или страницы) **вытеснить**, инкапсулировано в **политике замещения**; исторически это было одно из самых важных решений, принимаемых ОС в ранних системах виртуальной памяти, потому что тогда физической памяти было мало. Это интересный набор политик, с которым стоит познакомиться поближе.

СУЩЕСТВО ПРОБЛЕМЫ: КАК РЕШИТЬ, КАКУЮ СТРАНИЦУ ВЫТЕСНИТЬ

На основе чего ОС решает, какую страницу (или страницы) вытеснить из памяти? Это решение принимается политикой замещения, которая обычно основана на общих принципах (обсуждаемых ниже), но включает также специальные приемы для обработки редких ситуаций.

22.1. УПРАВЛЕНИЕ КЕШЕМ

Прежде чем приступать к политикам, опишем более подробно проблему, которую мы пытаемся решить. Учитывая, что в основной памяти находится некоторое подмножество всех страниц в системе, мы можем с полным правом рассматривать ее как **кеш** для страниц виртуальной памяти. Поэтому наша цель при выборе политики замещения в этом кеше – минимизировать коли-

чество **непопаданий в кеш**, т. е. случаев, когда приходится подкачивать страницу с диска. Можно также поставить целью максимизацию количества **попаданий в кеш**, т. е. случаев, когда нужная страница уже находится в памяти.

Зная количество попаданий и непопаданий в кеш, мы можем вычислить **среднее время доступа к памяти** (average memory access time – AMAT) для конкретной программы (такую метрику архитекторы компьютерных систем вычисляют для аппаратных кешей [HP06]):

$$AMAT = T_M + (P_{Miss} \cdot T_D), \quad (22.1)$$

где T_M – время доступа к памяти, T_D – время доступа к диску, а P_{Miss} – вероятность отсутствия данных в кеше (непопадания). P_{Miss} изменяется от 0.0 до 1.0, но иногда коэффициент непопаданий выражается не в виде вероятности, а в виде процентной доли (например, коэффициент 10 % – то же самое, что $P_{Miss} = 0.10$). Заметим, что за доступ к данным в памяти мы платим всегда, но если данные приходится подкачивать с диска, то несем еще и дополнительные расходы.

Например, представим себе машину с крохотным адресным пространством размером 4 КБ и 256-байтовыми страницами. Тогда виртуальный адрес состоит из двух частей: 4-битовый VPN (старшие биты) и 8-битовое смещение (младшие биты). Поэтому всего процесс может адресовать $2^4 = 16$ виртуальных страниц. Предположим, что процесс генерирует виртуальные адреса 0x000, 0x100, 0x200, 0x300, 0x400, 0x500, 0x600, 0x700, 0x800, 0x900, соответствующие первому байту одной из первых десяти страниц адресного пространства (номер страницы – первая шестнадцатеричная цифра виртуального адреса).

Предположим далее, что все страницы, кроме виртуальной страницы 3, уже находятся в памяти. Тогда наша последовательность обращений к памяти столкнется со следующим поведением: попадание, попадание, попадание, непопадание, попадание, попадание, попадание, попадание, попадание, попадание. **Коэффициент попаданий** (процентная доля адресов, найденных в памяти) равен 90 %, потому что 9 из 10 страниц присутствуют в памяти. Следовательно, **коэффициент непопаданий** равен 10 % ($P_{Miss} = 0.1$). В общем случае $P_{Hit} + P_{Miss} = 1.0$; сумма обоих коэффициентов равна 100 %.

Для вычисления AMAT необходимо знать стоимость доступа к памяти и к диску. В предположении, что время доступа к памяти (T_M) составляет приблизительно 100 наносекунд, а время доступа к диску (T_D) приблизительно 10 миллисекунд, AMAT будет равно $100 \text{ нс} + 0.1 \cdot 10 \text{ мс} = 1.0001 \text{ мс}$, т. е. приблизительно 1 миллисекунда. Если бы коэффициент попаданий был равен 99.9 % ($P_{Miss} = 0.001$), то результат получился бы совсем другим: $AMAT = 10.1 \text{ микросекунд}$, т. е. примерно в 100 раз быстрее. Когда коэффициент попаданий стремится к 100 %, AMAT стремится к 100 нс.

Как показывает этот пример, стоимость доступа к диску в современных системах настолько высока, что даже при небольшом коэффициенте непопаданий она будет преобладать в величине AMAT. Очевидно, что нужно всеми силами избегать непопадания в кеш, иначе программа будет работать медленно – со скоростью диска. Один из способов добиться этой цели – тщательно проектировать политику замещения, чем мы сейчас и займемся.

22.2. ОПТИМАЛЬНАЯ ПОЛИТИКА ЗАМЕЩЕНИЯ

Для лучшего понимания принципов работы конкретной политики замещения полезно было бы сравнить ее с лучшей из возможных политик. Такая **оптимальная** политика была разработана Беладии много лет назад [B66] (он назвал ее MIN). При использовании оптимальной политики общее число непападаний будет наименьшим. Беладии показал, что оптимальным является простой (но, к сожалению, трудный для реализации) подход, когда замещается страница, которая *не потребуется в будущем дольше всего*.

СОВЕТ: СРАВНЕНИЕ С ОПТИМУМом ПОЛЕЗНО

Хотя оптимальная политика не очень полезна на практике, она невероятно полезна в качестве эталона для сравнения при моделировании или иных исследованиях. Утверждение о том, что ваш новый блистательный алгоритм дает коэффициент попаданий 80 %, само по себе бессодержательно, а вот если известно, что оптимальный алгоритм достигает коэффициента 82 % (и, значит, новый подход очень близок к оптимальному), то результат становится куда более осмысленным. Поэтому в любом исследовании знание оптимума позволяет лучше проводить сравнение – мы понимаем, к чему стремиться и когда можно прекратить попытки улучшения политики, т. к. она и без того уже близка к идеальной [AD03].

Надеемся, что на интуитивном уровне правильность оптимальной политики не вызывает у вас сомнений. Давайте рассуждать следующим образом: если уж приходится вытеснять какую-то страницу, то почему не ту, необходимость в которой возникнет позже всего? Поступая так, мы по существу говорим, что все остальные страницы в кеше важнее, чем самая отдаленная по времени. И понятно, почему это верно: ко всем остальным страницам мы обратимся раньше.

Разберем простой пример, чтобы понять решения, принимаемые оптимальной политикой. Пусть программа обращается к виртуальным страницам в такой последовательности: 0, 1, 2, 0, 1, 3, 0, 3, 1, 2, 1. На рис. 22.1 показано поведение оптимальной политики в предположении, что в кеш помещается три страницы.

Неудивительно, что первые три обращения заканчиваются непаданием, потому что в самом начале кеш пуст; такие непадания часто называются **непаданиями из-за холодного старта** (или **вынужденными непаданиями**). Далее мы снова обращаемся к страницам 0 и 1, и обе они уже находятся в кеше. Наконец, мы снова сталкиваемся с непаданием (страница 3), но на этот раз кеш заполнен – какую-то страницу необходимо заместить! И какую же? Следуя оптимальной политике, мы рассматриваем будущее каждой находящейся в кеше страницы (0, 1, 2) и видим, что к странице 0 будет обращение почти сразу же, к странице 1 чуть позже, а к странице 2 позже всего. Поэтому у оптимальной политики выбор простой: вытеснить страницу 2, в результате чего в кеше останутся страницы 0, 1 и 3. Следующие три обращения заканчиваются попаданием, а затем мы обращаемся к странице 2, которая давно была вытеснена, – и снова непадание. Оптимальная

политика опять исследует будущее каждой страницы в кеше (0, 1, 3) и видит, что если не вытеснить страницу 1 (к которой программа вот-вот обратится), то все будет хорошо. В примере вытеснена страница 3, но с тем же успехом можно было бы выбрать страницу 0. Наконец, мы обращаемся к странице 1, имеет место попадание, и трассировка заканчивается.

Обращение	Попадание/непопадание	Вытеснить	Новое состояние кеша
0	Непопадание		0
1	Непопадание		0, 1
2	Непопадание		0, 1, 2
0	Попадание		0, 1, 2
1	Попадание		0, 1, 2
3	Непопадание	2	0, 1, 3
0	Попадание		0, 1, 3
3	Попадание		0, 1, 3
1	Попадание		0, 1, 3
2	Непопадание	3	0, 1, 2
1	Попадание		0, 1, 2

Рис. 22.1 ❖ Трассировка оптимальной политики

Отступление: типы непопадания в кеш

Архитекторы компьютеров иногда находят полезным характеризовать непопадания по типу и выделяют три категории: вынужденное, из-за емкости и из-за конфликта. Поскольку по-английски категории называются compulsory, capacity и conflict, эту классификацию иногда называют **Три С** [Н87]. **Вынужденное непопадание** (или **непопадание из-за холодного старта** [ЕF78]) имеет место, потому что кеш в самом начале пуст, а это первое обращение к элементу. **Непопадание из-за емкости** возникает, потому что в кеше закончилось место и необходимо вытеснить какой-то элемент, чтобы поместить новый. И наконец, **непопадание из-за конфликта** возникает в аппаратных кешах из-за ограничений на место размещения элемента, вызванных **секторной ассоциативностью** (set associativity); в программных кешах страниц такое невозможно, потому что они всегда **полностью ассоциативны** и ограничений на место размещения страницы в кеше не существует. Детали см. в работе [НР06].

Можно также вычислить коэффициент попаданий для этого кеша: при 6 попаданиях и 5 непопаданиях он равен $\text{Hits}/(\text{Hits} + \text{Misses}) = 6/(6 + 5) = 54.5\%$. Еще можно вычислить коэффициент попаданий без учета вынужденных непопаданий (когда *первое* непопадание при доступе к каждой странице игнорируется); получится 85.7 %.

К сожалению, как мы видели при разработке политик планирования, будущее в общем случае неизвестно, так что построить оптимальную политику для операционной системы общего назначения невозможно¹. Поэтому при разработке реальной политики мы обратимся к подходам, в которых используются другие способы определения страницы, подлежащей вытеснению.

¹ Если вам удастся, дайте знать! Мы сможем обогатиться вместе. Или, подобно «изобретателям» холодного ядерного синтеза, подвергнуться всеобщему презрению и осмеянию [FP89].

А оптимальная политика послужит нам лишь эталоном для сравнения, чтобы оценить близость к идеалу.

22.3. Простая политика: FIFO

Во многих ранних системах стремились избегать сложности, сопровождающей попытки приблизиться к идеалу, и применяли очень простые политики замещения. Например, использовалась политика **FIFO** (первым пришел, первым ушел): страницы помещаются в конец очереди при поступлении в систему, а когда наступает время замещения, вытесняется таблица, оказавшаяся в начале очереди («пришедшая первой»). Политика FIFO обладает одним важным достоинством: ее легко реализовать.

Посмотрим, как FIFO применяется к последовательности обращений на рис. 22.2. Как и раньше, все начинается с трех вынужденных непопаданий для страниц 0, 1 и 2, а затем имеют место попадания для страниц 0 и 1. После этого обращение к странице 3 заканчивается непопаданием. В случае FIFO решение о вытеснении простое: взять страницу, которая была «первой» (состояние кеша на рисунке представлено в порядке FIFO – первая помещенная в кеш страница находится слева), т. е. страницу 0. Увы, в следующий раз обращение производится как раз к странице 0, что приводит к непопаданию и еще одному замещению (страницы 1). Далее имеет место попадание для страницы 3, непопадания для страниц 1 и 2 и, наконец, попадание для страницы 1.

Обращение	Попадание/непопадание	Вытеснить	Новое состояние кеша
0	Непопадание		0
1	Непопадание		0, 1
2	Непопадание		0, 1, 2
0	Попадание		0, 1, 2
1	Попадание		0, 1, 2
3	Непопадание	0	1, 2, 3
0	Непопадание	1	2, 3, 0
3	Попадание		2, 3, 0
1	Непопадание	2	3, 0, 1
2	Непопадание	3	0, 1, 2
1	Попадание		0, 1, 2

Рис. 22.2 ❖ Трассировка политики FIFO

По сравнению с оптимальной, политика FIFO выглядит заметно хуже: коэффициент попаданий всего 36.4 % (или 57.1 %, если исключить вынужденные непопадания). FIFO просто неспособна оценить важность блоков: хотя к странице 0 было несколько обращений, FIFO все равно вытесняет ее, т. к. она была помещена в кеш первой.

ОТСТУПЛЕНИЕ: АНОМАЛИЯ БЕЛАДИ

Белади (автор оптимальной политики) с сотрудниками обнаружили интересную последовательность обращений – 1, 2, 3, 4, 1, 2, 5, 1, 2, 3, 4, 5 – с неожиданным поведением [BNS69]. Они изучали политику FIFO. Любопытно, как изменяется коэффициент попаданий при увеличении размера кеша с 3 до 4 страниц.

Вообще говоря, естественно ожидать, что коэффициент попаданий в кеш будет *увеличиваться* (улучшаться), когда кеш становится больше. Но в данном случае он уменьшается! Вычислите сами и убедитесь. Это странное поведение получило название **аномалии Белади**.

Другие политики, например LRU, не подвержены этой проблеме. Догадаетесь, почему? Дело в том, что LRU обладает **свойством стека** [M+70]. Для алгоритмов, обладающих этим свойством, кеш размера $N + 1$ естественно включает содержимое кеша размера N . Поэтому при увеличении размера кеша коэффициент попаданий останется неизменным или улучшится. Политика FIFO и случайная политика (и некоторые другие) свойством стека, очевидно, не обладают, поэтому подвержены аномальному поведению.

22.4. ЕЩЕ ОДНА ПРОСТАЯ ПОЛИТИКА: СЛУЧАЙНАЯ

Случайная политика замещения просто вытесняет случайную страницу в случае дефицита памяти. Эта политика аналогична FIFO – ее легко реализовать, но она даже не пытается сколько-нибудь разумно выбирать вытесняемые блоки. На рис. 22.3 показано, как случайная политика применяется к той же последовательности обращений.

Обращение	Попадание/непопадание	Вытеснить	Новое состояние кеша
0	Непопадание		0
1	Непопадание		0, 1
2	Непопадание		0, 1, 2
0	Попадание		0, 1, 2
1	Попадание		0, 1, 2
3	Непопадание	0	1, 2, 3
0	Непопадание	1	2, 3, 0
3	Попадание		2, 3, 0
1	Непопадание	3	2, 0, 1
2	Попадание		2, 0, 1
1	Попадание		2, 0, 1

Рис. 22.3 ❖ Трассировка случайной политики

Разумеется, результат случайной политики целиком и полностью зависит от того, насколько удачно (или неудачно) был произведен выбор. В показанном примере она оказалась чуть лучше FIFO и чуть хуже оптимальной политики. Можно провести тысячи экспериментов и оценить, как обстоит дело

в общем случае. На рис. 22.4 показано, сколько попаданий было в более чем 10 000 независимых случайных испытаний. Как видим, иногда (более чем в 40 % случаев) случайная политика не хуже оптимальной, т. е. дает 6 попаданий при данной последовательности обращений, но иногда она оказывается гораздо хуже – 2 попадания, а то и меньше. Успех случайной политики зависит от везения.

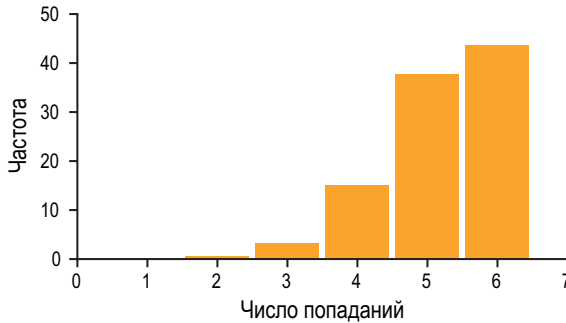


Рис. 22.4 ❖ Результаты случайной политики при более 10 000 испытаний

22.5. УЧЕТ ИСТОРИИ: LRU

К сожалению, любой политике, настолько простой, как FIFO или случайная, свойственна общая проблема: она может вытеснить важную страницу, к которой возможно обращение в ближайшем будущем. FIFO вытесняет страницу, которая была помещена в кеш первой, и если таковой окажется страница, содержащая важный код или данные, она все равно будет замещена, пусть даже вскоре придется загружать ее заново. Поэтому FIFO, случайная политика и им подобные вряд ли будут близки к оптимальной, нужно что-то поумнее.

Как и в случае с политикой планирования, чтобы улучшить гипотезу о будущем поведении, обратимся к прошлому и будем опираться на *историю*. Например, если программа обращалась к странице в недавнем прошлом, она, вероятно, обратится к ней и в ближайшем будущем.

В качестве исторической информации политика замещения страниц могла бы воспользоваться **частотой**; если к странице было много обращений, то, быть может, ее не следует вытеснять, потому что она, очевидно, представляет какую-то ценность. Другое часто используемое свойство страницы – **недавность** обращения; если к странице обращались недавно, то, быть может, обратятся снова, и чем недавнее было обращение, тем выше вероятность повторного.

Это семейство политик основано на **принципе локальности** [D70] – наблюдении за поведением программ. Заключается он в том, что у программ имеется тенденция часто обращаться к определенным последовательностям

команд (например, в цикле) и структурам данных (например, элементам массива, перебираемым в цикле). Поэтому имеет смысл воспользоваться историей, чтобы понять, какие страницы важны, и оставить их в памяти, когда придет время вытеснения.

Так и появилось на свет семейство простых основанных на истории алгоритмов. Политика вытеснения **наименее часто используемой** (Least Frequently Used – **LFU**) замещает страницу, которая использовалась наименее часто. А политика вытеснения **по давности использования** (Least Recently Used – **LRU**) замещает страницу, которая не использовалась дольше других. Алгоритмы легко запоминаются, стоит увидеть название – и сразу понятно, что он делает; очень полезное свойство названия.

Чтобы лучше разобраться в LRU, посмотрим, как эта политика применяется к нашей последовательности обращений (рис. 22.5). Из рисунка видно, как LRU использует историю, чтобы добиться результатов, лучших, чем политики без учета состояния (случайная или FIFO). Когда впервые возникает необходимость заместить страницу, LRU выбирает страницу 2, поскольку страницы 0 и 1 недавно использовались. Затем по той же причине она замещает страницу 0. В обоих случаях принятое LRU решение, основанное на истории, оказывается правильным, поэтому следующие обращения заканчиваются попаданием. Таким образом, в нашем примере LRU не уступает оптимальной политике¹.

Обращение	Попадание/непопадание	Вытеснить	Новое состояние кеша
0	Непопадание		0
1	Непопадание		0, 1
2	Непопадание		0, 1, 2
0	Попадание		1, 2, 0
1	Попадание		2, 0, 1
3	Непопадание	1	0, 1, 3
0	Попадание		1, 3, 0
3	Попадание		1, 0, 3
1	Попадание		0, 3, 1
2	Непопадание	0	3, 1, 2
1	Попадание		3, 2, 1

Рис. 22.5 ❖ Трассировка политики LRU

ОТСТУПЛЕНИЕ: ТИПЫ ЛОКАЛЬНОСТИ

В программах встречается два типа локальности. **Пространственная локальность** означает, что если было обращение к странице P , то с большой вероятностью будет и обращение к расположенной по соседству странице (скажем, $P - 1$ или $P + 1$). **Временная локальность** означает, что если к странице было обращение в недавнем прошлом, то, вероятно, будет и обращение в ближайшем будущем. Предположение о наличии таких типов локальности играет важную роль при проектировании иерархий аппаратных кешей, когда организация многоуровневых кешей команд, данных и трансляций адресов ускоряет работу программы, если локальность действительно имеет место.

¹ Признаем, мы подогнали результаты к ответу. Но иногда небольшое жульничество необходимо для доказательства правоты.

Разумеется, **принцип локальности** не является непреложным правилом, которому должны подчиняться все программы. Есть программы, которые обращаются к памяти (или к диску) совершенно случайным образом, не демонстрируя никакой локальности. Поэтому, хотя локальность стоит иметь в виду при проектировании любых кешей (аппаратных или программных), *гарантии* успеха она не дает. Это всего лишь эвристика, часто оказывающаяся полезной при конструировании вычислительных систем.

Следует также отметить противоположные алгоритмы: вытеснение **наиболее часто используемой** (Most Frequently Used – **MFU**) и **по недавности использования** (Most Recently Used – **MRU**). В большинстве случаев (но не во всех!) эти политики работают не очень хорошо, потому что игнорируют локальность, а не обращают ее себе на пользу.

22.6. ПРИМЕРЫ РАБОЧЕЙ НАГРУЗКИ

Рассмотрим еще несколько примеров, чтобы лучше понять, как ведут себя некоторые из описанных политик. Теперь вместо небольших трасс нас будут интересовать более сложные **рабочие нагрузки**. Но все равно эти нагрузки будут сильно упрощенными, для более качественного анализа нужно было бы брать трассы реальных приложений.

В нашей первой рабочей нагрузке нет вообще никакой локальности: доступ производится к случайно выбранной странице из некоторого набора. В этом простом примере набор содержит 100 страниц, а всего производится 10 000 обращений к страницам. В нашем эксперименте размер кеша варьировался от 1 страницы (совсем мало) до 100 страниц (все вообще), чтобы изучить поведение политик во всем диапазоне размеров кеша.

На рис. 22.6 показаны результаты экспериментов для разных политик: оптимальной, LRU, случайной и FIFO. По оси y отложен коэффициент попаданий для каждой политики, а по оси x – размер кеша.

Из этого рисунка можно сделать несколько выводов. Во-первых, если в рабочей нагрузке нет локальности, то не важно, какой политикой пользоваться; все четыре дают одинаковые результаты, а коэффициент попаданий определяется только размером кеша. Во-вторых, если кеш достаточно велик и вмещает все страницы, то политика не играет никакой роли; все политики, даже случайная, сходятся к 100%-му коэффициенту попаданий, когда все адресуемые страницы находятся в кеше. Наконец, видно, что оптимальная политика заметно лучше реалистичных; если бы можно было заглянуть в будущее, то замещение страниц было бы куда эффективнее.

Далее рассмотрим нагрузку «80-20», в которой локальность присутствует: 80 % обращений производится к 20 % «горячих» страниц, а остальные 20 % обращений – к 80 % «холодных» страниц. Всего у нас будет снова 100 страниц. На рис. 22.7 показано, как функционируют политики при такой рабочей нагрузке.

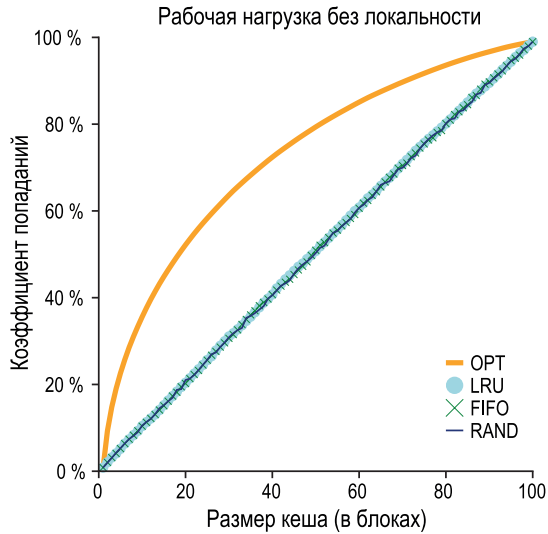


Рис. 22.6 ❖ Рабочая нагрузка без локальности

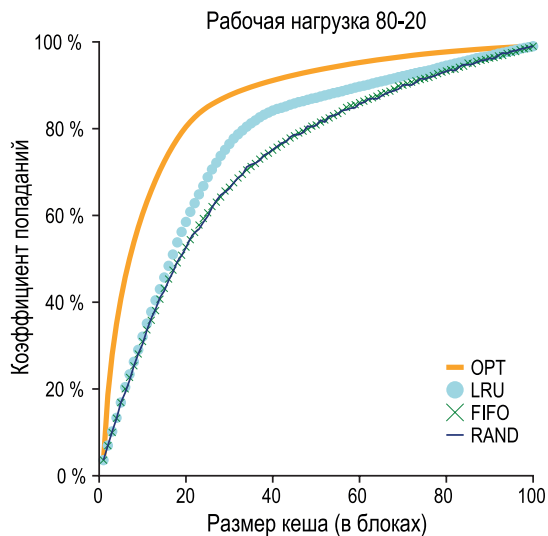


Рис. 22.7 ❖ Рабочая нагрузка 80-20

Видно, что случайная политика и FIFO ведут себя неплохо, но LRU лучше, поскольку она с большей готовностью оставляет горячие страницы в кеше: поскольку к ним часто обращались в прошлом, велика вероятность, что будут обращения и в ближайшем будущем. Оптимальная политика и на этот раз ведет себя лучше всех, и это показывает, что историческая информация, учитываемая LRU, не гарантирует совершенства.

Может возникнуть вопрос: так ли существенно улучшение, которое дает LRU по сравнению со случайной политикой и FIFO? Ответ, как обычно, «зависит от обстоятельств». Если каждое непопадание обходится очень дорого (такое бывает – и довольно часто), то даже небольшое увеличение коэффициента попаданий может сильно сказаться на производительности. Если же непопадания не столь дороги, то, конечно, выигрыш от использования LRU не так важен.

И напоследок рассмотрим еще одну рабочую нагрузку. Назовем ее «последовательно циклической», потому что мы последовательно обращаемся к 50 страницам (0, 1, ..., 49), затем возвращаемся в начало цикла и повторяем всю последовательность. Всего производится 10 000 обращений. На рис. 22.8 показано поведение политик при такой рабочей нагрузке.



Рис. 22.8 ❖ Циклическая рабочая нагрузка

Такой тип рабочей нагрузки, встречающийся во многих приложениях (в т. ч. и таких важных, как базы данных [CD85]), является худшим случаем для политик LRU и FIFO. Эти алгоритмы вытесняют старые страницы, но в силу циклического характера данной рабочей нагрузки к старым страницам последующее обращение будет произведено раньше, чем к тем, которые политики предпочитают оставить в кеше. Действительно, если размер кеша равен 49, то последовательно циклическая рабочая нагрузка с 50 страницами дает коэффициент попаданий 0. Интересно, что случайная политика работает заметно лучше; хотя она и не приближается к оптимальной, но коэффициент попаданий все-таки ненулевой. Вообще, случайная политика обладает некоторыми хорошими свойствами, в частности не имеет аномальных граничных случаев.

22.7. РЕАЛИЗАЦИЯ АЛГОРИТМОВ, УЧИТЫВАЮЩИХ ИСТОРИЮ

Как видим, алгоритм типа LRU в общем случае работает лучше более простых политик типа FIFO или случайной, которые могут вытеснять важные страницы. К сожалению, учитывающие историю политики ставят перед нами новый вопрос: как их реализовать?

Возьмем, к примеру, LRU. Для идеальной реализации нужно проделать немало работы. Именно, при каждом *доступе к странице* (т. е. при любом доступе к памяти, будь то выборка команды, загрузка или сохранение) мы должны обновить некоторую структуру данных, переместив эту страницу в начало списка (туда, где находятся часто используемые страницы). Сравните с FIFO, когда очередь страниц модифицируется только в момент *вытеснения* страницы (удаляется страница в начале очереди) и в момент помещения новой страницы в кеш (страница добавляется в конец очереди). Чтобы следить за тем, какие страницы используются чаще всего и реже всего, система должна выполнять какие-то действия *при каждом обращении к памяти*. Очевидно, что без должной аккуратности такой учет обойдется в копеечку.

Некоторого ускорения поможет добиться толика поддержки со стороны оборудования. Например, машина могла бы при каждом доступе к странице обновлять некоторое поле времени в памяти (это могло бы быть поле в таблице страниц процесса или в каком-то отдельном массиве в памяти, содержащем по одному элементу на каждую физическую страницу). Таким образом, при любом доступе к странице оборудование будет записывать в это поле текущее время. Затем при замещении страницы ОС могла бы просмотреть все такие поля в системе и найти, какая страница не использовалась дольше всех прочих.

К сожалению, поскольку количество страниц в системе растет, просмотр гигантского массива только ради нахождения подлежащей вытеснению страницы оказывается запретительно дорогим. Представьте современную машину с 4 ГБ памяти, разбитой на страницы размером 4 КБ. В такой машине миллион страниц, поэтому поиск LRU-страницы займет много времени даже при современных скоростных процессорах. Что приводит к вопросу: действительно ли нам нужно искать страницу, которая не использовалась строго дольше всех прочих? Нельзя ли обойтись аппроксимацией?

СУЩЕСТВО ПРОБЛЕМЫ: КАК РЕАЛИЗОВАТЬ ПОЛИТИКУ ЗАМЕЩЕНИЯ LRU

Учитывая высокую стоимость точной реализации политики LRU, нельзя ли каким-то способом аппроксимировать ее, сохранив желаемое поведение?

22.8. Аппроксимация LRU

Ответ на этот вопрос положительный: приближенная LRU более практична с точки зрения накладных расходов, именно так и делается во многих современных системах. Для реализации идеи необходима аппаратная поддержка в форме **бита использования** (или **бита обращения**), первый вариант был реализован в первой системе со страничной организацией – системе Atlas с одноуровневым хранением [KE+62]. Для каждой страницы в системе имеется один бит использования, и все эти биты находятся где-то в памяти (в таблицах страниц процессов или в каком-то отдельном массиве). При каждом обращении к странице (чтении или записи) оборудование устанавливает бит использования в 1. Оборудование никогда не сбрасывает бит использования в 0, это задача операционной системы.

Как ОС применяет бит использования для аппроксимации LRU? Способов может быть много, один из них – простой **алгоритм часов** (clock algorithm) [C69]. Предположим, что все страницы в системе организованы в виде циклического списка. В начальный момент **стрелка часов** указывает на какую-то страницу (какую именно, не важно). Когда нужно вытеснить какую-то страницу, ОС проверяет, чему равен бит использования в странице P , на которую указывает стрелка: 1 или 0. Если 1, значит, страница P недавно использовалась и потому не является хорошим кандидатом на замещение. Поэтому ее бит использования сбрасывается в 0 (очищается), а стрелка часов сдвигается к следующей странице ($P + 1$). Алгоритм продолжает работу, пока не найдет страницу с битом использования 0, который означает, что страница давно не использовалась (в худшем случае он обойдет все страницы и очистит все биты).

Заметим, что это не единственный способ применить бит использования для аппроксимации LRU. Подойдет любой алгоритм, который периодически сбрасывает биты использования и выбирает для вытеснения страницу, в которой этот бит равен 1. Алгоритм часов, предложенный Корбатом, был всего лишь одним из первых успешных подходов и обладал полезным свойством: не просматривал раз за разом всю память в поисках давно не использовавшейся страницы.

Поведение одного из вариантов алгоритма часов показано на рис. 22.9. В этом варианте в момент вытеснения производится просмотр случайно выбранных страниц; когда встречается страница с битом использования 1, этот бит очищается, а первая встретившаяся страница с битом использования 0 выбирается в качестве жертвы. Как видим, этот алгоритм ведет себя хуже, чем строгая политика LRU, но лучше, чем политики, в которых история не учитывается вовсе.

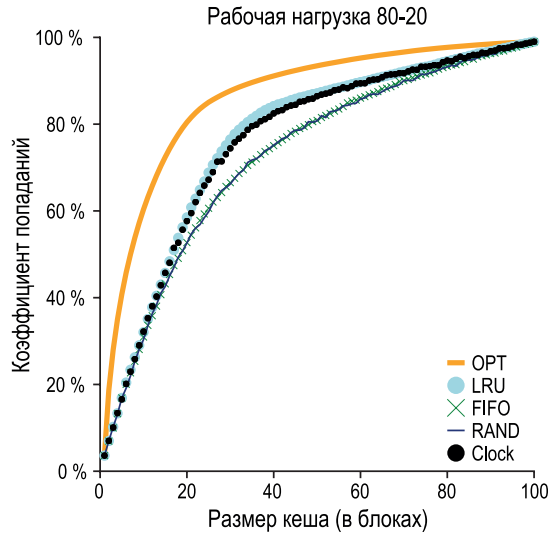


Рис. 22.9 ❖ Рабочая нагрузка 80-20 с часами

22.9. УЧЕТ МОДИФИЦИРОВАННЫХ СТРАНИЦ

Небольшая и часто встречающаяся модификация алгоритма часов (также впервые предложенная Корбатто [С69]) – дополнительно учитывать, была ли страница изменена во время нахождения в памяти. Обоснование простое: если страница была **модифицирована** и, значит, является **грязной**, то при вытеснении ее необходимо записать на диск, что обойдется дорого. Если же страница не была модифицирована (т. е. является **чистой**), то вытеснение ничего не стоит; физическую рамку можно просто повторно использовать для других целей, не неся расходов на ввод-вывод. Поэтому некоторые системы ВП предпочитают вытеснять чистые страницы, а не грязные.

Для поддержки такого поведения оборудование должно также устанавливать **бит изменения**. Он выставляется при каждой записи на страницу, поэтому может быть включен в алгоритм замещения страниц. Например, алгоритм часов можно модифицировать, так чтобы он сначала искал страницы, которые давно не использовались и при этом чистые; если таких нет, то те, что давно не использовались и грязные, и т. д.

22.10. Другие политики ВП

Замещение страниц – не единственная политика, применяемая в подсистеме ВП (хотя, пожалуй, самая важная). Например, ОС также должна решать, *когда* подкачать страницу в память. Эта политика **выбора страницы** (так ее назвал Деннинг [D70]) открывает перед ОС несколько вариантов действий.

Для большинства страниц используется просто **подкачка по запросу**, т. е. ОС загружает страницу в память, когда к ней производится обращение – «запрос». Конечно, ОС могла бы догадаться, что страницу скоро нужно будет использовать, и подкачать ее заранее; такое поведение называется **предвыборкой**, но прибегать к нему следует только тогда, когда догадка с большой вероятностью верна. Например, некоторые системы предполагают, что если страница кода P загружена в память, то весьма вероятно, что вскоре понадобится и страница кода $P + 1$, поэтому загружают и ее тоже.

Еще одна политика определяет, как ОС выгружает страницы на диск. Конечно, можно было бы выгружать их по одной, но во многих системах вместо этого набирается группа страниц, ожидающих записи, и сбрасывается на диск одной операцией (это более эффективно). Такое поведение обычно называют **кластеризацией**, или просто **группировкой** операций записи, его эффективность объясняется устройством дисков, которым быстрее выполнить одну длинную операцию записи, чем несколько коротких.

22.11. Пробуксовка

Перед тем как закругляться, рассмотрим последний вопрос: что должна делать ОС, когда спрос на память со стороны работающих процессов превышает размер доступной физической памяти? В таком случае система будет постоянно выгружать и подкачивать страницы – такую ситуацию иногда называют **пробуксовкой** [D70].

В некоторых ранних операционных системах существовали довольно сложные механизмы обнаружения пробуксовки и борьбы с ней. Например, для данного множества процессов система могла принять решение не запускать некоторое его подмножество в надежде, что **рабочие наборы** оставшихся процессов (страницы, которые они активно используют) поместятся в память и таким образом удастся достичь какого-то прогресса. Идея этого подхода, называемого **контролем допуска**, заключается в том, что иногда лучше делать меньше работы, чем пытаться делать все сразу, но плохо, – такую ситуацию мы встречаем как в реальной жизни, так и в современных вычислительных системах (как это ни грустно).

В некоторых современных системах практикуется драконовский подход к перегрузке памяти. Например, в Linux при обнаружении серьезного дефицита памяти запускается **убийца пожирателей памяти** (out-of-memory killer); этот демон выбирает процесс, который особенно интенсивно потребляет память, и принудительно завершает его, бесцеремонно снижая потребление. Хотя этот подход успешно справляется с дефицитом памяти, у него

есть свои проблемы: например, если он снимает X-сервер, то все приложения, которым требуется дисплей, перестанут работать.

22.12. РЕЗЮМЕ

Мы познакомились с несколькими политиками замещения страниц (и некоторыми другими), являющимися частью подсистемы ВП всех современных операционных систем. В современных системах применяются модификации прямолинейных аппроксимаций LRU, таких как алгоритм часов; например, **противодействие сканированию** – важная часть многих современных алгоритмов, в частности ARC [MM03]. Алгоритмы, противодействующие сканированию, обычно похожи на LRU, но стараются избежать свойственного LRU поведения в худшем случае, которое мы наблюдали при обсуждении последовательно циклической рабочей нагрузки. Таким образом, развитие алгоритмов замещения страниц продолжается.

Но во многих случаях важность вышеупомянутых алгоритмов уменьшилась, потому что увеличился разрыв между временем доступа к памяти и к диску. Поскольку страничный обмен с диском обходится так дорого, плата за частую выгрузку и подкачку становится запретительно высокой. Поэтому лучшим решением в этом случае зачастую является самое простое (и не приносящее интеллектуального удовлетворения): докупить память.

Литература

[AD03] «Run-Time Adaptation in River» by Remzi H. Arpaci-Dusseau. ACM TOCS, 21:1, February 2003. *Реферат диссертации одного из авторов, посвященной системе River, в которой он понял, что сравнение с эталоном – важный метод проектирования систем.*

[B66] «A Study of Replacement Algorithms for Virtual-Storage Computer» by Laszlo A. Belady. IBM Systems Journal 5(2): 78–101, 1966. *В этой статье впервые описан простой способ вычисления оптимальной политики (алгоритм MIN).*

[BNS69] «An Anomaly in Space-time Characteristics of Certain Programs Running in a Paging Machine» by L. A. Belady, R. A. Nelson, G. S. Shedler. Communications of the ACM, 12:6, June 1969. *Описание короткой последовательности обращений к памяти, которая теперь известна как аномалия Белади. Интересно, как к этому названию относятся Нельсон и Шедлер.*

[CD85] «An Evaluation of Buffer Management Strategies for Relational Database Systems» by Hong-Tai Chou, David J. DeWitt. VLDB '85, Stockholm, Sweden, August 1985. *Знаменитая статья из области баз данных, в которой описываются стратегии буферизации, рекомендуемые при нескольких типичных паттернах доступа к базе. Более общий урок: если что-то известно о рабочей нагрузке, то можно спроектировать политики, которые будут работать лучше универсальных, применяемых в ОС.*

[C69] «A Paging Experiment with the Multics System» by F. J. Corbato. Included in a Festschrift published in honor of Prof. P. M. Morse. MIT Press, Cambridge, MA, 1969. *Оригинальное (и его еще надо было поискать!) описание алгоритма часов, хотя и не первое предложение по применению бита использования. Благодарим Х. Балакришнана из MIT, который раскопал для нас эту статью.*

[D70] «Virtual Memory» by Peter J. Denning. Computing Surveys, Vol. 2, No. 3, September 1970. *Ранний и очень известный обзор систем виртуальной памяти, написанный Деннингом.*

[EF78] «Cold-start vs. Warm-start Miss Ratios» by Malcolm C. Easton, Ronald Fagin. Communications of the ACM, 21:10, October 1978. *Хорошее обсуждение непопаданий в кеш при холодном и теплом старте.*

[FP89] «Electrochemically Induced Nuclear Fusion of Deuterium» by Martin Fleischmann, Stanley Pons. Journal of Electroanalytical Chemistry, Volume 26, Number 2, Part 1, April, 1989. *Знаменитая статья, которая должна была совершить революцию, поскольку был предложен простой способ генерировать почти бесконечную энергию, имея кувшин воды, в который добавлено немного металла. К сожалению, результаты, опубликованные (и широко разрекламированные) Понсом и Фейшманом, не удалось воспроизвести, поэтому оба ученых, желавших только добра, были дискредитированы (и, конечно, подвергнуты осмеянию). Единственным, кому этот исход принес удовлетворение, был Марвин Хокинс; его имя было исключено из перечня авторов, хотя он и принимал участие в работе, и таким образом он избежал ассоциаций с одним из величайших научных провалов XX века.*

[HP06] «Computer Architecture: A Quantitative Approach» by John Hennessy and David Patterson. Morgan-Kaufmann, 2006. *Великолепная книга о компьютерной архитектуре. Прочитайте ее!*

[H87] «Aspects of Cache Memory and Instruction Buffer Performance» by Mark D. Hill. Ph. D. Dissertation, U.C. Berkeley, 1987. *В своей диссертации Марк Хилл впервые описал концепцию «трех С», которая стала весьма популярной после описания в книге Хеннесси и Паттерсона [HP06]. Цитата: «Я обнаружил, что полезно разделять непопадания ... на три категории, интуитивно связанные с причиной непопадания (стр. 49)».*

[KE+62] «One-level Storage System» by T. Kilburn, D. B. G. Edwards, M. J. Lanigan, F. H. Sumner. IRE Trans. EC-11:2, 1962. *Хотя в системе Atlas бит использования существовал, количество страниц было настолько мало, что сканирование всех страниц в большой памяти не составляло проблемы, и авторы не стремились ее решить.*

[M+70] «Evaluation Techniques for Storage Hierarchies» by R. L. Mattson, J. Gecsei, D. R. Slutz, I. L. Traiger. IBM Systems Journal, Volume 9:2, 1970. *Статья в основном посвящена эффективному моделированию иерархий кешей; в этом отношении она, безусловно, является классической, но интересно также обсуждение некоторых свойств различных алгоритмов замещения. Догадаетесь ли вы, почему стековое свойство может быть полезно для эмуляции одновременно наличия большого числа кешей разного размера?*

[MM03] «ARC: A Self-Tuning, Low Overhead Replacement Cache» by Nimrod Megiddo and Dharmendra S. Modha. FAST 2003, February 2003, San Jose, California. *Великолепная современная статья об алгоритмах замещения, в которой описывается новая политика, ARC, ныне применяемая в некоторых системах. В 2014 году на конференции FAST '14 была удостоена премии «Test of Time», присуждаемой сообществом систем хранения.*

Домашнее задание (эмуляция)

Эмулятор `raging-policy.py` позволит поэкспериментировать с различными политиками замещения страниц. Детали см. в файле README.

Вопросы

1. Сгенерируйте случайные адреса, запустив программу со следующими аргументами: `-s 0 -n 10`, `-s 1 -n 10`, `-s 2 -n 10`. Вместо политики FIFO выберите LRU или OPT. Вычислите, чем является доступ к каждому из сгенерированных адресов: попаданием или непопаданием.
2. Для кеша размером 5 сгенерируйте худшую последовательность обращений для каждой из следующих политик: FIFO, LRU и MRU («худшая» означает, что количество непопаданий в кеш максимально). Насколько больше должен быть кеш, чтобы для худшей последовательности обращений с применением политики OPT значительно улучшить производительность?
3. Сгенерируйте случайную последовательность адресов (написав программу на Python или perl). Какого результата вы ожидаете от различных политик при такой последовательности обращений?
4. Теперь сгенерируйте последовательность обращений с некоторой локальностью. Как это вообще сделать? Как на ней работает LRU? Насколько LRU лучше случайной политики? Как работает алгоритм часов? А если в этом алгоритме использовать разное число битов часов?
5. Воспользуйтесь программой типа `valgrind`, чтобы оснастить реальное приложение инструментальными средствами и сгенерировать поток обращений к виртуальным страницам. Например, команда `valgrind --tool=lackey --trace-mem=yes ls` выведет почти полную последовательность всех обращений к командам и данным из программы `ls`. Чтобы сделать эту последовательность полезной для эмулятора, нужно сначала преобразовать каждый виртуальный адрес в номер виртуальной страницы (замаскировав смещение и сдвинув старшие биты вправо). Кеш какого размера необходим, чтобы вместить большую часть запросов к памяти? Постройте график рабочего набора при увеличении размера кеша.

Глава 23

Полные примеры систем виртуальной памяти

Прежде чем завершить изучение виртуализации памяти, посмотрим, как все рассмотренные компоненты собираются в полную систему. Мы изучили основные элементы таких систем: разнообразные схемы таблицы страниц, взаимодействие с TLB (иногда даже реализуемое самой ОС) и стратегии выбора страниц, оставляемых в памяти и выгружаемых из нее. Но полная система виртуальной памяти этим не исчерпывается, есть еще многочисленные средства, отвечающие за производительность, функциональность и безопасность.

СУЩЕСТВО ПРОБЛЕМЫ: КАК ПОСТРОИТЬ ПОЛНУЮ СИСТЕМУ ВП

Какие средства необходимы для реализации полной системы виртуальной памяти? Как они позволяют увеличить производительность, повысить безопасность и вообще улучшить систему?

Для этого мы рассмотрим две системы. Первая – один из самых ранних примеров «современного» диспетчера виртуальной памяти в операционной системе **VAX/VMS** [LL82], разработанной в 1970-х и начале 1980-х годов; на удивление большое число различных методов и подходов, примененных в этой системе, дожило до наших дней, поэтому она заслуживает изучения. Некоторые идеи, даже 50-летней давности, до сих пор полезно знать. Эту мысль, хорошо известную ученым, работающим в большинстве других областей (например, в физике), следует особо подчеркнуть для дисциплин, связанных на технологические достижения (например, информатики).

Вторая система – **Linux**, тут причины очевидны. Linux – широко распространенная система, которая эффективно работает как на совсем небольших и маломощных устройствах, в частности смартфонах, так и на большинстве масштабируемых многоядерных систем, установленных в современных центрах обработки данных. Поэтому ее система ВП должна быть достаточно гибкой для работы во всех этих ситуациях. Мы обсудим обе системы, чтобы проиллюстрировать, как изложенные в предыдущих главах концепции соединяются в полном диспетчере памяти.

23.1. Виртуальная память в VAX/VMS

Мини-компьютер VAX-11 был разработан в конце 1970-х годов компанией **Digital Equipment Corporation (DEC)**. В эру мини-компьютеров DEC была одним из главных игроков в индустрии вычислительной техники. К сожалению, серия неудачных решений и наступление ПК медленно (но неуклонно) привели ее к упадку [C03]. Архитектура была реализована в нескольких моделях, включая VAX-11/780 и менее мощный VAX-11/750.

ОС для этой системы называлась VAX/VMS (или просто VMS). Одним из ее главных архитекторов был Дэйв Катлер, который впоследствии возглавил работу над Microsoft Windows NT [C93]. Перед VMS стояла общая проблема: она должна была работать на широком спектре машин, начиная от совсем дешевых VAX'ов и кончая дорогими и мощными машинами, принадлежащими тому же архитектурному семейству. Поэтому в ОС необходимо было встроить механизмы и политики, которые могли работать (и хорошо работали) на всех этих столь разнообразных машинах.

Кроме того, VMS являет прекрасный пример программных инноваций, призванных скрыть некоторые внутренние изъяны архитектуры. Хотя ОС часто полагается на оборудование для реализации эффективных абстракций и иллюзий, иногда конструкторы справляются не в полной мере, и в оборудовании VAX мы увидим несколько таких примеров. Тогда вмешивается операционная система VMS, чтобы система эффективно работала, несмотря на дефекты оборудования.

Оборудование управления памятью

VAX-11 предоставляла каждому процессу 32-разрядное адресное пространство, разбитое на 512-байтовые страницы. Следовательно, виртуальный адрес состоял из 23-битового VPN и 9-битового смещения. Кроме того, два старших бита VPN использовались, чтобы обозначить, какому сегменту принадлежит страница, т. е. в системе использовался гибридный сегментации и страничной организации – мы видели это раньше.

Нижняя половина адресного пространства называлась «пространством процесса» и была уникальна для каждого процесса. В первой половине пространства процесса (известной как P0) находилась пользовательская программа, а также растущая вниз куча. Во второй же его половине (P1) находился стек, растущий вверх. Верхняя половина адресного пространства называлась системным пространством (S), хотя использовалась только его половина. Здесь располагался защищенный код и данные ОС, и таким образом ОС разделялась между процессами.

Одной из основных трудностей, с которыми столкнулись проектировщики VMS, был очень малый размер страницы (512 байт). Из-за этого решения, принятого по историческим причинам, простые линейные таблицы страниц оказывались чрезмерно большими. Поэтому одной из первых целей при проектировании VMS было добиться того, чтобы ОС не забила всю память таблицами страниц.

Чтобы уменьшить потребление памяти таблицами страниц, было применено два метода. Во-первых, благодаря сегментированию адресного пространства пользователя на две части VAX-11 предоставляет каждому процессу по одной таблице страниц для обеих этих областей (P0 и P1); таким образом, в таблице страниц не нужно резервировать место для неиспользуемой части адресного пространства между стеком и кучей. Регистры базы и границы используются как обычно: в регистре базы хранится адрес таблицы страниц для сегмента, а в регистре границы – ее размер (количество записей в таблице страниц).

ОТСТУПЛЕНИЕ: ПРОКЛЯТИЕ ОБЩНОСТИ

Операционные системы часто сталкиваются с **проклятием общности**, поскольку вынуждены предоставлять общую поддержку для широкого класса систем и приложений. А отсюда следует, что ОС, скорее всего, не сможет поддержать каждое конкретное приложение оптимальным образом. В случае VMS эта проблема была более чем реальной, потому что архитектура VAX-11 была реализована на большом числе разных моделей. Не стала она менее реальной и сегодня, когда от Linux ожидают хорошей работы и на телефоне, и на ТВ-приставке, и на ноутбуке, и на настольном компьютере, и на дорогом сервере, исполняющем тысячи процессов в облачном ЦОДе.

Во-вторых, ОС дополнительно снижает потребление памяти, помещая пользовательские таблицы страниц (для сегментов P0 и P1, т. е. две на каждый процесс) в виртуальную память ядра. Таким образом, создавая новую таблицу страниц или увеличивая размер существующей, ядро выделяет место в своей собственной виртуальной памяти, в сегменте S. В случае острого дефицита памяти ядро может выгрузить эти таблицы страниц на диск, освободив физическую память для других нужд.

Размещение таблиц страниц в виртуальной памяти ядра означает, что трансляция адресов становится еще сложнее. Например, чтобы транслировать виртуальный адрес в P0 или P1, оборудование должно сначала найти соответствующую странице запись в таблице страниц сегмента P0 или P1 данного процесса, но при этом оно должно предварительно проконсультироваться с системной таблицей страниц (находящейся в физической памяти). После того как эта трансляция завершится, оборудование будет знать адрес страницы в таблице страниц и только тогда сможет найти желаемый физический адрес. По счастью, эта работа ускоряется аппаратно управляемым TLB, который обычно (хочется надеяться) позволяет обойтись без такого трудоемкого поиска.

Реальное адресное пространство

Изучение VMS хорошо тем, что мы можем посмотреть, как устроено реальное адресное пространство (рис. 23.1). До сих пор предполагалось, что адресное пространство включает только код, данные и кучу, но, как было сказано выше, реально оно устроено гораздо сложнее.

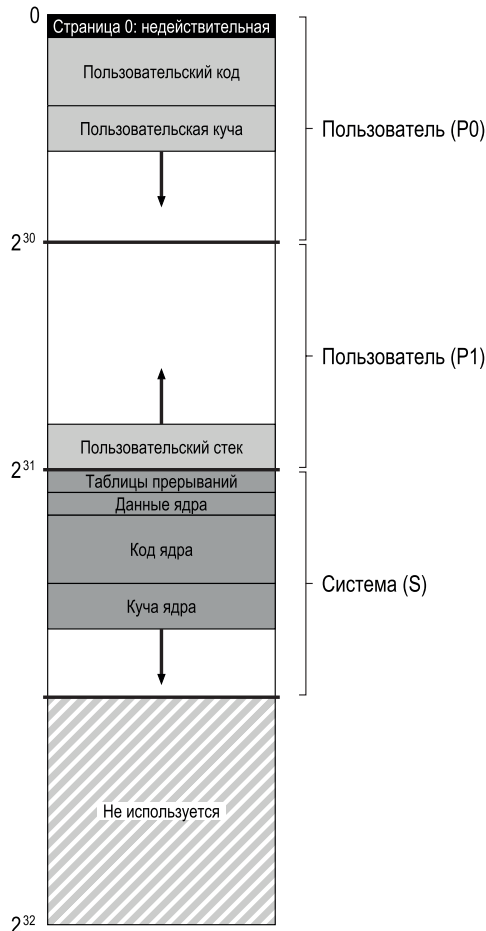


Рис. 23.1 ❖ Адресное пространство VAX/VMS

Например, сегмент кода не начинается на странице 0. Эта страница помечена как недоступная, чтобы можно было обнаружить доступ по **нулевому указателю**. Таким образом, при проектировании адресного пространства следует помнить, в частности, об отладке, одной из форм которой является недоступность нулевой страницы.

Но, пожалуй, важнее то, что виртуальное адресное пространство ядра (его структуры данных и код) является частью каждого адресного пространства пользователя. При контекстном переключении ОС изменяет регистры $P0$ и $P1$, так чтобы они указывали на таблицы страниц готового к выполнению процесса, но не изменяет регистры базы и границы S , поэтому на каждое адресное пространство пользователя отображаются «одни и те же» структуры ядра.

Ядро отображается в каждое адресное пространство по нескольким причинам. Прежде всего так ядру проще работать; например, получив от пользовательской программы указатель (скажем, при выполнении системного вызова `write()`), ОС может легко скопировать адресуемые этим указателем

данные в собственные структуры. ОС естественно пишется и компилируется, т. к. не нужно думать о том, откуда берутся данные, к которым она обращается. Напротив, если бы ядро целиком располагалось в физической памяти, то было бы очень трудно, например, выгружать на диск страницы таблицы страниц; если бы ядру было выделено отдельное адресное пространство, то перемещение данных между ядром и пользовательскими приложениями тоже усложнилось бы. А описанная конструкция (ныне широко используемая) позволяет приложениям рассматривать ядро почти как библиотеку, правда, защищенную.

ОТСТУПЛЕНИЕ:

ПОЧЕМУ ДОСТУП ПО НУЛЕВОМУ УКАЗАТЕЛЮ ВЫЗЫВАЕТ ОШИБКУ СЕГМЕНТАЦИИ

К настоящему моменту вы должны хорошо понимать, что произойдет при попытке разыменования нулевого указателя. Процесс генерирует виртуальный адрес 0, выполняя примерно такой код:

```
int *p = NULL; // установить p = 0
*p = 10;       // попытаться сохранить значение 10 по виртуальному адресу 0
```

Оборудование пытается найти VPN (в данном случае 0) в TLB, но там его не находит. Тогда оно заглядывает в таблицу страниц и обнаруживает, что запись, соответствующая нулевому VPN, помечена как недействительная. Следовательно, мы имеем недопустимый доступ, управление передается ОС, которая, скорее всего, снимет процесс (в системах Unix процессу посылается сигнал, который оставляет им возможность отреагировать на ошибку, но если этот сигнал не перехвачен, то процесс завершается).

Последнее, что мы хотели сказать об этом адресном пространстве, связано с защитой. Очевидно, ОС не хочет, чтобы пользовательские приложения могли читать или изменять данные либо код ОС. Поэтому оборудование должно поддерживать различные уровни защиты. В VAX для этого с помощью битов защиты в таблице страниц задается, на каком уровне привилегий должен работать процессор, чтобы ему было разрешено обратиться к конкретной странице. Таким образом, данные и код системы имеют более высокий уровень защиты, чем данные и код пользователя; любая попытка обратиться к такой информации из пользовательского кода приведет к системному прерыванию, и ОС, вероятно, принудительно завершит процесс-нарушитель.

Замещение страниц

Запись таблицы страниц (PTE) в VAX содержит следующие биты: бит достоверности, поле защиты (4 бита), бит изменения, поле, зарезервированное для нужд ОС (5 бит), и номер физической рамки (PFN), в котором хранится местоположение страницы в физической памяти. Внимательный читатель, вероятно, заметил, что **бита обращения** нет! Поэтому алгоритм замещения в VMS должен обходиться без аппаратной поддержки определения активных страниц.

Разработчики также озаботились усмирением **пожирателей памяти** – программ, которые потребляют очень много памяти и не дают другим программам работать. Большинство рассмотренных выше политик уязвимы перед этой проблемой, например LRU является *глобальной* политикой и не старается распределить память между процессами справедливо.

Для решения этих двух проблем была предложена политика **FIFO с сегментированием** [RL81]. Идея проста: каждому процессу назначается максимальное число страниц, которые он может хранить в памяти; оно называется **размером резидентного набора** (resident set size – RSS). Все такие страницы хранятся в FIFO-очереди; если процесс выходит за пределы своего RSS, то страница, находящаяся в начале очереди, вытесняется. Очевидно, что для политики FIFO поддержка со стороны оборудования не нужна, поэтому ее легко реализовать.

ОТСТУПЛЕНИЕ: ЭМУЛЯЦИЯ БИТОВ ОБРАЩЕНИЯ

Чтобы получить представление о том, какие страницы используются, на самом деле аппаратные биты обращения необязательны. В начале 1980-х годов Бабоглу и Джой показали, что для эмуляции битов обращения в VAX можно использовать биты защиты [BJ81]. Основная идея выглядит так: если мы хотим понимать, какие страницы активно используются, то пометим все страницы таблицы страниц как недоступные (но сохраним информацию о том, какие страницы в действительности доступны процессу, например в поле записи таблицы страниц, «зарезервированном для ОС»). Когда процесс попытается обратиться к странице, будет сгенерировано системное прерывание; ОС проверит, должна ли данная страница быть доступной, и, если так, восстановит нормальную защиту (например, только для чтения или для чтения и записи). В момент замещения ОС может проверить, какие страницы остались недоступными, и таким образом узнать, какие недавно использовались.

Чтобы такая «эмуляция» битов обращения стала возможной, необходимо уменьшить накладные расходы, сохранив в то же время возможность получить приблизительную информацию об использовании страниц. ОС не должна слишком агрессивно помечать страницы как недоступные, иначе накладные расходы будут слишком велики. Но ОС не должна быть и слишком пассивной в этом деле, иначе окажется, что ко всем страницам были обращения, и ОС не будет знать, какую вытеснять.

Разумеется, чистая политика FIFO работает не особенно хорошо – и мы это уже видели. Чтобы повысить ее производительность, в VMS включены **списки второго шанса**, куда страницы помещаются, перед тем как будут вытеснены из памяти, а именно: глобальные списки *чистых страниц* и *грязных страниц*. Когда процесс *P* превышает свою квоту RSS, страницы удаляются из очереди FIFO данного процесса; если она чистая (немодифицированная), то помещается в конец списка чистых страниц, а если грязная (модифицированная), то в конец списка грязных страниц.

Если какому-то другому процессу *Q* понадобится страница, он возьмет первую страницу из глобального списка чистых страниц. Но если в исходном процессе *P* происходит отказ при доступе к этой странице, *до того* как она будет передана другому процессу, то *P* заберет ее из списка чистых (или

грязных) страниц и таким образом избежит дорогостоящего доступа к диску. Чем больше глобальные списки второго шанса, тем ближе алгоритм FIFO с сегментированием к LRU [RL81].

Еще одна оптимизация, предпринятая в VMS, также предназначена для преодоления проблемы малого размера страницы. Именно, при таких маленьких страницах дисковый ввод-вывод во время страничного обмена может оказаться крайне неэффективным, поскольку диски лучше работают с блоками большой длины. Для повышения эффективности ввода-вывода в VMS реализован целый ряд оптимизаций, но самая важная из них – **кластеризация**. Для этой цели VMS объединяет большие пакеты страниц из глобального списка грязных страниц в группу и записывает их на диск одним махом (заодно делая их чистыми). Кластеризация используется в большинстве современных систем, поскольку возможность помещать страницы в любое место области подкачки позволяет ОС группировать страницы, выполнять меньше операций записи большего размера и тем самым повышать производительность.

Другие хитрости

В VMS впервые предложено два ставших стандартными приема: обнуление по запросу и копирование при записи. Опишем эти **ленивые** оптимизации. Одна из форм ленивого выполнения в VMS (и большинстве современных систем) – **обнуление страниц по запросу**. Чтобы разобраться в этом, рассмотрим добавление страницы в ваше адресное пространство, например в кучу. При наивной реализации в ответ на запрос добавить страницу в кучу ОС находит страницу в физической памяти, обнуляет ее (это необходимо из соображений безопасности, иначе вы могли бы увидеть, что было на странице, когда она использовалась другим процессом!) и отображает на ваше адресное пространство (т. е. прописывает эту физическую страницу в таблице страниц). Но наивная реализация может оказаться дорогой, особенно если страница так и не понадобится процессу.

В случае обнуления по запросу ОС проделывает очень мало работы на этапе добавления страницы в адресное пространство – только помещает в таблицу страниц запись, в которой эта страница помечена как недоступная. Если затем процесс попытается прочитать или изменить эту страницу, то произойдет системное прерывание. Во время его обработки ОС замечает (обычно просматривая биты в той части записи таблицы страниц, которая «зарезервирована для ОС»), что эта страница требует обнуления; только тогда ОС находит физическую страницу, обнуляет ее и отображает в адресное пространство процесса. Если процесс так никогда и не обратится к странице, то вся эта работа не будет выполнена, и ОС сэкономит время.

Еще одна весьма полезная оптимизация, включенная в VMS (а вслед за ней и практически во все современные ОС), – **копирование при записи** (сору-on-write, или для краткости **COW**). Идея, восходящая еще к операционной системе TENEX [BB+72], проста: когда ОС нужно скопировать страницу из одного адресного пространства в другое, она может вместо копирования

отобразить страницу в целевое адресное пространство и пометить в обоих адресных пространствах как доступную только для чтения. Если оба процесса будут только читать страницу, то и делать ничего не придется, и, стало быть, ОС осуществила быстрое копирование, вообще не перемещая никаких данных.

Если же один из процессов все-таки захочет что-то записать на страницу, то произойдет системное прерывание. Его обработчик заметит, что эта копируемая при записи страница, и (лениво) выделит новую страницу, заполнит ее данными и отобразит в адресное пространство процесса, в котором произошла ошибка. Процесс продолжит выполнение, но теперь будет иметь собственную частную копию страницы.

COW полезно по ряду причин. Конечно, любую разделяемую библиотеку можно с помощью копирования при записи отобразить в адресные пространства многих процессов и тем сэкономить ценную память. В Unix-системах COW еще важнее из-за семантики системных вызовов `fork()` и `exec()`. Напомним, что `fork()` создает точную копию адресного пространства вызывающего процесса, а если оно велико, то копирование будет выполняться медленно и затрагивать много данных. Хуже того, большая часть адресного пространства немедленно перезаписывается следующим вызовом `exec()`, который замещает адресное пространство вызывающего процесса адресным пространством программы, которая вот-вот начнет выполняться. Если вместо этого `fork()` выполняет копирование при записи, то ОС пропускает большую часть ненужного копирования, сохраняя при этом правильную семантику, но с существенным повышением производительности.

Совет: быть ленивым хорошо

Лень может быть достоинством как в жизни, так и в операционных системах. Ленивый откладывает работу на потом, а в ОС это бывает полезно по целому ряду причин. Во-первых, откладывание работы может уменьшить задержку текущей операции и тем самым улучшить отзывчивость системы; например, операционные системы зачастую сразу же сообщают, что запись в файл завершилась успешно, а реально записывают данные на диск позже в фоновом режиме. Во-вторых, и это важнее, ленивому иногда удастся вообще «отбояриться» от работы; например, если отложить запись до момента, когда файл будет удален, то и записывать ничего не придется. Быть ленивым хорошо и в жизни: например, отложив на время свой проект по ОС, вы можете потом обнаружить, что ошибки в спецификации проекта уже исправлены однокурсниками; впрочем, общий курсовой проект вряд ли отменят, поэтому лень может стать и причиной проблем: не сданный вовремя проект, плохая оценка и опечаленный профессор. Не нужно печалиться профессоров!

23.2. СИСТЕМА ВИРТУАЛЬНОЙ ПАМЯТИ В LINUX

Теперь обсудим некоторые наиболее интересные аспекты системы ВП в Linux. Разработка Linux всегда велась инженерами, стремившимися решить реальные проблемы эксплуатации, а потому многие функции потихоньку

включались в то, что ныне является полнофункциональной, изобилующей разнообразными возможностями системой виртуальной памяти.

Мы не сможем обсудить *все* аспекты ВП Linux, но затронем наиболее важные, особенно те, которые выходят за рамки классических систем типа VAX/VMS. Мы также попытаемся подчеркнуть то общее, что Linux разделяет с прежними системами. В этом обсуждении мы ограничимся Linux на платформе Intel x86. Хотя Linux может работать и работает на процессорах с самой разной архитектурой, больше всего экземпляров Linux развернуто именно на x86, так что наше внимание к этой платформе оправдано.

Адресное пространство Linux

Как и в других современных операционных системах и в VAX/VMS, адресное пространство в Linux¹ состоит из пользовательской части (в ней располагаются код программы, стек, куча и другие разделы) и части ядра (код ядра, стеки, куча и другие вещи). Как и в других системах, при контекстном переключении изменяется только пользовательская часть адресного пространства текущего процесса, а ядерная остается без изменения. Как и в других системах, программа, работающая в режиме пользователя, не может обращаться к виртуальным страницам ядра; доступ к этой памяти возможен только в результате системного прерывания и после перехода в привилегированный режим.

В классической 32-разрядной Linux (т. е. Linux с 32-разрядным виртуальным адресным пространством) граница между пользовательской и ядерной частями адресного пространства проходит по адресу 0xC0000000, т. е. отделяет три четверти пространства. Следовательно, виртуальные адреса от 0 до 0xBFFFFFFF принадлежат пользователю, а остальные (от 0xC0000000 до 0xFFFFFFFF) – ядру. В 64-разрядной Linux граница аналогична, но проходит в другом месте. На рис. 23.2 показано типичное (упрощенное) адресное пространство.

Интересно, что в Linux есть два типа виртуальных адресов ядра. Во-первых, это **логические адреса ядра** [O16]. Это то, что мы назвали бы нормальным виртуальным адресным пространством ядра; чтобы выделить память этого типа, код ядра должен вызвать функцию `kmalloc`. Здесь размещается большая часть структур данных ядра, в т. ч. таблицы страниц, стеки ядра каждого процесса и т. д. В отличие от остальных частей памяти в системе, логическая память ядра *не может быть* выгружена на диск.

Самая интересная черта логических адресов ядра – их связь с физической памятью. Именно, существует прямое отображение между логическими адресами ядра и первой частью физической памяти. То есть логический адрес 0xC0000000 транслируется в физический адрес 0x00000000, 0xC0000FFF в 0x00000FFF и т. д. Из этого факта вытекает два следствия. Первое – то, что трансляция между логическими адресами ядра и физическими адресами

¹ До недавно внесенных изменений, связанных с угрозами безопасности. О деталях этого изменения читайте ниже в разделах, посвященных безопасности в Linux.

осуществляется очень просто, поэтому эти адреса часто трактуются так, будто и в самом деле являются физическими. Второе – что непрерывный участок памяти в логическом адресном пространстве ядра является непрерывным и в физической памяти. Поэтому память, выделенная в этой части адресного пространства ядра, пригодна для операций, которым необходима непрерывная физическая память, например для ввода-вывода посредством **прямого доступа к памяти (ПДП, англ. DMA)** (об этом мы еще поговорим в третьей части книги).

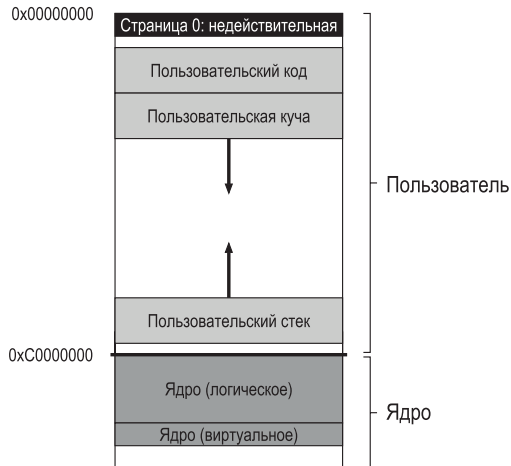


Рис. 23.2 ❖ Адресное пространство в Linux

Другой тип адресов называется **виртуальными адресами ядра**. Чтобы получить память этого типа, ядро вызывает функцию `vmalloc`, которая возвращает указатель на виртуально непрерывный участок памяти нужного размера. В отличие от логической памяти ядра, виртуальная память ядра обычно не является непрерывной; любая виртуальная страница ядра может отображаться на несоседние физические страницы (поэтому для ПДП такая память непригодна). Однако благодаря этому такую память легко выделить, и потому она используется для больших буферов, для которых трудно найти непрерывный участок физической памяти.

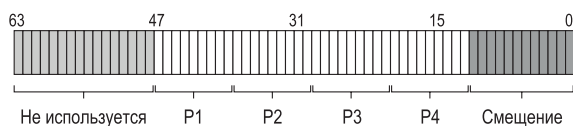
В 32-разрядной Linux есть еще одна причина существования виртуальных адресов ядра – они позволяют ядру адресовать более 1 Гб памяти (приблизительно). Много лет назад машины располагали куда меньшей памятью, поэтому невозможность доступа к памяти за пределами 1 Гб вообще не считалась проблемой. Но прогресс технологий не останавливается, и вскоре возникла необходимость предоставить ядру доступ к памяти большего размера. Благодаря виртуальным адресам ядра и их независимости от взаимно однозначного отображения на физическую память это стало возможно. Однако с переходом на 64-разрядную Linux эта потребность стала не такой настоятельной, потому что ядро не ограничено последним гигабайтом виртуального адресного пространства.

Структура таблицы страниц

Коль скоро мы говорим о Linux для x86, то и обсуждать будем структуру таблицы страниц, предлагаемую процессором x86, поскольку она определяет, что Linux может, а чего не может. Как уже было сказано, x86 предлагает аппаратно управляемую многоуровневую структуру таблицы страниц, по одной таблице на каждый процесс; ОС просто организует отображения в своей памяти, записывает в привилегированный регистр адрес начала каталога страниц, а оборудование делает все остальное. Как и следовало ожидать, ОС привлекается в момент создания и удаления процесса и при контекстном переключении, ее задача – гарантировать, что устройство управления памятью (MMU) использует правильную таблицу страниц для выполнения трансляций.

Пожалуй, самым большим изменением за последние годы был переход от 32-разрядного x86 к 64-разрядному. Как следует из обсуждения системы VAX/VMS, 32-разрядные адресные пространства существовали долгое время, но по мере развития технологий наконец стали реальным ограничением для программ. Виртуальная память упрощает программирование систем, но в современных системах, имеющих много гигабайтов памяти, 32 разрядов просто не хватает для ее адресации. Поэтому пришло время для следующего скачка.

Переход к 64-разрядному адресному пространству, естественно, оказал влияние и на структуру таблицы страниц. Поскольку x86 использует многоуровневую таблицу страниц, в современных 64-разрядных системах таких уровней четыре. Но пока 64-разрядное адресное пространство используется не полностью, задействованы только младшие 48 бит. Таким образом, виртуальный адрес имеет следующую структуру:



Как видно на рисунке, старшие 16 бит виртуального адреса не используются (а потому не играют никакой роли в трансляции), младшие 12 бит (напомним, что размер страницы равен 4 КБ) интерпретируются как смещение (а значит, используются непосредственно и трансляции не подвергаются), так что в трансляции принимают участие только средние 36 бит. Часть P1 служит индексом в каталог страниц верхнего уровня, оттуда и начинается трансляция, переходя от уровня к уровню, пока дело не дойдет до самой страницы таблицы страниц; последняя индексируется частью P4, в результате чего получается искомая запись.

По мере того как память системы будет и дальше расти, в действие вступят и другие части этого обширного адресного пространства, что приведет к пятиуровневому и в конечном итоге шестиуровневому дереву таблицы страниц. Только представьте себе: для простого поиска в таблице страниц

понадобится шесть уровней трансляции – и все только для того, чтобы найти, где в памяти располагаются нужные данные.

Поддержка больших страниц

Intel x86 допускает использование страниц нескольких размеров, а не только стандартных 4-килобайтовых. Именно, последние модели аппаратно поддерживают страницы размера 2 МБ и даже 1 ГБ. Таким образом, в ходе эволюции Linux стала разрешать приложениям использовать такие **гигантские страницы** (huge pages – так они называются в мире Linux).

Использование гигантских страниц, как уже было сказано выше, дает многочисленные преимущества. При изучении VAX/VMS мы видели, что это уменьшает количество отображений в таблице страниц: чем больше страницы, тем меньше отображений. Но не стремление уменьшить число записей в таблицах страниц стало стимулом для внедрения гигантских страниц, а желание повысить эффективность TLB и, как следствие, производительность.

Если процесс активно использует много памяти, то TLB быстро заполняется трансляциями. Если транслируются страницы размером 4 КБ, то обратиться, не опасаясь непопадания в TLB, можно будет лишь к небольшой части всей памяти. И выходит, что современные рабочие нагрузки, требующие «большой памяти» и работающие на машинах, оснащенных многими гигабайтами памяти, испытывают заметную потерю производительности. Недавние исследования показали, что в некоторых приложениях 10 % процессорного времени тратится на обслуживание непопаданий в TLB [B+13].

Гигантские страницы позволяют процессу обращаться к большому участку памяти без непопаданий в TLB, поскольку в TLB занято меньше позиций. Именно это и является их главным преимуществом. Но есть и другие: в случае непопадания в TLB путь к нужной странице короче, а значит, непопадание обслуживается быстрее. Кроме того, выделить память можно очень быстро (в некоторых ситуациях) – небольшое, но иногда важное преимущество.

Интересный аспект поддержки гигантских страниц в Linux – то, как постепенно она внедрялась. Сначала разработчики Linux считали, что такая поддержка важна только для небольшого числа приложений, например больших баз данных с жесткими требованиями к производительности. Поэтому было принято решение разрешить приложениям явно запрашивать выделение памяти большими страницами (с помощью системных вызовов `mmap()` или `shmget()`). При этом большинству приложений это было безразлично – они продолжали использовать 4-килобайтовые страницы, а для тех нескольких особо требовательных приложений, которые необходимо было изменить, чтобы получить доступ к новым интерфейсам, игра стоила свеч.

Впоследствии, осознав, что повышение эффективности TLB – потребность, присущая многим приложениям, разработчики Linux добавили **прозрачную** поддержку гигантских страниц. Теперь операционная система автоматически ищет возможность выделять память гигантскими страницами (обычно 2 МБ, но в некоторых системах 1 ГБ), не требуя модификации приложения.

Совет: постепенность – это хорошо

В жизни нас часто подталкивают к революционности. «Мечтай по-крупному!» – говорят они. «Измени мир!» – призывают нас. И мы понимаем, почему это находит отклик; бывает, что большие перемены необходимы, поэтому подталкивание к ним вполне оправдано. И если мы попробуем пойти по этому пути, то, по крайней мере, на нас перестанут орать – возможно.

Но во многих случаях уместнее более медленное и постепенное движение к цели. Гигантские страницы в Linux – пример инженерной постепенности; вместо того чтобы занять ортодоксальную позицию и настаивать на том, что только большими шагами можно прийти в будущее, разработчики приняли взвешенный подход: сначала добавили специализированную поддержку и, лишь узнав больше о ее плюсах и минусах и убедившись, что основания действительно имеются, включили общую поддержку для всех приложений.

Постепенность, пусть и презируемая кем-то, часто ведет к медленному, обдуманному и благоразумному прогрессу. При построении систем именно такой подход может оказаться наиболее плодотворным. Собственно, к обычной жизни это тоже относится.

У гигантских страниц есть своя цена. И главный потенциальный недостаток – **внутренняя фрагментация**, когда страница велика, но используется разреженно. Из-за такой формы непроизводительных потерь память может оказаться заполнена большими страницами, которые почти не используются. Выгрузка гигантских страниц, если она разрешена, работает неэффективно и иногда значительно увеличивает общий объем ввода-вывода в системе. Накладные расходы на выделение тоже могут быть велики (в некоторых случаях). В целом понятно одно: 4-килобайтовые страницы, которые верой и правдой служили системе много лет, больше не являются универсальным решением; неуклонный рост доступного объема памяти требует рассмотрения больших страниц и других решений – это необходимая часть эволюции систем ВП. Постепенное взятие на вооружение этих аппаратных технологий в Linux – свидетельство надвигающихся перемен.

Страничный кеш

Чтобы уменьшить стоимость доступа к постоянным запоминающим устройствам (тема третьей части книги), в большинстве систем применяется агрессивное **кеширование** с целью хранить наиболее востребованные данные в памяти. В этом отношении Linux ничем не отличается от традиционных операционных систем.

Страничный кеш в Linux унифицирован, в нем хранятся страницы, поступающие из трех основных источников: **файлы, отображенные в память**, данные и метаданные файлов (доступ к ним обычно производится с помощью вызовов `read()` и `write()`, адресованных файловой системе), а также страницы кучи и стека, являющиеся частью каждого процесса (иногда они называются **анонимной памятью**, потому что за ними стоит не именованный файл, а область подкачки). Эти данные хранятся в **хеш-таблице страничного кеша**, что позволяет быстро находить их при необходимости.

ОТСТУПЛЕНИЕ: ПОВСЕМЕСТНОСТЬ ОТОБРАЖЕНИЯ В ПАМЯТЬ

Отображение в память на несколько лет опережает появление Linux и используется во многих местах как Linux, так и других современных систем. Идея проста: вызвав `mmap()` для уже открытого файлового дескриптора, процесс получает в ответ указатель на начало области виртуальной памяти, в которой, как кажется, находится файл. Затем, просто переименовав этот указатель, процесс может обратиться к любой части файла.

Попытки доступа к тем частям отображенного в память файла, которые еще не были загружены в память, вызывают **отказ страницы**, в этот момент ОС подкачивает нужную страницу и делает ее доступной, обновляя таблицу страниц процесса (**подкачка по запросу**).

Любой обычный процесс в Linux пользуется отображенными в память файлами, хотя `main()` и не вызывает `mmap()` напрямую – просто потому, что именно так Linux загружает код из исполняемого файла и разделяемых библиотек. Ниже приведен (сильно сокращенный) вывод команды `map`, показывающий части, из которых складывается виртуальное адресное пространство работающей программы (в данном случае – оболочки `tcsh`). Мы видим четыре столбца: виртуальный адрес отображенной области, ее размер, биты защиты и источник отображения:

```
0000000000400000    372K r-x-- tcsh
00000000019d5000   1780K rw--- [anon ]
00007f4e7cf06000   1792K r-x-- libc-2.23.so
00007f4e7d2d0000    36K r-x-- libcrypt-2.23.so
00007f4e7d508000   148K r-x-- libtinfo.so.5.9
00007f4e7d731000   152K r-x-- ld-2.23.so
00007f4e7d932000    16K rw--- [stack ]
```

Как видно, коды двоичного файла `tcsh`, а также библиотек `libc`, `libcrypt`, `libtinfo` и самого динамического компоновщика (`ld.so`) отображены в адресное пространство. Также присутствуют две анонимные области: куча (вторая строка сверху, обозначенная `anon`) и стек (обозначена `stack`). Отображение файлов в память дает ОС простой и эффективный способ построить современное адресное пространство.

В страничном кеше страницы помечены как **чистые** (прочитаны, но не изменены) или **грязные (модифицированные)**. Грязные данные периодически записываются на постоянное запоминающее устройство (т. е. в конкретный файл, если данные были прочитаны из файла, или в область подкачки в случае анонимных участков) фоновыми потоками (`pdflush`), и тем самым гарантируется, что модифицированные данные рано или поздно окажутся в постоянном хранилище. Эта фоновая операция происходит по срабатыванию таймера или когда накопилось слишком много грязных страниц (интервал срабатывания и пороговое число страниц – настраиваемые параметры).

В некоторых случаях система обнаруживает, что памяти осталось мало, и тогда Linux должна решить, какие страницы вытеснить из памяти, чтобы освободить место. Для этого используется модифицированный вариант алгоритма вытеснения **2Q** [JS94], который описывается ниже.

Основная идея проста: стандартный алгоритм замещения LRU эффективен, но теряется, сталкиваясь с некоторыми распространенными паттернами доступа. Например, если процесс многократно обращается к большому файлу (особенно приближающемуся к размеру памяти или большему), то

LRU будет вытеснять все остальные файлы из памяти. Хуже того: сохранение частей этого файла в памяти бесполезно, потому что процесс не обратится к ним повторно раньше, чем они будут вытеснены из памяти.

В варианте алгоритма вытеснения 2Q Linux решает эту проблему, поддерживая два списка, так что каждая страница попадает в один из них. При первом обращении страница помещается в один список (в оригинальной статье он назван A1, но в Linux называется **списком неактивных**); при повторном обращении страница перемещается в другой список (в оригинале Aq, а в Linux – **список активных**). Когда нужно произвести вытеснение, страница-кандидат берется из списка неактивных. Кроме того, Linux периодически перемещает страницы, оказавшиеся в конце списка активных, в список неактивных, стремясь к тому, чтобы список активных занимал примерно две трети всего страничного кеша [G04].

В идеале Linux должна была бы хранить эти списки в порядке LRU, но, как было отмечено выше, поддерживать такую дисциплину дорого. Поэтому, как и во многих операционных системах, применяется приближение к LRU, похожее на алгоритм **часов**.

В общем случае алгоритм 2Q ведет себя почти так же, как LRU, но гораздо лучше обрабатывает случай циклического доступа к большому файлу, поскольку страницы, к которым при этом происходят обращения, остаются в списке неактивных. Так как к этим страницам никогда не производится повторный доступ перед вытеснением из памяти, они не вытесняют другие полезные страницы, хранящиеся в списке активных.

Безопасность и переполнение буфера

Пожалуй, самое существенное отличие современных систем БП (Linux, Solaris и различные варианты BSD) и старых (VAX/VMS) – упор на безопасность. Защите всегда уделялось серьезное внимание со стороны операционных систем, но теперь, когда машины взаимосвязаны в гораздо большей степени, чем раньше, неудивительно, что разработчики реализовали разнообразные защитные средства, чтобы помешать злокозненным хакерам получить контроль над системой.

Одна из основных угроз – **атаки с переполнением буфера** [W18], которые можно применить как против обычных пользовательских программ, так и против самого ядра. Идея в том, чтобы найти в системе дефект, который позволит атакующему внедрить произвольные данные в ее адресное пространство. Иногда такие уязвимости возникают, потому что автор программы ошибочно предположил, что входные данные не слишком велики, и доверчиво скопировал их в буфер. Но поскольку на самом деле данные оказались длиннее, чем ожидалось, буфер переполнился, и часть памяти целевой системы оказалась перезаписана. Источником проблемы может выглядеть следующий, с первого взгляда невинный, код:

```
int some_function(char *input) {
    char dest_buffer[100];
    strcpy(dest_buffer, input); // опа, копирование без ограничения!
}
```

Во многих случаях такое переполнение не приводит к катастрофе, например неправильные входные данные, без злого умысла переданные пользовательской программе или даже ОС, скорее всего, приведут к аварийному останову – и только. Однако злонамеренный программист может специально подготовить данные, чтобы, переполнив буфер, внедрить собственный код в атакуемую систему, перехватить управление ей и затем делать все, что ему заблагорассудится. Если атакующему удастся проделать такое с пользовательской программой, к которой он подключился по сети, то он сможет запускать произвольные вычисления и даже сдавать в аренду время работы в скомпрометированной системе. А если будет успешна атака против самой операционной системы, то он сможет получить доступ к дополнительным ресурсам, это форма **эскалации привилегий** (когда пользовательский код получает права ядра). Если вы еще не догадались, намеком – все это очень Плохие Вещи.

Первая и самая простая линия обороны против переполнения буфера – предотвратить выполнение любого кода в некоторых областях адресного пространства (например, в стеке). **Бит NX** (No-eXecute – запрета выполнения), включенный компанией AMD в свой вариант x86 (аналогичный бит XD теперь есть и в процессорах Intel), – одно из таких средств; если в записи таблицы страниц для некоторой страницы поднят этот бит, то выполнение любого кода на ней запрещено. При этом атакующий не сможет выполнить код, внедренный в стек, так что проблема в какой-то степени сглаживается.

Однако изобретательные хакеры ... ну да, изобретательны, и даже когда внедренный код невозможно выполнить явно, специально написанная программа может выполнить произвольные последовательности кода. В своей самой общей форме эта идея известна под названием **возвратно-ориентированное программирование** (return-oriented programming – **ROP**) [S07], в остроумии ей и вправду не откажешь. Основана она на том, что в адресном пространстве любой программы, а особенно программы на С, которая компонуется с очень объемными С-библиотеками, и так имеются самые разные фрагменты кода (в терминологии ROP – **гаджеты**). Поэтому атакующий может перезаписать стек таким образом, чтобы адрес возврата из выполняемой в данный момент функции указывал на нужную ему команду (или последовательность команд), за которой следует команда возврата. Сцепляя много гаджетов (т. е. сделав так, чтобы каждая команда возврата переходила к следующему гаджету), атакующий сможет выполнить произвольный код. Изумительно!

Для защиты от ROP (и, в частности, от его ранней формы, **атаки возвратом в libc** [S+04]) в Linux (и в других системах) добавлена еще одна линия обороны – **рандомизация распределения адресного пространства** (address space layout randomization – **ASLR**). Вместо того чтобы размещать код, стек и кучу в фиксированных местах виртуального адресного пространства, ОС размещает их случайным образом, затрудняя сборку сложной последовательности кода, необходимой для осуществления этого класса атак. Поэтому большинство атак на уязвимые пользовательские программы заканчиваются крахом, но не перехватом управления.

Интересно, что результат такой рандомизации легко наблюдать на практике. Следующий код демонстрирует это в современной системе Linux:

```
#include <stdio.h>
int main(int argc, char *argv[]) {
    int stack = 0;
    printf("%p\n", &stack);
    return 0;
}
```

Этот код просто печатает виртуальный адрес переменной в стеке. В старых системах, где еще не было механизма ASLR, при каждом запуске печаталось бы одно и то же значение. Но теперь, как показано ниже, значение всякий раз другое:

```
prompt> ./random
0x7ffd3e55d2b4
prompt> ./random
0x7ffe1033b8f4
prompt> ./random
0x7ffe45522e94
```

ASLR – настолько полезное средство защиты пользовательских программ, что оно внедрено и в ядро в форме **рандомизации распределения адресного пространства ядра (KASLR)**. Но, как выясняется, у ядра есть проблемы и посерьезнее.

Другие проблемы безопасности: Meltdown и Spectre

Когда мы писали этот текст (август 2018 года), мир компьютерной безопасности всколыхнуло сообщение о двух новых взаимосвязанных атаках. Первая получила название **Meltdown**, вторая – **Spectre**. Они были обнаружены примерно в одно время четырьмя разными группами исследователей и инженеров и заставили глубоко задуматься о фундаментальных мерах защиты, предлагаемых оборудованием и ОС. На сайтах meltdownattack.com и spectreattack.com эти атаки описаны во всех подробностях. Spectre считается большей проблемой.

Общее слабое место, эксплуатируемое обеими атаками, заключается в том, что процессоры, установленные в современных системах, завуалированно применяют разнообразные приемы с целью повышения производительности. Класс приемов, приведших к проблеме, называется **упреждающим выполнением** (speculative execution), смысл его в том, что CPU предполагает, какие команды могут быть выполнены в ближайшем будущем, и начинает выполнять их заранее. Если предположение было правильным, то программа будет работать быстрее, а если нет, то CPU отменит все произведенные изменения в архитектурном состоянии (например, в регистрах) и попробует снова, на этот раз выбрав правильный путь.

Беда в том, что в различных частях системы остаются следы отмененного выполнения, например в кешах процессора, в предсказателях ветвлений и т. д. А отсюда и проблема: как показали авторы атак, это состояние может

сделать уязвимым содержимое памяти, даже той, которая, как нам казалось, защищена средствами MMU.

Один из способов повысить защищенность ядра – убрать большую часть его адресного пространства из каждого пользовательского процесса и вместо этого завести отдельную таблицу страниц ядра для большинства данных ядра (это называется **изоляция таблицы страниц ядра**, или **KPTI**) [G+17]. Таким образом, вместо того чтобы отображать код и структуры данных ядра в каждый процесс, мы оставляем там только самый минимум, поэтому при переходе в ядро теперь необходимо переключаться на таблицу страниц ядра. Это повышает безопасность и позволяет избежать некоторых векторов атак, но ценой производительности. Переключение таблиц страниц обходится дорого. Да, такова цена безопасности: удобство и производительность.

К сожалению, KPTI решает лишь часть описанных выше проблем безопасности. А простые решения, например полное отключение упреждающего выполнения, тоже не имеют смысла, потому что тогда системы стали бы работать в тысячи раз медленнее. Так что вы живете в интересное время, если ваша работа – безопасность систем.

Чтобы до конца разобраться в этих атаках, нужно сначала многому научиться. Начните с архитектуры современных компьютеров, изложенной в учебниках повышенного типа; особое внимание обратите на упреждающее выполнение и все необходимые для его реализации механизмы. Безусловно, нужно будет прочитать об атаках Meltdown и Spectre на вышеупомянутых сайтах; там, кстати, тоже есть полезное введение в упреждающее выполнение, так что, может, отсюда и стоит начать. И изучайте операционную систему на предмет других уязвимостей. Кто знает, какие еще проблемы остались?

23.3. РЕЗЮМЕ

Вы только что прочитали обзор двух систем виртуальной памяти. Надеемся, что детали не вызвали у вас затруднений, потому что вы уже неплохо понимаете базовые механизмы и политики. Дополнительные сведения о VAX/VMS имеются в великолепной (и короткой) статье Леви и Липмана [LL82]. Рекомендуем прочитать ее, поскольку это позволит понять, на каком исходном материале были построены данные главы.

Мы также узнали кое-что о Linux. Хотя это большая и сложная система, многие идеи, которые мы не имели возможности обсудить детально, она унаследовала от своих предшественников. Например, Linux производит ленивое копирование страниц при записи после `fork()`, уменьшая тем самым накладные расходы на ненужное копирование. Linux также обнуляет страницы по запросу (отображая в память устройство `/dev/zero`) и включает фоновый демон выгрузки (`swapon`), который выгружает страницы на диск, борясь с дефицитом памяти. Вообще, ВП изобилует хорошими идеями, рожденными в прошлом, но также включает немало собственных изобретений.

Для дальнейшего изучения рекомендуем хорошие, но, увы, устаревшие книги [BC05, G04]. Лучше прочитать их самостоятельно, потому что мы мо-

жем продемонстрировать лишь каплю из океана сложности. Но откуда-то ведь надо начинать. Что есть океан, как не множество капель? [M04].

Литература

[B+13] «Efficient Virtual Memory for Big Memory Servers» by A. Basu, J. Gandhi, J. Chang, M. D. Hill, M. M. Swift. ISCA '13, June 2013, Tel-Aviv, Israel. *Недавняя работа, в которой показано, что на работу с TLB приходится целых 10 % процессорного времени для рабочих нагрузок, требующих много памяти. Решение: один массивный сегмент для хранения больших наборов данных. Мы возвращаемся назад, чтобы продвинуться вперед!*

[BB+72] «TENEX, A Paged Time Sharing System for the PDP-10» by D. G. Bobrow, J. D. Burchfiel, D. L. Murphy, R. S. Tomlinson. CACM, Volume 15, March 1972. *Ранняя статья по ОС с разделением времени, где впервые высказано много блестящих идей. Одной из них было копирование при записи, но отсюда же черпали вдохновение разработчики и других аспектов современных систем, в т. ч. управления процессами, виртуальной памяти и файловых систем.*

[BJ81] «Converting a Swap-Based System to do Paging in an Architecture Lacking Page-Reference Bits» by O. Babaoglu, W. N. Joy. SOSP '81, Pacific Grove, California, December 1981. *Как воспользоваться существующими механизмами защиты для эмуляции битов обращения. Написано членом группы, работавшей в Беркли над собственным вариантом Unix: **Berkeley Systems Distribution (BSD)**. Эта группа оказала большое влияние на разработку виртуальной памяти, файловых систем и сетевых механизмов.*

[BC05] «Understanding the Linux Kernel» by D. P. Bovet, M. Cesati. O'Reilly Media, November 2005. *Одна из многих книг по Linux, уже устаревшая, но все еще достойная внимания.*

[C03] «The Innovator's Dilemma» by Clayton M. Christenson. Harper Paperbacks, January 2003. *Фантастическая книга об индустрии производства дисковых накопителей и о том, как инновации отправляют в утиль старые идеи. Полезное чтение как для руководителей предприятий, так и для компьютерщиков. Дает пищу для размышлений о том, как большие и успешные компании терпят крах.*

[C93] «Inside Windows NT» by H. Custer, D. Solomon. Microsoft Press, 1993. *Книга о Windows NT, в которой рассматривается вся система снизу доверху с большим числом деталей, чем вы, возможно, хотели бы знать. Но серьезно, очень хорошая книжка.*

[G04] «Understanding the Linux Virtual Memory Manager» by M. Gorman. Prentice Hall, 2004. *Углубленное изложение виртуальной памяти в Linux, но, к сожалению, уже немного устаревшее.*

[G+17] «KASLR is Dead: Long Live KASLR» by D. Gruss, M. Lipp, M. Schwarz, R. Fellner, C. Maurice, S. Mangard. Engineering Secure Software and Systems, 2017. Доступно по адресу <https://gruss.cc/files/kaizer.pdf>. *Богатая информация о KASLR, KPTI, и не только.*

[JS94] «2Q: A Low Overhead High Performance Buffer Management Replacement Algorithm» by T. Johnson, D. Shasha. VLDB '94, Santiago, Chile. *Простой, но эффективный подход к построению алгоритма замещения страниц.*

[LL82] «Virtual Memory Management in the VAX/VMS Operating System» by H. Levy, P. Lipman. IEEE Computer, Volume 15:3, March 1982. *Прочитайте источник большей части материала в этой главе. Особенно важно, если вы собираетесь поступать в магистратуру, где от вас только и требуется, что читать статьи, работать, снова читать статьи, опять работать и наконец написать статью, а затем еще поработать.*

[M04] «Cloud Atlas» by D. Mitchell. Random House, 2004. *Трудно выбрать любимую книгу. Ведь их слишком много! И каждая по-своему уникальна. Но авторам этой книги трудно не выбрать «Cloud Atlas» – фантастическую, растянувшуюся во времени и пространстве сагу о доле людской, из которой взята последняя в этой главе цитата. Если вы умный человек, а мы думаем, что так оно и есть, то кончайте читать эти косноязычные комментарии, а лучше прочтите-ка «Облачный атлас»¹ – вы потом скажете нам спасибо.*

[O16] «Virtual Memory and Linux» by A. Ott. Embedded Linux Conference, April 2016. https://events.static.linuxfound.org/sites/events/files/slides/elc_2016_mem.pdf. *Полезный набор слайдов с обзором ВП в Linux.*

[RL81] «Segmented FIFO Page Replacement» by R. Turner, H. Levy. SIGMETRICS'81, Las Vegas, Nevada, September 1981. *Коротенькая статья, в которой показано, что для некоторых рабочих нагрузок FIFO с сегментированием по производительности может приближаться к LRU.*

[S07] «The Geometry of Innocent Flesh on the Bone: Return-into-libc without Function Calls (on the x86)» by H. Shacham. CCS '07, October 2007. *Обобщение возврата в libc. Д-р Бет Гарнер говорил в «Основном инстинкте»: «Она безумна! Она великолепна!» То же самое мы можем сказать о ROP.*

[S+04] «On the Effectiveness of Address-space Randomization» by H. Shacham, M. Page, B. Pfaff, E. J. Goh, N. Modadugu, D. Boneh. CCS '04, October 2004. *Описание атаки возвратом в libc и ее ограничений. Начните читать, но помните: кроличья нора системной безопасности глубока...*

¹ Дэвид Митчелл. Облачный атлас. Азбука, 2020.

Глава 24

Заключительный диалог о виртуализации памяти

Студент (сглатывая). Уфф, нехилый материал.

Профессор. Да, и?

Студент. И как мне все это запомнить? В смысле, для экзамена?

Профессор. О, надеюсь, это не единственная причина, по которой ты пытаешься это запомнить.

Студент. А для чего же тогда?

Профессор. Я-то думал, ты знаешь. Ты ведь хочешь здесь чему-то научиться, чтобы, выйдя из стен университета, понимать, как в действительности работают системы.

Студент. Гм... нельзя ли примерчик из жизни?

Профессор. Разумеется! Когда я учился в магистратуре, мы с друзьями измеряли, сколько времени занимает доступ к памяти, и иногда цифры оказывались больше ожидаемых. Видишь ли, мы думали, что все данные помещаются в аппаратный кеш второго уровня, поэтому доступ должен быть очень быстрым.

Студент (кивает).

Профессор. И никак не могли понять, что происходит. Ну, и что бы ты стал делать в такой ситуации? Конечно, спросил бы у профессора! Вот мы и спросили у одного из профессоров, а тот только кинул взгляд на наш график и просто сказал: «TLB». Ага! Ну конечно, непопадания в TLB! Почему же мы об этом не подумали? Очень полезно иметь хорошую модель работы виртуальной памяти, тогда можно будет диагностировать разные интересные проблемы с производительностью.

Студент. Кажется, понимаю. Я должен попытаться создать мысленные модели работы того-сего, чтобы, когда начну работать самостоятельно, неожиданное поведение системы не застало меня врасплох. Я даже должен предвидеть, как поведет себя система, просто поразмыслив над этим.

Профессор. Точно. Так чему же ты научился? Какую составил мысленную модель работы виртуальной памяти?

Студент. Думаю, у меня в голове сложилось неплохое представление о том, что происходит, когда процесс обращается к памяти, что, как вы неоднократно говорили, происходит при каждой выборке команды, а также при выполнении явных команд загрузки и сохранения.

Профессор. Неплохо – продолжай.

Студент. Я навсегда запомню, что адрес, который мы видим в пользовательской программе, написанной, к примеру, на С...

Профессор. А разве есть еще какие-то языки?

Студент (продолжает) ...Да, я знаю, что вы любите С. Я тоже! В общем, как я говорил, я теперь знаю, что все наблюдаемые в программе адреса виртуальные; что я как программист вижу только иллюзию того, как данные и код размещены в памяти. Я-то думал, что печатать адрес в указателе – это круто, но вы опустили меня с небес на землю – это всего лишь виртуальный адрес! Я не могу узнать истинные физические адреса, по которым находятся данные.

Профессор. Не можешь, ОС намеренно прячет их от тебя. Что еще?

Студент. Еще я думаю, что TLB и вправду очень важная штука, он предоставляет системе небольшой аппаратный кеш трансляций адресов. Таблицы страниц обычно очень велики, поэтому размещаются в большой и медленной памяти. Не будь TLB, программы работали бы куда медленнее. Похоже, что именно благодаря TLB виртуализация памяти вообще возможна. Я так не могу представить себе систему без него! И содрогаюсь от мысли о программе, рабочий набор которой превосходит емкость TLB: со всеми этими непопаданиями в TLB смотреть, как она работает, было бы тяжело.

Профессор. Да уж, прикройте глаза детям! А кроме TLB, чему ты научился?

Студент. Еще я теперь понимаю, что таблица страниц – одна из тех структур данных, о которых необходимо знать; но это всего лишь структура данных, а значит, можно было бы использовать почти любую. Мы начали с простых структур типа массивов (они же линейные таблицы страниц), а потом добрались до многоуровневых таблиц (похожих на дерево) и даже еще более безумных вещей типа выгружаемых таблиц страниц в виртуальной памяти ядра. И все, чтобы сэкономить немного памяти!

Профессор. Точно.

Студент. И еще одна важная вещь: я понял, что структуры для трансляции адресов должны быть достаточно гибкими, чтобы поддержать все, что программисты хотят делать со своими адресными пространствами. В этом смысле структуры типа многоуровневой таблицы идеальны; они позволяют выделять место только тогда, когда пользователю нужна часть адресного пространства, поэтому мало что пропадает даром. Первые попытки – простая пара регистров базы и границы – были попросту недостаточно гибкими; структуры должны подстраиваться под то, что пользователи ожидают от своих систем виртуальной памяти.

Профессор. *Заманчивая перспектива. Ну а как насчет того, чтобы мы узнали о выгрузке на диск?*

Студент. *Учиться, доложу я вам, интересно, и знать, как работает замещение страниц, тоже полезно. Некоторые базовые политики очевидны (LRU, например), но мне интереснее построение реальной системы виртуальной памяти, такой как в VMS. Только почему-то меня больше интересуют механизмы, а политики – меньше.*

Профессор. *Да что ты?*

Студент. *Ну, вы же сами сказали, в конечном итоге лучшее решение проблем политики простое: купить побольше памяти. Зато механизмы нужно понимать, если хочешь знать, как работает система. И раз уж мы об этом заговорили...*

Профессор. *Так, так...*

Студент. *Что-то мой комп в последнее время тормозит... а память не так уж дорого стоит...*

Профессор. *То-то и оно! На вот, возьми пару монет. Сходи и докупи себе немного DRAM, скряга.*

Студент. *Спасибо, профессор! Я больше никогда не буду выгружать страницы на диск – а если такое случится, то хотя бы буду знать, что происходит!*

Часть II



КОНКУРЕНТНОСТЬ

Глава 25

Диалог о конкурентности

Профессор. Вот мы и добрались до второго из трех столпов операционных систем: **конкурентности**.

Студент. А я думал, что столпов четыре...

Профессор. Нет, так было в прежнем варианте книги.

Студент. Угу... Ну ладно. Так что же такое конкурентность, о чудный профессор?

Профессор. Ну, представь, что у тебя есть персик...

Студент (прерывая). Опять персики! Что вам дались эти персики?

Профессор. А ты не читал Т. С. Элиота? «Песнь о любви» Дж. Альфреда Прурока: «Я смею есть персик?» – и все такое?

Студент. А как же! В школе, на уроках английского. Отличная вещь! Вот, помню, мне особенно нравилось...

Профессор (прерывая). Впрочем, это к делу не относится – просто я люблю персики. Как бы то ни было, представь, что на столе лежит куча персиков, и есть множество людей, желающих их съесть. Допустим, что каждый едок сначала намечает себе персик глазами, а затем пытается схватить его и слопать. Что не так в этом подходе?

Студент. Гм... ну, наверное, я могу наметить себе персик, который наметил еще кто-то. Если этот кто-то доберется до него первым, то я останусь без персика!

Профессор. Точно! И что с этим делать?

Студент. Надо придумать способ получше. Может быть, встать в очередь и, когда подойдет мой черед, взять персик и отойти в сторонку?

Профессор. Хорошо! А что в этом подходе не так?

Студент. Это что же, мне самому все делать?

Профессор. Да.

Студент. Ладно, дайте подумать. Если бы все хватало персики сразу, то дело пошло бы быстрее. А при моем способе надо брать персики по одному, это правильно, но медленно. А надо бы сделать так, чтобы было и правильно, и быстро.

Профессор. Ты начинаешь мне нравиться. На самом деле ты только что сказал самую суть того, что нужно знать о конкурентности! Отличная работа.

Студент. Я сказал? Я-то думал, мы говорим о персиках. А Вы опять перешили к компьютерам.

Профессор. Твоя правда. Приношу извинения! Никогда не следует забывать о конкретике. В общем, есть такие программы, которые называются **многопоточными приложениями**; каждый **поток** – это своего рода независимый агент, работающий внутри программы и делающий что-то от ее имени. Но эти потоки обращаются к памяти, и для них каждая область памяти – что-то вроде персика. Если мы не будем координировать доступ к памяти со стороны разных потоков, то программа будет работать не так, как мы ожидаем. Это понятно?

Студент. Более-менее. Но почему мы говорим об этом на курсе по ОС? Разве это не относится больше к прикладному программированию?

Профессор. Хороший вопрос! Причин несколько. Во-первых, ОС должна поддерживать многопоточные приложения с помощью таких примитивов, как **блокировки** и **условные переменные**, о которых мы скоро поговорим. Во-вторых, сама ОС была первой многопоточной программой – она должна обращаться к своей памяти очень аккуратно, иначе могут произойти странные и ужасные вещи. Настоящий кошмар может приключиться.

Студент. Понимаю. Звучит интригующе. Но, как я понимаю, есть кое-какие детали?

Профессор. А как же...

Глава 26

Конкурентность: введение

До сих пор мы говорили только о разработке базовых абстракций операционной системы. Мы видели, как ОС превращает единственный физический процессор в несколько **виртуальных CPU**, создавая иллюзию одновременной работы нескольких программ. Мы также видели, как создается иллюзия большой частной **виртуальной памяти** у каждого процесса; эта абстракция **адресного пространства** позволяет каждой программе вести себя так, будто ей принадлежит вся память, тогда как на самом деле ОС тайком распределяет физическую память (а иногда и место на диске) между адресными пространствами.

Сейчас мы введем новую абстракцию для одного работающего процесса: **поток**. На смену классическому представлению о существовании единственной точки выполнения внутри программы (т. е. единственного счетчика команд, который указывает на очередную выбираемую из памяти и исполняемую команду) приходит идея **многопоточной** программы, в которой точек выполнения несколько (несколько регистров PC, указывающих на разные команды). Есть и другой способ представлять себе потоки – как отдельные процессы – с тем отличием, что они *разделяют* общее адресное пространство и, следовательно, могут обращаться к одним и тем же данным.

Таким образом, состояние потока очень похоже на состояние процесса. У него есть счетчик команд (PC), который указывает, откуда программа выбирает команды. У каждого потока имеется свой набор регистров, используемых при вычислениях. Поэтому, если два потока работают на одном процессоре, то когда приостанавливается выполнение одного (T1) и начинается выполнение другого (T2), должно происходить **контекстное переключение**. Эта процедура очень похожа на контекстное переключение между процессами, поскольку необходимо сохранить состояние регистров T1 и восстановить состояние регистров T2. В случае процессов состояние сохранялось в **блоке управления процессом (PCB)**, теперь же нам нужно несколько **блоков управления потоком** (thread control block – TCB), в которых будет сохраняться состояние отдельных потоков процесса. Но есть одно важное различие между контекстным переключением процессов и потоков: адресное пространство остается тем же самым (т. е. нет необходимости переключать таблицу страниц).

Еще одно существенное различие между процессами и потоками касается стека. В нашей простой модели адресного пространства классического про-

цесса (который теперь можно назвать **однопоточным процессом**) имеется всего один стек, который обычно располагается в конце адресного пространства (рис. 26.1, слева).

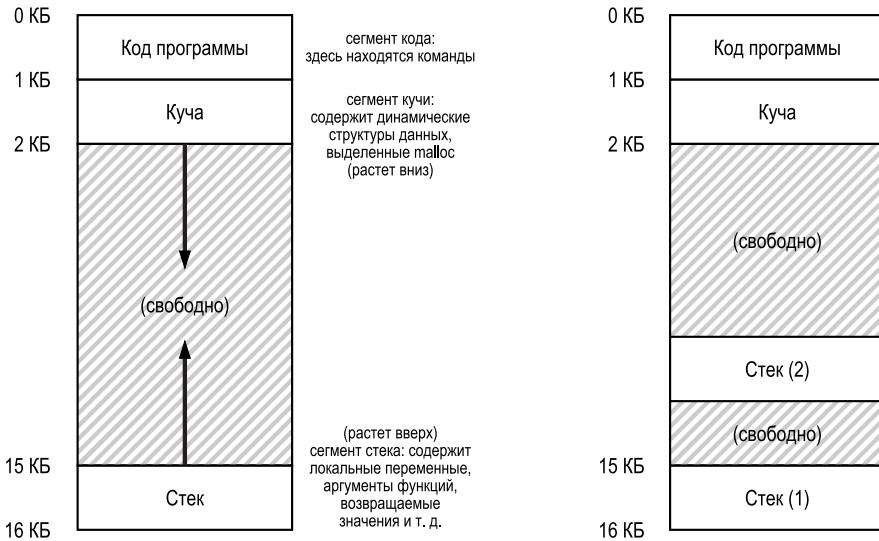


Рис. 26.1 ❖ Адресное пространство однопоточного и многопоточного процесса

Однако в многопоточном процессе каждый поток работает независимо от остальных и, конечно, может вызывать по ходу дела различные функции. Вместо единственного стека в адресном пространстве будет по одному стеку на каждый поток. Предположим, что имеется процесс с двумя потоками, его адресное пространство выглядит по-другому и показано на рис. 26.1 справа.

Мы видим, что в адресном пространстве процесса разместилось два стека. Таким образом, все выделенные в стеке переменные, параметры, возвращаемые значения и вообще всё, что помещается в стек, будут находиться в так называемой **поточно-локальной** памяти, т. е. в стеке соответствующего потока.

Вы, конечно, обратили внимание, как это разрушает нашу красивую схему размещения адресного пространства. Раньше стек и куча могли расти независимо, а проблема возникала, только когда в адресном пространстве заканчивалось место. Теперь же все не так приятно. По счастью, обычно в этом нет ничего страшного, потому что стеки редко бывают очень большими (исключение составляют программы, в которых активно используется рекурсия).

26.1. ЗАЧЕМ НУЖНЫ ПОТОКИ?

Прежде чем переходить к детальному обсуждению потоков и проблем, связанных с написанием многопоточных программ, ответим на простой вопрос: зачем вообще нужны потоки?

Есть, по крайней мере, две важные причины. Первая проста: **параллелизм**. Допустим, что мы пишем программу, которая выполняет операции над очень большими массивами, скажем складывает их поэлементно или увеличивает каждый элемент массива на какую-то величину. Если программа работает на одном процессоре, то все очевидно: нужно выполнить операции над всеми элементами – и всё. Но если в системе несколько процессоров, то программу можно было бы значительно ускорить, поручив каждому процессору часть работы. Задача преобразования стандартной **однопоточной** программы в программу, которая выполняет свою работу на нескольких CPU, называется **распараллеливанием**. А самый естественный и типичный способ ускорить работу программ на современном оборудовании – запустить по одному потоку на каждом процессоре.

Вторая причина не столь очевидна – избежать приостановки программы из-за медленного ввода-вывода. Представьте, что вы пишете программу, выполняющую различные виды ввода-вывода: она может ожидать отправки или получения сообщения, завершения явной операции дискового ввода-вывода или даже (неявно) завершения обработки отказа страницы. Но вместо ожидания программа могла бы заняться чем-то еще, например использовать CPU для вычислений или отправить другие запросы ввода-вывода. Потоки – естественный способ избежать затора: пока один поток программы ждет (т. е. блокирован в ожидании завершения ввода-вывода), планировщик может переключиться на другой готовый к выполнению поток и сделать что-то полезное. Многопоточность открывает возможность **перекрывтия** ввода-вывода с другими действиями *внутри* одной программы, тогда как **мультипрограммирование** предназначено для той же цели, но между разными программами. Поэтому многие современные серверные приложения (веб-серверы, системы управления базами данных и т. п.) используют потоки.

Конечно, во всех вышеупомянутых случаях можно было бы использовать несколько *процессов* вместо потоков. Но, поскольку потоки разделяют одно адресное пространство, становится проще совместно использовать общие данные, так что при создании программ такого типа потоки – естественный выбор. Процессы разумнее использовать, когда имеются логически различные задачи, для которых общие структуры данных совсем или почти не нужны.

26.2. ПРИМЕР: СОЗДАНИЕ ПОТОКА

Теперь займемся деталями. Мы хотим написать программу, которая создает два потока, выполняющих независимые действия, в данном случае – печать «А» или «В». Код приведен на рис. 26.2.

Главная программа создает два потока, каждый из которых выполняет функцию `mythread()`, но с разными аргументами (строка А или В). Созданный поток может сразу начать выполнение или же (в зависимости от настройки планировщика) в момент создания может оказаться в состоянии «готов», а не «выполняется». Конечно, в системе с несколькими процессорами потоки

могли бы даже запуститься одновременно, но пока не будем рассматривать эту возможность.

```

1 #include <stdio.h>
2 #include <assert.h>
3 #include <pthread.h>
4
5 void *mythread(void *arg) {
6     printf("%s\n", (char *) arg);
7     return NULL;
8 }
9
10 int
11 main(int argc, char *argv[]) {
12     pthread_t p1, p2;
13     int rc;
14     printf("main: begin\n");
15     rc = pthread_create(&p1, NULL, mythread, "A"); assert(rc == 0);
16     rc = pthread_create(&p2, NULL, mythread, "B"); assert(rc == 0);
17     // join ждет завершения потока
18     rc = pthread_join(p1, NULL); assert(rc == 0);
19     rc = pthread_join(p2, NULL); assert(rc == 0);
20     printf("main: end\n");
21     return 0;
22 }

```

Рис. 26.2 ❖ Простая программа создания потоков (t0.c)

Создав оба потока (назовем их T1 и T2), главная программа вызывает функцию `pthread_join()`, которая ждет завершения конкретного потока. Делает она это дважды, так что T1 и T2 должны будут завершиться, прежде чем главная программа сможет продолжить выполнение. Всего в прогоне участвовало три потока: главный, T1 и T2.

Рассмотрим возможные порядки выполнения этой небольшой программы. На диаграмме выполнения (рис. 26.3) время увеличивается в направлении сверху вниз, а в каждом столбце показано, когда выполняется один из трех потоков.

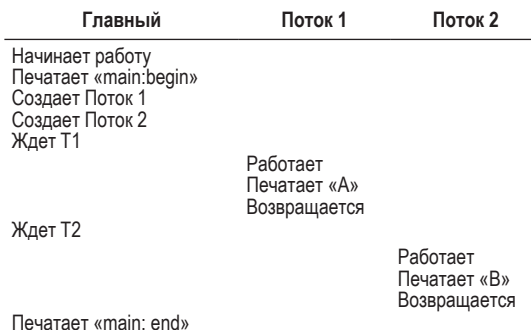


Рис. 26.3 ❖ Трассировка потоков (1)

Заметим, однако, что это не единственно возможный порядок. На самом деле для любой заданной последовательности команд существует много порядков их выполнения, зависящих от того, какой поток выберет планировщик в разные моменты времени. Например, если только что созданный поток начинает выполняться немедленно, то порядок выполнения будет таким, как на рис. 26.4.

Главный	Поток 1	Поток 2
Начинает работу Печатает «main:begin» Создает Поток 1		
	Работает Печатает «А» Возвращается	
Создает Поток 2		
		Работает Печатает «В» Возвращается
Ждет T1 <i>возвращается немедленно: T1 завершен</i> Ждет T2 <i>возвращается немедленно: T2 завершен</i> Печатает «main: end»		

Рис. 26.4 ❖ Трассировка потоков (2)

Можно даже увидеть, что «В» печатается раньше «А», если планировщик решит запустить поток 2 первым, несмотря на то что поток 1 был создан раньше; нет никаких причин полагать, что поток, созданный раньше, раньше и начнет выполняться. На рис. 26.5 показан этот последний порядок выполнения, при котором поток 2 приступает к работе раньше потока 1.

Главный	Поток 1	Поток 2
Начинает работу Печатает «main:begin» Создает Поток 1 Создает Поток 2		
		Работает Печатает «В» Возвращается
Ждет T1		
	Работает Печатает «А» Возвращается	
Ждет T2 <i>возвращается немедленно: T2 завершен</i> Печатает «main: end»		

Рис. 26.5 ❖ Трассировка потоков (3)

Как вы, наверное, поняли, создание потока можно представлять себе как что-то вроде вызова функции, но только вместо выполнения функции и последующего возврата управления система создает отдельный поток выпол-

нения вызванной функции и выполняет его независимо от вызывающей стороны – быть может, еще до возврата из `create`, а быть может, значительно позже. Что именно происходит, определяется **планировщиком** ОС, и хотя планировщик реализует вполне разумный алгоритм, предсказать, какой поток он выберет в каждый момент времени, трудно.

Из этого примера видно, что потоки усложняют жизнь: трудно даже сказать наверняка, какой поток когда будет выполняться! Компьютеры с трудом поддаются пониманию и без конкурентности. А уж с конкурентностью дела становятся еще хуже. Гораздо хуже.

26.3. ПОЧЕМУ СТАНОВИТСЯ ХУЖЕ: РАЗДЕЛЯЕМЫЕ ДАННЫЕ

Приведенный выше простой пример показывает, как потоки создаются и что они могут выполняться в разном порядке в зависимости от решений планировщика. А вот чего он не показывает, так это как потоки взаимодействуют при доступе к разделяемым данным.

Возьмем простой пример, когда два потока обновляют глобальную разделяемую переменную. Интересующий нас код показан на рис. 26.6.

Сделаем несколько замечаний об этом коде. Прежде всего, как предлагает Стивенс [SR05], мы обернули функции создания и ожидания потока в макросы, которые реализуют выход из программы в случае ошибки; для такой простой программы мы хотим только знать, что ошибка имела место, но как-то хитро обрабатывать ее не будем, а просто выйдем. Таким образом, `Pthread_create()` вызывает `pthread_create()` и проверяет, что код возврата равен 0, а если это не так, то печатает сообщение и выходит из программы.

Далее, вместо того чтобы заводить две разные функции в качестве тел рабочих потоков, мы используем одну и ту же функцию, но передаем ей аргумент (в данном случае – строку), чтобы она могла напечатать разные буквы в сообщениях.

Наконец, и это самое главное, посмотрим, что же потоки делают: 10 миллионов ($1e7$) раз в цикле прибавляют число к разделяемой переменной `counter`. Следовательно, ожидаемый конечный результат – 20 000 000.

Откомпилируем и запустим программу. Иногда все работает, как мы и ожидаем:

```
prompt> gcc -o main main.c -Wall -pthread
prompt> ./main
main: begin (counter = 0)
A: begin
B: begin
A: done
B: done
main: done with both (counter = 20000000)
```

```
1 #include <stdio.h>
2 #include <pthread.h>
3 #include "mythreads.h"
4
5 static volatile int counter = 0;
6
7 //
8 // mythread()
9 //
10 // В цикле прибавляет 1 к counter.
11 // Нет, прибавлять 10 000 000 к counter таким
12 // способом не надо, но суть проблемы становится понятна.
13 //
14 void *
15 mythread(void *arg)
16 {
17     printf("%s: begin\n", (char *) arg);
18     int i;
19     for (i = 0; i < 1e7; i++) {
20         counter = counter + 1;
21     }
22     printf("%s: done\n", (char *) arg);
23     return NULL;
24 }
25
26 //
27 // main()
28 //
29 // Запускает два потока (pthread_create)
30 // и ждет их завершения (pthread_join)
31 //
32 int
33 main(int argc, char *argv[])
34 {
35     pthread_t p1, p2;
36     printf("main: begin (counter = %d)\n", counter);
37     Pthread_create(&p1, NULL, mythread, "A");
38     Pthread_create(&p2, NULL, mythread, "B");
39
40     // join ждет завершения потока
41     Pthread_join(p1, NULL);
42     Pthread_join(p2, NULL);
43     printf("main: done with both (counter = %d)\n", counter);
44     return 0;
45 }
```

Рис. 26.6 ❖ Разделение данных: ай и ой (t1.c)

Но, к сожалению, даже на машине с одним процессором результат может быть и иным. Бывает, что получается такое:

```
prompt> ./main
main: begin (counter = 0)
A: begin
```

```
B: begin
A: done
B: done
main: done with both (counter = 19345221)
```

Давайте-ка еще раз, чтобы убедиться, что мы не спятили. Ведь вас же учили, что компьютеры порождают **детерминированный** результат! Может быть, профессора вас обманывали? (*потрясенный выдох*)

```
prompt> ./main
main: begin (counter = 0)
A: begin
B: begin
A: done
B: done
main: done with both (counter = 19221041)
```

Мало того что результат неверный, он еще и *другой*! Возникает недоуменный вопрос: почему так происходит?

СОВЕТ: ЗНАЙТЕ И ИСПОЛЬЗУЙТЕ СВОИ ИНСТРУМЕНТЫ

Всегда изучайте новые инструменты, которые помогают писать, отлаживать и понимать компьютерные системы. Здесь мы воспользуемся одним таким инструментом – **дизассемблером**. Если передать дизассемблеру исполняемый файл, то он покажет составляющие его машинные команды. Например, если мы хотим увидеть низкоуровневый код модификации счетчика (как в примере выше), то нужно запустить программу `objdump` (в Linux):

```
prompt> objdump -d main
```

Будет выведен длинный список всех команд программы, снабженный аккуратными метками (особенно если программа компилировалась с флагом `-g`, который включает в код информацию о символах). Программа `objdump` – лишь один из многих инструментов, которыми нужно научиться пользоваться: отладчик типа `gdb`, профилировщики памяти типа `valgrind` или `purify` и, конечно, сам компилятор. Чем лучше вы освоите инструменты, тем лучше будут созданные вами системы.

26.4. Суть проблемы: НЕКОНТРОЛИРУЕМОЕ ПЛАНИРОВАНИЕ

Чтобы понять, почему такое происходит, нужно разобраться, какой код генерирует компилятор для модификации `counter`. В данном случае мы просто хотим прибавить к `counter` число (1). Поэтому код (для x86) выглядит примерно так:

```
mov 0x8049a1c, %eax
add $0x1, %eax
mov %eax, 0x8049a1c
```

Здесь предполагается, что переменная `counter` хранится по адресу `0x8049a1c`. В этой последовательности трех команд x86 первая команда `mov` используется, чтобы взять значение, хранящееся в памяти по указанному адресу, и поместить его в регистр `eax`. Затем выполняется команда `add`, которая прибавляет единицу (`0x1`) к регистру `eax`, и напоследок содержимое `eax` сохраняется в памяти по тому же адресу.

Представим, что один из наших потоков (Поток 1) входит в эту область программы и, стало быть, собирается увеличить `counter` на 1. Он загружает значение `counter` (допустим, что оно в этот момент равно 50) в регистр `eax`. Теперь `eax=50` в Поток 1. Затем он прибавляет к регистру единицу, так что `eax=51`. И в этот момент происходит непредвиденное: срабатывает таймер прерываний. В ответ ОС сохраняет состояние текущего потока (его РС, регистры, включая `eax`, и т. д.) в TCB потока.

А потом случается страшное: запускается Поток 2, который входит в ту же область кода. Он тоже выполняет первую команду: читает значение `counter` и помещает его в `eax` (напомним, что у каждого потока свой собственный набор регистров, регистры **виртуализируются** кодом контекстного переключения, который их сохраняет и восстанавливает). В этот момент значение `counter` по-прежнему равно 50, а потому в Поток 2 `eax=50`. Предположим, что Поток 2 выполнил и следующие две команды, т. е. увеличил `eax` на 1 (теперь `eax=51`) и сохранил содержимое `eax` в переменной `counter` (по адресу `0x8049a1c`). Стало быть, глобальная переменная `counter` теперь имеет значение 51.

Наконец, происходит еще одно контекстное переключение, и Поток 1 возобновляет выполнение. Напомним, что он только выполнил команды `mov` и `add` и собирается выполнить последнюю команду `mov`. В данный момент `eax=51`. Итак, выполняется последняя команда `mov`, которая сохраняет значение в памяти, переменная `counter` снова равна 51.

Проще говоря, произошло следующее: код увеличения `counter` был выполнен дважды, но значение `counter`, вначале равное 50, теперь равно только 51. В результате работы «правильной» версии этой программы переменная `counter` должна была бы равняться 52.

Рассмотрим подробно трассу выполнения, чтобы лучше разобраться в проблеме. Пусть рассмотренный выше код находится в памяти, начиная с адреса 100, как показано ниже (примечание для тех, кто привык к наборам команд RISC-процессоров: в x86 команды переменной длины, в частности данная команда `mov` занимает 5 байт, а `add` только 3):

```
100 mov    0x8049a1c, %eax
105 add    $0x1, %eax
108 mov    %eax, 0x8049a1c
```

В этих предположениях происходящее показано на рис. 26.7. Считая, что начальное значение `counter` равно 50, протрассируйте этот пример и убедитесь, что понимаете, что происходит.

Явление, которое мы продемонстрировали, называется **состоянием гонки** (точнее, **гонки за данные**): результат зависит от хронометража выполнения кода. Если нам не повезло (контекстное переключение имеет место в неудачные моменты времени), то результат будет неверен. На самом деле мы

можем каждый раз получить другой результат, так что вместо пристойного **детерминированного** вычисления (которое мы привыкли ждать от компьютеров) мы имеем **недетерминированный** результат, когда неизвестно, что получится на выходе, и фактически каждый раз получается что-то новое.

ОС	Поток 1	Поток 2	(После команды)		
			PC	%eax	counter
Прерывание сохранить состояние T1 восстановить состояние T2	До критической секции mov 0x8049a1c, %eax add \$0x1, %eax		100	0	50
			105	50	50
			108	51	50
			100	0	50
		mov 0x8049a1c, %eax add \$0x1, %eax mov %eax, 0x8049a1c	105	50	50
			108	51	50
			113	51	51
			108	51	51
			113	51	51
Прерывание сохранить состояние T2 восстановить состояние T1					

Рис. 26.7 ❖ Проблема с близкого расстояния

Поскольку выполнение этого кода несколькими потоками может приводить к состоянию гонки, он называется **критической секцией**. Это такой участок кода, в котором производится доступ к разделяемой переменной (или вообще к любому разделяемому ресурсу) и который должен конкурентно выполняться более чем одним потоком.

В действительности нам хотелось бы, чтобы этот код обладал свойством **взаимного исключения**. Оно гарантирует, что если один поток находится в критической секции, то всем остальным вход в нее запрещен.

Кстати, практически все эти термины были введены в оборот Эдсгером Дейкстрой, первопроходцем в этой области, который был удостоен премии Тьюринга за эту и другие работы; см. его статью 1968 года о взаимодействующих последовательных процессах [D68], где приведено исключительно ясное описание проблемы. Мы еще не раз встретимся с Дейкстрой в этой части книги.

26.5. Жажда атомарности

Один из способов решить проблему заключается в использовании более мощных команд, которые за один шаг делают все необходимое, так что прерывание просто не может возникнуть не вовремя. Например, что, если бы у нас была такая суперкоманда:

memory-add 0x8049a1c, \$0x1

Допустим, что эта команда прибавляет некоторое значение к содержимому ячейки памяти и что оборудование гарантирует ее **атомарное** выполнение. Это значит, что команда не может быть прервана посередине, ведь именно

такую гарантию дает нам оборудование: если возникает прерывание, то либо команда не выполнена вовсе, либо выполнена до конца, никакого промежуточного состояния быть не может. Хорошая вещь оборудование, правда?

В этом контексте «атомарно» означает «как неделимое целое», или, по-другому, «все или ничего». Мы хотели бы, чтобы следующие три команды выполнялись атомарно:

```
mov 0x8049a1c, %eax
add $0x1, %eax
mov %eax, 0x8049a1c
```

Как уже сказано, если бы у нас была одна команда для этой цели, то можно было бы ее выполнить и спать спокойно. Но в общем случае такой команды быть не может. Представьте, что мы создаем конкурентное В-дерево и хотим его обновить; не станете же вы всерьез требовать, чтобы оборудование подерживало команду «атомарное обновление В-дерева»? Во всяком случае, в разумном наборе команд ничего такого быть не может.

Поэтому вместо этого мы хотим, чтобы оборудование предоставляло несколько полезных команд, на основе которых мы могли бы построить общие **примитивы синхронизации**. С помощью аппаратных примитивов синхронизации в сочетании с кое-какой помощью от операционной системы мы сумеем написать многопоточный код, который в критических секциях работает контролируемо и, следовательно, надежно порождает правильный результат, несмотря на все вызовы конкурентного выполнения. Звучит от-лично, не правда ли?

Совет: использование атомарных операций

Атомарные операции – один из самых мощных методов построения вычислительных систем, они применяются и при определении архитектуры компьютеров, и в конкурентном коде (тема, которую мы сейчас изучаем), и в файловых системах (тема третьей части книги), и в системах управления базами данных, и даже в распределенных системах [L+93].

Идея атомарной последовательности действий выражается простой фразой: «все или ничего»; извне должно казаться, что либо были выполнены все действия, которые мы хотели сгруппировать, либо ни одно из них, а промежуточные состояния при этом не видны. Иногда объединение нескольких действий в одно атомарное называется **транзакцией**, эта концепция тщательно разработана в области систем управления базами данных [GR92].

При обсуждении конкурентности мы будем использовать примитивы синхронизации, чтобы превратить короткие последовательности команд в атомарные блоки выполнения, но сама идея атомарности гораздо шире, в чем мы вскоре и убедимся. Например, в файловых системах применяются такие техники, как журналирование и копирование при записи, чтобы атомарно перенести данные на диск, – это абсолютно необходимо для корректного функционирования даже при условии отказа системы. Если вам пока не понятно, о чем речь, не страшно – ниже мы во всем разберемся.

Этой проблемой мы и будем заниматься в данной части книги. Это удивительная и трудная проблема, так что ваша голова пойдет кругом (чуть-чуть). А не пойдет – значит, чего-то не поняли! Продолжайте напрягать мозги, пока

все не встанет на свои места. А потом сделайте перерыв – нам не нужно, чтобы ваша голова разболелась не на шутку.

СУЩЕСТВО ПРОБЛЕМЫ: КАК ПОДДЕРЖАТЬ СИНХРОНИЗАЦИЮ

Какая поддержка со стороны оборудования необходима для построения полезных примитивов синхронизации? А какая поддержка от ОС? Как построить правильные и эффективные примитивы? Как использовать их в программах для получения желаемых результатов?

26.6. ЕЩЕ ОДНА ПРОБЛЕМА: ОЖИДАНИЕ ДРУГОГО ПОТОКА

В этой главе проблема конкурентности поставлена так, будто единственным типом взаимодействия между потоками является доступ к разделяемым переменным и нужда в поддержке атомарности в критических секциях. Но есть и другой распространенный тип взаимодействия: когда один поток должен ждать, пока другой завершит свои действия. Например, такой тип взаимодействия возникает, когда процесс начинает операцию дискового ввода-вывода и засыпает; по завершении ввода-вывода процесс необходимо пробудить ото сна, чтобы он мог продолжить работу.

Поэтому в последующих главах мы будем изучать не только построение примитивов синхронизации для поддержки атомарности, но и механизмы такого засыпания-пробуждения, часто применяемые в многопоточных программах. Не расстраивайтесь, если не все поняли. Все прояснится уже скоро, в главе об **условных переменных**. А если не прояснится, тогда плохо, и вам придется перечитать главу (а возможно, и не один раз), пока материал не уложится в голове.

26.7. РЕЗЮМЕ: ПОЧЕМУ НА КУРСЕ ПО ОС?

Прежде чем закруглиться, ответим на вопрос, который, наверное, вас мучит: почему мы изучаем все это на курсе по ОС? Простой ответ – «исторически сложилось»; ОС была первой конкурентной программой, поэтому многие методы были разработаны для использования *внутри* ОС. Впоследствии, когда появились многопоточные процессы, осваивать все это пришлось и прикладным программистам.

Например, представьте себе два работающих процесса. Предположим, что оба вызывают функцию `write()` для записи в файл и оба хотят дописать данные в конец файла. Для этого каждый процесс должен выделить новый блок, запомнить индексный дескриптор файла, в котором находится этот блок, и увеличить размер файла (это не все, но подробнее о файлах мы будем

говорить в третьей части книги). Поскольку в любой момент может произойти прерывание, код, в котором обновляются эти разделяемые структуры (например, битовая карта выделений или индексный дескриптор файла), является критической секцией. Поэтому проектировщики ОС с того самого момента, как появились прерывания, должны были позаботиться о том, как именно ОС обновляет свои внутренние структуры данных. Именно несвоевременные прерывания – корень всех описанных выше проблем. Неудивительно, что доступ к таблицам страниц, спискам процессов, системным структурам описания файлов и вообще практически всем структурам данных в ядре должен быть защищен примитивами синхронизации, иначе система будет работать неправильно.

ОТСТУПЛЕНИЕ: ОСНОВНЫЕ ТЕРМИНЫ КОНКУРЕНТНОСТИ – КРИТИЧЕСКАЯ СЕКЦИЯ, СОСТОЯНИЕ ГОНКИ, НЕДЕТЕРМИНИРОВАННОСТЬ, ВЗАИМНОЕ ИСКЛЮЧЕНИЕ

Эти четыре термина настолько важны для описания конкурентного кода, что мы сочли необходимым выделить их специально. Дополнительные сведения см. в ранних работах Дейкстры [D65, D68].

- **Критическая секция** – участок кода, в котором производится доступ к *разделяемому* ресурсу, обычно переменной или структуре данных.
- **Состояние гонки** (или **гонка за данные** [NM92]) возникает, когда несколько потоков выполнения входят в критическую область приблизительно в одно и то же время; все они пытаются обновить разделяемую структуру данных, что приводит к неожиданному (и, возможно, нежелательному) результату.
- **Недетерминированная** программа содержит одно или несколько состояний гонки; при каждом запуске программа дает другой результат, зависящий от того, какие потоки выполнялись. Обычно мы ожидаем от компьютерных программ **детерминированного** поведения, но в данном случае это не так.
- Чтобы избежать подобных проблем, потоки должны пользоваться примитивами **взаимного исключения**, которые гарантируют, что только один поток сможет войти в критическую секцию; тогда никаких гонок не возникнет, и программа снова станет детерминированной.

Литература

[D65] «Solution of a problem in concurrent programming control» by E. W. Dijkstra. Communications of the ACM, 8(9):569, September 1965. *Считается первой статьей Дейкстры, в которой была поставлена и решена проблема взаимного исключения. Это решение, однако, практически не используется на практике; необходима развитая поддержка со стороны оборудования и ОС, в чем мы убедемся в последующих главах.*

[D68] «Cooperating sequential processes» by Edsger W. Dijkstra. 1968. Доступно по адресу <http://www.cs.utexas.edu/users/EWD/ewd01xx/EWD123.PDF>. *Дейкстра оставил (для вечности) поразительное количество старых работ, заметок и мыслей на этом сайте на месте своей последней работы, в Техасском уни-*

верситете. Но большая часть его основополагающих работ была выполнена гораздо раньше, в Высшей технической школе Эйндховена (*Technische Hochschule of Eindhoven – THE*); к ним относится и его знаменитая статья о «взаимодействующих последовательных процессах», где намечены все проблемы, которые нужно учитывать при написании многопоточных программ. Многие из этих мыслей пришли в голову Дейкстре во время работы над операционной системой, названной в честь его учебного заведения: «THE» (произносится «ти», «эйч», «и», а не одним словом «зе»).

[GR92] «Transaction Processing: Concepts and Techniques» by Jim Gray and Andreas Reuter. Morgan Kaufmann, September 1992. Это библия транзакционной обработки, написанная одним из легендарных персонажей в этой области, Джимом Греем. Она считается «дампом мозга» Грея, т. к. он изложил в ней все, что знал о работе систем управления базами данных. К сожалению, несколько лет назад (в 2007 году) Грей трагически погиб, а многие из нас, включая и авторов этой книги, имевших счастье общаться с Греем во время своей магистратуры, потеряли друга и замечательного наставника.

[L+93] «Atomic Transactions» by Nancy Lynch, Michael Merritt, William Weihl, Alan Fekete. Morgan Kaufmann, August 1993. Учебник по теории и практике атомарных транзакций в распределенных системах. Кому-то книга может показаться излишне формальной, но содержит много полезных материалов.

[NM92] «What Are Race Conditions? Some Issues and Formalizations» by Robert H. B. Netzer and Barton P. Miller. ACM Letters on Programming Languages and Systems, Volume 1:1, March 1992. Прекрасное обсуждение различных типов гонок, встречающихся в конкурентных программах. В этой главе (и нескольких последующих) мы акцентировали внимание на гонках за данные, но затем обсудим также **гонки общего вида**.

[SR05] «Advanced Programming in the Unix Environment» by W. Richard Stevens and Stephen A. Rago. Addison-Wesley, 2005. Как мы уже не раз говорили, купите эту книгу и читайте ее. Понемножку, лучше перед сном. Во-первых, скорее заснете, но важнее, что вы больше узнаете о том, как стать серьезным программистом в Unix.

Домашнее задание (эмуляция)

Программа `x86.py` позволит наблюдать, как чередование потоков приводит к состояниям гонки или обходит их. Детали см. в файле README.

Вопросы

1. Исследуйте простую программу `loop.s`. Сначала посмотрите ее код и поймите, что она делает. Затем запустите ее с двумя аргументами (`./x86.py -p loop.s -t 1 -i 100 -R dx`). Это означает один поток, прерывание через каждые 100 команд и трассировку регистра `%dx`. Какие значения принимает `%dx` во время выполнения? Для проверки своих ответов задай-

те флаг `-с`, тогда слева будут печататься значения регистра (или ячейки памяти) *после* выполнения команды, показанной справа.

2. Запустите ту же программу с другими флагами: `./x86.py -p loop.s -t 2 -i 100 -a dx=3, dx=3 -R dx`. Это означает два потока, причем в каждом из них регистр `%dx` инициализируется значением 3. Какие значения принимает `%dx`? Для проверки задайте флаг `-с`. Влияет ли наличие нескольких потоков на вычисления? Существует ли гонка в этом коде?
3. Запустите программу с такими аргументами: `./x86.py -p loop.s -t 2 -i 3 -г -a dx=3, dx=3 -R dx`. Теперь промежуток времени между прерываниями мал и случаен. Чтобы наблюдать разные чередования, указывайте разные начальные значения (`-s`). Влияет ли на что-нибудь частота прерываний?
4. Теперь рассмотрим другую программу, `looping-race-nolock.s`, которая обращается к разделяемой переменной по адресу 2000; назовем ее `value`. Запустите программу с одним потоком, чтобы понять, как она работает: `./x86.py -p looping-race-nolock.s -t 1 -M 2000`. Какие значения принимает `value` (содержимое памяти по адресу 2000) на протяжении работы? Для проверки задайте флаг `-с`.
5. Запустите программу, задав несколько итераций и потоков: `./x86.py -p looping-race-nolock.s -t 2 -a bx=3 -M 2000`. Почему каждый поток выполняет три итерации цикла? Каково конечное значение `value`?
6. Запустите программу со случайным интервалом прерываний: `./x86.py -p looping-race-nolock.s -t 2 -M 2000 -i 4 -г -s 0`, задавая разные начальные значения (`-s 1`, `-s 2` и т. д.). Можете ли вы, глядя на чередование потоков, сказать, каким будет конечное значение `value`? Влияет ли на что-нибудь интервал между прерываниями? Где возникновение прерывания безопасно? А где нет? Иными словами, точно опишите критические секции в этой программе.
7. Теперь рассмотрим фиксированные интервалы между прерываниями: `./x86.py -p looping-race-nolock.s -a bx=1 -t 2 -M 2000 -i 1`. Чему равно конечное значение разделяемой переменной `value`? А что, если задать флаги `-i 2`, `-i 3` и т. д.? При каких интервалах между прерываниями программа дает «правильный» ответ?
8. Запустите ту же программу с большим числом циклов (например, задайте флаг `-a bx=100`). При каких интервалах между прерываниями (`-i`) получается правильный результат? Какие интервалы ведут к неожиданностям?
9. И последняя программа: `wait-for-me.s`. Запустите ее следующим образом: `./x86.py -p wait-for-me.s -a ax=1, ax=0 -R ax -M 2000`. При этом программа записывает в регистр `%ax` значение 1 в потоке 0 и значение 0 в потоке 1, а затем наблюдает за регистром `%ax` и ячейкой памяти по адресу 2000. Как должен вести себя код? Как значение по адресу 2000 используется потоками? Каким будет конечное значение?
10. Теперь поменяйте начальные значения местами: `./x86.py -p wait-for-me.s -a ax=0, ax=1 -R ax -M 2000`. Как ведут себя потоки? Что делает поток 0? Как изменение интервала между прерываниями (например, задайте `-i 1000` или случайные интервалы) влияет на результат трассировки? Эффективно ли программа использует процессор?

Глава 27

Интерлюдия: API потоков

В этой главе мы кратко рассмотрим основные части API потоков. Все они будут подробнее описаны в последующих главах, когда мы будем обсуждать использование API. Дополнительные сведения можно почерпнуть в различных книгах и онлайн-ресурсах [B89, B97, B+96, K+96]. Отметим, что в последующих главах блокировки и условные переменные будут рассматриваться в более медленном темпе, с многочисленными примерами, а эту главу лучше использовать как справочник.

СУЩЕСТВО ПРОБЛЕМЫ: КАК СОЗДАВАТЬ ПОТОКИ И УПРАВЛЯТЬ ИМИ

Какие интерфейсы должна предоставить ОС для создания и управления потоками? Как они должны быть спроектированы, чтобы ими было легко пользоваться, но не в ущерб функциональности?

27.1. СОЗДАНИЕ ПОТОКА

Первое, чему нужно научиться, если мы хотим писать многопоточные программы, – создавать новые потоки, стало быть, должен существовать соответствующий интерфейс. В POSIX это делается так:

```
#include <pthread.h>
int
pthread_create(      pthread_t *      thread,
                    const pthread_attr_t * attr,
                    void *            (*start_routine)(void*),
                    void *            arg);
```

Объявление выглядит довольно сложно (особенно если вы не привыкли к указателям на функции в C), но на самом деле все не так плохо. Функция передается четыре аргумента: `thread`, `attr`, `start_routine` и `arg`. Первый, `thread`, указатель на структуру типа `pthread_t`; мы используем ее для взаимодействия

с потоком, поэтому должны передать `pthread_create()`, чтобы та ее инициализировала.

Второй аргумент, `attr`, задает атрибуты потока, например размер стека или информацию о приоритете планирования. Атрибуты инициализируются путем отдельного обращения к функции `pthread_attr_init()`; подробности см. на странице руководства. Впрочем, в большинстве случаев значений по умолчанию достаточно, и тогда можно просто передать вместо этого аргумента `NULL`.

Третий аргумент самый сложный, но по существу это просто функция, которая должна выполняться в потоке. В С такая конструкция называется **указателем на функцию**, в данном случае это функция `start_routine`, которой передается один аргумент типа `void *` (о чем говорят скобки после имени функции) и которая возвращает значение типа `void *` (т. е. **указатель на void**).

Если бы функции требовался целый аргумент, а не указатель на `void`, объявление выглядело бы так:

```
int pthread_create(..., // первые два аргумента не меняются
                  void * (*start_routine)(int),
                  int     arg);
```

А если бы функция принимала в качестве аргумента указатель на `void`, но возвращала целое, то объявление было бы таким:

```
int pthread_create(..., // первые два аргумента не меняются
                  int     (*start_routine)(void *),
                  void *  arg);
```

Наконец, четвертый аргумент, `arg`, — это и есть аргумент, передаваемый функции, в которой поток начинает выполнение. Вы можете спросить зачем нужны эти указатели на `void`? Да все просто: функции, принимающей указатель на `void` (в данном случае `start_routine`), можно передать аргумент *любого* типа, а функция, возвращающая такой указатель, может вернуть результат *любого* типа.

Рассмотрим пример на рис. 27.1. Здесь создается поток, которому передается два аргумента, упакованных в определенный нами тип (`myarg_t`). После создания поток может привести аргумент к ожидаемому типу и, стало быть, распаковать аргументы нужным ему способом.

Вот и всё! Создав поток, мы получили еще одну допускающую исполнение сущность со своим стеком, которая работает в *той же* адресном пространстве, что и все уже существующие потоки программы. И теперь начинается самое интересное!

```

1 #include <pthread.h>
2
3 typedef struct __myarg_t {
4     int a;
5     int b;
6 } myarg_t;
7
8 void *mythread(void *arg) {
9     myarg_t *m = (myarg_t *) arg;
10    printf("%d %d\n", m->a, m->b);
11    return NULL;
12 }
13
14 int
15 main(int argc, char *argv[]) {
16     pthread_t p;
17     int rc;
18
19     myarg_t args;
20     args.a = 10;
21     args.b = 20;
22     rc = pthread_create(&p, NULL, mythread, &args);
23     ...
24 }

```

Рис. 27.1 ❖ Создание потока

27.2. ЗАВЕРШЕНИЕ ПОТОКА

В примере выше показано, как создать поток. Но что, если мы хотим подождать завершения потока? Для этого нужно вызвать функцию `pthread_join()`.

```
int pthread_join(pthread_t thread, void **value_ptr);
```

Эта функция принимает два аргумента. Первый аргумент типа `pthread_t` определяет, какого потока дожидаться. Он должен быть инициализирован функцией создания потока `pthread_create()`, которой передается в качестве первого аргумента.

Второй аргумент – указатель на переменную, в которой мы хотим получить возвращенное значение. Поскольку функция может вернуть что угодно, он определен как указатель на `void`, а т. к. `pthread_join()` *изменяет* значение переданного аргумента, необходимо передавать указатель на значение, а не само значение.

Рассмотрим еще один пример (рис. 27.2). Программа снова создает один поток и передает два аргумента, упакованных в структуру `myarg_t`. Для возврата значения используется тип `myret_t`. После того как поток завершился, функция `pthread_join()`¹, которая ожидала его завершения, возвращает управление главному потоку, и тот может получить доступ к возвращенному значению в структуре `myret_t`.

```

1 #include <stdio.h>
2 #include <pthread.h>
3 #include <assert.h>
4 #include <stdlib.h>
5
6 typedef struct __myarg_t {
7     int a;
8     int b;
9 } myarg_t;
10
11 typedef struct __myret_t {
12     int x;
13     int y;
14 } myret_t;
15
16 void *mythread(void *arg) {
17     myarg_t *m = (myarg_t *) arg;
18     printf("%d %d\n", m->a, m->b);
19     myret_t *r = Malloc(sizeof(myret_t));
20     r->x = 1;
21     r->y = 2;
22     return (void *) r;
23 }
24
25 int
26 main(int argc, char *argv[]) {
27     pthread_t p;
28     myret_t *m;
29
30     myarg_t args = {10, 20};
31     Pthread_create(&p, NULL, mythread, &args);
32     Pthread_join(p, (void **) &m);
33     printf("returned %d %d\n", m->x, m->y);
34     free(m);
35     return 0;
36 }

```

Рис. 27.2 ❖ Ожидание завершения потока

¹ Обратите внимание на использование функций-оберток: мы вызываем `Malloc()`, `Pthread join()` и `Pthread create()`, которые просто вызывают соответствующую функцию с именем, содержащим только строчные буквы, и проверяют, что она не вернула ничего неожиданного.

Сделаем несколько замечаний об этом примере. Во-первых, нам зачастую не нужно заниматься утомительной упаковкой и распаковкой аргументов. Например, если мы просто создаем поток без аргументов, то можем при создании передать в качестве аргумента NULL. Аналогично, можно передать NULL функции `pthread_join()`, если возвращенное значение нас не интересует.

Во-вторых, если функции передается всего одно значение (например, типа `int`), то упаковывать его в структуру нет необходимости. Пример приведен на рис. 27.3. В таком случае код становится немного проще, потому что пропадает шаг упаковки аргументов и распаковки возвращенных значений.

```
void *mythread(void *arg) {
    int m = (int) arg;
    printf("%d\n", m);
    return (void *) (arg + 1);
}

int main(int argc, char *argv[]) {
    pthread_t p;
    int rc, m;
    Pthread_create(&p, NULL, mythread, (void *) 100);
    Pthread_join(p, (void **) &m);
    printf("returned %d\n", m);
    return 0;
}
```

Рис. 27.3 ❖ Упрощенная передача аргументов потоку

В-третьих, нужно быть очень осторожными с тем, как значения возвращаются из потока. В частности, никогда не возвращайте указатель на что-то, размещенное в стеке потока. Подумайте, что будет, если пренебречь этим советом. Ниже приведен пример опасного кода, полученный модификацией примера на рис. 27.3.

```
1 void *mythread(void *arg) {
2     myarg_t *m = (myarg_t *) arg;
3     printf("%d %d\n", m->a, m->b);
4     myret_t r; // ПАМЯТЬ ВЫДЕЛЕНА В СТЕКЕ: ПЛОХО!
5     r.x = 1;
6     r.y = 2;
7     return (void *) &r;
8 }
```

Здесь память для переменной `r` выделена в стеке потока `mythread`. Но при возврате из функции эта память автоматически освобождается (именно поэтому стек так легко использовать!), а передача указателя на уже не существующую переменную чревата всяческими неприятностями. И действительно, при попытке напечатать возвращенное значение вы, вероятно (но не обязательно!), будете немало удивлены. Попробуйте и убедитесь сами¹.

¹ К счастью, компилятор `gcc`, скорее всего, ругнется на такой код, и это еще одна причина обращать внимание на предупреждения компилятора.

Наконец, вы, наверное, пришли к выводу, что использование `pthread_create()` и сразу вслед за этим обращение к `pthread_join()` – довольно странное занятие. Действительно, существует более простой способ сделать то же самое: **вызов процедуры**. Очевидно, что обычно нам нужно нечто большее, чем просто создание одного потока и ожидание его завершения, а иначе в потоках не было бы никакого смысла.

Отметим, что не в любой многопоточной программе используется функция ожидания (`join`). Например, многопоточный веб-сервер мог бы создать несколько рабочих потоков, а затем в главном потоке в бесконечном цикле принимать запросы и отдавать их рабочим потокам. В таких долго работающих программах нет необходимости ожидать завершения. Но в параллельной программе, которая создает поток для выполнения конкретной задачи (параллельно), скорее всего, ожидать придется, потому что мы хотим, чтобы работа завершилась перед выходом из программы или переходом к следующему этапу вычислений.

27.3. Блокировки

Следующими по полезности после создания и ожидания завершения потока, пожалуй, являются описанные в стандарте POSIX функции, обеспечивающие взаимное исключение в критических секциях с помощью **блокировок**. Вот две самые главные функции для этой цели:

```
int pthread_mutex_lock(pthread_mutex_t *mutex);
int pthread_mutex_unlock(pthread_mutex_t *mutex);
```

Эти функции понятны, и использовать их просто. Блокировки весьма полезны, когда требуется защитить **критическую секцию** кода и тем самым гарантировать корректность операций. Такой код мог бы выглядеть следующим образом:

```
pthread_mutex_t lock;
pthread_mutex_lock(&lock);
x = x + 1; // или что там еще делается в критической секции
pthread_mutex_unlock(&lock);
```

Смысл этого кода следующий: если никакой другой поток не удерживает блокировку в момент вызова функции `pthread_mutex_lock()`, то поток захватит блокировку и войдет в критическую секцию. Если же другой поток удерживает блокировку, то поток, желающий ее захватить, не вернет управление из этой функции, пока не получит блокировку в свое распоряжение (в предположении, что поток, удерживающий блокировку, рано или поздно освободит ее путем обращения к `pthread_mutex_unlock`). Разумеется, в каждый момент времени получения блокировки может ожидать много потоков, но только поток, захвативший блокировку, может вызвать `unlock`.

К сожалению, в этом коде есть ошибки – и даже две. Во-первых, это **отсутствие должной инициализации**. Все блокировки должны быть инициа-

Они используются для захвата блокировки. Функция `trylock` возвращает код ошибки, если блокировка уже захвачена, а функция `timedlock` возвращает управление, если блокировка успешно захвачена или произошел тайм-аут. Если тайм-аут равен 0, то `timedlock` сводится к `trylock`. В общем случае этими функциями лучше не пользоваться, но есть несколько ситуаций, когда крайне желательно избежать зависания (возможно, бесконечного) в функции захвата блокировки. Мы обсудим это в последующих главах (в частности, при изучении взаимоблокировки).

27.4. УСЛОВНЫЕ ПЕРЕМЕННЫЕ

Еще один важный компонент любой библиотеки для работы с потоками и, конечно же, потоков POSIX – **условные переменные**. Они полезны, если нужно организовать механизм обмена сигналами между потоками, когда один поток может продолжить работу только после того, как другой что-то сделает. Вот две основные функции из этого ряда:

```
int pthread_cond_wait(pthread_cond_t *cond, pthread_mutex_t *mutex);
int pthread_cond_signal(pthread_cond_t *cond);
```

Чтобы воспользоваться условной переменной, необходимо иметь блокировку, ассоциированную с данным условием. При вызове любой из показанных выше функций эта блокировка будет захватываться.

После вызова первой функции, `pthread_cond_wait()`, поток засыпает и пребывает в таком состоянии, пока какой-то другой поток не сигнализирует ему. Обычно это бывает, когда в программе произошло какое-то изменение, интересное спящему потоку. Типичный сценарий показан ниже:

```
pthread_mutex_t lock = PTHREAD_MUTEX_INITIALIZER;
pthread_cond_t cond = PTHREAD_COND_INITIALIZER;
pthread_mutex_lock(&lock);
while (ready == 0)
    pthread_cond_wait(&cond, &lock);
pthread_mutex_unlock(&lock);
```

Здесь после инициализации блокировки и условной переменной¹ поток проверяет, верно ли, что переменная `ready` все еще равна нулю. Если да, поток вызывает функцию `wait` и спит, пока какой-нибудь другой поток не разбудит его.

В каком-то другом потоке для пробуждения данного потока должен быть выполнен код такого вида:

```
pthread_mutex_lock(&lock);
ready = 1;
```

¹ Можно было бы использовать функцию `pthread_cond_init()` (и парную ей `pthread_cond_destroy()`) вместо статического инициализатора `PTHREAD_COND_INITIALIZER`. Кажется, что больше работы? Так оно и есть.

```
Pthread_cond_signal(&cond);
Pthread_mutex_unlock(&lock);
```

Тут надо сказать несколько слов. Во-первых, в момент сигнализации (а также при модификации глобальной переменной `ready`) мы обязательно должны удерживать блокировку. Это дает уверенность, что мы по неосторожности не внесем в программу состояние гонки.

Во-вторых, вы, наверное, обратили внимание, что функция `wait` принимает блокировку в качестве второго параметра, тогда как функция `signal` принимает только условную переменную. Это различие объясняется тем, что `wait` должна не только усыпить вызывающий поток, но и *освободить* при этом блокировку. Подумайте сами: ведь если бы она этого не сделала, то как бы другой поток смог захватить блокировку и просигнализировать о необходимости просыпаться? Однако *перед* тем как вернуть управление после пробуждения, функция `pthread_cond_wait()` заново захватывает блокировку, гарантируя тем самым, что в любой момент времени между захватом блокировки в начале и освобождением ее в конце ожидающий поток работает, удерживая блокировку.

И последняя странность: ожидающий поток повторно проверяет условие в цикле `while`, а не просто в предложении `if`. Мы вернемся к этой детали при изучении условных переменных в следующей главе, но уже сейчас отметим, что в общем случае использование цикла `while` – самый простой и безопасный способ в этой ситуации. Конечно, перепроверку условия можно счесть накладными расходами, пусть и небольшими, но дело в том, что в некоторых реализациях `pthread` ожидающий поток может быть пробужден по ошибке, поэтому без перепроверки он будет считать, что условие изменилось, хотя на самом деле это не так. Так что лучше рассматривать пробуждение как предположение о том, что нечто могло измениться, а не как непреложный факт.

Отметим, что иногда возникает искушение использовать в качестве средства сигнализации между потоками простой флаг, а не условную переменную и ассоциированную с ней блокировку. Например, можно было бы переписать код ожидания таким образом:

```
while (ready == 0)
; // крутиться в цикле
```

А код сигнализации тогда выглядел бы так:

```
ready = 1;
```

Ни в коем случае не поступайте так – и вот почему. Во-первых, подобный код медленно работает (проверка в цикле в течение длительного времени просто транжирит процессорное время). Во-вторых, такой код чреват ошибками. Как показало недавнее исследование [X+10], очень легко допустить ошибку при таком использовании флагов для синхронизации потоков. Утверждается, что примерно половина изученных ситуативных синхронизаций такого рода содержала ошибки! Не ленитесь, используйте условные переменные, даже когда вам кажется, что без них можно обойтись.

Если вы пока не разобрались до конца в условных переменных, не переживайте – в следующей главе мы рассмотрим их гораздо подробнее. А до тех пор достаточно знать, что они существуют, и примерно представлять, как и для чего они используются.

27.5. Компиляция и выполнение

Все приведенные в этой главе примеры несложно собрать и выполнить. Для компиляции необходимо включить в код заголовочный файл `pthread.h`. При вызове компилятора нужно скомпоновать программу с библиотекой `pthread`, добавив флаг `-pthread`.

Например, чтобы откомпилировать простую многопоточную программу, нужно лишь ввести команду

```
prompt> gcc -o main main.c -Wall -pthread
```

Коль скоро `main.c` включает заголовок `pthread.h`, конкурентная программа будет успешно откомпилирована. А уж работает она или нет – это другой вопрос.

27.6. Резюме

Мы познакомились с основами библиотеки `pthread`: создание потока, взаимное исключение с помощью блокировок, ожидание и сигнализация с помощью условных переменных. Для написания надежного и эффективного многопоточного кода больше почти ничего и не нужно – только терпение и аккуратность!

Завершая эту главу, мы дадим несколько советов, которые могут помочь вам при написании многопоточного кода (см. врезку «Отступление» ниже). У API есть и другие возможности, которые могут показаться вам интересными; если хотите получить дополнительные сведения, наберите команду `man -k pthread` в своей системе Linux – и вы увидите свыше сотни функций, составляющих интерфейс потоков. Но и рассматриваемых в книге основ должно хватить для создания достаточно функциональных (и, хочется надеяться, правильных и производительных) многопоточных программ. Сложность при работе с потоками представляет не API, а продумывание логики конкурентной программы. Но об этом ниже.

ОТСТУПЛЕНИЕ: РЕКОМЕНДАЦИИ ПО ИСПОЛЬЗОВАНИЮ API ПОТОКОВ

При работе с библиотекой потоков POSIX (да и любой другой) стоит помнить о ряде простых, но важных вещей.

- **Будьте проще.** Помимо всего прочего, код блокировки и межпоточной сигнализации должен быть как можно проще. Запутанные взаимодействия между потоками ведут к ошибкам.

- **Минимизируйте взаимодействие потоков.** Старайтесь свести к минимуму количество способов взаимодействия потоков. Каждое взаимодействие следует тщательно продумывать и при реализации применять проверенные временем подходы (о многих из них мы узнаем в последующих главах).
- **Инициализируйте блокировки и условные переменные.** Если этого не сделать, то программа иногда будет работать, а иногда «вылетать» таинственным образом.
- **Проверяйте коды ошибок.** Конечно, при программировании на C в Unix необходимо проверять все без исключения возвращенные значения, и к данному случаю это правило тоже относится. Пренебрежение им ведет к загадочному и трудно объяснимому поведению, из-за которого хочется (а) кричать, (b) рвать на себе волосы или (c) делать то и другое.
- **Внимательно следите за тем, как вы передаете аргументы потокам и получаете от них возвращаемые значения.** В частности, передавая указатель на переменную, хранящуюся в стеке, вы, скорее всего, совершаете ошибку.
- **У каждого потока свой стек.** В дополнение к предыдущему совету не забывайте, что у каждого потока свой собственный стек. Поэтому, если вы завели локальную переменную внутри функции, исполняемой в потоке, то она принадлежит только данному потоку, и никакой другой не сможет к ней обратиться (по крайней мере, без труда). Если вы хотите, чтобы потоки могли сообща пользоваться данными, то память для них необходимо выделять в **куче** или еще в каком-то глобально доступном месте.
- **Для межпоточной сигнализации всегда используйте условные переменные.** Хотя соблазнительно ограничиться простым флагом, не делайте этого.
- **Читайте страницы руководства.** В частности, в Linux страницы руководства по библиотеке pthread в высшей степени информативны, в них обсуждаются многие описанные здесь нюансы, и зачастую даже подробнее. Читайте их внимательно!

Литература

[B89] «An Introduction to Programming with Threads» by Andrew D. Birrell. DEC Technical Report, January, 1989. Доступно по адресу <https://birrell.org/andrew/papers/035-Threads.pdf>. *Классическое, но уже устаревшее введение в многопоточное программирование. Все еще интересное и к тому же бесплатно доступное чтение.*

[B97] «Programming with POSIX Threads» by David R. Butenhof. Addison-Wesley, May 1997. *Еще одна книга, посвященная потокам.*

[B+96] «PThreads Programming: by A POSIX Standard for Better Multiprocessing». Dick Buttler, Jacqueline Farrell, Bradford Nichols. O'Reilly, September 1996. *Достойная книга от уважаемого издательства O'Reilly. На наших полках стоит много книг этого издательства, включая блистательные труды по Perl, Python и Javascript (в особенности книга Crockford «Javascript: The Good Parts»).*

[K+96] «Programming With Threads» by Steve Kleiman, Devang Shah, Bart Smalders. Prentice Hall, January 1996. *Быть может, одна из лучших книг на эту тему. Возьмите ее в местной библиотеке. Или украдите у мамы. Нет, серьезно, просто попросите ее у мамы – она даст почитать, не сомневайтесь.*

[X+10] «Ad Hoc Synchronization Considered Harmful» by Weiwei Xiong, Soyeon Park, Jiaqi Zhang, Yuanyuan Zhou, Zhiqiang Ma. OSDI 2010, Vancouver, Canada. *В этой статье показано, что обманчиво простой код синхронизации может приводить к удивительно большому числу ошибок. Пользуйтесь условными переменными и реализуйте сигнализацию правильно!*

Домашнее задание (код)

В этом задании мы напишем простые многопоточные программы и воспользуемся специальным инструментом **helgrind** для поиска ошибок в них. О том, как собирать программы и пользоваться **helgrind**, читайте в файле README.

Вопросы

1. Сначала соберите программу `main-race.c`. Изучите код, чтобы понять, где в нем происходит гонка за данные (надеемся, что это очевидно). Теперь запустите **helgrind** (введите команду `valgrind --tool=helgrind main-race`) и посмотрите, как она сообщает о гонке. Правильно ли указаны строки кода? Какую еще информацию дает инструмент?
2. Что будет, если удалить одну из ошибочных строк? Теперь защитите блокировкой сначала одно из обновлений разделяемой переменной, а затем оба. Что **helgrind** сообщает в каждом случае?
3. Теперь рассмотрим программу `main-deadlock.c`. Изучите ее код. В нем имеется **взаимоблокировка** (в следующей главе мы будем обсуждать эту проблему гораздо подробнее). Сможете ли вы понять, в чем потенциальная проблема?
4. Запустите для этой программы **helgrind**. Что он сообщает?
5. Теперь изучите программу `main-deadlock-global.c`. Есть ли в этом коде та же проблема, что в `main-deadlock.c`? Должен ли **helgrind** сообщать о такой же ошибке? Исходя из этого, что вы можете сказать об инструментах типа **helgrind**?
6. Далее рассмотрим программу `main-signal.c`. В ней используется переменная `done`, чтобы просигнализировать родителю о том, что потомок завершился и можно продолжать работу. Почему этот код неэффективен? (На что родительский процесс тратит время, особенно если потомок работает долго?)
7. Запустите для этой программы **helgrind**. Что он сообщает? Правильен ли код?
8. Затем рассмотрите немного модифицированную версию этого кода в файле `main-signal-cv.c`. В ней для сигнализации используется условная переменная (и ассоциированная с ней блокировка). Почему этот код предпочтительнее? Все дело в корректности, в производительности или в том и другом сразу?
9. Запустите **helgrind** для `main-signal-cv`. Сообщает ли инструмент об ошибках?

Глава 28

Блокировки

Во введении к части, посвященной конкурентности, мы описали одну из фундаментальных проблем конкурентного программирования: необходимо атомарно выполнить последовательность команд, но из-за прерываний при работе на одном процессоре (или из-за наличия нескольких потоков, конкурентно выполняемых на нескольких процессорах) сделать это не удастся. В данной главе мы атакуем эту проблему в лоб, введя в рассмотрение **блокировку**. Программисты вставляют блокировки в исходный код, окружая ими критические секции, и таким образом гарантируют, что код в критической секции выполняется как одна атомарная команда.

28.1. Блокировки: основная идея

В качестве примера предположим, что критическая секция состоит из канонического обновления разделяемой переменной:

```
balance = balance + 1;
```

Конечно, возможны и другие критические секции, скажем добавление элемента в связный список или более сложные обновления разделяемых структур данных, но пока ограничимся этим простым примером. Чтобы воспользоваться блокировкой, мы должны окружить критическую секцию специальным кодом, как показано ниже:

```
1 lock_t mutex; // глобально выделенная блокировка 'mutex'
2 ...
3 lock(&mutex);
4 balance = balance + 1;
5 unlock(&mutex);
```

Блокировка – это просто переменная, поэтому мы должны объявить **переменную блокировки** какого-то типа (в данном случае `mutex`). В переменной блокировки (или просто блокировке) хранится текущее состояние блокировки. Она либо **свободна** (**разблокирована**, **доступна**), т. е. ни один поток не удерживает ее, либо **захвачена** (**заблокирована**, **занята**), т. е. ее удерживает один и только один поток, вероятно, находящийся в критической секции. В переменной можно хранить и другую информацию, например какой поток

удерживает блокировку или очередь потоков, выстроившуюся к блокировке), но она скрыта от пользователя блокировки.

Семантика функций `lock()` и `unlock()` проста. Вызов `lock()` пытается захватить блокировку; если никакой другой поток ее не удерживает (блокировка свободна), то поток захватит ее и войдет в критическую секцию, иногда говорят, что поток стал **владельцем** блокировки. Если какой-то поток впоследствии вызовет `lock()` для той же самой переменной блокировки (в нашем примере `mutex`), то вызов не вернет управление, пока блокировка удерживается другим потоком. Таким образом, другие потоки не могут войти в критическую секцию, пока в ней находится владелец блокировки.

После того как владелец блокировки вызовет `unlock()`, блокировка снова станет доступной. Если другие потоки не ждут ее (т. е. ни один поток не вызвал ранее `lock()` и не оказался заблокированным), то состояние блокировки просто изменяется на «свободна». Если же имеются ожидающие потоки, то один из них будет (в конечном итоге) проинформирован об изменении состояния блокировки, захватит блокировку и войдет в критическую секцию.

Блокировки дают программисту минимальный контроль над планированием. Мы рассматриваем потоки как объекты, которые создаются программистом, но планируются ОС так, как она сочтет нужным. Блокировки позволяют программисту вернуть себе часть контроля, поскольку, окружив участок кода блокировкой, тот может гарантировать, что этот участок никогда не будет исполняться сразу несколькими процессами. Поэтому блокировка превращает хаос, характерный для традиционного планирования ОС, в более упорядоченную деятельность.

28.2. БЛОКИРОВКИ В PTHREAD

В стандарте POSIX для блокировок употребляется название **мьютекс** (`mutex`), поскольку они применяются для обеспечения **взаимного исключения** (**mutual exclusion**) потоков, т. е. поток, находящийся в критической секции, исключает остальные потоки, не давая им войти, пока не выйдет сам. Таким образом, следующий многопоточный код, написанный с соблюдением правил POSIX, делает то же самое, что и показанный выше код (мы снова пользуемся обертками для проверки ошибок при блокировке и разблокировке):

```
1 pthread_mutex_t lock = PTHREAD_MUTEX_INITIALIZER;
2
3 Pthread_mutex_lock(&lock); // обертка pthread_mutex_lock()
4 balance = balance + 1;
5 Pthread_mutex_unlock(&lock);
```

Вы, наверное, заметили, что в версии POSIX функциям блокировки и разблокировки передается переменная, потому что для защиты разных секций можно использовать *разные* блокировки. Это позволяет повысить степень конкурентности: вместо того чтобы использовать одну большую блокировку для доступа к любой критической секции (**крупная** блокировка), мы часто

защищаем разные структуры данных различными блокировками, допуская исполнение нуждающегося в защите кода сразу несколькими потоками (**мелкие** блокировки).

28.3. КОНСТРУИРОВАНИЕ БЛОКИРОВКИ

Теперь вы более-менее понимаете, как блокировка работает с точки зрения программиста. Но как создать блокировку? Какая поддержка необходима со стороны оборудования и ОС? На эти вопросы мы ответим в оставшейся части данной главы.

СУЩЕСТВО ПРОБЛЕМЫ: КАК СКОНСТРУИРОВАТЬ БЛОКИРОВКУ

Как сконструировать эффективную блокировку? Эффективная блокировка должна обеспечить взаимное исключение с низкими затратами и может также обладать другими свойствами, обсуждаемыми ниже. Какая поддержка необходима со стороны оборудования? А со стороны ОС?

Для построения работоспособной блокировки нам понадобится помощь от нашего старого друга, оборудования, и от нашего доброго приятеля, ОС. На протяжении многих лет в наборы команды различных компьютеров были добавлены разные аппаратные примитивы. Мы не будем изучать, как реализованы эти команды (это, скорее, тема курса по архитектуре компьютеров), но посмотрим, как с их помощью можно построить примитив взаимного исключения, в т. ч. блокировку. Мы также выясним, какое участие принимает в этом ОС и как она позволяет построить развитую библиотеку блокировки.

28.4. ОЦЕНИВАНИЕ БЛОКИРОВОК

Прежде чем конструировать блокировки, нужно понять, чего мы хотим достичь и, стало быть, как оценивать эффективность конкретной реализации. Необходимо установить базовые критерии работоспособности блокировок. Прежде всего блокировка должна отвечать своему главному предназначению – обеспечивать **взаимное исключение**. Работает ли вообще блокировка, т. е. предотвращает ли она нахождение нескольких потоков в критической области?

Второй критерий – **справедливость**. Получают ли все потоки, конкурирующие за блокировку, равные шансы захватить освободившуюся блокировку? Этот вопрос можно сформулировать по-другому, рассмотрев крайний случай: не может ли случиться ли так, что какой-то поток **зависнет** в процессе ожидания и никогда не получит блокировку?

И последний критерий – **производительность**, а именно дополнительные временные затраты, вызванные использованием блокировки. Тут следует

рассмотреть несколько случаев. Первый – отсутствие конкуренции: если единственный поток захватывает и освобождает блокировку, то какие накладные расходы он несет? Второй случай – когда несколько потоков конкурируют за доступ к блокировке на одном CPU; есть ли в этом случае какие-то соображения производительности? Наконец, насколько эффективно работает блокировка, когда имеется несколько CPU и потоки, исполняемые каждым, конкурируют за блокировку? Сравнив поведение в этих трех ситуациях, мы сможем лучше понять, какое влияние на производительность оказывают различные методы блокировки, описанные ниже.

28.5. УПРАВЛЕНИЕ ПРЕРЫВАНИЯМИ

Одним из самых первых решений проблемы взаимного исключения стало запрещение прерываний в критических секциях, это решение было предназначено для однопроцессорных систем. Код выглядел так:

```
1 void lock() {  
2     DisableInterrupts();  
3 }  
4 void unlock() {  
5     EnableInterrupts();  
6 }
```

Предположим, что программа работает в такой однопроцессорной системе. Запретив прерывания (с помощью специальной команды) перед входом в критическую секцию, мы гарантируем, что код внутри этой секции не будет прерван, а значит, будет выполняться атомарно. По выходе из секции мы вновь разрешаем прерывания (с помощью другой команды), и программа продолжает работать как обычно.

Основное достоинство этого подхода – простота. Не нужно чесать в затылке, чтобы понять, почему это решение работает. В отсутствие прерываний поток может быть уверен, что никакой другой поток не нарушит ход выполнения кода.

А вот недостатков, к сожалению, много. Во-первых, требуется, чтобы любому вызывающему потоку было разрешено выполнить *привилегированную* операцию (запрещение и разрешение прерываний), а значит, мы вынуждены *доверять* исполняемому в этом потоке коду. Но вы уже знаете, что, доверяя произвольной программе, мы роем себе яму. Проблемы могут проявляться по-разному. Например, жадная программа могла бы вызвать `lock()` в начале выполнения и тем самым монополизировать процессор. Хуже того, ошибочная или вредоносная программа могла бы вызвать `lock()` и войти в бесконечный цикл; в этом случае ОС никогда не сможет восстановить контроль над системой, и останется только один выход: перезагрузить систему. Запрещение прерываний как общий метод синхронизации означало бы, что мы слишком доверяем приложениям.

Во-вторых, этот подход не работает в системах с несколькими процессорами. Если несколько потоков работают на разных CPU и каждый пытается

войти в одну и ту же критическую секцию, то не важно, разрешены прерывания или запрещены: поток, работающий на другом процессоре, сможет это сделать. Поскольку мультипроцессоры теперь используются повсеместно, придется придумать решение получше.

В-третьих, если запретить прерывания на длительное время, то часть прерываний может быть потеряна, что ведет к серьезным общесистемным проблемам. Подумайте, например, что будет, если CPU не заметил того факта, что диск закончил обрабатывать запрос чтения. Как тогда ОС узнает, что пора будить процесс, ожидающий завершения этой операции?

Наконец, – и это самая меньшая из бед – такое решение может оказаться неэффективным. По сравнению с нормальным выполнением команд, код, в котором прерывания то запрещаются, то разрешаются, на современных процессорах обычно работает дольше.

По всем этим причинам запрещение прерываний в качестве примитива взаимного исключения используется только в ограниченных контекстах. Например, в некоторых случаях операционная система запрещает прерывания для гарантии атомарности при доступе к собственным структурам данных и чтобы предотвратить возможный хаос при обработке прерываний. Такое использование имеет смысл, потому что внутри ОС проблемы доверия нет – она в любом случае доверяет самой себе хотя бы для того, чтобы выполнять привилегированные операции.

28.6. Неудачная попытка: пробуем обойтись командами загрузки и сохранения

Потерпев неудачу с методами на основе прерываний, мы вынуждены обратиться за поддержкой к оборудованию и предоставляемым им командам. Сначала попробуем сконструировать простую блокировку, воспользовавшись одной переменной-флагом. И хотя ничего не получится, мы познакомимся с некоторыми базовыми идеями и увидим, почему одной переменной и доступа к ней с помощью стандартных команд загрузки и сохранения недостаточно.

Идея первой попытки (рис. 28.1) проста: использовать обычную переменную (flag), показывающую, владеет ли какой-нибудь поток блокировкой. Первый поток, вошедший в критическую область, вызовет функцию lock(), которая **проверит**, равен ли флаг 1 (в данном случае нет), а затем **установит** флаг в 1, чтобы показать, что теперь поток **удерживает** блокировку. Перед выходом из критической секции поток вызовет функцию unlock(), которая сбросит флаг и тем самым покажет, что блокировка больше не занята.

Если какой-нибудь другой поток вызовет lock(), когда первый поток находится в критической секции, то он будет **активно ждать** в цикле while, пока этот поток не вызовет unlock() и не очистит флаг. А как только первый поток это сделает, ожидающий поток выйдет из цикла, установит флаг в 1 для себя и войдет в критическую секцию.

```

1 typedef struct __lock_t { int flag; } lock_t;
2
3 void init(lock_t *mutex) {
4     // 0 -> блокировка свободна, 1 -> занята
5     mutex->flag = 0;
6 }
7
8 void lock(lock_t *mutex) {
9     while (mutex->flag == 1) // проверить флаг
10        ; // активное ожидание (ничего не делать)
11     mutex->flag = 1;        // теперь установить его!
12 }
13
14 void unlock(lock_t *mutex) {
15     mutex->flag = 0;
16 }

```

Рис. 28.1 ❖ Первая попытка: простой флаг

К сожалению, в этом коде есть две проблемы: корректность и производительность. Проблему корректности легко разглядеть, стоит только при-выкнуть к размышлениям о конкурентном программировании. Рассмотрим чередование кода на рис. 28.2; предположим, что в начале `flag=0`.

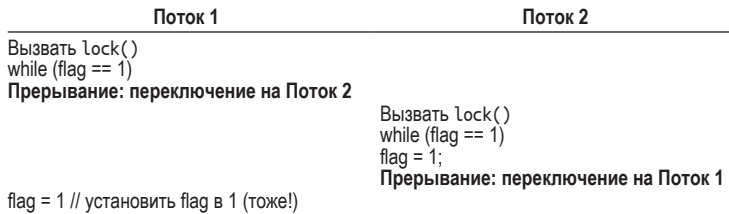


Рис. 28.2 ❖ Трассировка: взаимного исключения нет

Как видно из рисунка, если прерывания возникают в подходящий (скорее уж, неподходящий) момент, то легко может возникнуть ситуация, когда *оба* потока установили флаг в 1 и, значит, оба смогли войти в критическую секцию. Профессионалы называют такое поведение «плохим» – очевидно, что не выполнено самое главное требование: обеспечить взаимное исключение.

Проблема производительности, которую мы подробнее рассмотрим ниже, заключается в том, как именно поток ожидает захвата уже удерживаемой блокировки: он бесконечно проверяет значение флага, т. е. занимается **активным ожиданием** (spin-waiting). Активное ожидание – бессмысленное сжигание процессорного времени в ожидании освобождения блокировки другим потоком. Накладные расходы особенно высоки на машине с одним процессором, поскольку поток, которого мы с таким нетерпением ждем, даже не может работать (по крайней мере, пока не произойдет контекстное переключение)! Поэтому в процессе разработки более подходящих решений мы должны избегать такого рода потерь.

28.7. ПОСТРОЕНИЕ РАБОТОСПОСОБНЫХ СПИН-БЛОКИРОВОК С ПОМОЩЬЮ КОМАНДЫ ПРОВЕРКИ И УСТАНОВКИ

Поскольку при наличии нескольких процессоров запрещение прерываний не работает, как и простые подходы на основе команд загрузки и сохранения, конструкторы систем принялись изобретать аппаратную поддержку блокировки. Уже в самых ранних мультипроцессорных системах, в частности Burroughs B5000 в начале 1960-х годов [M82], такая поддержка была; ныне она имеется во всех системах, даже однопроцессорных.

Простейший для понимания элемент аппаратной поддержки – **команда проверки и установки** (test-and-set), или команда **атомарного обмена**¹. О том, что именно делает команда проверки и установки, дает представление следующий код на C:

```
1 int TestAndSet(int *old_ptr, int new) {
2     int old = *old_ptr; // выбрать значение old по адресу old_ptr
3     *old_ptr = new;     // сохранить new по адресу old_ptr
4     return old;         // вернуть old
5 }
```

Итак, команда проверки и установки возвращает старое значение, на которое указывал ptr, и одновременно записывает по этому адресу новое значение. Вся соль, конечно, в том, что эта последовательность операций выполняется **атомарно**. Свое название команда получила, потому что позволяет «проверить» старое значение (которое возвращается) и одновременно «установить» в память новое значение. Как выясняется, этой чуть более мощной команды уже достаточно для построения простой **спин-блокировки**, показанной на рис. 28.3. Мы сейчас объясним, как она устроена, а еще лучше – попробуйте сначала разобраться самостоятельно!

Давайте все-таки разберемся, почему эта блокировка работает. Сначала рассмотрим случай, когда поток вызывает функцию lock() в момент, когда никакой другой поток не удерживает блокировку, поэтому flag должна быть равна 0. Когда поток вызывает TestAndSet(flag, 1), функция возвращает старое значение флага, равное 0; поэтому вызывающий поток, который *проверяет* значение flag, не окажется в цикле while, а захватит блокировку. Кроме того, этот поток атомарно *установит* значение флага в 1, подтверждая тем самым, что удерживает блокировку. Перед выходом из критической секции поток вызовет unlock() и сбросит флаг в 0.

¹ В разных архитектурах команда проверки и установки называется по-разному. Например, в SPARC это команда загрузки и сохранения байта без знака (ldstub), а в x86 – команда обмена (xchg). Но мы будем придерживаться более общего названия: проверка и установка.

```

1 typedef struct __lock_t {
2     int flag;
3 } lock_t;
4
5 void init(lock_t *lock) {
6     // 0 означает, что блокировка свободна, 1 - что занята
7     lock->flag = 0;
8 }
9
10 void lock(lock_t *lock) {
11     while (TestAndSet(&lock->flag, 1) == 1)
12         ; // активное ожидание (ничего не делать)
13 }
14
15 void unlock(lock_t *lock) {
16     lock->flag = 0;
17 }

```

Рис. 28.3 ❖ Простая спин-блокировка
с применением команды проверки и установки

ОТСТУПЛЕНИЕ: АЛГОРИТМЫ ДЕККЕРА И ПЕТЕРСОНА

В 1960-х годах Дейкстра поставил проблему конкурентности перед своими друзьями, и один из них, математик Теодор Йозеф Деккер, нашел решение [D68]. В отличие от обсуждаемых в этой главе решений, требующих специальных команд и даже поддержки со стороны ОС, в **алгоритме Деккера** используются только команды загрузки и сохранения (в предположении, что они атомарны друг относительно друга, что было верно даже для раннего оборудования).

Решение Деккера впоследствии было усовершенствовано Петерсоном [P81]. И на этот раз использовались только команды загрузки и сохранения, а задача состояла в том, чтобы не допустить одновременного нахождения двух потоков в критической секции. Ниже приведен **алгоритм Петерсона** (для двух потоков), сможете ли вы в нем разобраться? Для чего используются переменные `flag` и `turn`?

```

int flag[2];
int turn;

void init() {
    flag[0] = flag[1] = 0;    // 1 -> поток хочет захватить блокировку
    turn = 0;                 // чья очередь (потока 0 или 1)?
}

void lock() {
    flag[self] = 1;           // self: идентификатор вызывающего потока
    turn = 1 - self;          // передать очередь другому потоку
    while ((flag[1-self] == 1) && (turn == 1 - self))
        ; // активное ожидание
}

void unlock() {
    flag[self] = 0;           // просто отменить свое намерение
}

```


По какой-то причине разработка блокировок, работающих без специальной аппаратной поддержки, на протяжении некоторого времени стала безумно популярной, поставив кучу проблем перед теоретиками. Конечно, все это направление оказалось бесполезным с осознанием того, что при наличии совсем небольшой поддержки со стороны оборудования (а такая поддержка вообще-то существовала уже в первых работах по мультипроцессированию) все становится куда проще. К тому же алгоритмы, подобные приведенному выше, не работают на современном оборудовании (из-за ослабленной модели непротиворечивости памяти), что делает их еще более бесполезными, чем прежде. Как и многие другие, эти исследования были погребены на свалке истории...

Второй случай возникает, когда какой-то поток уже захватил блокировку (т. е. `flag` равно 1). Тогда наш поток вызовет `lock()`, а затем `TestAndSet(flag, 1)`. Но на этот раз `TestAndSet()` вернет старое значение флага, равное 1 (потому что блокировка занята), и одновременно установит флаг в 1. Пока блокировка удерживается другим потоком, `TestAndSet()` будет возвращать 1, поэтому поток будет крутиться в цикле до тех пор, пока блокировка не освободится. Когда какой-то другой поток наконец сбросит флаг в 0, этот поток снова вызовет `TestAndSet()`, которая на этот раз вернет 0 и одновременно атомарно установит флаг в 1, а это будет означать, что поток захватил блокировку и вошел в критическую секцию.

Сделав **проверку** (старого значения блокировки) и **установку** (нового значения) одной атомарной операцией, мы гарантировали, что только один поток может захватить блокировку. И стало быть, создали работающий примитив взаимного исключения!

Теперь вы, вероятно, понимаете, почему блокировки этого типа обычно называются **спин-блокировками**. Она просто крутится (*spin*) в цикле, потребляя процессорное время, пока блокировка не станет доступна. Для правильной работы данного решения на одном процессоре необходим **вытесняющий планировщик** (который периодически прерывает поток по таймеру, чтобы дать возможность поработать другому потоку). Без вытеснения спин-блокировки на одном процессоре были бы бессмысленны, потому что крутящийся в цикле поток никогда не уступил бы CPU.

28.8. ОЦЕНКА СПИН-БЛОКИРОВОК

Оценим эффективность спин-блокировки по вышеупомянутым критериям. Самый важный аспект – **корректность**: обеспечивает ли блокировка взаимное исключение? Ответ утвердительный: спин-блокировка позволяет находиться в критической секции не более чем одному потоку.

Наш следующий критерий – **справедливость**. Насколько спин-блокировка справедлива по отношению к ожидающему потоку? Ответ, увы, неутешительный: не дает спин-блокировка никаких гарантий справедливости. В условиях конкуренции блокировка может крутиться в цикле бесконечно. Простые спин-блокировки (рассмотренные выше) несправедливы и могут приводить к зависанию потока.

Совет: взгляд на конкурентность как на вредоносный планировщик

Из этого примера можно вынести представление о взгляде на вещи, полезном для понимания конкурентного выполнения. Представьте, что вы **вредоносный планировщик**, который стремится прерывать потоки в самый неподходящий момент, чтобы воспрепятствовать нашим потугам сконструировать примитивы синхронизации. О, каким зловредным планировщиком должны вы стать! Пусть даже необходимая последовательность прерываний *маловероятна*, она все же *возможна*, а этого достаточно для демонстрации неработоспособности подхода. Думать, как злоумышленник, иногда полезно!

И последний критерий – **производительность**. Какова плата за использование спин-блокировки? Чтобы проанализировать этот вопрос, рассмотрим несколько случаев. Сначала представим, что потоки конкурируют за блокировку на одном процессоре, а потом – что они работают на нескольких процессорах.

Для спин-блокировок на одном CPU накладные расходы могут быть весьма велики; представьте, что поток, удерживающий блокировку, вытесняется, когда находится в критической секции. Планировщик может после этого выбрать любой другой поток (допустим, что таковых $N - 1$), и каждый из них может попытаться захватить блокировку. В таком случае каждый поток будет крутиться в цикле на протяжении своего временного кванта, не уступая CPU, – совершенно бесполезная трата процессорного времени.

Но при наличии нескольких CPU спин-блокировки работают вполне прилично (если количество потоков приблизительно равно количеству CPU). Будем рассуждать следующим образом: пусть поток А работает на CPU 1, а поток В – на CPU 2, и оба конкурируют за блокировку. Если поток А (CPU 1) захватит блокировку, а затем то же самое попытается сделать поток В, то В будет активно ожидать в цикле (на CPU 2). Однако критическая секция предположительно короткая, так что блокировка скоро освободится и будет захвачена потоком В. В этом случае активное ожидание на другом процессоре не потребляет слишком много времени и может оказаться эффективным решением.

28.9. Сравнить и обменять

Еще один аппаратный примитив, предоставляемый некоторыми системами, – команда **сравнить и обменять** (в архитектуре SPARC она называется `compare-and-swap`, а в архитектуре x86 – `compare-and-exchange`). Псевдокод этой команды на С показан на рис. 28.4.

Идея команды «сравнить и обменять» заключается в том, чтобы проверить, равно ли значение по адресу, на который указывает `ptr`, ожидаемому. Если да, то в память, адресуемую `ptr`, записывается новое значение, а если нет, то ничего и не делается. В любом случае возвращается значение по указанному адресу, что позволяет коду, вызвавшему команду, узнать, успешно она завершилась или нет.

```

1 int CompareAndSwap(int *ptr, int expected, int new) {
2     int actual = *ptr;
3     if (actual == expected)
4         *ptr = new;
5     return actual;
6 }

```

Рис. 28.4 ❖ Сравнить и обменять

Имея команду «сравнить и обменять», мы можем построить блокировку примерно так же, как с помощью команды «проверить и установить». Например, можно было бы просто заменить функцию `lock()` следующей:

```

1 void lock(lock_t *lock) {
2     while (CompareAndSwap(&lock->flag, 0, 1) == 1)
3         ; // активное ожидание
4 }

```

Остальной код точно такой же, как в примере выше. Работает он аналогично: проверяет, равен ли флаг 0, и если да, атомарно обменивает его с 1, захватывая таким образом блокировку. Потоки, которые пытаются захватить удерживаемую блокировку, будут крутиться в цикле, пока блокировка не освободится.

Если вы хотите знать, как реально написать на платформе x86 код блокировки, основанный на команде «сравнить и обменять» и допускающий вызов из C, то взгляните на следующую функцию ([S05]):

```

1 char CompareAndSwap(int *ptr, int old, int new) {
2     unsigned char ret;
3
4     // Заметим, что устанавливается значение байта, а не слова
5     __asm__ __volatile__ (
6         " lock\n"
7         " cmpxchgl %2,%1\n"
8         " sete %0\n"
9         : "=q" (ret), "=m" (*ptr)
10        : "r" (new), "m" (*ptr), "a" (old)
11        : "memory");
12    return ret;
13 }

```

Наконец, как вы, возможно, поняли, команда «сравнить и обменять» мощнее, чем «проверить и установить». Мы воспользуемся этим фактом в будущем, когда будем кратко обсуждать вопрос о **безблокировочной синхронизации** [H91]. Однако если требуется лишь сконструировать простую спин-блокировку, то ни одна команда не лучше другой.

28.10. ЗАГРУЗИТЬ ПО СВЯЗИ И СОХРАНИТЬ УСЛОВНО

На некоторых платформах предлагается пара команд, работающих совместно для организации критических секций. Например, в архитектуре MIPS [H93] имеются команды **загрузить по связи** (load-linked) и **сохранить условно** (store-conditional), совместное использование которых позволяет строить блокировки и другие конкурентные структуры. Псевдокод этих команд на C показан на рис. 28.5. В архитектурах Alpha, PowerPC и ARM имеются аналогичные команды [W09].

```

1 int LoadLinked(int *ptr) {
2     return *ptr;
3 }
4
5 int StoreConditional(int *ptr, int value) {
6     if (никто не обновлял *ptr с момента выполнения LoadLinked по этому адресу) {
7         *ptr = value;
8         return 1; // успешно!
9     } else {
10        return 0; // не удалось обновить
11    }
12 }
```

Рис. 28.5 ❖ Команды «загрузить по связи» и «сохранить условно»

Загрузка по связи работает, как обычная команда загрузки, – просто выбирает значение из памяти и помещает его в регистр. Главное отличие кроется в команде условного сохранения, которая завершается успешно (и изменяет значение по адресу, из которого только что произошла загрузка), если в промежутке не было никакого сохранения по этому адресу. В случае успеха команда условного сохранения возвращает 1 и записывает значение *value* по адресу, на который указывает *ptr*. В случае же неудачи значение по адресу *ptr* не изменяется и возвращается 0.

И вот вам задача: подумайте, как сконструировать блокировку с помощью команд «загрузить по связи» и «сохранить условно». А потом изучите код на рис. 28.6, в котором предложено простое решение. Давайте!

Интерес представляет только код `lock()`. Сначала поток активно ожидает, пока флаг примет значение 0 (это будет означать, что блокировка свободна). После этого поток пытается захватить блокировку с помощью команды условного сохранения; если получилось, значит, поток атомарно изменил значение флага на 1 и может войти в критическую секцию.

Обратите внимание, из-за чего команда условного сохранения может завершиться неудачно. Один поток вызывает `lock()` и выполняет команду «загрузить по связи», которая возвращает 0, если блокировка свободна. Но проинформировать условное сохранение он не успевает, потому что его прервал другой

поток, который тоже выполнил команду загрузки по связи, тоже получил 0 и продолжает выполнение. В этот момент оба потока выполнили загрузку по связи и оба пытаются выполнить условное сохранение. Но весь смысл этих команд в том, что только один поток сможет успешно установить флаг в 1 и тем самым захватить блокировку, а попытка второго выполнить условное сохранение терпит провал (потому что другой поток успел обновить флаг между своей загрузкой по связи и условным сохранением), и потому он вынужден снова пытаться захватить блокировку.

```

1 void lock(lock_t *lock) {
2     while (1) {
3         while (LoadLinked(&lock->flag) == 1)
4             ; // ждать, пока не 0
5         if (StoreConditional(&lock->flag, 1) == 1)
6             return; // если установка в 1 прошла успешно, то все сделано
7                     // иначе начать все сначала
8     }
9 }
10
11 void unlock(lock_t *lock) {
12     lock->flag = 0;
13 }

```

Рис. 28.6 ❖ Применение пары команд LL/SC для построения блокировки

СОВЕТ: ЧЕМ МЕНЬШЕ КОДА, ТЕМ ЛУЧШЕ (ЗАКОН ЛАУЭРА)

Программисты любят хвастаться, как много кода они написали для решения какой-то задачи. Но это принципиально порочный подход. Хвастаться надо тем, как *мало* кода вы написали для решения задачи. Короткий лаконичный код всегда предпочтительнее: его легче понять, и он содержит меньше ошибок. Как говорил Хью Лауэр, обсуждая устройство операционной системы Pilot: «Если бы у тех же людей было в два раза больше времени, они создали бы не менее хорошую систему, написав вдвое меньше кода» [L81]. Мы назовем это правило **законом Лауэра** и рекомендуем его хорошенько запомнить. В следующий раз, когда захотите похвалиться, как много кода написали при выполнении проекта, прикусите язык, а еще лучше перепишите код, сделав его максимально ясным и лаконичным.

Несколько лет назад студент нашего курса Дэвид Кейпл предложил более короткую форму этого кода – она понравится тем из вас, кто обожает закороченные условия. Подумайте, почему оба варианта эквивалентны. Но второй-то уж точно короче!

```

1 void lock(lock_t *lock) {
2     while (LoadLinked(&lock->flag)||!StoreConditional(&lock->flag, 1))
3         ; // активное ожидание
4 }

```

28.11. ВЫБРАТЬ И ПРИБАВИТЬ

И последний аппаратный примитив – команда **выбрать и прибавить** (fetch-and-add), которая атомарно увеличивает значение по указанному адресу и возвращает старое. Псевдокод этой команды на С показан ниже:

```
1 int FetchAndAdd(int *ptr) {
2     int old = *ptr;
3     *ptr = old + 1;
4     return old;
5 }
```

Далее мы воспользуемся командой «выбрать и прибавить» для конструирования интересной **билетной блокировки** (ticket lock), впервые описанной в работе Мэллора-Крамми и Скотта [MS91]. Код блокировки и разблокировки показан на рис. 28.7.

```
1 typedef struct __lock_t {
2     int ticket;
3     int turn;
4 } lock_t;
5
6 void lock_init(lock_t *lock) {
7     lock->ticket = 0;
8     lock->turn = 0;
9 }
10
11 void lock(lock_t *lock) {
12     int myturn = FetchAndAdd(&lock->ticket);
13     while (lock->turn != myturn)
14         ; // активное ожидание
15 }
16
17 void unlock(lock_t *lock) {
18     lock->turn = lock->turn + 1;
19 }
```

Рис. 28.7 ❖ Билетная блокировка

Для построения блокировки в этом решении вместо одной переменной используются две: ticket (билет) и turn (очередность). Основная операция проста: поток, желающий захватить блокировку, сначала выполняет атомарную команду «выбрать и прибавить» над значением билета и далее рассматривает это значение как свою «очередность» (myturn). Затем lock->turn используется, чтобы определить, чья сейчас очередь; если данного потока (myturn == turn), значит, он может войти в критическую секцию. Для разблокировки нужно просто увеличить очередность на 1, так чтобы следующий ожидающий поток мог войти в критическую секцию.

Отметим важное отличие этого решения от предыдущих: оно гарантирует продвижение всем потокам. Коль скоро потоку был выдан билет, то в буду-

щем он обязательно будет запланирован (после того как потоки, оказавшиеся в очереди перед ним, посетят критическую секцию и освободят блокировку). Предыдущие попытки такой гарантии не давали: поток, пытающийся выполнить команду проверки и установки (например), мог крутиться в цикле вечно, безутешно наблюдая за тем, как другие потоки захватывают и освобождают блокировку.

28.12. Слишком много активного ожидания: и как с этим быть?

Наши аппаратные блокировки просты (всего несколько строчек кода) и работают (это даже можно строго доказать), т. е. два важных свойства любой системы мы имеем. Но в некоторых ситуациях эти решения могут быть крайне неэффективны. Пусть работают два потока на одном процессоре. Допустим, что поток 0 находится в критической секции и, стало быть, удерживает блокировку, но в этот момент, по несчастливому стечению обстоятельств, оказывается прерван. Теперь поток 1 пытается захватить блокировку, но обнаруживает, что она занята. Тогда он переходит к активному ожиданию. И ждет. И еще ждет. И еще немного. Наконец, срабатывает таймер, поток 0 возобновляется, освобождает блокировку, и в конечном итоге (будем надеяться, что со второго раза) потоку 1 не придется так долго ждать, и он сможет захватить блокировку. Таким образом, всякий раз, как поток оказывается в такой ситуации, он вынужден впустую потратить весь свой временной квант, только и занимаясь проверкой значения, которое в принципе не может измениться! Все становится еще хуже, когда количество N потоков, конкурирующих за блокировку, растет; при этом будет бездарно потрачено $N - 1$ временных квантов в ожидании момента, когда один-единственный поток освободит блокировку.

Существо проблемы: как избежать активного ожидания

Как разработать блокировку, которая не будет бесполезно тратить время, крутясь в цикле?

Одной аппаратной поддержки для решения этой проблемы недостаточно. Необходима еще и поддержка со стороны ОС! Посмотрим, как это можно сделать.

28.13. Простой подход: уступи

Аппаратная поддержка позволила продвинуться довольно далеко: мы имеем работоспособные блокировки и даже (как показывает билетная блокировка) справедливое отношение к конкурирующим потокам. Но проблема остается

ся: что делать, когда контекстное переключение происходит в критической области и потоки начинают крутиться в бесконечном цикле, ожидая, когда же прерванный и удерживающий блокировку поток наконец возобновит работу?

Наша первая попытка проста и дружелюбна: если собираешься перейти в режим активного ожидания, лучше сразу уступи процессор другому потоку. Этот подход показан на рис. 28.8.

```

1 void init() {
2     flag = 0;
3 }
4
5 void lock() {
6     while (TestAndSet(&flag, 1) == 1)
7         yield(); // уступить CPU
8 }
9
10 void unlock() {
11     flag = 0;
12 }
```

Рис. 28.8 ❖ Блокировка на основе команды «проверить и установить» и уступки

Здесь мы предполагаем, что операционная система предоставляет примитив `yield()`, который поток может вызвать, чтобы уступить процессор другому потоку. Поток может находиться в одном из трех состояний (выполняется, готов или заблокирован), а системный вызов `yield` просто переводит вызывающий поток из состояния **выполняется** в состояние **готов**, тем самым давая возможность поработать другому потоку. Иначе говоря, уступающий процесс сознательно отказывается от процессора.

Вернемся к примеру двух потоков на одном CPU; в этом случае основанное на уступке решение работает отлично. Если поток вызвал `lock()` и обнаружил, что блокировка удерживается другим потоком, то он просто уступает CPU, так чтобы другой поток смог закончить работу в критической секции.

Теперь рассмотрим случай, когда потоков много (скажем, 100) и все они конкурируют за блокировку. Если один поток захватил блокировку, после чего оказался вытеснен, то каждый из оставшихся 99 вызовет `lock()`, обнаружит, что блокировка занята, и уступит CPU. В предположении, что планировщик циклический, все 99 потоков должны будут проделать этот трюк «запустись и уступи», прежде чем процессор перейдет к удерживающему блокировку потоку. Это, конечно, лучше, чем активное ожидание (тогда было бы впустую потрачено 99 временных квантов), но все равно дорого: стоимость контекстного переключения велика, так что напрасно израсходовано много времени.

Хуже того, мы еще не затронули проблему зависания. А ведь поток может попасть в бесконечный цикл уступки, беспомощно наблюдая, как другие потоки входят в критическую секцию и выходят из нее. Нужен какой-то подход, который позволит разрешить эту проблему.

28.14. ОЧЕРЕДИ: ЗАСЫПАНИЕ ВМЕСТО АКТИВНОГО ОЖИДАНИЯ

Проблема предыдущих подходов заключается в том, что они слишком многое оставляют на волю слепого случая. Планировщик определяет, какой поток будет выполняться следующим; если он примет неудачное решение, то запустится поток, который либо активно ожидает блокировку (первый подход), либо немедленно уступает CPU (второй подход). В любом случае открывается возможность для непроизводительных затрат и зависания.

Поэтому нам нужен явный контроль над тем, какой поток получит блокировку, после того как текущий владелец освободит ее. Для этого потребуется чуть больше поддержки со стороны ОС, а также очередь для хранения потоков, ожидающих возможности захватить блокировку.

Для простоты воспользуемся поддержкой, предоставляемой Solaris в виде двух системных вызовов: `park()` усыпляет вызвавший поток, а `unpark(threadID)` пробуждает поток, заданный своим идентификатором `threadID`. С помощью этих двух функций можно построить блокировку, которая усыпляет поток, попытавшийся захватить занятую блокировку, и пробуждает его, когда блокировка освободится. На рис. 28.9 показан пример возможного использования этих примитивов.

Здесь есть два интересных момента. Во-первых, мы объединили старую идею проверки и установки с явной очередью потоков, ожидающих блокировку, чтобы повысить эффективность. Во-вторых, очередь позволяет управлять тем, кто получит блокировку следующим, и тем самым избежать зависаний.

Обратите внимание, как используется поле `guard`; по существу это спин-блокировка, защищающая манипуляции с флагом и очередью, которые нужны для реализации блокировки. При таком подходе активное ожидание не исключено полностью; поток может быть прерван в процессе захвата или освобождения блокировки, и тогда другие потоки должны будут активно ждать, когда его выполнение возобновится. Но время, проведенное в ожидании, крайне ограничено (всего несколько команд в коде блокировки и разблокировки, а не вся определенная пользователем критическая секция), поэтому такое решение может оказаться приемлемым.

Далее вы, возможно, обратили внимание, что внутри `lock()`, когда поток не может захватить блокировку (она уже занята), он добавляет себя в очередь (функция `gettid()` возвращает идентификатор текущего потока), сбрасывает `guard` в 0 и уступает CPU. Вопрос читателю: что произойдет, если блокировка `guard` освобождается *после*, а не до вызова `park()`? Подсказка: нехорошо будет.

Вы, наверное, заметили интересный факт: флаг не сбрасывается в 0, когда просыпается какой-то другой поток. Почему? Это не ошибка, а суровая необходимость! Проснувшийся поток выглядит так, как будто он вернулся из `park()`, но в этот момент он не владеет блокировкой `guard` и потому даже пытаться не должен установить флаг в 1. Таким образом, мы просто передаем блокировку от потока, только что освободившего ее, следующему желающему, в промежутке флаг не сбрасывается в 0.

```

1 typedef struct __lock_t {
2     int flag;
3     int guard;
4     queue_t *q;
5 } lock_t;
6
7 void lock_init(lock_t *m) {
8     m->flag = 0;
9     m->guard = 0;
10    queue_init(m->q);
11 }
12
13 void lock(lock_t *m) {
14     while (TestAndSet(&m->guard, 1) == 1)
15         ; // захватить блокировку guard с помощью активного ожидания
16     if (m->flag == 0) {
17         m->flag = 1; // блокировка захвачена
18         m->guard = 0;
19     } else {
20         queue_add(m->q, getpid());
21         m->guard = 0;
22         park();
23     }
24 }
25
26 void unlock(lock_t *m) {
27     while (TestAndSet(&m->guard, 1) == 1)
28         ; // захватить блокировку guard с помощью активного ожидания
29     if (queue_empty(m->q))
30         m->flag = 0; // освободить блокировку; она никому не нужна
31     else
32         unpark(queue_remove(m->q)); // удержать блокировку (для следующего потока!)
33     m->guard = 0;
34 }

```

Рис. 28.9 ❖ Блокировка на основе очередей,
команды «проверить и установить», уступки и пробуждения

Наконец, отметим возможное состояние гонки в этом решении, прямо перед вызовом `park()`. Поток собирается заснуть, предполагая, что должен спать до тех пор, пока блокировку не освободят. Если в этот момент произойдет переключение на другой поток (например, удерживающий блокировку) и он освободит блокировку, то может возникнуть проблема – последующее выполнение `park()` первым потоком может привести к засыпанию навсегда – эту проблему иногда называют **гонкой между пробуждением и ожиданием**.

В Solaris для решения этой проблемы добавлен третий системный вызов: `setpark()`. Вызвав эту функцию, поток сообщает, что *готов* заснуть. Если после этого он будет прерван и другой поток вызовет `unpark`, до того как первый

успел вызвать `park`, то последующий вызов `park` немедленно вернет управление, вместо того чтобы усыпить поток. Код функции `lock()` нужно изменить совсем чуть-чуть:

```
1      queue_add(m->q, gettid());
2      setpark(); // новый код
3      m->guard = 0;
```

Отступление:

ЕЩЕ ОДНА ПРИЧИНА ИЗБЕГАТЬ АКТИВНОГО ОЖИДАНИЯ – ИНВЕРСИЯ ПРИОРИТЕТОВ

Убедительная причина избегать спин-блокировок – производительность: как было описано в основном тексте, если поток прерывается в момент, когда владеет блокировкой, то спин-блокировки в других потоках будут тратить много времени на ожидание ее освобождения. Но оказывается, что в некоторых системах есть еще одна интересная причина избегать спин-блокировок, а именно корректность. Эта проблема **инверсии приоритетов** уже вышла на межпланетный уровень, поскольку проявилась как на Земле [M15], так и на Марсе [R97]!

Пусть в системе имеется два потока. У Потока 2 (T2) высокий приоритет планирования, а у Потока 1 (T1) поменьше. Будем предполагать, что в случае, когда оба потока готовы к выполнению, планировщик всегда предпочитает T2, а T1 может выполняться, только когда у T2 такой возможности нет (например, потому что он блокирован в ожидании завершения ввода-вывода).

А теперь опишем проблему. Предположим, что T2 по какой-то причине блокирован. Поэтому T1 начинает выполняться, захватывает блокировку и входит в критическую секцию. Затем T2 разблокируется (например, потому что операция ввода-вывода завершилась), и планировщик незамедлительно отдает ему процессор (отбирая его у T1). T2 пытается захватить блокировку, но не может (ее удерживает T1), поэтому переходит к активному ожиданию. Так как это спин-блокировка, T2 будет крутиться в цикле вечно, и система зависнет.

К сожалению, отказ от использования спин-блокировок не устраняет проблему инверсии. Рассмотрим три потока, T1, T2 и T3, и предположим, что T3 имеет высший приоритет, а T1 – низший. Допустим, что T1 захватил блокировку. Затем запускается T3 и, поскольку его приоритет выше, чем у T1, сразу же вытесняет T1. T3 пытается захватить блокировку, удерживаемую T1, но вынужден ждать, поскольку T1 все еще удерживает ее. Если сейчас запустится T2, то, имея приоритет, больший, чем у T1, он вытеснит T1 и начнет работать. T3 с приоритетом, большим, чем у T2, висит, ожидая T1, который может вообще никогда не получить процессор, потому что сейчас работает T2. Не правда ли, грустно наблюдать, как могущественный T3 не может продвинуться ни на шаг, тогда как низший по рангу T2 распоряжается процессором? Высокий приоритет не спас от прозябания.

Есть несколько подходов к решению проблемы инверсии приоритетов. В частном случае, когда причиной являются спин-блокировки, можно отказаться от них (это описано ниже). А в общем случае высокоприоритетный поток, ожидающий низкоприоритетного, может временно повысить приоритет последнего, дав ему возможность поработать и тем самым преодолеть инверсию. Эта техника называется **наследованием приоритетов**. И последнее решение самое простое – назначать всем потокам одинаковые приоритеты.

Другое возможное решение – передать ответственность ядру. Тогда ядро могло бы предпринять меры, чтобы атомарно освободить блокировку и удалить из очереди работающий поток.

28.15. РАЗНЫЕ ОС, РАЗНАЯ ПОДДЕРЖКА

До сих пор мы видели только один тип поддержки, оказываемой ОС с целью повысить эффективность блокировки в библиотеке для работы с потоками. Другие ОС предлагают аналогичную поддержку, но детали могут варьироваться.

Например, в Linux имеются **фьютексы** (futex) – интерфейс, аналогичный Solaris, но с большей функциональностью внутри ядра. Именно, с каждым фьютексом ассоциирована ячейка в физической памяти и расположенная внутри ядра очередь. Поток вызывает функции фьютекса (описанные ниже) для засыпания и пробуждения.

Всего имеется две функции. Обращение к `futex_wait(address, expected)` усыпляет вызвавший поток, при условии что значение по адресу `address` равно ожидаемому. Если это не так, то функция возвращает управление немедленно. Обращение к `futex_wake(address)` пробуждает один из потоков, ждущих в очереди. Использование этих системных вызовов для реализации мьютексов в Linux показано на рис. 28.10.

Этот фрагмент кода, взятый из файла `lowlevellock.h`, входящего в библиотеку `nptl` (часть библиотеки `gnu libc`) [L09], интересен по нескольким причинам. Во-первых, используется всего одно целое число для отслеживания сразу двух вещей: свободна блокировка или занята (старший бит) и количества потоков, ожидающих в очереди (остальные биты). Таким образом, если число отрицательно, то блокировка занята (поскольку старший бит определяет знак числа и он установлен).

Во-вторых, из этого фрагмента видно, как оптимизируется самый типичный случай, когда за блокировку никто не конкурирует; если всего один поток захватывает и освобождает блокировку, то почти ничего не нужно делать (атомарная проверка и установка для захвата блокировки и атомарное сложение для ее освобождения).

Попробуйте самостоятельно разобраться в остальных деталях этой «реальной» блокировки. Если получится, станете мастером блокировки в Linux.

```

1 void mutex_lock (int *mutex) {
2     int v;
3     /* Бит 31 был сброшен, мы захватили мьютекс (это быстрая ветвь) */
4     if (atomic_bit_test_set (mutex, 31) == 0)
5         return;
6     atomic_increment (mutex);
7     while (1) {
8         if (atomic_bit_test_set (mutex, 31) == 0) {
9             atomic_decrement (mutex);
10            return;
11        }
12        /* Теперь придется подождать. Сначала убедимся, что значение фьютекса,
13        за которым мы наблюдаем, отрицательно (т. е. он занят). */
14        v = *mutex;
15        if (v >= 0)
16            continue;
17        futex_wait (mutex, v);
18    }
19 }
20
21 void mutex_unlock (int *mutex) {
22     /* Прибавление 0x80000000 к счетчику дает 0 тогда и только тогда, когда
23     нет других заинтересованных потоков */
24     if (atomic_add_zero (mutex, 0x80000000))
25         return;
26
27     /* Существуют другие потоки, ожидающие этого мьютекса,
28     разбудить один из них. */
29     futex_wake (mutex);
30 }

```

Рис. 28.10 ❖ Фьютексная блокировка в Linux

28.16. ДВУХФАЗНАЯ БЛОКИРОВКА

И последнее замечание: реализованное в Linux решение несет в себе дух старого подхода, который на протяжении многих лет то принимался, то отвергался. Он известен, по крайней мере, со времен блокировок Дэма в начале 1960-х годов [М82] и ныне называется **двухфазной блокировкой**. Этот подход признает, что активное ожидание может быть полезным, особенно если блокировку вот-вот освободят. Поэтому на первом этапе функция активно ждет, правда очень недолго, в надежде, что удастся захватить блокировку.

Но если блокировку не удалось захватить на первом этапе, то наступает второй этап, на котором вызывающий поток засыпает и пробуждается, только когда блокировка освободится. Показанная выше блокировка в Linux – разновидность такого подхода всего с одной итерацией цикла ожидания; обобщением был бы цикл, рассчитанный на фиксированное время, по истечении которого средства **фьютекса** используются для засыпания.

Двухфазная блокировка – еще один пример **гибридного** подхода, когда сочетание двух хороших идей может дать отличный результат. Правда, может и не дать, – это зависит от многих факторов, включая аппаратное окружение, количество потоков и другие детали рабочей нагрузки. Как всегда, реализация одной универсальной блокировки, пригодной для всех случаев, – весьма трудная задача.

28.17. РЕЗЮМЕ

Мы описали, как в наши дни конструируются реальные блокировки: нужна поддержка со стороны оборудования (в форме специальных команд) и со стороны операционной системы (например, в форме примитивов `park()` и `unpark()` в Solaris или **фьютексов** в Linux). Конечно, детали различаются, а точный код реализации блокировки обычно высокооптимизирован. Если вас интересуют дополнительные сведения, загляните в исходный код Solaris или Linux – это потрясающе увлекательное чтение [L09, S09]. Также см. работу David et al. [D+13], в которой имеется сравнение стратегий блокировки в современных мультипроцессорах.

Литература

[D+13] «Everything You Always Wanted to Know about Synchronization but Were Afraid to Ask» by Tudor David, Rachid Guerraoui, Vasileios Trigonakis. SOSP '13, Nemaquin Woodlands Resort, Pennsylvania, November 2013. *Отличная статья, в которой сравнивается много разных способов построения блокировок с использованием аппаратных примитивов. Удивительно, сколько идей работают на современном оборудовании.*

[D68] «Cooperating sequential processes» by Edsger W. Dijkstra. 1968. Доступно по адресу <http://www.cs.utexas.edu/users/EWD/ewd01xx/EWD123.PDF>. *Одна из ранних основополагающих статей. Обсуждается оригинальная постановка проблемы конкурентности Дейкстры и ее решение, предложенное Деккером.*

[H93] «MIPS R4000 Microprocessor User's Manual» by Joe Heinrich. Prentice-Hall, June 1993. Доступно по адресу http://cag.csail.mit.edu/raw/documents/R4400_Uman_book_Ed2.pdf. *Старое руководство пользователя по архитектуре MIPS. Скачайте, пока его не убрали.*

[H91] «Wait-free Synchronization» by Maurice Herlihy. ACMTOPLAS, Volume 13: 1, January 1991. *Знаковая статья, в которой впервые описан другой подход к построению конкурентных структур данных. Из-за сложности изложенных идей для их усвоения и практического воплощения потребовалось много времени.*

[L81] «Observations on the Development of an Operating System» by Hugh Lauer. SOSP '81, Pacific Grove, California, December 1981. *Обязательный для прочтения рассказ о разработке ОС Pilot для ранних ПК. Любопытно и местами неожиданно.*

[L09] «glibc 2.9 (include Linux pthreads implementation)» by Many authors. Доступно по адресу <http://ftp.gnu.org/gnu/glibc>. *В частности, обратите внимание на подкаталог nptl, в котором находится большая часть современной поддержки pthread в Linux.*

[M82] «The Architecture of the Burroughs B5000: 20 Years Later and Still Ahead of the Times?» by A. Mayer. 1982. Доступно по адресу www.ajwm.net/amayer/papers/B5000.html. *«Это (RDLK) неделимая операция, которая читает и записывает в ячейку памяти». Таким образом, RDLK – не что иное, как команда проверить и установить! Дэйв Дэм создал спин-блокировки («Buzz Locks») и двухфазные блокировки («Dahm Locks»).*

[M15] «OSSpinLock Is Unsafe» by J. McCall. mjtsai.com/blog/2015/12/16/oss핀lock-is-unsafe. *Вызов OSSpinLock в Mac небезопасен, если используются потоки с разными приоритетами – можно заикнуться навсегда! Поэтому будьте осторожны, фанатики Mac, даже ваша могучая система может оказаться неидеальной...*

[MS91] «Algorithms for Scalable Synchronization on Shared-Memory Multiprocessors» by John M. Mellor-Crummey and M. L. Scott. ACM TOCS, Volume 9, Issue 1, February 1991. *Великолепный и весьма дотошный обзор различных алгоритмов блокировки. Однако не используется никакой поддержки со стороны операционных систем, только причудливые аппаратные команды.*

[P81] «Myths About the Mutual Exclusion Problem» by G. L. Peterson. Information Processing Letters, 12(3), pages 115–116, 1981. *Здесь изложен алгоритм Петерсона.*

[R97] «What Really Happened on Mars?» by Glenn E. Reeves. research.microsoft.com/en-us/um/people/mbj/Mars_Pathfinder/Authoritative_Account.html. *Описание инверсии приоритетов, имевшей место в проекте Mars Pathfinder. Корректность конкурентного кода имеет значение, особенно в космосе!*

[S05] «Guide to porting from Solaris to Linux on x86» by Ajay Sood, April 29, 2005. Доступно по адресу <http://www.ibm.com/developerworks/linux/library/l-solar/>.

[S09] «OpenSolaris Thread Library» by Sun. Код: src.opensolaris.org/source/xref/onnv/onnv-gate/usr/src/lib/libc/port/threads/synch.c. *Довольно интересно, хотя кто знает, что будет теперь, после того как Oracle приобрел компанию Sun. Спасибо Майку Свифту за ссылочку.*

[W09] «Load-Link, Store-Conditional» by Many authors. en.wikipedia.org/wiki/Load-Link/Store-Conditional. *Можете поверить – мы ссылаемся на Википедию? Однако именно там мы нашли эту информацию, и было бы нечестно не упомянуть источник. К тому же статья полезна, в ней перечислены команды для разных архитектур: ld_l/stl_c и ldq_l/stq_c (Alpha), lwarx/stwxc (PowerPC), ll/sc (MIPS) и ldrex/strex (ARM). Вообще-то, Википедия – удивительное место, так что не будьте уж слишком бескомпромиссны, ладно?*

[WG00] «The SPARC Architecture Manual: Version 9» by D. Weaver, T. Germond. SPARC International, 2000. <http://www.sparc.org/standards/SPARCV9.pdf>. *Допол-*

нительные сведения об атомарных операциях см. по адресу developers.sun.com/solaris/articles/atomic_sparc/.

Домашнее задание (эмуляция)

Программа `x86.ru` дает возможность посмотреть, как различный порядок чередования потоков провоцирует или предотвращает состояния гонки. Об аргументах программы см. в файле `README`.

Вопросы

1. Изучите файл `flag.s`. В нем «реализована» блокировка с одним флагом в памяти. Понятен ли вам ассемблерный код?
2. Работает ли `flag.s` с аргументами по умолчанию? Задайте флаги `-M` и `-R` для трассировки переменных и регистров (и флаг `-c`, чтобы увидеть их значения). Можете ли вы предсказать конечное значение в переменной `flag`?
3. Измените значение регистра `%bx` с помощью флага `-a` (например, `-a bx=2, bx=2`, если запущено всего два потока). Что делает код? Как изменится ответ на предыдущий вопрос?
4. Присвойте `bx` большое значение в каждом потоке, а затем с помощью флага `-i` задайте разные частоты прерываний. При каких значениях получается плохой результат? А хороший при каких?
5. Теперь изучите программу `test-and-set.s`. Сначала попробуйте понять код, в котором используется команда `xchg` для создания простого примитива блокировки. Как производится захват блокировки? А как производится освобождение?
6. Теперь запустите код, изменив величину интервала между прерываниями (`-i`) и позаботившись о том, чтобы было выполнено несколько итераций цикла. Всегда ли код работает, как вы ожидаете? Бывает ли так, что CPU используется неэффективно? Как можно выразить это количественно?
7. С помощью флага `-P` сгенерируйте конкретные тесты кода блокировки. Например, спланируйте так, чтобы блокировка успешно захватывалась в первом потоке, а во втором производилась попытка захвата. Все ли работает правильно? Что еще следовало бы протестировать?
8. Теперь взгляните на код в файле `peterson.s`, реализующий алгоритм Петерсона (описанный на врезке). Понятен ли вам код?
9. Запустите этот код с разными значениями флага `-i`. Какие виды поведения вы наблюдаете? Не забудьте правильно задать идентификаторы потоков (например, `-a bx=0, bx=1`), поскольку это предполагается в коде.
10. Можете ли вы с помощью управления планированием (флаг `-P`) «доказать», что код работает правильно? Какие случаи необходимо рассмотреть? Подумайте о взаимном исключении и предотвращении взаимоблокировок.

11. Перейдем к коду билетной блокировки в файле `ticket.s`. Соответствует ли он коду, приведенному в тексте главы? Запустите программу с флагами `-a bx=1000,bx=1000` (при этом каждый поток проходит критическую секцию 1000 раз). Что происходит: много ли времени потоки проводят в активном ожидании блокировки?
12. Как поведет себя код, если увеличить число потоков?
13. Рассмотрите файл `yield.s`, в котором команда `yield` позволяет одному потоку уступить процессор (в реальности это должен быть примитив ОС, но для простоты предположим, что с задачей справляется команда). При каких обстоятельствах `test-and-set.s` транжирит время на активное ожидание, а `yield.s` – нет? Сколько команд сэкономлено? При каких сценариях имеет место такая экономия?
14. Наконец, рассмотрите файл `test-and-test-and-set.s`. Что делает эта блокировка? Какой экономии она позволяет достичь по сравнению с `test-and-set.s`?

Глава 29

Конкурентные структуры данных с блокировками

Прежде чем расстаться с блокировками, опишем, как они используются в некоторых популярных структурах данных. Блокировки добавляются в структуру данных, чтобы ее можно было использовать в потоках, благодаря этому структура становится **потокобезопасной**. Разумеется, корректность и производительность структуры данных зависят от того, как именно добавлены блокировки.

Существо проблемы: как добавлять блокировки в структуры данных

Пусть дана конкретная структура данных. Как добавить в нее блокировки, чтобы она работала корректно? И как при этом обеспечить высокую производительность в условиях, когда к структуре одновременно, т. е. **конкурентно**, обращается много потоков?

Конечно, мы не сможем рассмотреть все структуры данных, равно как и все методы реализации конкурентности, поскольку эта тема исследовалась годами и ей посвящены тысячи (буквально) научных статей. Но мы надеемся дать введение, которого будет достаточно, чтобы понять направление мысли, а также предложим источники для самостоятельного изучения. Мы считаем обзор Муара и Шавита [MS04] великолепным источником информации.

29.1. Конкурентные счетчики

Одна из самых простых структур данных – счетчик. Она используется сплошь и рядом, а интерфейс проще некуда. Неконкурентный счетчик определен на рис. 29.1.

Простой, но немасштабируемый

Как видим, несинхронизированный счетчик – это тривиальная структура данных, для реализации которой нужно совсем немного кода. Обратимся

теперь к следующей задаче: как сделать этот код **потокобезопасным**? Наше решение показано на рис. 29.2.

```

1 typedef struct __counter_t {
2     int value;
3 } counter_t;
4
5 void init(counter_t *c) {
6     c->value = 0;
7 }
8
9 void increment(counter_t *c) {
10     c->value++;
11 }
12
13 void decrement(counter_t *c) {
14     c->value--;
15 }
16
17 int get(counter_t *c) {
18     return c->value;
19 }
```

Рис. 29.1 ❖ Счетчик без блокировок

```

1 typedef struct __counter_t {
2     int value;
3     pthread_mutex_t lock;
4 } counter_t;
5
6 void init(counter_t *c) {
7     c->value = 0;
8     Pthread_mutex_init(&c->lock, NULL);
9 }
10
11 void increment(counter_t *c) {
12     Pthread_mutex_lock(&c->lock);
13     c->value++;
14     Pthread_mutex_unlock(&c->lock);
15 }
16
17 void decrement(counter_t *c) {
18     Pthread_mutex_lock(&c->lock);
19     c->value--;
20     Pthread_mutex_unlock(&c->lock);
21 }
22
23 int get(counter_t *c) {
24     Pthread_mutex_lock(&c->lock);
25     int rc = c->value;
26     Pthread_mutex_unlock(&c->lock);
27     return rc;
28 }
```

Рис. 29.2 ❖ Счетчик с блокировками

Этот конкурентный счетчик прост и работает корректно. На самом деле он следует паттерну проектирования, типичному для самых простых конкурентных структур данных: добавляется одна блокировка, которая захватывается при вызове любой функции, манипулирующей структурой данных, и освобождается при возврате из этой функции. В этом смысле имеется сходство со структурой данных, построенной с применением **мониторов** [BH73], когда блокировки захватываются и освобождаются автоматически при вызове методов объекта и возврате из них.

Сейчас мы уже имеем работающую конкурентную структуру данных. Проблема только в производительности. Если структура данных работает слишком медленно, то, возможно, добавлением всего одной блокировки ограничиться не удастся; подобные оптимизации (если они необходимы) – тема этой главы. Отметим, что если структура данных работает достаточно быстро, то больше ничего делать не нужно! Ни к чему изобретать хитроумный механизм, если хватает и простого.

Чтобы оценить издержки простого подхода, мы запустили тест производительности, в котором каждый поток обновляет один разделяемый счетчик фиксированное число раз; количество потоков можно изменять. На рис. 29.3 показано общее время работы, когда число потоков изменяется от 1 до 4, а каждый поток обновляет счетчик миллион раз. Эксперимент был поставлен на компьютере iMac с четырьмя процессорами Intel i5 2.7 ГГц; мы надеемся, что чем больше процессоров, тем больше объем выполненной работы в единицу времени.

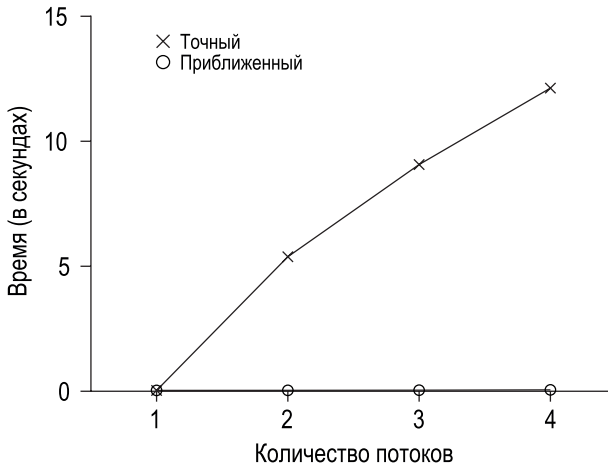


Рис. 29.3 ❖ Производительность традиционного и приближенного счетчиков

Верхняя линия на рисунке (с меткой «Точный») показывает, что производительность синхронизированного счетчика масштабируется плохо. В то время как один поток может выполнить миллион обновлений счетчика за ничтожно малое время (приблизительно 0.03 с), при наличии всего двух потоков время резко возрастает (более 5 секунд!). А с увеличением числа потоков все становится только хуже.

В идеале мы хотели бы, чтобы несколько потоков завершалось на нескольких процессорах так же быстро, как один поток на одном. Достижение этой цели называется **идеальным масштабированием**; хотя работы больше, но выполняется она параллельно, поэтому общее время не увеличивается.

Масштабируемый подсчет

Как это ни удивительно, исследователи изучали построение лучше масштабируемых счетчиков в течение многих лет [MS04]. Еще удивительнее тот факт, что масштабируемые счетчики – не пустая прихоть, поскольку недавняя работа по анализу производительности операционных систем [B+10] показала, что без масштабируемого подсчета некоторые рабочие нагрузки в Linux испытывают серьезные проблемы с производительностью на многоядерных машинах.

Для решения этой проблемы разработано много методов. Мы опишем один подход – **приближенный счетчик** [C06]. В этом случае один логический счетчик представляется несколькими локальными *физическими* счетчиками, по одному для каждого процессорного ядра, а также одним глобальным счетчиком. Конкретно, на машине с четырьмя CPU будет четыре логических и один глобальный счетчик. Помимо этих счетчиков есть еще блокировки: по одной для каждого локального счетчика¹ и еще одна для глобального.

Идея приближенного подсчета такова. Когда поток, работающий на данном ядре, хочет увеличить счетчик, он увеличивает свой локальный счетчик; доступ к локальному счетчику синхронизирован с помощью соответствующей локальной блокировки. Поскольку у каждого CPU свой собственный локальный счетчик, потоки на разных CPU могут обновлять локальные счетчики, не встречая конкуренции, поэтому операция обновления счетчика масштабируема.

Однако чтобы поддерживать глобальный счетчик в актуальном состоянии (на случай, если какой-то поток захочет прочитать его значение), локальные значения периодически передаются в глобальный счетчик. Для этого захватывается глобальная блокировка, и глобальный счетчик увеличивается на величину локального, после чего локальный счетчик сбрасывается в 0.

Частота передачи локального счетчика в глобальный определяется порогом S . Чем меньше S , тем ближе поведение счетчика к описанному выше немасштабируемому случаю, а чем больше S , тем более масштабируемым является счетчик, но тем сильнее значение глобального счетчика может отличаться от истинного. Чтобы получить точное значение, можно просто захватить глобальную и все локальные блокировки (в определенном порядке, чтобы избежать взаимоблокировки), но такое решение не масштабируется.

Для иллюстрации этого алгоритма рассмотрим пример на рис. 29.4. Здесь порог S равен 5, а потоки на каждом из четырех процессоров обновляют свои локальные счетчики $L_1, \dots, L_4$. Показано также значение глобального счетчика (G). На каждом временном шаге локальный счетчик может увеличиться; как только локальное значение достигает порога S , оно переносится в глобальный счетчик, а локальный счетчик сбрасывается.

Время	L_1	L_2	L_3	L_4	G
0	0	0	0	0	0
1	0	0	1	1	0
2	1	0	2	1	0
3	2	0	3	1	0
4	3	0	3	2	0
5	4	1	3	3	0
6	5 → 0	1	3	4	5 (из L_1)
7	0	2	4	5 → 9	10 (из L_4)

Рис. 29.4 ❖ Трассировка приближенных счетчиков

¹ Локальные блокировки нужны, поскольку мы предполагаем, что на каждом ядре работает несколько потоков. Если бы каждое ядро исполняло только один поток, то локальные блокировки не понадобились бы.

Нижняя линия на рис. 29.3 (с меткой «Приближенный») показывает производительность приближенных счетчиков для порога $S = 1024$. Результат отличный – на четыре миллиона обновлений счетчика ушло едва ли больше времени, чем на обновление счетчика на одном процессоре миллион раз.

На рис. 29.5 иллюстрируется важность выбора порогового значения S в случае, когда четыре потока увеличивают счетчик по миллиону раз на четырех

```

1 typedef struct __counter_t {
2     int global;                // глобальный счетчик
3     pthread_mutex_t glock;     // глобальная блокировка
4     int local[NUMCPUS];        // локальные счетчики (по одному на cpu)
5     pthread_mutex_t llock[NUMCPUS]; // ... и блокировки
6     int threshold;             // частота обновления
7 } counter_t;
8
9 // init: запомнить порог, инициализировать блокировки, значения
10 // всех локальных и глобального счетчиков
11 void init(counter_t *c, int threshold) {
12     c->threshold = threshold;
13     c->global = 0;
14     pthread_mutex_init(&c->glock, NULL);
15     int i;
16     for (i = 0; i < NUMCPUS; i++) {
17         c->local[i] = 0;
18         pthread_mutex_init(&c->llock[i], NULL);
19     }
20 }
21
22 // update: обычно просто захватывает локальную блокировку и обновляет локальный
23 // счетчик, но когда он достигает порога, захватывает глобальную блокировку
24 // и переносит локальное значение в глобальный счетчик
25 void update(counter_t *c, int threadID, int amt) {
26     int cpu = threadID % NUMCPUS;
27     pthread_mutex_lock(&c->llock[cpu]);
28     c->local[cpu] += amt;           // предполагается, что amt > 0
29     if (c->local[cpu] >= c->threshold) { // перенести в глобальный
30         pthread_mutex_lock(&c->glock);
31         c->global += c->local[cpu];
32         pthread_mutex_unlock(&c->glock);
33         c->local[cpu] = 0;
34     }
35     pthread_mutex_unlock(&c->llock[cpu]);
36 }
37
38 // get: вернуть значение глобального счетчика (возможно, неточное)
39 int get(counter_t *c) {
40     pthread_mutex_lock(&c->glock);
41     int val = c->global;
42     pthread_mutex_unlock(&c->glock);
43     return val; // всего лишь приближение!
44 }

```

Рис. 29.5 ❖ Реализация приближенного счетчика

процессорах. Если S мало, то производительность низкая (но глобальный счетчик почти всегда точен). При большом S производительность отличная, но глобальный счетчик отстает (однако не более чем на величину S , умноженную на количество процессоров). Именно компромисс между точностью и производительностью и лежит в основе приближенного счетчика.

Набросок реализации приближенного счетчика показан на рис. 29.6. Прочитайте код, а еще лучше поэкспериментируйте, чтобы понять, как он работает.

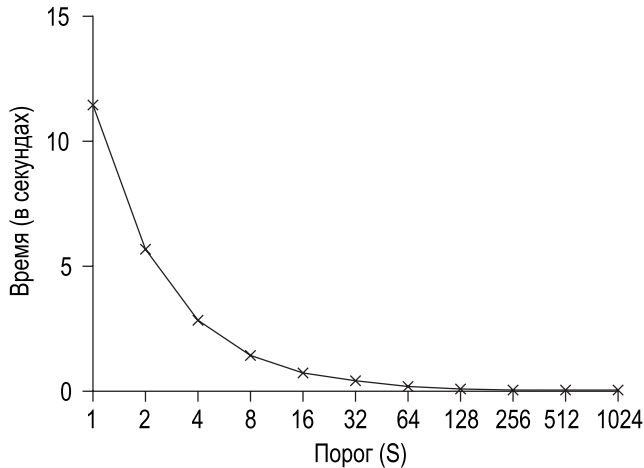


Рис. 29.6 ❖ Масштабируемость приближенных счетчиков

29.2. КОНКУРЕНТНЫЕ СВЯЗНЫЕ СПИСКИ

Далее мы рассмотрим более сложную структуру – связный список. Снова начнем с базового подхода. Для простоты мы опустим некоторые очевидные функции, которые должны присутствовать в списке, а сконцентрируем внимание на конкурентной вставке; оставляем читателю возможность поразмыслить о поиске, удалении и т. д. На рис. 29.7 приведен код этой предельно упрощенной структуры данных.

Как видим, код просто захватывает блокировку при входе в функцию вставки и освобождает ее перед выходом. Небольшая сложность возникает в редком случае, когда `malloc()` завершается ошибкой, тогда код должен освободить блокировку, перед тем как возвращать код ошибки.

Показано, что такой поток управления в исключительных ситуациях провоцирует массу ошибок; в недавнем исследовании исправлений ядра Linux обнаружилось, что огромная доля дефектов (примерно 40 %) находится как раз на таких редких путях (это наблюдение вдохновило нас на собственное исследование, в ходе которого мы исправили в файловой системе Linux все пути, на которых происходит ошибка выделения памяти, получив в итоге более надежную систему [S+11]).

```

1 // базовая структура узла
2 typedef struct __node_t {
3     int key;
4     struct __node_t *next;
5 } node_t;
6
7 // базовая структура списка (по одной на каждый список)
8 typedef struct __list_t {
9     node_t *head;
10    pthread_mutex_t lock;
11 } list_t;
12
13 void List_Init(list_t *L) {
14     L->head = NULL;
15     pthread_mutex_init(&L->lock, NULL);
16 }
17
18 int List_Insert(list_t *L, int key) {
19     pthread_mutex_lock(&L->lock);
20     node_t *new = malloc(sizeof(node_t));
21     if (new == NULL) {
22         perror("malloc");
23         pthread_mutex_unlock(&L->lock);
24         return -1; // fail
25     }
26     new->key = key;
27     new->next = L->head;
28     L->head = new;
29     pthread_mutex_unlock(&L->lock);
30     return 0; // success
31 }
32
33 int List_Lookup(list_t *L, int key) {
34     pthread_mutex_lock(&L->lock);
35     node_t *curr = L->head;
36     while (curr) {
37         if (curr->key == key) {
38             pthread_mutex_unlock(&L->lock);
39             return 0; // успешно
40         }
41         curr = curr->next;
42     }
43     pthread_mutex_unlock(&L->lock);
44     return -1; // ошибка
45 }

```

Рис. 29.7 ❖ Конкурентный связный список

Итак, вопрос: можно ли переписать функции вставки и обновления, сохранив корректность конкурентной вставки, но избежав случая, в котором возникновение ошибки требует добавления разблокировки на некотором пути?

Ответ утвердительный. Именно, мы можем немного реорганизовать код, так чтобы захват и освобождение блокировки окружали только саму критическую секцию в коде вставки, а в коде поиска существовал общий путь выхода. Первая идея работает, потому что часть кода вставки на самом деле блокировать не нужно; в предположении, что сама `malloc()` потокобезопасна, все потоки могут обращаться к ней, не опасаясь возникновения гонки или других ошибок, связанных с конкурентностью. И лишь при обновлении разделяемого списка блокировку необходимо удерживать. Детали внесенных изменений см. на рис. 29.8.

```

1 void List_Init(list_t *L) {
2     L->head = NULL;
3     pthread_mutex_init(&L->lock, NULL);
4 }
5
6 void List_Insert(list_t *L, int key) {
7     // синхронизация не нужна
8     node_t *new = malloc(sizeof(node_t));
9     if (new == NULL) {
10         perror("malloc");
11         return;
12     }
13     new->key = key;
14
15     // защитить блокировкой только критическую секцию
16     pthread_mutex_lock(&L->lock);
17     new->next = L->head;
18     L->head = new;
19     pthread_mutex_unlock(&L->lock);
20 }
21
22 int List_Lookup(list_t *L, int key) {
23     int rv = -1;
24     pthread_mutex_lock(&L->lock);
25     node_t *curr = L->head;
26     while (curr) {
27         if (curr->key == key) {
28             rv = 0;
29             break;
30         }
31         curr = curr->next;
32     }
33     pthread_mutex_unlock(&L->lock);
34     return rv; // возвращается либо код успеха, либо код ошибки
35 }

```

Рис. 29.8 ❖ Конкурентный связный список: переписанный вариант

Что касается функции поиска, то мы проделали простое преобразование кода, чтобы после выхода из цикла поиска попасть на общий путь выхода.

Это уменьшает количество точек захвата и освобождения блокировок в коде, а значит, и шансы случайно внести ошибку (например, забыть об освобождении блокировки перед возвратом).

Масштабирование связанных списков

Хоть мы и получили конкурентный связный список, но снова оказались в ситуации, когда он плохо масштабируется. Для повышения масштабируемости списка была изучена техника **блокировки вперехват** (hand-over-hand locking) (или **цепной блокировки** [lock coupling]) [MS04].

Идея довольно проста. Нужно завести не одну блокировку на весь список, а по одной на каждый узел. В процессе обхода списка мы сначала захватываем блокировку следующего узла, а затем освобождаем блокировку текущего (откуда и название «вперехват»).

Теоретически блокировка связного списка вперехват имеет смысл, поскольку позволяет увеличить степень конкурентности. Но на практике трудно сделать такую структуру более быстрой, чем структуру с одной блокировкой, поскольку накладные расходы на захват и освобождение блокировок для каждого узла непомерно велики. Даже для очень больших списков и при большом числе потоков одновременный обход несколькими потоками вряд ли окажется быстрее, чем захват одной блокировки, выполнение операции и освобождение блокировки. Быть может, имеет смысл изучить какой-нибудь гибридный подход (например, захватывать новую блокировку через каждые N узлов).

СОВЕТ: КОНКУРЕНТНЕЕ НЕОБЯЗАТЕЛЬНО БЫСТРЕЕ

Если придуманная вами схема привносит большие накладные расходы (например, захватывает и освобождает блокировку не один раз, а многократно), то, возможно, повышение степени конкурентности окажется не столь важным. Простые схемы обычно работают хорошо, особенно если дорогостоящие функции вызываются редко. Увеличение количества блокировок и сложности может стать причиной падения производительности. Впрочем, есть только один способ узнать точно: реализовать обе версии (простую, но менее конкурентную, и сложную, но более конкурентную) и измерить производительность. Тут не обманешь: либо ваша идея дает ускорение программы, либо нет.

СОВЕТ: БУДЬТЕ ВНИМАТЕЛЬНЫ К БЛОКИРОВКАМ НА РАЗНЫХ ПУТЯХ ВЫПОЛНЕНИЯ

Общий совет, полезный при написании не только конкурентного, но и любого другого кода, – внимательно следить за изменениями потока управления, ведущими к возврату из функции, выходу из программы и другим способам прекращения выполнения функции. Многие функции в начале работы захватывают блокировку, выделяют память или производят еще какие-то операции, приводящие к изменению состояния, поэтому при возникновении ошибки функция должна вернуться к прежнему состоянию перед возвратом – и вот тут-то можно напортачить. Так что лучше всего структурировать код так, чтобы минимизировать число таких ситуаций.

29.3. КОНКУРЕНТНЫЕ ОЧЕРЕДИ

Как вы уже знаете, всегда есть стандартный способ сделать структуру данных конкурентной: добавить большую блокировку. При обсуждении очереди мы опустим этот подход, веря, что вы сами разберетесь.

А рассмотрим вместо этого несколько более конкурентную очередь, спроектированную Майклом и Скоттом [MS98]. Соответствующие структуры данных и код показаны на рис. 29.9.

```

1 typedef struct __node_t {
2     int value;
3     struct __node_t *next;
4 } node_t;
5
6 typedef struct __queue_t {
7     node_t *head;
8     node_t *tail;
9     pthread_mutex_t headLock;
10    pthread_mutex_t tailLock;
11 } queue_t;
12
13 void Queue_Init(queue_t *q) {
14     node_t *tmp = malloc(sizeof(node_t));
15     tmp->next = NULL;
16     q->head = q->tail = tmp;
17     pthread_mutex_init(&q->headLock, NULL);
18     pthread_mutex_init(&q->tailLock, NULL);
19 }
20
21 void Queue_Enqueue(queue_t *q, int value) {
22     node_t *tmp = malloc(sizeof(node_t));
23     assert(tmp != NULL);
24     tmp->value = value;
25     tmp->next = NULL;
26
27     pthread_mutex_lock(&q->tailLock);
28     q->tail->next = tmp;
29     q->tail = tmp;
30     pthread_mutex_unlock(&q->tailLock);
31 }
32
33 int Queue_Dequeue(queue_t *q, int *value) {
34     pthread_mutex_lock(&q->headLock);
35     node_t *tmp = q->head;
36     node_t *newHead = tmp->next;
37     if (newHead == NULL) {
38         pthread_mutex_unlock(&q->headLock);
39         return -1; // очередь была пуста
40     }
41     *value = newHead->value;
42     q->head = newHead;
43     pthread_mutex_unlock(&q->headLock);
44     free(tmp);
45     return 0;
46 }
```

Рис. 29.9 ❖ Конкурентная очередь Майкла и Скотта

Обратите внимание, что в этом коде две блокировки: для начала и для конца очереди. Цель в том, чтобы обеспечить конкурентность операции помещения в очередь и извлечения из очереди. В типичном случае функция помещения в очередь обращается только к блокировке конца, а функция извлечения из очереди – только к блокировке начала.

Майкл и Скотт предложили добавить фиктивный узел (память для него выделяется в коде инициализации очереди), который гарантирует разделение операций над началом и концом. Изучите код, а еще лучше – введите его, выполните и измерьте производительность, чтобы глубже понять, как он работает.

Очереди часто используются в многопоточных приложениях. Но рассмотренный здесь тип очереди (только с блокировками) зачастую удовлетворяет потребности таких программ не в полной мере. Более развитая ограниченная очередь, которая позволяет потоку перейти в состояние ожидания, если очередь пуста или заполнена, – тема углубленного изучения в следующей главе, посвященной условным переменным. Не пропустите!

29.4. КОНКУРЕНТНАЯ ХЕШ-ТАБЛИЦА

В заключение обсудим простую и широко распространенную конкурентную структуру данных – хеш-таблицу. Мы сосредоточимся на простой хеш-таблице фиксированного размера; чтобы реализовать возможность изменения размера, нужно еще немного потрудиться, но это мы оставим в качестве упражнения для читателя (извините).

```

1 #define BUCKETS (101)
2
3 typedef struct __hash_t {
4     list_t lists[BUCKETS];
5 } hash_t;
6
7 void Hash_Init(hash_t *H) {
8     int i;
9     for (i = 0; i < BUCKETS; i++) {
10         List_Init(&H->lists[i]);
11     }
12 }
13
14 int Hash_Insert(hash_t *H, int key) {
15     int bucket = key % BUCKETS;
16     return List_Insert(&H->lists[bucket], key);
17 }
18
19 int Hash_Lookup(hash_t *H, int key) {
20     int bucket = key % BUCKETS;
21     return List_Lookup(&H->lists[bucket], key);
22 }
```

Рис. 29.10 ❖ Конкурентная хеш-таблица

Наша конкурентная хеш-таблица основана на разработанных выше конкурентных списках и работает на удивление хорошо. Причина в том, что вместо единственной блокировки на всю структуру данных мы используем по одной блокировке на каждую ячейку хеша (представленную списком). Поэтому можно одновременно выполнять несколько операций над таблицей.

На рис. 29.11 показана производительность конкурентных обновлений хеш-таблицы (от 10 000 до 50 000 обновлений в каждом из четырех потоков на том же четырехпроцессорном iMac). Для сравнения показана также производительность связного списка (с одной блокировкой). Как видно из графика, эта простая конкурентная таблица великолепно масштабируется, в отличие от связного списка.

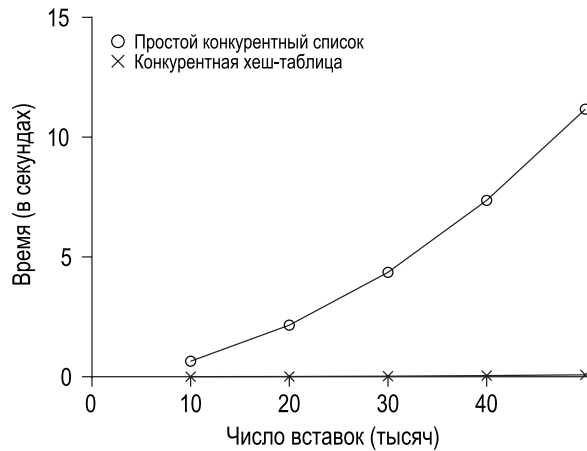


Рис. 29.11 ❖ Масштабирование хеш-таблицы

Совет: избегайте преждевременной оптимизации (закон Кнута)

Проектируя конкурентную структуру данных, начните с самого простого подхода, т. е. заведите одну большую блокировку для синхронизации доступа. Это, скорее всего, позволит получить *правильное* решение, а если обнаружится, что оно недостаточно производительное, то можно будет уточнить его. Структура должна быть настолько быстрой, насколько необходимо, а добиваться большего не имеет смысла. Напомним знаменитое высказывание **Кнута**: «Преждевременная оптимизация – корень всех зол».

Во многих операционных системах, в т. ч. Sun OS и Linux, на начальных этапах перехода на мультипроцессоры использовалась единственная блокировка. В Linux у нее даже было имя: **большая блокировка ядра** (big kernel lock – **BKL**). На протяжении многих лет этот простой подход считался хорошим, но когда многопроцессорные системы стали обыденностью, ограничение – один активный поток в ядре в каждый момент времени – превратилось в узкое место. Тогда наконец настало время включить оптимизацию с целью повысить степень конкурентности. В Linux был принят прямолинейный подход: заменить одну блокировку несколькими. Разработчики Sun поступили более радикально: создали новую операционную систему под маркой Solaris, в которой конкурентность пронизывала все с самого начала. Если хотите узнать больше об этих удивительных системах, почитайте книги по устройству ядра Linux и Solaris [BC05, MM00].

29.5. РЕЗЮМЕ

Мы познакомились с несколькими конкурентными структурами данных: счетчиками, списками, очередями и, наконец, с самой распространенной – хеш-таблицей. По ходу дела мы извлекли несколько важных уроков: что при изменении потока управления нужно внимательно следить за захватом и освобождением блокировок; что повышение степени конкурентности необязательно означает повышение производительности; что проблемами производительности нужно заниматься, только когда они действительно существуют. Последний пункт – избегать **преждевременной оптимизации** – должен накрепко запомнить любой разработчик, озабоченный производительностью: ни к чему тратить силы на ускорение какой-то операции, если при этом общая производительность приложения не повысится.

Конечно, наше введение в высокопроизводительные структуры было очень поверхностным. Дополнительные сведения см. в блестящем обзоре Маура и Шавита [MS04], где приведены ссылки и на другие источники. Быть может, вас заинтересуют иные структуры данных (например, B-деревья), в таком случае самое подходящее место – курс по базам данных. Возможно, вам будут интересны методы, в которых традиционные блокировки вообще не используются, такие **безблокировочные структуры данных** мы вскользь затронем в главе, посвященной типичным ошибкам, связанным с конкурентностью, но, откровенно говоря, эта тема – отдельная отрасль знаний, требующая больше внимания, чем мы можем уделить ей в этой скромной книге. Если интересно, займитесь самостоятельными изысканиями (как всегда!).

Литература

[B+10] «An Analysis of Linux Scalability to Many Cores» by Silas Boyd-Wickizer, Austin T. Clements, Yandong Mao, Aleksey Pesterev, M. Frans Kaashoek, Robert Morris, Nickolai Zeldovich . OSDI '10, Vancouver, Canada, October 2010. *Изучается вопрос о производительности Linux на многоядерных машинах, а также некоторые простые решения. Описан **небрежный счетчик** для решения одной разновидности проблемы масштабируемого подсчета.*

[BH73] «Operating System Principles» by Per Brinch Hansen. Prentice-Hall, 1973. Доступно по адресу <http://portal.acm.org/citation.cfm?id=540365>. *Одна из первых книг по операционным системам, намного опередившая свое время. Впервые описан монитор как примитив конкурентности.*

[BC05] «Understanding the Linux Kernel (Third Edition)» by Daniel P. Bovet and Marco Cesati. O'Reilly Media, November 2005. *Классическая книга о ядре Linux. Обязательно прочитайте.*

[C06] «The Search For Fast, Scalable Counters» by Jonathan Corbet. February 1, 2006. Доступно по адресу <https://lwn.net/Articles/170003>. *На сайте LWN много замечательных статей о последних новостях Linux. В этой кратко описан алгоритм масштабируемого приближенного подсчета; прочитайте ее и другие статьи, если хотите быть в курсе событий в Linux.*

[L+13] «A Study of Linux File System Evolution» by Lanyue Lu, Andrea C. Arpaci-Dusseau, Remzi H. Arpaci-Dusseau, Shan Lu. FAST '13, San Jose, CA, February 2013. *Наша статья, в которой изучаются все исправления в файловых системах Linux за последние десять лет. Описывается много интересных находок – почитайте! Но далась эта статья тяжело – наш бедный магистрант, Лэнью Лю, вынужден был глазами просмотреть все исправления и понять, что они делают.*

[MS98] «Nonblocking Algorithms and Preemption-safe Locking on by Multiprogrammed Shared-memory Multiprocessors». M. Michael, M. Scott. Journal of Parallel and Distributed Computing, Vol. 51, No. 1, 1998. *Профессор Скотт со своими студентами на протяжении многих лет был на переднем крае исследований по конкурентным алгоритмам и структурам данных. Загляните на его веб-страницу, прочитайте многочисленные статьи и книги, если хотите узнать больше.*

[MS04] «Concurrent Data Structures» by Mark Moir and Nir Shavit. In Handbook of Data Structures and Applications (Editors D. Metha and S.Sahni). Chapman and Hall/CRC Press, 2004. Доступно по адресу www.cs.tau.ac.il/~shanir/concurrent-data-structures.pdf. *Краткий, но относительно полный справочник по конкурентным структурам данных. Хотя некоторые последние работы в этой области не представлены (все-таки книга вышла довольно давно), остается невероятно полезным источником.*

[MM00] «Solaris Internals: Core Kernel Architecture» by Jim Mauro and Richard McDougall. Prentice Hall, October 2000. *Книга про Solaris. Обязательно почитайте ее, если хотите узнать о чем-нибудь, кроме Linux.*

[S+11] «Making the Common Case the Only Case with Anticipatory Memory Allocation» by Swaminathan Sundararaman, Yupu Zhang, Sriram Subramanian, Andrea C. Arpaci-Dusseau, Remzi H. Arpaci-Dusseau. FAST '11, San Jose, CA, February 2011. *Наша работа по удалению потенциально способных привести к ошибке операций выделения памяти из путей выполнения в ядре. Благодаря выделению необходимой памяти до начала работы мы предотвращаем ошибки в глубинах подсистемы хранения.*

Домашнее задание (код)

В этом домашнем задании вы поднаберетесь опыта в написании конкурентного кода и измерении его производительности. Умение создавать производительный код очень важно, поэтому немного практики никак не помешает.

Вопросы

1. Для начала повторите измерения, продемонстрированные в этой главе. Для измерения времени внутри программы пользуйтесь функцией `gettimeofday()`. Насколько точен этот таймер? Какой минимальный интервал он может измерить? Вы должны быть уверены в механизме его работы,

поскольку это понадобится для решения следующих задач. Можете также поинтересоваться другими таймерами, например счетчиком тактов процессора, реализованным командой `rdtsc` на платформе `x86`.

2. Теперь напишите простой конкурентный счетчик и измерьте время, необходимое для его многократного инкремента при росте количества потоков. Сколько процессоров в системе, на которой вы работаете? Зависит ли результат измерения от количества процессоров?
3. Реализуйте приближенный счетчик. Снова измерьте его производительность при увеличении количества потоков с разными порогами. Совпадают ли ваши результаты с приведенными в этой главе?
4. Напишите код связанного списка с блокировкой вперехват [MS04], описанной в этой главе. Сначала прочитайте оригинальную статью и разберитесь в алгоритме, а затем реализуйте его. Измерьте производительность. При каких условиях список с блокировкой вперехват работает лучше стандартного списка?
5. Возьмите свою любимую структуру данных, например B-дерево или еще более интересную. Реализуйте ее, начав с простой стратегии, допустим с одной блокировкой. Измерьте производительность при увеличении количества потоков.
6. Подумайте о более интересной стратегии блокировки для этой структуры данных. Реализуйте ее и измерьте производительность. Сравните с прямым подходом.

Глава 30

Условные переменные

До сих пор мы занимались понятием блокировки и показали, как можно их сконструировать при надлежащей поддержке со стороны оборудования и ОС. Увы, блокировки – не единственные примитивы, необходимые для построения конкурентных программ.

В частности, есть много случаев, когда поток хочет проверить, выполняется ли какое-то **условие**, прежде чем продолжить выполнение. Например, когда родительский поток проверяет, завершился ли его потомок (в этой конкретной ситуации часто применяется функция `join()`). Как реализовать такое ожидание? Взгляните на рис. 30.1.

```
1 void *child(void *arg) {
2     printf("потомок\n");
3     // XXX как показать, что мы все сделали?
4     return NULL;
5 }
6
7 int main(int argc, char *argv[]) {
8     printf("родитель: начало\n");
9     pthread_t c;
10    Pthread_create(&c, NULL, child, NULL); // создать потомка
11    // XXX как дождаться потомка?
12    printf("родитель: конец\n");
13    return 0;
14 }
```

Рис. 30.1 ❖ Родитель, ожидающий потомка

Мы хотели бы увидеть такую распечатку:

```
родитель: начало
потомок
родитель: конец
```

Можно было бы воспользоваться разделяемой переменной, как показано на рис. 30.2. Вообще говоря, такое решение работает, но чудовищно неэффективно, потому что родитель крутится в цикле и напрасно транжирит процессорное время. Хотелось бы, чтобы вместо этого родитель спал, пока не окажется выполненным условие, которого мы ждем (например, потомок закончил выполнение).

```

1 volatile int done = 0;
2
3 void *child(void *arg) {
4     printf("потомок\n");
5     done = 1;
6     return NULL;
7 }
8
9 int main(int argc, char *argv[]) {
10    printf("родитель: начало\n");
11    pthread_t c;
12    Pthread_create(&c, NULL, child, NULL); // создать потомка
13    while (done == 0)
14        ; // крутиться в цикле
15    printf("родитель: конец\n");
16    return 0;
17 }

```

Рис. 30.2 ❖ Родитель, ожидающий потомка

СУЩЕСТВО ПРОБЛЕМЫ: КАК ЖДАТЬ ВЫПОЛНЕНИЯ УСЛОВИЯ

В многопоточных программах поток часто должен дождаться выполнения какого-то условия, прежде чем продолжить. Простое решение – крутиться в цикле, пока условие не станет истинным, очень неэффективно, тратит много процессорного времени и в некоторых случаях может оказаться некорректным. И как же все-таки реализовать ожидание условия?

30.1. ОПРЕДЕЛЕНИЕ И ФУНКЦИИ

Чтобы дождаться выполнения условия, поток должен воспользоваться **условной переменной**. По существу, это явная очередь, в которую поток может поместить себя, если в данный момент некоторое **условие** ложно и требуется **ждать**, когда оно станет истинным. Другой поток, изменив это условие, может пробудить один или несколько ожидающих его потоков, дав им возможность продолжить выполнение (или, как говорят, **сигнализировав** условию). Идея восходит к использованию «частных семафоров» Дейкстрой [D68]; аналогичная идея впоследствии была названа «условной переменной» в работе Хоара по мониторам [H74].

Для объявления условной переменной с достаточно написать `pthread_cond_t c`; (замечание: необходимо еще инициализировать ее). Для условных переменных определены две операции: `wait()` и `signal()`. Функция `wait()` вызывается, когда поток хочет заснуть, а функция `signal()` – когда поток что-то изменил в состоянии программы и хочет пробудить спящий поток, ожидающий этого условия. В стандарте POSIX эти функции объявлены следующим образом:

```
pthread_cond_wait(pthread_cond_t *c, pthread_mutex_t *m);
pthread_cond_signal(pthread_cond_t *c);
```

Мы часто будем называть эти функции просто `wait()` и `signal()`. Обратите внимание, что `wait()` принимает мьютекс в качестве параметра и предполагает, что в момент вызова этот мьютекс захвачен. На `wait()` возлагается обязанность освободить блокировку и усыпить вызвавший поток (атомарно), а когда поток пробудится (после сигнала из другого потока), он должен заново захватить блокировку, перед тем как вернуть управление вызывающей стороне. Смысл этой процедуры в том, чтобы предотвратить некоторые состояния гонки, возможные, когда поток засыпает. Чтобы лучше понять все это, рассмотрим, как решается проблема ожидания дочернего потока (рис. 30.3).

```
1 int done = 0;
2 pthread_mutex_t m = PTHREAD_MUTEX_INITIALIZER;
3 pthread_cond_t c = PTHREAD_COND_INITIALIZER;
4
5 void thr_exit() {
6     Pthread_mutex_lock(&m);
7     done = 1;
8     Pthread_cond_signal(&c);
9     Pthread_mutex_unlock(&m);
10 }
11
12 void *child(void *arg) {
13     printf("потомок\n");
14     thr_exit();
15     return NULL;
16 }
17
18 void thr_join() {
19     Pthread_mutex_lock(&m);
20     while (done == 0)
21         Pthread_cond_wait(&c, &m);
22     Pthread_mutex_unlock(&m);
23 }
24
25 int main(int argc, char *argv[]) {
26     printf("родитель: начало\n");
27     pthread_t p;
28     Pthread_create(&p, NULL, child, NULL);
29     thr_join();
30     printf("родитель: конец\n");
31     return 0;
32 }
```

Рис. 30.3 ❖ Родитель, ожидающий потомка:
использование условной переменной

Необходимо рассмотреть два случая. В первом родитель создает дочерний поток, но по-прежнему владеет процессором (предполагается, что про-

цессор один) и, значит, немедленно вызывает `thr_join()`, чтобы дождаться завершения потомка. В этом случае он захватывает блокировку, проверяет, завершился ли потомок (не завершился), и засыпает, вызвав `wait()` (при этом блокировка освобождается). Рано или поздно потомок начнет выполняться, напечатает сообщение «потомок» и вызовет `thr_exit()`, чтобы разбудить родительский поток; этот код просто захватывает блокировку, устанавливает переменную состояния `done` в единицу и сигнализирует родителю о том, что пора пробуждаться. Наконец, родитель возобновляет выполнение (с точки, следующей за `wait()`, с захваченной блокировкой), освобождает блокировку и печатает сообщение «родитель: конец».

Во втором случае дочерний поток начинает работать сразу после создания, устанавливает `done` в 1, вызывает `signal`, чтобы пробудить спящий поток (но, поскольку такового нет, функция возвращает управление, ничего не сделав), и на этом завершается. Затем возобновляет работу родитель, вызывает `thr_join()`, видит, что `done` равно 1, поэтому ничего не ждет, а сразу возвращает управление.

И последнее замечание: обратите внимание, что родитель проверяет `done` в цикле `while`, а не однократно в предложении `if`. В данном случае это не обязательно, но, вообще говоря, лучше поступать именно так, а почему, мы скоро увидим.

Чтобы вы лучше поняли важность каждой части кода `thr_exit()` и `thr_join()`, рассмотрим несколько альтернативных реализаций. Во-первых, вы, возможно, недоумеваете, зачем нужна переменная `done`. Не достаточно ли кода, показанного ниже?

```

1 void thr_exit() {
2     Pthread_mutex_lock(&m);
3     Pthread_cond_signal(&c);
4     Pthread_mutex_unlock(&m);
5 }
6
7 void thr_join() {
8     Pthread_mutex_lock(&m);
9     Pthread_cond_wait(&c, &m);
10    Pthread_mutex_unlock(&m);
11 }
```

К сожалению, такое решение не годится. Допустим, что потомок сразу начинает выполняться и вызывает `thr_exit()`; в таком случае он просигнализирует, но никакого потока, ждущего данного условия, нет. Когда родитель начнет работать, он просто вызовет `wait` и зависнет, его никто и никогда не разбудит. Это показывает важность переменной состояния `done`, в ней запоминается значение, интересующее потоки. Засыпание, пробуждение и блокировка – все завязано на эту переменную.

А вот еще одна дефектная реализация. В ней мы вообразили, будто для ожидания и сигнализации никакая блокировка не нужна. Какая беда может здесь произойти? Напрягите мозги!

```

1 void thr_exit() {
2     done = 1;
3     Pthread_cond_signal(&c);
4 }
5
6 void thr_join() {
7     if (done == 0)
8         Pthread_cond_wait(&c);
9 }

```

Здесь проблема в тонком состоянии гонки. Именно если родитель вызовет `thr_join()` и затем проверит значение `done`, то увидит, что оно равно 0, и поэтому попытается заснуть. Но прямо перед тем как родитель погрузился в сон, его прервали, и начал работать потомок. Потомок устанавливает переменную состояния в 1 и сигнализирует, но никто этого сигнала не ждет, поэтому никакой поток не пробуждается. Возобновив выполнение, родитель заснет навечно – печальный итог.

Совет: сигнализируя, всегда удерживайте блокировку

Удерживать блокировку в момент сигнализации условной переменной, наверное, проще и лучше всего, пусть даже не всегда это необходимо. В примере выше показан случай, когда блокировку удерживать *обязательно*, иначе программа окажется некорректной. Бывают случаи, когда этого можно и не делать, но надежнее **всегда удерживать блокировку в момент сигнализации**.

Обратная сторона этого совета – удерживать блокировку при вызове `wait` – это и не совет вовсе, а обязательная семантика ожидания, которая предполагает, (а) что в момент вызова блокировка захвачена, (б) что блокировка освобождается перед усыплением процесса и (с) что блокировка повторно захватывается перед возвратом управления. Таким образом, совет можно представить в обобщенной форме: **удерживайте блокировку в момент сигнализации или ожидания**, и все у вас будет хорошо.

Надеемся, что на этом простом примере вы поняли основы правильного использования условных переменных. А чтобы закрепить навыки, рассмотрим пример посложнее: задачу о **производителе и потребителе**, или об **ограниченном буфере**.

30.2. Задача о производителе и потребителе (об ограниченном буфере)

Следующая задача синхронизации, которую мы рассмотрим в этой главе, называется проблемой **производителя и потребителя**, или проблемой **ограниченного буфера**; впервые она была поставлена Дейкстрой [D72]. Именно при решении этой задачи Дейкстра с сотрудниками изобрел обобщенный

семафор (который можно использовать и как блокировку, и как условную переменную) [D01]; ниже мы еще поговорим о семафорах подробнее.

Пусть имеется один или несколько потоков-производителей и один или несколько потоков-потребителей. Производители порождают элементы данных и помещают их в буфер, а потребители забирают элементы из буфера и что-то с ними делают.

Такая ситуация возникает во многих реальных системах. Например, в многопоточном веб-сервере производитель помещает HTTP-запросы в очередь работ (т. е. в ограниченный буфер), а потребитель выбирает запросы из очереди и обрабатывает их.

Ограниченный буфер используется также, когда выход одной программы по конвейеру передается на вход другой, например `grep foo file.txt | wc -l`. Здесь два процесса работают конкурентно: `grep` записывает строки файла `file.txt`, содержащие последовательность символов `foo`, в свой стандартный вывод, а оболочка Unix перенаправляет этот вывод в канал (созданный с помощью системного вызова **pipe**). Второй конец канала соединен со стандартным вводом процесса `wc`, который просто подсчитывает число строк во входном потоке и печатает результат. Таким образом, процесс `grep` является производителем, процесс `wc` – потребителем, а между ними находится ограниченный буфер, расположенный в памяти ядра. Вы же – просто довольный пользователь.

Поскольку ограниченный буфер – разделяемый ресурс, доступ к нему должен быть синхронизирован, иначе возможно состояние гонки. Чтобы лучше понять проблему, обратимся к коду.

Прежде всего нам нужен разделяемый буфер, в который производитель будет помещать данные и из которого потребитель будет их извлекать. Для простоты пусть это будет одно целое число (можете, конечно, представить, что это указатель на некоторую структуру данных). Существует две внутренние функции: одна помещает значение в буфер, другая извлекает из него значение. Детали показаны на рис. 30.4.

```

1 int buffer;
2 int count = 0; // в начальном состоянии пуст
3
4 void put(int value) {
5     assert(count == 0);
6     count = 1;
7     buffer = value;
8 }
9
10 int get() {
11     assert(count == 1);
12     count = 0;
13     return buffer;
14 }
```

Рис. 30.4 ❖ Функции `put` и `get` (версия 1)

Довольно просто, правда? Функция `put()` предполагает, что буфер пуст (и проверяет это с помощью утверждения), затем помещает в него значение и сообщает, что буфер полон, установив переменную `count` в 1. Функция `get()` делает прямо противоположное: сообщает, что буфер пуст (сбрасывает `count` в 0), и возвращает значение. Не важно, что сейчас разделяемый буфер содержит всего один элемент, позже мы обобщим его, превратив в очередь, которая может содержать несколько элементов, и это будет даже интереснее, чем кажется.

Теперь нужно написать функции, которые знают, когда можно помещать данные в буфер и извлекать из него. Условия очевидны: помещать данные можно, когда `count` равна 0 (т. е. буфер пуст), а извлекать – когда `count` равна 1 (т. е. буфер полон). Если код синхронизации написан так, что производитель может поместить данные в заполненный буфер или потребитель может извлечь данные из пустого буфера, значит, что-то неправильно (и в нашем коде в этом случае сработает утверждение).

Эта работа выполняется потоками двух типов: **производителями** и **потребителями**. На рис. 30.5 показан код производителя, который помещает целое число в разделяемый буфер `loops` раз, и потребитель, который извлекает данные из этого буфера (в бесконечном цикле) и всякий раз печатает полученное значение.

```

1 void *producer(void *arg) {
2     int i;
3     int loops = (int) arg;
4     for (i = 0; i < loops; i++) {
5         put(i);
6     }
7 }
8
9 void *consumer(void *arg) {
10    int i;
11    while (1) {
12        int tmp = get();
13        printf("%d\n", tmp);
14    }
15 }

```

Рис. 30.5 ❖ Потоки производителя и потребителя
(версия 1)

Неправильное решение

Теперь предположим, что имеется всего один производитель и один потребитель. Очевидно, что внутри функций `put()` и `get()` имеются критические секции, поскольку `put()` обновляет буфер, а `get()` читает из него. Но окружить этот код блокировкой недостаточно, необходимо кое-что еще. Неудивительно, что это «кое-что» – условные переменные. В первой (неправильной) вер-

сии (рис. 30.6) мы завели только одну условную переменную `cond` и ассоциированную с ней блокировку `mutex`.

```

1 int loops; // где-то необходимо инициализировать...
2 cond_t cond;
3 mutex_t mutex;
4
5 void *producer(void *arg) {
6     int i;
7     for (i = 0; i < loops; i++) {
8         Pthread_mutex_lock(&mutex);           // p1
9         if (count == 1)                       // p2
10             Pthread_cond_wait(&cond, &mutex); // p3
11         put(i);                               // p4
12         Pthread_cond_signal(&cond);          // p5
13         Pthread_mutex_unlock(&mutex);        // p6
14     }
15 }
16
17 void *consumer(void *arg) {
18     int i;
19     for (i = 0; i < loops; i++) {
20         Pthread_mutex_lock(&mutex);           // c1
21         if (count == 0)                       // c2
22             Pthread_cond_wait(&cond, &mutex); // c3
23         int tmp = get();                      // c4
24         Pthread_cond_signal(&cond);          // c5
25         Pthread_mutex_unlock(&mutex);        // c6
26         printf("%d\n", tmp);
27     }
28 }

```

Рис. 30.6 ❖ Производитель–потребитель:
одна условная переменная и предложение `if`

Рассмотрим логику сигнализации между производителями и потребителями. Производитель, желающий заполнить буфер, ожидает, пока он окажется пуст (p1–p3). У потребителя похожая логика, только ожидает он другого условия: буфер заполнен (c1–c3).

При наличии всего одного производителя и одного потребителя код на рис. 30.6 работает. Но если таких потоков больше (например, два потребителя), то проявляются две критические проблемы. Какие?

... (пауза на размышление) ...

Сначала разберемся с первой проблемой, она связана с предложением `if` перед ожиданием. Пусть имеется два потребителя (T_{c1} и T_{c2}) и один производитель (T_p). Сначала работает производитель (T_{c1}); он захватывает блокировку (c1), проверяет, есть ли данные в буфере (c2), обнаруживает, что нет, и ждет (c3) (освобождая блокировку).

Затем начинает работать производитель (T_p). Он захватывает блокировку (p1), проверяет, заполнен ли буфер (p2), обнаруживает, что нет, и помещает

в него данные (р4). Затем производитель сигнализирует о том, что буфер заполнен (р5). Важно, что при этом первый потребитель (T_{c1}) пробуждается ото сна в ожидании условной переменной и переходит в очередь готовых; теперь T_{c1} готов к выполнению (но еще не выполняется). Производитель продолжает работать, но, поняв, что буфер полон, засыпает (р6, р1–р3).

Вот тут-то и возникает проблема: встречается другой потребитель (T_{c2}) и забирает единственное значение из буфера (с1, с2, с4, с5, с6, пропуская ожидание в точке с3, потому что буфер заполнен). Предположим теперь, что возобновляется T_{c1} ; прямо перед тем как выйти из состояния ожидания, он заново захватывает блокировку и возвращает управление. Затем он вызывает `get()` (с4), но в буфере нет данных! Срабатывает утверждение – и вот она, ошибка! Очевидно, нужно было как-то предотвратить попытку T_{c1} потребить данные, потому что вмешавшийся T_{c2} забрал из буфера единственное находившееся там значение. На рис. 30.7 показано, какие действия предпринимает каждый поток, а также его состояние (Готов, Выполняется, Спит) в разные моменты времени.

T_{c1}	Состояние	T_{c2}	Состояние	T_{c3}	Состояние	Счетчик	Примечание
c1	Выполняется		Готов		Готов	0	
c2	Выполняется		Готов		Готов	0	
c3	Спит		Готов		Готов	0	Нечего получать
	Спит		Готов	p1	Выполняется	0	
	Спит		Готов	p2	Выполняется	0	
	Спит		Готов	p4	Выполняется	1	Буфер полон
	Готов		Готов	p5	Выполняется	1	T_{c1} пробудился
	Готов		Готов	p6	Выполняется	1	
	Готов		Готов	p1	Выполняется	1	
	Готов		Готов	p2	Выполняется	1	
	Готов		Готов	p3	Спит	1	Буфер полон, спать
	Готов	c1	Выполняется		Спит	1	T_{c2} встречается...
	Готов	c2	Выполняется		Спит	1	
	Готов	c4	Выполняется		Спит	0	...и забирает данные
	Готов	c5	Выполняется		Готов	0	T_p пробудился
	Готов	c6	Выполняется		Готов	0	
c4	Выполняется		Готов		Готов	0	Нет данных

Рис. 30.7 ❖ Трассировка потоков: неправильное решение (версия 1)

Причина проблемы проста: после того как производитель разбудил T_{c1} , но до того как T_{c1} начал работать, состояние ограниченного буфера изменилось (из-за T_{c2}). Полученный сигнал только пробуждает поток, это не более чем *предположение* о том, что состояние мира могло измениться (в данном случае в буфер было помещено значение), но не гарантия того, что когда разбуженный поток начнет работать, состояние *все еще* будет таким, как ему хотелось бы. Эту интерпретацию сигнала часто называют **семантикой Mesa** по названию языка, в котором условная переменная была впервые построена таким образом [LR80]. **Семантику Хоара** реализовать труднее, но она дает более сильную гарантию, поскольку разбуженный поток начинает работать сразу после пробуждения [H74]. Практически во всех когда-либо созданных системах используется семантика Mesa.

Лучше, но все равно неправильно: While, а не If

К счастью, это легко исправить (рис. 30.8): нужно только заменить if на while. Подумаем, почему такое решение работает; теперь потребитель T_{c1} пробуждается (с захваченной блокировкой) и сразу проверяет состояние разделяемой переменной (c2). Если в этот момент буфер пуст, потребитель просто возвращается ко сну (c3). Соответственно, в производителе if тоже нужно заменить на while (p2).

```

1 int loops;
2 cond_t cond;
3 mutex_t mutex;
4
5 void *producer(void *arg) {
6     int i;
7     for (i = 0; i < loops; i++) {
8         Pthread_mutex_lock(&mutex);           // p1
9         while (count == 1)                    // p2
10            Pthread_cond_wait(&cond, &mutex); // p3
11         put(i);                               // p4
12         Pthread_cond_signal(&cond);           // p5
13         Pthread_mutex_unlock(&mutex);         // p6
14     }
15 }
16
17 void *consumer(void *arg) {
18     int i;
19     for (i = 0; i < loops; i++) {
20         Pthread_mutex_lock(&mutex);           // c1
21         while (count == 0)                    // c2
22            Pthread_cond_wait(&cond, &mutex); // c3
23         int tmp = get();                      // c4
24         Pthread_cond_signal(&cond);           // c5
25         Pthread_mutex_unlock(&mutex);         // c6
26         printf("%d\n", tmp);
27     }
28 }
```

Рис. 30.8 ❖ Производитель–потребитель:
одна условная переменная и цикл while

Благодаря семантике Меса при работе с условными переменными действует простое правило: **всегда использовать циклы while**. Иногда нет острой необходимости повторно проверять условие, но поступать так всегда безопасно; просто делайте, как сказано, и наслаждайтесь жизнью.

Однако и в этом коде имеется ошибка – и это вторая из вышеупомянутых проблем. Связана она с тем, что условная переменная всего одна. Попробуйте понять, в чем проблема, прежде чем читать дальше. НУ ЖЕ!

... (еще одна пауза на размышление, или прикройте на секунду глаза) ...

Проверим, правильна ли ваша догадка, или хотя бы что вы проснулись и продолжаете читать. Проблема возникает, когда сначала выполняются два потребителя (T_{c1} и T_{c2}), а затем оба засыпают (с3). Потом запускается производитель, помещает значение в буфер и пробуждает одного из потребителей (скажем, T_{c1}). После этого производитель возвращается в начало цикла (по ходу дела освобождая и вновь захватывая блокировку) и пытается поместить в буфер дополнительные данные, но, поскольку буфер уже заполнен, вместо этого засыпает в ожидании выполнения условия. Теперь один потребитель готов к выполнению (T_{c1}), а два потока спят по условию (T_{c2} и T_p). Проблема вот-вот возникнет, напряжение нарастает!

Потребитель T_{c1} пробуждается, выходя из `wait()` (с3), проверяет условие (с2), обнаруживает, что буфер полон, и потребляет значение (с4). Затем – внимание! – этот потребитель сигнализирует условию (с5), пробуждая только один из спящих потоков. Но какой именно поток он должен пробудить?

Поскольку потребитель опустошил буфер, очевидно, что он должен пробудить производителя. Однако если он пробудит потребителя T_{c2} (что, безусловно, возможно и зависит от способа управления очередью), то у нас проблема. Именно, потребитель T_{c2} просыпается, обнаруживает, что буфер пуст (с2), и снова засыпает (с3). Производитель T_p , который мог бы поместить значение в буфер, по-прежнему спит. Другой поток-потребитель, T_{c1} , также заснул. Все три потока спят, чего, конечно же, не должно быть; см. трассировку на рис. 30.9, где шаг за шагом показан путь к этому смертному покою.

T_{c1}	Состояние	T_{c2}	Состояние	T_{c3}	Состояние	Счетчик	Примечание
c1	Выполняется		Готов		Готов	0	
c2	Выполняется		Готов		Готов	0	
c3	Спит		Готов		Готов	0	Нечего получать
	Спит	c1	Выполняется		Готов	0	
	Спит	c2	Выполняется		Готов	0	
	Спит	c3	Спит		Готов	1	Нечего получать
	Спит		Спит	p1	Выполняется	0	
	Спит		Спит	p2	Выполняется	0	
	Спит		Спит	p4	Выполняется	1	Буфер полон
	Готов		Спит	p6	Выполняется	1	T_{c1} пробудился
	Готов		Спит	p1	Выполняется	1	
	Готов		Спит	p2	Выполняется	1	
	Готов		Спит	p3	Спит	1	Должен спать (полон)
c2	Выполняется		Спит		Спит	1	Повторно проверить условие
c4	Выполняется		Спит		Спит	0	T_{c1} забирает данные
c5	Выполняется		Готов		Спит	0	Ой! T_{c1} пробудился
c6	Выполняется		Готов		Спит	0	
c1	Выполняется		Готов		Спит	0	
c2	Выполняется		Готов		Спит	0	
c3	Спит		Готов		Спит	0	
	Спит		Готов		Спит	0	
	Спит	c2	Выполняется		Спит	0	
	Спит	c3	Спит		Спит	0	

Рис. 30.9 ❖ Трассировка потоков: неправильное решение (версия 2)

Очевидно, сигнализация необходима, но она должна быть более целенаправленной. Потребитель должен будить не потребителей, а только производителей – и наоборот.

Решение задачи о производителе и потребителе с буфером на один элемент

Решение и в этом случае простое: использовать не одну, а *две* условные переменные, чтобы можно было правильно указать, какого типа потоки будить при изменении состояния системы. Получившийся код показан на рис. 30.10.

```

1 cond_t empty, fill;
2 mutex_t mutex;
3
4 void *producer(void *arg) {
5     int i;
6     for (i = 0; i < loops; i++) {
7         Pthread_mutex_lock(&mutex);
8         while (count == 1)
9             Pthread_cond_wait(&empty, &mutex);
10        put(i);
11        Pthread_cond_signal(&fill);
12        Pthread_mutex_unlock(&mutex);
13    }
14 }
15
16 void *consumer(void *arg) {
17     int i;
18     for (i = 0; i < loops; i++) {
19         Pthread_mutex_lock(&mutex);
20         while (count == 0)
21             Pthread_cond_wait(&fill, &mutex);
22         int tmp = get();
23         Pthread_cond_signal(&empty);
24         Pthread_mutex_unlock(&mutex);
25         printf("%d\n", tmp);
26     }
27 }
```

Рис. 30.10 ❖ Производитель–потребитель:
две условные переменные и цикл while

Здесь потоки-производители ждут по условию **empty**, а сигнализируют условию **fill**. Наоборот, потоки-потребители ждут по **fill**, а сигнализируют **empty**. Таким образом, вторая проблема исключается на корню: потребитель никогда не сможет случайно разбудить производителя, а производитель – потребителя.

Правильное решение задачи о производителе и потребителе

Итак, у нас теперь имеется работоспособное решение задачи о производителе и потребителе, хотя и не вполне общее. Напоследок мы повысим эффективность и степень конкурентности, а именно увеличим емкость буфера, чтобы производитель мог произвести больше элементов, прежде чем отойти ко сну, а потребитель соответственно мог извлечь больше элементов за один период бодрствования. При наличии всего одного производителя и потребителя этот подход более эффективен, потому что уменьшает количество контекстных переключений, а если производителей и (или) потребителей несколько, то даже позволяет конкурентно производить и потреблять данные, что повышает степень конкурентности. По счастью, в существующее решение придется внести совсем немного изменений.

Первым делом мы должны изменить структуру самого буфера и соответствующие функции `put()` и `get()` (рис. 30.11). Также нужно немного изменить условия, которые производители и потребители проверяют, чтобы решить, засыпать или нет. На рис. 30.12 показана правильная логика ожидания и сигнализации. Производитель засыпает, только если весь буфер заполнен (`p2`), а потребитель – только если буфер пуст (`c2`). Таким образом, мы решили проблему производителя и потребителя, можно откинуться на спинку кресла и выпить чего-нибудь освежающего.

```

1 int buffer[MAX];
2 int fill_ptr = 0;
3 int use_ptr = 0;
4 int count = 0;
5
6 void put(int value) {
7     buffer[fill_ptr] = value;
8     fill_ptr = (fill_ptr + 1) % MAX;
9     count++;
10 }
11
12 int get() {
13     int tmp = buffer[use_ptr];
14     use_ptr = (use_ptr + 1) % MAX;
15     count--;
16     return tmp;
17 }
```

Рис. 30.11 ❖ Правильные функции `put` и `get`

```

1 cond_t empty, fill;
2 mutex_t mutex;
3
4 void *producer(void *arg) {
5     int i;
6     for (i = 0; i < loops; i++) {
7         Pthread_mutex_lock(&mutex);           // p1
8         while (count == MAX)                  // p2
9             Pthread_cond_wait(&empty, &mutex); // p3
10        put(i);                               // p4
11        Pthread_cond_signal(&fill);           // p5
12        Pthread_mutex_unlock(&mutex);         // p6
13    }
14 }
15
16 void *consumer(void *arg) {
17     int i;
18     for (i = 0; i < loops; i++) {
19         Pthread_mutex_lock(&mutex);           // c1
20         while (count == 0)                    // c2
21             Pthread_cond_wait(&fill, &mutex); // c3
22         int tmp = get();                      // c4
23         Pthread_cond_signal(&empty);          // c5
24         Pthread_mutex_unlock(&mutex);         // c6
25         printf("%d\n", tmp);
26     }
27 }

```

Рис. 30.12 ❖ Правильная синхронизация
производителя и потребителя

Совет: используйте while, а не if для проверки условий

Для проверки условия в многопоточной программе использовать цикл while всегда правильно и безопасно, а корректность применения if зависит от семантики сигнализации. Поэтому используйте while – и ваш код будет работать, как задумано.

Обертывание проверки циклом while также решает проблему **ложного пробуждения**. В некоторых библиотеках для работы с потоками из-за деталей реализации возможно пробуждение сразу двух потоков при поступлении всего одного сигнала [L11]. Ложное пробуждение – еще одна причина перепроверить условие, которого ждет поток.

30.3. ПОКРЫВАЮЩИЕ УСЛОВИЯ

Теперь мы рассмотрим еще один пример возможного использования условных переменных. Он заимствован из статьи Лэмпсона и Рэдella об ОС Pilot [LR80], которую разрабатывала та же группа, что впервые реализовала описанную выше **семантику** (они использовали язык Mesa, отсюда и название).

Проблему, с которой они столкнулись, проще всего продемонстрировать на простом примере, в данном случае – простой многопоточной библиотеки выделения памяти. Суть проблемы видна из кода на рис. 30.13.

```

1 // сколько байтов свободно в куче?
2 int bytesLeft = MAX_HEAP_SIZE;
3
4 // нужны блокировка и условная переменная
5 cond_t c;
6 mutex_t m;
7
8 void *
9 allocate(int size) {
10     Pthread_mutex_lock(&m);
11     while (bytesLeft < size)
12         Pthread_cond_wait(&c, &m);
13     void *ptr = ...; // получить память из кучи
14     bytesLeft -= size;
15     Pthread_mutex_unlock(&m);
16     return ptr;
17 }
18
19 void free(void *ptr, int size) {
20     Pthread_mutex_lock(&m);
21     bytesLeft += size;
22     Pthread_cond_signal(&c); // кому сигнализировать??
23     Pthread_mutex_unlock(&m);
24 }
```

Рис. 30.13 ❖ Покрывающие условия: пример

Поток, обратившийся к функции выделения памяти, должен ждать, когда память освободится. Наоборот, поток, освобождающий память, сигнализирует о том, что появилась свободная память. Однако возникает проблема: какой из ожидающих потоков (а их может быть несколько) пробудить?

Рассмотрим следующую ситуацию. Допустим, что нет ни одного свободного байта; поток T_a вызывает `allocate(100)`, а затем поток T_b запрашивает меньше памяти, вызвав `allocate(10)`. И T_a , и T_b засыпают в ожидании одного и того же условия, поскольку удовлетворить их запросы прямо сейчас невозможно.

В этот момент третий поток, T_c , вызывает `free(50)`. Он сигнализирует, но, к сожалению, может случиться, что разбудит не поток T_b , которому нужно всего 10 байт, а поток T_a , который должен спать дальше, т. к. для него памяти пока недостаточно. Поэтому показанный на рисунке код не работает, т. к. сигнализирующий поток не знает, какой поток (или потоки) пробуждать.

Лэмпсон и Рэдделл предложили прямолинейное решение: заменить вызов `pthread_cond_signal()` вызовом `pthread_cond_broadcast()`, который будит все ожидающие условия потоки. Конечно, у этого решения есть недостаток, связанный с производительностью, т. к. мы, возможно, будем без нужды бу-

дить много потоков, которым еще стоило бы поспать. Эти потоки проснутся, перепроверят условие и сразу же заснут опять.

Лэмпсон и Рэдделл назвали такое условие **покрывающим**, поскольку оно покрывает все случаи, когда поток должен проснуться (пусть и с излишком); но, как мы сказали, за это приходится платить тем, что может быть разбужено слишком много потоков. Проницательный читатель, возможно, обратил внимание, что мы могли бы использовать этот подход и раньше (при решении задачи о производителе и потребителе с помощью одной условной переменной). Однако тогда существовало более подходящее решение, и мы выбрали его. В общем же случае, если оказывается, что ваша программа работает, только если заменить направленный сигнал широковещательным (хотя вам не кажется, что так должно быть), вероятно, в ней есть ошибка: исправьте ее! Но в таких примерах, как описанный выше распределитель памяти, широковещание является, пожалуй, самым простым решением.

30.4. РЕЗЮМЕ

Мы познакомились еще с одним важным примитивом синхронизации, отличным от блокировок: условными переменными. Они позволяют потокам спать, пока не будет выполнено интересующее их условие, и тем самым решают целый ряд важных проблем синхронизации, включая знаменитую (и по-прежнему важную) задачу о производителе и потребителе, а также проблему покрывающих условий. Здесь будет уместно драматическое заключительное предложение, например: «Он полюбил Большого Брата» [O49].

Литература

[D68] «Cooperating sequential processes» by Edsger W. Dijkstra. 1968. Доступно по адресу <http://www.cs.utexas.edu/users/EWD/ewd01xx/EWD123.PDF>. *Еще одна классическая работа Дейкстры; читая его ранние работы по конкурентности, вы узнаете многое из того, что должны знать.*

[D72] «Information Streams Sharing a Finite Buffer» by E.W. Dijkstra. Information Processing Letters 1: 179180, 1972. Доступно по адресу <http://www.cs.utexas.edu/users/EWD/ewd03xx/EWD329.PDF>. *Знаменитая статья, в которой была поставлена задача о производителе и потребителе.*

[D01] «My recollections of operating system design» by E.W. Dijkstra. April, 2001. Доступно по адресу <http://www.cs.utexas.edu/users/EWD/ewd13xx/EWD1303.PDF>. *Увлекательное чтение для тех, кого интересует, как основоположники нашей отрасли приходили к фундаментальным идеям, в т. ч. прерывания и даже стека.*

[H74] «Monitors: An Operating System Structuring Concept» by C. A. R. Hoare. Communications of the ACM, 17:10, pages 549–557, October 1974. *Хоар – автор важных теоретических работ по конкурентности. Однако, пожалуй, больше известен его алгоритм Quicksort – самый лучший алгоритм сортировки в мире, по крайней мере по мнению авторов этой книги.*

[L11] «Pthread cond signal Man Page» by Mysterious author. March, 2011. Доступно по адресу http://linux.die.net/man/3/pthread_cond_signal. На этой странице руководства по Linux приведен простой пример ситуации, в которой возможно ложное пробуждение из-за состояния гонки в коде сигнализации и пробуждения.

[LR80] «Experience with Processes and Monitors in Mesa» by B. W. Lampson, D. R. Redell. Communications of the ACM. 23:2, pages 105–117, February 1980. Потрясающая статья о том, как реализованы сигнализация и условные переменные в реальной системе, о том, как возник термин «семантика Mesa», и о более старой семантике Тони Хоара [H74], о которой трудно рассказывать в аудитории на полном серьезе.

[O49] «1984» by George Orwell. Secker and Warburg, 1949. Немного тяжеловесная, но, безусловно, достойная прочтения книга. Правда, мы раскрыли, чем она кончается, в последнем предложении этой главы. Уж простите! А если эту книгу читают члены правительства, то мы готовы поставить такому правительству оценку «отлично». Слышите нас, ребята из АНБ?

Домашнее задание (код)

В этом домашнем задании вы будете исследовать реальный код, в котором блокировки и условные переменные используются для реализации различных форм очереди в рассмотренной выше задаче о производителе и потребителе. Вы будете иметь дело с реальным кодом, выполнять его с разными параметрами и на его примере изучать, что работает, а что нет, а также другие тонкие моменты. Детали см. в файле README.

Вопросы

1. Наш первый вопрос касается файла `main-two-cvs-while.c` (работоспособное решение). Для начала изучите код. Понимаете ли вы, что должно произойти при выполнении этой программы?
2. Выполните программу с одним производителем и одним потребителем, заставив производителя произвести несколько значений. Начните с буфера размером 1 и постепенно увеличивайте его. Как изменяется (и изменяется ли вообще) поведение программы при увеличении размера буфера? Каким, на ваш взгляд, будет значение `num_full` при различных размерах буфера (например, если задан флаг `-m 10`) и различном числе произведенных элементов (например, при `-l 100`) при умалчиваемой строке засыпания потребителя (без засыпания) и строке `-C 0,0,0,0,0,0,1`?
3. Если есть такая возможность, выполните код в разных системах (например, Mac и Linux). Наблюдаете ли вы какие-нибудь различия в поведении?
4. Поговорим о хронометраже. Как вы думаете, сколько времени займет выполнение программы при следующих условиях: один производитель, три потребителя, буфер на один элемент, каждый потребитель в точке `c3` делает задержку на одну секунду: `./main-two-cvs-while -p 1 -c 3 -m 1 -C 0,0,0,1,0,0,0:0,0,0,1,0,0,0:0,0,0,1,0,0,0 -l 10 -v -t?`

5. Измените размер буфера на 3 (-m 3). Изменяется ли при этом общее время?
6. Измените место задержки на s6 (при этом моделируется потребитель, который выбирает из очереди элемент, а затем что-то делает с ним), вернувшись к буферу на один элемент. Какое время вы прогнозируете в этом случае? `./main-two-cvs-while -p 1 -c 3 -m 1 -C 0,0,0,0,0,0,1:0,0,0,0,0,0,1:0,0,0,0,0,0,1 -l 10 -v -t`
7. Наконец, снова возьмите буфер размером 3 (-m 3). Каков теперь ваш прогноз?
8. Обратимся к программе `main-one-cv-while.c`. В предположении, что имеется один производитель, один потребитель и буфер размером 1, попробуйте задать такую строку засыпания, чтобы в программе возникла проблема.
9. Измените число потребителей, сделав его равным 2. Попробуйте построить такие строки засыпания для производителя и потребителей, которые вызывают проблемы в программе.
10. Теперь рассмотрим программу `main-two-cvs-if.c`. Сможете ли вы спровоцировать в ней проблему? Снова рассмотрите случаи одного и нескольких потребителей.
11. Наконец, поработаем с программой `main-two-cvs-while-extra-unlock.c`. Какая проблема возникает, если освободить блокировку до выполнения `put` или `get`? Сможете ли вы уверенно воспроизвести эту проблему, задавая строку засыпания? Что плохого может произойти?

Глава 31

Семафоры

Как нам уже известно, для решения широкого круга важных и интересных задач конкурентности необходимы как блокировки, так и условные переменные. Одним из первых это много лет назад осознал **Эдсгер Дейкстра** (хотя точную историю восстановить трудно [GR92]), известный, среди прочего, своим знаменитым алгоритмом «кратчайших путей» в теории графов [D59], ранней полемикой на тему структурного программирования в статье «Goto Statements Considered Harmful» (О вреде оператора GOTO) [D68a] (какое замечательное название!) и – возвращаясь к интересующей нас теме – первым описанием примитива синхронизации, названного **семафором** [D68b, D72]. На самом деле Дейкстра с сотрудниками предложили семафор как единственный примитив для всего связанного с синхронизацией; ниже мы покажем, как можно использовать семафор в качестве блокировки и условной переменной.

СУЩЕСТВО ПРОБЛЕМЫ: КАК ПОЛЬЗОВАТЬСЯ СЕМАФОРАМИ

Как использовать семафоры вместо блокировок и условных переменных? Как определяется семафор? Что такое двоичный семафор? Легко ли построить семафор с помощью блокировок и условных переменных? А блокировки и условные переменные помощью семафоров?

31.1. СЕМАФОРЫ: ОПРЕДЕЛЕНИЕ

Семафор – это объект, принимающий целочисленное значение, которым можно манипулировать с помощью двух операций, в стандарте POSIX они называются `sem_wait()` и `sem_post()`¹. Поскольку начальное значение семафора определяет его поведение, перед вызовом любой операции необходимо инициализировать семафор, как показано в коде на рис. 31.1.

¹ Исторически Дейкстра употреблял названия `P()` вместо `sem_wait()` и `V()` вместо `sem_post()`. `P()` происходит от «prolaag» – сплав слов «probeer» (голландское *пытаться*) и «verlaag» (*уменьшать*), а `V()` – от голландского «verhoog» (*увеличивать*) (спасибо Марту Оскампу за эту информацию). Иногда эти операции называют «опустить» и «поднять». Можете поразить друзей голландскими словами – или привести их в смятение, или то и другое сразу.

```

1 #include <semaphore.h>
2 sem_t s;
3 sem_init(&s, 0, 1);

```

Рис. 31.1 ❖ Инициализация семафора

Здесь мы объявили семафор `s` и инициализировали его значением 1, передав его в третьем аргументе. Второй аргумент `sem_init()` во всех последующих примерах будет равен 0, это означает, что семафор разделяется между потоками внутри одного процесса. О других применениях семафоров (как использовать их для синхронизации доступа со стороны *разных* процессов), когда второй аргумент принимает другое значение, см. страницы руководства.

После инициализации семафора для работы с ним вызываются функции `sem_wait()` и `sem_post()`. Их семантика описана на рис. 31.2.

```

1 int sem_wait(sem_t *s) {
2     уменьшить значение семафора s на единицу
3     ждать, если значение семафора s отрицательно
4 }
5
6 int sem_post(sem_t *s) {
7     увеличить значение семафора s на единицу
8     если в очереди к семафору стоит один или более потоков, разбудить один из них
9 }

```

Рис. 31.2 ❖ Семафор: определение операций wait и post

Пока что нас не будет интересовать реализация этих функций, которая, безусловно, требует аккуратности; когда несколько потоков вызывают `sem_wait()` или `sem_post()`, очевидно, требуется управлять критическими секциями. Мы сосредоточимся на использовании этих примитивов, отложив рассмотрение деталей внутреннего устройства на потом.

Обсудим наиболее интересные аспекты интерфейса. Во-первых, мы видим, что `sem_wait()` либо возвращает управление сразу (потому что в момент вызова значение семафора было больше или равно 1), либо приостанавливает выполнение вызывающей программы в ожидании следующей операции `post`. Конечно, `sem_wait()` могут вызвать несколько потоков, и тогда все они будут поставлены в очередь.

Во-вторых, `sem_post()` не ждет выполнения какого-то условия, как `sem_wait()`. Она просто увеличивает значение семафора, и если в очереди к семафору стоят потоки, то будит один из них.

В-третьих, если значение семафора отрицательно, то оно равно количеству стоящих в очереди потоков [D68b]. Хотя пользователи семафора обычно не видят его значения, об этом инварианте полезно знать, потому что это поможет запомнить, как семафор функционирует.

Пока не стоит тревожиться по поводу кажущихся состояний гонки внутри семафора; предположим, что все действия выполняются атомарно. А чуть позже мы гарантируем это с помощью блокировок и условных переменных.

31.2. ДВОИЧНЫЕ СЕМАФОРЫ (БЛОКИРОВКИ)

Теперь мы готовы использовать семафор. С первым способом использования мы уже знакомы: семафор в роли блокировки. Взгляните на фрагмент кода на рис. 31.3; мы просто окружили критическую секцию парой функций `sem_wait()` и `sem_post()`. Но для того чтобы этот код работал, очень важно правильно инициализировать семафор. Чему же должно быть равно начальное значение `X`?

... (Сначала подумайте, потом читайте дальше) ...

```
1 sem_t m;
2 sem_init(&m, 0, X); // инициализировать семафор значением X; чему должно быть равно X?
3
4 sem_wait(&m);
5 // здесь критическая секция
6 sem_post(&m);
```

Рис. 31.3 ❖ Двоичный семафор (т. е. блокировка)

Из определений операций `sem_wait()` и `sem_post()` ясно, что начальное значение должно быть равно 1.

Чтобы убедиться в этом, рассмотрим случай двух потоков. Первый поток (Поток 0) вызывает `sem_wait()`, при этом значение семафора уменьшается и становится равным 0. Семафор переходит в состояние ожидания, только если значение *не* является большим или равным 0. Поскольку оно равно 0, то `sem_wait()` просто возвращает управление, вызывающий Поток 0 продолжает работать и может войти в критическую секцию. Если никакой другой поток не попытается захватить блокировку, когда Поток 0 находится в критической секции, то вызов `sem_post()` просто восстановит значение семафора, сделав его равным 1 (и не будет никого пробуждать, потому что некого). Этот сценарий показан на рис. 31.4.

Значение семафора	Поток 0	Поток 1
1		
1	вызов <code>sem_wait()</code>	
0	<code>sem_wait()</code> вернулась	
0	(критическая секция)	
0	вызов <code>sem_post()</code>	
1	<code>sem_post()</code> вернулась	

Рис. 31.4 ❖ Трассировка потоков: один поток использует семафор

Более интересный случай возникает, когда Поток 0 «удерживает блокировку» (т. е. вызвал `sem_wait()`, но еще не вызывал `sem_post()`), а другой поток (Поток 1) пытается войти в критическую секцию, вызвав `sem_wait()`. В этом случае Поток 1 уменьшит значение семафора до -1 и, следовательно, будет ждать (заснет и уступит процессор). Когда Поток 0 возобновится, он рано или поздно вызовет `sem_post()`, увеличит значение семафора до 0 и разбудит ждущий Поток 1, который затем сможет захватить блокировку. Выходя из

критической секции, Поток 1 еще раз увеличит значение семафора, снова сделав его равным 1.

На рис. 31.5 показана трассировка этого примера. Помимо действий потоков показано также состояние каждого потока: Выполняется, готов (т. е. может выполняться, но еще не выполняется) или Спит. Заметим, в частности, что Поток 1 засыпает, пытаясь захватить уже занятую блокировку, и лишь после того как Поток 0 возобновится, Поток 1 может быть разбужен и потенциально получит возможность поработать.

Значение	Поток 0	Состояние	Поток 1	Состояние
1		Выполняется		Готов
1	вызов sem_wait()	Выполняется		Готов
0	sem_wait() вернулась	Выполняется		Готов
0	(критическая секция: начало)	Готов		Готов
0	Прерывание: переключение → T1	Готов		Выполняется
0		Готов	вызов sem_wait()	Выполняется
-1		Готов	декремент sem	Готов
-1		Готов	(sem < 0) → спать	Спит
-1		Выполняется	Переключение → T0	Спит
-1	(критическая секция: конец)	Выполняется		Спит
-1	вызов sem_post()	Выполняется		Спит
0	инкремент sem	Выполняется		Спит
0	разбудить (T1)	Выполняется		Готов
0	sem_post() вернулась	Выполняется		Готов
0	Прерывание: переключение → T1	Готов		Выполняется
0		Готов	sem_wait() вернулась	Выполняется
0		Готов	(критическая секция)	Выполняется
0		Готов	вызов sem_post()	Выполняется
1		Готов	sem_post() вернулась	Выполняется

Рис. 31.5 ❖ Трассировка потоков: два потока используют семафор

Если вы готовы проработать собственный пример, рассмотрите случай, когда несколько потоков стоят в очереди к блокировке. Какие значения будет принимать семафор в процессе выполнения?

Итак, мы можем использовать семафоры в качестве блокировок. Поскольку блокировка может находиться только в двух состояниях (занята или свободна), иногда семафор, используемый в таком качестве, называют **двоичным**. Отметим, что если мы не собираемся использовать семафор никак иначе, то его можно реализовать проще, чем обсуждаемые здесь семафоры общего вида.

31.3. ИСПОЛЬЗОВАНИЕ СЕМАФОРОВ для упорядочения

Семафоры полезны также для упорядочения событий в конкурентной программе. Например, иногда потоку нужно дождаться, чтобы список стал непустым, и потом удалить из него элемент. При таком способе использования часто оказывается, что один поток *ждет* чего-то, а другой совершает это

«что-то» и *сигнализирует* о происшедшем, тем самым пробуждая ожидающий поток. Таким образом, мы используем семафор как примитив **упорядочения** (что близко к описанному выше использованию **условных переменных**).

Приведем простой пример. Пусть поток создает другой поток и хочет дождаться его завершения (рис. 31.6). При выполнении этой программы мы хотим видеть такую картину:

```
родитель: начало
потомок
родитель: конец
```

```
1 sem_t s;
2
3 void *
4 child(void *arg) {
5     printf("потомок\n");
6     sem_post(&s); // здесь сигнал: потомок закончил
7     return NULL;
8 }
9
10 int
11 main(int argc, char *argv[]) {
12     sem_init(&s, 0, X); // чему должно быть равно X?
13     printf("родитель: начало\n");
14     pthread_t c;
15     Pthread_create(&c, NULL, child, NULL);
16     sem_wait(&s); // ждать потомка
17     printf("родитель: конец\n");
18     return 0;
19 }
```

Рис. 31.6 ❖ Родитель ждет потомка

Вопрос, таким образом, заключается в том, как воспользоваться семафором для достижения желаемого эффекта. Оказывается, что это не так уж сложно. Из кода видно, что родитель просто вызывает `sem_wait()`, а потомок – `sem_post()`, и в результате родитель дожидается, когда условие «потомок завершил выполнение» будет выполнено. Но вот вопрос: каким должно быть начальное значение семафора?

(Снова сначала поразмыслите над вопросом, а потом читайте дальше.)

Ответ такой: начальное значение должно быть равно 0. Снова рассмотрим два случая. Сначала предположим, что родитель создал потомка, но потомок начал работать не сразу (он находится в очереди в состоянии готовности, но еще не выполняется). В этом случае (рис. 31.7) родитель вызовет `sem_wait()` еще до того, как потомок вызовет `sem_post()`; мы хотели бы, чтобы родитель дождался начала работы потомка. Это может произойти, только если значение семафора не больше нуля, а значит, начальное значение должно быть равно 0. Родитель уменьшает значение (оно становится равно –1), а затем ждет (засыпает). Когда потомок начнет выполняться, он вызовет `sem_post()`,

увеличит значение семафора до 0 и разбудит родителя, который вернется из `sem_wait()` и завершит программу.

Значение	Родитель	Состояние	Потомок	Состояние
0	<code>create(Child)</code>	Выполняется	<i>(Потомок существует, готов к выполнению)</i>	Готов
0	вызвать <code>sem_wait()</code>	Выполняется		Готов
-1	декремент <code>sem</code>	Выполняется		Готов
-1	$(sem < 0) \rightarrow$ спать	Спит		Готов
-1	<i>Переключение $\rightarrow$ потомок</i>	Спит	потомок выполняется	Выполняется
-1		Спит	вызвать <code>sem_post()</code>	Выполняется
0		Спит	инкремент <code>sem</code>	Выполняется
0		Готов	разбудить (родитель)	Выполняется
0		Готов	<code>sem_post()</code> возвращается	Выполняется
0		Готов	<i>Переключение $\rightarrow$ родитель</i>	Готов
0	<code>sem_wait()</code> возвращается	Выполняется		Готов

Рис. 31.7 ❖ Трассировка потоков: родитель ждет потомка (случай 1)

Второй случай (рис. 31.8) – когда потомок заканчивает работу еще до того, как родитель получил шанс вызвать `sem_wait()`. Тогда потомок первым делом вызовет `sem_post()`, увеличив тем самым значение семафора с 0 до 1. Когда родитель получит шанс поработать, он вызовет `sem_wait()` и обнаружит, что семафор имеет значение 1. Родитель уменьшит значение до 0 и вернется из `sem_wait()` без ожидания – именно то, что нам нужно.

Значение	Родитель	Состояние	Потомок	Состояние
0	<code>create(Child)</code>	Выполняется	<i>(Потомок существует, готов к выполнению)</i>	Готов
0	<i>Переключение $\rightarrow$ потомок</i>	Готов	потомок выполняется	Выполняется
0		Готов	вызвать <code>sem_post()</code>	Выполняется
1		Готов	инкремент <code>sem</code>	Выполняется
1		Готов	разбудить (никого)	Выполняется
1		Готов	<code>sem_post()</code> возвращается	Выполняется
1	родитель выполняется	Выполняется	<i>Прерывание; Переключение $\rightarrow$ родитель</i>	Готов
1	вызвать <code>sem_wait()</code>	Выполняется		Готов
0	декремент <code>sem</code>	Выполняется		Готов
0	$(sem \geq 0) \rightarrow$ разбудить	Выполняется		Готов
0	<code>sem_wait()</code> возвращается	Выполняется		Готов

Рис. 31.8 ❖ Трассировка потоков: родитель ждет потомка (случай 2)

31.4. Задача о производителе и потребителе (об ограниченном буфере)

Далее в этой главе мы рассмотрим задачу **о производителе и потребителе**, или об **ограниченном буфере** [D72]. Она была подробно описана в предыдущей главе, в разделе об условных переменных, детали постановки смотрите там.

Первая попытка

Сначала попробуем решить эту задачу с помощью двух семафоров, `empty` и `full`, которые показывают соответственно, что буфер пуст или заполнен. Код функций `put` и `get` приведен на рис. 31.9, а наша попытка решить задачу показана на рис. 31.10.

```

1 int buffer[MAX];
2 int fill = 0;
3 int use = 0;
4
5 void put(int value) {
6     buffer[fill] = value;    // Строка F1
7     fill = (fill + 1) % MAX; // Строка F2
8 }
9
10 int get() {
11     int tmp = buffer[use];    // Строка G1
12     use = (use + 1) % MAX;    // Строка G2
13     return tmp;
14 }
```

Рис. 31.9 ❖ Функции `put` и `get`

В этом примере производитель ждет, когда буфер станет пуст, чтобы поместить в него данные, а потребитель ждет, когда в буфере появятся данные, чтобы можно было их забрать. Для начала допустим, что `MAX=1` (в буфере может быть только один элемент), и посмотрим, работает ли наше решение.

Снова предположим, что имеется два потока, производитель и потребитель. Рассмотрим случай одного CPU. Пусть потребитель начинает работать первым. Тогда он дойдет до строки C1 на рис. 31.10 и вызовет `sem_wait(&full)`. Поскольку `full` был инициализирован значением 0, этот вызов уменьшит `full` до -1, заблокирует потребителя и будет ждать, когда какой-нибудь другой поток вызовет `sem_post()` для `full`.

Затем начинает работать производитель. Он доходит до строки P1 и вызывает `sem_wait(&empty)`. Но, в отличие от потребителя, он не засыпает в этой строке, а идет дальше, потому что `empty` был инициализирован значением `MAX` (в данном случае 1). Поэтому `empty` будет уменьшен до 0, и производитель поместит данные в первую позицию буфера (строка P2). Затем производитель перейдет к строке P3 и вызовет `sem_post(&full)`, изменив тем самым значение семафора `full` с -1 до 0 и разбудив потребителя (например, переводя его в состояние «готов» из состояния «заблокирован»).

В этом случае может произойти одно из двух. Если производитель продолжает работать, то он вернется в начало цикла и снова окажется на строке P1. Но на этот раз он будет заблокирован, поскольку значение семафора `empty` равно 0. Если, напротив, производитель будет прерван, а выполняться начнет потребитель, то он вызовет `sem_wait(&full)` (строка C1), обнаружит, что буфер заполнен, и потребит его. В любом случае мы имеем желаемое поведение.

```

1 sem_t empty;
2 sem_t full;
3
4 void *producer(void *arg) {
5     int i;
6     for (i = 0; i < loops; i++) {
7         sem_wait(&empty);    // Строка P1
8         put(i);              // Строка P2
9         sem_post(&full);     // Строка P3
10    }
11 }
12
13 void *consumer(void *arg) {
14     int i, tmp = 0;
15     while (tmp != -1) {
16         sem_wait(&full);     // Строка C1
17         tmp = get();         // Строка C2
18         sem_post(&empty);    // Строка C3
19         printf("%d\n", tmp);
20     }
21 }
22
23 int main(int argc, char *argv[]) {
24     // ...
25     sem_init(&empty, 0, MAX); // Сначала в буфере свободно MAX позиций...
26     sem_init(&full, 0, 0);    // ... а занято 0
27     // ...
28 }

```

Рис. 31.10 ❖ Добавление семафоров full и empty

Можете разобрать тот же пример с большим числом потоков (несколько производителей и несколько потребителей). Все по-прежнему должно работать.

Теперь допустим, что MAX больше 1 (скажем, MAX = 10). Предположим также, что имеется несколько производителей и потребителей. Тогда возникает проблема: состояние гонки. Видите ли вы, где имеет место гонка (сделайте паузу и поищите)? Если не видите, вот вам подсказка: внимательно присмотритесь к коду функций put() и get().

Ладно, давайте разбираться. Пусть два производителя (Pa и Pb) вызывают put() примерно в одно и то же время. Предположим, что Pa работает первым и начинает заполнять первую позицию в буфере (fill = 0 в строке F1). Но прежде чем Pa успевает увеличить счетчик fill до 1, его прерывают. Начинает работать производитель Pb и в строке F1 также помещает данные в нулевую позицию буфера, т. е. старые данные оказываются перезаписанными! Это никуда не годится: мы не хотим терять никакие данные, порожденные производителем.

Решение: добавление взаимного исключения

Как видите, мы забыли о *взаимном исключении*. Заполнение буфера и увеличение индекса в нем – это критическая секция, которая должна быть тщательно защищена. Поэтому воспользуемся нашим старым приятелем, двоичным семафором, и добавим блокировки. Эта попытка показана на рис. 31.11.

```

1 sem_t empty;
2 sem_t full;
3 sem_t mutex;
4
5 void *producer(void *arg) {
6     int i;
7     for (i = 0; i < loops; i++) {
8         sem_wait(&mutex); // строка P0 (НОВАЯ СТРОКА)
9         sem_wait(&empty); // строка P1
10        put(i);           // строка P2
11        sem_post(&full);   // строка P3
12        sem_post(&mutex);  // строка P4 (НОВАЯ СТРОКА)
13    }
14 }
15
16 void *consumer(void *arg) {
17     int i;
18     for (i = 0; i < loops; i++) {
19         sem_wait(&mutex); // строка C0 (НОВАЯ СТРОКА)
20         sem_wait(&full);  // строка C1
21         int tmp = get();  // строка C2
22         sem_post(&empty); // строка C3
23         sem_post(&mutex); // строка C4 (НОВАЯ СТРОКА)
24         printf("%d\n", tmp);
25     }
26 }
27
28 int main(int argc, char *argv[]) {
29     // ...
30     sem_init(&empty, 0, MAX); // Сначала в буфере свободно MAX позиций...
31     sem_init(&full, 0, 0);    // ... а занято 0
32     sem_init(&mutex, 0, 1);   // mutex=1, потому что это блокировка (НОВАЯ СТРОКА)
33     // ...
34 }
```

Рис. 31.11 ❖ Добавление взаимного исключения (некорректное)

Итак, мы окружили части `put()` и `get()` блокировками, обозначенными фразой «НОВАЯ СТРОКА» в комментариях. Идея, на первый взгляд, здравая, но тоже не работает. В чем же дело? Во взаимоблокировке. А почему возникает взаимоблокировка? Попробуйте сами предъявить ситуацию, когда это происходит. Какая последовательность шагов ведет к взаимоблокировке в программе?

Предотвращение взаимоблокировки

Вы, наверное, уже разобрались, поэтому дадим ответ. Пусть имеется два потока, производитель и потребитель. Потребитель начинает работать первым. Он захватывает мьютекс (строка C0) и вызывает `sem_wait()` для семафора `full` (строка C1); поскольку данных еще нет, этот вызов приводит к блокировке потребителя и уступке CPU. Но гораздо важнее, что потребитель продолжает удерживать блокировку.

Затем начинает работать производитель. Производить ему нечего, и если бы ему дали возможность поработать, то он смог бы разбудить поток потребителя, и все закончилось бы хорошо. Но, к несчастью, первым делом он вызывает `sem_wait()` для двоичного семафора-мьютекса (строка P0). Однако блокировка уже занята, поэтому производитель тоже засыпает в ожидании.

Что же мы имеем? Потребитель *удерживает* мьютекс и *ждет*, когда кто-нибудь *просигнализирует* семафор `full`. Производитель мог бы *просигнализировать* `full`, но *ждет* мьютекса. Таким образом, производитель и потребитель замерли в ожидании друг друга: классическая взаимоблокировка.

Наконец-то правильное решение

Для решения проблемы нужно лишь уменьшить область действия блокировки. Правильное решение показано на рис. 31.12. Мы просто сдвинули операции захвата и освобождения мьютекса ближе к критической секции, а код ожидания и сигнализации семафорам `full` и `empty` остался снаружи. В результате получился простой и работоспособный ограниченный буфер, часто встречающийся в многопоточных программах. Как следует разобраться в нем сейчас и можете использовать в будущем. Вы еще не раз скажете нам спасибо. Или, по крайней мере, вспомните нас с благодарностью, если этот вопрос попадет на выпускном экзамене.

31.5. Блокировки чтения-записи

Еще одна классическая проблема проистекает из желания иметь более гибкий примитив блокировки, допускающий, что для разных способов доступа к структуре данных могут понадобиться разные виды блокировки. Например, представим себе последовательность конкурентных операций со списком, включающую операции вставки и поиска. Если вставка изменяет состояние списка (а значит, традиционная критическая секция оправдана), то поиск только *читает* структуру данных; если мы сможем гарантировать, что одновременно не производится вставка, то несколько операций поиска могут выполняться конкурентно. Для поддержки такого способа работы мы разработаем специальный тип блокировки, называемой **блокировкой чтения-записи** [CNP71]. Соответствующий код показан на рис. 31.13.

```

1 sem_t empty;
2 sem_t full;
3 sem_t mutex;
4
5 void *producer(void *arg) {
6     int i;
7     for (i = 0; i < loops; i++) {
8         sem_wait(&empty);    // строка P1
9         sem_wait(&mutex);    // строка P1.5 (ПЕРЕНЕСЛИ МЬЮТЕКС СЮДА...)
10        put(i);              // строка P2
11        sem_post(&mutex);    // строка P2.5 (... И СЮДА)
12        sem_post(&full);     // строка P3
13    }
14 }
15
16 void *consumer(void *arg) {
17     int i;
18     for (i = 0; i < loops; i++) {
19         sem_wait(&full);     // строка C1
20         sem_wait(&mutex);    // строка C1.5 (ПЕРЕНЕСЛИ МЬЮТЕКС СЮДА...)
21         int tmp = get();     // строка C2
22         sem_post(&mutex);    // строка C2.5 (... И СЮДА)
23         sem_post(&empty);    // строка C3
24         printf("%d\n", tmp);
25     }
26 }
27
28 int main(int argc, char *argv[]) {
29     // ...
30     sem_init(&empty, 0, MAX); // Сначала в буфере свободно MAX позиций...
31     sem_init(&full, 0, 0);    // ... а занято 0
32     sem_init(&mutex, 0, 1);   // mutex=1, потому что это блокировка
33     // ...
34 }

```

Рис. 31.12 ❖ Добавление взаимного исключения (корректное)

Код довольно простой. Если какой-то поток хочет обновить структуру данных, то должен вызвать одну из двух новых операций синхронизации: `rwlock_acquire_writelock()`, чтобы захватить блокировку записи, и `rwlock_release_writelock()`, чтобы освободить ее. На внутреннем уровне в них используется семафор `writelock`, гарантирующий, что только один писатель сможет захватить блокировку и войти в критическую секцию, где производится обновление.

Более интересна пара функций для захвата и освобождения блокировки чтения. В этом случае читатель сначала захватывает блокировку `lock`, а затем увеличивает переменную `readers`, равную количеству читателей, находящихся в критической области. Если это первый читатель, то он производит еще одно важное действие: захватывает блокировку записи, вызывая функцию `sem_wait()` для семафора `writelock`. Перед выходом из функции любой читатель освобождает блокировку `lock`, вызывая `sem_post()`.

```

1 typedef struct _rwlock_t {
2     sem_t lock;        // двоичный семафор (базовая блокировка)
3     sem_t writelock;   // допускает ОДНОГО писателя и МНОГО читателей
4     int readers;       // количество читателей в критической области
5 } rwlock_t;
6
7 void rwlock_init(rwlock_t *rw) {
8     rw->readers = 0;
9     sem_init(&rw->lock, 0, 1);
10    sem_init(&rw->writelock, 0, 1);
11 }
12
13 void rwlock_acquire_readlock(rwlock_t *rw) {
14     sem_wait(&rw->lock);
15     rw->readers++;
16     if (rw->readers == 1)
17         sem_wait(&rw->writelock); // первый читатель захватывает блокировку записи
18     sem_post(&rw->lock);
19 }
20
21 void rwlock_release_readlock(rwlock_t *rw) {
22     sem_wait(&rw->lock);
23     rw->readers--;
24     if (rw->readers == 0)
25         sem_post(&rw->writelock); // последний читатель освобождает блокировку записи
26     sem_post(&rw->lock);
27 }
28
29 void rwlock_acquire_writelock(rwlock_t *rw) {
30     sem_wait(&rw->writelock);
31 }
32
33 void rwlock_release_writelock(rwlock_t *rw) {
34     sem_post(&rw->writelock);
35 }

```

Рис. 31.13 ❖ Простая блокировка чтения записи

Таким образом, после захвата блокировки чтения ее смогут захватить и другие читатели, но любой поток, желающий захватить блокировку записи, должен будет ждать, пока *все* читатели закончат работу; последний читатель, покидающий критическую область, вызывает `sem_post()` для семафора `writelock` и тем самым даст ожидающему писателю возможность захватить блокировку.

Этот подход работает правильно, но имеет ряд недостатков, особенно в части справедливости. Именно, читатели без труда смогут лишить писателей обслуживания. Существуют и более изощренные решения этой задачи; быть может, вы сами придумаете реализацию получше? Подсказка: подумайте, что необходимо для того, чтобы воспрепятствовать входу читателей в критическую область, если какой-нибудь писатель ожидает освобождения блокировки?

СОВЕТ: НЕЗАМЫСЛОВАТОЕ МОЖЕТ ОКАЗАТЬСЯ ЛУЧШЕ (ЗАКОН ХИЛЛА)

Никогда не нужно недооценивать идею о том, что незамысловатый подход может оказаться самым лучшим. Иногда простая спин-блокировка дает оптимальный результат, потому что работает быстро и легко реализуется. Хотя «блокировка чтения-записи» звучит пафосно, этот механизм сложен, а «сложный» и «медленный» – зачастую одно и то же. Поэтому всегда начинайте с бесхитростного решения.

Идея отдавать предпочтение простоте встречается во многих местах. Один из ранних источников – диссертация Марка Хилла [H87], в которой изучалось проектирование процессорных кешей. Хилл обнаружил, что простые кеши с прямым отображением работают лучше изошренных наборно-ассоциативных кешей (одной из причин является тот факт, что при кешировании чем проще конструкция, тем быстрее поиск). Хилл подвел такой итог своей работе: «Большое и примитивное лучше». Поэтому похожую рекомендацию мы называем **законом Хилла**.

Наконец, отметим, что блокировки чтения-записи следует использовать с осторожностью. Часто они увеличивают накладные расходы (особенно при более сложной реализации), а потому не дают повышения производительности по сравнению с простыми и быстрыми примитивами блокировки [CB08]. Но как бы то ни было, это еще один пример интересного и полезного применения семафоров.

31.6. ОБЕДАЮЩИЕ ФИЛОСОФЫ

Одна из самых знаменитых задач на тему конкурентности, поставленная и решенная Дейкстрой, – **задача об обедающих философах** [D71]. Задача знаменита тем, что формулируется забавно и представляет интеллектуальный вызов, но практическая ее полезность невысока. Однако ее слава побудила нас включить ее в эту книгу, да и, возможно, вас когда-нибудь спросят о ней на собеседовании, и если вы не ответите и не получите работу, то будете проклинать профессора, читавшего курс по ОС. И наоборот, если это поможет вам получить работу, то не забудьте послать профессору восточку с благодарностью, а то и опцион на покупку акций.

В базовой постановке (рис. 31.14) задача формулируется так: предположим, что вокруг стола сидят пять философов. Между каждыми двумя философами лежит одна вилка (всего, стало быть, вилок пять). Каждый философ иногда думает и в это время не нуждается в вилке, а иногда ест. Чтобы покушать, философу нужно две вилки: та, что слева, и та, что справа. Конкуренция за вилки и возникающие при этом проблемы синхронизации – вот причина, по которой мы изучаем эту задачу при обсуждении конкурентного программирования.

Вот как выглядит базовый цикл каждого философа:

```
while (1) {
    think();
    getforks();
    eat();
    putforks();
}
```

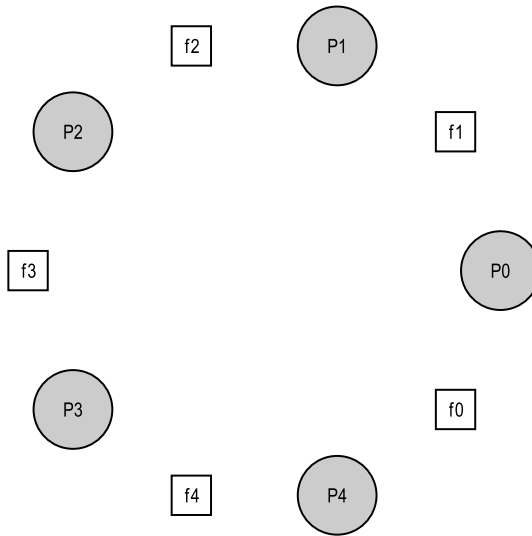


Рис. 31.14 ❖ Обедаящие философы

Требуется написать функции `getforks()` и `putforks()`, так чтобы избежать взаимоблокировки, чтобы ни один философ не умер с голоду, не получая доступа к пище, и чтобы степень конкурентности была высокой (т. е. чтобы в каждый момент времени возможность покушать была у максимального количества философов).

Повторяя ход рассуждений Дауни [D08], заведем несколько вспомогательных функций, которые приблизят нас к решению:

```
int left(int p) { return p; }
int right(int p) { return (p + 1) % 5; }
```

Когда философ p желает обратиться к левой от себя вилке, он вызывает `left(p)`. А для обращения к правой вилке философ вызывает `right(p)`; оператор деления по модулю правильно обрабатывает случай, когда последний философ ($p=4$) пытается взять вилку справа от себя, т. е. вилку 0.

Для решения задачи нам еще понадобятся семафоры. Заведем пять семафоров, по числу вилок: `sem_t forks[5]`.

Неправильное решение

Предпримем первую попытку. Инициализируем каждый семафор (в массиве `forks`) значением 1. Предположим также, что каждый философ знает свой номер (p). Тогда функции `getforks()` и `putforks()` можно написать, как показано на рис. 31.15.

К такому решению приводят следующие интуитивные соображения (неправильные). Чтобы получить вилки, мы просто захватываем «блокировку» на каждую: сначала на левую, потом на правую. А когда покушаем, освобож-

даем их. Просто, не правда ли? К сожалению, в данном случае «просто» не значит «правильно». Видите ли вы, в чем проблема?

```

1 void getforks() {
2     sem_wait(forks[left(p)]);
3     sem_wait(forks[right(p)]);
4 }
5
6 void putforks() {
7     sem_post(forks[left(p)]);
8     sem_post(forks[right(p)]);
9 }

```

Рис. 31.15 ❖ Функции `getforks()` и `putforks()`

Проблема во **взаимоблокировке**. Если каждый философ возьмет вилку слева от себя раньше, чем успеет взять правую, то все они будут обладать одной вилкой и вечно ждать друг друга. Действительно, философ 0 берет вилку 0, философ 1 – вилку 1, философ 2 – вилку 2, философ 3 – вилку 3, философ 4 – вилку 4; все вилки разобраны, и каждый философ ждет вилку, которой завладел его сосед. Вскоре мы изучим эту взаимоблокировку подробнее, а пока достаточно того, что такое решение не годится.

Решение: разрыв зависимости

Чтобы решить задачу, проще всего изменить порядок захвата вилок хотя бы одним философом, именно так решил ее сам Дейкстра. Именно, предположим, что философ 4 (с наибольшим номером) берет вилки в *другом* порядке. Этой идее соответствует такой код:

```

1 void getforks() {
2     if (p == 4) {
3         sem_wait(forks[right(p)]);
4         sem_wait(forks[left(p)]);
5     } else {
6         sem_wait(forks[left(p)]);
7         sem_wait(forks[right(p)]);
8     }
9 }

```

Поскольку последний философ пытается взять правую вилку раньше левой, не возникает ситуации, когда каждый философ завладевает одной вилкой и тщетно ждет другой; порочный круг ожидания разорван. Поразмыслите об этом решении и убедитесь, что оно работает.

Есть и другие «знаменитые» задачи, например **задача о курильщиках** и **задача о спящем парикмахере**. По большей части они просто дают повод задуматься о конкурентности и иногда имеют образные названия. Поищите в сети, если хотите узнать больше или попрактиковаться в конкурентных рассуждениях [D08].

31.7. КАК РЕАЛИЗУЮТСЯ СЕМАФОРЫ

Наконец, воспользуемся низкоуровневыми примитивами синхронизации, блокировками и условными переменными для построения собственной версии семафоров, которую назовем... (*барабанная дробь*)... **земафорами**. Задача решается довольно просто, как видно из рис. 31.16.

```

1 typedef struct __Zem_t {
2     int value;
3     pthread_cond_t cond;
4     pthread_mutex_t lock;
5 } Zem_t;
6
7 // only one thread can call this
8 void Zem_init(Zem_t *s, int value) {
9     s->value = value;
10    Cond_init(&s->cond);
11    Mutex_init(&s->lock);
12 }
13
14 void Zem_wait(Zem_t *s) {
15     Mutex_lock(&s->lock);
16     while (s->value <= 0)
17         Cond_wait(&s->cond, &s->lock);
18     s->value--;
19     Mutex_unlock(&s->lock);
20 }
21
22 void Zem_post(Zem_t *s) {
23     Mutex_lock(&s->lock);
24     s->value++;
25     Cond_signal(&s->cond);
26     Mutex_unlock(&s->lock);
27 }
```

Рис. 31.16 ❖ Реализация земафоров
с помощью блокировок и условных переменных

Мы используем одну блокировку и одну условную переменную плюс переменную состояния, в которой запоминается значение семафора. Изучите этот код и убедитесь, что вы его понимаете. Вперед!

Между нашими земафорами и чистыми семафорами, определенными Дейкстрой, имеется одно тонкое различие: мы не поддерживаем инвариант, согласно которому отрицательное значение семафора равно числу потоков, стоящих к нему в очереди; на самом деле значение нашего земафора не бывает меньше нуля. Такое поведение проще реализовать, и оно совпадает с текущей реализацией в Linux.

Совет: осторожнее с обобщениями

Абстрактная идея обобщения бывает весьма полезна при проектировании систем, когда удачное решение можно немного обобщить и распространить на более широкий класс задач. Но не следует увлекаться; Лэмпсон предостерегает нас: «Не обобщайте; обобщения в общем случае неверны» [L83].

Семафоры можно было бы рассматривать как обобщения блокировок и условных переменных, но нужно ли такое обобщение? А учитывая сложность реализации условной переменной поверх семафора, может статься, что это обобщение не такое общее, как кажется.

Любопытно, что построение условных переменных с помощью семафоров – гораздо более трудное упражнение. Некоторые весьма опытные в конкурентном программировании разработчики пытались сделать это в Windows, но дело кончилось многочисленными и разнообразными ошибками [B04]. Попробуйте сами – и вы поймете, почему конструирование условных переменных из семафоров сложнее, чем могло бы показаться.

31.8. РЕЗЮМЕ

Семафоры – мощный и гибкий примитив для написания конкурентных программ. Некоторые программисты только ими и пользуются, сторонясь блокировок и условных переменных, поскольку ценят их за простоту и полезность.

В этой главе мы рассмотрели несколько классических задач и их решения. Если вам показалось недостаточно, то есть много других источников информации. Один из лучших (и притом бесплатных) – книга Аллена Дауни о конкурентности и программировании с помощью семафоров [D08]. В ней множество задач, которые вы можете порешать, чтобы лучше понять семафоры и конкурентность в целом. Чтобы стать настоящим специалистом по конкурентности, требуются годы работы, поэтому если вы не будете ограничиваться материалом этого курса, то, без сомнения, сделаете важный шаг на пути к овладению этой темой.

Литература

[B04] «Implementing Condition Variables with Semaphores» by Andrew Birrell. December 2004. *Интересная статья на тему о том, как трудно реализовать условные переменные с помощью семафоров, и об ошибках, которые допустили на этом пути автор с коллегами. Кстати, Биррелл известен (среди прочего) различными руководствами по многопоточному программированию.*

[CB08] «Real-world Concurrency» by Bryan Cantrill, Jeff Bonwick. ACM Queue. Volume 6, No. 5. September 2008. *Статья, написанная разработчиками ядра из компании, когда-то называвшейся Sun, о реальных проблемах, с которыми они сталкивались в конкурентном коде.*

[CHP71] «Concurrent Control with Readers and Writers» by P. J. Courtois, F. Heymans, D. L. Parnas. Communications of the ACM, 14:10, October 1971. *Введение в задачу о читателях и писателях, содержащее ее простое решение. В позднейших работах были предложены более изощренные решения, которые здесь опущены из-за своей сложности.*

[D59] «A Note on Two Problems in Connexion with Graphs» by E. W. Dijkstra. Numerische Mathematik 1, 269271, 1959. Доступно по адресу <http://www-m3.ma.tum.de/twiki/pub/MN0506/WebHome/dijkstra.pdf>. *Можете ли вы поверить, что люди работали над алгоритмами еще в 1959 году? Мы не можем. Еще до того, как компьютеры стали забавой, эти люди понимали, что когда-нибудь машины сделают мир другим...*

[D68a] «Go-to Statement Considered Harmful» by E.W. Dijkstra. CACM, volume 11(3), March 1968. <http://www.cs.utexas.edu/users/EWD/ewd02xx/EWD215.PDF>. *Иногда высказанная мысль кладет начало целой отрасли программной инженерии.*

[D68b] «The Structure of the THE Multiprogramming System» by E.W. Dijkstra. CACM, volume 11(5), 1968. *Одна из первых работ, показавших, что системное программирование в информатике требует серьезных интеллектуальных усилий. Здесь же приводятся убедительные аргументы в пользу модульности в форме многоуровневого строения систем.*

[D72] «Information Streams Sharing a Finite Buffer» by E. W. Dijkstra. Information Processing Letters 1, 1972. <http://www.cs.utexas.edu/users/EWD/ewd03xx/EWD329.PDF>. *Изобрел ли Дейкстра всё вообще? Нет, но, пожалуй, был близок к этому. Безусловно, он первым ясно указал на проблемы в конкурентном коде. Однако правда и то, что люди, занимавшиеся практическим проектированием операционных систем, знали о многих проблемах, описанных Дейкстрой, поэтому приписывать ему все заслуги было бы, пожалуй, исторической несправедливостью.*

[D08] «The Little Book of Semaphores» by A. B. Downey. Доступно по адресу <http://greenteapress.com/semaphores/>. *Симпатичная (и бесплатная!) книга о семафорах. Множество интересных задач для самостоятельного решения для тех, кому это нравится.*

[D71] «Hierarchical ordering of sequential processes» by E. W. Dijkstra. Доступно по адресу <http://www.cs.utexas.edu/users/EWD/ewd03xx/EWD310.PDF>. *Здесь поставлено много задач на тему конкурентности, включая и задачу об обедающих философах. Статья в Википедии на эту тему тоже весьма информативна.*

[GR92] «Transaction Processing: Concepts and Techniques» by Jim Gray, Andreas Reuter. Morgan Kaufmann, September 1992. *Точная цитата, которая кажется нам исполненной особого юмора, находится на стр. 485, в начале раздела 8.8: «Первые мультимикропроцессоры, появившиеся примерно в 1960 году, имели команду проверить и установить... предположительно разработчики ОС разработали соответствующие алгоритмы, хотя обычно изобретение семафоров приписывают Дейкстре, несмотря на то что это случилось спустя много лет». Вот же блин!*

[H87] «Aspects of Cache Memory and Instruction Buffer Performance» by Mark D. Hill. Ph. D. Dissertation, U. C. Berkeley, 1987. *Диссертация Хилла, которая будет интересна тем, кто поглощен кешированием в ранних системах. Прекрасный пример количественного исследования.*

[L83] «Hints for Computer Systems Design» by Butler Lampson. ACM Operating Systems Review, 15:5, October 1983. *Лэмпсон, знаменитый системщик, обожал использовать рекомендации при проектировании вычислительных систем. Рекомендация (hint) – это утверждение, которое чаще всего правильно, но может быть и ложным. При такой интерпретации `signal()` сообщает ожидающему потоку о том, что условие, которого он ждет, изменилось, но не стоит слепо доверять тому, что новое состояние совпадает с ожидаемым. В этой статье, посвященной проектированию систем, содержится одна из общих рекомендаций Лэмпсона – использовать рекомендации. Между прочим, не такой нелепый совет, как кажется.*

Домашнее задание (код)

В этом домашнем задании вы будете использовать семафоры для решения некоторых хорошо известных проблем конкурентности. Многие из них взяты из замечательной книжки Дауни «Little Book of Semaphores»¹, в которой сведены классические задачи и предложено несколько новых вариантов.

Во всех задачах предлагается заготовка кода, а вы должны дописать код, работающий с заданными семафорами. В Linux можно использовать платформенные семафоры, а в Mac (где семафоры не поддерживаются) нужно сначала написать свою реализацию (с помощью блокировок и условных переменных, как описано в этой главе). Успехов!

Вопросы

1. Для начала реализуйте и протестируйте решение **задачи о разветвлении и соединении**, описанной в тексте главы. Хотя это решение приведено в тексте, самостоятельный ввод кода – полезное упражнение; даже Бах переписывал ноты Вивальди – будущий маэстро учился у здравствующего. Детали см. в файле `fork-join.c`. Добавьте обращение к `sleep(1)` в код потомка, чтобы убедиться, что он работает.
2. Теперь рассмотрим небольшое обобщение – **задачу о randevu**, которая ставится следующим образом: имеется два потока, каждый из которых собирается войти в точку randevu в программе. Ни один не должен покинуть эту область раньше, чем другой войдет в нее. Подумайте, как решить задачу с помощью двух семафоров, детали см. в файле `rendezvous.c`.
3. Сделаем еще один шаг и реализуем общее решение задачи о **барьерной синхронизации**. Пусть в последовательном участке кода имеются две точки, P_1 и P_2 . Размещение барьера между P_1 и P_2 гарантирует, что все

¹ Доступна по адресу <http://greenteapress.com/semaphores/downey08semaphores.pdf>.

потоки выполняют P_1 , прежде чем хотя бы один выполнит P_2 . Ваша задача: написать функцию `barrier()`, которую можно использовать описанным образом. Можно предполагать, что известно общее число потоков в программе N и что все N потоков пытаются подойти к барьеру. В решении, вероятно, будет использовано два семафора и несколько целых чисел для подсчета. Детали см. в файле `barrier.c`.

4. Теперь решим задачу о **читателях и писателях**, также описанную в тексте. При первой попытке не думайте о возможном зависании. Детали см. в файле `reader-writer.c`. Включите в код вызовы `sleep()`, демонстрирующие, что код работает, как задумано. Можете ли вы продемонстрировать зависание некоторых потоков (невозможность получить в свое распоряжение процессор)?
5. Снова рассмотрим задачу о читателях и писателях, но на этот раз уделим внимание зависанию. Как гарантировать, что все читатели и писатели в конечном итоге будут продвигаться вперед? Детали см. в файле `reader-writer-nostarve.c`.
6. Воспользуйтесь семафорами для построения **справедливого мьютекса**, гарантирующего, что всякий поток, пытающийся захватить мьютекс, в конечном итоге преуспееет. Дополнительные сведения см. в файле `mutex-nostarve.c`.
7. Вам понравились эти задачи? В бесплатной книге Дауни есть еще много таких. И не забывайте получать удовольствие! Впрочем, вы ведь всегда его получаете, когда пишете код, разве нет?

Типичные ошибки в конкурентных программах

Исследователи потратили немало времени и сил на изучение ошибок, связанных с конкурентностью. Значительная часть этой работы была посвящена **взаимоблокировкам** – проблеме, которую мы затрагивали в предыдущих главах, а сейчас намереваемся рассмотреть глубже [С+71]. В более поздних работах изучаются другие типичные ошибки в конкурентных программах (помимо взаимоблокировок). В этой главе мы кратко рассмотрим примеры ошибок в реальных программах, чтобы лучше понимать, на что обращать особое внимание.

СУЩЕСТВО ПРОБЛЕМЫ:

КАК БЫТЬ С ТИПИЧНЫМИ ОШИБКАМИ В КОНКУРЕНТНЫХ ПРОГРАММАХ

Есть несколько типов ошибок, связанных с конкурентностью. Знать, чего остерегаться, – первый шаг на пути к написанию надежного и правильного конкурентного кода.

32.1. КАКИЕ БЫВАЮТ ОШИБКИ?

Сразу напрашивается вопрос: как классифицировать ошибки, встречающиеся в сложных конкурентных программах? В общем случае на этот вопрос трудно ответить, но, к счастью, эту работу за нас уже проделали. Мы будем опираться на исследование Lu et al. [L+08], в котором детально проанализирован ряд популярных конкурентных приложений с целью понять, какие типы ошибок встречаются на практике.

В работе рассматриваются четыре крупных и важных приложения с открытым исходным кодом: MySQL (популярная система управления базами данных), Apache (хорошо известный веб-сервер), Mozilla (знаменитый веб-браузер) и OpenOffice (свободная версия комплекта офисных программ

MS Office, которой действительно пользуются). Авторы изучают связанные с конкурентностью дефекты, которые были найдены и исправлены в каждой из этих программ, превращая тем самым труд разработчика в количественный анализ дефектов. Полученные результаты помогут нам понять, какие проблемы реально встречаются в зрелых кодовых базах.

На рис. 32.1 показана сводная информация об ошибках, изученных Лю с сотрудниками. Видно, что всего было рассмотрено 105 ошибок, большая часть которых (74) не связана с взаимоблокировками, а остальные 31 связаны. Кроме того, мы видим, что в разных приложениях число ошибок неодинаково: в OpenOffice с конкурентностью связано всего 8 ошибок, а в Mozilla – почти 60.

Приложение	Что делает	Не взаимоблокировки	Взаимоблокировки
MySQL	Сервер баз данных	14	9
Apache	Веб-сервер	13	4
Mozilla	Веб-браузер	41	16
OpenOffice	Офисные программы	6	2
Всего		74	31

Рис. 32.1 ❖ Ошибки в современных приложениях

Далее мы рассмотрим эти классы ошибок (не взаимоблокировки и взаимоблокировки) более пристально. В первом случае мы будем иллюстрировать обсуждение примерами, взятыми из исследования, а во втором обсудим длинный перечень того, что было сделано для предотвращения или обработки взаимоблокировок.

32.2. Ошибки, НЕ СВЯЗАННЫЕ С ВЗАИМОБЛОКИРОВКОЙ

Такие ошибки, согласно исследованию Лю, составляют большинство дефектов, связанных с конкурентностью. Но что это за ошибки? Как они возникают? Как их исправить? Мы обсудим два основных типа ошибок этого класса: **нарушение атомарности** и **нарушение порядка**.

Ошибки нарушения атомарности

Рассмотрим простой пример ошибки, встретившейся в MySQL. Прежде чем читать объяснение, попробуйте понять, в чем ошибка.

```

1 Поток 1::
2 if (thd->proc_info) {
3     ...
4     fputs(thd->proc_info, ...);
5     ...

```



```

6 }
7
8 Поток 2::
9 thd->proc_info = NULL;

```

Здесь два потока обращаются к полю `proc_info` в структуре `thd`. Первый поток проверяет, равно ли это поле `NULL`, и если нет, то печатает значение. Вторым устанавливает поле в `NULL`. Очевидно, что если первый поток выполнит проверку, а затем будет прерван до обращения к `fputs`, то второй поток вклинится и сбросит указатель в `NULL`. Когда первый поток возобновит работу, он аварийно завершится при попытке `fputs` разыменовать нулевой указатель.

Формально нарушение атомарности в работе Лю и др. определено так: «Нарушена желаемая сериализуемость нескольких операций доступа к памяти (т. е. участок кода предполагается атомарным, но во время выполнения атомарность не обеспечивается)». В примере выше в коде имеется *предположение атомарности* (по выражению Лю) при проверке отличия `proc_info` от `NULL` и использовании `proc_info` в обращении к `fputs()`; если это предположение неверно, то код работает не так, как задумано.

Исправление проблем подобного типа часто (но не всегда) не вызывает трудностей. Понимаете ли вы, как исправить приведенный выше код?

В решении ниже мы просто окружили доступ к разделяемой переменной блокировками, так что любое обращение к полю `proc_info` из потока производится под защитой блокировки (`proc_info_lock`). Конечно, захватывать эту блокировку должен также любой другой код, обращающийся к данной структуре.

```

1 pthread_mutex_t proc_info_lock = PTHREAD_MUTEX_INITIALIZER;
2
3 Поток 1::
4 pthread_mutex_lock(&proc_info_lock);
5 if (thd->proc_info) {
6     ...
7     fputs(thd->proc_info, ...);
8     ...
9 }
10 pthread_mutex_unlock(&proc_info_lock);
11
12 Поток 2::
13 pthread_mutex_lock(&proc_info_lock);
14 thd->proc_info = NULL;
15 pthread_mutex_unlock(&proc_info_lock);

```

Ошибка нарушения порядка

Еще один распространенный тип ошибок, выявленный Лю и др., называется **нарушением порядка**. Ниже приведен простой пример; сможете ли вы понять, почему этот код неправилен?

```

1 Поток 1::
2 void init() {
3     ...

```

```

4   mThread = PR_CreateThread(mMain, ...);
5   ...
6 }
7
8 Поток 2::
9 void mMain(...) {
10    ...
11    mState = mThread->State;
12    ...
13 }

```

Как вы, наверное, поняли, код в Поток 2 предполагает, что переменная `mThread` уже инициализирована (и не равна `NULL`). Однако если Поток 2 начнет работать сразу после создания, то `mThread` не будет присвоено никакого значения при доступе из функции `mMain()`, поэтому программа, скорее всего, аварийно завершится при попытке разыменовать нулевой указатель. И это мы еще предположили, что `mThread` инициализирована значением `NULL`; если же это не так, то при разыменовании в Поток 2 могут происходить еще более странные вещи, поскольку программа обращается к произвольному адресу в памяти.

Формальное определение нарушения порядка звучит так: «Изменен желаемый порядок выполнения двух (групп) операций доступа к памяти (т. е. А должна выполняться раньше В, но этот порядок никак не обеспечен)» [L+08].

Чтобы исправить ошибку, обычно нужно принудительно гарантировать определенный порядок. Как мы подробно обсуждали ранее, простой и надежный способ такой синхронизации дают **условные переменные**. Приведенный выше пример можно было бы переписать таким образом.

```

1 pthread_mutex_t mtLock = PTHREAD_MUTEX_INITIALIZER;
2 pthread_cond_t mtCond  = PTHREAD_COND_INITIALIZER;
3 int mtInit             = 0;
4
5 Thread 1::
6 void init() {
7     ...
8     mThread = PR_CreateThread(mMain, ...);
9
10    // сигнализировать о создании потока...
11    pthread_mutex_lock(&mtLock);
12    mtInit = 1;
13    pthread_cond_signal(&mtCond);
14    pthread_mutex_unlock(&mtLock);
15    ...
16 }
17
18 Thread 2::
19 void mMain(...) {
20    ...
21    // ждать инициализации потока...
22    pthread_mutex_lock(&mtLock);
23    while (mtInit == 0)

```

```

24     pthread_cond_wait(&mtCond, &mtLock);
25     pthread_mutex_unlock(&mtLock);
26
27     mState = mThread->State;
28     ...
29 }

```

Здесь мы добавили блокировку (`mtLock`) и ассоциированную с ней условную переменную (`mtCond`), а также переменную состояния (`mtInit`). В процессе инициализации `mtInit` присваивается значение 1 и сигнализируется о том, что это сделано. Если Поток 2 начнет работать раньше, то будет ждать сигнала и соответствующего изменения состояния, а если позже, то проверит состояние, увидит, что инициализация уже состоялась (т. е. `mtInit = 1`), и продолжит спокойно работать. Заметим, что можно было бы использовать в качестве переменной состояния саму `mThread`, но ради простоты не стали этого делать. Если порядок выполнения потоков имеет значение, то на помощь приходят условные переменные (или семафоры).

Ошибки, не связанные с взаимоблокировкой: резюме

Среди изученных Лю и др. ошибок, не связанных с взаимоблокировкой, значительная доля (97 %) – нарушения атомарности или порядка. Поэтому, помня об этих видах ошибок, программисты имеют больше шансов избежать их. Кроме того, вновь разрабатываемые инструменты проверки кода, вероятно, будут обращать особое внимание на такие ошибки, поскольку они являются наиболее типичными.

К сожалению, не все ошибки исправляются так легко, как в примерах выше. Некоторые требуют более глубокого понимания работы программы либо значительной реорганизации кода или структур данных. Дополнительные сведения см. в блестящей (и понятно написанной) статье Лю и др.

32.3. Ошибки, связанные с взаимоблокировкой

Помимо вышеупомянутых ошибок, в конкурентных программах со сложными протоколами блокировки часто встречается классическая проблема **взаимоблокировки**. Например, это может случиться, когда Поток 1 удерживает блокировку (L1) и ждет освобождения другой блокировки (L2), а в то же время Поток 2, удерживающий блокировку L2, ждет освобождения L1. В следующем фрагменте демонстрируется потенциальная взаимоблокировка:

Поток 1:	Поток 2:
<code>pthread_mutex_lock(L1);</code>	<code>pthread_mutex_lock(L2);</code>
<code>pthread_mutex_lock(L2);</code>	<code>pthread_mutex_lock(L1);</code>

Заметим, что при выполнении этого кода взаимоблокировки может и не случиться; она возможна, но не предопределена, и произойдет, например, когда Поток 1 успешно захватил L1, после чего произошло контекстное переключение на Поток 2. В этот момент Поток 2 захватывает L2 и пытается захватить L1. В итоге имеем взаимоблокировку, поскольку каждый поток ждет другого и ни один не может сдвинуться с места. Наглядно это представлено на рис. 32.2; наличие **цикла** в графе является признаком взаимоблокировки.

Проблема хорошо видна из этого рисунка. Но каким образом следует писать код, чтобы не нарваться на взаимоблокировку?

СУЩЕСТВО ПРОБЛЕМЫ: КАК БЫТЬ С ВЗАИМБЛОКИРОВКОЙ

Как нужно строить системы, чтобы предотвратить взаимоблокировки или, по крайней мере, обнаружить их и восстановить нормальное выполнение? Является ли эта проблема насущной в современных системах?

Почему возникают взаимоблокировки?

Вы, наверное, думаете, что таких простых взаимоблокировок, как показанная выше, избежать легко. Например, если потоки 1 и 2 всегда будут захватывать блокировки в одном и том же порядке, то взаимоблокировка никогда не случится. Так почему же они возникают?

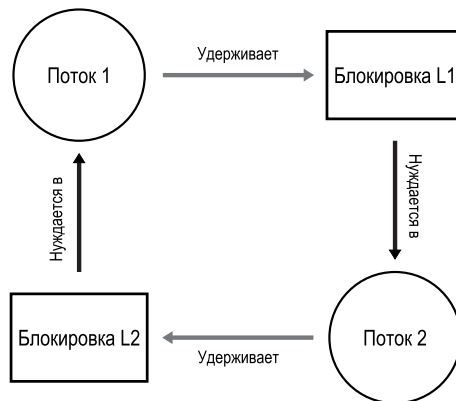


Рис. 32.2 ❖ Граф зависимостей при взаимоблокировке

Одна из причин заключается в том, что в больших кодовых базах между компонентами имеются сложные зависимости. Взять, к примеру, операционную систему. Подсистема виртуальной памяти может обратиться к файловой системе, чтобы подкачать блок с диска; впоследствии файловая система

может затребовать страницу памяти, чтобы прочитать в нее блок, и таким образом обратится к подсистеме виртуальной памяти. Поэтому в больших системах проектировать стратегии блокировки нужно очень внимательно, чтобы избежать взаимоблокировок из-за циклических зависимостей, естественно возникающих в коде.

Еще одна причина – сама природа **инкапсуляции**. Разработчики программного обеспечения научены скрывать детали реализации, поскольку так легче строить модульные программы. К сожалению, модульность плохо уживается с блокировкой. Как показано в работе Julia et al. [J+08], некоторые невинные, на первый взгляд, интерфейсы являются чуть ли не приглашением к взаимоблокировке. Возьмем, например, класс `Vector` в Java и его метод `AddAll()`. Он вызывается следующим образом:

```
Vector v1, v2;
v1.AddAll(v2);
```

Внутри, поскольку этот метод должен быть потокобезопасным, захватываются блокировки на оба вектора `v1` и `v2`. Код захватывает блокировки в каком-то произвольном порядке (скажем, сначала `v1`, потом `v2`), чтобы добавить содержимое `v2` в `v1`. Если какой-то другой поток вызовет примерно в то же время метод `v2.AddAll(v1)`, то возникнет угроза взаимоблокировки, причем так, что вызывающее приложение об этом даже не узнает.

Условия возникновения взаимоблокировки

Для возникновения взаимоблокировки необходимо выполнение четырех условий [C+71]:

- **взаимное исключение.** Потоки требуют монопольного контроля над необходимыми им ресурсами (например, захватывают блокировку);
- **ожидание с удержанием.** Потоки удерживают выделенные им ресурсы (например, уже захваченные блокировки) и при этом ожидают выделения дополнительных ресурсов (например, хотят захватить новые блокировки);
- **отсутствие вытеснения.** Ресурсы (например, блокировки) невозможно насильно отобрать у потоков, которые их удерживают;
- **циклическое ожидание.** Существует замкнутая цепочка потоков – такая, что каждый поток удерживает один или несколько ресурсов (например, блокировок), которые хочет получить следующий поток в цепочке.

Если хотя бы одно из этих четырех условий не выполняется, взаимоблокировки не случится. Таким образом, мы для начала рассмотрим методы *предотвращения* взаимоблокировок; все описываемые ниже стратегии имеют общую цель: предотвратить наступление одного из вышеперечисленных условий.

Предотвращение

Циклическое ожидание

Пожалуй, самым практически полезным методом предотвращения (и уж точно самым часто применяемым) является написание кода захвата блокировок таким образом, чтобы циклическое ожидание никогда не возникало. И проще всего это сделать, определив **полное упорядочение** захвата блокировок. Например, если в системе имеется всего две блокировки (L1 и L2), то для предотвращения взаимоблокировки достаточно всегда первой захватывать L1, а потом L2. Такое строгое упорядочение гарантирует отсутствие циклического ожидания, а значит, и взаимоблокировки.

Разумеется, в более сложных системах количество блокировок больше, поэтому обеспечить их полное упорядочение трудно (да, может быть, и не нужно). Поэтому для избегания взаимоблокировок полезно **частичное упорядочение** захвата. Отличный практический пример частичного упорядочения блокировок можно найти в коде отображения памяти в Linux [Т+94]; в комментариях в начале исходного кода описано десять групп порядка захвата блокировок, начиная с таких простых, как «i_mutex раньше i_mmap_mutex», и кончая такими сложными, как «i_mmap_mutex раньше private_lock раньше swap_lock раньше mapping->tree_lock».

Понятно, что и полное, и частичное упорядочения требуют тщательного проектирования стратегий блокировки. Кроме того, упорядочение – не более чем соглашение, а небрежный программист легко может проигнорировать протокол блокировки и тем самым открыть возможность для взаимоблокировки. Наконец, для упорядочения блокировок нужно глубоко понимать кодовую базу и порядок вызова различных функций; достаточно одной ошибки – и взаимоблокировка тут как тут.

Совет: упорядочивайте блокировки по адресу

Бывает, что функция должна захватить две (или более) блокировки, это уже причина для повышенного внимания к возможности взаимоблокировки. Допустим, что функция вызывается следующим образом: `do_something(mutex t *m1, mutex t *m2)`. Если код всегда захватывает m1 раньше m2 (или m2 раньше m1 – но тоже всегда), то возможна взаимоблокировка, потому что один поток мог бы вызвать `do_something(L1, L2)`, а другой – `do_something(L2, L1)`.

Чтобы избежать этой конкретной проблемы, умный программист может упорядочить блокировки по *адресу* в памяти. Захватывая сначала блокировку с меньшим адресом, а потом с большим (или наоборот), `do_something()` может гарантировать, что блокировки всегда захватываются в одном и том же порядке, вне зависимости от того, как они переданы. Код мог бы выглядеть следующим образом:

```
if (m1 > m2) { // захватывать сначала блокировку с большим адресом
    pthread_mutex_lock(m1);
    pthread_mutex_lock(m2);
} else {
```

```
pthread_mutex_lock(m2);
pthread_mutex_lock(m1);
}
// Предполагается, что m1 != m2 (т. е. это не одна и та же блокировка)
```

Эта техника дает простую и эффективную реализацию захвата нескольких блокировок без риска взаимоблокировки.

Ожидание с удержанием

Избежать ожидания с удержанием можно, захватывая все блокировки разом – атомарно. На практике это могло бы выглядеть так:

```
1 pthread_mutex_lock(prevention); // начать захват блокировок
2 pthread_mutex_lock(L1);
3 pthread_mutex_lock(L2);
4 ...
5 pthread_mutex_unlock(prevention); // закончить
```

Захватив сначала блокировку `prevention`, этот код гарантирует, что поток не будет прерван в середине последовательности захватов, и тем самым предотвращает возникновение взаимоблокировки. Конечно, при этом требуется, чтобы любой поток перед захватом блокировок захватил сначала глобальную блокировку `prevention`. Например, если другой поток попытается захватить блокировки `L1` и `L2` в ином порядке, то ничего страшного не произойдет, потому что в это время он удерживает блокировку `prevention`.

Заметим, что это решение может оказаться проблематичным по нескольким причинам. Как и раньше, инкапсуляция работает против нас: при вызове функции мы должны точно знать, какие блокировки нужно удерживать, и захватить их заранее. Кроме того, при таком подходе может снизиться степень конкурентности, потому что блокировки захватываются предварительно (и все сразу), а не тогда, когда они действительно необходимы.

Отсутствие вытеснения

Захват нескольких блокировок часто приводит к беде, потому что, ожидая одну блокировку, мы удерживаем другую. Многие библиотеки для работы с потоками предлагают более гибкий набор интерфейсов, чтобы избежать такой ситуации. Именно, функция `pthread_mutex_trylock()` либо успешно захватывает блокировку (если та свободна), либо возвращает код ошибки, указывающий, что блокировка занята; в последнем случае можно попробовать захватить ее еще раз позже.

Такой интерфейс позволяет следующим образом построить свободный от взаимоблокировок устойчивый к порядку протокол захвата блокировок:

```
1 top:
2 pthread_mutex_lock(L1);
3 if (pthread_mutex_trylock(L2) != 0) {
```

```

4     pthread_mutex_unlock(L1);
5     goto top;
6 }

```

Заметим, что если этому же протоколу следует еще один поток, но захватывает блокировки в другом порядке (L2, затем L1), то взаимоблокировки все равно не возникнет. Однако появляется новая проблема: **активная блокировка** (livelock). Может случиться (хотя это и маловероятно), что оба потока будут раз за разом пытаться выполнить эту последовательность команд, но ни разу не смогут захватить обе блокировки. В этом случае обе системы все время что-то делают (так что это не взаимоблокировка, когда все стоят), но прогресса все равно нет. Для проблемы активной блокировки тоже есть решения: например, можно добавить случайную задержку перед переходом на начало цикла, уменьшив тем самым вероятность периодической интерференции конкурирующих потоков.

Сделаем одно замечание по поводу этого решения: мы стыдливо уклонились от обсуждения трудных вопросов, сопряженных с использованием trylock. Первая проблема, которая, скорее всего, возникнет, вновь связана с инкапсуляцией: если одна из блокировок спрятана глубоко внутри какой-то вызываемой функции, то осуществить возврат в начало цикла будет нелегко. Если код попутно захватил еще какие-то ресурсы (помимо L1), то нужно позаботиться об их освобождении тоже; например, если после захвата L1 код выделил память, то после неудачной попытки захватить L2, но до перехода на метку top для повторения всей последовательности эту память нужно будет освободить. Впрочем, в некоторых конкретных случаях (например, в вышеупомянутом методе класса Vector в Java Vector) это решение работает прекрасно.

Вероятно, вы также заметили, что этот подход не означает какого-то реального вытеснения (принудительного отобрания блокировки у владеющего ей потока), он лишь позволяет разработчику отказаться от владения блокировкой (т. е. вытеснить себя добровольно). Однако на практике этот подход полезен, потому мы и включили его, несмотря на все несовершенства.

Взаимное исключение

Последний метод предотвращения – вообще избежать взаимного исключения. Мы знаем, что в общем случае это трудно, поскольку в программах действительно встречаются критические секции. Так что же делать?

Херлихи придумал, как можно спроектировать различные структуры данных вообще без блокировок [Н91, Н93]. Идея **безблокировочных** подходов (и родственных им подходов **без ожидания**) проста: воспользовавшись мощными аппаратными командами, построить структуры данных способом, при котором явные блокировки не нужны.

В качестве простого примера предположим, что имеется команда «сравнить и обменять», которая, как вы, возможно, помните, атомарно выполняет следующие действия:

```

1 int CompareAndSwap(int *address, int expected, int new) {
2     if (*address == expected) {
3         *address = new;

```



```

4     return 1; // успех
5 }
6     return 0; // ошибка
7 }

```

Теперь допустим, что нам нужно атомарно увеличить значение на некоторую величину. Это можно сделать так:

```

1 void AtomicIncrement(int *value, int amount) {
2     do {
3         int old = *value;
4     } while (CompareAndSwap(value, old, old + amount) == 0);
5 }

```

Вместо того чтобы захватывать блокировку, выполнять обновление, а затем освобождать блокировку, мы поступили иначе: повторно пытаемся обновить значение с помощью команды «сравнить и обменять». При этом никакая блокировка не захватывается, так что и взаимоблокировка невозможна (хотя активная блокировка все же может возникнуть).

Рассмотрим чуть более сложный пример: вставку в список. Ниже показано, как вставить элемент в начало списка:

```

1 void insert(int value) {
2     node_t *n = malloc(sizeof(node_t));
3     assert(n != NULL);
4     n->value = value;
5     n->next = head;
6     head = n;
7 }

```

Код простой, но если его «одновременно» будут выполнять несколько потоков, то возникнет состояние гонки (видите ли вы, в каком месте?). Конечно, проблему можно было бы решить, обернув критическую секцию захватом и освобождением блокировки:

```

1 void insert(int value) {
2     node_t *n = malloc(sizeof(node_t));
3     assert(n != NULL);
4     n->value = value;
5     pthread_mutex_lock(listlock); // начало критической секции
6     n->next = head;
7     head = n;
8     pthread_mutex_unlock(listlock); // конец критической секции
9 }

```

Здесь мы используем блокировки традиционным способом¹. Но давайте вместо этого выполним вставку в безблокировочной манере, воспользовавшись командой «сравнить и обменять». Вот один из возможных подходов:

¹ Внимательный читатель может спросить, почему мы захватываем блокировку так поздно, а не прямо после входа в `insert()`. Предлагаем внимательному читателю ответить на вопрос, почему это правильно. Например, какие предположения делает код относительно вызова `malloc()`?

```

1 void insert(int value) {
2     node_t *n = malloc(sizeof(node_t));
3     assert(n != NULL);
4     n->value = value;
5     do {
6         n->next = head;
7     } while (CompareAndSwap(&head, n->next, n) == 0);
8 }

```

Здесь код изменяет указатель на следующий элемент, так чтобы он указывал на текущее начало списка, а затем пытается обменять вновь созданный узел, сделав его новым началом списка. Однако это не получится, если тем временем какой-то другой поток успешно заменил начальный элемент, и тогда первому потоку придется повторить вставку в новое начало списка.

Конечно, для построения полезного списка одной вставки недостаточно, и неудивительно, что безблокировочная реализация вставки, удаления и поиска в списке – нетривиальная задача. Дополнительные сведения о синхронизации без блокировок и ожидания можно найти в обширной литературе на эту тему [H01, H91, H93].

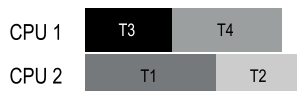
Избегание взаимоблокировок с помощью планирования

Вместо предотвращения взаимоблокировок в некоторых случаях лучше их **избегать**. Для этого нужно иметь глобальную информацию о том, какие блокировки могут затребовать различные потоки в ходе своего выполнения, и планировать эти потоки таким образом, чтобы взаимоблокировка гарантированно не возникла.

Например, предположим, что имеется два процессора и четыре потока, которые нужно на них запланировать. Предположим далее, что мы знаем, что Поток 1 (T1) захватывает блокировки L1 и L2 (в каком-то порядке), T2 также захватывает L1 и L2, T3 захватывает только L2, а T4 вообще ничего не захватывает. Мы можем представить требования потоков к захвату блокировок в табличном виде:

	T1	T2	T3	T4
L1	да	да	нет	нет
L2	да	да	да	нет

Умный планировщик мог бы отсюда сделать вывод, что если T1 и T2 не будут работать одновременно, то взаимоблокировка заведомо не случится. Вот возможный пример планирования:



Заметим, что T3 может перекрываться с T1 или с T2. Хотя T3 и захватывает блокировку L2, взаимоблокировка при конкурентной работе с другими потоками все равно невозможна, потому что он захватывает только одну блокировку.

Рассмотрим еще один пример, в котором степень конкуренции за те же ресурсы (блокировки L1 и L2) выше:

	T1	T2	T3	T4
L1	да	да	да	нет
L2	да	да	да	нет

Именно, каждый из потоков T1, T2 и T3 в какой-то момент своего выполнения хочет захватить блокировки L1 и L2. Ниже показан вариант планирования, при котором взаимоблокировка точно не возникнет:



Как видим, статическое планирование приводит к консервативному решению, когда потоки T1, T2 и T3 выполняются на одном процессоре, поэтому суммарное время работы оказывается заметно больше. Хотя возможность конкурентного выполнения потоков имеется, из-за опасения взаимоблокировки мы от нее отказываемся, а ценой является производительность.

Знаменитый пример такого подхода дает предложенный Дейстрой алгоритм банкира [D64], а в литературе описано еще много подобных примеров. К сожалению, они полезны только в очень специфических обстоятельствах, например во встраиваемой системе, когда весь набор задач и необходимых им блокировок известен заранее. Кроме того, эти подходы ограничивают степень конкурентности, как мы видели во втором примере выше. Поэтому избегание взаимоблокировок посредством планирования нельзя назвать широко распространенным универсальным решением.

СОВЕТ: НЕ ВСЕГДА НАДО СТРЕМИТЬСЯ К СОВЕРШЕНСТВУ (ЗАКОН ТОМА УЭСТА)

Тому Уэсту, герою классической книги компьютерной отрасли «Soul of a New Machine» [K81], принадлежит знаменитое высказывание: «Не все достойное быть сделанным достойно быть сделано хорошо», и эту блистательную максиму стоит запомнить любому инженеру. Если изредка случается какая-то неприятность, это не значит, что нужно тратить гигантские усилия на ее предотвращение, особенно если цена неприятности невелика. С другой стороны, если вы конструируете космический челнок, а ценой неправильного действия может стать взрыв челнока, то, пожалуй, этот совет стоит проигнорировать.

Некоторые читатели возражат: «Э, да вы, похоже, предлагаете посредственность в качестве решения!» Быть может, они правы, и к таким рекомендациям следует относиться настороженно. Но наш опыт показывает, что в мире техники, где большую роль играют сроки и другие практические соображения, всегда приходится решать, какие части системы сделать хорошо, а о каких подумать когда-нибудь потом. Самое трудное – знать, что когда делать, а эта интуиция приходит с опытом и преданностью своему делу.

Найди и исправь

И последняя наша стратегия заключается в том, чтобы смириться с возможностью редких взаимоблокировок, а меры предпринимать после того, как эта неприятность обнаружена. Например, если бы ОС зависала раз в год, то мы могли бы просто перезагрузить ее и бодро (или ворча) продолжить работу. Если взаимоблокировки – редкость, то такое «не решение» является вполне прагматичным выходом.

Во многих системах баз данных применяются методы обнаружения взаимоблокировок и восстановления после них. Детектор взаимоблокировок запускается периодически, строит граф ресурсов и ищет в нем циклы. Если цикл существует (взаимоблокировка), то систему нужно перезапустить. Если сначала требуется более тонкое исправление структур данных, то к процессу можно привлечь человека.

Дополнительные сведения о конкурентности в базах данных, о взаимоблокировках и смежных вопросах можно найти в литературе [B+87, K87]. Почитайте эти работы, а еще лучше – запишитесь на курс по базам данных, чтобы больше узнать об этой насыщенной и интересной теме.

32.4. РЕЗЮМЕ

В этой главе мы изучили различные типы ошибок, встречающихся в конкурентных программах. Ошибки первого класса, не связанные с взаимоблокировками, встречаются на удивление часто, но исправлять их обычно проще. Это нарушения атомарности, когда последовательность команд, которая должна была бы выполняться как единое целое, на самом деле так не выполняется, и нарушения порядка, когда не обеспечивается требуемый порядок выполнения двух потоков.

Мы также кратко обсудили взаимоблокировку: почему она возникает и что с этим делать. Проблема стара, как сама конкурентность, на эту тему написаны сотни статей. На практике лучшее решение – проявлять осторожность, установить порядок захвата блокировок и тем самым предотвратить саму возможность взаимоблокировки. Многообещающими являются также подходы без ожидания, и некоторые структуры данных без ожидания уже проложили себе путь в популярные библиотеки и критически важные системы, включая и Linux. Однако недостаток общности и сложность разработки новых безблокировочных структур данных, скорее всего, ограничат полезность этого подхода. Быть может, лучшее решение – разработка новых моделей конкурентного программирования: в системах типа MapReduce [GD02] программист может описать некоторые типы параллельных вычислений вообще без блокировок. Блокировки несут проблемы в силу самой своей природы; наверное, их следует избегать, если есть какое-то другое решение.

Литература

[B+87] «Concurrency Control and Recovery in Database Systems» by Philip A. Bernstein, Vassos Hadzilacos, Nathan Goodman. Addison-Wesley, 1987. *Классическая книга, посвященная конкурентности в системах управления базами данных. Вы, наверное, понимаете, что конкурентность, взаимоблокировки и смежные вопросы в области баз данных – это отдельный мир. Откройте его для себя.*

[C+71] «System Deadlocks» by E. G. Coffman, M. J. Elphick, A. Shoshani. ACM Computing Surveys, 3:2, June 1971. *Классическая статья, в которой сформулированы условия возникновения взаимоблокировок и описано, что с этим можно сделать. Конечно, есть и более ранние работы на эту тему, ссылки на них приведены в статье.*

[D64] «Een algorithmie ter voorkoming van de dodelijke omarming» by Edsger Dijkstra. 1964. Доступно по адресу <http://www.cs.utexas.edu/users/EWD/ewd01xx/EWD108.PDF>. *Дейкстра не только предложил ряд решений проблемы взаимоблокировки, но и первым обратил внимание на ее существование, по крайней мере в письменной форме. Он, правда, называл эту ситуацию «смертельными объятиями», в которые (слава богу) не угодил.*

[GD02] «MapReduce: Simplified Data Processing on Large Clusters» by Sanjay Ghemawat, Jeff Dean. OSDI '04, San Francisco, CA, October 2004. *Статья о MapReduce положила начало эре крупномасштабной обработки данных, в ней же описана инфраструктура для выполнения таких вычислений с помощью кластера, вообще говоря, ненадежных компьютеров.*

[H01] «A Pragmatic Implementation of Non-blocking Linked-lists» by Tim Harris. International Conference on Distributed Computing (DISC), 2001. *Относительно современный пример трудностей, возникающих при построении такой простой структуры, как конкурентный связный список, без применения блокировок.*

[H91] «Wait-free Synchronization» by Maurice Herlihy. ACM TOPLAS, 13:1, January 1991. *В работе Херлихи впервые высказаны идеи, лежащие в основу подходов к написанию конкурентных программ без ожидания. Эти решения оказались сложными и зачастую сталкиваются с большими трудностями, чем правильное использование блокировок, поэтому их практическое использование, скорее всего, будет ограниченным.*

[H93] «A Methodology for Implementing Highly Concurrent Data Objects» by Maurice Herlihy. ACM TOPLAS, 15:5, November 1993. *Хороший обзор структур без блокировок и ожидания. В обоих случаях блокировки не используются, но решения без ожидания труднее реализовать, потому что они пытаются гарантировать, что любая операция с конкурентной структурой завершится за конечное число шагов (т. е. не возникнет закливания).*

[J+08] «Deadlock Immunity: Enabling Systems To Defend Against Deadlocks» by Horatiu Julia, Daniel Tralamazza, Cristian Zamfir, George Candea. OSDI '08, San

Diego, CA, December 2008. *Прекрасная недавняя статья о взаимоблокировках и о том, как избежать попадания в один и тот же капкан в конкретной системе.*

[K81] «Soul of a New Machine» by Tracy Kidder. Backbay Books, 2000 (reprint of 1980 version). *Обязательно к прочтению любым конструктором систем и инженером. Описывается, как в давние времена группа в компании Data General (DG), возглавляемая Томом Уэстом, работала над созданием «новой машины». Другие книги Киддера тоже великолепны, в т. ч. «Mountains beyond Mountains» Или вы с нами не согласны?*

[K87] «Deadlock Detection in Distributed Databases» by Edgar Knapp. ACM Computing Surveys, 19:4, December 1987. *Прекрасный обзор обнаружения взаимоблокировок в распределенных системах баз данных. Содержит ссылки на другие работы, поэтому является хорошей отправной точкой для изучения данной темы.*

[L+08] «Learning from Mistakes – A Comprehensive Study on Real World Concurrency Bug Characteristics» by Shan Lu, Soyeon Park, Eunsoo Seo, Yuanyuan Zhou. ASPLOS '08, March 2008, Seattle, Washington. *Первое углубленное изучение ошибок, связанных с конкурентностью, в реальных программах. Легла в основу этой главы. Загляните на страницы Яньянь Чжоу или Шань Лю, там вы найдете много интересных статей об ошибках.*

[T+94] «Linux File Memory Map Code» by Linus Torvalds and many others. Доступно по адресу <http://lxr.free-electrons.com/source/mm/filemap.c>. *Спасибо Майклу Уолфишу из Нью-Йоркского университета за ссылку на этот бесценный пример. Как видно из данного файла, реальный мир может оказаться малость сложнее, чем кристальная ясность, представляющая в учебниках...*

Домашнее задание (код)

В этом домашнем задании вы будете исследовать реальный код, в котором возникают взаимоблокировки (или удастся их избежать). Разные версии кода соответствуют разным подходам к предотвращению взаимоблокировок в простой функции `vector_add()`. Подробные сведения об этих программах и их общей основе имеются в файле README.

Вопросы

1. Сначала убедитесь, что понимаете общие принципы работы этих программ и некоторые основные параметры. Изучите код в файлах `vector-deadlock.c`, `main-common.c` и связанных с ними. Теперь выполните программу `./vector-deadlock -n 2 -l 1 -v`, создающую два потока (`-n 2`), каждый из которых добавляет в вектор один элемент (`-l 1`), причем делает это в режиме подробной диагностики (`-v`). Убедитесь, что понимаете напечатанный результат. Как изменяется результат от запуска к запуску?

2. Добавьте флаг `-d` и измените число итераций (`-l`), сделав его больше 1. Что происходит? Возникает ли взаимоблокировка (при каждом запуске)?
3. Как изменение числа потоков (`-n`) влияет на результат программы? Существуют ли такие значения `-n`, при которых гарантируется отсутствие взаимоблокировки?
4. Теперь рассмотрим код в файле `vector-global-order.c`. Разберитесь, что пытается сделать программа; понимаете ли вы, почему она избегает взаимоблокировки? Еще вопрос: почему в функции `vector_add()` выделен специальный случай, когда исходный и конечный векторы совпадают?
5. Запустите программу со следующими флагами: `-t -n 2 -l 100000 -d`. Сколько времени она выполняется? Как изменяется общее время при увеличении количества циклов или количества потоков?
6. Что будет, если добавить флаг распараллеливания (`-p`)? Как, на ваш взгляд, должна измениться производительность, если каждый поток будет работать над добавлением в разные векторы (это и делает флаг `-p`), а не в один и тот же?
7. Теперь изучим программу `vector-try-wait.c`. Для начала убедитесь, что понимаете код. Действительно ли необходимо первое обращение к `pthread_mutex_trylock()`? Теперь выполните программу. Насколько быстро она работает по сравнению с глобальным упорядочением? Как изменяется число повторных попыток, подсчитываемое в программе, при увеличении числа потоков?
8. Перейдем к файлу `vector-avoid-hold-and-wait.c`. Какова главная проблема такого подхода? Сравните производительность с другими версиями при наличии флага `-p` и без него?
9. Наконец, рассмотрим файл `vector-nolock.c`. В этой версии блокировки не используются вовсе, но совпадает ли семантика кода с другими версиями? Объясните свой ответ.
10. Теперь сравните производительность этой версии с другими, когда потоки работают с одними и теми же двумя векторами (без флага `-p`) и когда каждый поток работает с отдельными векторами (с флагом `-p`). Какова производительность этой безблокировочной версии?

Глава 33

Событийно-управляемая конкурентность (материал повышенной сложности)

До сих пор мы представляли дело так, будто единственный способ создания конкурентных приложений – использование потоков. Но, как многое в жизни, это не вся правда. В графических приложениях [O96], а также в некоторых типах интернет-серверов [PDZ99] применяется другой стиль конкурентного программирования. Этот стиль, известный под названием «**событийно-управляемая конкурентность**», стал популярен в некоторых современных системах, в т. ч. основанных на сервере **node.js** [N13], но корнями он уходит в системы на базе C и Unix, обсуждаемые ниже.

Событийно-управляемая конкурентность решает двоякую проблему. Во-первых, управление конкурентностью в многопоточных приложениях – нетривиальная задача; мы уже видели, что возможны разнообразные проблемы, в т. ч. пропущенные блокировки, взаимоблокировки и т. д. Во-вторых, в многопоточном приложении программист почти или совсем не контролирует, какой поток будет запланирован в данный момент времени; он просто создает потоки и надеется, что ОС будет разумно распределять их между имеющимися процессорами. А учитывая, как трудно сконструировать планировщик общего назначения, который хорошо работает на всех рабочих нагрузках, неудивительно, что иногда ОС решает эту задачу неоптимально.

Существо проблемы: как построить конкурентный сервер без потоков

Как построить конкурентный сервер, не используя потоков, и тем самым восстановить контроль над конкурентностью, а также избежать некоторых проблем, преследующих многопоточные приложения?

33.1. Основная идея: цикл событий

Как уже сказано, мы будем использовать **событийно-управляемую конкурентность**. Подход довольно прост: мы ждем, когда произойдет некое «событие», а дождавшись, проверяем тип события и обрабатываем его – это обычно небольшой объем работы (отправка запросов ввода-вывода, планирование следующих событий и т. д.). Вот и всё!

Прежде чем переходить к деталям, рассмотрим, как выглядит канонический событийно-управляемый сервер. В основе подобных приложений лежит **цикл событий**. Вот его псевдокод:

```
while (1) {
    events = getEvents();
    for (e in events)
        processEvent(e);
}
```

Действительно просто. В главном цикле программа ждет возможности что-то сделать (вызвав функцию `getEvents()`), а по мере поступления событий обрабатывает их одно за другим в **обработчиках событий**. Важно, что в каждый момент времени в системе обрабатывается только одно событие и одновременно с этим ничего не происходит, поэтому решение о том, какое событие обработать следующим, эквивалентно планированию. Этот явный контроль над планированием – одно из фундаментальных преимуществ событийно-управляемого подхода.

Но тогда остается большой вопрос: как именно событийно-управляемый сервер узнаёт, какие события имеют место, в частности связанные с сетью и дисковым вводом-выводом? То есть откуда сервер знает, что ему поступило событие?

33.2. Важный API: `select()` (или `poll()`)

Помня о главном цикле событий, мы теперь должны решить вопрос о том, как получать события. В большинстве систем имеется API, построенный на базе системного вызова `select()` или `poll()`.

Эти интерфейсы позволяют программе проверить, имеется ли операция ввода-вывода, требующая ее внимания. Например, сетевому приложению (скажем, веб-серверу) нужно проверять, пришел ли сетевой пакет, чтобы обслужить его. Именно это и позволяют сделать эти системные вызовы.

Возьмем, к примеру, `select()`. На странице руководства (для Mac) этот API описывается следующим образом:

```
int select(int nfds,
           fd_set *restrict readfds,
           fd_set *restrict writefds,
           fd_set *restrict errorfds,
           struct timeval *restrict timeout);
```

И далее следует такой текст: *select()* *смотрит, имеются ли в наборах дескрипторов ввода-вывода, переданных в аргументах readfds, writefds и errorfds, дескрипторы, готовые соответственно для чтения, для записи или содержащие информацию об ошибках. В каждом наборе проверяются первые nfds дескрипторов с индексами от 0 до nfds-1. После возврата из select() переданные наборы дескрипторов заменяются подмножествами, включающими только дескрипторы, готовые для соответствующей операции. select() возвращает общее число готовых дескрипторов во всех наборах.*

ОТСТУПЛЕНИЕ: БЛОКИРУЮЩИЕ И НЕБЛОКИРУЮЩИЕ ИНТЕРФЕЙСЫ

Блокирующие (или **синхронные**) интерфейсы выполняют всю работу до возврата управления вызывающей стороне, а неблокирующие (или **асинхронные**) начинают работу, но сразу же возвращают управление, так что основная ее часть выполняется в фоновом режиме.

Обычно блокирующие вызовы связаны с обработкой ввода-вывода. Например, если для завершения операции вызов должен прочитать данные с диска, то он может блокировать выполнение программы, пока не придет ответ на запрос ввода-вывода к диску.

Неблокирующие интерфейсы можно использовать при любом стиле программирования (в частности, совместно с потоками), но в случае событийно-управляемого подхода это на- сущная необходимость, потому что иначе вызов заблокировал бы работу всей программы.

Сделаем два замечания относительно *select()*. Во-первых, этот вызов позволяет проверить, какие дескрипторы готовы для *чтения* или для *записи*; в первом случае сервер может узнать, что прибыл новый пакет, нуждающийся в обработке, а во втором – что уже можно отвечать (т. е. в исходящей очереди есть место).

Во-вторых, обратите внимание на аргумент *timeout*. Часто он задается равным NULL, это означает, что *select()* блокируется неопределенно долго – пока какой-нибудь дескриптор не окажется готов. Однако более надежные серверы обычно задают тайм-аут; так, очень распространена техника задания тайм-аута 0 – тогда *select()* возвращает управление немедленно.

Системный вызов *poll()* очень похож. Детали см. на странице руководства или в книге Стивенса и Раго [SR05].

Любой из этих примитивов дает возможность построить неблокирующий цикл событий, в котором проверяется наличие входящих пакетов, производится чтение сообщений из сокетов и отправка ответов на них.

33.3. Использование *select()*

Чтобы добавить конкретики, воспользуемся *select()*, дабы узнать, на каких сетевых дескрипторах имеются ожидающие обработки сообщения. Простой пример показан на рис. 33.1.

```

1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <sys/time.h>
4 #include <sys/types.h>
5 #include <unistd.h>
6
7 int main(void) {
8     // открыть и подготовить сокеты (не показано)
9     // главный цикл
10    while (1) {
11        // инициализировать fd_set нулями
12        fd_set readFDs;
13        FD_ZERO(&readFDs);
14
15        // установить биты, соответствующие дескрипторам,
16        // интересующим сервер
17        // (для простоты считаем, что таковы все дескрипторы от min до max)
18        int fd;
19        for (fd = minFD; fd < maxFD; fd++)
20            FD_SET(fd, &readFDs);
21
22        // выполнить select
23        int rc = select(maxFD+1, &readFDs, NULL, NULL, NULL);
24
25        // проверить с помощью FD_ISSET(), на каких дескрипторах есть данные
26        int fd;
27        for (fd = minFD; fd < maxFD; fd++)
28            if (FD_ISSET(fd, &readFDs))
29                processFD(fd);
30    }
31 }

```

Рис. 33.1 ❖ Простой код с применением select()

Разобраться в этом коде нетрудно. После инициализации сервер входит в бесконечный цикл. В цикле сначала используется макрос `FD_ZERO()`, чтобы очистить набор файловых дескрипторов, а затем макрос `FD_SET()`, чтобы включить в набор все дескрипторы с номерами от `minFD` до `maxFD`. Этот набор дескрипторов мог бы, к примеру, представлять все сетевые сокеты, требующие внимания сервера. Наконец, сервер вызывает `select()`, чтобы узнать, на каких подключениях имеются ожидающие данные. Затем, вызывая в цикле макрос `FD_ISSET()`, сервер может выяснить, на каких конкретно дескрипторах имеются данные, и обработать их.

Разумеется, реальный сервер был бы сложнее, в нем присутствовал бы код для отправки сообщений, запросов дискового ввода-вывода и много других деталей. Дополнительные сведения об API см. в книге Стивенса и Паго [SR05], а хороший обзор событийно-управляемых серверов – статьи Pai et. al или Welsh et al. [PDZ99, WCB01].

33.4. ПОЧЕМУ ПРОЩЕ? ПОТОМУ ЧТО НЕ НУЖНЫ БЛОКИРОВКИ

При наличии одного CPU в событийно-управляемом приложении отсутствуют проблемы, характерные для конкурентных программ. Действительно, поскольку в каждый момент времени обрабатывается только одно событие, нет необходимости захватывать и освобождать блокировки; событийно-управляемый сервер не может быть прерван другим потоком, потому что он сознательно выбрал однопоточную модель. Так что ошибки, связанные с конкурентностью, столь типичные для многопоточных программ, в событийно-управляемых не возникают.

СОВЕТ: ОСТЕРЕГАЙТЕСЬ БЛОКИРОВАНИЯ В СОБЫТИЙНО-УПРАВЛЯЕМЫХ СЕРВЕРАХ

В событийно-управляемых серверах применяется мелкоструктурный контроль над планированием задач. Но чтобы это было возможным, ни один вызов функции не должен блокировать выполнение вызывающей стороны, иначе вы столкнетесь с недовольными клиентами и вопросом, а точно ли вы прочли эту часть книги.

33.5. ПРОБЛЕМА: БЛОКИРУЮЩИЕ СИСТЕМНЫЕ ВЫЗОВЫ

До этого момента идея событийно-управляемого программирования выглядела великолепно, не так ли? Нужно лишь написать простой цикл и обрабатывать события по мере поступления. А о блокировках вообще забыть! Но вот вопрос: что, если для обработки события нужно выполнить системный вызов, который может заблокировать выполнение?

Например, предположим, что сервер получил от клиента запрос прочитать файл с диска и вернуть его содержимое (очень похоже на простой HTTP-запрос). Для его обслуживания какой-то обработчик события рано или поздно должен будет обратиться к системному вызову `open()` для открытия файла, а затем несколько раз вызвать `read()`, чтобы этот файл прочитать. Когда файл окажется в памяти, сервер, вероятно, начнет отправлять его данные клиенту.

И `open()`, и `read()` отправляют запросы ввода-вывода системе хранения (если требуемые метаданные или данные отсутствуют в памяти), а их обслуживание может занять много времени. Для многопоточного сервера это не проблема: пока один поток ждет завершения ввода-вывода, могут работать другие, так что сервер будет занят. Именно это естественное **перекрывание** ввода-вывода с другими вычислениями и делает многопоточное программирование столь привлекательным.

Но в случае событийно-управляемого подхода других потоков нет, есть только главный цикл событий. А это значит, что когда обработчик события

выполняет блокирующий вызов, в ожидании его завершения блокируется *весь* сервер. Система простаивает, и мы имеем масштабное недоиспользование ресурсов. Поэтому в событийно-управляемых системах действует жесткое правило: никаких блокирующих вызовов.

33.6. РЕШЕНИЕ: АСИНХРОННЫЙ ВВОД-ВЫВОД

Для преодоления этого ограничения во многие операционные системы добавлены новые способы отправки запросов дисковой системе – **асинхронный ввод-вывод**. Эти интерфейсы позволяют приложению отправить запрос ввода-вывода и сразу же вернуть управление, не дожидаясь его завершения, а впоследствии получить уведомление о том, что ввод-вывод завершился.

Например, рассмотрим интерфейс, предлагаемый в Mac (в других ОС есть аналогичные). Весь API построен вокруг базовой структуры `struct aiocb`, или **блока управления асинхронным вводом-выводом** (AIO control block). Упрощенный вариант этой структуры показан ниже (дополнительные сведения см. на страницах руководства).

```
struct aiocb {
    int aio_fildes;      /* дескриптор файла */
    off_t aio_offset;    /* смещение от начала файла */
    volatile void *aio_buf; /* адрес буфера */
    size_t aio_nbytes;   /* длина передачи */
};
```

Чтобы отправить асинхронный запрос чтения из файла, приложение должно сначала заполнить поля структуры: дескриптор файла (`aio_fildes`), смещение от начала файла (`aio_offset`), количество запрашиваемых байтов (`aio_nbytes`) и адрес области в памяти, куда будут помещены результаты чтения (`aio_buf`).

Затем приложение должно выполнить операцию **асинхронного чтения** файла:

```
int aio_read(struct aiocb *aiocbp);
```

Этот вызов пытается выполнить ввод-вывод; в случае успеха он сразу же возвращает управление, и приложение (событийно-управляемый сервер) может продолжить работу.

Но остается еще один кусочек пазла, который нужно поставить на место. Откуда мы узнаем, что ввод-вывод завершен и, следовательно, буфер (на который указывает `aio_buf`) содержит запрошенные данные?

Нужен еще один API. В Mac он называется (не вполне удачно) `aio_error()`:

```
int aio_error(const struct aiocb *aiocbp);
```

Этот системный вызов проверяет, завершился ли запрос, на который указывает `aiocbp`. Если да, то функция возвращает код успеха, 0, иначе – код `EINPROGRESS`. Таким образом, для каждой еще не завершенной асинхронной

операции ввода-вывода приложение должно периодически **опрашивать** систему, вызывая `aio_errgr()`.

Но вы наверняка пришли к выводу, что проверять, завершился ли ввод-вывод, очень утомительно; если в каждый момент времени в программе могут быть десятки, а то и сотни незавершенных операций, то что мы должны делать: непрерывно проверять каждую в цикле, или делать небольшую паузу между проверками, или ...?

Для решения этой проблемы в некоторых системах предлагается подход, основанный на **прерывании**. Для данной цели используются **сигналы** Unix, чтобы информировать приложение о завершении асинхронного ввода-вывода и тем самым избавить его от необходимости раз за разом опрашивать систему. Эта дихотомия «опрос–прерывания» используется и при работе с устройствами, как вы увидите (или уже видели) в главе, посвященной устройствам ввода-вывода.

ОТСТУПЛЕНИЕ: СИГНАЛЫ UNIX

Обширная и удивительная инфраструктура сигналов присутствует во всех современных вариантах Unix. В своей простейшей форме сигналы дают способ взаимодействия с процессом. Именно, сигнал может быть доставлен приложению, при этом приложение приостанавливает текущую работу и начинает выполнять **обработчик сигнала**, а по его завершении возобновляет работу с прерванного места.

У каждого сигнала есть имя, например **HUP** («повесить трубку»), **INT** (прерывание), **SEGV** (ошибка сегментации) и т. д., подробности см. на странице руководства. Интересно, что иногда сигнал посылает само ядро. Например, в случае ошибки сегментации ядро посылает сигнал **SIGSEGV** (добавление префикса **SIG** к имени сигнала – обычная практика). Если программа рассчитана на обработку этого сигнала, то он может как-то отреагировать на извещение об ошибочном поведении (что полезно при отладке). Если программа не рассчитана на обработку сигнала, то выполняется действие по умолчанию – в случае **SEGV** процесс снимается.

Ниже показана программа, которая входит в бесконечный цикл, предварительно установив обработчик сигнала **SIGHUP**:

```
#include <stdio.h>
#include <signal.h>

void handle(int arg) {
    printf("хватит меня будить...\n");
}

int main(int argc, char *argv[]) {
    signal(SIGHUP, handle);
    while (1)
        ; // ничего не делать, кроме перехвата некоторых сигналов
    return 0;
}
```

Послать ей сигнал можно с помощью утилиты **kill** (да, вот такое странное агрессивное имя). При этом главный цикл прерывается и выполняется обработчик `handle()`:

```

prompt> ./main &
[3] 36705
prompt> kill -HUP 36705
stop wakin' me up...
prompt> kill -HUP 36705
stop wakin' me up...
prompt> kill -HUP 36705
stop wakin' me up...

```

О сигналах можно сказать столько, что даже целой главы, не говоря уже об одной страничке, не хватит. Как обычно, есть замечательный источник: книга Стивенса и Rago [SR05]. Почитайте, если хотите узнать больше.

В системах без асинхронного ввода-вывода реализовать событийно-управляемый подход в чистом виде невозможно. Но изобретательные разработчики придумали приемлемые замены. Например, в работе Pai et al. [PDZ99] описан гибридный подход, при котором для обработки сетевых пакетов используются события, а для управления начатыми операциями ввода-вывода – пул потоков. Детали см. в оригинальной статье.

33.7. ЕЩЕ ОДНА ПРОБЛЕМА: УПРАВЛЕНИЕ СОСТОЯНИЕМ

Еще одна проблема событийно-управляемого подхода состоит в том, что такой код обычно сложнее, чем традиционный многопоточный. А причина в том, что когда обработчик события запрашивает асинхронный ввод-вывод, он должен подготовить информацию о состоянии программы, которую сможет использовать обработчик следующего события, когда ввод-вывод завершится. Этой дополнительной работы нет в многопоточных программах, поскольку вся необходимая программе информация находится в стеке потока. В работе Adua et al. [A+02] эта деятельность, принципиально важная для событийно-управляемого программирования, называется **ручным управлением стеком**.

Чтобы прояснить данный момент, рассмотрим простой пример, когда многопоточный сервер должен прочитать данные из файла с дескриптором `fd` и по завершении записать прочитанные данные в сокет с дескриптором `sd`. Код (без проверки ошибок) выглядит так:

```

int rc = read(fd, buffer, size);
rc = write(sd, buffer, size);

```

Как видим, в многопоточной программе эта задача решается тривиально; после возврата из `read()` программа уже знает, в какой сокет записывать данные, потому что эта информация находится в стеке потока (в переменной `sd`).

В событийно-управляемой системе всё не так просто. Для решения той же задачи мы сначала иницилируем асинхронное чтение с помощью описанного выше API. Допустим, что затем мы периодически проверяем результат операции с помощью `io_egrgr()`; когда эта функция сообщает, что чтение закончилось, откуда сервер узнает, что делать дальше?

В работе Adua et al. [A+02] предложено использовать старую языковую конструкцию, называемую **продолжением** [FHK84]. Несмотря на кажущуюся сложность, идея проста: нужно сохранить информацию, необходимую для завершения обработки события, в некоторой структуре данных, а когда событие произойдет (т. е. дисковый ввод-вывод завершится), заглянуть в эту структуру и обработать его.

В данном конкретном случае мы должны записать дескриптор сокета (`sd`) в какую-то структуру данных (например, в хеш-таблицу), индексированную файловым дескриптором (`fd`). Когда операция дискового ввода-вывода завершится, обработчик события обратится к продолжению по файловому дескриптору и получит в ответ дескриптор сокета. В этот момент (наконец-то) сервер сможет довести свое дело до конца: записать данные в сокет.

33.8. КАКИЕ ЕЩЕ ТРУДНОСТИ СОПРЯЖЕНЫ С СОБЫТИЯМИ

Стоит упомянуть еще несколько трудностей, присущих событийно-управляемому подходу. В частности, при переходе от одного к нескольким процессорам простота событийной управляемости частично пропадает. Именно, чтобы воспользоваться несколькими CPU, событийный сервер должен параллельно запустить несколько обработчиков событий, а тогда возникают обычные проблемы синхронизации (например, критические секции) и обычные способы их решения (например, блокировки). Поэтому в современных многоядерных системах простая обработка событий без блокировок невозможна.

Еще одна проблема заключается в том, что событийно-управляемый подход плохо сочетается с некоторыми системными действиями, в т. ч. **страничным обменом**. Так, если в обработчике события возникает отказ страницы, то он окажется заблокированным и, следовательно, весь сервер не сможет ничего делать, пока обработка отказа страницы не завершится. Да, сервер умеет избегать *явных* блокировок, но ничего не может поделать с *неявными*, примером которых является отказ страницы, и если такие ситуации возникают часто, то неминуемы серьезные потери производительности.

Третья проблема состоит в сложности сопровождения событийно-управляемого кода при изменении семантики функций [A+02]. Например, если некая функция, бывшая ранее неблокирующей, становится блокирующей, то обработчик события, вызывающий эту функцию, должен быть адаптирован к ее новой природе, что потребует его разбиения на две части. Поскольку блокирование смертельно для событийно-управляемых серверов, программист

всегда должен быть на чеку, отслеживая семантику API, которыми пользуется каждый обработчик события.

Наконец, хотя ныне асинхронный дисковый ввод-вывод реализован на большинстве платформ, этот путь был нелегким [PDZ99], и простой и единообразной интеграции с асинхронным сетевым вводом-выводом добиться так и не удалось. Например, хотелось бы просто использовать функцию `select()` для управления всеми незавершенными операциями ввода-вывода, но обычно требуется сочетание `select()` для сетевого ввода-вывода и вызовов функций из семейства AIO для дискового.

33.9. РЕЗЮМЕ

Мы представили краткое введение в различные стили конкурентного программирования на основе событий. В событийно-управляемых серверах контроль над планированием передан самому приложению, но ценой некоторого увеличения сложности и трудностей интеграции с другими аспектами современных систем (например, страничной организацией). Из-за этих проблем ни один подход нельзя назвать лучшим, так что потоки и события, вероятно, будут еще много лет существовать как два разных подхода к решению одной и той же проблемы. Чтобы узнать больше, почитайте научные статьи (например, [A+02, PDZ99, vB+03, WCB01]), а еще лучше – попробуйте сами написать событийно-управляемый код.

Литература

[A+02] «Cooperative Task Management Without Manual Stack Management» by Atul Adya, Jon Howell, Marvin Theimer, William J. Bolosky, John R. Douceur. USENIX ATC '02, Monterey, CA, June 2002. *В этой статье впервые были сформулированы некоторые трудности событийно-управляемой конкурентности и предложены простые решения, а также исследована еще более безумная идея о комбинации двух типов управления конкурентностью в одном приложении!*

[FHK84] «Programming With Continuations» by Daniel P. Friedman, Christopher T. Haynes, Eugene E. Kohlbecker. In Program Transformation and Programming Environments, Springer Verlag, 1984. *Классический возврат к старой идее продолжения из мира языков программирования. В некоторых современных языках идея становится все более популярной.*

[N13] «Node.js Documentation» by the folks who built node.js. Available: nodejs.org/api. *Один из многих новых каркасов, позволяющих легко создавать веб-сервисы и приложения. Любой специалист по современным системам должен свободно ориентироваться в таких каркасах (и, вероятно, не в одном). Потратьте некоторое время на разработку в каком-нибудь из них и станьте экспертом.*

[O96] «Why Threads Are A Bad Idea (for most purposes)» by John Ousterhout. Invited Talk at USENIX '96, San Diego, CA, January 1996. *Замечательная лекция на тему о том, почему потоки плохо сочетаются с приложениями на основе GUI (хотя высказанные идеи носят более общий характер). Оустерхаут пришел к этим мыслям, когда разрабатывал Tcl/Tk, очень интересный скриптовый язык и инструментарий, который в сто раз упростил создание графических приложений по сравнению с существовавшими на тот момент средствами. Хотя комплект инструментов Tk GUI все еще жив (например, к нему есть интерфейс из Python), язык Tcl медленно умирает (а жаль).*

[PDZ99] «Flash: An Efficient and Portable Web Server» by Vivek S. Pai, Peter Druschel, Willy Zwaenepoel. USENIX '99, Monterey, CA, June 1999. *Пионерская работа о том, как структурировать веб-сервер в эру наступающего интернета. Прочитайте ее, чтобы разобраться в основах и познакомиться с идеями авторов о построении гибридных решений при отсутствии поддержки асинхронного ввода-вывода.*

[SR05] «Advanced Programming in the Unix Environment» by W. Richard Stevens and Stephen A. Rago. Addison-Wesley, 2005. *И снова отсылаем к классической книге по программированию в системах Unix, которая должна стоять на вашей книжной полке. Если нужно узнать о какой-то детали, то вам сюда.*

[vB+03] «Capriccio: Scalable Threads for Internet Services» by Rob von Behren, Jeremy Condit, Feng Zhou, George C. Necula, Eric Brewer. SOSP '03, Lake George, New York, October 2003. *Статья о том, как добиться высокой степени масштабируемости потоков; возражение против всех проводимых в то время работ по событийно-управляемому программированию.*

[WCB01] «SEDA: An Architecture for Well-Conditioned, Scalable Internet Services» by Matt Welsh, David Culler, and Eric Brewer. SOSP '01, Banff, Canada, October 2001. *Интересные соображения о подходе к обслуживанию запросов, сочетающем потоки, очереди и событийно-управляемую обработку. Некоторые идеи нашли применение в инфраструктуре таких компаний, как Google, Amazon и др.*

Домашнее задание (код)

В данном (коротком) домашнем задании вы приобретете опыт работы с событийно-управляемым кодом и некоторыми важнейшими концепциями в этой области. Удачи!

Вопросы

1. Для начала напишите простой сервер, который может принимать и обслуживать подключения по протоколу TCP. Пошарьте в интернете, если не знаете, как это делается. Сервер должен обслуживать по одному запросу за раз; запросы пусть будут простыми, например вернуть текущее время.

2. Теперь добавим вызов `select()`. Напишите главную программу, которая может принимать несколько соединений, и цикл событий, в котором проверяется, на каких файловых дескрипторах имеются данные, а затем эти данные читаются и обрабатываются. Тщательно протестируйте правильность использования `select()`.
3. Сделайте запросы более интересными, имитирующими простой веб-сервер или сервер файлов. В ответ на каждый запрос сервер должен прочитать содержимое файла (имя которого указано в запросе) в буфер, а затем вернуть его клиенту. При реализации пользуйтесь стандартными системными вызовами `open()`, `read()` и `close()`. Но будьте осторожны: если такой сервер будет работать долго, кто-нибудь может догадаться, как с его помощью прочитать все файлы на вашем компьютере!
4. Теперь вместо стандартных системных вызовов воспользуйтесь интерфейсами асинхронного ввода-вывода, описанными в этой главе. Трудно ли оказалось включить асинхронные интерфейсы в программу?
5. Ради интереса добавьте обработку сигналов. Типичное применение сигналов – побудить сервер перезагрузить конфигурационный файл или предпринять какое-то административное действие. Естественный путь для экспериментов – добавить пользовательский кеш, в котором сервер будет хранить недавно востребованные файлы. Реализуйте обработчик сигналов, который будет очищать кеш в ответ на посланный серверному процессу сигнал.
6. И наконец, трудная часть: как определить, стоило ли построение асинхронного событийно-управляемого сервера затраченных усилий? Можете ли вы придумать эксперимент, демонстрирующий преимущества этого подхода? Оцените дополнительную сложность, сопутствующую вашему подходу.

Глава 34

Итоговый диалог о конкурентности

Профессор. Ну как, голова не разболелась?

Студент (принимая две таблетки Мотрина). *Есть немножко. Трудно представлять, как все эти потоки чередуются.*

Профессор. *Воистину трудно. Я и сам всегда поражаюсь: стоит появиться конкурентному выполнению, как всего несколько строчек кода становится невозможно понять.*

Студент. *Вот и я о том же! Ну, стыдно же – специалист по информатике, и не может разобраться в пяти строчках кода.*

Профессор. *Ну, не надо так себя корить. Ты почитай первые статьи по конкурентным алгоритмам, они почти все содержат ошибки! А ведь авторы часто были профессорами!*

Студент (открыв рот в изумлении). *Профессора могут... э-э-э... допускать ошибки?*

Профессор. *Бывает. Ты только никому не говори – это наша тайна фирмы.*

Студент. *Все-все, рот на замке. Но если о конкурентном коде так трудно рассуждать и если его так трудно написать правильно, то как же мы можем быть уверены, что написали правильную конкурентную программу?*

Профессор. *Вот в чем вопрос-то, верно? Я думаю, начинать надо с нескольких простых вещей. Первое – будь проще! Избегай сложных взаимодействий между потоками, используй хорошо известные и проверенные практикой способы управления потоками.*

Студент. *Вы имеете в виду простые блокировки и, быть может, очередь между производителем и потребителем.*

Профессор. *Вот-вот! Это распространенные парадигмы, и с помощью того, что ты уже знаешь, можно построить работающее решение. И второе – используй конкурентность, только когда это абсолютно необходимо, а по возможности держись от нее подальше. Нет ничего хуже преждевременной оптимизации программы.*

Студент. Понимаю – зачем добавлять потоки, если в них нет нужды?

Профессор. Точно. Третье – если без распараллеливания не обойтись, поищи другие, более простые формы. Например, технология Map-Reduce для параллельного анализа данных – прекрасный пример распараллеливания без ужасающих сложностей, связанных с блокировками, условными переменными и другими противными вещами, о которых мы говорили.

Студент. Map-Reduce, говорите? Интересно – надо будет почитать самостоятельно.

Профессор. Правильно! Почитай. Да и в любом случае, теперь придется этим заняться, поскольку то, что мы рассмотрели, – лишь малая толика обширнейших знаний в этой области. Читай, читай и еще раз читай! А потом экспериментируй, напиши код, а затем напиши еще. Как пишет Гладуэлл в своей книге «Outliers»¹, нужно потратить примерно 10 000 часов, чтобы стать настоящим экспертом в чем-то. Нельзя сделать все за время, отведенное на курсе!

Студент. Уж и не знаю, огорчаться или радоваться. Лучше выберу второе – и за работу! Пойду писать конкурентный код...

¹ Малькольм Гладуэлл. Гении и аутсайдеры. Почему одним все, а другим ничего? М.: Манн, Иванов и Фарбер, 2019.

Часть III



ХРАНЕНИЕ

Глава 35

Диалог о хранении

Профессор. Вот мы и добрались до третьего из четырех... виноват... из трех столпов операционных систем: **хранения**.

Студент. Так сколько столпов-то, три или четыре? Какой четвертый?

Профессор. Три, мой юный ученик, всего три. Будем стараться не усложнять.

Студент. Ну, ладно. Но что такое хранение, о прекрасный и мудрый профессор?

Профессор. Собственно, что это такое в жизни, ты и сам знаешь, правда¹?

Студент. Ну, это как учиться на вашем курсе: требуется упорство.

Профессор. Ха! Это точно. Но у этого слова есть и другой смысл. Попробую объяснить. Представь, что ты где-то на природе и сорвал...

Студент (прерывая). Я знаю! Персик! С персикового дерева!

Профессор. Я собирался сказать «яблоко», с яблони. Ну да ладно, пусть будет по-твоему.

Студент (тупо уставившись).

Профессор. Итак, ты сорвал персик. На самом деле много-много персиков. И хочешь как-то сохранить их надолго. Ведь зима в Висконсине трудная и злая. Что будешь делать?

Студент. Ну, разные есть способы. Можно законсервировать! Или испечь пирог. Или сварить варенье. Ой, такое удовольствие!

Профессор. Удовольствие? Ну, может быть. Но точно тебе придется немало потрудиться, чтобы **сохранить** персики. Вот и с информацией так же; сохранить информацию, несмотря на выход компьютера из строя, на отказы дисков, на сбои электропитания, – трудная и интересная задача.

Студент. Ух ты, какой переход, да вы просто мастер в этом деле.

Профессор. Спасибо! Профессор всегда сумеет найти нужные слова, знаешь ли.

Студент. Попробую запомнить. Я так думаю, хватит уже о персиках, пора поговорить о компьютерах?

Профессор. Да, самое время...

¹ Игра слов. В оригинале употребляется слово «persistence», которое в обыденной речи означает, скорее, настойчивость. И дается его словарное определение «твердое или упорное продолжение действия, несмотря на трудности или сопротивление». – Прим. перев.

Глава 36

Устройства ВВОДА-ВЫВОДА

Прежде чем переходить к основной теме этой части книги (хранению), познакомимся с понятием **устройства ввода-вывода** и покажем, как операционная система может с ним взаимодействовать. Конечно, ввод-вывод – критически важная часть вычислительной системы; ведь программа, которая ничего не вводит, будет каждый раз давать один и тот же результат, а если программа ничего не выводит, то какой смысл был ее запускать? Очевидно, что если мы хотим, чтобы вычислительная система представляла хоть какой-нибудь интерес, она должна что-то вводить и что-то выводить.

СУЩЕСТВО ПРОБЛЕМЫ: КАК ВКЛЮЧИТЬ В СИСТЕМУ ВВОД-ВЫВОД?

Как интегрировать ввод-вывод в систему? Какие имеются общие механизмы? Как сделать их эффективными?

36.1. АРХИТЕКТУРА СИСТЕМЫ

В начале обсуждения рассмотрим «классическую» диаграмму типичной системы (рис. 36.1). Мы видим один процессор, подключенный к основной памяти системы с помощью **шины памяти**. К системе также подключены устройства с помощью **шины ввода-вывода**; во многих современных системах в этом качестве цели используется шина **PCI** (или одна из ее многочисленных производных). Среди этих устройств могут быть графические карты и другие высокоскоростные устройства ввода-вывода. Наконец, еще ниже расположена одна или несколько **периферийных шин**, например **SCSI**, **SATA** или **USB**. С их помощью к системе подключаются сравнительно медленные устройства: **диски, мыши и клавиатуры**.

Возникает вопрос: зачем нужна такая иерархическая структура? Если просто, то дело в физике и в цене. Чем шина быстрее, чем она должна быть короче, поэтому на высокоскоростной шине памяти нет места для подключения устройств. Кроме того, изготовление высокоскоростной шины обходится

очень дорого. Поэтому конструкторы и приняли иерархический подход, при котором компоненты, нуждающиеся в высокой производительности (например, графические карты), располагаются ближе к процессору, а более медленные – дальше от него. У размещения дисков и других медленных устройств на периферийной шине есть много преимуществ: в частности, к ней можно подключить больше устройств.

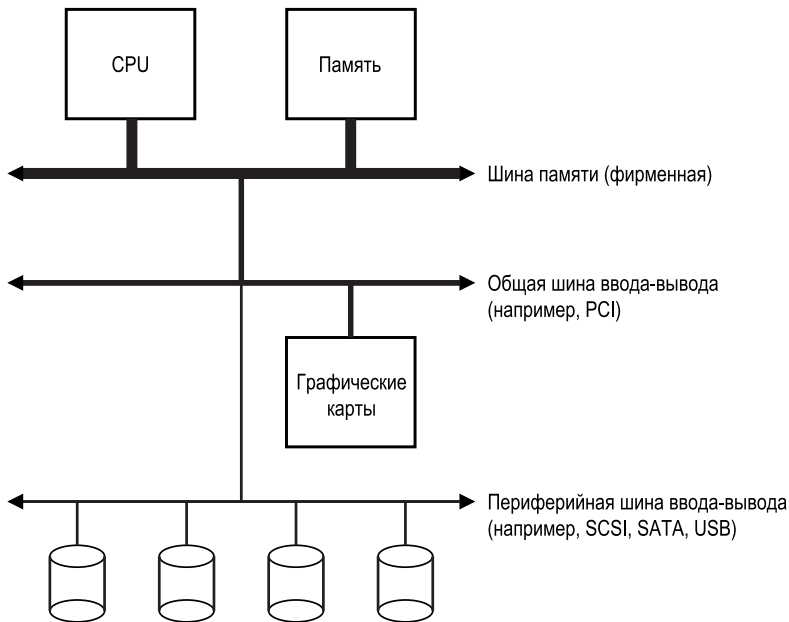


Рис. 36.1 ❖ Архитектура типовой системы

Конечно, в современных системах все чаще используются специализированные наборы микросхем и более быстрые двухточечные межсоединения для повышения производительности. На рис. 36.2 показана упрощенная диаграмма набора микросхем Intel Z270 [H17]. В верхней части CPU соединен с близко расположенными микросхемами памяти, но также имеет высокоскоростное соединение с графической картой (а значит, и с дисплеем) для поддержки игровых (о, ужас!) и других приложений, интенсивно работающих с графикой.

CPU соединен с микросхемой ввода-вывода разработанной Intel шиной **DMI (Direct Media Interface)**, а остальные устройства подключены к этой микросхеме с помощью других межсоединений. Справа показаны жесткие диски, подключенные к системе через интерфейс **eSATA**; интерфейс **ATA (AT Attachment)** – отсылка к IBM PC AT, сменивший его **SATA (Serial ATA)** и, наконец, **eSATA (external SATA)** представляют эволюцию интерфейсов хранения за прошедшие десятилетия, на каждом этапе которой производительность увеличивалась, чтобы соответствовать вновь появляющимся устройствам хранения.

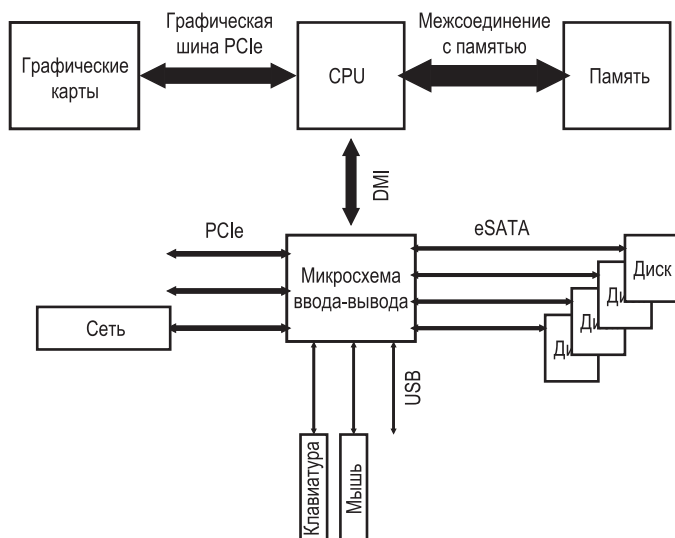


Рис. 36.2 ❖ Архитектура современной системы

Под микросхемой ввода-вывода расположено несколько подключений по шине **USB (Universal Serial Bus)**, предназначенной, например, для присоединения клавиатуры, мыши и других низкоскоростных устройств.

Наконец, слева на рисунке показано подключение других высокоскоростных устройств через интерфейс **PCIe (Peripheral Component Interconnect Express)**. В данном случае так подключена сетевая карта, но часто сюда подключаются и высокопроизводительные устройства хранения (например, **NVMe**).

36.2. КАНОНИЧЕСКОЕ УСТРОЙСТВО

Теперь рассмотрим каноническое устройство (не настоящее) и на его примере постараемся разобраться в механизмах, которые необходимы для эффективного взаимодействия с устройством. На рис. 36.3 показано, что устройство состоит из двух основных компонентов. Первый – аппаратный **интерфейс**, предоставляемый остальным частям системы. Как и программы, оборудование должно предоставлять какой-то интерфейс, чтобы программные компоненты могли управлять его работой. Поэтому все устройства имеют четко определенный интерфейс и протокол взаимодействия.

Второй компонент любого устройства – его **внутренняя структура**. Она зависит от реализации и отвечает за реализацию абстракции, которую устройство предлагает системе. В случае очень простых устройств функциональность реализуется одной или несколькими микросхемами, а устройства посложнее включают простой процессор, память общего назначения и несколько специализированных микросхем. Например, современный RAID-контроллер может насчитывать сотни тысяч строк **прошивки** (программного обеспечения внутри аппаратного изделия), реализующей его функциональность.



Рис. 36.3 ❖ Каноническое устройство

36.3. КАНОНИЧЕСКИЙ ПРОТОКОЛ

На рисунке выше интерфейс (упрощенный) состоит из трех регистров: регистр **состояния** можно читать для получения текущего состояния устройства; регистр **команд** сообщает устройству, что нужно выполнить определенное действие; регистр **данных** служит для передачи данных устройств или для получения данных от него. Читая и записывая эти регистры, операционная система может управлять поведением устройства.

Опишем теперь типичное взаимодействие ОС с устройством, призванное побудить устройство выполнить какую-нибудь операцию. Протокол выглядит так:

```
while (STATUS == BUSY)
    ; // ждать, когда устройство освободится
Записать данные в регистр DATA
Записать команду в регистр COMMAND
    (В результате устройство активизируется и выполняет команду)
while (STATUS == BUSY)
    ; // ждать, когда устройство закончит выполнять запрос
```

Протокол состоит из четырех шагов. На первом ОС ждет, когда устройство будет готово принять команду, для чего в цикле проверяет регистр состояния; это называется **опросом** устройства (по существу ОС спрашивает, как дела). На втором шаге ОС записывает данные в регистр данных; если бы речь шла о диске, то понадобилось бы несколько операций записи, чтобы поместить блок данных (скажем, размером 4 КБ) на устройство. Если в перемещении данных участвует главный CPU (как в этом примере протокола), то говорят о **программируемом вводе-выводе** (programmed I/O – PIO). На третьем шаге ОС помещает команду в регистр команд; это является для устройства неявным указанием на то, что данные переданы и можно начинать выполнение команды. Наконец, ОС ждет завершения команды, снова опрашивая устройство в цикле (и в результате получит код успеха или ошибки).

Базовый протокол хорош тем, что прост и работает. Но в некоторых отношениях он неудобен и неэффективен. Первая проблема – неэффективность опроса, процессор тратит много времени, просто ожидая, когда устройство (возможно, медленное) закончит работу, вместо того чтобы переключиться на какой-нибудь готовый к выполнению процесс.

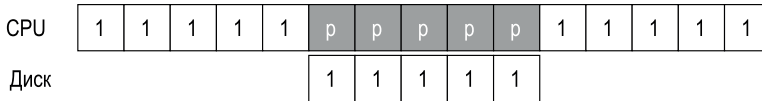
Сущность проблемы: как избежать дорогостоящего опроса

Как ОС может проверить состояние устройства, не выполняя опроса, и тем самым снизить затраты CPU на обслуживание устройства?

36.4. ПРЕРЫВАНИЯ ПОМОГАЮТ СНИЗИТЬ ЗАТРАТЫ CPU

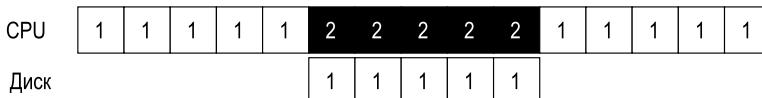
Инженеры много лет назад придумали, как улучшить это взаимодействие; это механизм **прерываний**, с которым мы уже встречались. Вместо того чтобы опрашивать устройство в цикле, ОС выдает команду, усыпляет вызывающий процесс и переключает контекст, переходя к следующей задаче. Когда устройство завершит операцию, оно возбудит аппаратное прерывание, в результате чего CPU перейдет по предопределенному адресу **программы обработки прерывания** (interrupt service routine – **ISR**), или, проще, **обработчика прерывания**. Обработчик – это часть операционной системы, которая завершает выполнение запроса (например, читает данные и, возможно, код ошибки, возвращенный устройством) и будит процесс, ожидающий результата ввода-вывода.

Поэтому прерывания открывают возможность **перекрывтия** вычислений и ввода-вывода, что является ключом к эффективному использованию процессора. Ниже схематически показана хронология действий, иллюстрирующая проблему:



На этом рисунке Процесс 1 некоторое время занимает CPU (это показано повторяющейся цифрой 1 в строке line), а затем инициирует запрос дискового ввода-вывода для чтения данных. Не будь прерываний, система просто крутилась бы в цикле, опрашивая устройство и ожидая завершения ввода-вывода (показано буквой р). Диск обслуживает запрос, и в конце концов Процесс 1 возобновляет выполнение.

Но если воспользоваться прерываниями и допустить перекрывтие, то ОС может делать что-то еще, пока диск выполняет операцию:



В данном случае ОС отдает CPU Процессу 2, пока диск обрабатывает запрос Процесса 1. По завершении обработки возникает прерывание, и ОС будит Процесс 1 и возобновляет его выполнение. Таким образом, на среднем участке хронологии в полную силу работают и диск, и ОС.

Отметим, что прерывания не *всегда* являются наилучшим решением. Например, представьте устройство, которое выполняет операции очень быстро, так что уже на первой итерации цикла опроса процессор обнаруживает, что устройство все сделало. В таком случае прерывание лишь *замедлило* бы работу системы, поскольку переключение на другой процесс, обработка прерывания и возврат к исходному процессу – все это дорогостоящие действия. Поэтому быстрое устройство лучше опрашивать, а при работе с медленным использовать прерывания. Если скорость устройства неизвестна или иногда оно работает медленно, а иногда быстро, то оптимальным будет **гибридное** решение: сначала несколько раз опросить, а если оно занято, перейти в режим прерывания. Такой **двухфазный** подход позволяет взять лучшее из обоих миров.

Совет: прерывания не всегда лучше PIO

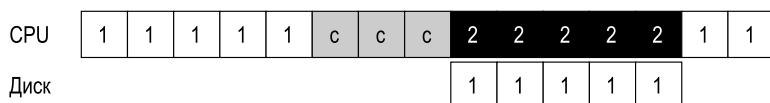
Хотя прерывания допускают перекрытие вычислений и ввода-вывода, использовать их имеет смысл только для относительно медленных устройств. В противном случае стоимость обработки прерываний и контекстного переключения может перевесить выгоду. Также бывают ситуации, когда система не успевает справляться с потоком прерываний, что приводит к активной блокировке [MR96]; в таких случаях опрос дает ОС больше контроля над планированием и, стало быть, выгоду.

Еще одна причина не использовать прерывания свойственна сетям [MR96]. Если каждый из огромного потока входящих пакетов будет генерировать прерывание, то ОС может оказаться в ситуации **активной блокировки**, когда занята только обработкой прерываний и не дает пользовательским процессам возможности поработать и обслужить запросы. Например, представьте себе веб-сервер, который испытывает резкий рост нагрузки, т. е. попал в верхнюю строчку хакерских новостей [H18]. В таком случае лучше время от времени использовать опрос для лучшего контроля над происходящим в системе и дать серверу шанс обслужить несколько запросов, прежде чем снова обратить внимание на устройство и проверить, пришли ли еще пакеты.

Существует также оптимизация, называемая **объединением прерываний**. В этом случае устройство, прежде чем генерировать прерывание, немного ждет. Во время ожидания могут завершиться другие запросы, и тогда устройство сможет доставить сразу несколько прерываний, объединенных в одно, снизив тем самым накладные расходы на обработку прерываний. Конечно, если ждать слишком долго, то увеличится задержка выполнения запроса, но это стандартный компромисс при проектировании систем. Хороший обзор см. в статье Ahmad et al. [A+11].

36.5. БОЛЕЕ ЭФФЕКТИВНОЕ ПЕРЕМЕЩЕНИЕ ДАННЫХ С ПОМОЩЬЮ ПДП

К сожалению, у канонического протокола есть еще один недостаток, требующий внимания. Именно, при использовании программируемого ввода-вывода (PIO) для передачи устройству большого блока данных приходится снова занимать CPU для решения довольно тривиальной задачи, а потому тратить много времени, которое лучше было бы направить на выполнение других процессов. Хронология проблемы показана на рисунке ниже.



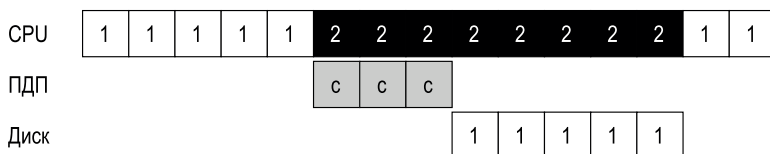
Мы видим, что Процесс 1 сначала работает, а потом хочет записать данные на диск. Он иницирует операцию ввода-вывода, которая должна явно скопировать данные из памяти на устройство, по одному слову за раз (обозначено буквой *c* на диаграмме). По завершении копирования начинается ввод-вывод на диск, а CPU можно использовать для чего-то другого.

СУЩЕСТВО ПРОБЛЕМЫ: КАК СНИЗИТЬ ИЗДЕРЖКИ РЮ

При программируемом вводе-выводе CPU тратит слишком много времени на перемещение данных на устройство и от него. Как можно снять с процессора эту нагрузку и тем самым повысить эффективность его использования?

Эта проблема решается с помощью технологии **прямого доступа к памяти (ПДП, англ. DMA)**. Контроллер ПДП – это очень специфическое устройство в системе, которое координирует передачу данных между устройствами и основной памятью почти без участия CPU.

ПДП работает следующим образом. Чтобы передать данные устройству, ОС программирует контроллер ПДП, сообщая ему, где в памяти находятся данные, сколько данных копировать и какому устройству их передать. На этом участие ОС в передаче заканчивается, она может заняться другой работой. По завершении передачи контроллер ПДП генерирует прерывание, сообщая тем самым ОС, что работа выполнена. Вот как выглядит модифицированная хронология событий:



Отсюда видно, что теперь копированием данных занимается контроллер ПДП. Поскольку CPU в это время свободен, ОС может делать что-то еще, например выбрать Процесс 2, который занимает CPU до того, как Процесс 1 возобновится.

36.6. МЕТОДЫ ВЗАИМОДЕЙСТВИЯ С УСТРОЙСТВАМИ

Теперь, получив представление о вопросах эффективности ввода-вывода, рассмотрим несколько других проблем, которые нужно решить при включении устройств в современные системы. И одна из них заключается в том, что мы до сих пор ничего не сказали о том, как ОС фактически взаимодействует с устройством!

СУЩЕСТВО ПРОБЛЕМЫ: КАК ВЗАИМОДЕЙСТВОВАТЬ С УСТРОЙСТВАМИ

Как оборудование должно взаимодействовать с устройством? Существуют ли какие-то специальные команды? Или это делается иначе?

Со временем развитие получили два основных способа взаимодействия с устройствами. Первый, самый старый (много лет использовался в мейн-фреймах IBM), – явные **команды ввода-вывода**. Они определяют, как ОС должна записывать данные в регистры конкретного устройства, и потому позволяют строить описанные выше протоколы.

Например, в x86 для взаимодействия с устройствами используются команды *in* и *out*. Чтобы отправить данные устройству, программа указывает, в каком регистре находятся данные, а также *порт*, идентифицирующий устройство. Выполнение команды дает желаемое поведение.

Обычно такие команды **привилегированные**. Устройствами управляет ОС, поэтому только ОС и разрешено взаимодействовать с ними напрямую. Подумайте, что было бы, если бы любой программе было позволено читать или записывать на диск. Воцарился бы хаос (как всегда), потому что любая пользовательская программа смогла бы использовать эту брешь, чтобы получить полный контроль над машиной.

Второй способ взаимодействия с устройствами называется **ввод-вывод с отображением памяти**. При таком подходе оборудование представляет регистры устройства так, будто они являются ячейками памяти. Для доступа к конкретному регистру ОС выполняет команду загрузки (чтобы прочитать данные) или сохранения (чтобы записать данные), а оборудование переадресует команды устройству вместо основной памяти.

Ни один подход нельзя назвать наилучшим. Отображение в память хорошо тем, что для его поддержки не нужны специальные команды, но в настоящее время используются оба подхода.

36.7. Сопряжение с ОС: ДРАЙВЕР УСТРОЙСТВА

И последняя проблема, которую мы обсудим: как организовать сопряжение устройств, имеющих сильно различающиеся интерфейсы, с ОС, которую хотелось бы сделать максимально общей. Рассмотрим, к примеру, файловую систему. Мы хотели бы построить ее так, чтобы она могла работать на SCSI-дисках, на IDE-дисках, на флеш-дисках, подключенных к шине USB, и т. д., не заботясь о деталях формирования запросов чтения и записи для каждого типа устройств.

СУЩЕСТВО ПРОБЛЕМЫ: КАК ПОСТРОИТЬ НЕЗАВИСИМУЮ ОТ УСТРОЙСТВ ОС

Как сделать большую часть ОС независимой от устройств и тем самым скрыть детали взаимодействия с устройствами от основных подсистем ОС?

Проблема решается применением старой как мир техники **абстрагирования**. На самом низком уровне существует какая-то часть ОС, которая знает обо всех деталях работы устройства. Эта часть называется **драйвером устройства**, именно в ней инкапсулирована вся специфика взаимодействия с устройством.

Посмотрим, как эта абстракция помогает в проектировании и реализации ОС на примере программного стека файловой системы в Linux. На рис. 36.4 очень приблизительно показана организация Linux. Как видим, файловая система (и, конечно, расположенные выше нее приложения) ничего не знает о том, диски какого класса используются; она просто отправляет запросы чтения и записи общему уровню блочных устройств, а тот уже переправляет их подходящему драйверу устройства, который берет на себя все детали выполнения конкретного запроса. Несмотря на упрощения, эта диаграмма показывает, как можно скрыть такие детали от основной части ОС.

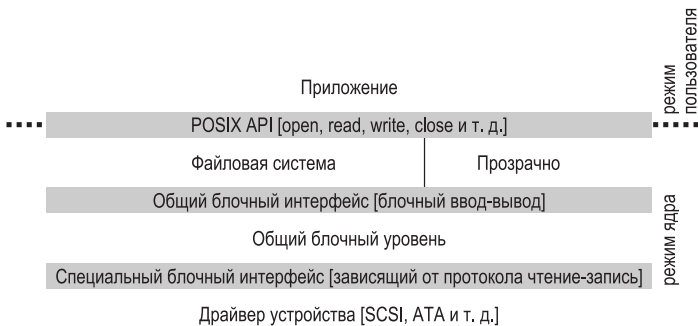


Рис. 36.4 ❖ Стек файловой системы

На диаграмме показан также **прозрачный (raw) интерфейс** с устройствами, который позволяет специальным приложениям (**средству проверки файловой системы**, описанной ниже [AD14], или **средству дефрагментации диска**) напрямую читать и записывать блоки, не прибегая к абстракции файла. В большинстве систем такой интерфейс предоставляется для поддержки подобных низкоуровневых приложений управления системой хранения.

Заметим, что у такой инкапсуляции есть и недостаток. Например, если устройство имеет много специальных возможностей, но должно раскрывать ядру только общий интерфейс, то эти возможности останутся невостребованными. Такая ситуация возникает, например, в Linux с SCSI-устройствами, которые обладают очень широким функционалом для извещения об ошибках, но, поскольку у других блочных устройств (например, ATA и IDE) аналогичный функционал гораздо проще, верхние уровни программного обеспечения получают только обобщенную ошибку ввода-вывода, а все дополнительные детали, которое могло бы сообщить SCSI-устройство, остаются потерянными для файловой системы [G08].

Интересно, что поскольку драйверы устройств нужны для всех устройств, которые потенциально могут быть включены в систему, со временем их код стал занимать большую часть ядра. Исследование ядра Linux показало, что свыше 70 % кода ОС приходится на драйверы устройств [C01]; для систем на базе Windows дело, вероятно, обстоит примерно так же. Поэтому, когда говорят, что ОС насчитывает миллионы строк кода, на самом деле имеются в виду миллионы строк кода драйверов. Конечно, в любой конкретной установке большая часть этого кода не активна (поскольку одновременно к системе подключено лишь несколько устройств). Но гораздо печальнее то, что драйверы часто пишутся «любителями» (а не профессиональными разработчиками ядра), поэтому содержат много ошибок и являются основными причинами краха ядра [S03].

36.8. ПРАКТИЧЕСКИЙ ПРИМЕР: ПРОСТОЙ ДРАЙВЕР IDE-ДИСКА

Копнем немного глубже и рассмотрим реальное устройство: IDE-диск [L94]. Его протокол описан в справочном руководстве [W10], а исходный код простого, но работоспособного драйвера мы взяли из системы xv6 [CK+08].

IDE-диск раскрывает системе простой интерфейс, состоящий из четырех типов регистров: управления, блока команд, состояния и ошибки. Все они доступны путем чтения или записи по определенным «адресам ввода-вывода» (например, 0x3F6 в коде ниже) с использованием команд `in` и `out` (на платформе x86).

Регистр управления:

Адрес 0x3F6 = 0x08 (0000 1RE0): R=сброс, E=0 означает “разрешить прерывание”

Блок регистров команд:

Адрес 0x1F0 = порт данных

Адрес 0x1F1 = ошибка

Адрес 0x1F2 = число секторов

Адрес 0x1F3 = младший байт ЛАБ

Адрес 0x1F4 = средний байт ЛАБ

Адрес 0x1F5 = старший байт ЛАБ

Адрес 0x1F6 = 1B1D TOP4LBA: B=ЛАБ, D=диск

Адрес 0x1F7 = Команда/состояние

Регистр состояния (адрес 0x1F7):

7	6	5	4	3	2	1	0
BUSY	READY	FAULT	SEEK	DRQ	CORR	IDDEX	ERROR

Регистр ошибки (адрес 0x1F1): (проверять, когда бит ERROR регистра состояния == 1)

7	6	5	4	3	2	1	0
BBK	UNC	MC	IDNF	MCR	ABRT	T0NF	AMNF

BBK = сбойный блок

UNC = неисправимая ошибка данных

MC = сменился носитель

IDNF = не найден маркер ID

MCR = запрошена смена носителя

ABRT = команда отменена

T0NF = не найдена дорожка 0

AMNF = не найден маркер адреса

Рис. 36.5 ❖ Интерфейс IDE

Ниже описан базовый протокол взаимодействия с устройством в предположении, что оно уже инициализировано.

- **Ждать готовности диска.** Читать регистр состояния (0x1F7), пока не окажется, что бит READY поднят, а бит BUSY сброшен.
- **Записать параметры в регистры команд.** Записать число секторов, логический адрес блока (ЛАБ, англ. *LBA*) секторов и номер диска (ведущий=0x00 или ведомый=0x10, поскольку IDE допускает только два диска) в регистры команд (0x1F2-0x1F6).
- **Начать ввод-вывод,** для чего записать команду чтения или записи в регистр команды (0x1F7).
- **Передача данных (для операций записи).** Ждать, когда в регистре состояния будут подняты биты READY и DRQ (drive request for data – запрос данных к диску); писать данные в порт данных.
- **Обработка прерываний.** В простейшем случае обработать прерывание для каждого переданного сектора; в более сложных сценариях допускается объединение прерываний, т. е. генерируется одно прерывание по завершении всей передачи.
- **Обработка ошибок.** После каждой операции читать регистр состояния. Если бит ERROR поднят, читать подробную информацию из регистра ошибки.

Большая часть этого протокола реализована в драйвере IDE-дисков в xv6 (рис. 36.6). Не считая инициализации, он состоит из четырех функций. Функция `ide_rw()` ставит запрос в очередь (если имеются другие еще не обработанные запросы) или отправляет его напрямую диску (с помощью функции `start_request()`); в любом случае функция ждет завершения запроса, а вызвавший процесс усыпляет. Функция `ide_start_request()` служит для отправки

```
static int ide_wait_ready() {
    while (((int r = inb(0x1f7)) & IDE_BSY) || !(r & IDE_DRDY))
        ; // цикл, пока диск не готов
}

static void ide_start_request(struct buf *b) {
    ide_wait_ready();
    outb(0x3f6, 0); // генерировать прерывания
    outb(0x1f2, 1); // сколько секторов?
    outb(0x1f3, b->sector & 0xff); // сюда пишем ЛАБ ...
    outb(0x1f4, (b->sector >> 8) & 0xff); // ... и сюда
    outb(0x1f5, (b->sector >> 16) & 0xff); // ... и сюда!
    outb(0x1f6, 0xe0 | ((b->dev&1)<<4) | ((b->sector>>24)&0x0f));
    if(b->flags & B_DIRTY){
        outb(0x1f7, IDE_CMD_WRITE); // это ЗАПИСЬ
        outsl(0x1f0, b->data, 512/4); // данные тоже передать!
    } else {
        outb(0x1f7, IDE_CMD_READ); // это ЧТЕНИЕ (данных нет)
    }
}

void ide_rw(struct buf *b) {
    acquire(&ide_lock);
    for (struct buf **pp = &ide_queue; *pp; pp=&(*pp)->qnext)
        ; // обойти очередь
    *pp = b; // добавить запрос в конец
    if (ide_queue == b) // если q пуста
        ide_start_request(b); // отправить запрос диску
    while ((b->flags & (B_VALID|B_DIRTY)) != B_VALID)
        sleep(b, &ide_lock); // ждать завершения
    release(&ide_lock);
}

void ide_intr() {
    struct buf *b;
    acquire(&ide_lock);
    if (!(b->flags & B_DIRTY) && ide_wait_ready() >= 0)
        insl(0x1f0, b->data, 512/4); // если ЧТЕНИЕ: получить данные
    b->flags |= B_VALID;
    b->flags &= ~B_DIRTY;
    wakeup(b); // разбудить ожидающий процесс
    if ((ide_queue = b->qnext) != 0) // начать новый запрос
        ide_start_request(ide_queue); // (если таковой существует)
    release(&ide_lock);
}
```

Рис. 36.6 ❖ Драйвер IDE-диска в xv6 (упрощенный)

запроса (и, в случае операции записи, прилагаемых к нему данных) диску; для чтения и записи регистров используются соответственно команды `x86 in` и `out`. Функция `ide_start_request` пользуется функцией `ide_wait_ready()`, чтобы дождаться готовности диска перед отправкой ему запроса. Наконец, функция `ide_intr()` вызывается в ответ на прерывание; она читает данные от устройства (если имел место запрос чтения, а не записи), пробуждает процесс, ожидающий завершения ввода-вывода, и (если в очереди есть еще запросы) запускает следующий запрос, обращаясь к `ide_start_request()`.

36.9. ИСТОРИЧЕСКИЕ ЗАМЕЧАНИЯ

Прежде чем закончить эту главу, мы хотим сделать несколько исторических замечаний о происхождении некоторых основополагающих идей. Если хотите узнать больше, обратитесь к великолепному обзору Смотермана [S08].

Прерывания – очень старая идея, восходящая к самым первым компьютерам. Например, в машине UNIVAC, разработанной в начале 1950-х годов, уже присутствовала форма вектора прерываний, хотя неизвестно, в каком точно году она появилась [S08]. Грустно, что, еще не выйдя из младенческого возраста, мы начинаем забывать исторические вехи компьютерной индустрии.

Спорят по поводу того, в какой машине впервые воплотилась идея ПДП. Например, Кнут и другие указывают на DYSEAC («мобильный» компьютер – в те времена это означало, что его можно перевозить в грузовом прицепе), а кто-то полагает, что впервые этот механизм появился в IBM SAGE [S08]. Как бы то ни было, к середине 1950-х годов уже существовали системы, в которых устройства ввода-вывода могли обмениваться данными непосредственно с памятью и прерывать CPU по завершении операции.

Проследить историю трудно, потому что изобретения привязаны к реальным, но иногда малоизвестным машинам. Например, некоторые считают, что впервые векторные прерывания появились в машине Lincoln Labs TX-2 [S08], но вряд ли это так.

Поскольку идеи витают в воздухе – не нужно эйнштейновского озарения, чтобы прийти к мысли о том, что процессор может делать что-то еще во время медленной операции ввода-вывода, – наша сосредоточенность на вопросе «кто первый?», быть может, не оправдана. Но ясно одно: уже когда создавались первые машины, была осознана необходимость в поддержке ввода-вывода. Прерывания, ПДП и смежные идеи – всё это прямые следствия того, что процессор работает быстро, а устройства – медленно; если бы вы жили в то время, то, наверное, пришли бы к таким же идеям.

36.10. РЕЗЮМЕ

Сейчас вы в очень общих чертах понимаете, как ОС взаимодействует с устройством. Мы познакомились с двумя методами, прерываниями и ПДП, которые позволяют добиться эффективной работы устройств, а также с двумя под-

ходами к работе с регистрами устройств – явные команды ввода-вывода и отображение регистров на память. Наконец, мы рассказали о драйверах устройств – той части ОС, в которой инкапсулированы низкоуровневые детали и благодаря которой все остальные части можно сделать независимыми от устройств.

Литература

[A+11] «vIC: Interrupt Coalescing for Virtual Machine Storage Device IO» by Irfan Ahmad, Ajay Gulati, Ali Mashtizadeh. USENIX '11. *Прекрасный обзор объединения прерываний в традиционной и виртуализированной среде.*

[AD14] «Operating Systems: Three Easy Pieces» (Chapters: Crash Consistency: FSCCK and Journaling and Log-Structured File Systems) by Remzi Arpaci-Dusseau and Andrea Arpaci-Dusseau. Arpaci-Dusseau Books, 2014. *Описание принципов работы программы проверки файловой системы, которой необходим низкоуровневый доступ к дискам, обычно не предоставляемый самой файловой системой.*

[C01] «An Empirical Study of Operating System Errors» by Andy Chou, Junfeng Yang, Benjamin Chelf, Seth Hallem, Dawson Engler. SOSP '01. *Одна из первых работ, в которой систематически исследуется вопрос о количестве ошибок в операционных системах. Среди прочих интересных находок, авторы показывают, что в драйверах устройств ошибок примерно в семь раз больше, чем в основном коде ядра.*

[CK+08] «The xv6 Operating System» by Russ Cox, Frans Kaashoek, Robert Morris, Nikolai Zeldovich. Доступно по адресу <http://pdos.csail.mit.edu/6.828/2008/index.html>. В файле `ide.c` находится драйвер IDE-устройства, содержащий чуть больше деталей.

[D07] «What Every Programmer Should Know About Memory» by Ulrich Drepper. November, 2007. Доступно по адресу <http://www.akkadia.org/drepper/cpumemory.pdf>. *Фантастическая статья о современных системах памяти, начиная с DRAM и далее до виртуализации и алгоритмов оптимизации кеша.*

[G08] «EIO: Error-handling is Occasionally Correct» by Haryadi Gunawi, Cindy Rubio-Gonzalez, Andrea Arpaci-Dusseau, Remzi Arpaci-Dusseau, Ben Liblit. FAST '08, San Jose, CA, February 2008. *Наша собственная работа о создании инструмента, который ищет в коде файловых систем Linux места, где неправильно обрабатываются ошибки. Мы нашли сотни ошибок, многие из которых теперь исправлены.*

[H17] «Intel Core i7-7700K review: Kaby Lake Debuts for Desktop» by Joel Hruska. January 3, 2017. www.extremetech.com/extreme/241950-intels-core-i7-7700k-reviewed-kaby-lake-debuts-desktop. *Подробный обзор недавних комплектов микросхем Intel, включая CPU и подсистему ввода-вывода.*

[H18] «Hacker News» by Many contributors. Available: <https://news.ycombinator.com>. *Один из лучших агрегаторов технических материалов. Как-то раз*

в 2014 году эта книга заняла первую позицию в рейтинге, что привело к миллиону скачиваний глав всего за один день! Печально, но больше такого не повторилось.

[L94] «AT Attachment Interface for Disk Drives» by Lawrence J. Lamers. Reference number: ANSI X3.221, 1994. Доступно по адресу <ftp://ftp.t10.org/t13/project/d0791r4c-ATA-1.pdf>. Довольно сухо написанный документ об интерфейсах устройств. Читайте на свой страх и риск.

[MR96] «Eliminating Receive Livelock in an Interrupt-driven Kernel» by Jeffrey Mogul, K. K. Ramakrishnan. USENIX '96, San Diego, CA, January 1996. Могул с сотрудниками выполнили много первопроходческих работ по сетевой производительности веб-серверов. Эта статья – один из примеров.

[S08] «Interrupts» by Mark Smotherman. July '08. Доступно по адресу <http://people.cs.clemson.edu/~mark/interrupts.html>. Сокровищница сведений по истории прерываний, ПДП и смежных идей на заре развития вычислительной техники.

[S03] «Improving the Reliability of Commodity Operating Systems» by Michael M. Swift, Brian N. Bershad, Henry M. Levy. SOSP '03. Работа Свифта возродила интерес к конструированию операционных систем с микроядром; как минимум, в ней наконец-то приведены убедительные причины, почему защита, основанная на адресном пространстве, может быть полезна в современной ОС.

[W10] «Hard Disk Driver» by Washington State Course Homepage. Доступно по адресу <http://eecs.wsu.edu/~cs460/cs560/HDdriver.html>. Краткое описание интерфейса IDE-диска и построения драйвера для него.

Глава 37

Жесткие диски

В предыдущей главе мы познакомились с общей концепцией устройства ввода-вывода и показали, как с ним может взаимодействовать ОС. В этой главе мы будем подробнее говорить об одном конкретном устройстве: **жестком диске**¹. Такие диски в течение многих десятилетий были основной формой хранения данных в компьютерных системах, и развитие технологии файловых систем (о них чуть позже) в значительной системе обусловлено их поведением. Поэтому, прежде чем переходить к изучению файловых систем, полезно разобраться в деталях работы дисков, для управления которыми файловые системы и предназначены. Многие из этих деталей описаны в прекрасных статьях Ruemmler and Wilkes [RW92] и Anderson, Dykes, and Riedel [ADR03].

СУЩЕСТВО ПРОБЛЕМЫ: КАК ХРАНИТЬ И ОБРАЩАТЬСЯ К ДАННЫМ НА ДИСКЕ

Как хранятся данные на современных жестких дисках? Какой интерфейс эти диски предоставляют? Как данные размещаются, и как к ним производится доступ? Как планирование диска повышает производительность?

37.1. ИНТЕРФЕЙС

Для начала разберемся в интерфейсе диска. Базовый интерфейс всех современных дисков прост и прямолинеен. Диск состоит из большого числа секторов (512-байтовых блоков), каждый из которых можно читать и записывать. Сектора диска, содержащего n секторов, нумеруются от 0 до $n - 1$. Таким образом, диск можно рассматривать как массив секторов, а множество чисел от 0 до $n - 1$ составляет **адресное пространство** диска.

Возможны операции, затрагивающие сразу несколько секторов; на самом деле многие файловые системы читают и записывают блоками по 4 КБ (или больше). Но производители дисков гарантируют лишь, что **атомарной** явля-

¹ Раньше было принято различать *накопитель на жестком магнитном диске* (hard disk drive) и сам *жесткий диск*. Но постепенно это терминологическое различие исчезло, и сейчас под жестким диском понимают не только сам набор пластин, но и всю технологическую обвязку, включая контроллер. – Прим. перев.

ется только запись длиной 512 байт (она либо выполняется целиком, либо не выполняется вовсе), поэтому в случае пропадания энергоснабжения может оказаться, что записана лишь часть порции данных (иногда это называют **разорванной записью**).

Большинство клиентов делают определенные предположения о дисках, хотя в самом интерфейсе они не специфицированы; Шлоссер и Гангер называют это «неписанным контрактом» дисков [SG04]. Именно, обычно предполагается, что доступ к двум блокам¹, расположенным рядом в адресном пространстве диска, быстрее, чем к блокам, находящимся далеко друг от друга. Обычно можно также предполагать, что доступ к непрерывной цепочке блоков (т. е. последовательное чтение или запись) – самый быстрый способ доступа, гораздо быстрее, чем доступ в произвольном порядке.

37.2. БАЗОВАЯ ГЕОМЕТРИЯ

Опишем некоторые компоненты современного диска. Начнем с **пластины**, круглой поверхности, на которую записываются данные. Диск может состоять из одной или нескольких пластин, каждая пластина имеет две стороны, называемые **поверхностями**. Пластины обычно изготавливаются из твердого материала (например, алюминия), а затем покрываются тонким магнитным слоем, на котором данные сохраняются даже после отключения питания.

Все пластины насажены на **шпиндель**, подключенный к электродвигателю, вращающему пластины (если подается питание) с постоянной скоростью. Скорость вращения обычно измеряют в **оборотах в минуту (об/мин, англ. RPM)**, для современных дисков она чаще всего составляет от 7200 до 15 000 об/мин. Заметим, что нас нередко интересует время одного полного оборота, например если диск вращается со скоростью 10 000 об/мин, значит, время одного оборота составляет примерно 6 миллисекунд (6 мс).

Данные на поверхности располагаются в концентрических окружностях, одна такая окружность называется **дорожкой**. На одной поверхности могут размещаться тысячи и тысячи дорожек, упакованных так плотно, что сотни дорожек занимают место шириной в человеческий волос.

Для чтения и записи дорожки необходим механизм, который позволяет распознавать (читать) силовые линии магнитного поля или вызывать их изменение (записывать). Этот процесс чтения и записи осуществляется **головкой диска**; на каждую поверхность приходится одна такая головка. Головка диска крепится на **кронштейне**, который перемещается поперек поверхности для позиционирования головки над нужной дорожкой.

¹ Мы, как и многие другие, употребляем термины **блок** и **сектор** как синонимы, предполагая, что читатель понимает, о чем речь, из контекста. Просим нас извинить!

37.3. Простой диск

Рассмотрим принцип работы диска на примере простого диска с одной дорожкой (рис. 37.1).

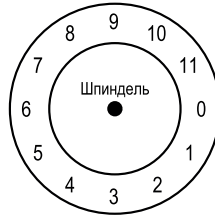


Рис. 37.1 ❖ Диск с одной дорожкой

Дорожка состоит из 12 секторов размером по 512 байт (это, напомним, типичный размер сектора), которые, следовательно, имеют адреса от 0 до 11. Единственная пластина вращается вокруг шпинделя, который соединен с двигателем. Конечно, сама дорожка не очень интересна, мы хотим читать и записывать сектора, поэтому нужна головка, крепящаяся на кронштейне (рис. 37.2).

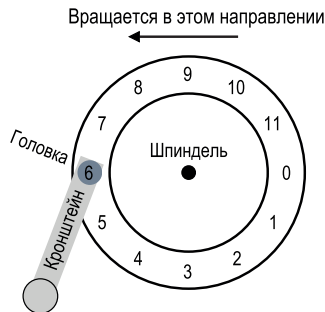


Рис. 37.2 ❖ Дорожка и головка

На рисунке головка диска, закрепленная на конце кронштейна, позиционирована над сектором 6, а поверхность вращается против часовой стрелки.

Одна дорожка: задержка вращения

Чтобы понять, как будет обрабатываться запрос к нашему простому диску с одной дорожкой, представим, что получен запрос на чтение блока 0. Что должен сделать диск для обслуживания этого запроса?

Наш диск простой, так что работы у него немного. Он должен подождать, пока нужный сектор не окажется под головкой. Это ожидание в современных дисках случается достаточно часто и является настолько важной частью вре-

мени обслуживания, что получило специальное название: **задержка вращения**. В нашем примере если полная задержка вращения равна R , то в среднем диску придется подождать время $R/2$, пока сектор 0 не окажется под головкой чтения-записи (если первоначально под ней находился сектор 6). В худшем случае, когда головка находится над сектором 0, для обслуживания запроса придется ждать почти столько времени, сколько занимает полный оборот, т. е. почти R .

Несколько дорожек: время поиска

Диск с одной дорожкой на практике не встречается, современные диски содержат миллионы дорожек. Поэтому рассмотрим чуть более реалистичную поверхность с тремя дорожками (рис. 37.3 слева).

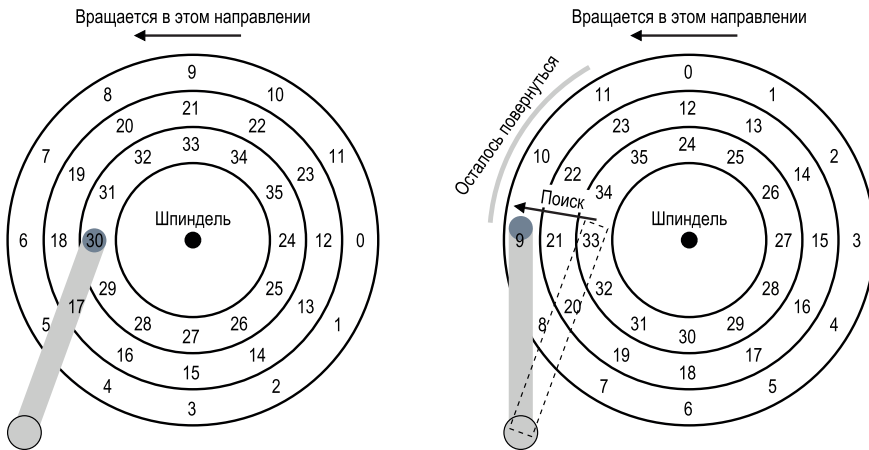


Рис. 37.3 ❖ Три дорожки и головка (справа: с поиском)

В настоящий момент головка позиционирована над самой внутренней дорожкой (содержащей сектора с 24 по 35). Следующая дорожка содержит сектора с 12 по 23, а самая внешняя – первые сектора (с 0 по 11).

Чтобы понять, как диск может обратиться к заданному сектору, посмотрим, что происходит, когда запрос адресован какому-нибудь далекому сектору, например требуется прочитать сектор 11. Чтобы обслужить этот запрос, диск должен сначала переместить кронштейн к нужной дорожке (в данном случае – самой внешней), этот процесс называется **поиском**. Поиск, наряду с вращением, – одна из самых дорогостоящих операций.

Отметим, что поиск состоит из нескольких этапов: сначала *ускорение*, когда кронштейн начинает движение, затем *движение по инерции*, потом *торможение*, когда кронштейн замедляется, и, наконец, *установление*, когда головка точно позиционируется над нужной дорожкой. **Время установления** весьма значительно, например от 0,5 до 2 мс, поскольку головка должна встать точно над дорожкой (а не просто рядом!).

По завершении поиска головка позиционирована строго над нужной дорожкой. Процесс поиска схематично изображен на рис. 37.3 (справа).

Итак, в процессе поиска кронштейн переведен в нужное положение, а пластина все это время, естественно, вращалась – в данном случае она повернулась примерно на три сектора. Таким образом, скоро под головкой пройдет сектор 9, и мы должны лишь немного подождать, прежде чем начать считывание.

Когда сектор 11 проходит под головкой, наступает последний этап ввода-вывода – **передача**, когда данные читаются с поверхности или записываются на нее. Полное время ввода-вывода складывается из трех частей: поиск, задержка вращения и передача.

Дополнительные детали

Хотя мы не можем уделить этому много времени, все же отметим ряд других интересных аспектов функционирования диска. Во многих дисках практикуется так называемый **сдвиг дорожек** (track skew), чтобы было проще обслужить последовательное чтение с пересечением границ дорожек. В нашем простом примере это может выглядеть, как показано на рис. 37.4.

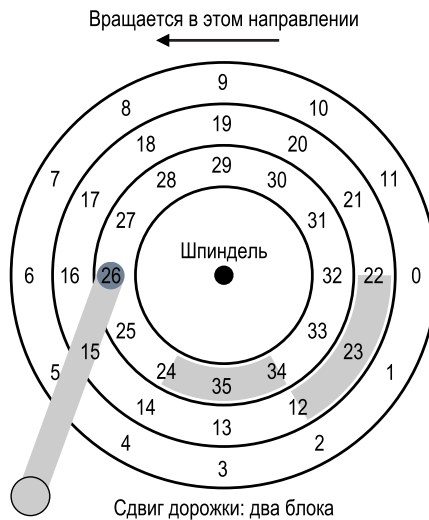


Рис. 37.4 ❖ Три дорожки: сдвиг дорожек на 2

Сектора сдвигаются подобным образом, потому что при переходе с одной дорожки на другую диску нужно время для перепозиционирования головки (даже если дорожки соседние). Не будь сдвига, головка, конечно, переместилась бы на следующую дорожку, но пластина уже провернулась бы, так что для доступа к искомому следующему блоку пришлось бы ждать в течение почти полной задержки вращения.

Еще одна проблема заключается в том, что на внешних дорожках секторов больше, чем на внутренних; это элементарная геометрия – на внешних до-

рожках просто больше места. Поэтому диски часто бывают **многозонными**, т. е. разбиты на несколько зон, где зоной называется множество соседних дорожек на одной поверхности. В пределах каждой зоны количество секторов на дорожке одинаково, но во внешних зонах секторов больше, чем во внутренних.

Наконец, важной частью любого современного диска является **кеш**, который, по историческим причинам, называется **буфером дорожки**. Обычно кеш представляет собой память небольшого объема (8 или 16 МБ), в которой контроллер хранит данные, недавно записанные на диск или прочитанные с него. Например, при чтении одного сектора с диска контроллер может прочитать сразу все сектора с данной дорожки и кешировать их в памяти; тогда последующие запросы к этим секторам можно будет удовлетворить очень быстро.

При записи у диска есть выбор: подтвердить завершение операции, когда данные помещены в память, или после того, как они реально попали на диск. Первый подход называется **кешированием с отложенной записью** (или **немедленной отчетностью**), второй – **кешированием со сквозной записью**. Благодаря отложенной записи диск кажется «быстрее», но это опасно; если файловая система или приложение требует, чтобы данные записывались на диск в определенном порядке, то отложенная запись может стать источником проблем (детали см. в главе о журналировании в файловых системах).

ОТСТУПЛЕНИЕ: АНАЛИЗ РАЗМЕРНОСТЕЙ

Помните, как на уроках по химии вы решали задачи, подгоняя единицы измерения, и в результате таинственным образом получался правильный ответ? У этой химической магии есть высокопарное название – **анализ размерностей**, иногда она оказывается полезной и при анализе вычислительных систем.

Покажем на примере, как работает анализ размерностей и почему он полезен. В данном случае нам нужно вычислить, сколько времени в миллисекундах занимает один оборот диска. К сожалению, в технической документации приведена только скорость вращения в **оборотах в минуту**. Пусть имеется диск со скоростью вращения 10 000 об/мин. Как с помощью анализа размерностей получить время одного оборота в миллисекундах?

Для этого напишем слева желаемые единицы измерения; в данном случае мы хотим получить время (в миллисекундах) одного оборота, поэтому так и пишем: $\frac{\text{Время (мс)}}{1 \text{ об.}}$. Затем выпишем все, что мы знаем, сокращая единицы измерения там, где это возможно. Сначала пишем $\frac{1 \text{ мин}}{10\,000 \text{ об.}}$ (число оборотов в знаменателе, потому что так написано слева), затем преобразуем минуты в секунды: $\frac{60 \text{ с}}{1 \text{ мин}}$, а потом секунды в миллисекунды: $\frac{1000 \text{ мс}}{1 \text{ с}}$. И вот окончательный результат (после сокращения единиц измерения):

$$\frac{\text{Время (мс)}}{1 \text{ об.}} = \frac{1 \text{ мин}}{10\,000 \text{ об.}} \cdot \frac{60 \text{ с}}{1 \text{ мин}} \cdot \frac{1000 \text{ мс}}{1 \text{ с}} = \frac{60\,000 \text{ мс}}{10\,000 \text{ об.}} = \frac{6 \text{ мс}}{\text{об.}}$$

Как видим, анализ размерностей превращает интуитивно очевидное в простую повторяемую процедуру. Он полезен не только при вычислении скорости вращения, но и во

многих других случаях. Например, для диска часто задается скорость передачи, скажем, 100 МБ/с, а требуется узнать, сколько времени займет передача блока размером 512 КБ (в миллисекундах). Анализ размерностей позволяет легко ответить на этот вопрос:

$$\frac{\text{Время (мс)}}{1 \text{ запрос}} = \frac{512 \text{ КБ}}{1 \text{ запрос}} \cdot \frac{1 \text{ МБ}}{1024 \text{ КБ}} \cdot \frac{1 \text{ с}}{100 \text{ МБ}} \cdot \frac{1000 \text{ мс}}{1 \text{ с}} = \frac{5 \text{ мс}}{\text{запрос}}.$$

37.4. ВРЕМЯ ВВОДА-ВЫВОДА: НЕМНОГО АРИФМЕТИКИ

Теперь, имея абстрактную модель диска, проведем простой анализ, чтобы лучше понять, из чего складывается его производительность. Время ввода-вывода можно представить в виде суммы трех компонентов:

$$T_{I/O} = T_{seek} + T_{rotation} + T_{transfer} \quad (37.1)$$

Заметим, что скорость ввода-вывода I/O ($R_{I/O}$), которую проще всего использовать для сравнения дисков (как будет показано ниже), легко вычисляется. Нужно просто разделить размер передачи на время передачи:

$$R_{I/O} = \text{Size}_{\text{Transfer}} / T_{I/O}. \quad (37.2)$$

Чтобы лучше прочувствовать время ввода-вывода, произведем вычисление. Предположим, что нас интересуют две рабочие нагрузки. Первая – **произвольный доступ** – предполагает небольшие операции чтения (скажем, 4 КБ) в случайных местах на диске. Произвольный доступ встречается во многих важных приложениях, включая системы управления базами данных. Вторая – **последовательный доступ** – предполагает, что читается много соседних секторов без случайных переходов из одного места в другое. Последовательный доступ также часто встречается и не менее важен.

Чтобы понять различие в производительности между произвольным и последовательным доступом, сделаем несколько предположений о диске. На рис. 37.5 приведены спецификации двух современных дисков компании Seagate. Первый, Cheetah 15K.5 [S09b], – высокопроизводительный SCSI-диск. Второй, Barracuda [S09a], – диск большой емкости.

	Cheetah 15K.5	Barracuda
Емкость	300 ГБ	1 ТБ
Об/мин	15 000	7200
Среднее время поиска	4 мс	9 мс
Макс. скорость передачи	125 МБ/с	105 МБ/с
Число пластин	4	4
Кеш	16 МБ	16/32 МБ
Интерфейс	SCSI	SATA

Рис. 37.5 ❖ Спецификации дисков: сравнение SCSI и SATA

Как видно, характеристики дисков сильно различаются, и это во многих отношениях показательно для рынка дисковых накопителей. Во-первых, имеется рынок «высокопроизводительных» дисков, для которых важна максимальная скорость вращения, низкое время поиска и быстрая передача данных. Во-вторых, есть рынок «емких» дисков, для которого самое главное – стоимость в расчете на один байт; такие диски медленнее, но позволяют разместить максимальное число байтов в заданном пространстве.

Зная эти характеристики, мы можем вычислить, как оба диска будут работать в двух описанных выше режимах. Сначала рассмотрим произвольный доступ. В предположении, что каждое чтение 4 КБ происходит в случайных местах диска, можно вычислить, сколько времени займет операция. Для Cheetah:

$$T_{seek} = 4 \text{ мс}, T_{rotation} = 2 \text{ мс}, T_{transfer} = 30 \text{ мкс.} \quad (37.3)$$

СОВЕТ: РАБОТАЙТЕ С ДИСКАМИ ПОСЛЕДОВАТЕЛЬНО

Если возможно, передавайте данные на диск и с диска последовательно. Если это никак невозможно, то хотя бы постарайтесь передавать данные большими блоками: чем больше, тем лучше. Если ввод-вывод производится мелкими порциями в произвольных местах диска, то производительность сильно падает. Пострадают пользователи. Да и вы тоже, сознавая, какие страдания причинили своим беззаботным произвольным доступом.

Среднее время поиска (4 мс) – это всего лишь среднее время, сообщаемое производителем; полное время поиска (от одного края поверхности до другого), скорее всего, в 2–3 раза больше. Средняя задержка вращения вычисляется по скорости вращения в об/мин. 15 000 об/мин равно 250 об/с (оборотов в секунду), поэтому каждый оборот занимает 4 мс. В среднем для подвода к нужному сектору диск должен совершить пол-оборота, т. е. среднее время составляет 2 мс. Наконец, время передачи равно размеру передачи, умноженному на пиковую скорость передачи, в данном случае оно пренебрежимо мало (30 *микросекунд*, напомним, что в одной миллисекунде 1000 *микросекунд*).

Таким образом, согласно приведенному выше соотношению, $T_{I/O}$ для Cheetah приближенно равно 6 мс. Чтобы найти скорость ввода-вывода, нужно разделить размер передачи на среднее время, поэтому для Cheetah $R_{I/O}$ при произвольном доступе составляет приблизительно 0.66 МБ/с. Для Barracuda то же вычисление дает $T_{I/O} \approx 13.2$ мс, т. е. в два с лишним раза медленнее, поэтому скорость составляет приблизительно 0.31 МБ/с.

Теперь рассмотрим последовательный доступ. Здесь можно предполагать, что очень длинной передаче предшествует один поиск и одно вращение. Для простоты предположим, что размер передачи равен 100 МБ. Тогда $T_{I/O}$ для Cheetah и Barracuda равно приблизительно 800 мс и 950 мс соответственно. Поэтому скорости ввода-вывода очень близки к пиковым значениям 125 МБ/с и 105 МБ/с.

Этот рисунок позволяет сделать ряд важных выводов. Во-первых, и это самое главное, существует огромный разрыв в производительности между

произвольным и последовательным доступом, почти в 200 раз для Cheetah и свыше 300 раз для Barracuda. И это подводит нас к самому очевидному совету в истории вычислительной техники.

Второй, более тонкий момент: имеет место существенное различие в производительности между дорогими «высокопроизводительными» дисками и дешевыми дисками «большой емкости». По этой и другим причинам люди часто готовы платить много за первые, но вторые стараются купить как можно дешевле.

		Cheetah	Barracuda
$R_{\text{вч}}$	Произвольный	0.66 МБ/с	0.31 МБ/с
$R_{\text{пч}}$	Последовательный	125 МБ/с	105 МБ/с

Рис. 37.6 ❖ Производительность дисков: сравнение SCSI и SATA

ОТСТУПЛЕНИЕ: ВЫЧИСЛЕНИЕ «СРЕДНЕГО» ВРЕМЕНИ ПОИСКА

Во многих книгах и статьях утверждается, что среднее время поиска по диску приближенно равно одной трети полного времени поиска. Откуда это следует?

Из простого вычисления среднего *расстояния* (а не времени) поиска. Представим диск в виде множества дорожек, пронумерованных от 0 до N . Расстояние поиска между двумя дорожками вычисляется как абсолютная величина разности их номеров: $|x - y|$.

Чтобы вычислить среднее расстояние поиска, нужно сначала сложить все возможные расстояния поиска:

$$\sum_{x=0}^N \sum_{y=0}^N |x - y|. \quad (37.4)$$

Затем разделим эту сумму на количество возможных расстояний поиска: N^2 . Для вычисления суммы представим ее в интегральной форме:

$$\int_{x=0}^N \int_{y=0}^N |x - y| dy dx. \quad (37.5)$$

Для вычисления внутреннего интеграла раскроем абсолютную величину:

$$\int_{y=0}^x (x - y) dy + \int_{y=x}^N (y - x) dy. \quad (37.6)$$

Вычисление дает $\left(xy - \frac{1}{2} y^2 \right) \Big|_0^x + \left(\frac{1}{2} y^2 - xy \right) \Big|_x^N$, и после упрощений получаем $\left(x^2 - Nx + \frac{1}{2} N^2 \right)$. Теперь вычислим внешний интеграл:

$$\int_{x=0}^N \left(x^2 - Nx + \frac{1}{2} N^2 \right) dx, \quad (37.7)$$

откуда

$$\left(\frac{1}{3}x^3 - \frac{N}{2}x^2 + \frac{N^2}{2}x \right) \Big|_0^N = \frac{N^3}{3}. \quad (37.8)$$

Для вычисления среднего расстояния поиска это нужно разделить на общее число состояний N^2 : $\left(\frac{N^3}{3} \right) / (N^2) = \frac{1}{3}N$. Таким образом, среднее расстояние поиска на диске равно одной трети полного расстояния. Так что теперь, услышав, что среднее время поиска равно одной трети полного, вы будете знать, откуда ноги растут.

37.5. ПЛАНИРОВАНИЕ ДИСКА

Из-за высокой стоимости ввода-вывода ОС исторически играла важную роль в установлении порядка операций дискового ввода-вывода. Именно, имея набор запросов ввода-вывода, **планировщик диска** решает, какие запросы выполнять следующими [SCO90, JW91].

В отличие от планирования заданий, при котором продолжительность каждого задания обычно неизвестна, в случае планирования диска можно высказать обоснованную гипотезу о том, сколько времени займет «задание» (т. е. запрос к диску). Оценив время поиска и возможную задержку вращения, планировщик диска будет знать время каждого запроса и сможет (жадно) назначить первым запрос с наименьшим временем, т. е. воспользоваться **принципом SJF** (shortest job first – **кратчайшее задание первым**).

SSTF: с наименьшим временем поиска первым

Один из ранних подходов к планированию диска назывался **SSTF** (shortest-
seek-time-first – **с наименьшим временем поиска первым**), или **SSF** (short-
est-
seek-first). Алгоритм SSTF упорядочивает очередь запросов ввода-вывода по номеру дорожки и первым выполняет запросы к секторам на ближайшей дорожке. Например, если головка сейчас находится над внутренней дорожкой и имеются запросы к секторам 21 (на средней дорожке) и 2 (на внешней дорожке), то первым будет запланирован запрос к сектору 21, а когда он закончится – запрос к сектору 2 (рис. 37.7).

В этом примере SSTF работает хорошо – сначала подводит головку к средней дорожке, а потом к внешней. Но в общем случае SSTF не панацея, и вот почему. Во-первых, ОС не знает о геометрии диска, а просто рассматривает его как массив блоков. По счастью, эту проблему решить довольно легко – вместо SSTF ОС может реализовать алгоритм **NBF** (nearest-block-first – **ближайший блок первым**), который планирует следующим запрос к блоку с ближайшим адресом.

Вторая проблема серьезнее: **зависание**. Представьте, что в примере выше имеется непрекращающийся поток запросов к внутренней дорожке, над которой позиционирована головка. Тогда, если не предпринять никаких мер, SSTF будет просто игнорировать все остальные дорожки.

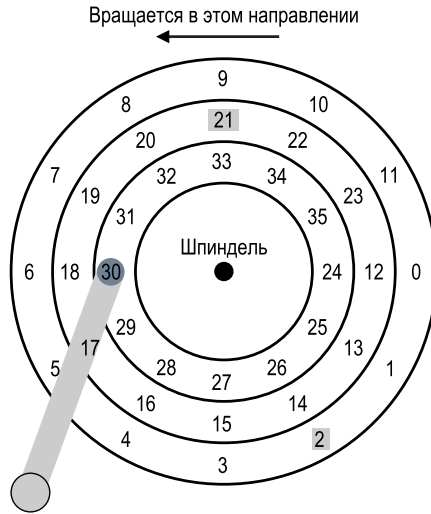


Рис. 37.7 ❖ SSTF: планирование запросов к секторам 21 и 2

СУЩЕСТВО ПРОБЛЕМЫ: КАК РЕШИТЬ ПРОБЛЕМУ ЗАВИСАНИЯ ДИСКА

Как реализовать планирование по принципу SSTF, избежав при этом зависания?

Лифт (он же SCAN или C-SCAN)

На этот вопрос сравнительно простой ответ был дан довольно давно (см., например, [CKR72]). Алгоритм, первоначально названный **SCAN**, перемещает головку поперек диска, обслуживая по ходу дела запросы. Назовем один проход поперек диска (от внешних дорожек к внутренним) *прометанием* (sweep). Если поступает запрос к сектору на дорожке, которая уже была обслужена в процессе этого прометания, то он не обслуживается сразу, а ставится в очередь и будет обслужен при следующем прометании (в обратном направлении).

У алгоритма SCAN есть несколько вариантов, и все они делают примерно одно и то же. Например, в работе Коффмана и др. описан алгоритм **F-SCAN**, который замораживает очередь подлежащих обслуживанию запросов на время прометания [CKR72]; это означает, что все запросы, поступающие во время прометания, помещаются в очередь и обслуживаются позже. Это позволяет избежать игнорирования запросов к далеким секторам, отложив обслуживание поздно поступивших (но более близких) запросов.

Популярен также вариант **C-SCAN**, или **Circular SCAN** (циклический SCAN). Вместо прометания в обоих направлениях этот алгоритм прометает диск только от внешних дорожек к внутренним, а затем возвращается к внешней дорожке и начинает все сначала. Это чуть более справедливо по отношению к внутренним и внешним дорожкам, поскольку чистый SCAN с прометанием

в обоих направлениях отдает предпочтение средним дорожкам, т. к. после обслуживания внешней дорожки SCAN проходит через средние дважды, прежде чем снова вернется к внешней.

По причинам, которые теперь должны быть понятны, алгоритм SCAN (и его варианты) иногда называют алгоритмом **лифта**, потому что он ведет себя как лифт, который движется вверх или вниз, а не просто обслуживает вызовы с этажей, к которым оказался ближе. Представьте, как бы вы разозлились, если бы хотели спуститься с десятого этажа на первый, а кто-то вошедший на третьем этаже нажал 4, и лифт поехал бы на четвертый этаж, потому что он ближе, чем первый! Как видите, алгоритм лифта в реальной жизни предотвращает драки в кабине лифта. А в случае дисков он предотвращает зависания.

К сожалению, SCAN и его варианты не являются примером оптимального планирования. В частности, SCAN (и даже SSTF) не отвечают принципу SJF в полной мере. Например, они игнорируют вращение.

СУЩЕСТВО ПРОБЛЕМЫ: КАК УЧЕСТЬ СТОИМОСТЬ ВРАЩЕНИЯ ДИСКА

Как реализовать алгоритм, который лучше аппроксимирует SJF, принимая в расчет как время поиска, так и вращение диска?

SPTF: с наименьшим временем позиционирования первым

Прежде чем обсуждать алгоритм планирования **SPTF** (shortest positioning time first – **с наименьшим временем позиционирования первым**), который иногда называют также **SATF** (shortest access time first – **с наименьшим временем доступа первым**), дающий решение нашей проблемы, разберемся в самой проблеме более детально. На рис. 37.8 приведен пример.

В этом примере головка позиционирована над сектором 30 на внутренней дорожке. Планировщик должен решить, какой сектор планировать для следующего запроса: 16 (на средней дорожке) или 8 (на внешней дорожке)?

Ответ, разумеется, – «смотря по обстоятельствам». В технике такая ситуация возникает сплошь и рядом и отражает компромиссы, которые являются частью профессии инженера; такой ответ хорош также в трудную минуту, например если вы не знаете, как ответить на вопрос начальника, можете прибегнуть к этой фразе – вдруг прокатит. Но почти всегда лучше знать, *почему* ответ зависит от обстоятельств, что мы сейчас и обсудим.

В данном случае обстоятельства – это соотношение времени поиска и вращения. В нашем случае если время поиска гораздо больше задержки вращения, то SSTF (и его варианты) вполне устраивает. Но допустим, что поиск чуть быстрее вращения. Тогда имело бы прямой смысл сдвинуть кронштейн *подальше*, чтобы обслужить запрос к сектору 8 на внешней дорожке, чем сдвинуть его ближе, к средней дорожке, и обслужить сектор 16, который еще должен сделать полный оборот, чтобы пройти под головкой.

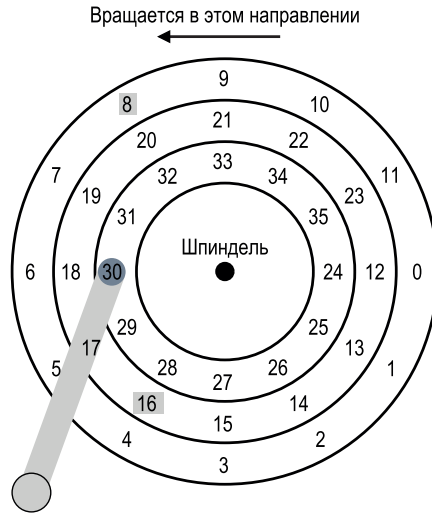


Рис. 37.8 ❖ SSTF: иногда недостаточно хорош

В современных дисках, как мы видели выше, поиск и вращение приблизительно эквивалентны (но, конечно, все зависит от конкретного запроса), поэтому алгоритм SPTF полезен и повышает производительность. Но его труднее реализовать в ОС, которая обычно не знает ни точных границ дорожек, ни текущего положения головки (в смысле вращения). Поэтому SPTF обычно реализуется в контроллере диска, как описано ниже.

Совет: всё всегда зависит от обстоятельств (закон Ливны)

Почти на любой вопрос можно ответить «смотря по обстоятельствам», как всегда говорит наш коллега Мирон Ливны. Однако будьте осторожны: если вы слишком часто будете отвечать таким образом, то вам вообще перестанут задавать вопросы. Например, вас спрашивают: «Пойдешь обедать?» А вы отвечаете: «Смотря по обстоятельствам, а ты пойдешь?»

Другие проблемы планирования

Есть много других вопросов, которые мы не обсуждаем в этом кратком введении в основы работы дисков, планирования и т. п. Один такой вопрос: где осуществляется планирование диска в современных системах? Раньше всем планированием занималась операционная система; рассмотрев все множество ожидающих запросов, ОС выбирала наилучший и отправляла его диску. По завершении запроса выбирался следующий и т. д. Диски тогда были проще, а вместе с ними проще была и жизнь.

В современных системах диски могут накапливать несколько ожидающих выполнения запросов и располагают изощренными внутренними планировщиками (которые могут точно реализовать алгоритм SPTF, поскольку

контроллеру диска известны все необходимые детали, включая и позицию головок). Поэтому планировщик ОС обычно выбирает несколько лучших, на его взгляд, запросов (скажем, 16) и передает их диску разом, а диск, пользуясь своими знаниями о положении головки и геометрии дорожек, обслуживает эти запросы в оптимальном порядке (следуя алгоритму SPTF).

Еще одна важная задача, решаемая планировщиком диска, – **объединение ввода-вывода**. Пусть, например, имеется последовательность запросов на чтение блоков 33, затем 8, потом 34, как показано на рис. 37.8. В этом случае планировщик должен объединить запросы к блокам 33 и 34 в один, а последующее переупорядочение применяется уже к объединенным запросам. Объединение особенно важно на уровне ОС, потому что уменьшает число запросов, передаваемых диску, а значит, и накладные расходы.

И последняя проблема, стоящая перед современными планировщиками: сколько времени должна ждать система, прежде чем передать запрос ввода-вывода диску? Наивный подход заключается в том, что, получив даже один-единственный запрос, система должна незамедлительно переадресовать его диску; такой подход называется «с поддержанием загрузки» (work-conserving), потому что диск никогда не простаивает, если есть требующие обслуживания запросы. Однако исследования по **упреждающему планированию диска** показали, что иногда лучше немного подождать [ID01], т. е. применить подход **без поддержания загрузки** (non-work-conserving). За время ожидания может поступить новый, в каком-то смысле «лучший» запрос к диску, поэтому общая эффективность повысится. Конечно, решение о том, когда стоит ждать и сколько времени, дается нелегко; дополнительные сведения можно найти в научных статьях или посмотреть, как эти идеи воплощаются в жизнь в коде ядра Linux (если ваши амбиции высоки).

37.6. РЕЗЮМЕ

Мы вкратце описали, как работают диски. На самом деле мы представили только подробную функциональную модель, не затрагивая поражающие воображение физику, электронику и материаловедение, без которых невозможно производство дисков. Тем, кому интересны детали такого рода, следовало бы выбрать другой профилирующий предмет (или, быть может, записаться на спецкурс). А если вам хватает модели, то и ладно! Мы же теперь можем воспользоваться этой моделью для построения более интересных систем на основе этих невероятных устройств.

Литература

[ADR03] «More Than an Interface: SCSI vs. ATA» by Dave Anderson, Jim Dykes, Erik Riedel. FAST '03, 2003. *Одна из лучших сравнительно недавних работ по устройству современных дисков, обязательное чтение для тех, кто хочет узнать больше.*

[CKR72] «Analysis of Scanning Policies for Reducing Disk Seek Times» E. G. Coffman, L. A. Klimko, B. Ryan SIAM Journal of Computing, September 1972, Vol 1. No 3. *Одна из ранних работ по планированию доступа к дискам.*

[HK+17] «The Unwritten Contract of Solid State Drives» by JunHe, Sudarsun Kannan, Andrea C. Arpaci-Dusseau, Remzi H. Arpaci-Dusseau. EuroSys '17, Belgrade, Serbia, April 2017. *Мы распространили идею неписаного контракта на SSD-диски. Использование SSD-дисков представляет не меньшие, а иногда и большие трудности, чем жестких дисков.*

[ID01] «Anticipatory Scheduling: A Disk-scheduling Framework To Overcome Deceptive Idleness In Synchronous I/O» by Sitaram Iyer, Peter Druschel. SOSP '01, October 2001. *Интересная статья, в которой показано, как ожидание может повысить качество планирования диска: возможно, на подходе лучшие запросы!*

[JW91] «Disk Scheduling Algorithms Based On Rotational Position» by D. Jacobson, J. Wilkes. Technical Report HPL-CSP-91-7rev1, Hewlett-Packard, February 1991. *Более современный подход к планированию дисков. Работа осталась техническим отчетом (а не опубликованной статьей), потому что авторы выступили с докладом [SCOS90].*

[RW92] «An Introduction to Disk Drive Modeling» by C. Ruemmler, J. Wilkes. IEEE Computer, 27:3, March 1994. *Потрясающее введение в основы функционирования дисков. Некоторые положения устарели, но большая часть по-прежнему актуальна.*

[SCO90] «Disk Scheduling Revisited» by Margo Seltzer, Peter Chen, John Ousterhout. USENIX 1990. *В этой статье обсуждается вопрос о том, почему вращение играет важную роль при планировании доступа к диску.*

[SG04] «MEMS-based storage devices and standard disk interfaces: A square peg in a round hole?» Steven W. Schlosser, Gregory R. Ganger FAST '04, pp. 87–100, 2004. *Хотя высказанные в этой статье идеи, связанные с MEMS (микроэлектромеханическими системами), пока еще не оказали влияния, обсуждение контракта между файловыми системами и дисками – удивительный и сохраняющий актуальность вклад. Позже мы воспользовались этой работой при изучении «неписаного контракта твердотельных дисков» [HK+17].*

[S09a] «Barracuda ES.2 data sheet» by Seagate, Inc. Доступно по следующему адресу, по крайней мере было доступно: http://www.seagate.com/docs/pdf/data-sheet/disc/ds_barracuda_es.pdf. *Техническое описание. Читайте на свой страх и риск. Риск чего? Умереть от скуки.*

[S09b] «Cheetah 15K.5» by Seagate, Inc. Доступно по следующему адресу, мы почти уверены, что так оно и есть: <http://www.seagate.com/docs/pdf/datasheet/disc/ds-cheetah-15k-5-us.pdf>. *См. замечание о технических описаниях выше.*

Домашнее задание (эмуляция)

В этом домашнем задании вы будете использовать программу disk.py, чтобы познакомиться с работой современного диска. У нее много параметров

и, в отличие от большинства других эмуляций, имеется графический аниматор, который показывает, что происходит, когда диск работает. Детали см. в файле README.

1. Вычислите время поиска, время вращения и время передачи для следующих наборов запросов: -а 0, -а 6, -а 30, -а 7,30,8 и, наконец, -а 10,11,12,13.
2. То же самое, но измените скорость поиска, задав следующие значения: -S 2, -S 4, -S 8, -S 10, -S 40, -S 0.1. Как изменяется каждое время?
3. То же самое, но измените скорость вращения: -R 0.1, -R 0.5, -R 0.01. Как изменяется каждое время?
4. FIFO-очередь – не всегда лучшее решение. Например, в каком порядке лучше обрабатывать поток запросов -а 7,30,8? Сначала подайте эту рабочую нагрузку SSTF-планировщику (-р SSTF); сколько времени займет обслуживание (поиск, вращение, передача) каждого запроса?
5. Теперь воспользуйтесь SATF-планировщиком (-р SATF). Есть ли какая-нибудь разница на рабочей нагрузке -а 7,30,8? Подберите такой набор запросов, для которого SATF лучше SSTF. Вообще, при каких условиях SATF лучше SSTF?
6. Рассмотрим такой набор запросов: -а 10,11,12,13. Что не так при его обработке? Попробуйте добавить сдвиг дорожек для решения проблемы (-о skew). При каком сдвиге производительность максимальна, если скорость поиска выбирается по умолчанию? А при других значениях скорости поиска (например, -S 2, -S 4)? Попробуйте вывести общую формулу для вычисления сдвига.
7. Задайте многозонный диск с различными плотностями дорожек, например -z 10,20,30, где параметр описывает разные секторные углы на внешней, средней и внутренней дорожках. Выполните несколько случайных запросов (например, -а -1 -A 5,-1,0, где -а -1 означает, что запросы случайные, и далее говорится, что нужно сгенерировать пять запросов к секторам с номерами от 0 до max) и вычислите время поиска, вращения и передачи. Задавайте разные начальные значения генератора случайных чисел. Какова пропускная способность (в секторах в единицу времени) на внешней, средней и внутренней дорожках?
8. Окно планирования определяет, сколько запросов диск может анализировать одновременно. Сгенерируйте случайную рабочую нагрузку (например, -A 1000, -1,0 с различными начальными значениями) и посмотрите, сколько времени займет обслуживание в соответствии с алгоритмом SATF, когда окно планирования изменяется от 1 до полного числа запросов. При каком окне планирования производительность максимальна? Подсказка: задайте флаг -с и не включайте графику (-G), чтобы выполнить запросы быстро. Имеет ли значение выбранный алгоритм, если окно планирования равно 1?
9. Создайте последовательность запросов, при которой один конкретный запрос зависает, в предположении, что используется политика SATF. Какой будет производительность на этой последовательности, если

использовать алгоритм планирования **BSATF (ограниченный SATF)**? В этом алгоритме задается окно планирования (например, -w 4) и планировщик переходит к следующему окну запросов только после того, как все запросы в текущем окне обслужены. Решает ли это проблему зависания? Какова производительность по сравнению с SATF? Как в общем случае выбирать компромисс между производительностью и избеганием зависаний?

10. Все рассмотренные выше алгоритмы планирования **жадные**, т. е. выбирают следующий наилучший вариант, вместо того чтобы поискать глобально оптимальный план. Попробуйте найти набор запросов, на котором жадное планирование не оптимально.

Глава 38

Избыточный массив недорогих дисков (RAID)

Работая с диском, мы иногда мечтаем, чтобы он был побыстрее; операции ввода-вывода медленные, поэтому могут стать узким местом в системе. А иногда мы мечтаем, чтобы диск был побольше: данные из сети поступают и поступают, так что наши диски постепенно заполняются. А иногда хочется, чтобы диск был понадежнее; если диск отказывает, а резервной копии нет, то все наши ценные данные растворяются в воздухе.

СУЩЕСТВО ПРОБЛЕМЫ: КАК СДЕЛАТЬ БОЛЬШОЙ, БЫСТРЫЙ И НАДЕЖНЫЙ ДИСК

Как сделать так, чтобы система хранения была емкой, быстрой и надежной? Какие есть методы? Какие существуют компромиссы?

В этой главе мы познакомимся с **избыточным массивом недорогих дисков** (Redundant Array of Inexpensive Disks), больше известным под названием **RAID [P+88]**, – техникой объединения нескольких дисков для построения более быстрой, емкой и надежной системы. Термин был введен в обиход в конце 1980-х годов группой исследователей из Калифорнийского университета в Беркли (под руководством профессоров Дэвида Паттерсона и Рэнди Каца и тогда еще студента Гарта Гибсона). Примерно в то время многие ученые одновременно пришли к мысли использовать несколько дисков для построения улучшенной системы хранения [BG88, K86, K88, PB86, SG86].

Извне RAID-массив выглядит как один диск: группа блоков, доступных для чтения и записи. Но внутри RAID устроен весьма сложно, он состоит из нескольких дисков, памяти (энергозависимой и энергонезависимой) и одного или нескольких процессоров для управления системой. Аппаратный RAID – это, по существу, компьютерная система, предназначенная для решения узкой задачи: управления группой дисков. RAID предлагает ряд преимуществ по сравнению с одним диском. Одно из них – *производительность*. Благодаря параллельному использованию нескольких дисков скорость ввода-вывода существенно возрастает. Другое преимущество – *емкость*. Для больших наборов данных требуются большие диски. Наконец, RAID-массив может повысить *надежность*; распределение данных между несколькими дисками (без

применения методов RAID) уязвимо к выходу из строя одного диска, но если добавить какую-то форму **избыточности**, то RAID-массив даже при отказе диска будет работать как ни в чем не бывало.

СОВЕТ: ПРОЗРАЧНОСТЬ СПОСОБСТВУЕТ ВНЕДРЕНИЮ

При рассмотрении вопроса о том, как добавить новую функциональность в систему, следует также задуматься о том, можно ли сделать это **прозрачно**, т. е. так, чтобы не вносить изменений в остальные части системы. Если требуется полностью переписать существующее программное обеспечение (или внести радикальные изменения в оборудование), то шансы на принятие идеи уменьшаются. RAID – идеальный пример, и, конечно же, прозрачность немало способствовала успеху этой технологии; администратору достаточно было установить RAID-массив с интерфейсом SCSI вместо SCSI-диска, ни на йоту не изменяя остальной системы (основного компьютера, ОС и т. д.), – и можно было начинать использовать. Благодаря решению проблемы **внедрения** RAID добился успеха с самого первого дня.

Удивительно, что технология RAID предоставляет системе все эти преимущества **прозрачно**, т. к. с точки зрения хост-системы RAID-массив выглядит как один большой диск. Красота прозрачности в том и состоит, что она позволяет заменить диск RAID-массивом, не изменяя ни единой строки в существующем коде; операционная система и клиентские приложения продолжают работать, как работали. В этом смысле прозрачность значительно улучшает пригодность RAID к внедрению, поскольку дает пользователям и администраторам возможность установить RAID, не беспокоясь о программной совместимости.

Теперь обсудим некоторые важные аспекты RAID. Начнем с интерфейса, модели отказов, а затем перейдем к оценке конструкции RAID по трем критериям: емкость, надежность и производительность. После этого мы перейдем к другим вопросам, важным для проектирования и реализации.

38.1. ИНТЕРФЕЙС И ВНУТРЕННЕЕ УСТРОЙСТВО RAID

Для файловой системы RAID-массив выглядит как большой и, хочется надеяться, быстрый и надежный диск. Как и один диск, он представляется линейным массивом блоков, каждый из которых файловая система (или другой клиент) может читать и записывать.

Когда файловая система отправляет запрос *логического ввода-вывода* RAID-массиву, тот должен определить, к какому диску (или дискам) обратиться для выполнения запроса, а затем инициировать одну или несколько операций *физического ввода-вывода*. Точная природа этих физических операций зависит от уровня RAID, но это мы подробно обсудим позже. Пока же в качестве простого примера рассмотрим RAID-массив, в котором хранятся две копии

каждого блока (по одной на двух разных дисках). При записи в такую **зеркалированную** систему RAID должен выполнить две физические операции ввода-вывода для каждой логической.

RAID-система часто выглядит как отдельный ящик со стандартным подключением к хост-компьютеру (например, SCSI или SATA). Но внутри RAID-массив устроен весьма сложно, он состоит из микроконтроллера, управляющего работой RAID, энергозависимой памяти, например DRAM, для буферизации прочитанных и записанных блоков и в некоторых случаях энергонезависимой памяти для более надежной буферизации операций записи. Иногда могут включаться даже специализированные схемы для вычисления четности (полезные при некоторых уровнях RAID, как мы увидим ниже). На верхнем уровне RAID представляет собой специализированную компьютерную систему с процессором, памятью и дисками, но вместо приложений он выполняет специальную программу, обеспечивающую функционирование RAID-массива.

38.2. Модель отказов

Чтобы лучше разобраться в технологии RAID и сравнить различные подходы, мы должны иметь в виду модель отказов. RAID-массивы проектируются, так чтобы обнаруживать некоторые виды отказов диска и восстанавливаться после них, поэтому, чтобы выбрать подходящую конструкцию, нужно точно знать, какие могут быть отказы.

Первая из рассматриваемых нами моделей отказов очень проста, она называется **«прекращение работы при ошибке»** (fail-stop) [S84]. В этом случае считается, что диск может быть ровно в одном из двух состояний: работает или отказал. Если диск работает, то все блоки можно читать и записывать. Если же он отказал, то потерян навсегда.

Критически важный аспект этой модели – предположения касательно обнаружения отказа. Считается, что если диск отказал, то отказ легко обнаружить. Например, в случае RAID-массива предполагается, что оборудование (или программное обеспечение) контроллера сможет сразу заметить, что диск вышел из строя.

Так что пока не будем задумываться о более сложных «тихих» отказах вроде повреждения данных на диске. Также оставим в стороне вопрос о недоступности одного блока на работающем в остальных отношениях диске (иногда это называется скрытой ошибкой сектора). Эти более сложные (и, к сожалению, более реалистичные) отказы мы рассмотрим позже.

38.3. Как оценивать RAID

Как мы вскоре узнаем, существует несколько подходов к построению RAID-массива. У всех них разные характеристики, которые стоит оценить, чтобы понять сильные и слабые стороны каждого.

Мы будем оценивать каждый вариант RAID по трем критериям. Первый – **емкость**: если имеется набор N дисков, каждый из которых содержит B блоков, то сколько полезной емкости доступно клиентам RAID-массива? Без избыточности ответ был бы $N \cdot B$, но если система хранит две копии каждого блока (это называется **зеркалированием**), то полезная емкость равна $(N \cdot B)/2$. Существуют и другие схемы (например, с контролем четности), которые дают промежуточные результаты.

Второй критерий оценки – **надежность**. Сколько отказов может выдержать данная конструкция? В соответствии с нашей моделью отказов мы предполагаем, что отказывает весь диск; в последующих главах (посвященных целостности данных) мы подумаем, как быть с более сложными сценариями отказов.

Наконец, третий критерий – **производительность**. Оценивать ее трудно, потому что она сильно зависит от рабочей нагрузки. Поэтому, прежде чем приступать к оценке производительности, мы опишем типовые рабочие нагрузки, которые необходимо учесть.

Рассмотрим теперь три важных варианта RAID: RAID уровня 0 (чередование), RAID уровня 1 (зеркалирование) и RAID уровня 4/5 (избыточность по четности). Слово «уровень» берет начало в основополагающей работе Паттерсона, Гибсона и Каца [P+88].

38.4. RAID уровня 0: ЧЕРЕДОВАНИЕ

Этот уровень по существу даже нельзя назвать RAID, потому что он не предусматривает никакой избыточности. Однако RAID уровня 0, или, как его чаще называют, **чередование**, очень полезен в качестве верхней границы производительности и емкости, поэтому о нем стоит знать.

Простейшая форма чередования – поочередная запись на имеющиеся диски в следующем порядке (предполагается массив из четырех дисков):

Диск 0	Диск 1	Диск 2	Диск 3
0	1	2	3
4	5	6	7
8	9	10	11
12	13	14	15

Рис. 38.1 ❖ RAID-0: простое чередование

Основная идея ясна из рис. 38.1: циклически распределять блоки массива по дискам. Задача в том, чтобы добиться максимального распараллеливания при доступе к соседним блокам массива (например, в случае длинной операции последовательного чтения). Блоки, составляющие один ряд, называются **полосой**, т. е. блоки 0, 1, 2 и 3 на рисунке выше образуют полосу.

Мы сделали упрощающее предположение о том, что блоки (скажем, размера 4 КБ) помещаются на каждый диск по одному. Однако это не обязательно должно быть так. Например, блоки можно чередовать, как показано на рис. 38.2:

Диск 0	Диск 1	Диск 2	Диск 3	
0	2	4	6	Размер порции
1	3	5	7	2 блока
8	10	12	14	
9	11	13	15	

Рис. 38.2 ❖ Чередование с большим размером порции

В этом примере мы помещаем на каждый диск по два блока размером 4 КБ и только потом переходим к следующему диску. Поэтому **размер порции** в таком RAID-массиве равен 8 КБ, а полоса состоит из 4 порций, или 32 КБ.

ОТСТУПЛЕНИЕ: ПРОБЛЕМА ОТОБРАЖЕНИЯ В RAID

Прежде чем переходить к изучению характеристик емкости, надежности и производительности RAID, ненадолго отвлечемся и поговорим о **проблеме отображения**, свойственной RAID-массивам. Если в операции чтения или записи указан логический блок, то откуда RAID знает соответствующий ему физический диск и смещение от его начала?

Для простых уровней RAID нет ничего сложного в отображении логических блоков на физические. Возьмем первый пример чередования (размер порции = 1 блок = 4 КБ). В этом случае RAID, зная логический адрес блока A, легко может вычислить номер физического диска и смещение в нем по формулам:

Диск = $A \% \text{number_of_disks}$
 Смещение = $A / \text{number_of_disks}$

Отметим, что все это целочисленные операции (например, $4/3 = 1$, а не $1.33333...$). Покажем на простом примере, как работают эти формулы. Допустим, что поступает запрос на блок 12. Поскольку всего имеется 4 диска, это означает, что нужно вычислить значение $14 \% 4 = 2$, т. е. блоку соответствует диск 2. А точный номер физического блока равен $14/4 = 3$. Итак, логическому блоку 14 соответствует четвертый блок (номер 3, а нумерация начинается с 0) третьего диска (номер 2, нумерация начинается с 0).

Подумайте, как следует модифицировать эти формулы для других размеров порции. Давайте, это не трудно!

Размеры порций

Размер порции влияет прежде всего на производительность массива. Если размер порции мал, то многие файлы будут разбросаны по разным дискам, что увеличивает степень параллелизма при чтении и записи файла. Однако при этом возрастает время позиционирования при доступе к блокам на разных дисках, потому что время позиционирования запроса в целом равно максимуму из времен позиционирования подзапросов на каждом диске.

С другой стороны, если размер порции велик, то степень такого внутри-файлового параллелизма меньше, поэтому для достижения высокой произ-

водительности нужно много конкурентных запросов. Однако время позиционирования уменьшается; если файл помещается в одной порции и, значит, находится на одном диске, то время позиционирования при доступе к нему совпадает со временем позиционирования этого конкретного диска.

Поэтому определить «наилучший» размер порции трудно, поскольку нужно хорошо знать характеристики рабочей нагрузки в данной системе [CL95]. Далее мы будем предполагать, что порция состоит из одного блока (4 КБ). В большинстве реальных массивов размер порции больше (например, 64 КБ), но для обсуждаемых нами вопросов это не имеет значения, поэтому для простоты ограничимся одним блоком.

Возвращаясь к анализу RAID-0

Теперь оценим емкость, надежность и производительность чередования. С точки зрения емкости, этот метод идеален: если имеется N дисков размером B блоков каждый, то чередование дает полезную емкость $N \cdot B$ блоков. В плане надежности чередование тоже идеально, только в худшем смысле: любой отказ диска приводит к потере данных. Наконец, производительность отличная: для обслуживания запросов ввода-вывода используются все диски, и часто параллельно.

Оценка производительности RAID

При анализе производительности RAID можно рассматривать две разные метрики. Первая – *задержка одного запроса*. Понимать, какова задержка одного запроса к RAID-массиву, полезно, потому что позволяет оценить потенциальную степень параллелизма при выполнении одной логической операции ввода-вывода. Вторая характеристика – *пропускная способность в установившемся режиме*, т. е. общая пропускная способность при большем числе конкурентных запросов. Поскольку RAID-массивы часто используются в высокопроизводительных средах, стационарная пропускная способность критически важна и потому будет в фокусе нашего внимания.

Чтобы лучше разобраться в пропускной способности, опишем некоторые особенно интересные рабочие нагрузки. В этом обсуждении будем предполагать, что существует два типа рабочей нагрузки: **последовательная** и **произвольная**. При последовательной рабочей нагрузке предъявляются запросы на доступ к большим непрерывным сериям блоков; например, запрос (или несколько запросов) для доступа к 1 МБ данных, начиная с блока x и заканчивая блоком $(x + 1\text{МБ})$, будет считаться последовательным. Последовательные рабочие нагрузки встречаются во многих ситуациях (например, при поиске слова в большом файле), поэтому считаются важными.

В случае произвольной рабочей нагрузки каждый запрос сравнительно мал и адресован разным блокам в произвольных местах диска. Например, сначала может быть обращение к блоку по логическому адресу 10, затем к блоку по логическому адресу 550 000, потом 20 100 и т. д. Такой тип досту-

па характерен, например, для транзакционной рабочей нагрузки в системах управления базами данных (СУБД), поэтому тоже считается важным.

Разумеется, реальные рабочие нагрузки не так просты и часто представляют собой смесь последовательного и произвольного доступов со всеми промежуточными поведением. Но для простоты будем рассматривать только эти два варианта.

Легко понять, что характеристики производительности диска для последовательной и произвольной рабочей нагрузок совершенно различны. В случае последовательного доступа диск работает в самом эффективном режиме, тратя мало времени на поиск и ожидание вращения, а больше всего на передачу данных. В случае произвольного доступа ситуация прямо противоположная: больше времени тратится на поиск и ожидание, а сравнительно мало на передачу данных. Чтобы отразить это различие в нашем анализе, будем предполагать, что диск может передавать данные со скоростью S МБ/с при последовательной нагрузке и R МБ/с при произвольной. В общем случае S много больше R ($S \gg R$).

Чтобы убедиться, что вы понимаете это различие, сделаем простое упражнение – вычислим S и R при заданных характеристиках диска. Предположим, что средний размер последовательной передачи равен 10 МБ, а средний размер произвольной – 10 КБ. Предположим также, что диск имеет следующие характеристики:

Среднее время поиска	7 мс
Средняя задержка вращения	3 мс
Скорость передачи	50 МБ/с

Для вычисления S нужно сначала вычислить, сколько времени занимает типичная передача длиной 10 МБ. Сначала мы тратим 7 мс на поиск, затем 3 мс на вращение, после чего начинается передача: 10 МБ со скоростью 50 МБ/с будут переданы за 1/5 секунды, т. е. 200 мс. Итого на завершение каждого запроса длиной 10 МБ мы тратим 210 мс. Теперь вычислим S :

$$S = \frac{\text{Количество данных}}{\text{Время доступа}} = \frac{10 \text{ МБ}}{210 \text{ мс}} = 47.62 \text{ МБ/с}.$$

Поскольку большая часть времени тратится на передачу данных, S очень близко к пиковой пропускной способности диска (затраты на поиск и вращение амортизируются).

R вычисляется аналогично. Время поиска и ожидания вращения такие же, а время, потраченное на передачу, равно 10 КБ со скоростью 50 МБ/с = 0.195 мс.

$$R = \frac{\text{Количество данных}}{\text{Время доступа}} = \frac{10 \text{ КБ}}{10.195 \text{ мс}} = 0.981 \text{ МБ/с}.$$

Как видим, R меньше 1 МБ/с, а S/R равно почти 50.

Снова возвращаемся к анализу RAID-0

Теперь вычислим производительность чередования. Мы уже говорили, что в общем и целом она высокая. Например, задержка запроса к одному блоку должна быть почти такой же, как при наличии единственного диска, ведь RAID-0 просто переадресует запрос к одному из дисков.

Что касается пропускной способности в установившемся режиме, мы ожидаем достичь максимальной пропускной способности системы, т. е. N (число дисков), умноженное на S (пропускная способность одного диска при последовательном доступе). При большом числе произвольных запросов ввода-вывода мы снова можем использовать все диски, а значит, получить пропускную способность $N \cdot R$ МБ/с. Ниже мы увидим, что оба этих значения проще всего вычислить, и они будут служить эталоном для сравнения с другими уровнями RAID.

38.5. RAID уровня 1: зеркалирование

Первый уровень RAID после чередования называется зеркалированием. В зеркалированной системе имеется несколько копий каждого блока, и, конечно, копии хранятся на разных дисках. Это позволяет справляться с отказами дисков.

Будем предполагать, что в типичной зеркалированной системе имеются две физические копии каждого логического блока, например:

Диск 0	Диск 1	Диск 2	Диск 3
0	0	1	1
2	2	3	3
4	4	5	5
6	6	7	7

Рис. 38.3 ❖ Простой RAID-1: зеркалирование

В этом примере содержимое дисков 0 и 1 одинаково, как и дисков 2 и 3; данные чередуются между этими зеркальными парами. Как вы, наверное, поняли, есть несколько способов распределить копии блоков между дисками. Показанный выше – наиболее распространенный, иногда его называют **RAID-10** (или **RAID 1+0**), потому что используются зеркальные пары (RAID-1) с чередованием (RAID-0). Другой, тоже популярный, способ – **RAID-01** (или **RAID 0+1**), когда имеется два больших массива с чередованием (RAID-0), а поверх них организовано зеркалирование (RAID-1). Пока что, говоря о зеркалировании, будем иметь в виду показанную выше схему.

При чтении блока из зеркалированного массива у RAID есть выбор: прочитать одну или другую копию. Например, если запрашивается логический блок 5, то RAID вправе прочитать его как с диска 2, так и с диска 3. Но при

записи такого выбора нет: RAID-массив обязан обновить обе копии данных во имя надежности. Отметим, однако, что обе операции записи можно выполнять параллельно; например, запись логического блока 5 на диски 2 и 3 может производиться одновременно.

Анализ RAID-1

Давайте оценим RAID-1. С точки зрения емкости, RAID-1 обходится дорого; если уровень зеркалирования равен 2, то мы получаем только половину полезной емкости. А при наличии N по B блоков каждый полезная емкость RAID-1 равна $(N \cdot B)/2$.

С надежностью у RAID-1 все хорошо. Он устойчив к отказам одного любого диска. На самом деле если немного повезет, то надежность RAID-1 даже лучше. Представьте, что на рисунке выше оба диска 0 и 1 отказали. А данные все равно не будут потеряны! Вообще, зеркалированная система (с уровнем зеркалирования 2) наверняка сохраняет работоспособность при отказе 1 диска и может вынести $N/2$ отказов в зависимости от того, какие диски вышли из строя. Но на практике мы не хотим оставлять такие вещи на волю случая, поэтому большинство пользователей считают, что зеркалирование рассчитано на одиночный отказ.

Наконец, проанализируем производительность. Задержка одного запроса чтения такая же, как на одиночном диске, поскольку RAID-1 просто перенаправляет запрос к одной из копий. С записью все немного иначе: запрос считается выполненным только после завершения двух физических операций записи. Правда, эти операции могут производиться параллельно, и тогда время будет примерно равно времени одной записи, но поскольку необходимо дожидаться обеих физических операций записи, задержка определяется максимумом из двух значений времени поиска и задержки вращения, а потому в среднем может быть немного выше, чем при записи на один диск.

ОТСТУПЛЕНИЕ: ПРОБЛЕМА СОГЛАСОВАННОГО ОБНОВЛЕНИЯ RAID

Прежде чем анализировать RAID-1, обсудим проблему, которая имеет место в любой RAID-системе с несколькими дисками и носит название **проблемы согласованного обновления** [DAA05]. Она возникает при записи в RAID-массив, когда нужно обновить несколько дисков в составе одной логической операции. Для конкретности предположим, что речь идет о зеркалированном массиве дисков.

Пусть инициирована запись в RAID-массив и RAID решил, что необходимо записать на два диска: 0 и 1. Массив начинает запись на диск 0, но еще до того, как инициирована запись на диск 1, отключается питание (или система «падает»). Предположим, что в этом несчастливом случае запись на диск 0 завершилась (а запись на диск 1 нет, ведь она даже не начиналась).

В результате этого так не вовремя случившегося сбоя питания две копии блока оказались несогласованными: на диске 0 находится новая версия, а на диске 1 старая. Мы хотели бы, чтобы состояние обоих дисков изменялось атомарно, т. е. обе копии были либо новыми, либо старыми.

Общий способ решения этой задачи – **журнал с упреждающей записью**, в который сначала записывается, что RAID собирается сделать (т. е. записать на оба диска некоторые данные), а только потом выполняется сама операция. В таком случае есть уверенность, что даже в случае сбоя произойдет то, что планировалось; выполнив процедуру **восстановления**, которая воспроизводит все ожидающие транзакции на RAID-массиве, мы сможем гарантировать, что все зеркальные копии (в случае RAID-1) синхронизированы.

И последнее замечание: поскольку ведение журнала всех операций записи на диске обошлось бы слишком дорого, в большинстве аппаратных реализаций RAID имеется небольшая энергонезависимая память (например, питаемая от аккумулятора), в которой находится этот журнал. Поэтому для согласованного обновления не требуется создавать дорогой журнал на диске.

Чтобы проанализировать пропускную способность в установившемся режиме, начнем с последовательной рабочей нагрузки. При последовательной записи на диск каждая логическая операция записи порождает две физические; например, при записи логического блока 0 (на рисунке выше) RAID пишет на оба диска 0 и 1. Поэтому можно заключить, что максимальная пропускная способность последовательной записи на зеркалированный массив равна $N/2 \cdot S$, или половине пиковой пропускной способности.

К сожалению, точно такая же производительность получается при последовательном чтении. Можно было бы подумать, что последовательное чтение должно работать быстрее, потому что прочитать нужно только одну копию данных, а не обе. Но покажем на примере, почему это не сильно помогает. Допустим, требуется прочитать блоки 0, 1, 2, 3, 4, 5, 6 и 7. Пусть блок 0 читается с диска 0, блок 1 – с диска 2, блок 2 – с диска 1, а блок 3 – с диска 3. Затем блоки 4, 5, 6, 7 читаются соответственно с дисками 0, 2, 1, 3. Кажется, что раз мы задействовали все диски, то к нашим услугам вся пропускная способность массива.

Чтобы понять, почему это необязательно так, рассмотрим запросы, которые получает какой-то один диск (скажем, диск 0). Сначала он получает запрос на чтение блока 0, затем запрос на чтение блока 4 (а блок 2 пропускается). Фактически каждый диск получает запрос на чтение каждого второго блока. И когда под головкой проходит пропущенный блок, никакой пользы клиент не ощущает. Таким образом, каждый диск использует только половину своей пиковой пропускной способности. А значит, пропускная способность при последовательном чтении равна лишь $N/2 \cdot S$ МБ/с.

Для зеркалированного RAID-массива наилучшей является произвольная рабочая нагрузка. В этом случае операции чтения можно распределить между всеми дисками, достигнув максимально возможной пропускной способности. Стало быть, для произвольного чтения RAID-1 обеспечивает производительность $N \cdot R$ МБ/с.

Наконец, произвольная запись работает, как и можно было предполагать, со скоростью $N/2 \cdot R$ МБ/с. Каждая логическая запись преобразуется в две физические, и потому, несмотря на использование всех дисков, клиент получает только половину располагаемой пропускной способности. Хотя запись логического блока x приводит к двум параллельным записям на два разных

физического диска, пропускная способность при большом числе коротких запросов составляет только половину той, что мы наблюдали при чередовании. Но, как мы скоро увидим, половинная пропускная способность – это не так плохо!

38.6. RAID уровня 4: экономия места ЗА СЧЕТ ЧЕТНОСТИ

Теперь покажем другой метод организации избыточности в массиве дисков – вычисление **четности**. Основанные на этой идее подходы имеют целью сэкономить потребление места и тем самым уменьшить гигантский штраф, который приходится платить за использование зеркалированной системы. Но расплачиваться приходится производительностью.

На рис. 38.4 приведен пример системы RAID-4 с пятью дисками. С каждой полосой данных связан один блок **четности**, в котором хранится контрольная информация для этой полосы. Например, в блоке четности P1 хранится информация, вычисленная по блокам 4, 5, 6 и 7.

Диск 0	Диск 1	Диск 2	Диск 3	Диск 4
0	1	2	3	P0
4	5	6	7	P1
8	9	10	11	P2
12	13	14	15	P3

Рис. 38.4 ❖ RAID-4 с четностью

Для вычисления четности необходима математическая функция, которая позволила бы восстановить данные в случае утраты любого блока в полосе. Оказывается, что с этой задачей отлично справляется функция **XOR**. В применении к множеству битов XOR дает 0, если в этом множестве четное число единиц, и 1, если число единиц нечетное. Например:

C0	C1	C2	C3	P
0	0	1	1	$XOR(0,0,1,1) = 0$
0	1	0	0	$XOR(0,1,0,0) = 1$

В первой строке (0,0,1,1) есть две единицы (C2, C3), поэтому XOR всех битов равно 0 (P); аналогично во второй строке только одна единица (C1), поэтому XOR должно быть равно 1 (P). Запомнить это легко: число единиц в каждой строке должно быть четным, и этот **инвариант** должен поддерживаться для всего RAID-массива.

Данный пример также показывает, как можно использовать информацию о четности для восстановления в случае отказа. Допустим, что столбец C2 потерян. Чтобы вычислить, какими должны быть значения в этом столбце, мы

должны просто прочитать все остальные значения в той же строке (включая бит четности) и **реконструировать** недостающее. Предположим, что в первой строке значение в столбце C2 утрачено (а находилась там 1); прочитав остальные значения в этой строке (0 в столбце C0, 0 в C1, 1 в C3 и 0 в столбце четности P), будем иметь 0, 0, 1, 0. Мы знаем, что в каждой строке должно быть четное число единиц, а стало быть, недостающее значение равно 1. Именно так работает реконструкция в схеме четности на основе XOR! Отметим еще, как именно вычисляется реконструированное значение: мы применяем XOR ко всем битам данных и четности – точно так же, как вычисляли бит четности изначально.

У вас наверняка возник вопрос: мы тут распространяемся о применении XOR к битам, а ведь RAID пишет на каждый диск блоки размером 4 КБ (или больше); и как же применить XOR к целым блокам для вычисления четности? Да просто. Нужно применять XOR поразрядно, а результаты применения к каждой группе одноименных разрядов помещать в соответствующий бит блока четности. Например, если бы блок состоял из двух битов (да, это очень мало по сравнению с 4 КБ, но достаточно, чтобы понять происходящее), то получилась бы такая картина:

Блок0	Блок1	Блок2	Блок3	Четность
00	10	11	10	11
10	01	00	01	10

Как видим, четность вычисляется для каждого бита каждого блока, и результат помещается в блок четности.

Анализ RAID-4

Проанализируем RAID-4. С точки зрения емкости, в RAID-4 используется один диск для хранения информации о четности для каждой группы защищаемых дисков. Поэтому полезная емкость для группы RAID равна $(N - 1) \cdot B$.

С надежностью тоже все понятно: RAID-4 допускает отказ одного диска, и не более. Если из строя выйдет больше дисков, то реконструировать потерянные данные не получится.

И наконец, о производительности. На этот раз начнем с анализа пропускной способности в установившемся режиме. При последовательном чтении можно задействовать все диски, кроме диска четности, поэтому максимальная эффективная пропускная способность равна $(N - 1) \cdot S$ МБ/с (простой случай).

Чтобы оценить производительность последовательной записи, нужно сначала понять, как эта запись производится. При записи большой порции данных на диск массив RAID-4 может выполнить простую оптимизацию, называемую **цельнополосной записью** (full-stripe write). Например, допустим, что в запросе записи участвуют блоки 0, 1, 2 и 3 (рис. 38.5).

Диск 0	Диск 1	Диск 2	Диск 3	Диск 4
0	1	2	3	P0
4	5	6	7	P1
8	9	10	11	P2
12	13	14	15	P3

Рис. 38.5 ❖ Цельнополосная запись в RAID-4

В этом случае RAID-массив может просто вычислить новое значение P0 (применив XOR к блокам 0, 1, 2 и 3), а затем параллельно записать все блоки (включая и блок четности) на пять дисков (выделены на рисунке серым цветом). Таким образом, цельнополосная запись – самый эффективный способ записи на диск в массиве RAID-4.

Разобравшись с цельнополосной записью, вычислить производительность последовательной записи в RAID-4 уже просто; эффективная пропускная способность также равна $(N - 1) \cdot S$ МБ/с. Хотя диск четности в течение операции используется постоянно, клиент не получает от этого никакой выгоды.

Теперь проанализируем производительность произвольного чтения. Как видно из рисунка выше, множество операций чтения произвольных оди-ночных блоков можно распределить по дискам данных, но диск четности в их число не входит. Поэтому эффективная производительность равна $(N - 1) \cdot R$ МБ/с.

Произвольная запись, которую мы оставили на сладкое, – самый интерес-ный случай в массиве RAID-4. Допустим, что мы хотим перезаписать блок 1 в примере выше. Можно было бы, ничтоже сумняшеся, так и поступить, но тогда возникает проблема: блок четности P0 не отражает новое значение четности полосы, т. е. мы должны обновить также P0. А как это сделать пра-вильно и эффективно?

Есть два метода. Первый называется **аддитивная четность** и подразуме-вает следующие действия. Чтобы вычислить новое значение блока четности, нужно параллельно прочесть все остальные блоки в полосе (в нашем при-мере – блоки 0, 2 и 3) и применить XOR к ним и к новому блоку (1). Результат и есть новый блок четности. Для завершения операции записи мы должны записать новые данные и новый блок четности на соответствующие диски, тоже параллельно.

Проблема этого метода в том, что он масштабируется при увеличении чис-ла дисков, т. е. чем больше дисков в RAID-массиве, тем больше должно быть операций чтения для вычисления четности. Поэтому был предложен метод **субтрактивной четности**.

Рассмотрим, к примеру, следующую битовую схему (4 бита данных, 1 бит четности):

C0	C1	C2	C3	P
0	0	1	1	XOR(0,0,1,1) = 0

Допустим, что требуется перезаписать бит C2 новым значением, которое мы обозначим $C2_{new}$. Субтрактивный метод состоит из трех шагов. Сначала

мы читаем старое значение $C2$ ($C2_{old} = 1$) и старую четность ($P_{old} = 0$). Затем сравниваем старые и новые данные; если они совпадают ($C2_{new} = C2_{old}$), то и бит четности не изменится, т. е. $P_{new} = P_{old}$. Если же они различны, то старый бит четности следует инвертировать, т. е. если $P_{old} = 1$, то P_{new} будет равно 0, а если $P_{old} = 0$, то P_{new} следует положить равным 1. Эту длинную фразу можно элегантно выразить с помощью оператора XOR (ниже он обозначен символом $\oplus$):

$$P_{new} = (C_{old} \oplus C_{new}) \oplus P_{old}. \quad (38.1)$$

Поскольку мы имеем дело с блоками, а не с битами, то это вычисление следует применить ко всем битам в блоке (т. е. 4096×8 раз для блока размером 4 КБ). Поэтому в большинстве случаев новый блок будет отличаться от старого, а вместе с ним будет отличаться и блок четности.

Теперь вы, наверное, понимаете, когда следует использовать аддитивный метод вычисления четности, а когда субтрактивный. Подумайте, при каком количестве дисков в системе аддитивный метод выполняет меньше операций ввода-вывода, чем субтрактивный; где находится точка перелома?

Для анализа производительности предположим, что используется субтрактивный метод. Тогда для каждой операции записи RAID-массив должен выполнить 4 физические операции ввода-вывода (два чтения и две записи). Представим теперь, что таких операций записи много, сколько из них RAID-4 сможет выполнить параллельно? Чтобы ответить на этот вопрос, снова обратимся к схеме размещения данных на дисках массива RAID-4 (рис. 38.6).

Диск 0	Диск 1	Диск 2	Диск 3	Диск 4
0	1	2	3	P0
*4	5	6	7	+P1
8	9	10	11	P2
12	*13	14	15	+P3

Рис. 38.6 ❖ Пример: запись в блоки 4 и 13 и соответствующие блоки четности

Допустим, что массив попросили выполнить 2 короткие операции записи приблизительно в одно время: в блоки 4 и 13 (отмечены * на рисунке). Эти блоки находятся на дисках 0 и 1, поэтому чтение и запись данных можно производить параллельно – и это хорошо. Проблема возникает с диском четности: оба запроса должны прочитать блоки четности: первый – блок 1, второй – блок 3 (отмечены знаками +). Надеемся, что теперь проблема ясна: диск четности является узким местом при рабочей нагрузке такого типа; иногда говорят о **проблеме короткой записи** для RAID-массивов с четностью. Таким образом, хотя к дискам данных и можно обращаться параллельно, диск четности мешает реализоваться параллелизму: из-за него все операции записи оказываются сериализованы. Поскольку необходимы две операции ввода-вывода (чтение и запись) с диском четности для каждой логической операции записи, для вычисления производительности коротких произвольных операций записи в массиве RAID-4 мы должны учесть

производительность диска четности на этих двух операциях, так что получается $R/2$ МБ/с. Пропускная способность RAID-4 при коротких произвольных операциях записи отвратительна и не становится лучше при добавлении дисков в систему.

В заключение проанализируем задержку ввода-вывода в массиве RAID-4. Как вы уже знаете, одиночное чтение (при условии что не было отказа) отображается на один диск, поэтому задержка эквивалентна задержке запроса к одному диску. При одиночной записи требуется два чтения, а затем две записи; и чтение, и запись могут производиться параллельно, поэтому полная задержка примерно в два раза больше, чем для одного диска (небольшое отличие есть, потому что мы должны ждать завершения обеих операций чтения, так что время позиционирования соответствует худшему случаю, но зато обновление не сопровождается потерями на поиск и стоимость позиционирования может оказаться лучше, чем в среднем).

38.7. RAID уровня 5: ротация четности

Для решения проблемы короткой записи (по крайней мере, частичного) Паттерсон, Гибсон и Кац придумали RAID-5. Этот вариант работает почти так же, как RAID-4, но блок четности **ротится**, перемещаясь между дисками (рис. 38.7).

Диск 0	Диск 1	Диск 2	Диск 3	Диск 4
0	1	2	3	P0
5	6	7	P1	4
10	11	P2	8	9
15	P3	12	13	14
P4	16	17	18	19

Рис. 38.7 ❖ RAID-5 с чередованием четности

Как видим, блоки четности соседних полос находятся на разных дисках, и тем самым расширяется узкое место, связанное с диском четности в RAID-4.

Анализ RAID-5

Анализ RAID-5 по большей части повторяет анализ RAID-4. Так, эффективная емкость и отказоустойчивость в обоих случаях одинаковы, как и производительность последовательного чтения и записи. Задержка одиночного запроса (чтения или записи) тоже такая же, как в RAID-4.

Производительность произвольного чтения немного лучше, потому что теперь мы можем задействовать все диски. Наконец, производительность произвольной записи значительно лучше, потому что становится возможным распараллеливание запросов. Пусть производится запись в блоки 1 и 10, она сводится к обращениям к дискам 1 и 4 (блок 1 и его четность) и к дис-

кам 0 и 2 (блок 10 и его четность). Поэтому все операции могут выполняться параллельно. На самом деле и в общем случае можно предполагать, что при большом количестве произвольных запросов мы сможем обеспечить равномерную загрузку дисков. Если это так, то полная пропускная способность для коротких операций записи равна $N/4 \cdot R$ МБ/с. Четырехкратная потеря обусловлена тем, что в случае RAID-5 по-прежнему генерируется 4 операции ввода-вывода – это та цена, которую приходится платить за контроль четности.

Поскольку RAID-5 мало чем отличается от RAID-4, но в некоторых случаях лучше, он почти полностью вытеснил RAID-4 с рынка. Единственное место, где он еще применяется, – системы, где заведомо выполняются только длинные операции записи, так что проблемы короткой записи не существует вовсе [HLM94]; в таких случаях RAID-4 иногда используется, потому что немного проще строится.

38.8. СРАВНЕНИЕ RAID: ИТОГИ

Теперь подведем итоги нашему упрощенному сравнению уровней RAID (см. рис. 38.8). Для простоты мы опустили ряд деталей в процессе анализа. Например, при записи в зеркалированную систему среднее время поиска немного больше, чем при записи на один диск, поскольку оно равно максимуму из двух времен поиска (по одному на каждый диск). Поэтому производительность произвольной записи на два диска в общем случае чуть меньше, чем при записи на один диск. Кроме того, при обновлении диска четности в RAID-4/5 первое чтение старой четности, скорее всего, будет сопровождаться полным временем поиска и полной задержкой вращения, тогда как при второй записи четности поиска не будет вообще, а останется только вращение.

	RAID-0	RAID-1	RAID-4	RAID-5
Емкость	$N \cdot B$	$(N \cdot B) / 2$	$(N - 1) \cdot B$	$(N - 1) \cdot B$
Надежность	0	1 (наверняка) $N/2$ (если повезет)	1	1
Пропускная способность				
Последовательное чтение	$N \cdot S$	$N/2 \cdot S$	$(N - 1) \cdot S$	$(N - 1) \cdot S$
Последовательная запись	$N \cdot S$	$N/2 \cdot S$	$(N - 1) \cdot S$	$(N - 1) \cdot S$
Произвольное чтение	$N \cdot R$	$N \cdot R$	$(N - 1) \cdot R$	$N \cdot R$
Произвольная запись	$N \cdot R$	$N/2 \cdot R$	$1/2 \cdot R$	$N/4 \cdot R$
Задержка				
Чтения	T	T	T	T
Записи	T	T	$2T$	$2T$

Рис. 38.8 ❖ Емкость, надежность и производительность RAID

Однако на рис. 38.8 все же отражены существенные различия, и он полезен для осмысления компромиссов, принимаемых при выборе уровня. При анализе задержки T обозначает время запроса к одному диску.

В заключение отметим, что если вас интересует только производительность, а надежность не важна, то лучшим вариантом является чередование. Если же нужна высокая производительность произвольного ввода-вывода и надежность, то лучше выбрать зеркалирование, но ценой будет потеря емкости. Если на первом месте стоят емкость и надежность, выбирайте RAID-5; расплачиваться придется производительностью коротких операций записи. Наконец, если вы всегда производите только последовательный ввод-вывод и хотите максимизировать емкость, то и в этом случае имеет смысл остановиться на RAID-5.

38.9. ДРУГИЕ ИНТЕРЕСНЫЕ ВОПРОСЫ RAID

Есть еще ряд интересных идей, которые стоило бы (и, наверное, следовало бы) обсудить в контексте RAID. Возможно, мы рано или поздно напишем о них.

Например, есть еще много вариантов RAID, в т. ч. уровни 2 и 3 из оригинальной классификации и уровень 6, устойчивый к отказам нескольких дисков [C+04]. Интересно также, что делает RAID в случае отказа диска; иногда имеется диск **горячей замены**, готовый в любой момент подменить вышедший из строя диск. А что происходит с производительностью при отказе и в период реконструкции отказавшего диска? Существуют также более реалистичные модели отказа, принимающие во внимание **скрытые ошибки секторов** или **искажение блоков** [B+08], а также многочисленные методы обработки таких отказов (см. главу, посвященную целостности данных). Наконец, RAID можно даже реализовать программно, такие системы с **программным RAID** дешевле, но имеют другие проблемы, в т. ч. с согласованным обновлением [DAA05].

38.10. РЕЗЮМЕ

Мы обсудили технологию RAID, которая позволяет преобразовать несколько независимых дисков в более емкий и надежный массив, причем сделать это прозрачно, так что расположенные выше уровня программного и аппаратного обеспечения почти ничего не будут знать об изменении.

Существует много уровней RAID, а какой выбрать, сильно зависит от того, что конечный пользователь считает важным. Например, RAID с зеркалированием прост, надежен и в целом предлагает хорошую производительность, но ценой потери части емкости. Наоборот, RAID-5 надежен и обладает хорошими характеристиками с точки зрения емкости, но демонстрирует низкую производительность, когда рабочая нагрузка состоит из коротких операций записи. Правильный выбор уровня RAID и задание параметров (размер порции, количество дисков и т. д.) – трудная задача, пока остается скорее искусством, чем наукой.

Литература

[B+08] «An Analysis of Data Corruption in the Storage Stack» by Lakshmi N. Bairava-sundaram, Garth R. Goodson, Bianca Schroeder, Andrea C. Arpaci-Dusseau, Remzi H. Arpaci-Dusseau. FAST '08, San Jose, CA, February 2008. *Наша собственная работа, в которой анализируется, насколько часто повреждаются данные на дисках. Не часто, но бывает! И поэтому нужно подумать о надежной системе хранения.*

[BJ88] «Disk Shadowing» by D. Bitton and J. Gray. VLDB 1988. *Одна из первых работ, в которой обсуждается зеркалирование, которое авторы называют «теневым копированием».*

[CL95] «Striping in a RAID level 5 disk array» by Peter M. Chen and Edward K. Lee. SIGMETRICS 1995. *Хороший анализ важных параметров массива дисков RAID-5.*

[C+04] «Row-Diagonal Parity for Double Disk Failure Correction» by P. Corbett, B. English, A. Goel, T. Grcanac, S. Kleiman, J. Leong, S. Sankar. FAST '04, February 2004. *Не первая работа по RAID-системам с двумя дисками четности, но сравнительно недавнее и в высшей степени понятное изложение этой идеи. Прочитайте, если хотите узнать больше.*

[DAA05] «Journal-guided Resynchronization for Software RAID» by Timothy E. Denehy, A. Arpaci-Dusseau, R. Arpaci-Dusseau. FAST 2005. *Наша собственная работа по проблеме согласованного обновления. Мы решаем ее для программного RAID, соединив механизм журналирования файловой системы на верхнем уровне с программной реализацией RAID на нижнем.*

[HLM94] «File System Design for an NFS File Server Appliance» by Dave Hitz, James Lau, Michael Malcolm. USENIX Winter 1994, San Francisco, California, 1994. *Лаконичная статья, в которой описывается исторически фундаментальный для систем хранения продукт – файловая система WAFL (write-anywhere file layout – файловая структура с записью повсюду), лежащая в основе сетевых систем хранения данных компании NetApp.*

[K86] «Synchronized Disk Interleaving» by M. Y. Kim. IEEE Transactions on Computers, Volume C-35: 11, November 1986. *Одна из самых первых работ по технологии RAID.*

[K88] «Small Disk Arrays – The Emerging Approach to High Performance» by F. Kurzweil. Presentation at Spring COMPCON '88, March 1, 1988, San Francisco, California. *Еще одна ранняя работа по RAID.*

[P+88] «Redundant Arrays of Inexpensive Disks» by D. Patterson, G. Gibson, R. Katz. SIGMOD 1988. *Эта знаменитая статья Паттерсона, Гибсона и Каца считается самой главной на тему RAID. Она прошла испытание временем и открыла эру RAID, включая и само название RAID!*

[PB86] «Providing Fault Tolerance in Parallel Secondary Storage Systems» by A. Park, K. Balasubramaniam. Department of Computer Science, Princeton, CS-TR-O57-86, November 1986. *Еще одна ранняя работа по RAID.*

[SG86] «Disk Striping» by K. Salem, H. Garcia-Molina. IEEE International Conference on Data Engineering, 1986. *И да, еще одна ранняя работа по RAID. Их много,*

кажется, что они полезли из всех щелей после публикации основополагающей работы по RAID, доложенной на конференции SIGMOD.

[S84] «Byzantine Generals in Action: Implementing Fail-Stop Processors» by F. B. Schneider. ACM Transactions on Computer Systems, 2(2):145154, May 1984. *И напоследок статья не о RAID! В ней описываются типы системных отказов и как сделать так, чтобы компонент прекращал работу в случае ошибки.*

Домашнее задание (эмуляция)

В этом разделе вы будете работать с программой `raid.py`, простым эмулятором RAID, который поможет закрепить знания о работе RAID-систем. Детали см. в файле README.

Вопросы

1. Воспользуйтесь эмулятором для простых тестов отображения в RAID. Запустите его с разными уровнями (0, 1, 4, 5) и попробуйте вычислить отображения для набора запросов. В случае RAID-5 попробуйте определить различие между левосимметричной и левоасимметричной структурой. Для генерирования других задач используйте различные начальные значения генератора случайных чисел.
2. То же, что в первом вопросе, но запускайте с разными размерами порций (флаг `-c`). Как размер порции влияет на отображения?
3. То же, что и выше, но запускайте с флагом `-r`, чтобы изменить природу каждой задачи на противоположную.
4. Снова используйте флаг `-r`, но увеличьте размер каждого запроса с помощью флага `-s`. Попробуйте размеры 8k, 12k и 16k при разных уровнях RAID. Что происходит с паттерном ввода-вывода при увеличении размера запроса? Попробуйте также с последовательной рабочей нагрузкой (`-w sequential`); при каких размерах запросов RAID-4 и RAID-5 демонстрируют существенно более высокую производительность ввода-вывода?
5. Воспользуйтесь встроенными в эмулятор средствами хронометража (`-t`), чтобы оценить производительность на 100 операциях произвольного чтения при разных уровнях RAID и 4 дисках.
6. То же, что и выше, но увеличьте количество дисков. Насколько хорошо масштабируется производительность RAID каждого уровня при увеличении числа дисков?
7. То же, что и выше, но вместо операции чтения задайте операции записи (`-w 100`). Насколько хорошо теперь масштабируется производительность RAID каждого уровня? Попробуйте грубо оценить время завершения 100 операций произвольной записи.
8. Снова воспользуйтесь режимом хронометража, но на этот раз для последовательной рабочей нагрузки (`-w sequential`). Как производительность зависит от уровня RAID и типа операции: чтение или запись? А от размера запроса? Каким должен быть размер запроса записи при использовании RAID-4 или RAID-5?

Глава 39

Интерлюдия: файлы и каталоги

До сих пор мы наблюдали за развитием двух ключевых абстракций операционной системы: процесса в части, посвященной виртуализации CPU, и адресного пространства в части, посвященной виртуализации памяти. В совокупности эти две абстракции позволяют выполнять программу, как если бы она существовала в собственном изолированном мире: имела свой личный процессор (или несколько процессоров) и свою личную память. Благодаря этой иллюзии программирование системы значительно упрощается, поэтому она сегодня присутствует не только на серверах и настольных компьютерах, но все чаще и на других программируемых платформах, включая мобильные телефоны и т. п.

В этом разделе мы добавим еще один кусочек в этот пазл виртуализации: систему **долговременного хранения**. Устройство для постоянного хранения, например классический **жесткий диск** или более современное **твердотельное запоминающее устройство**, позволяет хранить информацию в течение длительного времени. В отличие от памяти, его содержимое не пропадает при выключении питания. Поэтому ОС должна особенно внимательно относиться к таким устройствам, ведь там хранятся ценные для пользователей данные.

Существо проблемы:

КАК УПРАВЛЯТЬ УСТРОЙСТВОМ ДОЛГОВРЕМЕННОГО ХРАНЕНИЯ

Как ОС должна управлять устройством постоянного хранения? Какие имеются API? Каковы наиболее важные аспекты реализации?

В нескольких следующих главах мы будем изучать основные методы управления средствами долговременного хранения, обращая особое внимание на производительность и надежность. Но начнем мы с обзора API: интерфейсов для взаимодействия с файловой системой Unix.

39.1. ФАЙЛЫ И КАТАЛОГИ

По мере развития систем хранения кристаллизовались две ключевые абстракции. Первая из них – **файл**. Файл – это просто линейный массив байтов, каждый из которых можно прочитать или записать. У каждого файла имеется **низкоуровневое имя**, обычно какое-то число; пользователь зачастую ничего не знает об этом имени (как мы вскоре увидим). По историческим причинам, низкоуровневое имя файла часто называют **номером индексного дескриптора** (inode number). Мы подробно расскажем об индексных дескрипторах в последующих главах, а пока просто будем предполагать, что с каждым файлом связан номер индексного дескриптора.

В большинстве систем ОС ничего не знает о структуре файла (картинка это, текстовый файл или исходный код на С); задача файловой системы – просто сохранить данные на диске и гарантировать, что, обратившись впоследствии к этим данным, мы получим то, что сохранили. А добиться этого не так просто, как кажется!

Вторая абстракция – **каталог**. Каталог, как и файл, имеет низкоуровневое имя (номер индексного дескриптора), но его содержимое весьма специфическое: список пар (понятное человеку внешнее имя, низкоуровневое имя). Предположим, к примеру, что имеется файл с низкоуровневым именем «10», к которому обращаются по внешнему имени «foo». Тогда каталог, где находится файл «foo», будет содержать запись («foo», «10»), которая отображает внешнее имя на низкоуровневое. Каждая запись в каталоге ссылается либо на файл, либо на другую каталог. Помещая один каталог внутрь другого, мы можем построить произвольное **дерево каталогов** (или **иерархию каталогов**), в котором хранятся все файлы и каталоги.

Иерархия каталогов начинается с **корневого каталога** (в Unix-системах он обозначается /), а для разделения **имен подкаталогов** используется какой-нибудь символ-**разделитель**. Например, если пользователь создал каталог foo в корневом каталоге /, а затем файл bar.txt в каталоге foo, то на этот файл можно будет сослаться по **абсолютному пути**, который в данном случае равен /foo/bar.txt. Более сложное дерево каталогов показано на рис. 39.1; в нем присутствуют каталоги /, /foo, /bar, /bar/bar, /bar/foo и файлы /foo/bar.txt и /bar/foo/bar.txt. Каталоги и файлы могут иметь одинаковые имена, при условии что находятся в разных ветвях дерева файловой системы (например, на рисунке есть два файла с именем bar.txt: /foo/bar.txt и /bar/foo/bar.txt).

Отметим также, что имя файла нередко состоит из двух частей, например bar и txt, разделенных точкой. Первая часть – произвольное имя, а вторая обычно обозначает **тип** файла, например код на С (.c), изображение (.jpg) или музыкальный файл (.mp3). Однако это всего лишь **соглашение**: обычно никто не заставляет хранить в файле main.c именно исходный код на С.

Таким образом, мы видим, что файловая система предоставляет важнейшую вещь: удобный способ **поименовать** все интересующие нас файлы. Имена – первый шаг на пути, открывающем доступ к системным ресурсам. В Unix файловая система дает единообразный способ получить доступ к фай-

лам на диске, к USB-диску, к CD-ROM и ко многим другим устройствам, поскольку все они находятся в едином дереве каталогов.

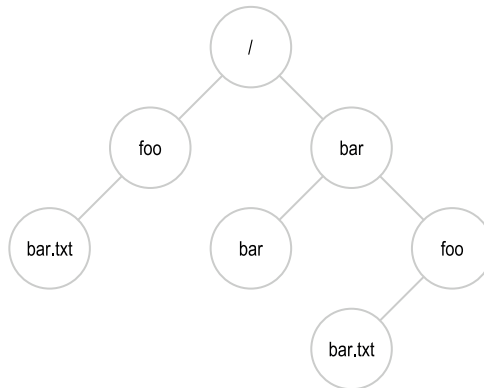


Рис. 39.1 ❖ Пример дерева каталогов

СОВЕТ: ОТНОСИТЕСЬ К ИМЕНАМ СЕРЬЕЗНО

Именованное – важный аспект компьютерных систем [SK09]. В Unix практически все объекты представлены именами в файловой системе. Помимо файлов, в старой доброй файловой системе могут встречаться устройства, каналы и даже процессы [K84]. Такое единообразие упрощает концептуальную модель системы и делает ее более модульной. Поэтому при создании системы или интерфейса тщательно обдумывайте используемые имена.

39.2. ИНТЕРФЕЙС ФАЙЛОВОЙ СИСТЕМЫ

Теперь обсудим интерфейс файловой системы более подробно. Начнем с основ: создания файла, доступа к нему и удаления. Быть может, вам кажется, что тут нет ничего сложного, но по ходу дела мы столкнемся с таинственным системным вызовом для удаления файлов, который носит название `unlink()`. Надеемся, что в конце этой главы тайна развеется!

39.3. СОЗДАНИЕ ФАЙЛОВ

Начнем с самой главной операции: создания файла. Для этой цели предназначен системный вызов `open()`; вызвав `open()` и передав флаг `O_CREAT`, программа создает новый файл. Ниже показано, как создать файл «foo» в текущем рабочем каталоге:

```
int fd = open("foo", O_CREAT|O_WRONLY|O_TRUNC, S_IRUSR|S_IWUSR);
```

Функция `open()` принимает ряд флагов. В данном случае флаг `O_CREAT` во втором параметре означает создание файла, если такого еще не существует, флаг `O_WRONLY` – что файл предназначен только для записи, а флаг `O_TRUNC` – что если файл уже существует, то его необходимо сократить до нулевой длины, удалив тем самым все прежнее содержимое. Третий параметр задает разрешения – в данном случае файл может читать и записывать только его владелец.

ОТСТУПЛЕНИЕ: СИСТЕМНЫЙ ВЫЗОВ `creat()`

Раньше применялся еще один способ создания – `creat()`:

```
int fd = creat("foo"); // можно задать второй флаг, описывающий разрешения
```

Можно считать, что `creat()` – то же самое, что `open()` с флагами `O_CREAT | O_WRONLY | O_TRUNC`. Поскольку `open()` умеет создавать файлы, использование функции `creat()` вышло из моды (ее можно реализовать в виде библиотечной функции, вызывающей `open()`), но она занимает особое место в истории Unix. Именно, когда Кена Томпсона спросили, что бы он сделал по-другому, если бы проектировал Unix заново, тот ответил: «Написал бы `creat` с буквой `e` в конце».

Важно, что `open()` возвращает **дескриптор файла** – целое число, уникальное в рамках процесса и используемое ОС для доступа к файлу. Открыв файл, мы используем его дескриптор для чтения и записи в файл (в предположении, что у нас есть на это право). Таким образом, дескриптор файла – это **возможность** [L84], т. е. непрозрачный описатель, позволяющий выполнять определенные операции. По-другому можно представлять себе дескриптор файла как указатель на объект типа файла; имея такой объект, мы можем вызывать его «методы», чтобы получить доступ к файлу, например `read()` и `write()` (как это делается, мы увидим ниже).

Как уже было сказано, дескрипторы файлов управляются операционной системой на уровне процесса. Это значит, что внутри структуры `proc` в Unix имеется какая-то простая структура (например, массив). Вот как выглядит соответствующий код в ядре xv6 [CK+08]:

```
struct proc {
    ...
    struct file *ofile[NFILE]; // открытые файлы
    ...
};
```

Простой массив (рассчитанный максимум на `NFILE` открытых файлов) позволяет отследить, какие файлы открыты в процессе. Каждый элемент массива – указать на структуру `struct file`, в которой хранится информация о записываемом или читаемом файле; мы вернемся к этому вопросу ниже.

39.4. ЧТЕНИЕ И ЗАПИСЬ ФАЙЛОВ

Имея файл, мы, конечно, захотим его читать или записывать. Начнем с чтения существующего файла. Если бы мы работали на уровне командной строки, то могли бы просто воспользоваться программой `cat`, чтобы вывести содержимое файла на экран:

```
prompt> echo hello > foo
prompt> cat foo
hello
prompt>
```

СОВЕТ: ПОЛЬЗУЙТЕСЬ STRACE (И ДРУГИМИ ПОДОБНЫМИ ИНСТРУМЕНТАМИ)

Инструмент `strace` – замечательный способ увидеть, что делает программа. Он трассирует все выполняемые программой системные вызовы, показывая их аргументы и возвращаемые значения.

Инструмент трассировки также принимает аргументы, что дополнительно увеличивает его полезность. Например, флаг `-f` означает, что нужно трассировать все дочерние процессы; флаг `-t` говорит, что нужно выводить время каждого вызова; флаг `-e trace=open,close,read,write` – что нужно трассировать только эти системные вызовы и игнорировать все остальные. Есть много других интересных флагов – почитайте страницу руководства, чтобы узнать, как воспользоваться всей мощью этого чудесного инструмента.

В примере выше мы перенаправляем вывод программы `echo` в файл `foo`, который будет содержать слово «hello». Затем мы используем программу `cat`, чтобы посмотреть на содержимое файла. Но как `cat` получает доступ к файлу `foo`?

Чтобы ответить на этот вопрос, воспользуемся невероятно полезным инструментом трассировки системных вызовов, выполняемых программой. В Linux он называется `strace`; в других системах есть похожие инструменты (**dtruss** в Mac, **truss** в некоторых старых вариантах Unix). Программа `strace` показывает на экране все системные вызовы, сделанные указанной программой. Ниже приведен пример использования `strace` для выяснения того, что делает `cat` (некоторые вызовы удалены для большей понятности):

```
prompt> strace cat foo
...
open("foo", O_RDONLY|O_LARGEFILE)    = 3
read(3, "hello\n", 4096)              = 6
write(1, "hello\n", 6)                = 6
hello
read(3, "", 4096)                    = 0
close(3)                             = 0
...
prompt>
```


Первым делом `cat` открывает файл для чтения. Тут следует отметить три вещи: во-первых, что файл открывается только для чтения (не для записи), на что указывает флаг `O_RDONLY`; во-вторых, что используется 64-разрядное смещение (`O_LARGEFILE`); в-третьих, что вызов `open()` завершился успешно и вернул дескриптор файла 3.

Почему первый вызов `open()` вернул 3, а не 0 или 1, как вы, возможно, ожидали? Потому что в любом процессе уже открыто три файла: стандартный ввод (из которого процесс может читать входные данные), стандартный вывод (куда процесс может выводить данные, которые хочет показать на экране) и стандартный вывод для ошибок (куда процесс выводит сообщения об ошибках). Они-то и представлены дескрипторами 0, 1 и 2. Поэтому первый открытый программой (в данном случае `cat`) файл почти наверняка будет иметь дескриптор 3.

После успешного открытия `cat` обращается к системному вызову `read()`, чтобы прочитать байты из файла. Первый аргумент `read()` – дескриптор файла, который сообщает файловой системе, из какого файла читать; разумеется, процесс может открыть сразу несколько файлов, так что дескриптор дает операционной системе возможность узнать, какой именно файл имеется в виду. Второй аргумент – указатель на буфер, в который будет помещен результат `read()`; в приведенной выше трассе системных вызовов показано, что именно помещено в этот буфер («hello»). Третий аргумент – размер буфера, в данном случае 64 КБ. Вызов `read()` завершился успешно и вернул количество прочитанных байтов (6, из которых 5 – число букв в слове «hello», и один символ занят признаком конца строки).

Далее `strace` показывает еще одну интересную вещь: системный вызов `write()` с дескриптором файла 1. Выше мы отметили, что этот дескриптор называется стандартным выводом и служит для записи слова «hello» на экран, что и должна делать программа `cat`. Но вызывает ли она `write()` напрямую? Возможно – если сильно оптимизирована. Но если нет, то `cat` может вызывать библиотечную функцию `printf()`, которая берет на себя все вопросы форматирования и в конечном итоге пишет на стандартный вывод, т. е. на экран.

Затем программа `cat` пытается продолжить чтение из файла, но поскольку в нем больше ничего нет, `read()` возвращает 0 – для программы это означает, что весь файл прочитан. Тогда программа вызывает `close()`, сообщая системе, что закончила работать с файлом «foo», дескриптор которого передан ей в качестве аргумента. Файл закрывается, и чтение из него заканчивается.

Запись в файл требует аналогичных шагов. Сначала файл открывается для записи, затем вызывается `write()`, быть может, не один раз, если файл велик, а потом `close()`. Воспользуйтесь `strace`, чтобы протрассировать запись в файл: для программы, которую написали сами, или, например, для утилиты `dd`, вызываемой так: `dd if=foo of=bar`.

39.5. НЕПОСЛЕДОВАТЕЛЬНЫЕ ЧТЕНИЕ И ЗАПИСЬ

Мы обсудили, как читать и записывать файлы, но до сих доступ был **последовательным**, т. е. мы либо читали, либо записывали файл от начала до конца.

Однако иногда требуется читать или записывать файл, начиная с заданного смещения от его начала; например, если мы построили индекс над текстовым документом и хотим найти в нем определенное слово, то, вероятно, придется читать с **произвольной** позиции внутри файла. Для этого нам понадобится системный вызов `lseek()`. Вот как выглядит его прототип::

```
off_t lseek(int fildes, off_t offset, int whence);
```

С первым аргументом мы уже знакомы, это дескриптор файла. Вторым аргументом `offset` – **смещение** искомой позиции. А третий аргумент, в силу исторических причин названный `whence`, определяет, как именно осуществляется поиск. Приведем выдержку из страницы руководства:

Если `whence` равно `SEEK_SET`, то смещение от начала файла равно `offset` байт.

Если `whence` равно `SEEK_CUR`, то смещение от начала файла равно смещению текущей позиции плюс `offset` байт.

Если `whence` равно `SEEK_END`, то смещение от начала файла равно размеру файла плюс `offset` байт.

ОТСТУПЛЕНИЕ: СТРУКТУРА ДАННЫХ – ТАБЛИЦА ОТКРЫТЫХ ФАЙЛОВ

В каждом процессе имеется массив дескрипторов файлов, ссылающихся на запись в системной **таблице открытых файлов**. Каждая запись в этой таблице описывает файл, на который ссылается дескриптор, текущее смещение в нем и другие детали, например доступен ли файл для чтения или записи.

Как следует из описания, для каждого файла, открытого процессом, ОС хранит «текущее» смещение, определяющее, откуда начнется следующая операция чтения или записи. Следовательно, частью абстракции открытого файла является наличие текущего смещения, которое обновляется одним из двух способов. Первый – чтение или запись N байт, в этом случае к текущему смещению прибавляется N , так что операция чтения или записи *неявно* обновляет смещение. Второй – *явное* обновление с помощью `lseek`, при котором смещение изменяется, как описано выше.

Как вы, наверное, догадались, смещение хранится в структуре `struct file`, на которую ссылается `struct proc`. Ниже приведено ее упрощенное определение в исходном коде системы xv6:

```
struct file {
    int ref;
    char readable;
    char writable;
    struct inode *ip;
    uint off;
};
```

Как видим, из этой структуры ОС может узнать, допускает файл только чтение, только запись или то и другое, на какой файл эта структура ссылается (на него ведет указатель `ip` типа `struct inode *`), а также текущее смещение (`off`). Имеется также счетчик ссылок (`ref`), который мы обсудим ниже.

Структуры `file` описывают все открытые в данный момент в системе файлы, иногда их совокупность называется **таблицей открытых файлов**. В ядре `xv6` эта таблица хранится в виде массива, с которым ассоциирована блокировка:

```
struct {
    struct spinlock lock;
    struct file file[NFILE];
} ftable;
```

Для лучшего усвоения приведем несколько примеров. Сначала протрассируем процесс, который открывает файл (размером 300 байт) и несколько раз читает из него по 100 байт, вызывая `read()`. Ниже показана трасса системных вызовов, значения, возвращаемые каждым, и значение текущего смещения в таблице открытых файлов после каждого обращения к файлу.

Системные вызовы	Код возврата	Текущее смещение
<code>fd = open("file", O_RDONLY);</code>	3	0
<code>read(fd, buffer, 100);</code>	100	100
<code>read(fd, buffer, 100);</code>	100	200
<code>read(fd, buffer, 100);</code>	100	300
<code>read(fd, buffer, 100);</code>	0	300
<code>close(fd);</code>	0	—

В этой трассе стоит обратить внимание на несколько моментов. Во-первых, текущее смещение инициализируется нулем в момент открытия файла. Во-вторых, оно увеличивается после каждого вызова `read()`, поэтому для чтения следующей порции процессу нужно просто вызывать `read()`. Наконец, по достижении конца файла вызов `read()` возвращает 0, это означает, что файл прочитан полностью.

Далее протрассируем процесс, который открыл *один и тот же* файл дважды, после чего читал из обоих экземпляров.

Системные вызовы	Код возврата	OFT[10] Текущее смещение	OFT[11] Текущее смещение
<code>fd1 = open("file", O_RDONLY);</code>	3	0	—
<code>fd2 = open("file", O_RDONLY);</code>	4	0	0
<code>read(fd1, buffer1, 100);</code>	100	100	0
<code>read(fd2, buffer2, 100);</code>	100	100	100
<code>close(fd1);</code>	0	—	100
<code>close(fd2);</code>	0	—	—

В этом примере выделено два дескриптора файла (3 и 4), ссылающихся на *разные* записи таблицы открытых файлов (в данном случае – 10 и 11, как показано в заголовке таблицы, где OFT обозначает таблицу открытых файлов). Протрассировав происходящее, мы увидим, что смещения в каждом файле обновляются независимо.

И в последнем примере мы воспользуемся функцией `lseek()`, чтобы изменить текущее смещение перед чтением; в этом случае нам необходима только одна запись в таблице открытых файлов (как в первом примере).

Системные вызовы	Код возврата	Текущее смещение
fd = open("file", O_RDONLY);	3	0
lseek(fd, 200, SEEK SET);	200	200
read(fd, buffer, 50);	50	250
close(fd);	0	—

Здесь `lseek()` сначала устанавливает текущее смещение равным 200. Последующий вызов `read()` читает 50 байт и соответственно обновляет текущее смещение.

ОТСТУПЛЕНИЕ: ВЫЗОВ **LSEEK()** НЕ ПРИВОДИТ К ПОИСКУ НА ДИСКЕ

Неудачное имя системного вызова `lseek()` смущает студентов, стремящихся разобраться в работе дисков и построенных поверх них файловых систем. Не путайте одно с другим! Вызов `lseek()` просто изменяет переменную в памяти ОС, где хранится смещение от начала файла, с которого процесс должен начать следующую операцию чтения или записи. Поиск на диске имеет место, когда головка находится не над той дорожкой, к которой относилась последняя операция чтения или записи, и, следовательно, головку нужно переместить в другое место. Неразбериху только усугубляет тот факт, что вызов `lseek()` для чтения или записи конкретной части файла с последующим реальным чтением или записью этой части действительно приводит к поиску на диске. Поэтому вызов `lseek()`, конечно, может приводить к поиску в ходе выполнения чтения или записи в будущем, но сам по себе уж точно не вызывает никакого дискового ввода-вывода.

39.6. РАЗДЕЛЯЕМЫЕ ЗАПИСИ ТАБЛИЦЫ ФАЙЛОВ: `fork()` и `dup()`

Во многих случаях (и, в частности, в приведенных выше примерах) отображение дескриптора файла на запись в таблице открытых файлов взаимно однозначно. Например, выполняемый процесс может открыть файл, прочитать его и затем закрыть, и в этой ситуации в таблице открытых файлов будет создана единственная запись. Даже если другой процесс одновременно читает тот же файл, у него будет собственная запись в таблице открытых файлов. Поэтому все логические операции чтения и записи файла независимы, и для каждой хранится отдельное смещение.

Однако есть несколько интересных случаев, когда запись в таблице открытых файлов *разделяется*. Один из них возникает, когда родительский процесс создает дочерний с помощью `fork()`. На рис. 39.2 приведен фрагмент кода, в котором родитель создает потомка и ждет его завершения. Потомок изменяет текущее смещение с помощью `lseek()`, после чего выходит. А родитель, дождавшись завершения потомка, печатает текущее смещение.

При выполнении этой программы мы увидим такие строки:

```
prompt> ./fork-seek
потомок: смещение 10
родитель: смещение 10
prompt>
```

```

int main(int argc, char *argv[]) {
    int fd = open("file.txt", O_RDONLY);
    assert(fd >= 0);
    int rc = fork();
    if (rc == 0) {
        rc = lseek(fd, 10, SEEK_SET);
        printf("потомок: смещение %d\n", rc);
    } else if (rc > 0) {
        (void) wait(NULL);
        printf("родитель: смещение %d\n", (int) lseek(fd, 0, SEEK_CUR));
    }
    return 0;
}

```

Рис. 39.2 ❖ Разделяемая родителем и потомком запись в таблице файлов (fork-seek.c)

На рис. 39.3 показано, как связаны частные массивы дескрипторов каждого процесса, запись в таблице открытых файлов и ссылка на соответствующий индексный дескриптор в файловой системе. Наконец-то мы увидели использование **счетчика ссылок**. Если запись в таблице файлов разделяется между двумя процессами, то ее счетчик ссылок увеличивается, и лишь когда оба процесса закроют файл (или завершатся), эта запись будет удалена.

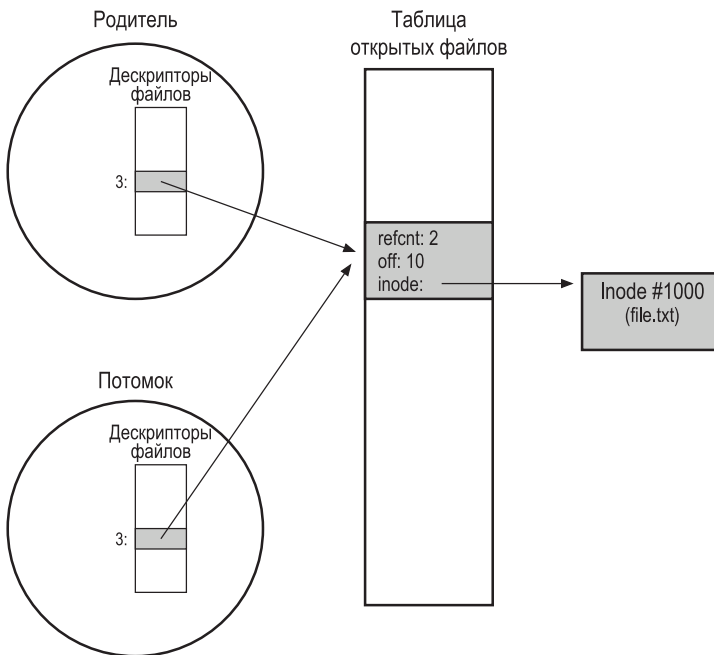


Рис. 39.3 ❖ Процессы, разделяющие запись в таблице открытых файлов

Разделение записи таблицы открытых файлов между родителем и потомком иногда бывает полезно. Например, если мы создали несколько процес-

сов, совместно работающих над некоторой задачей, то все они могут писать в один и тот же файл без дополнительной координации. Дополнительные сведения о том, что еще разделяют процессы после вызова `fork()`, смотрите на страницах руководства.

Еще один интересный и, пожалуй, более полезный случай разделения имеет место при обращении к системному вызову `dup()` (и его близким родственникам `dup2()` и `dup3()`). Вызов `dup()` позволяет процессу создать новый дескриптор файла, ссылающийся на тот же открытый файл, что и существующий дескриптор. На рис. 39.4 показан пример использования `dup()`.

```
int main(int argc, char *argv[]) {
    int fd = open("README", O_RDONLY);
    assert(fd >= 0);
    int fd2 = dup(fd);
    // теперь fd и fd2 можно использовать на равных правах
    return 0;
}
```

Рис. 39.4 ❖ Разделение записи таблицы файлов
с помощью `dup()` (`dup.c`)

Вызов `dup()` (и в особенности `dup2()`) полезен при написании оболочек Unix и выполнении операций типа перенаправления вывода; поразмыслите немного, почему это так! Вы, наверное, теперь недоумеваете, почему вам не сказали об этом, когда вы делали свой проект оболочки? Ну, что поделать, не получается все делать в правильном порядке, даже в такой потрясающей книге об операционных системах. Извините!

39.7. БЕЗОТЛАГАТЕЛЬНАЯ ЗАПИСЬ с помощью `fsync()`

Обычно, вызывая `write()`, программа говорит файловой системе: запиши эти данные в долговременное хранилище, можно не прямо сейчас, а когда-нибудь в будущем. Из соображений производительности файловая система буферизует такие операции записи в памяти на некоторое время (скажем, 5 или 30 секунд), а позже записывает все разом на запоминающее устройство. Вызывающему приложению кажется, что запись производится быстро, и лишь в редких случаях данные теряются (например, если машина «падает» после вызова `write()`, но перед записью на диск).

Однако некоторым приложениям недостаточно гарантии записи в конечном счете. Например, в системе управления базами данных (СУБД) протокол восстановления иногда требует форсированной записи на диск.

Для поддержки таких приложений в большинстве файловых систем имеются дополнительные управляющие API. В Unix соответствующий интерфейс называется `fsync(int fd)`. Когда процесс вызывает `fsync()` для какого-то де-

скриптора файла, файловая система принудительно сбрасывает на диск все **грязные** (т. е. еще не записанные) данные, относящиеся к заданному файлу. Функция `fsync()` возвращает управление, после того как все эти данные записаны.

Ниже приведен простой пример использования `fsync()`. Программа открывает файл `foo`, записывает в него одну порцию данных, а затем вызывает `fsync()`, чтобы форсировать перенос этих данных на диск. После возврата из `fsync()` приложение может спокойно работать дальше, зная, что данные сохранены (если, конечно, `fsync()` реализована правильно).

```
int fd = open("foo", O_CREAT|O_WRONLY|O_TRUNC, S_IRUSR|S_IWUSR);
assert(fd > -1);
int rc = write(fd, buffer, size);
assert(rc == size);
rc = fsync(fd);
assert(rc == 0);
```

Интересно, что эта последовательность не гарантирует всего, что хотелось бы; в некоторых случаях нужно еще выполнить `fsync()` для каталога, содержащего файл. Этот шаг гарантирует, что не только сам файл помещен на диск, но – в случае, когда это новый файл, – что зафиксирована его принадлежность каталогу. Неудивительно, что об этой детали часто забывают, в результате чего появляются многочисленные ошибки в приложениях [P+13,P+14].

39.8. ПЕРЕИМЕНОВАНИЕ ФАЙЛОВ

Иногда требуется дать существующему файлу новое имя. Для этой цели предназначена команда `mv`: ниже файл `foo` переименовывается в `bar`:

```
prompt> mv foo bar
```

Воспользовавшись `strace`, мы увидим, что `mv` обращается к системному вызову `rename(char *old, char *new)`, который принимает ровно два аргумента: старое имя (`old`) и новое имя (`new`).

Интересно, что `rename()` обычно предоставляет гарантию **атомарности** в случае крахов системы: если система «падает» во время переименования, то файл будет иметь либо старое, либо новое имя, но точно никакого промежуточного. Поэтому вызов `rename()` необходим в некоторых приложениях, требующих атомарного изменения состояния файла.

Давайте немного уточним этот момент. Пусть мы работаем с редактором файлов (например, `emacs`) и вставляем строку в середину файла. Файл называется, к примеру, `foo.txt`. Редактор мог бы следующим образом дать гарантию, что новый файл будет содержать все то, что содержал исходный, плюс новую строку (проверку ошибок для простоты опускаем):

```
int fd = open("foo.txt.tmp", O_WRONLY|O_CREAT|O_TRUNC,
              S_IRUSR|S_IWUSR);
```

```
write(fd, buffer, size); // записать новую версию файла
fsync(fd);
close(fd);
rename("foo.txt.tmp", "foo.txt");
```

Что делает редактор? Сохраняет новую версию файла с временным именем (foo.txt.tmp), форсирует запись на диск с помощью fsync(), а затем, будучи уверен, что новые метаданные и содержимое файла находятся на диске, переименовывает временный файл, возвращая ему исходное имя. Последний шаг атомарно переименовывает новый файл и вместе с тем удаляет его старую версию, так что достигается атомарное обновление файла.

39.9. ПОЛУЧЕНИЕ ИНФОРМАЦИИ О ФАЙЛАХ

От файловой системы мы ожидаем не только возможности доступа к файлам, но и предоставления информации о хранящихся в ней файлах. Такая информация называется **метаданными**. Для получения метаданных файла служат системные вызовы stat() и fstat(). Они принимают путь к файлу или дескриптор файла соответственно и заполняют структуру stat:

```
struct stat {
    dev_t st_dev;           /* идентификатор устройства, содержащего файл */
    ino_t st_ino;           /* номер индексного дескриптора */
    mode_t st_mode;         /* режим доступа */
    nlink_t st_nlink;       /* количество жестких ссылок */
    uid_t st_uid;           /* идентификатор пользователя-владельца */
    gid_t st_gid;           /* идентификатор группы-владельца */
    dev_t st_rdev;          /* идентификатор устройства (если это устройство) */
    off_t st_size;          /* полный размер в байтах */
    blksize_t st_blksize;   /* размер блока ввода-вывода файловой системы */
    blkcnt_t st_blocks;     /* количество выделенных блоков */
    time_t st_atime;        /* время последнего доступа */
    time_t st_mtime;        /* время последней модификации */
    time_t st_ctime;        /* время последнего изменения состояния */
};
```

Как видим, в системе хранится много информации о каждом файле, в т. ч. его размер (в байтах), низкоуровневое имя (номер файлового дескриптора), информация о владельце и о том, когда было последнее обращение к файлу и когда он был в последний раз изменен. Чтобы увидеть всю эту информацию, можно воспользоваться командной утилитой stat:

```
prompt> echo hello > file
prompt> stat file
  File: 'file'
  Size: 6 Blocks: 8 IO Block: 4096 regular file
Device: 811h/2065d Inode: 67158084 Links: 1
Access: (0640/-rw-r-----) Uid: (30686/  remzi) Gid: (30686/  remzi)
Access: 2011-05-03 15:50:20.157594748 -0500
```

Modify: 2011-05-03 15:50:20.157594748 -0500

Change: 2011-05-03 15:50:20.157594748 -0500

Файловая система обычно хранит эту информацию в структуре **inode**¹. Мы многое узнаем об индексных дескрипторах, когда будем говорить о реализации файловой системы. А пока достаточно считать, что индексный дескриптор – это находящаяся на запоминающем устройстве структура данных, в которой файловая система хранит описанную выше информацию о файле. Все индексные дескрипторы находятся на диске, а копии активных кешируются еще и в памяти для ускорения доступа.

39.10. УДАЛЕНИЕ ФАЙЛОВ

Сейчас мы знаем, как создавать файлы и обращаться к ним, последовательно или иначе. Но как удалить файл? Если вы когда-нибудь работали с Unix, то, наверное, знаете – выполнить команду `rm`. Но каким системным вызовом пользуется `rm` для удаления файла?

Воспользуемся нашим старым знакомым `strace` еще раз. Удалим этот всем изрядно поднадоевший файл «foo»:

```
prompt> strace rm foo
...
unlink("foo")          = 0
...
```

Мы опустили не относящиеся к делу строки из результата трассировки, оставив только системный вызов с загадочным именем `unlink()`. Как видите, ему передается только имя удаляемого файла, и в случае успеха вызов возвращает 0. Но остается вопрос: почему этот системный вызов называется «`unlink`», а не, скажем, «`remove`» или «`delete`»? Чтобы ответить на него, нужно сначала лучше разобраться не только в файлах, но и в каталогах.

39.11. СОЗДАНИЕ КАТАЛОГА

Существуют системные вызовы, позволяющие создавать, читать и удалять каталоги. Заметим, что прямая запись в каталог невозможна; поскольку формат каталога считается деталью реализации файловой системы, обновлять каталог можно только опосредованно, например путем создания в нем файлов, каталогов и других объектов. Так файловая система может быть уверена, что содержимое каталога всегда будет таким, как она ожидает.

Для создания каталога предназначен системный вызов `mkdir()`. Программа с таким же именем, `mkdir`, используется для создания каталога из оболочки.

¹ В некоторых файловых системах эта структура называется похоже, но по-другому, например `dnodes`; идея, впрочем, та же.

Посмотрим, что происходит, когда мы выполняем программу `mkdir` для создания каталога `foo`:

```
prompt> strace mkdir foo
...
mkdir("foo", 0777) = 0
...
prompt>
```

Сразу после создания каталог считается «пустым», хотя кое-что в нем все же есть. Именно, пустой каталог содержит две записи: одна ссылается на него самого, а вторая на его родителя. Первая называется каталогом «.» (точка), а вторая – «..» (две точки). Увидеть эти каталоги можно, запустив программу `ls` с флагом `-a`:

```
prompt> ls -a
./ ../
prompt> ls -al
total 8
drwxr-x--- 2 remzi remzi 6 Apr 30 16:17 ./
drwxr-x--- 26 remzi remzi 4096 Apr 30 16:17 ../
```

Совет: будьте осторожны с мощными командами

Программа `rm` являет пример мощной команды, а вместе с тем демонстрирует, что слишком большая мощь может обернуться бедой. Например, чтобы одним движением удалить целую группу файлов, можно набрать

```
prompt> rm *
```

где `*` сопоставляется с любым файлом в текущем каталоге. Но иногда мы хотим удалить также и каталоги вместе с их содержимым. Для этого можно попросить `rm` рекурсивно заходить в каждый каталог и удалять его содержимое:

```
prompt> rm -rf *
```

Беда происходит, если эта скромно выглядящая команда случайно выполнена, когда текущим является корневой каталог файловой системы, – в этом случае будут удалены вообще все файлы и каталоги. Ой!

Помните, что мощные команды – это обоюдоострый меч; они позволяют выполнить много работы, нажав всего несколько клавиш, но при неосторожном обращении могут причинить огромный вред.

39.12. ЧТЕНИЕ КАТАЛОГОВ

Итак, создавать каталоги мы научились, теперь неплохо бы научиться читать их. Именно для этого предназначена программа `ls`. Но давайте сами напишем простенькую программку типа `ls`, чтобы узнать, как это делается.

Вместо того чтобы открывать каталог так же, как файл, мы будем использовать другой набор системных вызовов. Приведенная ниже программа печатает содержимое каталога. В ней используется три системных вызова: `opendir()`, `readdir()` и `closedir()`, а интерфейс, как видите, очень простой: мы в цикле читаем по одной записи каталога и печатаем имя и номер индексного дескриптора, хранящиеся в этой записи:

```
int main(int argc, char *argv[]) {
    DIR *dp = opendir(".");
    assert(dp != NULL);
    struct dirent *d;
    while ((d = readdir(dp)) != NULL) {
        printf("%lu %s\n", (unsigned long) d->d_ino, d->d_name);
    }
    closedir(dp);
    return 0;
}
```

В объявлении ниже представлена информация, хранящаяся в каждой записи каталога – структуре `struct dirent`:

```
struct dirent {
    char d_name[256];           /* имя файла */
    ino_t d_ino;                /* номер индексного дескриптора */
    off_t d_off;               /* смещение следующей dirent */
    unsigned short d_reclen;    /* длина этой записи */
    unsigned char d_type;      /* тип файла */
};
```

Поскольку каталог содержит не слишком много информации (по существу, только отображение имени в индексный дескриптор и еще несколько деталей), программа, вероятно, захочет вызвать `stat()` для каждого файла, чтобы получить дополнительные сведения, например длину файла и т. п. Именно это и делает `ls`, если задан флаг `-l`; запустите `strace` для `ls` с этим флагом и без него и посмотрите, что получится.

39.13. УДАЛЕНИЕ КАТАЛОГОВ

Наконец, мы можем удалить каталог, вызвав `rmdir()` (этот вызов используется в одноименной программе `rmdir`). Но, в отличие от удаления файла, удаление каталогов – более опасная операция, потому что можно ненароком удалить одной командой целую кучу данных. Поэтому `rmdir()` предъявляет дополнительное требование: удаляемый каталог должен быть пуст (содержать только записи «.» и «..»). При попытке удалить непустой каталог `rmdir()` завершится с ошибкой.

39.14. ЖЕСТКИЕ ССЫЛКИ

Теперь вернемся к загадке – почему функция удаления файла называется `unlink()`? Для этого мы расскажем о еще одном способе создать узел в дереве файловой системы – системном вызове `link()`. Этот вызов принимает два аргумента: старое и новое путевое имя. При создании нового файла, «ссылающегося» на старый, мы по сути дела создаем другой способ обратиться к тому же файлу. Для этой цели служит командная программа `ln`:

```
prompt> echo hello > file
prompt> cat file
hello
prompt> ln file file2
prompt> cat file2
hello
```

Здесь мы создали файл, содержащий слово «hello», и назвали его `file`¹. Затем мы создали жесткую ссылку на этот файл с помощью программы `ln`. А потом можем напечатать содержимое файла, открыв его по имени `file` или `file2`.

Системный вызов `link` создает в каталоге еще одно имя, которое ссылается на *тот же* номер индексного дескриптора, что и исходный файл. Файл при этом не копируется, просто мы имеем два имени (`file` и `file2`), указывающих на один и тот же файл. В этом легко убедиться, напечатав номера индексных дескрипторов файлов:

```
prompt> ls -i file file2
67158084 file
67158084 file2
prompt>
```

Программа `ls` с флагом `-i` печатает номера индексных дескрипторов файлов (в дополнение к имени). Так что мы видим, что на самом деле представляет собой ссылка: не более чем другое имя того же индексного дескриптора (в данном случае 67158084).

Теперь вы, надо думать, понимаете, почему `unlink()` так называется. При создании файла мы в действительности делаем *две* вещи. Во-первых, создается структура (индексный дескриптор), в которой сохраняется вся существенная информация о файле, включая его размер, местонахождение его блоков на диске и т. д. Во-вторых, мы *связываем* (англ. *link*) понятное человеку внешнее имя с этим файлом и помещаем эту связь или ссылку в каталог.

После создания жесткой ссылки на файл для файловой системы безразлично, какое имя было у файла первоначально (`file`), а какое появилось позже (`file2`); оба имени – просто ссылки на один и тот же набор метаданных, которому соответствует номер индексного дескриптора 67158084.

¹ Обратите внимание на богатую фантазию авторов. Когда-то у нас была кошка по прозвищу Кошка (правда, правда). Но она умерла, и теперь у нас живет хомячок по прозвищу Хомми. Поправка: Хомми тоже умер. Кладбище домашних питомцев.

Поэтому, чтобы удалить файл из файловой системы, мы вызываем `unlink()` (букв. отменить ссылку). В примере выше мы могли бы удалить файл `file`, сохранив возможность доступа к нему по другому имени:

```
prompt> rm file
removed 'file'
prompt> cat file2
hello
```

Это работает, потому что файловая система, выполняя операцию `unlink` для файла, смотрит на **счетчик ссылок**, хранящийся в индексном дескрипторе. Этот счетчик информирует систему о том, сколько имен связано с этим дескриптором. При вызове `unlink()` удаляется связь между внешним именем (указанным при удалении) и данным индексным дескриптором, а счетчик ссылок уменьшается на 1. И лишь когда счетчик ссылок опустится до нуля, файловая система удаляет индексный дескриптор и все связанные с ним блоки данных, так что файл, наконец, «удаляется» по-настоящему.

Разумеется, `stat()` умеет показывать счетчик ссылок. Посмотрим, как он изменяется при создании и удалении жестких ссылок на файл. В примере ниже мы создали три ссылки на один файл, а затем удалили их. Следите за счетчиком ссылок!

```
prompt> echo hello > file
prompt> stat file
... Inode: 67158084      Links: 1 ...
prompt> ln file file2
prompt> stat file
... Inode: 67158084      Links: 2 ...
prompt> stat file2
... Inode: 67158084      Links: 2 ...
prompt> ln file2 file3
prompt> stat file
... Inode: 67158084      Links: 3 ...
prompt> rm file
prompt> stat file2
... Inode: 67158084      Links: 2 ...
prompt> rm file2
prompt> stat file3
... Inode: 67158084      Links: 1 ...
prompt> rm file3
```

39.15. СИМВОЛИЧЕСКИЕ ССЫЛКИ

Есть еще один полезный тип ссылок – **символические**, или **мягкие**. Дело в том, что у жестких ссылок есть ограничения: нельзя создать жесткую ссылку на каталог (потому что есть опасность появления циклов в дереве каталогов), нельзя создать ссылку на файл в другом разделе диска (потому что номера индексных дескрипторов уникальны только в пределах одной файловой си-

стемы, а в разных могут повторяться) и т. д. Поэтому и были предложены символические ссылки, лишенные этих недостатков.

Для создания такой ссылки применяется та же программа, но с флагом `-s`, например:

```
prompt> echo hello > file
prompt> ln -s file file2
prompt> cat file2
hello
```

Как видим, символическая ссылка выглядит почти так же, как жесткая, а к исходному файлу можно обращаться как по имени `file`, так и по имени символической ссылки `file2`.

Однако на проверку символические ссылки сильно отличаются от жестких. Первое отличие в том, что символическая ссылка – это настоящий файл, хотя и другого типа. До сих пор мы рассматривали регулярные файлы и каталоги; символические ссылки – это третий тип файла, известный файловой системе. Всю правду о ней раскрывает программа `stat`:

```
prompt> stat file
... regular file ...
prompt> stat file2
... symbolic link ...
```

Программа `ls` тоже сообщает об этом факте. Обратите внимание на первый символ в длинном формате выдачи `ls` – вы увидите, что для регулярных файлов печатается `-`, для каталогов `d`, а для символических ссылок `l`. Также для символических ссылок показан размер (в данном случае 4 байта) и файл, на который ссылка указывает (`file`).

```
prompt> ls -al
drwxr-x--- 2 remzi remzi 29 May 3 19:10 ./
drwxr-x--- 27 remzi remzi 4096 May 3 15:14 ../
-rw-r----- 1 remzi remzi 6 May 3 19:10 file
lrwxrwxrwx 1 remzi remzi 4 May 3 19:10 file2 -> file
```

Размер `file2` равен 4 байтам, потому что в символической ссылке хранится путевое имя файла, на который она указывает. В данном случае ссылка `file2` указывает на файл `file`, поэтому ее размер равен 4 байтам. Если бы она указывала на файл с более длинным именем, то и размер ссылки был бы больше:

```
prompt> echo hello > alongerfilename
prompt> ln -s alongerfilename file3
prompt> ls -al alongerfilename file3
-rw-r----- 1 remzi remzi 6 May 3 19:17 alongerfilename
lrwxrwxrwx 1 remzi remzi 15 May 3 19:17 file3 -> alongerfilename
```

Наконец, из-за самого способа создания символических ссылок возможно появление **висячих ссылок**:

```
prompt> echo hello > file
prompt> ln -s file file2
```

```
prompt> cat file2
hello
prompt> rm file
prompt> cat file2
cat: file2: No such file or directory
```

Как видим, в отличие от жестких ссылок, удаление исходного файла `file` приводит к тому, что ссылка указывает на несуществующее путевое имя.

39.16. Биты полномочий и списки КОНТРОЛЯ ДОСТУПА

Абстракция процесса привела нас к двум основным видам виртуализации: процессора и памяти. Они создают иллюзию, будто у процесса имеется свой *частный* процессор и своя *частная* память; в реальности ОС применяет различные методы, чтобы безопасно разделить ограниченные физические ресурсы между конкурирующими претендентами.

Файловая система также предлагает виртуальное представление диска, преобразуя совокупность физических блоков в более удобные для работы файлы и каталоги, как описано в этой главе. Но эта абстракция принципиально отличается от абстракций процессора и памяти, потому что файлы *разделяются* между всеми пользователями и процессами, а не являются частной собственностью (по крайней мере, не всегда). Поэтому внутри файловой системы обычно присутствует более полный набор механизмов, определяющих различные степени такого разделения.

Первый из таких механизмов – **биты полномочий**. Чтобы увидеть их для файла `foo.txt`, достаточно выполнить команду

```
prompt> ls -l foo.txt
-rw-r--r-- 1 remzi wheel 0 Aug 24 16:29 foo.txt
```

Обратите внимание на первую часть напечатанной строки: `-rw-r--r--`. Первый символ обозначает тип файла: `-` для регулярного файла (каковым является `foo.txt`), `d` для каталога, `l` для символической ссылки и т. д. Он (как правило) не связан с полномочиями, поэтому пока не будем обращать на него внимания.

Интересующие нас биты полномочий представлены следующими девятью символами (`rw-r--r--`). Они определяют, кто и как может обращаться к файлу, каталогу и другим объектам.

Полномочия разделены на три группы: что разрешено **владельцу** файла, что разрешено членам группы и, наконец, что разрешено всем (иногда всех называют **прочими**). Владелец, **группа** и прочие могут читать, записывать и исполнять файл – это и есть полномочия.

В примере выше первые три символа означают, что владелец может читать и записывать файл (`rw-`), а члены группы `wheel` и все прочие могут только читать (повторенное два раза `r--`).

Владелец файла может изменить эти полномочия, например, с помощью команды `chmod` (*change file mode* – изменить **режим файла**). Чтобы отобразить возможность доступа к файлу у всех, кроме владельца, нужно ввести команду

```
prompt> chmod 600 foo.txt
```

Эта команда устанавливает бит чтения (4) и бит записи (2) для владельца (объединяя их с помощью связки OR, что и дает 6), но сбрасывает все биты полномочий для группы и прочих (биты 0 и 0), так что строка полномочий имеет вид `gw-----`.

Особенно интересен бит выполнения. Для регулярных файлов он показывает, можно ли выполнять файл как программу. Например, чтобы выполнить простой скрипт оболочки `hello.csh`, нужно набрать:

```
prompt> ./hello.csh
hello, from shell world.
```

Но если для этого файла не установлен бит выполнения, то мы увидим такой результат:

```
prompt> chmod 600 hello.csh
prompt> ./hello.csh
./hello.csh: Permission denied.
```

ОТСТУПЛЕНИЕ: РОЛЬ СУПЕРПОЛЬЗОВАТЕЛЯ В ФАЙЛОВОЙ СИСТЕМЕ

Какому пользователю разрешено выполнять привилегированные операции с целью администрирования файловой системы? Например, кто имеет право удалить файлы неактивного пользователя, чтобы освободить место на диске?

В локальной файловой системе обычно предполагается наличие какого-то **суперпользователя** (с именем **root**), которому разрешен доступ ко всем файлам независимо от прописанных в них полномочий. В распределенной файловой системе, например AFS (где имеются **списки контроля доступа**), все облеченные доверием пользователи включены в группу `system:administrators`. В обоих случаях существование доверенных пользователей сопряжено с риском; если противник каким-то образом сможет выдать себя за такого пользователя, то он получит доступ ко всей информации в системе, обратив в прах все ожидаемые гарантии защиты и конфиденциальности.

Для каталогов бит выполнения интерпретируется иначе. Именно, он позволяет пользователю (или группе, или всем) заходить в данный каталог (командой `cd`), а в сочетании с битом записи – создавать в нем файлы. Лучший способ разобраться в этих битах – поэкспериментировать! Не пугайтесь, маловероятно, что вы что-то серьезно испортите.

Помимо битов полномочий, в некоторых файловых системах, включая распределенную файловую систему AFS (обсуждаемую в следующей главе), имеются более развитые средства контроля. Конкретно в AFS это **список контроля доступа** (access control list – **ACL**) для каждого каталога. Это более общий способ описать, кто имеет право обращаться к данному ресурсу. В файловой системе данный механизм позволяет создать точный список

всех, кто может или не может читать набор файлов, что открывает более широкие возможности, чем описанная выше ограниченная модель битов полномочий для владельца, группы и прочих.

Например, ниже показан список контроля доступа для каталога `private` в системе AFS, принадлежащего одному из авторов; для его распечатки служит команда `fs listacl`:

```
prompt> fs listacl private
Access list for private is
Normal rights:
  system:administrators rlidwka
  gemzi rlidwka
```

Это означает, что системные администраторы и пользователь `gemzi` могут просматривать, добавлять, удалять и администрировать файлы в этом каталоге, а также читать, записывать и блокировать эти файлы.

Чтобы разрешить кому-то (в данном случае – другому автору) доступ к этому каталогу, пользователь `gemzi` мог бы выполнить следующую команду:

```
prompt> fs setacl private/ andrea rl
```

Прощай, конфиденциальность `gemzi`! Но зато мы выучили более важный урок: в удачном браке не может быть секретов, даже внутри файловой системы¹.

СОВЕТ: ОСТЕРЕГАЙТЕСЬ ТОСТТОУ

В 1974 году Макфи (McPhee) обратил внимание на одну проблему в компьютерных системах. Именно, он заметил, что «...если проходит некоторое время между проверкой допустимости и действием, основанным на результате этой проверки, то в многозадачном режиме в течение этого времени можно намеренно изменить проверенные условия и тем самым открыть возможность для выполнения программой недопустимой операции». В настоящее время мы называем эту проблему «от времени проверки до времени использования» (Time Of Check To Time Of Use – **ТОСТТОУ**), и, как это ни печально, она все еще существует.

Простой пример, описанный в работе Бишоп и Дилджера [BD96], показывает, как пользователь может обмануть обремененную доверием службу и навлечь беды. Представим, к примеру, что почтовая служба работает от имени `root` (и, значит, полномочна обращаться к любому файлу в системе). Служба добавляет входящее сообщение в почтовый ящик пользователя следующим образом. Сначала она вызывает `lstat()`, чтобы получить информацию о файле, проверяя, что это действительно регулярный файл, принадлежащий адресату сообщения, а не ссылка на другой файл, который почтовый сервер не должен обновлять. Затем, если проверка прошла успешно, сервер записывает в файл новое сообщение.

К сожалению, наличие промежутка времени между проверкой и обновлением создает проблему: противник (в данном случае получатель сообщения, который имеет полномо-

¹ Если кому интересно, мы состоим в счастливом браке с 1996 года. Понимаем, вам не интересно.

чия для доступа к почтовому ящику) подменяет файл ящика (вызывая `rename()`), так чтобы он указывал на файл, который должен быть ему недоступен, например `/etc/passwd` (в нем хранится информация о пользователях и их паролях). Если совершить такую подмену в нужный момент времени (между проверкой и обновлением), то сервер, пребывая в блаженном неведении, запишет в секретный файл содержимое почтового сообщения. Таким образом, противник сможет добавить в `/etc/passwd` свою учетную запись с полномочиями `root` и, следовательно, получить полный контроль над системой.

У проблемы TOCTTOU нет простого и полного решения [Т+08]. Один из подходов – уменьшить число служб, которым необходимы полномочия `root`, это отчасти помогает. Флаг `O_NOFOLLOW` при вызове `open()` запрещает следовать по символической ссылке, т. е. если системному вызову передана символическая ссылка, то файл, на которую она указывает, не будет открыт. Это позволяет противостоять атакам с открытием таких ссылок. Более радикальные подходы, например **транзакционные файловые системы** [Н+18], могли бы решить данную проблему, но подобных систем развернуто мало. Поэтому дадим обычный («отстойный») совет: будьте осторожны при написании кода, работающего с высоким уровнем полномочий!

39.17. СОЗДАНИЕ И МОНТИРОВАНИЕ ФАЙЛОВОЙ СИСТЕМЫ

Мы кратко рассмотрели основные интерфейсы для работы с файлами, каталогами и некоторыми специальными типами ссылок. Но нужно обсудить еще одну тему: как собрать полное дерево каталогов, содержащее несколько файловых систем. Для этого нужно сначала создать файловые системы, а затем смонтировать, сделав тем самым доступным их содержимое.

Для создания файловой системы обычно предоставляется программа `mkfs` (произносится «make fs»). Ей передается устройство (например, раздел диска `/dev/sda1`) и тип файловой системы (например, `ext3`), а она создает пустую файловую систему с корневым каталогом в указанном разделе диска. И сказала `mkfs`: «Да будет файловая система!»

Однако после того как файловая система создана, ее нужно сделать доступной из единого дерева каталогов. Эту задачу решает программа `mount` (она обращается к системному вызову `mount()`, который и делает всю работу). Программа берет существующий каталог, называемый **точкой монтирования**, и по существу вставляет указанную файловую систему в дерево каталогов в этой точке.

Пример поможет прояснить дело. Пусть имеется несмонтированная файловая система типа `ext3`, находящаяся в разделе `/dev/sda1`, которая содержит корневой каталог с двумя подкаталогами `a` и `b`, каждый из которых содержит только файл `foo`. Мы хотим смонтировать эту файловую систему на каталог `/home/users`. Это делается так:

```
prompt> mount -t ext3 /dev/sda1 /home/users
```

Если все получилось, то новая файловая система становится доступной. Но обратите внимание, как производится доступ к ней. Чтобы просмотреть содержимое корневого каталога, нужно выполнить `ls` следующим образом:

```
prompt> ls /home/users/
a b
```

Как видим, путь `/home/users/` теперь относится к корню только что смонтированного каталога. Аналогично мы могли бы получить доступ к каталогам `a` и `b`, указав пути `/home/users/a` и `/home/users/b`. Наконец, к файлам `foo` можно было бы обратиться по именам `/home/users/a/foo` и `/home/users/b/foo`. В этом и состоит изящество монтирования: вместо того чтобы работать с рядом отдельных файловых систем, смонтировать их все в одном дереве, сделав именование удобным и единообразным.

Чтобы узнать, какие файловые системы смонтированы в вашей системе и на какие точки, запустите программу `mount` без параметров. Будет напечатано что-то вроде:

```
/dev/sda1 on / type ext3 (rw)
proc on /proc type proc (rw)
sysfs on /sys type sysfs (rw)
/dev/sda5 on /tmp type ext3 (rw)
/dev/sda7 on /var/vice/cache type ext3 (rw)
tmpfs on /dev/shm type tmpfs (rw)
AFS on /afs type afs (rw)
```

Эта сумасшедшая смесь показывает, что целый ряд разных файловых систем с такими типами, как `ext3` (стандартная файловая система на диске), `proc` (файловая система для получения информации о текущих процессах), `tmpfs` (файловая система для временных файлов) и `afs` (распределенная файловая система), объединены в единое дерево.

ОТСТУПЛЕНИЕ: ТЕРМИНОЛОГИЯ ФАЙЛОВЫХ СИСТЕМ

- **Файл** – массив байтов, который можно создать, прочитать, записать и удалить. Имеет низкоуровневое имя (число), однозначно идентифицирующее его. Низкоуровневое имя часто называют **номером индексного дескриптора**, или **i-номером**.
- **Каталог** – совокупность кортежей, каждый из которых содержит понятное человеку внешнее имя и соответствующее ему низкоуровневое имя. Каждый элемент ссылается либо на другой каталог, либо на файл. У каждого каталога также имеется низкоуровневое имя (i-номер). В любом каталоге всегда есть два специальных элемента: `.`, ссылающийся на сам каталог, и `..`, ссылающийся на родительский каталог.
- **Дерево каталогов**, или **иерархия каталогов** – способ организации всех файлов и каталогов в одно большое дерево, начинающееся с **корня**.
- Для доступа к файлу процесс должен использовать системный вызов (обычно `open()`), чтобы запросить разрешение у операционной системы. Если разрешение дано, то ОС возвращает **дескриптор файла**, который затем можно использовать для чтения или записи в соответствии с тем, на что испрашивалось разрешение.

- Каждый дескриптор файла представляет собой уникальное в рамках процесса число – индекс записи в **таблице открытых файлов**. В этой записи среди прочего хранится информация о том, к какому файлу она относится, о **текущем смещении** (откуда начнется следующая операция чтения или записи).
- Обращения к `read()` и `write()` естественным образом изменяют текущее смещение. Дополнительно процесс может изменить его с помощью системного вызова `lseek()`, что позволяет выполнять произвольный доступ к любому участку файла.
- Для форсированной записи обновлений на запоминающее устройство процесс должен воспользоваться вызовом `fsync()` или родственными ему. Но сделать это правильно и притом сохранить высокую производительность – непростая задача [P+14], поэтому дважды подумайте, перед тем как решиться.
- Если вы хотите, чтобы несколько внешних имен ссылались на один и тот же файл, пользуйтесь **жесткими** или **символическими ссылками**. У каждого типа есть своя сфера применения, так что учитывайте их сильные и слабые стороны. И помните, что удаление файла производится после удаления последней жесткой ссылки на него вызовом `unlink()`.
- В большинстве файловых систем имеются механизмы разрешения и запрета совместного использования. Рудиментарной формой такого контроля являются **биты полномочий**; более развитые списки контроля доступа обеспечивают более точный контроль над тем, кто и как может манипулировать информацией.

39.18. РЕЗЮМЕ

Интерфейс файловой системы в Unix (да и вообще в любой ОС) на первый взгляд кажется примитивным, но, чтобы овладеть им в полной мере, нужно во многом разобраться. Конечно, нет ничего лучше практики (обширной). Не пренебрегайте ей! И читайте, больше читайте; как всегда, начинать следует с книги Стивенса [SR05].

Литература

[BD96] «Checking for Race Conditions in File Accesses» by Matt Bishop, Michael Dilger. Computing Systems 9:2, 1996. *Прекрасное описание проблемы TOCTTOU и ее примеров в файловых системах.*

[CK+08] «The xv6 Operating System» by Russ Cox, Frans Kaashoek, Robert Morris, Nickolai Zeldovich. Доступно по адресу <https://github.com/mit-pdos/xv6-public>. *Простая и элегантная реализация Unix. Мы пользовались довольно старой версией (2012-01-30-1-g1c41342), поэтому некоторые примеры в книге могут не соответствовать исходному коду на сайте.*

[H+18] «TxFS: Leveraging File-System Crash Consistency to Provide ACID Transactions» by Y. Hu, Z. Zhu, I. Neal, Y. Kwon, T. Cheng, V. Chidambaram, E. Witchel. USENIX ATC '18, June 2018. *Лучшая работа из представленных на конференции USENIX ATC '18 и хорошая отправная точка для изучения транзакционных файловых систем.*

[K84] «Processes as Files» by Tom J. Killian. USENIX, June 1984. *В этой работе впервые была описана псевдофайловая система /proc, в которой каждый процесс рассматривается как файл. Изобретательная идея, которая до сих пор используется в современных системах Unix.*

[L84] «Capability-Based Computer Systems» by Henry M. Levy. Digital Press, 1984. Доступно по адресу <http://homes.cs.washington.edu/~levy/capabook>. *Отличный обзор ранних мандатных систем защиты.*

[P+13] «Towards Efficient, Portable Application-Level Consistency» by Thanumalayan S. Pillai, Vijay Chidambaram, Joo-Young Hwang, Andrea C. Arpaci-Dusseau, and Remzi H. Arpaci-Dusseau. HotDep '13, November 2013. *Наша собственная работа, в которой показаны ошибки записи данных на диск, встречающиеся в реальных приложениях. В частности, приложение может делать некоторые допущения о файловой системе, в результате правильно работает только с конкретной файловой системой.*

[P+14] «All File Systems Are Not Created Equal: On the Complexity of Crafting Crash-Consistent Applications» by Thanumalayan S. Pillai, Vijay Chidambaram, Ramnathan Alagappan, Samer Al-Kiswani, Andrea C. Arpaci-Dusseau, and Remzi H. Arpaci-Dusseau. OSDI '14, Broomfield, Colorado, October 2014. *Доклад на ту же тему, представленный на конференции, – гораздо более детальный и содержащий больше интересных фактов, чем предыдущая работа.*

[SK09] «Principles of Computer System Design» by Jerome H. Saltzer and M. Frans Kaashoek. Morgan-Kaufmann, 2009. *Эта превосходная книга – обязательное чтение для всех, интересующихся принципами проектирования вычислительных систем. Так преподают эту дисциплину в MIT. Прочитайте один раз, а потом еще несколько, чтобы как следует усвоить материал.*

[SR05] «Advanced Programming in the UNIX Environment» by W. Richard Stevens and Stephen A. Rago. Addison-Wesley, 2005. *Мы, наверное, ссылались на эту книгу сотни тысяч раз. Вот так она полезна всем, кто собирается стать хорошим системным программистом.*

[T+08] «Portably Solving File TOCTTOU Races with Hardness Amplification» by D. Tsafir, T. Hertz, D. Wagner, D. Da Silva. FAST '08, San Jose, California, 2008. *Это не первая работа, посвященная TOCTTOU, но сравнительно свежее и качественное описание проблемы и переносимого способа ее решения.*

Домашнее задание (код)

В этом домашнем задании вы ближе познакомитесь с работой описанных в данной главе API. Для этого нужно будет написать несколько программ на основе различных утилит Unix.

Вопросы

1. **Stat.** Напишите свою версию командной утилиты `stat`, которая должна просто обращаться к системному вызову `stat()` для указанного файла или каталога. Напечатайте размер файла, количество выделенных ему блоков, число ссылок и т. д. Посмотрите, что происходит с числом ссылок на каталог, когда изменяется количество элементов в нем. Полезные интерфейсы: `stat()`.
2. **Список файлов.** Напишите программу, которая выводит список файлов в указанном каталоге. При вызове без аргументов программа должна напечатать только имена файлов. При вызове с флагом `-l` программа должна печатать подробную информацию о каждом файле: владельца, группу, разрешения и т. д., получая ее от системного вызова `stat()`. Программа может также принимать один дополнительный аргумент: имя подлежащего чтению каталога, например `mys -l directory`. Если каталог не задан, используется текущий рабочий каталог. Полезные интерфейсы: `stat()`, `opendir()`, `readdir()`, `getcwd()`.
3. **Tail.** Напишите программу, которая печатает несколько последних строк файла. Программа должна быть эффективной: переходит в место, близкое к концу файла, читает блок данных с этого места до конца файла, а затем идет от конца блока к началу, отсчитывая запрошенное количество строк, после чего печатает все строки с этого места до конца файла. Программа вызывается так: `mytail -n file`, где `n` – количество печатаемых строк файла `file`. Полезные интерфейсы: `stat()`, `lseek()`, `open()`, `read()`, `close()`.
4. **Рекурсивный поиск.** Напишите программу, которая печатает имена всех файлов и каталогов в дереве файловой системы, начиная с указанного места в нем. Например, при запуске без аргументов программа должна напечатать содержимое текущего рабочего каталога и рекурсивно всех его подкаталогов. Если задан один аргумент (имя каталога), то обход должен начинаться с него. Обогадите рекурсивный поиск несколькими дополнительными аргументами по аналогии с мощной командной утилитой `find`. Полезные интерфейсы: разберитесь самостоятельно.

Глава 40

Реализация файловой системы

В этой главе мы познакомимся с реализацией простой файловой системы под названием **vsfs (Very Simple File System)** – очень простая файловая система). Это упрощенная версия типичной файловой системы Unix, призванная продемонстрировать некоторые базовые структуры данных на диске, методы доступа и различные политики, встречающиеся во многих современных файловых системах.

Файловая система – это чисто программная конструкция; в отличие от подходов к виртуализации процессора и памяти, мы не станем добавлять никаких аппаратных средств для улучшения работы файловой системы (хотя будем учитывать характеристики устройства, чтобы обеспечить ее эффективную работу). Вследствие большой гибкости, доступной проектировщику, было разработано много файловых систем: от первой буквы алфавита – AFS (Andrew File System) [H+88] – до последней – ZFS (Zettabyte File System компании Sun) [B07]. В них используются разные структуры данных, и каждая имеет свои достоинства и недостатки. Так что изучение файловых систем будет построено на примерах: в данной главе мы рассмотрим простую файловую систему (vsfs), чтобы познакомиться с основными концепциями, а в последующих обратимся к реальным файловым системам, чтобы понять, чем они различаются.

СУЩЕСТВО ПРОБЛЕМЫ: КАК РЕАЛИЗОВАТЬ ПРОСТУЮ ФАЙЛОВУЮ СИСТЕМУ

Как построить простую файловую систему? Какие необходимы структуры данных на диске? Что в них нужно запоминать? Как к ним обращаться?

40.1. Ход мыслей

Обычно мы предлагаем размышлять о двух аспектах файловой системы; усвоив оба, вы будете понимать принципы работы файловой системы.

Первый – **структуры данных** файловой системы. То есть какие структуры на диске нужны файловой системе для организации данных и метаданных?

В первых системах, которые мы рассмотрим (в т. ч. vsfs), используются простые структуры, например массивы блоков или других объектов, тогда как в более развитых системах типа XFS компании SGI применяются сложные древовидные структуры [S+96].

ОТСТУПЛЕНИЕ: МЫСЛЕННЫЕ МОДЕЛИ ФАЙЛОВЫХ СИСТЕМ

Как уже отмечалось выше, именно мысленную модель мы пытаемся построить при изучении систем. В случае файловых систем мысленная модель должна включать следующие вопросы: в каких структурах на диске хранятся данные и метаданные? что происходит при открытии файла? к каким структурам на диске производятся обращения при чтении или записи файла? Развивая и уточняя мысленную модель, мы приходим к абстрактному пониманию происходящего, вместо того чтобы пытаться разобраться в хитросплетениях кода конкретной файловой системы (хотя это, конечно, тоже полезно!).

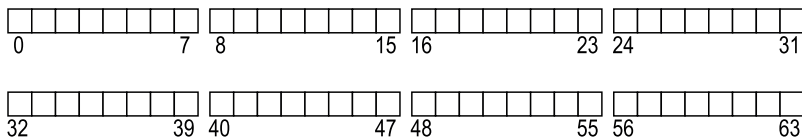
Второй аспект файловой системы – ее **методы доступа**. Как она отображает вызовы со стороны процесса, например `open()`, `read()`, `write()` и т. д., на свои структуры? Какие структуры читаются при выполнении конкретного системного вызова? Как в них производится запись? Насколько эффективны эти шаги?

Если вы разобрались в структурах данных и методах доступа файловой системы, значит, составили хорошую мысленную модель ее работы, а это и есть то главное, что необходимо для системного образа мышления. Трудитесь над построением мысленной модели по мере того, как мы будем углубляться в детали нашей первой реализации.

40.2. ОБЩАЯ ОРГАНИЗАЦИЯ

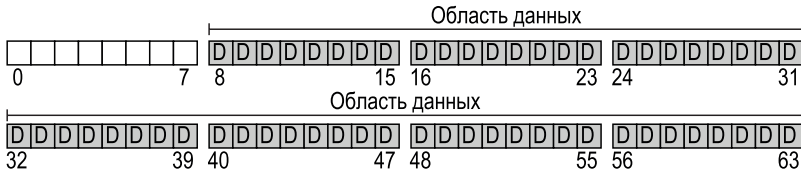
Мы приступаем к разработке общей организации дисковых структур данных файловой системы vsfs. Первым делом нужно разделить диск на **блоки**; в простых файловых системах используются диски одного размера, и мы поступим точно так же. Выберем типичный размер 4 КБ.

Таким образом, наше представление раздела диска, в котором создается файловая система, очень простое: линейная последовательность блоков размером 4 КБ. Блоки нумеруются с 0 до $N - 1$, так что раздел содержит N 4-килобайтовых блоков. Предположим, что диск совсем маленький и насчитывает всего 64 блока.



Теперь подумаем, что должно храниться в этих блоках, чтобы можно было построить файловую систему. Разумеется, первое, что приходит на

ум, – пользовательские данные. Действительно, большая часть места в любой файловой системе занята данными пользователей. Назовем ту часть диска, которая отведена под пользовательские данные, **областью данных** и, опять же для простоты, зарезервируем под нее фиксированную долю диска, скажем последние 56 из 64 блоков.



Как было сказано (вскользь) в предыдущей главе, файловая система должна хранить информацию о каждом файле. Это ключевая часть **метаданных**, сюда входит информация о том, какие блоки (в области данных) принадлежат файлу, о размере файла, о его владельце и правах доступа, о времени последнего доступа и модификации и т. п. Для хранения всего этого обычно используется структура, называемая **индексным дескриптором** (мы поговорим о них позже).

Для размещения индексных дескрипторов нужно зарезервировать для них место на диске. Назовем эту область диска **таблицей индексных дескрипторов** и будем считать, что она является просто массивом. Таким образом, теперь наше представление диска выглядит, как показано на рисунке ниже, в предположении, что 5 из имеющихся 64 блоков отведено под индексные дескрипторы (обозначенные буквой I):



Отметим, что обычно размер индексных дескрипторов невелик, например 128 или 256 байт. В предположении, что один дескриптор занимает 256 байт, в блоке размером 4 КБ помещается 16 дескрипторов, а всего в нашей файловой системе может быть 80 дескрипторов. В нашей простой файловой системе, построенной в крохотном разделе, содержащем 64 блока, это и есть максимальное число файлов. Но в точно такой же файловой системе, построенной на большом диске, можно было бы отвести под таблицу индексных дескрипторов гораздо больше места, увеличив тем самым максимально допустимое число файлов.

Итак, в нашей файловой системе имеются блоки данных (D) и индексные дескрипторы (I), но кое-чего еще не хватает. И прежде всего, как вы, навер-

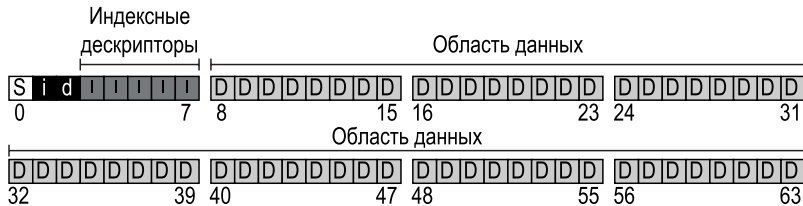
ное, догадались, не хватает способа определения, какие индексные дескрипторы или блоки данных свободны, а какие заняты. Подобные **структуры учета выделения** – обязательный элемент любой файловой системы.

Разумеется, можно предложить много методов учета выделения. Например, можно было бы использовать **список свободных**, т. е. хранить указатель на первый свободный блок, который указывает на следующий свободный, и т. д. Но мы вместо этого выберем простую и популярную структуру **битовой карты**, по одной для области данных (**битовая карта данных**) и для таблицы индексных дескрипторов (**битовая карта индексных дескрипторов**). В битовой карте каждый бит показывает, свободен соответствующий объект/блок (0) или занят (1). Ниже показана новая схема диска, включающая битовую карту индексных дескрипторов (i) и битовую карту данных (d):



Вообще-то, расточительно использовать целый блок размером 4 КБ под битовые карты; такая карта позволяет следить за выделением 32К объектов, тогда как у нас всего 80 индексных дескрипторов и 56 блоков данных. Но для простоты мы отведем под каждую битовую карту целый блок.

Внимательный читатель (т. е. тот, который еще не спит), вероятно, заметил, что в нашей структуре файловой системы остался еще один блок слева. Он зарезервирован для **суперблока**, обозначенного буквой S на рисунке ниже. В суперблоке хранится информация о данной файловой системе, в частности количество индексных дескрипторов и блоков данных (в нашем примере – 80 и 56), адрес начала таблицы индексных дескрипторов (блок 3) и т. д. Вероятно, здесь же присутствует какая-то сигнатура, определяющая тип файловой системы (в данном случае vsfs).



В момент монтирования файловой системы операционная система первым читает суперблок, инициализирует различные параметры, после чего присоединяет том к дереву каталогов. При доступе к файлам на этом томе ОС будет точно знать, где на диске искать необходимые структуры.

40.3. ОРГАНИЗАЦИЯ ФАЙЛА: ИНДЕКСНЫЙ ДЕСКРИПТОР

Одна из самых важных дисковых структур файловой системы – **индексный дескриптор**; практически во всех файловых системах есть подобная структура. Исторически название **индексный дескриптор** (англ. *index node*, сокращенно *inode*) появилось в Unix, а быть может, и в более ранних системах, потому что первоначально дескрипторы были организованы в виде массива, доступ к которому производился по *индексу*.

ОТСТУПЛЕНИЕ: СТРУКТУРА ДАННЫХ – ИНДЕКСНЫЙ ДЕСКРИПТОР

Индексный дескриптор – общий термин, используемый во многих файловых системах для описания структуры, в которой хранятся метаданные файла, в т. ч. длина, права доступа и адреса принадлежащих ему блоков. Название восходит по меньшей мере к временам создания Unix (а может быть, Multics или даже еще раньше) и связано с тем, что номер индексного дескриптора является индексом массива дескрипторов на диске. Как мы увидим, устройство индексного дескриптора – один из ключевых аспектов проектирования файловой системы. В большинстве современных ОС есть подобная структура для описания файлов, но иногда она называется иначе (*dnode*, *fnode* и т. д.).

С каждым индексным дескриптором неявно связано число (**номер индексного дескриптора**, или **i-номер**), которое мы ранее называли **низкоуровневым именем** файла. В *vsfs* (и других простых файловых системах), зная *i*-номер, можно вычислить, где на диске расположен соответствующий индексный дескриптор. Например, возьмем описанную выше таблицу индексных дескрипторов в *vsfs*: ее размер 20 КБ (5 блоков по 4 КБ), т. е. она содержит 80 дескрипторов (в предположении, что каждый дескриптор занимает 256 байт); предположим далее, что область индексных дескрипторов начинается по адресу 12 КБ (т. е. суперблок начинается с 0 КБ, битовая карта индексных дескрипторов – с 4 КБ, битовая карта данных – с 8 КБ, а сразу после нее идет таблица индексных дескрипторов). В *vsfs*, следовательно, раздел файловой системы устроен следующим образом (крупным планом):

Таблица индексных дескрипторов
(крупным планом)

			i-блок 0				i-блок 1				i-блок 2				i-блок 3				i-блок 4			
Супер- блок	Битовая карта индексных дескрипторов	Битовая карта данных	0	1	2	3	16	17	18	19	32	33	34	35	48	49	50	51	64	65	66	67
			4	5	6	7	20	21	22	23	36	37	38	39	52	53	54	55	68	69	70	71
			8	9	10	11	24	25	26	27	40	41	42	43	56	57	58	59	72	73	74	75
			12	13	14	15	28	29	30	31	44	45	46	47	60	61	62	63	76	77	78	79
0 КБ	4 КБ	8 КБ	12 КБ	16 КБ	20 КБ	24 КБ	28 КБ	32 КБ														

Чтобы прочитать индексный дескриптор номер 32, файловая система должна сначала вычислить смещение относительно начала области индекс-

ных дескрипторов ($32 \cdot \text{sizeof}(\text{inode}) = 8192$), прибавить к нему адрес начала таблицы индексных дескрипторов на диске ($\text{inodeStartAddr} = 12 \text{ KB}$) и получить в результате адрес нужного блока индексных дескрипторов: 20 KB. Напомним, что диск не адресуется побайтово, а состоит из большого числа адресуемых секторов, обычно размером 512 байт. Поэтому, чтобы прочитать блок индексных дескрипторов, содержащий дескриптор 32, файловая система должна прочитать сектор под номером $(20 \times 1024) / 512 = 40$. Вообще, адрес iaddr сектора, содержащего блок индексных дескрипторов, вычисляется следующим образом:

```
blk = (inumber * sizeof(inode_t)) / blockSize;
sector = ((blk * blockSize) + inodeStartAddr) / sectorSize;
```

В индексном дескрипторе хранится вся информация, которую необходимо знать о файле: его *тип* (например, регулярный файл, каталог и т. д.), *размер*, количество принадлежащих ему *блоков*, *права доступа* (кто владеет файлом и кто имеет к нему доступ), временные метки (когда файл был создан, когда модифицирован, когда к нему в последний раз обращались) и адреса *блоков данных* на диске (своего рода указатели). Вся эта информация о файле называется **метаданными**; собственно, так называется любая хранящаяся в файловой системе информация, кроме самих данных. Пример индексного дескриптора в файловой системе ext2 [P09] приведен на рис. 40.1¹.

Размер	Имя	Назначение
2	mode	Разрешено ли читать, записывать, исполнять этот файл
2	uid	Кто владеет файлом?
4	size	Сколько байтов в этом файле?
4	time	Время последнего доступа к файлу
4	ctime	Время создания файла
4	mtime	Время последней модификации файла
4	dtime	Время удаления этого дескриптора
2	gid	Какой группе принадлежит файл?
2	links_count	Количество жестких ссылок на файл
4	blocks	Сколько блоков выделено этому файлу?
4	flags	Как ext2 должна использовать файл?
4	osd1	Поле, зависящее от ОС
60	block	Множество дисковых указателей (всего 15)
4	generation	Версия файла (используется NFS)
4	file_acl	Новая модель полномочий, выходящая за рамки mode,
4	dir_acl	называемая «список контроля доступа»

Рис. 40.1 ❖ Упрощенный индексный дескриптор в ext2

Одно из самых важных решений при проектировании индексного дескриптора – как хранить информацию о выделенных файлу блоках. Мы применим простой подход – иметь внутри дескриптора один или более **прямых указа-**

¹ Информация о типе хранится в элементе каталога, поэтому в самом индексном дескрипторе ее нет.

телей (дисковых адресов), каждый из которых указывает на один блок, принадлежащий файлу. У этого подхода есть ограничение: если файл большой (больше, чем размер блока, умноженный на количество прямых указателей в дескрипторе), то нам не повезло – сохранить его не удастся.

Многоуровневый индекс

Для поддержки больших файлов проектировщики файловых систем были вынуждены включить в индексный дескриптор другие структуры. Популярна идея специального **косвенного указателя**. Он указывает не на блок, содержащий пользовательские данные, а на блок, содержащий другие указатели, каждый из которых уже указывает на пользовательские данные. Поэтому индексный дескриптор может содержать фиксированное число прямых указателей (скажем, 12) и один косвенный. Если файл слишком велик, то выделяется косвенный блок (из области данных на диске), на который будет указывать косвенный указатель в дескрипторе. В предположении, что размер блока 4 КБ, а адреса на диске занимают 4 байта, мы дополнительно получаем 1024 указателя, и максимальный размер файла увеличивается до $(12 + 1024) \cdot 4 \text{ КБ} = 4144 \text{ КБ}$.

СОВЕТ: ПОДУМАЙТЕ О РЕШЕНИЯХ НА ОСНОВЕ ЭКСТЕНТОВ

Другой подход – использовать не указатели, а **экстенты**. Экстент – это просто дисковый указатель плюс длина (в блоках). Тогда вместо того чтобы хранить по одному указателю на каждый блок, мы можем сохранить указатель на начало непрерывной области блоков и длину этой области. Одного экстенента недостаточно, потому что не всегда можно найти непрерывную свободную область на диске. Поэтому в системах на основе экстенентов часто разрешается иметь несколько экстенентов, что дает файловой системе больше свободы при выделении места файлу.

Если сравнивать оба подхода, то подход на основе указателей более гибкий, но требует больше метаданных в расчете на один файл (особенно большой). Подходы на основе экстенентов менее гибкие, зато более компактные; особенно хорошо они работают, когда на диске много свободного места и файлы можно разместить в непрерывной области (а это вообще желанная цель практически для любой политики выделения места файлу).

Неудивительно, что возникает желание поддержать еще большие файлы. Для этого нужно всего лишь добавить еще один указатель на индексный дескриптор: **двойной косвенный указатель**. Он ссылается на блок, содержащий указатели на косвенные блоки, каждый из которых содержит указатели на блоки данных. Таким образом, двойной косвенный блок дает возможность увеличить размер файлов еще на $1024 \cdot 1024$, или миллион блоков по 4 КБ, т. е. более 4 ГБ. Можно и еще больше, и вы наверняка уже понимаете, к чему мы клоним: **тройной косвенный указатель**.

В общем и целом такое несбалансированное дерево называется **многоуровневым индексом**. Рассмотрим пример, включающий 12 прямых указателей, один косвенный и один двойной косвенный блок. В предположении, что размер блока равен 4 КБ, а размер указателя – 4 байта, такая структура допускает создание файла размером чуть больше 4 ГБ (точнее $(12 + 1024 + 1024^2) \times 4$ КБ). Попробуйте посчитать максимальный размер файла при добавлении тройного косвенного блока (подсказка: очень большой).

Многоуровневые индексы применяются во многих файловых системах, включая такие широко распространенные, как ext2 [P09] и ext3 в Linux, WAFL компании NetApp и оригинальную файловую систему Unix. В других файловых системах, в частности SGI XFS и Linux ext4, применяются не простые указатели, а **экстенты**; о том, как работают экстенты, см. врезку выше (они в чем-то похожи на сегменты, которые мы обсуждали в части, посвященной виртуальной памяти).

Возникает вопрос: зачем использовать такое несбалансированное дерево? Что, нет другого решения? На самом деле многие исследователи изучали файловые системы и способы их практического использования и чуть ли не каждый раз открывали некие «истины», не меняющиеся на протяжении многих десятилетий. Одна из них заключается в том, что *большинство файлов малы*. И описанный выше несбалансированный подход отражает эту реальность: если в большинстве своем файлы действительно малы, то имеет смысл оптимизировать решение для такого случая. При небольшом числе прямых указателей (12 – типичное значение) индексный дескриптор может напрямую адресовать 48 КБ данных, а для более крупных понадобится один (или несколько) косвенный блок. Недавнее исследование см. в работе Agrawal et al. [A+07]; итоги подведены на рис. 40.2.

Файлы в основном малы	Типичный размер 2 КБ
Средний размер файлов растет	В среднем почти 200 КБ
Большая часть данных хранится в больших файлах	Несколько больших файлов занимают почти все место
Файловые системы содержат много файлов	В среднем почти 100К
Файловые системы примерно наполовину пусты	Даже при увеличении размера диска файловая система остается заполненной на 50 %
Типичный каталог мал	Во многих каталогах мало записей, по большей части не более 20

Рис. 40.2 ❖ Результаты измерения файловых систем

Конечно, существуют и иные подходы к проектированию индексных дескрипторов; в конце концов, индексный дескриптор – всего лишь структура данных, и подойдет любая структура, в которой хранится релевантная информация и которая допускает эффективные запросы к ней. Поскольку файловую систему легко заменить, имеет смысл изучить различные решения на случай, если изменится характер рабочей нагрузки или технология.

Отступление: подходы на основе связей

Еще один более простой подход к проектированию индексных дескрипторов – использование **связного списка**. В этом случае вместо нескольких указателей в дескрипторе хранится только один, указывающий на первый блок файла. Если этого недостаточно, в конце блока данных размещается указатель на следующий блок и т. д.

Легко понять, что для некоторых видов рабочей нагрузки этот подход малопригоден; подумайте, например, как прочитать последний блок файла или как организовать произвольный доступ. Поэтому, чтобы повысить производительность связного списка блоков, в некоторых системах таблица связей хранится в памяти, а не в самих блоках. Эта таблица индексируется адресом блока данных D , а в записи таблицы хранится адрес следующего за D блока. Допускается также нулевой указатель (обозначающий конец файла) и другие признаки, означающие, что блок свободен. При наличии такой таблицы указателей на следующий схема со связным списком блоков может эффективно применяться для реализации произвольного доступа – нужно просто пробежаться по таблице в памяти до нужного блока, а затем обратиться к нему на диске.

Не кажется ли вам эта идея знакомой? То, что мы только что описали, – основа файловой системы **FAT** (file allocation table – **таблица распределения файлов**). Да, в основе классической файловой системы в старых версиях Windows, существовавшей до NTFS [С94], лежали связные списки блоков. Эта схема имеет и другие отличия от стандартной файловой системы в Unix; например, индексных дескрипторов как таковых не существует, а есть лишь записи каталогов, в которых хранятся метаданные файла и указатель на первый блок файла, из-за чего создание жестких ссылок невозможно в принципе. Описание других не слишком элегантных деталей см. в работе Брауэра [B02].

40.4. ОРГАНИЗАЦИЯ КАТАЛОГОВ

В vsfs (как и во многих файловых системах) каталоги имеют простую организацию: по существу, каталог содержит список пар (имя элемента, номер индексного дескриптора). Для каждого файла или каталога в блоках данных хранится строка и число. Может также храниться длина строки (если допускаются имена переменной длины).

Например, предположим, что каталог `dir` (с номером индексного дескриптора 5) содержит три файла (`foo`, `bar` и `foobar_is_a_pretty_longname`) с номерами индексных дескрипторов 12, 13 и 24. Тогда на диске будут храниться следующие данные:

inum	reclen	strlen	name
5	12	2	.
2	12	3	..
12	12	4	foo
13	12	4	bar
24	36	28	foobar_is_a_pretty_longname

Здесь в каждой записи хранится номер индексного дескриптора, длина записи (общее число байтов, занятых именем, плюс неиспользованное место), длина строки (фактическая длина имени) и, наконец, имя элемента. На-

помним, что в каждом каталоге есть две специальные записи: . (точка) и .. (две точки); первый соответствует текущему каталогу (в данном случае `dir`), второй – родительскому (в данном случае корневому).

Удаление файла (например, вызов `unlink()`) может оставить пустое место в середине каталога, поэтому должен быть предусмотрен какой-то способ обозначить этот факт (например, зарезервировать номер индексного дескриптора 0). Удаление – одна из причин использования длины записи: новая запись может быть размещена на месте старой, которая была больше, поэтому в ней осталось лишнее место.

Возникает вопрос: где именно хранятся каталоги? Зачастую файловая система рассматривает каталог как специальный тип файла. Это значит, что каталогу соответствует индексный дескриптор, находящийся где-то в таблице индексных дескрипторов (и помеченный типом «каталог», а не «регулярный файл»). Дескриптор указывает на блоки данных каталога (и, быть может, косвенные блоки), которые в нашей простой файловой системе находятся в области данных. Наша структура на диске остается неизменной.

Снова подчеркнем, что простой линейный список записей каталога – не единственный способ хранения этой информации. Как и в других случаях, возможна любая структура данных. Например, в системе XFS [S+96] каталоги хранятся в виде В-дерева, поэтому операция создания файла (которая должна гарантировать уникальность имени) выполняется быстрее, чем в системах с простыми списками, которые нужно просмотреть от начала до конца.

40.5. УПРАВЛЕНИЕ СВОБОДНЫМ МЕСТОМ

Файловая система должна отслеживать свободные индексные дескрипторы и блоки данных, чтобы можно было найти место для нового файла или каталога. Поэтому **управление свободным местом** важно во всех файловых системах. В *vsfs* для этой цели используются простые битовые карты.

ОТСТУПЛЕНИЕ: УПРАВЛЕНИЕ СВОБОДНЫМ МЕСТОМ

Существует много способов управления свободным местом, битовые карты – лишь один из них. В некоторых ранних файловых системах использовался **список свободных**, когда в суперблоке хранился указатель на первый свободный блок, а внутри каждого свободного блока – указатель на следующий, так что в итоге формировался список свободных блоков в системе. Когда нужно было получить блок, система брала его из начала списка и соответственно обновляла указатели.

В современных файловых системах применяются более сложные структуры данных. Например, в системе XFS компании SGI [S+96] используется вариант **В-дерева** для компактного представления свободных областей диска. Как и для любой структуры данных, имеют место компромиссы между временем и пространством.

Например, желая создать файл, мы должны выделить для него индексный дескриптор. Поэтому файловая система просматривает битовую карту

в поисках свободного дескриптора и отдает его файлу; при этом система помечает дескриптор как занятый (устанавливая соответствующий бит в 1) и в конечном итоге записывает на диск правильную информацию. Аналогичные действия производятся при освобождении блока данных.

При выделении блоков данных для нового файла тоже возникают некоторые вопросы. Например, в файловых системах Linux ext2 и ext3 после создания файла производится поиск последовательности соседних свободных блоков данных (скажем, восьми) для него. Отыскав такую последовательность, файловая система гарантирует, что часть файла будет занимать непрерывный участок диска, что повышает производительность. Такая политика **предварительного выделения** нередко применяется при выделении места для данных.

40.6. Пути доступа: чтение и запись

Получив представление о том, как файлы и каталоги хранятся на диске, мы можем проследить, что делается при чтении и записи файла. Понимание происходящего на таком **пути доступа** – второй ключ к уяснению принципов работы файловой системы. Будьте внимательны!

В следующих примерах предполагается, что файловая система смонтирована, и, следовательно, суперблок уже находится в памяти. Все остальное (индексные дескрипторы и каталоги) пока остается на диске.

Чтение файла с диска

В этом простом примере мы предположим, что требуется просто открыть файл (скажем, /foo/bar), прочитать его и закрыть. Будем считать, что размер файла равен 12 КБ (т. е. он занимает 3 блока).

При выполнении системного вызова `open("/foo/bar", 0_RDONLY)` файловая система сначала должна найти индексный дескриптор файла bar и получить информацию о нем (права доступа, размер файла и т. д.). Для этого нужно уметь находить индексный дескриптор, но пока что у файловой системы есть только полный путь к файлу. Система должна **пройти** по этому пути и добраться до нужного индексного дескриптора.

Любой проход начинается от **корневого каталога** файловой системы, обозначаемого /. Поэтому первым делом файловая система читает с диска индексный дескриптор корневого каталога. Но где находится этот дескриптор? Чтобы найти дескриптор, мы должны знать его i-номер. Обычно i-номер файла или каталога ищется в его родительском каталоге, но у корня нет родителя (по определению). Следовательно, номер индексного дескриптора корня должен быть «хорошо известен»; файловая система должна знать его в момент монтирования. В большинстве файловых систем для Unix i-номер корневого каталога равен 2. Поэтому процесс начинается с чтения блока, содержащего дескриптор с номером 2 (первого блока индексных дескрипторов).

После того как дескриптор прочитан, файловая система берет из него указатели на блоки данных, в которых хранится содержимое корневого каталога. Зная эти указатели, файловая система может прочитать каталог, и в данном случае она ищет элемент `foo`. Прочитав один или несколько блоков данных каталога, она найдет запись, соответствующую `foo`, а значит, будет знать номер индексного дескриптора `foo` (скажем, 44), который понадобится на следующем шаге.

Далее система рекурсивно проходит по пути, пока не дойдет до нужного индексного дескриптора. В нашем примере она читает блок, содержащий индексный дескриптор `foo`, а затем данные этого каталога и в конечном итоге найдет дескриптор `bar`. Последний шаг `open()` – прочитать индексный дескриптор `bar` в память; там файловая система проверяет права доступа, выделяет дескриптор файла в таблице открытых файлов процесса и возвращает этот дескриптор вызывающей программе.

После того как файл открыт, программа может обратиться к системному вызову `read()` для чтения из него. Первая операция чтения (со смещения 0, если до этого не вызывалась функция `lseek()`) прочитает первый блок файла, узнав из индексного дескриптора, где этот блок находится; возможно, она также обновит дескриптор, записав в него время последнего доступа. Затем операция чтения обновит запись находящейся в памяти таблицы открытых файлов, относящуюся к этому дескриптору файла, записав в нее текущее смещение, так чтобы следующая операция чтения прочитала второй блок, и т. д.

ОТСТУПЛЕНИЕ: ПРИ ЧТЕНИИ НЕ НУЖЕН ДОСТУП К СТРУКТУРАМ ДЛЯ ВЫДЕЛЕНИЯ

По нашим наблюдениям, многие студенты плохо понимают структуры, связанные с выделением места, в частности битовые карты. Многие думают, что при чтении файла, когда новые блоки не выделяются, система все равно обращается к битовым картам. Это не так! Доступ к структурам для выделения производится, только когда нужно выделить место. В индексных дескрипторах, каталогах и косвенных блоках уже содержится вся информация, необходимая для завершения запроса чтения. Зачем проверять, что блок выделен, если индексный дескриптор уже на него указывает?

В какой-то момент файл нужно будет закрыть. Тут работы гораздо меньше; понятно, что дескриптор файла следует освободить, но это и все, что должна сделать файловая система. Дисковый ввод-вывод при этом не производится.

Весь процесс изображен на рис. 40.3 (время течет сверху вниз). Мы видим, что открытие файла приводит к многочисленным операциям чтения, призванным найти индексный дескриптор файла. После этого для чтения каждого блока файловая система должна сначала заглянуть в индексный дескриптор, затем прочитать блок и, наконец, обновить время последнего доступа к файлу, выполнив операцию записи. Потратьте некоторое время, чтобы понять последовательность событий.

Отметим также, что объем ввода-вывода в результате открытия файла пропорционален длине пути. Для каждого дополнительного каталога в пути нам придется прочитать его индексный дескриптор и данные. Еще больше

осложняет задачу наличие больших каталогов; в нашем примере нужно было прочитать только один блок, в котором помещалось все содержимое каталога, а если каталог велик, то для нахождения нужной записи, возможно, придется прочитать много блоков. Да, жизнь тяжела даже при чтении файла, но, как мы скоро увидим, запись (а особенно создание нового файла) еще хуже.

	Битовая карта данных	Битовая карта ИД	Корневой ИД	ИД foo	ИД bar	Корневые данные	Данные foo	data[0] bar	data[1] bar	data[2] bar
open(bar)			чтение			чтение				
			чтение			чтение				
			чтение			чтение				
read()			чтение			чтение				
			запись			чтение				
read()			чтение			чтение				
			запись			чтение				
read()			чтение			чтение				
			запись			чтение				

Рис. 40.3 ❖ Хронология чтения файла

Запись на диск

Процесс записи в файл аналогичен. Сначала файл нужно открыть (как описано выше). Затем приложение вызывает `write()`, чтобы записать в файл новые данные. И в конце файл закрывается.

Но, в отличие от чтения, при записи в файл нужно **выделить** блок (если только не перезаписывается уже существующий блок). Каждая операция записи должна не только записать данные на диск, но предварительно определить, какой блок выделить файлу, и соответственно обновить структуры на диске (в частности, битовую карту данных и индексный дескриптор). Поэтому каждая запись в файл порождает пять логических операций ввода-вывода: чтение битовой карты данных (которая затем обновляется, чтобы пометить вновь выделенный блок как занятый), запись в эту битовую карту (чтобы отразить ее новое состояние на диске), чтение и запись в индексный дескриптор (в котором нужно зарегистрировать местоположение нового блока) и последнюю запись в сам блок данных.

Количество операций записи еще возрастает при выполнении такой простой и распространенной операции, как создание файла. Чтобы создать файл, файловая система должна не только выделить индексный дескриптор, но и найти место в каталоге, содержащем файл. Общий объем ввода-вывода при этом весьма солидный: чтение битовой карты индексных дескрипторов

(чтобы найти свободный дескриптор), запись в эту битовую карту (чтобы пометить, что дескриптор занят), запись в сам индексный дескриптор (инициализация), запись в блок данных каталога (чтобы связать внешнее имя файла с номером его индексного дескриптора), а также чтение индексного дескриптора каталога и запись в него, чтобы отразить новое состояние. А если каталог приходится увеличивать, чтобы он вместил новую запись, то понадобятся дополнительные операции ввода-вывода (запись в битовую карту данных и в новый блок каталога). И это все, только чтобы создать файл!

Рассмотрим конкретный пример: создание файла `/foo/bar` и запись в него трех блоков. На рис. 40.4 показано, что происходит во время вызова `open()` (для создания файла) и каждой из трех операций записи блока размером 4 КБ.

	Битовая карта данных	Битовая карта ИД	Корневой ИД	ИД foo	ИД bar	Корневые данные	Данные foo	data[0] bar	data[1] bar	data[2] bar
create(/foo/bar)		чтение запись	чтение			чтение				
				чтение			чтение			
					чтение запись	запись				
				запись						
write()	чтение запись				чтение					
					запись			запись		
write()	чтение запись				чтение					
					запись				запись	
write()	чтение запись				чтение					
					запись					запись

Рис. 40.4 ❖ Хронология создания файла

На рисунке операции чтения и записи на диск сгруппированы в соответствии с породившим их системным вызовом и перечислены сверху вниз в порядке выполнения. Видно, как много работы приходится проделать для создания файла: в данном случае 10 операций ввода-вывода – чтобы прой-

ти по пути и в самом конце создать файл. Видно также, что каждая запись с выделением нового блока порождает 5 операций ввода-вывода: две, чтобы прочитать и обновить индексный дескриптор, еще две, чтобы прочитать и обновить битовую карту данных, и, наконец, операцию записи в сам блок данных. И как же файловая система может обеспечить при этом приемлемую эффективность?

СУЩЕСТВО ПРОБЛЕМЫ:

КАК УМЕНЬШИТЬ ЗАТРАТЫ ФАЙЛОВОЙ СИСТЕМЫ НА ВВОД-ВЫВОД

Даже простейшие операции открытия, чтения и записи в файл порождают огромное число операций ввода-вывода, разбросанных по всему диску. Что может сделать файловая система для уменьшения сопряженных с этим затрат?

40.7. КЕШИРОВАНИЕ И БУФЕРИЗАЦИЯ

Как показывают рассмотренные выше примеры, чтение и запись файлов могут обходиться дорого, поскольку порождают много операций ввода-вывода на (медленный) диск. Чтобы решить эту колоссальную проблему производительности, в большинстве файловых систем агрессивно используется системная память (ДЗУПВ, англ. *DRAM*) для кеширования важных блоков.

Возьмем приведенный выше пример открытия файла: без кеширования каждая такая операция потребовала бы по меньшей мере двух операций чтения для каждого уровня иерархии каталогов (чтобы прочитать индексный дескриптор каталога и его данные). Если путь длинный (например, `/1/2/3/.../100/file.txt`), то файловой системе пришлось бы выполнять сотни операций чтения, только чтобы открыть файл!

Поэтому еще в ранних файловых системах был добавлен **кеш постоянного размера** для хранения востребованных блоков. Ранее при обсуждении виртуальной памяти мы говорили о применении стратегии **LRU** и различных ее вариантов для решения о том, какие блоки оставлять в кеше; эти стратегии актуальны и в данном случае. Кеш постоянного размера обычно создается на этапе начальной загрузки и занимает приблизительно 10 % доступной памяти.

Но такое **статическое разбиение** памяти может оказаться чрезмерно расточительным: что, если файловой системе не нужно 10 % памяти в данный момент времени? Если размер кеша постоянный, то неиспользуемые страницы кеша не могут быть отданы под другие цели, и мы имеем непроизводительный расход памяти.

В современных системах применяется **динамическое разбиение**, т. е. страницы виртуальной памяти и страницы файловой системы объединяются в **унифицированный страничный кеш** [S00]. Это позволяет более гибко распределять физическую память между виртуальной памятью и файловой системой в зависимости от текущих потребностей.

Снова рассмотрим пример открытия файла, на сей раз с кешированием. При первом открытии может генерироваться много операций ввода-вывода для чтения индексного дескриптора и данных каталогов, но при последующем открытии того же файла (или файлов из того же каталога) большая часть блоков уже будет находиться в кеше, поэтому ввод-вывод не понадобится.

Рассмотрим также влияние кеширования на запись. Если достаточно большой кеш позволяет вообще избежать чтения с диска, то для операций записи обращение к диску неизбежно, иначе данные не сохранить надолго. Поэтому кеш не позволяет повысить производительность записи в той же мере, как для чтения. Однако **буферизация записи** (так ее иногда называют), безусловно, приносит определенный выигрыш. Во-первых, откладывая на время запись, система может собрать обновления в **пакет**; например, если битовая карта индексных дескрипторов обновляется при создании одного файла, а затем, спустя несколько мгновений, еще раз при создании другого файла, то файловая система может сэкономить на вводе-выводе, если отложит запись после первого обновления. Во-вторых, буферизуя несколько операций записи в памяти, система может **спланировать** последующие операции ввода-вывода и тем самым повысить производительность. Наконец, если отложить некоторые операции записи, то, возможно, их вообще не придется выполнять; например, если приложение создает файл, а затем удаляет его, то откладывание записи на диск при создании позволяет **избежать** ее полностью. В этом случае лень (при записи блоков на диск) является достоинством.

СОВЕТ: СРАВНЕНИЕ СТАТИЧЕСКОГО И ДИНАМИЧЕСКОГО РАЗБИЕНИЙ

При разделении ресурса между несколькими клиентами (пользователями) можно использовать как **статическое**, так и **динамическое разбиение**. При статическом подходе ресурс делится один раз в фиксированной пропорции; например, если у памяти два потенциальных потребителя, то можно отдать фиксированную часть памяти одному, а остальное – другому. Динамический подход более гибкий, он позволяет делить ресурс по-разному в разные моменты времени; например, система может выделить одному пользователю большую долю пропускной способности диска в течение нескольких промежутков времени, а затем передумать и отдать предпочтение другому пользователю.

У каждого подхода есть свои плюсы. Статическое разбиение, гарантирующее, что каждый пользователь получит какую-то долю ресурса, обычно имеет более предсказуемую производительность и зачастую проще реализуется. При динамическом разбиении можно добиться лучшего использования ресурса (нуждающиеся пользователи потребляют ресурс, который в противном случае простаивал бы без движения), но реализовать эту политику труднее, и она может негативно отразиться на пользователях, чей ресурс был отдан другим во время простоя, а потом долго не возвращается. Как всегда в таких случаях, ни один метод не является лучшим, необходимо тщательно проанализировать конкретную проблему и решить, какой подход для нее более подходящий. Впрочем, так ведь всегда нужно делать, правда?

По указанным выше причинам, в большинстве современных файловых систем операции записи буферизуются в памяти на время от пяти до тридцати секунд – и это еще один компромисс: если система «упадет» до фиксации

обновлений на диске, то эти обновления будут потеряны; с другой стороны, если дольше хранить обновления в памяти, то производительность можно повысить за счет пакетных операций, планирования и даже избегания записи.

Для некоторых приложений (например, баз данных) такой компромисс неприемлем. И чтобы избежать непредвиденной потери данных из-за буферизации записи, они форсируют запись на диск, вызывая `fsync()`, т. е. пользуются интерфейсом **прямого ввода-вывода** в обход кеша или **прозрачным** интерфейсом с диском, вообще минуя файловую систему¹. Хотя большинству приложений вполне комфортно жить в условиях, диктуемых файловой системой, существует достаточно средств заставить систему делать то, что вам нужно, если режим по умолчанию не устраивает.

Совет: компромисс между долговечностью и производительностью

Системы хранения часто предлагают пользователям компромисс между долговечностью и производительностью. Если пользователь хочет записывать данные безотлагательно, чтобы обеспечить их долговечность, то система должна приложить все усилия к тому, чтобы новые данные сразу попали на диск, поэтому запись будет медленной (но безопасной). Если же пользователь готов потерять немного данных, то система может буферизовать операции записи в памяти на некоторое время, а затем сбросить их на диск (в фоновом режиме). При этом будет казаться, что запись завершается быстро, т. е. воспринимаемая производительность высока, но если произойдет крах, то все еще не зафиксированные операции записи будут потеряны, так что нужно искать компромисс. Чтобы понять, какой компромисс приемлем, надо знать требования приложения, использующего систему хранения; например, смириться с потерей нескольких последних изображений, скачанных браузером, можно, но потерять часть транзакции базы данных, которая переводит деньги на ваш банковский счет, – вряд ли. Если, конечно, вы не миллионер, в таком случае чего уж так пекаться о каждой копейке?

40.8. Резюме

Мы познакомились с основными механизмами построения файловой системы. Должна храниться информация о каждом файле (метаданные), обычно для этого используется структура под названием индексный дескриптор. Каталог – это просто специальный тип файла, в котором хранятся отображения имя → номер индексного дескриптора. Нужны и другие структуры; например, часто используется битовая карта для отслеживания свободных и занятых индексных дескрипторов и блоков данных.

Что поражает в файловой системе, так это свобода действий при проектировании; в следующих главах мы рассмотрим, как разные файловые системы пользуются этой свободой для оптимизации того или иного аспекта работы.

¹ Запишитесь на курс по базам данных, чтобы больше узнать о базах данных старой школы и их непременном желании обойти ОС и всем управлять самостоятельно. Но будьте осторожны! Такие базы данных всегда пытаются ошельмовать ОС. Стыдно, базовики. Стыдно и неприлично.

Конечно, многие политики мы пока не упомянули. Например, где на диске должен размещаться вновь созданный файл? Это и другие решения будут предметом следующих глав. Или не будут¹?

Литература

[A+07] «A Five-Year Study of File-System Metadata» by Nitin Agrawal, William J. Bolosky, John R. Douceur, Jacob R. Lorch. FAST'07, San Jose, California, February 2007. *Прекрасный, сравнительно свежий анализ фактического использования файловых систем. В библиографии перечислены работы по файловым системам, начиная с 1980-х годов.*

[B07] «ZFS: The Last Word in File Systems» by Jeff Bonwick and Bill Moore. Доступно по адресу http://www.ostep.org/Citations/zfs_last.pdf. *Одна из последних важных файловых систем, исключительно функционально насыщенная. Ей надо бы посвятить отдельную главу, и, возможно, скоро мы это сделаем.*

[B02] «The FAT File System» by Andries Brouwer. September, 2002. Доступно по адресу <http://www.win.tue.nl/~aeb/linux/fs/fat/fat.html>. *Удивительно ясное описание файловой системы FAT.*

[C94] «Inside the Windows NT File System» by Helen Custer. Microsoft Press, 1994. *Короткая книжка о NTFS; вероятно, есть и другие, содержащие больше технических деталей.*

[H+88] «Scale and Performance in a Distributed File System» by John H. Howard, Michael L. Kazar, Sherri G. Menees, David A. Nichols, M. Satyanarayanan, Robert N. Sidebotham, Michael J. West. ACM TOCS, Volume 6:1, February 1988. *Классическая распределенная файловая система. Мы еще вернемся к этой теме, не беспокойтесь.*

[P09] «The Second Extended File System: Internal Layout» by Dave Poirier. 2009. Доступно по адресу <http://www.nongnu.org/ext2-doc/ext2.html>. *Описание некоторых деталей ext2, очень простой файловой системы для Linux, основанной на FFS, Berkeley Fast File System. Мы будем рассматривать ее в следующей главе.*

[RT74] «The Unix Time-Sharing System» by M. Ritchie, K. Thompson. CACM Volume 17:7, 1974. *Оригинальная статья о Unix. Прочитайте, если хотите знать об истоках многих современных операционных систем.*

[S00] «UBC: An Efficient Unified I/O and Memory Caching Subsystem for NetBSD» by Chuck Silvers. FREENIX, 2000. *Статья об интеграции буферизуемого кеширования файловой системы и страничного кеша виртуальной памяти в ОС NetBSD. Это делается и во многих других системах.*

[S+96] «Scalability in the XFS File System» by Adan Sweeney, Doug Doucette, Wei Hu, Curtis Anderson, Mike Nishimoto, Geoff Peck. USENIX '96, January 1996,

¹ Звучит таинственная интригующая музыка, побуждающая поскорее узнать о других аспектах файловых систем.

San Diego, California. *Первая попытка сделать упор на масштабируемости операций, в т. ч. при наличии миллионов файлов в одном каталоге. Блестящий пример доведения идеи до логического завершения. Ключевая мысль этой файловой системы: все является деревом. Было бы хорошо посвятить главу и этой файловой системе.*

Домашнее задание (эмуляция)

Вы будете пользоваться программой `vsfs.py` для изучения того, как изменяется состояние файловой системы при различных операциях. Первоначально файловая система пуста и содержит только корневой каталог. В ходе эмуляции выполняются различные действия, в результате которых состояние файловой системы на диске медленно изменяется. Детали см. в файле `README`.

Вопросы

1. Запустите эмулятор с различными начальными значениями генератора случайных чисел (например, 17, 18, 19, 20) и попробуйте определить, какие операции имели место при каждом изменении состояния.
2. Теперь сделайте то же самое с другими начальными значениями (скажем, 21, 22, 23, 24), но задайте также флаг `-g`: вам нужно будет высказать гипотезу об изменении состояния, видя операцию. Что можно сказать о том, какие блоки предпочитают выделять алгоритмы выделения индексного дескриптора и блока данных?
3. Уменьшите количество блоков данных в файловой системе до очень низкого значения (скажем, 2) и запустите эмулятор, так чтобы он выполнил примерно сотню запросов. Файлы какого типа будут преобладать в системе при таких сильных ограничениях? Какие операции будут завершаться неудачно?
4. То же самое, но ограничьте число индексных дескрипторов. Если индексных дескрипторов очень мало, то какие операции смогут успешно завершиться? А какие обычно будут завершаться неудачно? Каково вероятное конечное состояние файловой системы?

Глава 41

Локальность и быстрая файловая система

Когда операционная система Unix только появилась, один из ее авторов Кен Томпсон написал первую файловую систему, которую будем называть «старой файловой системой Unix». Она была очень простой. Структуры данных на диске располагались примерно так:

S	Индексные дескрипторы	Данные
---	-----------------------	--------

В суперблоке (S) хранилась информация о файловой системе в целом: размер тома, количество индексных дескрипторов, указатель на начало списка свободных блоков и т. д. В области индексных дескрипторов находились все индексные дескрипторы файловой системы. А остаток диска был занят блоками данных.

Плюсом старой файловой системы была простота и то, что она поддерживала базовые абстракции, ожидаемые от файловой системы: файлы и иерархию каталогов. Эта простая система стала большим шагом вперед по сравнению с неуклюжими основанными на записях системами хранения прошлых лет, а иерархия каталогов явилась настоящим прорывом по сравнению с прежними одноуровневыми иерархиями.

41.1. ПРОБЛЕМА: НИЗКАЯ ПРОИЗВОДИТЕЛЬНОСТЬ

Но была и проблема: ужасающая производительность. Согласно измерениям Кирка Маккьюсика (Kirk McKusick) и его коллег по университету в Беркли [MJLF84], производительность была низкой с самого начала, а со временем только ухудшалась, так что в конечном итоге система задействовала лишь 2 % полной пропускной способности диска!

Основной причиной было то, что старая файловая система Unix рассматривала диск как память с произвольным доступом; данные были разбросаны по всему диску без учета того, что диск несет вполне реальные и высокие

затраты на позиционирование. Например, блоки данных файла зачастую находились очень далеко от его индексного дескриптора, поэтому, когда система сначала читала индексный дескриптор, а потом блоки данных файла (весьма распространенная операция), тратилось много времени на поиск по диску, т. е. подвод головки.

Хуже того, файловая система в итоге становится сильно фрагментированной, поскольку свободным пространством никто толком не управляет. Список свободных будет указывать на блоки, разбросанные по всему диску, а файлам выделяется первый же встретившийся свободный блок. В результате для доступа к логически непрерывному файлу придется метаться из одного места диска в другое, что катастрофически снижает производительность.

Например, рассмотрим следующую область данных, которая содержит четыре файла (A, B, C, D), по 2 блока каждый:

A1	A2	B1	B2	C1	C2	D1	D2
----	----	----	----	----	----	----	----

Если удалить блоки B и D, получится такая картина:

A1	A2			C1	C2		
----	----	--	--	----	----	--	--

Как видим, свободное место распалось на две части по два блока, а хотелось бы, чтобы оно занимало четыре соседних блока. Теперь мы хотим выделить место для файла E размером 4 блока:

A1	A2	E1	E2	C1	C2	E3	E4
----	----	----	----	----	----	----	----

Сами видите, что происходит: E оказывается в разных местах диска, поэтому при доступе к нему мы не сможем добиться от диска максимальной производительности (достигаемой при последовательном доступе). Вместо этого мы сначала прочитаем блоки E1 и E2, затем выполним поиск, а потом прочитаем E3 и E4. Такая фрагментация постоянно происходила в старой файловой системе Unix и сводила производительность на нет. Попутное замечание: именно эту проблему призваны решать средства **дефрагментации** диска; они реорганизуют данные на диске, так чтобы файлы занимали непрерывные участки, а свободное место также располагалось в соседних блоках. Для этого блоки данных перемещаются из одного места в другое, и эти изменения отражаются в индексных дескрипторах.

Еще одна проблема: первоначально размер блока был слишком мал (512 байт). Поэтому передача данных с диска производилась неэффективно.

Небольшие блоки хороши тем, что минимизируют **внутреннюю фрагментацию** (потерю места внутри блока), но плохи с точки зрения скорости передачи из-за накладных расходов на позиционирование.

**СУЩЕСТВО ПРОБЛЕМЫ: КАК ОРГАНИЗОВАТЬ ДАННЫЕ НА ДИСКЕ,
ЧТОБЫ ПОВЫСИТЬ ПРОИЗВОДИТЕЛЬНОСТЬ**

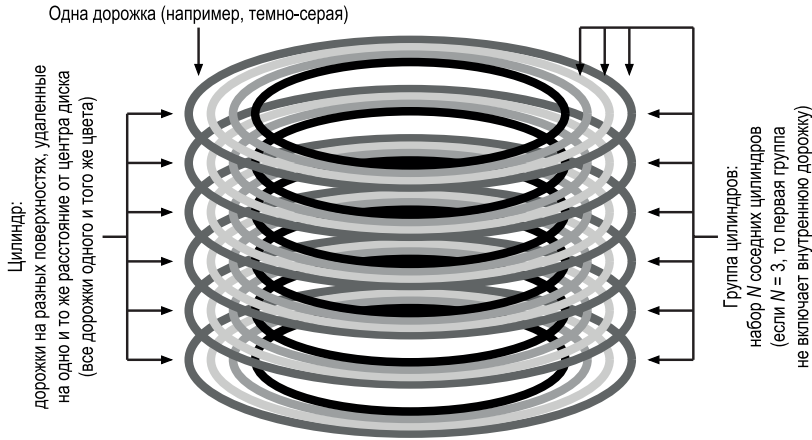
Как организовать структуры данных файловой системы, чтобы повысить производительность? Какие политики выделения следует надстроить над этими структурами данных? Как сообщить файловой системе о характеристиках диска?

41.2. FFS: РЕШЕНИЕ – ОСВЕДОМЛЕННОСТЬ О ДИСКЕ

Группа исследователей из университета в Беркли решила создать улучшенную, более быструю файловую систему, которую так и назвали – **Fast File System** (быстрая файловая система – **FFS**). Идея заключалась в том, чтобы сделать структуры данных и политики выделения «осведомленными о диске» и тем самым повысить производительность. Таким образом, FFS открыла новую эру в изучении файловых систем: сохранив старый *интерфейс* (те же самые API, включая `open()`, `read()`, `write()`, `close()` и другие системные вызовы), но изменив внутреннюю *реализацию*, авторы проложили путь к новым конструкциям файловых систем, и эта работа продолжается по сей день. Практически все современные файловые системы придерживаются существующего интерфейса (а стало быть, сохраняют совместимость с приложениями), но изменяют внутреннее устройство ради достижения большей производительности, надежности или по другим причинам.

41.3. ОРГАНИЗАЦИОННАЯ СТРУКТУРА: ГРУППА ЦИЛИНДРОВ

Первым шагом стало изменение структур на диске. FFS разбивает диск на ряд **групп цилиндров**. **Цилиндром** называется множество дорожек на разных поверхностях жесткого диска, удаленных от центра на одинаковое расстояние; название напоминает об одноименном геометрическом теле. FFS объединяет N соседних цилиндров в группу, поэтому весь диск можно рассматривать как совокупность групп цилиндров. На рисунке ниже показаны четыре внешние дорожки диска с шестью пластинами и группа, состоящая из трех цилиндров.



Отметим, что современные диски не сообщают файловой системе достаточно информации для того, чтобы можно было безошибочно понять, используется ли конкретный цилиндр (см. [AD14a]); диски экспортируют лишь логическое адресное пространство блоков и скрывают от клиентов свою внутреннюю геометрию. Поэтому в современных файловых системах (в частности, ext2, ext3 и ext4 в Linux) диск организуется в виде **групп блоков**, каждая из которых представляет собой непрерывную часть адресного пространства диска. В примере ниже каждые 8 блоков объединены в группу (отметим, что в действительности группы включают гораздо больше блоков).



Но как бы ни называть – группа цилиндров или группа блоков, – именно эти группы являются основным механизмом, позволяющим FFS повысить производительность. Важно, что, помещая два файла в одну группу, FFS может гарантировать, что доступ к одному вслед за другим не приведет к долгому поиску поперек диска. Чтобы использовать группы для хранения файлов и каталогов, FFS должна уметь помещать файлы и каталоги в конкретную группу и хранить всю необходимую информацию о них. Для этого FFS включает все структуры, ожидаемые от файловой системы, внутри каждой группы, т. е. выделяет в ней место для индексных дескрипторов, блоков данных и структур, необходимых для отслеживания свободных и занятых. Ниже показано, что FFS хранит в одной группе цилиндров:



Рассмотрим компоненты одной группы цилиндров более подробно. Для надежности FFS хранит в каждой группе копию **суперблока** (S). Суперблок необходим для монтирования файловой системы; если какая-то копия повредится, систему все равно можно будет смонтировать и работать с ней, воспользовавшись исправной копией.

Внутри каждой группы FFS следит за тем, какие индексные дескрипторы и блоки данных заняты. Для этого служат соответственно **битовая карта индексных дескрипторов** (ib) и **битовая карта данных** (db). Битовые карты – очень эффективный способ управления свободным пространством в файловой системе, потому что позволяют легко найти большой свободный участок и выделить его файлу, избегая тем самым некоторых проблем фрагментации, внутренне присущих механизму списка свободных в старой файловой системе.

Наконец, области **индексных дескрипторов** и **блоков данных** играют ту же роль, что в рассмотренной выше очень простой файловой системе (VSFS). Большая часть каждой группы цилиндров, как обычно, отведена под блоки данных.

ОТСТУПЛЕНИЕ: СОЗДАНИЕ ФАЙЛА В FFS

В качестве примера подумаем, какие структуры данных необходимо обновить при создании файла. Будем считать, что пользователь создает файл /foo/bar.txt длиной один блок (4 КБ). Файл новый, поэтому потребуются новый индексный дескриптор, а значит, на диск нужно будет записать битовую карту индексных дескрипторов и вновь выделенный дескриптор. Файл содержит данные, и для них тоже нужно выделить место, поэтому рано или поздно на диск придется записать обновленную битовую карту данных и сам блок данных. Итак, в текущей группе цилиндров уже насчитывается четыре операции записи (напомним, что эти операции могут быть на время буферизованы в памяти). Но это еще не всё! При создании файла необходимо отыскать ему место в иерархии файловой системы, т. е. обновить каталог. Точнее, нужно обновить родительский каталог foo, добавив в него запись о bar.txt; быть может, эта запись поместится в уже выделенный для foo блок данных, а быть может, понадобится выделить новый блок (и отразить это в битовой карте). Также необходимо обновить индексный дескриптор foo – отразить в нем новую длину каталога и обновить временные метки (в частности, время последней модификации). В общем, куча работы только для того, чтобы создать новый файл! В следующий раз, когда будете это делать, возблагодарите судьбу или, по крайней мере, поразитесь тому, как здорово все работает!

41.4. Политики: как выделять место для файлов и каталогов

Определившись с групповой структурой, FFS далее должна решить, как размещать файлы, каталоги и ассоциированные с ними метаданные на диске, чтобы повысить производительность. Основная мантра проста: *храните ло-*

гически связанные вещи рядом (и, как следствие, храните логически не связанные вещи порознь).

Следуя этой мантре, FFS должна решить, что такое «логически связанные» объекты, и поместить их в одну группу блоков; и наоборот, логически не связанные объекты следует помещать в разные группы блоков. Для этого FFS использует несколько простых эвристических правил размещения.

Первое касается размещения каталогов. В FFS принят простой подход: найти группу цилиндров с малым числом размещенных каталогов (чтобы сбалансировать количество каталогов в каждой группе) и большим числом свободных индексных дескрипторов (чтобы впоследствии можно было разместить много файлов) и поместить каталог и индексный дескриптор в эту группу. Конечно, на этом шаге можно использовать и другие эвристики (например, принимать в расчет количество свободных блоков данных).

Что касается файлов, то FFS делает две вещи. Во-первых, старается (в общем случае), чтобы блоки данных файла оказались в той же группе, что его индексный дескриптор, поскольку это предотвращает долгий поиск при переходе от дескриптора к данным (как было в старой файловой системе). Во-вторых, все файлы, находящиеся в одном каталоге, помещаются в ту группу цилиндров, где находится сам каталог. Таким образом, если пользователь создает четыре файла, /a/b, /a/c, /a/d и /b/f, то FFS постарается поместить первые три рядом (в одну группу), а четвертый подальше от них (в другую группу).

Рассмотрим пример такого размещения. Предположим, что в каждой группе всего 10 индексных дескрипторов и 10 блоков данных (нереалистично мало) и что в соответствии с политиками FFS в них размещено три каталога (корень /, /a и /b) и четыре файла (/a/c, /a/d, /a/e, /b/f). Предположим еще, что длина каждого регулярного файла равна двум блокам и что для каталогов выделен ровно один блок данных. На рисунке ниже файлы и каталоги обозначены очевидными символами (/ вместо корня, a вместо /a, f вместо /b/f и т. д.).

группа	дескрипторы	данные
0	/-----	/-----
1	acde-----	acdde-----
2	bf-----	bff-----
3	-----	-----
4	-----	-----
5	-----	-----
6	-----	-----
7	-----	-----
...		

Отметим два положительных аспекта политик FFS: блоки данных каждого файла находятся недалеко от его индексного дескриптора, а файлы, принадлежащие одному каталогу, находятся рядом (именно, /a/c, /a/d и /a/e в группе 1, а каталог /b и его файл /b/f в группе 2).

Для сравнения посмотрим на политику выделения индексных дескрипторов, которая просто распределяет дескрипторы по группам, стремясь к тому,

чтобы ни в одной группе таблица дескрипторов не заполнилась слишком быстро. В этом случае конечное распределение могло бы выглядеть так:

группа	дескрипторы	данные
0	/-----	/-----
1	a-----	a-----
2	b-----	b-----
3	c-----	cc-----
4	d-----	dd-----
5	e-----	ee-----
6	f-----	ff-----
7	-----	-----
...		

Как видим, эта политика действительно хранит данные файла (и каталога) рядом с соответствующим индексным дескриптором, но файлы, находящиеся в одном каталоге, произвольно разбросаны по диску, так что локальность по именам отсутствует. Доступ к файлам */a/c*, */a/d* и */a/e* теперь затрагивает три группы, а не одну, как в FFS.

Эвристики FFS основаны не на тщательном изучении реального трафика в файловых системах или каких-то особых нюансов, а на старом добром **здравом смысле**. Файлы из одного каталога действительно часто используются вместе; вспомните о компиляции группы файлов и их последующей компоновке в исполняемый файл. Поскольку такая локальность пространства имен реально существует, FFS часто улучшает производительность, гарантируя, что поиск по диску между обращениями к связанным файлам будет коротким.

41.5. ИЗМЕРЕНИЕ ЛОКАЛЬНОСТИ ФАЙЛОВ

Чтобы лучше понять, оправданы ли эти эвристики, проанализируем трассы доступа к файловой системе и посмотрим, существует ли эта пресловутая локальность пространства имен. По непонятной причине в литературе эта тема, похоже, не изучена.

Конкретно, мы будем использовать трассировку в системе SEER [K94] и проанализируем, как «далеко» отстоят друг от друга в дереве каталогов файлы, к которым производятся обращения. Например, если файл *f* открыт, а затем еще раз открыт следующим в трассе (прежде чем были открыты другие файлы), то расстояние между этими двумя операциями открытия в дереве каталогов равно нулю (это один и тот же файл). Если открыт файл *f* в каталоге *dir* (т. е. *dir/f*), а вслед за ним открыт файл *g* в том же каталоге (т. е. *dir/g*), то расстояние между этими операциями доступа равно 1, поскольку каталог один, но файлы разные. Иными словами, наша метрика измеряет, как далеко нужно подняться по дереву каталогов, прежде чем встретится *общий предок* двух файлов; чем они ближе в дереве, тем меньше метрика.

На рис. 41.1 показана локальность, наблюдавшаяся в процессе SEER-трассировки на всех рабочих станциях в кластере SEER по всему множеству

трасс. На графике вдоль оси x отложена описанная выше метрика, а по оси y – накопительная процентная доля операций открытия файлов с данным расстоянием между ними. Мы видим, что для SEER-трасс (обозначенных кружочками на графике) примерно 7 % операций доступа относились к ранее открытому файлу, а примерно 40 % – к тому же файлу или к файлу в том же каталоге (т. е. отстоящему на расстояние 0 или 1). Так что, похоже, гипотеза локальности, принятая в FFS, имеет смысл (по крайней мере, для этих трасс).

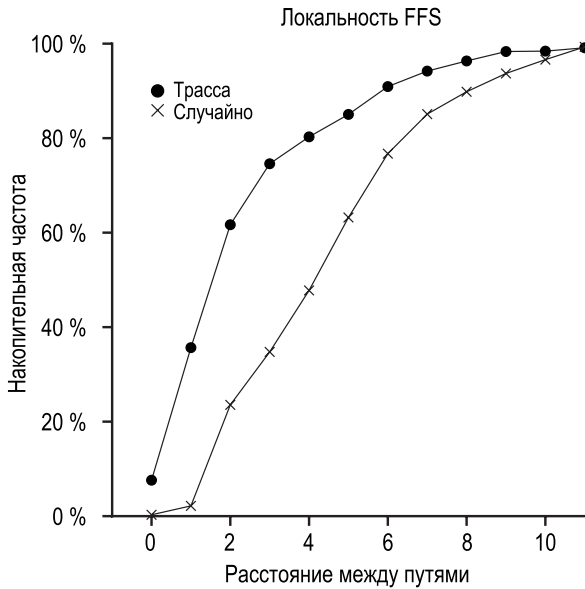


Рис. 41.1 ❖ Локальность FFS для SEER-трассировки

Интересно, что еще примерно 25 % обращений приходится на файлы с расстоянием 2. Такой тип локальности имеет место, когда пользователь организовал многоуровневую иерархию взаимосвязанных каталогов и постоянно переходит из одного в другой. Например, если файлы с исходным кодом хранятся в каталоге `src`, а объектные файлы (с расширением `.o`) строятся в каталоге `obj`, причем оба являются подкаталогами главного каталога `proj`, то будет наблюдаться устойчивый паттерн доступа: сначала `proj/src/foo.c`, затем `proj/obj/foo.o`. Расстояние между этими двумя файлами равно 2, поскольку общим предком является `proj`. FFS не улавливает такой тип локальности в своих политиках, поэтому между подобными операциями доступа время поиска будет больше.

Для сравнения на графике показана также локальность для «случайной» трассы, которая была сгенерирована путем выборки файлов из существующей SEER-трассы в случайном порядке и вычисления расстояния между ними. Как видим, локальность пространства имен в случайных трассах меньше, чего и следовало ожидать. Однако поскольку у любых двух файлов есть общий предок (например, корень), какая-то локальность все же существует, поэтому случайная трасса полезна в качестве эталона для сравнения.

41.6. ИСКЛЮЧЕНИЕ ДЛЯ БОЛЬШИХ ФАЙЛОВ

В FFS имеется одно важное исключение из общей политики размещения файлов, речь идет о больших файлах. Если бы не было другого правила, то большой файл целиком заполнил бы группу блоков, в которую был помещен первоначально (и, возможно, другие тоже). Такое заполнение группы блоков нежелательно, поскольку не позволяет поместить последующие «логически связанные» файлы в ту же группу, что может повредить локальности доступа к файлам.

Поэтому для больших файлов FFS поступает иначе. После того как несколько блоков выделены в первой группе (например, 12 блоков или столько, сколько имеется прямых указателей в индексном дескрипторе), FFS помещает следующую «большую» порцию файла (например, блоки, на которые указывает первый косвенный блок) в другую группу (быть может, выбранную по критерию малой загруженности). Следующая порция помещается в третью группу блоков и т. д.

Чтобы лучше разобраться в этой политике, рассмотрим несколько примеров. Не будь исключения для больших файлов, все блоки одного большого файла оказались бы в одной части диска. Мы изучим небольшой файл (/a) с 30 блоками в FFS, сконфигурированной так, что имеется 10 индексных дескрипторов и 40 блоков данных в каждой группе. Ниже изображено размещение в FFS, если исключений для больших файлов не делается:

группа	дескрипторы	данные
0	/a-----	/aaaaaaaaa aaaaaaaaaa aaaaaaaaaa a-----
1	-----	-----
2	-----	-----
...		

Как видно из рисунка, /a заполнил большинство блоков данных в группе 0, а остальные группы остались пустыми. Если теперь создать другие файлы в корневом каталоге /, то для их данных в этой группе почти нет места.

С исключением для больших файлов (в данном случае пять блоков в каждой порции) FFS распределяет файл между группами, поэтому коэффициент заполнения каждой группы не такой высокий:

группа	дескрипторы	данные
0	/a-----	/aaaaa-----
1	-----	aaaaa-----
2	-----	aaaaa-----
3	-----	aaaaa-----
4	-----	aaaaa-----
5	-----	aaaaa-----
6	-----	aaaaa-----
...		

Внимательный читатель (то есть вы) заметит, что распределение блоков файла по всему диску снижает производительность, особенно в сравнительно распространенном случае последовательного доступа к файлу (например,

когда приложение читает порции с 0 по 20 по порядку). И будет прав! Однако эту проблему можно решить, если тщательно выбирать размер порции.

Именно, если размер порции достаточно велик, то система будет тратить большую часть времени на передачу данных с диска и относительно немного на поиск при переходе от одной порции к другой. Процесс уменьшения накладных расходов за счет увеличения объема работы при той же величине затрат называется **амортизацией**; эта техника часто применяется в вычислительных системах.

Рассмотрим пример: предположим, что среднее время позиционирования (т. е. поиска и вращения) диска равно 10 мс, а скорость передачи данных – 40 МБ/с. Если наша цель – расходовать половину времени на поиск при переходе к новой порции и половину на передачу данных (т. е. достичь 50 % от максимальной пропускной способности диска), то требуется тратить 10 мс на передачу данных на каждые 10 мс, потраченные на позиционирование. Тогда вопрос ставится так: каким должен быть размер порции, чтобы на ее передачу уходило 10 мс? На помощь приходит наш добрый приятель, анализ размерностей, с которым мы познакомились в главе, посвященной дискам [AD14a]:

$$\frac{40 \text{ МБ}}{\cancel{\text{с}}} \cdot \frac{1024 \text{ КБ}}{1 \text{ МБ}} \cdot \frac{1 \cancel{\text{с}}}{1000 \text{ мс}} \cdot 10 \text{ мс} = 409.6 \text{ КБ.} \quad (41.1)$$

Эта формула говорит: если передавать данные со скоростью 40 МБ/с, то для того чтобы половина времени уходила на передачу, а половина на поиск, нужно каждый раз передавать 409.6 КБ. Точно так же можно вычислить размер порции, необходимый для достижения 90 % от максимальной пропускной способности (будет 3.69 МБ) или 99 % (будет целых 40.6 МБ!). Как видим, чем ближе мы хотим подойти к пику, тем больше должны быть порции (на рис. 41.2 построен соответствующий график).

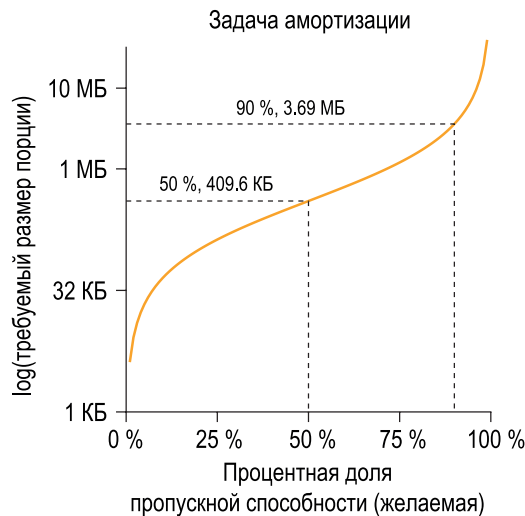


Рис. 41.2 ❖ Амортизация: каким должен быть размер порции?

Но в FFS не выполняются такие вычисления при распределении больших файлов по группам. Вместо этого принят простой подход, основанный на структуре самого индексного дескриптора. Первые 12 прямых блоков помещаются в ту же группу, что сам дескриптор, а каждый последующий косвенный блок и все блоки, на которые он указывает, – в другую группу. Если размер блока равен 4 КБ, то при 32-разрядных дисковых адресах эта стратегия означает, что каждые 1024 блока файла (4 МБ) помещаются в отдельную группу, а единственным исключением являются первые 48 КБ, адресуемые прямыми указателями.

Заметим, что по мере развития технологий производства дисков скорость передачи довольно быстро увеличивается, поскольку изготовители с успехом размещают все больше битов на неизменной площади поверхности, но механические характеристики, от которых зависит время поиска (скорость перемещения кронштейна и угловая скорость вращения), улучшаются гораздо медленнее [Р98]. Поэтому со временем удельный вес механических затрат растет, и для амортизации этих затрат требуется передавать больше данных между операциями поиска.

41.7. ДРУГИЕ АСПЕКТЫ FFS

В FFS были впервые апробированы и другие новшества. В частности, проектировщики сильно заботились о малых файлах; в то время размер многих файлов составлял примерно 2 КБ, поэтому использование 4-килобайтовых блоков было хорошим решением с точки зрения эффективности, но куда менее хорошим с точки зрения эффективности использования места. Из-за **внутренней фрагментации** в типичной файловой системе впустую расходовалась примерно половина диска.

Проектировщики FFS решили эту проблему просто. Они ввели понятие **подблока**; это небольшие блоки размером 512 байт, которые файловая система могла выделять файлам. Поэтому для небольшого файла (скажем, размером 1 КБ) система отдала бы ему два подблока и не тратила целый 4-килобайтовый блок. По мере роста файла система продолжала бы выделять подблоки, пока размер не достиг бы 4 КБ. В этот момент FFS находит блок размером 4 КБ, *копирует* в него подблоки и освобождает их для будущего использования.

Вы, вероятно, заметили, что этот процесс неэффективен, поскольку требует от файловой системы много дополнительной работы (в частности, ввода-вывода при копировании). И снова вы правы! Поэтому в общем случае FFS стремилась избежать такого скверного поведения путем модификации библиотеки `libc`; библиотека должна была буферизовать операции записи и передавать их файловой системе порциями по 4 КБ; как правило, это позволяло вообще избежать выделения подблоков.

Второе интересное новшество FFS заключалось в оптимизации схемы нумерации секторов на диске с целью повышения производительности. В те времена (еще до появления SCSI и других современных интерфейсов) диски

были куда менее сложными устройствами, и для управления ими требовались действия со стороны основного процессора. Проблема возникала, когда файл размещался в соседних секторах диска, как показано на рис. 41.3 слева.

В особенности это относилось к последовательному чтению. FFS сначала читала блок 0. Когда чтение заканчивалось и FFS отправляла команду чтения блока 1, было уже поздно: блок 1 прошел под головкой, и теперь для его чтения нужно было дождаться завершения полного оборота.

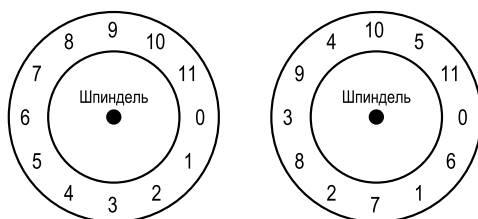


Рис. 41.3 ❖ FFS:
стандартное и параметризованное размещение

FFS решила эту проблему, изменив схему нумерации секторов, как показано на рис. 41.3 справа. Пропуская каждый второй блок (в данном примере), FFS оставляла себе достаточно времени для запроса следующего блока до того, как он прошел под головкой. На самом деле FFS даже умела определять, сколько секторов пропускать на конкретном диске, чтобы избежать лишних оборотов; эта техника получила название **параметризация**, потому что FFS умела вычислять параметры производительности диска и использовать их при выборе такой «шахматной» схемы нумерации.

Быть может, вам кажется, что эта схема не так уж хороша. Ведь мы получаем только 50 % максимальной пропускной способности, потому что должны совершить два оборота, чтобы по одному разу прочитать каждый блок. По счастью, современные диски гораздо «умнее»: они самостоятельно читают всю дорожку и сохраняют ее во внутреннем кеше (который часто называют **буфером дорожки**). Поэтому при последующем чтении сектора на той же дорожке диск просто возвращает данные из своего кеша. Поэтому файловым системам не нужно заботиться об этих деталях очень низкого уровня. Абстрагирование и высокоуровневые интерфейсы – вещь хорошая, если правильно спроектированы.

Были добавлены и другие усовершенствования. FFS была одной из первых файловых систем, в которой допускались **длинные имена файлов**, что позволяло именовать файлы более осмысленно, чем раньше, когда длина была фиксирована (скажем, 8 знаков). Тогда же появилось понятие **символической ссылки**. В предыдущей главе [AD14b] мы объяснили, что жесткие ссылки ограничены, потому что не могут указывать на каталоги (из-за опасений создать циклы в иерархии файловой системы) и могут указывать только на файлы в пределах одного и того же тома (чтобы номер индексного дескриптора имел смысл). Символические ссылки позволяют создавать «псевдонимы» файла или каталога, поэтому обладают гораздо большей гибкостью.

Кроме того, в FFS была впервые реализована атомарная операция `rename()` для переименования файлов. Усовершенствования как в базовой технологии, так и в удобстве пользования снискали FFS широкую популярность.

Совет: системой должно быть удобно пользоваться

Пожалуй, главный урок FFS не в концептуально блестящей идее сделать систему осведомленной об особенностях диска, а в том, что был включен ряд средств, делающих работу с системой более удобной. Длинные имена файлов, символические ссылки и атомарная операция переименования – все это улучшало потребительские характеристики системы. Хотя о таких вещах трудно написать научную работу (ну, представьте себе статью на 14 страниц под названием «Символическая ссылка: давно потерявшийся родственник жесткой ссылки»), именно такие мелочи сделали FFS более полезной и тем повысили ее шансы на признание. Удобство работы с системой зачастую не менее важно, чем глубокие технические инновации.

41.8. РЕЗЮМЕ

Появление FFS стало переломным моментом в истории файловых систем, поскольку ясно показало, что проблема управления файлами – одна из самых интересных в операционной системе, и продемонстрировало, как можно обращаться с самым важным из всех устройств – жестким диском. С тех пор были разработаны сотни новых файловых систем, но и по сей день многие заимствуют идеи FFS (так, `ext2` и `ext3` для Linux – ее очевидные интеллектуальные преемники). И уж точно, все современные системы усвоили главный урок FFS: обращаться с диском именно как с диском.

Литература

[AD14a] «Operating Systems: Three Easy Pieces» (Chapter: Hard Disk Drives) by Remzi Arpaci-Dusseau and Andrea Arpaci-Dusseau. Arpaci-Dusseau Books, 2014. *Невозможно читать о FFS, не поняв сначала, хотя бы в общих чертах, как устроены жесткие диски. Если хотите попробовать, лучше сразу отправляйтесь в тюрьму, не пользуйтесь правом выхода и, самое главное, не получайте такие желанные 200 условных единиц¹.*

[AD14b] «Operating Systems: Three Easy Pieces» (Chapter: File System Implementation) by Remzi Arpaci-Dusseau and Andrea Arpaci-Dusseau. Arpaci-Dusseau Books, 2014. *Как и в предыдущем случае, не имеет смысла читать эту главу, если вы предварительно не прочли (и не поняли) главу о реализации файловой системы. В противном случае щедро употребляемые нами термины «индексный дескриптор» и «косвенный блок» будут вызывать только недоуменное «что это значит?», а это не нужно ни вам, ни нам.*

¹ Отсылка к широко известной настольной игре «Монополия». – Прим. перев.

[K94] «The Design of the SEER Predictive Caching System» by G. H. Kuenning. MOBICOMM '94, Santa Cruz, California, December 1994. *Согласно Кюннингу, это лучший обзор проекта SEER, из которого мы взяли набор трасс (и не только).*

[MJLF84] «A Fast File System for Unix» by Marshall K. McKusick, William N. Joy, Sam J. Leffler, Robert S. Fabry. ACM TOCS, 2:3, August 1984. *Маккьюсик недавно получил премию IEEE имени Рейнольда Б. Джосона за вклад в развитие файловых систем и, прежде всего, за работу по созданию FFS. Ответную речь он посвятил обсуждению оригинальной системы FFS: всего-то 1200 строк кода! Современные версии гораздо сложнее, например преемники BSD FFS занимают в районе 50 тысяч строк.*

[P98] «Hardware Technology Trends and Database Opportunities» by David A. Patterson. Keynote Lecture at SIGMOD '98, June 1998. *Замечательно простой обзор тенденций в технологиях производства диска и их эволюции.*

Домашнее задание (эмуляция)

В этом задании вы познакомитесь с программой `ffs.py`, простым эмулятором FFS, который можно использовать для лучшего понимания того, как FFS выделяет место для файлов и каталогов. Детали см. в файле `README`.

Вопросы

1. Познакомьтесь с файлом `in.largefile`, а затем запустите эмулятор с флагами `-f in.largefile` и `-L 4`. Последний задает исключение для больших файлов – 4 блока. Как будет выглядеть результирующее распределение? Запустите эмулятор с флагом `-c`, чтобы проверить свой ответ.
2. Теперь запустите с флагом `-L 30`. Что вы ожидаете увидеть? Снова проверьте себя, задав флаг `-c`. Можете также задать флаг `-S`, чтобы увидеть, какие именно блоки были выделены файлу `/a`.
3. Теперь вычислим некоторые статистические характеристики файла. Первую назовем *размахом файла*, это максимальное расстояние между парой блоков данных файла или между индексным дескриптором и блоком данных. Вычислите размах файла `/a`. Чтобы узнать, чему он равен, запустите `ffs.py -f in.largefile -L 4 -T -c`. Сделайте то же самое для флага `-L 100`. Как, на ваш взгляд, должен изменяться размах файла с увеличением параметра исключения для большого файла?
4. Теперь рассмотрим входной файл `in.manyfiles`. Как, по вашему мнению, FFS должна распределить эти файлы между группами? (Можете запустить эмулятор с флагом `-v`, чтобы увидеть, какие созданы файлы и каталоги, или просто выполнить `cat in.manyfiles`). Проверьте свое предположение, запустив эмулятор с флагом `-c`.
5. Метрика для оценки FFS называется *размах каталога*. Она описывает разбросанность файлов в конкретном каталоге и равна максимальному расстоянию между индексными дескрипторами и блоками данных всех

файлов в каталоге и между индексным дескриптором и блоком данных самого каталога. Запустите эмулятор для `in.manyfiles` с флагом `-T` и вычислите размах всех трех каталогов. Для проверки задайте флаг `-c`. Хорошо ли FFS справляется с минимизацией размаха каталога?

6. Задайте размер таблицы индексных дескрипторов в группе равным 5 (`-I 5`). Как вы думаете, изменится ли при этом распределение файлов? Проверьте себя, задав флаг `-c`. Как это отражается на размахе каталога?
7. В какую группу FFS должна поместить индексный дескриптор нового каталога? По умолчанию эмулятор ищет группу с максимальным числом свободных индексных дескрипторов. Альтернативная политика – искать набор групп с максимальным числом свободных индексных дескрипторов. Например, если эмулятор запущен с флагом `-A 2`, то при выделении нового каталога эмулятор будет рассматривать группы парами и выбирать лучшую пару. Запустите `./ffs.py -f in.manyfiles -I 5 -A 2 -c`, чтобы увидеть, как изменяется выделение при такой стратегии. Как она влияет на размах каталога? Почему эта стратегия может оказаться хорошей?
8. И напоследок рассмотрим изменение политики, связанное с фрагментацией файлов. Запустите `./ffs.py -f in.fragmented -v` и попробуйте предсказать, как будут распределены оставшиеся файлы. Для проверки задайте флаг `-c`. Что интересного в распределении данных файла `/i`? В чем тут проблема?
9. Новая политика, которую мы будем называть *непрерывным выделением* (`-C`), старается выделить для каждого файла непрерывную область блоков. Именно, при задании флага `-C n` файловая система пытается найти `n` соседних блоков в одной группе, прежде чем выделить блок. Запустите `./ffs.py -f in.fragmented -v -C 2 -c`, чтобы увидеть разницу. Как изменяется распределение при увеличении параметра, сопровождающего флаг `-C`? А как `-C` влияет на размах файла и каталога?

Согласованность после отказа: FSCK и журналирование

Как мы видели, файловая система управляет рядом структур данных, стремясь реализовать ожидаемые от нее абстракции: файлы, каталоги и прочее. Но, в отличие от большинства структур данных (например, хранящихся в памяти исполняемой программы), структуры файловой системы должны храниться **постоянно**, т. е. в течение длительного времени, несмотря на выключение питания (как, например, на жестких дисках или на SSD-дисках).

Одна из главных проблем, стоящих перед файловой системой, – как обеспечить сохранность структур данных в условиях, когда возможно **отключение питания** или **аварийный отказ** (крах) системы. Что произойдет, если в процессе обновления дисковых структур кто-то выдернет питающий кабель? Или если вследствие ошибки в коде случится сбой операционной системы? Из-за возможности пропадания питания и аварийных отказов обновление постоянно хранимой структуры данных может вызывать большие сложности, и это ставит новую интересную проблему при реализации файловой системы – **проблему согласованности после отказа**.

Понять, в чем заключается проблема, легко. Представим, что для завершения некоторой операции нужно обновить две структуры на диске, *A* и *B*. Поскольку диск обслуживает запросы по одному, какая-то из этих операций будет выполняться первой. Если отказ системы или выключение питания произойдет после завершения первой записи, то дисковая структура окажется в **несогласованном** состоянии. И следовательно, возникает проблема, которую должна решать любая файловая система.

СУЩЕСТВО ПРОБЛЕМЫ: КАК ОБНОВИТЬ ДИСК, НЕСМОТРИ НА АВАРИЙНЫЕ ОТКАЗЫ

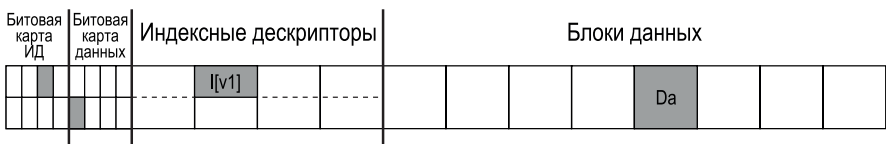
Отказ системы или пропадание питания может возникнуть между двумя операциями записи, в результате чего состояние структур на диске будет обновлено лишь частично. После отказа система перезагружается и хочет снова смонтировать файловую систему (чтобы иметь возможность обращаться к файлам). Учитывая, что отказ может произойти в любой момент времени, как гарантировать, что данные на диске окажутся в разумном состоянии?

В этой главе мы опишем проблему более подробно и рассмотрим некоторые методы ее решения. Начнем с подхода, применявшегося в старых файловых системах, – **средства проверки файловой системы**, или **fsck**. Затем мы перейдем к другому подходу – **журналированию** (или **упреждающей записи в журнал**), – который добавляет небольшие накладные расходы к каждой операции записи, но позволяет быстрее восстанавливаться после отказа или пропадания питания. Мы обсудим основные механизмы журналирования, включая несколько его вариантов, реализованных в файловой системе ext3 для Linux [T98,РАА05] (сравнительно современной файловой системе с журналированием).

42.1. ПОДРОБНЫЙ ПРИМЕР

Чтобы приступить к изучению журналирования, рассмотрим пример. Нам нужна **рабочая нагрузка**, которая каким-то образом обновляет структуры данных на диске. Пусть это будет простое добавление одного блока данных в конец существующего файла. Для этого нужно открыть файл, вызвать `lseek()`, чтобы перейти в конец файла, выполнить запись блока размером 4 КБ, а затем закрыть файл.

Предположим, что используются стандартные дисковые структуры файловой системы, изученные в предыдущих главах. В нашем крохотном примере имеется **битовая карта индексных дескрипторов** (всего 8 бит, по одному на каждый дескриптор), **битовая карта данных** (тоже 8 бит), индексные дескрипторы (в количестве 8 штук, пронумерованных от 0 до 7 и распределенных по четырем блокам) и блоки данных (8 блоков, пронумерованных от 0 до 7). Вот как выглядит эта файловая система:



На рисунке показано, что выделен один индексный дескриптор (с номером 2), закрашенный серым цветом в битовой карте, и один блок данных (с номером 4), тоже закрашенный, но уже в битовой карте данных. Индексный дескриптор обозначен `I[v1]`, поскольку это первая его версия; скоро она будет обновлена.

Заглянем внутрь этого упрощенного индексного дескриптора. Вот что мы увидим:

```
владелец: gemzi
права доступа : чтение-запись
размер : 1
указатель : 4
```

```
указатель : null
указатель : null
указатель : null
```

Здесь размер файла равен 1 (выделен один блок), первый прямой указатель указывает на блок 4 (первый блок данных файла, Da), а три других равны null (т. е. не используются). Конечно, в реальных индексных дескрипторах полей гораздо больше, о них написано в предыдущих главах.

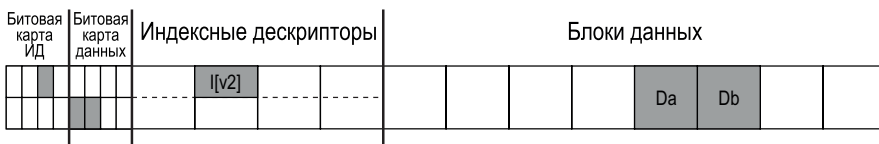
При записи в конец файла мы добавляем в файл новый блок, после чего должны обновить три структуры на диске: индексный дескриптор (он должен указывать на новый блок, а размер файла должен стать больше), новый блок данных Db и битовую карту данных (назовем новую версию B[v2]), которая должна отразить выделение нового блока.

Следовательно, в памяти системы имеется три структуры, которые нужно записать на диск. Обновленный индексный дескриптор (версии 2, I[v2]) теперь выглядит следующим образом:

```
владелец: gemzi
права доступа : чтение-запись
размер : 2
указатель : 4
указатель : 5
указатель : null
указатель : null
```

Обновленная битовая карта данных (B[v2]) теперь имеет вид: 00001100. Наконец, имеется блок данных (Db), заполненный тем, что пользователь пожелал записать в файл. Может быть, ворованной музыкой, кто знает?

Мы хотим, чтобы в конечном итоге образ файловой системы на диске выглядел так:



Для перехода в это состояние файловая система должна произвести три отдельные операции записи на диск: в индексный дескриптор (I[v2]), в битовую карту (B[v2]) и в блок данных (Db). Заметим, что запись обычно производится не сразу в момент системного вызова write(); на самом деле в течение некоторого времени измененные дескриптор, битовая карта и данные хранятся в памяти (в **страничном кеше** или **буферном кеше**), и лишь когда файловая система сочтет нужным (скажем, по истечении 5 или 30 секунд), выполняются соответствующие операции записи. К сожалению, внезапный аварийный отказ может смешать все планы. Если отказ произойдет после одной или двух операций записи, но до завершения третьей, то файловая система останется в некорректном состоянии.

Сценарии отказа

Чтобы лучше понять проблему, рассмотрим несколько возможных сценариев отказа. Допустим, что успешно завершилась только одна операция записи, тогда возможны следующие исходы.

- **На диск записан только блок данных (Db).** В этом случае данные находятся на диске, но на них не указывает никакой индексный дескриптор и в битовой карте не отмечено, что блок выделен. Все выглядит так, будто записи никогда и не было. С точки зрения согласованности файловой системы, проблемы вообще нет¹.
- **На диск записан только обновленный индексный дескриптор (I[v2]).** В этом случае индексный дескриптор содержит адрес на диске (5), по которому система собиралась записать, но так и не записала блок Db. Поэтому, доверившись указателю, мы прочтем с диска мусор (старое содержимое блока по адресу 5).
Более того, мы имеем и еще одну проблему, именуемую **несогласованностью файловой системы**. Битовая карта на диске говорит, что блок 5 еще не был выделен, а индексный дескриптор – что был. Рассогласование между битовой картой и индексным дескриптором – признак несогласованности структур файловой системы; чтобы файловой системой можно было пользоваться, мы должны как-то разрешить эту проблему (подробнее об этом ниже).
- **На диск записана только обновленная битовая карта (B[v2]).** В этом случае битовая карта говорит, что блок 5 выделен, но на него не указывает ни один индексный дескриптор. Таким образом, система снова несогласована; если оставить все как есть, то запись приведет к **утечке места** на диске, поскольку блок 5 невозможно использовать.

Существует еще три сценария отказа, когда две операции записи закончились успешно, а до третьей дело не дошло.

- **На диск записаны индексный дескриптор (I[v2]) и битовая карта (B[v2]), но не блок данных.** В этом случае метаданные файловой системы согласованы: индексный дескриптор указывает на блок 5, и в битовой карте отмечено, что блок 5 занят, так что, с точки зрения метаданных, все в порядке. Но есть проблема: блок 5 содержит мусор.
- **На диск записаны индексный дескриптор (I[v2]) и блок данных (Db), но не битовая карта (B[v2]).** В этом случае дескриптор указывает на правильные данные, но имеет место рассогласование между дескриптором и старой версией битовой карты (B1). И опять-таки эту проблему следует решить, перед тем как пользоваться файловой системой.
- **На диск записаны битовая карта (B[v2]) и блок данных (Db), но не индексный дескриптор (I[v2]).** И снова мы имеем рассогласование между дескриптором и битовой картой. Но теперь, хотя блок был

¹ Однако проблема есть у пользователя, который потерял данные!

записан и битовая карта говорит, что он занят, мы не знаем, какому файлу он принадлежит, потому что на блок не указывает ни один дескриптор файла.

Проблема согласованности после отказа

Надеемся, что эти сценарии показали вам, как много проблем может случиться с образом файловой системы на диске из-за отказов: возможно рас-согласование структур данных, утечка места на диске, возврат мусорных данных пользователю и т. д. В идеале мы хотели бы, чтобы файловая система переходила из одного согласованного состояния (перед добавлением блока) в другое (когда записаны индексный дескриптор, битовая карта и сам блок) **атомарно**. К сожалению, сделать это не так просто, потому что диск выполняет только одну операцию за раз, а отказ или выключение питания может произойти между обновлениями. Эта общая проблема называется проблемой **согласованности после отказа** (или **проблемой согласованного обновления**).

42.2. РЕШЕНИЕ 1: СРЕДСТВО ПРОВЕРКИ ФАЙЛОВОЙ СИСТЕМЫ

В ранних файловых системах к проблеме согласованности подходили просто: согласованность может быть нарушена, но должна быть восстановлена позже (во время перезагрузки). Классический пример такого ленивого подхода – инструмент **fsck**¹. **fsck** – средство поиска и исправления таких несогласованностей в Unix [M86]; аналогичные инструменты проверки и ремонта дискового раздела есть и в других системах. Заметим, что все проблемы так исправить невозможно; вспомним, например, случай, когда файловая система кажется согласованной, но индексный дескриптор указывает на мусорные данные. Единственная реальная цель – восстановить внутреннюю согласованность метаданных файловой системы.

Программа **fsck** состоит из нескольких шагов, описанных в статье Маккьюсика и Ковальски [MK96]. Она запускается *до* того, как файловая система смонтирована и сделана доступной (**fsck** предполагает, что во время ее работы к файловой системе больше никто не обращается). После завершения файловая система должна быть согласованной, так что пользователи могут с ней работать. Ниже кратко описано, что делается на каждом шаге.

- **Суперблок.** Первым делом **fsck** проверяет, нет ли очевидных повреждений суперблока, например размер файловой системы должен быть больше количества выделенных блоков. Цель таких проверок – найти

¹ Произносится по-разному: «эф-эс-си-кей», «эф-эс-чек», а если инструмент вам не нравится, то даже «эф-сак». Правда-правда, серьезные профессионалы так говорят.

подозрительный (поврежденный) суперблок, в этом случае система (или администратор) может использовать альтернативную копию суперблока.

- **Свободные блоки.** Далее fsck проверяет индексные дескрипторы, косвенные блоки, двойные косвенные блоки и т. д., чтобы понять, какие блоки в данный момент выделены. Эта информация используется для создания правильных битовых карт. Таким образом, если между битовыми картами и индексными дескрипторами есть рассогласование, то за основу система берет дескрипторы. Такая же проверка производится для всех индексных дескрипторов – если дескриптор занят, то он должен быть помечен как таковой в битовой карте индексных дескрипторов.
- **Состояние индексных дескрипторов.** Все индексные дескрипторы проверяются на наличие повреждений и других проблем. Например, fsck проверяет, что во всех выделенных дескрипторах поле типа содержит допустимое значение (регулярный файл, каталог, символическая ссылка и т. д.). Если обнаружена проблема, которую трудно исправить, то дескриптор считается подозрительным, и fsck его очищает, после чего соответственно обновляет битовую карту индексных дескрипторов.
- **Счетчики ссылок в индексных дескрипторах.** Также fsck проверяет счетчики ссылок во всех выделенных индексных дескрипторах. Напомним, что счетчик ссылок показывает, сколько имеется файлов, ссылающихся на данный дескриптор. Для проверки fsck обходит все дерево каталогов, начиная с корня, и вычисляет собственные версии счетчиков ссылок для каждого файла и каталога в файловой системе. Если обнаружено расхождение между вычисленным и хранящимся значением, то необходимо выполнить коррекцию – обычно исправляется счетчик внутри индексного дескриптора. Если обнаружен выделенный дескриптор, на который не указывает ни один каталог, то он перемещается в каталог `lost+found`.
- **Дубликаты.** fsck проверяет также наличие указателей-дубликатов, когда два индексных дескриптора указывают на один и тот же блок. Если один из блоков очевидно испорчен, то его следует очистить. В противном случае блок, на который ведут указатели, можно скопировать, так чтобы у каждого дескриптора была своя копия.
- **Испорченные блоки.** В процессе просмотра списка всех указателей проверяются также испорченные указатели на блоки. Указатель считается «испорченным», если указывает куда-то за пределы допустимого диапазона, например на блок с номером, большим, чем размер раздела. В таком случае fsck не может сделать ничего разумного, она просто очищает указатель в индексном дескрипторе или в косвенном блоке.
- **Проверки каталогов.** fsck ничего не знает о содержимом пользовательских файлов, но каталоги содержат информацию в известном формате, поскольку создавались самой системой. Поэтому fsck выполняет дополнительные проверки целостности содержимого каждого каталога: что «.» и «..» – первые элементы, что все узлы, упоминаемые в записях каталогов, выделены и что ни на один каталог не ведет более одного указателя.

Как видите, для написания работоспособной версии `fsck` необходимо очень хорошо знать устройство файловой системы; гарантировать, что такая программа правильно работает во всех случаях, – нетривиальная задача [G+08]. Однако у `fsck` (и других подобных решений) есть более серьезная и фундаментальная проблема: они работают *очень медленно*. Для очень большого тома полная проверка всех выделенных блоков и чтение всего дерева каталогов может занимать много минут, а то и часов. По мере появления дисков большой емкости и RAID-массивов низкая производительность стала непреодолимым препятствием (несмотря на недавние успехи [M+13]).

На верхнем уровне исходный замысел `fsck` представляется несколько нерациональным. Рассмотрим приведенный выше пример с записью всего трех блоков на диск; невероятно расточительно просматривать весь диск, чтобы исправить проблемы, связанные с обновлением каких-то трех блоков. Можно провести аналогию: вы обронили ключи в спальне, но запустили алгоритм *поиска-ключей-во-всем-доме*, начиная с подвала и с посещением каждого помещения. Ключи-то вы, наверное, найдете, но какой ценой! Поэтому, видя, как растут размеры дисков (и RAID-массивов), исследователи и инженеры-практики стали искать другие решения.

42.3. РЕШЕНИЕ 2: ЖУРНАЛИРОВАНИЕ (или УПРЕЖДАЮЩАЯ ЗАПИСЬ В ЖУРНАЛ)

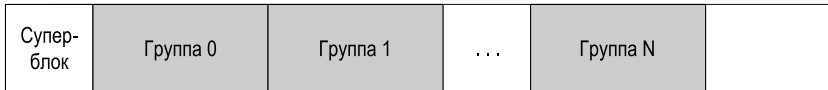
Самое популярное решение проблемы согласованного обновления пришло из мира баз данных. Эта идея, именуемая **упреждающей записью в журнал**, была предложена для решения именно такого рода проблем. В файловых системах упреждающая запись в журнал обычно называется **журналированием** – по историческим причинам. Первой такой файловой системой стала Cedar [H87], но потом идея была реализована во многих системах, включая `ext3`, `ext4` и `reiserfs` для Linux, JFS компании IBM, XFS компании SGI и Windows NTFS.

Основная идея проста. При выполнении операции записи на диск, прежде чем перезаписывать структуры данных на месте, сначала оставить небольшое примечание (где-то в хорошо известном месте на диске) с информацией о том, что мы собираемся делать. Это и есть «упреждающая запись», а сохраняется она в структуре, которая называется «журналом», отсюда и название.

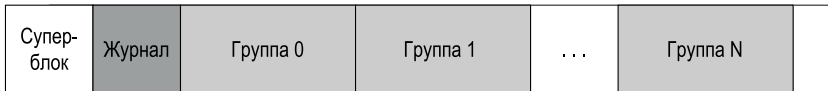
Записывая примечание на диск, мы гарантируем, что если во время обновления структур произойдет крах, то мы сможем из примечания узнать, что собирались делать, и повторить операцию. Таким образом, мы будем точно знать, что (и как) исправлять после отказа, и не нужно будет просматривать весь диск. Согласно замыслу, журналирование добавляет немного работы при каждом обновлении, но существенно уменьшает объем работы при восстановлении.

Теперь опишем, как идея журналирования реализована в популярной файловой системе **Linux ext3**. Структуры данных на диске по большей ча-

сти такие же, как в **Linux ext2**, т. е. диск разбит на группы блоков и в каждом блоке имеется битовая карта индексных дескрипторов, битовая карта данных, индексные дескрипторы и блоки данных. Новой структурой является сам журнал, который занимает немного места внутри раздела или на другом устройстве. В итоге файловая система ext2 (без журналирования) выглядит так:



В предположении, что журнал находится внутри самой файловой системы (хотя иногда его помещают на отдельное устройство или создают как файл), файловая система ext3 с журналированием выглядит так:

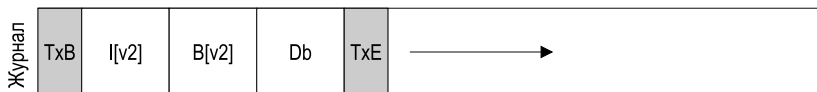


Единственная разница – наличие журнала и, конечно, порядок его использования.

Журналирование данных

На простом примере покажем, как работает **журналирование данных**. Это один из режимов файловой системы Linux ext3, на которой во многом основано данное обсуждение.

Снова будем рассматривать каноническое обновление, когда требуется записать на диск индексный дескриптор ($I[v2]$), битовую карту ($B[v2]$) и блок данных (Db). Прежде чем окончательно записывать все это на диск, произведем запись в журнал. Вот как будет выглядеть журнал после этого:



Как видите, мы записали пять блоков. В блоке начала транзакции (TxV) содержится информация о еще не выполненном обновлении файловой системы (например, адреса блоков $I[v2]$, $B[v2]$ и Db) и некоторый **идентификатор транзакции (TID)**. Следующие три блока содержат точное содержимое самих обновленных элементов; это называется **физическим журналированием**, потому что мы помещаем в журнал точные физические копии обновляемых данных (альтернативная идея **логического журналирования** заключается

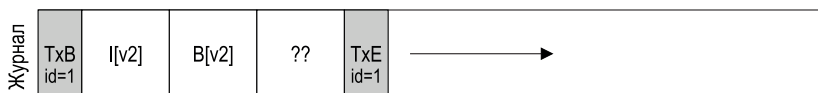
в том, что в журнал помещается логическое описание обновления, например «в этом обновлении мы хотим добавить блок данных Db в конец файла X», это немного сложнее, но позволяет сэкономить место в журнале и, возможно, повысить производительность). Последний блок (TxE) обозначает конец транзакции и тоже содержит TID.

После того как транзакция записана на диск, мы готовы перезаписать старые структуры в файловой системе; этот процесс называется **созданием контрольной точки**. Таким образом, чтобы создать **контрольную точку** файловой системы (т. е. синхронизировать ее с обновлением, записанным в журнал), мы записываем I[v2], B[v2] и Db туда, где они должны находиться на диске. Если запись пройдет успешно, то контрольная точка системы создана, и операция завершена. Итак, вот наша последовательность операций.

1. **Запись в журнал.** Записать транзакцию, включая блок начала транзакции, все запланированные обновления данных и метаданных и блок конца транзакции; дождаться завершения записи.
2. **Контрольная точка.** Записать запланированные обновления данных и метаданных в файловую систему.

В нашем примере мы должны сначала записать TxB, I[v2], B[v2], Db и TxE в журнал. Когда все эти операции записи завершатся, можно будет довести обновление до конца, создав контрольную точку, т. е. записав I[v2], B[v2] и Db туда, где им и место.

Ситуация немного осложняется, если во время записи в журнал происходит аварийный отказ. Мы пытаемся записать на диск набор блоков, составляющий транзакцию (например, TxB, I[v2], B[v2], Db, TxE). Проще всего было бы выполнять операции по одной, дожидаясь каждый раз завершения. Но это долго. В идеале мы хотели бы выполнить все пять операций записи сразу, поскольку это превратило бы пять отдельных операций в одну операцию последовательной записи и потому было бы быстрее. Однако это небезопасно по следующей причине: планировщик диска может выполнять пять частей большой операции записи в любом порядке. То есть диск мог бы (1) записать TxB, I[v2], B[v2] и TxE, а только потом (2) записать Db. Но если питание пропадет между (1) и (2), то вот что окажется на диске:



ОТСТУПЛЕНИЕ: ФОРСИРОВАННАЯ ЗАПИСЬ НА ДИСК

Чтобы обеспечить правильный порядок двух операций записи на диск, современные файловые системы должны принимать дополнительные меры предосторожности. Во время оно форсировать порядок операций записи A и B было просто: инициировать запись A, дождаться, когда диск прервет ОС, сигнализируя о завершении записи, после чего инициировать запись B.

Ситуация осложнилась с распространением кеширования записи самим диском. Если включен режим буферизации записи (иногда его называют **немедленной отчетностью**), то диск информирует ОС о завершении записи, когда лишь поместил данные в свой кеш, хотя на сам диск они еще не попали. Если затем ОС инициирует следующую операцию записи, то нет никакой гарантии, что она достигнет диска раньше предыдущей, поэтому порядок записи не сохраняется. Возможное решение – отключить буферизацию записи. Однако более современные системы принимают дополнительные меры предосторожности и устанавливают явные **барьеры записи**; такой барьер гарантирует, что все операции записи, инициированные до барьера, попадут на диск раньше любой операции записи, инициированной после барьера.

Все эти механизмы требуют полного доверия к правильности работы диска. К сожалению, недавние исследования показали, что некоторые производители дисков в погоне за «высочайшей производительностью» явным образом игнорируют запросы на установление барьера записи, поэтому кажется, что диск работает быстрее, но необязательно правильно [C+13, R+11]. Как говорил Кахан, быстрое почти всегда победит медленное, даже если быстрое неправильно.

Так в чем же здесь проблема? Транзакция выглядит вполне пристойно (у нее есть начало и конец, и идентификаторы в них совпадают). Но файловая система не может понять, что четвертый блок неправилен, ведь это всего лишь пользовательские данные. Поэтому, когда система перезагрузится и начнет восстановление, она воспроизведет эту транзакцию и, не ведая, что творит, скопирует мусор в блоке, помеченном «??», туда, где должен находиться блок Db. Это плохо, даже когда речь идет о пользовательских данных в файле, но многократно хуже, если блок является критическим элементом файловой системы, например суперблоком, – тогда файловую систему вообще не удастся смонтировать.

ОТСТУПЛЕНИЕ: ОПТИМИЗАЦИЯ ЗАПИСИ В ЖУРНАЛ

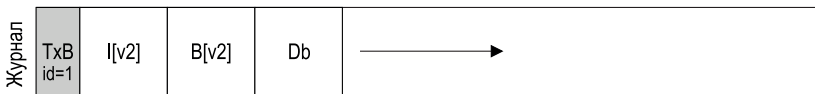
Вы, наверное, обратили внимание на крайнюю неэффективность записи в журнал. Файловая система должна сначала записать блок начала транзакции и ее содержимое и только после завершения этих операций может отправить диску блок конца транзакции. Если вспомнить, как работает диск, то влияние на производительность станет очевидно: обычно это дополнительный оборот диска (подумайте, почему).

Один из наших бывших магистрантов, Виджаян Прабхакаран, придумал простой способ исправить эту проблему [P+05]. При записи транзакции в журнал нужно включить контрольную сумму содержимого в блоки начала и конца. Это позволит файловой системе записать всю транзакцию сразу, без промежуточного ожидания; если в процессе восстановления файловая система обнаружит, что хранимая и вычисленная контрольные суммы не совпадают, то заключит, что во время записи транзакции произошел сбой, и отбросит это обновление файловой системы. Таким образом, путем небольшого изменения в протоколе записи и восстановления можно повысить производительность файловой системы в типичном случае. Заодно система станет чуть надежнее, потому что любое чтение из журнала теперь защищено контрольной суммой.

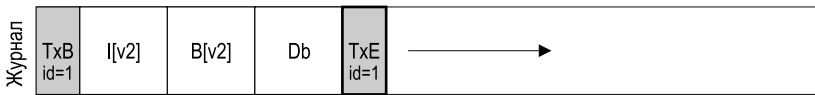
Это простое изменение оказалось настолько заманчивым, что привлекло внимание разработчиков Linux, которые включили его в следующее поколение файловой системы, на-

званное (ну, вы уже догадались!) **Linux ext4**. Теперь она стоит на миллионах машин по всему миру, включая мобильные устройства на платформе Android. Так что знайте – всякий раз, как вы пишете на диск в своей системе Linux, код, разработанный в Висконсине, делает вашу систему чуть быстрее и надежнее.

Чтобы решить эту проблему, файловая система выполняет транзакционную запись в два этапа. Сначала в журнал записываются все блоки, кроме TxЕ, причем все операции записи инициируются сразу. По завершении этой записи журнал будет выглядеть так (опять же в предположении, что речь идет о записи в конец файла):



После этого файловая система записывает блок TxЕ, оставляя журнал в конечном безопасном состоянии:



В этом процессе важна гарантия атомарности, предоставляемая диском. Именно, диск гарантирует, что любая запись длиной 512 байт либо выполнена полностью, либо не выполнена вовсе, поэтому, чтобы запись блока TxЕ была атомарной, он должен занимать ровно 512 байт. Итак, мы можем описать текущий протокол обновления файловой системы.

1. **Запись в журнал.** Записать в журнал содержимое транзакции (блок TxВ, метаданные и данные); дождаться завершения этих операций записи.
2. **Фиксация журнала.** Записать в журнал блок фиксации транзакции (включающий только TxЕ) и дождаться завершения записи. После этого транзакция считается **зафиксированной**.
3. **Контрольная точка.** Записать содержимое обновления (метаданные и данные) туда, где они должны находиться на диске.

Восстановление

Теперь рассмотрим, как файловая система может воспользоваться журналом для **восстановления** после отказа. Отказ может произойти в любой момент описанной выше последовательности обновлений. Если это случилось перед тем, как транзакция полностью записана в журнал (т. е. до завершения шага 2 выше), то все просто: ожидающее обновление просто отбрасывается.

Если отказ произошел после фиксации транзакции в журнале, но до записи контрольной точки, то файловая система может **восстановить** обновление следующим образом. На этапе начальной загрузки процесс восстановления файловой системы просматривает журнал в поисках зафиксированных транзакций; эти транзакции **воспроизводятся** (по порядку), т. е. файловая система снова пытается поместить включенные в транзакцию блоки на диск по месту назначения. Эта простейшая из имеющихся форм журналирования называется **журналом с повтором**. Восстановив зафиксированные в журнале транзакции, система может быть уверена, что структуры данных на диске согласованы, поэтому файловую систему можно смонтировать и приступить к новым запросам.

Отметим, что отказ может произойти в любой момент во время записи контрольной точки, даже после того, как часть операций записи в конечные блоки завершена. В худшем случае некоторые обновления просто будут повторены в процессе восстановления. Поскольку восстановление – редкая операция (происходит только после неожиданного аварийного отказа системы), несколько лишних операций записи – не причина для беспокойства¹.

Группировка обновлений журнала

Вы, конечно, заметили, что базовый протокол сильно увеличивает количество дисковых операций. Представьте, что мы создаем один за другим два файла, `file1` и `file2`, в одном каталоге. Чтобы создать один файл, нужно обновить ряд структур на диске, как минимум битовую карту индексных дескрипторов (чтобы выделить новый дескриптор), вновь созданный индексный дескриптор файла, блок данных родительского каталога, в который нужно поместить новую запись, и индексный дескриптор родительского каталога (чтобы изменить время последней модификации). Если включено журналирование, то мы должны логически зафиксировать все эти операции в журнале при создании каждого из двух файлов. Но файлы создаются в одном каталоге, и если предположить, что их индексные дескрипторы находятся в одном и том же блоке индексных дескрипторов, то мы будем перезаписывать одни и те же блоки снова и снова – если, конечно, не принять никаких мер.

Чтобы смягчить эту проблему, некоторые файловые системы (например, Linux ext3) не фиксируют все обновления по одному, а буферизуют их в одной глобальной транзакции. В примере выше при создании двух файлов система просто помечает, что хранящиеся в памяти битовая карта индексных дескрипторов, индексные дескрипторы файлов, данные каталога и индексный дескриптор каталога модифицированы, и добавляет их в список блоков, входящих в состав текущей транзакции. Когда подойдет время записи этих блоков на диск (скажем, через 5 секунд), эта глобальная транзакция, содержащая все описанные выше обновления, фиксируется. Таким образом, буферизация

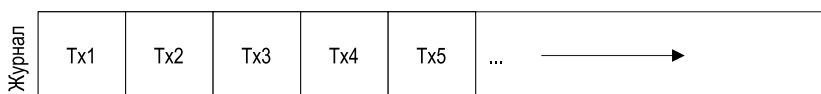
¹ Если, конечно, вас не терзает беспокойство по любому поводу, но тут уж мы бесцельны. Хватит тревожиться, это вредно для здоровья! Но теперь вы, возможно, начнете тревожиться из-за избыточной тревожности.

обновлений во многих случаях позволяет файловой системе избежать избыточного дискового трафика.

Ограничение размера журнала

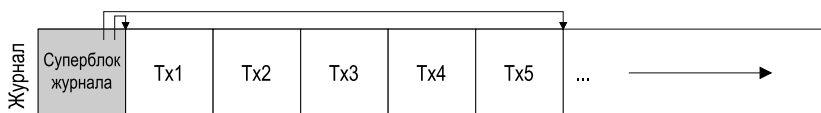
Мы описали базовый протокол обновления дисковых структур файловой системы. Файловая система в течение некоторого времени буферизует обновления в памяти, а когда наконец записывает их на диск, сначала предусмотрительно помещает детали транзакции в журнал упреждающей записи, после чего создает контрольную точку, т. е. записывает все блоки по месту их назначения.

Однако же размер журнала конечен. Если мы так и будем добавлять в него транзакции (как показано на рисунке), то скоро он заполнится. И что тогда?



В связи с переполнением журнала возникает две проблемы. Первая проще, но и не так критична: чем журнал больше, тем больше времени займет восстановление, потому что необходимо воспроизвести все хранящиеся в журнале транзакции (по порядку). Вторая серьезнее: когда журнал заполнен (или почти заполнен), мы не можем зафиксировать ни одной транзакции, поэтому файловая система становится бесполезной.

Для решения этих проблем журнал рассматривается файловой системой как циклическая структура данных, в которую запись производится по кругу, поэтому журнал часто называют **циклическим**. Для этого после создания контрольной точки транзакция файловой системы должна освободить место, которое эта транзакция занимала в журнале, чтобы его можно было использовать повторно. Достичь этого можно разными способами, например просто пометить самую старую и самую новую транзакции, для которых еще не создана контрольная точка, в **суперблоке журнала**; все остальное место свободно. Вот как это выглядит:



В суперблоке журнала (не путать с суперблоком самой файловой системы) хранится достаточно информации, для того чтобы система журналирования знала, для каких транзакций еще не создана контрольная точка. Это уменьшает время восстановления и заодно позволяет организовать циклическое ведение журнала. Таким образом, в наш базовый протокол добавлен еще один шаг.

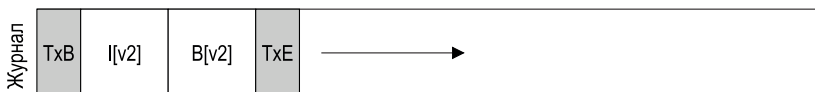
1. **Запись в журнал.** Записать в журнал содержимое транзакции (блок TxV, метаданные и данные); дождаться завершения этих операций записи.
2. **Фиксация журнала.** Записать в журнал блок фиксации транзакции (включающий только TxE) и дождаться завершения записи. После этого транзакция считается **зафиксированной**.
3. **Контрольная точка.** Записать содержимое обновления (метаданные и данные) туда, где они должны находиться на диске.
4. **Освобождение.** Спустя какое-то время пометить транзакцию в журнале как свободную, обновив суперблок журнала.

Это и есть окончательный протокол журнализации. Но осталась одна проблема: каждый блок данных записывается на диск *дважды*, а это очень высокая цена, особенно с учетом того, как редко случаются аварийные отказы. Нельзя ли как-нибудь обеспечить согласованность, не записывая данные дважды?

Журналирование метаданных

Хотя восстановление теперь стало быстрым (просмотр журнала и воспроизведение нескольких транзакций, а не просмотр всего диска), в обычном режиме файловая система работает медленнее, чем хотелось бы. При каждой записи на диск мы должны сначала записать в журнал, что удваивает объем дискового ввода-вывода; особенно болезненно это удвоение при последовательной записи, которая теперь задействует лишь половину пропускной способности диска. Кроме того, между записью в журнал и записью в саму файловую систему производится дорогостоящая операция поиска, что также добавляет заметные накладные расходы при некоторых рабочих нагрузках.

Поскольку двойная запись каждого блока обходится так дорого, было принято несколько попыток повысить производительность. Описанный выше режим журналирования часто называют **журналированием данных** (как в Linux ext3), т. к. журналируются все пользовательские данные (помимо метаданных самой файловой системы). Более простое (и шире распространенное) **упорядоченное журналирование** (или **журналирование метаданных**) отличается тем, что пользовательские данные в журнал *не* записываются. То есть при том же обновлении, что и выше, в журнал была бы записана следующая информация:



Блок данных Db, который раньше записывался в журнал, теперь записывается только в саму файловую систему, что позволяет отказаться от лишней записи. Поскольку большая часть дискового ввода-вывода связана именно с данными, отказ от двойной записи данных существенно уменьшает нагруз-

ку на систему журналирования. Но эта модификация поднимает интересный вопрос: когда именно следует записывать блоки данных на диск?

Снова рассмотрим наш пример записи в конце файла, чтобы лучше разобраться в проблеме. Требуется обновить три блока: I[v2], B[v2] и Db. Первые два содержат метаданные и будут записаны в журнал, а затем в файловую систему (при создании контрольной точки), третий – только в файловую систему. Когда мы должны записать Db на диск? Имеет ли это значение?

Как выясняется, порядок записи данных имеет значение, если журналируются только метаданные. Например, что, если мы запишем Db на диск *после* завершения транзакции (содержащей I[v2] и B[v2])? К сожалению, при таком подходе возникает проблема: файловая система согласована, но I[v2] может указывать на мусорные данные. Именно, предположим, что I[v2] и B[v2] записаны на диск, а Db нет. После этого начинается восстановление файловой системы. Поскольку Db *отсутствует* в журнале, файловая система произведет запись в I[v2] и B[v2] и окажется согласованной (с точки зрения метаданных). Но I[v2] будет указывать на мусорные данные, т. е. на то, что находилось в том блоке, куда мы намеревались записать Db, да не успели.

Чтобы такая ситуация не возникала, некоторые файловые системы (в частности, Linux ext3) записывают блоки данных (регулярных файлов) *сначала*, до того как относящиеся к ним метаданные записаны на диск. Точнее, протокол выглядит следующим образом.

1. **Запись данных.** Записать данные по месту назначения и дождаться завершения (ждать необязательно, см. разъяснения ниже).
2. **Запись метаданных в журнал.** Записать в журнал блок начала транзакции и метаданные; дождаться завершения этих операций записи.
3. **Фиксация журнала.** Записать в журнал блок фиксации транзакции (включающий только TxE) и дождаться завершения записи. После этого транзакция считается **зафиксированной**.
4. **Контрольная точка метаданных.** Записать обновленные метаданные туда, где они должны находиться в файловой системе.
5. **Освобождение.** Спустя какое-то время пометить транзакцию в журнале как свободную в суперблоке журнала.

Записывая данные вначале, файловая система может гарантировать, что указатель никогда не будет вести на мусор. Правило «запиши объект, на который указывают, раньше, чем объект, который на него указывает» – основа согласованности после отказа, оно находит дальнейшее развитие и в других схемах обеспечения согласованности [GP94] (подробности см. ниже).

В большинстве систем журналирование метаданных (сродни упорядоченному журналированию в ext3) популярнее полного журналирования. Например, в Windows NTFS и SGI XFS используются разные формы журналирования метаданных. Linux ext3 предлагает на выбор журналирование данных, а также режимы упорядоченного и неупорядоченного журналирования (в неупорядоченном режиме данные могут записываться в любое время). В любом режиме метаданные согласованы, а различаются они семантикой записи данных.

Наконец, отметим, что ждать завершения записи данных (шаг 1) до инициирования записи в журнал (шаг 2) для гарантии корректности необязательно.

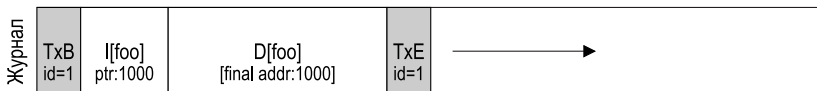
Именно, можно было бы одновременно инициировать запись данных, блока начала транзакции и журналируемых метаданных; единственное непреклонное требование заключается в том, что шаги 1 и 2 должны завершиться раньше, чем в журнал будет записан блок фиксации (шаг 3).

Интересный случай: повторное использование блока

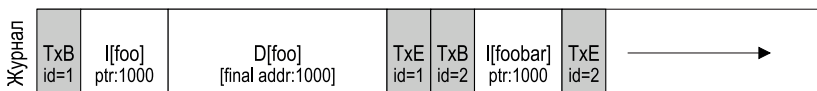
Есть несколько интересных редких случаев, усложняющих журналирование и потому заслуживающих обсуждения. Многие из них связаны с повторным использованием блока. Вот что писал Стивен Твиди (один из главных создателей ext3):

Что самое противное в файловой системе? ... Удаление файлов. Все, что необходимо сделать при удалении, непросто. По ночам вас будут мучить кошмары – что произойдет, если блок удален, а затем снова выделен. [T00]

И Твиди приводит конкретный пример. Допустим, что используется какая-то форма журналирования метаданных (т. е. блоки данных для файлов *не* журналируются). Пусть имеется каталог *foo*. Пользователь добавил в *foo* запись (скажем, создал файл), поэтому содержимое *foo* (ведь каталоги считаются метаданными) записано в журнал; предположим, что данные каталога *foo* находятся в блоке 1000. Таким образом, журнал выглядит примерно так:



В этот момент пользователь удаляет всё из каталога, а затем и сам каталог, так что блок 1000 можно использовать повторно. Наконец, пользователь создает новый файл *foobag*, которому повторно выделяется тот же блок 1000, ранее принадлежавший *foo*. Индексный дескриптор *foobag* записывается на диск, как и данные этого файла. Заметим, однако, что поскольку журналируются метаданные, в журнал попадает только индексный дескриптор *foobag*, а новые данные, записанные в блок 1000 файла *foobag*, *не* журналируются.



Предположим теперь, что произошел аварийный отказ и вся эта информация по-прежнему находится в журнале. Процесс восстановления просто воспроизводит все, что видит в журнале, включая запись данных каталога

в блок 1000; в результате данные текущего файла foobar будут перезаписаны старым содержимым каталога! Очевидно, что это некорректное действие, и пользователь будет немало удивлен, когда попытается прочесть файл foobar.

У этой проблемы есть несколько решений. Например, можно запретить повторное использование блоков до того, как операция удаления не будет отражена в контрольной точке. Но в Linux ext3 вместо этого добавлен новый тип записи в журнале – **отозвать**. В описанном выше случае удаление каталога приведет к помещению в журнал записи отзыва. При воспроизведении журнала система сначала ищет записи отзыва; данные в таких записях никогда не воспроизводятся, что снимает описанную проблему.

Подводя итоги: хронология журналирования

Прежде чем завершить обсуждение журналирования, проиллюстрируем наглядно все описанные выше протоколы. На рис. 42.1 показан протокол журналирования данных и метаданных, а на рис. 42.2 – протокол журналирования только метаданных.

TxV	Журнал		TxE	Файловая система	
	Содержимое (метаданные)	(данные)		Метаданные	Данные
начата	начата	начата			
завершена	завершена	завершена			
		начата			
		завершена			
				начата	начата
				завершена	завершена

Рис. 42.1 ❖ Хронология журналирования данных

TxV	Журнал		TxE	Файловая система	
	Содержимое (метаданные)	(данные)		Метаданные	Данные
начата	начата				начата
завершена	завершена				завершена
		начата			
		завершена			
				начата	
				завершена	

Рис. 42.2 ❖ Хронология журналирования метаданных

На обоих рисунках время течет сверху вниз, а каждая строка соответствует логическому моменту начала или завершения записи. Например, в протоколе журналирования данных (рис. 42.1) операции записи блока начала

транзакции (TxV) и содержимого транзакции логически могут быть начаты одновременно, а завершаться в любом порядке; однако запись блока конца транзакции (TxE) должна быть начата не раньше, чем предыдущие операции завершатся. Аналогично запись контрольной точки данных и метаданных не может начинаться, пока транзакция не будет зафиксирована. Горизонтальные штриховые линии показывают моменты, когда должны соблюдаться эти требования к упорядочению записи.

Аналогичная хронология показана для протокола журналирования метаданных. Заметим, что операция записи данных логически может быть начата одновременно с записью в журнал блоков начала и содержимого транзакции, но она должна завершиться раньше, чем будет начата запись конца транзакции.

Наконец, отметим, что время завершения каждой операции записи в хронологиях показано произвольно. В реальной системе время завершения определяется подсистемой ввода-вывода, которая может переупорядочивать операции записи ради повышения производительности. Единственные гарантии упорядоченности, на которые мы можем рассчитывать, – те, что должны быть обеспечены для корректности протокола (показаны горизонтальными штриховыми линиями на рисунках).

42.4. РЕШЕНИЕ 3: ДРУГИЕ ПОДХОДЫ

До сих пор мы описали два подхода к обеспечению согласованности метаданных файловой системы: ленивый на основе *fsck* и более активный на основе журналирования. Но этим разнообразие не исчерпывается. Еще один подход, получивший название «мягкое обновление», описан в работе Гейнджера и Пэтта [GP94]. В нем все операции записи в файловую систему тщательно упорядочиваются, чтобы структуры данных на диске никогда не оказывались в несогласованном состоянии. Например, если блок данных всегда записывается раньше индексного дескриптора, который на него указывает, то есть уверенность, что индексный дескриптор никогда не будет указывать на мусор. Аналогичные правила можно вывести для всех структур файловой системы. Но реализация мягких обновлений может столкнуться с трудностями; если описанный выше слой журналирования можно реализовать, мало что зная о структурах файловой системы, то метод мягких обновлений требует знания всех деталей структур данных файловой системы, поэтому значительно увеличивает сложность системы.

Другой подход, называемый **копированием при записи** (*copy-on-write*, или **COW**), применяется во многих популярных файловых системах, в т. ч. ZFS компании Sun [B07]. В этом случае ни файлы, ни каталоги не перезаписываются на месте, вместо этого обновления помещаются в свободное место на диске. После завершения какого-то числа обновлений файловая система типа COW изменяет корневую структуру файловой системы, включая в нее указатели на только что обновленные структуры. При таком подходе поддерживать согласованность файловой системы нетрудно. Мы еще поговорим

об этом методе, когда будем обсуждать файловую систему со структурой журнала (log-structured file system – LFS) в следующей главе; LFS – ранний пример COW.

Еще одно решение мы разработали у себя в Висконсинском университете. Мы назвали его **согласованностью на основе обратных указателей** (backpointer-based consistency – BBC), оно не требует определенного порядка операций записи. Для достижения согласованности в каждый блок системы включается дополнительный **обратный указатель**; например, каждый блок данных указывает на индексный дескриптор, которому принадлежит. При обращении к файлу файловая система может установить, согласован ли файл, проверив, что прямой указатель (например, адрес в индексном дескрипторе или прямом блоке) указывает на блок, который, в свою очередь, указывает на него. Если это так, то все, что нужно, было записано на диск, и файл согласован; в противном случае файл не согласован, и система возвращает ошибку. Благодаря добавлению обратных указателей можно реализовать новую форму ленивой согласованности после отказов [C+12].

Наконец, мы также исследовали, как можно уменьшить количество ожиданий завершения записи в протоколе журналирования. В этом новом подходе, названном **оптимистическая согласованность после отказа** [C+13], иницируется столько операций записи на диск, сколько возможно, благодаря применению обобщенной формы **контрольной суммы транзакции** [P+05]. Реализовано и еще несколько методов обнаружения несогласованностей. Для некоторых рабочих нагрузок оптимистические методы способны повысить производительность на порядок. Но если мы хотим, чтобы они работали по-настоящему хорошо, нужно немного изменить интерфейс с диском [C+13].

42.5. РЕЗЮМЕ

Мы познакомились с проблемой согласованности после отказа и обсудили различные подходы к ее решению. Старый подход на основе средства проверки файловой системы работает, но для современных дисков восстановление производится слишком медленно. Поэтому во многих файловых системах теперь применяется журналирование, позволяющее уменьшить время восстановления с $O(\text{размер дискового тома})$ до $O(\text{размер журнала})$, а значит, гораздо быстрее перезапустить систему после аварии. Мы также видели, что журналирование может принимать разные формы; чаще всего встречается упорядоченное журналирование метаданных, поскольку при этом уменьшается объем записи в журнал, но сохраняются разумные гарантии согласованности как метаданных файловой системы, так и пользовательских данных. В конечном итоге строгие гарантии сохранности пользовательских данных – вероятно, главное, что нужно обеспечить. Как ни странно, эта область все еще остается предметом исследований [P+14].

Литература

[B07] «ZFS: The Last Word in File Systems» by Jeff Bonwick and Bill Moore. Доступно по адресу http://www.ostep.org/Citations/zfs_last.pdf. *В ZFS используется копирование при записи и журналирование, потому что в некоторых случаях протоколирование операций записи на диск работает быстрее.*

[C+12] «Consistency Without Ordering» by Vijay Chidambaram, Tushar Sharma, Andrea C. Arpaci-Dusseau, Remzi H. Arpaci-Dusseau. FAST '12, San Jose, California. *Наша недавняя статья о новой форме согласованности после отказов, основанной на обратных указателях. Прочитайте, если хотите узнать об интересных деталях!*

[C+13] «Optimistic Crash Consistency» by Vijay Chidambaram, Thanu S. Pillai, Andrea C. Arpaci-Dusseau, Remzi H. Arpaci-Dusseau. SOSP '13, Nemaquin Woodlands Resort, PA, November 2013. *Наша работа по оптимистическому и более производительному протоколу журналирования. Для рабочих нагрузок с массовым применением `fsync()` производительность может заметно повыситься.*

[GP94] «Metadata Update Performance in File Systems» by Gregory R. Ganger and Yale N. Patt. OSDI '94. *Статья посвящена применению тщательного упорядочения операций записи как основного способа достижения согласованности. Впоследствии идея реализована в системах на базе BSD.*

[G+08] «SQCK: A Declarative File System Checker» by Haryadi S. Gunawi, Abhishek Rajimwale, Andrea C. Arpaci-Dusseau, Remzi H. Arpaci-Dusseau. OSDI '08, San Diego, California. *Наша статья о новом улучшенном способе построения средства проверки файловой системы с применением SQL-запросов. Мы также обращаем внимание на проблемы в существующей реализации `fsck`, отмечая многочисленные ошибки и странности поведения – прямой результат сложности этой программы.*

[H87] «Reimplementing the Cedar File System Using Logging and Group Commit» by Robert Hagmann. SOSP '87, Austin, Texas, November 1987. *Первая известная нам работа о применении упреждающей записи в журнал (журналирования) к файловой системе.*

[M+13] «ffsck: The Fast File System Checker» by Ao Ma, Chris Dragg, Andrea C. Arpaci-Dusseau, Remzi H. Arpaci-Dusseau. FAST '13, San Jose, California, February 2013. *Недавняя наша работа о том, как ускорить `fsck` на порядок. Некоторые идеи уже были включены в средство проверки файловой системы для BSD [MK96].*

[MK96] «Fsk – The UNIX File System Check Program» by Marshall Kirk McKusick and T. J. Kowalski. Revised in 1996. *Описывается первый полный инструмент проверки файловой системы, `fsck`. Среди авторов встречаются те самые люди, которые подарили нам FFS.*

[MJLF84] «A Fast File System for UNIX» by Marshall K. McKusick, William N. Joy, Sam J. Leffler, Robert S. Fabry. ACM Transactions on Computing Systems, Volume 2:3, August 1984. *Вы уже достаточно знаете о FFS, верно? Но все равно сослаться на важные статьи не вредно.*

[P+14] «All File Systems Are Not Created Equal: On the Complexity of Crafting Crash-Consistent Applications» by Thanumalayan Sankaranarayanan Pillai, Vijay Chidambaram, Ramnathan Alagappan, Samer Al-Kiswany, Andrea C. Arpaci-Dusseau, Remzi H. Arpaci-Dusseau. OSDI'14, Broomfield, Colorado, October 2014. *Статья, в которой изучается, какие гарантии дают файловые системы после аварий, и показано, что приложение ожидает совершенно другого, что в результате приводит к разнообразным интересным проблемам.*

[P+05] «IRON File Systems» by Vijayan Prabhakaran, Lakshmi N. Bairavasundaram, Nitin Agrawal, Haryadi S. Gunawi, Andrea C. Arpaci-Dusseau, Remzi H. Arpaci-Dusseau. SOSP '05, Brighton, England, October 2005. *Основное внимание уделяется тому, как файловые системы реагируют на отказы дисков. В конце описывается контрольная сумма транзакций, позволяющая ускорить запись в журнал. Эта идея в итоге была включена в Linux ext4.*

[PAA05] «Analysis and Evolution of Journaling File Systems» by Vijayan Prabhakaran, Andrea C. Arpaci-Dusseau, Remzi H. Arpaci-Dusseau. USENIX '05, Anaheim, California, April 2005. *Наша ранняя статья, посвященная анализу работы файловых систем.*

[R+11] «Coerced Cache Eviction and Discreet-Mode Journaling» by Abhishek Rajimwale, Vijay Chidambaram, Deepak Ramamurthi, Andrea C. Arpaci-Dusseau, Remzi H. Arpaci-Dusseau. DSN '11, Hong Kong, China, June 2011. *Наша статья о проблеме дисков, буферизующих операции записи в памяти вместо форсированной записи на диск, даже когда их об этом явно просили! Наше решение проблемы: если вы хотите, чтобы A было записано на диск раньше B, то сначала запишите A, потом отправьте диску много «фиктивных» операций записи, в надежде, что A все-таки будет записано, чтобы освободить место в кеше. Решение, может, и изящное, но не слишком практичное.*

[T98] «Journaling the Linux ext2fs File System» by Stephen C. Tweedie. The Fourth Annual Linux Expo, May 1998. *Твиди проделал большую работу, увенчанную добавлением журналирования в файловую систему Linux ext2; неудивительно, что новая система была названа ext3. Ряд важных проектных решений касается обратной совместимости, например мы можем просто добавить файл журнала в существующую файловую систему ext2, а затем смонтировать ее как ext3.*

[T00] «EXT3, Journaling Filesystem» by Stephen Tweedie. Talk at the Ottawa Linux Symposium, July 2000. olstrans.sourceforge.net/release/OLS2000-ext3/OLS2000-ext3.html. *Стенограмма выступления Твиди, посвященного ext3.*

[T01] «The Linux ext2 File System» by Theodore Ts'o, June, 2001. Доступно по адресу <http://e2fsprogs.sourceforge.net/ext2.html>. *Простая файловая система для Linux на основе идей, заложенных в FFS. На протяжении некоторого времени использовалась очень активно, теперь же осталась в ядре в качестве примера простой файловой системы.*

Домашнее задание (эмуляция)

В этом задании вы познакомитесь с программой `fsck.py`, простым эмулятором, который позволит лучше понять, как обнаруживаются (и потенциально исправляются) повреждения файловой системы. Подробнее о флагах программы см. в файле `README`.

Вопросы

1. Для начала запустите `fsck.py -D`; этот флаг отключает все повреждения, так что вы можете сгенерировать случайную файловую систему и попробовать определить, какие в ней есть файлы и каталоги. Вперед! Задайте флаг `-r`, чтобы проверить себя. Прodelайте это для нескольких случайным образом сгенерированных файловых систем, задавая различные начальные значения (`-s`), например 1, 2, 3.
2. Теперь добавим повреждение. Начнем с запуска `fsck.py -S 1`. Понимаете ли вы, какая принесена несогласованность? Как бы вы исправили ее с помощью реального инструмента ремонта файловой системы? Проверьте себя, задав флаг `-c`.
3. Измените начальное значение, задав флаг `-S 3` или `-S 19`; теперь какую несогласованность вы видите? Проверьте свой ответ, задав флаг `-c`. Чем различаются эти два случая?
4. Измените начальное значение, задав флаг `-S 5`; какая теперь имеет место несогласованность? Насколько трудно было бы исправить ее автоматически? Проверьте ответ, задав флаг `-c`. Затем добавьте похожую несогласованность, задав флаг `-S 38`; насколько труднее ее обнаружить, и возможно ли это вообще? И наконец, `-S 642`; можно ли обнаружить такую несогласованность? Если да, то как бы вы исправили файловую систему?
5. Измените начальное значение, задав флаг `-S 6` или `-S 13`; какую несогласованность вы видите? Проверьте свой ответ, задав флаг `-c`. Чем различаются эти два случая? Что должен делать инструмент ремонта, встретив такую ситуацию?
6. Измените начальное значение, задав флаг `-S 9`; какую несогласованность вы видите? Проверьте свой ответ, задав флаг `-c`. Какой информации должен доверять инструмент ремонта в этом случае?
7. Измените начальное значение, задав флаг `-S 15`; какую несогласованность вы видите? Что может сделать инструмент ремонта в этом случае? Если ремонт невозможен, то сколько данных будет потеряно?
8. Измените начальное значение, задав флаг `-S 10`; какую несогласованность вы видите? Существует ли в структуре файловой системы избыточность, которая могла бы помочь при ремонте?
9. Измените начальное значение, задав флаг `-S 16` или `-S 20`; какую несогласованность вы видите? Проверьте свой ответ, задав флаг `-c`. Как инструмент ремонта должен был бы исправить такую проблему?

Глава 43

Файловые системы со структурой журнала

В начале 1990-х годов группа в Беркли под руководством профессора Джона Оустерхаута и магистранта Менделя Розенблюма разработала новую файловую систему, получившую название «файловая система со структурой журнала» (log-structured file system) [RO91]. Заняться этим их побудили следующие наблюдения.

- **Объем памяти в системах растет.** Раз памяти больше, то больше данных можно кешировать в памяти. А чем больше данных кешируется, тем сильнее дисковый трафик смещается в сторону записи, потому что операции чтения обслуживаются из кеша. Поэтому производительность файловой системы определяется прежде всего производительностью записи.
- **Существует большой разрыв между производительностью последовательного и произвольного ввода-вывода.** Со временем пропускная способность жесткого диска значительно увеличилась [P98], поскольку возросла плотность упаковки данных на поверхности диска. Но время поиска и задержка вращения уменьшаются медленно, т. к. очень трудно изготовить недорогой и компактный электродвигатель, который вращал бы пластины и передвигал кронштейн с головкой считывания быстрее. Поэтому если обращаться к дискам последовательно, то можно будет добиться заметного повышения производительности по сравнению с подходами, требующими поиска и ожидания вращения.
- **Существующие файловые системы показывают низкую производительность для многих типичных рабочих нагрузок.** Например, FFS [MJLF84] должна выполнить много операций записи для создания нового файла, занимающего один блок: создать новый индексный дескриптор, обновить карту индексных дескрипторов, обновить блок данных каталога, содержащего файл, обновить индексный дескриптор каталога, записать в новый блок данных, принадлежащий файлу, и обновить битовую карту данных, пометив, что блок выделен. И хотя FFS помещает все эти блоки в одну группу, все равно приходится выполнять много коротких операций поиска и затем нести потери на вращение, поэтому пропускная способность далеко не достигает максимальной, возможной при последовательном доступе.

- **Файловые системы не осведомлены о RAID.** Например, и RAID-4, и RAID-5 страдают от **проблемы короткой записи**, когда логическая запись в один блок приводит к четырем физическим операциям ввода-вывода. Существующие файловые системы даже не пытаются избежать такого поведения при записи в RAID в худшем случае.

СОВЕТ: ДЕТАЛИ ИМЕЮТ ЗНАЧЕНИЕ

Все интересные системы основаны на немногих общих идеях, но включают множество деталей. Иногда, читая про такую систему, вы думаете: «Ага, общую идею я уловил, остальное – детали» – и не утруждаете себя изучением того, как все работает на самом деле. Не поступайте так! Очень часто именно детали критичны. Как мы увидим при обсуждении LFS, общую-то идею понять легко, а вот чтобы построить работоспособную систему, нужно продумать *все* заковыристые случаи.

Поэтому в фокусе внимания идеальной файловой системы должна лежать производительность записи, и при этом она должна по возможности задействовать пропускную способность последовательного доступа. Кроме того, она должна хорошо работать на типичных рабочих нагрузках, которые характеризуются не только записью данных, но и частым обновлением метаданных на диске. Наконец, она должна хорошо работать как с отдельными дисками, так и с RAID-массивами.

Розенблум и Оустерхаут назвали файловую систему нового типа **LFS**, что означало Log-structured File System (файловая система со структурой журнала). При записи на диск LFS сначала буферизует все обновления (включая метаданные!) в **сегменте**, находящемся в памяти; когда сегмент заполняется, он переносится в свободную часть диска одной длинной операцией записи. LFS никогда не перезаписывает существующие данные, сегменты *всегда* записываются в свободную область. Поскольку сегменты велики, диск (или RAID-массив) используется эффективно, а производительность файловой системы приближается к максимуму.

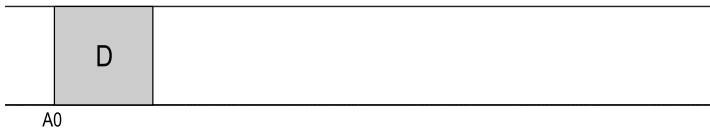
СУЩЕСТВО ПРОБЛЕМЫ: КАК СДЕЛАТЬ ВСЕ ОПЕРАЦИИ ЗАПИСИ ПОСЛЕДОВАТЕЛЬНЫМИ?

Как файловая система может преобразовать все операции записи в последовательные? Для операций чтения это невозможно, потому что требуемый блок может находиться в любом месте диска. Но в случае записи у файловой системы всегда есть выбор, и вот этим-то мы и намереваемся воспользоваться.

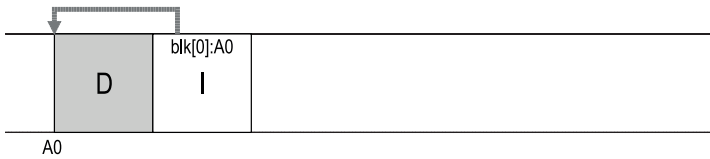
43.1. ЗАПИСЫВАТЬ НА ДИСК ПОСЛЕДОВАТЕЛЬНО

Итак, наша первая задача: как преобразовать все обновления состояния файловой системы в серию операций последовательной записи на диск? Чтобы лучше понять, в чем тут дело, рассмотрим простой пример. Пусть требуется

записать блок данных D в файл. Результат записи блока данных на диск может выглядеть, как показано ниже (D записан по адресу $A0$):



Но ведь записью блока данных дело не ограничивается, нужно еще обновить **метаданные**. В этом случае запишем на диск также **индексный дескриптор** (I) файла, так чтобы он указывал на блок D . На диске блок данных и индексный дескриптор выглядят, как показано ниже (отметим, что дескриптор показан таким же большим, как блок данных, хотя, вообще говоря, это не так; в большинстве систем блок данных занимает 4 КБ, а индексный дескриптор около 128 байт):



Вот эта идея – просто записывать все обновления (блоки данных, индексные дескрипторы и т. д.) на диск последовательно – и легла в основу LFS. Если вы это понимаете, прекрасно. Но, как во всех сложных системах, дьявол кроется в деталях.

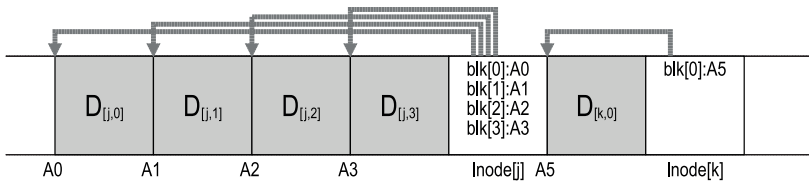
43.2. ЗАПИСЫВАТЬ ПОСЛЕДОВАТЕЛЬНО И ЭФФЕКТИВНО

К сожалению, одна лишь последовательная запись еще не гарантирует эффективности. Представим, к примеру, что в момент T мы записываем один блок по адресу A . Затем немножко ждем и записываем блок по адресу $A + 1$ (следующий по порядку), но в момент $T + \delta$. Увы, между первой и второй записями диск провернулся, поэтому, чтобы выполнить вторую запись, придется подождать, пока он совершит почти полный оборот (если время одного оборота $T_{rotation}$, то ждать придется $T_{rotation} - \delta$). Так что сами видите, что писать на диск последовательно недостаточно для достижения максимальной производительности; для этого нужно выполнять много операций записи одну за другой без перерывов (или одну длинную операцию записи).

Для достижения данной цели LFS использует старую как мир технику **буферизации записи**¹. Прежде чем записывать на диск, LFS сохраняет обновления в памяти, а, накопив достаточное число обновлений, записывает их на диск все сразу, добиваясь тем самым эффективного использования диска.

Большая порция обновлений, которую LFS записывает за один раз, называется **сегментом**. В информатике этот термин сильно перегружен, но здесь означает сравнительно большую порцию для группировки операций записи. Итак, при записи на диск LFS буферизует обновления в сегменте в памяти, а затем выводит весь сегмент одной операцией записи. Если сегмент достаточно велик, то запись будет эффективна.

В примере ниже LFS буферизует два набора обновлений в небольшом сегменте; на самом деле сегменты больше (несколько МБ). Первое обновление состоит из записи четырех блоков в файл j , второе – добавление одного блока в конец файла k . Затем LFS записывает весь сегмент из семи блоков на диск одной операцией. В результате блоки на диске будут расположены следующим образом:



43.3. Сколько буферизовать?

Тогда возникает следующий вопрос: сколько обновлений буферизовать перед записью на диск? Конечно, ответ зависит от диска, точнее от соотношения накладных расходов на позиционирование и скорости передачи; см. главу, посвященную FFS, где имеется подобный анализ.

Предположим, к примеру, что позиционирование (т. е. вращение и поиск) перед каждой записью занимает T_{position} секунд, а скорость передачи данных равна R_{peak} МБ/с. Сколько байтов должна буферизовать LFS перед записью на такой диск?

Будем считать, что при каждой записи мы несем фиксированные затраты на позиционирование. Вопрос: сколько байтов нужно записать, чтобы **амортизировать** эти затраты? Чем больше, тем лучше (это понятно) и тем ближе мы подойдем к максимальной пропускной способности.

¹ И вправду, трудно найти первоисточник этой идеи, потому что она, вероятно, многократно открывалась разными людьми на самых ранних этапах истории вычислительной техники. О преимуществах буферизации записи см. работу Solworth and Orji [SO90], а о ее потенциальных рисках – работу Mogul [M94].

Чтобы получить конкретный ответ, предположим, что записывается D МБ. Время записи этой порции данных (T_{write}) складывается из времени позиционирования и времени передачи блока размером D (D/R_{peak}):

$$T_{write} = T_{position} + \frac{D}{R_{peak}}. \quad (43.1)$$

Поэтому эффективная скорость записи ($R_{effective}$), т. е. объем данных, деленный на время его записи, равна:

$$R_{effective} = \frac{D}{T_{write}} = \frac{D}{T_{position} + \frac{D}{R_{peak}}}. \quad (43.2)$$

Мы хотим, чтобы эффективная скорость ($R_{effective}$) была близка к максимальной. Обозначим F отношение эффективной скорости к максимальной, $0 < F < 1$ (типичное значение F может составлять 0.9, или 90 %). Математически это записывается в виде $R_{effective} = F \times R_{peak}$.

Теперь можно решить это уравнение относительно D :

$$R_{effective} = \frac{D}{T_{position} + \frac{D}{R_{peak}}} = F \times R_{peak}; \quad (43.3)$$

$$D = F \times R_{peak} \times \left(T_{position} + \frac{D}{R_{peak}} \right); \quad (43.4)$$

$$D = (F \times R_{peak} \times T_{position}) + \left(F \times R_{peak} \times \frac{D}{R_{peak}} \right); \quad (43.5)$$

$$D = \frac{F}{1 - F} \times R_{peak} \times T_{position}. \quad (43.6)$$

Пусть время позиционирования диска равно 10 мс, а максимальная скорость передачи данных 100 МБ/с и требуется добиться эффективной пропускной способности, равной 90 % от максимальной ($F = 0.9$). В этом случае $D = 0.9/0.1 \times 100 \text{ МБ/с} \times 0.01 \text{ с} = 9 \text{ МБ}$. Посчитайте сами, каким должен быть размер буфера, чтобы еще сильнее приблизиться к максимальной пропускной способности, например 95 % или 99%.

43.4. ПРОБЛЕМА: НАХОЖДЕНИЕ ИНДЕКСНЫХ ДЕСКРИПТОРОВ

Чтобы понять, как найти индексный дескриптор в LFS, давайте вспомним, как ищется индексный дескриптор в типичной файловой системе Unix. Найдите дескрипторы в FFS и даже в старой файловой системе Unix легко, потому

что они организованы в виде массива, который находится в хорошо известном месте на диске.

Так, в старой файловой системе массив дескрипторов размещен по фиксированному адресу. Зная номер дескриптора и начальный адрес массива, мы можем вычислить адрес дескриптора, умножив его номер на размер и прибавив произведение к начальному адресу.

В FFS найти индексный дескриптор, зная его номер, немного труднее, поскольку таблица дескрипторов разделена на части, и в каждой группе цилиндров находится одна часть. Поэтому нужно знать размер каждой части таблицы и ее начальный адрес. А при наличии этой информации вычисления такие же простые, как и выше.

В LFS жизнь устроена сложнее. Почему? Да потому, что мы разбросали индексные дескрипторы по всему диску! Хуже того, мы никогда не перезаписываем данные (и метаданные) на месте, поэтому последняя версия дескриптора (та, что нам нужна) постоянно перемещается.

43.5. РЕШЕНИЕ ДАЕТ КОСВЕННОСТЬ: КАРТА ИНДЕКСНЫХ ДЕСКРИПТОРОВ

Чтобы решить проблему, проектировщики LFS ввели дополнительный **уровень косвенности** между номерами индексных дескрипторов и самими дескрипторами – структуру данных, названную **картой индексных дескрипторов (i-картой)**. Она позволяет получить дисковый адрес последней версии индексного дескриптора, зная его номер. Часто она реализуется в виде простого *массива*, в котором каждый элемент занимает 4 байта (дисковый адрес). При каждой записи индексного дескриптора на диск в i-карту записывается его новое положение.

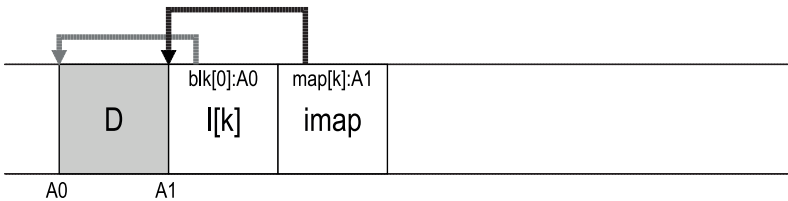
СОВЕТ: КОСВЕННОСТЬ – ЭТО ХОРОШО

Часто можно услышать, что любую проблему в информатике можно решить, добавив **уровень косвенности**. Это, конечно, неправда, так можно решить лишь *большинство* проблем (данное утверждение тоже слишком сильное, но мысль вы поняли). Все изученные нами виды виртуализации, в т. ч. виртуальная память и даже само понятие файла, – это просто дополнительный уровень косвенности. И понятно, что карта индексных дескрипторов в LFS – это виртуализация номеров индексных дескрипторов. Надеемся, что на этих примерах вы оценили мощь косвенности, которая позволяет свободно перемещать структуры данных (скажем, страницы ВП или дескрипторы LFS), не изменяя все ссылки на них. Разумеется, у косвенности есть и недостаток: **дополнительные накладные расходы**. Поэтому, когда в следующий раз столкнетесь с какой-нибудь проблемой, попробуйте решить ее с помощью косвенности, но не забудьте сначала обдумать, какие издержки это повлечет. Напомним знаменитое высказывание Уилера: «Все проблемы информатики можно решить с помощью дополнительного уровня косвенности, кроме, разумеется, проблемы слишком большого числа уровней косвенности».

К сожалению, *i*-карту нужно хранить постоянно (т. е. на диске), поскольку только так LFS не будет забывать местоположения индексных дескрипторов после аварии. И тогда возникает вопрос: а где на диске хранить *i*-карту?

Конечно, ее можно было разместить в каком-то фиксированном месте. Но поскольку она часто обновляется, это означало бы, что после каждого обновления файловых структур нужно было бы записывать в карту индексных дескрипторов, из-за чего пострадала бы производительность (между обновлением и записью по фиксированному адресу вклинивалась бы операция поиска).

Вместо этого LFS помещает части карты индексных дескрипторов рядом со всей остальной новой информацией. Например, при дописывании блока данных в конец файла *k* LFS записывает новый блок, его индексный дескриптор и кусок *i*-карты встык:



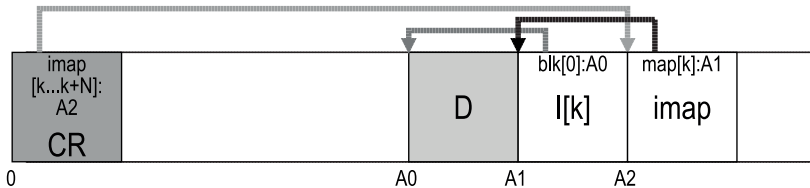
На этом рисунке кусок массива *i*-карты, хранящийся в блоке с меткой *imap*, сообщает LFS, что индексный дескриптор *k* находится на диске по адресу *A1*, а этот дескриптор, в свою очередь, сообщает LFS, что блок данных *D* находится по адресу *A0*.

43.6. Полное решение: область контрольной точки

Проницательный читатель (это ведь вы, правда?), вероятно, заметил проблему. Как найти карту индексных дескрипторов, части которой разбросаны по всему диску? В конце концов, волшебства-то не существует: файловая система должна начать поиск с *какого-то* фиксированного известного ей места на диске.

В LFS есть такое место: **область контрольной точки** (checkpoint region – **CR**). Она содержит указатели (т. е. адреса) последних частей карты индексных дескрипторов, поэтому для чтения этих частей нужно сначала прочитать CR. Заметим, что область контрольной точки обновляется периодически (примерно раз в 30 секунд), поэтому на производительность это почти не влияет. Таким образом, информация на диске хранится следующим образом: область контрольной точки (указывающая на последние фрагменты карты индексных дескрипторов); сами фрагменты карты индексных дескрипторов, которые содержат адреса дескрипторов; индексные дескрипторы, указывающие на файлы (и каталоги), как в типичной файловой системе Unix.

На рисунке ниже показана область контрольной точки (она расположена в начале диска, по адресу 0), один фрагмент *i*-карты, индексный дескриптор и блок данных. Конечно, в настоящей файловой системе CR значительно больше (на самом деле их даже две, но об этом мы поговорим позже), много фрагментов *i*-карты и гораздо больше индексных дескрипторов, блоков данных и т. д.



43.7. ЧТЕНИЕ ФАЙЛА С ДИСКА: ПОВТОРЕНИЕ ПРОЙДЕННОГО

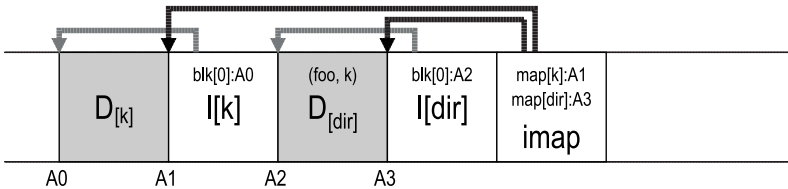
Чтобы вы лучше поняли, как работает LFS, разберем, что должно происходить при чтении файла с диска. Предположим, что в начальный момент в памяти ничего нет. Прежде всего мы должны прочитать с диска область контрольной точки. Она содержит указатели (дисковые адреса) на все части карты индексных дескрипторов, поэтому LFS читает всю карту и кеширует ее в памяти. После этого, получив номер индексного дескриптора файла, LFS находит в *i*-карте соответствие между этим номером и адресом дескриптора и читает последнюю версию дескриптора. Дальше, чтобы прочитать блок данных из файла, LFS действует точно так же, как типичная файловая система Unix, переходя по прямым, косвенным или дважды косвенным указателям. В типичном случае при чтении файла с диска LFS выполняет столько же операций ввода-вывода, сколько и типичная файловая система; вся *i*-карта кеширована в памяти, поэтому при чтении дополнительная работа LFS сводится только к поиску адреса индексного дескриптора в *i*-карте.

43.8. А КАК НАСЧЕТ КАТАЛОГОВ?

До сих пор мы немного упрощали обсуждение, рассматривая только индексные дескрипторы и блоки данных. Но чтобы найти файл в файловой системе (например, `/home/remzi/foo`, одно из наших любимых дурацких имен файлов), нужно обращаться и к каталогам. И как же хранятся данные каталогов в LFS?

По счастью, структура каталогов по сути дела такая же, как в классических файловых системах Unix, т. е. каталог – это просто совокупность отображений вида (имя, номер индексного дескриптора). При создании файла LFS должна записать новый индексный дескриптор, данные файла, а также данные ка-

талога и его индексный дескриптор, чтобы можно было сослаться на файл. Напомним, что LFS записывает все это последовательно (а перед тем какое-то время хранит обновления в буфере). Поэтому в результате создания файла `foo` на диске окажутся такие структуры:



Фрагмент карты индексных дескрипторов содержит информацию о местоположении каталога `dir` и вновь созданного файла `f`. Таким образом, для доступа к файлу `foo` (с номером индексного дескриптора k) мы сначала должны заглянуть в карту индексных дескрипторов (обычно она кеширована в памяти) и найти в ней местоположение дескриптора каталога `dir` (A3); затем мы должны прочитать дескриптор каталога, который даст нам местоположение данных каталога (A2); чтение этого блока данных даст соответствие между именем и индексным дескриптором (`foo, k`). Затем мы снова обращаемся к карте индексных дескрипторов и находим в ней местоположение дескриптора с номером k (A1). И наконец, читаем нужный нам блок данных по адресу A0.

В LFS существует еще одна серьезная проблема, которую решает карта индексных дескрипторов: **проблема рекурсивного обновления** [Z+12]. Она возникает в любой файловой системе, где обновление не производится по месту, а записывается в новое место на диске (как в LFS).

Именно, при каждом обновлении индексного дескриптора его положение на диске изменяется. При отсутствии должной осмотрительности это повлекло бы за собой обновление каталога, указывающего на данный файл, что вынудило бы нас изменить родителя этого каталога и т. д. вплоть до корня файловой системы.

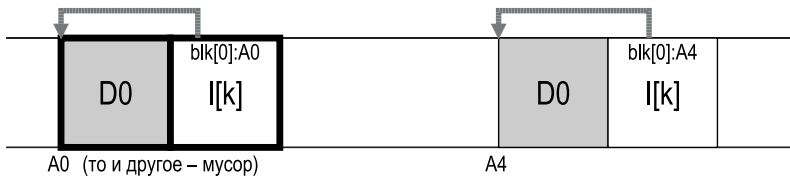
Благодаря карте индексных дескрипторов LFS мудро обходит эту проблему. Хотя местоположение индексного дескриптора может измениться, это изменение никогда не отражается в самом каталоге; обновляется лишь i -карта, тогда как в каталоге остается то же соответствие между именем и номером индексного дескриптора, что и раньше. Таким образом, косвенность позволяет LFS избежать проблемы рекурсивного обновления.

43.9. НОВАЯ ПРОБЛЕМА: СБОРКА МУСОРА

Вы, наверное, обратили внимание еще на одну проблему LFS; она раз за разом записывает последнюю версию файла (включая его индексный дескриптор и данные) в новые места на диске. Хотя это повышает эффектив-

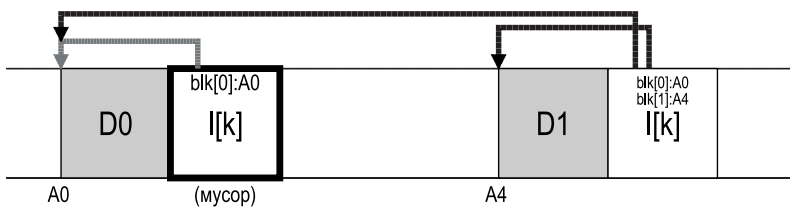
ность записи, получается, что старые версии файловых структур остаются и разбросаны по всему диску. Мы (довольно бесцеремонно) называем эти старые версии **мусором**.

Например, рассмотрим случай, когда существующий файл описывается индексным дескриптором с номером k , который указывает на единственный блок данных $D0$. Мы обновляем этот блок, что приводит к появлению нового индексного дескриптора и нового блока данных. В результате картина на диске будет следующей (для простоты мы опустили i -карту и другие структуры, а, по идее, нужно было бы записать на диск новый фрагмент i -карты, указывающий на новый индексный дескриптор):



Мы видим, что на диске присутствуют две версии индексного дескриптора и блока данных: старая (слева) и текущая, т. е. **живая** (справа). В результате простого логического обновления одного блока данных образовалось несколько новых структур, которые LFS обязана сохранить, а старые версии так и остались на диске.

Рассмотрим другой пример – дописывание блока в конец файла k . В этом случае порождается новая версия индексного дескриптора, но она указывает и на старый блок данных, который, стало быть, по-прежнему жив и является частью файловой системы:



Так что же нам делать с этими старыми версиями индексных дескрипторов, блоков данных и т. д.? Можно было бы хранить их, дав пользователям возможность восстановить старые версии (например, после случайной перезаписи или удаления файла это было бы ой как полезно); такие файловые системы называются **версионными**, потому что хранят разные версии файла.

Однако LFS хранит только последнюю версию файла, поэтому должна периодически (в фоновом режиме) находить старые, уже мертвые версии данных, индексных дескрипторов и других структур и **очищать** их, т. е. делать блоки доступными для использования при будущих операциях записи. Отметим, что процесс очистки является формой **сборки мусора** – метода, при-

меняемого в языках программирования для автоматического освобождения памяти, занятой программой.

Выше мы говорили о важности сегментов как механизма, обеспечивающего длинную запись на диск в LFS. Но они также неотъемлемая часть эффективной очистки. Подумайте, что было бы, если бы LFS просто пробегала по всему диску, удаляя одиночные блоки данных, индексные дескрипторы и т. д. В результате на диске образовались бы **дырки** между выделенными участками. Производительность записи резко упала бы, поскольку LFS не смогла бы найти длинного непрерывного участка, в который можно было писать последовательно.

Поэтому чистильщик, входящий в LFS, освобождает место на диске сегментами, так что образуются большие непрерывные области. Принцип работы процесса очистки следующий: периодически он читает несколько старых (частично используемых) сегментов, определяет, какие принадлежащие им блоки еще «живы», и записывает новый набор сегментов, содержащий только живые блоки, а старые освобождает для записи. Точнее, мы ожидаем, что чистильщик прочитает M существующих сегментов, **уплотнит** их содержимое, создав $N < M$ новых сегментов, и запишет эти N сегментов в новые места на диске. После этого M старых сегментов освобождаются и могут быть использованы файловой системой для записи.

Но остается две проблемы. Первая – механизм: откуда LFS знает, какие блоки в сегменте еще живы, а какие мертвы? Вторая – политика: как часто следует запускать чистильщик, и какие сегменты он должен выбирать для очистки?

43.10. Нахождение живых блоков

Сначала рассмотрим механизм. Для блока данных D в сегменте S LFS должна уметь определять, жив ли D . Для этого LFS включает в каждый сегмент дополнительную информацию, описывающую блоки, а именно для каждого блока D запоминается номер его индексного дескриптора (какому файлу он принадлежит) и смещение от начала файла. Эта информация хранится в начале сегмента в так называемом **сводном блоке сегмента**.

Имея эти сведения, легко определить, жив сегмент или нет. Для блока D , находящегося на диске по адресу A , найдем в сводном блоке сегмента номер его индексного дескриптора N и смещение T . Затем по i -карте определим, где на диске находится дескриптор с номером N , и прочитаем его (если он уже находится в памяти, тем лучше). Наконец, зная смещение T , заглянем в индексный дескриптор (или какой-то косвенный блок) и узнаем, где находится T -й блок этого файла на диске. Если дескриптор указывает точно на адрес A , то LFS может заключить, что блок D жив. Если же он указывает куда-то еще, значит, D не используется (мертв), т. е. эта версия блока больше не нужна. Ниже приведен псевдокод данного алгоритма:

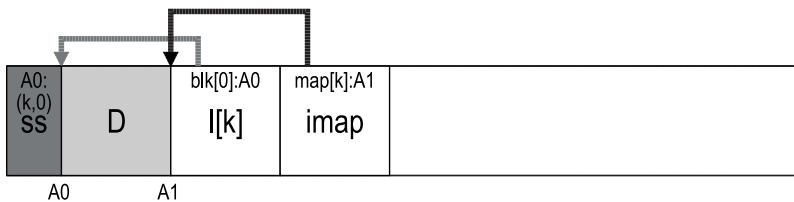
```
(N, T) = SegmentSummary[A];
inode = Read(imap[N]);
```



```

if (inode[T] == A)
    // блок D жив
else
    // блок D содержит мусор
    
```

На рисунке ниже изображен этот механизм: в сводном блоке сегмента (обозначен SS) хранится информация о том, что блок данных по адресу A0 является частью файла k и имеет в нем смещение 0. Заглянув в карту индексных дескрипторов, мы видим, что соответствующий дескриптор действительно указывает на этот блок.



Есть несколько приемов, благодаря которым процесс нахождения живых блоков можно сделать более эффективным. Например, при усечении (удалении всех данных) или удалении файла LFS увеличивает его **номер версии** и сохраняет новый номер в i -карте. Запоминая номер версии также в сегменте на диске, LFS может сократить описанную выше долгую проверку, просто сравнив номера версий на диске и в i -карте, это позволит обойтись без дополнительных операций чтения.

43.11. Политика: КАКИЕ БЛОКИ ОЧИЩАТЬ И КОГДА?

Над описанным выше механизмом LFS должна надстроить ряд политик для определения того, когда и какие блоки очищать. С тем, когда очищать, дело обстоит просто: либо периодически, во время простоя, либо когда диск заполнится.

А вот решить, какие блоки очищать, сложнее, и эта задача стала предметом многих научных работ. В оригинальной статье по LFS [RO91] авторы пытаются разделить сегменты на *горячие* и *холодные*. Горячим считается сегмент, который часто перезаписывается; такой сегмент лучше очищать не сразу, потому что все больше и больше блоков перезаписывается (и оказывается в новых сегментах), т. е. освобождается для повторного использования. Напротив, в холодном сегменте может быть всего несколько мертвых блоков, а остальное его содержимое относительно стабильно. Поэтому, делают вывод авторы, холодные сегменты лучше очищать поскорее, а горячие попозже, и предлагают соответствующую эвристику. Однако, как часто бывает, эта политика не идеальна, в более поздних работах показано, как ее можно улучшить [MR+97].

43.12. СТРУКТУРА ЖУРНАЛА И ВОССТАНОВЛЕНИЕ ПОСЛЕ АВАРИИ

И последняя проблема: что, если произойдет авария в тот момент, когда LFS пишет на диск? Как было сказано в предыдущей главе, посвященной журналированию, аварии во время обновления представляют сложность для файловых систем, поэтому и LFS должна уделить внимание этому вопросу.

В процессе нормальной работы LFS буферизует операции записи в сегменте, а затем (когда сегмент заполнен или прошло некоторое время) записывает весь сегмент на диск. LFS организует сегменты в виде **журнала** (log), т. е. область контрольной точки указывает на начальный и конечный сегменты, а каждый сегмент указывает на следующий, подлежащий записи. Кроме того, LFS периодически обновляет область контрольной точки. Очевидно, что аварийный отказ может произойти во время любой из этих операций (записи в сегмент, записи в CR). Как LFS должна обрабатывать отказы во время записи в эти структуры?

Рассмотрим сначала второй случай. Стремясь гарантировать атомарность обновления CR, LFS в действительности хранит две области контрольной точки, по одной на каждом конце диска, и записывает в них попеременно. LFS также реализует четкий протокол обновления CR при записи актуальных указателей на карту индексных дескрипторов и другой информации; именно, сначала записывается заголовок (содержащий временную метку), затем тело CR и, наконец, последний блок (тоже с временной меткой). Если система «падает» во время обновления CR, то LFS сможет это обнаружить, поскольку два временных штампа будут не согласованы. LFS всегда выбирает самую позднюю CR с согласованными временными метками, и таким образом достигается согласованное обновление CR.

Теперь рассмотрим первый случай. Поскольку LFS записывает CR один раз в 30 секунд (или около того), то последний снимок файловой системы может оказаться довольно старым. Во время перезагрузки LFS может легко восстановиться, прочитав область контрольной точки, фрагменты i-карты, на которую она указывает, и последующие файлы и каталоги, но при этом могут быть потеряны обновления за много последних секунд.

Чтобы улучшить ситуацию, LFS старается перестроить как можно больше сегментов, применяя технику, которую в сообществе баз данных называют **накатом**. Идея в том, чтобы, отправляясь от области последней контрольной точки, найти конец журнала (указатель на него хранится в CR), а затем читать следующие за ним сегменты и смотреть, есть ли в них действительные обновления. Если есть, то LFS соответственно обновляет файловую систему и таким образом восстанавливает значительную часть данных и метаданных, записанных с момента последней контрольной точки. Детали см. в удостоенной премии диссертации Розенблюма [R92].

43.13. РЕЗЮМЕ

LFS знаменует новый подход к обновлению информации на диске. Вместо того чтобы перезаписывать файлы на месте, LFS всегда пытается найти свободную часть диска, а позже освобождает уже не используемое место путем очистки. Этот подход, который в системах баз данных называется **механизмом теневых страниц** [L77], а в файловых системах – **копированием при записи**, позволяет записывать очень эффективно, поскольку LFS собирает все обновления в сегменте, находящемся в памяти, а затем записывает их последовательно.

Совет: обратить недостатки в достоинства

Если у системы имеется фундаментальный изъян, попробуйте обратить его себе на пользу. Файловая система WAFL компании NetApp именно так и поступает со старым содержимым файлов; делая старые версии доступными, WAFL может не так часто заниматься очисткой (хотя со временем все-таки удаляет старые версии, в фоновом режиме) – в результате одним махом и предлагает полезную возможность, и решает мучащую LFS проблему очистки. Если ли еще подобные примеры? Без сомнения, но вы уж подумайте об этом сами. Потому что эта глава закончилась. Всё. Амба. Капут. Покойся с миром!

Длинная запись, выполняемая LFS, очень хороша с точки зрения производительности на многих различных устройствах. Для жестких дисков длинная запись гарантирует минимизацию времени позиционирования, а для RAID-массивов с контролем четности, таких как RAID-4 и RAID-5, позволяет полностью избежать проблемы короткой записи. Недавние исследования даже показали, что длинные операции ввода-вывода необходимы для обеспечения высокой производительности на SSD-дисках [H+17], так что, как это ни странно, файловые системы в духе LFS могут оказаться отличным выбором и для этих новых носителей.

Недостаток данного подхода – порождение мусора; старые копии данных разбросаны по всему диску, и если требуется освободить это место для последующего использования, то нужно периодически очищать старые сегменты. Очистка – главный предмет споров по поводу LFS, а опасения в связи с ее потенциальной стоимостью [SS+95], наверное, встали на пути широкого внедрения LFS на начальном этапе. Однако в некоторых современных коммерческих файловых системах, включая NetApp WAFL [HLM94], Sun ZFS [B07] и Linux **btrfs** [R+13], и даже в современных SSD-дисках на базе технологии флеш-памяти [AD14] принят аналогичный подход к записи на диск, поэтому интеллектуальное наследие LFS живет и процветает. В частности, WAFL нашла способ решить проблему очистки, превратив ее в функциональную особенность: поскольку старые версии файловой системы доступны в виде **снимков**, пользователи получают возможность восстановить случайно удаленные файлы.

Литература

[AD14] «Operating Systems: Three Easy Pieces» (Chapter: Flash-based Solid State Drives) by Remzi Arpaci-Dusseau and Andrea Arpaci-Dusseau. Arpaci-Dusseau Books, 2014. *Пожалуй, неучтиво давать ссылку на другую главу этой же книги, но кто мы такие, чтобы судить?*

[B07] «ZFS: The Last Word in File Systems» by Jeff Bonwick and Bill Moore. Доступно по адресу http://www.ostep.org/Citations/zfs_last.pdf. *Слайды, посвященные ZFS; к сожалению, пока еще нет хорошей статьи на тему ZFS. Но, быть может, вы ее напишете, и тогда мы сможем поместить на нее ссылку.*

[H+17] «The Unwritten Contract of of Solid State Drives» by Jun He, Sudarsun Kannan, Andrea C. Arpaci-Dusseau, Remzi H. Arpaci-Dusseau. EuroSys '17, April 2017. *Каким неписаным правилам нужно следовать, чтобы добиться максимальной производительности от SSD-диска? Интересно, что даже для SSD важны масштаб запроса (длинные или параллельные запросы) и локальность. Чем больше вещи меняются...*

[HLM94] «File System Design for an NFS File Server Appliance» by Dave Hitz, James Lau, Michael Malcolm. USENIX Spring '94. *WAFL заимствует немало идей у LFS и RAID и воплощает их в сетевом устройстве на основе NFS, производимом компанией NetApp, стоящей миллиарды долларов.*

[L77] «Physical Integrity in a Large Segmented Database» by R. Lorie. ACM Transactions on Databases, Volume 2:1, 1977. *Идея механизма теневого страниц впервые предложена в этой статье.*

[MJLF84] «A Fast File System for UNIX» by Marshall K. McKusick, William N. Joy, Sam J. Leffler, Robert S. Fabry. ACM TOCS, Volume 2:3, August 1984. *Оригинальная статья по FFS; детали см. в главе, посвященной FFS.*

[MR+97] «Improving the Performance of Log-structured File Systems with Adaptive Methods» by Jeanna Neeffe Matthews, Drew Roselli, Adam M. Costello, Randolph Y. Wang, Thomas E. Anderson. SOSP 1997, pages 238-251, October, Saint Malo, France. *Относительно недавняя статья, в которой описаны усовершенствованные политики очистки в LFS.*

[M94] «A Better Update Policy» by Jeffrey C. Mogul. USENIX ATC '94, June 1994. *В этой статье Могул обнаружил, что если буферизовать записи в течение слишком долгого времени, а затем передавать их диску одной большой порцией, то может снизиться производительность чтения. Поэтому он рекомендует производить запись чаще и небольшими порциями.*

[P98] «Hardware Technology Trends and Database Opportunities» by David A. Patterson. ACM SIGMOD '98 Keynote, 1998. Доступно по адресу <http://www.cs.berkeley.edu/~pattsrn/talks/keynote.html>. *Комплект слайдов на тему тенденций развития компьютерных технологий. Надеемся, что Паттерсон в скорости подготовит еще один такой комплект.*

[R+13] «BTRFS: The Linux B-Tree Filesystem» by Ohad Rodeh, Josef Bacik, Chris Mason. ACM Transactions on Storage, Volume 9 Issue 3, August 2013. *Наконец-то*

хорошая статья по BTRFS, современному варианту файловой системы с копированием при записи.

[RO91] «Design and Implementation of the Log-structured File System» by Mendel Rosenblum and John Ousterhout. SOSP '91, Pacific Grove, CA, October 1991. *Оригинальная работа о LFS, доложенная на симпозиуме по принципам операционных систем (SOSP). Цитировалась в сотнях других работ и стала источником вдохновения для многих реальных систем.*

[R92] «Design and Implementation of the Log-structured File System» by Mendel Rosenblum. <http://www.eecs.berkeley.edu/Pubs/TechRpts/1992/CSD-92-696.pdf>. *Удостоенная премии диссертация на тему LFS, в которой восполнены многочисленные детали, описанные в докладе.*

[SS+95] «File system logging versus clustering: a performance comparison» by Margo Seltzer, Keith A. Smith, Hari Balakrishnan, Jacqueline Chang, Sara McMains, Venkata Padmanabhan. USENIX 1995 Technical Conference, New Orleans, Louisiana, 1995. *В этой работе показано, что у LFS имеются проблемы с производительностью, в частности для рабочих нагрузок с большим числом обращений к fsync() (характерных для баз данных). В свое время статья вызвала много споров.*

[SO90] «Write-Only Disk Caches» by Jon A. Solworth, Cyril U. Orji. SIGMOD '90, Atlantic City, New Jersey, May 1990. *Раннее исследование буферизации записи и его преимуществ. Однако буферизация в течение слишком длительного времени может быть опасной; см. работу Могула [M94].*

[Z+12] «De-indirection for Flash-based SSDs with Nameless Writes» by Yiyang Zhang, Leo Prasath Arulraj, Andrea C. Arpaci-Dusseau, Remzi H. Arpaci-Dusseau. FAST '13, San Jose, California, February 2013. *Наша работа о новом способе построения запоминающих устройств на основе флеш-памяти, который позволяет избежать лишних соответствий в файловой системе и FTL. Идея в том, чтобы устройство получало физический адрес записи и возвращало его файловой системе, которая сохраняет соответствие.*

Домашнее задание (эмуляция)

В этом домашнем задании вы будете работать с программой `lfs.py`, простым эмулятором LFS, позволяющим лучше понять принципы работы LFS-подобных файловых систем. О параметрах эмулятора см. файл README.

Вопросы

1. Выполните команду `./lfs.py -n 3`, возможно, изменив начальное значение (-s). Сможете ли вы понять, какие команды выполнялись для генерирования конечного состояния файловой системы? А в каком порядке? Наконец, попробуйте для каждого блока в конечном состоянии определить, жив он или мертв. Чтобы увидеть, какие команды выполнялись,

задайте флаг `-o`, а чтобы увидеть конечное состояние блоков – флаг `-c`. Насколько труднее станет эта задача, если увеличить количество выполненных команд (т. е. вместо `-n 3` задать `-n 5`)?

2. Если предыдущая задача показалась вам трудной, то можете немного помочь себе, попросив программу показать набор обновлений, индуцированных каждой командой. Для этого запустите программу так: `./lfs.py -n 3 -i`. Проще ли стало понять, какие команды выполнялись? Измените начальное значение генератора случайных чисел, чтобы получить и интерпретировать разные результаты (например, `-s 1`, `-s 2`, `-s 3` и т. д.).
3. Продолжим опыты на тему определения того, к каким обновлениям привела каждая команда. Наберите `./lfs.py -o -F -s 100` (а потом, быть может, запустите с другими начальными значениями). Будет показан только набор команд, но не показано конечное состояние файловой системы. Можете ли вы сказать, каким должно быть это состояние?
4. Теперь посмотрим, сможете ли определить, какие файлы и каталоги остались в живых после ряда операций. Запустите `tt ./lfs.py -n 20 -s 1` и изучите конечное состояние файловой системы. Какие имена путей допустимы? Запустите `tt ./lfs.py -n 20 -s 1 -c -v`, чтобы увидеть результаты. Запускайте программу с флагом `-o`, чтобы проверить свои ответы для серии случайных команд. Если хотите получить дополнительные задачи, запускайте с другими начальными значениями.
5. Теперь выполните несколько конкретных команд. Для этого вам понадобится флаг `-L`, который позволяет выполнять конкретные команды. Создайте файл `/foo` и четыре раза запишите в него: `-L c, /foo:w, /foo,0,1:w, /foo,1,1:w, /foo,2,1:w, /foo,3,1 -o`. Можете ли вы сказать, какие блоки живы в конечном состоянии файловой системы; для проверки своего ответа задайте флаг `-c`.
6. Сделайте то же самое, но вместо четырех операций записи оставьте одну. Выполните `./lfs.py -o -L c, /foo:w, /foo,0,4`, чтобы создать файл `/foo` и записать в него четыре блока одной операцией. Снова определите, какие блоки живы, и проверьте себя с помощью флага `-c`. В чем основное различие между записью всего файла сразу (как здесь) и по одному блоку (как в предыдущем упражнении)? Что это говорит нам о важности буферизации обновлений в памяти, как то делает реальная LFS?
7. Рассмотрим еще один конкретный пример. Сначала запустите `./lfs.py -L c, /foo:w, /foo,0,1`. Что делает этот набор команд? Теперь запустите `./lfs.py -L c, /foo:w, /foo,7,1`. Что делают эти команды? В чем различие? Что вы можете сказать о поле `size` в индексном дескрипторе после выполнения этих двух наборов команд?
8. Теперь сравним создание файла и создание каталога. Выполните команды `./lfs.py -L c, /foo` и `./lfs.py -L d, /foo`, чтобы создать файл, а затем каталог. Что общего между этими запусками, и чем они различаются?
9. Эмулятор LFS поддерживает также жесткие ссылки. Выполните следующую команду, чтобы понять, как они работают: `./lfs.py -L c, /foo:l, /foo, /bar:l, /foo, /goo -o -i`. Какие блоки записываются при создании жесткой ссылки? Чем это похоже на создание файла, а чем отличается? Как изменяется поле счетчика ссылок при создании жестких ссылок?

10. LFS принимает много решений и политик. Мы не будем здесь исследовать их подробно – быть может, когда-нибудь в будущем, – но одно решение все же рассмотрим: выбор номера индексного дескриптора. Сначала выполните `./lfs.py -p c100 -n 10 -o -a s`, чтобы продемонстрировать обычное поведение – «последовательную» политику выделения, которая пытается использовать свободные номера индексных дескрипторов, ближайшие к нулю. Затем задайте «произвольную» политику, выполнив `./lfs.py -p c100 -n 10 -o -a g` (флаг `-p c100` говорит, что 100 процентов случайных операций – создание файлов). Как выбор последовательной или произвольной политики отражается на состоянии диска? Что это говорит о важности выбора номера индексного дескриптора в реальной LFS?
11. Мы предполагали, что эмулятор LFS обновляет область контрольной точки после каждого обновления. В реальной LFS это не так: эта область обновляется периодически, чтобы избежать долгих операций поиска. Выполните `./lfs.py -N -i -o -s 1000`, чтобы увидеть некоторые операции, а также промежуточные и конечные состояния файловой системы, когда нет форсированной записи области контрольной точки на диск. Что будет, если область контрольной точки никогда не обновляется? А если она обновляется периодически? Попробуйте разобраться в том, как восстановить файловую систему в последнем состоянии, накатив журнал.

Глава 44

SSD-диски на основе флеш-памяти

После многих десятилетий доминирования жестких дисков в мир пришла и широко распространилась новая форма постоянных запоминающих устройств. Эти устройства, получившие общее название **твердотельных запоминающих устройств**¹, не имеют механических подвижных частей, как жесткие диски; они целиком построены на базе транзисторов, как память и процессоры. Но, в отличие от типичного запоминающего устройства с произвольной выборкой (например, ДЗУПВ, англ. *DRAM*), **твердотельное запоминающее устройство (SSD)** сохраняет информацию после отключения питания, так что является идеальным кандидатом для постоянного хранения данных.

Нас будет в первую очередь интересовать технология **флеш-памяти** (точнее, **флеш-памяти типа NAND**), которая была разработана Фудзио Масуока в 1980-х годах [М+14]. Как мы увидим, флеш-память обладает рядом уникальных свойств. Например, чтобы записать некоторый участок памяти (**флеш-страницу**), нужно сначала стереть больший участок (**флеш-блок**), что может стоить очень дорого. Кроме того, если слишком часто записывать страницу, то она **изнашивается**. Из-за этих двух свойств конструирование SSD на базе технологии флеш-памяти оказывается интересной и трудной задачей.

СУЩЕСТВО ПРОБЛЕМЫ: КАК ПОСТРОИТЬ SSD НА БАЗЕ ФЛЕШ-ПАМЯТИ

Как создать SSD на базе флеш-памяти? Как справиться с принципиально дорогостоящим стиранием? Как создать устройство, которое будет служить долго, притом что многократная перезапись приводит к износу? Прекратится ли когда-нибудь технологический прогресс? Или он перестанет удивлять?

¹ Правильнее было бы говорить «полупроводниковые», но перевод «твердотельные» уже прижился, и с этим ничего не поделаешь. – *Прим. перев.*

44.1. СОХРАНЕНИЕ ОДНОГО БИТА

Микросхемы флеш-памяти проектируются так, что в одном транзисторе хранится один или несколько битов; уровень захваченного транзистором заряда отображается в двоичное значение. В технологии флеш-памяти с **одноуровневыми ячейками** (single-level cell – **SLC**) в транзисторе хранится только один бит (1 или 0), а в памяти с **многоуровневыми ячейками** (multi-level cell – **MLC**) два бита кодируются разными уровнями заряда, т. е. комбинации 00, 01, 10, 11 представляются соответственно низким, пониже, повыше и высоким уровнями. Существует даже флеш-память с **трехуровневыми ячейками** (triple-level cell – **TLC**), в которой одна ячейка кодирует три бита. В общем и целом SLC-память обеспечивает более высокую производительность и стоит дороже.

СОВЕТ: ОСТОРОЖНЕЕ С ТЕРМИНОЛОГИЕЙ

Вы, наверное, заметили, что некоторые термины, много раз встречавшиеся прежде (блоки, страницы), теперь используются в контексте флеш-памяти, но несколько в ином смысле, чем раньше. Новые термины создаются не для того, чтобы усложнить вам жизнь (хотя именно так, возможно, и получается), а потому, что нет никакого центрального органа, отвечающего за терминологию. Что кажется блоком вам, кому-то другому естественнее называть страницей, и наоборот; все зависит от контекста. Ваше дело простое: знать термины, употребляемые в каждой предметной области, и использовать их так, чтобы люди, разбирающиеся в предмете, поняли, о чем вы говорите. Это один из тех случаев, когда есть единственное решение – простое, но иногда болезненное – пользоваться своей памятью.

Разумеется, есть много деталей, касающихся принципов работы таких запоминающих устройств, оперирующих битами, – вплоть до физического уровня. Они выходят за рамки этой книги, но при желании можете почитать самостоятельно [J10].

44.2. ОТ БИТОВ К БАНКАМ И ПЛОСКОСТЯМ

Как говаривали еще в Древней Греции, запоминание одного (или нескольких) битов еще не делает систему хранения. Поэтому микросхемы флеш-памяти организованы в виде **банков**, или **плоскостей**, состоящих из большого числа ячеек.

Из банка можно получать единицы двух размеров: блоки (иногда их называют **стираемыми блоками**), обычно размером 128 или 256 КБ, и страницы размером несколько килобайтов (например, 4 КБ). В каждом банке много блоков, а в каждом блоке – много страниц. В разговорах о флеш-памяти помните об этой новой терминологии, в которой смысл слова «блок» не такой, как в дисках или RAID-массивах, а «страница» означает не то же, что в системах виртуальной памяти.

На рис. 44.1 показан пример плоскости флеш-памяти, состоящей из блоков и страниц: имеется три блока по четыре страницы каждый. Ниже мы увидим, зачем это разделение на блоки и страницы; оказывается, оно принципиально важно для таких операций, как чтение и запись, и еще важнее для обеспечения производительности устройства. Самая главная (и загадочная) вещь – то, что для записи на страницу внутри блока сначала необходимо стереть весь блок, из-за этой замысловатой детали построение SSD на базе флеш-памяти оказывается интересной и трудной задачей, которой посвящена вторая часть этой главы.

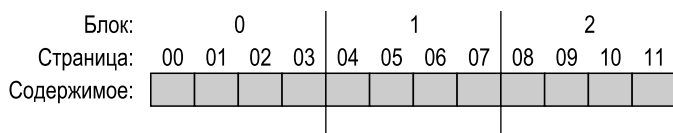


Рис. 44.1 ❖ Организация простой флеш-памяти: страницы с несколькими блоками

44.3. Основные операции с флеш-памятью

Флеш-память поддерживает три низкоуровневые операции. Команда **чтения** позволяет прочитать одну страницу флеш-памяти, а команды **стирания** и **программирования** совместно используются для записи. Опишем их подробнее.

- **Читать (страницу).** Клиент может прочитать любую страницу (например, 2 или 4 КБ), выполнив команду чтения и указав номер страницы. Обычно команда занимает десятки микросекунд независимо от места страницы внутри устройства и почти независимо от адреса, указанного в предыдущем запросе (в отличие от диска). Поскольку время доступа не зависит от местоположения данных, флеш-память является устройством с **произвольной выборкой**.
- **Стереть (блок).** Прежде чем записать *страницу* во флеш-память, необходимо стереть *весь блок*, в котором эта страница находится, – такова природа этого устройства. Важно, что операция стирания уничтожает содержимое блока (все биты устанавливаются в 1), поэтому нужно предварительно куда-то скопировать представляющие интерес данные (в память или, возможно, в другой блок флеш-памяти). Команда стирания обходится дорого – занимает несколько миллисекунд. По ее завершении весь блок сброшен в начальное состояние и готов к программированию.
- **Программировать (страницу).** После стирания блока можно выполнить команду программирования, чтобы заменить часть единиц на странице нулями и тем самым записать на страницу нужные данные. Программирование страницы стоит дешевле, чем стирание блока, но все равно дороже, чем чтение страницы, – в современных устройствах оно занимает сотни микросекунд.

Можно считать, что с каждой страницей флеш-памяти ассоциировано состояние. Изначально любая страница находится в состоянии INVALID. Стерев блок, которому принадлежит страница, мы переводим ее (и все остальные страницы в этом блоке) в состояние ERASED, в котором содержимое страницы сброшено в начальное состояние и (это очень важно) страница допускает программирование. В процессе программирования страница переходит в состояние VALID, т. е. ее содержимое установлено и может быть прочитано. Операции чтения никак не влияют на состояние (но читать можно только запрограммированные страницы). После того как страница запрограммирована, изменить ее содержимое можно только одним способом – стереть блок, которому она принадлежит. Ниже приведен пример перехода состояний после различных операций стирания и программирования в 4-страничном блоке:

	iiii	<i>начальное: страницы в блоке недействительны (i)</i>
Erase()	→ EEEE	<i>состояние страниц в блоке изменено на ERASED (E)</i>
Program(0)	→ VEEE	<i>программировать страницу 0; новое состояние VALID (V)</i>
Program(0)	→ ошибка	<i>нельзя перепрограммировать страницу после программирования</i>
Program(1)	→ VVEE	<i>программировать страницу 1</i>
Erase()	→ EEEE	<i>содержимое стерто; все страницы допускают программирование</i>

Подробный пример

Поскольку процесс записи (стирание и программирование) настолько необычен, разберем подробный пример, чтобы вы лучше поняли происходящее. Будем считать, что имеется четыре 8-битовые страницы, входящие в состав 4-страничного блока (размеры гораздо меньше реальных, но для примера так даже лучше); состояние всех страниц VALID, потому что они уже были запрограммированы ранее.

Страница 0	Страница 1	Страница 2	Страница 3
00011000	11001110	00000001	00111111
VALID	VALID	VALID	VALID

Допустим, что требуется записать на страницу 0, изменив ее содержимое. Для записи любой страницы мы должны сначала стереть весь блок. После этого блок окажется в таком состоянии:

Страница 0	Страница 1	Страница 2	Страница 3
11111111	11111111	11111111	11111111
ERASED	ERASED	ERASED	ERASED

Прекрасно! Теперь можно пойти дальше и запрограммировать страницу 0, например записать в нее 00000011 (вместо 00011000). После этого блок будет выглядеть следующим образом:

Страница 0	Страница 1	Страница 2	Страница 3
00000011	11111111	11111111	11111111
VALID	ERASED	ERASED	ERASED

А теперь плохие новости: предыдущее содержимое страниц 1, 2 и 3 пропало! Поэтому, перед тем как перезаписывать любую страницу *внутри* блока, нужно переместить все ценные данные в другое место (например, в основную память или в другой блок флеш-памяти). Природа стирания оказывает сильное влияние на проектирование SSD на основе флеш-памяти, к чему мы скоро перейдем.

Резюме

Подведем итоги. Чтение страницы не вызывает никаких трудностей. Флеш-память прекрасно с этим справляется, а скорость во много раз больше, чем скорость произвольного чтения в современных жестких дисках, которое замедляется из-за поиска и вращения.

С записью страницы сложнее; нужно сначала стереть весь блок (позаботившись о том, чтобы переместить данные в другое место), а затем запрограммировать требуемую страницу. Мало того что это дорого, так еще частое повторение цикла программирование–стирание порождает серьезную проблему надежности флеш-памяти: **износ**. При проектировании системы хранения на флеш-памяти производительность и надежность находятся в центре внимания. Скоро мы узнаем, как решаются эти проблемы в современных SSD, которые отличаются высокими характеристиками производительности и надежности, несмотря на указанные ограничения.

44.4. Производительность и надежность флеш-памяти

Поскольку нас интересует создание запоминающего устройства из микросхем флеш-памяти, хорошо было бы понимать исходные характеристики производительности. На рис. 44.2 показаны приблизительные цифры, взятые из популярного издания [V12]. Автор приводит задержки операций чтения, программирования и стирания для микросхем типа SLC, MLC и TLC, позволяющих хранить 1, 2 и 3 бита информации в одной ячейке соответственно.

Как видно из таблицы, задержки чтения выглядят прекрасно, всего десятки микросекунд. Задержки программирования выше, и их разброс шире:

начинаются с 200 мкс для SLC, но растут вместе с количеством битов, упакованных в одну ячейку; для получения хорошей производительности придется использовать несколько микросхем флеш-памяти параллельно. Наконец, стирание обходится очень дорого – порядка нескольких миллисекунд. Центральная задача проектирования современных устройств на базе флеш-памяти – борьба с этой дороговизной.

Устройство	Чтение (мкс)	Программирование (мкс)	Стирание (мкс)
SLC	25	200–300	1500–2000
MLC	50	600–900	~3000
TLC	~75	~900–1350	~4500

Рис. 44.2 ❖ Характеристики аппаратной производительности флеш-памяти

Теперь обратимся к надежности флеш-памяти. В отличие от механических дисков, которые могут отказать по самым разным причинам (включая печальную и самую что ни на есть физическую **аварию головки**, когда головка диска касается его поверхности), флеш-память состоит только из кремния, поэтому не так уязвима. Главная беда – **износ**; по мере того как блок флеш-памяти стирается и программируется, в нем медленно накапливается избыточный заряд. Со временем из-за этого становится трудно отличить 0 от 1. Когда это оказывается вообще невозможно, блок становится бесполезным.

В настоящее время типичный срок службы блока неизвестен. Производители MLC-памяти оценивают его в 10 000 циклов стирания-программирования (P/E циклов). Для SLC-памяти, в которой хранится один бит на транзистор, срок службы дольше – обычно 100 000 P/E циклов. Но недавние исследования показали, что реальный срок службы гораздо больше ожидаемого [BD10].

Еще одна проблема надежности флеш-памяти – **интерференция**. При доступе к конкретной странице может случиться, что некоторые биты в соседних страницах меняют состояние; такие инверсии называются **интерференцией чтения** или **интерференцией программирования** в зависимости от того, на каком этапе они произошли.

СОВЕТ: ВАЖНОСТЬ ОБРАТНОЙ СОВМЕСТИМОСТИ

Обратная совместимость всегда должна учитываться в многоуровневых системах. Определив стабильный интерфейс между двумя системами, мы можем модифицировать любую его сторону, не препятствуя взаимодействию систем. Такой подход снискал успех во многих предметных областях: операционные системы предлагают относительно стабильный API приложениям, диски предоставляют единый блочный интерфейс файловым системам, а каждый уровень сетевого стека IP предлагает фиксированный и неизменный интерфейс вышестоящему уровню.

Понятно, что у такой негибкости могут быть и недостатки, потому что интерфейсы, определенные для одного поколения аппаратного или программного обеспечения, могут быть непригодны для следующего. В некоторых случаях полезно подумать о полном перепроектировании всей системы. Отличный пример дает файловая система Sun ZFS [B07]; переосмыслив взаимодействие файловых систем с RAID-массивами, создатели ZFS узрели (а затем и реализовали) более эффективное целое.

44.5. От голой флеш-памяти к SSD НА ЕЕ ОСНОВЕ

Итак, мы поняли, как устроены микросхемы флеш-памяти, и можем перейти к следующей задаче: как превратить базовый набор микросхем в нечто похожее на типичное запоминающее устройство. Стандартный интерфейс систем хранения блочный, предполагающий, что блоки (сектора) размером 512 байт (или больше) можно читать и записывать, зная только номер блока. Задача SSD на базе флеш-памяти – предоставить стандартный блочный интерфейс поверх составляющих микросхем.

На внутреннем уровне SSD состоит из нескольких микросхем флеш-памяти для долговременного хранения. SSD также включает сколько-то энергозависимой памяти (например, SRAM), которая используется для кеширования и буферизации данных, а также для таблиц отображения, о которых мы расскажем ниже. Наконец, SSD содержит логические схемы для управления устройством в целом. Дополнительные сведения см. в работе Agrawal et. al [A+08], а упрощенная блок-схема показана на рис. 44.3.

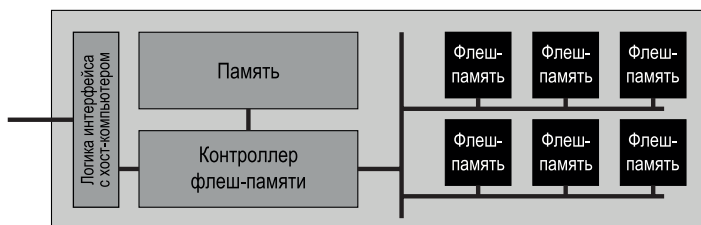


Рис. 44.3 ❖ SSD на основе флеш-памяти: логическая диаграмма

Одна из основных функций управляющей логики – удовлетворять запросы чтения и записи со стороны клиентов, преобразуя их во внутренние операции флеш-памяти. Именно этим занимается **уровень трансляции флеш-памяти** (flash translation layer – **FTL**). FTL принимает запросы чтения и записи *логических блоков* (составляющих интерфейс устройства) и преобразует их в низкоуровневые команды чтения, стирания и программирования *физических блоков* и *физических страниц* (составляющих само устройство). При этом FTL должен обеспечивать высокую производительность и надежность.

Как мы увидим, для достижения высокой производительности можно применить сочетание разных методов. Один из главных – работать с несколькими микросхемами флеш-памяти **параллельно**; мы не будем обсуждать эту технику подробно, но скажем, что во всех современных SSD для достижения высокой производительности применяется несколько микросхем. Еще одна цель – снизить **коэффициент расширения записи** (write amplification), который определяется как полное количество байтов, переданных микросхемам флеш-памяти уровнем FTL, поделенное на количество байтов, переданных клиентом SSD. Как мы увидим ниже, наивные подходы к конструи-

рованию FTL ведут к высокому коэффициенту расширения записи и низкой производительности.

Высокая надежность достигается комбинированием нескольких подходов. Главная проблема, как уже было сказано, – **износ**. Блок, который стирается и программируется слишком часто, становится бесполезным, поэтому FTL должен распределять операции записи между блоками флеш-памяти как можно более равномерно, так чтобы все блоки изнашивались примерно одинаково. Эта техника называется **выравниванием износа** и является важной частью современного FTL.

С надежностью связана еще одна важная проблема – интерференция программирования. Для ее минимизации FTL обычно программирует страницы, принадлежащие стертому блоку, *по порядку* – от младшей к старшей.

44.6. Организация FTL: неправильный подход

Простейшую организацию FTL можно было бы назвать **прямым отображением**. В этом случае чтение логической страницы N напрямую отображается в чтение физической страницы N . С записью логической страницы N дело обстоит сложнее; FTL должен сначала прочитать весь блок, которому принадлежит страница N , затем стереть этот блок и, наконец, запрограммировать старые и новую страницу.

Как вы, наверное, догадываетесь, FTL с прямым отображением имеет много проблем в плане как производительности, так и надежности. Первые проявляются при каждой записи: устройство должно прочитать блок целиком (дорого), стереть его (очень дорого), а затем запрограммировать (дорого). В результате коэффициент расширения записи (пропорциональный числу страниц в блоке) оказывается очень велик, а скорость записи ужасающая, даже медленнее, чем у типичного жесткого диска с его задержками поиска и вращения.

Еще печальнее дело обстоит с надежностью. Если метаданные файловой системы и пользовательские файлы постоянно перезаписываются, то один и тот же блок будет стираться и программироваться снова и снова, что быстро приведет к его износу и может закончиться потерей данных. Прямое отображение оставляет слишком много контроля над износом на волю клиентской рабочей нагрузки; если та не распределена равномерно по логическим блокам, то физические блоки, содержащие востребованные данные, будут быстро изнашиваться. Так что и с точки зрения производительности, и с точки зрения надежности FTL с прямым отображением – никуда не годная идея.

44.7. FTL со структурой журнала

По вышеуказанным причинам современные FTL имеют **структуру журнала**, эта идея полезна при реализации как самих запоминающих устройств (ниже мы в этом убедимся), так и построенных поверх них файловых систем (это мы увидим в главе, посвященной **файловым системам со структурой**

журнала). Получив запрос записи в логический блок N , устройство производит запись в свободное место в текущем записываемом блоке; такой стиль записи называется **протоколированием**. Чтобы можно было впоследствии читать из блока N , устройство хранит **таблицу отображения** (в своей памяти и в каком-то виде на самом устройстве); в этой таблице хранятся физические адреса всех логических блоков в системе.

Разберем пример, чтобы лучше понять, как работает этот подход. Клиенту устройство кажется типичным диском, с которого можно читать и на который можно записывать секторами по 512 байт (или группами таких секторов). Для простоты будем считать, что клиент читает и записывает порциями по 4 КБ. Далее предположим, что SSD содержит много блоков размером 16 КБ, каждый из которых разбит на четыре 4-килобайтовые страницы; это нереалистичные параметры (блоки флеш-памяти содержат гораздо больше страниц), но в педагогических целях удобно.

Пусть клиент выполняет такую последовательность операций:

- Write(100), содержимое a_1 ;
- Write(101), содержимое a_2 ;
- Write(2000), содержимое b_1 ;
- Write(2001), содержимое b_2 .

Эти **адреса логических блоков** (например, 100) клиент SSD (например, файловая система) использует, чтобы запомнить, где находится информация.

Устройство должно преобразовать эти операции записи блоков в операции стирания и программирования, поддерживаемые оборудованием, и каким-то образом для каждого адреса логического блока запомнить, в какой **физической странице** SSD находятся его данные. Предположим, что в настоящий момент все блоки SSD недействительны и должны быть стерты, прежде чем запрограммировать любую страницу. Ниже показано начальное состояние SSD, где все страницы имеют состояние INVALID (i):

Блок:	0				1				2			
Страница:	00	01	02	03	04	05	06	07	08	09	10	11
Содержимое:												
Состояние:	i	i	i	i	i	i	i	i	i	i	i	i

Когда SSD получает первый запрос записи (в логический блок 100), FTL решает записать данные в физический блок 0, содержащий четыре физические страницы: 0, 1, 2, 3. Поскольку блок не стерт, мы не можем в него писать; сначала устройство должно выполнить команду стирания блока 0. После этого возникнет следующее состояние:

Блок:	0				1				2			
Страница:	00	01	02	03	04	05	06	07	08	09	10	11
Содержимое:												
Состояние:	E	E	E	E	i	i	i	i	i	i	i	i

Теперь блок 0 готов к программированию. Большинство SSD записывают страницы по порядку (от младших к старшим), снижая риск **интерференции программирования**. Поэтому SSD направляет запись в логический блок 100 на физическую страницу 0:

Блок:	0				1				2			
Страница:	00	01	02	03	04	05	06	07	08	09	10	11
Содержимое:	a1											
Состояние:	V	E	E	E	i	i	i	i	i	i	i	i

Но что, если клиент захочет *прочитать* логический блок 100? Как он найдет его? SSD должен преобразовать запрос чтения из логического блока 100 в команду чтения физической страницы 0. Для этого FTL запоминает тот факт, что логический блок 100 был записан в физическую страницу 0, в **таблице отображения в памяти**. Мы будем отслеживать состояние таблицы отображения на рисунках:

Таблица:	100 → 0												Память
Блок:	0				1				2				Микросхема флеш-памяти
Страница:	00	01	02	03	04	05	06	07	08	09	10	11	
Содержимое:	a1												
Состояние:	V	E	E	E	i	i	i	i	i	i	i	i	

Теперь вы видите, что происходит, когда клиент записывает на SSD. SSD находит место для записи – обычно просто выбирает следующую свободную страницу, затем программирует эту страницу, перенося на нее содержимое блока, и запоминает соответствие между логическим блоком и физической страницей в таблице отображения. Последующие операции чтения пользуются этой таблицей для **трансляции** адреса логического блока, переданного клиентом, в номер физической страницы, где хранятся данные.

Рассмотрим другие операции записи: 101, 2000 и 2001. После записи этих блоков состояние устройства будет таким:

Таблица:	100 → 0 101 → 1 2000 → 2 2001 → 3												Память
Блок:	0				1				2				Микросхема флеш-памяти
Страница:	00	01	02	03	04	05	06	07	08	09	10	11	
Содержимое:	a1	a2	b1	b2									
Состояние:	V	V	V	V	i	i	i	i	i	i	i	i	

Подход на основе структуры журнала по самой своей природе улучшает производительность (стирание необходимо только изредка, а дорогостоящие

циклы чтение–модификация–запись, свойственные прямому отображению, вообще отсутствуют) и значительно повышает надежность. Теперь FTL может распределять запись по всем страницам и тем самым **выравнивать износ**, продлевая срок службы устройства. Выравнивание износа мы обсудим ниже.

ОТСТУПЛЕНИЕ: СОХРАНЕНИЕ ИНФОРМАЦИИ ОБ ОТОБРАЖЕНИИ В FTL

Может возникнуть вопрос: а что будет, если отключится питание устройства? Пропадет ли таблица отображения, хранящаяся в памяти? Очевидно, что такая информация не должна пропасть, иначе устройство не может рассматриваться как запоминающее. У SSD должны быть какие-то средства восстановления информации об отображении.

Проще всего было бы запоминать информацию об отображении в каждой странице, в так называемой **пограничной зоне** (out-of-band – OOB). После выключения питания и перезапуска устройство должно реконструировать свою таблицу отображения в памяти, просканировав все пограничные зоны. Но у этой хорошей в общем-то идеи есть проблемы: сканирование большого SSD в поисках всей необходимой информации занимает много времени. Чтобы справиться с этим, в некоторых устройствах высокой ценовой категории применяются более сложные схемы **протоколирования** и **контрольных точек** для ускорения восстановления; подробнее об этих методах написано в главах, посвященных согласованности после аварийных отказов и файловым системам со структурой журнала [AD14].

К сожалению, у базового подхода, основанного на структуре журнала, имеются недостатки. Первый заключается в том, что перезаписывание логических блоков ведет к накоплению **мусора**, т. е. старых версий данных, разбросанных по всему диску и занимающих место. Устройство должно периодически выполнять **сборку мусора** (garbage collection – GC), чтобы найти эти блоки и освободить их для будущей записи, но частая сборка мусора приводит к сильному расширению записи и снижению производительности. Вторая проблема – высокая стоимость хранения таблиц отображения в памяти; чем больше устройство, тем больше места эти таблицы занимают. Обсудим обе проблемы по очереди.

44.8. СБОРКА МУСОРА

Главный недостаток любого подхода на основе структуры журнала заключается в порождении мусора, для борьбы с которым необходимо выполнять **сборку мусора** (т. е. вводить мертвые блоки в повторный оборот). Поясним это на прежнем примере. Напомним, что на устройство были записаны логические блоки 100, 101, 2000 и 2001.

Предположим, что в блоки 100 и 101 повторно произведена запись данных с1 и с2 соответственно. Данные попадут в следующие свободные страницы (в данном случае – физические страницы 4 и 5), и таблица отображения будет

соответственно обновлена. Отметим, что устройство должно предварительно стереть блок 1, чтобы такое программирование стало возможным:

Таблица:	100 → 4				101 → 5				2000 → 2				2001 → 3				Память
Блок:	0				1				2				Микросхема флеш-памяти				
Страница:	00	01	02	03	04	05	06	07	08	09	10	11					
Содержимое:	a1	a2	b1	b2	c1	c2											
Состояние:	V	V	V	V	V	V	E	E	i	i	i	i					

Теперь понятно, в чем проблема: физические страницы 0 и 1, хотя и помечены как VALID, содержат **мусор**, т. е. старые версии блоков 100 и 101. К их появлению привело именно структурирование устройства как журнала, и теперь устройство должно как-то вернуть эти блоки в оборот, чтобы в них можно было писать в будущем.

Процесс поиска **мусорных**, или **мертвых, блоков** и их введение в оборот для будущего использования называется **сборкой мусора**, это важная составная часть любого современного SSD. Базовый процесс прост: найти блок, содержащий одну или несколько мусорных страниц, прочитать живые (немусорные) страницы из этого блока, записать их в журнал и (наконец) сделать весь блок доступным для записи в будущем.

Проиллюстрируем это на примере. Устройство решает, что хочет ввести в повторный оборот мертвые страницы в блоке 0, где есть две мертвые страницы (0 и 1) и две живые (2 и 3, содержащие соответственно логические блоки 2000 и 2001). Для этого устройство должно:

- прочитать живые страницы (2 и 3) из блока 0;
- записать живые данные в конец журнала;
- стереть блок 0 (освободив его для последующего использования).

Для нормальной работы сборщика мусора в каждом блоке должно быть достаточно информации, чтобы SSD мог решить, какие страницы живы, а какие мертвы. С этой целью было бы естественно сохранить где-то в блоке информацию о том, какие логические блоки хранятся в каждой странице. Тогда устройство могло бы использовать таблицу отображения, чтобы определить, в каких страницах хранятся живые данные, а в каких – только мусор.

В нашем примере (до сборки мусора) блок 0 содержит логические блоки 100, 101, 2000, 2001. Справившись с таблицей отображения (которая до сборки мусора содержала соответствия 100->4, 101->5, 2000->2, 2001->3), устройство легко установит, какие страницы этого блока содержат актуальную информацию. Например, логические блоки 2000 и 2001, очевидно, еще находятся в физическом блоке 0, тогда как логические блоки 100 и 101 уже нет и, стало быть, являются кандидатами на сборку мусора.

Когда процесс сборки мусора завершится, устройство окажется в следующем состоянии:

Таблица:	100 → 4		101 → 5		2000 → 6		2001 → 7		Память				
Блок:	0				1				2				Микросхема флеш-памяти
Страница:	00	01	02	03	04	05	06	07	08	09	10	11	
Содержимое:					c1	c2	b1	b2					
Состояние:	E	E	E	E	V	V	V	V	i	i	i	i	

Как видим, сборка мусора может обойтись недешево, поскольку требует чтения и перезаписи живых данных. Идеальным кандидатом для возврата в повторный оборот является блок, содержащий только мертвые страницы, его можно было бы сразу же стереть и использовать для размещения новых данных без дорогостоящего переноса данных.

ОТСТУПЛЕНИЕ: НОВЫЙ API СИСТЕМ ХРАНЕНИЯ – УСЕЧЕНИЕ

Размышляя о жестких дисках, мы обычно имеем в виду самый простой интерфейс чтения и записи: читать и писать (часто в него входит также команда **сброса кеша**, гарантирующая, что записанные данные попали на запоминающее устройство, но для простоты мы ее иногда опускаем). Для SSD со структурой журнала, да и вообще для любого устройства, поддерживающего гибкое и переменное отображение логических блоков в физические, полезен новый интерфейс – операция **усечения** (trim).

Операция усечения принимает адрес (возможно, еще и длину) и информирует устройство о том, что блок по этому адресу (или последовательность блоков заданной длины) удален; устройство больше не обязано хранить информацию об указанном диапазоне адресов. Для стандартного жесткого диска операция усечения не особенно полезна, потому что для диска определено статическое отображение адресов блоков на конкретную пластину, дорожку и сектор(а). Но для SSD со структурой журнала очень полезно знать, что блок больше не нужен, поскольку тогда SSD может удалить эту информацию из FTL и впоследствии вернуть в оборот физическое пространство в процессе сборки мусора.

Хотя иногда мы рассматриваем интерфейс и реализацию как разные сущности, в этом случае реализация диктует интерфейс. При наличии сложных отображений знание о том, какие блоки больше не нужны, позволяет построить более эффективную реализацию.

Чтобы уменьшить накладные расходы на сборку мусора, некоторые SSD производят **резервирование** (overprovision) устройства [A+08]; оставляя часть флеш-памяти под свои нужды, они могут отсрочить очистку и перенести ее в **фоновый режим**, т. е. выполнять, когда устройство не очень занято. Резервирование емкости также увеличивает внутреннюю пропускную способность, так что ее избыток можно использовать для очистки, не уменьшая пропускную способность, воспринимаемую клиентом. Многие современные накопители поступают подобным образом, это один из способов повысить общую производительность.

44.9. РАЗМЕР ТАБЛИЦЫ ОТОБРАЖЕНИЯ

Вторая проблема, связанная со структурой журнала, – потенциально очень большие таблицы отображения, в которых каждой 4-килобайтовой странице устройства соответствует одна запись. Для SSD объемом 1 ТБ понадобится 1 Гб памяти только для хранения отображений! Поэтому такая схема FTL **страничного уровня** практически неприемлема.

Блочное отображение

Один из способов уменьшить расходы на отображение – хранить указатели на блоки, а не на страницы, что уменьшает объем таблицы в $\frac{\text{Размер блока}}{\text{Размер страницы}}$ раз. Такой FTL блочного уровня сродни увеличению размера страницы в системе виртуальной памяти, когда мы отводим меньше битов под VPN, но увеличиваем длину смещения в каждом виртуальном адресе.

К сожалению, блочное отображение не очень хорошо работает с FTL со структурой журнала с точки зрения производительности. Самая серьезная проблема возникает при «короткой записи» (меньшей, чем размер физического блока). В этом случае FTL должен прочитать большой объем живых данных из старого блока и скопировать их в новый (вместе с данными короткой записи). Из-за этого копирования резко увеличивается коэффициент расширения записи и, следовательно, падает производительность.

Чтобы прояснить проблему, рассмотрим пример. Предположим, что ранее клиент записал логические блоки 2000, 2001, 2002 и 2003 (с данными а, б, с, d) и что они расположены внутри физического блока 1 на физических страницах 4, 5, 6 и 7. При страничном отображении в таблице трансляции должны были бы храниться следующие четыре записи для логических блоков: 2000→4, 2001→5, 2002→6, 2003→7.

Если же вместо этого используется отображение блочного уровня, то в FTL будет храниться только одна трансляция адреса для всех этих данных. Но отображение адресов будет не совсем таким, как в предыдущих примерах. Именно, мы рассматриваем логическое адресное пространство устройства как разбитое на порции, размер которых равен размеру физического блока флеш-памяти. Поэтому адрес логического блока состоит из двух частей: номер порции и смещение. Поскольку мы предполагаем, что в физическом блоке помещается четыре логических, смещение должно занимать 2 бита, а оставшиеся (старшие) биты образуют номер порции.

В каждом из логических блоков 2000, 2001, 2002 и 2003 номер порции один и тот же (500), но смещения разные (соответственно, 0, 1, 2 и 3). Таким образом, при блочном отображении FTL запоминает, что порция 500 отображается в блок 1 (начинающийся с физической страницы 4), как показано на рисунке ниже:

Таблица:	500 → 4												Память
Блок:	0				1				2				Микросхема флеш-памяти
Страница:	00	01	02	03	04	05	06	07	08	09	10	11	
Содержимое:					a	b	c	d					
Состояние:	i	i	i	i	V	V	V	V	i	i	i	i	

Для блочного FTL чтение не вызывает трудностей. Сначала FTL выделяет номер порции из адреса логического блока, переданного клиентом, – это старшие биты адреса. Затем FTL ищет в таблице отображение номера порции в физическую страницу. И наконец, FTL вычисляет адрес нужной страницы флеш-памяти, прибавляя смещение логического адреса к физическому адресу блока.

Например, если клиент просит прочитать логический блок с адресом 2002, то устройство выделяет из адреса номер логической порции (500), находит в таблице отображения трансляцию (4) и прибавляет смещение из логического адреса (2) к результату трансляции (4). Физическая страница с получившимся адресом (6) и есть то место, где хранятся данные. Теперь FTL может выполнить команду чтения по физическому адресу и получить искомые данные (с).

Но что, если клиент хочет записать в логический блок 2002 (данные с)? В этом случае FTL должен прочитать блоки 2000, 2001 и 2003, а затем записать все четыре блока в новое место, соответственно обновив таблицу отображения. Блок 1 (там, где данные были раньше) после этого можно стереть и использовать повторно, как показано на рисунке ниже.

Таблица:	500 → 8												Память
Блок:	0				1				2				Микросхема флеш-памяти
Страница:	00	01	02	03	04	05	06	07	08	09	10	11	
Содержимое:									a	b	c'	d	
Состояние:	i	i	i	i	E	E	E	E	V	V	V	V	

Из этого примера видно, что хотя отображение на блочном уровне значительно уменьшает объем памяти, необходимой для хранения трансляций, вместе с тем оно создает проблемы производительности, когда запись короче размера физического блока устройства, а поскольку физические блоки занимают 256 КБ и больше, то такие записи, вероятно, будут встречаться часто. Поэтому необходимо решение получше. Может быть, вы сами догадаетесь, какое, прежде чем читать дальше?

Гибридное отображение

Чтобы повысить гибкость записи и одновременно снизить затраты на отображение, во многих современных FTL применяется метод **гибридного отобра-**

жения. При таком подходе FTL запоминает несколько стертых блоков и направляет в них все операции записи; эти блоки называются **журнальными**. Поскольку FTL хочет иметь возможность записывать любую страницу в любое место журнального блока без копирования, сопровождающего чистое блочное отображение, для таких блоков хранятся *страничные* отображения.

Таким образом, в памяти FTL хранятся два логически различающихся типа таблиц отображения: небольшой набор страничных отображений в так называемой *журнальной таблице* и большой набор блочных отображений в *таблице данных*. При поиске логического блока FTL сначала заглядывает в журнальную таблицу, а если логического блока там нет, то ищет его в таблице данных и затем обращается к запрошенным данным.

Важная особенность гибридной стратегии отображения – небольшое число журнальных блоков. А чтобы оно оставалось небольшим, FTL должен периодически просматривать журнальные блоки (для каждого из которых имеется указатель на страницу) и *переключать* их на блоки, допускающие адресацию с помощью одного указателя. Существует три основных метода такого переключения в зависимости от содержимого блока [KK+02].

Например, предположим, что ранее FTL записал логические страницы 1000, 1001, 1002 и 1003, поместив их в физический блок 2 (физические страницы 8, 9, 10, 11), и пусть в них записаны данные a, b, c, d соответственно.

Журнальная таблица:													Память
Таблица данных:	250 → 8												
Блок:	0				1				2				Микросхема флеш-памяти
Страница:	00	01	02	03	04	05	06	07	08	09	10	11	
Содержимое:									a	b	c	d	
Состояние:	i	i	i	i	i	i	i	i	V	V	V	V	

Теперь предположим, что клиент перезаписывает каждый из этих блоков (данными a', b', c' и d') в том же порядке, и они оказываются в одном из доступных журнальных блоков (скажем, в физическом блоке 0, физические страницы 0, 1, 2, 3). Тогда FTL переходит в такое состояние:

Журнальная таблица:	1000 → 8	1001 → 1	1002 → 2	1003 → 3	
Таблица данных:	250 → 8				Память

Блок:	0				1				2				Микросхема флеш-памяти
Страница:	00	01	02	03	04	05	06	07	08	09	10	11	
Содержимое:	a'	b'	c'	d'					a	b	c	d	
Состояние:	V	V	V	V	i	i	i	i	V	V	V	V	

Поскольку эти блоки записаны точно таким же образом, как и прежде, FTL может выполнить **переключающее объединение**. В этом случае журналь-

ный блок (0) становится местом хранения блоков 0, 1, 2 и 3, и на него указывает один указатель на блок, а старый блок (2) стирается и используется как журнальный. Это наилучший случай, потому что все необходимые указатели на страницы заменяются одним указателем на блок.

Журнальная таблица:

Таблица данных: 250 → 0

Память

Блок:	0				1				2				Микросхема флеш-памяти
Страница:	00	01	02	03	04	05	06	07	08	09	10	11	
Содержимое:	a'	b'	c'	d'									
Состояние:	V	V	V	V	i	i	i	i	i	i	i	i	

Переключающее объединение – идеальный случай для гибридного FTL. Увы, так везет не всегда. Рассмотрим случай, когда начальные условия такие же (логические блоки 1000...1003 хранятся в физическом блоке 2), но затем клиент перезаписывает логические блоки 1000 и 1001. Как вы думаете, что произойдет? Почему этот случай труднее? (*Подумайте, прежде чем читать дальше.*)

Журнальная таблица: 1000 → 0 1001 → 1

Таблица данных: 250 → 8

Память

Блок:	0				1				2				Микросхема флеш-памяти
Страница:	00	01	02	03	04	05	06	07	08	09	10	11	
Содержимое:	a'	b'							a	b	c	d	
Состояние:	V	V	i	i	i	i	i	i	V	V	V	V	

Чтобы воссоединить другие страницы этого физического блока и получить возможность обращаться к ним с помощью одного указателя на блок, FTL выполняет **частичное объединение**. В этом случае логические блоки 1002 и 1003 читаются из физического блока 2, а затем добавляются в журнал. Конечное состояние SSD такое же, как в переключающем объединении выше, но для этого FTL пришлось выполнить дополнительные операции ввода-вывода, так что коэффициент расширения записи увеличился.

Последний случай называется **полным объединением** и требует еще больше работы. Теперь для выполнения очистки FTL должен собрать страницы из многих блоков. Например, предположим, что в журнальный блок А записаны логические блоки 0, 4, 8 и 12. Чтобы переключить этот журнальный блок на страницу, отображенную на блок, FTL должен сначала создать блок данных, содержащий логические блоки 0, 1, 2 и 3, т. е. прочитать откуда-то блоки 1, 2 и 3, а потом записать все четыре блока 0, 1, 2, 3 вместе. Затем точно такую же операцию нужно проделать для логического блока 4, т. е. найти блоки 5, 6, 7 и объединить их в один физический блок. То же самое нужно

сделать для логических блоков 8 и 12, и только потом можно будет освободить журнальный блок А. Неудивительно, что частые полные объединения сильно снижают производительность, поэтому их нужно избегать любой ценой [GY+09].

Страничное отображение плюс кеширование

Учитывая сложность описанного выше гибридного подхода, были предложены более простые способы снизить потребление памяти для FTL со страничным отображением. Пожалуй, самое простое – кешировать только активные части FTL в памяти, уменьшив тем самым объем необходимой памяти [GY+09].

Этот подход может давать неплохие результаты. Например, если рабочая нагрузка такова, что обращения производятся к небольшому набору страниц, то их трансляции будут храниться в памяти FTL, так что производительность будет отличной при минимальном расходе памяти. Но, конечно, этот подход может работать и отвратительно. Если в памяти не помещается **рабочий набор** необходимых трансляций, то каждое обращение будет как минимум требовать дополнительного чтения флеш-памяти, чтобы сначала прочитать отсутствующее отображение и только потом сами данные. Хуже того, чтобы освободить место для нового отображения, FTL должен **вытеснить** какое-то старое, а если вытесняемое отображение **грязное** (т. е. страница модифицирована, но еще не записана во флеш-память на постоянное хранение), то потребуются и дополнительная операция записи. Впрочем, зачастую в рабочей нагрузке присутствует локальность, так что описанный метод кеширования все же позволяет уменьшить потребление памяти, не снижая производительности.

44.10. ВЫРАВНИВАНИЕ ИЗНОСА

Наконец, современные FTL в фоновом режиме реализуют **выравнивание износа**, о котором уже заходила речь выше. Идея проста: поскольку многократные циклы стирания-программирования приводят к износу блока флеш-памяти, FTL должен приложить все усилия к равномерному распределению работы по всем блокам. Тогда блоки полностью изнашиваются примерно в одно время и не окажется, что несколько особо «популярных» блоков вообще стали непригодны.

Подход на основе структуры журнала в целом неплохо справляется с распределением записи, да и сборка мусора в этом помогает. Но некоторые блоки содержат неизменные данные и вообще не перезаписываются, в таком случае сборщик мусора никогда не будет трогать этот блок, так что он не получит своей доли записи.

Для решения этой проблемы FTL должен периодически читать данные из всех таких блоков и записывать их в какое-то другое место, делая блок

доступным для записи. Этот процесс выравнивания износа увеличивает коэффициент расширения записи SSD, а значит, снижает производительность, поскольку, чтобы все блоки изнашивались примерно с одинаковой скоростью, необходимы дополнительные операции ввода-вывода. В литературе описано много алгоритмов решения этой задачи [A+08, M+14], почитайте, если вам интересно.

44.11. Производительность и стоимость SSD

Прежде чем закрыть тему, рассмотрим производительность и стоимость современных SSD, чтобы лучше понять, насколько вероятно их использование в составе систем долговременного хранения. В обоих случаях для сравнения будем использовать классические жесткие диски (HDD) и отмечать существенные различия.

Производительность

В отличие от жестких дисков, SSD на основе флеш-памяти не имеют механических частей и во многих отношениях больше похожи на ДЗУПВ (англ. *DRAM*) тем, что являются устройствами «с произвольной выборкой». Различие в производительности по сравнению с жесткими дисками проявляется прежде всего при выполнении операций произвольного чтения и записи: если типичный диск может выполнить несколько сотен операций произвольного ввода-вывода в секунду, то для SSD эта величина гораздо больше. Мы приведем некоторые данные о современных SSD, чтобы понять, насколько они производительнее; особенно нас будет интересовать, в какой мере FTL скрывают проблемы, характерные для исходных микросхем флеш-памяти.

На рис. 44.4 приведены данные о производительности трех разных SSD и одного жесткого диска верхней ценовой категории; данные взяты из нескольких сетевых источников [S13, T15]. В двух левых столбцах показаны данные о производительности произвольного ввода-вывода, а в двух правых – то же для последовательного. Первые три строки относятся к трем SSD разных производителей (Samsung, Seagate, Intel), а последняя – к **жесткому диску (HDD)**, в данном случае высокопроизводительному изделию компании Seagate.

Устройство	Произвольно		Последовательно	
	Чтение (МБ/с)	Запись (МБ/с)	Чтение (МБ/с)	Запись (МБ/с)
Samsung 840 Pro SSD	103	287	421	384
Seagate 600 SSD	84	252	424	374
Intel SSD 335 SSD	39	222	344	354
Seagate Savvio 15K.3 HDD	2	2	223	223

Рис. 44.4 ❖ Сравнение производительности SSD и жестких дисков

Эта таблица раскрывает несколько интересных фактов. Первый и самый заметный – колоссальное различие в производительности произвольного ввода-вывода. Если SSD достигают десятков и сотен МБ/с, то «высокопроизводительный» жесткий диск показывает в пике жалкие пару МБ/с (и это мы еще округлили с избытком). Второе – что для производительности последовательного ввода-вывода различие гораздо меньше; хотя SSD по-прежнему быстрее, жесткий диск все же является хорошей альтернативой, если нам важен только последовательный ввод-вывод. Третье – производительность произвольного чтения SSD не так впечатляет, как производительность произвольной записи. Причина такого неожиданно хорошего поведения – структура журнала, реализованная во многих SSD, благодаря которой операции произвольной записи преобразуются в последовательные, что повышает производительность. Наконец, поскольку SSD демонстрируют некоторые различия в производительности последовательного и произвольного ввода-вывода, многие методы построения файловых систем для жестких дисков, описанные в предыдущих главах, применимы и к SSD; хотя по абсолютной величине различие меньше, оно все же достаточно велико, чтобы задуматься о проектировании файловых систем, минимизирующих объем произвольного ввода-вывода.

Стоимость

Как мы видели, производительность SSD заметно превышает производительность современных жестких дисков, даже при выполнении последовательного ввода-вывода. Так почему же тогда SSD не вытеснили полностью жесткие диски в качестве носителя? Ответ прост: стоимость, а точнее стоимость на единицу емкости. В настоящее время [A15] SSD-диск объемом 250 ГБ стоит примерно 150 долларов, т. е. цена за 1 ГБ составляет 60 центов. Типичный жесткий диск объемом 1 ТБ стоит около 50 долларов, т. е. 1 ГБ обходится в 5 центов. Разница более чем десятикратная.

Различия в производительности и стоимости диктуют принципы построения крупномасштабных систем хранения. Если во главу угла ставится производительность, то у SSD нет конкурентов, особенно когда важна производительность произвольного чтения. С другой стороны, если вы создаете большой центр обработки данных, где собираетесь хранить огромные массивы информации, то из-за высоких ценовых различий предпочтение стоит отдать жестким дискам. Конечно, гибридный подход тоже имеет право на существование – в состав некоторых систем хранения входят как SSD, так и жесткие диски, причем количество SSD сравнительно невелико, и используются они для хранения «горячих» данных, когда нужна максимальная производительность, а остальные данные «попрохладнее» (менее востребованные) хранятся на жестких дисках для экономии. Пока существует значительный ценовой разрыв, жесткие диски, скорее всего, останутся.

44.12. РЕЗЮМЕ

SSD-диски на основе флеш-памяти все чаще встречаются в ноутбуках, настольных компьютерах и серверах в центрах обработки данных, которые являются движущей силой современной экономики. Поэтому о них хоть что-то нужно знать, правда?

Но вот вам плохие новости: эта глава (как и многие другие в данной книге) – лишь первый шаг на пути к пониманию современного состояния дел. Из других источников, в которых можно почерпнуть сведения о базовой технологии, упомянем исследования по производительности конкретных устройств (например, Chen et al. [CK+09] и Grupp et al. [GC+09]). По вопросам проектирования FTL см. работы Agrawal et al. [A+08], Gupta et al. [GY+09], Huang et al. [H+14], Kim et al. [KK+02], Lee et al. [L+07] и Zhang et al. [Z+12], о распределенных системах на основе флеш-памяти см. работы Gordon [CG+09] и CORFU[B+12]. А по-настоящему хороший обзор всего, что нужно знать для выжимания высокой производительности из SSD, приведен в статье по «неписаному контракту» [HK+17].

Не ограничивайтесь чтением научных статей, читайте также о последних достижениях в научно-популярных изданиях (например, [V12]). Там вы найдете информации более практического свойства (но все равно полезную), например о том, как компания Samsung использует ячейки типа TLC и SLC в одном SSD с целью увеличения одновременно производительности (SLC может быстро буферизовать операции записи) и емкости (в одной TLC-ячейке можно хранить больше битов). И это, как говорится, только вершина айсберга. А то, что скрыто под водой, вы можете исследовать сами, начав, к примеру, с великолепного (и недавнего) обзора [M+14]. Но будьте осторожны – айсберги могут потопить даже самые могучие корабли [W15].

ОТСТУПЛЕНИЕ: ОСНОВНЫЕ ТЕРМИНЫ SSD

- **Микросхема флеш-памяти** состоит из большого числа банков, каждый из которых включает **стираемые блоки** (иногда их называют просто **блоками**). Каждый блок разбит на несколько **страниц**.
- Блоки велики (128 КБ – 2 МБ) и содержат много относительно небольших (1–8 КБ) страниц.
- Для чтения из флеш-памяти нужно выполнить команду чтения, указав адрес и длину; так можно прочесть одну или несколько страниц.
- Запись во флеш-память сложнее. Сначала клиент должен **стереть** весь блок (при этом вся хранившаяся в блоке информация теряется). Затем клиент может **запрограммировать** каждую страницу ровно один раз и тем самым завершить запись.
- Новая операция **усечения** полезна, когда требуется сообщить устройству, что некоторый блок (или диапазон блоков) больше не нужен.
- Надежность флеш-памяти определяется в основном **износом**; если блок стирается и программируется заново слишком часто, то он становится непригодным к использованию.

- **Твердотельное запоминающее устройство (SSD)** на основе флеш-памяти ведет себя как обычный блочный диск; благодаря **уровню трансляции флеш-памяти (FTL)** оно преобразует операции чтения и записи, запрашиваемые клиентом, в команды чтения, стирания и программирования составляющих микросхем флеш-памяти.
- Чаще всего FTL имеет **структуру журнала**, что позволяет снизить стоимость записи посредством уменьшения количества циклов стирания-программирования. В находящейся в памяти таблице отображения запоминается, в каком месте физического носителя находится записанный логический блок.
- Одна из главных проблем FTL со структурой журнала – стоимость **сборки мусора**, которая приводит к **расширению записи**.
- Еще одна проблема – потенциально очень большой размер таблицы отображения. Возможные решения – **гибридное отображение** или просто **кеширование** особенно востребованных блоков FTL.
- И последняя проблема – **выравнивание износа**; FTL должен время от времени переносить данные из блоков, которые в основном читаются, чтобы и эти блоки получили свою долю стирания и программирования.

Литература

[A+08] «Design Tradeoffs for SSD Performance» by N. Agrawal, V. Prabhakaran, T. Wobber, J. D. Davis, M. Manasse, R. Panigrahy. USENIX '08, San Diego California, June 2008. *Великолепный обзор проблем проектирования SSD.*

[AD14] «Operating Systems: Three Easy Pieces» by *Chapters: Crash Consistency: FSCCK and Journaling and Log-Structured File Systems.* Remzi Arpaci-Dusseau and Andrea Arpaci-Dusseau. *Здесь гораздо подробнее описаны файловые системы со структурой журнала; некоторые из этих идей применимы и к самому устройству.*

[A15] «Amazon Pricing Study» by Remzi Arpaci-Dusseau. February, 2015. *Это не статья в буквальном понимании слова; просто один из авторов зашел на сайт Amazon и посмотрел цены на жесткие диски и на SSD. Можете проделать то же самое и посмотреть, сколько они стоят сегодня.*

[B+12] «CORFU: A Shared Log Design for Flash Clusters» by M. Balakrishnan, D. Malkhi, V. Prabhakaran, T. Wobber, M. Wei, J. D. Davis. NSDI '12, San Jose, California, April 2012. *Новый подход к проектированию высокопроизводительного реплицированного журнала в кластерах с применением флеш-памяти.*

[BD10] «Write Endurance in Flash Drives: Measurements and Analysis» by Simona Boboila, Peter Desnoyers. FAST '10, San Jose, California, February 2010. *Интересная статья о сроке службы устройств на основе флеш-памяти. Иногда он значительно превосходит прогнозы производителей, бывает, что и в сто раз.*

[B07] «ZFS: The Last Word in File Systems» by Jeff Bonwick and Bill Moore. Доступно по адресу http://www.ostep.org/Citations/zfs_last.pdf. *Было ли это последним словом в области файловых систем? Нет, но, возможно, близко к тому.*

[CG+09] «Gordon: Using Flash Memory to Build Fast, Power-efficient Clusters for Data-intensive Applications» by Adrian M. Caulfield, Laura M. Grupp, Steven Swanson. ASPLOS '09, Washington, D. C., March 2009. *Ранняя статья о сборке крупных кластеров из микросхем флеш-памяти; определенно заслуживает прочтения.*

[CK+09] «Understanding Intrinsic Characteristics and System Implications of Flash Memory based Solid State Drives» by Feng Chen, David A. Koufaty, and Xiaodong Zhang. SIGMETRICS/Performance '09, Seattle, Washington, June 2009. *Прекрасный обзор на тему проблем производительности SSD в 2009 году (правда, уже немного устарел).*

[G14] «The SSD Endurance Experiment» by Geoff Gasior. The Tech Report, September 19, 2014. Доступно по адресу <http://techreport.com/review/27062>. *Описание ряда простых экспериментов по измерению производительности SSD по прошествии времени. Есть много других исследований подобного типа, погуглите сами.*

[GC+09] «Characterizing Flash Memory: Anomalies, Observations, and Applications» by L. M. Grupp, A. M. Caulfield, J. Coburn, S. Swanson, E. Yaakobi, P. H. Siegel, J. K. Wolf. IEEE MICRO '09, New York, New York, December 2009. *Еще одна прекрасная работа на тему производительности флеш-памяти.*

[GY+09] «DFTL: a Flash Translation Layer Employing Demand-Based Selective Caching of Page-Level Address Mappings» by Aayush Gupta, Youngjae Kim, Bhuvan Ugaonkar. ASPLOS '09, Washington, D. C., March 2009. *Отличный обзор различных стратегий очистки гибридных SSD, а также описание новой схемы, которая экономит место в таблице отображения и повышает производительность при различных рабочих нагрузках.*

[HK+17] «The Unwritten Contract of Solid State Drives» by Jun He, Sudarsun Kannan, Andrea C. Arpaci-Dusseau, Remzi H. Arpaci-Dusseau. EuroSys '17, Belgrade, Serbia, April 2017. *Наша собственная работа, где сформулировано пять правил, которых должен придерживаться клиент, чтобы добиться максимальной производительности от современных SSD. Правила касаются масштаба запросов, локальности, выравнивания на границу при последовательном доступе, группировки по времени смерти и равномерного срока службы. Детали смотрите в самой статье!*

[H+14] «An Aggressive Worn-out Flash Block Management Scheme To Alleviate SSD Performance Degradation» by Ping Huang, Guanying Wu, Xubin He, Weijun Xiao. EuroSys '14, 2014. *Недавняя работа о том, как получить максимум возможного от изношенных блоков флеш-памяти. Интересно!*

[J10] «Failure Mechanisms and Models for Semiconductor Devices» by Unknown author. Report JEP122F, November 2010. Доступно по адресу <http://www.jedec.org/sites/default/files/docs/JEP122F.pdf>. *Очень подробное обсуждение того, что происходит на уровне устройства и как такие устройства отказывают. Не для слабых духом. Или для физиков. Или для тех, кто отвечает обоим критериям.*

[KK+02] «A Space-Efficient Flash Translation Layer For Compact Flash Systems» by Jesung Kim, Jong Min Kim, Sam H. Noh, Sang Lyul Min, Yookun Cho. IEEE Transactions on Consumer Electronics, Volume 48, Number 2, May 2002. *Одно из самых первых предложений по использованию гибридных отображений.*

[L+07] «A Log Buffer-Based Flash Translation Layer by Using Fully-Associative Sector Translation.» Sang-won Lee, Tae-Sun Chung, Dong-Ho Lee, Sangwon Park, Ha-Joo Song. ACM Transactions on Embedded Computing Systems, Volume 6, Number 3, July 2007. *Потрясающая статья о построении гибридных отображений на уровне журнала и блоков.*

[M+14] «A Survey of Address Translation Technologies for Flash Memories» by Dongzhe Ma, Jianhua Feng, Guoliang Li. ACM Computing Surveys, Volume 46, Number 3, January 2014. *Пожалуй, один из лучших обзоров флеш-памяти и применяющих технологий.*

[S13] «The Seagate 600 and 600 Pro SSD Review» by Anand Lal Shimpi. AnandTech, May 7, 2013. Доступно по адресу <http://www.anandtech.com/show/6935/seagate-600-ssd-review>. *Одно из многих измерений производительности SSD, доступных в интернете. Не слышали об интернете? Не страшно. Просто откройте свой браузер и наберите в поисковой строке «интернет». Вы будете поражены открывшимся.*

[T15] «Performance Charts Hard Drives» by Tom's Hardware. January 2015. Доступно по адресу <http://www.tomshardware.com/charts/enterprise-hdd-charts>. *Еще один сайт, содержащий данные о производительности, на этот раз жестких дисков.*

[V12] «Understanding TLC Flash» by Kristian Vatto. AnandTech, September, 2012. Доступно по адресу <http://www.anandtech.com/show/5067/understanding-tlc-nand>. *Краткое описание флеш-памяти типа TLC и ее характеристик.*

[W15] «List of Ships Sunk by Icebergs» by Many authors. Доступно по адресу http://en.wikipedia.org/wiki/List_of_ships_sunk_by_icebergs. *Да, в Википедии, оказывается, есть страница, посвященная кораблям, потопленным айсбергами. Довольно скучная, тем более что есть всего один корабль, который у всех на слуху, – Титаник.*

[Z+12] «De-indirection for Flash-based SSDs with Nameless Writes» by Yiying Zhang, Leo Prasath Arulraj, Andrea C. Arpaci-Dusseau, Remzi H. Arpaci-Dusseau. FAST '13, San Jose, California, February 2013. *Наше исследование новой идеи о том, как уменьшить размер таблицы отображения. Основная идея – повторно использовать указатели в надстроенной поверх файловой системе для хранения местоположений блоков, а не добавлять еще один уровень косвенности.*

Домашнее задание (эмуляция)

В этом задании вы познакомитесь с программой `ssd.py`, простым эмулятором, который позволит лучше понять, как работает SSD. Описание параметров программы см. в файле `README`. Файл длинный, поэтому запаситесь

чашечкой чая (и лучше с кофеином), наденьте очки для чтения, примостите на колени кошку¹ и приступайте к работе.

Вопросы

1. В этом домашнем задании нас будет интересовать в первую очередь SSD со структурой журнала, для эмуляции которого нужно задать флаг `-T log`. Для сравнения мы будем использовать и другие типы SSD. Прежде всего запустите программу с флагами `-T log -s 1 -n 10 -q`. Какие операции будут выполнены? Проверьте свой ответ, задав флаг `-c` (или флаг `-C` вместо `-q -c`). Для генерирования других случайных рабочих нагрузок задайте другие значения флага `-s`.
2. Теперь требуется определить промежуточные состояния флеш-памяти после выполнения команд. Запустите программу с флагами `-T log -s 2 -n 10 -c`, чтобы увидеть все команды. Определите состояние флеш-памяти между каждой парой команд; задайте флаг `-F`, чтобы показать состояния и сравнить их со своими ответами. Чтобы проверить, как быстро вы набираете опыт, задайте другие начальные значения генератора случайных чисел.
3. Усложним задачу, добавив флаг `-r 20`. Как при этом изменятся команды? Для проверки своего ответа снова задайте флаг `-c`.
4. Производительность зависит от количества операций стирания, программирования и чтения (мы предполагаем, что операции усечения ничего не стоят). Запустите программу с такой же рабочей нагрузкой, что и выше, но без показа промежуточных состояний (например, с флагами `-T log -s 1 -n 10`). Попробуйте оценить, сколько времени будет потрачено на выполнение этой рабочей нагрузки (по умолчанию время стирания 1000 мкс, время программирования 40 мкс, а время чтения 10 мкс). Для проверки ответа задайте флаг `-S`. Время стирания, программирования и чтения можно задать с помощью флагов `-E`, `-W` и `-R` соответственно.
5. Теперь сравним производительность двух подходов: со структурой журнала и прямого (очень плохого), для чего нужно будет задать соответственно флаги `-T log` и `-T direct`. Сначала прикиньте, сколько времени будет работать прямой подход, а затем проверьте себя с помощью флага `-S`. В общем случае насколько подход на основе структуры журнала лучше прямого?
6. Далее изучим поведение сборщика мусора. Для этого нужно будет задать подходящие значения верхнего (`-G`) и нижнего (`-g`) пределов. Для начала посмотрите, что происходит, когда SSD-диск со структурой журнала подается большей рабочей нагрузкой, но без сборки мусора. С этой целью запустите программу с флагами `-T log -n 1000` (по умолчанию верхний предел равен 10, так что GC в этой конфигурации работать не будет). Что, на ваш взгляд, произойдет? Для проверки задайте флаг `-C` и, возможно, `-F`.

¹ Тут вы можете воскликнуть «Но я собачник!». На это мы можем только сказать: «Очень жаль». Заведите кошку, примостите ее на колени и приступайте к домашнему заданию! А как вы вообще собираетесь чему-то научиться, если не можете выполнить даже самые простые инструкции?

7. Чтобы активировать сборщик мусора, задайте значения поменьше. Верхний предел ($-G\ N$) говорит системе, что сборку мусора нужно начинать, когда использовано N блоков, а нижний предел ($-g\ M$) – что сборку нужно прекратить, когда число использованных блоков станет равно M . Как вы думаете, какие значения пределов следует задать в рабочей системе? Воспользуйтесь флагами $-C$ и $-F$, чтобы показать команды и промежуточные состояния устройства.
8. Еще один полезный флаг $-J$ показывает, какие действия выполняет сборщик мусора. Запустите программу с флагами $-T\ log\ -n\ 1000\ -C\ -J$, чтобы увидеть как команды, так и поведение GC. Что вы можете сказать о GC? Главным критерием оценки GC, конечно, является производительность. Задайте флаг $-S$, чтобы вывести финальную статистику; сколько дополнительных операций чтения и записи производится из-за сборки мусора? Сравните с идеальным SSD ($-T\ ideal$); сколько дополнительных операций чтения, записи и стирания производится вследствие самой природы флеш-памяти? Сравните также с прямым подходом; чем (с точки зрения операций стирания, чтения и программирования) подход на основе структуры журнала лучше?
9. Последний аспект, который мы исследуем, – **асимметрия рабочей нагрузки**. Добавление асимметрии изменяет рабочую нагрузку так, что большее количество операций записи относится к меньшей части пространства логических блоков. Например, если запустить программу с флагом $-k\ 80/20$, то 80 % операций записи приходятся на 20 % блоков. Задайте другие коэффициенты асимметрии и выполните много случайно выбранных операций (например, $-n\ 1000$), сначала в режиме $-T\ direct$, чтобы понять влияние асимметрии, а затем в режиме $-T\ log$, чтобы увидеть, какое воздействие она оказывает на устройство со структурой журнала. Какого результата вы ожидаете? Есть еще один способ управления асимметрией: флаг $-k\ 100$. Если задать его для асимметричной рабочей нагрузки, то на первые 100 операций записи асимметрия не распространяется. Идея в том, что сначала создается много данных, а затем обновляется только часть из них. Как это может повлиять на сборщик мусора?

Глава 45

Целостность и защита данных

Помимо рассмотренных выше основных направлений прогресса в области файловых систем, существует еще ряд особенностей, заслуживающих изучения. В этой главе предметом нашего внимания снова будет надежность (ранее мы изучали надежность систем хранения в главе, посвященной RAID). А точнее, как файловая система или система хранения должна обеспечивать безопасность данных, несмотря на ненадежность, внутренне присущую современным запоминающим устройствам?

Это общее направление называется **целостностью данных**, или **защитой данных**. Таким образом, мы будем изучать методы, которые дают уверенность, что система хранения вернет именно те данные, которые были в нее помещены.

СУЩЕСТВО ПРОБЛЕМЫ: КАК ОБЕСПЕЧИТЬ ЦЕЛОСТНОСТЬ ДАННЫХ

Как система может гарантировать, что данные, помещенные на хранение, защищены? Какие методы для этого необходимы? Как сделать эти методы эффективными с точки зрения времени и пространства?

45.1. Виды отказа дисков

В главе, посвященной RAID, мы узнали, что диски несовершенны и иногда выходят из строя. В ранних RAID-системах модель отказов была простой: либо диск работает, либо отказывает целиком, и обнаружить такой отказ было несложно. Благодаря подобной модели **прекращения работы при ошибке** конструировать RAID-массивы было сравнительно легко [S90].

А вот о чем мы не говорили, так это обо всех остальных видах отказа современных дисков. Именно, как очень подробно описано в работах Bairavasundaram et al. [B+07, B+08], современные диски в целом работают, но испытывают проблемы при доступе к одному или нескольким блокам. Наиболее распространенными и заслуживающими внимания являются два типа

отказов одного блока: **скрытая ошибка сектора** (latent-sector error – LSE) и **искажение блока**. Рассмотрим их более внимательно.

LSE возникают, когда сектор (или группа секторов) диска каким-то образом поврежден. Например, если по какой-то причине головка диска коснулась поверхности (это называется **аварией головки** и не должно происходить при нормальной работе диска), то поверхность может оказаться поврежденной, а данные на ней – нечитаемыми. Космическое излучение также может привести к инверсии битов и, как следствие, к изменению записанных данных. По счастью, встроенные в контроллер диска **коды, исправляющие ошибки** (error correcting code – ECC), позволяют обнаружить ошибки в битах на диске и в некоторых случаях исправить их; если у диска недостаточно информации для исправления, то при попытке прочитать такие данные будет возвращена ошибка.

Бывает также, что блок диска **искажается** таким образом, что сам диск не может этого обнаружить. Например, из-за ошибки в прошивке блок может быть записан не в то место, тогда встроенный в диск ECC покажет, что содержимое блока правильно, но, с точки зрения клиента, результаты чтения блока некорректны. Блок может быть искажен также во время передачи с компьютера на диск по неисправной шине; тогда на диск будут записаны искаженные данные, а не те, что имел в виду клиент. Такие типы отказов особенно коварны, потому что никак себя не проявляют – диск не сообщает о наличии проблемы, а просто возвращает неверные данные.

В работе Prabhakaran et al. [P+05] этот более современный взгляд на отказы дисков назван моделью **частичной неработоспособности при ошибке** (fail-partial). Диск по-прежнему может отказывать целиком (как в традиционной модели прекращения работы при ошибке), но иногда кажется, что диск работает, хотя один или несколько блоков уже недоступны (LSE) или в них хранится неверная информация (искажение). Таким образом, при доступе к «мнимоработающему» диску иногда возвращается ошибка при попытке прочитать или записать определенный блок (явный частичный отказ) или же просто возвращаются неверные данные (тихий частичный отказ).

Отказы обоих типов редки, но насколько редки? На рис. 45.1 представлены некоторые результаты двух исследований Байравасундарамы [B+07, B+08].

	Дешевые	Дорогие
LSE	9.40 %	1.40 %
Повреждение	0.50 %	0.05 %

Рис. 45.1 ❖ Частота скрытых ошибок сектора и повреждений блоков

Показана процентная доля дисков, для которых в ходе исследования (более 3 лет, свыше 1,5 млн дисков) хотя бы раз встречалась скрытая ошибка сектора или искажение блока. Отдельно приведены результаты для «дешевых» (обычно SATA) и «дорогих» (обычно SCSI или FibreChannel) дисков. Как видим, хотя покупка более дорогих дисков уменьшает частоту ошибок обоих видов

(примерно на порядок), все равно они встречаются достаточно часто, чтобы задуматься о том, как обрабатывать их в системе хранения.

Приведем дополнительные факты, касающиеся LSE:

- если в дорогом диске имеет место более одной LSE, то, скорее всего, со временем возникнет еще больше, как и в дешевых дисках;
- для большинства дисков частота ошибок возрастает на втором году эксплуатации;
- количество LSE увеличивается с ростом емкости диска;
- в большинстве дисков количество LSE меньше 50;
- если на диске имеют место LSE, то со временем их число с большой вероятностью будет увеличиваться;
- имеет место выраженная пространственная и временная локальность ошибок;
- чистка диска полезна (большинство LSE было найдено именно таким способом).

А вот несколько фактов, относящихся к искажению:

- вероятность искажения сильно варьируется среди различных моделей дисков одного и того же класса;
- возраст также по-разному влияет на различные модели;
- рабочая нагрузка и емкость диска почти не влияют на искажение;
- в большинстве дисков с искажениями число искажений невелико;
- искажения на одном диске или во всем RAID-массиве не являются независимыми;
- существует слабая корреляция с LSE.

Если хотите подробнее узнать об этих отказах, прочитайте оригинальные статьи [В+07, В+08]. Но надеемся, что основная мысль ясна: чтобы построить надежную систему хранения, необходимо включить какой-то механизм обнаружения и исправления LSE и искажений блоков.

45.2. ОБРАБОТКА СКРЫТЫХ ОШИБОК СЕКТОРОВ

Зная о двух новых видах частичного отказа дисков, мы должны решить, что с этим делать. Сначала займемся более простым случаем – скрытыми ошибками секторов.

СУЩЕСТВО ПРОБЛЕМЫ: КАК ОБРАБАТЫВАТЬ СКРЫТЫЕ ОШИБКИ СЕКТОРОВ

Как система хранения должна обрабатывать скрытые ошибки секторов? Какие для этого нужны дополнительные механизмы?

Как выясняется, обработать скрытые ошибки секторов довольно просто, потому что они (по определению) легко обнаруживаются. Если при попытке обратиться к блоку диск возвращает ошибку, то система хранения должна

просто задействовать встроенные механизмы избыточности, чтобы вернуть правильные данные. Например, в зеркалированном RAID-массиве система должна обратиться к альтернативной копии, а в массивах типа RAID-4 или RAID-5 с контролем четности – реконструировать блок по другим блокам в той же группе четности. Таким образом, легко обнаруживаемые ошибки типа LSE столь же легко исправляются с помощью стандартных механизмов избыточности.

Преобладание LSE с годами привело к изменениям в конструкции RAID-массивов. Особенно интересная проблема возникает в системах типа RAID-4/5, когда отказы типа переполнения диска и LSE встречаются вместе. Именно, когда отказывает диск целиком, RAID пытается **реконструировать** диск (скажем, на диске из горячего резерва), прочитав все остальные диски в той группе четности и вычислив отсутствующие значения. Если в процессе реконструкции на одном из других дисков встречается LSE, то успешно довести реконструкцию до конца не получится – налицо проблема.

Для борьбы с этой неприятностью в некоторых системах добавлена дополнительная степень избыточности. Например, в системе **RAID-DP** компании NetApp реализован эквивалент двух дисков четности вместо одного [C+04]. Если в процессе реконструкции обнаружена LSE, то для восстановления отсутствующего блока на помощь приходит второй диск четности. Как всегда, это обходится не даром, поскольку поддержание двух блоков четности для каждой полосы стоит дороже. Но во многих случаях файловая система Net-App **WAFL**, имеющая структуру журнала, амортизирует эти затраты [HLM94]. Остается только стоимость дополнительного места в форме еще одного диска для хранения второго блока четности.

45.3. ОБНАРУЖЕНИЕ ИСКАЖЕНИЯ: КОНТРОЛЬНАЯ СУММА

Теперь займемся более трудной проблемой: тихие отказы диска вследствие искажения данных. Как предотвратить возврат пользователям некорректных данных?

СУЩЕСТВО ПРОБЛЕМЫ:

КАК ОБЕСПЕЧИТЬ ЦЕЛОСТНОСТЬ ДАННЫХ ВОПРЕКИ ИСКАЖЕНИЮ

Что система хранения может сделать для обнаружения искажений, которые не проявляют себя? Какие методы для этого необходимы? Как реализовать их эффективно?

В отличие от скрытых ошибок секторов, главной проблемой является само *обнаружение* искажения. Откуда клиент может узнать, что блок неправильный? Коль скоро это известно, *восстановление* происходит так же, как раньше: нужно где-то найти другую копию блока (желательно неискаженную!). Поэтому сосредоточимся на методах обнаружения.

Основной механизм обеспечения целостности данных в современных системах хранения – **контрольная сумма**. Контрольная сумма – это краткий дайджест (скажем, 4 или 8 байт), который вычисляется функцией, получающей на входе какой-то блок данных (например, размером 4 КБ). Зная контрольную сумму, можно обнаружить, что данные были искажены; для этого система сохраняет ее вместе с данными и при последующем доступе проверяет, что вновь вычисленная контрольная сумма совпадает с сохраненной.

Совет: бесплатных завтраков не бывает

Бесплатных завтраков не бывает – старая американская поговорка, означающая, что даже если вам кажется, что вы что-то получаете задаром, на самом деле за это приходится чем-то расплачиваться. Появилась она в те давние времена, когда владельцы едальных заведений рекламировали бесплатные завтраки, надеясь заманить клиента, а уж, оказавшись внутри, клиент понимал, что получить «бесплатный» завтрак он может, только заказав порцию выпивки, а то и не одну. Конечно, не всегда это такая уж проблема, особенно если вы начинающий алкоголик (или типичный студент).

Распространенные функции вычисления контрольной суммы

Существует несколько функций для вычисления контрольных сумм, различающихся силой (способностью защищать целостность данных) и быстродействием (скоростью вычисления). Как всегда, нужно искать компромисс: чем сильнее защита, тем она дороже. Бесплатных завтраков не бывает.

Иногда используют простую контрольную сумму на основе ИСКЛЮЧАЮЩЕГО ИЛИ (XOR). В этом случае сумма вычисляется путем применения XOR к каждому фрагменту блока данных, в результате чего порождается одно значение.

Для примера рассмотрим вычисление 4-байтовой контрольной суммы для 16-байтового блока (это, конечно, слишком мало для сектора диска, но для иллюстрации в самый раз). Пусть в шестнадцатеричной форме эти 16 байт имеют вид:

```
365e c4cd ba14 8a92 ecef 2c3a 40be f666
```

В двоичной форме это записывается так:

```
0011 0110 0101 1110 1100 0100 1100 1101
1011 1010 0001 0100 1000 1010 1001 0010
1110 1100 1110 1111 0010 1100 0011 1010
0100 0000 1011 1110 1111 0110 0110 0110
```

Представив блок в виде строк по 4 байта, мы легко найдем контрольную сумму – нужно только применить XOR к группам в каждом столбце, в результате чего получим:

```
0010 0000 0001 1011 1001 0100 0000 0011
```

В шестнадцатеричной форме это равно 0x201b9403.

У контрольной суммы на базе XOR есть ограничения. Например, изменение двух битов в одной и той же позиции группы, к которой применяется XOR, не обнаруживается. Поэтому были изучены другие контрольные суммы.

Еще одна простая контрольная сумма основана на сложении. Преимущество этого подхода – высокая скорость, нужно только складывать элементы групп в дополнительном коде, игнорируя переполнение. Эта контрольная сумма может обнаружить многие изменения данных, но не годится, например, в случае, когда изменение сводит к поразрядному сдвигу.

Чуть более сложный алгоритм называется **контрольной суммой Флетчера** по имени изобретателя [F82]. Вычисляется она очень быстро, и в результате получается два контрольных байта, s_1 и s_2 . Именно, предположим, что блок D состоит из байтов $d_1 \dots d_n$; тогда $s_1 = (s_1 + d_i) \bmod 255$ (суммируется по всем d_i), а $s_2 = (s_2 + s_1) \bmod 255$ (суммирование снова по всем d_i) [F04]. Контрольная сумма Флетчера обладает почти такой же силой, как CRC (см. ниже), она способна обнаружить все ошибки в одном и в двух битах, а также многие пакетные ошибки [F04].

И последняя часто используемая контрольная сумма называется **циклическим избыточным кодом** (cyclic redundancy check – **CRC**). Предположим, что требуется вычислить контрольную сумму блока данных D . Для этого будем рассматривать D как большое двоичное число (в конце концов, это всего лишь строка битов) и поделим его на согласованное обеими сторонами значение (k). Остаток от деления и есть значение CRC. Реализовать двоичные операции по модулю можно очень эффективно, поэтому CRC находит широкое применение также в сетях. Дополнительные сведения см. в работе [M13].

Очевидно, что какой бы метод ни использовать, идеальной контрольной суммы не получится: всегда можно найти два разных блока данных с одинаковой контрольной суммой, эту ситуацию часто называют **коллизией**. Данный факт интуитивно очевиден: ведь мы берем что-то большое (например, 4 КБ) и получаем гораздо меньший дайджест (4 или 8 байт). Поэтому хорошая контрольная сумма отличается тем, что минимизирует вероятность коллизии и при этом легко вычисляется.

Хранение контрольных сумм

Теперь, понимая, как вычисляется контрольная сумма, нужно понять, как ее можно использовать в системе хранения. Первый вопрос, который нужно решить, – где хранить контрольные суммы на диске?

Самое простое – хранить контрольную сумму вместе с каждым сектором (или блоком). Обозначим $C(D)$ контрольную сумму блока данных D . Без контрольных сумм данные размещаются на диске следующим образом:

D0	D1	D2	D3	D4	D5	D6
----	----	----	----	----	----	----

А с контрольными суммами в каждый блок включается его сумма:

C(D0)	D0	C(D1)	D1	C(D2)	D2	C(D3)	D3	C(D4)	D4
-------	----	-------	----	-------	----	-------	----	-------	----

Поскольку контрольные суммы обычно короткие (например, 8 байт), а диск может записывать данные только секторами (512 байт) или порциями, кратными сектору, то возникает проблема: как реализовать такую схему размещения? Некоторые производители дисков форматируют диск, разбивая его на сектора длиной 520 байт, и в дополнительных 8 байтах хранят контрольную сумму.

Если диск не обладает такой возможностью, то файловая система должна исхитриться и упаковывать контрольные суммы в 512-байтовые блоки. Ниже показан один из таких способов:

C(D0)	C(D1)	C(D2)	C(D3)	C(D4)	D0	D1	D2	D3	D4
-------	-------	-------	-------	-------	----	----	----	----	----

В этой схеме n контрольных сумм хранятся вместе в одном секторе, вслед за ними расположено n блоков данных, дальше еще один сектор контрольных сумм, следующие n блоков и т. д. Это решение хорошо тем, что работает для любых дисков, но оно не так эффективно; например, желая перезаписать блок $D1$, файловая система должна прочитать сектор контрольных сумм, содержащий $C(D1)$, обновить в нем $C(D1)$ и записать сектор контрольных сумм и блок данных (т. е. получается одно чтение и две записи). В первом случае (когда в каждом секторе хранится его контрольная сумма) требуется только одна запись.

45.4. ИСПОЛЬЗОВАНИЕ КОНТРОЛЬНЫХ СУММ

Определившись со способом хранения контрольных сумм, продолжим разбираться с тем, как они *используются*. При чтении блока D клиент (файловая система или контроллер диска) читает его контрольную сумму $C_s(D)$, которая называется **хранимой контрольной суммой**. Затем клиент *вычисляет* контрольную сумму прочитанного блока D , которая называется **вычисленной контрольной суммой** $C_c(D)$. Далее клиент сравнивает обе суммы; если они равны (т. е. $C_s(D) == C_c(D)$), то данные, вероятно, не были искажены, поэтому их можно вернуть пользователю. Если же суммы *не* совпадают, значит, данные изменились с момента сохранения (поскольку сохраненная контрольная сумма отражает состояние данных на тот момент времени). В таком случае налицо искажение, и контрольная сумма помогла его обнаружить.

Итак, искажение имеется, но что нам с этим делать? Если в системе хранения имеется копия, то ответ ясен: попытаться использовать ее. Если же копии нет, то, вероятно, следует вернуть ошибку. В любом случае надо понимать, что обнаружение искажения – не чудодейственное средство; если не существует никакого способа получить неискаженные данные, значит, нам не повезло.

45.5. НОВАЯ ПРОБЛЕМА: ЗАПИСЬ НЕ ПО АДРЕСУ

Описанная выше базовая схема хорошо работает в общем случае искаженных блоков. Но в современных дисках встречается два необычных вида отказов, для которых требуется иное решение.

Первый из них называется **записью не по адресу** (misdirected write). Так бывает, когда контроллер диска или RAID записывает правильные данные, но в *неправильное* место. В системе с одним диском это означает, что диск записал D_x не по адресу x (как было нужно), а по адресу y (тем самым «исказив» D_y). В системе с несколькими дисками контроллер может также отправить блок $D_{i,x}$ по адресу x не на диске i , а на каком-то другом диске j .

СУЩЕСТВО ПРОБЛЕМЫ: КАК ОБРАБОТАТЬ ЗАПИСЬ НЕ ПО АДРЕСУ

Как система хранения или контроллер диска может обнаружить запись не по адресу? Какими дополнительными свойствами должна обладать контрольная сумма?

Ответ, как ни странно, простой: добавить немного информации к каждой контрольной сумме. В данном случае полезно добавить **физический идентификатор**. Например, если хранимая информация содержит контрольную сумму $C(D)$, а также номер диска и номер сектора блока, то клиент легко сможет определить, правильная ли информация находится в данном секторе. Именно, если клиент читает блок 4 на диске 10 ($D_{10,4}$), то хранимая информация должна включать этот номер диска и номер сектора, как показано на рисунке ниже. Если информация не совпадает, значит, имела место запись не по адресу, а теперь обнаружено искажение. Ниже показано, как может выглядеть добавленная информация в системе с двумя дисками. Отметим, что на этом рисунке, как и на предыдущих, масштаб не соблюден, поскольку контрольные суммы обычно короткие (например, 8 байт), а блоки гораздо больше (4 КБ и более).

Диск 1	$C[D0]$	диск=1	блок=0	D0	$C[D1]$	диск=1	блок=1	D1	$C[D2]$	диск=1	блок=2	D2
Диск 0	$C[D0]$	диск=0	блок=0	D0	$C[D1]$	диск=0	блок=1	D1	$C[D2]$	диск=0	блок=2	D2

Как видим, на диске хранится довольно много избыточной информации: в каждом блоке повторяется номер диска и смещение самого этого блока. Впрочем, это не должно вызывать удивления, т. к. избыточность – ключ к обнаружению ошибок (в данном случае) и восстановлению (в других). Немного дополнительной информации, которая не нужна, когда диск работает идеально, может оказаться неоценимым подспорьем для обнаружения проблем, если они вдруг возникнут.

45.6. Последняя проблема: ПОТЕРЯННЫЕ ЗАПИСИ

К сожалению, запись не по адресу – не последняя из рассматриваемых проблем. Именно, в некоторых современных запоминающих устройствах существует еще проблема **потерянной записи**, которая возникает, когда устройство сообщает расположенному выше уровню, что запись завершена, а на самом деле данные так и не были сохранены, т. е. на носителе осталось старое содержимое блока.

Возникает очевидный вопрос: позволяет ли какая-нибудь из описанных выше стратегий (базовая контрольная сумма или дополненная физическим идентификатором) помочь в обнаружении потерянных записей? К сожалению, нет: в старом блоке, скорее всего, правильная контрольная сумма, да и физический идентификатор (номер диска и смещение блока) тоже правилен.

Существо проблемы: КАК ОБРАБАТЫВАТЬ ПОТЕРЯННУЮ ЗАПИСЬ

Как система хранения или контроллер диска может обнаружить потерянную запись? Какими дополнительными свойствами должна обладать контрольная сумма?

Существует несколько решений [K+08]. Классический подход [BS04] – выполнять **верификацию записи**, или **чтение после записи**; прочитав данные сразу же после записи, система может удостовериться, что данные попали на поверхность диска. Но это очень медленно, поскольку количество операций ввода-вывода, необходимых для завершения записи, удваивается.

В некоторых системах потерянные записи обнаруживаются благодаря добавлению контрольной суммы в другие области системы. Например, в файловой системе **Zettabyte File System (ZFS)** компании Sun контрольные суммы для каждого блока файла включены в его индексный дескриптор и косвенные блоки. Таким образом, если запись в блок данных будет потеряна, контрольная сумма в индексном дескрипторе не будет соответствовать старым данным. Эта схема не сработает, только если будут одновременно потеряны записи в индексный дескриптор и в блок данных, что крайне маловероятно (но, увы, все-таки возможно!).

45.7. ОЧИСТКА

С учетом всего этого обсуждения возникает вопрос: а когда же проверяются эти контрольные суммы? Конечно, какие-то проверки производятся при доступе к данным со стороны приложений, но к большей части данных обращения производятся редко, поэтому данные остаются непроверенными. А непроверенные данные составляют проблему в надежной системе хранения, поскольку «гниение битов» может рано или поздно постигнуть все копии конкретного блока данных.

Для решения этой проблемы во многих системах применяется **очистка диска** (disk scrubbing) в той или иной форме [K+08]. Система периодически читает *все без исключения* блоки и проверяет совпадение контрольных сумм, это снижает вероятность одновременного искажения всех копий данных. Обычно данная процедура планируется каждую ночь или еженедельно.

45.8. НАКЛАДНЫЕ РАСХОДЫ КОНТРОЛЬНЫХ СУММ

Прежде чем закрыть тему, обсудим накладные расходы, сопряженные с использованием контрольных сумм для защиты данных. Как всегда в вычислительной системе, есть два типа накладных расходов: пространственные и временные.

Пространственные накладные расходы бывают двух видов. Во-первых, на самом диске (или другом носителе); каждая хранимая контрольная сумма занимает место, которое могло бы быть отдано под пользовательские данные. Типичная потеря – 8 байт контрольной суммы на 4-килобайтовый блок данных, т. е. 0.19 %.

Второй вид пространственных накладных расходов связан с памятью системы. При доступе к данным необходимо выделять память не только для самих данных, но и для контрольных сумм. Но если система просто проверяет контрольную сумму, а потом отбрасывает ее, то эти затраты краткосрочные и на них можно не обращать внимания. Лишь если контрольные суммы хранятся в памяти (в качестве дополнительного уровня защиты от повреждения памяти [Z+13]), то эти накладные расходы становятся заметными.

Хотя пространственные накладные расходы невелики, временные, обусловленные проверкой контрольных сумм, могут быть весьма значительны. Как минимум, процессор должен вычислить контрольную сумму каждого блока – как при сохранении данных (чтобы знать, что сохранять), так и при доступе к ним (чтобы сравнить с хранимой контрольной суммой). Во многих системах, где применяются контрольные суммы (в т. ч. сетевых стеках), для снижения процессорных накладных расходов копирование данных и вычисление контрольных сумм объединяются в составе одной операции; поскольку копирование в любом случае необходимо (например, чтобы скопировать данные из страничного кеша в ядре в пользовательский буфер), то объединение копирования с вычислением контрольной суммы может оказаться весьма эффективным.

Помимо процессорных накладных расходов, некоторые схемы контрольной суммы влекут за собой накладные расходы на ввод-вывод, особенно если контрольные суммы хранятся отдельно от данных (так что для доступа к ним требуются дополнительные операции). Также дополнительные затраты связаны с фоновой очисткой диска. Первый вид накладных расходов можно снизить путем правильного проектирования, а влияние второго ограничить, например, настройкой времени запуска очистки. Середина ночи, когда большинство (но не все!) продуктивных работников спят, – самое время для того, чтобы почистить диски и тем самым повысить надежность системы хранения.

45.9. РЕЗЮМЕ

Мы обсудили защиту данных в современных системах хранения, уделив особое внимание реализации и применению контрольных сумм. Различные контрольные схемы защищают от разных типов отказов; с развитием технологий хранения, несомненно, появятся и новые виды отказов. Возможно, это заставит исследователей и промышленность пересмотреть базовые подходы, а то и придумать совершенно новые. Время покажет. Или не покажет. Время – вообще забавная штука.

Литература

[B+07] «An Analysis of Latent Sector Errors in Disk Drives» by L. Bairavasundaram, G. Goodson, S. Pasupathy, J. Schindler. SIGMETRICS '07, San Diego, CA. *Первая статья, посвященная подробному изучению скрытых ошибок секторов. Стала победителем конкурса выдающихся студенческих работ имени Кеннета К. Севчика (OSTEP) – блестящего ученого и чудесного парня, который слишком рано ушел от нас. Чтобы доказать участникам OSTEP, что можно переехать из США в Канаду, Кен однажды спел канадский гимн, стоя посреди ресторана. Мы выбрали США, но воспоминание об этом случае сохранили.*

[B+08] «An Analysis of Data Corruption in the Storage Stack» by Lakshmi N. Bairavasundaram, Garth R. Goodson, Bianca Schroeder, Andrea C. Arpaci-Dusseau, Remzi H. Arpaci-Dusseau. FAST '08, San Jose, CA, February 2008. *Первая работа, в которой очень подробно изучено искажение данных на диске с упором на частоту подобных явлений на протяжении более трех лет. Исследование проводилось на выборке, насчитывающей более 1,5 млн дисков.*

[BS04] «Commercial Fault Tolerance: A Tale of Two Systems» by Wendy Bartlett, Lisa Spahnower. IEEE Transactions on Dependable and Secure Computing, Vol. 1:1, January 2004. *Классическая работа по построению отказоустойчивых систем – прекрасный обзор текущего состояния дел, выполненный сотрудниками компаний IBM и Tandem. Еще одна статья, обязательная для прочтения всеми интересующимися этим вопросом.*

[C+04] «Row-Diagonal Parity for Double Disk Failure Correction» by P. Corbett, B. English, A. Goel, T. Grcanac, S. Kleiman, J. Leong, S. Sankar. FAST '04, San Jose, CA, February 2004. *Ранняя работа о том, как дополнительная избыточность помогает решить проблему полного и частичного отказа диска. Прекрасный пример плодотворного объединения теории с практикой.*

[F04] «Checksums and Error Control» by Peter M. Fenwick. Доступно по адресу <http://www.ostep.org/Citations/checksums-03.pdf>. *Простое пособие по контрольным суммам, распространяемое совершенно бесплатно.*

[F82] «An Arithmetic Checksum for Serial Transmissions» by John G. Fletcher. IEEE Transactions on Communication, Vol. 30:1, January 1982. *Оригинальная работа Флетчера по контрольной сумме, названной в его честь. Сам-то он не называл ее контрольной суммой Флетчера, собственно, вообще никак не называл, позже это название стали употреблять другие люди. Так что не будем упрекать старину Флетча в похвальбе. Эта история приводит на память Рубика: сам Рубик никогда не говорил «кубик Рубика», а называл его «мой кубик».*

[HLM94] «File System Design for an NFS File Server Appliance» by Dave Hitz, James Lau, Michael Malcolm. USENIX Spring '94. *Первопроходческая работа, в которой описываются идеи и продукт, легший в основу компании NetApp. Впоследствии NetApp выросла в производителя систем хранения стоимостью миллиарды долларов. Если хотите получше узнать о NetApp, почитайте автобиографию Хитца «How to Castrate a Bull» (Как кастрировать быка) (так книга и называется, это никакая не шутка). А вы, небось, думали, что сможете избежать кастрации быков, занявшись информатикой?*

[K+08] «Parity Lost and Parity Regained» by Andrew Krioukov, Lakshmi N. Bairavasundaram, Garth R. Goodson, Kiran Srinivasan, Randy Thelen, Andrea C. Arpaci-Dusseau, Remzi H. Arpaci-Dusseau. FAST '08, San Jose, CA, February 2008. *В этой работе исследуется эффективность (или неэффективность) различных контрольных сумм для защиты данных. Мы выявили ряд интересных дефектов в современных стратегиях защиты.*

[M13] «Cyclic Redundancy Checks» by unknown. Доступно по адресу <http://www.mathpages.com/home/kmath458.htm>. *Исключительно ясное и краткое описание CRC. В интернете вообще полно информации, если поискать.*

[P+05] «IRON File Systems» by V. Prabhakaran, L. Bairavasundaram, N. Agrawal, H. Gunawi, A. Arpaci-Dusseau, R. Arpaci-Dusseau. SOSP '05, Brighton, England. *Наша работа о частичных отказах дисков, в которой подробно рассматривается реакция современных файловых систем на такие отказы. Плохо они реагируют! Мы нашли многочисленные ошибки, изъяны проектирования и другие странные вещи. Некоторые наши результаты нашли отклик в сообществе Linux и привели к повышению надежности файловых систем. Мы только рады!*

[RO91] «Design and Implementation of the Log-structured File System» by Mendel Rosenblum and John Ousterhout. SOSP '91, Pacific Grove, CA, October 1991. *Настолько блестящая работа, что мы еще раз приводим на нее ссылку.*

[S90] «Implementing Fault-Tolerant Services Using The State Machine Approach: A Tutorial» by Fred B. Schneider. ACM Surveys, Vol. 22, No. 4, December 1990. *Как конструируются отказоустойчивые службы. Обязательна для прочтения всеми, кто создает распределенные системы.*

[Z+13] «Zettabyte Reliability with Flexible End-to-end Data Integrity» by Y. Zhang, D. Myers, A. Arpaci-Dusseau, R. Arpaci-Dusseau. MSST '13, Long Beach, California, May 2013. *Как добавить защиту данных в страничный кеш системы. Места маловато, а то бы мы написали...*

Домашнее задание (эмуляция)

В этом домашнем задании мы воспользуемся программой `checksum.py` для изучения различных аспектов контрольных сумм.

Вопросы

1. Для начала запустите `checksum.py` без аргументов. Вычислите разные контрольные суммы: аддитивную, основанную на XOR и Флетчера. Проверьте свои ответы, задав флаг `-s`.
2. То же самое, но с различными начальными значениями генератора случайных чисел (флаг `-s`).
3. Иногда аддитивная и основанная на XOR контрольные суммы дают одинаковые значения (например, если все биты нулевые). Попробуйте найти такое 4-байтовое значение (задавая флаг `-D`, например `-D a,b,c,d`), не состоящее из одних нулей, для которого аддитивная и основанная на XOR контрольные суммы равны. Какое общее условие должно для этого выполняться? Проверьте свои ответы, задав флаг `-s`.
4. Теперь задайте 4-байтовое значение, для которого аддитивная и основанная на XOR контрольные суммы заведомо не равны. При каком общем условии такое бывает?
5. С помощью эмулятора вычислите контрольные суммы дважды (по одному разу для двух разных наборов чисел — `-D a1,b1,c1,d1` и `-D a2,b2,c2,d2`). Требуется, чтобы аддитивные контрольные суммы были при этом равны. При каком общем условии аддитивные контрольные суммы разных данных одинаковы? Проверьте ответ на конкретном примере с помощью флага `-s`.
6. Тот же вопрос для контрольной суммы на основе XOR.
7. Теперь рассмотрим конкретный набор данных. Каковы контрольные суммы (аддитивная, на основе XOR и Флетчера) данных `-D 1,2,3,4`? А для `-D 4,3,2,1`? Что интересного в этих трех контрольных суммах? Чем сумма Флетчера отличается от двух других? Чем сумма Флетчера в общем случае «лучше» простой аддитивной контрольной суммы?
8. Ни одна контрольная сумма не идеальна. Попробуйте найти два разных набора данных, для которых контрольная сумма Флетчера одинакова. Какое для этого должно выполняться общее условие? Начните с простой строки данных (например, `-D 0,1,2,3`) и посмотрите, можно ли заменить

одно число, так чтобы сумма Флетчера осталась той же? Как всегда, проверьте свой ответ с помощью флага -с.

Домашнее задание (код)

В этой части домашнего задания вам предстоит написать свою реализацию различных контрольных сумм.

Вопросы

1. Напишите короткую программу на С (назовите файл `check-xor.c`), которая вычисляет основанную на XOR контрольную сумму входного файла и печатает результат. Для хранения (однобайтовой) контрольной суммы используйте 8-разрядный тип `unsigned char`. Напишите тесты для проверки правильности своей программы.
2. Напишите программу на С (назовите файл `check-fletcher.c`), которая вычисляет контрольную сумму Флетчера входного файла. Снова протестируйте программу.
3. Сравните производительность обеих программ: какая быстрее? Как изменяется производительность при изменении размера файла? Для хронометража воспользуйтесь функцией `gettimeofday`. Какой функции вы отдадите предпочтение, если важна прежде всего производительность? А если способность обнаруживать ошибки?
4. Прочитайте о 16-разрядном CRC-коде и реализуйте его. Протестируйте на нескольких наборах входных данных. Сравните производительность с простой контрольной суммой на основе XOR и контрольной суммой Флетчера. А что можно сказать о способности обнаруживать ошибки?
5. Напишите программу (`create-csum.c`), которая вычисляет однобайтовую контрольную сумму для каждого 4-килобайтового блока файла и сохраняет результаты в выходном файле (заданном в командной строке). Напишите дополнительную программу (`check-csum.c`), которая читает файл, вычисляет контрольные суммы блоков и сравнивает результаты с хранящимися в другом файле. При обнаружении несоответствия программа должна напечатать, что файл поврежден. Протестируйте программу, повредив файл вручную.

Глава 46

.....

Итоговый диалог о долговременном хранении

Студент. Уф-ф, файловые системы – вещь вроде бы интересная (!), но сложная.

Профессор. Потому-то мы с супругой ими и занимаемся.

Студент. Секундочку. А Вы что, один из тех профессоров, кто написал эту книгу? Я-то думал, мы оба – мысленные конструкции для подведения итогов, ну и для внесения толики юмора в изучение операционных систем.

Профессор. Э... ну... может быть. Вообще-то не твое дело! Да и кто, по твоему мнению, все это писал? (вздыхая) Ну ладно, перейдем к делу: чему ты научился?

Студент. Думаю, что главное я понял – долговременное хранение данных организовать гораздо труднее, чем кратковременное (в оперативной памяти). Ведь если машина «падает», то все содержимое памяти пропадает. А данные в файловой системе должны храниться вечно.

Профессор. Ну, как любил говаривать мой друг Кевин Халтквист, «вечно – это довольно долго». Он, правда, говорил о пластиковых подставках для мячей в гольфе, но это верно и для мусора, который встречается в большинстве файловых систем.

Студент. Ну, Вы же понимаете, о чем я! По крайней мере, длительное время. И даже такие простые вещи, как обновление данных в запоминающем устройстве, сложны, потому что нужно думать о том, что может случиться в случае аварии. Восстановление – штука, о которой я даже не задумывался, когда мы говорили о виртуализации памяти, – теперь выходит на первое место!

Профессор. Именно так. Обновление устройства долговременного хранения всегда было и остается увлекательной и трудной задачей.

Студент. Я также узнал много интересного о планировании диска и о методах защиты данных вроде RAID-массивов и даже контрольных сумм. Это круто.

Профессор. Мне эти темы тоже нравятся. Хотя если вникать в них глубоко, то придется заняться математикой. Почитай последние материалы по стирающим кодам, если хочешь поломать мозги.

Студент. Обязательно читаю.

Профессор (хмурясь). Что-то мне слышится в твоем тоне сарказм. Ладно, что еще тебе понравилось?

Студент. Еще понравились идеи о создании систем, осведомленных о технологии, например FFS и LFS. Классная штука! Знать о диске – это круто. Но будет ли это нужно – ведь на смену идут флеш-память и еще более новые технологии?

Профессор. Хороший вопрос! И кстати, не забыть бы поработать над главой о флеш-памяти... (что-то пишет в блокноте)... Но даже при наличии флеш-памяти все эти идеи остаются важными и нужными. Например, уровень трансляции флеш-памяти (FTL) внутри имеет структуру журнала, что позволяет повысить производительность и надежность SSD на основе флеш-памяти. А помнить о локальности вообще никогда не вредно. Так что технологии могут меняться, но многие рассмотренные нами идеи по-прежнему останутся полезными, по крайней мере еще некоторое время.

Студент. Это здорово. Я столько времени потратил на их изучение, и не хотелось бы, чтобы все это было впустую!

Профессор. Профессора так с тобой не поступят, верно?

Глава 47

.....

Диалог о распределенности

Профессор. Вот мы и добрались до последнего кусочка в мире операционных систем: распределенных систем. Поскольку мы не можем уделить им много места, ограничимся кратким введением в этой части, посвященной хранению, акцентировав внимание в основном на распределенных файловых системах. Надеюсь, что это будет хорошо.

Студент. Звучит неплохо. Но что такое распределенная система, о славный и всезнающий профессор?

Профессор. Э, держу пари, ты знаешь, что я собираюсь сказать...

Студент. Опять персик?

Профессор. Точно! Но на этот раз он далеко от тебя, и чтобы до него добраться, нужно потратить некоторое время. А персиков-то полно! Но вот беда – иногда персик оказывается гнилым. А мы хотим, чтобы каждый, вонзивший зубы в сочную мякоть, насладился божественным вкусом.

Студент. Эта аналогия с персиками мне все меньше нравится.

Профессор. Спокойно! Это последняя, уж потерпи немного.

Студент. Ладно.

Профессор. Впрочем, бог с ними, с персиками. Строить распределенные системы трудно, т. к. то одно, то другое постоянно ломается. Сообщения теряются, машины выходят из строя, данные на дисках повреждаются. Весь мир – против тебя!

Студент. Но ведь я же постоянно пользуюсь распределенными системами, разве не так?

Профессор. Да! Пользуешься. И..?

Студент. Да все вроде работает. Ведь когда я отправляю запрос гуглу, ответ обычно приходит мгновенно, и результаты отличные! То же самое с Facebook, Amazon и т. д.

Профессор. *Да, просто поразительно. И это несмотря на все отказы! Эти компании встроили в свои системы уйму разных механизмов, чтобы они продолжали работать, даже когда часть машин выходит из строя. Для этого применяются самые разные методы: репликация, повторение и куча других трюков, разработанных за многие годы для обнаружения и восстановления после отказов.*

Студент. *Интересно. Не пора ли что-нибудь уже изучить?*

Профессор. *Пора. За работу! Но всему свое время... (кусает персик, который все время держал в руке, но тот, к сожалению, оказывается подгнившим).*

Глава 48

Распределенные системы

Распределенные системы изменили мир. Когда наш браузер подключается к веб-серверу где-то в другой точке планеты, кажется, что он принимает участие в простой форме распределенной системы – **клиент-серверной**. Но, обращаясь к современной веб-службе, например Google или Facebook, мы взаимодействуем не с одной машиной; за кулисами эти сложные службы состоят из большой совокупности (тысяч) машин, и все они слаженно работают во имя предоставления требуемой функциональности. Таким образом, должно быть понятно, почему изучение распределенных систем представляет такой интерес. Эта тема заслуживает отдельного курса, мы же кратко изложим лишь несколько основных вопросов.

При построении распределенной системы возникает ряд новых проблем. Больше всего нас будет интересовать проблема **отказа**; компьютеры, диски, сети и программы время от времени дают сбой, поскольку мы не умеем (и, вероятно, никогда не научимся) конструировать «идеальные» компоненты и системы. Но мы хотим, чтобы клиентам казалось, что веб-служба никогда не отказывает. Как достичь этой цели?

Существо проблемы:

КАК СТРОИТЬ СИСТЕМЫ, КОТОРЫЕ РАБОТАЮТ, НЕСМОТЯ НА ОТКАЗЫ КОМПОНЕНТОВ

Как построить правильно работающую систему из компонентов, которые иногда могут работать неправильно? Этот главный вопрос воскрешает в памяти некоторые темы, которые мы обсуждали при рассмотрении RAID-массивов, только сейчас проблемы, а вместе с ними и решения, более сложные.

Интересно, что отказ, являясь центральной проблемой конструирования распределенных систем, в то же время открывает возможность. Да, машины выходят из строя, но из этого еще не следует, что и система в целом должна перестать работать. Объединив несколько машин, мы можем построить систему, которая будет отказывать редко, хотя ее компоненты отказывают регулярно. В этом и состоят красота и ценность распределенных систем, именно поэтому они стоят практически за любой современной веб-службой, включая Google, Facebook и др.

Совет: связь принципиально ненадежна

Практически всегда разумно считать, что связь – принципиально ненадежная часть системы. Искажение битов, неработающие каналы и машины, нехватка памяти в буфере для входящих пакетов – все это ведет к одному результату: пакеты иногда не доходят до места назначения. Чтобы построить надежную службу поверх таких ненадежных сетей, необходимы методы, устойчивые к потере пакетов.

Есть и другие важные вопросы. Часто на первый план выходит **производительность**; в условиях, когда части распределенной системы соединены сетью, проектировщики должны продумывать, как достичь поставленных целей, а для этого нужно постараться уменьшить количество циркулирующих в сети сообщений и принять другие меры к повышению эффективности взаимодействия (снижение задержки, повышение пропускной способности).

Наконец, обязательно следует учитывать **безопасность**. При подключении к удаленному сайту нужно быть уверенным, что человек (или программа) на другом конце – именно тот, кем представляется. Кроме того, важно, чтобы никто не мог подслушать или изменить состав сообщений, которыми обмениваются стороны.

В этом введении мы рассмотрим самый главный и новый аспект работы распределенных систем: **коммуникация**. Как машины, входящие в состав распределенной системы, взаимодействуют между собой? Начнем с основного из имеющихся примитивов, сообщений, и надстроим над ним примитивы более высокого уровня. Как уже было сказано, в фокусе нашего внимания находятся отказы: как уровни взаимодействия должны обрабатывать отказы?

48.1. Основы коммуникации

Главный постулат современных сетей связи – коммуникация принципиально ненадежна. Будь то глобальный интернет или локальная высокоскоростная сеть типа Infiniband, пакеты регулярно теряются, искажаются или не достигают места назначения по какой-то другой причине.

Причин потери или искажения пакетов много. Иногда во время передачи некоторые биты инвертируются из-за воздействия электрических полей или сходных проблем. Иногда какой-то элемент системы, например линия связи или маршрутизатор пакетов, дает сбой. Бывает, что повреждаются сетевые кабели.

Но более фундаментальной причиной потери пакетов является отсутствие буферизации в сетевом коммутаторе, маршрутизаторе или оконечной точке. Даже если бы мы могли гарантировать правильную работу всех линий связи и компонентов системы (коммутаторов, маршрутизаторов и оконечных компьютеров), потери все равно возможны по следующей причине. Представьте себе, что маршрутизатору поступил пакет; чтобы его обработать, маршрутизатор должен поместить пакет в свою память. Но если одновременно поступило очень много пакетов, то памяти может и не хватить. И тогда

у маршрутизатора остается только один выбор – **отбросить** один или несколько пакетов. То же самое происходит в оконечном компьютере: когда одной машине поступает так много сообщений, что ее ресурсов не хватает, имеет место потеря пакетов.

Таким образом, потеря пакетов – фундаментальное свойство сетей. Вопрос: что с этим делать?

48.2. НЕНАДЕЖНЫЕ УРОВНИ КОММУНИКАЦИИ

Есть простой способ: ничего не делать. Поскольку некоторые приложения знают, как обрабатывать потерю пакетов, иногда полезно дать им взаимодействовать, пользуясь простым, но ненадежным уровнем обмена сообщениями; в обоснование этого принципа часто приводят **сквозной аргумент** (см. отступление в конце главы). Прекрасный пример такого ненадежного уровня дает протокол **UDP/IP**, который есть практически во всех современных системах. Чтобы воспользоваться UDP, процесс создает **оконечную точку коммуникации** с помощью API **сокетов**; процессы на разных машинах (или на одной машине) обмениваются **дейтаграммами** UDP (дейтаграммой называется сообщение фиксированного размера, не превышающего максимальный предел).

На рис. 48.1 и 48.2 показаны простой клиент и сервер на базе протокола UDP/IP. Клиент может послать сообщение серверу, тот в ответ посылает свое сообщение. Написав совсем немного кода, мы положили начало распределенной системе!

UDP – пример ненадежного коммуникационного уровня. Используя его, вы столкнетесь с ситуациями, когда пакеты теряются (отбрасываются) и потому не доходят до места назначения; при этом отправитель не получает никакой информации о потере. Но это не значит, что UDP вообще никак не защищен от сбоев. Например, в состав UDP-пакета входит **контрольная сумма** для обнаружения некоторых видов искажения пакетов.

Однако, поскольку многие приложения хотят отправлять данные получателю, не заботясь о потере пакетов, нам этого недостаточно. Мы должны построить надежный механизм коммуникации поверх ненадежной сети.

```

// код клиента
int main(int argc, char *argv[]) {
    int sd = UDP_Open(20000);
    struct sockaddr_in addrSnd, addrRcv;
    int rc = UDP_FillSockAddr(&addrSnd, "machine.cs.wisc.edu", 10000);
    char message[BUFFER_SIZE];
    sprintf(message, "hello world");
    rc = UDP_Write(sd, &addrSnd, message, BUFFER_SIZE);
    if (rc > 0) {
        int rc = UDP_Read(sd, &addrRcv, message, BUFFER_SIZE);
    }
    return 0;
}

// код сервера
int main(int argc, char *argv[]) {
    int sd = UDP_Open(10000);
    assert(sd > -1);
    while (1) {
        struct sockaddr_in addr;
        char message[BUFFER_SIZE];
        int rc = UDP_Read(sd, &addr, message, BUFFER_SIZE);
        if (rc > 0) {
            char reply[BUFFER_SIZE];
            sprintf(reply, "goodbye world");
            rc = UDP_Write(sd, &addr, reply, BUFFER_SIZE);
        }
    }
    return 0;
}

```

Рис. 48.1 ❖ Пример клиент-серверного кода с применением протокола UDP/IP

Совет: о применении контрольных сумм для проверки целостности

Контрольные суммы – распространенный способ быстро и эффективно обнаружить повреждение данных в современных системах. Для вычисления простой контрольной суммы нужно лишь сложить байты блока данных, но, конечно, придумано много более изощренных контрольных сумм, в том числе циклический избыточный код (CRC), контрольная сумма Флетчера и другие [МК09].

В компьютерных сетях контрольные суммы используются следующим образом. Отправитель вычисляет контрольную сумму байтов сообщения и посылает ее вместе с самим сообщением. Получатель также вычисляет контрольную сумму поступившего сообщения; если она совпадает с отправленной, то есть надежда, что данные не были повреждены во время передачи.

Контрольные суммы оцениваются по нескольким критериям. Самый важный – сила, или способность к обнаружению ошибок: приводит ли изменение данных к изменению контрольной суммы? Чем сильнее контрольная сумма, тем больше шансов, что изменение данных не останется незамеченным. Еще один важный критерий – производительность: каковы затраты на вычисление контрольной суммы? К сожалению, сила и производительность редко уживаются, т. е. высококачественную контрольную сумму труднее вычислить. В который раз повторим, что нет совершенства в мире.

```

int UDP_Open(int port) {
    int sd;
    if ((sd = socket(AF_INET, SOCK_DGRAM, 0)) == -1) { return -1; }
    struct sockaddr_in myaddr;
    bzero(&myaddr, sizeof(myaddr));
    myaddr.sin_family = AF_INET;
    myaddr.sin_port = htons(port);
    myaddr.sin_addr.s_addr = INADDR_ANY;
    if (bind(sd, (struct sockaddr *) &myaddr, sizeof(myaddr)) == -1) {
        close(sd);
        return -1;
    }
    return sd;
}

int UDP_FillSockAddr(struct sockaddr_in *addr, char *hostName, int port) {
    bzero(addr, sizeof(struct sockaddr_in));
    addr->sin_family = AF_INET;           // порядок байтов хост-компьютера
    addr->sin_port = htons(port);         // сетевой порядок байтов
    struct in_addr *inAddr;
    struct hostent *hostEntry;
    if ((hostEntry = gethostbyname(hostName)) == NULL) { return -1; }
    inAddr = (struct in_addr *) hostEntry->h_addr;
    addr->sin_addr = *inAddr;
    return 0;
}

int UDP_Write(int sd, struct sockaddr_in *addr, char *buffer, int n) {
    int addrLen = sizeof(struct sockaddr_in);
    return sendto(sd, buffer, n, 0, (struct sockaddr *) addr, addrLen);
}

int UDP_Read(int sd, struct sockaddr_in *addr, char *buffer, int n) {
    int len = sizeof(struct sockaddr_in);
    return recvfrom(sd, buffer, n, 0, (struct sockaddr *) addr,
                    (socklen_t *) &len);
}

```

Рис. 48.2 ❖ Простая библиотека UDP

48.3. НАДЕЖНЫЕ КОММУНИКАЦИОННЫЕ УРОВНИ

Для построения надежного коммуникационного уровня нам нужны новые механизмы и методы обработки потери пакетов. Рассмотрим простой пример, когда клиент посылает серверу сообщение по ненадежному соединению. Вот вам первый вопрос: откуда отправитель знает, что сообщение дошло до получателя?

Для этой цели используется техника **подтверждения**, или **квитирования** (acknowledgment, для краткости **ack**). Идея проста: отправитель посылает сообщение получателю; в ответ получатель посылает короткое сообщение, *подтверждающее* получение. Этот процесс изображен на рис. 48.3.

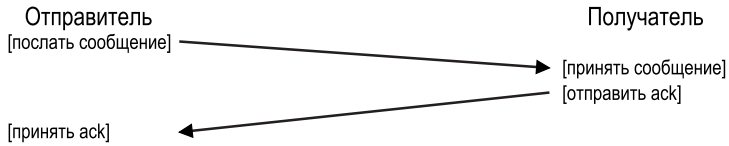


Рис. 48.3 ❖ Сообщение и подтверждение

Получив подтверждение, отправитель может быть уверен, что получатель действительно получил исходное сообщение. Но что должен делать отправитель, если подтверждение не получено?

Для обработки этого случая нам необходим дополнительный механизм – **тайм-аут**. Посылая сообщение, отправитель взводит таймер на определенное время. Если в течение этого времени подтверждение не приходит, то отправитель делает вывод, что сообщение потеряно. Тогда отправитель делает **повторную попытку** послать сообщение, надеясь, что на этот раз получится. Чтобы эта идея работала, отправитель должен хранить копию сообщения на случай, если его придется еще раз отправить позже. На рис. 48.4 показан пример.

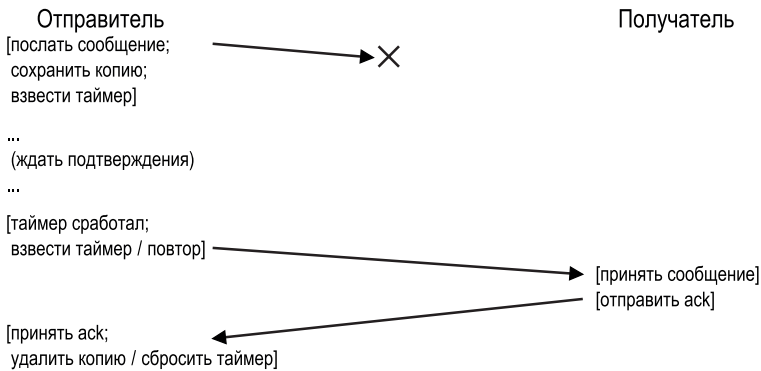


Рис. 48.4 ❖ Сообщение и подтверждение: отброшенный запрос

К сожалению, такого тайм-аута с повтором все же недостаточно. На рис. 48.5 приведен пример проблематичной потери пакета. Здесь потеряно не исходное сообщение, а подтверждение. С точки зрения отправителя, ситуация точно такая же: подтверждение не получено, поэтому по истечении тайм-аута сообщение посылается еще раз. Но с точки зрения получателя все выглядит иначе: одно и то же сообщение получено дважды! Бывают, конечно, ситуации, когда это нормально, но в общем случае это не так; представьте, что случилось бы, если бы дополнительные пакеты появились при скачивании файла. Поэтому для обеспечения надежного уровня передачи сообщений мы обычно хотим гарантировать, что каждое сообщение будет получено **ровно один раз**.

Чтобы получатель мог обнаружить дубликаты сообщений, отправитель должен каким-то образом однозначно идентифицировать каждое сообще-

ние, а получатель должен следить, не получал ли он такого сообщения раньше. Увидев дубликат сообщения, получатель подтверждает его, но (и это очень важно!) *не* передает сообщение принимающему приложению. Таким образом, отправитель получит подтверждение, но сообщение не будет принято дважды, т. е. семантика однократного получения будет соблюдена.

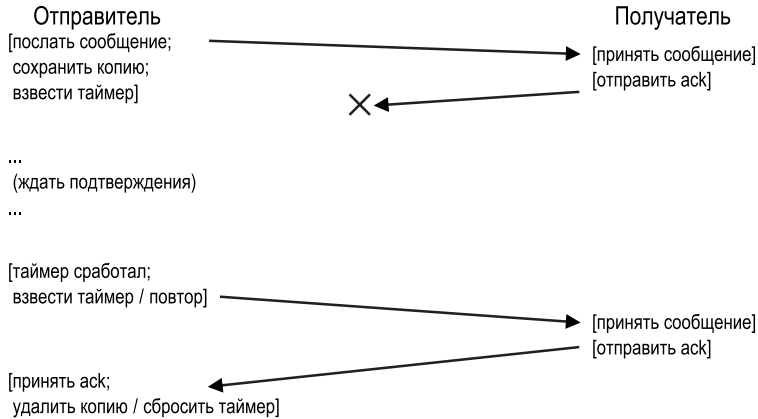


Рис. 48.5 ❖ Сообщение и подтверждение: отброшенный ответ

Существует масса способов обнаружить сообщения-дубликаты. Например, отправитель может генерировать для каждого сообщения уникальный идентификатор, а получатель – следить, встречался ли такой идентификатор раньше. Но подобный подход практически неприменим, потому что требует неограниченной памяти для хранения всех идентификаторов.

Решить проблему можно проще – с помощью механизма **порядкового номера**, который требует совсем немного памяти. В этом случае отправитель и получатель договариваются о начальном значении (например, 1) счетчика, поддерживаемого обеими сторонами. В состав каждого отправленного сообщения включается счетчик (N), который и играет роль идентификатора сообщения. Послав сообщение, отправитель увеличивает счетчик на 1.

Получатель сравнивает свое значение счетчика с пришедшим в сообщении. Если они совпадают, то получатель подтверждает сообщение и передает его приложению; в этом случае получатель понимает, что сообщение пришло впервые, увеличивает свой счетчик на 1 и ждет следующего сообщения.

Если подтверждение было потеряно, то на стороне отправителя срабатывает таймер, поэтому он посылает сообщение N еще раз. Но теперь счетчик получателя равен $N + 1$, поэтому получатель понимает, что такое сообщение он уже принимал. Раз так, то получатель подтверждает сообщение, но *не* передает его приложению. Таким простым способом счетчик порядковых номеров позволяет избежать дубликатов.

Самый популярный надежный коммуникационный уровень называется **ТСР/IP**, или просто **ТСР**. В этот протокол встроено куда больше хитростей, чем описано выше, включая механизмы для обработки перегрузки сети [VJ88], нескольких недоставленных запросов, а также различные оптими-

зации. Если интересно, почитайте дополнительную литературу, или, еще лучше, запишитесь на курс по программированию сетей.

48.4. АБСТРАКЦИИ КОММУНИКАЦИИ

Итак, мы разобрали базовый уровень передачи сообщений, и теперь встает следующий вопрос: какие абстракции коммуникации нужно использовать при построении распределенной системы?

СОВЕТ: ОСТОРОЖНЕЕ ПРИ УСТАНОВКЕ ТАЙМ-АУТА

Как вы, наверное, поняли, правильное задание величины тайм-аута – важная часть использования тайм-аутов для повторной отправки сообщений. Если тайм-аут слишком мал, то отправитель будет повторно посылать сообщения без нужды, напрасно транжиря ресурсы процессора и сети. Если же он слишком велик, то отправитель будет долго ждать перед повторной посылкой, поэтому воспринимаемая производительность снизится. Поэтому при наличии одного клиента и одного сервера «правильно» задать такую величину, чтобы ждать достаточно долго для обнаружения потери пакета, но не дольше.

Однако часто бывает, что в распределенной системе клиентов и серверов несколько. Когда много клиентов обмениваются сообщениями с одним сервером, потеря пакетов на стороне сервера может означать, что сервер перегружен. Если это так, то клиенты могут вести себя адаптивно, например после первого тайм-аута клиент увеличивает его величину, скажем, в два раза. Такая схема **экспоненциальной задержки** впервые была применена в ранней сети Aloha и затем принята в ранних версиях сети Ethernet [A70]. Она позволяет избежать перегрузки ресурсов из-за слишком частых повторных передач – в надежной системе подобного быть не должно.

С годами системщики разработали целый ряд подходов. Одно из направлений – обобщить абстракции ОС на распределенное окружение. Например, системы с **распределенной разделяемой памятью** (distributed shared memory – DSM) позволяют процессам, работающим на разных машинах, сообщая пользоваться большим виртуальным адресным пространством [LN89]. Эта абстракция превращает распределенное вычисление в подобие многопоточного приложения; единственное различие заключается в том, что потоки работают на разных компьютерах, а не на разных процессорах одного компьютера.

Большинство DSM-систем работают с помощью системы виртуальной памяти ОС. При обращении к странице на одной машине может случиться одно из двух. В первом (лучшем) случае нужная страница локальна для данной машины, поэтому данные выбираются быстро. Во втором случае страница находится на другой машине. Происходит страничный отказ, и его обработчик отправляет какой-то другой машине сообщение с просьбой выбрать страницу, помещает ее в таблицу страниц запросившего процесса и продолжает выполнение.

Этот подход сегодня не слишком распространен по ряду причин. Самая серьезная проблема DSM – обработка отказов. Допустим, что некоторая ма-

шина вышла из строя; что будет с находящимися на ней страницами? Что, если структуры данных, необходимые для распределенного вычисления, разбросаны по всему адресному пространству? В таком случае части этих структур внезапно станут недоступными. Обработать отказ в ситуации, когда часть адресного пространства пропала, трудно; представьте только связный список, в котором указатель на следующий элемент ведет в исчезнувшую часть адресного пространства. Кошмар!

Еще одна проблема – производительность. При написании кода обычно предполагается, что доступ к памяти обходится дешево. Но в DSM-системах некоторые обращения действительно дешевы, тогда как другие влекут за собой страничные отказы и дорогостоящее получение данных с другой машины. Поэтому программист должен очень внимательно относиться к организации вычислений, так чтобы по возможности избегать удаленного доступа, но тогда вся привлекательность этого подхода сходит на нет. Несмотря на многочисленные исследования в этой области, практической отдачи мало; на данный момент никто не занимается созданием надежных распределенных DSM-систем.

48.5. Удаленный вызов процедур (RPC)

Абстракции ОС оказались неважным выбором для построения распределенных систем, но вот абстракции языков программирования имеют куда больше смысла. Самая распространенная основана на идее **удаленного вызова процедур** (remote procedure call – **RPC**) [BN84]¹.

Все пакеты удаленного вызова процедур преследуют простую цель: сделать процесс выполнения кода на удаленной машине таким же простым и естественным, как вызов локальной функции. Клиент производит вызов процедуры и спустя некоторое время получает результат. Сервер же определяет процедуры, которые готов экспортировать. Всю магию вершит система RPC, которая в общем случае состоит из двух частей: **генератора заглушек** (иногда его называют **компилятором протокола**) и **библиотеки времени выполнения**. Рассмотрим их подробнее.

Генератор заглушек

Задача генератора заглушек проста: автоматизировать упаковку аргументов и результатов функции в сообщения, сняв это бремя с плеч программиста. Преимущества такого подхода многообразны: по определению, отсутствуют простые ошибки, характерные для написания подобного кода вручную, оптимизируется сам код, и, значит, повышается его производительность.

¹ В современных языках программирования иногда говорят об **удаленном вызове методов** (remote method invocation – **RMI**), но кому они нужны, эти современные языки с их дурацкими объектами?

На вход компилятору подается набор вызовов, которые сервер хочет экспортировать клиентам. Концептуально это может выглядеть так:

```
interface {
    int func1(int arg1);
    int func2(int arg1, int arg2);
};
```

Генератор заглушек принимает такой интерфейс и порождает различные фрагменты кода. Для клиента генерируется **клиентская заглушка**, содержащая все определенные в интерфейсе функции. Клиентская программа, желающая воспользоваться службой RPC, компонуется с заглушкой и вызывает ее, когда требуется вызвать удаленную процедуру.

Под капотом каждая функция в клиентской заглушке делает все необходимое для выполнения удаленного вызова процедуры. Клиенту код кажется обычным вызовом функции (например, `func1(x)`), тогда как на самом деле происходит следующее.

- **Создать буфер сообщения.** Обычно это непрерывный массив байтов определенного размера.
- **Упаковать в буфер необходимую информацию.** В состав этой информации входит некоторый идентификатор вызываемой функции, а также все ее аргументы (в нашем примере один целый аргумент для функции `func1`). Процесс упаковки информации в непрерывный буфер иногда называют **маршалингом** аргументов, или **сериализацией** сообщения.
- **Отправить сообщение адресату – RPC-серверу.** Все детали взаимодействия с RPC-сервером берет на себя библиотека времени выполнения RPC, описанная ниже.
- **Дождаться ответа.** Поскольку вызовы функций обычно синхронные, заглушка ждет завершения вызова.
- **Распаковать код возврата и другие аргументы.** Если функция возвращает всего одно значение, то с этим проблем не возникает. Но бывают функции, возвращающие более сложные результаты (например, список), и тогда заглушка должна распаковать их тоже. Этот процесс называется **демаршалингом**, или **десериализацией**.
- **Возврат управления вызывающей стороне.** Наконец, нужно вернуть управление из заглушки клиентскому коду.

Генерирует также код для сервера, который должен выполнять следующие действия.

- **Распаковать сообщение.** На этом шаге демаршалинга, или десериализации, извлекается информация из поступившего сообщения: идентификатор функции и ее аргументы.
- **Вызвать функцию.** Наконец-то! Вот мы и добрались до места, где выполняется удаленная функция. Библиотека выполнения RPC вызывает функцию, зная ее идентификатор, и передает ей аргументы.
- **Упаковать результаты.** Возвращаемое значение (или несколько значений) упаковывается в буфер ответа.

- **Отправить ответ.** На последнем шаге ответ отправляется вызывающей стороне.

Следует упомянуть еще несколько важных аспектов генератора заглушек. Первая – сложные аргументы: как упаковать и отправить сложную структуру данных? Например, системный вызов `write()` принимает три аргумента: целочисленный дескриптор файла, указатель на буфер и размер этого буфера (сколько байтов в него можно записать). Получив указатель, пакет RPC должен решить, как его интерпретировать, и выполнить правильное действие. Обычно эта задача решается либо с помощью хорошо известных типов (например, тип `buffer_t` используется для передачи блоков данных заданного размера, и компилятор RPC его понимает), либо посредством аннотирования структур данных дополнительной информацией, которая сообщает компилятору, сколько байтов нужно сериализовать).

Еще один важный вопрос – организация сервера с точки зрения конкурентности. Простой сервер ожидает поступления запросов в цикле и обрабатывает по одному запросу за раз. Но, как вы понимаете, это совершенно неэффективно: если один вызов RPC блокирует выполнение (например, ожидает ввода-вывода), то ресурсы сервера недоиспользуются. Поэтому обычно сервер представляет собой конкурентную программу. Часто организуется **пул потоков**. В этом случае на этапе инициализации сервера создается конечный набор потоков. Поступившее сообщение передается одному из этих рабочих потоков, который выполняет все, что требуется от вызова RPC, и в конечном итоге отвечает. В течение этого времени главный поток продолжает принимать запросы и распределять их между другими рабочими потоками. При такой организации вычисления внутри сервера можно выполнять конкурентно, так что коэффициент использования его ресурсов повышается. Правда, это сопровождается стандартными издержками, в основном в виде дополнительной сложности программирования, поскольку теперь вызовы RPC должны использовать блокировки и другие примитивы синхронизации для обеспечения корректности операций.

Библиотека времени выполнения

На библиотеку времени выполнения приходится основной груз работы в RPC-системе, именно от нее зависят производительность и надежность. Обсудим основные проблемы, стоящие перед построением этого слоя программного обеспечения.

Первым делом нужно как-то найти удаленную службу. Эта проблема **именования** общая для всех распределенных систем и в некотором смысле выходит далеко за рамки нашего обсуждения. Проще всего положить в основу существующие системы именования, например имя хоста и номер порта, являющиеся частью протоколов интернета. В такой системе клиент должен знать имя или IP-адрес машины, на которой работает нужная ему RPC-служба, а также используемый ей номер порта (номер порта – это способ определить конкретную коммуникационную службу, одна машина может обслуживать сразу несколько портов). Затем набор протоколов должен предоставить ме-

ханизм маршрутизации пакетов на конкретный адрес от любого другого компьютера в системе. Полное обсуждение систем именования смотрите в других источниках, например почитайте о системе доменных имен (DNS) в интернете, а еще лучше – замечательную главу в книге Saltzer and Kaashoek [SK09].

Решив вопрос о том, как клиент узнает, с каким сервером взаимодействовать для получения конкретной удаленной службы, нужно подумать, на базе какого протокола транспортного уровня строить RPC. Должна ли система RPC использовать надежный протокол типа TCP/IP или ненадежный коммуникационный уровень типа UDP/IP? На первый взгляд, ответ ясен: конечно же, мы хотим обеспечить надежную доставку запроса удаленному серверу и столь же надежную доставку ответа. Значит, нужно выбирать надежный транспортный протокол, например TCP, разве не так?

К сожалению, построение RPC поверх надежного коммуникационного уровня может резко снизить эффективность. Вспомните, как работает такой уровень: нужны подтверждения и тайм-аут вместе с повторной передачей. Когда клиент отправляет RPC-запрос серверу, тот посылает в ответ подтверждение, чтобы клиент знал, что запрос благополучно пришел. Аналогично, когда сервер отправляет ответ клиенту, тот подтверждает получение. Если протокол запрос–ответ (каковым является и RPC) построен поверх надежного коммуникационного уровня, то отправляется два «лишних» сообщения.

Поэтому многие RPC-пакеты построены поверх ненадежных коммуникационных уровней, например UDP. Так уровень RPC получается более эффективным, но ответственность за обеспечение надежности возлагается на саму систему RPC. Надежность достигается с помощью таких же тайм-аутов, повторных передач и подтверждений, что были описаны выше. Применяя ту или иную форму порядковой нумерации, коммуникационный уровень может гарантировать, что каждый удаленный вызов происходит ровно один раз (если отказов не было) или не более одного раза (при наличии отказов).

Другие проблемы

Существуют и другие проблемы, которые должна решать библиотека времени выполнения RPC. Например, что, если удаленный вызов занимает много времени? Из-за механизма тайм-аута длительный вызов может показаться клиенту отказом, поэтому он инициирует повторную попытку, и это необходимо как-то обрабатывать. Одно из возможных решений – явное подтверждение (от получателя отправителю), если ответ не возвращается немедленно; таким образом, клиент будет знать, что сервер получил запрос. Затем клиент может периодически спрашивать сервер, работает ли еще тот над запросом; если сервер отвечает «да», клиент соглашается подождать еще (в конце концов, иногда для завершения вызова действительно нужно много времени).

Библиотека времени выполнения должна также учитывать, что аргументы функции могут быть длинными и не помещаться в один пакет. Некоторые сетевые протоколы низкого уровня допускают **фрагментацию** на стороне отправителя (разбиение больших пакетов на меньшие части) и **сборку** (час-

тей в логическое целое) на стороне получателя. Если же этот механизм не поддерживается, то библиотека времени выполнения RPC должна реализовать его самостоятельно. Дополнительные сведения см. в статье Birrell and Nelson [BN84], посвященной RPC.

Один из вопросов, стоящих перед многими системами, – **порядок байтов**. Как вы знаете, одни машины хранят байты в **прямом порядке** (little endian), а другие – в **обратном** (big endian). Обратный порядок означает, что байты (например, целого числа) хранятся от старших битов к младшим, как цифры в арабских числительных, а прямой порядок – наоборот. Ни один порядок ничем не лучше другого, проблема же в том, как организовать взаимодействие машин с *разным* порядком байтов.

ОТСТУПЛЕНИЕ: СКВОЗНОЙ АРГУМЕНТ

Сквозной аргумент утверждает, что самый верхний уровень системы, т. е. обычно «оконечное» приложение, в конечном счете является единственным местом в многоуровневой системе, где некоторая функциональность может быть реализована по-настоящему. В своей основополагающей работе [SRC84] Зальцер с коллегами доказывают это на убедительном примере: надежной передаче файла между двумя компьютерами. Если мы хотим передать файл с компьютера *A* на компьютер *B* и при этом гарантировать, что все байты, оказавшиеся на *B*, в точности совпадают с исходными байтами на *A*, то должны осуществить «сквозную» проверку; механизмы обеспечения надежности на нижних уровнях, например в сети или на диске, не могут дать такой гарантии.

Противоположностью этому подходу является попытка решить проблему надежной передачи файла путем добавления механизмов обеспечения надежности на нижние уровни системы. Допустим, к примеру, что мы строим надежный коммуникационный протокол, который хотим использовать для передачи файлов. Протокол гарантирует, что каждый байт, переданный отправителем, будет принят получателем в правильном порядке, для чего используются, например, тайм-аут с повторной передачей, подтверждения и порядковые номера. К сожалению, такой протокол не гарантирует надежной передачи файла; действительно, представьте, что байты были как-то искажены в памяти отправителя еще перед началом передачи или что с ними что-то случилось при записи получателем на диск. Тогда, несмотря на надежность доставки данных по сети, вся процедура передачи файла ненадежна. А чтобы добиться надежной передачи, необходимо включить сквозные проверки, например после завершения передачи прочитать файл с диска получателя, вычислить его контрольную сумму и сравнить ее с контрольной суммой того же файла на стороне отправителя.

Отсюда следует, что иногда включение в нижние уровни дополнительной функциональности может повысить производительность системы или еще как-то оптимизировать ее. Поэтому не следует отказываться от таких низкоуровневых механизмов, но нужно тщательно оценивать их полезность с точки зрения конечного применения в системе или приложении в целом.

Пакеты RPC решают эту проблему путем точного определения порядка байтов в форматах сообщений. В пакете **XDR (eXternal Data Representation)** компании Sun такой порядок определен. Если внутренний порядок байтов на машине, которая отправляет или принимает сообщение, совпадает с тем, что определен в XDR, то сообщения не подвергаются никакому преобразованию.

Если же машина на какой-то стороне соединения имеет другой порядок байтов, то все информационные элементы сообщения следует преобразовать. Поэтому различие в порядке байтов влечет небольшие затраты.

И последний вопрос: следует ли раскрывать асинхронную природу коммуникации клиентам, что открыло бы дорогу некоторым оптимизациям. В типичном пакете RPC коммуникация **синхронная**, т. е., вызвав процедуру, клиент должен дождаться возврата управления от нее и только потом сможет продолжить работу. Поскольку ожидание может затянуться, а у клиента, возможно, есть и другие дела, то некоторые пакеты RPC допускают **асинхронные** вызовы. При асинхронном удаленном вызове процедуры пакет RPC отправляет запрос и сразу же возвращает управление; клиент может заняться чем-то еще, например произвести другие RPC-вызовы или выполнить полезные вычисления. Когда клиент захочет увидеть результаты асинхронного RPC-вызова, он снова обратится к уровню RPC и попросит его дождаться завершения начатого вызова, после чего сможет получить доступ к возвращенным значениям.

48.6. РЕЗЮМЕ

Мы познакомились с новой темой, распределенными системами, и важным вопросом, относящимся к ней: как обрабатывать отказы. В компании Google говорят, что на настольных компьютерах отказы – редкое явление, а в ЦОДе, насчитывающем тысячи компьютеров, отказы случаются постоянно. Ключ к успеху распределенной системы – как она справляется с такими отказами.

Мы также видели, что коммуникация – сердце распределенной системы. Распространенная абстракция такой коммуникации – удаленный вызов процедуры (RPC) – позволяет клиентам обращаться к функциям на стороне сервера; пакет RPC берет на себя всю грязную работу, включая подтверждения, тайм-ауты и повторную передачу, стремясь предоставить службу, которая мало чем отличается от локального вызова процедур.

Конечно, лучший способ разобраться в пакете RPC – попробовать им воспользоваться. Система RPC от компании Sun с ее генератором заглушек *grsdp* считается устаревшей; более современные Google gRPC и Apache Thrift преследуют те же цели. Попробуйте и оцените сами.

Литература

[A70] «The ALOHA System – Another Alternative for Computer Communications» by Norman Abramson. The 1970 Fall Joint Computer Conference. *В сети ALOHA впервые были представлены такие базовые идеи организации сетей, как экспоненциальная задержка и повторная передача, много лет составлявшие основу взаимодействия в сетях Ethernet с общей шиной.*

[BN84] «Implementing Remote Procedure Calls» by Andrew D. Birrell, Bruce Jay Nelson. ACMTOCS, Volume 2:1, February 1984. *Первопроходческая система RPC,*

на базе которой построены все остальные. Да, еще одна пионерская работа наших друзей из Xerox PARC.

[МК09] «The Effectiveness of Checksums for Embedded Control Networks» by Theresa C. Maxino and Philip J. Koopman. IEEE Transactions on Dependable and Secure Computing, 6:1, January '09. Обзор базовых механизмов контрольной суммы и сравнение их производительности и надежности.

[LH89] «Memory Coherence in Shared Virtual Memory Systems» by Kai Li and Paul Hudak. ACM TOCS, 7:4, November 1989. Построение программно разделяемой памяти с помощью виртуальной памяти. Идея выглядела интригующе, но в конечном итоге не пошла и надолго не задержалась.

[SK09] «Principles of Computer System Design» by Jerome H. Saltzer and M. Frans Kaashoek. Morgan-Kaufmann, 2009. Прекрасная книга о системах, обязательно должна стоять на каждой книжной полке. Одно из лучших обсуждений систем именования, которые нам встречались.

[SRC84] «End-To-End Arguments in System Design» by Jerome H. Saltzer, David P. Reed, David D. Clark. ACM TOCS, 2:4, November 1984. Великолепное обсуждение многоуровневых структур, абстрагирования и конечного места функциональности в компьютерных системах.

[VJ88] «Congestion Avoidance and Control» by Van Jacobson. SIGCOMM '88. Пионерская работа о приспособлении клиентов к воспринимаемой перегрузке сети; безусловно, одна из важнейших составных частей технологии интернета и обязательное чтение для любого, кто серьезно занимается системами. И для родственников ван Якобсона, потому что родственники просто обязаны читать все ваши статьи.

Домашнее задание (код)

В этом разделе вы напишете простой код, который позволит лучше познакомиться с коммуникацией. Удачи!

Вопросы

1. Пользуясь кодом, приведенным в этой главе, напишите простой сервер и клиента на основе UDP. Сервер должен принимать сообщения от клиента и посылать в ответ подтверждение. Пока что не добавляйте повторной передачи и не пытайтесь добиться надежности (считайте, что сеть работает идеально). Тестирование организуйте сначала на одной машине, а затем на двух.
2. Преобразуйте свой код в **коммуникационную библиотеку**. То есть спроектируйте свой собственный API, включающий функции отправки и получения сообщений и всё, что еще необходимо. Перепишите код клиента и сервера с использованием этой библиотеки без обращения к API сокетов.

3. Добавьте в свою развивающуюся библиотеку средства обеспечения надежности в виде **тайм-аута и повторной попытки**. Библиотека должна сохранять копии всех отправляемых сообщений. В момент отправки она должна взвести таймер, чтобы знать, сколько прошло времени. На стороне получателя библиотека должна **подтверждать** принятые сообщения. Клиентская функция отправки должна **блокировать** выполнение, т. е. ждать подтверждения, прежде чем вернуть управления. Она также должна быть готова повторять отправку бесконечно. Размер одного сообщения не должен превышать максимума, допустимого протоколом UDP. Наконец, тайм-аут и повторная передача должны быть реализованы эффективно, т. е. вызывающий процесс должен засыпать до прибытия подтверждения или срабатывания таймера; *не* пытайтесь крутиться в цикле, транжиря процессорное время!
4. Сделайте библиотеку более эффективной и функционально насыщенной. Для начала добавьте возможность передачи очень больших сообщений. Хотя сеть налагает ограничения на максимальный размер сообщения, библиотека должна принимать от клиента сообщения произвольного размера и передавать их серверу. Передача таких сообщений должна осуществляться по частям; библиотечный код на стороне сервера должен собирать из частей единое целое и передавать приложению один большой буфер.
5. Теперь сделайте то же самое, но более эффективно. Вместо того чтобы передавать фрагменты по одному, быстро передайте много фрагментов, чтобы полнее загрузить сеть. Но при этом помечайте каждый фрагмент, чтобы при сборке на стороне получателя фрагменты не перепутались.
6. И последнее усовершенствование реализации: асинхронная передача сообщений с доставкой по порядку. То есть клиент должен повторно вызывать функцию отправки для передачи одного сообщения за другим, а получатель должен надежно принимать сообщения по порядку, притом что одновременно в процессе передачи может находиться несколько сообщений. Кроме того, добавьте на стороне отправителя вызов, который позволил бы клиенту ждать подтверждения всех отправленных разом сообщений.
7. И еще одна болевая точка: измерение. Измерьте пропускную способность каждого из описанных выше подходов; сколько данных получается передать с одной машины на другую в единицу времени, с какой скоростью? Также измерьте задержку: сколько времени проходит от отправки одного пакета до получения подтверждения? Кажутся ли полученные цифры логичными? Чего вы ожидали? Как уточнить ожидания, чтобы лучше понимать, есть проблема или код работает хорошо?

Глава 49

Сетевая файловая система Sun (NFS)

Одним из первых применений клиент-серверных вычислений стали распределенные файловые системы. В этом случае имеется несколько клиентских машин и один или небольшое число серверов; сервер хранит данные на своих дисках, а клиенты запрашивают данные по четко определенному протоколу. Схематически система изображена на рис. 49.1.

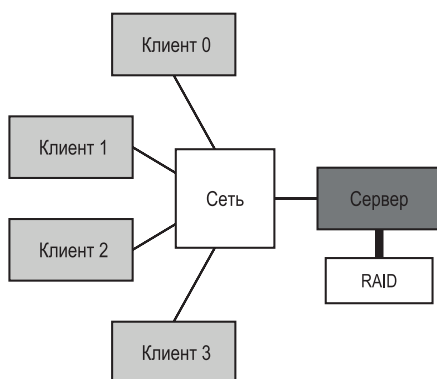


Рис. 49.1 ❖ Обобщенная клиент-серверная система

Как видим, сервер располагает дисками, а клиенты отправляют по сети сообщения, чтобы получить доступ к файлам и каталогам на этих дисках. Зачем нужна такая организация (т. е. почему бы клиентам просто не пользоваться своими локальными дисками)? Прежде всего это позволяет упростить **разделение** данных между клиентами. То есть Клиент 0 и Клиент 2, нуждающиеся в доступе к одному и тому же файлу, будут видеть единое представление файловой системы. Данные естественным образом разделяются. Второе преимущество – **централизованное администрирование**; например, резервное копирование файлов достаточно выполнить на небольшом числе серверов, а не на всех многочисленных клиентах. И наконец, **безопасность**; размещение серверов в запертом машинном зале предотвращает некоторые проблемы.

СУЩЕСТВО ПРОБЛЕМЫ: КАК ПОСТРОИТЬ РАСПРЕДЕЛЕННУЮ ФАЙЛОВУЮ СИСТЕМУ

Как построить распределенную файловую систему? О каких аспектах следует подумать в первую очередь? Что может пойти не так? Чему можно научиться на опыте существующих систем?

49.1. ПРОСТАЯ РАСПРЕДЕЛЕННАЯ ФАЙЛОВАЯ СИСТЕМА

Изучим архитектуру упрощенной распределенной файловой системы. Такая система состоит из большего числа компонентов, чем мы видели ранее. На стороне клиента имеются приложения, которые обращаются к файлам и каталогам с помощью **клиентской файловой системы**. Приложение выполняет **системные вызовы** к этой файловой системе (например, `open()`, `read()`, `write()`, `close()`, `mkdir()` и т. д.), чтобы получить доступ к файлам, хранящимся на сервере. Таким образом, с точки зрения клиентского приложения, файловая система ничем не отличается от локальной (размещенной на диске), разве что производительность. Следовательно, распределенные файловые системы предоставляют **прозрачный** доступ к файлам, это и есть очевидная цель – кто же захочет пользоваться файловой системой с другим набором API и вообще неудобной?

Роль клиентской файловой системы – выполнять действия, необходимые для обслуживания этих системных вызовов. Например, если клиент вызывает функцию `read()`, то клиентская файловая система должна отправить сообщение **серверной файловой системе** (или просто **файловому серверу**) с просьбой прочитать определенный блок; файловый сервер читает блок с диска (или из своего кеша в памяти) и отправляет клиенту сообщение, содержащее запрошенные данные. Клиентская файловая система копирует данные в пользовательский буфер, указанный при вызове `read()`, и на этом запрос завершается. Отметим, что последующий вызов `read()` для чтения того же блока клиентом может быть удовлетворен из **кеша** в памяти клиента или даже на его диске; при благоприятных обстоятельствах это вообще не потребует передачи по сети.

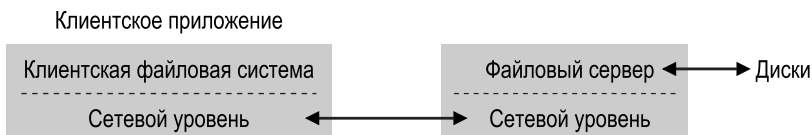


Рис. 49.2 ❖ Архитектура распределенной файловой системы

Из этого простого обзора ясно, что клиент-серверная распределенная файловая система состоит из двух важных частей: клиентской файловой системы

и файлового сервера. В совокупности они определяют поведение распределенной файловой системы. Теперь перейдем к изучению конкретной системы: сетевой файловой системы от компании Sun (Network File System – NFS).

ОТСТУПЛЕНИЕ: ПОЧЕМУ ПАДАЮТ СЕРВЕРЫ

Прежде чем переходить к обсуждению деталей протокола NFSv2, следует задаться вопросом: почему серверы «падают»? Причин, как вы, наверное, понимаете, много. Возможно, просто произошло **отключение электропитания** (временное), и как только питание восстановится, компьютеры перезапустятся. Зачастую код сервера насчитывает сотни тысяч или даже миллионы строк, поэтому в нем есть **дефекты** (даже в хорошей программе на каждую сотню или тысячу строк встречается несколько дефектов), которые рано или поздно проявятся и приведут к краху сервера. Бывают также утечки памяти, а даже небольшая утечка в конечном итоге приведет к нехватке памяти и как следствие краху. И наконец, в распределенных системах между клиентом и сервером находится сеть, и если она ведет себя аномально (например, становится **разделенной**, так что клиенты и серверы работают, но не могут взаимодействовать), то может показаться, будто удаленная машина упала, хотя в действительности она просто недостижима по сети.

49.2. ВПЕРЕД К NFS

Одна из самых ранних и весьма успешных распределенных систем разработана компанией Sun Microsystems и называется Sun Network File System (или NFS) [S86]. При этом Sun выбрала необычный подход: вместо того чтобы строить закрытую проприетарную систему, опубликовала **открытый протокол**, точно определяющий формат сообщений, которыми должны обмениваться клиенты и серверы. Различные группы получили возможность разрабатывать свои реализации NFS-серверов и конкурировать с NFS на рынке, сохраняя интероперабельность. Это сработало: сегодня немало компаний продают NFS-серверы (Oracle/Sun, NetApp [HLM94], EMC, IBM и другие), и широкое распространение NFS, вероятно, связано с таким выходом на «открытый рынок».

49.3. АКЦЕНТ НА ПРОСТОМ И БЫСТРОМ ВОССТАНОВЛЕНИИ ПОСЛЕ АВАРИИ ФАЙЛОВОГО СЕРВЕРА

В этой главе мы обсудим классический протокол NFS версии 2 (NFSv2), который много лет был стандартом; небольшие изменения были внесены при переходе на версию NFSv3, а более обширные – в версию NFSv4. Однако NFSv2 одновременно очаровывает и разочаровывает, поэтому именно он будет интересовать нас больше всего.

Главной целью, поставленной при проектировании NFSv2, было *простое и быстрое восстановление после аварии файлового сервера*. В среде, где работает один сервер и много клиентов, эта цель более чем осмыслена: в тот самый момент, когда сервер «падает» (или становится недоступным), все клиентские машины (и их пользователи) ничего не могут делать – и горько плачут. Насколько хорошо серверу, настолько хорошо и всей системе.

49.4. Ключ к быстрому восстановлению: ОТСУТСТВИЕ ИНФОРМАЦИИ О СОСТОЯНИИ

Эта цель реализована в протоколе NFSv2 путем отказа от хранения информации о состоянии. По определению, сервер не запоминает никакой информации о происходящем на стороне клиентов. Например, сервер не знает, какие клиенты какие блоки кешируют, какие файлы открыты каждым клиентом, текущую позицию файлового указателя и т. д. Проще говоря, сервер не знает, что делают клиенты, в каждом запросе передается *вся информация*, необходимая для его выполнения. Если сейчас этот подход не кажется вам разумным, подождите, пока мы дойдем до подробного обсуждения протокола.

В качестве примера протокола **с запоминанием состояния** (в отличие от протокола без запоминания) рассмотрим системный вызов `open()`. Получив путь к файлу, `open()` возвращает дескриптор файла (целое число). Этот дескриптор указывается в последующих вызовах `read()` и `write()` для получения доступа к блокам файла, как в показанном ниже прикладном коде (надлежащая проверка ошибок для краткости опущена):

```
char buffer[MAX];
int fd = open("foo", O_RDONLY); // получить дескриптор "fd"
read(fd, buffer, MAX);         // читать MAX байт из foo (через fd)
read(fd, buffer, MAX);         // читать MAX байт из foo
...
read(fd, buffer, MAX);         // читать MAX байт из foo
close(fd);                     // закрыть файл
```

Рис. 49.3 ❖ Клиентский код: чтение из файла

Представьте теперь, что клиентская файловая система открывает файл, отправляя серверу сообщение «открой файл 'foo' и верни мне дескриптор». Файловый сервер открывает файл локально и отправляет его дескриптор клиенту. При последующих операциях чтения клиент передает этот дескриптор системному вызову `read()`, а клиентская файловая система отправляет дескриптор файловому серверу в сообщении такого содержания: «прочитай байты из файла, на который указывает дескриптор, который я тебе передаю».

В этом примере дескриптор файла – часть **разделяемого** клиентом и сервером состояния (Оустерхаут называет его **распределенным состоянием** [O91]). Как мы уже намекнули, разделяемое состояние затрудняет восста-

новление после аварии. Допустим, что сервер «падает» после завершения первого чтения, но до того, как клиент отправил следующий запрос. Когда сервер «поднимется», клиент отправит второй запрос чтения. Увы, сервер понятия не имеет, на что ссылается `fd`; эта информация была эфемерной (хранилась только в памяти), поэтому после аварии сервера потеряна. Чтобы выйти из этой ситуации, клиент и сервер должны договориться о **протоколе восстановления**, требующем, чтобы клиент сохранял в памяти достаточно информации, чтобы сообщить серверу все, что тому нужно знать (в данном случае – что дескриптор `fd` ссылается на файл `foo`).

Картина становится еще мрачнее, если принять во внимание, что сервер с запоминанием состояния должен обрабатывать отказы клиентов. Представьте, к примеру, что клиент открыл файл и после этого потерпел аварию. Вызов `open()` занял дескриптор файла на сервере; как сервер узнает, что этот файл можно закрыть? При нормальной работе клиент в конечном итоге вызвал бы `close()`, проинформировав тем самым сервер о том, что файл можно закрывать. Но поскольку клиент «упал», сервер не получил этой информации, и потому для закрытия файла ему нужно как-то узнать об аварии клиента.

В силу этих причин проектировщики NFS выбрали подход без запоминания состояния: сообщение о каждой клиентской операции содержит все необходимое для выполнения запроса. И не нужно никакого изощренного восстановления; сервер просто перезапускается, а клиент в худшем случае должен будет повторить запрос.

49.5. Протокол NFSv2

Вот мы и подошли к определению протокола NFSv2.

СУЩЕСТВО ПРОБЛЕМЫ: КАК ОПРЕДЕЛИТЬ ФАЙЛОВЫЙ ПРОТОКОЛ БЕЗ ЗАПОМИНАНИЯ ИНФОРМАЦИИ О СОСТОЯНИИ

Как определить сетевой протокол, допускающий работу без запоминания состояния? Очевидно, что такие вызовы, как `open()`, сразу отпадают (т. к. это потребовало бы от сервера помнить, какие файлы открыты), однако же клиентское приложение хочет вызывать функции `open()`, `read()`, `write()`, `close()` и другие для обращения к файлам и каталогам. Поэтому уточним вопрос: как определить протокол, который одновременно не требовал бы запоминания состояния и поддерживал API файловой системы, определенные в POSIX?

Ключом к пониманию устройства протокола NFS является **описатель файла** (file handle). Описатель файла позволяет однозначно описать файл или каталог, к которому будет применена операция, поэтому многие входящие в протокол запросы включают описатели.

Можно считать, что описатель файла состоит из трех важных компонентов: *идентификатор тома*, *номер индексного дескриптора* и *номер поколения*; в совокупности они образуют уникальный идентификатор файла или каталога,

к которому хочет обратиться клиент. Идентификатор тома сообщает серверу, к какой файловой системе относится запрос (NFS-сервер может экспортировать несколько файловых систем); номер индексного дескриптора говорит серверу, к какому файлу в данном разделе относится запрос. Наконец, номер поколения нужен, если индексный дескриптор используется повторно; увеличивая его на 1 при каждом повторном использовании, сервер гарантирует, что клиент, знающий старый описатель файла, не сможет случайно обратиться к вновь созданному файлу.

Ниже приведена сводка наиболее важных частей протокола, полное описание можно найти в другом месте (например, в книге Каллагана [C00], содержащей очень хороший и подробный обзор NFS).

```

NFSPROC_GETATTR
    ожидает: описатель файла
    возвращает: атрибуты
NFSPROC_SETATTR
    ожидает: описатель файла, атрибуты
    возвращает: ничего
NFSPROC_LOOKUP
    ожидает: описатель каталога, имя искомого файла или каталога
    возвращает: описатель файла
NFSPROC_READ
    ожидает: описатель файла, смещение, счетчик
    возвращает: данные, атрибуты
NFSPROC_WRITE
    ожидает: описатель файла, смещение, счетчик, данные
    возвращает: атрибуты
NFSPROC_CREATE
    ожидает: описатель каталога, имя файла, атрибуты
    возвращает: ничего
NFSPROC_REMOVE
    ожидает: описатель каталога, имя подлежащего удалению файла
    возвращает: ничего
NFSPROC_MKDIR
    ожидает: описатель каталога, имя каталога, атрибуты
    возвращает: описатель файла
NFSPROC_RMDIR
    ожидает: описатель каталога, имя подлежащего удалению каталога
    возвращает: ничего
NFSPROC_READDIR
    ожидает: описатель каталога, количество подлежащих чтению байтов, кук
    возвращает: элементы каталога, кук (для получения дополнительных элементов)
    
```

Рис. 49.4 ❖ Протокол NFS: примеры

Кратко рассмотрим наиболее важные элементы протокола. Сначала сообщение LOOKUP используется для получения описателя файла, который затем применяется для доступа к данным файла. Клиент передает описатель каталога и имя искомого файла (или каталога), а сервер возвращает описатель и атрибуты файла.

Например, предположим, что у клиента уже имеется описатель, соответствующий корневому каталогу файловой системы (/) (на самом деле он становится известен в результате выполнения **протокола монтирования** NFS, посредством которого клиент подключается к серверу, но для краткости мы не станем здесь обсуждать этот протокол). Если приложение, работающее на стороне клиента, хочет открыть файл /foo.txt, то клиентская файловая система посылает серверу запрос на поиск, передавая описатель корневого каталога и имя foo.txt. В случае успеха будет возвращен описатель (и атрибуты) файла foo.txt.

Для особо любопытных скажем, что атрибуты – это метаданные, которые файловая система хранит о каждом файле, в т. ч. время создания файла, время последней модификации, размер, идентификатор владельца, сведения о правах доступа и т. д. – в общем, та самая информация, которую возвращает вызов stat().

Получив описатель файла, клиент может отправлять сообщения READ и WRITE для чтения и записи файла соответственно. Протокол требует, чтобы в сообщении READ был передан описатель файла, смещение от начала файла и количество подлежащих чтению байтов. В этом случае сервер сможет прочитать файл (поскольку описатель содержит том и индексный дескриптор, а в сообщении указаны смещение и счетчик байтов) и вернуть клиенту данные (или код ошибки). Сообщение WRITE обрабатывается аналогично с тем отличием, что данные передаются от клиента к серверу, а возвращается только код успеха или ошибки.

Последним мы рассмотрим сообщение протокола GETATTR; получив описатель файла, сервер возвращает его атрибуты, в т. ч. время последней модификации. Мы увидим, почему этот запрос так важен в NFSv2, когда будем обсуждать кеширование (а сами не догадаетесь?).

49.6. От протокола к распределенной файловой системе

Надеемся, вы уже прикинули, как этот протокол можно преобразовать в файловую систему, протянувшуюся между клиентом и сервером. Клиентская файловая система отслеживает открытые файлы и транслирует запросы приложения в соответствующие сообщения протокола. Сервер просто отвечает на сообщения, каждое из которых содержит достаточно информации для выполнения запроса.

Например, рассмотрим простое приложение, читающее файл. На рис. 49.5 показана последовательность системных вызовов, выполняемых приложением, и реакция на эти вызовы со стороны клиентской файловой системы и файлового сервера.

Сделаем несколько замечаний. Во-первых, отметим, как клиент запоминает все релевантное **состояние** для доступа к файлу, в т. ч. отображение целочисленного дескриптора файла на NFS-описатель файла и текущий ука-

затель файла. Это позволяет клиенту преобразовать каждый запрос чтения (в котором, заметим, явно *не* указано смещение) в правильно отформатированное сообщение протокола, которое сообщает серверу, какие именно байты следует прочитать из файла. Если чтение завершилось успешно, то клиент обновляет текущую позицию в файле, так что последующие операции чтения из файла с тем же описателем будут начинаться с другого смещения.

Клиент	Сервер
fd = open("/foo", ...); Послать LOOKUP (rootdir FH, «foo»)	Принять запрос LOOKUP искать «foo» в корневом каталоге вернуть описатель файла (FH) foo + атрибуты
Принять ответ на LOOKUP найти свободный дескриптор файла в таблице открытых файлов сохранить FH foo в таблице сохранить текущую позицию в файле (0) вернуть приложению дескриптор файла	
read(fd, buffer, MAX); найти запись в таблице открытых файлов по fd получить NFS-описатель файла (FH) использовать текущую позицию в файле в качестве смещения Послать READ (FH, offset=0, count=MAX)	Принять запрос READ выделить из FH том и номер индексного дескриптора прочитать индексный дескриптор с диска (или из кеша) вычислить местоположение блока (с помощью смещения) прочитать данные с диска (или из кеша) вернуть данные клиенту
Принять ответ на READ обновить позицию в файле (+число прочитанных байтов) установить текущую позицию в файле = MAX вернуть данные или код ошибки приложению	
read(fd, buffer, MAX); То же, но offset=MAX, а текущая позиция в файле устанавливается равной 2*MAX	
read(fd, buffer, MAX); То же, но offset=2*MAX, а текущая позиция в файле устанавливается равной 3*MAX	
close(fd); Нужно только освободить локальные структуры данных Освободить дескриптор fd в таблице открытых файлов (Никакого взаимодействия с сервером не происходит)	

Рис. 49.5 ❖ Чтение файла: действия на стороне клиента и файлового сервера

Во-вторых, обратите внимание, как происходят взаимодействия с сервером. Когда файл открывается в первый раз, клиентская файловая система посылает сообщение LOOKUP. Если бы нужно было пройти по длинному пути (например, /home/gemzi/foo.txt), то клиент послал бы три сообщения LOOKUP: одно для поиска home в каталоге /, другое – для поиска gemzi в home и третье – для поиска foo.txt в gemzi.

В-третьих, вы, наверное, заметили, что в каждом запросе серверу присутствует вся информация, необходимая для выполнения запроса. Именно это проектное решение и позволяет восстановиться после отказа сервера,

поскольку для ответа на запрос серверу не нужна никакая информация о состоянии.

СОВЕТ: о важности идемпотентности

Идемпотентность – полезное свойство для построения надежных систем. Если операцию можно выполнять более одного раза, то обрабатывать отказ становится гораздо проще – можно всего лишь повторить операцию. Если же операция не является идемпотентной, то жизнь усложняется.

49.7. ОБРАБОТКА ОТКАЗОВ СЕРВЕРА БЛАГОДАРЯ ИДЕМПОТЕНТНЫМ ОПЕРАЦИЯМ

Отправив сообщение серверу, клиент иногда не получает ответа. Причин может быть много. Бывает, что сообщение потеряно в сети, причем это может быть как запрос, так и ответ, но в любом случае клиент не дожидется ответа.

Возможно также, что сервер «упал» и в настоящий момент не отвечает на сообщения. Спустя некоторое время сервер перезагрузится и снова начнет работать, но пока все сообщения теряются. Как бы то ни было, клиент должен решить, что делать, если сервер не отвечает вовремя.

В NFSv2 клиент реагирует на все отказы единообразно и элегантно: он просто *повторяет* запрос. Точнее, после отправки запроса клиент взводит таймер. Если ответ получен до срабатывания таймера, то таймер отменяется и все заканчивается хорошо. Если же таймер срабатывает *раньше*, чем получен ответ, то клиент предполагает, что запрос не был обработан, и посылает его повторно. Если сервер ответил, все хорошо, и клиент считает, что проблема благополучно разрешилась.

Тот факт, что клиент может просто повторить запрос (какова бы ни была причина отказа), – важное свойство большинства запросов в NFS: они **идемпотентны**. Операция называется идемпотентной, если при многократном ее повторении результат получается такой же, как при однократном. Например, если сохранить некоторое значение в памяти три раза, то эффект будет таким же, как при однократном сохранении, поэтому операция «сохранение значения в памяти» идемпотентна. С другой стороны, если увеличить счетчик три раза, то, очевидно, результат будет не таким, как при однократном увеличении, следовательно, операция «увеличить счетчик» не идемпотентна. Вообще, любая операция, которая только читает данные, идемпотентна, а операции, обновляющие данные, нужно тщательно анализировать на предмет наличия этого свойства.

В основе восстановления после отказов в NFS лежит идемпотентность большинства типичных операций. Запросы LOOKUP и READ, очевидно, идемпотентны, поскольку только читают данные с файлового сервера и ничего не обновляют. Интереснее, что и запросы WRITE тоже идемпотентны. Если, например, запрос WRITE завершается ошибкой, то клиент может просто по-

вторить его. Сообщение WRITE содержит данные, счетчик и (это важно) смещение, по которому записывать данные. Поэтому запрос можно повторять, будучи уверенным, что результат нескольких операций записи такой же, как результат одной.

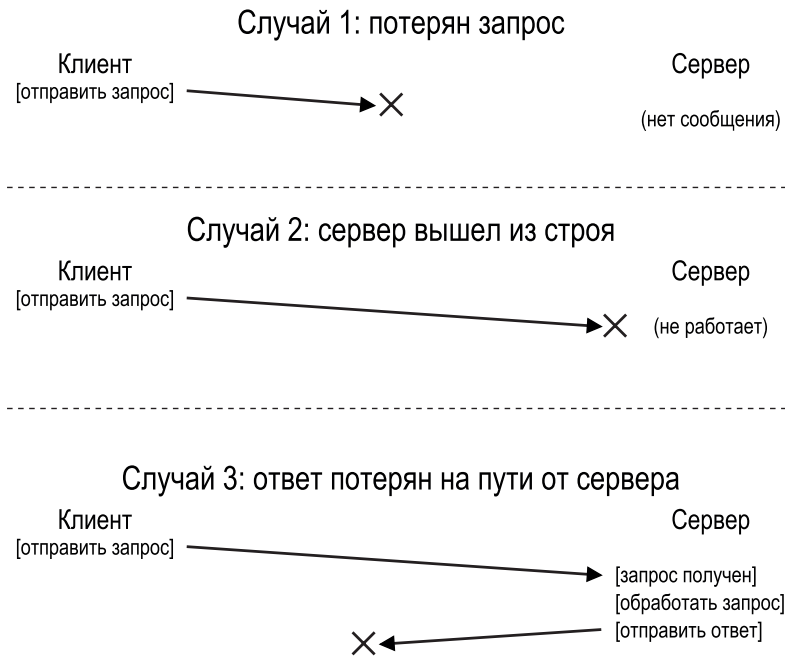


Рис. 49.6 ❖ Три вида потерь

Таким образом, клиент способен обработать все тайм-ауты единообразно. Если запрос WRITE был просто потерян (случай 1 выше), то клиент повторит его, сервер выполнит запись, и все будет хорошо. То же самое произойдет, если сервер «упал», после того как был отправлен первый запрос, но успел подняться до отправки второго (случай 2). Наконец, возможно, что сервер получил запрос WRITE, записал данные на диск и отправил ответ. Этот ответ может быть потерян (случай 3), что вынудит клиента повторить запрос. Получив запрос снова, сервер просто выполнит те же самые действия: запишет данные на диск и ответит, что все сделал. Если на этот раз клиент получит ответ, то все хорошо, так что клиент обрабатывает потерю сообщения и отказ сервера одинаково. Замечательно!

Небольшое отступление: некоторые операции трудно сделать идемпотентными. Например, при попытке создать уже существующий каталог нам сообщат, что функция `mkdir` завершилась неудачно. В NFS, если файловый сервер получил сообщение MKDIR, успешно обработал его, но ответ потерялся, то клиент повторит запрос, но получит в ответ код ошибки, потому что операция была выполнена в первый раз, а на второй раз закончилась неудачно. Жизнь все-таки не совершенна.

Совет: лучшее – враг хорошего (закон Вольтера)

Даже в самом лучшем проекте системы иногда остаются редкие случаи, когда система работает не совсем так, как хочется. Взять, к примеру, приведенный выше пример `mkdir`; можно было бы изменить семантику операции `mkdir`, сделав ее идемпотентной (подумайте, как это можно сделать), только зачем? Проект NFS покрывает наиболее важные случаи, и в целом обработка отказов в системе получилась ясной и простой. Поэтому принятие того факта, что жизнь не совершенна, а системы все равно нужно строить, – признак хорошего инженерного подхода. Следующий афоризм приписывается Вольтеру: «...один мудрый итальянец говорит, что лучшее – враг хорошего» [V72], – поэтому мы называем его **законом Вольтера**.

49.8. ПОВЫШЕНИЕ ПРОИЗВОДИТЕЛЬНОСТИ:

КЕШИРОВАНИЕ НА СТОРОНЕ КЛИЕНТА

Распределенные файловые системы хороши по ряду причин, но передача всех запросов чтения и записи по сети может привести к серьезным проблемам с производительностью: в общем случае сеть работает не быстро, особенно по сравнению с локальной памятью или диском. И следовательно, перед нами встает очередной вопрос: как улучшить производительность распределенной файловой системы?

Ответ ясен из заголовка этого раздела: **кеширование** на стороне клиента. В NFS клиентская файловая система кеширует данные и метаданные файлов, прочитанных с сервера, в памяти клиента. Таким образом, хотя первый доступ к файлу обходится дорого (т. к. требует передачи по сети), последующие запросы обслуживаются из памяти клиента, поэтому выполняются быстро.

Кеш также служит временным буфером записи. Когда клиентское приложение в первый раз записывает в файл, клиент буферизует данные в своей памяти (в том же кеше, где хранятся данные, прочитанные с файлового сервера), перед тем как передавать их серверу. Такая **буферизация записи** полезна, потому что разрывает связь между задержкой вызова `write()` и воспринимаемой производительностью записи: вызов `write()` немедленно возвращает управление (но всего лишь сохраняет данные в кеше клиентской файловой системы), а лишь спустя некоторое время данные передаются файловому серверу.

Таким образом, NFS-клиенты кешируют данные, производительность в большинстве случаев высока, и можно радоваться жизни. Так? К сожалению, не совсем. Добавление кеширования в любую систему с несколькими кеширующими клиентами порождает серьезную и интересную **проблему согласованности кешей**.

49.9. ПРОБЛЕМА СОГЛАСОВАННОСТИ КЕШЕЙ

Проблему согласованности кешей проще всего проиллюстрировать на примере трех клиентов и одного сервера. Пусть клиент C1 читает файл F и сохраняет копию файла в своем локальном кеше. Далее другой клиент, C2, перезаписывает файл F, изменяя его содержимое; назовем новую версию файла F[v2], а старую F[v1], чтобы их различать (но, конечно, на самом деле имя файла не изменилось, поменялось только его содержимое). И есть еще третий клиент, C3, который пока не обращался к файлу F.

Вы уже, наверное, видите, в чем проблема (рис. 49.7). На самом деле мы имеем две подпроблемы. Первая заключается в том, что клиент C2 может в течение некоторого времени хранить записанные данные в своем кеше, прежде чем отправить их серверу; в таком случае, пока F[v2] мирно пребывает в памяти C2, любое обращение к F со стороны другого клиента (скажем, C3) получит старую версию файла (F[v1]). Таким образом, буферизация записи на стороне клиента может привести к тому, что другие клиенты будут работать с устаревшими версиями файла, что нежелательно; действительно, представьте, что вы зашли на машину C2, обновили F, затем зашли на машину C3, прочитали тот же файл и обнаружили старую версию! Ясно, что никакой радости вы не испытаете. Назовем этот аспект проблемы согласованности кешей **видимостью обновлений**; когда обновления, произведенные одним клиентом, становятся видны остальным?

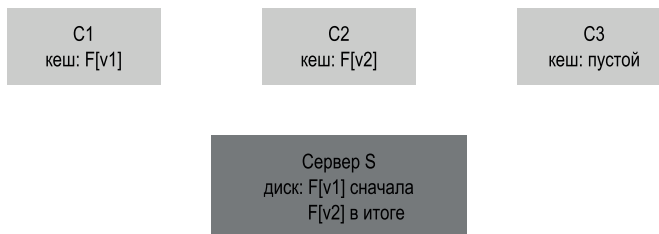


Рис. 49.7 ❖ Проблема согласованности кешей

Вторая подпроблема согласованности кешей – **устаревший кеш**; в этом случае C2 сбросил свои обновления на файловый сервер, так что сервер располагает последней версией (F[v2]). Но C1 по-прежнему хранит в своем кеше F[v1]; попытавшись прочитать файл F, программа, работающая на C1, получит устаревшую версию (F[v1]), а не самую свежую (F[v2]), что зачастую нежелательно.

В реализации NFSv2 эти проблемы согласованности кешей решены двумя способами. Что касается видимости обновлений, клиенты реализуют семантику согласованности, которая иногда называется **сбросом при закрытии**

(flush-on-close) (или **закрытием перед открытием**, англ. *close-to-open*). Именно, если клиентское приложение записывает в файл, а затем закрывает его, то клиент сбрасывает все обновления (модифицированные страницы в своем кеше) на сервер. При этом NFS гарантирует, что при последующем открытии файла на любой другой машине клиент увидит самую свежую версию.

Что же касается проблемы устаревшего кеша, клиент NFSv2, прежде чем воспользоваться данными из своего кеша, проверяет, изменился ли файл на сервере. Точнее, перед тем как использовать кешированный блок, клиентская файловая система отправляет серверу запрос GETATTR, чтобы получить атрибуты файла. В состав атрибутов входит, в частности, время последней модификации файла; если оно позже времени помещения файла в кеш, то клиент объявляет файл **недействительным**, т. е. удаляет его из своего кеша, так что последующее чтение обязательно приведет к запросу файла с сервера. Если же клиент видит, что в кеше хранится последняя версия файла, то может воспользоваться содержимым кеша и, значит, улучшить производительность.

Реализовав это решение проблемы устаревшего кеша, разработчики оригинальной системы NFS в Sun обнаружили новую проблему: внезапно на NFS-сервер стало поступать множество запросов GETATTR. Здоровый инженерный принцип – проектировать в расчете на **типичный случай**, добиваясь именно в нем хорошей работы. Здесь же типичным случаем считался доступ к файлу со стороны одного клиента (быть может, повторяющийся), однако клиент всегда должен был отправлять запросы GETATTR, чтобы убедиться, что больше никто не изменил файл. То есть клиент бомбардирует сервер запросами «изменил ли кто-нибудь этот файл?», хотя чаще всего ничего такого не было.

Чтобы выправить ситуацию (хотя бы частично), на каждого клиента был добавлен **кеш атрибутов**. Клиент по-прежнему проверяет актуальность файла перед обращением к нему, но чаще всего просто выбирает атрибуты из своего кеша. Атрибуты файла помещаются в кеш при первом обращении к нему и хранятся в течение некоторого времени (скажем, 3 секунды). В течение этих трех секунд любой запрос к файлу будет считать, что использовать кешированный файл можно, поэтому не будет беспокоить сервер.

49.10. Оценка согласованности кешей в NFS

И напоследок еще несколько слов о согласованности кешей в NFS. Сброс при закрытии был добавлен, потому что это «имеет смысл», но породил проблему с производительностью. Именно, даже если клиент создавал временный или краткосрочный файл, который вскоре удалялся, все равно его приходилось копировать на сервер. Лучше было бы хранить такие файлы в памяти до момента удаления, вообще устранив взаимодействие с сервером; это позволило бы повысить производительность.

Еще важнее, что из-за добавления кеша атрибутов стало очень трудно понять, с какой именно версией файла имеет дело клиент. Иногда он получал последнюю версию, а иногда устаревшую – просто потому, что тайм-аут кеша

атрибутов еще не истек, поэтому клиент радостно отдавал приложению версию, хранящуюся в его памяти. В большинстве случаев этого было (да и сейчас остается) достаточно, но временами приводит к странному поведению.

Таким образом, мы описали странности кеширования на стороне NFS-клиента. Это интересный пример того, как детали реализации определяют наблюдаемую пользователем семантику, а не наоборот.

49.11. Последствия для буферизации записи на стороне сервера

До сих пор мы акцентировали внимание на кешировании на стороне клиента, именно там возникают наиболее интересные вопросы. Но в качестве NFS-серверов обычно используются мощные компьютеры с большим объемом памяти, поэтому для них кеширование тоже актуально. Прочитав данные (и метаданные) с диска, NFS-сервер хранит их в памяти, так что при последующем чтении не обращается к диску, что может принести (возможно, небольшое) повышение производительности.

Но более интригующим является вопрос о буферизации записи. NFS-сервер *не* должен возвращать информацию об успешном выполнении запроса WRITE, пока данные не записаны на запоминающее устройство длительного хранения (например, на диск). Хотя он может поместить копию данных в свою память, сообщить клиенту об успехе было бы неправильно; понимаете ли вы, почему?

Ответ кроется в наших предположениях о том, как клиенты обрабатывают отказ сервера. Рассмотрим следующую последовательность операций записи со стороны клиента:

```
write(fd, a_buffer, size); // первый блок заполнить символами a
write(fd, b_buffer, size); // второй блок заполнить символами b
write(fd, c_buffer, size); // третий блок заполнить символами c
```

Эти операции записывают в три блока файла соответственно символы a, b и c. Если вначале файл имел вид:

```
xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx
yyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyy
zzzzzzzzzzzzzzzzzzzzzzzzzzzzzzzzzzzzzzzzzzzzzzzzzzzzzzzzzzzzzz
```

то было бы естественно ожидать, что после этих операций он примет такой вид:

```
aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa
bbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbb
cccccccccccccccccccccccccccccccccccccccccccccccccccccccccccc
```

Теперь предположим, что эти три записи были отправлены серверу в трех разных сообщениях протокола WRITE. Допустим, что первое сообщение было

быстро отвечать на запросы WRITE, не опасаясь потерять данные и не неся затрат на немедленную запись на диск. Второй метод – использовать файловую систему, специально спроектированную, так чтобы запись на диск осуществлялась максимально быстро, когда в этом все-таки возникает необходимость [HLM94, RO91].

49.12. РЕЗЮМЕ

Мы познакомились с распределенной файловой системой NFS. В основе NFS лежит идея простого и быстрого восстановления после аварийного отказа сервера. Эта цель достигается посредством тщательного проектирования протокола. Очень важна идемпотентность операций; клиент может безопасно повторить сбойную операцию вне зависимости от того, выполнил ее сервер раньше или нет.

ОТСТУПЛЕНИЕ: ТЕРМИНОЛОГИЯ NFS

- Ключом к реализации главной цели NFS – быстрого и простого восстановления после аварии – является протокол **без сохранения информации о состоянии**. После аварии сервер может быстро перезапуститься и возобновить обслуживание запросов; клиенты просто пытаются **повторять** запросы, пока не добьются успеха.
- **Идемпотентность** запросов – центральный аспект протокола NFS. Операция называется идемпотентной, если при многократном ее повторении результат получается такой же, как при однократном. В NFS идемпотентность позволяет клиенту без опаски повторять операции и унифицировать обработку клиентами отказов сервера.
- Соображения производительности диктуют необходимость **кеширования** и **буферизации записи** на стороне клиента, но при этом возникает **проблема согласованности кешей**.
- В реализациях NFS предложено инженерное решение проблемы согласованности кешей, включающее сочетание нескольких методов. **Сброс при закрытии (закрытие перед открытием)** гарантирует, что в момент закрытия файла его содержимое принудительно передается на сервер, что позволяет другим клиентам видеть изменения. Кеш атрибутов позволяет не так часто проверять, изменился ли файл на сервере (с помощью запросов GETATTR).
- NFS-сервер обязан записать данные на устройство долговременного хранения, прежде чем сообщать клиенту об успехе, в противном случае была бы возможна потеря данных.
- Для поддержки интеграции NFS с операционной системой Sun реализовала интерфейс **VFS-Vnode**, благодаря которому в одной операционной системе могут сосуществовать несколько реализаций файловой системы.

Мы также видели, как включение кеширования в систему с одним сервером и несколькими клиентами может создать дополнительные проблемы. В частности, система, претендующая на разумное поведение, должна решить проблему согласованности кешей. Однако NFS несколько ослабила это требо-

вание, поэтому иногда можно наблюдать странное поведение. Наконец, мы видели, что кеширование на стороне сервера может оказаться непростым делом: прежде чем сообщать клиенту об успехе, сервер обязан записать данные на устройство длительного хранения, иначе возможна потеря данных.

Мы не упомянули о других, не менее важных вопросах и, прежде всего, о безопасности. В ранних реализациях NFS безопасность была слабым местом; любой пользователь мог без особых усилий притвориться любым другим и таким образом получить доступ практически к любому файлу. Впоследствии интеграция с более серьезными службами аутентификации (например, Kerberos [NT94]) устранила эти очевидные недостатки.

Литература

[AKW88] «The AWK Programming Language» by Alfred V. Aho, Brian W. Kernighan, Peter J. Weinberger. Pearson, 1988 (1st edition). *Замечательная и лаконичная книга о языке awk. Когда-то мы имели удовольствие встречаться с Питером Вейнбергером. Он представился так: «Я Питер Вейнбергер, тот самый, который ‘W’ в awk». Являясь большими поклонниками awk, мы были в восторге. Один из нас (Ремзи) тогда сказал: «Я обожаю awk! А особенно мне нравится книга, где все так чудесно разложено по полочкам». Вейнбергер ответил (приуныв): «А, книгу-то написал Керниган».*

[C00] «NFS Illustrated» by Brent Callaghan. Addison-Wesley Professional Computing Series, 2000. *Великолепный справочник по NFS; невероятно подробное и скрупулезное описание самого протокола.*

[ES03] «New NFS Tracing Tools and Techniques for System Analysis» by Daniel Ellard and Margo Seltzer. LISA '03, San Diego, California. *Тщательный анализ NFS посредством пассивной трассировки. Авторы демонстрируют, как, всего лишь наблюдая за сетевым трафиком, можно многое узнать о работе файловой системы.*

[HLM94] «File System Design for an NFS File Server Appliance» by Dave Hitz, James Lau, Michael Malcolm. USENIX Winter 1994. San Francisco, California, 1994. *На Хитца с коллегами сильное влияние оказала предыдущая работа по файловым системам со структурой журнала.*

[K86] «Vnodes: An Architecture for Multiple File System Types in Sun UNIX» by Steve R.

Kleiman. USENIX Summer '86, Atlanta, Georgia. *В этой работе показано, как встроить гибкую архитектуру файловой системы в операционную систему, что позволяет сосуществовать нескольким разным файловым системам. В той или иной форме используется во всех современных операционных системах.*

[NT94] «Kerberos: An Authentication Service for Computer Networks» by B. Clifford Neuman, Theodore Ts'o. IEEE Communications, 32(9):33–38, September 1994. *Kerberos – ранняя служба аутентификации, оказавшая огромное влияние. Надо будет когда-нибудь написать книгу о ней...*

[O91] «The Role of Distributed State» by John K. Ousterhout. 1991. Доступно по адресу <ftp://ftp.cs.berkeley.edu/ucb/sprite/papers/state.ps>. *Редко упоминаемое обсуждение распределенного состояния; более широкий взгляд на проблемы и вызовы.*

[P+94] «NFS Version 3: Design and Implementation» by Brian Pawlowski, Chet Juszczak, Peter Staubach, Carl Smith, Diane Lebel, Dave Hitz. USENIX Summer 1994, pages 137–152. *Описание небольших модификаций, включенных в NFS версии 3.*

[P+00] «The NFS version 4 protocol» by Brian Pawlowski, David Noveck, David Robinson, Robert Thurlow. 2nd International System Administration and Networking Conference (SANE 2000). *Безусловно, самая литературная статья из всех написанных о NFS.*

[RO91] «The Design and Implementation of the Log-structured File System» by Mendel Rosenblum, John Ousterhout. Symposium on Operating Systems Principles (SOSP), 1991. *И снова LFS. Нет, насытиться LFS решительно невозможно.*

[S86] «The Sun Network File System: Design, Implementation and Experience» by Russel Sandberg. USENIX Summer 1986. *Оригинальная статья о NFS; читать ее сложно, но все равно стоит, ведь это источник изложенных замечательных идей.*

[Sun89] «NFS: Network File System Protocol Specification» by Sun Microsystems, Inc. Request for Comments: 1094, March 1989. Доступно по адресу <http://www.ietf.org/rfc/rfc1094.txt>. *Скучная спецификация. Читайте, если обязаны по работе, т. е. если вам платят за такое чтение. И надеюсь, неплохо платят. Наличными!*

[V72] «La Begueule» by Francois-Marie Arouet a. k. a. Voltaire. Published in 1772. *Вольтер наговорил много умных вещей, это лишь один пример. Еще Вольтер сказал: «Если в вашей стране всего две религии, то они вцепятся друг другу в горло, но если их тридцать, то все будут жить в мире». Как вам это, демократы и республиканцы?*

Домашнее задание (измерение)

В этом домашнем задании мы займемся трассировкой NFS на реальных данных. Источником трасс является работа Ellard and Seltzer [ES03]. Не забудьте предварительно прочитать файл README и скачать нужный архив с веб-страницы OSTER.

Вопросы

1. Первый вопрос по анализу трасс: с помощью временных меток в первом столбце определите, за какой период времени были созданы трассы. Какова длина этого периода? В каком году (неделе, месяце, дне) это было (соответствуют ли ваши выводы имени файла)? Указание: воспользуйтесь

командами `head -1` и `tail -1`, чтобы получить первую и последнюю строки файла, и выполните вычисление.

2. Теперь посчитаем операции. Сколько операций каждого типа присутствует в трассе? Отсортируйте операции по частоте; какая операция встречается чаще других? Отвечает ли NFS своей репутации?
3. Теперь рассмотрим некоторые операции подробнее. Например, запрос `GETATTR` возвращает много информации о файлах, в т. ч. идентификатор пользователя, от имени которого выполняется запрос, размер файла и т. д. Постройте гистограмму распределения встречающихся в трассе файлов по размерам; каков средний размер файла? Сколько различных пользователей обращались к файлам? Верно ли, что большая часть трафика генерируется немногими пользователями, или распределение более равномерное? Какую еще интересную информацию можно извлечь из ответов на запросы `GETATTR`?
4. Можно также изучить запросы к определенному файлу и определить, как именно пользователи обращаются к файлам. Например, верно ли, что данный файл читается или записывается последовательно? Или произвольно? Проанализируйте детально запросы `READ` и `WRITE` и ответы на них, чтобы ответить на этот вопрос.
5. Трафик генерируется многими машинами и адресован одному серверу (в этой трассе). Вычислите матрицу трафика, показывающую, сколько различных клиентов присутствует в трассе и сколько запросов-ответов приходится на каждого. Верно ли, что доминирует несколько машин, или трафик более сбалансирован?
6. Хронометрическая информация и уникальные идентификаторы в запросах и ответах помогут вычислить задержку каждого запроса. Вычислите задержки для всех пар запрос-ответ и постройте соответствующее распределение. Какова средняя задержка? Максимальная? Минимальная?
7. Иногда запросы отправляются повторно, поскольку запрос или ответ на него потерян или отброшен. Видны ли следы таких повторных попыток в трассе?
8. Можно задать еще много вопросов, на которые даст ответы анализ. Какие вопросы, на ваш взгляд, важны? Отправьте их нам, и, возможно, мы включим их в это домашнее задание!

Глава 50

Файловая система Andrew (AFS)

Файловая система Andrew была разработана в университете Карнеги-Меллона (CMU)¹ в 1980-х годах [Н+88]. Основная цель проекта под руководством хорошо известного профессора М. Сатьянараянана (для краткости «Сатья») была простой: **масштабируемость**. Точнее, как спроектировать распределенную файловую систему, чтобы сервер мог поддерживать максимально возможное количество клиентов?

Интересно, что на масштабируемость влияют многочисленные аспекты проектирования и реализации. Самым главным является **протокол** взаимодействия клиентов и серверов. Например, в NFS протокол вынуждает клиентов периодически опрашивать сервер, чтобы узнать, не изменились ли кешированные ими файлы; поскольку каждая проверка потребляет ресурсы сервера (CPU и пропускную способность сети), излишне частые действия такого рода ограничивают число клиентов, которые сервер может обслужить, и тем самым масштабируемость.

AFS отличается от NFS и тем, что с самого начала на первое место ставилось разумное видимое пользователю поведение. В NFS трудно описать согласованность кешей, потому что она напрямую зависит от низкоуровневых деталей реализации, в т. ч. от величины тайм-аута кеша на стороне клиента. В AFS согласованность кешей проста и понятна: в момент открытия файла клиент в общем случае получает от сервера последнюю согласованную версию.

50.1. AFS версии 1

Мы обсудим две версии AFS [Н+88, S+85]. Первая (которую мы называем AFSv1, хотя исторически оригинальная система называлась распределен-

¹ Изначально названный университетом Карнеги-Меллона, впоследствии CMU отбросил дефис и возродился под современным названием «университет Карнеги Меллона». Поскольку AFS основана на работе, выполненной в начале 1980-х годов, мы используем прежнее название с дефисом. Если у вас есть лишняя минутка, можете найти дополнительные сведения об этой истории на странице <https://www.quora.com/When-did-Carnegie-Mellon-University-remove-the-hyphen-in-the-university-name>.

ной файловой системой ITC [S+85]) уже обладала основными проектными особенностями, но масштабировалась не так хорошо, как хотели авторы, что стало причиной перепроектирования, в результате чего родился окончательный протокол (который мы называем AFSv2, или просто AFS) [H+88]. Сейчас мы обсудим первую версию.

TestAuth	Проверить, изменился ли файл (используется для подтверждения действительности кешированных элементов)
GetFileStat	Получить статистические сведения о файле
Fetch	Прочитать содержимое файла
Store	Сохранить файл на сервере
SetFileStat	Установить статистические сведения о файле
ListDir	Получить содержимое каталога

Рис. 50.1 ❖ Основы протокола AFSv1

Один из основополагающих принципов всех версий AFS – **кеширование файла целиком на локальном диске** клиентской машины, обратившейся к файлу. После вызова `open()` весь файл (если он существует) забирается с сервера и сохраняется на локальном диске. Последующие операции `read()` и `write()` со стороны приложения переадресуются локальной файловой системе, поэтому не требуют передачи данных по сети и выполняются быстро. Наконец, после вызова `close()` файл (если он был изменен) передается обратно серверу. Отметим очевидное отличие от NFS, которая кеширует *блоки* (а не целые файлы, хотя, конечно, NFS могла бы кешировать все блоки файла), причем в *памяти* клиента (а не на локальном диске).

Теперь перейдем к деталям. Когда клиентское приложение первый раз вызывает `open()`, клиентский код AFS (который разработчики AFS называют **Venus**) посылает сообщение `Fetch` серверу. В этом сообщении передается полный путь к требуемому файлу (например, `/home/remzi/notes.txt`) файловому серверу (группу которых разработчики называют **Vice**), который проходит по этому пути, находит файл и отправляет его целиком клиенту. Клиентский код кеширует файл на локальном диске клиента. Как уже было сказано, последующие системные вызовы `read()` и `write()` строго *локальны* (отсутствует всякая коммуникация с сервером), они просто переадресуются локальной копии файла. Поскольку вызовы `read()` и `write()` работают в точности как в локальной файловой системе, однажды прочитанный блок можно кешировать в памяти клиента. Таким образом, AFS также использует память клиента для кеширования блоков, правда, находящихся на локальном диске. Наконец, по завершении AFS-клиент проверяет, был ли файл изменен (т. е. открывался ли он для записи), и если да, то отправляет новую версию обратно серверу в сообщении `Store`, передавая весь файл и путь к нему.

При следующем обращении к файлу AFSv1 действует гораздо эффективнее. Сначала клиентский код просит сервер проверить (с помощью сообщения `TestAuth`), был ли файл изменен. Если нет, то клиент пользуется локальной кешированной копией, избегая передачи по сети, что повышает производительность. На рисунке выше показаны некоторые сообщения протокола

AFSv1. Отметим, что в этой ранней версии кешировались только файлы, а, к примеру, каталоги хранились на сервере.

СОВЕТ: СНАЧАЛА ИЗМЕРЬТЕ, ПОТОМ СТРОЙТЕ (ЗАКОН ПАТТЕРСОНА)

Один из наших научных руководителей, Дэвид Паттерсон (прославившийся в области RISC и RAID), всегда рекомендовал нам выполнить измерения системы и продемонстрировать наличие проблемы, *прежде* чем приступать к построению новой системы, устраняющей эту проблему. Располагая экспериментальным свидетельством, а не просто интуитивными догадками, вы сможете подвести научное обоснование под построение системы. К тому же вам придется поломать голову над технологией измерений еще до разработки улучшенной версии. И когда, наконец, дело дойдет до построения новой системы, вы будете лучше вооружены в двух отношениях: во-первых, будете уверены, что решается реальная проблема, а во-вторых, будете знать, как измерить новую систему и показать, что положение дел действительно улучшилось. Мы называем это правило **законом Паттерсона**.

50.2. ПРОБЛЕМЫ ВЕРСИИ 1

Несколько принципиальных проблем в первой версии AFS заставили разработчиков пересмотреть свою файловую систему. Для изучения этих проблем проектировщики потратили много времени на хронометраж существующего прототипа, чтобы найти, что в нем не так. Такие эксперименты – похвальная вещь, потому что **измерение** – ключ к пониманию того, как система работает и как ее можно улучшить; получение заслуживающих доверия данных – необходимая часть конструирования системы. В ходе исследований авторы обнаружили две основные проблемы в AFSv1.

- **Стоимость обхода пути слишком высока.** При выполнении запросов протокола Fetch и Store клиент передает серверу путь целиком (например, /home/gemzi/notes.txt). Сервер, чтобы получить доступ к файлу, должен пройти по всему пути: начать с корневого каталога, найти home, затем в home найти gemzi и т. д., пока не доберется до требуемого файла. Проектировщики AFS обнаружили, что когда много клиентов обращаются к серверу одновременно, сервер тратит значительную часть процессорного времени на обход путей.
- **Клиент отправляет слишком много сообщений TestAuth.** Как NFS переполнена сообщениями протокола GETATTR, так и AFSv1 порождала чрезмерно большой трафик для проверки актуальности локальной копии файла (или информации о его состоянии) с помощью сообщений TestAuth. Таким образом, сервер тратит много времени, чтобы сообщить клиентам, что с их кешированными копиями все в порядке. Причем чаще всего он отвечает, что файл не изменился.

В AFSv1 было и еще две проблемы: нагрузка на серверы не сбалансирована, и сервер создавал отдельный процесс для каждого клиента, что приводило к избыточному контекстному переключению и другим накладным расхо-

дам. Проблема дисбаланса была решена введением **томов**, которые администратор мог перемещать с одного сервера на другой для балансировки нагрузки. Проблема контекстного переключения была решена в AFSv2 путем использования потоков вместо процессов. Но из-за нехватки места мы сосредоточимся только на двух главных проблемах протокола, ограничивших масштабируемость системы.

50.3. УЛУЧШЕНИЕ ПРОТОКОЛА

Две описанные выше проблемы ограничивают масштабируемость AFS; процессор сервера стал узким местом в системе: каждый сервер мог обслужить не более 20 клиентов, после чего испытывал перегрузку. Серверы получали слишком много сообщений TestAuth, а когда приходило сообщение Fetch или Store, тратили слишком много времени на проход по иерархии каталогов. Таким образом, проектировщики AFS столкнулись со следующей проблемой.

СУЩЕСТВО ПРОБЛЕМЫ:

КАК СПРОЕКТИРОВАТЬ МАСШТАБИРУЕМЫЙ ФАЙЛОВЫЙ ПРОТОКОЛ

Как следует перепроектировать протокол, чтобы свести к минимуму взаимодействие с сервером, т. е. уменьшить количество сообщений TestAuth? Кроме того, как повысить эффективность этих взаимодействий? Решив эти вопросы, новый протокол стал бы стержнем гораздо более масштабируемой версии AFS.

50.4. AFS ВЕРСИИ 2

В версии AFSv2 введено понятие **обратного вызова** с целью уменьшить количество взаимодействий между клиентом и сервером. Обратный вызов – это просто обещание сервера информировать клиента о том, что кешированный им файл был модифицирован. Благодаря добавлению в систему этого **состояния** клиенту больше не нужно контактировать с сервером, чтобы выяснить, актуален ли еще кешированный файл. Вместо этого клиент предполагает, что файл актуален, пока сервер явно не уведомит его об обратном; можно провести аналогию с **опросом** и **прерываниями**.

В AFSv2 также введено понятие **идентификатора файла (FID)** (аналогичное **описателю файла** в NFS), заменившего пути, – он позволяет указать, какой файл интересует клиента. FID в AFS включает идентификатор тома, идентификатор файла и «уникальный идентификатор» (чтобы можно было повторно использовать идентификаторы тома и файла, когда файл удален). Таким образом, вместо того чтобы отправлять серверу полный путь и заставлять его проходить по этому пути для поиска нужного файла, клиент сам обходит путь, по одной компоненте за раз, и кеширует при этом результаты – в надежде уменьшить нагрузку на сервер.

Например, если бы клиент хотел обратиться к файлу `/home/remzi/notes.txt` и `home` был бы каталогом AFS, смонтированным на `/` (т. е. `/` – локальный корневой каталог, но `home` и его потомки находятся в AFS), то клиент сначала отправил бы сообщение `Fetch`, чтобы получить содержимое `home`, поместил бы его в кеш на локальном диске и установил бы обратный вызов для `home`. Затем клиент отправил бы сообщение `Fetch`, чтобы получить содержимое `remzi`, поместил бы его в локальный кеш и установил бы обратный вызов для `remzi`. Наконец, клиент отправил бы сообщение `Fetch notes.txt`, кешировал бы регулярный файл на локальном диске, установил бы для него обратный вызов и, наконец, вернул бы дескриптор файла вызывающему приложению. Эта последовательность показана на рис. 50.2.

Клиент (C ₁)	Сервер
<code>fd = open("/home/remzi/notes.txt", ...);</code> Отправить <code>Fetch</code> (<code>home</code> FID, "remzi")	Получить запрос <code>Fetch</code> искать <code>remzi</code> в каталоге <code>home</code> установить <code>callback(C1)</code> для <code>remzi</code> вернуть содержимое <code>remzi</code> и FID
Получить ответ на <code>Fetch</code> записать <code>remzi</code> в кеш на локальном диске запомнить состояние обратного вызова для <code>remzi</code> Отправить <code>Fetch</code> (<code>remzi</code> FID, "notes.txt")	
Получить ответ на <code>Fetch</code> записать <code>notes.txt</code> в кеш на локальном диске запомнить состояние обратного вызова для <code>notes.txt</code> локальный <code>open()</code> кешированного <code>notes.txt</code> вернуть дескриптор файла приложению	Получить запрос <code>Fetch</code> искать <code>notes.txt</code> в каталоге <code>remzi</code> установить <code>callback(C1)</code> для <code>notes.txt</code> вернуть содержимое <code>notes.txt</code> и FID
<code>read(fd, buffer, MAX);</code> выполнить локальный <code>read()</code> для кешированной копии	
<code>close(fd);</code> выполнить локальный <code>close()</code> для кешированной копии если файл изменился, отправить на сервер	
<code>fd = open("/home/remzi/notes.txt", ...);</code> Для каждого каталога <code>dir</code> (<code>home</code> , <code>remzi</code>) if (<code>callback(dir) == VALID</code>) искать в локальной копии <code>dir</code> else Fetch (как и выше) if (<code>callback(remzi) == VALID</code>) открыть локальную кешированную копию вернуть ее дескриптор файла else Fetch (как и выше), затем <code>open</code> и вернуть <code>fd</code>	

Рис. 50.2 ❖ Чтение файла: действия на стороне клиента и сервера

Но ключевое отличие от NFS заключается в том, что при каждом получении каталога или файла AFS-клиент устанавливает обратный вызов, так чтобы сервер извещал клиента об изменении в кешированном им состоянии. Преимущество очевидно: хотя при *первом* обращении к файлу `/home/remzi/notes.txt` генерирует много сообщений от клиента серверу (как описано выше), одновременно устанавливаются обратные вызовы для файла `notes.txt` и всех

каталогов на пути к нему, поэтому все последующие обращения обрабатываются локально и вообще не требуют взаимодействия с сервером. Таким образом, в типичном случае, когда файл кеширован на стороне клиента, AFS ведет себя почти так же, как локальная файловая система на диске. Если приложение обращается к файлу более одного раза, то второе обращение должно быть таким же быстрым, как обращение к локальному файлу.

ОТСТУПЛЕНИЕ: СОГЛАСОВАННОСТЬ КЕШЕЙ – НЕ ПАНАЦЕЯ

При обсуждении распределенных файловых систем много внимания уделяется реализованной в них согласованности кешей. Однако базовые средства согласованности не решают всех проблем, связанных с доступом к файлу со стороны нескольких клиентов. Например, если мы создаем репозиторий кода, позволяющий клиентам помещать и извлекать версии программ, то не можем полагаться на то, что файловая система сделает за нас всю работу, а должны использовать явную **блокировку на уровне файлов**, чтобы при конкурентном доступе происходили «правильные» вещи. И действительно, любое приложение, которое серьезно относится к конкурентным обновлениям, включает дополнительные механизмы обработки конфликтов. Базовые средства согласованности, описанные в этой и в предыдущей главах, полезны лишь для нерегулярного применения – заходя на другую машину, пользователь хочет видеть на ней разумные версии своих файлов. Ожидать от этих протоколов большего – значит напрашиваться на ошибки, разочарование и горькие слезы.

50.5. СОГЛАСОВАННОСТЬ КЕШЕЙ

Обсуждая NFS, мы рассматривали два аспекта согласованности кешей: **видимость обновлений** и **устаревание кеша**. В первом случае вопрос ставится так: когда на сервер попадет новая версия файла? А во втором нас интересует следующее: если на сервер попала новая версия, то через какое время клиенты увидят новую версию вместо кешированной старой?

Благодаря обратным вызовам и кешированию файлов целиком концепцию согласованности кешей, предлагаемую AFS, легко описать и понять. Нужно рассмотреть два важных случая: согласованность между процессами на *разных* машинах и на *одной* машине.

В случае разных машин AFS делает обновления видимыми на сервере и одновременно объявляет недействительными кешированные копии – это делается в момент закрытия обновленного файла. Клиент открывает файл, затем записывает в него (быть может, несколько раз). Когда файл в конечном итоге закрывается, новый файл отправляется на сервер (и становится видимым). В этот момент сервер «разрывает» обратные вызовы со всеми клиентами, имеющими кешированные копии; разрыв означает, что сервер связывается с каждым клиентом и информирует его, что тот хранит ставший неактуальным файл. После этого клиенты перестают читать устаревшие копии файла; при последующем открытии клиент затребует новую версию файла от сервера (и заново установит обратный вызов для полученной версии файла).

AFS делает исключение из этой простой модели для процессов на одной машине. В этом случае запись в файл сразу становится видна другим локальным процессам (т. е. процесс не должен ждать закрытия файла, чтобы увидеть обновления). Поэтому поведение на одной машине в точности соответствует ожидаемому, основанному на стандартной семантике Unix. И только при переходе на другую машину мы сможем увидеть более общий механизм согласованности в AFS.

Существует еще один заслуживающий внимания случай с участием нескольких машин, а именно редкая ситуация, когда процессы на разных машинах модифицируют файл одновременно. Тогда AFS считает, что **последний записавший выигрывает** (хотя правильнее было бы назвать этот подход «**последний закрывший выигрывает**»). Именно, тот клиент, который вызвал `close()` последним, запишет на сервер весь файл и таким образом окажется «победителем», т. е. именно его версия файла останется на сервере и будет видна всем остальным. Не может быть так, что часть файла создана одним клиентом, а часть другим. Отметим отличие от блочных протоколов типа NFS: в NFS отдельные блоки могут быть сброшены на сервер после обновления каким-то клиентом, поэтому окончательный файл на сервере может состоять из блоков, записанных разными клиентами. Во многих случаях такая мешанина не имеет особого смысла, например представьте себе, что получится, если изображение в формате JPEG модифицируется двумя клиентами по частям: вряд ли результат окажется правильным JPEG-файлом.

На рис. 50.3 показана хронология нескольких сценариев. В столбцах представлено поведение двух процессов (P_1 и P_2) на машине Клиент₁ и состояние кеша на ней, один процесс (P_3) на машине Клиент₂ и ее состояние кеша, а также состояние на сервере. Предполагается, что все машины работают с одним файлом F. Для сервера показано просто состояние файла после завершения операции слева. Изучите хронологию – понимаете ли вы, почему каждое чтение возвращает именно такой результат, как показано. Если зайдете в тупик, поле комментария справа должно помочь.

50.6. ВОССТАНОВЛЕНИЕ ПОСЛЕ АВАРИИ

Из описания выше может сложиться впечатление, что восстановление после аварии сложнее, чем в случае NFS. И это чистая правда. Представьте, что в течение короткого времени сервер (S) потерял связь с клиентом (C1), например потому что клиент перезагружается. Пока C1 недоступен, S мог попытаться послать ему одно или несколько сообщений обратного вызова; например, предположим, что C1 кешировал файл F на своем локальном диске, затем другой клиент C2 обновил F, понуждая S отправить всем клиентам, хранящим копию этого файла, сообщение о необходимости удалить ее из локального кеша. Поскольку C1 мог не получить эти критически важные сообщения, пока перезагружался, после присоединения к системе он должен считать все содержимое своего кеша подозрительным. Поэтому при следующем обращении к файлу F C1 должен сначала спросить у сервера (с помощью сообщения

TestAuth), актуален ли еще файл F; если да, то C1 может его использовать, а иначе должен получить новую версию от сервера.

P ₁	Клиент ₁		Клиент ₂		Диск сервера	Примечания
	P ₂	Кеш	P ₃	Кеш		
open(F)		—		—	—	Файл создан
write(A)		A		—	—	
close()		A		—	A	
	open(F)	A		—	A	
	read()→A	A		—	A	
	close()	A		—	A	Локальные процессы видят запись немедленно
open(F)		A		—	A	
write(B)		B		—	A	
	open(F)	B		—	A	
	read()→B	B		—	A	
	close()	B		—	A	Удаленные процессы не видят запись...
		B	open(F)	A	A	
		B	read()→A	A	A	
		B	close()	A	A	
close()		B		A	B	
		B	open(F)	B	B	...пока не будет выполнен close()
		B	read()→B	B	B	
		B	close()	B	B	
		B	open(F)	B	B	
open(F)		B		B	B	
write(D)		D		B	B	К сожалению для P ₃ , последний записавший выигрывает
		D	write(C)	C	B	
		D	close()	C	C	
close()		D		C	D	
		D	open(F)	D	D	
		D	read()→D	D	D	
		D	close()	D	D	

Рис. 50.3 ❖ Хронология согласованности кешей

Восстановление сервера после аварии также сложнее. Проблема в том, что обратные вызовы хранятся в памяти, поэтому после перезагрузки сервер не знает, какие файлы кешированы на каких клиентских машинах. Таким образом, после перезапуска сервера все клиенты должны понять, что сервер потерпел аварию, считать свои кеши подозрительными и восстановить актуальность любого файла, прежде чем пользоваться им. Следовательно, авария сервера – это крупное событие, о котором необходимо своевременно уведомить всех клиентов, иначе возникнет риск, что клиент работает с устаревшим файлом. Есть много способов реализовать такое восстановление; например, после перезапуска сервер может разослать всем клиентам сообщение «не доверяй содержимому своего кеша!», или клиенты могут периодически проверять, что сервер жив (посылая ему **контрольное сообщение**). Как видим, построение лучше масштабируемой модели, обладающей разумным поведением кеширования, обходится недаром; в NFS клиенты едва замечали аварию сервера.

50.7. Масштабируемость и производительность AFSv2

После измерения производительности новый протокол AFSv2 был признан гораздо лучше масштабируемым, чем первоначальная версия. Каждый сервер мог поддерживать примерно 50 клиентов (а не 20, как раньше). Кроме того, производительность клиента часто была почти такой же, как при локальном доступе, поскольку в типичном случае все обращения к файлам и были локальными; операции чтения обычно производились из локального кеша на диске (и потенциально даже из локальной памяти). И лишь когда клиент создавал новый файл или записывал в существующий, возникала необходимость отправить серверу сообщение Store и обновить содержимое файла.

Оценим производительность AFS и еще одним способом: качественно сравним с NFS поведение в типичных ситуациях (рис. 50.4).

Рабочая нагрузка	NFS	AFS	AFS/NFS
1. Маленький файл, последовательное чтение	$N_s \cdot L_{net}$	$N_s \cdot L_{net}$	1
2. Маленький файл, последовательное повторное чтение	$N_s \cdot L_{mem}$	$N_s \cdot L_{mem}$	1
3. Средний файл, последовательное чтение	$N_m \cdot L_{net}$	$N_m \cdot L_{net}$	1
4. Средний файл, последовательное повторное чтение	$N_m \cdot L_{mem}$	$N_m \cdot L_{mem}$	1
5. Большой файл, последовательное чтение	$N_L \cdot L_{net}$	$N_L \cdot L_{net}$	1
6. Большой файл, последовательное повторное чтение	$N_L \cdot L_{net}$	$N_L \cdot L_{disk}$	L_{disk} / L_{net}
7. Большой файл, одиночное чтение	L_{net}	$N_L \cdot L_{net}$	N_L
8. Маленький файл, последовательная запись	$N_s \cdot L_{net}$	$N_s \cdot L_{net}$	1
9. Большой файл, последовательная запись	$N_L \cdot L_{net}$	$N_L \cdot L_{net}$	1
10. Большой файл, последовательная перезапись	$N_L \cdot L_{net}$	$2 \cdot N_L \cdot L_{net}$	2
11. Большой файл, одиночная запись	L_{net}	$2 \cdot N_L \cdot L_{net}$	$2 \cdot N_L$

Рис. 50.4 ❖ Сравнение AFS с NFS

Мы сравниваем типичные виды чтения и записи для файлов разных размеров. Маленький файл содержит N_s блоков, средний – N_m блоков, а большой – N_L блоков. Предполагается, что маленький и средний файлы помещаются в память клиента, а большие файлы хранятся на локальном диске, но не в памяти.

Также предполагается, что доступ по сети к одному блоку на удаленном сервере занимает L_{net} единиц времени. Доступ к блоку в локальной памяти занимает L_{mem} единиц, а доступ к локальному диску – L_{disk} единиц; при этом $L_{net} > L_{disk} > L_{mem}$.

Наконец, будем предполагать, что при первом обращении файл отсутствует в кеше, а при последующих обращениях (повторное чтение) он уже находится в кеше, если только емкости кеша достаточно для хранения файла.

В столбцах таблицы показано примерное время каждой операции (например, последовательного чтения маленького файла) в NFS и в AFS, а в самом правом столбце – отношение AFS к NFS.

Можно сделать следующие наблюдения. Во-первых, во многих случаях производительность обеих систем приблизительно эквивалентна. Например, при

первом чтении файла (рабочие нагрузки 1, 3, 5) время получения его от удаленного сервера преобладает и примерно одинаково в обеих системах. Можно было бы подумать, что AFS в этом случае должна быть медленнее, т. к. ей приходится записывать файл на локальный диск, но на самом деле эта запись буферизуется кешем локальной (клиентской) файловой системы, поэтому затраты, скорее всего, будут скрыты. Аналогично можно было бы предположить, что чтение из локальной кешированной копии в AFS должно быть медленнее, опять же потому, что AFS хранит кешированную копию на диске. Однако же AFS и в этом случае пользуется всеми преимуществами кеширования в локальной файловой системе; скорее всего, чтение будет удовлетворено из кеша в памяти клиента, так что производительность будет схожей с NFS.

Во-вторых, интересное отличие наблюдается при повторном последовательном чтении большого файла (рабочая нагрузка 6). Поскольку AFS имеет большой локальный кеш на диске, она будет читать файл именно оттуда при повторном обращении. С другой стороны, NFS кеширует в памяти клиента только блоки, поэтому при повторном чтении большого (т. е. не помещающегося в локальную память) файла NFS-клиент должен будет заново получить весь файл от удаленного сервера. Следовательно, в этом случае AFS быстрее NFS в L_{net}/L_{disk} раз в предположении, что удаленный доступ действительно медленнее, чем доступ к локальному диску. Отметим также, что NFS в этом случае увеличивает нагрузку на сервер, что, конечно, отражается на производительности.

В-третьих, мы отмечаем, что последовательная запись (нескольких файлов) должна работать примерно одинаково в обеих системах (рабочие нагрузки 8, 9). AFS в этом случае производит запись в локальную копию, а в момент закрытия файла AFS-клиент отправляет файл серверу, как предписано протоколом. NFS буферизует результаты записи в памяти клиента, быть может, отправляя некоторые блоки серверу, если памяти не хватает, но, безусловно, отправляет все блоки серверу в момент закрытия файла, как предписано политикой сброса при закрытии. Можно было бы предположить, что AFS должна быть медленнее в этом случае, потому что она записывает все данные на локальный диск. Но вспомним, что речь идет о локальной файловой системе, поэтому сначала запись производится в страничный кеш и лишь позже (в фоновом режиме) – на диск, поэтому AFS пожинает плоды инфраструктуры кеширования в памяти ОС на стороне клиента, что повышает производительность.

В-четвертых, AFS хуже работает при последовательной перезаписи файла (рабочая нагрузка 10). До сих пор мы предполагали, что при записи также создается файл, но в данном случае файл уже существует и перезаписывается. Запись в существующий файл – особенно плохой случай для AFS, потому что клиент должен сначала получить файл целиком и только потом вносить в него изменения. NFS, с другой стороны, просто перезаписывает блоки, избегая начального (бесполезного) чтения¹.

¹ Мы здесь предполагаем, что NFS записывает блоками и с выравниванием на границу блока. Если это не так, то NFS-клиенту также пришлось бы сначала прочитать блок. Еще предполагается, что при открытии файла не задавался флаг `O_TRUNC`, иначе в момент открытия AFS не стала бы скачивать файл, который скоро все равно придется усесть.

Наконец, если обращения производятся к небольшому подмножеству данных в большом файле, NFS работает гораздо лучше, чем AFS (рабочие нагрузки 7, 11). В этих случаях AFS скачивает весь файл целиком в момент открытия, но лишь с тем, чтобы прочитать или записать его малую часть. Хуже того, если файл изменится, то придется передать его обратно на сервер, что удваивает потери производительности. NFS, будучи блочным протоколом, несет на такой ввод-вывод затраты, пропорциональные длине чтения или записи.

В общем и целом мы видим, что NFS и AFS делают разные предположения, и неудивительно, что их производительность оказывается различной. Существенные эти различия или нет, как всегда, зависит от характера рабочей нагрузки.

ОТСТУПЛЕНИЕ: ВАЖНОСТЬ РАБОЧЕЙ НАГРУЗКИ

При оценивании системы важно принимать во внимание **рабочую нагрузку**. Поскольку вычислительные системы используются по-разному, видов рабочих нагрузок много. Как проектировщик системы хранения должен выбрать, какие из них важны, чтобы принять разумные решения?

Проектировщики AFS, основываясь на своем опыте наблюдений за эксплуатацией файловых систем, сделали определенные допущения о рабочей нагрузке, в частности предположили, что файлы, как правило, не используются совместно и что доступ к ним последовательный. При таких предположениях дизайн AFS идеален.

Однако эти предположения не всегда оправдываются. Рассмотрим, например, приложение, которое периодически дописывает данные в конец журнала. Такие короткие операции записи в большой существующий файл весьма проблематичны для AFS. Есть и много других неприятных рабочих нагрузок, например произвольные обновления в транзакционной базе данных.

Было выполнено много исследований, из которых можно получить информацию о типичных рабочих нагрузках, например [B+91, H+11, R+00, V99], а также ретроспективный анализ AFS [H+88].

50.8. AFS: ДРУГИЕ УСОВЕРШЕНСТВОВАНИЯ

Как и авторы описанной выше файловой системы Berkeley FFS (куда были добавлены символические ссылки и ряд других средств), проектировщики AFS воспользовались возможностью включить механизмы, упрощающие использование и управление системой. Например, AFS предоставляет клиентам по-настоящему глобальное пространство имен, гарантирующее, что любой файл получает одно и то же имя на всех клиентских машинах. Напротив, NFS позволяет каждому клиенту монтировать NFS-серверы, как ему заблагорассудится, поэтому только благодаря соглашениям (и немалым усилиям администратора) файл будет иметь одно и то же имя на разных клиентах.

AFS также серьезно относится к безопасности: включает механизмы аутентификации пользователей и гарантирует конфиденциальность группы файлов, если их владелец так пожелает. У NFS же в течение многих лет была весьма примитивная поддержка безопасности.

AFS включает также гибкие средства управляемого пользователем контроля доступа. То есть пользователь AFS может строго контролировать, кто именно имеет доступ к его файлам. NFS, как и большинство файловых систем в Unix, гораздо хуже поддерживает такие средства совместного использования.

Наконец, как уже отмечалось, AFS добавляет инструменты, упрощающие управление серверами администраторам системы. В этом отношении AFS намного опередила современные ей системы.

50.9. РЕЗЮМЕ

Пример AFS показывает, что распределенные файловые системы могут сильно отличаться от NFS. Дизайн протокола AFS особенно важен; поскольку взаимодействие с сервером сведено к минимуму (благодаря кешированию файлов целиком и обратным вызовам), каждый сервер может поддерживать много клиентов, так что количество серверов, необходимых для работы конкретного вычислительного центра, уменьшается. У AFS есть также много других приятных черт: единое пространство имен, безопасность и списки контроля доступа. Модель согласованности, предлагаемая AFS, проста для понимания и не приводит к атипичному поведению, как иногда бывает в NFS.

Но, как это ни печально, AFS, по-видимому, клонится к закату. Поскольку NFS – открытый стандарт, ее поддерживают многие производители, и наряду с CIFS (протокол распределенной файловой системы в Windows) NFS доминирует на рынке. Время от времени AFS все еще можно встретить (например, в образовательных учреждениях, в т. ч. в Висконсинском университете), но на какое-то влияние могут рассчитывать лишь идеи AFS, а не сама система. И действительно, в версии NFSv4 добавлено состояние сервера (например, сообщение протокола «open»), так что сходство с базовым протоколом AFS увеличивается.

Литература

[B+91] «Measurements of a Distributed File System» by Mary Baker, John Hartman, Martin Kupfer, Ken Shirriff, John Ousterhout. SOSP '91, Pacific Grove, California, October 1991. *Ранняя работа, посвященная измерению того, как на практике используются распределенные файловые системы. Результаты во многом подтверждают интуитивные представления AFS.*

[H+11] «A File is Not a File: Understanding the I/O Behavior of Apple Desktop Applications» by Tyler Harter, Chris Dragga, Michael Vaughn, Andrea C. Arpaci-Dusseau, Remzi H. Arpaci-Dusseau. SOSP '11, New York, New York, October 2011. *Наша собственная работа, посвященная изучению рабочих нагрузок на настольных компьютерах Apple; как оказалось, они немного отличаются от нагрузок в серверных системах, находящихся в фокусе внимания исследователей. Дополнительно может служить относительно свежим источником ссылок на смежные работы.*

[H+88] «Scale and Performance in a Distributed File System» by John H. Howard, Michael L. Kazar, Sherri G. Menees, David A. Nichols, M. Satyanarayanan, Robert N. Sidebotham, Michael J. West. ACM Transactions on Computing Systems (ACM TOCS), Volume 6:1, February 1988. *Длинная журнальная статья о знаменитой системе AFS, которая до сих пор используется в различных странах, и, пожалуй, самое раннее ясное рассуждение о том, как нужно строить распределенные файловые системы. Удивительное сочетание научных измерений и образцовой инженерной практики.*

[R+00] «A Comparison of File System Workloads» by Drew Roselli, Jacob R. Lorch, Thomas E. Anderson. USENIX '00, San Diego, California, June 2000. *Сравнительно недавние по сравнению с работой Бейкера [B+91] результаты трассировки, есть несколько интересных поворотов.*

[S+85] «The ITC Distributed File System: Principles and Design» by M. Satyanarayanan, J. H. Howard, D. A. Nichols, R. N. Sidebotham, A. Spector, M. J. West. SOSP '85, Orcas Island, Washington, December 1985. *Довольно старая статья о распределенной файловой системе. Многие базовые идеи AFS присутствуют в этой системе, но не усовершенствования с целью повышения масштабируемости. Название было изменено на «Andrew» как дань памяти двум людям, носившим это имя: Эндрю Карнеги и Эндрю Меллону. Эти два денежных мешка основали соответственно Технологический институт Карнеги и Институт промышленных исследований Меллона, которые впоследствии объединились в то, что теперь называется университетом Карнеги-Меллона.*

[V99] «File system usage in Windows NT 4.0» by Werner Vogels. SOSP '99, Kiawah Island Resort, South Carolina, December 1999. *Увлекательное исследование рабочих нагрузок в Windows, которые принципиально отличаются от описанных во многих предшествующих работах для систем на базе Unix.*

Домашнее задание (эмуляция)

В этом задании вы познакомитесь с программой `afs.py`, простым эмулятором, который поможет лучше понять, как работает файловая система Andrew File System. Детали см. в файле README.

Вопросы

1. Рассмотрите несколько простых случаев и попытайтесь предсказать, какие значения будут прочитаны клиентами. Задайте разные начальные значения генератора случайных чисел (-s) и попробуйте предсказать промежуточные и окончательные значения в файлах. Чтобы немного усложнить задание, измените количество файлов (-f), количество клиентов (-c) и коэффициент операций чтения (-r, должен принимать значение от 0 до 1). Можете также сгенерировать более длинные трассы (например, -n 2 или еще больше), чтобы сделать взаимодействие интереснее.

2. То же самое, но предсказать требуется обратные вызовы, инициируемые AFS-сервером. Попробуйте разные начальные значения и укажите высокий уровень детализации вывода (например, -d 3), чтобы при запуске в режиме проверки (с флагом -с) было видно, когда имеют место обратные вызовы. Можете ли вы точно сказать, в какой момент возникает каждый обратный вызов? При каком условии он возникает?
3. То же, что и выше, но запускайте с разными начальными значениями и попытайтесь предсказать точное состояние кеша на каждом шаге. Для проверки задайте флаги -с и -d 7.
4. Теперь построим несколько конкретных рабочих нагрузок. Выполните программу с флагом -A oa1:w1:c1,oa1:r1:c1. Какие значения может наблюдать клиент 1 при чтении файла а, если запускается под управлением случайного планировщика (попробуйте задать разные начальные значения генератора случайных чисел, чтобы получались различные результаты)? Сколько из возможных случайных чередований операций двух клиентов приводят к чтению значения 1 клиентом 1, а сколько к чтению значения 0?
5. Теперь построим несколько конкретных планов. Одновременно с флагом -A oa1:w1:c1,oa1:r1:c1 задайте следующие планы: -S 01, -S 100011, -S 011100 и еще какие-нибудь по своему выбору. Какое значение прочтет клиент 1?
6. Запустите программу с такой рабочей нагрузкой: -A oa1:w1:c1,oa1:w1:c1 – и указанными выше планами. Что происходит при задании флага -S 011100? А при задании флага -S 010011? Что важно для определения конечного содержимого файла?

Глава 51

.....

Заключительный диалог о распределенных файловых системах

Студент. Как же скоро все закончилось. На мой взгляд, слишком скоро!

Профессор. Да, распределенные системы – вещь сложная и увлекательная, они заслуживают изучения, но только не в этой книге (и не на этом курсе).

Студент. Как жаль, а я так хотел узнать больше! Но кое-чему я все-таки научился.

Профессор. Например?

Студент. Ну, что любая вещь может отказать.

Профессор. Неплохое начало.

Студент. Но если таких вещей много (дисков, машин, да всего, чего угодно), то многие отказы можно скрыть.

Профессор. Продолжай!

Студент. Есть ряд полезных базовых методов, например повторная передача.

Профессор. Это правда.

Студент. И нужно тщательно обдумывать протоколы: какой именно информацией обмениваются компьютеры. От протокола зависит всё, в т. ч. как система реагирует на отказ и в какой мере она масштабируема.

Профессор. Наблюдается несомненный прогресс в способности учиться.

Студент. Спасибо! Так и Вы – неплохой учитель!

Профессор. И тебе большое спасибо.

Студент. Так что, это конец книги?

Профессор. Не уверен. Мне пока не сказали.

Студент. Мне тоже. Давайте выйдем отсюда.

Профессор. *Давай.*

Студент. *Прошу Вас.*

Профессор. *Только после Вас.*

Студент. *Пожалуйста, профессора первыми.*

Профессор. *Нет-нет, только после Вас.*

Студент (сердито). *Ну хорошо!*

Профессор (в ожидании). *...Так почему ты еще не ушел?*

Студент. *Я не знаю, как. Получается, что единственное, что я могу делать, – участвовать в этих диалогах.*

Профессор. *Вот и со мной такая же беда. И значит, ты выучил последний урок...*

Предметный указатель

Символы

2Q, 315

A

ACL, 537

AIX, 53

ASID, 249

ASLR, 317

ATA, 467

B

B-дерево, 553

bash, 79

Berkeley Systems Distribution, 53, 320

BKL, 391

BSATF, 497

BSD, 53, 320

btrfs, 613

C

C, язык программирования, 53

CFS, 134

Circular SCAN, 491

CISC, 245, 246

CRC, 649

C-SCAN, 491

D

DEC, 303

Digital Equipment Corporation, 303

Direct Media Interface, 467

DMA, 472

DMI, 467

DOS, 52

dtruss, 521

dup(), 527

dup2(), 527

dup3(), 527

E

ECC, 645

eSATA, 467

exec(), 81

eXternal Data Representation, 674

external SATA, 467

F

Fast File System (FFS), 565

FAT, 552

FCFS, 103

FIFO, 103, 288

FIFO с сегментированием, 307

fork(), 73, 81

free(), 173

F-SCAN, 491

fsck, 579, 582

FTL, 624

G

gcc, 57

H

helgrind, 354

HPUX, 53

HUP, сигнал, 456

I

INT, сигнал, 456

IRIX, 53

K

kill, 456

kmalloc, 310

L

LFS, 601

LFU, 291

Linux, 54, 302

Linux ext2, 585

Linux ext3, 584

Linux ext4, 588

lmbench, 96

LRU, 250, 291, 558

M

Mac OS X, 53

Meltdown, 318

MenuMeters, 81
Mesa семантика, 403, 408
MFU, 292
Microsoft, 52
MLC, 619
MQMS, 148
MRU, 292
Multics, 53

N

NBF, 490
new, 173
NeXTStep, 54
nice, 136
node.js, 450
NVMe, 468

P

PCB, 69
PCI, 466
PCIe, 468
PDE, 262
Peripheral Component Interconnect Express, 468
PFN, 229
PID, 73
pipe, системный вызов, 400
PPN, 229
PSJF, 106
purify, 177

R

RAID, 498
 программный, 514
RAID 0+1, 505
RAID-01, 505
RAID 1+0, 505
RAID-10, 505
RAID-DP, 647
RAM, 252
RISC, 245, 246
RMI, 670
root, 80, 537
RPC, 670
RSS, 307

S

SATA, 466, 467
SATF, 492
SCAN, 491
sched_latency, 135
SCSI, 466

SEGV, 456
SIGSEGV, 456
SLC, 619
Spectre, 318
SPTF, 492
SQMS, 147
SSD, 46, 618, 639
SSD-диск на основе флеш-памяти, 64, 613
SSF, 490
STTF, 490
STCF, 106
strace, 521
SunOS, 53
swapon, 319

T

TCP/IP, 668
tcsh, 79
TLB, 240
 аппаратно управляемый, 241, 245
 емкость, 252
 непопадание, 241, 276
 полностью ассоциативный, 247
 попадание, 241, 275
 сброс в начальное состояние, 249
TLC, 619
TOCTTOU, 538
truss, 521

U

UDP/IP, 664
USB, 466, 468

V

valgrind, 177
VAX/VMS, 302
Venus, 698
VFS, 692
Vice, 698
vmalloc, 311
vmstat, 281
vsfs, 544

W

WAFL, 613, 647
wait(), 81, 83

X

x86, 235

XDR, 674

XOR, 508

Z

Zettabyte File System (ZFS), 613, 652

A

Абсолютный путь, 518

Абстракция, 48, 164, 474

Аварийный отказ, 578

Авария головки, 623, 645

Автоматическая память, 170

Автоматическое управление памятью, 173

Аддитивная четность, 510

Адрес, 42

Адреса логических блоков, 626

Адресное пространство, 44, 62, 69, 159, 162, 183, 328, 481

Активная блокировка, 442, 471

Активное ожидание, 359, 360

Алгоритм близнецов, 203

Алгоритм часов, 296, 316

Амортизация, 108, 572, 603

Анализ размерностей, 486

Анонимная область памяти, 177, 314

Аппаратная трансляция адресов, 182

Аппаратный уровень привилегий, 51

Асинхронный ввод-вывод, 455

Атаки возвратом в libc, 317

Атомарный, 46, 337, 338, 481, 506, 528, 582

Атомарный обмен, 361

Б

База, 185, 187, 260

Банк, 619

Барьер записи, 587

Барьерная синхронизация, 431

Безблокировочные структуры данных, 145, 392, 442

Без ожидания (структуры данных), 442

Безопасность, 49, 54, 80, 89, 120, 663, 678

Безотлагательно, 38, 162

Безотлагательное выполнение, 64

Без поддержания загрузки, 494

Белади аномалия, 289

Библиотека времени выполнения, 670

Билет, 127, 129

валюта, 129

инфляция, 129

передача, 129

Билл Джой, 53

Бит

достоверности, 248, 262

доступа, 232

запрета выполнения (NX), 317

изменения, 232, 248, 297

использования, 296

обращения, 232, 296, 306

Битовая карта, 547

данных, 547, 567, 579

индексных дескрипторов, 547, 567, 579

Бит присутствия, 232, 276, 280

Биты защиты, 200, 232

Биты полномочий, 536, 541

Блок, 482, 545, 619

Блокировка, 96, 146, 147, 327, 348, 355

билетная, 368

вперехват, 388

доступная, 355

занятая, 355

захваченная, 355

крупная, 356

мелкая, 357

на уровне файлов, 702

свободная, 355

цепная, 388

чтения-записи, 422

Блок управления асинхронным вводом-выводом, 455

Блок управления потоком, 328

Большая блокировка ядра, 391

Буфер ассоциативной трансляции (TLB), 240, 253

Буфер дорожки, 486, 574

Буферизация записи, 559, 603, 688, 693

В

Ввод-вывод, 46

команды, 473

перенаправление, 81

с отображением памяти, 473

шина, 466

Верификация записи, 652

Верхний предел, 280

Взаимное исключение, 337, 340, 356, 357

Взаимоблокировка, 354, 427, 433, 437

Взаимоблокировки

избегание, 444

Видимость обновлений, 689

Виртуализация, 39, 59
 памяти, 43, 164
 процессора, 41
Виртуальная машина, 39
Виртуальная память, 328
Виртуальная файловая система, 692
Виртуальное адресное пространство, 44
Виртуальное время работы, 134
Виртуальный адрес, 164, 165, 186
Виртуальный процессор, 328
Виртуальный узел (vnode), 692
Виртуальные адреса ядра, 311
Висячая ссылка, 535
Висячий указатель, 175
Владелец, 356, 536
Внедрение, 499
Внутренняя структура, 468
Возврат из системного прерывания, 51, 88, 97
Возвратно-ориентированное программирование (ROP), 317
 гаджет, 317
Вольтера закон, 688
Воспроизведение транзакций, 589
Восстановление после отказа, 507, 588
Вполне равномерный планировщик, 134
Вредоносный планировщик, 364
Временной квант, 107
Время
 отклика, 107
 простоя, 280
 установления, 484
Выборка команды, 38
Выбор страницы, 298
Выбрать и прибавить, 368
Выгрузка, 232, 274
Выгрузка страницы, 278
Выделение, 556
Вызов процедуры, 51, 62, 69, 348
Выравнивание износа, 625, 628, 635, 639
Вытеснение, 284, 635
Вытеснение страниц, 232
Вычисления со сложным набором команд (CISC), 246
Вычисления с сокращенным набором команд (RISC), 246

Г

Генератор заглушек, 670
Гибрид, 258, 261, 376, 471
Гибридное отображение, 633, 639
Гигантские страницы, 313

Главная управляющая программа, 39
Головка диска, 482
Голодная смерть, 119
Гонка
 за данные, 336, 340
 общего вида, 341
Горячая замена, 514
Граница, 185, 187, 260
Группа
 блоков, 566
 пользователей, 536
 процессов, 80
 цилиндров, 565
Группировка, 298
Группировка страниц, 280
Грязная страница, 297, 315, 528, 635

Д

Двоичный семафор, 416
Двойное освобождение, 176
Двойной косвенный указатель, 550
Двухфазная блокировка, 375
Двухфазный подход, 471
Дейтаграмма, 664
Деккера алгоритм, 362
Декодирование, 38
Демаршalling, 671
Демон выгрузки, 280
Демон страничного обмена, 280
Дерево каталогов, 518, 540
Десериализация, 671
Дескриптор файла, 65, 520, 540
Детерминированность, 74, 335, 337, 340
Дефекты, 680
Дефицит памяти, 284
Дефрагментация, 564
Дизассемблер, 235
Динамическое перемещение, 185
Динамическое разбиение, 558, 559
Диск, 64
Дисковый адрес, 274
Диспетчер ресурсов, 39, 42
Дисциплина планирования, 101
Длинные имена файлов, 574
Долговременное хранение, 46, 465, 517, 578
Дорожка, 482
Драйвер устройства, 48, 474
Дырки, 610

Е

Емкость, 501

Ж

Жадный алгоритм, 497
 Жесткая ссылка, 541
 Жесткий диск, 46, 273, 481, 517, 636
 Журнал, 612
 с повтором, 589
 с упреждающей записью, 507
 Журналирование, 48, 579, 584
 данных, 585, 591
 логическое, 585
 метаданных, 591
 физическое, 585
 Журнальные блоки, 633

З

Зависание, 490
 Заголовок, 211
 Загрузка
 в память, 64
 по связи, 366
 Загрузчик, 186
 Задания, 102
 Задача
 об обедающих философах, 425
 о курильщиках, 427
 о разветвлении и соединении, 431
 о randevu, 431
 о спящем парикмахере, 427
 о читателях и писателях, 432
 Задержка вращения, 484
 Заимствование работ, 151
 Заккрытие перед открытием, 690, 693
 Запись каталога страниц, 262
 Запись не по адресу, 651
 Запись таблицы страниц (PTE), 230, 276
 Запоминающее устройство
 с произвольной выборкой (ЗУПВ), 252
 Запрет прерываний, 96
 Зашитые трансляции, 246
 Защита, 49, 89, 159, 162, 165, 248
 Защита данных, 644
 Защита памяти, 52
 Здравый смысл, 569
 Земафор, 428
 Зеркалирование, 500, 501
 Зомби, 69

И

Идеальное масштабирование, 382
 Идемпотентность, 686, 693
 Идентификатор
 адресного пространства, 249

процесса, 73, 249
 процесса (PID), 81, 169
 транзакции (TID), 585
 файла (FID), 700
 Иерархия запоминающих устройств, 273
 Иерархия каталогов, 518, 540
 Избыточность, 499
 Избыточный массив недорогих
 дисков, 498
 Измерение, 699
 Износ, 618, 622, 623, 625, 638
 Изоляция, 49, 159, 164
 Изоляция таблицы страниц ядра
 (KPTI), 319
 Иллюзия, 41, 182
 Именованное, 519, 672
 Инвариант, 508
 Инверсия приоритетов, 373
 Инвертированная таблица страниц, 228, 269
 Индекс каталога страниц, 265
 Индекс справедливости Джейна, 102
 Индекс таблицы страниц, 265
 Индексный дескриптор, 546, 548, 567, 602
 Инкапсуляция, 439
 Интерактивность, 161
 Интерфейс, 468
 Интерференция, 623
 программирования, 623, 627
 чтения, 623
 Искажение блока, 514, 645
 Исключение, 189
 Исполняемый формат, 64

К

Каллера закон, 252
 Канал, 79
 Карта индексных дескрипторов
 (i-карта), 605
 Каталог, 518, 540
 Каталог страниц, 262, 266
 Квантование времени, 107
 Квант планирования, 107
 Кеш, 284, 486
 аппаратный, 143
 атрибутов, 690
 буферный, 580
 виртуально индексируемый, 253
 вытеснение из, 250
 когерентность, 145
 непопадание в, 285
 объектный, 221
 полностью ассоциативный, 287

попадание в, 285
 постоянного размера, 558
 проблема согласованности, 688, 693
 сброс, 630
 страничный, 314, 580
 трансляции адресов, 240
 устаревший, 689
 физически индексируемый, 253
 Кеширование, 314, 639, 688, 693
 со сквозной записью, 486
 с отложенной записью, 486
 Кеширование файла целиком, 698
 Кластеризация, 280, 298, 308
 Клиент-серверная архитектура, 662
 Клиентская заглушка, 671
 Клиентская файловая система, 679
 Кнут, 391
 Код, 162
 Коды, исправляющие ошибки, 645
 Колдовские константы, 120
 Коллизия, 649
 Коммуникация, 663
 библиотека, 676
 оконечная точка, 664
 Компилятор протокола, 670
 Компромисс, 109
 Конвейер, 53, 81
 Конкурентность, 44, 48, 52, 326
 Контекстное переключение, 61, 68, 94, 96, 105, 328
 Контроль допуска, 298
 Контрольная сумма, 648, 664
 вычисленная, 650
 хранямая, 650
 контрольная точка, 586, 628
 Контрольное сообщение, 704
 Кооперативный подход, 91
 Копирование при записи, 48, 308, 595, 613
 Корневой каталог, 518, 540, 554
 Корректность, 363
 Косвенный указатель, 550
 Коэффициент
 использования, 161
 непопаданий, 250, 285
 попаданий, 243, 250, 285
 Красно-черное дерево, 137
 Критическая секция, 337, 340, 348
 Кронштейн, 482
 Куча, 65, 162, 171, 353

Л

Лауэра закон, 367
 Ленивое выполнение, 64, 308

Линейная таблица страниц, 231, 241
 Линус Торвальдс, 54
 Лифт, 492
 Логические адреса ядра, 310
 Ложное пробуждение, 408
 Локальность, 244
 временная, 144, 244, 291
 переменная, 355
 пространственная, 144, 244, 291
 Лотерейное планирование, 127
 Лучший подходящий, 203, 218

М

Маршalling, 671
 Масштабируемость, 147, 697
 Машинное состояние, 62
 Мертвые блоки, 629
 Метаданные, 529, 546, 549, 602
 Метод близнецов, 221
 Методы доступа, 545
 Метрика неравномерности, 131
 Метрика планирования, 102
 Механизм теневых страниц, 613
 Механизмы, 42, 61, 101, 156, 166
 Миграция, 148, 150
 Микроядро, 70, 164
 Мини-компьютер, 51
 Многозонный диск, 486
 Многопоточность, 44, 74, 327, 328
 Многоуровневая аналитическая очередь (MLFQ), 112, 114
 очереди, 115
 уровень приоритета, 115
 Многоуровневая таблица страниц, 245, 261
 Многоуровневый индекс, 551
 Многоядерный, 142
 Мобильность, 49
 Модульность, 63
 Монитор, 381
 Мониторинг протокол, 684
 Мультипрограммирование, 51, 161, 330
 Мультипроцессор, 142
 Мусор, 581, 609, 628
 Мьютекс, 356
 справедливый, 432
 Мягкая ссылка, 534

Н

Надежность, 49, 501
 Накат, 612
 Нарушение атомарности, 434
 Нарушение порядка, 434, 435
 Наследование приоритетов, 373

Небрежный счетчик, 392
 Недавность, 290
 Недействительность, 231, 690
 Недетерминированность, 74, 337, 340
 Немедленная отчетность, 486, 587
 Непопадание
 вынужденное, 286, 287
 из-за емкости, 287
 из-за конфликта, 287
 из-за холодного старта, 286, 287
 Несбалансированность нагрузки, 149
 Нижний предел, 280
 Низкоуровневое имя, 518, 548
 Номер версии, 611
 Номер виртуальной страницы (VPN), 228
 Номер индексного дескриптора, 518, 540, 548
 Номер физической рамки, 229
 Нулевой указатель, 305

О

Облако, 139
 Область
 данных, 546
 контрольной точки, 606
 подкачки, 177, 274
 Обновление, 42, 145
 Обнуление страниц по запросу, 308
 Оболочка, 53, 81
 Обратное время, 102
 Оборотов в минуту, 482, 486
 Обработчик
 исключения, 189, 190
 отказа страницы, 277, 281
 прерывания, 470
 сигнала, 456
 событий, 451
 Обратный вызов, 700
 Обратный указатель, 596
 Объединение
 ввода-вывода, 494
 переключающее, 633
 полное, 634
 прерываний, 471
 свободных блоков, 210, 217
 частичное, 634
 Оверлейная память, 274
 Ограниченное прямое выполнение (LDE), 86, 90, 97, 156, 181, 192
 Ограниченный буфер, 399, 418
 Ограниченный SATF, 497
 Операционная система (OC), 39

Описатель файла, 682, 700
 Опрос, 456, 469, 700
 Оптимальность, 104, 286
 Оптимистическая согласованность после отказа, 596
 Отбрасывание пакетов, 664
 От времени проверки до времени использования, 538
 Отзыв, 594
 Отказ, 662
 Отказ страницы, 276, 315
 Отключение питания, 578, 680
 Открытый протокол, 680
 Отсутствие страницы, 276
 Оустерхаута закон, 122
 Очистка диска, 653
 Ошибка сегментации, 174, 198

П

Пакет, 559
 Пакетный режим, 50
 Параллелизм, 330
 Параметризация, 574
 Паттерсона закон, 699
 ПДП, 472
 Первый подходящий, 203, 219
 Первым пришел, первым обслужен, 103
 Первым пришел, первым ушел, 103
 Передача, 485
 Перезагрузка машины, 92
 Переиграть планировщик, 119
 Перекрытие, 109, 111, 277, 330, 454, 470
 Перемещение, 184
 динамическое, 186
 статическое, 186
 Перенаправление, 77
 Переполнение буфера, 174, 316
 Периферийная шина, 466
 Персональный компьютер, 52
 Петерсона алгоритм, 362
 Планирование диска
 ближайший блок первым, 490
 принцип SJF (кратчайшее задание первым), 490
 с наименьшим временем позиционирования первым, 492
 с наименьшим временем поиска первым, 490
 Планирование мультипроцессора с несколькими очередями (MQMS), 148
 Планирование мультипроцессора с одной очередью (SQMS), 147

- Планирование процессора
в многопроцессорных системах, 142, 143
некооперативное, 97
политики, 62, 101
циклическое, 107, 149
- Планировщик, 67, 74, 94, 333
вытесняющий, 105, 363
диска, 490
невывтесняющий, 105
пропорциональный, 127
равномерный, 127
- Пластина, 482
- Плоскость, 619
- Поверхность, 482
- Повторная попытка, 667, 677
- Повышение приоритета, 120
- Пограничная зона, 628
- Подблок, 573
- Подкаталог, 518
- Подкачка, 64
- Подкачка по запросу, 298, 315
- Подкачка страницы, 277
- Пожиратели памяти, 307
- Поиск, 484
- Покрывающее условие, 410
- Политика вытеснения
наиболее часто используемая
(MFU), 292
наименее часто используемая
(LFU), 291
по давности использования (LRU), 250, 291
по недавности использования
(MRU), 292
- Политика замещения страниц, 278
- Политики, 42, 62, 166
- Полное упорядочение, 440
- Полоса, 501
- Пользователь, 80, 81
- Понижение приоритета, 123
- Порядковый номер, 668
- Порядок байтов, 674
обратный, 674
прямой, 674
- Последний закрывший выигрывает, 703
- Последний записавший выигрывает, 703
- Последовательный доступ, 487, 503, 522
- ПО с открытым исходным кодом, 53
- Потерянная запись, 652
- Поток, 44, 142, 163, 327, 328
- Потокобезопасность, 380
- Потомок (дочерний процесс), 73, 81
- Поточно-локальная память, 329
- Потребитель, 401
- Предварительное выделение, 554
- Предвыборка, 298
- Предел, 185, 187
- Преждевременная оптимизация, 392
- Прекращение работы при ошибке, 500, 644
- Прерывание, 456, 470, 700
- Прерывание от таймера, 92, 97
- Приближенный счетчик, 383
- Приведение типа, 172
- Привилегированные операции, 89, 252, 473
- Привилегированный режим, 188
- Привязка к процессору, 146, 154
- Приглашение, 77
- Примитивы синхронизации, 338
- Принцип локальности, 290, 292
- Проблема короткой записи, 511, 601
- Проблема отображения, 502
- Проблема рекурсивного обновления, 608
- Проблема согласованного обновления, 506, 582
- Проблема согласованности после отказа, 578, 582
- Пробуксовка, 298
- Проверка, 363
- Проверка и установка, 361
- Программа обработки прерывания, 470
- Программируемый ввод-вывод, 469
- Прогресс, 133
- Продолжение, 458
- Прозрачность, 165, 183, 281, 313, 499, 679
- Прозрачный интерфейс, 475
- Прозрачный интерфейс с диском, 560
- Производитель, 401
- Производительность, 49, 102, 357, 364, 501, 663
- Произвольный доступ, 487, 503, 523
- Проклятие общности, 304
- Простота использования, 159
- Противодействие сканированию, 299
- Протокол
без сохранения информации
о состоянии, 693
с запоминанием состояния, 681
- Протокол восстановления, 682
- Протоколирование, 626, 628
- Процесс, 61
блок управления (PCB), 69, 190, 328
дескриптор, 69

слово состояния, 188
 состояния, 69
 API, 69
 Прочие, 536
 Прошивка, 468
 Прямое отображение, 625
 Прямой ввод-вывод, 560
 Прямой доступ к памяти, 472
 Прямой указатель, 550
 Пул потоков, 672
 Путь доступа, 554
Р
 Рабочая нагрузка, 101, 292, 579, 707
 асимметрия, 643
 Рабочий набор, 298, 635
 Разделение, 46, 200, 678
 блока памяти, 210, 213
 времени, 61, 85, 86, 161
 Разделение кода, 200
 Разделенная сеть, 680
 Разделитель, 518
 Разделяемое состояние, 681
 Размер порции, 502
 Размер резидентного набора, 307
 Разорванная запись, 482
 Разреженное адресное пространство, 197
 Рандомизация, 128
 Рандомизация распределения адресного пространства, 317
 Рандомизация распределения адресного пространства ядра (KASLR), 318
 Распараллеливание, 142, 330
 Распределенная разделяемая память, 669
 Распределенное состояние, 681
 Расширение записи, 624, 639
 Регистр
 данных, 469
 команд, 469
 состояния, 469
 Регистр базы таблицы страниц, 233, 245, 276
 Регистровый контекст, 68
 Режим пользователя, 51, 87, 97, 188
 Режим файла, 537
 Режим ядра, 51, 87, 97, 188
 Резервирование, 630
 Рекомендация, 123
 Реконструкция, 509, 647
 Ресурс, 39, 48
 Ровно один раз, 667
 Родитель, 73, 81

Ротация четности, 512
 Ручное управление стеком, 457

С

Сборка, 673
 Сборка мусора, 609, 628, 639
 Сборщик мусора, 173
 Сброс при закрытии, 689, 693
 Сверхлинейное ускорение, 155
 Сводный блок сегмента, 610
 Свойство стека, 289
 Связный список, 552
 Сдвиг дорожек, 485
 Сегмент, 195, 601, 603
 Сегментация, 193, 195, 207, 226
 крупноструктурная, 201
 мелкоструктурная, 201
 Сегрегированные списки, 220
 Сектор, 482
 Секторная ассоциативность, 287
 Семафор, 413
 Серверная файловая система, 679
 Сериализация, 671
 Сигнализация, 396
 Сигналы, 79, 81, 456
 Символическая ссылка, 534, 541, 575
 Синхронный, 452, 675
 Система управления базами данных, 252
 Системное прерывание, 51, 88, 97
 Системный вызов, 51, 88, 679
 номер, 89
 Сквозной аргумент, 664, 674
 Скрытые ошибки секторов, 514, 645
 Следующий подходящий, 219
 Слежение за шиной, 145
 Слябовый распределитель, 221
 Смещение
 в файле, 523
 на странице, 229
 Сначала самое короткое
 с вытеснением, 106
 Сначала самое короткое (SJF), 104
 Сначала с наименьшим временем
 до завершения, 106
 Снимки, 613
 Снятие с процессора, 66
 Событийно-управляемая
 конкурентность, 451
 Согласованность на основе обратных
 указателей, 596
 Сокеты, 664

Состояние, 65
 готов, 65
 заблокирован, 66
 исполняется, 65
Состояние гонки, 336, 340
Сохранить условно, 366
Спин-блокировка, 361, 363
Списки второго шанса, 307
Список
 активных, 316
 задач, 69
 контроля доступа, 537, 541
 неактивных, 316
 процессов, 67, 69
 свободных, 188, 189, 208, 228, 547, 553
С поддержанием загрузки, 494
Справедливость, 102, 109, 357, 363
Сравнить и обменять, 364
Среднее время доступа к памяти, 285
Среднее оборотное время, 103
Стандартная библиотека, 39, 48
Стандартный вывод, 77
Статическое разбиение, 558, 559
Стек, 65, 162, 170
Стек ядра, 88
Стираемые блоки, 619, 638
Стирание, 639
Страница, 226
Страницы руководства, 79
Страничная организация, 64, 207, 226, 237, 458
Страничная рамка, 226
Страничный уровень, 631
Стрелка часов, 296
Структура учета выделения, 547
Структуры данных, 69, 544
СУБД, 252
Субтрактивная четность, 510
Суперблок, 547, 567
Суперблок журнала, 590
Супервизор, 39
Суперпользователь, 80, 81, 537
Суперстраницы, 271
Счетчик команд, 62
Счетчик ссылок, 526, 534

Т

Таблица
 индексных дескрипторов, 546
 открытых файлов, 523, 541

 отображения, 626
 отображения в памяти, 627
 прерываний, 89, 97
 распределения файлов, 552
 сегментов, 201
 страниц, 228, 234
Тайм-аут, 667
Твердотельное запоминающее устройство, 517, 618, 639
Текущее смещение, 541
Типичный случай, 690
Тип файла, 518
Тихие отказы, 500
Точка монтирования, 539
Транзакция, 338
Трансляция адреса, 182, 186, 192
Трансляция адресов, 228
Трансляция виртуальных адресов в физические, 234
Три С, 287
Тройной косвенный указатель, 550
Тьюринга премия, 114

У

Убийца пожирателей памяти, 298
Удаленный вызов методов, 670
Удаленный вызов процедур, 670
Указание, 123
Указатель
 кадра, 63
 на функцию, 344
 на void, 208, 344
 стека, 63, 69
Унифицированный страничный кеш, 558
Уплотнение, 202, 610
Упорядочение, 417
Упорядоченное журналирование, 591
Управление памятью, 182
Управление свободным пространством, 207, 553
Упреждающая запись в журнал, 579, 584
Упреждающее выполнение, 318
Упреждающее планирование диска, 494
Уровень косвенности, 263, 605
Усечение, 630, 638
Условная переменная, 327, 339, 350, 396, 417, 436
Установка, 363
Устройство ввода-вывода, 466
Устройство управления памяти (MMU), 187, 188, 240
Уступать, 91

Утечка места, 581
 Утечка памяти, 175
 Учет, 121

Ф

Файл, 518, 540
 Файловая система, 46, 47, 50
 версионная, 609
 несогласованность, 581
 со структурой журнала, 601, 626, 639
 средство проверки, 475, 579
 транзакционная, 539
 Файловый сервер, 679
 Файлы, отображенные в память, 314
 Физическая память, 42
 Физическая страница, 626
 Физический адрес, 186
 Физический идентификатор, 651
 Флетчера контрольная сумма, 649
 Флеш-память, 618
 блок, 618, 638
 микросхема, 638
 параллельная работа, 624
 программирование страницы, 620, 638
 произвольная выборка, 620
 с одноуровневыми ячейками, 619
 стирание блока, 620, 638
 страница, 618, 638
 типа NAND, 618
 уровень трансляции, 639
 уровень трансляции (FTL), 624
 чтение страницы, 620
 Фон Нейман, 38
 Фоновый режим, 280, 630
 Фрагментация, 226, 673
 внешняя, 203, 207, 208
 внутренняя, 192, 208, 258, 314, 565, 573
 Фьютекс, 374

Х

Хеш-таблица страничного кеша, 314
 Хилла закон, 425

Хоара семантика, 403
 Худший подходящий, 203, 218

Ц

Целостность данных, 644
 Цельнополосная запись, 509
 Централизованное
 администрирование, 678
 Цикл в графе, 438
 Циклический журнал, 590
 Циклический избыточный код, 649
 Цикл событий, 451
 Цилиндр, 565

Ч

Частичная неработоспособность
 при ошибке, 645
 Частичное упорядочение, 440
 Частота, 290
 Чередование, 501
 Четность, 508
 Чистая страница, 297, 315
 Чтение неинициализированной
 памяти, 175
 Чтение после записи, 652

Ш

Шаг, 132
 Шаговое планирование, 132
 Шина памяти, 466
 Шпиндель, 482

Э

Эдсгер Дейкстра, 413
 Экспоненциальная задержка, 669
 Экстент, 550
 Энергозависимый, 46
 Энергоэффективность, 49
 Эскалация привилегий, 317
 Эффективность, 161, 165
 Эффект сопровождения, 104

Список врезок

«Отступление»

Отступление: важность Unix	53
Отступление: и тогда пришла LINUX	54
Отступление: структура данных – список процессов.....	69
Отступление: термины, связанные с процессом	69
Отступление: интерлюдии	72
Отступление: RTFM – читайте страницы руководства.....	79
Отступление: суперпользователь (root)	80
Отступление: основные термины API процессов	81
Отступление: задачи на кодирование	83
Отступление: почему системный вызов выглядит как вызов функции	87
Отступление: сколько времени занимает контекстное переключение	96
Отступление: основные термины, относящиеся к виртуализации CPU (механизмы)	97
Отступление: домашние задания на измерение	99
Отступление: вытесняющие планировщики.....	105
Отступление: главы повышенной сложности	143
Отступление: все адреса, которые вы видите, виртуальные.....	165
Отступление: почему память не утекает после завершения процесса	176
Отступление: программное перемещение	186
Отступление: структура данных – список свободных.....	188
Отступление: ошибка сегментации	198
Отступление: великие инженеры действительно великие	221
Отступление: структура данных – таблица страниц	234
Отступление: RISC и CISC	246
Отступление: бит достоверности в TLB ≠ бит достоверности в таблице страниц	247
Отступление: страницы нескольких размеров	258
Отступление: технологии хранения данных	274
Отступление: терминология и другие вопросы	276
Отступление: почему оборудование не обрабатывает отказы страниц.....	277
Отступление: типы непопадания в кеш	287
Отступление: аномалия Беладии.....	289
Отступление: типы локальности	291
Отступление: проклятие общности	304
Отступление: почему доступ по нулевому указателю вызывает ошибку сегментации.....	306
Отступление: эмуляция битов обращения	307
Отступление: повсеместность отображения в память.....	315

Отступление: основные термины конкурентности – критическая секция, состояние гонки, недетерминированность, взаимное исключение	340
Отступление: рекомендации по использованию API потоков	352
Отступление: алгоритмы Деккера и Петерсона	362
Отступление: еще одна причина избегать активного ожидания – инверсия приоритетов	373
Отступление: блокирующие и неблокирующие интерфейсы	452
Отступление: сигналы UNIX	456
Отступление: анализ размерностей	486
Отступление: вычисление «среднего» времени поиска	489
Отступление: проблема отображения в RAID	502
Отступление: проблема согласованного обновления RAID	506
Отступление: системный вызов CREAT()	520
Отступление: структура данных – таблица открытых файлов	523
Отступление: вызов LSEEK() не приводит к поиску на диске	525
Отступление: роль суперпользователя в файловой системе	537
Отступление: терминология файловых систем	540
Отступление: мысленные модели файловых систем	545
Отступление: структура данных – индексный дескриптор	548
Отступление: подходы на основе связей	552
Отступление: управление свободным местом	553
Отступление: при чтении не нужен доступ к структурам для выделения	555
Отступление: создание файла в FFS	567
Отступление: форсированная запись на диск	586
Отступление: оптимизация записи в журнал	587
Отступление: сохранение информации об отображении в FTL	628
Отступление: новый API систем хранения – усечение	630
Отступление: основные термины SSD	638
Отступление: сквозной аргумент	674
Отступление: почему падают серверы	680
Отступление: инновации рожают инновации	692
Отступление: терминология NFS	693
Отступление: согласованность кешей – не панацея	702
Отступление: важность рабочей нагрузки	707

Список врезок «Совет»

Совет: применяйте разделение времени (и пространства)	62
Совет: различайте политику и механизм	63
Совет: сделать правильно (закон Лэмпсона)	77
Совет: применяйте защищенную передачу управления	88
Совет: берегитесь данных от пользователя в безопасных системах	89
Совет: обработка неправильного поведения приложения	92
Совет: использование таймера для восстановления контроля	93
Совет: перезагрузка полезна	93
Совет: принцип «сначала самое короткое»	104
Совет: амортизация может уменьшить стоимость	108
Совет: перекрытие открывает возможность для более высокой степени использования	109
Совет: обучаться на исторических данных	115
Совет: планирование должно быть защищено от атак	120
Совет: избегайте колдовских констант (закон Оустерхаута)	122
Совет: по возможности пользуйтесь механизмом указаний	123
Совет: о пользе случайности	128
Совет: используйте билеты для представления долей	129
Совет: используйте эффективные структуры данных там, где это оправдано	139
Совет: принцип изоляции	164
Совет: сомневаешься – попробуй	172
Совет: компилируется и запускается ≠ правильна	174
Совет: вмешательство – это круто	183
Совет: аппаратное динамическое перемещение	187
Совет: если существует 1000 решений, то лучшего среди них нет	203
Совет: при любой возможности задействуйте кеширование	244
Совет: ЗУПВ не всегда ЗУПВ (закон Каллера)	252
Совет: применяйте гибридные подходы	261
Совет: разберитесь в компромиссах между пространством и временем	263
Совет: берегитесь сложности	264
Совет: работайте в фоновом режиме	280
Совет: сравнение с оптимумом полезно	286
Совет: быть ленивым хорошо	309
Совет: постепенность – это хорошо	314
Совет: знайте и используйте свои инструменты	335
Совет: использование атомарных операций	338
Совет: взгляд на конкурентность как на вредоносный планировщик	364
Совет: чем меньше кода, тем лучше (закон Лауэра)	367
Совет: конкурентнее необязательно быстрее	388
Совет: будьте внимательны к блокировкам на разных путях выполнения	388
Совет: избегайте преждевременной оптимизации (закон Кнута)	391

Совет: сигнализируя, всегда удерживайте блокировку.....	399
Совет: используйте while, а не if для проверки условий.....	408
Совет: незамысловатое может оказаться лучше (закон Хилла).....	425
Совет: осторожнее с обобщениями.....	429
Совет: упорядочивайте блокировки по адресу	440
Совет: не всегда надо стремиться к совершенству (закон Тома Уэста)	445
Совет: остерегайтесь блокирования в событийно-управляемых серверах	454
Совет: прерывания не всегда лучше РЮ	471
Совет: работайте с дисками последовательно	488
Совет: всё всегда зависит от обстоятельств (закон Ливны).....	493
Совет: прозрачность способствует внедрению	499
Совет: относитесь к именам серьезно	519
Совет: пользуйтесь STRACE (и другими подобными инструментами)	521
Совет: будьте осторожны с мощными командами	531
Совет: остерегайтесь TOCTTOU	538
Совет: подумайте о решениях на основе экстенгов.....	550
Совет: сравнение статического и динамического разбиений	559
Совет: компромисс между долговечностью и производительностью.....	560
Совет: системой должно быть удобно пользоваться.....	575
Совет: детали имеют значение	601
Совет: косвенность – это хорошо	605
Совет: обратить недостатки в достоинства	613
Совет: осторожнее с терминологией	619
Совет: важность обратной совместимости	623
Совет: бесплатных завтраков не бывает	648
Совет: связь принципиально ненадежна	663
Совет: о применении контрольных сумм для проверки целостности	665
Совет: осторожнее при установке тайм-аута	669
Совет: о важности идемпотентности.....	686
Совет: лучшее – враг хорошего (закон Вольтера)	688
Совет: сначала измерьте, потом стройте (закон Паттерсона)	699

Список врезок

«Существо проблемы»

Существо проблемы: как виртуализировать ресурсы	39
Существо проблемы: как правильно писать конкурентные программы.....	44
Существо проблемы: как сохранять данные на постоянной основе	47
Существо проблемы: как создать иллюзию существования многих процессоров?	61
Существо проблемы: как создавать процессоры и управлять ими?	72
Существо проблемы: как эффективно виртуализировать CPU, сохранив над ним контроль.....	85
Существо проблемы: как выполнять операции с ограниченным доступом	87
Существо проблемы: как вернуть себе контроль над CPU.....	91
Существо проблемы: как вернуть себе контроль без кооперации	92
Существо проблемы: как разработать политику планирования	101
Существо проблемы: как планировать, не имея точных знаний?	115
Существо проблемы: как разделить CPU пропорционально	127
Существо проблемы: как планировать задания на нескольких CPU	143
Существо проблемы: как справиться с несбалансированной нагрузкой.....	150
Существо проблемы: как виртуализировать память.....	164
Существо проблемы: как выделить память и управлять ей.....	170
Существо проблемы: как эффективно и гибко виртуализировать память.....	181
Существо проблемы: как поддерживать большое адресное пространство.....	195
Существо проблемы: как управлять свободным пространством	208
Существо проблемы: как виртуализировать память с помощью страниц	226
Существо проблемы: как ускорить трансляцию адресов.....	240
Существо проблемы: как управлять содержимым TLB при контекстном переключении	249
Существо проблемы: как спроектировать политику вытеснения.....	250
Существо проблемы: как уменьшить размер таблиц страниц?.....	257
Существо проблемы: как выйти за пределы физической памяти	273
Существо проблемы: как решить, какую страницу вытеснить.....	284
Существо проблемы: как реализовать политику замещения LRU	295
Существо проблемы: как построить полную систему ВП.....	302
Существо проблемы: как поддерживать синхронизацию	339
Существо проблемы: как создавать потоки и управлять ими	343
Существо проблемы: как сконструировать блокировку	357
Существо проблемы: как избежать активного ожидания.....	369
Существо проблемы: как добавлять блокировки в структуры данных	380
Существо проблемы: как ждать выполнения условия	396
Существо проблемы: как пользоваться семафорами	413

Существо проблемы: как быть с типичными ошибками в конкурентных программах	433
Существо проблемы: как быть с взаимоблокировкой.....	438
Существо проблемы: как построить конкурентный сервер без потоков.....	450
Существо проблемы: как включить в систему ввод-вывод?	466
Существо проблемы: как избежать дорогостоящего опроса.....	470
Существо проблемы: как снизить издержки РЮ	472
Существо проблемы: как взаимодействовать с устройствами	473
Существо проблемы: как построить независимую от устройств ОС	474
Существо проблемы: как хранить и обращаться к данным на диске	481
Существо проблемы: как решить проблему зависания диска	491
Существо проблемы: как учесть стоимость вращения диска	492
Существо проблемы: как сделать большой, быстрый и надежный диск.....	498
Существо проблемы: как управлять устройством долговременного хранения.....	517
Существо проблемы: как реализовать простую файловую систему.....	544
Существо проблемы: как уменьшить затраты файловой системы на ввод-вывод.....	558
Существо проблемы: как организовать данные на диске, чтобы повысить производительность.....	565
Существо проблемы: как обновить диск, несмотря на аварийные отказы	578
Существо проблемы: как сделать все операции записи последовательными?.....	601
Существо проблемы: как построить SSD на базе флеш-памяти.....	618
Существо проблемы: как обеспечить целостность данных.....	644
Существо проблемы: как обрабатывать скрытые ошибки секторов	646
Существо проблемы: как обеспечить целостность данных вопреки искажению	647
Существо проблемы: как обработать запись не по адресу	651
Существо проблемы: как обрабатывать потерянную запись.....	652
Существо проблемы: как строить системы, которые работают, несмотря на отказы компонентов	662
Существо проблемы: как построить распределенную файловую систему	679
Существо проблемы: как определить файловый протокол без запоминания информации о состоянии.....	682
Существо проблемы: как спроектировать масштабируемый файловый протокол	700

Книги издательства «ДМК ПРЕСС»
можно купить оптом и в розницу
в книготорговой компании «Галактика»
(представляет интересы издательств
«ДМК ПРЕСС», «СОЛОН ПРЕСС», «КТК Галактика»).

Адрес: г. Москва, пр. Андропова, 38;
тел.: (499) 782-38-89, электронная почта: books@aliants-kniga.ru.
При оформлении заказа следует указать адрес (полностью),
по которому должны быть высланы книги;
фамилию, имя и отчество получателя.

Желательно также указать свой телефон и электронный адрес.
Эти книги вы можете заказать и в интернет-магазине: www.a-planeta.ru.

Ремзи Х. Арпачи-Дюссо, Андреа К. Арпачи-Дюссо

Операционные системы

Главный редактор	<i>Мовчан Д. А.</i> dmkpress@gmail.com
Зам. главного редактора	<i>Сенченкова Е. А.</i>
Перевод	<i>Слинкин А. А.</i>
Корректор	<i>Синяева Г. И.</i>
Верстка	<i>Чаннова А. А.</i>
Дизайн обложки	<i>Мовчан А. Г.</i>

Гарнитура PT Serif. Печать цифровая.
Усл. печ. л. 59,31. Тираж 200 экз.

Веб-сайт издательства: www.dmkpress.com